

THE ZCASH ARBORETUM

The Zcash Arboretum

Foundations, deployed protocol, and frontier designs

Marek
m@rek.onl

Contents

Introduction	7
I Math Guide: Mathematical foundations for Halo 2 and Zcash Orchard	8
1 Notation: sets, functions, and relations	8
2 The integers and modular arithmetic	14
3 Groups	23
4 Rings and fields	37
5 Polynomials over a field	47
6 Finite fields	63
7 Number theory for cryptography	72
8 Linear algebra over a field	82
9 Roots of unity and evaluation domains	93
10 Elliptic curves	108
11 Probability, asymptotics, and computation	127
II Crypto Guide: Cryptographic primitives from scratch	152
1 The provable-security model	152
2 Hardness assumptions	166
3 Hash functions and the random oracle model	178
4 Commitment schemes	191
5 Pseudorandom functions, block ciphers, and format-preserving encryption	210
6 Symmetric encryption, AEAD, and key derivation	219
7 Key agreement	230
8 Digital signatures	244

9	Interactive proofs, zero knowledge, and SNARKs	260
10	Merkle trees and commitments to sets	279
III	Halo 2 Guide: The Halo 2 proof system, from arithmetisation to recursion	293
1	Introduction	294
2	Arithmetisation: from a computation to committed polynomials	297
3	The inner-product polynomial commitment	316
4	The Halo 2 proof system	345
5	A rigorous treatment of knowledge soundness	369
6	The verifier's decision and its implication chain	377
7	Recursive proof composition and accumulation schemes	383
IV	Consensus Guide: Nakamoto consensus: the double-spend race, the arithmetic of confirmations, and the deployed most-work rule	397
1	Scope, sources, and the two double-spends	397
2	The block tree and the most-work rule	398
3	A probability toolkit	400
4	Playing catch-up	402
5	Waiting for confirmations	404
6	What the numbers say	408
7	The economics of the attack	410
8	NU7: 25-second blocks and bounded shielded work	412
9	Layer boundary	417
V	Ironwood Guide: The Zcash Orchard protocol and its Ironwood pool	418
1	Introduction	418

2	Notation and primitive instances	423
3	Keys and addresses	435
4	Notes and note commitments	444
5	The note commitment tree	451
6	Nullifiers	459
7	Spend authorisation	465
8	Value commitments and the binding signature	473
9	The Action statement and its proof	479
10	Note encryption	490
11	Transactions and consensus rules	499
12	Security	517
VI	Wallet Guide: The Zcash wallet layer: keys, transactions, and light-client synchronisation	530
1	Introduction	530
2	Hierarchical key derivation (ZIP 32)	533
3	Addresses and viewing keys (ZIP 316)	546
4	Compact blocks (ZIP 307)	564
5	The light-client service	572
6	Note discovery	577
7	Commitment-tree synchronisation	587
8	Leakage to the light-client server	599
9	Anchors, confirmations, and expiry	608
10	Fees (ZIP 317)	617
11	Transaction construction	623
12	Partially created transactions (PCZT)	638

13	The transaction lifecycle	658
14	Payment requests (ZIP 321)	664
VII	FlyClient Guide: Merkle mountain ranges, the chain-history commitment, and the FlyClient sampling protocol	671
1	Introduction	671
2	The verification problem	674
3	Merkle mountain ranges	680
4	The FlyClient security model	690
5	The FlyClient protocol	698
6	The chain-history commitment	715
7	The deployment boundary	728
VIII	ZSA Guide: Zcash Shielded Assets: asset identity, issuance, transfer, and burn	737
1	Introduction	737
2	Asset identity	744
3	Notes and note commitments	751
4	OrchardZSA Actions and Action Groups	758
5	Burn and the binding signature	769
6	Issuance of Custom Assets	776
7	The global issuance state	785
8	The transaction carrier	794
9	Security	799
10	Classification of the components	813

IX FROST Guide: Threshold RedPallas: FROST and its Zcash integration surfaces	819
1 Introduction	819
2 Threshold Schnorr signatures	824
3 Key generation	829
4 Signing and aggregation	835
5 Re-randomised FROST for spend authorisation	846
6 The aggregate as a spend-authorisation signature	856
7 Security	867

Introduction

The Zcash Arboretum is a non-normative guide to the mathematics, cryptography, and engineering of the Zcash protocol. It covers the foundations of Halo 2 and Orchard, deployed consensus and wallet protocols, and designs being explored beyond them. The protocol specification, applicable ZIPs, and consensus rules remain authoritative.

The order is layered. The *Math*, *Crypto*, and *Halo 2* Guides construct the foundations. The *Consensus*, *Ironwood*, and *Wallet* Guides explain the deployed system and its boundaries. The remaining parts examine the unbuilt FlyClient bridge, shielded assets, and threshold authorization.

Three reading paths cover most uses. For prerequisites, begin with the first three parts; readers new to proof systems may start with the worked example that closes the Halo 2 Guide. For the life of a shielded payment, read *Ironwood*, then *Wallet* and *Consensus*. For proposed changes, read the relevant frontier part only after its lower-layer dependencies. Each part restarts its own section numbering so that citations agree with the separately published volume.

I

Math Guide

Mathematical foundations for Halo 2 and Zcash Orchard

Abstract

This volume of *The Zcash Arboretum* develops the mathematics underlying the cryptography of Zcash from first principles, assuming only elementary algebra and calculus. Its subjects, in order, are the notation of sets, functions, and relations in which every later claim is stated; the integers and their remainders modulo a prime, where the extended Euclidean algorithm computes inverses and so reduces division to multiplication by an inverse; the abstract structures—groups, rings, and fields—that axiomatise the laws this arithmetic obeys; polynomials over a field, whose bounded number of roots allows two polynomials of large degree to be compared by evaluation at a single random point; the finite fields, their possible sizes, and their cyclic multiplicative structure; the number theory of squares, square roots, and the discrete logarithm; linear algebra and the inner products underlying short commitments to long vectors; the roots of unity that supply evaluation domains and the fast Fourier transform; elliptic curves, whose point groups, when chosen appropriately, admit no known subexponential discrete-logarithm attack; and the probability, asymptotics, and model of computation required to state a claim of 128-bit security with a precise meaning.

Later volumes of the series depend on three parts of this volume: the Pasta curve cycle, constructed in §10; the polynomial machinery on which Halo 2 rests, developed in §5 and §9; and the language of security statements, fixed in §11. A first reading should take the sections in order; a reader who needs one of these parts may begin at its section and follow its references back to the earlier sections on which it depends.

1 Notation: sets, functions, and relations

This section fixes the set-theoretic and logical language of the series. It defines sets, functions and relations; proves that a function has a two-sided inverse exactly when it is bijective (Theorem 1.1), that the classes of an equivalence relation partition the set (Proposition 1.2), and that a partial order and its strict companion determine each other (Proposition 1.3); fixes the conventions of proof; and closes with three tables giving the spoken form of every recurring symbol (§1.5).

1.1 Sets

A *set* is a collection of distinct objects, its *elements*; one writes $x \in A$ for membership and $x \notin A$ for its negation. A set may be given by listing, $\{a, b, c\}$, or by *set-builder* notation $\{x \in A : P(x)\}$ (“the x in A such that $P(x)$ ”). One writes $A \subseteq B$ when every element of A lies in B , and $A = B$ exactly when $A \subseteq B$ and $B \subseteq A$ — so proving two sets equal means proving two inclusions. The usual operations are union $A \cup B$, intersection $A \cap B$, difference $A \setminus B$, and the empty set $\emptyset$. The *Cartesian product* is $A \times B = \{(a, b) : a \in A, b \in B\}$, and A^n is the set of n -tuples over A . For a finite set A , $|A|$ denotes its number of elements. The standard number systems are the naturals $\mathbb{N}$, the integers $\mathbb{Z}$, the rationals $\mathbb{Q}$, the reals $\mathbb{R}$, and the complex numbers $\mathbb{C}$, with $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$; we construct the underlying arithmetic of the prime field $\mathbb{F}_p$ in §2, certify it as a field—and first call it $\mathbb{F}_p$ —in §4, and treat general finite fields $\mathbb{F}_q$ in §6.

1.2 Functions

A *function* $f: A \rightarrow B$ assigns to each $a \in A$ a unique value $f(a) \in B$; A is the *domain* and B the *codomain*. It is *injective* (one-to-one) if $f(a) = f(a')$ implies $a = a'$; *surjective* (onto) if every $b \in B$ equals $f(a)$ for some $a \in A$; and *bijective* if both. The *composition* of $f: A \rightarrow B$ and $g: B \rightarrow C$ is $(g \circ f)(a) = g(f(a))$; composition is associative, $h \circ (g \circ f) = (h \circ g) \circ f$, both sides sending a to $h(g(f(a)))$. The *identity* on A is $\text{id}_A: A \rightarrow A$, $\text{id}_A(a) = a$.

Theorem 1.1 (Inverse functions). *A function $f: A \rightarrow B$ has a two-sided inverse $f^{-1}: B \rightarrow A$, meaning $f^{-1} \circ f = \text{id}_A$ and $f \circ f^{-1} = \text{id}_B$, if and only if f is bijective. When it exists, the inverse is unique.*

Proof. Suppose a two-sided inverse g exists. If $f(a) = f(a')$, applying g gives $a = g(f(a)) = g(f(a')) = a'$, so f is injective; and every $b \in B$ satisfies $b = f(g(b))$, so f is surjective. Conversely, suppose f is bijective. Each $b \in B$ has at least one preimage by surjectivity and at most one by injectivity, so we may define $g(b)$ to be the unique $a \in A$ with $f(a) = b$; then $g(f(a)) = a$ and $f(g(b)) = b$ hold by construction. Finally, if g and g' are both two-sided inverses, associativity of composition gives $g = g \circ \text{id}_B = g \circ (f \circ g') = (g \circ f) \circ g' = \text{id}_A \circ g' = g'$; the inverse is unique, and we write it f^{-1} . $\square$

For $S \subseteq A$ the *image* is $f(S) = \{f(a) : a \in S\}$, and for $T \subseteq B$ the *preimage* is $f^{-1}(T) = \{a \in A : f(a) \in T\}$, defined whether or not f is invertible. The two uses of f^{-1} agree: when the inverse function exists, the preimage of T under f is exactly the image of T under f^{-1} .

1.3 Relations and equivalences

A *binary relation* on A is a subset $R \subseteq A \times A$; one writes $a \sim b$ for $(a, b) \in R$. It is an *equivalence relation* if it is *reflexive* ($a \sim a$), *symmetric* ($a \sim b \implies b \sim a$), and *transitive* ($a \sim b$ and $b \sim c$ imply $a \sim c$). The *equivalence class* of a is $[a] = \{x \in A : x \sim a\}$, and the set of distinct classes is the *quotient* $A/\sim$.

Proposition 1.2 (Classes partition the set). *Let $\sim$ be an equivalence relation on a set A . The distinct equivalence classes partition A : they are nonempty, pairwise disjoint, and their union is A .*

Proof. Reflexivity gives $a \in [a]$ for every $a \in A$, so each class is nonempty and every element lies in some class; the union of the classes is therefore A . For disjointness, suppose the classes $[a]$ and $[b]$ share an element x , and let $y \in [a]$ be arbitrary. From $y \sim a$ and $a \sim x$ (the latter by symmetry from $x \sim a$), transitivity gives $y \sim x$; combining with $x \sim b$ gives $y \sim b$, so $y \in [b]$. Thus $[a] \subseteq [b]$, and exchanging the roles of a and b gives the reverse inclusion; two classes that meet coincide. $\square$

The volume uses this construction to form the integers modulo n (§2) and the cosets of a subgroup (§3).

A second kind of relation orders elements rather than identifying them. A *partial order* on A is a relation $\leq$ that is reflexive, transitive, and *antisymmetric*: $a \leq b$ and $b \leq a$ imply $a = b$. Four partial orders recur in the series: the usual $\leq$ on $\mathbb{Z}$; inclusion $\subseteq$ on the subsets of a fixed set, whose antisymmetry is the two-inclusions rule of §1.1; divisibility $|$ on $\mathbb{N}$ (§2.2); and the *prefix* relation on finite sequences, under which a sequence precedes every sequence that extends it. Two elements are *comparable* if $a \leq b$ or $b \leq a$, and a partial order under which every two elements are comparable is a *total* (or *linear*) order. The order $\leq$ on $\mathbb{Z}$ is total; the other three are not: $\{1\}$ and $\{2\}$ are incomparable under inclusion, as are 2 and 3 under divisibility and the one-term sequences (1) and (2) under prefix.

Every partial order has a *strict companion*, $a < b$ meaning $a \leq b$ and $a \neq b$.

Proposition 1.3 (Strict and non-strict orders). *If $\leq$ is a partial order on A , its strict companion $<$ is irreflexive ($a < a$ holds for no a) and transitive. Conversely, if $<$ is irreflexive and transitive, then “ $a < b$ or $a = b$ ” defines a partial order on A whose strict companion is $<$.*

Proof. Irreflexivity of the companion is the clause $a \neq a$. For transitivity let $a < b < c$; then $a \leq c$ because $\leq$ is transitive, and $a = c$ would give $a \leq b \leq a$, hence $a = b$ by antisymmetry, contradicting $a < b$. Conversely, the relation “ $a < b$ or $a = b$ ” is reflexive by construction and transitive by cases: if either hypothesis is an equality, the conclusion follows from the other by substitution, and otherwise from transitivity of $<$. It is antisymmetric, since $a \neq b$ with both $a \leq b$ and $b \leq a$ would mean $a < b$ and $b < a$, whence $a < a$, against irreflexivity. Its strict companion is $<$ again: “($a < b$ or $a = b$) and $a \neq b$ ” reduces to $a < b$, which already excludes $a = b$ by irreflexivity. $\square$

A strict order $<$ is a *strict total order* when its non-strict companion is total, equivalently when for all a, b exactly one of $a < b$, $a = b$, $b < a$ holds (*trichotomy*): at least one holds by totality, and any two together would give $a < a$, by substitution or by transitivity. The usual $<$ on $\mathbb{Z}$ is the standard example. A function $f: A \rightarrow B$ between ordered sets is *monotone* (order-preserving) if $a \leq a'$ implies $f(a) \leq f(a')$, and *strictly increasing* if $a < a'$ implies $f(a) < f(a')$. The volume assumes only one further property of an order, the well-ordering of $\leq$ on $\mathbb{N}$, stated at the start of §2.1.

1.4 Logic and proof conventions

We employ the symbols $\forall$ (“for all”), $\exists$ (“there exists”), $\exists!$ (“there exists a unique”), and “:” or “s.t.” for “such that”. Negation swaps the quantifiers: $\neg(\forall x P(x))$ is $\exists x \neg P(x)$, and $\neg(\exists x P(x))$ is $\forall x \neg P(x)$. The order of unlike quantifiers matters: $\forall x \exists y P(x, y)$ (the y may depend on x) is weaker than $\exists y \forall x P(x, y)$ (one y works for all x).

Throughout the series we prove an implication $P \Rightarrow Q$ directly (assume P , derive Q), by contraposition (assume $\neg Q$, derive $\neg P$), or by contradiction; we prove a statement $\forall x P(x)$ by taking an arbitrary x , and $\exists x P(x)$ by exhibiting a witness. We prove statements about all natural numbers by induction.

1.5 Symbols and their pronunciation

Tables 1–3 list the three alphabets of the series—blackboard-bold letters for the standard number systems and structures, Greek letters such as λ and ρ for parameters and challenges, and relational and operator symbols—with their English readings: the Greek letters in the International Phonetic Alphabet, the other symbols as the words read aloud. Readings vary between speakers and regions; the alternatives shown are not exhaustive. Each symbol is defined where it is first used; the notes here are reminders, not definitions.

Table 1. Blackboard-bold letters. “Blackboard bold” names the double-struck style itself; in speech one usually says only the letter or the system it denotes. Elliptic curves are written italic E or calligraphic $\mathcal{E}$, never blackboard bold. The squared $\mathbb{P}^2$ of §10 is unrelated: it denotes the projective plane.

Symbol	Name	Said	Typical role in the series
$\mathbb{N}$	blackboard N	“natural numbers”	the naturals
$\mathbb{Z}$	blackboard Z	“the integers”	the integers (German <i>Zahlen</i>)
$\mathbb{Q}$	blackboard Q	“the rationals”	the rationals (<i>quotients</i>)
$\mathbb{R}$	blackboard R	“the reals”	the real numbers
$\mathbb{C}$	blackboard C	“the complexes”	the complex numbers
$\mathbb{F}_q$	blackboard F	“eff-cue”	the finite field of q elements
$\mathbb{G}$	blackboard G	“group G”	an abstract group
$\mathbb{E}$	blackboard E	“expectation”	expectation of a random variable (§11)
$\mathbb{P}$	blackboard P	“the Pallas curve”	the Pallas group (§10; protocol-spec notation)

Table 2. The 24 letters of the Greek alphabet in lower and upper case. The English names are given in the International Phonetic Alphabet, British (Received Pronunciation) and American (General American), as in the Cambridge, Collins and Oxford dictionaries and Wiktionary, the most widely attested form first; rarer variants are omitted. The last column gives the Modern Greek name in the simplified spelling of the Triantafyllidis dictionary, with its pronunciation; traditional spellings such as $\gamma\acute{\alpha}\mu\mu\alpha$, $\kappa\acute{\alpha}\pi\pi\alpha$ and $\mu\upsilon$ are also current. The capital Σ is also the summation sign and names the Σ -protocol family of later volumes; the capital Π is also the product sign and, in those volumes, a conventional name for a protocol or problem.

Lower	Capital	Name	British	American	Greek
α	A	alpha	/ˈælfə/	/ˈælfə/	άλφα [ˈalfa]
β	B	beta	/ˈbi:tə/	/ˈbeɪtə/	βήτα [ˈvita]
γ	Γ	gamma	/ˈgæmə/	/ˈgæmə/	γάμα [ˈɣama]
δ	Δ	delta	/ˈdeltə/	/ˈdeltə/	δέλτα [ˈdelta]
ϵ, ε	E	epsilon	/ˈepsɪlɒn, ɛpˈsaɪlən/	/ˈepsələ:n, ˈepsɪlə:n/	έψιλον [ˈepsilon]
ζ	Z	zeta	/ˈzi:tə/	/ˈzeɪtə, ˈzi:tə/	ζήτα [ˈzita]
η	H	eta	/ˈi:tə/	/ˈeɪtə, ˈi:tə/	ήτα [ˈita]
θ	Θ	theta	/ˈθi:tə/	/ˈθeɪtə, ˈθi:tə/	θήτα [ˈθita]
ι	I	iota	/aɪˈəʊtə/	/aɪˈoʊtə/	ιώτα [ˈjota]
κ	K	kappa	/ˈkæpə/	/ˈkæpə/	κάπα [ˈkapa]
λ	Λ	lambda	/ˈlæmdə/	/ˈlæmdə/	λάμδα [ˈlamða]
μ	M	mu	/mju:/	/mu:, mju:/	μι [mi]
ν	N	nu	/nju:/	/nu:, nju:/	νι [ni]
ξ	Ξ	xi	/saɪ, zaɪ, ksai/	/saɪ, zaɪ, ksai/	ξι [ksi]
$\omicron$	O	omicron	/əʊˈmaɪkrɒn, ˈɒmɪkrɒn/	/ˈoʊmɪkra:n, ˈɑ:mɪkra:n/	όμικρον [ˈomikron]
π	Π	pi	/paɪ/	/paɪ/	πι [pi]
ρ	P	rho	/rəʊ/	/rou/	ρο [ro]
σ	Σ	sigma	/ˈsɪgmə/	/ˈsɪgmə/	σίγμα [ˈsiɣma]
τ	T	tau	/taʊ, tɔ:/	/taʊ, tɔ:/	ταυ [taf]
υ	Υ	upsilon	/ju:pˈsaɪlən, ˈʌpsɪlɒn/	/ˈʌpsələ:n, ˈʊpsɪlə:n/	ύψιλον [ˈipsilon]
ϕ, φ	Φ	phi	/faɪ/	/faɪ, fi:/	φι [fi]
χ	X	chi	/kaɪ/	/kaɪ, ki:/	χι [çi]
ψ	Ψ	psi	/psaɪ, saɪ/	/saɪ, psai, psi:/	ψι [psi]
ω	Ω	omega	/ˈəʊmɪgə, əʊˈmi:gə/	/ouˈmeɪgə, ouˈmegə/	ωμέγα [oˈmɛɣa]

Table 3. Relational and operator symbols. Every use of $\oplus$ in the series is the bitwise exclusive or. The encoding P^* is protocol-spec notation, defined in later volumes.

Symbol	Said	Meaning
$\in, \notin$	“in”, “not in”	set membership
$\subseteq, \subset$	“subset of”	(proper) inclusion
$\cup, \cap$	“union”, “intersect”	set union and intersection
$A \setminus B$	“A minus B”	set difference
$\times$	“times” / “cross”	Cartesian product
$, \nmid$	“divides”, “does not divide”	divisibility
$\preceq, <$	“precedes”	partial order, strict companion
$\equiv$	“congruent to”	congruence, qualified (mod n)
$a \bmod n$	“a mod n”	remainder of division by n
$:=, =:$	“is defined as”	definitional equality
$\mapsto$	“maps to”	the action of a function
$g \circ f$	“g after f”	function composition
$\lfloor x \rfloor, \lceil x \rceil$	“floor”, “ceiling”	round down, round up
$\log x, \log_2 x, \log_g b$	“log”, “log base two”, “log base g ”	natural, base-2, and discrete (base g) logarithm
$\langle a, b \rangle$	“inner product”	inner product; $\langle g \rangle$ the subgroup generated by g
$\oplus$	“x-or”	bitwise exclusive or
$x \parallel y$	“x concat y”	concatenation of bit or byte strings
$x \leftarrow D, x \xleftarrow{\$} S$	“drawn from”	sampling ($\$$ marks uniform); also assignment
$[k]P$	“k times P”	scalar multiplication of a point ($\$10$)
P^*	“P star”	canonical bit-encoding of a point or element
$\cdot$	“dot”	product; argument placeholder, as in $F(\cdot)$
$ A $	“size of A” / “order of A”	cardinality

2 The integers and modular arithmetic

This section develops the integers $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ from a small set of assumptions and constructs the integers modulo n —the set $\mathbb{Z}/n\mathbb{Z}$ of congruence classes—with its units and the algorithms that compute with them. It proves that division modulo n is multiplication by a *modular inverse*, that the class of a has one exactly when $\gcd(a, n) = 1$ (Theorem 2.24), and that an extension of the algorithm of Euclid’s *Elements* computes it efficiently, as cryptography requires. Every step of a prover or verifier in the proof systems of this series is an addition, a multiplication, or a division of remainders modulo a fixed large prime; addition and multiplication are computed in $\mathbb{Z}$, after which multiples of the modulus are discarded. The finite number systems underlying elliptic curves, the scalar arithmetic of Halo 2, and the polynomial commitments of Orchard (short values that bind a prover to entire polynomials, which a verifier then tests by evaluation) all reduce to computations with congruence classes.

2.1 Well-ordering and the integers

We take the basic algebraic properties of $\mathbb{Z}$ for granted: addition and multiplication are associative and commutative, and multiplication distributes over addition; the integers 0 and 1 are identities for addition and multiplication respectively; every integer a has a negative $-a$; there are no zero divisors (if $ab = 0$ then $a = 0$ or $b = 0$); and $\mathbb{Z}$ carries a total order $\leq$ compatible with the arithmetic. The one further property assumed as an axiom is the following.

Theorem 2.1 (Well-ordering principle). *Every nonempty subset $S \subseteq \mathbb{N} = \{0, 1, 2, \dots\}$ of the natural numbers has a least element: there exists $m \in S$ such that $m \leq s$ for all $s \in S$.*

The well-ordering principle is logically equivalent to the principle of mathematical induction; we use both freely. The existence results below (division with remainder, termination of the Euclidean algorithm, Bézout’s identity, prime factorisation) all rest on it.

2.2 Divisibility

Definition 2.2. Let $a, b \in \mathbb{Z}$. One says that a *divides* b , written $a \mid b$, if there exists an integer $c \in \mathbb{Z}$ with $b = ac$. In this case a is called a *divisor* or *factor* of b , and b is a *multiple* of a . If no such c exists one writes $a \nmid b$.

The following consequences are used throughout, usually without citation.

Proposition 2.3. *Let $a, b, c, d \in \mathbb{Z}$. Then:*

1. $a \mid a$ (reflexivity), $1 \mid a$, and $a \mid 0$.
2. If $a \mid b$ and $b \mid c$ then $a \mid c$ (transitivity).
3. If $a \mid b$ and $a \mid c$ then $a \mid (bx + cy)$ for all $x, y \in \mathbb{Z}$; in particular $a \mid (b \pm c)$.
4. If $a \mid b$ and $b \neq 0$ then $|a| \leq |b|$.

5. If $a \mid b$ and $b \mid a$ then $a = \pm b$.

6. $0 \mid b$ if and only if $b = 0$.

Proof. Part (1) is immediate: $a = a \cdot 1$, $a = 1 \cdot a$, $0 = a \cdot 0$. For (2), write $b = ae$ and $c = bf$; then $c = a(e f)$, so $a \mid c$. For (3), write $b = ae$ and $c = af$; then $bx + cy = a(ex + fy)$. For (4), write $b = ac$ with $b \neq 0$, so $c \neq 0$ and hence $|c| \geq 1$; then $|b| = |a| |c| \geq |a|$. For (5), if either is 0 then by (6) so is the other and the claim holds; otherwise (4) gives $|a| \leq |b|$ and $|b| \leq |a|$, so $|a| = |b|$ and $a = \pm b$. For (6), $0 \mid b$ means $b = 0 \cdot c = 0$; conversely $0 \mid 0$. $\square$

Restricted to the natural numbers, where (5) becomes $a = b$, the relation $\mid$ is a partial order in the sense of §1.3; on $\mathbb{Z}$ it fails to be antisymmetric because of signs. The *units* of $\mathbb{Z}$ —the integers dividing 1—are exactly ± 1 , and two integers that divide each other are called *associates*; in $\mathbb{Z}$ the associates of a are $\pm a$.

2.3 The division algorithm

Theorem 2.4 (Division algorithm). *Let $a \in \mathbb{Z}$ and $b \in \mathbb{Z}$ with $b > 0$. Then there exist unique integers q (the quotient) and r (the remainder) such that*

$$a = bq + r, \quad 0 \leq r < b.$$

Proof. Existence. Consider the set

$$S = \{a - bk : k \in \mathbb{Z}, a - bk \geq 0\} \subseteq \mathbb{N}.$$

First, $S \neq \emptyset$. If $a \geq 0$ then $a = a - b \cdot 0 \in S$. If $a < 0$ then, since $b \geq 1$, taking $k = a$ gives $a - ba = a(1 - b) \geq 0$ (a product of a nonpositive and a nonpositive factor), so $a - ba \in S$. Thus S is a nonempty subset of $\mathbb{N}$, and by the well-ordering principle (Theorem 2.1) it has a least element $r = a - bq$ for some $q \in \mathbb{Z}$. By construction $r \geq 0$. If $r \geq b$, then $r - b = a - b(q + 1) \geq 0$ would lie in S and be strictly smaller than r , contradicting minimality. Hence $0 \leq r < b$.

Uniqueness. Suppose $a = bq + r = bq' + r'$ with $0 \leq r, r' < b$. Then $b(q - q') = r' - r$, so $b \mid (r' - r)$. But $-b < r' - r < b$, and the only multiple of b strictly between $-b$ and b is 0. Hence $r' = r$, and then $b(q - q') = 0$ with $b \neq 0$ forces $q = q'$. $\square$

Remark 2.5. The hypothesis $b > 0$ is only for definiteness. For general $b \neq 0$ one obtains unique q, r with $a = bq + r$ and $0 \leq r < |b|$, by applying the theorem to $|b|$ and adjusting the sign of q . The remainder convention $0 \leq r < |b|$ differs from the truncation behaviour of many programming languages, where the remainder of a negative dividend may be negative; the results below use the former.

We denote the remainder r produced by the division algorithm by $a \bmod b$, and the quotient q by $\lfloor a/b \rfloor$ when $b > 0$ (the *floor* of the rational number a/b). With this notation, $b \mid a$ holds precisely when $a \bmod b = 0$.

2.4 The greatest common divisor and the Euclidean algorithm

Definition 2.6. A *common divisor* of integers a and b is an integer d with $d \mid a$ and $d \mid b$. The *greatest common divisor* of a and b , not both zero, is the largest such d ; it is denoted $\gcd(a, b)$. By convention $\gcd(0, 0) = 0$. Integers a and b are *coprime* (or *relatively prime*) if $\gcd(a, b) = 1$.

The greatest common divisor is well defined when $(a, b) \neq (0, 0)$: the set of common divisors is nonempty (it contains 1) and bounded above (every common divisor of, say, the nonzero a has absolute value at most $|a|$ by Proposition 2.3(4)), so it has a largest element, which is positive since 1 is a common divisor. The computation of the gcd rests on the following lemma.

Lemma 2.7. Let $a, b \in \mathbb{Z}$. For any $q \in \mathbb{Z}$,

$$\gcd(a, b) = \gcd(b, a - qb).$$

In particular, if $a = bq + r$ then $\gcd(a, b) = \gcd(b, r)$, and $\gcd(a, 0) = |a|$.

Proof. It suffices to show the two pairs (a, b) and $(b, a - qb)$ have exactly the same common divisors; equal sets of common divisors have equal greatest elements (and the same coprimality status). If $d \mid a$ and $d \mid b$, then $d \mid (a - qb)$ by Proposition 2.3(3), so d is a common divisor of b and $a - qb$. Conversely, if $d \mid b$ and $d \mid (a - qb)$, then $d \mid ((a - qb) + qb) = a$, so d is a common divisor of a and b . The two sets of common divisors coincide. Finally $\gcd(a, 0) = |a|$ because the common divisors of a and 0 are exactly the divisors of a , the largest of which in absolute value is $|a|$ (and $|a| > 0$ when $a \neq 0$; the case $a = 0$ is the convention $\gcd(0, 0) = 0$). $\square$

Theorem 2.8 (Euclidean algorithm). Let a, b be integers with $b \neq 0$. Define a sequence of remainders by $r_0 = |a|$, $r_1 = |b|$, and, as long as $r_i \neq 0$, let $r_{i+1} = r_{i-1} \bmod r_i$, so that

$$r_{i-1} = r_i q_i + r_{i+1}, \quad 0 \leq r_{i+1} < r_i.$$

The sequence $r_1 > r_2 > \dots$ of positive remainders is strictly decreasing, so some $r_{n+1} = 0$; the last nonzero remainder satisfies $r_n = \gcd(a, b)$.

Proof. The remainders $r_1, r_2, \dots$ are nonnegative integers and, while nonzero, strictly decreasing because $r_{i+1} = r_{i-1} \bmod r_i < r_i$. A strictly decreasing sequence of nonnegative integers cannot be infinite (again by well-ordering: otherwise the set of values would have no least element), so the algorithm terminates with some $r_{n+1} = 0$ and $r_n > 0$. By Lemma 2.7, each step preserves the gcd:

$$\gcd(a, b) = \gcd(r_0, r_1) = \gcd(r_1, r_2) = \dots = \gcd(r_n, r_{n+1}) = \gcd(r_n, 0) = r_n,$$

using $\gcd(|a|, |b|) = \gcd(a, b)$ (signs do not affect common divisors). $\square$

Example 2.9. Compute $\gcd(1071, 462)$:

$$1071 = 2 \cdot 462 + 147,$$

$$462 = 3 \cdot 147 + 21,$$

$$147 = 7 \cdot 21 + 0.$$

The last nonzero remainder is 21, so $\gcd(1071, 462) = 21$. As a check, $1071 = 21 \cdot 51$ and $462 = 21 \cdot 22$, with $\gcd(51, 22) = 1$.

Remark 2.10. The number of division steps of the Euclidean algorithm for inputs $a > b > 0$ is $O(\log b)$; the worst case occurs at consecutive Fibonacci numbers (Lamé's theorem), where every quotient q_i equals 1 except the final one (which equals 2) and the remainders shrink as slowly as possible. With schoolbook arithmetic on k -bit inputs the whole computation costs $O(k^2)$ bit operations. This bound makes gcd-based operations such as modular inversion practical at the sizes used in Halo 2 and Orchard.

2.5 The Bézout identity and the extended Euclidean algorithm

Back-substitution through the equations of the Euclidean algorithm expresses the gcd as an *integer linear combination* of the inputs.

Theorem 2.11 (Bézout's identity). *Let $a, b \in \mathbb{Z}$, not both zero, and let $d = \gcd(a, b)$. Then there exist integers $u, v \in \mathbb{Z}$ with*

$$au + bv = d.$$

Moreover d is the smallest positive integer expressible in the form $ax + by$ with $x, y \in \mathbb{Z}$, and the set of all such combinations is exactly the set of multiples of d :

$$\{ax + by : x, y \in \mathbb{Z}\} = d\mathbb{Z} = \{kd : k \in \mathbb{Z}\}.$$

Proof. Let $I = \{ax + by : x, y \in \mathbb{Z}\}$. This set contains a and b and is closed under addition and under multiplication by any integer; concretely, if $s = ax_1 + by_1$ and $t = ax_2 + by_2$ lie in I , then for any $m, n \in \mathbb{Z}$ one has

$$ms + nt = a(mx_1 + nx_2) + b(my_1 + ny_2) \in I.$$

Since a, b are not both zero, I contains a positive element (one of $\pm a, \pm b$), so the set $I \cap \mathbb{N}_{>0}$ of positive elements of I is nonempty. By well-ordering it has a least element $g = au + bv > 0$.

We claim that g divides every element of I . Indeed, let $c \in I$ and write $c = gq + r$ with $0 \leq r < g$ by the division algorithm. Then $r = c - gq \in I$ because I is closed under the operations just described. By minimality of g among positive elements of I , and $0 \leq r < g$, necessarily $r = 0$. Hence $g \mid c$ for every $c \in I$; in particular $g \mid a$ and $g \mid b$, so g is a common divisor of a and b . Conversely, any common divisor e of a and b divides $au + bv = g$ by Proposition 2.3(3). Therefore g is a common divisor that is divisible by every common divisor; being positive, it is in particular the *greatest* common divisor, so $g = d$. The displayed equation $au + bv = d$ follows, d is the least positive element of I , and $I = g\mathbb{Z} = d\mathbb{Z}$ because every element of I is a multiple of g , as shown, and conversely every multiple $kg = a(ku) + b(kv)$ lies in I . $\square$

Remark 2.12. The proof yields a characterisation of the gcd that is independent of the order $\leq$: $d = \gcd(a, b)$ is, up to sign, the unique common divisor of a and b that is divisible by every common divisor. Later sections use this formulation outside the integers, where “largest” may make no sense but “divisible by every common divisor” still does. We call a nonempty subset $I \subseteq$

$\mathbb{Z}$ closed under addition and under multiplication by arbitrary integers an *ideal*; the content of Bézout's identity is that every ideal of $\mathbb{Z}$ has the form $d\mathbb{Z}$ for a single generator d . Ideals recur in Section 4 in a construction that generalises the passage from integers to congruence classes below.

The Bézout coefficients u, v are not unique. If $au + bv = d$, then for every $k \in \mathbb{Z}$,

$$a\left(u + k\frac{b}{d}\right) + b\left(v - k\frac{a}{d}\right) = d,$$

since the added terms $k\frac{ab}{d}$ and $-k\frac{ab}{d}$ cancel; and these are all the solutions. To prove completeness, observe first that a/d and b/d are coprime: a common divisor e of both makes de a common divisor of a and b , so $de \mid au + bv = d$ by Proposition 2.3(3), forcing $e = \pm 1$. Now suppose $au' + bv' = d$. Subtracting $au + bv = d$ and dividing by $d > 0$ gives $\frac{a}{d}(u' - u) = -\frac{b}{d}(v' - v)$, so b/d divides $\frac{a}{d}(u' - u)$; by Proposition 2.15(1) below (whose proof rests only on Bézout's identity, already established), b/d divides $u' - u$, say $u' = u + k\frac{b}{d}$ with $k \in \mathbb{Z}$. Substituting back gives $b(v' - v + k\frac{a}{d}) = 0$, so $v' = v - k\frac{a}{d}$ when $b \neq 0$; and when $b = 0$ we have $u' = u$, $a/d = \pm 1$, and $k = (v - v')\frac{a}{d}$ places (u', v') in the family.

Theorem 2.13 (Extended Euclidean algorithm). *There is an algorithm that, on input $a, b \in \mathbb{Z}$ not both zero, outputs $d = \gcd(a, b)$ together with integers u, v satisfying $au + bv = d$, using the same number of division steps as the Euclidean algorithm.*

Proof. Alongside the remainder sequence of Theorem 2.8, maintain two auxiliary sequences (s_i) and (t_i) with the invariant

$$r_i = as_i + bt_i \quad \text{for all } i. \quad (*)$$

Initialise (taking $a, b \geq 0$ for clarity; we restore signs at the end)

$$r_0 = a, s_0 = 1, t_0 = 0; \quad r_1 = b, s_1 = 0, t_1 = 1,$$

so $(*)$ holds for $i = 0, 1$. At step $i \geq 1$, having $r_{i-1} = r_i q_i + r_{i+1}$ from the division algorithm, set

$$s_{i+1} = s_{i-1} - q_i s_i, \quad t_{i+1} = t_{i-1} - q_i t_i.$$

Then

$$as_{i+1} + bt_{i+1} = (as_{i-1} + bt_{i-1}) - q_i(as_i + bt_i) = r_{i-1} - q_i r_i = r_{i+1},$$

so induction preserves $(*)$. When the algorithm terminates with $r_{n+1} = 0$ and $r_n = d = \gcd(a, b)$, the invariant for $i = n$ reads $d = as_n + bt_n$; output $u = s_n, v = t_n$. If $b = 0$, then $a \neq 0$ and termination is immediate with $n = 0$: no division step runs, the last nonzero remainder is $r_0 = a = \gcd(a, 0)$ (Lemma 2.7), and the invariant at $i = 0$ already gives the output $(d, u, v) = (a, 1, 0)$. If $b \neq 0$, termination and the value of d are exactly as in Theorem 2.8. In either case each step does only a constant amount of extra work. For general inputs, run the algorithm on $(|a|, |b|)$ to obtain $|a|u' + |b|v' = d$ (recall $\gcd(|a|, |b|) = \gcd(a, b)$) and output $u = \operatorname{sgn}(a)u', v = \operatorname{sgn}(b)v'$, so that $au + bv = d$. $\square$

Example 2.14. Run the extended algorithm on $a = 1071, b = 462$ (continuing Example 2.9). The quotients are $q_1 = 2, q_2 = 3, q_3 = 7$.

i	r_i	s_i	t_i
0	1071	1	0
1	462	0	1
2	147	1	-2
3	21	-3	7
4	0	22	-51

For instance, row 2 is row 0 minus $q_1 = 2$ times row 1: $s_2 = 1 - 2 \cdot 0 = 1, t_2 = 0 - 2 \cdot 1 = -2$, and $1071 \cdot 1 + 462 \cdot (-2) = 1071 - 924 = 147$. The last nonzero remainder is $r_3 = 21 = \gcd(1071, 462)$, with

$$1071 \cdot (-3) + 462 \cdot 7 = -3213 + 3234 = 21.$$

Thus $(u, v) = (-3, 7)$. The final row $(s_4, t_4) = (22, -51) = (b/d, -a/d)$ up to sign records the homogeneous solution: $1071 \cdot 22 + 462 \cdot (-51) = 0$.

Proposition 2.15 (Euclid's lemma). *Let $a, b, c \in \mathbb{Z}$.*

1. *If $c \mid ab$ and $\gcd(c, a) = 1$, then $c \mid b$.*
2. *If $\gcd(a, b) = 1, a \mid n$, and $b \mid n$, then $ab \mid n$.*
3. *If $\gcd(a, b) = 1$ and $\gcd(a, c) = 1$, then $\gcd(a, bc) = 1$.*

Proof. (1) By Bézout there are u, v with $cu + av = 1$. Multiply by b : $b = cub + avb = c(ub) + (ab)v$. Since $c \mid ab, c$ divides both terms on the right, hence $c \mid b$.

(2) Write $n = ak$. Since $b \mid n = ak$ and $\gcd(b, a) = 1$, part (1) gives $b \mid k$, say $k = bm$. Then $n = a(bm) = (ab)m$, so $ab \mid n$.

(3) By Bézout, $ax_1 + by_1 = 1$ and $ax_2 + cy_2 = 1$ for suitable integers. Multiplying,

$$1 = (ax_1 + by_1)(ax_2 + cy_2) = a \underbrace{(ax_1x_2 + x_1cy_2 + by_1x_2)}_{\in \mathbb{Z}} + bc(y_1y_2),$$

which exhibits 1 as an integer combination of a and bc . Hence $\gcd(a, bc)$ divides 1, so it equals 1. $\square$

2.6 Primes and the fundamental theorem of arithmetic

Definition 2.16. An integer $p > 1$ is *prime* if its only positive divisors are 1 and p . An integer $n > 1$ that is not prime is *composite*; thus n is composite iff $n = ab$ for some integers $1 < a, b < n$. The integer 1 is a *unit* and is by convention neither prime nor composite.

Excluding 1 from the primes is necessary for factorisation into primes to be unique, since otherwise arbitrarily many factors of 1 could be inserted. The property of primes on which uniqueness rests is the prime case of Euclid's lemma.

Lemma 2.17 (Euclid's lemma for primes). *Let p be prime and $a, b \in \mathbb{Z}$. If $p \mid ab$ then $p \mid a$ or $p \mid b$. More generally, if $p \mid a_1 a_2 \cdots a_k$ then $p \mid a_i$ for some i .*

Proof. The only positive divisors of p are 1 and p , so $\gcd(p, a) \in \{1, p\}$. If $\gcd(p, a) = p$ then $p \mid a$ and the claim holds; otherwise $\gcd(p, a) = 1$, and Proposition 2.15(1) gives $p \mid b$. The general statement follows by induction on k : from $p \mid a_1(a_2 \cdots a_k)$ it follows that $p \mid a_1$ or $p \mid a_2 \cdots a_k$, and the inductive hypothesis handles the latter. $\square$

Theorem 2.18 (Fundamental theorem of arithmetic). *Every integer $n > 1$ can be written as a product of primes,*

$$n = p_1 p_2 \cdots p_k,$$

and this factorisation is unique up to the order of the factors. Equivalently, there is a unique expression

$$n = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r}, \quad p_1 < p_2 < \cdots < p_r \text{ prime, } e_i \geq 1.$$

Proof. Existence. Suppose, for contradiction, that some integer > 1 is not a product of primes, and let n be the least such (well-ordering). Then n is not prime (a prime is trivially a product of one prime), so $n = ab$ with $1 < a, b < n$. By minimality, each of a, b is a product of primes; concatenating these factorisations expresses n as a product of primes, a contradiction. Hence every $n > 1$ factors into primes.

Uniqueness. Suppose

$$p_1 p_2 \cdots p_k = q_1 q_2 \cdots q_\ell$$

are two factorisations of the same integer into primes. The argument proceeds by induction on k . If $k = 0$ the product is the empty product 1, forcing $\ell = 0$ as well (a nonempty product of primes is > 1). For the inductive step, p_1 divides the right-hand product, so by Lemma 2.17 $p_1 \mid q_j$ for some j . Since q_j is prime, its only divisor > 1 is itself, so $p_1 = q_j$. Cancelling this common factor (legitimate as $\mathbb{Z}$ has no zero divisors) yields

$$p_2 \cdots p_k = q_1 \cdots \widehat{q_j} \cdots q_\ell,$$

where the hat denotes omission. By the inductive hypothesis these two factorisations agree up to order, with $k - 1 = \ell - 1$; restoring $p_1 = q_j$ shows the original factorisations agree up to order. Collecting equal primes into prime powers gives the canonical form, whose uniqueness is immediate from the uniqueness of the multiset of primes—the factors counted with multiplicity, without regard to order. $\square$

Remark 2.19. Existence rests on the order structure (well-ordering); uniqueness rests on Euclid's lemma, hence on Bézout, hence on the division algorithm. In some number systems with a division-like structure, Euclid's lemma fails and factorisation into irreducibles is *not* unique.

2.7 Congruences and residue classes

Congruence formalises the arithmetic of remainders. Fix an integer $n \geq 1$, the *modulus*.

Definition 2.20. Let $n \geq 1$ and $a, b \in \mathbb{Z}$. The integer a is *congruent to b modulo n* , written

$$a \equiv b \pmod{n},$$

if $n \mid (a - b)$, equivalently if a and b leave the same remainder upon division by n .

The two formulations agree: if $a = nq_1 + r_1$ and $b = nq_2 + r_2$ with $0 \leq r_1, r_2 < n$, then $a - b = n(q_1 - q_2) + (r_1 - r_2)$, and $n \mid (a - b)$ iff $n \mid (r_1 - r_2)$ iff $r_1 = r_2$ (since $|r_1 - r_2| < n$).

Proposition 2.21. Fix $n \geq 1$. Congruence modulo n is an equivalence relation on $\mathbb{Z}$ that is moreover compatible with addition and multiplication: if $a \equiv a' \pmod{n}$ and $b \equiv b' \pmod{n}$, then

$$a + b \equiv a' + b' \pmod{n}, \quad ab \equiv a'b' \pmod{n}.$$

Consequently $a^k \equiv (a')^k \pmod{n}$ for all $k \geq 0$, and one may substitute congruent values freely in any polynomial expression with integer coefficients.

Proof. Reflexivity ($n \mid 0$), symmetry ($n \mid (a - b) \Rightarrow n \mid (b - a)$), and transitivity ($n \mid (a - b)$ and $n \mid (b - c)$ give $n \mid (a - c)$ by Proposition 2.3(3)) are immediate. For compatibility, write $a - a' = ns$ and $b - b' = nt$. Then

$$(a + b) - (a' + b') = n(s + t),$$

and

$$ab - a'b' = ab - a'b + a'b - a'b' = (a - a')b + a'(b - b') = n(sb + a't),$$

so both $n \mid ((a + b) - (a' + b'))$ and $n \mid (ab - a'b')$. The statement about powers and polynomial expressions follows by induction from the multiplicative case. $\square$

Because congruence is an equivalence relation, it partitions $\mathbb{Z}$ into disjoint *congruence classes* (also called *residue classes*).

Definition 2.22. The *congruence class* (or *residue class*) of a modulo n is

$$[a]_n = \{b \in \mathbb{Z} : b \equiv a \pmod{n}\} = \{a + kn : k \in \mathbb{Z}\} = a + n\mathbb{Z}.$$

We write the set of all congruence classes as

$$\mathbb{Z}/n\mathbb{Z} = \{[a]_n : a \in \mathbb{Z}\}.$$

By the division algorithm every integer is congruent modulo n to exactly one of $0, 1, \dots, n - 1$, namely its remainder; hence the n classes $[0]_n, [1]_n, \dots, [n - 1]_n$ are distinct and exhaust $\mathbb{Z}/n\mathbb{Z}$. The set $\{0, 1, \dots, n - 1\}$ is the standard *system of representatives*, called the *least nonnegative residues*.

2.8 Units modulo n and modular inverses

Division is the inverse of multiplication, so division of remainders modulo n reduces to determining the a for which the congruence $ax \equiv 1 \pmod{n}$ admits a solution.

Definition 2.23. Let $n \geq 1$. A class $[a]_n \in \mathbb{Z}/n\mathbb{Z}$ is a *unit* (or is *invertible*) if there exists an integer x with $ax \equiv 1 \pmod{n}$. Any class $[x]_n$ with this property is the (*modular*) *inverse* of $[a]_n$, written $[a]_n^{-1}$; one also calls x a modular inverse of a modulo n . We write the set of units as

$$(\mathbb{Z}/n\mathbb{Z})^\times = \{ [a]_n : [a]_n \text{ is a unit} \}.$$

The definition is well posed on classes: by Proposition 2.21, if $a' \equiv a$ and $x' \equiv x \pmod{n}$ then $a'x' \equiv ax \pmod{n}$, so whether $ax \equiv 1 \pmod{n}$ holds depends only on the classes $[a]_n$ and $[x]_n$. The uniqueness part of the next theorem justifies the definite article in “the inverse”.

Theorem 2.24. Let $n \geq 1$ and $a \in \mathbb{Z}$. The class $[a]_n$ is a unit in $\mathbb{Z}/n\mathbb{Z}$ if and only if $\gcd(a, n) = 1$. When $\gcd(a, n) = 1$, the inverse is unique, and the extended Euclidean algorithm computes a representative: if $au + nv = 1$, then $[a]_n^{-1} = [u]_n$.

Proof. Suppose $\gcd(a, n) = 1$. By Bézout (Theorem 2.11) there are u, v with $au + nv = 1$. Reducing modulo n , $au \equiv 1 \pmod{n}$, so $[u]_n$ is an inverse of $[a]_n$. The extended Euclidean algorithm (Theorem 2.13) produces such a u explicitly.

Conversely, if $[a]_n$ is a unit, say $ax \equiv 1 \pmod{n}$, then $ax - 1 = nk$ for some k , i.e. $ax - nk = 1$. Any common divisor of a and n divides the left side, hence divides 1; thus $\gcd(a, n) = 1$.

Uniqueness: if $ax \equiv 1$ and $ax' \equiv 1 \pmod{n}$, then $x \equiv x(ax') \equiv (xa)x' \equiv x' \pmod{n}$ —a computation using only the associativity and commutativity of integer multiplication together with Proposition 2.21—so $[x]_n = [x']_n$. $\square$

Example 2.25. Consider the computation of $[17]_{43}^{-1}$. The extended Euclidean algorithm on $(43, 17)$:

$$\begin{aligned} 43 &= 2 \cdot 17 + 9, \\ 17 &= 1 \cdot 9 + 8, \\ 9 &= 1 \cdot 8 + 1, \\ 8 &= 8 \cdot 1 + 0, \end{aligned}$$

so $\gcd(43, 17) = 1$. Back-substituting:

$$1 = 9 - 8 = 9 - (17 - 9) = 2 \cdot 9 - 17 = 2(43 - 2 \cdot 17) - 17 = 2 \cdot 43 - 5 \cdot 17.$$

Hence $-5 \cdot 17 \equiv 1 \pmod{43}$, and $[17]_{43}^{-1} = [-5]_{43} = [38]_{43}$. Check: $17 \cdot 38 = 646 = 15 \cdot 43 + 1$, so $17 \cdot 38 \equiv 1 \pmod{43}$.

Later sections use the number of units.

Definition 2.26 (Euler totient). *Euler’s totient function* $\varphi(n)$ is the number of integers a with $1 \leq a \leq n$ and $\gcd(a, n) = 1$. Equivalently, $\varphi(n) = |(\mathbb{Z}/n\mathbb{Z})^\times|$.

The two counts agree: the classes of $1, 2, \dots, n$ run exactly once through all of $\mathbb{Z}/n\mathbb{Z}$ (the class of n being $[0]_n$), and by Theorem 2.24 the class $[a]_n$ is a unit precisely when $\gcd(a, n) = 1$.

Example 2.27. We have $\varphi(1) = 1$, and $\varphi(p) = p - 1$ for prime p : for $1 \leq a < p$ the positive divisor $\gcd(a, p)$ of p is 1 or p , and $p \nmid a$ rules out p , so every one of the $p - 1$ nonzero classes is a unit. By direct enumeration, $\varphi(12) = |\{1, 5, 7, 11\}| = 4$.

Division by a modulo n is therefore multiplication by the inverse class $[a]_n^{-1}$. By Theorem 2.24 that inverse exists precisely when $\gcd(a, n) = 1$, hence for every nonzero class when the modulus is prime, and the extended Euclidean algorithm computes it in $O(\log n)$ division steps (Remark 2.10). The laws used in this section—associativity, commutativity, distributivity, identities, negatives, and inverses—are the defining laws of the structures that Sections 3 and 4 introduce, with $\mathbb{Z}/n\mathbb{Z}$ as an instance.

3 Groups

This section defines the *group*, a set with one associative operation that has an identity and inverses, and proves the results on groups that the series uses: Lagrange's theorem (Theorem 3.29), with the corollary on which the rest of the volume relies, Euler's theorem and Fermat's little theorem (Corollary 3.32); the subgroups and the classification of cyclic groups (Theorems 3.24 and 3.43); the quotient of an abelian group by a subgroup; and the basic theory of homomorphisms and isomorphisms. Almost every object met later in this series is a group: the integers modulo a prime under addition, the nonzero residues under multiplication, the points of an elliptic curve, the symmetries of a finite field. Key agreement over a public channel needs this structure: associativity makes repeated combination of a public element with itself independent of bracketing, and identity and inverses make repetition counts obey an arithmetic.

3.1 The group axioms

Definition 3.1 (Binary operation). Let S be a set. A *binary operation* on S is a function

$$*: S \times S \rightarrow S, \quad (a, b) \mapsto a * b.$$

Closure, the property that $a * b$ lies in S , is part of the statement that $*$ maps *into* S . It is named separately because $*$ restricts to a binary operation on a subset $H \subseteq S$ exactly when H is *closed* under $*$, the condition on which subgroups (§3.4) rest.

Definition 3.2 (Group). A *group* is a pair $(G, *)$ consisting of a set G and a binary operation $*$ on G satisfying three axioms.

(G1) *Associativity*. For all $a, b, c \in G$, $(a * b) * c = a * (b * c)$.

(G2) *Identity*. There exists $e \in G$ such that $e * a = a * e = a$ for all $a \in G$.

(G3) *Inverses*. For each $a \in G$ there exists $b \in G$ such that $a * b = b * a = e$, where e is an identity as in (G2).

A set with an operation satisfying only (G1) is a *semigroup*; if it also satisfies (G2) it is a *monoid*. A group is thus a monoid in which every element is invertible.

The number of elements of G , written $|G|$, is the *order* of the group; if $|G|$ is finite, G is a *finite group*, and otherwise an *infinite group*. When the operation is understood, one writes “the group G ”.

The axioms assert only the *existence* of an identity and of inverses; their uniqueness, proved next, justifies the notations e and a^{-1} .

Proposition 3.3. *Let $(G, *)$ be a group.*

1. *The identity element is unique.*
2. *Each $a \in G$ has a unique inverse, denoted a^{-1} .*
3. (Cancellation.) *For all $a, x, y \in G$: if $a * x = a * y$ then $x = y$, and if $x * a = y * a$ then $x = y$.*
4. *For all $a, b \in G$, $(a^{-1})^{-1} = a$ and $(a * b)^{-1} = b^{-1} * a^{-1}$.*

Proof. (1) Suppose e and e' both satisfy (G2). Using that e' is an identity, $e = e * e'$; using that e is an identity, $e * e' = e'$. Hence $e = e'$.

(2) Suppose b and b' are both inverses of a . Then

$$b = b * e = b * (a * b') = (b * a) * b' = e * b' = b',$$

using (G2), the inverse property of b' , associativity, and the inverse property of b .

(3) Suppose $a * x = a * y$. Multiplying on the left by a^{-1} and reassociating,

$$x = (a^{-1} * a) * x = a^{-1} * (a * x) = a^{-1} * (a * y) = (a^{-1} * a) * y = y.$$

The right-cancellation law is proved symmetrically, multiplying on the right by a^{-1} .

(4) Since $a * a^{-1} = a^{-1} * a = e$, the element a is an inverse of a^{-1} ; by (2) it is *the* inverse, so $(a^{-1})^{-1} = a$. For the product formula, compute

$$(a * b) * (b^{-1} * a^{-1}) = a * (b * b^{-1}) * a^{-1} = a * e * a^{-1} = e,$$

and symmetrically $(b^{-1} * a^{-1}) * (a * b) = e$; by (2), $(a * b)^{-1} = b^{-1} * a^{-1}$. The reversal of order is known as the “socks–shoes” rule. $\square$

Two notational conventions are standard. *Multiplicative* notation writes the operation as juxtaposition ab (or $a \cdot b$), the identity as 1, the inverse as a^{-1} , and powers as $a^n = \underbrace{a \cdots a}_{n \text{ factors}}$ for $n \geq 1$, with $a^0 = 1$ and $a^{-n} = (a^{-1})^n$. *Additive* notation, reserved almost exclusively for commutative groups because the symbol $+$ connotes commutativity, writes $a + b$, the identity as 0, the inverse as $-a$, and na for the n -fold sum. The exponent laws

$$a^m a^n = a^{m+n}, \quad (a^m)^n = a^{mn}$$

hold for all integers m, n , proved by induction together with Proposition 3.3. Henceforth ab denotes $a * b$, and the symbol $*$ is omitted unless needed for clarity.

The power notation presupposes that an n -fold product does not depend on its bracketing, whereas axiom (G1) concerns only three factors. The next proposition proves this; thereafter products and powers are written without brackets.

Proposition 3.4 (Generalised associativity). *Let $a_1, \dots, a_n$, $n \geq 1$, be elements of a group—or of any set with an associative operation. Every way of bracketing the product $a_1 a_2 \dots a_n$ yields the same element, namely the left-normed product $L_n = (\dots ((a_1 a_2) a_3) \dots) a_n$. In particular a^n is a single well-defined element, however the n factors are combined.*

Proof. By strong induction on n . For $n \leq 2$ there is only one bracketing, and for $n = 3$ the two bracketings agree by (G1). Let $n \geq 3$ and assume the claim for all shorter products. Any bracketed product P of $a_1, \dots, a_n$ has an outermost multiplication, splitting it as $P = QR$ with Q a bracketed product of $a_1, \dots, a_k$ and R one of $a_{k+1}, \dots, a_n$, for some $1 \leq k < n$. The induction hypothesis evaluates both halves: $Q = L_k$, and R is the left-normed product of $a_{k+1}, \dots, a_n$. If $k = n - 1$, then $R = a_n$ and $P = L_{n-1} a_n = L_n$ directly. If $k < n - 1$, write $R = R' a_n$ with R' the left-normed product of $a_{k+1}, \dots, a_{n-1}$; then (G1) and the induction hypothesis, applied to the bracketed product $L_k R'$ of the $n - 1$ elements $a_1, \dots, a_{n-1}$, give

$$P = L_k(R' a_n) = (L_k R') a_n = L_{n-1} a_n = L_n.$$

Every bracketing therefore equals L_n . □

Definition 3.5 (Abelian group). A group $(G, *)$ is *abelian* (or *commutative*) if $a * b = b * a$ for all $a, b \in G$; otherwise it is *non-abelian*.

The term honours Niels Henrik Abel. In an abelian group the factors of a product may be reordered arbitrarily, and the socks–shoes formula becomes $(ab)^{-1} = b^{-1} a^{-1} = a^{-1} b^{-1}$.

3.2 First examples

Example 3.6. The pair $(\mathbb{Z}, +)$ is an infinite abelian group: addition is associative and commutative, the identity is 0, and the inverse of n is $-n$. Likewise $(\mathbb{Q}, +)$, $(\mathbb{R}, +)$, and $(\mathbb{C}, +)$ are abelian groups. By contrast $(\mathbb{Z}, \cdot)$ is *not* a group: the integer 2 has no multiplicative inverse in $\mathbb{Z}$.

Example 3.7. The nonzero rationals $\mathbb{Q}^\times = \mathbb{Q} \setminus \{0\}$ form an abelian group under multiplication, with identity 1 and inverse $1/a$; likewise $\mathbb{R}^\times$ and $\mathbb{C}^\times$. Zero must be excluded because it has no multiplicative inverse. More generally, for any field F (for example $\mathbb{Q}, \mathbb{R}, \mathbb{C}$; the abstract notion is defined in Section 4) the nonzero elements $F^\times = F \setminus \{0\}$ form an abelian group under multiplication. This property is part of the definition of a field.

Most finite examples below are built from congruence classes. On the set $\mathbb{Z}/n\mathbb{Z}$ of residue classes $[a]_n$ of Section 2, define operations through representatives,

$$[a]_n + [b]_n = [a + b]_n, \quad [a]_n [b]_n = [ab]_n.$$

By Proposition 2.21 the right-hand sides do not depend on the representatives chosen, so both operations are well defined, and each law of integer arithmetic—associativity, commutativity, the identities $[0]_n$ and $[1]_n$, negatives—passes to classes by computing on representatives. (The full structural statement, that these operations make $\mathbb{Z}/n\mathbb{Z}$ a commutative ring, is Theorem 4.6 in Section 4; the present section uses only the laws listed above.) Under addition, all of $\mathbb{Z}/n\mathbb{Z}$ is a group, as Example 3.9 below records. For $n > 1$, multiplication does not make it a group, since $[0]_n$ has no inverse; Section 2 identified the invertible classes as the *units*, the classes $[a]_n$ with $\gcd(a, n) = 1$ (Theorem 2.24). The units form a group.

Proposition 3.8. *Under multiplication of classes, the set $(\mathbb{Z}/n\mathbb{Z})^\times$ of units of $\mathbb{Z}/n\mathbb{Z}$ is an abelian group. Its order is $|(\mathbb{Z}/n\mathbb{Z})^\times| = \varphi(n)$, Euler's totient of Definition 2.26: the number of integers a with $1 \leq a \leq n$ and $\gcd(a, n) = 1$.*

Proof. Multiplication of classes is associative and commutative with identity $[1]_n$, as derived above from Proposition 2.21 by computing on representatives, and $[1]_n$ is a unit (it is its own inverse). The operation is closed on units: if $[a]_n$ and $[b]_n$ are units with inverses $[a]_n^{-1}$ and $[b]_n^{-1}$, then $[ab]_n$ has inverse $[b]_n^{-1}[a]_n^{-1}$, since $(ab)(b^{-1}a^{-1}) \equiv 1 \pmod{n}$ for any representatives of the inverse classes. Every unit has an inverse by definition, and that inverse is itself a unit (it is invertible, with inverse the original class). Hence $(\mathbb{Z}/n\mathbb{Z})^\times$ is an abelian group. By Theorem 2.24 its elements are exactly the classes $[a]_n$ with $\gcd(a, n) = 1$ and $1 \leq a \leq n$, of which there are $\varphi(n)$ by Definition 2.26. $\square$

Example 3.9. The pair $(\mathbb{Z}/n\mathbb{Z}, +)$ is an abelian group of order n : the discussion above derived associativity, commutativity, the identity $[0]_n$, and negatives from Proposition 2.21, and the classes number exactly n (the least nonnegative residues, Definition 2.22). The invertible classes form the abelian group $(\mathbb{Z}/n\mathbb{Z})^\times$ of order $\varphi(n)$ under multiplication (Theorem 2.24 and Proposition 3.8). When $n = p$ is prime, every nonzero class is a unit (Example 2.27), so $(\mathbb{Z}/p\mathbb{Z})^\times$ has order $p - 1$ —this is Example 3.7 specialised to $\mathbb{Z}/p\mathbb{Z}$, which Section 4 shows to be a field and denotes $\mathbb{F}_p$.

Example 3.10. The one-element set $\{e\}$, with its only possible operation $e * e = e$, is the *trivial group*. The group $\mathbb{Z}/2\mathbb{Z} = \{0, 1\}$ is the smallest nontrivial group. A further example of small order is the *Klein four-group* $V = \{e, a, b, c\}$, in which every element composed with itself gives e and the product of any two distinct nonidentity elements is the third; the verification that this table satisfies the axioms is routine. The group V is abelian of order 4.

The groups V and $\mathbb{Z}/4\mathbb{Z}$ both have order 4 but are not isomorphic, that is, structurally different (§3.8): in V every element combined with itself gives the identity, so V has no element of order 4 in the sense defined next, whereas in $\mathbb{Z}/4\mathbb{Z}$ the class 1 must be added to itself four times to give 0.

3.3 The order of an element

Definition 3.11 (Order of an element). Let G be a group and $a \in G$. The *order* of a , written $\text{ord}(a)$, is the least positive integer n with $a^n = e$, if such an n exists; a then has *finite order*. If no positive power of a equals e , then a has *infinite order* and one writes $\text{ord}(a) = \infty$.

The order $\text{ord}(a)$ of an element shares its name with the order $|G|$ of a group because it equals the order of the smallest subgroup of G containing a (Proposition 3.21).

Proposition 3.12. *Let a have finite order $d = \text{ord}(a)$, and let $m, k \in \mathbb{Z}$.*

1. $a^m = e$ if and only if $d \mid m$.
2. $a^m = a^k$ if and only if $m \equiv k \pmod{d}$.
3. The distinct powers of a are exactly $e = a^0, a^1, \dots, a^{d-1}$.

Proof. (1) If $d \mid m$, write $m = dq$; then $a^m = (a^d)^q = e^q = e$. Conversely, suppose $a^m = e$ and divide with remainder (Theorem 2.4): $m = dq + r$ with $0 \leq r < d$. Then

$$e = a^m = (a^d)^q a^r = a^r,$$

so r is a nonnegative integer smaller than d with $a^r = e$. Minimality of d among *positive* exponents forces $r = 0$, whence $d \mid m$.

(2) We have $a^m = a^k$ if and only if $a^{m-k} = e$ (multiply by a^{-k} and use the exponent laws), which by (1) holds if and only if $d \mid (m - k)$, i.e. $m \equiv k \pmod{d}$ (Definition 2.20).

(3) By (2) the powers $a^0, a^1, \dots, a^{d-1}$ are pairwise distinct, their exponents being pairwise incongruent modulo d ; and every power a^m equals $a^{m \bmod d}$, again by (2). $\square$

The final step of part (1) is the *division-minimality argument*: to show that every witness of some property is a multiple of the least positive witness m , divide an arbitrary witness $t = mq + r$ with $0 \leq r < m$, show that r is itself a witness, and conclude $r = 0$ by minimality. It has the form of the proof of Bézout's identity (Theorem 2.11), carried over to a group, and recurs for the least positive power of g lying in a subgroup (Theorem 3.24) and for the least positive element of a subgroup of $\mathbb{Z}$ (Proposition 3.26).

Example 3.13. In $(\mathbb{Z}/n\mathbb{Z}, +)$, with the powers of Definition 3.11 read additively, the k -fold sum of the class 1 is the class k , which is 0 exactly when $n \mid k$, so $\text{ord}(1) = n$. More generally $\text{ord}(a) = n / \gcd(a, n)$ for any class a , by Corollary 3.23 below read additively. In $\mathbb{C}^\times$ the element i has order 4: its successive powers are

$$i^1 = i, \quad i^2 = -1, \quad i^3 = -i, \quad i^4 = 1.$$

In $(\mathbb{Z}, +)$ every nonzero element has infinite order: no nonzero multiple of a nonzero integer is 0.

3.4 Subgroups

Definition 3.14 (Subgroup). A subset $H \subseteq G$ of a group $(G, *)$ is a *subgroup*, written $H \leq G$, if H is itself a group under the restriction of $*$ to $H \times H$: explicitly, H is nonempty, closed under $*$, contains the identity e of G , and contains a^{-1} whenever it contains a . Every group has the *trivial* subgroup $\{e\}$ and G itself; a subgroup other than G is *proper*.

The definition is unambiguous. The identity of a subgroup coincides with the identity of G : if $e_H \in H$ satisfies $e_H * e_H = e_H$, then cancelling e_H in G against $e_H * e = e_H$ (Proposition 3.3(3)) gives $e_H = e$. Inverses computed in H agree with those computed in G , by uniqueness of inverses in G .

Since a subset inherits associativity from G , a single condition suffices.

Proposition 3.15 (One-step subgroup test). *Let G be a group and $H \subseteq G$. Then $H \leq G$ if and only if*

1. H is nonempty, and
2. for all $a, b \in H$, the element ab^{-1} lies in H .

Proof. If $H \leq G$, then H is nonempty and, for $a, b \in H$, it contains b^{-1} and hence ab^{-1} ; both conditions hold.

Conversely, suppose (1) and (2) hold. Pick $a \in H$ by (1); taking $b = a$ in (2) gives $e = aa^{-1} \in H$. For any $b \in H$, taking the pair (e, b) in (2) gives $b^{-1} = eb^{-1} \in H$, so H is closed under inverses. Finally, for $a, b \in H$ the previous step gives $b^{-1} \in H$, and applying (2) to the pair (a, b^{-1}) gives $a(b^{-1})^{-1} = ab \in H$, so H is closed under the operation. Associativity is inherited from G , so H is a group. $\square$

Remark 3.16. For a *finite* nonempty subset H , closure under the operation alone forces $H \leq G$. Given $a \in H$, closure puts all the powers $a, a^2, a^3, \dots$ in H ; since H is finite they cannot all be distinct, so $a^i = a^j$ for some $i < j$, giving $a^{j-i} = e \in H$. Then $a^{-1} = a^{j-i-1} \in H$: this is a positive power of a when $j - i \geq 2$, and when $j - i = 1$ we have $a = e$, whose inverse e already lies in H .

Example 3.17. The even integers $2\mathbb{Z} = \{2k : k \in \mathbb{Z}\}$ form a subgroup of $(\mathbb{Z}, +)$: the set is nonempty, and $2k - 2l = 2(k - l)$ is again even (the subgroup test, written additively). More generally $n\mathbb{Z} = \{nk : k \in \mathbb{Z}\} \leq \mathbb{Z}$ for every $n \geq 0$, by the same computation—and these are *all* the subgroups of $\mathbb{Z}$, as Proposition 3.26 will show.

Proposition 3.18. *The intersection $\bigcap_{i \in I} H_i$ of any family of subgroups $H_i \leq G$ is a subgroup of G .*

Proof. The identity e lies in every H_i , so the intersection is nonempty. If a and b lie in the intersection, then for each i both lie in H_i , so $ab^{-1} \in H_i$ by Proposition 3.15; hence ab^{-1} lies in the intersection, and the one-step test applies. $\square$

Definition 3.19 (Generated subgroup). For a subset $S \subseteq G$, the *subgroup generated by S* , written $\langle S \rangle$, is the intersection of all subgroups of G containing S . By Proposition 3.18 it is a subgroup, and by construction it is the smallest subgroup of G containing S .

Concretely, $\langle S \rangle$ consists of all finite products $s_1^{\varepsilon_1} s_2^{\varepsilon_2} \dots s_k^{\varepsilon_k}$ with $s_i \in S$ and $\varepsilon_i \in \{+1, -1\}$, the empty product being e . The set of such products contains S , contains e , and is closed under multiplication (concatenate) and under inversion (reverse the factors and negate the exponents, by the socks–shoes rule), so it is a subgroup containing S ; conversely, every subgroup containing S is closed under these operations and so contains every such product. Hence the two descriptions coincide.

3.5 Cyclic groups

Definition 3.20 (Cyclic group). A group G is *cyclic* if there exists $g \in G$ with

$$G = \langle g \rangle = \{g^k : k \in \mathbb{Z}\};$$

such an element g is a *generator* of G .

The displayed equality agrees with Definition 3.19: the set $\{g^k : k \in \mathbb{Z}\}$ contains g and is closed under products and inverses by the exponent laws, so it is a subgroup containing g ; conversely, any subgroup containing g contains all its powers. Every cyclic group is abelian, since $g^m g^n = g^{m+n} = g^n g^m$.

Proposition 3.21. Let $G = \langle g \rangle$ be cyclic.

1. If g has infinite order, the powers g^k ($k \in \mathbb{Z}$) are pairwise distinct, so G is infinite.
2. If g has finite order d , then $G = \{e, g, g^2, \dots, g^{d-1}\}$ has exactly d elements.

In both cases $|\langle g \rangle| = \text{ord}(g)$.

Proof. (1) Suppose $g^i = g^j$ with $i \neq j$; without loss of generality $i > j$. Then $g^{i-j} = e$ with $i - j > 0$, contradicting infinite order. So the powers are pairwise distinct and G is infinite.

(2) This is Proposition 3.12(3): the distinct powers of g are exactly $g^0, g^1, \dots, g^{d-1}$, and these exhaust $G = \{g^k : k \in \mathbb{Z}\}$. $\square$

Example 3.22. The group $(\mathbb{Z}, +)$ is infinite cyclic, generated by 1 and also by -1 . The group $(\mathbb{Z}/n\mathbb{Z}, +)$ is cyclic of order n , generated by 1 (Example 3.13). The unit group $(\mathbb{Z}/8\mathbb{Z})^\times = \{1, 3, 5, 7\}$ is *not* cyclic: each of its four elements squares to 1 ($3^2 = 9 \equiv 1$, $5^2 = 25 \equiv 1$, $7^2 = 49 \equiv 1 \pmod{8}$), so no element has order 4. By contrast $(\mathbb{Z}/5\mathbb{Z})^\times = \{1, 2, 3, 4\}$ is cyclic, generated by 2: its powers run

$$2^1 \equiv 2, \quad 2^2 \equiv 4, \quad 2^3 \equiv 3, \quad 2^4 \equiv 1 \pmod{5},$$

exhausting the group.

Corollary 3.23. Let g have finite order n and let $k \in \mathbb{Z}$. Then

$$\text{ord}(g^k) = \frac{n}{\gcd(n, k)}.$$

In particular g^k generates $\langle g \rangle$ if and only if $\gcd(n, k) = 1$, so a cyclic group of order n has exactly $\phi(n)$ generators.

Proof. Set $d = \gcd(n, k)$ and write $k = dk'$, $n = dn'$ with $\gcd(n', k') = 1$. For $m \geq 1$,

$$(g^k)^m = g^{km} = e \iff n \mid km \iff dn' \mid dk'm \iff n' \mid k'm \iff n' \mid m,$$

using Proposition 3.12(1) for the first equivalence and Euclid's lemma (Proposition 2.15(1), with $\gcd(n', k') = 1$) for the last. The least positive such m is $n' = n/d$, which is therefore $\text{ord}(g^k)$.

For the second claim, $\langle g^k \rangle \subseteq \langle g \rangle$ always, and by Proposition 3.21 the subgroup $\langle g^k \rangle$ has $\text{ord}(g^k) = n / \gcd(n, k)$ elements; it equals the n -element set $\langle g \rangle$ exactly when $\gcd(n, k) = 1$. Since the powers $g^1, g^2, \dots, g^n$ run through each element of $\langle g \rangle$ exactly once (Proposition 3.12(3)), the generators correspond to the $k \in \{1, \dots, n\}$ with $\gcd(n, k) = 1$, of which there are $\varphi(n)$ by Definition 2.26. $\square$

Theorem 3.24 (Subgroups of cyclic groups). *Let $G = \langle g \rangle$ be a cyclic group.*

1. *Every subgroup of G is cyclic.*
2. *If G is infinite, its subgroups are exactly $\langle g^m \rangle$ for $m \geq 0$; distinct $m \geq 0$ give distinct subgroups, and for $m \geq 1$ the subgroup $\langle g^m \rangle$ is isomorphic to $(\mathbb{Z}, +)$.*
3. *If $|G| = n$, then for each positive divisor $d \mid n$ there is exactly one subgroup of G of order d , namely $\langle g^{n/d} \rangle$, and these are all the subgroups of G . Subgroups of G thus correspond bijectively to positive divisors of n .*

Proof. (1) Let $H \leq G$. If $H = \{e\}$, then $H = \langle e \rangle$ is cyclic. Otherwise H contains some $g^t \neq e$, so $t \neq 0$; since H also contains $(g^t)^{-1} = g^{-t}$, it contains a power of g with positive exponent. Let m be the least positive integer with $g^m \in H$ (well-ordering, Theorem 2.1). We claim $H = \langle g^m \rangle$. The inclusion $\supseteq$ holds by closure. For $\subseteq$, take any element of H ; it is some g^t , since $H \subseteq G = \langle g \rangle$. Divide $t = mq + r$ with $0 \leq r < m$; then

$$g^r = g^{t-mq} = g^t (g^m)^{-q} \in H,$$

and minimality of m forces $r = 0$, by the division-minimality argument. Hence $g^t = (g^m)^q \in \langle g^m \rangle$.

(2) Let G be infinite. Then g has infinite order, for otherwise G would be finite by Proposition 3.21(2). By part (1) every subgroup is $\{e\} = \langle g^0 \rangle$ or $\langle g^m \rangle$ with $m \geq 1$ least positive, and conversely each $\langle g^m \rangle$ is a subgroup. For distinctness, observe that for $m \geq 1$ the positive exponents t with $g^t \in \langle g^m \rangle$ are exactly the positive multiples of m : the elements of $\langle g^m \rangle$ are the powers g^{mk} , and $g^t = g^{mk}$ forces $t = mk$ because distinct exponents give distinct powers (Proposition 3.21(1)). The least such exponent recovers m , so distinct $m \geq 1$ give distinct subgroups, and each differs from $\{e\}$ because $g^m \neq e$. Finally, for $m \geq 1$ the element g^m has infinite order (if $(g^m)^t = g^{mt} = e$ then $mt = 0$, so $t = 0$), so $\langle g^m \rangle$ is an infinite cyclic group and is therefore isomorphic to $(\mathbb{Z}, +)$ by Theorem 3.43(1) below. No circularity arises: the proof of that theorem uses only Propositions 3.12 and 3.21.

(3) Let $|G| = n$, so $\text{ord}(g) = n$ by Proposition 3.21.

Existence. Let $d \mid n$ be a positive divisor. Since n/d divides n , we have $\gcd(n, n/d) = n/d$, so Corollary 3.23 gives

$$\text{ord}(g^{n/d}) = \frac{n}{\gcd(n, n/d)} = \frac{n}{n/d} = d,$$

and $\langle g^{n/d} \rangle$ is a subgroup of order d by Proposition 3.21.

Uniqueness. Let $H \leq G$ with $|H| = d$. If $d = 1$ then $H = \{e\} = \langle g^n \rangle = \langle g^{n/1} \rangle$, since $g^n = e$. Otherwise, as in part (1), $H = \langle g^m \rangle$ for the least positive m with $g^m \in H$, and Propositions 3.12 and 3.21 together with Corollary 3.23 give

$$d = |H| = \text{ord}(g^m) = \frac{n}{\gcd(n, m)}, \quad \text{so} \quad \gcd(n, m) = \frac{n}{d}.$$

In particular n/d divides m , say $m = (n/d) m'$; then $g^m = (g^{n/d})^{m'}$, so $H = \langle g^m \rangle \subseteq \langle g^{n/d} \rangle$. Both sides have exactly d elements, so $H = \langle g^{n/d} \rangle$.

All subgroups arise. Any $H \leq G$ equals some $\langle g^m \rangle$ (or $\{e\}$), whose order $n / \gcd(n, m)$ is a positive divisor d of n ; by uniqueness $H = \langle g^{n/d} \rangle$. The assignment $d \mapsto \langle g^{n/d} \rangle$ is therefore surjective onto the set of subgroups, and it is injective because subgroups of different orders are distinct. This is the asserted bijection. $\square$

Remark 3.25. Specialised to $G = \mathbb{Z}/n\mathbb{Z}$ with generator 1 (written additively), Theorem 3.24(3) reads: for each divisor $d \mid n$ there is exactly one subgroup of order d , namely

$$\langle n/d \rangle = \{0, n/d, 2n/d, \dots, (d-1)n/d\},$$

itself cyclic of order d , hence isomorphic to $\mathbb{Z}/d\mathbb{Z}$ by the classification below. For $n = 12$ the six subgroups correspond to the divisors 1, 2, 3, 4, 6, 12, and inclusion between subgroups mirrors divisibility between the corresponding divisors: the subgroup lattice of $\mathbb{Z}/12\mathbb{Z}$ is the divisor lattice of 12, with chains such as $1 \mid 2 \mid 4, 1 \mid 2 \mid 6$, and $3 \mid 6$.

The classification of the subgroups of $\mathbb{Z}$ announced in Example 3.17 is Theorem 3.24(2) read additively for $G = (\mathbb{Z}, +) = \langle 1 \rangle$; a direct proof follows.

Proposition 3.26. *Every subgroup of $(\mathbb{Z}, +)$ has the form $n\mathbb{Z}$ for a unique integer $n \geq 0$.*

Proof. Let $H \leq \mathbb{Z}$. If $H = \{0\}$, then $H = 0\mathbb{Z}$. Otherwise H contains a nonzero integer and, being closed under negation, contains a positive one; let n be its least positive element (Theorem 2.1). Closure under addition and negation gives $n\mathbb{Z} \subseteq H$. Conversely, for any $h \in H$, divide $h = nq + r$ with $0 \leq r < n$; then $r = h - nq \in H$, and minimality of n forces $r = 0$, so $h \in n\mathbb{Z}$. Hence $H = n\mathbb{Z}$. For uniqueness, the integer n is recovered from H as its least positive element (and $n = 0$ exactly for $H = \{0\}$), so distinct $n \geq 0$ give distinct subgroups. $\square$

3.6 Cosets and Lagrange's theorem

The cosets of a subgroup $H \leq G$, defined next, partition G into blocks of size $|H|$; counting the blocks gives Lagrange's theorem.

Definition 3.27 (Cosets and index). Let $H \leq G$ and $a \in G$. The *left coset* of H by a is

$$aH = \{ah : h \in H\},$$

and the *right coset* is $Ha = \{ha : h \in H\}$. In additive notation one writes $a + H = \{a + h : h \in H\}$. The set of left cosets is denoted G/H , and the *index* of H in G is $[G : H] = |G/H|$, the number of left cosets.

Lemma 3.28 (Cosets partition the group). *Let $H \leq G$.*

1. *Each $a \in G$ lies in aH ; the left cosets cover G .*
2. *Two left cosets are either equal or disjoint: $aH = bH$ if and only if $a^{-1}b \in H$, and otherwise $aH \cap bH = \emptyset$.*
3. *Every left coset has the same cardinality as H : the map $h \mapsto ah$ is a bijection from H onto aH .*

The same statements hold for right cosets, with $ba^{-1} \in H$ as the criterion in (2).

Proof. (1) Since $e \in H$, we have $a = ae \in aH$.

(2) First suppose the cosets meet: $ab_1 = bb_2$ for some $b_1, b_2 \in H$. Then $a^{-1}b = b_1b_2^{-1} \in H$. Conversely, suppose $a^{-1}b = h \in H$, so $b = ah$. Then every element $bb' \in bH$ equals $a(hb') \in aH$, giving $bH \subseteq aH$; and since $b^{-1}a = h^{-1} \in H$, the same argument with the roles of a and b exchanged gives $aH \subseteq bH$. Hence $aH = bH$. Combining the two directions: if two cosets share even one element they coincide, so distinct cosets are disjoint.

(3) The map $\lambda_a: H \rightarrow aH, h \mapsto ah$, is surjective by the definition of aH and injective by left cancellation (Proposition 3.3(3)): $ah = ah'$ implies $h = h'$. Hence $|aH| = |H|$.

For right cosets, the same three arguments apply with sides exchanged; in (2) the criterion becomes $ba^{-1} \in H$. $\square$

Theorem 3.29 (Lagrange's theorem). *Let G be a finite group and $H \leq G$. Then $|H|$ divides $|G|$, and*

$$|G| = [G : H] |H|.$$

Proof. By Lemma 3.28, the distinct left cosets of H partition G : they cover G by part (1) and are pairwise disjoint by part (2). Since G is finite there are finitely many of them, say $[G : H] = k$, and each has exactly $|H|$ elements by part (3). Summing the sizes of the blocks of the partition,

$$|G| = \sum_{\text{distinct cosets } aH} |aH| = k |H| = [G : H] |H|. \quad \square$$

Corollary 3.30. *In a finite group G , the order of every element divides $|G|$.*

Proof. For $a \in G$, the cyclic subgroup $\langle a \rangle$ has order $\text{ord}(a)$ by Proposition 3.21 (the order is finite, since $\langle a \rangle \subseteq G$ is finite), and Theorem 3.29 makes $|\langle a \rangle|$ a divisor of $|G|$. $\square$

Corollary 3.31. *If G is a finite group of order N , then $a^N = e$ for every $a \in G$.*

Proof. Let $d = \text{ord}(a)$. By Corollary 3.30, $d \mid N$, say $N = dm$; then $a^N = (a^d)^m = e^m = e$. $\square$

Corollary 3.32 (Euler's theorem; Fermat's little theorem). *Let $n \geq 1$ and $a \in \mathbb{Z}$ with $\gcd(a, n) = 1$. Then*

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

In particular, for a prime p and a not divisible by p ,

$$a^{p-1} \equiv 1 \pmod{p},$$

and $a^p \equiv a \pmod{p}$ holds for all integers a .

Proof. Apply Corollary 3.31 to the group $(\mathbb{Z}/n\mathbb{Z})^\times$, which has order $\varphi(n)$ (Proposition 3.8). For $\gcd(a, n) = 1$ the class $[a]_n$ is a unit (Theorem 2.24), so $[a]_n^{\varphi(n)} = [1]_n$, which is the first congruence. For $n = p$ prime, $\varphi(p) = p - 1$ (Example 2.27), and $p \nmid a$ gives $\gcd(a, p) = 1$, yielding the second congruence. Multiplying it by a gives $a^p \equiv a \pmod{p}$; and when $p \mid a$ both sides are congruent to 0, so the last congruence holds for all a . $\square$

Remark 3.33. The congruences of Corollary 3.32 are proved only here; Section 7 restates them under the heading *Fermat's little theorem and Euler's theorem*, cites this proof, and develops their cryptographic role.

Corollary 3.34. *Every group G of prime order p is cyclic, hence isomorphic to $\mathbb{Z}/p\mathbb{Z}$, and has no proper nontrivial subgroups.*

Proof. Pick $a \in G$ with $a \neq e$ (possible since $p \geq 2$). The subgroup $\langle a \rangle$ has at least two elements, and by Theorem 3.29 its order divides p ; a divisor of p exceeding 1 equals p , so $\langle a \rangle = G$ and G is cyclic. The isomorphism with $\mathbb{Z}/p\mathbb{Z}$ is Theorem 3.43 below. Finally, any subgroup has order 1 or p by Lagrange, i.e. is $\{e\}$ or G . $\square$

3.7 Quotients of abelian groups

The quotient of an abelian group by a subgroup underlies the quotient rings of Section 4 and, through them, $\mathbb{Z}/n\mathbb{Z}$ and the polynomial quotients used throughout the series.

Proposition 3.35 (Quotient of an abelian group). *Let A be an abelian group, written additively, and let $H \leq A$. The operation*

$$(a + H) + (b + H) = (a + b) + H$$

on the coset set A/H is well defined and makes A/H an abelian group—the quotient group of A by H —with identity $H = 0 + H$ and inverse $-(a + H) = (-a) + H$. If A is finite, then

$$|A/H| = [A : H] = \frac{|A|}{|H|}.$$

Proof. The substantive claim is that the operation is well defined: its output is stated in terms of chosen representatives a and b , and replacing them by other representatives of the same cosets

must leave the result unchanged. Suppose then that $a + H = a' + H$ and $b + H = b' + H$. By Lemma 3.28(2), written additively, this means $a' - a \in H$ and $b' - b \in H$. Then

$$(a' + b') - (a + b) = (a' - a) + (b' - b) \in H,$$

since H is closed under addition; applying Lemma 3.28(2) once more, $(a' + b') + H = (a + b) + H$. The operation is well defined. This representative-independence check is the template for every coset-wise construction in the volume; it recurs, in the same words, for quotient-ring multiplication in Section 4.

The axioms then pass coset-wise from A : for associativity,

$$\begin{aligned} ((a + H) + (b + H)) + (c + H) &= ((a + b) + c) + H = (a + (b + c)) + H \\ &= (a + H) + ((b + H) + (c + H)); \end{aligned}$$

commutativity follows the same way from commutativity in A ; the coset $0 + H = H$ is an identity; and $(-a) + H$ is an inverse of $a + H$. Hence A/H is an abelian group. When A is finite, the number of cosets is $[A : H] = |A|/|H|$ by Theorem 3.29—the same partition-into-equal-blocks count as before. $\square$

For $A = (\mathbb{Z}, +)$ and $H = n\mathbb{Z}$, the cosets are the residue classes $[a]_n = a + n\mathbb{Z}$ of Definition 2.22 and coset addition is addition of classes. The notation $\mathbb{Z}/n\mathbb{Z}$, introduced in Section 2 as a name for the set of classes, is thus an instance of the quotient notation A/H , and $(\mathbb{Z}/n\mathbb{Z}, +)$ is the quotient group of $\mathbb{Z}$ by $n\mathbb{Z}$.

Remark 3.36. For a non-abelian group G the same construction succeeds exactly for those subgroups $N \leq G$ whose left and right cosets coincide, $gN = Ng$ for all $g \in G$; these are the *normal* subgroups. Commutativity makes every subgroup of an abelian group normal, which is why no extra hypothesis was needed in Proposition 3.35. The present volume needs only the abelian case and does not develop the general theory.

3.8 Homomorphisms and isomorphisms

Groups are compared through the maps that respect their operations.

Definition 3.37 (Group homomorphism). Let $(G, *)$ and $(G', \circ)$ be groups. A function $\phi: G \rightarrow G'$ is a *group homomorphism* if

$$\phi(a * b) = \phi(a) \circ \phi(b) \quad \text{for all } a, b \in G.$$

The set of all homomorphisms from G to G' is denoted $\text{Hom}(G, G')$.

Proposition 3.38. Let $\phi: G \rightarrow G'$ be a homomorphism, and write e, e' for the identities of G and G' .

1. $\phi(e) = e'$.
2. $\phi(a^{-1}) = \phi(a)^{-1}$ for all $a \in G$.

3. $\phi(a^n) = \phi(a)^n$ for all $a \in G$ and $n \in \mathbb{Z}$.

4. The image $\phi(G)$ is a subgroup of G' .

Proof. (1) From $e = e * e$ we get $\phi(e) = \phi(e) \circ \phi(e)$; cancelling $\phi(e)$ in G' (Proposition 3.3(3)) gives $e' = \phi(e)$.

(2) Applying ϕ to $a * a^{-1} = e = a^{-1} * a$ and using (1),

$$\phi(a) \circ \phi(a^{-1}) = e' = \phi(a^{-1}) \circ \phi(a),$$

so $\phi(a^{-1})$ is an inverse of $\phi(a)$; by uniqueness of inverses, $\phi(a^{-1}) = \phi(a)^{-1}$.

(3) For $n \geq 0$, induct: the case $n = 0$ is (1), and $\phi(a^{n+1}) = \phi(a^n * a) = \phi(a)^n \circ \phi(a) = \phi(a)^{n+1}$. For $n < 0$, write $n = -m$ with $m > 0$ and combine with (2): $\phi(a^{-m}) = \phi((a^{-1})^m) = \phi(a^{-1})^m = (\phi(a)^{-1})^m = \phi(a)^{-m}$.

(4) The image is nonempty, containing $e' = \phi(e)$. For $x = \phi(a)$ and $y = \phi(b)$ in $\phi(G)$,

$$x \circ y^{-1} = \phi(a) \circ \phi(b)^{-1} = \phi(a * b^{-1}) \in \phi(G),$$

using (2); the one-step subgroup test (Proposition 3.15) concludes. $\square$

Definition 3.39 (Kernel and image). The *kernel* and *image* of a homomorphism $\phi: G \rightarrow G'$ are

$$\ker(\phi) = \{a \in G : \phi(a) = e'\} = \phi^{-1}(\{e'\}), \quad \text{im}(\phi) = \phi(G) = \{\phi(a) : a \in G\}.$$

Proposition 3.40. For any homomorphism $\phi: G \rightarrow G'$, the kernel is a subgroup, $\ker(\phi) \leq G$, and ϕ is injective if and only if $\ker(\phi) = \{e\}$.

Proof. The kernel contains e by Proposition 3.38(1), so it is nonempty; and for $a, b \in \ker(\phi)$,

$$\phi(ab^{-1}) = \phi(a) \circ \phi(b)^{-1} = e' \circ (e')^{-1} = e',$$

so $ab^{-1} \in \ker(\phi)$; the one-step test gives $\ker(\phi) \leq G$.

If ϕ is injective and $\phi(a) = e' = \phi(e)$, then $a = e$; the kernel is trivial. Conversely, suppose $\ker(\phi) = \{e\}$ and $\phi(a) = \phi(b)$. Then $\phi(ab^{-1}) = \phi(a) \circ \phi(b)^{-1} = e'$, so $ab^{-1} \in \ker(\phi) = \{e\}$, forcing $ab^{-1} = e$, i.e. $a = b$. $\square$

Kernels are exactly the subgroups by which one may quotient: for abelian G , Proposition 3.35 forms $G/\ker(\phi)$ directly, and in general the kernel satisfies the normality condition of Remark 3.36—for $n \in \ker(\phi)$ and $g \in G$, one computes $\phi(gng^{-1}) = \phi(g) \circ e' \circ \phi(g)^{-1} = e'$, so conjugating the kernel by g lands back in the kernel, whence $g\ker(\phi) = \ker(\phi)g$.

Definition 3.41 (Isomorphism). An *isomorphism* is a bijective homomorphism $\phi: G \rightarrow G'$; when one exists, the groups are *isomorphic*, written $G \cong G'$. An isomorphism from G to itself is an *automorphism*. An injective homomorphism is a *monomorphism*; a surjective one is an *epimorphism*.

Proposition 3.42. *The set-theoretic inverse of an isomorphism is an isomorphism, and $\cong$ is an equivalence relation on any collection of groups.*

Proof. Let $\phi: G \rightarrow G'$ be an isomorphism, with inverse function ϕ^{-1} (which exists since ϕ is a bijection, Theorem 1.1). For $x, y \in G'$, write $x = \phi(a)$ and $y = \phi(b)$; then

$$\phi^{-1}(x \circ y) = \phi^{-1}(\phi(a * b)) = a * b = \phi^{-1}(x) * \phi^{-1}(y),$$

so ϕ^{-1} is a homomorphism, and it is a bijection; hence an isomorphism. For the equivalence relation: reflexivity is witnessed by the identity automorphism id_G ; symmetry by the inverse just constructed; and transitivity because a composite of homomorphisms is a homomorphism (apply the defining property twice) and a composite of bijections is a bijection. $\square$

Any property expressible through the operation alone, such as the orders of elements, the lattice of subgroups, or commutativity, transfers across an isomorphism, so groups are classified *up to isomorphism*; the next theorem does so for cyclic groups.

Theorem 3.43 (Classification of cyclic groups). *Every cyclic group is isomorphic to exactly one of the following:*

1. *the infinite cyclic group $(\mathbb{Z}, +)$, if it is infinite;*
2. *the finite cyclic group $(\mathbb{Z}/n\mathbb{Z}, +)$, if it has order n .*

In particular, two cyclic groups are isomorphic if and only if they have the same order.

Proof. Let $G = \langle g \rangle$ and define

$$\phi: \mathbb{Z} \rightarrow G, \quad \phi(k) = g^k.$$

The exponent law $g^{k+l} = g^k g^l$ says exactly that ϕ is a homomorphism from $(\mathbb{Z}, +)$ to G , and ϕ is surjective because every element of G is a power of g .

Case 1: g has infinite order. Distinct exponents give distinct powers (Proposition 3.21(1)), so ϕ is injective, hence a bijective homomorphism, and $\mathbb{Z} \cong G$.

Case 2: $\text{ord}(g) = n$. By Proposition 3.12(1),

$$\ker(\phi) = \{k \in \mathbb{Z} : g^k = e\} = n\mathbb{Z}.$$

Define the induced map

$$\bar{\phi}: \mathbb{Z}/n\mathbb{Z} \rightarrow G, \quad \bar{\phi}([k]_n) = g^k.$$

It is well defined: if $[k]_n = [l]_n$ then $n \mid (k - l)$, so $g^k = g^l$ by Proposition 3.12(2). It is a homomorphism: $\bar{\phi}([k]_n + [l]_n) = \bar{\phi}([k + l]_n) = g^{k+l} = g^k g^l = \bar{\phi}([k]_n) \bar{\phi}([l]_n)$. It is surjective because ϕ is. And it is injective by Proposition 3.40: if $\bar{\phi}([k]_n) = g^k = e$, then $n \mid k$ (Proposition 3.12(1)), so $[k]_n = [0]_n$; the kernel of $\bar{\phi}$ is trivial. Hence $\mathbb{Z}/n\mathbb{Z} \cong G$.

Distinctness. Isomorphic groups have equal order, an isomorphism being in particular a bijection. The group $\mathbb{Z}$ is infinite and the groups $\mathbb{Z}/n\mathbb{Z}$ for $n = 1, 2, 3, \dots$ have the pairwise distinct finite orders n , so no two of the listed groups are isomorphic, and a cyclic group is isomorphic to exactly

one of them—the one matching its order. The final claim follows: cyclic groups of equal order are isomorphic to the same listed group, hence to each other (Proposition 3.42), and isomorphic groups have equal order. $\square$

In key agreement over a public channel, two parties fix a finite group G and a public element $g \in G$, of order $n = \text{ord}(g)$; each combines g with itself a secret number of times, a and b respectively, publishes the result, and applies its own count to the other's. By generalised associativity (Proposition 3.4), a repetitions of g give the element g^a however bracketed, and the exponent laws give

$$(g^a)^b = g^{ab} = (g^b)^a,$$

so the two parties obtain a common secret element, while an eavesdropper sees only g and the two published values. By the classification of cyclic groups, $\langle g \rangle$ is isomorphic to $\mathbb{Z}/n\mathbb{Z}$, so exponents add, negate, and matter only modulo n (Proposition 3.12), in the modular arithmetic of Section 2. By Lagrange's theorem, n divides $|G|$. That publishing g^a hides a is not a theorem of algebra but a statement about computation: it fails in some groups, such as $(\mathbb{Z}/n\mathbb{Z}, +)$, where recovering a from ag is a single modular division, and appears to hold in others. The sections that follow and later volumes make this hiding property precise and construct the groups for which it is credible.

4 Rings and fields

An arithmetic circuit, the object a verifier checks in a proof system, is a network of gates, each adding or multiplying two previously computed values; inverting a random challenge, or cancelling a common factor from both sides of an identity, requires *division* as well. This section defines the *ring*, a set with two operations obeying the arithmetic laws of $\mathbb{Z}$, and the *field*, a ring in which division by every nonzero element is possible. It shows that the residue classes $\mathbb{Z}/n\mathbb{Z}$ of §2, in which this monograph computes, form a ring (Theorem 4.6), the quotient of $\mathbb{Z}$ by the ideal $n\mathbb{Z}$, and a field exactly when n is prime (Corollary 4.26); it proves that every finite integral domain is a field (Theorem 4.29) and that the characteristic of a domain is 0 or prime (Theorem 4.32). Polynomials, linear algebra, roots of unity, and elliptic curves are developed over fields in the sections that follow, and the finite fields $\mathbb{F}_p$ and $\mathbb{F}_q$ underlie essentially all the elliptic-curve and proof-system constructions of the monograph. From §3 we assume only the theory of *abelian* groups; additive structure is written additively with identity 0, multiplicative structure multiplicatively with identity 1.

4.1 Rings

Definition 4.1 (Ring). A *ring* is a set R equipped with two binary operations $+$ and $\cdot$ such that:

1. $(R, +)$ is an abelian group: addition is associative and commutative, there is an element 0 with $a + 0 = a$ for all a , and every a has an *additive inverse* $-a$ with $a + (-a) = 0$;
2. multiplication is associative: $a \cdot (b \cdot c) = (a \cdot b) \cdot c$;

3. there is a *multiplicative identity* 1 with $1 \cdot a = a \cdot 1 = a$ for all a ;
4. multiplication distributes over addition on both sides: $a \cdot (b + c) = a \cdot b + a \cdot c$ and $(a + b) \cdot c = a \cdot c + b \cdot c$.

The ring is *commutative* if moreover $a \cdot b = b \cdot a$ for all $a, b \in R$. We write ab for $a \cdot b$, let multiplication bind tighter than addition, and call 0 the *zero* and 1 the *unity* of the ring.

Remark 4.2. Some authors omit requirement (3), saying “ring with unity” for the notion above and “rng” for the weaker one. In this monograph *every ring has a 1*, and the structure-preserving maps between rings—the ring homomorphisms of Definition 4.21 below—are required to preserve it. In the examples used here “ring” usually means “commutative ring”; commutativity is always stated explicitly when a result depends on it.

The distributive law, the only axiom coupling the two operations, yields the following identities, which are used without comment hereafter.

Proposition 4.3 (Elementary ring arithmetic). *Let R be a ring and $a, b \in R$. Then:*

1. $0 \cdot a = a \cdot 0 = 0$;
2. $(-a)b = a(-b) = -(ab)$;
3. $(-a)(-b) = ab$;
4. for all integers $m, n \in \mathbb{Z}$, $(ma)(nb) = (mn)(ab)$, where na denotes the n -fold additive multiple ($0a = 0$, $(n+1)a = na + a$, and $(-n)a = -(na)$, as for any abelian group, §3).

Proof. (1) From $0 = 0 + 0$ and distributivity, $0 \cdot a = (0 + 0) \cdot a = 0 \cdot a + 0 \cdot a$; adding $-(0 \cdot a)$ to both sides leaves $0 = 0 \cdot a$. The computation for $a \cdot 0$ is symmetric.

(2) By distributivity and (1), $ab + (-a)b = (a + (-a))b = 0 \cdot b = 0$, so $(-a)b$ is the additive inverse of ab , that is, $(-a)b = -(ab)$; likewise $ab + a(-b) = a(b + (-b)) = 0$ gives $a(-b) = -(ab)$.

(3) Applying (2) twice, $(-a)(-b) = -(a(-b)) = -(-(ab)) = ab$, the last step because negation is an involution in the abelian group $(R, +)$.

(4) First fix a, b and show $a(nb) = n(ab)$ for $n \geq 0$ by induction: the case $n = 0$ is (1), and $a((n+1)b) = a(nb + b) = a(nb) + ab = n(ab) + ab = (n+1)(ab)$ by distributivity and the inductive hypothesis. For $n < 0$ write $n = -m$ with $m > 0$; then $a(nb) = a(-mb) = -(a(mb)) = -(m(ab)) = n(ab)$ by (2). The same argument in the first factor gives $(ma)c = m(ac)$ for all c , and combining,

$$(ma)(nb) = m(a(nb)) = m(n(ab)) = (mn)(ab),$$

the final equality being the usual bookkeeping for iterated multiples in an abelian group, which we omit. $\square$

Remark 4.4 (The zero ring). Nothing in Definition 4.1 forces $1 \neq 0$. If $1 = 0$ in a ring R , then every $a \in R$ satisfies $a = 1 \cdot a = 0 \cdot a = 0$ by Proposition 4.3(1), so $R = \{0\}$: the *zero ring* (or *trivial ring*), a ring whose two operations are the only possible ones. Whenever the zero ring must

be excluded we impose the assumption $1 \neq 0$ explicitly; the definition of a field below builds $1 \neq 0$ in as a standing assumption.

Example 4.5 (Basic rings). 1. The integers $\mathbb{Z}$ form the prototype commutative ring; much of ring theory consists of determining which properties of $\mathbb{Z}$ hold in general.

2. The rationals $\mathbb{Q}$, the reals $\mathbb{R}$, and the complexes $\mathbb{C}$ are commutative rings, and moreover fields in the sense of §4.4.

3. The residue classes $\mathbb{Z}/n\mathbb{Z} = \{\overline{0}, \overline{1}, \dots, \overline{n-1}\}$ for $n \geq 1$ —in this section’s examples we abbreviate the class $[a]_n$ of Definition 2.22 to $\overline{a}$ when the modulus is clear—under the induced operations form a commutative ring whose additive group is the cyclic group of order n (§3); this is Theorem 4.6 below. We write $\mathbb{Z}/n\mathbb{Z}$, or $\mathbb{Z}_n$ when no ambiguity arises. The case $n = 1$ gives the zero ring of Remark 4.4.

4. The *polynomial ring* $R[x]$ over a commutative ring R consists of the polynomials $a_0 + a_1x + \dots + a_dx^d$ with coefficients $a_i \in R$, added coefficient-wise and multiplied by expanding and collecting powers of x ; it is a commutative ring with the same 0 and 1 as R (the constant polynomials). The theory of polynomials over a field is developed in its own section later in the volume.

5. The *matrix ring* $M_n(R)$ of $n \times n$ matrices over a commutative ring R , $n \geq 1$, is a ring under matrix addition and multiplication. It is *not* commutative for $n \geq 2$ and $R \neq \{0\}$, even when R is commutative: in $M_2(\mathbb{Z})$,

$$\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad \text{but} \quad \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}.$$

It is the basic noncommutative example.

6. The *product ring* $R \times S$ of two rings carries the componentwise operations, with zero $(0, 0)$ and identity $(1, 1)$; likewise any finite product $R_1 \times \dots \times R_k$.

7. The zero ring $\{0\}$.

Theorem 4.6. *Let $n \geq 1$. The operations*

$$[a]_n + [b]_n := [a + b]_n, \quad [a]_n \cdot [b]_n := [ab]_n$$

are well defined and make $\mathbb{Z}/n\mathbb{Z}$ a commutative ring with additive identity $[0]_n$ and multiplicative identity $[1]_n$. It has exactly n elements.

Proof. The right-hand sides are defined in terms of chosen representatives a, b , so we must check that different choices give the same class. This is exactly the compatibility half of Proposition 2.21: if $a' \equiv a$ and $b' \equiv b \pmod{n}$, then $a' + b' \equiv a + b$ and $a'b' \equiv ab \pmod{n}$, so $[a' + b']_n = [a + b]_n$ and $[a'b']_n = [ab]_n$. The operations are therefore well defined on classes.

Every ring axiom is now inherited from the corresponding law of $\mathbb{Z}$ by passing to classes. For instance, distributivity:

$$\begin{aligned} [a]_n([b]_n + [c]_n) &= [a]_n[b + c]_n = [a(b + c)]_n = [ab + ac]_n \\ &= [ab]_n + [ac]_n = [a]_n[b]_n + [a]_n[c]_n, \end{aligned}$$

using only the definitions and the distributive law of the integers. Associativity and commutativity of both operations follow by the identical pattern. The class $[0]_n$ is an additive identity and $[1]_n$ a multiplicative one, since $[a + 0]_n = [a]_n$ and $[1 \cdot a]_n = [a]_n$; and $-[a]_n = [-a]_n$ because $[a]_n + [-a]_n = [a + (-a)]_n = [0]_n$.

Finally, by the division algorithm every integer is congruent modulo n to exactly one of $0, 1, \dots, n-1$ (the least nonnegative residues, Definition 2.22 and the discussion following it), so the classes $[0]_n, \dots, [n-1]_n$ are distinct and exhaust $\mathbb{Z}/n\mathbb{Z}$: the ring has exactly n elements. $\square$

4.2 Units, zero divisors, and integral domains

Units are the elements by which one can divide, and zero divisors are the obstruction to *cancellation*, the removal of a nonzero common factor from both sides of an equation. The integers admit cancellation, since a product of nonzero integers is nonzero, but only ± 1 divide 1.

Definition 4.7 (Unit). An element u of a ring R is a *unit* (or is *invertible*) if there exists $v \in R$ with $uv = vu = 1$. Such a v is unique: if $uv = vu = 1$ and $uw' = v'u = 1$, then $v = v \cdot 1 = v(uv') = (vu)v' = 1 \cdot v' = v'$. It is called the *inverse* of u and written u^{-1} . The set of units of R is denoted $R^\times$.

Proposition 4.8. For any ring R , the set $R^\times$ of units is a group under the multiplication of R , called the group of units (or multiplicative group) of R .

Proof. The identity 1 is a unit (it is its own inverse), so $1 \in R^\times$. For closure, let $u, w \in R^\times$; then

$$(uw)(w^{-1}u^{-1}) = u(ww^{-1})u^{-1} = u \cdot 1 \cdot u^{-1} = 1,$$

and symmetrically $(w^{-1}u^{-1})(uw) = 1$, so uw is a unit with $(uw)^{-1} = w^{-1}u^{-1}$. Multiplication in $R^\times$ is associative because it is associative in R . Finally each u^{-1} is itself a unit, its inverse being u ; so every element of $R^\times$ has an inverse in $R^\times$, and the group axioms are verified. $\square$

Example 4.9 (Units). 1. The units of the integers are $\mathbb{Z}^\times = \{1, -1\}$: if $uv = 1$ in $\mathbb{Z}$ then $|u| |v| = 1$ forces $|u| = 1$.

2. In $\mathbb{Q}$, $\mathbb{R}$, and $\mathbb{C}$ every nonzero element is a unit: $\mathbb{Q}^\times = \mathbb{Q} \setminus \{0\}$, $\mathbb{R}^\times = \mathbb{R} \setminus \{0\}$, $\mathbb{C}^\times = \mathbb{C} \setminus \{0\}$. Commutative rings with $1 \neq 0$ in which *every* nonzero element is a unit are precisely the fields of §4.4; this is their defining feature. Both qualifiers are needed: the zero ring of Remark 4.4 satisfies the unit condition vacuously, and noncommutative rings satisfying it—the *division rings*—exist but are not fields.

3. In $\mathbb{Z}/n\mathbb{Z}$, the class $\bar{a}$ is a unit if and only if $\gcd(a, n) = 1$, by Theorem 2.24; thus $(\mathbb{Z}/n\mathbb{Z})^\times$ consists of the residue classes coprime to the modulus, and its order is Euler's totient $\varphi(n)$ (Definition 2.26).

4. In the matrix ring, $M_n(R)^\times$ is the group of invertible $n \times n$ matrices. Over a field it is the *general linear group* GL_n .

Definition 4.10 (Zero divisor). A nonzero element a of a ring R is a *left zero divisor* if $ab = 0$ for some nonzero $b \in R$, and a *right zero divisor* if $ba = 0$ for some nonzero b . In a commutative ring the two notions coincide and we say *zero divisor*. A nonzero element that is neither is called *regular* (or *cancellable*).

The term “cancellable” is justified by the following equivalence.

Proposition 4.11. A nonzero element a of a ring R is not a left zero divisor if and only if it is left-cancellable: $ab = ac$ implies $b = c$ for all $b, c \in R$.

Proof. Suppose a is not a left zero divisor and $ab = ac$. Then $a(b - c) = ab - ac = 0$, using Proposition 4.3(2) to expand $a(b + (-c))$; since a is nonzero and not a left zero divisor, the factor $b - c$ must be 0, so $b = c$. Conversely, suppose a is a left zero divisor, say $ab = 0$ with $b \neq 0$. Then $ab = 0 = a \cdot 0$ while $b \neq 0$: cancellation fails. $\square$

Remark 4.12. A unit is never a zero divisor: if $u \in R^\times$ and $ub = 0$, then $b = 1 \cdot b = (u^{-1}u)b = u^{-1}(ub) = u^{-1} \cdot 0 = 0$. The converse fails in general—in $\mathbb{Z}$ every nonzero element is a non-zero-divisor, yet only ± 1 are units—but every nonzero non-zero-divisor is a unit in a finite commutative ring, by the injectivity-implies-surjectivity argument that proves Theorem 4.29 below.

Example 4.13 (Zero divisors). 1. In $\mathbb{Z}/6\mathbb{Z}$, $\bar{2} \cdot \bar{3} = \bar{6} = \bar{0}$, so $\bar{2}$ and $\bar{3}$ are zero divisors. In general a nonzero class $\bar{a} \in \mathbb{Z}/n\mathbb{Z}$ is a zero divisor precisely when $d := \gcd(a, n) > 1$: in that case $a \cdot (n/d) = (a/d) \cdot n \equiv 0 \pmod{n}$ while $n/d \neq 0$ (as $0 < n/d < n$); and when $d = 1$ the class is a unit (Example 4.9(3)), hence no zero divisor by Remark 4.12. Thus in $\mathbb{Z}/n\mathbb{Z}$ every nonzero element is either a unit or a zero divisor, a dichotomy revisited structurally by Theorem 4.29.

2. In a product ring $R \times S$ with both factors nonzero, $(1, 0)(0, 1) = (0, 0)$: product rings always have zero divisors.
3. In $M_2(\mathbb{R})$, the diagonal matrices $\text{diag}(1, 0)$ and $\text{diag}(0, 1)$ multiply to the zero matrix; more generally every nonzero *singular* matrix—the standard name for a square matrix that is not invertible—is a zero divisor there.

Definition 4.14 (Integral domain). An *integral domain* (or simply *domain*) is a commutative ring R with $1 \neq 0$ and no zero divisors: $ab = 0$ implies $a = 0$ or $b = 0$. Equivalently, by Proposition 4.11, a domain is a nonzero commutative ring in which every nonzero element is cancellable.

Proposition 4.15. Let $n \geq 2$. The ring $\mathbb{Z}/n\mathbb{Z}$ has no zero divisors if and only if n is prime; that is, $\mathbb{Z}/n\mathbb{Z}$ is an integral domain if and only if n is prime.

Proof. If $n = ab$ is composite with $1 < a, b < n$, then $[a]_n[b]_n = [n]_n = [0]_n$ with both factors nonzero (their representatives lie strictly between 0 and n), so $\mathbb{Z}/n\mathbb{Z}$ has zero divisors. Conversely let $n = p$ be prime and suppose $[a]_p[b]_p = [0]_p$, that is, $p \mid ab$. Euclid's lemma for primes (Lemma 2.17) gives $p \mid a$ or $p \mid b$, that is, $[a]_p = [0]_p$ or $[b]_p = [0]_p$. Since $p \geq 2$ we also have $[1]_p \neq [0]_p$, so $\mathbb{Z}/p\mathbb{Z}$ is an integral domain. $\square$

Example 4.16 (Integral domains). The integers $\mathbb{Z}$ form an integral domain, the example from which the name derives. Every field is a domain (Proposition 4.28 below). The ring $\mathbb{Z}/n\mathbb{Z}$ is a domain exactly when n is prime (Proposition 4.15; the composite case is witnessed concretely by Example 4.13(1)). If R is a domain, so is the polynomial ring $R[x]$: let f and g be nonzero of degrees m and n —the largest indices carrying nonzero coefficients f_m and g_n , the *leading coefficients*. In fg the coefficient of x^{m+n} is $\sum_{i+j=m+n} f_i g_j$; every term with $i > m$ or $j > n$ has a vanishing factor, and $i \leq m, j \leq n, i + j = m + n$ force $i = m, j = n$, so the coefficient equals $f_m g_n$, nonzero because R is a domain. Hence $fg \neq 0$. Non-domains include $\mathbb{Z}/6\mathbb{Z}$, product rings with two nonzero factors, and the matrix rings $M_n(R)$ for $n \geq 2$ (Example 4.13).

4.3 Ideals and quotient rings

The *ideals* of a ring are the subsets by which one can form a quotient preserving *both* operations, just as any subgroup of an abelian group yields a quotient group (Proposition 3.35). They are needed here to construct $\mathbb{Z}/n\mathbb{Z}$ as a quotient and to indicate the origin of the finite fields $\mathbb{F}_q$; the general theory belongs to commutative algebra and is omitted.

Definition 4.17 (Ideal). Let R be a commutative ring. An *ideal* of R is a subset $I \subseteq R$ such that

1. I is a subgroup of the additive group $(R, +)$: $0 \in I$, and $a - b \in I$ whenever $a, b \in I$; and
2. I *absorbs* multiplication by arbitrary ring elements: $ra \in I$ whenever $r \in R$ and $a \in I$.

In noncommutative rings one distinguishes left, right, and two-sided ideals according to the side on which absorption holds; commutative rings suffice for this volume, and there the three notions coincide.

No ideal other than R contains a unit, so none is a subring: if an ideal I contains a unit u , then $1 = u^{-1}u \in I$ by absorption, so $r = r \cdot 1 \in I$ for every $r \in R$ and $I = R$. Consequently a field has no ideals other than the two trivial ones, and hence no quotients other than a copy of itself and the zero ring.

Example 4.18 (Ideals). 1. In any commutative ring R , the subsets $\{0\}$ and R are ideals—the *trivial* and *improper* ideals respectively.

2. For $a \in R$, the *principal ideal generated by a* is

$$(a) := aR = \{ar : r \in R\},$$

the set of multiples of a ; absorption holds because $r'(ar) = a(r'r)$. In $\mathbb{Z}$, the principal ideal $(n) = n\mathbb{Z}$ is the set of multiples of n . Moreover *every* ideal of $\mathbb{Z}$ is principal: an ideal

$I \neq \{0\}$ contains a nonzero element and hence (closing under negation) a positive one, so it contains a least positive element n by well-ordering; for any $a \in I$ the division algorithm gives $a = qn + r$ with $0 \leq r < n$, whence $r = a - qn \in I$ by absorption and subgroup closure, forcing $r = 0$ by minimality; so $I = n\mathbb{Z}$. This sharpens the observation of Remark 2.12, where ideals of $\mathbb{Z}$ first appeared as the sets of Bézout combinations.

3. For $a_1, \dots, a_k \in R$, the *ideal generated by* $a_1, \dots, a_k$ is $(a_1, \dots, a_k) = \{r_1 a_1 + \dots + r_k a_k : r_i \in R\}$, the smallest ideal containing them all.

An integral domain in which every ideal is principal, as in (2), is called a *principal ideal domain* (PID).

Definition 4.19 (Quotient ring operations). Let I be an ideal of a commutative ring R . Since I is a subgroup of the abelian group $(R, +)$, the additive cosets $a + I$ form the quotient group R/I under coset addition $(a + I) + (b + I) = (a + b) + I$ (Proposition 3.35). Define a multiplication of cosets by

$$(a + I)(b + I) := ab + I.$$

Proposition 4.20. *With the operations of Definition 4.19, the set R/I is a commutative ring—the quotient ring of R by I —with zero $0 + I = I$ and identity $1 + I$. In particular the coset multiplication is well defined.*

Proof. Well-definedness is where absorption is needed. Let $a' = a + s$ and $b' = b + t$ be other representatives of the same cosets, with $s, t \in I$. Then

$$a'b' = (a + s)(b + t) = ab + at + sb + st,$$

and each of at, sb, st lies in I by the absorption property; hence $a'b' - ab \in I$ and $a'b' + I = ab + I$. The product coset is therefore independent of the representatives chosen.

The ring axioms now pass to cosets exactly as in the proof of Theorem 4.6. For instance, distributivity reads

$$\begin{aligned} (a + I)((b + I) + (c + I)) &= (a + I)((b + c) + I) = a(b + c) + I \\ &= (ab + ac) + I = (a + I)(b + I) + (a + I)(c + I), \end{aligned}$$

and associativity and commutativity of both operations follow by the same pattern from the corresponding laws in R . The coset $I = 0 + I$ is the additive identity, $1 + I$ the multiplicative identity, and $-(a + I) = (-a) + I$. $\square$

Ring homomorphisms, the maps preserving ring structure, make precise the sense in which the quotient construction generalises the passage from $\mathbb{Z}$ to $\mathbb{Z}/n\mathbb{Z}$.

Definition 4.21 (Ring homomorphism). A *ring homomorphism* between rings R and S is a map $\phi: R \rightarrow S$ with

$$\phi(a + b) = \phi(a) + \phi(b), \quad \phi(ab) = \phi(a)\phi(b), \quad \phi(1) = 1$$

for all $a, b \in R$. Its *kernel* is $\ker \phi = \{a \in R : \phi(a) = 0\}$.

The kernel of a ring homomorphism out of a commutative ring is always an ideal: it is a subgroup of $(R, +)$ because ϕ is in particular a homomorphism of additive groups, and it absorbs because $a \in \ker \phi$ gives $\phi(ra) = \phi(r)\phi(a) = \phi(r) \cdot 0 = 0$.

Example 4.22. Take $R = \mathbb{Z}$ and $I = n\mathbb{Z}$. The cosets $a + n\mathbb{Z}$ are literally the residue classes $[a]_n$ of Definition 2.22, and the coset operations of Definition 4.19 are exactly residue arithmetic: the quotient construction recovers the ring $\mathbb{Z}/n\mathbb{Z}$ of Example 4.5(3), so modular arithmetic is an instance of it. The same construction with a different ring produces the general finite fields: quotients $\mathbb{F}_p[x]/(f)$ of a polynomial ring by the principal ideal of an *irreducible* polynomial f of degree k —irreducibility, the polynomial analogue of primality, is defined in §5—furnish the finite fields $\mathbb{F}_q$ with $q = p^k$ elements. The construction is recorded here and developed in §6; the curves of Halo 2 and Orchard are defined over *prime* fields, so the prime case $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$ is the main case in what follows.

Remark 4.23. The quotient map

$$\pi: \mathbb{Z} \rightarrow \mathbb{Z}/n\mathbb{Z}, \quad \pi(a) = [a]_n,$$

is a surjective ring homomorphism with kernel $n\mathbb{Z}$. Every quotient construction in the volume’s algebra follows this pattern: one chooses a ring and an ideal of elements to be “declared zero”, and computes with cosets. On notation: we abbreviate $[a]_n$ to $\bar{a}$ (as in the examples above) or drop the decoration altogether, writing plain a , whenever the modulus is clear from context; $\mathbb{Z}/n\mathbb{Z}$ and $\mathbb{Z}_n$ are used interchangeably; and $\mathbb{Z}_n$ always means this ring, never the n -adic integers (which do not appear in this series).

4.4 Fields

Definition 4.24 (Field). A *field* is a commutative ring F with $1 \neq 0$ in which every nonzero element is a unit. Equivalently, a field is a set F with two operations $+$ and $\cdot$ such that $(F, +)$ is an abelian group with identity 0, the nonzero elements $(F \setminus \{0\}, \cdot)$ form an abelian group with identity 1, and multiplication distributes over addition. Concretely, in a field one may add, subtract, multiply, and divide by any nonzero element, with $a/b := ab^{-1}$. The group $F^\times = (F \setminus \{0\}, \cdot)$ is the *multiplicative group* of the field. (Two steps of the equivalence require an argument. Passing from the first formulation to the second, one must show $F \setminus \{0\}$ is closed under multiplication; that is Proposition 4.28 below. Passing back, the ring axioms must be checked for products involving 0: distributivity forces $0 \cdot a = a \cdot 0 = 0$ by the computation of Proposition 4.3(1), whereupon every product with a zero factor vanishes and the associativity, commutativity, and identity laws extend from $F \setminus \{0\}$ to all of F ; and $1 \neq 0$ holds because the group $F \setminus \{0\}$ contains its identity 1.)

Definition 4.25 (Subfield and extension field). A subset $F \subseteq K$ of a field K is a *subfield* when it contains 0 and 1 and is itself a field under the operations of K restricted to F . When F is a subfield of K , the larger field K is called an *extension field* (or simply an *extension*) of F ; the two phrases name the same relation. More generally, an injective ring homomorphism $F \hookrightarrow$

K between fields exhibits K as an extension of the copy of F inside it; by the usual abuse of language we identify F with that copy.

The unit criterion of §2 identifies the residue rings that are fields.

Corollary 4.26. *If p is prime, then every nonzero class in $\mathbb{Z}/p\mathbb{Z}$ is a unit, so $\mathbb{Z}/p\mathbb{Z}$ is a field; it is denoted $\mathbb{F}_p$. More generally, $\mathbb{Z}/n\mathbb{Z}$ is a field if and only if n is prime.*

Proof. Let p be prime and $[a]_p \neq [0]_p$, so that $p \nmid a$. The only positive divisors of p are 1 and p , so $\gcd(a, p) = 1$, and Theorem 2.24 makes $[a]_p$ a unit. Since $[1]_p \neq [0]_p$ for $p \geq 2$, the ring $\mathbb{Z}/p\mathbb{Z}$ is a field. Conversely, if $n = ab$ is composite with $1 < a, b < n$, then $[a]_n \neq [0]_n$ but $\gcd(a, n) = a > 1$, so $[a]_n$ is not a unit (Theorem 2.24 again) and the ring is not a field; and $n = 1$ gives the zero ring, which is not a field because a field has $1 \neq 0$ by definition. $\square$

Example 4.27 (Fields). 1. The rationals $\mathbb{Q}$, the reals $\mathbb{R}$, and the complexes $\mathbb{C}$ are fields. The integers are not: 2 has no inverse in $\mathbb{Z}$.

2. For p prime, $\mathbb{F}_p := \mathbb{Z}/p\mathbb{Z}$ is a field (Corollary 4.26; Theorem 4.29 below recovers the same fact abstractly). It is called the *prime field* of characteristic p —Definition 4.30 names the invariant—and underlies every later cryptographic construction in the series. Its finite extensions $\mathbb{F}_q$ with $q = p^k$ elements exist for every prime power, arising from the quotient construction of Example 4.22, and are developed in §6.

3. The subset $\mathbb{Q}(i) = \{a + bi : a, b \in \mathbb{Q}\} \subseteq \mathbb{C}$ is a field: the inverse of a nonzero $a + bi$ is $(a - bi)/(a^2 + b^2)$, which again has rational coordinates. More generally the *number fields*, obtained by adjoining to $\mathbb{Q}$ roots of polynomial equations with rational coefficients, are fields.

4. For any field F , the *rational functions* $F(x) = \{f/g : f, g \in F[x], g \neq 0\}$ form a field under the usual arithmetic of fractions—the field of fractions of the polynomial ring $F[x]$.

Proposition 4.28. *Every field is an integral domain.*

Proof. Let F be a field. Commutativity and $1 \neq 0$ hold by definition. Suppose $ab = 0$ with $a \neq 0$. Then a is a unit, and multiplying by its inverse,

$$b = 1 \cdot b = (a^{-1}a)b = a^{-1}(ab) = a^{-1} \cdot 0 = 0.$$

Hence F has no zero divisors. $\square$

The converse fails: $\mathbb{Z}$ is a domain but not a field (Example 4.16). For finite domains the converse holds, by a counting argument that recurs, the *injectivity-implies-surjectivity argument*: to invert an element a of a finite structure, one shows that multiplication by a is injective, hence onto by finiteness, and reads off a preimage of 1. The same argument proves the claim of Remark 4.12.

| **Theorem 4.29.** *Every finite integral domain is a field.*

Proof. Let R be a finite integral domain and $a \in R$ nonzero. Consider the multiplication map

$$\mu_a: R \rightarrow R, \quad \mu_a(x) = ax.$$

It is injective: if $ax = ay$, then $a(x - y) = 0$, and since R is a domain and $a \neq 0$ this forces $x - y = 0$, that is, $x = y$ (equivalently, a is cancellable by Proposition 4.11). An injective map from a finite set to itself is surjective: the image $\mu_a(R)$ has $|R|$ distinct elements and sits inside R , so it is all of R . In particular 1 lies in the image, so $ab = 1$ for some $b \in R$; commutativity gives $ba = 1$ as well, so $b = a^{-1}$ and a is a unit. Since R is a domain, $1 \neq 0$, and every nonzero element has been shown invertible: R is a field. $\square$

With Proposition 4.28, the theorem shows that a nonzero finite commutative ring is a field exactly when it has no zero divisors. Applied to $\mathbb{Z}/p\mathbb{Z}$, a finite domain by Proposition 4.15, it recovers Corollary 4.26, but establishes only that each inverse *exists*; the extended Euclidean algorithm of §2 *computes* it, in $O(\log p)$ division steps. Cryptography uses the existence statement in proofs and the algorithm in computation.

4.5 The characteristic

Every ring receives a canonical map from the integers; the characteristic, an invariant of the ring, describes that map and determines the image of $\mathbb{Z}$ inside the ring.

Definition 4.30 (Characteristic). Let R be a ring, and for $n \in \mathbb{Z}$ let $n \cdot 1$ denote the n -fold additive multiple of 1 (so $n \cdot 1 = 1 + \dots + 1$ with n summands for $n > 0$, $0 \cdot 1 = 0$, and $(-n) \cdot 1 = -(n \cdot 1)$). The *characteristic* $\text{char}(R)$ is the least positive integer n with $n \cdot 1 = 0$, if such an n exists, and 0 otherwise.

The map

$$\iota: \mathbb{Z} \rightarrow R, \quad \iota(n) = n \cdot 1,$$

is a ring homomorphism: additivity is the bookkeeping of iterated multiples in the abelian group $(R, +)$, multiplicativity is Proposition 4.3(4) with $a = b = 1$ —namely $\iota(m)\iota(n) = (m \cdot 1)(n \cdot 1) = (mn)(1 \cdot 1) = \iota(mn)$ —and $\iota(1) = 1$. Its kernel is an ideal of $\mathbb{Z}$, hence of the form $n\mathbb{Z}$ for a unique $n \geq 0$ by Example 4.18(2), and $\text{char}(R)$ is exactly this nonnegative generator: $\text{char}(R) = 0$ precisely when ι is injective, so that a faithful copy of $\mathbb{Z}$ sits inside R , and $\text{char}(R) = n > 0$ when the kernel is $n\mathbb{Z}$.

Example 4.31 (Characteristics). The rings $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$, and $\mathbb{C}$ all have characteristic 0: no positive multiple of 1 vanishes. For the residue rings, $\text{char}(\mathbb{Z}/n\mathbb{Z}) = n$: the multiple $n \cdot \bar{1} = \bar{n} = \bar{0}$ vanishes, and no smaller positive multiple does, since $k \cdot \bar{1} = \bar{k} \neq \bar{0}$ for $0 < k < n$. In particular $\text{char}(\mathbb{F}_p) = p$.

| **Theorem 4.32.** *The characteristic of an integral domain—in particular, of a field—is either 0 or a prime number.*

Proof. Let R be an integral domain with $\text{char}(R) = n > 0$; we show n is prime. First $n \neq 1$: otherwise $1 \cdot 1 = 1 = 0$, making R the zero ring (Remark 4.4) and contradicting $1 \neq 0$ in a domain. Suppose $n = ab$ with $1 < a, b < n$. By Proposition 4.3(4),

$$0 = n \cdot 1 = (ab)(1 \cdot 1) = (a \cdot 1)(b \cdot 1),$$

so $a \cdot 1 = 0$ or $b \cdot 1 = 0$ since R is a domain. Either case exhibits a positive integer smaller than n whose multiple of 1 vanishes, contradicting the minimality of $n = \text{char}(R)$. Hence n admits no such factorisation, and being greater than 1, it is prime. $\square$

Remark 4.33 (The prime subfield). Let F be a field. If $\text{char}(F) = 0$, the homomorphism ι embeds a copy of $\mathbb{Z}$ in F , and dividing—possible in a field—a copy of $\mathbb{Q}$ as well. If $\text{char}(F) = p$, the image of ι is a copy of $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$. The smallest subfield so obtained is called the *prime subfield* of F ; every field is thus an extension of $\mathbb{Q}$ or of some $\mathbb{F}_p$. The fields $\mathbb{F}_q$ underlying the cryptography treated in this series all have prime characteristic p . In characteristic p the “freshman’s dream” identity $(a + b)^p = a^p + b^p$ holds and gives rise to the Frobenius endomorphism; both are developed in the finite-fields section (Lemma 6.13).

5 Polynomials over a field

Throughout this section F denotes a field. The section constructs the ring $F[X]$ with its division, gcd and factorisation theory, and proves that a nonzero polynomial of degree d over a field has at most d roots. From this bound it derives Lagrange interpolation (§5.7) and the Schwartz–Zippel lemma (§5.10), which justifies the polynomial identity testing of the later volumes: two polynomials encoding a computation are compared by evaluating both at one random α , not coefficient by coefficient at the cost of reading the computation. Halo 2 and Zcash Orchard use these results in their polynomial commitment schemes and their *arithmetisation*, the encoding of a computation as identities between polynomials; polynomials also underlie finite fields and error-correcting codes.

5.1 The polynomial ring

A polynomial is a formal expression, a finite list of coefficients, and *not a function*: evaluating it yields a function, which over a finite field carries less information than the polynomial (§5.9).

Definition 5.1 (Polynomial). A *polynomial* over F in the *indeterminate* X is a formal expression

$$f = a_0 + a_1X + a_2X^2 + \dots + a_nX^n = \sum_{i=0}^n a_iX^i,$$

where $n \in \mathbb{N} = \{0, 1, 2, \dots\}$ and the *coefficients* $a_0, \dots, a_n$ lie in F . Formally, f is identified with its coefficient sequence $(a_0, a_1, a_2, \dots)$, in which only finitely many entries are nonzero. Two polynomials are equal precisely when their coefficient sequences agree term by term:

$$\sum_i a_iX^i = \sum_i b_iX^i \iff a_i = b_i \text{ for every } i \in \mathbb{N}.$$

The symbol X is an *indeterminate* (a formal variable): it is not an element of F and denotes no particular value; it is a placeholder organising the coefficients. The set of all polynomials over F in X is denoted $F[X]$.

Remark 5.2 (The sequence model). In the rigorous form of Definition 5.1, $F[X]$ is the set of all functions $a: \mathbb{N} \rightarrow F$ that are *eventually zero*: there is some N with $a(i) = 0$ for all $i > N$. The expression $\sum_i a_i X^i$ is notation for the sequence $(a_0, a_1, \dots)$, and the two viewpoints are used interchangeably: the arithmetic of sequences defined next is the arithmetic the notation suggests.

Definition 5.3 (Ring operations on $F[X]$). Let $f = \sum_i a_i X^i$ and $g = \sum_i b_i X^i$ lie in $F[X]$, with the shorter coefficient sequence zero-padded so that both run over the same index set. Their *sum* and *product* are

$$f + g = \sum_{i \geq 0} (a_i + b_i) X^i, \quad fg = \sum_{k \geq 0} c_k X^k \quad \text{where} \quad c_k = \sum_{i+j=k} a_i b_j = \sum_{i=0}^k a_i b_{k-i}.$$

The coefficient c_k is the *convolution* of the two coefficient sequences. Both defining sums are finite—only finitely many a_i and b_j are nonzero, hence only finitely many c_k are nonzero—so the sum and the product are again polynomials.

The convolution formula results from multiplying out the product $(\sum_i a_i X^i)(\sum_j b_j X^j)$ and collecting like powers of X using $X^i \cdot X^j = X^{i+j}$; as a convolution it involves no substitution of a value for X . The proof of the ring axioms (Definition 4.1) likewise involves no evaluation: it writes the coefficient of X^m on each side of an identity as a finite sum over index decompositions, then reindexes or regroups that sum, as in the associativity step below.

Theorem 5.4 ($F[X]$ is a commutative ring). *With the operations of Definition 5.3, the set $F[X]$ is a commutative ring whose additive identity is the zero polynomial $0 = (0, 0, 0, \dots)$ and whose multiplicative identity is the constant polynomial $1 = (1, 0, 0, \dots)$. Moreover the map $c \mapsto (c, 0, 0, \dots)$ embeds F into $F[X]$ as a subring, the constant polynomials; in this sense $F \subseteq F[X]$.*

Proof. Additive structure. Addition is coordinatewise, so its commutativity and associativity are inherited directly from the corresponding laws of the additive group of F , the sequence $(0, 0, \dots)$ is an additive identity, and $(-a_0, -a_1, \dots)$ is an additive inverse of $(a_0, a_1, \dots)$. Hence $(F[X], +)$ is an abelian group.

Commutativity of multiplication. The coefficient of X^k in fg is $\sum_{i+j=k} a_i b_j$; reindexing the sum by $(i, j) \mapsto (j, i)$ and using commutativity of multiplication in F ,

$$\sum_{i+j=k} a_i b_j = \sum_{i+j=k} b_i a_j,$$

which is the coefficient of X^k in gf .

Associativity of multiplication. Write $h = \sum_l c_l X^l$. The coefficient of X^m in $(fg)h$ is

$$\sum_{k+l=m} \left(\sum_{i+j=k} a_i b_j \right) c_l = \sum_{i+j+l=m} a_i b_j c_l,$$

and expanding $f(gh)$ instead produces $\sum_{i+j+l=m} a_i(b_j c_l)$, the same symmetric triple sum by associativity in F . Hence $(fg)b = f(gh)$ coefficient by coefficient.

Distributivity. The coefficient of X^k in $f(g + h)$ is $\sum_{i+j=k} a_i(b_j + c_j)$, which distributivity in F splits as $\sum_{i+j=k} a_i b_j + \sum_{i+j=k} a_i c_j$: the coefficient of X^k in $fg + fh$. The other distributive law follows from this one and commutativity.

Identity. With $1 = (1, 0, 0, \dots)$, the k -th coefficient of $1 \cdot f$ is $\sum_{i+j=k} \mathbf{1}[i=0] a_j = a_k$, where $\mathbf{1}[i=0]$ denotes the coefficient sequence of 1 (equal to 1 at index 0 and 0 elsewhere); so $1 \cdot f = f$.

Embedding. Let $\iota(c) = (c, 0, 0, \dots)$. The map ι preserves sums coordinatewise, and $\iota(c)\iota(d)$ has k -th coefficient cd for $k = 0$ and 0 otherwise, so $\iota(c)\iota(d) = \iota(cd)$; also $\iota(1) = 1$. Distinct constants give distinct sequences, so ι is an injective ring homomorphism (Definition 4.21) and its image is a subring of $F[X]$ isomorphic to F . $\square$

Remark 5.5 (Scalar multiplication). Multiplying $f = \sum_i a_i X^i$ by a constant $c \in F$, viewed as the constant polynomial $\iota(c)$, gives the *scalar multiple* $cf = \sum_i (ca_i) X^i$. With addition, this makes $F[X]$ a *vector space* over F : for the present purpose, a set closed under addition and under scaling by field elements, treated in the linear-algebra section later in the volume. The monomials $1, X, X^2, X^3, \dots$ form an infinite coordinate system (a *basis*). Later arguments use this linear structure, for instance to count polynomials of bounded degree.

5.2 Degree

Definition 5.6 (Degree, leading coefficient, monic). Let $f = \sum_i a_i X^i \in F[X]$ be nonzero. The *degree* $\deg f$ is the largest index n with $a_n \neq 0$; the coefficient a_n is the *leading coefficient* $\text{lc}(f)$, the term $a_n X^n$ is the *leading term*, and a_0 is the *constant term*. A nonzero polynomial is *monic* if its leading coefficient is 1. The constant polynomials are the elements of F embedded by Theorem 5.4; the nonzero ones have degree 0. The zero polynomial is assigned $\deg 0 = -\infty$, with the conventions $-\infty < n$, $-\infty + n = -\infty$, and $-\infty + (-\infty) = -\infty$ for every $n \in \mathbb{Z}$. Polynomials of degree 1, 2, and 3 are called *linear*, *quadratic*, and *cubic* respectively.

The convention $\deg 0 = -\infty$ makes the degree formulas below hold without exception, so later inductions on degree need no special case for the zero polynomial.

Proposition 5.7 (Degree of sums and products). For all $f, g \in F[X]$,

$$\deg(f + g) \leq \max\{\deg f, \deg g\}, \quad (1)$$

with equality whenever $\deg f \neq \deg g$, and

$$\deg(fg) = \deg f + \deg g. \quad (2)$$

Proof. If either polynomial is zero, both statements reduce to the $-\infty$ conventions of Definition 5.6. So let f and g be nonzero, of degrees m and n with leading coefficients a_m and b_n .

Sum. The coefficient of X^k in $f + g$ is $a_k + b_k$, which vanishes for $k > \max\{m, n\}$; this is (1). If $m > n$, the coefficient of X^m is $a_m + 0 = a_m \neq 0$, giving equality (symmetrically for $n > m$). When

$m = n$, cancellation $a_m + b_m = 0$ can lower the degree, which is why the sum bound is only an inequality.

Product. The coefficient $c_{m+n} = \sum_{i+j=m+n} a_i b_j$ has only the surviving term $i = m, j = n$: terms with $i > m$ have $a_i = 0$, and terms with $i < m$ force $j > n$, so $b_j = 0$. Hence $c_{m+n} = a_m b_n$, and the same argument shows $c_k = 0$ for $k > m + n$. Since F is a field it has no zero divisors (Proposition 4.28), so $a_m b_n \neq 0$; therefore $\deg(fg) = m + n$ and $\text{lc}(fg) = \text{lc}(f)\text{lc}(g)$. $\square$

The product formula (2) is the first result to use the field hypothesis, through the absence of zero divisors in F ; the next three results and the rest of the section depend on it.

Corollary 5.8 ($F[X]$ is an integral domain). *The ring $F[X]$ has no zero divisors: if $fg = 0$, then $f = 0$ or $g = 0$.*

Proof. If f and g are both nonzero, then $\deg(fg) = \deg f + \deg g \geq 0$ by (2), so $fg \neq 0$. Contrapositively, $fg = 0$ forces $f = 0$ or $g = 0$. $\square$

Corollary 5.9 (Cancellation). *Let $f, g, h \in F[X]$ with $h \neq 0$. If $fh = gh$, then $f = g$.*

Proof. From $fh = gh$ we get $(f - g)h = 0$; since $h \neq 0$, Corollary 5.8 forces $f - g = 0$. $\square$

Proposition 5.10 (Units of $F[X]$). *A polynomial $f \in F[X]$ is a unit—has a multiplicative inverse in $F[X]$ —if and only if it is a nonzero constant. Thus $F[X]^\times = F^\times$.*

Proof. A nonzero constant c has the inverse $c^{-1} \in F \subseteq F[X]$. Conversely, $fg = 1$ gives $\deg f + \deg g = \deg 1 = 0$ by (2); since degrees of nonzero polynomials are nonnegative, $\deg f = \deg g = 0$, and f is a nonzero constant. $\square$

Definition 5.11 (Associates). Two polynomials $f, g \in F[X]$ are *associates* if $f = ug$ for some unit $u \in F^\times$, that is, if they differ only by a nonzero scalar factor.

Every nonzero polynomial f has exactly one monic associate, $\text{lc}(f)^{-1}f$. Choosing the monic representative, as below, makes *the* greatest common divisor and *the* irreducible factor unique.

5.3 Division with remainder

Division with remainder, with *degree* in the role of absolute value on $\mathbb{Z}$, carries the arithmetic of $\mathbb{Z}$ (§2) over to $F[X]$: greatest common divisors, the Euclidean algorithm, Bézout's identity, irreducibles in place of primes, and unique factorisation.

Theorem 5.12 (Division algorithm). *Let $f, g \in F[X]$ with $g \neq 0$. There exist unique polynomials $q, r \in F[X]$ such that*

$$f = qg + r \quad \text{and} \quad \deg r < \deg g.$$

We call q the quotient and r the remainder, and write $q = f \text{ div } g$ and $r = f \bmod g$.

Proof. Existence. Fix g , of degree n with leading coefficient b_n , and induct on $\deg f$. If $\deg f < n$ —including $f = 0$ with $\deg f = -\infty$ —take $q = 0$ and $r = f$. Otherwise let $m = \deg f \geq n$, with leading coefficient a_m , and form

$$f_1 = f - \frac{a_m}{b_n} X^{m-n} g.$$

The subtracted term has degree m and leading coefficient $(a_m/b_n) b_n = a_m$, so it cancels the leading term of f : either $f_1 = 0$ or $\deg f_1 < m$. By the induction hypothesis $f_1 = q_1 g + r$ with $\deg r < n$, and then $f = (q_1 + (a_m/b_n) X^{m-n})g + r$. The field hypothesis enters at this step: division by the leading coefficient b_n requires b_n^{-1} to exist.

Uniqueness. Suppose $f = qg + r = q'g + r'$ with both remainders of degree $< n$. Then $(q - q')g = r' - r$. If $q \neq q'$, then $\deg((q - q')g) = \deg(q - q') + n \geq n$ by (2), whereas $\deg(r' - r) \leq \max\{\deg r', \deg r\} < n$ by (1)—a contradiction. Hence $q = q'$, and then $r = f - qg = r'$. $\square$

Remark 5.13 (Polynomial long division). The existence proof is constructive: it is *polynomial long division*, which repeatedly eliminates the current leading term by subtracting a monomial multiple of g . Each step strictly lowers the degree of the running remainder, so the process terminates after at most $\deg f - \deg g + 1$ steps when $\deg f \geq \deg g$, and takes no steps when $f = 0$ or $\deg f < \deg g$. Later inductions on degree use the same leading-term elimination.

For example, let $f = X^3 + 2X + 1$ and $g = X^2 + 1$ over $\mathbb{Q}$. Subtracting $X \cdot g = X^3 + X$ leaves $X + 1$, whose degree 1 is below $\deg g = 2$; hence $q = X$ and $r = X + 1$:

$$X^3 + 2X + 1 = X(X^2 + 1) + (X + 1).$$

Definition 5.14 (Divisibility). For $f, g \in F[X]$, we say g *divides* f , written $g \mid f$, if $f = qg$ for some $q \in F[X]$; equivalently, for $g \neq 0$, if the remainder $f \bmod g$ is 0. In that case g is a *divisor* (or *factor*) of f .

5.4 Roots and the factor theorem

This subsection defines evaluation, where a polynomial first gives rise to a function, and proves the root bound (Theorem 5.18). Every later construction of the volume on polynomials rests on that bound, including the cyclicity of the multiplicative group of a finite field.

Definition 5.15 (Evaluation and roots). Let $f = \sum_{i=0}^n a_i X^i \in F[X]$ and $\alpha \in F$. The *evaluation* of f at α is the field element

$$f(\alpha) = \sum_{i=0}^n a_i \alpha^i \in F,$$

obtained by substituting α for the indeterminate X . An element $\alpha \in F$ is a *root* (or *zero*) of f if $f(\alpha) = 0$.

Proposition 5.16 (Evaluation is a ring homomorphism). Fix $\alpha \in F$. The evaluation map $\text{ev}_\alpha: F[X] \rightarrow F, f \mapsto f(\alpha)$, satisfies

$$(f + g)(\alpha) = f(\alpha) + g(\alpha), \quad (fg)(\alpha) = f(\alpha)g(\alpha), \quad 1(\alpha) = 1$$

for all $f, g \in F[X]$; that is, ev_α is a ring homomorphism.

Proof. Additivity is immediate from the coordinatewise definition of addition. For multiplicativity, with $c_k = \sum_{i+j=k} a_i b_j$ as in Definition 5.3,

$$(fg)(\alpha) = \sum_k c_k \alpha^k = \sum_k \sum_{i+j=k} a_i b_j \alpha^{i+j} = \left(\sum_i a_i \alpha^i \right) \left(\sum_j b_j \alpha^j \right) = f(\alpha) g(\alpha),$$

the middle equality because expanding the product of the two finite sums and grouping terms by $k = i + j$ produces exactly the double sum. The formal reason evaluation respects the algebra is that the convolution defining polynomial multiplication collapses to ordinary multiplication in F via $\alpha^i \alpha^j = \alpha^{i+j}$. Finally $1(\alpha) = 1$ by direct substitution. $\square$

Theorem 5.17 (Remainder theorem; factor theorem). *Let $f \in F[X]$ and $\alpha \in F$.*

1. (Remainder theorem.) *The remainder of f upon division by the linear polynomial $X - \alpha$ is the constant $f(\alpha)$: there is $q \in F[X]$ with $f = q(X - \alpha) + f(\alpha)$.*
2. (Factor theorem.) *The element α is a root of f if and only if $(X - \alpha) \mid f$.*

Proof. (1) Divide f by $X - \alpha$ (Theorem 5.12): since $\deg(X - \alpha) = 1$, the remainder has degree < 1 , hence is a constant $r \in F$. Evaluating $f = q(X - \alpha) + r$ at α via Proposition 5.16 gives $f(\alpha) = q(\alpha)(\alpha - \alpha) + r = r$.

(2) By (1) and the uniqueness of the remainder, $(X - \alpha) \mid f$ holds exactly when the remainder $f(\alpha)$ is 0, that is, when α is a root of f . $\square$

Theorem 5.18 (At most d roots). *A nonzero polynomial $f \in F[X]$ of degree d has at most d distinct roots in F .*

Proof. We induct on $d = \deg f$. If $d = 0$, then f is a nonzero constant, which evaluates to itself everywhere and so has $0 \leq d$ roots. Let $d \geq 1$. If f has no root in F the claim holds; otherwise let α be a root. By the factor theorem, $f = (X - \alpha)q$, and $\deg q = d - 1$ by (2). Let $\beta \neq \alpha$ be any other root of f . Evaluating with Proposition 5.16,

$$0 = f(\beta) = (\beta - \alpha)q(\beta);$$

since $\beta - \alpha \neq 0$ and F has no zero divisors (Proposition 4.28), we get $q(\beta) = 0$. Every root of f other than α is therefore a root of q , and by the induction hypothesis q has at most $d - 1$ distinct roots. Hence f has at most $(d - 1) + 1 = d$. $\square$

Remark 5.19 (Why a field is needed). Both ingredients of the proof, the absence of zero divisors and the degree bookkeeping, are necessary. Over a ring with zero divisors the bound fails: in $\mathbb{Z}/8\mathbb{Z}$ the degree-2 polynomial $X^2 - 1$ has the four roots $\bar{1}, \bar{3}, \bar{5}, \bar{7}$, since $1^2 = 1$, $3^2 = 9 \equiv 1$, $5^2 = 25 \equiv 1$, and $7^2 = 49 \equiv 1 \pmod{8}$. The polynomial methods of Halo 2 therefore require a field, specifically a finite field $\mathbb{F}_p$ or $\mathbb{F}_q$.

Corollary 5.20 (Few points determine a low-degree polynomial). *Let $f, g \in F[X]$ both have degree at most d . If $f(\alpha) = g(\alpha)$ for $d + 1$ distinct values $\alpha \in F$, then $f = g$ as polynomials.*

Proof. The difference $h = f - g$ has $\deg h \leq \max\{\deg f, \deg g\} \leq d$ by (1) and vanishes at $d + 1$ distinct points. Were h nonzero, it would be a polynomial of degree at most d with $d + 1$ distinct roots, contradicting Theorem 5.18. Hence $h = 0$, that is, $f = g$. $\square$

5.5 Greatest common divisors

Definition 5.21 (Greatest common divisor). Let $f, g \in F[X]$, not both zero. A *greatest common divisor* of f and g is a polynomial d such that

1. $d \mid f$ and $d \mid g$; and
2. every common divisor c of f and g satisfies $c \mid d$.

A greatest common divisor is determined only up to multiplication by a unit; the unique *monic* one (Theorem 5.23) is singled out and denoted $\gcd(f, g)$. When $\gcd(f, g) = 1$, the polynomials f and g are called *coprime* (or *relatively prime*). Since every polynomial divides 0, we have $\gcd(f, 0) = \text{lc}(f)^{-1}f$ for $f \neq 0$.

Lemma 5.22 (Euclidean step). *Let $f, g \in F[X]$ with $g \neq 0$, and let $f = qg + r$ be the division of Theorem 5.12. Then the pairs (f, g) and (g, r) have exactly the same common divisors; in particular, their greatest common divisors coincide.*

Proof. If $c \mid f$ and $c \mid g$, then $c \mid (f - qg) = r$; so c is a common divisor of (g, r) . Conversely, if $c \mid g$ and $c \mid r$, then $c \mid (qg + r) = f$. The two sets of common divisors are therefore equal, hence so are their maximal elements under divisibility. $\square$

Theorem 5.23 (Euclidean algorithm; Bézout's identity). *Let $f, g \in F[X]$, not both zero. Then $\gcd(f, g)$ exists and is unique as a monic polynomial, and there exist $s, t \in F[X]$ with*

$$\gcd(f, g) = sf + tg. \quad (3)$$

Concretely: set $r_0 = f$ and $r_1 = g$, and while $r_i \neq 0$ divide

$$r_{i-1} = q_i r_i + r_{i+1}, \quad \deg r_{i+1} < \deg r_i.$$

The degrees strictly decrease, so some $r_{N+1} = 0$; the last nonzero remainder r_N , made monic, is $\gcd(f, g)$.

Proof. Termination. If $r_1 = 0$, the algorithm stops at once with $N = 0$ and $r_N = f \neq 0$. Otherwise the degrees $\deg r_1 > \deg r_2 > \dots$ form a strictly decreasing sequence of nonnegative integers, which cannot continue forever; so some $r_{N+1} = 0$ with $r_N \neq 0$.

The gcd property. Lemma 5.22, applied at each division, preserves the set of common divisors:

$$\begin{aligned}\{\text{common divisors of } (f, g)\} &= \{\text{common divisors of } (r_1, r_2)\} \\ &= \dots = \{\text{common divisors of } (r_N, 0)\},\end{aligned}$$

and the last set is exactly the set of divisors of r_N , since every polynomial divides 0. Thus r_N divides both f and g , and every common divisor of f and g divides r_N : the monic associate $\text{lc}(r_N)^{-1}r_N$ is a monic greatest common divisor.

Uniqueness. Two monic greatest common divisors divide each other, so they differ by a unit $u \in F^\times$ (Proposition 5.10); comparing leading coefficients of monic polynomials forces $u = 1$.

Bézout. We show by induction on i that each remainder r_i is an $F[X]$ -linear combination of f and g . The bases $r_0 = 1 \cdot f + 0 \cdot g$ and $r_1 = 0 \cdot f + 1 \cdot g$ are trivial, and the recurrence $r_{i+1} = r_{i-1} - q_i r_i$ expresses r_{i+1} as a combination once r_{i-1} and r_i are. Hence $r_N = s_0 f + t_0 g$ for some $s_0, t_0 \in F[X]$; dividing through by $\text{lc}(r_N)$ yields (3). $\square$

Remark 5.24 (Extended Euclidean algorithm). Carrying the linear-combination coefficients along with the remainders, as in the integer case (Theorem 2.13), produces s and t alongside the gcd. The standard application is inversion in a quotient $F[X]/(m)$ for irreducible m : if $\gcd(f, m) = 1$, then $sf + tm = 1$, so s is the inverse of f modulo m . Division in the extension fields $\mathbb{F}_q$ constructed in Section 6 is performed this way, and the algorithm also computes gcds of the polynomials a proof system manipulates. The curves of Halo 2 and Orchard, however, are defined over *prime* fields $\mathbb{F}_p$, not extension fields, and field inversion there uses the integer extended Euclidean algorithm or Fermat's little theorem, not the polynomial algorithm in $F[X]/(m)$.

Corollary 5.25 (Euclid's lemma in $F[X]$). *Let $p, f, g \in F[X]$.*

1. *If $\gcd(p, f) = 1$ and $p \mid fg$, then $p \mid g$.*
2. *If p is irreducible and $p \mid fg$, then $p \mid f$ or $p \mid g$.*

Here irreducible is the polynomial analogue of prime, made precise in Definition 5.26 below.

Proof. (1) Multiplying the Bézout identity $sp + tf = 1$ by g gives $spg + tfg = g$. The polynomial p divides both terms on the left—the second because $p \mid fg$ —hence divides g .

(2) Suppose $p \nmid f$. The gcd $\gcd(p, f)$ is a monic divisor of p that is not the monic associate of p (otherwise $p \mid f$). By irreducibility the only monic divisors of p are 1 and its monic associate, so $\gcd(p, f) = 1$ and part (1) applies. $\square$

5.6 Unique factorisation

Definition 5.26 (Irreducible polynomial). A polynomial $p \in F[X]$ is *irreducible over F* if $\deg p \geq 1$ and p admits no factorisation $p = gh$ with both $\deg g \geq 1$ and $\deg h \geq 1$; equivalently, p is nonconstant and its only factorisations are the trivial ones, (unit) $\times$ (associate of p). A nonconstant polynomial that is not irreducible is *reducible*.

Remark 5.27 (Irreducibility is relative to the base field). The polynomial $X^2 + 1$ is irreducible over $\mathbb{R}$ (it has no real root, and a real quadratic factors precisely when it has a root, by the test below), yet splits as $(X - i)(X + i)$ over $\mathbb{C}$, and factors as $(X + 1)^2$ over $\mathbb{F}_2$, by direct expansion in characteristic 2. Degree-1 polynomials are irreducible over every field. The *rootlessness test* states that a polynomial of degree 2 or 3 is irreducible over F if and only if it has no root in F , because any nontrivial factorisation of a quadratic or cubic must include a linear factor, which by the factor theorem (Theorem 5.17) supplies a root. The test fails from degree 4 onward: a product of two irreducible quadratics has no roots yet is reducible.

Theorem 5.28 ($F[X]$ is a unique factorisation domain). *Every nonconstant $f \in F[X]$ factors as*

$$f = c p_1 p_2 \cdots p_k$$

with $c = \text{lc}(f) \in F^\times$ a constant and each p_i monic irreducible. The factorisation is unique up to order: if also $f = c' q_1 \cdots q_l$ with $c' \in F^\times$ and each q_j monic irreducible, then $c = c'$, $k = l$, and the q_j are a reordering of the p_i .

Proof. The proof is modelled on the fundamental theorem of arithmetic (Theorem 2.18), with degree in place of absolute value.

Existence, by induction on $\deg f \geq 1$. If f is irreducible, then $f = \text{lc}(f) \cdot p_1$ with p_1 its monic associate. If $f = gh$ is reducible with $1 \leq \deg g, \deg h < \deg f$, the induction hypothesis factors g and h , and multiplying the two factorisations—collecting the two constants into one—factors f .

Uniqueness. Comparing leading coefficients in the two factorisations gives $c = c' = \text{lc}(f)$, since monic factors contribute leading coefficient 1 by (2). Cancelling the constants (Corollary 5.9) leaves $p_1 \cdots p_k = q_1 \cdots q_l$; we induct on k . For $k = 1$ the left side is irreducible, so $l = 1$ and $p_1 = q_1$. For $k \geq 2$: p_1 divides $q_1 \cdots q_l$, so iterating Euclid's lemma (Corollary 5.25(2)) gives $p_1 \mid q_j$ for some j . A monic irreducible dividing a monic irreducible equals it: $q_j = p_1 h$ forces $\deg h = 0$ by irreducibility of q_j , and comparing leading coefficients gives $h = 1$. Cancel $p_1 = q_j$ from both sides (Corollary 5.9). If $l = 1$, the cancellation leaves $p_2 \cdots p_k = 1$, which is impossible: the left side is a product of $k - 1 \geq 1$ nonconstant polynomials and so has degree at least 1 by (2), while $\deg 1 = 0$. Hence $l \geq 2$, and the induction hypothesis, applied to $p_2 \cdots p_k = \prod_{j' \neq j} q_{j'}$, concludes $k = l$ and matches the remaining factors. $\square$

Remark 5.29 (The ring $F[X]$ is a principal ideal domain). The degree function and Theorem 5.12 make $F[X]$ a *Euclidean domain*—an integral domain with division with remainder relative to a size measure—and every Euclidean domain is a *principal ideal domain* (PID): each ideal (Definition 4.17) of $F[X]$ is $(g) = \{qg : q \in F[X]\}$ for a single generator g , namely any nonzero element of least degree in a nonzero ideal (the zero ideal is generated by 0), by the same division-minimality argument that proved every ideal of $\mathbb{Z}$ principal (Example 4.18(2)). The generator of the ideal $\{sf + tg : s, t \in F[X]\}$ is a gcd of f and g , and Bézout's identity (3) is the explicit form of this principality. The results above follow the standard chain

$$\text{Euclidean} \implies \text{PID} \implies \text{UFD}$$

for $F[X]$, ending at Theorem 5.28.

5.7 Lagrange interpolation

Lagrange interpolation, the constructive converse of Corollary 5.20, underlies secret sharing, Reed–Solomon error-correcting codes, and the polynomial encodings in Halo 2.

Theorem 5.30 (Lagrange interpolation). *Let $x_0, \dots, x_d \in F$ be distinct (the nodes) and let $y_0, \dots, y_d \in F$ be arbitrary targets. There exists a unique $f \in F[X]$ with $\deg f \leq d$ and $f(x_i) = y_i$ for all i , given explicitly by*

$$f(X) = \sum_{i=0}^d y_i \ell_i(X), \quad \text{where} \quad \ell_i(X) = \prod_{j \neq i} \frac{X - x_j}{x_i - x_j}$$

are the Lagrange basis polynomials for the nodes.

Proof. The ℓ_i are polynomials. Each denominator $\prod_{j \neq i} (x_i - x_j)$ is a product of nonzero field elements, the nodes being distinct, hence is an invertible constant; so ℓ_i is a polynomial, of degree exactly d (a product of d linear factors divided by a nonzero constant).

The delta property. Writing δ_{im} for the Kronecker symbol (equal to 1 if $i = m$ and 0 otherwise),

$$\ell_i(x_m) = \delta_{im} \quad (0 \leq i, m \leq d) : \quad (4)$$

at $m = i$ every factor $(x_i - x_j)/(x_i - x_j)$ is 1, while at $m \neq i$ the factor with $j = m$ has numerator $x_m - x_m = 0$. Consequently $f(x_m) = \sum_i y_i \ell_i(x_m) = y_m$ for each m , and $\deg f \leq d$ by (1), each summand having degree at most d .

Uniqueness. Two interpolants of degree at most d agree at the $d + 1$ distinct nodes, hence are equal by Corollary 5.20. $\square$

Example 5.31 (A worked interpolation over $\mathbb{Q}$). This example computes the polynomial of degree at most 2 with $f(0) = 1$, $f(1) = 3$, and $f(2) = 7$. The basis polynomials for the nodes 0, 1, 2 are

$$\begin{aligned} \ell_0 &= \frac{(X-1)(X-2)}{(0-1)(0-2)} = \frac{(X-1)(X-2)}{2}, \\ \ell_1 &= \frac{(X-0)(X-2)}{(1-0)(1-2)} = -X(X-2), \\ \ell_2 &= \frac{(X-0)(X-1)}{(2-0)(2-1)} = \frac{X(X-1)}{2}. \end{aligned}$$

Hence

$$f = 1 \cdot \ell_0 + 3 \cdot \ell_1 + 7 \cdot \ell_2 = \frac{1}{2}(X^2 - 3X + 2) + 3(2X - X^2) + \frac{7}{2}(X^2 - X) = X^2 + X + 1,$$

which satisfies $f(0) = 1$, $f(1) = 3$, $f(2) = 7$ as required.

Remark 5.32 (The evaluation isomorphism). Fix $d + 1$ distinct nodes and let $V = \{f \in F[X] : \deg f \leq d\}$. In the language of the linear-algebra section later in the volume, V is a $(d + 1)$ -dimensional vector space over F with basis $1, X, \dots, X^d$, and the evaluation map

$$\text{ev}: V \rightarrow F^{d+1}, \quad f \mapsto (f(x_0), \dots, f(x_d)),$$

is F -linear. Its surjectivity is the existence half of Theorem 5.30, and its injectivity the uniqueness half; so ev is a linear isomorphism $V \cong F^{d+1}$. By the delta property (4), the Lagrange basis $\{\ell_i\}$ is the basis of V matched to the standard coordinates of F^{d+1} : the polynomial with prescribed values $(\gamma_0, \dots, \gamma_d)$ has coordinates $(\gamma_0, \dots, \gamma_d)$ in the basis $\{\ell_i\}$. Conversion between a polynomial's *coefficient representation* and its *evaluation representation* underlies fast polynomial arithmetic via the fast Fourier transform, treated with the roots of unity later in the volume, and the evaluation-domain encodings used throughout Halo 2.

Remark 5.33 (The vanishing polynomial). Alongside the Lagrange basis, the node set $\{x_0, \dots, x_d\}$ determines its *vanishing polynomial*

$$Z(X) = \prod_{i=0}^d (X - x_i),$$

the unique monic polynomial of degree $d + 1$ vanishing at every node. A polynomial g vanishes on the whole node set if and only if $Z \mid g$. One direction is immediate from Proposition 5.16. For the other, apply the factor theorem repeatedly: $g(x_0) = 0$ gives $g = (X - x_0)g_1$; evaluating at $x_1 \neq x_0$ gives $0 = (x_1 - x_0)g_1(x_1)$, so $g_1(x_1) = 0$ since F has no zero divisors, and $g_1 = (X - x_1)g_2$; continuing through the remaining nodes strips off one linear factor per node. The repetition requires the nodes to be distinct, so that the linear factors $X - x_i$ are pairwise coprime. A proof system uses the Lagrange basis and the vanishing polynomial to argue that a committed polynomial agrees with prescribed values on an evaluation domain.

5.8 Multiplicity and the formal derivative

A linear factor may divide a polynomial more than once, as in $X^2 - 2X + 1 = (X - 1)^2$. This subsection defines the multiplicity of a root, bounds the number of roots counted with multiplicity, and detects repeated roots with a derivative defined without limits.

Definition 5.34 (Multiplicity of a root). Let $f \in F[X]$ be nonzero and let $\alpha \in F$ be a root of f . The *multiplicity* $\text{mult}_\alpha(f)$ is the largest integer $m \geq 1$ with $(X - \alpha)^m \mid f$. A root of multiplicity 1 is *simple*; a root of multiplicity at least 2 is *repeated* (or *multiple*).

The largest such m exists: the powers $(X - \alpha)^m$ have strictly increasing degrees, so only finitely many can divide the fixed f , and at least one does by the factor theorem. In an equivalent characterisation, $m = \text{mult}_\alpha(f)$ holds exactly when $f = (X - \alpha)^m g$ with $g(\alpha) \neq 0$. Writing $f = (X - \alpha)^m g$ with m maximal forces $g(\alpha) \neq 0$, else the factor theorem would split a further factor $X - \alpha$ off g ; conversely, if $f = (X - \alpha)^m g$ with $g(\alpha) \neq 0$, then $(X - \alpha)^{m+1} \mid f$ would give $(X - \alpha)^m g = (X - \alpha)^{m+1} h$, and cancelling (Corollary 5.9) $g = (X - \alpha)h$, making α a root of g .

The number of roots counted *with* multiplicity is also bounded by the degree; the proof uses the following lemma.

Lemma 5.35 (Coprime factors multiply). Let $f_1, \dots, f_r \in F[X]$ be pairwise coprime ($\gcd(f_i, f_j) = 1$ for $i \neq j$) and suppose each f_i divides f . Then $f_1 f_2 \dots f_r \mid f$.

Proof. First let $r = 2$, and write $f = f_1 u = f_2 v$. Bézout (3) gives $s f_1 + t f_2 = 1$, so

$$f = s f_1 f + t f_2 f = s f_1 (f_2 v) + t f_2 (f_1 u) = f_1 f_2 (s v + t u),$$

and $f_1 f_2 \mid f$. For $r > 2$, induct: by hypothesis $f_1 \cdots f_{r-1} \mid f$, and the product $f_1 \cdots f_{r-1}$ is coprime to f_r —a nonconstant common divisor would have a monic irreducible factor (Theorem 5.28), which by Euclid's lemma (Corollary 5.25(2)) would divide some f_i with $i < r$ as well as f_r , contradicting $\gcd(f_i, f_r) = 1$. The case $r = 2$ then applies. $\square$

Proposition 5.36 (Counting roots with multiplicity). *Let $f \in F[X]$ be nonzero of degree d , with all its distinct roots $\alpha_1, \dots, \alpha_r \in F$ of multiplicities $m_1, \dots, m_r$. Then*

$$f = (X - \alpha_1)^{m_1} \cdots (X - \alpha_r)^{m_r} g$$

for some $g \in F[X]$ with no roots in F , and $\sum_i m_i \leq d$. The number of roots counted with multiplicity is thus at most d , with equality if and only if f splits into linear factors over F .

Proof. We first check that the powers $(X - \alpha_i)^{m_i}$ are pairwise coprime. Each $X - \alpha_i$ is monic irreducible (degree 1), and by unique factorisation the monic divisors of $(X - \alpha_i)^{m_i}$ are exactly the powers $(X - \alpha_i)^e$ with $0 \leq e \leq m_i$. For $i \neq j$ the only monic polynomial that is simultaneously a power of $X - \alpha_i$ and of $X - \alpha_j$ is 1, since $X - \alpha_i = X - \alpha_j$ would force $\alpha_i = \alpha_j$; so $\gcd((X - \alpha_i)^{m_i}, (X - \alpha_j)^{m_j}) = 1$.

Each $(X - \alpha_i)^{m_i}$ divides f by the definition of multiplicity, so Lemma 5.35 gives $f = (X - \alpha_1)^{m_1} \cdots (X - \alpha_r)^{m_r} g$. Maximality of each m_i gives $g(\alpha_i) \neq 0$: a further factor $X - \alpha_i$ inside g would raise the multiplicity. Nor has g any other root: a root β of g is a root of f , hence one of the α_i . Taking degrees with (2),

$$m_1 + \cdots + m_r + \deg g = d,$$

so $\sum_i m_i \leq d$, with equality precisely when $\deg g = 0$, that is, when f is a constant times a product of linear factors. $\square$

Definition 5.37 (Formal derivative). The *formal derivative* is the map $D: F[X] \rightarrow F[X]$ defined on $f = \sum_{i=0}^n a_i X^i$ by

$$f' = Df = \sum_{i=1}^n i a_i X^{i-1} = a_1 + 2a_2 X + \cdots + n a_n X^{n-1},$$

where the coefficient $i a_i$ means a_i added to itself i times in F —that is, a_i multiplied by the image of the integer i under the canonical map $\mathbb{Z} \rightarrow F$ (Definition 4.30). No limits are involved: the definition is formal and algebraic, valid over any field, and is introduced for one purpose, to detect repeated roots without appeal to analysis.

Proposition 5.38 (Rules of formal differentiation). *For all $f, g \in F[X]$, $c \in F$, $\alpha \in F$, and $m \geq 1$:*

1. (Linearity.) $(f + g)' = f' + g'$ and $(cf)' = cf'$.
2. (Product rule.) $(fg)' = f'g + fg'$.
3. (Power rule.) $((X - \alpha)^m)' = m(X - \alpha)^{m-1}$.

Proof. (1) Linearity is immediate from the coordinatewise definition of D , which is linear in the coefficients.

(2) Both sides are *bilinear* in the pair (f, g) : by part (1), each is linear in f with g fixed and linear in g with f fixed. It therefore suffices to verify the identity on monomials $f = X^a, g = X^b$, since every polynomial is a linear combination of monomials and the general case follows by expanding $f = \sum_a u_a X^a, g = \sum_b v_b X^b$ and applying the monomial case termwise. On monomials,

$$(X^a X^b)' = (X^{a+b})' = (a+b) X^{a+b-1} = aX^{a-1} X^b + X^a bX^{b-1} = (X^a)' X^b + X^a (X^b)',$$

with the convention that a term carrying the integer coefficient 0 (the case $a = 0$ or $b = 0$) is the zero polynomial.

(3) Induct on m . The base is $(X - \alpha)' = 1$. For the step, the product rule gives

$$\begin{aligned} ((X - \alpha)^m)' &= ((X - \alpha)(X - \alpha)^{m-1})' \\ &= (X - \alpha)^{m-1} + (X - \alpha)(m-1)(X - \alpha)^{m-2} \\ &= m(X - \alpha)^{m-1}. \end{aligned} \quad \square$$

The proof of part (3) of the next theorem needs a repeated root in an extension field not given in advance. The following lemma constructs that extension by applying the quotient construction of Example 4.22 to polynomial rings; its proof transposes the argument that made $\mathbb{Z}/p\mathbb{Z}$ a field (Corollary 4.26) from $\mathbb{Z}$ to $F[X]$.

Lemma 5.39 (Adjoining a root). *Let $h \in F[X]$ be irreducible. Then the quotient ring $K = F[X]/(h)$ is a field; the map $a \mapsto a + (h)$ embeds F into K as a subfield, so that K is an extension of F ; and the class $\alpha = X + (h)$ is a root of h in K .*

Proof. The principal ideal $(h) = hF[X]$ is an ideal of the commutative ring $F[X]$ (Example 4.18(2)), so K is a commutative ring under the coset operations (Proposition 4.20). Its identity is not zero: $1 + (h) = 0 + (h)$ would say $h \mid 1$, impossible since a factorisation $1 = hk$ has degree $\deg h + \deg k = 0$ by (2), while $\deg h \geq 1$.

Every nonzero class is a unit. Let $f + (h) \neq 0 + (h)$, i.e. $h \nmid f$. The gcd $\gcd(h, f)$ is then a monic divisor of the irreducible h other than its monic associate, hence equals 1, exactly as in the proof of Corollary 5.25(2); Bézout's identity (3) supplies $s, t \in F[X]$ with $sf + th = 1$, and reducing modulo (h) gives $(s + (h))(f + (h)) = 1 + (h)$. So K is a field.

The map $\iota(a) = a + (h)$ on constants preserves sums, products, and 1 directly from the coset operations of Definition 4.19, and it is injective: $\iota(a) = 0$ with $a \neq 0$ would make h divide a nonzero constant, again impossible by degrees. We identify F with its image, making K an extension field of F .

Finally, write $h = \sum_i c_i X^i$ and compute in K :

$$h(\alpha) = \sum_i \iota(c_i) (X + (h))^i = \left(\sum_i c_i X^i \right) + (h) = h + (h) = 0 + (h),$$

so α is a root of h in K . □

Theorem 5.40 (Derivative test for repeated roots; separability). *Let $f \in F[X]$ be nonzero and $\alpha \in F$.*

1. *The element α is a repeated root of f —a root of multiplicity at least 2—if and only if $f(\alpha) = 0$ and $f'(\alpha) = 0$.*
2. *If f has a repeated root in F , then $\gcd(f, f') \neq 1$.*
3. *Call f separable if it has no repeated root in any field containing F . Then f is separable if and only if $\gcd(f, f') = 1$ in $F[X]$.*

Proof. (1) Let α be a root of multiplicity $m \geq 1$ and write $f = (X - \alpha)^m g$ with $g(\alpha) \neq 0$. The product and power rules give

$$f' = m(X - \alpha)^{m-1}g + (X - \alpha)^m g' = (X - \alpha)^{m-1} [mg + (X - \alpha)g'].$$

If $m = 1$, the first factor is 1 and evaluating at α gives $f'(\alpha) = 1 \cdot g(\alpha) = g(\alpha) \neq 0$: a simple root of f is not a root of f' . If $m \geq 2$, then $(X - \alpha)^{m-1}$ divides f' and vanishes at α , so $f'(\alpha) = 0$. Together with the factor theorem for $f(\alpha) = 0$, this proves the equivalence.

(2) A repeated root $\alpha \in F$ is a common root of f and f' by (1), so the factor theorem puts the common factor $X - \alpha$ into both, and $(X - \alpha) \mid \gcd(f, f')$.

(3) The argument rests on the fact that *gcds are stable under field extension*: for an extension field K of F (Definition 4.25), the Euclidean algorithm of Theorem 5.23 run on $f, f' \in F[X]$ uses only field operations on their coefficients, which lie in F ; run inside $K[X]$ on the same inputs it produces the identical quotients and remainders. Hence the gcd computed in $K[X]$ equals the gcd computed in $F[X]$.

Suppose $\gcd(f, f') = 1$ in $F[X]$, and let $K \supseteq F$ be any extension. By stability, $\gcd(f, f') = 1$ in $K[X]$ as well, so by (2) applied over K , the polynomial f has no repeated root in K : f is separable.

Conversely, suppose $d = \gcd(f, f')$ is nonconstant; we exhibit a repeated root in some extension. Let b be a monic irreducible factor of d (Theorem 5.28), and let $K = F[X]/(b)$ be the extension field supplied by Lemma 5.39, in which b has a root α . The polynomial b divides d , which divides both f and f' ; evaluating the factorisations at α (Proposition 5.16) gives $f(\alpha) = f'(\alpha) = 0$. By (1), applied over K , the element α is a repeated root of f in K : f is not separable. $\square$

Remark 5.41 (Formal derivatives in positive characteristic). The integer coefficient i in $f' = \sum_i i a_i X^{i-1}$ is interpreted in F , and may vanish there. Over a field of characteristic p —where $p = 0$ in F , p prime, the setting of $\mathbb{F}_p$ and $\mathbb{F}_q$ (Definition 4.30)—the derivative of X^p is $p X^{p-1} = 0$. Thus $f = X^p - 1$ over $\mathbb{F}_p$ has $f' = 0$ and $\gcd(f, f') = \gcd(f, 0) = f \neq 1$; correspondingly,

$$X^p - 1 = (X - 1)^p \quad \text{in } \mathbb{F}_p[X],$$

a factorisation the finite-fields section establishes through the freshman's-dream identity $(a + b)^p = a^p + b^p$ of characteristic p (Lemma 6.13, applied to $(X + (-1))^p$ together with $(-1)^p = -1$ in every $\mathbb{F}_p$): a single root 1 of multiplicity p . Arguments over the finite fields of Orchard must account for this characteristic- p behaviour.

5.9 Polynomials versus polynomial functions

This subsection determines when a polynomial may be identified with the function it induces.

Definition 5.42 (Polynomial function). Every $f \in F[X]$ induces a *polynomial function*

$$\tilde{f}: F \rightarrow F, \quad \alpha \mapsto f(\alpha).$$

The assignment $f \mapsto \tilde{f}$ is a ring homomorphism from $F[X]$ to the ring F^F of *all* functions $F \rightarrow F$ under pointwise operations ($(u + v)(\alpha) = u(\alpha) + v(\alpha)$ and $(uv)(\alpha) = u(\alpha)v(\alpha)$), by Proposition 5.16 applied at every point.

Proposition 5.43 (When formal equals functional). Let $f, g \in F[X]$.

1. If F is infinite, then $\tilde{f} = \tilde{g}$ as functions implies $f = g$ as polynomials. Equivalently, $f \mapsto \tilde{f}$ is injective, and polynomials over an infinite field may be identified with polynomial functions.
2. If F is finite, the map is not injective. Over $\mathbb{F}_p$, the nonzero polynomial $X^p - X$ induces the zero function, since Fermat's little theorem (Corollary 3.32) gives $\alpha^p = \alpha$ for every $\alpha \in \mathbb{F}_p$.

Proof. (1) Let $h = f - g$, so that $\tilde{h}$ is the zero function: h vanishes at every point of the infinite set F . Were h nonzero of degree d , it would have at most d roots (Theorem 5.18), yet it has infinitely many—a contradiction. Hence $h = 0$.

(2) Let F be finite, with $q = |F|$ elements, and consider the *vanishing-polynomial witness*

$$w(X) = \prod_{\alpha \in F} (X - \alpha),$$

a monic polynomial of degree q —in particular nonzero as a formal object—which vanishes at every point of F by construction. Thus $\tilde{w} = \tilde{0}$ while $w \neq 0$: the map $f \mapsto \tilde{f}$ identifies distinct polynomials. Over $F = \mathbb{F}_p$ the witness specialises to $X^p - X$: by Fermat's little theorem every element of $\mathbb{F}_p$ is a root of the monic degree- p polynomial $X^p - X$, so Proposition 5.36 factors it as $\prod_{\alpha \in \mathbb{F}_p} (X - \alpha)$ —the p distinct linear factors exhaust the degree—and $X^p - X$ induces the zero function despite having degree $p \geq 1$. $\square$

Remark 5.44 (Why polynomials are defined formally). Part (2) of Proposition 5.43 is the reason polynomials are defined as coefficient sequences and not as functions. Cryptography works over finite fields, where the functional view loses information: the polynomials X and X^p induce the same function on $\mathbb{F}_p$, yet have degrees 1 and p . The security of polynomial commitment schemes rests on the *degree*, a property of the formal object invisible to the induced function over a finite field. A prover commits to a low-degree *formal* polynomial, and the verifier tests it by *evaluation*. The Schwartz–Zippel lemma quantifies the gap between agreement at the tested points and identity to the claimed formal polynomial.

5.10 The Schwartz–Zippel lemma

To draw α *uniformly at random* from a finite nonempty set S means to select each element with equal likelihood $1/|S|$; the *probability* of an event is then the fraction of elements of S producing it, so that $\Pr[f(\alpha) = 0] = |\{\alpha \in S : f(\alpha) = 0\}|/|S|$. The theorem needs only this counting case; probability is developed later in the volume.

Theorem 5.45 (Univariate Schwartz–Zippel). *Let $f \in F[X]$ be nonzero of degree at most d , let $S \subseteq F$ be finite and nonempty, and let α be drawn uniformly at random from S . Then*

$$\Pr[f(\alpha) = 0] \leq \frac{d}{|S|}. \quad (5)$$

More generally, if $f, g \in F[X]$ are distinct, each of degree at most d , then $\Pr[f(\alpha) = g(\alpha)] \leq d/|S|$.

Proof. By Theorem 5.18, the nonzero f has at most d roots in F , hence at most d in the subset S . Under the uniform draw,

$$\Pr[f(\alpha) = 0] = \frac{|\{\alpha \in S : f(\alpha) = 0\}|}{|S|} \leq \frac{d}{|S|}.$$

For the general form, apply this to $h = f - g$, which is nonzero and has degree at most d by (1), noting that $f(\alpha) = g(\alpha)$ exactly when $h(\alpha) = 0$. $\square$

Remark 5.46 (Polynomial identity testing). Inequality (5) is the probabilistic refinement of Corollary 5.20: a *single* random evaluation point detects unequal polynomials of degree at most d , except with probability at most $d/|S|$. With $|S|$ exponentially large ($S = \mathbb{F}_q$ with $q \approx 2^{254}$, the 255-bit fields underlying Orchard) and d polynomially bounded, the failure probability is exponentially small, hence *negligible*, a term given its precise asymptotic meaning later in the volume. The bound is the soundness argument for polynomial identity testing, and hence for the polynomial commitment schemes and arithmetisation checks of Halo 2, which verify a purported polynomial identity by evaluating both sides at a random challenge and comparing the results.

Remark 5.47 (Multivariate generalisation). The full Schwartz–Zippel lemma extends to polynomials in several indeterminates $f \in F[X_1, \dots, X_n]$: a nonzero polynomial of total degree d —the largest sum of exponents $i_1 + \dots + i_n$ over the monomials with nonzero coefficient—vanishes at a uniformly random point of S^n with probability at most $d/|S|$. The univariate case proved here is the base case of the multivariate inductive proof and the only version the later volumes invoke directly. Where an identity in several random challenges arises, for instance in a permutation argument drawing a pair of challenges (β, γ) , the multivariate bound applies and follows from the univariate lemma applied once per variable.

Over a 255-bit field $\mathbb{F}_q$, distinct polynomials f and g of degree at most d agree on a uniformly random challenge $\alpha \in \mathbb{F}_q$ with probability at most $d/q \approx d \cdot 2^{-254}$, astronomically small even with d in the millions.

6 Finite fields

This section studies the finite fields, the fields of §4 with finitely many elements. A verifier must divide by every nonzero value, which the arithmetic of a 64-bit processor word, modulo 2^{64} , does not permit: in $\mathbb{Z}/2^{64}\mathbb{Z}$ no even class has an inverse (Theorem 2.24). The section gives two inversion algorithms for $\mathbb{F}_p$ and proves that every finite field has p^k elements for a prime p (Theorem 6.9), so no field has 6, 10, or 12 elements. Fields of size 2^k exist for every positive k , but the field with 2^{64} elements is not $\mathbb{Z}/2^{64}\mathbb{Z}$: it is the polynomial quotient of Theorem 6.10, with different addition and multiplication. The section then defines the Frobenius map $x \mapsto x^p$ and proves that the multiplicative group $\mathbb{F}_q^\times$ is *cyclic*, the powers of a single element running through every nonzero value (Theorem 6.17). Cyclicity gives discrete logarithms their meaning and guarantees the roots of unity on which the polynomial constructions of later sections depend.

6.1 The prime field and two inversion routes

Most constructions of the series are carried out over the field $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$ of Corollary 4.26, for a prime p of roughly 255 bits. Its addition, subtraction, and multiplication are the residue arithmetic of §2; inversion requires an algorithm, since a proof that an inverse exists (§4) does not compute it.

Construction 6.1 (Inversion by the extended Euclidean algorithm). Given a with $0 < a < p$, run the extended Euclidean algorithm (Theorem 2.13) on the pair (a, p) . Since p is prime and $p \nmid a$, the gcd is 1, and the algorithm outputs integers u, v with $au + pv = 1$. Reducing modulo p gives $au \equiv 1 \pmod{p}$, that is,

$$a^{-1} = [u]_p \quad \text{in } \mathbb{F}_p.$$

The cost is $O(\log p)$ division steps (Theorem 2.13). This is the standard general-purpose method; a worked instance is Example 2.25, which computed $[17]_{43}^{-1} = [38]_{43}$ by exactly this route.

The second method rests on Fermat's little theorem and on the following fast exponentiation.

Construction 6.2 (Square-and-multiply). Let a be an element of any structure with an associative multiplication, and let $e \geq 1$ have binary expansion $e = \sum_i e_i 2^i$ with digits $e_i \in \{0, 1\}$. Squaring repeatedly produces the values

$$a^{2^0} = a, \quad a^{2^{i+1}} = (a^{2^i})^2,$$

and multiplying together the factors indexed by the nonzero digits gives

$$a^e = \prod_{i: e_i=1} a^{2^i}.$$

The computation uses $\lceil \log_2 e \rceil$ squarings and at most $\lceil \log_2 e \rceil$ further multiplications— $O(\log e)$ multiplications in all, against the $e - 1$ multiplications of the naive product $a \cdot a \cdots a$. For the 255-bit exponents of later sections the former is a few hundred multiplications and the latter a count beyond the reach of any computer.

Remark 6.3. Written additively, the same algorithm computes the multiple $e \cdot P$ of a group element P by repeated doubling and selective adding. Under the name *double-and-add* it is the algorithm for scalar multiplication on elliptic curves (Definition 10.20).

Construction 6.4 (Inversion by Fermat's little theorem). Let $a \in \mathbb{F}_p$ be nonzero. Fermat's little theorem (Corollary 3.32) gives $a^{p-1} = 1$, hence $a \cdot a^{p-2} = 1$, and so

$$a^{-1} = a^{p-2},$$

computed by square-and-multiply (Construction 6.2) with exponent $p - 2$, at a cost of $O(\log p)$ modular multiplications when $p > 2$. For $p = 2$, the only nonzero element is 1, whose inverse is 1; the exponent is zero and no multiplication is needed.

Remark 6.5 (Constant-time inversion). The two methods have comparable asymptotic cost. In Construction 6.4 the sequence of operations performed—which steps square, which multiply—depends only on the binary digits of the *public* exponent $p - 2$, never on the secret input a ; the method is *constant-time-friendly*. The extended Euclidean method, by contrast, performs a sequence of divisions whose lengths and quotients depend on a itself, so its running time and memory-access pattern can leak information about a to an observer timing the computation. Side-channel-resistant implementations therefore often prefer the Fermat method despite its slightly larger constant factor.

6.2 Characteristic, prime subfield, and prime-power order

This subsection determines the possible cardinalities of a finite field through the characteristic of Section 4.5, defined by the canonical homomorphism

$$\iota: \mathbb{Z} \rightarrow F, \quad \iota(n) = n \cdot 1_F,$$

whose kernel is an ideal $n\mathbb{Z}$ of $\mathbb{Z}$; the characteristic $\text{char}(F)$ is its nonnegative generator (Definition 4.30).

Theorem 6.6. *Every finite field F has prime characteristic: $\text{char}(F) = p$ for some prime p . In particular $\text{char}(\mathbb{F}_p) = p$.*

Proof. The map ι cannot be injective, since its domain $\mathbb{Z}$ is infinite and its codomain F is finite; two distinct integers $m < n$ must satisfy $\iota(m) = \iota(n)$, and then $n - m$ is a nonzero element of $\ker \iota$. Hence $\ker \iota = n\mathbb{Z}$ with $n > 0$, that is, $\text{char}(F) > 0$; and the characteristic of a field, being nonzero, is prime by Theorem 4.32. For $F = \mathbb{F}_p$ itself, Example 4.31 computed $\text{char}(\mathbb{Z}/p\mathbb{Z}) = p$ directly. $\square$

Every field of characteristic p contains a canonical copy of $\mathbb{F}_p$ (Remark 4.33), defined as follows.

Definition 6.7 (Subfield; prime subfield). A *subfield* of a field F is a subset $K \subseteq F$ containing 0 and 1 and closed under addition, negation, multiplication, and inversion of nonzero elements; with the restricted operations, K is itself a field. The *prime subfield* of F is the intersection of

all subfields of F —equivalently, the smallest subfield of F , and equivalently again the subfield generated by 1_F .

The intersection of any family of subfields is again a subfield, since each defining closure property is preserved under intersection; the intersection of *all* subfields is contained in every subfield, hence is the smallest one. That this smallest subfield is exactly what 1_F generates—the closure of $\{1_F\}$ under the field operations—follows from the classification below, whose proof exhibits the prime subfield inside that closure.

Proposition 6.8 (Classification of prime subfields). *Let F be a field. If $\text{char}(F) = p > 0$, the prime subfield of F is isomorphic to $\mathbb{F}_p$. If $\text{char}(F) = 0$, it is isomorphic to $\mathbb{Q}$.*

Proof. Suppose first that $\text{char}(F) = p > 0$, so that $\ker \iota = p\mathbb{Z}$. Define

$$\bar{\iota}: \mathbb{F}_p = \mathbb{Z}/p\mathbb{Z} \rightarrow F, \quad \bar{\iota}([a]_p) = a \cdot 1_F.$$

A single equivalence shows the map well defined and injective: $a \cdot 1_F = b \cdot 1_F$ holds if and only if $(a - b) \cdot 1_F = 0$, i.e. $a - b \in \ker \iota = p\mathbb{Z}$, i.e. $[a]_p = [b]_p$. It is a ring homomorphism because ι is one (Section 4.5) and $\bar{\iota}$ evaluates ι on representatives of residue classes. Its image $K = \{a \cdot 1_F : a \in \mathbb{Z}\}$ is therefore a subring of F isomorphic to the field $\mathbb{F}_p$, and K is a subfield: for $[a]_p \neq [0]_p$ the computation $\bar{\iota}([a]_p) \bar{\iota}([a]_p^{-1}) = \bar{\iota}([1]_p) = 1_F$ exhibits the inverse of each nonzero element of K inside K . Finally, K lies in every subfield: a subfield contains 1_F , hence all its additive multiples $a \cdot 1_F$ by closure under addition and negation. So K is contained in the intersection of all subfields and is itself a subfield, so K equals that intersection: the prime subfield is $K \cong \mathbb{F}_p$.

Now suppose $\text{char}(F) = 0$, so that ι is injective and $b \cdot 1_F \neq 0$ for every nonzero integer b . Define

$$j: \mathbb{Q} \rightarrow F, \quad j\left(\frac{a}{b}\right) = (a \cdot 1_F)(b \cdot 1_F)^{-1}.$$

The map is well defined: if $a/b = c/d$ with $b, d \neq 0$, then $ad = bc$ in $\mathbb{Z}$, so $(a \cdot 1_F)(d \cdot 1_F) = (b \cdot 1_F)(c \cdot 1_F)$ by applying ι , and multiplying both sides by $(b \cdot 1_F)^{-1}(d \cdot 1_F)^{-1}$ gives $(a \cdot 1_F)(b \cdot 1_F)^{-1} = (c \cdot 1_F)(d \cdot 1_F)^{-1}$. That j preserves sums, products, and 1 is the usual arithmetic of fractions, transported by the homomorphism ι and commutativity. The map is injective: $j(a/b) = 0$ forces $a \cdot 1_F = 0$, hence $a = 0$. The image of j is thus a subfield of F isomorphic to $\mathbb{Q}$; and it lies in every subfield K , since K contains 1_F , hence every $a \cdot 1_F$, hence—being closed under inversion and multiplication—every $(a \cdot 1_F)(b \cdot 1_F)^{-1}$. As before the image must equal the intersection of all subfields, and the prime subfield is a copy of $\mathbb{Q}$. $\square$

The size constraint follows by counting over the prime subfield. Every field F of characteristic p is a vector space over its prime subfield $\mathbb{F}_p$: vectors are the elements of F , vector addition is the field addition, and scalar multiplication $c \cdot x$ for $c \in \mathbb{F}_p \subseteq F$ is the field multiplication, so the vector-space axioms are instances of the ring axioms of F . The proof uses two facts, proved in Section 8 independently of the present theorem: a space with a finite spanning set has a basis, and coordinates with respect to a basis are unique.

Theorem 6.9 (Finite fields have prime-power order). *Every finite field F has cardinality $|F| = p^k$ for some prime p and integer $k \geq 1$, where $p = \text{char}(F)$.*

Proof. By Theorem 6.6 the characteristic of F is a prime p , and by Proposition 6.8 the prime subfield of F is a copy of $\mathbb{F}_p$, with which we identify it. As explained above, F is then a vector space over $\mathbb{F}_p$. It is spanned by the finite set F itself, so it has a finite basis $e_1, \dots, e_k$; and $k \geq 1$ because $1 \neq 0$ makes F nonzero. Every element of F is thus

$$c_1 e_1 + c_2 e_2 + \dots + c_k e_k$$

for a *unique* tuple $(c_1, \dots, c_k)$ with each $c_i \in \mathbb{F}_p$: existence because the e_i span, uniqueness because coordinates with respect to a basis are unique (Section 8). The assignment $(c_1, \dots, c_k) \mapsto \sum_i c_i e_i$ is therefore a bijection from $\mathbb{F}_p^k$ to F , and counting tuples gives $|F| = p \cdot p \cdots p = p^k$. $\square$

Theorem 6.10 (Existence and uniqueness of $\mathbb{F}_q$). *For every prime power $q = p^k$ there exists a field with exactly q elements, and any two finite fields of the same cardinality are isomorphic. One constructs such a field as the quotient $\mathbb{F}_p[x]/(f)$ of the polynomial ring by the ideal generated by an irreducible polynomial f of degree k —the construction recorded in Example 4.22.*

Remark 6.11. The fields on the critical path of Halo 2 and Orchard are *prime* fields, the case $q = p$ with $k = 1$, where the theorem is proved: existence is Corollary 4.26, and uniqueness holds because a field with p elements has order $p^k = p$ with $k = 1$ by Theorem 6.9, so it equals its own prime subfield, a copy of $\mathbb{F}_p$ (Proposition 6.8). The general case $k \geq 2$ would require polynomial factorisation over field extensions; no later result depends on it, so it is stated without proof. *Convention, fixed henceforth:* the letter $q = p^k$ denotes a prime power, and $\mathbb{F}_q$ a field with q elements, of characteristic p ; results are stated for general $\mathbb{F}_q$ whenever the proof is identical to the prime case, and for the applications to Halo 2 one may take $q = p$ throughout.

6.3 The Frobenius endomorphism

The identity $(a + b)^p = a^p + b^p$, a common error over $\mathbb{Q}$ and $\mathbb{R}$, where it is false for every $p \geq 2$, is a theorem in characteristic p . Its proof expands $(a + b)^p$ by the binomial theorem.

Lemma 6.12 (Binomial theorem; the coefficients count subsets). *Let R be a commutative ring. For all $a, b \in R$ and every integer $n \geq 0$,*

$$(a + b)^n = \sum_{k=0}^n \binom{n}{k} a^k b^{n-k},$$

where the integer coefficient $\binom{n}{k}$ acts as the additive multiple $\binom{n}{k} \cdot x = x + \dots + x$, equals the number of k -element subsets of an n -element set, and satisfies

$$\binom{n}{k} = \frac{n!}{k! (n-k)!}.$$

Proof. Expand the product $(a + b)^n = (a + b) \cdots (a + b)$ by distributivity without collecting terms: the result is a sum of 2^n products, one for each way of choosing, from every factor, either its a or its b . By commutativity such a product equals $a^k b^{n-k}$ exactly when a was chosen from k of the n factors, and it is determined by the set of those k factors; so the coefficient of $a^k b^{n-k}$ is the number of k -element subsets of the n factors, and collecting the terms gives the displayed sum.

For the formula, count the sequences of k distinct elements of an n -element set in two ways. Choosing the entries in turn, there are n choices for the first, $n - 1$ for the second, and $n - k + 1$ for the k -th: $n(n - 1) \cdots (n - k + 1) = n! / (n - k)!$ sequences. Alternatively, each such sequence is a k -element subset together with an ordering of it, and a k -element set has $k!$ orderings — the same count with k in place of n — so there are $k!$ sequences per subset, and the number of subsets is $n! / (k! (n - k)!)$. $\square$

Lemma 6.13 (Freshman's dream). *Let R be a commutative ring of prime characteristic p . Then for all $a, b \in R$,*

$$(a + b)^p = a^p + b^p,$$

and more generally $(a + b)^{p^n} = a^{p^n} + b^{p^n}$ for every $n \geq 1$.

Proof. By the binomial theorem (Lemma 6.12),

$$(a + b)^p = \sum_{i=0}^p \binom{p}{i} a^i b^{p-i},$$

where the integer coefficient $\binom{p}{i}$ acts as the additive multiple $\binom{p}{i} \cdot x = x + \cdots + x$. We claim that $p \mid \binom{p}{i}$ for $0 < i < p$. From the factorial identity of the same lemma

$$\binom{p}{i} i! (p - i)! = p!$$

the prime p divides the right-hand side. It does not divide $i! (p - i)!$: that product is a product of integers all smaller than p , and if p divided the product it would divide one of the factors by Euclid's lemma for primes (Lemma 2.17), which is impossible. Applying Euclid's lemma to the left-hand side instead, p must divide $\binom{p}{i}$, say $\binom{p}{i} = pm$.

Now let $0 < i < p$ and consider the corresponding term. Writing $\iota(n) = n \cdot 1_R$ for the canonical homomorphism of Section 4.5, the coefficient acts as multiplication by $\iota(\binom{p}{i}) = \iota(pm) = \iota(p) \iota(m) = 0 \cdot \iota(m) = 0$, since $\text{char}(R) = p$ means exactly $\iota(p) = p \cdot 1_R = 0$. Every middle term of the binomial expansion therefore vanishes, and the terms $i = p$ and $i = 0$ leave $(a + b)^p = a^p + b^p$.

The general case follows by induction on n , the base case $n = 1$ being the identity just proved. For $n \geq 2$, apply the base case to the elements $a^{p^{n-1}}$ and $b^{p^{n-1}}$:

$$(a + b)^{p^n} = ((a + b)^{p^{n-1}})^p = (a^{p^{n-1}} + b^{p^{n-1}})^p = a^{p^n} + b^{p^n},$$

using the inductive hypothesis in the middle step. $\square$

A homomorphism from a structure to itself is an *endomorphism*. In characteristic p the p -th power map is one: it preserves addition by the lemma, and multiplication by commutativity.

Definition 6.14 (Frobenius endomorphism). Let F be a field of characteristic $p > 0$. The *Frobenius map* of F is

$$\text{Frob}: F \rightarrow F, \quad \text{Frob}(x) = x^p.$$

Proposition 6.15. *Let F be a field of characteristic p . The Frobenius map is a ring homomorphism $F \rightarrow F$ fixing the prime subfield pointwise. If F is finite, Frob is an automorphism of F —a bijective ring homomorphism of F onto itself.*

Proof. Multiplicativity follows from commutativity and associativity alone: $(xy)^p = x^p y^p$. Additivity is the freshman's dream (Lemma 6.13), and $1^p = 1$; so Frob is a ring homomorphism.

Injectivity is an instance of a general fact used again later: *every ring homomorphism between fields is injective*. If $\phi: F \rightarrow F'$ had $\phi(x) = 0$ for some $x \neq 0$, then

$$1 = \phi(1) = \phi(x^{-1}x) = \phi(x^{-1})\phi(x) = \phi(x^{-1}) \cdot 0 = 0$$

in F' , contradicting $1 \neq 0$ in a field. In particular Frob is injective.

The prime subfield is fixed pointwise. By Proposition 6.8 its elements are the values $\bar{i}([a]_p) = a \cdot 1_F$, and since $\bar{i}$ is a ring homomorphism,

$$\text{Frob}(\bar{i}([a]_p)) = \bar{i}([a]_p)^p = \bar{i}([a]_p^p) = \bar{i}([a^p]_p) = \bar{i}([a]_p),$$

the last step because $a^p \equiv a \pmod{p}$ for all integers a (Corollary 3.32).

Finally, let F be finite. An injective map from a finite set to itself is surjective—the injectivity-implies-surjectivity argument of Section 4.4—so Frob is a bijective ring homomorphism of F onto itself, an automorphism. $\square$

The iterates $\text{Frob}^n(x) = x^{p^n}$ are again homomorphisms, by the general case of the freshman's dream. On $\mathbb{F}_p$ the Frobenius map is the identity; on larger fields of characteristic p it is not, and Remark 6.24 records its role in cryptography.

6.4 Cyclicity of the multiplicative group

The multiplicative group $\mathbb{F}_q^\times$ is a finite abelian group of order $q - 1$, since every nonzero element of a field is invertible. Finite abelian groups need not be cyclic: the unit group $(\mathbb{Z}/8\mathbb{Z})^\times$ of Example 3.22 has order 4 and no element of order 4. The proof that $\mathbb{F}_q^\times$ is cyclic is a counting argument of Gauss, resting on an identity for the totient function φ and using only its *counting definition* (Definition 2.26): $\varphi(n)$ is the number of integers a with $1 \leq a \leq n$ and $\gcd(a, n) = 1$.

Lemma 6.16 (Gauss's divisor-sum identity). *For every integer $n \geq 1$,*

$$\sum_{d|n} \varphi(d) = n,$$

the sum running over the positive divisors d of n .

Proof. Partition the set $\{1, 2, \dots, n\}$ according to the value of $\gcd(a, n)$. For each a the $\gcd$ is a positive divisor d of n , so the classes

$$C_d = \{a : 1 \leq a \leq n, \gcd(a, n) = d\}, \quad d \mid n,$$

are disjoint and cover $\{1, \dots, n\}$; hence $n = \sum_{d \mid n} |C_d|$.

We claim $|C_d| = \varphi(n/d)$. A member of C_d is divisible by d , so it has the form $a = db$ with $1 \leq b \leq n/d$; we show that, for such a ,

$$\gcd(db, n) = d \iff \gcd(b, n/d) = 1.$$

For the forward direction, suppose $e = \gcd(b, n/d) > 1$; then de divides both $db = a$ and $d \cdot (n/d) = n$, so de is a common divisor exceeding d and $\gcd(a, n) \geq de > d$. For the converse, suppose $\gcd(b, n/d) = 1$. The integer d is a common divisor of a and n , so d divides $g = \gcd(a, n)$ by the strongest-common-divisor property (Remark 2.12); write $g = dc$ with $c \geq 1$. From $dc \mid db$ we get $c \mid b$, and from $dc \mid n = d(n/d)$ we get $c \mid n/d$ (cancel d in each divisibility); so c divides $\gcd(b, n/d) = 1$, forcing $c = 1$ and $g = d$.

The members of C_d are therefore exactly the integers db with $1 \leq b \leq n/d$ and $\gcd(b, n/d) = 1$, and by the counting definition of the totient (Definition 2.26) there are precisely $\varphi(n/d)$ of them. Summing,

$$n = \sum_{d \mid n} \varphi\left(\frac{n}{d}\right) = \sum_{d \mid n} \varphi(d),$$

the second equality because $d \mapsto n/d$ is a bijection of the set of positive divisors of n onto itself. $\square$

Theorem 6.17 (Finite subgroups of $F^\times$ are cyclic). *Let F be any field and let $G \leq F^\times$ be a finite subgroup of the multiplicative group, of order N . Then G is cyclic. In particular, for a finite field $\mathbb{F}_q$ the whole multiplicative group $\mathbb{F}_q^\times$ is cyclic of order $q - 1$.*

Proof. For each divisor $d \mid N$, let

$$\psi(d) = |\{a \in G : \text{ord}(a) = d\}|$$

count the elements of G of order exactly d . Every element of the finite group G has finite order dividing $N = |G|$ (Corollary 3.30), so the sets counted by the $\psi(d)$ partition G and

$$\sum_{d \mid N} \psi(d) = N. \tag{6}$$

We claim that for each $d \mid N$, either $\psi(d) = 0$ or $\psi(d) = \varphi(d)$. Suppose $\psi(d) > 0$ and choose $a \in G$ of order d . The powers $a^0, a^1, \dots, a^{d-1}$ are d distinct elements of G (Proposition 3.12(3)), and each satisfies

$$(a^j)^d = (a^d)^j = 1^j = 1,$$

so each is a root in F of the polynomial $X^d - 1 \in F[X]$. That polynomial has degree d , so it has at most d roots in F (Theorem 5.18); the d distinct powers of a therefore account for all of its roots.

Now take any $b \in G$ of order d . Then $b^d = 1$, so b is a root of $X^d - 1$ and hence $b = a^j$ for some $0 \leq j < d$. By Corollary 3.23,

$$\text{ord}(a^j) = \frac{d}{\gcd(d, j)},$$

which equals d exactly when $\gcd(j, d) = 1$. The elements of order d in G are thus precisely the powers a^j with $0 \leq j < d$ and $\gcd(j, d) = 1$, and by the counting definition of the totient there are $\varphi(d)$ of them: $\psi(d) = \varphi(d)$. This proves the claim, and with it the inequality $\psi(d) \leq \varphi(d)$ for every $d \mid N$.

Summing the inequality over the divisors of N and comparing (6) with Gauss's identity (Lemma 6.16),

$$N = \sum_{d \mid N} \psi(d) \leq \sum_{d \mid N} \varphi(d) = N.$$

The two ends are equal, so the inequality is an equality term by term: $\psi(d) = \varphi(d)$ for every divisor $d \mid N$. In particular

$$\psi(N) = \varphi(N) \geq 1,$$

since $a = 1$ always witnesses $\gcd(1, N) = 1$ in the totient count. Hence G contains an element g of order N ; its powers form a subgroup of G with N distinct elements (Proposition 3.12(3)), which must be all of G . So $G = \langle g \rangle$ is cyclic.

For the final statement, let $F = \mathbb{F}_q$ be finite. The multiplicative group $\mathbb{F}_q^\times$ has $q - 1$ elements—every nonzero element of a field is a unit—and is a finite subgroup of itself, so it is cyclic of order $q - 1$. $\square$

Corollary 6.18 (Element-order census in cyclic groups). *A cyclic group G of order N contains, for each divisor $d \mid N$, exactly $\varphi(d)$ elements of order d , and no elements of any other order. In particular G has exactly $\varphi(N)$ generators.*

Proof. Write $G = \langle g \rangle$ with $\text{ord}(g) = N$; the elements of G are the powers g^j , $0 \leq j < N$, each appearing once (Proposition 3.12(3)). By Corollary 3.23, $\text{ord}(g^j) = N / \gcd(N, j)$, which is always a divisor of N ; and $\text{ord}(g^j) = d$ if and only if $\gcd(j, N) = N/d$. By the gcd computation in the proof of Lemma 6.16 (applied with $n = N$ and divisor N/d), the exponents $j \in \{1, \dots, N\}$ with $\gcd(j, N) = N/d$ are exactly $j = (N/d)t$ with $1 \leq t \leq d$ and $\gcd(t, d) = 1$, of which there are $\varphi(d)$ by Definition 2.26. The generators are the elements of order N , numbering $\varphi(N)$. $\square$

Definition 6.19 (Primitive root). A generator of the cyclic group $\mathbb{F}_q^\times$ is called a *primitive root* (or *primitive element*) of $\mathbb{F}_q$. Equivalently, an element $g \in \mathbb{F}_q^\times$ is primitive if and only if $\text{ord}(g) = q - 1$.

Primitive roots exist by Theorem 6.17; by Corollary 6.18 there are exactly $\varphi(q - 1)$ of them.

Remark 6.20. The group-theoretic term “generator of $\mathbb{F}_q^\times$ ” and the polynomial term “primitive $(q - 1)$ -th root of unity” (Definition 9.3) denote the same elements; the identification is restated once roots of unity have been defined in general.

Example 6.21 (Primitive roots of $\mathbb{F}_7$). The group $\mathbb{F}_7^\times$ has order 6. Testing $g = 3$, the successive powers modulo 7 are

$$3^1 = 3, \quad 3^2 = 2, \quad 3^3 = 6, \quad 3^4 = 4, \quad 3^5 = 5, \quad 3^6 = 1,$$

which run through all of $\{1, \dots, 6\}$: the element 3 is a primitive root of $\mathbb{F}_7$. By contrast 2 is not: its powers are 2, 4, 1, so $\text{ord}(2) = 3 < 6$. The complete set of primitive roots consists of the powers 3^j with $\gcd(j, 6) = 1$ (Corollary 3.23), namely $j = 1$ and $j = 5$: the primitive roots of $\mathbb{F}_7$ are $3^1 = 3$ and $3^5 = 5$, and there are $\phi(6) = 2$ of them, as the census predicts.

Cyclicity also determines the number of solutions of the equation $x^n = 1$ in a finite field, on which the evaluation domains of later sections depend.

Corollary 6.22 (Roots of unity in $\mathbb{F}_q$). *Let $\mathbb{F}_q$ be a finite field and $n \geq 1$. The solutions of $x^n = 1$ in $\mathbb{F}_q^\times$ number exactly $\gcd(n, q - 1)$, and they form the cyclic subgroup of $\mathbb{F}_q^\times$ of that order. In particular, when $n \mid q - 1$ the equation has exactly n solutions.*

Proof. Fix a primitive root g (Theorem 6.17) and write each element of $\mathbb{F}_q^\times$ uniquely as $x = g^j$ with $0 \leq j \leq q - 2$ (Proposition 3.12(3)). Then $x^n = g^{jn}$, and $g^{jn} = 1$ holds if and only if $(q - 1) \mid jn$ (Proposition 3.12(1)). Set $d = \gcd(n, q - 1)$ and write $q - 1 = dm$, $n = dn'$ with $\gcd(m, n') = 1$. Then

$$(q - 1) \mid jn \iff dm \mid jdn' \iff m \mid jn' \iff m \mid j,$$

the last step by Euclid's lemma (Proposition 2.15(1), with $\gcd(m, n') = 1$). The qualifying exponents in $\{0, \dots, q - 2\}$ are therefore the multiples of $m = (q - 1)/d$, namely $j = tm$ for $t = 0, 1, \dots, d - 1$: exactly d of them. The corresponding solutions $x = (g^m)^t$ are exactly the powers of g^m , which by Corollary 3.23 has order $(q - 1)/\gcd(q - 1, m) = (q - 1)/m = d$; so the solution set is the cyclic subgroup $\langle g^m \rangle$ of order $d = \gcd(n, q - 1)$. When $n \mid q - 1$ we have $d = n$, giving exactly n solutions. $\square$

Remark 6.23 (Two-adicity and proof-system fields). The corollary makes a curve such as Pallas or Vesta suitable for Halo 2: the field size q is chosen so that $q - 1$ is divisible by a large power of 2, and the corollary then *guarantees* a multiplicative subgroup of 2-power order. That subgroup is the evaluation domain of the number-theoretic transform, on which efficient polynomial arithmetic rests. The construction of these domains and the transform itself are treated in Section 9; the curves are introduced in Section 10.

Remark 6.24 (Cyclicity and Frobenius in cryptography). The cyclicity of $\mathbb{F}_q^\times$ (Theorem 6.17) underlies the discrete-logarithm problem—every nonzero element has the form g^j , the problem is to recover j , and the presumed hardness of doing so is a foundational assumption of later volumes—and, through Corollary 6.22, guarantees the existence of the roots of unity used in polynomial commitment schemes. The Frobenius endomorphism (Definition 6.14) induces, on an elliptic curve over $\mathbb{F}_p$, an endomorphism of the curve whose characteristic polynomial $T^2 - tT + p$ encodes the number of curve points as $N = p + 1 - t$, where t is the Hasse trace; the order of the cryptographic group is thus determined by the field-theoretic Frobenius. Both connections are taken up in Section 10.

7 Number theory for cryptography

This section treats the number theory behind two requirements of elliptic-curve cryptography. A point (x, y) of a curve $y^2 = x^3 + ax + b$ (Section 10) is determined by x up to the sign of y , so wire formats send x with one bit selecting the sign, halving the transmission; decompression must decide whether $x^3 + ax + b$ is a square modulo p and, if it is, *compute* a square root, in the time of a few modular exponentiations on numbers hundreds of bits long. Elliptic-curve cryptography also publishes powers g^x of a known group element while the exponent x is a secret key, so exponentiation must be efficient and its inversion infeasible. The section proves the Chinese Remainder Theorem and Euler's criterion, gives square-root algorithms for $p \equiv 3 \pmod{4}$ (Proposition 7.17) and in general (Theorem 7.19), and proves the Pohlig–Hellman reduction of a discrete logarithm to discrete logarithms in subgroups of prime order, whence the prime group orders of Halo 2 and Orchard (Corollary 7.24).

Throughout, n and m denote positive integers and p denotes a prime. Statements proved in Section 2, Section 3 and Section 6 (the cyclicity theorem) reappear in arithmetic form, with proofs by citation.

7.1 Euler's totient function and the Chinese Remainder Theorem

A class $[a]_n \in \mathbb{Z}/n\mathbb{Z}$ is a unit if and only if $\gcd(a, n) = 1$ (Theorem 2.24), the units form a finite abelian group $(\mathbb{Z}/n\mathbb{Z})^\times$ (Proposition 3.8), and Euler's totient $\varphi(n)$ counts them: $\varphi(n) = |(\mathbb{Z}/n\mathbb{Z})^\times|$ (Definition 2.26). In particular $\varphi(p) = p - 1$ for a prime p (Example 2.27), so $\mathbb{F}_p^\times$ has order $p - 1$. Through the orders of elliptic-curve groups, the totient enters the parameter choices of Halo 2 and Orchard. Since Section 2 computes $\varphi(n)$ only by enumeration, this subsection derives a formula from its values on prime powers and its multiplicativity on coprime factors.

Proposition 7.1 (Totient of a prime power). *For a prime p and an integer $k \geq 1$,*

$$\varphi(p^k) = p^k - p^{k-1} = p^k \left(1 - \frac{1}{p}\right).$$

Proof. Among $1, \dots, p^k$, we claim the integers *not* coprime to p^k are exactly the multiples of p . If $p \mid a$ then $p \mid \gcd(a, p^k)$, so the gcd exceeds 1. Conversely, if $g = \gcd(a, p^k) > 1$, then g has a prime factor ℓ ; from $\ell \mid g \mid p^k$ and Euclid's lemma for primes (Lemma 2.17) we get $\ell = p$, and $\ell \mid g \mid a$ gives $p \mid a$. The multiples of p in the range are $p, 2p, \dots, p^{k-1} \cdot p$, of which there are p^{k-1} ; subtracting them from the p^k integers in the range leaves $\varphi(p^k) = p^k - p^{k-1}$. $\square$

Multiplicativity follows from the Chinese Remainder Theorem, which the discrete-logarithm subsection uses again. The theorem is stated in the language of rings (Section 4), for the product ring $R \times S$ with componentwise operations (Example 4.5); a bijective ring homomorphism (Definition 4.21) is a *ring isomorphism*, as for groups (Definition 3.41).

Theorem 7.2 (Chinese Remainder Theorem). *If $\gcd(m, n) = 1$, then the map*

$$\mathbb{Z}/mn\mathbb{Z} \longrightarrow \mathbb{Z}/m\mathbb{Z} \times \mathbb{Z}/n\mathbb{Z}, \quad [a]_{mn} \longmapsto ([a]_m, [a]_n)$$

is a ring isomorphism. It restricts to a group isomorphism

$$(\mathbb{Z}/mn\mathbb{Z})^\times \cong (\mathbb{Z}/m\mathbb{Z})^\times \times (\mathbb{Z}/n\mathbb{Z})^\times.$$

Proof. The map is well defined: if $[a]_{mn} = [a']_{mn}$ then $mn \mid a - a'$, hence $m \mid a - a'$ and $n \mid a - a'$, so the two components agree. It is a ring homomorphism because each component is: reduction modulo m (and modulo n) sends sums to sums, products to products, and 1 to 1 (Theorem 4.6), and the product ring's operations are componentwise.

For injectivity it suffices, since the map is in particular a homomorphism of additive groups, to check that the kernel is trivial (Proposition 3.40). Suppose $[a]_m = [0]_m$ and $[a]_n = [0]_n$, that is, $m \mid a$ and $n \mid a$. Since $\gcd(m, n) = 1$, Euclid's lemma (Proposition 2.15(2)) gives $mn \mid a$, i.e. $[a]_{mn} = [0]_{mn}$.

Both sides have exactly mn elements (Theorem 4.6 for each factor), and an injective map between finite sets of equal size is bijective: the mn distinct inputs have mn distinct images, which must exhaust the codomain. Hence the map is a ring isomorphism.

For the multiplicative statement, a ring isomorphism ϕ matches units with units: if $uv = 1$ then $\phi(u)\phi(v) = \phi(1) = 1$, and the same applies to ϕ^{-1} . The units of the product ring are exactly the pairs of units, $(u, v)^{-1} = (u^{-1}, v^{-1})$ existing precisely when both components are invertible. Restricting ϕ to unit groups therefore gives a bijection $(\mathbb{Z}/mn\mathbb{Z})^\times \rightarrow (\mathbb{Z}/m\mathbb{Z})^\times \times (\mathbb{Z}/n\mathbb{Z})^\times$ that respects multiplication: a group isomorphism. $\square$

Corollary 7.3 (Multiplicativity and the product formula). *If $\gcd(m, n) = 1$ then $\phi(mn) = \phi(m)\phi(n)$. Hence if $n = \prod_i p_i^{k_i}$ is the prime factorisation of n , then*

$$\phi(n) = \prod_i (p_i^{k_i} - p_i^{k_i-1}) = n \prod_{p|n} \left(1 - \frac{1}{p}\right),$$

the last product running over the distinct primes dividing n .

Proof. The unit-group isomorphism of Theorem 7.2 is in particular a bijection, and the cardinality of a product of two sets is the product of the cardinalities, so $\phi(mn) = \phi(m)\phi(n)$.

For the product formula, the prime powers $p_i^{k_i}$ are pairwise coprime: a common divisor exceeding 1 of two of them would have a prime factor dividing both, which by Euclid's lemma for primes (Lemma 2.17) would equal both p_i and p_j . Moreover each $p_i^{k_i}$ is coprime to the product of the others, by repeated application of Proposition 2.15(3). Induction on the number of factors therefore extends multiplicativity to $\phi(n) = \prod_i \phi(p_i^{k_i})$, and Proposition 7.1 evaluates each factor:

$$\phi(n) = \prod_i (p_i^{k_i} - p_i^{k_i-1}) = \prod_i p_i^{k_i} \left(1 - \frac{1}{p_i}\right) = n \prod_{p|n} \left(1 - \frac{1}{p}\right). \quad \square$$

For example, the product formula gives $\phi(12) = 12\left(1 - \frac{1}{2}\right)\left(1 - \frac{1}{3}\right) = 4$, in agreement with the direct enumeration $\{1, 5, 7, 11\}$ of Example 2.27. Gauss's divisor-sum identity $\sum_{d|n} \phi(d) = n$ is Lemma 6.16.

7.2 Fermat's little theorem and Euler's theorem

Both congruences are Corollary 3.32, proved from Lagrange's theorem.

Theorem 7.4 (Euler's theorem). *If $\gcd(a, n) = 1$, then*

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

Proof. The hypothesis says that $[a]_n$ is a unit (Theorem 2.24), i.e. an element of the finite group $(\mathbb{Z}/n\mathbb{Z})^\times$, whose order is $\varphi(n)$ (Definition 2.26). By Corollary 3.31—every element g of a finite group of order N satisfies $g^N = e$, a consequence of Lagrange's theorem—we get $[a]_n^{\varphi(n)} = [1]_n$, which is the stated congruence. This is Corollary 3.32 verbatim. $\square$

Corollary 7.5 (Fermat's little theorem). *Let p be prime. If $p \nmid a$, then $a^{p-1} \equiv 1 \pmod{p}$. Moreover $a^p \equiv a \pmod{p}$ for every integer a .*

Proof. For $p \nmid a$ we have $\gcd(a, p) = 1$, so Theorem 7.4 with $n = p$ and $\varphi(p) = p - 1$ (Example 2.27) gives the first congruence; multiplying it by a gives $a^p \equiv a \pmod{p}$. When $p \mid a$ both a^p and a are congruent to 0, so the second congruence holds for all a . $\square$

Remark 7.6 (Cryptographic use of Euler and Fermat). Euler's theorem underlies RSA: if $ed \equiv 1 \pmod{\varphi(n)}$, say $ed = 1 + k\varphi(n)$, then

$$(a^e)^d = a^{1+k\varphi(n)} = a \cdot (a^{\varphi(n)})^k \equiv a \pmod{n}$$

whenever $\gcd(a, n) = 1$, so e can serve as a public encryption exponent and d as its private inverse. Orchard's elliptic-curve cryptography instead uses Fermat's little theorem in the field $\mathbb{F}_p$: every nonzero x satisfies $x^{p-1} = 1$, so inversion is computable as $x^{-1} = x^{p-2}$ (Construction 6.4).

7.3 Multiplicative order and primitive roots

Definition 7.7 (Multiplicative order). Let $\gcd(a, n) = 1$. The *multiplicative order* of a modulo n , written $\text{ord}_n(a)$, is the least positive integer k with $a^k \equiv 1 \pmod{n}$. Such a k exists because $a^{\varphi(n)} \equiv 1$ by Theorem 7.4.

The quantity $\text{ord}_n(a)$ is exactly the order of $[a]_n$ as an element of the group $(\mathbb{Z}/n\mathbb{Z})^\times$, so the order theory of Section 3 applies verbatim: $a^m \equiv 1 \pmod{n}$ if and only if $\text{ord}_n(a) \mid m$ (Proposition 3.12(1))—in particular $\text{ord}_n(a) \mid \varphi(n)$, by Euler's theorem—and

$$\text{ord}_n(a^j) = \frac{\text{ord}_n(a)}{\gcd(j, \text{ord}_n(a))}$$

(Corollary 3.23).

Definition 7.8 (Primitive root). A *primitive root* modulo n is an element g with $\text{ord}_n(g) = \varphi(n)$; equivalently, g generates $(\mathbb{Z}/n\mathbb{Z})^\times$ as a cyclic group, so that $\{g^0, g^1, \dots, g^{\varphi(n)-1}\}$ lists all the units.

Primitive roots need not exist for every modulus. The unit group $(\mathbb{Z}/8\mathbb{Z})^\times = \{1, 3, 5, 7\}$ has order $\varphi(8) = 4$, yet every one of its elements squares to 1 (Example 3.22): the element orders are 1 for the identity and 2 for each of 3, 5, 7, so no element has order 4 and no primitive root modulo 8 exists. The unit group is isomorphic to the Klein four-group of Example 3.10: every element is its own inverse, and the product of two distinct non-identity elements a and b is neither e (which would force $b = a^{-1} = a$) nor a nor b (which would force $b = e$ or $a = e$), so it is the third. The case that cryptography requires is $n = p$ prime, where the multiplicative group is always cyclic.

Corollary 7.9 ($\mathbb{F}_p^\times$ is cyclic). For every prime p , the group $\mathbb{F}_p^\times = (\mathbb{Z}/p\mathbb{Z})^\times$ is cyclic of order $p - 1$. In particular primitive roots modulo p exist, and there are exactly $\varphi(p - 1)$ of them; Example 6.21 exhibits the two primitive roots 3 and 5 of $\mathbb{F}_7$.

Proof. The group $\mathbb{F}_p^\times$ is a finite subgroup, of order $p - 1$, of the multiplicative group of the field $\mathbb{F}_p$, hence cyclic by Theorem 6.17. A generator g has order $p - 1$, and the powers g^k that again generate are those with $\gcd(k, p - 1) = 1$ (Corollary 3.23), numbering $\varphi(p - 1)$. $\square$

Remark 7.10 (Cyclicity and prime-order groups). Cyclicity is the reason the prime-order groups of elliptic-curve cryptography behave exactly like $\mathbb{Z}/q\mathbb{Z}$ under addition once a generator is fixed: a cyclic group of order q is isomorphic to $(\mathbb{Z}/q\mathbb{Z}, +)$ via $g^k \leftrightarrow k$ (Theorem 3.43). The full classification, stated without proof and not used later, is that primitive roots modulo n exist exactly when $n \in \{1, 2, 4, p^k, 2p^k\}$ for an odd prime p .

7.4 Quadratic residues and the Euler criterion

The squares modulo a prime decide whether a curve point with a given x -coordinate exists, the first decompression task.

Definition 7.11 (Quadratic residue). Let p be an odd prime and a an integer with $p \nmid a$. We say a is a *quadratic residue* modulo p (a QR) if the congruence $x^2 \equiv a \pmod{p}$ has a solution, and a *quadratic non-residue* (a QNR) otherwise.

Proposition 7.12 (Exactly half are residues). For an odd prime p , exactly $\frac{p-1}{2}$ of the nonzero residues modulo p are quadratic residues and $\frac{p-1}{2}$ are non-residues. The squaring map $x \mapsto x^2$ is exactly two-to-one on $\mathbb{F}_p^\times$.

Proof. Consider $\sigma: \mathbb{F}_p^\times \rightarrow \mathbb{F}_p^\times$, $\sigma(x) = x^2$; its image is exactly the set of quadratic residues. For $x, y \in \mathbb{F}_p^\times$,

$$\sigma(x) = \sigma(y) \iff x^2 - y^2 = 0 \iff (x - y)(x + y) = 0 \iff y = \pm x,$$

the middle equivalence because a field has no zero divisors (Proposition 4.28). Since p is odd, $x \neq -x$ for $x \neq 0$ (otherwise $2x = 0$ with 2 and x nonzero), so every fibre of σ over its image has exactly

two elements. Counting, $|\text{im } \sigma| = \frac{p-1}{2}$, and the complement—the non-residues—has the same size. $\square$

Theorem 7.13 (Euler’s criterion). *Let p be an odd prime and $p \nmid a$. Then*

$$a^{\frac{p-1}{2}} \equiv \begin{cases} 1 & (\text{mod } p) \quad \text{if } a \text{ is a quadratic residue,} \\ -1 & (\text{mod } p) \quad \text{if } a \text{ is a quadratic non-residue.} \end{cases}$$

Proof. First, the value is always ± 1 . Fermat’s little theorem (Corollary 7.5) gives $(a^{(p-1)/2})^2 = a^{p-1} \equiv 1$, and the only square roots of 1 modulo a prime are ± 1 : from $x^2 \equiv 1$ we get $p \mid (x-1)(x+1)$, so $p \mid x-1$ or $p \mid x+1$ by Euclid’s lemma for primes (Lemma 2.17), i.e. $x \equiv \pm 1$. Hence $a^{(p-1)/2} \equiv \pm 1 \pmod{p}$.

If a is a residue, say $a \equiv x^2$ with $p \nmid x$, then $a^{(p-1)/2} \equiv x^{p-1} \equiv 1$, again by Fermat. Thus every one of the $\frac{p-1}{2}$ quadratic residues (Proposition 7.12) is a root of the polynomial $t^{(p-1)/2} - 1$ over $\mathbb{F}_p$. That polynomial has degree $\frac{p-1}{2}$, so it has *at most* $\frac{p-1}{2}$ roots (Theorem 5.18); the residues therefore account for all of them. A non-residue a is consequently not a root, yet still satisfies $a^{(p-1)/2} \equiv \pm 1$; the remaining possibility is $a^{(p-1)/2} \equiv -1$. $\square$

Definition 7.14 (Legendre symbol). For an odd prime p and an integer a , the *Legendre symbol* is

$$\left(\frac{a}{p}\right) = \begin{cases} 0 & \text{if } p \mid a, \\ 1 & \text{if } a \text{ is a quadratic residue modulo } p, \\ -1 & \text{if } a \text{ is a quadratic non-residue modulo } p. \end{cases}$$

Euler’s criterion (Theorem 7.13) then reads, uniformly,

$$\left(\frac{a}{p}\right) \equiv a^{\frac{p-1}{2}} \pmod{p},$$

so the symbol is computable by a single modular exponentiation, at a cost of $O(\log p)$ modular multiplications by square-and-multiply (Construction 6.2). The criterion also shows that the symbol is *completely multiplicative* in a :

$$\left(\frac{ab}{p}\right) = \left(\frac{a}{p}\right) \left(\frac{b}{p}\right), \quad \text{since} \quad (ab)^{\frac{p-1}{2}} = a^{\frac{p-1}{2}} b^{\frac{p-1}{2}}.$$

Example 7.15 (13 is a non-square in both Pasta fields). Orchard uses the Pasta hash-to-curve construction, a procedure, treated in later volumes, that maps arbitrary byte strings to curve points and requires a fixed non-square constant in the base field. The construction fixes $Z = -13$ as a verifiably non-square constant in both base fields. Let p and q be the Pasta primes of Section 10; both satisfy $p \equiv q \equiv 1 \pmod{4}$. The verification below works modulo p ; the computation modulo q is identical.

First, -1 is a square modulo any prime $\equiv 1 \pmod{4}$: the exponent $\frac{p-1}{2}$ is even, so Euler's criterion gives

$$\left(\frac{-1}{p}\right) \equiv (-1)^{\frac{p-1}{2}} = 1.$$

By complete multiplicativity,

$$\left(\frac{-13}{p}\right) = \left(\frac{-1}{p}\right) \left(\frac{13}{p}\right) = \left(\frac{13}{p}\right),$$

so -13 is a non-square precisely when 13 is. Whether 13 is a square is settled by one Euler-criterion exponentiation in each field, a computation any computer-algebra system reproduces in milliseconds:

$$13^{\frac{p-1}{2}} \equiv p-1 \equiv -1 \pmod{p}, \quad 13^{\frac{q-1}{2}} \equiv q-1 \equiv -1 \pmod{q},$$

whence $\left(\frac{13}{p}\right) = \left(\frac{13}{q}\right) = -1$. Thus 13 , and with it $Z = -13$, is a quadratic non-residue modulo both Pasta primes, as the hash-to-curve construction requires.

Example 7.16 (5 is a non-square in the Pallas base field). The same test decides a second constant on which later volumes rely, in this case the constant term of the curve equation. Pallas is the curve $y^2 = x^3 + 5$ over $\mathbb{F}_p$ (Section 10), and the arithmetic circuits of Halo 2 encode its point at infinity as the coordinate pair $(0, 0)$ (Construction 10.13). The encoding is unambiguous only if no genuine point has x -coordinate 0 , i.e. only if $y^2 = 5$ has no solution in $\mathbb{F}_p$: the question is whether 5 is a square in the *base* field $\mathbb{F}_p$ of Pallas, not in its scalar field $\mathbb{F}_q$. Euler's criterion decides it in one exponentiation:

$$5^{\frac{p-1}{2}} \equiv p-1 \equiv -1 \pmod{p},$$

so $\left(\frac{5}{p}\right) = -1$. The constant 5 is a quadratic non-residue modulo the Pallas base-field prime, and no point of Pallas has $x = 0$. (The same computation modulo q gives $5^{(q-1)/2} \equiv -1 \pmod{q}$, so Vesta has no such point either.) That no point has $y = 0$ is a different fact— 5 is not a cube in $\mathbb{F}_p$ —which Remark 10.26 reads off from the oddness of the group order; the two together are the hypotheses of Proposition 10.14.

7.5 Square roots modulo a prime

Decompression must also *produce* an x with $x^2 \equiv a$ for a residue a , recovering y from y^2 ; the cost depends on $p \pmod{4}$.

Proposition 7.17 (The $p \equiv 3 \pmod{4}$ shortcut). *Let $p \equiv 3 \pmod{4}$ be prime and let a be a quadratic residue modulo p with $p \nmid a$. Then*

$$x \equiv a^{\frac{p+1}{4}} \pmod{p}$$

is a square root of a , and the two square roots of a are $\pm x$.

Proof. Since $p \equiv 3 \pmod{4}$, the quantity $\frac{p+1}{4}$ is a positive integer. Compute

$$x^2 = a^{\frac{p+1}{2}} = a^{\frac{p-1}{2}} \cdot a \equiv \left(\frac{a}{p}\right) \cdot a = a \pmod{p},$$

using Euler's criterion (Theorem 7.13) and the hypothesis that a is a residue, so $\left(\frac{a}{p}\right) = 1$. The roots come in the pair $\pm x$ by the two-to-one property of squaring (Proposition 7.12). $\square$

Remark 7.18 (Cost and the Pasta primes). Primes $p \equiv 3 \pmod{4}$ are often favoured for this single-exponentiation shortcut. For $p \equiv 5 \pmod{8}$ a closed-form variant (Atkin's formula) exists. In general the cost of square-root extraction grows with the 2-adic valuation s of $p-1$: writing $p-1 = 2^s d$ with d odd, the Tonelli–Shanks algorithm below performs up to s corrective iterations. The Pasta primes have $s = 32$, chosen for FFT-friendliness (Remark 9.10), so their implementations use Tonelli–Shanks.

For $p \equiv 1 \pmod{4}$, write $p-1 = 2^s d$ with d odd and $s \geq 2$. The algorithm works in the subgroup of $\mathbb{F}_p^\times$ of order 2^s , which contains the obstruction to the single-exponentiation shortcut, and removes that obstruction one power of two at a time. The subgroup is unique since $\mathbb{F}_p^\times$ is cyclic (Corollary 7.9 and Theorem 3.24(3)).

Theorem 7.19 (Tonelli–Shanks). *Let p be an odd prime, $p-1 = 2^s d$ with d odd, and let a be a quadratic residue with $p \nmid a$. Given any quadratic non-residue z modulo p , one can compute a square root of a modulo p using $O(s^2 + \log p)$ modular multiplications.*

Proof. Write $H \leq \mathbb{F}_p^\times$ for the unique cyclic subgroup of order 2^s . Set $c = z^d$. We claim c generates H . By Euler's criterion (Theorem 7.13) $z^{(p-1)/2} \equiv -1$, so

$$c^{2^{s-1}} = z^{2^{s-1}d} = z^{\frac{p-1}{2}} \equiv -1 \neq 1, \quad c^{2^s} = z^{p-1} \equiv 1$$

by Fermat (Corollary 7.5); hence $\text{ord}_p(c)$ divides 2^s but not 2^{s-1} , forcing $\text{ord}_p(c) = 2^s$ and $\langle c \rangle = H$.

Initialise

$$x = a^{\frac{d+1}{2}}, \quad t = a^d, \quad m = s$$

(the exponent $\frac{d+1}{2}$ is an integer because d is odd). We maintain three invariants:

1. $x^2 = a t$;
2. the current c has order exactly 2^m ;
3. the order of t divides 2^{m-1} .

Initially (1) holds since $x^2 = a^{d+1} = a \cdot a^d$; (2) was proved above; and (3) holds because $t^{2^{s-1}} = a^{(p-1)/2} \equiv 1$ by Euler's criterion (Theorem 7.13), a being a residue.

Each iteration proceeds as follows. If $t = 1$, then $x^2 = a$ by (1) and the algorithm terminates. Otherwise let $2^i = \text{ord}_p(t)$, found as the least $i \geq 1$ with $t^{2^i} = 1$ by repeatedly squaring t ; invariant (3) gives $1 \leq i \leq m-1$. Set

$$b = c^{2^{m-i-1}},$$

and update

$$x \leftarrow xb, \quad t \leftarrow tb^2, \quad c \leftarrow b^2, \quad m \leftarrow i.$$

The invariants are preserved. For (1), $(xb)^2 = x^2 b^2 = a t b^2$, which is a times the new t . For (2), the old c has order 2^m , so $\text{ord}_p(b) = 2^m / \gcd(2^m, 2^{m-i-1}) = 2^{i+1}$ (Corollary 3.23), and the new

$c = b^2$ has order 2^i —exactly 2 to the new m . For (3), both the old t and b^2 have order exactly 2^i , so $t^{2^{i-1}}$ and $(b^2)^{2^{i-1}}$ each have order 2; the only element of order 2 in $\mathbb{F}_p^\times$ is -1 , because the square roots of 1 are ± 1 (as in the proof of Theorem 7.13); hence

$$(t b^2)^{2^{i-1}} = t^{2^{i-1}} (b^2)^{2^{i-1}} = (-1)(-1) = 1,$$

so the new t has order dividing 2^{i-1} , which is 2^{m-1} for the new m .

Each iteration strictly decreases m , from s through a descending chain of nonnegative integers, so after at most s iterations the order of t reaches $2^0 = 1$, i.e. $t = 1$, and x is the desired root. For the cost: locating i and computing b each take at most $m \leq s$ squarings, so the loop costs $O(s)$ multiplications per iteration and $O(s^2)$ in total; the three initial exponentiations, with exponents smaller than p , cost $O(\log p)$ multiplications each by square-and-multiply (Construction 6.2). $\square$

Finding the non-residue z is the only randomised step in this section. A uniformly random element of $\mathbb{F}_p^\times$ is a non-residue with probability exactly $\frac{1}{2}$ (Proposition 7.12), and each candidate is checked by one Euler-criterion exponentiation (Definition 7.14), i.e. $O(\log p)$ modular multiplications, so a few random trials find z with high probability. Implementations hard-code one non-residue per field, verified once, as Example 7.15 verifies $Z = -13$. This volume uses no deterministic polynomial-time method for finding a non-residue.

Remark 7.20 (Which root, and detecting failure). The square roots of a residue come in a pair $\pm x$ (Proposition 7.12), so a *canonical* root is fixed by convention—for instance “choose the root whose least nonnegative representative is even”, as elliptic-curve point encodings do; the compressed point’s extra bit selects between the two. Both algorithms detect a non-residue a . The shortcut of Proposition 7.17 still outputs a candidate $a^{(p+1)/4}$, which fails the check $x^2 \equiv a$ at the cost of one squaring. The variable-time Tonelli–Shanks algorithm cannot complete its update: Euler’s criterion gives $a^{(p-1)/2} \equiv -1$, so $t = a^d$ satisfies $t^{2^{i-1}} \equiv -1$ and has order exactly 2^i , and the first search for the least $i \geq 1$ with $t^{2^i} = 1$ already returns $i = m$, which invariant (3) of the proof of Theorem 7.19 forbids and for which the update exponent 2^{m-i-1} is undefined. The algorithm can therefore report “not a residue” as soon as $i = m$ occurs, or verify $a^{(p-1)/2} \equiv 1$ up front, at the same $O(\log p)$ cost as the initial exponentiations.

7.6 The discrete logarithm problem

The statements below concern a cyclic group written multiplicatively. In Orchard the group is the group of points of an elliptic curve, of large prime order and written additively; every statement transfers by renaming the operation.

Definition 7.21 (Discrete logarithm problem). Let $G = \langle g \rangle$ be a finite cyclic group of order N with generator g . Given $h \in G$, the *discrete logarithm* of h to base g , written $\log_g h$, is the unique residue $x \in \mathbb{Z}/N\mathbb{Z}$ with $g^x = h$. The *discrete logarithm problem* (DLP) is to compute x from (G, g, h) .

Existence and uniqueness of $\log_g h$ are the content of the classification of cyclic groups: the map $x \mapsto g^x$ is a group isomorphism $(\mathbb{Z}/N\mathbb{Z}, +) \xrightarrow{\sim} G$ (Theorem 3.43, whose map $\tilde{\phi}$ this is), so it

has a well-defined inverse, and $\log_g$ is that inverse. Computing g^x from x takes $O(\log N)$ group operations by square-and-multiply (Construction 6.2); computing x from g^x is the DLP, for which no efficient algorithm is known in well-chosen groups. This asymmetry is the one-wayness on which elliptic-curve cryptography rests.

Remark 7.22 (Generic complexity and group choice). Two *generic* algorithms solve the DLP in any group of order N , using only the group operation. *Baby-step giant-step* is deterministic: set $m = \lceil \sqrt{N} \rceil$ and write the unknown as $x = im + j$ with $0 \leq i, j < m$; tabulate the “baby steps” g^j for all j , then walk the “giant steps” $h(g^{-m})^i$ for $i = 0, 1, 2, \dots$ until one equals a tabulated g^j , at which point $hg^{-im} = g^j$ gives $x = im + j$. Both the table and the walk are $O(\sqrt{N})$, so the cost is $O(\sqrt{N})$ time and space. *Pollard’s rho* (not developed here) is a randomised alternative with the same $O(\sqrt{N})$ expected running time—on a heuristic analysis, made explicit in Section 10—but only $O(1)$ space, and is the algorithm used in practical attacks. Conversely, Shoup’s theorem shows that $\Omega(\sqrt{N})$ group operations are *necessary* for any generic algorithm when N is prime, so for generic attacks the square-root cost is exact. Reaching a λ -bit security level—no attack cheaper than 2^λ operations—therefore requires $N \approx 2^{2\lambda}$. This is why the Pallas and Vesta scalar fields have prime order roughly 255 bits long, targeting roughly 128-bit security (Section 10). These costs hold for *generic* algorithms: in $(\mathbb{Z}/p\mathbb{Z})^\times$, sub-exponential index-calculus attacks exploit the ring structure of the integers behind the group. No such attack is known for elliptic-curve groups, which therefore permit much smaller parameters than multiplicative groups of comparable security.

The next theorem makes “well-chosen” precise.

Theorem 7.23 (Pohlig–Hellman reduction). *Let $G = \langle g \rangle$ be cyclic of order $N = \prod_{i=1}^r q_i^{e_i}$, the q_i distinct primes. The discrete logarithm in G is computable by solving discrete logarithms in subgroups of prime order q_i — e_i of them for each i —and combining the answers with the Chinese Remainder Theorem, at a total cost of*

$$O\left(\sum_{i=1}^r e_i (\log N + \sqrt{q_i})\right)$$

group operations. When N has a large prime factor $q_{\max}$ the $\sqrt{q_{\max}}$ term dominates; when N is smooth—a product of primes bounded polynomially in $\log N$ —every term is polynomial in $\log N$, and hence so is the cost of the DLP. Corollary 7.24 draws the design consequence.

Proof. Let $h = g^x$ with x the unknown. The reduction has two stages.

Stage (a): splitting across coprime factors. For each i set $N_i = N/q_i^{e_i}$ and project:

$$g_i = g^{N_i}, \quad h_i = h^{N_i}.$$

By Corollary 3.23, $\text{ord}(g_i) = N / \gcd(N, N_i) = N/N_i = q_i^{e_i}$, so g_i generates the unique subgroup of G of order $q_i^{e_i}$ (Theorem 3.24(3)). Moreover $h_i = g^{xN_i} = g_i^x$, and since g_i has order $q_i^{e_i}$, the exponent matters only modulo $q_i^{e_i}$ (Proposition 3.12(2)): writing $x_i = x \bmod q_i^{e_i}$,

$$\log_{g_i} h_i = x_i.$$

Solving the r smaller DLPs yields $x \bmod q_i^{e_i}$ for every i ; since the prime powers $q_i^{e_i}$ are pairwise coprime, the Chinese Remainder Theorem (Theorem 7.2, iterated across the factors as in the proof of Corollary 7.3) determines x modulo N uniquely. Each projection costs $O(\log N)$ operations by square-and-multiply.

Stage (b): lifting a prime power digit by digit. Fix one prime power q^e (dropping the subscript i), with $\gamma = g_i$ of order q^e and $\eta = b_i = \gamma^{x'}$, $x' = x_i$. Write x' in base q :

$$x' \equiv x_0 + x_1 q + \dots + x_{e-1} q^{e-1} \pmod{q^e}, \quad 0 \leq x_j < q.$$

The element $\delta = \gamma^{q^{e-1}}$ has order q (Corollary 3.23 again). Raise η to the power q^{e-1} : every digit beyond the first is multiplied by a multiple of q^e in the exponent and vanishes, leaving

$$\eta^{q^{e-1}} = \gamma^{x' q^{e-1}} = \delta^{x_0},$$

a single DLP in the group $\langle \delta \rangle$ of prime order q , which baby-step giant-step (Remark 7.22) solves in $O(\sqrt{q})$ operations. Once $x_0, \dots, x_{j-1}$ are known, they are removed and the step is repeated: the element $\eta_j = \gamma^{-(x_0 + x_1 q + \dots + x_{j-1} q^{j-1})}$ equals γ raised to $x_j q^j$ (multiples of q^{j+1}), so

$$\eta_j^{q^{e-1-j}} = \delta^{x_j},$$

and digit x_j is again one order- q DLP. Each of the e digits costs one such DLP plus $O(\log N)$ operations for the exponentiations.

Summing over the digits of all the prime-power factors gives $\sum_i e_i$ order- q_i DLPs at $O(\sqrt{q_i})$ apiece plus $O(\log N)$ bookkeeping for each, which is the stated bound. $\square$

Corollary 7.24 (Why prime order matters). *If N has a small prime factor q , an attacker can recover the discrete logarithm modulo q in $O(\log N + \sqrt{q})$ group operations— $O(\log N)$ to project into the order- q subgroup and $O(\sqrt{q})$ for the small DLP there—leaking part of the secret x . Security therefore requires N to have a large prime factor; the safest and simplest choice is to make N itself a large prime, so that G has no proper nontrivial subgroups and the Pohlig–Hellman reduction yields no smaller problem. For this reason the elliptic-curve groups used in Halo 2 and Orchard have large prime order.*

Proof. For the first claim, apply stage (a) of Theorem 7.23 to the single factor q : the projections $g^{N/q}, b^{N/q}$ cost $O(\log N)$ operations, and the resulting DLP in a group of order q costs $O(\sqrt{q})$ by baby-step giant-step—yielding $x \bmod q$. For the second, if N is prime then G has no subgroups besides $\{e\}$ and G (Corollary 3.34), so the reduction’s only output is the original problem, and the generic $\Omega(\sqrt{N})$ lower bound of Remark 7.22 applies in full. $\square$

Remark 7.25 (Cofactors and small-subgroup attacks). In practice an elliptic curve over $\mathbb{F}_p$ has group order $h \cdot q$, where q is a large prime and h is a small *cofactor*, and cryptographic operations stay in the subgroup of prime order q . A protocol that does not check that incoming points lie in that subgroup (or clear the cofactor) admits a practical *small-subgroup attack*: an adversary supplies a point of low order and runs Pohlig–Hellman in the small factor to extract information about the secret modulo h . The Pasta curves of Halo 2 have cofactor $h = 1$: their group orders are prime (the primes recorded in Section 10), which eliminates this class of attacks in the group itself rather than by protocol-level checks.

A group of large prime order q has the field $\mathbb{Z}/q\mathbb{Z}$ as its scalars. The curves, and the primes p and q taken as given here, are the subject of Section 10.

8 Linear algebra over a field

This section develops linear algebra from the axioms, assuming only the notion of a field (Section 4): *vector spaces*, whose elements can be added and scaled by elements of a field; bases and dimension (Theorem 8.17); linear maps and rank–nullity (Theorem 8.19); bilinear forms and the standard inner product on the coordinate space F^n , in practice over a finite field $\mathbb{F}_q$; and the mixed inner product of a vector of scalars with a vector of group elements. For fixed public elements $G_1, \dots, G_n$ of a suitable group, the mixed inner product $[a_1]G_1 + \dots + [a_n]G_n$ is the Pedersen-style *commitment* of Halo 2 and Orchard to a list $(a_1, \dots, a_n)$ of secrets, such as the *wire values* (intermediate values) of an *arithmetic circuit*, with n possibly in the millions: a single group element, independent of n , that computationally determines the list, can later be opened or reasoned about, and is additively homomorphic. Remark 8.29 derives the homomorphic property and the collisions that this compression necessarily introduces. Commitments of other kinds, such as hash-based ones, need not be linear. The next section applies these results to the polynomial spaces $F[X]_{<n}$ over structured evaluation domains.

The section also discharges two deferrals. Remark 5.5 left to it the linear structure of $F[X]$, with the monomials as an infinite coordinate system. The proof of Theorem 6.9 used two facts without proof: a space with a finite spanning set has a basis, proved in Theorem 8.17, and coordinates in a basis are unique, which follows from linear independence (Definition 8.14). Neither argument uses finite fields, so no circularity arises. Throughout, F denotes an arbitrary field with identities 0 and 1; the field axioms of Definition 4.24 are used without citation.

8.1 Vector spaces and subspaces

Definition 8.1 (Vector space). Let F be a field. A *vector space over F* (an *F -vector space*) is a set V equipped with two operations, *vector addition* $+: V \times V \rightarrow V$ and *scalar multiplication* $\cdot: F \times V \rightarrow V$, such that for all $u, v, w \in V$ and all $a, b \in F$:

1. $(V, +)$ is an abelian group: addition is associative and commutative, there is a *zero vector* $\mathbf{0} \in V$ with $v + \mathbf{0} = v$, and every v has an *additive inverse* $-v$ with $v + (-v) = \mathbf{0}$;
2. scalar multiplication is compatible with the multiplication of F : $a \cdot (b \cdot v) = (ab) \cdot v$;
3. the identity of F acts as the identity: $1 \cdot v = v$;
4. scalar multiplication distributes over vector addition: $a \cdot (u + v) = a \cdot u + a \cdot v$;
5. scalar multiplication distributes over scalar addition: $(a + b) \cdot v = a \cdot v + b \cdot v$.

The elements of V are called *vectors*, the elements of F *scalars*. We write av for $a \cdot v$, and we allow the scalar $0 \in F$ and the vector $\mathbf{0} \in V$ to share a symbol whenever context distinguishes them.

The following consequences of the axioms are used hereafter without citation.

Proposition 8.2. *Let V be an F -vector space, $v \in V$, and $a \in F$. Then:*

1. $0 \cdot v = \mathbf{0}$;
2. $a \cdot \mathbf{0} = \mathbf{0}$;
3. $(-1) \cdot v = -v$;
4. if $av = \mathbf{0}$, then $a = 0$ or $v = \mathbf{0}$.

Proof. (1) By distributivity over scalar addition, $0 \cdot v = (0 + 0) \cdot v = 0 \cdot v + 0 \cdot v$; adding $-(0 \cdot v)$ to both sides leaves $\mathbf{0} = 0 \cdot v$.

(2) Likewise $a \cdot \mathbf{0} = a \cdot (\mathbf{0} + \mathbf{0}) = a \cdot \mathbf{0} + a \cdot \mathbf{0}$, and cancelling in the abelian group $(V, +)$ gives $a \cdot \mathbf{0} = \mathbf{0}$.

(3) Using the identity axiom, distributivity, and (1),

$$v + (-1) \cdot v = 1 \cdot v + (-1) \cdot v = (1 + (-1)) \cdot v = 0 \cdot v = \mathbf{0},$$

so $(-1) \cdot v$ is the additive inverse of v ; by uniqueness of inverses in a group, $(-1) \cdot v = -v$.

(4) Suppose $av = \mathbf{0}$ and $a \neq 0$. Since F is a field, a^{-1} exists, and by the identity and compatibility axioms

$$v = 1 \cdot v = (a^{-1}a) \cdot v = a^{-1} \cdot (av) = a^{-1} \cdot \mathbf{0} = \mathbf{0},$$

the last step by (2). □

Example 8.3 (The coordinate space F^n). The principal example is the *coordinate space*

$$F^n = \{ (x_1, \dots, x_n) : x_i \in F \},$$

the set of n -tuples of scalars, under componentwise operations:

$$(x_1, \dots, x_n) + (y_1, \dots, y_n) = (x_1 + y_1, \dots, x_n + y_n), \quad a \cdot (x_1, \dots, x_n) = (ax_1, \dots, ax_n).$$

The zero vector is $(0, \dots, 0)$, and every axiom of Definition 8.1 follows from the corresponding field axiom applied in each coordinate. The degenerate case $n = 0$ is the *trivial* (or *zero*) vector space $\{\mathbf{0}\}$, consisting of the empty tuple alone. As a standing convention we write vectors of F^n in boldface and regard them as columns, $\mathbf{x} = (x_1, \dots, x_n)^T$.

Example 8.4 (Further examples). 1. *Polynomials.* The polynomials $F[X]$ form an F -vector space under addition and the scalar multiplication of Remark 5.5. So does the subset

$$F[X]_{<n} = \{ f \in F[X] : f = 0 \text{ or } \deg f < n \}$$

of polynomials of degree less than n : sums and scalar multiples of such polynomials again have degree less than n (or are zero), by Proposition 5.7, so the operations restrict and the axioms are inherited. Polynomial commitment schemes operate on this space.

2. *Functions.* For any set S , the set F^S of all functions $S \rightarrow F$ is an F -vector space under point-wise operations: $(f + g)(s) = f(s) + g(s)$ and $(af)(s) = a f(s)$. Example 8.3 is the special case $S = \{1, \dots, n\}$, a tuple being a function of its index.
3. *Matrices.* The set $F^{m \times n}$ of $m \times n$ arrays of scalars is an F -vector space under entrywise addition and scaling—the case of (2) in which S is the grid of index pairs (i, j) .
4. *Field extensions.* If F is a subfield of a larger field K (§4), then K is an F -vector space: vector addition is the addition of K , and scalar multiplication is the multiplication of K restricted to pairs in $F \times K$, so the axioms are instances of the field axioms of K . The dimension of this vector space—in the sense of Theorem 8.17 below—is called the *degree* of the extension, written $[K : F]$. For instance $\mathbb{C}$ is an $\mathbb{R}$ -vector space of dimension 2, with basis $1, i$, so $[\mathbb{C} : \mathbb{R}] = 2$; and the finite field $\mathbb{F}_q$ with $q = p^k$ is a k -dimensional $\mathbb{F}_p$ -vector space, $[\mathbb{F}_q : \mathbb{F}_p] = k$, which is the structure used in the proof of Theorem 6.9.

Definition 8.5 (Subspace). A subset $W \subseteq V$ of an F -vector space V is a (*linear*) *subspace* if W is itself a vector space under the operations inherited from V ; equivalently—as the next proposition makes precise—if and only if W is nonempty and closed under the vector-space operations.

Proposition 8.6 (Subspace criterion). *A subset $W \subseteq V$ is a subspace if and only if*

1. $\mathbf{0} \in W$;
2. W is closed under addition: $u, v \in W$ implies $u + v \in W$; and
3. W is closed under scalar multiplication: $a \in F$ and $v \in W$ imply $av \in W$.

Equivalently, W is a subspace if and only if $W \neq \emptyset$ and $au + bv \in W$ for all $a, b \in F$ and $u, v \in W$.

Proof. If W is a subspace, then (1)–(3) hold: closure (2) and (3) is what it means for the inherited operations to be operations *on* W , and the zero vector of W is the zero vector of V , since a zero vector $\mathbf{0}_W$ of W satisfies $\mathbf{0}_W + \mathbf{0}_W = \mathbf{0}_W$, and cancelling in V gives $\mathbf{0}_W = \mathbf{0}$.

Conversely, suppose (1)–(3) hold. Closure makes the two operations well defined on W . The additive inverse of any $v \in W$ is $(-1)v \in W$, by (3) and Proposition 8.2(3). Every remaining axiom of Definition 8.1 is a universally quantified identity—associativity, commutativity, compatibility, the two distributive laws, the action of 1—and each holds for all elements of V , in particular for all elements of W . Hence W is a vector space.

For the two-scalar form: if W is a subspace, then $au + bv \in W$ by (2) and (3) combined, and $W \ni \mathbf{0}$ is nonempty. Conversely, choosing $(a, b) = (1, 1)$ recovers (2), choosing $(a, b) = (a, 0)$ recovers (3) (using Proposition 8.2(1) to identify $0v$), and $\mathbf{0} = 0v \in W$ for any v in the nonempty W , which is (1). $\square$

Example 8.7. In F^3 the set $W = \{(x_1, x_2, x_3) : x_1 + x_2 + x_3 = 0\}$ is a subspace: it contains $\mathbf{0}$, and if the coordinates of $\mathbf{x}$ and $\mathbf{y}$ each sum to 0, then those of $a\mathbf{x} + b\mathbf{y}$ sum to $a \cdot 0 + b \cdot 0 = 0$. By contrast $\{(x_1, x_2, x_3) : x_1 + x_2 + x_3 = 1\}$ is *not* a subspace: it does not contain $\mathbf{0}$. The same computation

shows that the solution set of any single homogeneous linear equation $c_1x_1 + \cdots + c_nx_n = 0$ (the c_i fixed scalars) is a subspace of F^n , and the solution set of a finite *system* of such equations, being an intersection of subspaces, is a subspace by the next proposition. These solution sets are the kernels of linear maps (Definition 8.18).

Proposition 8.8 (Intersections and sums). *Let V be an F -vector space.*

1. *The intersection $\bigcap_{i \in I} W_i$ of an arbitrary family of subspaces of V is a subspace.*
2. *The sum $W_1 + \cdots + W_k = \{w_1 + \cdots + w_k : w_j \in W_j\}$ of finitely many subspaces is a subspace, and it is the smallest subspace of V containing every W_j .*

The union of two subspaces is in general not a subspace.

Proof. (1) Each W_i contains $\mathbf{0}$, so the intersection does. If u, v lie in every W_i and $a, b \in F$, then $au + bv \in W_i$ for every i by Proposition 8.6, hence $au + bv$ lies in the intersection, which is therefore a subspace by the same criterion.

(2) The sum contains $\mathbf{0} = \mathbf{0} + \cdots + \mathbf{0}$, and it contains each W_j (take every other summand $\mathbf{0}$). For closure, take two elements $\sum_j w_j$ and $\sum_j w'_j$ with $w_j, w'_j \in W_j$, and scalars a, b ; regrouping,

$$a \sum_j w_j + b \sum_j w'_j = \sum_j (aw_j + bw'_j),$$

and $aw_j + bw'_j \in W_j$ since W_j is a subspace, so the combination lies in the sum. Finally, any subspace U containing every W_j contains each summand w_j of any element of the sum, hence contains the element itself by closure under addition; so the sum is contained in U , and is the smallest such subspace.

For the union, take $V = F^2$ (any field has $|F| > 1$, since $1 \neq 0$) and the two axes $\{(x, 0) : x \in F\}$ and $\{(0, y) : y \in F\}$ —subspaces by the criterion. Their union contains $(1, 0)$ and $(0, 1)$ but not the sum $(1, 1) = (1, 0) + (0, 1)$, since $(1, 1)$ lies on neither axis; closure under addition fails. $\square$

8.2 Linear combinations, span, and linear independence

Besides being cut out by equations, a subspace can be generated by a set of vectors, as the set of all vectors obtained from them by addition and scaling.

Definition 8.9 (Linear combination and span). Let $S \subseteq V$. A *linear combination* of vectors in S is any finite sum

$$a_1v_1 + a_2v_2 + \cdots + a_kv_k, \quad a_i \in F, v_i \in S.$$

The *span* of S , written $\text{span}(S)$ or $\langle S \rangle$, is the set of all linear combinations of finitely many vectors of S ; by convention $\text{span}(\emptyset) = \{\mathbf{0}\}$. The set S *spans* (or *generates*) V if $\text{span}(S) = V$, and V is *finitely generated* (synonymously, *finite-dimensional*) if some finite set spans it.

Proposition 8.10. *For any $S \subseteq V$, the span $\text{span}(S)$ is the smallest subspace of V containing S : it is a subspace, it contains S , and it is contained in every subspace of V that contains S .*

Proof. The zero vector is the empty linear combination (or, for nonempty S , the combination $0v$), and a sum of two linear combinations of vectors in S , or a scalar multiple of one, is again such a linear combination—concatenate the lists and rescale the coefficients. By Proposition 8.6, $\text{span}(S)$ is a subspace, and it contains each $v \in S$ as the combination $1 \cdot v$. If W is any subspace containing S , then closure under addition and scaling forces every linear combination of elements of S into W , by induction on the number of summands; hence $\text{span}(S) \subseteq W$. $\square$

Definition 8.11 (Linear independence). A finite list of vectors $v_1, \dots, v_k \in V$ is *linearly independent* if the only way to express the zero vector as a linear combination of the list is with all coefficients zero:

$$a_1 v_1 + \dots + a_k v_k = \mathbf{0} \implies a_1 = \dots = a_k = 0.$$

Otherwise the list is *linearly dependent*, and a relation $\sum_i a_i v_i = \mathbf{0}$ with some $a_i \neq 0$ is called a *dependence relation*. An arbitrary—possibly infinite—set $S \subseteq V$ is linearly independent if every finite list of distinct vectors from S is linearly independent.

Remark 8.12. Linear independence expresses *non-redundancy*: a list is dependent exactly when some vector in it lies in the span of the others, which is the content of the next lemma. A single vector v is independent if and only if $v \neq \mathbf{0}$ (Proposition 8.2(4)); two vectors are dependent if and only if one is a scalar multiple of the other.

Lemma 8.13 (Dependence and redundancy). *A list $v_1, \dots, v_k$ with $k \geq 1$ is linearly dependent if and only if some v_j is a linear combination of the others. Moreover, if $v_1, \dots, v_k$ is independent but $v_1, \dots, v_k, w$ is dependent, then $w \in \text{span}(v_1, \dots, v_k)$, and the coefficients expressing w are unique.*

Proof. Suppose first that $v_j = \sum_{i \neq j} a_i v_i$. Then

$$\sum_{i \neq j} a_i v_i + (-1) v_j = \mathbf{0}$$

is a dependence relation, its coefficient -1 on v_j being nonzero (as $1 \neq 0$ in a field). Conversely, given a dependence relation $\sum_i a_i v_i = \mathbf{0}$ with $a_j \neq 0$, the field supplies a_j^{-1} , and solving for v_j gives $v_j = -a_j^{-1} \sum_{i \neq j} a_i v_i$, a linear combination of the others.

For the second statement, take a dependence relation $\sum_i a_i v_i + bw = \mathbf{0}$ for the extended list. The coefficient b cannot be 0: otherwise the relation would be a nontrivial relation among $v_1, \dots, v_k$ alone, contradicting their independence. Hence $w = -b^{-1} \sum_i a_i v_i \in \text{span}(v_1, \dots, v_k)$. For uniqueness, suppose $w = \sum_i c_i v_i = \sum_i c'_i v_i$; subtracting, $\sum_i (c_i - c'_i) v_i = \mathbf{0}$, and independence forces $c_i = c'_i$ for every i . $\square$

8.3 Bases, dimension, and linear maps

Definition 8.14 (Basis). A *basis* of V is a linearly independent set B that spans V . Every $v \in V$ is then a *unique finite* linear combination of elements of B : existence because B spans, uniqueness because two expressions subtract to a relation among finitely many distinct elements of the independent set (as in Lemma 8.13). For a finite basis $B = (b_1, \dots, b_n)$, write

$$v = c_1 b_1 + \dots + c_n b_n, \quad [v]_B = (c_1, \dots, c_n),$$

and call the scalars c_i the *coordinates* of v in B .

Example 8.15 (The standard and monomial bases). The *standard basis* of F^n is $e_1, \dots, e_n$, where e_i has a 1 in position i and 0 elsewhere. Both properties reduce to reading off coordinates: the combination $\sum_i c_i e_i$ is the tuple $(c_1, \dots, c_n)$, so every vector is such a combination (spanning) and only the zero combination gives $\mathbf{0}$ (independence). The *monomial basis* of $F[X]_{<n}$ is $1, X, \dots, X^{n-1}$: a polynomial of degree less than n is, by construction, a linear combination of these monomials, and independence is the fact that polynomials are equal exactly when their coefficient sequences agree (Definition 5.1).

That all bases of a space have the same length rests on the following lemma, whose proof replaces the vectors of a spanning list, one at a time, by the vectors of an independent list.

Lemma 8.16 (Steinitz exchange). *If $v_1, \dots, v_m$ are linearly independent in V and $w_1, \dots, w_n$ span V , then $m \leq n$.*

Proof. We prove by induction on $k = 0, 1, \dots, m$ that, after a suitable renumbering of the w_j , the hybrid list

$$v_1, \dots, v_k, w_{k+1}, \dots, w_n$$

spans V . The case $k = 0$ is the hypothesis on the w_j .

For the induction step, suppose the hybrid list at stage $k < m$ spans V . In particular it expresses v_{k+1} :

$$v_{k+1} = \sum_{i \leq k} a_i v_i + \sum_{j > k} c_j w_j.$$

Some coefficient c_j with $j > k$ must be nonzero: were they all zero, v_{k+1} would be a linear combination of $v_1, \dots, v_k$, and Lemma 8.13 would make the list $v_1, \dots, v_{k+1}$ dependent, contradicting the independence of the v_i . (This step requires a slot $j > k$ to exist, i.e. $k < n$; if instead $k = n$, the hybrid list is $v_1, \dots, v_n$ and the displayed equation already yields the same contradiction.) Renumber so that $c_{k+1} \neq 0$, and solve for the displaced vector:

$$w_{k+1} = c_{k+1}^{-1} \left(v_{k+1} - \sum_{i \leq k} a_i v_i - \sum_{j > k+1} c_j w_j \right).$$

Every vector of the stage- k list therefore lies in the span of the stage- $(k+1)$ list $v_1, \dots, v_{k+1}, w_{k+2}, \dots, w_n$ —the v_i and the remaining w_j trivially, w_{k+1} by the display—so by Proposition 8.10 the stage- $(k+1)$ list spans V , completing the induction.

Each step replaces one w by one v , so the induction reaches $k = m$ only if there are at least m slots to replace: $m \leq n$. $\square$

Theorem 8.17 (Dimension). *Let V be a finitely generated F -vector space. Then V has a basis, and any two bases of V have the same number of elements. This common number is the dimension $\dim_F V$ (written $\dim V$ when the field is clear). In particular $\dim_F F^n = n$ and $\dim_F F[X]_{<n} = n$.*

Proof. Existence. Let $w_1, \dots, w_n$ be a finite spanning list. While the list is dependent, Lemma 8.13 exhibits some w_j in the span of the others; discarding it leaves the span unchanged, since any combination using w_j can be rewritten without it (substitute and regroup, Proposition 8.10). Each discard shortens the list, so after at most n discards the process halts at an independent spanning list—a basis. (If V is the zero space, the process discards everything and halts at the empty list, a basis of $\{\mathbf{0}\}$ with $\text{span}(\emptyset) = \{\mathbf{0}\}$; the dimension is 0.)

Invariance. Every basis is finite: if a basis contained more than the n elements of a finite spanning list, any $n + 1$ of its elements would contradict Lemma 8.16. Let $b_1, \dots, b_m$ and $b'_1, \dots, b'_n$ be bases. Applying Lemma 8.16 with the first list as the independent one and the second as the spanning one gives $m \leq n$; exchanging the roles gives $n \leq m$. Hence $m = n$.

The two named spaces have the explicit bases of Example 8.15, each of length n . $\square$

Linear maps are the functions between vector spaces that preserve the linear structure; the Pedersen-style commitments of the series are such maps.

Definition 8.18 (Linear map, kernel, image). A function $T: V \rightarrow W$ between F -vector spaces is *linear* (an *F -linear map*) if

$$T(au + bv) = aT(u) + bT(v) \quad \text{for all } a, b \in F \text{ and } u, v \in V.$$

Its *kernel* and *image* are

$$\ker T = \{v \in V : T(v) = \mathbf{0}\} \subseteq V, \quad \text{im } T = \{T(v) : v \in V\} \subseteq W.$$

A bijective linear map is an *isomorphism*.

Three basic facts follow from the subspace criterion and linearity. First, taking $a = b = 0$ gives $T(\mathbf{0}) = \mathbf{0}$, so $\mathbf{0} \in \ker T$ and $\mathbf{0} \in \text{im } T$. Second, both kernel and image are subspaces: if $u, v \in \ker T$ then $T(au + bv) = aT(u) + bT(v) = \mathbf{0}$, and if $T(u), T(v) \in \text{im } T$ then $aT(u) + bT(v) = T(au + bv) \in \text{im } T$; Proposition 8.6 applies to each. Third, T is injective if and only if $\ker T = \{\mathbf{0}\}$: if $T(u) = T(v)$, then $T(u - v) = \mathbf{0}$ by linearity, so a trivial kernel forces $u = v$; conversely an injective T sends only $\mathbf{0}$ to $T(\mathbf{0}) = \mathbf{0}$. Moreover, for fixed V and W the set $\text{Hom}(V, W)$ of all linear maps $V \rightarrow W$ is itself an F -vector space under the pointwise operations of Example 8.4(2)—a sum or scalar multiple of linear maps is linear, so $\text{Hom}(V, W)$ is a subspace of the space of all functions $V \rightarrow W$.

Theorem 8.19 (Rank–nullity). *Let $T: V \rightarrow W$ be a linear map with V finite-dimensional. Then*

$$\dim V = \dim \ker T + \dim \text{im } T.$$

Proof. The kernel is a subspace of the finite-dimensional V , and is itself finite-dimensional: an independent list in $\ker T$ has length at most $\dim V$ by Lemma 8.16 (tested against a basis of V ,

which spans), so a longest independent list $u_1, \dots, u_k$ in $\ker T$ exists, and it spans $\ker T$ —any $u \in \ker T$ outside its span would extend it to a longer independent list by Lemma 8.13. Thus $u_1, \dots, u_k$ is a basis of $\ker T$.

Extend it to a basis of V : while the current independent list fails to span V , adjoin any vector outside its span—independence is preserved by Lemma 8.13—and the process terminates, again because independent lists cannot exceed $\dim V$ in length (Lemma 8.16 against any basis of V). Write the resulting basis as $u_1, \dots, u_k, v_1, \dots, v_r$, so that $\dim V = k + r$.

We claim $T(v_1), \dots, T(v_r)$ is a basis of $\operatorname{im} T$; the theorem follows, since then $\dim \operatorname{im} T = r$ and $\dim V = k + r = \dim \ker T + \dim \operatorname{im} T$.

Spanning. Any element of $\operatorname{im} T$ is $T(v)$ for some $v = \sum_i a_i u_i + \sum_j c_j v_j$, and linearity with $T(u_i) = \mathbf{0}$ gives $T(v) = \sum_j c_j T(v_j)$.

Independence. Suppose $\sum_j c_j T(v_j) = \mathbf{0}$. By linearity $T(\sum_j c_j v_j) = \mathbf{0}$, so $\sum_j c_j v_j \in \ker T$, and expressing this kernel element in the kernel basis gives $\sum_j c_j v_j = \sum_i a_i u_i$ for some scalars a_i . Rearranged, $\sum_i a_i u_i - \sum_j c_j v_j = \mathbf{0}$ is a linear relation among the combined basis of V , whose independence forces every coefficient—in particular every c_j —to vanish. $\square$

Rank–nullity implies that a linear map from a higher-dimensional space into a lower-dimensional one has a nontrivial kernel—the image has dimension at most that of the target, so the kernel has dimension at least the difference—and therefore that distinct inputs collide. This collision is what makes vector commitments compressing and only *computationally* binding (Remark 8.29).

8.4 Bilinear forms and the inner product on F^n

Commitment and proof systems use expressions linear in *each* of two arguments, chiefly the inner product of two vectors. Over a finite field such a product has no length, angle or positivity, and the applications use only its algebraic properties, which the *bilinear form* captures.

Definition 8.20 (Bilinear form). A *bilinear form* on an F -vector space V is a function $\beta: V \times V \rightarrow F$ that is linear in each argument separately:

$$\beta(au + a'u', v) = a\beta(u, v) + a'\beta(u', v), \quad \beta(u, bv + b'v') = b\beta(u, v) + b'\beta(u, v'),$$

for all $u, u', v, v' \in V$ and $a, a', b, b' \in F$. The form is *symmetric* if $\beta(u, v) = \beta(v, u)$ for all u, v , and *nondegenerate* if $\beta(u, v) = 0$ for all v implies $u = \mathbf{0}$.

Relative to a fixed basis $b_1, \dots, b_n$ of V , a bilinear form is determined by finitely many scalars: expanding both arguments in the basis and applying bilinearity termwise, $\beta(u, v) = \sum_{i,j} \beta(b_i, b_j) [u]_i [v]_j$, so β is represented by the unique $n \times n$ array M with entries $M_{ij} = \beta(b_i, b_j)$, via $\beta(u, v) = [u]_B^T M [v]_B$ in matrix shorthand. The representation is not used further; only the special case with M the identity array is needed below.

Definition 8.21 (Standard inner product on F^n). The *standard inner product* (or *dot product*)

on F^n is

$$\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{i=1}^n a_i b_i = \mathbf{a}^T \mathbf{b}, \quad \mathbf{a} = (a_1, \dots, a_n), \mathbf{b} = (b_1, \dots, b_n).$$

Bilinearity holds because each term $a_i b_i$ is linear in each factor, and symmetry because multiplication in F commutes; the representing array in the standard basis is the identity ($\langle e_i, e_j \rangle$ is 1 if $i = j$ and 0 otherwise). The form is nondegenerate: if $\langle \mathbf{a}, \mathbf{b} \rangle = 0$ for every $\mathbf{b}$, then in particular $a_i = \langle \mathbf{a}, e_i \rangle = 0$ for each i , so $\mathbf{a} = \mathbf{0}$.

Remark 8.22 (No positivity over finite fields). Over $\mathbb{R}$ the standard inner product is *positive definite*: $\langle \mathbf{a}, \mathbf{a} \rangle = \sum_i a_i^2 \geq 0$, with equality only at $\mathbf{0}$. Positivity is what yields the Euclidean norm $\|\mathbf{a}\| = \sqrt{\langle \mathbf{a}, \mathbf{a} \rangle}$ and the Cauchy–Schwarz inequality $|\langle \mathbf{a}, \mathbf{b} \rangle| \leq \|\mathbf{a}\| \|\mathbf{b}\|$ —the geometric structure of real inner products. Over a finite field $\mathbb{F}_q$ there is no ordering compatible with the arithmetic, so “ ≥ 0 ” is meaningless, and this structure is lost: there exist nonzero *isotropic* vectors $\mathbf{a} \neq \mathbf{0}$ with $\langle \mathbf{a}, \mathbf{a} \rangle = 0$. Over $\mathbb{F}_5$, for instance, the vector $\mathbf{a} = (1, 2)$ has

$$\langle \mathbf{a}, \mathbf{a} \rangle = 1^2 + 2^2 = 1 + 4 = 5 = 0$$

in $\mathbb{F}_5$. The arguments of proof systems use only the algebraic properties that remain—bilinearity, symmetry, and nondegeneracy—so the absence of positivity does not affect them.

Nondegeneracy identifies vectors with the scalar-valued linear maps on them.

Proposition 8.23 (Functionals via the inner product). *For each $\mathbf{a} \in F^n$, the map $\mathbf{b} \mapsto \langle \mathbf{a}, \mathbf{b} \rangle$ is a linear map $F^n \rightarrow F$ (a linear functional). The assignment*

$$F^n \longrightarrow \text{Hom}(F^n, F), \quad \mathbf{a} \longmapsto \langle \mathbf{a}, \cdot \rangle,$$

is an isomorphism of F -vector spaces onto the dual space $\text{Hom}(F^n, F)$ of all linear functionals on F^n . In particular both spaces have dimension n .

Proof. Each $\langle \mathbf{a}, \cdot \rangle$ is linear in $\mathbf{b}$ by bilinearity, and the assignment itself is linear in $\mathbf{a}$ for the same reason, mapping into the vector space $\text{Hom}(F^n, F)$ of Definition 8.18. It is injective: if $\langle \mathbf{a}, \cdot \rangle$ is the zero functional, then $\langle \mathbf{a}, \mathbf{b} \rangle = 0$ for every $\mathbf{b}$, and nondegeneracy (Definition 8.21) gives $\mathbf{a} = \mathbf{0}$, so the kernel is trivial. It is surjective: given a linear functional φ , set $a_i = \varphi(e_i)$ and $\mathbf{a} = (a_1, \dots, a_n)$; then for every $\mathbf{b} = \sum_i b_i e_i$, linearity of φ gives

$$\varphi(\mathbf{b}) = \sum_i b_i \varphi(e_i) = \sum_i a_i b_i = \langle \mathbf{a}, \mathbf{b} \rangle,$$

so $\varphi = \langle \mathbf{a}, \cdot \rangle$. A bijective linear map is an isomorphism, and $\dim \text{Hom}(F^n, F) = \dim F^n = n$ follows since an isomorphism carries a basis to a basis (its inverse is linear, and both directions preserve spanning and independence). $\square$

Remark 8.24 (Polynomial evaluation as an inner product). By Proposition 8.23, the notions “a linear functional on F^n ” and “a vector in F^n ” are interchangeable. For a fixed point $z \in F$, the map sending a polynomial to its value at z is linear in the coefficients, hence an inner product against a fixed vector: for $p \in F[X]_{<n}$ with coefficient vector $\mathbf{c} = (c_0, c_1, \dots, c_{n-1})$,

$$p(z) = c_0 + c_1 z + \dots + c_{n-1} z^{n-1} = \langle \mathbf{c}, (1, z, z^2, \dots, z^{n-1}) \rangle.$$

Evaluating a *committed* polynomial at a challenge point is therefore an inner-product claim about the committed coefficient vector, which is the form of statement an inner-product argument proves (Remark 8.32).

8.5 The mixed inner product with group elements

The commitments of Halo 2 and Orchard pair a vector of *scalars* with a vector of *group elements*. This subsection develops their algebra over an abstract abelian group; the group used in practice, the points of an elliptic curve, is constructed in the elliptic-curve section later in the volume, and the results apply to it unchanged.

Construction 8.25 (Scalar multiplication by $\mathbb{F}_r$ on a prime-order group). Let $(G, +)$ be a finite abelian group of prime order r , written additively with identity $\mathcal{O}$. For an integer $a \in \mathbb{Z}$ and $G \in G$, write $[a]G$ for the a -fold multiple of G —the additive reading of the powers of §3: $[0]G = \mathcal{O}$, $[a]G = G + \dots + G$ with a summands for $a > 0$, and $[-a]G = -([a]G)$. The brackets separate the scalar notationally from the group element. The multiple $[a]G$ depends only on the residue of a modulo r : the order of G divides $|G| = r$ (Corollary 3.30), and $[a]G = [b]G$ whenever $a \equiv b \pmod{\text{ord}(G)}$ (Proposition 3.12(2), read additively), so congruence modulo r suffices. The operation therefore *descends* to scalars in the field $\mathbb{F}_r = \mathbb{Z}/r\mathbb{Z}$ (Corollary 4.26): for $a \in \mathbb{F}_r$ and $G \in G$, define $[a]G$ using any integer representative of a . The rest of the section uses this descended operation, which scales group elements by elements of a *field*.

Proposition 8.26 (G as an $\mathbb{F}_r$ -vector space). *Let G be an abelian group of prime order r , with the scalar multiplication of Construction 8.25. Then G is an $\mathbb{F}_r$ -vector space of dimension 1; any element $G \neq \mathcal{O}$ is a basis, and every element of G is $[a]G$ for a unique $a \in \mathbb{F}_r$.*

Proof. The vector-space axioms are the standard laws of multiples. For integers a, b , the identities $[a + b]G = [a]G + [b]G$ and $[a]([b]G) = [ab]G$ are the exponent laws of §3 in additive notation, $[1]G = G$ is the definition, and $[a](G + H) = [a]G + [a]H$ holds in any *abelian* group: for $a > 0$ it is the rearrangement of a summands $G + H$ into a summands G followed by a summands H , valid by commutativity and associativity (formally, induction on a), and the cases $a \leq 0$ follow by negating. Each law respects reduction modulo r by Construction 8.25, so the four scalar axioms of Definition 8.1 hold with scalars in $\mathbb{F}_r$, and $(G, +)$ is an abelian group by hypothesis: G is an $\mathbb{F}_r$ -vector space.

Now let $G \neq \mathcal{O}$. By Lagrange’s theorem (Theorem 3.29, via Corollary 3.30) the order of G divides the prime r , and it exceeds 1 since G is not the identity; hence $\text{ord}(G) = r$. The r multiples $[0]G, [1]G, \dots, [r - 1]G$ are then pairwise distinct (Proposition 3.12(2), additively: $[a]G = [b]G$ forces $a \equiv b \pmod{r}$), and G has exactly r elements, so the multiples exhaust G . Thus $\{G\}$ spans; and it is independent, since $[a]G = \mathcal{O}$ forces $a = 0$ in $\mathbb{F}_r$, again because $\text{ord}(G) = r$. A one-element basis gives $\dim_{\mathbb{F}_r} G = 1$, and the uniqueness of a in $[a]G$ is the uniqueness of coordinates in a basis (Definition 8.14). $\square$

Definition 8.27 (Mixed inner product, multiscalar multiplication). Let $\mathbf{G} = (G_1, \dots, G_n) \in G^n$ be a fixed tuple of group elements and $\mathbf{a} = (a_1, \dots, a_n) \in \mathbb{F}_r^n$ a vector of scalars. Their *mixed in-*

ner product—in computational contexts a *multiscalar multiplication* (MSM), in cryptographic ones a *Pedersen-style inner product*—is the group element

$$\langle \mathbf{a}, \mathbf{G} \rangle = \sum_{i=1}^n [a_i] G_i \in \mathbb{G}.$$

The angle-bracket notation of Definition 8.21 is reused; the type of the second argument—scalar vector or vector of group elements—determines which product is meant.

Proposition 8.28 (Bilinearity of the mixed product). *Fix $\mathbf{G} \in \mathbb{G}^n$. The map $\mathbf{a} \mapsto \langle \mathbf{a}, \mathbf{G} \rangle$ is $\mathbb{F}_r$ -linear from $\mathbb{F}_r^n$ to $\mathbb{G}$:*

$$\langle c\mathbf{a} + c'\mathbf{a}', \mathbf{G} \rangle = [c]\langle \mathbf{a}, \mathbf{G} \rangle + [c']\langle \mathbf{a}', \mathbf{G} \rangle \quad \text{for all } c, c' \in \mathbb{F}_r \text{ and } \mathbf{a}, \mathbf{a}' \in \mathbb{F}_r^n.$$

Symmetrically, for fixed $\mathbf{a} \in \mathbb{F}_r^n$ the map $\mathbf{G} \mapsto \langle \mathbf{a}, \mathbf{G} \rangle$ is a group homomorphism $\mathbb{G}^n \rightarrow \mathbb{G}$, and is $\mathbb{F}_r$ -linear when $\mathbb{G}^n$ is viewed as an $\mathbb{F}_r$ -vector space (componentwise, as in Example 8.3 with $\mathbb{G}$ in place of F). Thus $\langle \cdot, \cdot \rangle: \mathbb{F}_r^n \times \mathbb{G}^n \rightarrow \mathbb{G}$ is bilinear over $\mathbb{F}_r$.

Proof. Both statements are the vector-space laws of $\mathbb{G}$ (Proposition 8.26) applied componentwise. For the first,

$$\langle c\mathbf{a} + c'\mathbf{a}', \mathbf{G} \rangle = \sum_i [ca_i + c'a'_i] G_i = \sum_i ([c][a_i]G_i + [c'][a'_i]G_i) = [c] \sum_i [a_i]G_i + [c'] \sum_i [a'_i]G_i,$$

using $[a + b]G = [a]G + [b]G$ and $[ab]G = [a]([b]G)$ in each component and then regrouping the sum in the abelian group $\mathbb{G}$. For the second, linearity in $\mathbf{G}$ follows by the symmetric computation on the other slot of the defining sum, using $[a](G + H) = [a]G + [a]H$ componentwise. $\square$

Remark 8.29 (The mixed product as a commitment). The element $\langle \mathbf{a}, \mathbf{G} \rangle$ is a *commitment* to the scalar vector $\mathbf{a}$ under the public “basis” $\mathbf{G}$, as at the opening of the section. Proposition 8.28 is its *homomorphic* property: the commitment to $c\mathbf{a} + c'\mathbf{a}'$ is the $[c]$ -, $[c']$ -scaled combination of the individual commitments, computable by anyone holding those commitments and the scalars c, c' , without knowledge of $\mathbf{a}$ or $\mathbf{a}'$. Provided the G_i are chosen so that no party can exhibit a nontrivial relation $\sum_i [a_i]G_i = \mathcal{O}$ —a computational hypothesis, the *discrete-logarithm hardness assumption*, made precise in later volumes of this series—the commitment is *binding*: the map $\mathbf{a} \mapsto \langle \mathbf{a}, \mathbf{G} \rangle$ is computationally injective, in the sense that no feasible computation produces two vectors with the same image. The map is not injective once $n \geq 2$: it is linear from the n -dimensional $\mathbb{F}_r^n$ into the one-dimensional $\mathbb{G}$ (Proposition 8.26), so by rank-nullity (Theorem 8.19) its kernel has dimension at least $n - 1$. Collisions therefore exist in large number; binding asserts only that they cannot be *found*, a computational rather than information-theoretic statement. The classical Pedersen commitment to a single value a with randomness ρ is the special case $n = 2$, with $H \neq \mathcal{O}$,

$$[a]G + [\rho]H = \langle (a, \rho), (G, H) \rangle,$$

which is moreover *perfectly hiding*: for fixed a , as ρ ranges uniformly over $\mathbb{F}_r$, the term $[\rho]H$ ranges uniformly over $\mathbb{G}$ (it is a bijection of $\mathbb{G}$, by unique representation in the basis $\{H\}$), so the commitment’s distribution is uniform and carries no information about a .

8.6 Linear algebra in commitment and proof systems

The following remarks locate the preceding results in commitment and proof systems. They are descriptive; later volumes develop each point rigorously.

Remark 8.30 (Witnesses, constraints, and assignments). An arithmetic circuit over $\mathbb{F}_q$ becomes, after *arithmetisation*—the encoding of a computation as a system of polynomial constraints on the wire values—a collection of equations that the vector $\mathbf{z} \in \mathbb{F}_q^N$ of all wire values must satisfy. The *linear* part of such a system—the wiring (or “copy”) constraints of the PLONK family, which assert that certain wires carry equal values—consists of homogeneous linear equations in the entries of $\mathbf{z}$, organised in practice as arrays of coefficients (one row per equation) acting on the assignment vector. Satisfying the linear constraints confines $\mathbf{z}$ to a subspace of $\mathbb{F}_q^N$ (Example 8.7), or, when public inputs fix some entries, to a translate $\mathbf{z}_0 + W$ of one—an *affine* subspace. The multiplicative constraints are handled separately, by the polynomial methods of the surrounding sections. Rank–nullity (Theorem 8.19) quantifies the degrees of freedom that remain after the linear constraints are imposed: the dimension of the witness space.

Remark 8.31 (Commitments as linear maps). A Pedersen vector commitment $\mathbf{a} \mapsto \langle \mathbf{a}, \mathbf{G} \rangle$ is a linear map $\mathbb{F}_r^n \rightarrow \mathbf{G}$ (Proposition 8.28), and every algebraic manipulation a verifier performs rests on this linearity: taking known linear combinations of commitments, folding two commitment keys $\mathbf{G}, \mathbf{G}'$ into $\mathbf{G} + [x]\mathbf{G}'$ under a challenge x , and checking claimed openings by re-evaluating the same linear map. A polynomial commitment is the same map applied to a coefficient vector, and an *evaluation* of the committed polynomial at a point z is the scalar inner product $\langle \mathbf{c}, (1, z, \dots, z^{n-1}) \rangle$ (Remark 8.24).

Remark 8.32 (The inner-product relation). The core statement an inner-product argument proves has the form: “I know vectors $\mathbf{a}, \mathbf{b} \in \mathbb{F}_r^n$ such that

$$P = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\langle \mathbf{a}, \mathbf{b} \rangle]U$$

for public $\mathbf{G}, \mathbf{H} \in \mathbf{G}^n$ and $U \in \mathbf{G}$.” The equation combines three of this section’s constructions: two mixed inner products $\langle \mathbf{a}, \mathbf{G} \rangle$ and $\langle \mathbf{b}, \mathbf{H} \rangle$, committing to the witness vectors; one scalar inner product $\langle \mathbf{a}, \mathbf{b} \rangle$, the value being argued; and the bilinearity of all three, which is what permits prover and verifier to *fold* a length- n instance into a length- $n/2$ one and recurse. The soundness of the argument reduces to the nondegeneracy of these forms together with the hardness of finding kernel elements of the commitment map.

9 Roots of unity and evaluation domains

This section constructs the evaluation domains on which polynomial proof systems of the kind underlying Halo 2 and Zcash Orchard encode computations, and the fast Fourier transform on them. Such a proof system records an n -step computation as a table over a finite field, one column per register and one row per step, and convinces a verifier that every row follows from its predecessor without the verifier replaying the computation: by Lagrange interpolation (Theorem 5.30) each column is the value table on n fixed points of a polynomial of degree less than n , so conditions on the table become polynomial identities, checkable at a single random point by the Schwartz–Zippel lemma (Theorem 5.45). The powers of a primitive n -th root of unity form a point set

on which conditions on every row, shifts to the next row and the polynomials attached to the set are cheap to evaluate, and on which conversion between values and coefficients, and polynomial multiplication, cost far fewer than the n^2 field operations of the schoolbook methods. Such a root exists in $\mathbb{F}_q$ exactly when $n \mid q - 1$ (Theorem 9.7); its powers form the evaluation domain H , whose vanishing polynomial is $X^n - 1$ (Theorem 9.13) and whose Lagrange basis has a closed form (Theorem 9.22); evaluation on H is the number-theoretic transform, which turns polynomial multiplication into entrywise multiplication of value vectors (Theorem 9.32); and the fast Fourier transform computes it in $O(n \log n)$ operations (Theorem 9.37).

Throughout, $\mathbb{F}$ denotes a field. When $\mathbb{F}$ is to be finite, the notation $\mathbb{F}_q$ denotes the field with q elements (so q is a prime power, Theorem 6.9); when we intend a prime field specifically we write $\mathbb{F}_p$. The multiplicative group of nonzero elements is $\mathbb{F}^\times = \mathbb{F} \setminus \{0\}$ (Example 3.7). We reserve the letter n for the order of the domain constructed below, with $n \geq 1$.

9.1 Roots of unity

Definition 9.1 (Root of unity). Let $\mathbb{F}$ be a field and $n \in \mathbb{N}_{>0}$. An element $\omega \in \mathbb{F}$ is an n -th root of unity if $\omega^n = 1$. We write

$$\mu_n(\mathbb{F}) = \{ \omega \in \mathbb{F} : \omega^n = 1 \}$$

for the set of all n -th roots of unity in $\mathbb{F}$.

Since $0^n = 0 \neq 1$, we have $0 \notin \mu_n(\mathbb{F})$; every n -th root of unity is therefore a unit, i.e. lies in $\mathbb{F}^\times$.

Proposition 9.2. The set $\mu_n(\mathbb{F})$ is a finite subgroup of $\mathbb{F}^\times$ of order at most n .

Proof. The identity 1 satisfies $1^n = 1$, so $1 \in \mu_n(\mathbb{F})$. If $\omega, \zeta \in \mu_n(\mathbb{F})$, then $(\omega\zeta)^n = \omega^n \zeta^n = 1 \cdot 1 = 1$ because $\mathbb{F}^\times$ is abelian, so $\omega\zeta \in \mu_n(\mathbb{F})$; and $(\omega^{-1})^n = (\omega^n)^{-1} = 1^{-1} = 1$, so $\omega^{-1} \in \mu_n(\mathbb{F})$. Hence $\mu_n(\mathbb{F})$ is a subgroup of $\mathbb{F}^\times$ (Definition 3.14). For the order bound, every element of $\mu_n(\mathbb{F})$ is a root of the polynomial $X^n - 1 \in \mathbb{F}[X]$, which is nonzero of degree n and so has at most n roots in $\mathbb{F}$ (Theorem 5.18). Thus $|\mu_n(\mathbb{F})| \leq n$. $\square$

The bound $|\mu_n(\mathbb{F})| \leq n$ need not be attained; it is attained exactly when $\mathbb{F}$ contains a *primitive* n -th root of unity.

Definition 9.3 (Primitive root of unity). An element $\omega \in \mathbb{F}$ is a *primitive n -th root of unity* if $\omega^n = 1$ and $\omega^k \neq 1$ for every integer k with $1 \leq k < n$. Equivalently, ω is a primitive n -th root of unity if its multiplicative order $\text{ord}(\omega)$ (Definition 3.11) equals n .

The order $\text{ord}(\omega)$ is the least positive integer d with $\omega^d = 1$; it exists for any $\omega \in \mu_n(\mathbb{F})$ because n itself is such an exponent and the positive integers are well-ordered. The two formulations in Definition 9.3 agree, since both state that n is the least positive exponent d with $\omega^d = 1$.

Remark 9.4 (Primitive roots as roots of unity). The group-theoretic term “generator of $\mathbb{F}_q^\times$ ” and the polynomial term “primitive $(q - 1)$ -th root of unity” name the same elements, as announced in Remark 6.20: a primitive root of a finite field $\mathbb{F}_q$, that is, a generator of the cyclic group $\mathbb{F}_q^\times$ in the sense of Definition 6.19, is an element of multiplicative order $q - 1$, which by Definition 9.3 is precisely a primitive $(q - 1)$ -th root of unity.

Lemma 9.5 (Order divides exponent). *Let $\omega \in \mathbb{F}^\times$ have finite order $d = \text{ord}(\omega)$. Then for any integer m we have $\omega^m = 1$ if and only if $d \mid m$. In particular, ω is an n -th root of unity if and only if $d \mid n$, and ω is a primitive n -th root of unity if and only if $d = n$.*

Proof. The first claim is Proposition 3.12(1), read in the abelian group $\mathbb{F}^\times$: if $d \mid m$, write $m = dk$, so $\omega^m = (\omega^d)^k = 1$; conversely, division with remainder $m = qd + r$, $0 \leq r < d$, gives $1 = \omega^m = \omega^r$, and minimality of d forces $r = 0$. The remaining clauses instantiate it: $\omega^n = 1 \iff d \mid n$ is the case $m = n$, and primitivity is the case $d = n$ by Definition 9.3. $\square$

Proposition 9.6. *If $\omega \in \mathbb{F}$ is a primitive n -th root of unity, then*

$$\mu_n(\mathbb{F}) = \langle \omega \rangle = \{1, \omega, \omega^2, \dots, \omega^{n-1}\}$$

is a cyclic group of order exactly n , and these n powers are pairwise distinct.

Proof. The powers $1, \omega, \dots, \omega^{n-1}$ are pairwise distinct: if $\omega^i = \omega^j$ with $0 \leq i \leq j \leq n-1$, then $\omega^{j-i} = 1$ with $0 \leq j-i < n$, so primitivity (Lemma 9.5, with $d = n$) forces $j-i = 0$. Thus $\langle \omega \rangle$ has exactly n elements, all of which lie in $\mu_n(\mathbb{F})$ since $(\omega^k)^n = (\omega^n)^k = 1$. But $|\mu_n(\mathbb{F})| \leq n$ by Proposition 9.2, so the inclusion $\langle \omega \rangle \subseteq \mu_n(\mathbb{F})$ is an equality of n -element sets. $\square$

Over the complex numbers a primitive n -th root of unity always exists, namely $\omega = e^{2\pi i/n}$; the n -th roots of unity are n equally spaced points on the unit circle, and the primitive ones lie at the angles $2\pi k/n$ with $\gcd(k, n) = 1$. Over a finite field, the case used in cryptography, one exists only for the n determined in the next subsection.

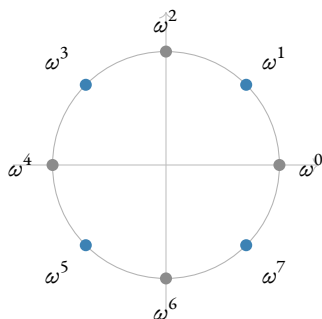


Figure 1. The eight 8-th roots of unity in $\mathbb{C}$, equally spaced on the unit circle at angles $2\pi k/8$, with $\omega = e^{2\pi i/8}$. The four primitive ones ($k = 1, 3, 5, 7$, exactly the k coprime to 8; marked in colour) each generate the whole cycle; the grey points $\omega^0 = 1, \omega^2 = i, \omega^4 = -1, \omega^6 = -i$ have orders 1, 4, 2, 4 and generate proper subgroups. Over a finite field the cyclic group structure persists without the geometry of the circle.

9.2 Existence over finite fields: the condition $n \mid q - 1$

The multiplicative group $\mathbb{F}_q^\times$ of a finite field is cyclic of order $q - 1$ (Theorem 6.17); the existence theorem for roots of unity rests on this fact.

Theorem 9.7 (Existence and count of primitive n -th roots of unity). *Let $\mathbb{F}_q$ be a finite field and $n \in \mathbb{N}_{>0}$ with $\gcd(n, q) = 1$ (automatic when $n < p$, where $p = \text{char } \mathbb{F}_q$; equivalently, in characteristic p , exactly the condition $p \nmid n$). Then:*

1. *The field $\mathbb{F}_q$ contains a primitive n -th root of unity if and only if $n \mid q - 1$.*
2. *When $n \mid q - 1$, the group $\mu_n(\mathbb{F}_q)$ is cyclic of order exactly n , and the number of primitive n -th roots of unity in $\mathbb{F}_q$ is $\varphi(n)$, where φ is Euler's totient function (Definition 2.26).*

Proof. Let g be a generator of $\mathbb{F}_q^\times$, which is cyclic of order $q - 1$ by Theorem 6.17.

($\Rightarrow$) Suppose ω is a primitive n -th root of unity in $\mathbb{F}_q$. Then $\omega \in \mathbb{F}_q^\times$ has order n by Definition 9.3, and by Lagrange's theorem the order of an element divides the group order $q - 1$ (Corollary 3.30). Hence $n \mid q - 1$.

($\Leftarrow$) Suppose $n \mid q - 1$, say $q - 1 = nm$. Set $\omega = g^m$. Then $\omega^n = g^{mn} = g^{q-1} = 1$, so $\omega \in \mu_n(\mathbb{F}_q)$. Its order is

$$\text{ord}(\omega) = \text{ord}(g^m) = \frac{\text{ord}(g)}{\gcd(m, \text{ord}(g))} = \frac{q-1}{\gcd(m, q-1)} = \frac{q-1}{m} = n,$$

using the order-of-a-power formula (Corollary 3.23) together with $m \mid q - 1$ (so $\gcd(m, q - 1) = m$). Thus ω is a primitive n -th root of unity, and by Proposition 9.6, $\mu_n(\mathbb{F}_q) = \langle \omega \rangle$ is cyclic of order exactly n .

Count. In the cyclic group $\mu_n(\mathbb{F}_q) = \langle \omega \rangle$ of order n , the element ω^k has order $n / \gcd(k, n)$, again by Corollary 3.23. This equals n precisely when $\gcd(k, n) = 1$, and the number of $k \in \{0, 1, \dots, n-1\}$ with $\gcd(k, n) = 1$ is by definition $\varphi(n)$ (Definition 2.26). Hence there are exactly $\varphi(n)$ primitive n -th roots of unity.

(On the coprimality hypothesis. The condition $\gcd(n, q) = 1$ is what guarantees that $X^n - 1$ is separable— $\gcd(X^n - 1, nX^{n-1}) = 1$ when $n \neq 0$ in $\mathbb{F}_q$ —hence has n distinct roots in a splitting field over $\mathbb{F}_q$ (Theorem 5.40); that all n of them lie in $\mathbb{F}_q$ itself is exactly the condition $n \mid q - 1$. If $p = \text{char } \mathbb{F}_q$ divided n , then $X^n - 1$ would have repeated roots and $\mu_n(\mathbb{F}_q)$ would be strictly smaller. But $n \mid q - 1$ already forces $\gcd(n, q) = 1$, since $\gcd(q - 1, q) = 1$, so the hypothesis is subsumed.) $\square$

Remark 9.8. The proof shows that for a finite field the statement reduces to $\mathbb{F}_q$ has a primitive n -th root of unity $\iff n \mid q - 1$, with no separate coprimality condition, because $n \mid q - 1$ makes n coprime to q ; the theorem states the condition $\gcd(n, q) = 1$ only to exhibit the role of the characteristic.

Example 9.9. Take $\mathbb{F}_q = \mathbb{F}_p$ with $p = 7$. Here $q - 1 = 6$, and 3 is a generator of $\mathbb{F}_7^\times$: its successive powers are 3, 2, 6, 4, 5, 1, all six nonzero residues (Example 6.21). For $n = 3 \mid 6$, set $m = 6/3 = 2$ and $\omega = 3^2 = 2$. Since $2^1 = 2$, $2^2 = 4$ and $2^3 = 8 \equiv 1$, the element $\omega = 2$ has order 3 and $\mu_3(\mathbb{F}_7) = \{1, 2, 4\}$. There are $\varphi(3) = 2$ primitive cube roots of unity, namely 2 and 4. By contrast $4 \nmid 6$, so $\mathbb{F}_7$ has no primitive 4-th root of unity: $X^4 - 1 = (X^2 - 1)(X^2 + 1)$, and $X^2 + 1$ is irreducible over $\mathbb{F}_7$ because it has no root there, -1 not being a square modulo 7.

Remark 9.10 (Two-adicity and FFT-friendly primes). For the fast algorithms below, n should be a power of two, or at least *smooth*—a product of small primes, ideally with many factors of two. By Theorem 9.7, a primitive 2^k -th root of unity exists exactly when $2^k \mid q - 1$. The largest k with $2^k \mid q - 1$ is called the *two-adicity* of the field. Designers choose primes p of high two-adicity, $p - 1 = 2^k \cdot c$ for large k , as scalar fields of proof systems so that large smooth evaluation domains exist. Both Pasta primes—those underlying the Pallas and Vesta curves of Section 10—satisfy $2^{32} \mid p - 1$, with two-adicity exactly 32 (the primes are specified in §5.4.9.6, “Pallas and Vesta”, of the Zcash protocol specification), so each Pasta field contains primitive 2^k -th roots of unity for every $k \leq 32$, hence power-of-two evaluation domains of every size up to 2^{32} ; every construction of this section therefore applies over the fields used by Halo 2 and Orchard.

9.3 The evaluation domain H and its vanishing polynomial

Definition 9.11 (Evaluation domain). Let $\mathbb{F}$ be a field containing a primitive n -th root of unity ω . The *evaluation domain* of order n is the multiplicative subgroup

$$H = \langle \omega \rangle = \{1, \omega, \omega^2, \dots, \omega^{n-1}\} = \mu_n(\mathbb{F}) \subseteq \mathbb{F}^\times,$$

a cyclic group of order n by Proposition 9.6. We index its elements as $\omega^0, \omega^1, \dots, \omega^{n-1}$, and abbreviate ω^i by b_i when convenient. Both $\mathbb{F}$ and ω remain fixed for the rest of the section.

Definition 9.12 (Vanishing polynomial). The *vanishing polynomial* (or *zerofier*) of H is

$$Z_H(X) = \prod_{b \in H} (X - b) \in \mathbb{F}[X].$$

On a generic n -point set the vanishing polynomial (Remark 5.33) is a product of n linear factors with no simpler form; on the group H it reduces to a binomial.

Theorem 9.13 (Closed form and factorisation of the vanishing polynomial). *With $H = \mu_n(\mathbb{F})$ of order n generated by a primitive n -th root of unity ω ,*

$$Z_H(X) = X^n - 1 = \prod_{i=0}^{n-1} (X - \omega^i).$$

Moreover Z_H is separable: it has n distinct roots and $\gcd(Z_H, Z'_H) = 1$.

Proof. Every $b \in H$ satisfies $b^n = 1$, hence is a root of $X^n - 1$. The n elements of H are distinct (Proposition 9.6), so $X^n - 1$ has n distinct roots, namely all of H , and factors as $X^n - 1 = c \prod_{b \in H} (X - b)$ for some constant $c \in \mathbb{F}^\times$ by iterating the factor theorem (Proposition 5.36, with the degree leaving no room for further factors). Comparing leading coefficients—both $X^n - 1$ and the product are monic—gives $c = 1$, whence $X^n - 1 = \prod_{b \in H} (X - b) = Z_H(X)$.

For separability, the formal derivative is $Z'_H(X) = nX^{n-1}$ (Definition 5.37). The existence of the n distinct roots already forces $\text{char } \mathbb{F} \nmid n$: if $\text{char } \mathbb{F} = \ell$ divided n , say $n = \ell m$, the freshman’s dream (Lemma 6.13) would give $X^n - 1 = (X^m - 1)^\ell$, a polynomial with at most $m < n$ distinct

roots—a contradiction. Hence $n \neq 0$ in $\mathbb{F}$, so nX^{n-1} is nonzero at every element of H , since each is nonzero. Thus it has no root in common with $X^n - 1$, so $\gcd(Z_H, Z'_H) = 1$ and Z_H is separable (Theorem 5.40). $\square$

The proof shows that $\text{char } \mathbb{F} \nmid n$, so that division by n is defined in $\mathbb{F}$. By the theorem, a polynomial f vanishes on all of H if and only if $X^n - 1$ divides it, and Z_H is evaluated at any point by a single exponentiation X^n , not n multiplications.

Proposition 9.14 (Cosets and shifts). *Let $\gamma \in \mathbb{F}^\times$ and let $\gamma H = \{\gamma h : h \in H\}$ be the corresponding coset (Definition 3.27). Then the polynomial vanishing on γH is*

$$Z_{\gamma H}(X) = \prod_{b \in H} (X - \gamma b) = X^n - \gamma^n.$$

In particular, distinct cosets of H in $\mathbb{F}^\times$ have vanishing polynomials $X^n - c$ for distinct constants $c = \gamma^n$, and the cosets of H partition $\mathbb{F}^\times$ (Lemma 3.28) into translates of H , on each of which X^n is constant.

Proof. Substitute $Y = X/\gamma$:

$$\prod_{b \in H} (X - \gamma b) = \gamma^n \prod_{b \in H} (Y - b) = \gamma^n (Y^n - 1) = \gamma^n ((X/\gamma)^n - 1) = X^n - \gamma^n,$$

using Theorem 9.13 for the middle equality. The constant is $c = \gamma^n$, and

$$\gamma_1 H = \gamma_2 H \iff \gamma_1/\gamma_2 \in H \iff (\gamma_1/\gamma_2)^n = 1 \iff \gamma_1^n = \gamma_2^n,$$

so distinct cosets give distinct constants c . $\square$

Lemma 9.15 (Finite geometric series). *Let $\mathbb{F}$ be a field, $\zeta \in \mathbb{F}$ with $\zeta \neq 1$, and $n \geq 1$. Then*

$$\sum_{i=0}^{n-1} \zeta^i = \frac{\zeta^n - 1}{\zeta - 1}.$$

Proof. Write $S = 1 + \zeta + \zeta^2 + \dots + \zeta^{n-1}$. Multiplying by ζ raises the exponent of every term by one:

$$\zeta S = \zeta + \zeta^2 + \dots + \zeta^{n-1} + \zeta^n.$$

In the difference $\zeta S - S$, the terms $\zeta, \zeta^2, \dots, \zeta^{n-1}$ appear in both lines and cancel; only the two extreme terms remain:

$$(\zeta - 1)S = \zeta S - S = \zeta^n - 1.$$

Since $\zeta \neq 1$, the factor $\zeta - 1$ is invertible, and dividing by it yields the claim. $\square$

Proposition 9.16 (Sum of roots of unity). *If $n > 1$ then $\sum_{i=0}^{n-1} \omega^i = 0$, and more generally for any integer k ,*

$$\sum_{i=0}^{n-1} \omega^{ki} = \begin{cases} n & \text{if } n \mid k, \\ 0 & \text{if } n \nmid k. \end{cases}$$

Proof. If $n \mid k$, then $\omega^{ki} = (\omega^n)^{ki/n} = 1$ for each i , so the sum is n . If $n \nmid k$, set $\zeta = \omega^k \neq 1$ (since ω has order n , we have $\omega^k = 1 \iff n \mid k$ by Lemma 9.5), and Lemma 9.15 gives

$$\sum_{i=0}^{n-1} \zeta^i = \frac{\zeta^n - 1}{\zeta - 1} = \frac{(\omega^n)^k - 1}{\zeta - 1} = \frac{1 - 1}{\zeta - 1} = 0,$$

the denominator being nonzero because $\zeta \neq 1$. The first claim is the case $k = 1$, valid for $n > 1$ since then $n \nmid 1$. $\square$

Remark 9.17 (Vanishing by rotation invariance). For $n \nmid k$ the vanishing also follows without the geometric-series formula. The n terms of $\sum_i \omega^{ki}$ run through the subgroup $\langle \zeta \rangle$ generated by $\zeta = \omega^k$, each element appearing equally often; multiplying the sum by ζ permutes the terms cyclically, so the sum satisfies $\zeta S = S$, and $(\zeta - 1)S = 0$ with $\zeta \neq 1$ forces $S = 0$. Figure 1 shows the complex case, in which rotation permutes the equally spaced points.

Remark 9.18 (Why “orthogonality”). Call two vectors in $\mathbb{F}^n$ *orthogonal* when their standard inner product (Definition 8.21) vanishes. For each frequency k let $v_k = (1, \omega^k, \omega^{2k}, \dots, \omega^{(n-1)k})$ be its vector of powers, and let $\bar{v}_k = (1, \omega^{-k}, \omega^{-2k}, \dots, \omega^{-(n-1)k})$ be the same construction with ω^{-1} in place of ω : the *mirror* of v_k , which traverses the cycle in the opposite direction and takes the role of the complex conjugate in the classical setting (on the unit circle $\bar{z} = z^{-1}$). Then $\langle \bar{v}_k, v_{k'} \rangle = \sum_i \omega^{i(k'-k)}$, which Proposition 9.16 evaluates to 0 for $k \neq k'$ and to n for $k = k'$: distinct frequency vectors are orthogonal, and each pairs with its own mirror to n . (Which factor carries the mirror is a convention: with the roles exchanged the exponent is $k - k'$, and the criterion $n \mid k' - k$ is symmetric; the order chosen matches the product $\bar{V}V$ computed in Theorem 9.29.) This is the finite-field counterpart of the orthogonality of the exponentials $\theta \mapsto e^{2\pi i k \theta}$ in classical Fourier analysis, with the sum over the cycle in place of the integral over the circle. The proof of the inverse transform (Theorem 9.29) rests on it, the v_k being the columns of the transform matrix.

9.4 The Lagrange basis on H

Since H has exactly n points, values on H determine polynomials of degree less than n (Corollary 5.20); the basis adapted to values on H is the Lagrange basis of §5.7.

Definition 9.19. Let $\mathbb{F}[X]_{<n}$ denote the $\mathbb{F}$ -vector space of polynomials of degree strictly less than n , together with the zero polynomial. It has the monomial basis $\{1, X, \dots, X^{n-1}\}$ (Definition 8.14), so $\dim_{\mathbb{F}} \mathbb{F}[X]_{<n} = n$ (Theorem 8.17).

Definition 9.20. For each $i \in \{0, 1, \dots, n-1\}$, the i -th *Lagrange basis polynomial* for $H = \{\omega^0, \dots, \omega^{n-1}\}$ is

$$L_i(X) = \prod_{\substack{j=0 \\ j \neq i}}^{n-1} \frac{X - \omega^j}{\omega^i - \omega^j} \in \mathbb{F}[X]_{<n},$$

the basis polynomial ℓ_i of Theorem 5.30 for the nodes $\omega^0, \dots, \omega^{n-1}$.

Proposition 9.21 (Interpolation property). *The polynomials $L_0, \dots, L_{n-1}$ satisfy the Kronecker-delta conditions*

$$L_i(\omega^j) = \delta_{ij} = \begin{cases} 1 & i = j, \\ 0 & i \neq j, \end{cases}$$

they form a basis of $\mathbb{F}[X]_{<n}$ and they give the unique interpolant: for any data $(v_0, \dots, v_{n-1}) \in \mathbb{F}^n$, the polynomial

$$f(X) = \sum_{i=0}^{n-1} v_i L_i(X)$$

is the unique element of $\mathbb{F}[X]_{<n}$ with $f(\omega^i) = v_i$ for all i .

Proof. Each L_i is well defined and of degree $n - 1$ because the nodes ω^j are distinct (Proposition 9.6), so the denominators $\omega^i - \omega^j$ with $j \neq i$ are nonzero. The delta conditions, the interpolation $f(\omega^j) = v_j$, and the uniqueness are Theorem 5.30 (with (4)) applied to these nodes with $d = n - 1$; uniqueness rests, as there, on the root count: two interpolants in $\mathbb{F}[X]_{<n}$ differ by a polynomial of degree $< n$ with n distinct roots, which is the zero polynomial (Theorem 5.18). It remains to see that the L_i form a basis of $\mathbb{F}[X]_{<n}$. They are linearly independent: if $\sum_i c_i L_i = 0$, evaluation at ω^j yields $c_j = 0$ for every j (Definition 8.11). Being n independent vectors in the n -dimensional space $\mathbb{F}[X]_{<n}$ (Definition 9.19), they form a basis (Theorem 8.17). $\square$

The root-count step recurs and is called the *root-count device*: two polynomials of degree $< n$ that agree on all n points of H are equal. It gives the uniqueness above, Proposition 9.24, and Example 9.25 below.

On a generic point set the Lagrange polynomials admit no closed form simpler than the defining product; on the *group* H they are expressed through the vanishing polynomial.

Theorem 9.22 (Closed form of the Lagrange basis on H). *For $H = \mu_n(\mathbb{F})$ with primitive root ω , and for each i ,*

$$L_i(X) = \frac{\omega^i}{n} \cdot \frac{X^n - 1}{X - \omega^i} = \frac{\omega^i (X^n - 1)}{n (X - \omega^i)}.$$

In particular, dividing out one linear factor and scaling yields all n Lagrange polynomials from the single polynomial $Z_H(X) = X^n - 1$.

Proof. Fix i . From Theorem 9.13, $Z_H(X) = \prod_{j=0}^{n-1} (X - \omega^j)$, so

$$\frac{Z_H(X)}{X - \omega^i} = \prod_{\substack{j=0 \\ j \neq i}}^{n-1} (X - \omega^j),$$

which is exactly the numerator of L_i in Definition 9.20. It remains to evaluate the denominator $\prod_{j \neq i} (\omega^i - \omega^j)$. Writing $Z_H = (X - \omega^i) g$ with $g = \prod_{j \neq i} (X - \omega^j)$, the product rule (Proposition 5.38) gives $Z'_H = g + (X - \omega^i) g'$, so

$$Z'_H(\omega^i) = g(\omega^i) = \prod_{\substack{j=0 \\ j \neq i}}^{n-1} (\omega^i - \omega^j)$$

—the derivative at a simple root equals the product of the differences to the other roots, the computation already made for simple roots in the proof of Theorem 5.40. Concretely $Z'_H(X) = nX^{n-1}$, so

$$Z'_H(\omega^i) = n(\omega^i)^{n-1} = n\omega^{in}\omega^{-i} = n \cdot 1 \cdot \omega^{-i} = \frac{n}{\omega^i},$$

using $\omega^{in} = (\omega^n)^i = 1$. Therefore

$$L_i(X) = \frac{Z_H(X)/(X - \omega^i)}{Z'_H(\omega^i)} = \frac{(X^n - 1)/(X - \omega^i)}{n/\omega^i} = \frac{\omega^i(X^n - 1)}{n(X - \omega^i)},$$

as claimed. $\square$

Remark 9.23 (Barycentric form). The closed form gives the *barycentric* interpolation formula on H a compact form. Writing $f(X) = \sum_i v_i L_i(X)$ and substituting Theorem 9.22,

$$f(X) = \frac{X^n - 1}{n} \sum_{i=0}^{n-1} \frac{\omega^i v_i}{X - \omega^i}.$$

Evaluating f at a point $z \notin H$ thus costs one evaluation of $z^n - 1$ and n terms of the sum, with no per-point polynomial reconstruction. A verifier can use this formula when the values v_i are known, as for public-input polynomials; a commitment to hidden values does not by itself permit this evaluation.

Proposition 9.24 (Lagrange polynomials sum to one). *The Lagrange basis polynomials for H satisfy $\sum_{i=0}^{n-1} L_i(X) = 1$ identically.*

Proof. The polynomial $g(X) = \sum_i L_i(X) - 1$ lies in $\mathbb{F}[X]_{<n}$ and vanishes at every ω^j , because $\sum_i L_i(\omega^j) = \sum_i \delta_{ij} = 1$ (Proposition 9.21). By the root-count device—a polynomial of degree $< n$ with n distinct roots is zero (Theorem 5.18)— $g \equiv 0$. $\square$

Example 9.25 (The all-ones value vector). The constant polynomial 1 interpolates the data $v_i = 1$ for all i ; Proposition 9.24 is the statement $\sum_i 1 \cdot L_i = 1$. Likewise the values of X on H are $v_i = \omega^i$, and, for $n \geq 2$, Proposition 9.21 gives $\sum_i \omega^i L_i(X) = X$: both sides have degree $< n$ and agree on all of H , so the root-count device applies. These identities are used to rewrite monomials in the Lagrange basis.

9.5 Evaluation and interpolation as mutually inverse linear maps

An element of $\mathbb{F}[X]_{<n}$ is described by its n coefficients or by its n values on H ; passage between these two coordinate systems is a linear isomorphism, the specialisation to H of the evaluation isomorphism for general nodes in Remark 5.32.

Definition 9.26 (Evaluation map). The *evaluation map* on H is the $\mathbb{F}$ -linear map (Definition 8.18)

$$\text{ev}_H: \mathbb{F}[X]_{<n} \longrightarrow \mathbb{F}^n, \quad \text{ev}_H(f) = (f(\omega^0), f(\omega^1), \dots, f(\omega^{n-1})).$$

Linearity is immediate from $(af + bg)(\omega^i) = af(\omega^i) + bg(\omega^i)$, an instance of Proposition 5.16.

Theorem 9.27 (Evaluation and interpolation are mutually inverse). *The evaluation map ev_H is a vector-space isomorphism. Its inverse is the interpolation map*

$$\text{int}_H: \mathbb{F}^n \longrightarrow \mathbb{F}[X]_{<n}, \quad \text{int}_H(v_0, \dots, v_{n-1}) = \sum_{i=0}^{n-1} v_i L_i(X).$$

That is, $\text{int}_H \circ \text{ev}_H = \text{id}_{\mathbb{F}[X]_{<n}}$ and $\text{ev}_H \circ \text{int}_H = \text{id}_{\mathbb{F}^n}$.

Proof. Both maps are linear and act between spaces of the same finite dimension n , so verifying one composition would suffice (a one-sided inverse makes ev_H injective, hence surjective by rank-nullity, Theorem 8.19); we verify both. For $v \in \mathbb{F}^n$, the polynomial $\text{int}_H(v) = \sum_i v_i L_i$ has value $\sum_i v_i L_i(\omega^j) = v_j$ at ω^j by Proposition 9.21, so $\text{ev}_H(\text{int}_H(v)) = v$. For $f \in \mathbb{F}[X]_{<n}$, the polynomial $\text{int}_H(\text{ev}_H(f)) = \sum_i f(\omega^i) L_i$ is, by the uniqueness clause of Proposition 9.21, the unique degree- $< n$ polynomial taking the values $f(\omega^i)$ on H —which is f itself. Hence the two maps are inverse; in particular ev_H is bijective and linear, i.e. an isomorphism (Definition 8.18). $\square$

In coordinates, ev_H is given by a matrix, which identifies evaluation on H with the Fourier transform. Matrix notation is fixed as follows: an $n \times n$ matrix over $\mathbb{F}$ is a doubly indexed family $A = (A_{ik})_{0 \leq i, k \leq n-1}$ of elements of $\mathbb{F}$, acting on a vector $c \in \mathbb{F}^n$ by $(Ac)_i = \sum_k A_{ik} c_k$ —the general form, written out through the standard basis (Example 8.15), of a linear map $\mathbb{F}^n \rightarrow \mathbb{F}^n$. The product of two matrices is defined so that $(AB)c = A(Bc)$, which forces $(AB)_{ij} = \sum_k A_{ik} B_{kj}$; the identity matrix I , with entries δ_{ik} , acts as the identity map; and A is invertible if some matrix A^{-1} satisfies $A^{-1}A = AA^{-1} = I$.

Definition 9.28 (Vandermonde / DFT matrix). Let $f(X) = \sum_{k=0}^{n-1} c_k X^k$ have coefficient vector $c = (c_0, \dots, c_{n-1})$. Then $\text{ev}_H(f)$ has i -th entry $f(\omega^i) = \sum_k c_k \omega^{ik}$. Thus, relative to the monomial basis on $\mathbb{F}[X]_{<n}$ and the standard basis on $\mathbb{F}^n$, the *Vandermonde matrix*

$$V = (\omega^{ik})_{0 \leq i, k \leq n-1} = \begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \dots & \omega^{2(n-1)} \\ \vdots & & & & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \dots & \omega^{(n-1)^2} \end{pmatrix}$$

—rows indexed by the evaluation point ω^i , columns by the power k —represents ev_H , so that $\text{ev}_H(f) = Vc$. We call V the (discrete Fourier) transform matrix on H .

Theorem 9.29 (Invertibility and the inverse transform). *The matrix V of Definition 9.28 is invertible, with*

$$(V^{-1})_{k,i} = \frac{1}{n} \omega^{-ik}.$$

Equivalently, $V^{-1} = \frac{1}{n}\bar{V}$, where $\bar{V} = (\omega^{-ik})_{i,k}$ is the transform matrix built from ω^{-1} (itself a primitive n -th root of unity). Consequently the coefficient and value representations are related by the mutually inverse formulas

$$c_k = \frac{1}{n} \sum_{i=0}^{n-1} f(\omega^i) \omega^{-ik}, \quad f(\omega^i) = \sum_{k=0}^{n-1} c_k \omega^{ik}.$$

Proof. First, $\omega^{-1} = \omega^{n-1}$ has order n by Corollary 3.23 (with $\gcd(n-1, n) = 1$), so $\bar{V}$ is again a matrix of the shape in Definition 9.28. We show $\bar{V}V = nI$. The (k, k') entry of $\bar{V}V$ is

$$\sum_{i=0}^{n-1} \omega^{-ik} \omega^{ik'} = \sum_{i=0}^{n-1} \omega^{i(k'-k)}.$$

By Proposition 9.16 this sum is n if $n \mid (k' - k)$ and 0 otherwise; for $k, k' \in \{0, \dots, n-1\}$ we have $n \mid (k' - k) \iff k = k'$. Hence $\bar{V}V = nI$, and the same computation with the roles of the two factors exchanged gives $V\bar{V} = nI$. Since $n \neq 0$ in $\mathbb{F}$ (Theorem 9.13), we may divide by n : $V^{-1} = \frac{1}{n}\bar{V}$. The component formulas write out the matrix identities $\text{ev}_H(f) = Vc$ and $c = V^{-1} \text{ev}_H(f)$. $\square$

Remark 9.30. Theorem 9.27 and Theorem 9.29 state one isomorphism in abstract and in matrix form. Since the inverse of V is, up to the scalar $1/n$, the same matrix with ω replaced by ω^{-1} , the fast inverse transform of Theorem 9.37 costs as much as the forward one.

9.6 The number-theoretic transform and convolution

The map ev_H , read as the matrix V , is the discrete Fourier transform over a finite field rather than over $\mathbb{C}$; since it involves no analytic structure (no complex exponentials, no convergence), it is called the *number-theoretic transform*.

Definition 9.31 (Number-theoretic transform). Let ω be a primitive n -th root of unity in $\mathbb{F}$. The *number-theoretic transform* (NTT) sends the coefficient vector of a polynomial to its value vector on H : it is the evaluation map ev_H written in coordinates. Explicitly, the NTT of $a = (a_0, \dots, a_{n-1}) \in \mathbb{F}^n$ is $\hat{a} = (\hat{a}_0, \dots, \hat{a}_{n-1})$ with

$$\hat{a}_j = \sum_{k=0}^{n-1} a_k \omega^{jk}, \quad j = 0, \dots, n-1,$$

each entry being the value at ω^j of the polynomial with coefficients a . The *inverse number-theoretic transform* is interpolation—the recovery of the coefficients from the values—and by Theorem 9.29 it too is a single explicit sum:

$$a_k = \frac{1}{n} \sum_{j=0}^{n-1} \hat{a}_j \omega^{-jk}.$$

The NTT has the convolution property of the classical DFT, the basis of fast polynomial multiplication. The coefficients of a product are a *convolution*: with $f = \sum_i a_i X^i$ and $g = \sum_j b_j X^j$, the coefficient of X^k in fg is $\sum_{i+j=k} a_i b_j$ (Definition 5.3). The *cyclic* variant folds indices modulo n : for $a, b \in \mathbb{F}^n$, the cyclic convolution $a * b \in \mathbb{F}^n$ has entries

$$(a * b)_k = \sum_{\substack{0 \leq i, j \leq n-1 \\ i+j \equiv k \pmod{n}}} a_i b_j, \quad k = 0, \dots, n-1.$$

Theorem 9.32 (Convolution theorem). *For $f, g \in \mathbb{F}[X]_{<n}$ with $\deg f + \deg g < n$, let $c = fg$ be their product (of degree $< n$). Then for every $h \in H$,*

$$c(h) = f(h) g(h),$$

i.e. pointwise multiplication of value vectors corresponds to polynomial multiplication. Equivalently, writing $a, b \in \mathbb{F}^n$ for the coefficient vectors of f, g (padded with zeros up to length n),

$$\widehat{a * b}_j = \hat{a}_j \hat{b}_j \quad (j = 0, \dots, n-1),$$

so the NTT turns convolution into entrywise multiplication.

Proof. The first identity is the statement that evaluation at h is a ring homomorphism $\mathbb{F}[X] \rightarrow \mathbb{F}$ (Proposition 5.16): $(fg)(h) = f(h) g(h)$. Since $\deg(fg) < n$, interpolation recovers the product $c = fg$ from its values on the n -point set H (Proposition 9.21), so the entrywise product of the value vectors uniquely determines c .

The convolution statement is proved in three steps: (i) multiplication in $\mathbb{F}[X]/(X^n - 1)$ is cyclic convolution of coefficient vectors; (ii) evaluation on H is a ring isomorphism from that quotient onto $\mathbb{F}^n$ with entrywise operations; (iii) the degree bound rules out wrap-around, so the cyclic product is the ordinary one. For (i), work in the quotient ring $\mathbb{F}[X]/(X^n - 1)$ (Definition 4.19 and Proposition 4.20, applied to the ideal of multiples of $X^n - 1$, Definition 4.17). Every class has a unique representative of degree $< n$, by division with remainder (Theorem 5.12); and multiplying two such representatives and reducing replaces each X^k with $k \geq n$ by X^{k-n} , since $X^n \equiv 1$. The coefficient of X^k in the reduced product of the representatives with coefficient vectors a and b is therefore $\sum_{i+j \equiv k \pmod{n}} a_i b_j = (a * b)_k$: multiplication in $\mathbb{F}[X]/(X^n - 1)$ is cyclic convolution of coefficient vectors.

Next consider the map $[p] \mapsto \text{ev}_H(p)$ from $\mathbb{F}[X]/(X^n - 1)$ to $\mathbb{F}^n$, where $\mathbb{F}^n$ is a commutative ring under entrywise addition and multiplication (the ring laws hold in each coordinate because they hold in $\mathbb{F}$), with multiplicative identity $(1, \dots, 1)$. The map is well defined: two representatives of a class differ by a multiple of $X^n - 1$, which vanishes on H (Theorem 9.13). It is a ring homomorphism (Definition 4.21), because evaluation at each point of H is one (Proposition 5.16) and the operations on $\mathbb{F}^n$ are entrywise. And it is bijective: classes correspond to their degree- $< n$ representatives, on which ev_H is a bijection by Theorem 9.27. Hence it is a ring isomorphism $\mathbb{F}[X]/(X^n - 1) \xrightarrow{\sim} \mathbb{F}^n$, and it sends the product of the classes of f and g —coefficient vector $a * b$ —to the entrywise product of $\hat{a}$ and $\hat{b}$. This is the displayed identity. Finally, when $\deg f + \deg g < n$, the ordinary product fg already has degree $< n$, so no folding occurs, the cyclic and ordinary products coincide, and the statement computes fg itself. $\square$

Remark 9.33 (Computational significance). Theorem 9.32 reduces multiplying two degree- $< n/2$ polynomials—a quadratic-cost operation by the schoolbook method—to: transform both (NTT), multiply the value vectors entrywise (n multiplications), and transform back (inverse NTT). If the NTT runs in $O(n \log n)$ operations, the whole multiplication costs $O(n \log n)$. The fast Fourier transform, constructed below, meets that hypothesis.

Example 9.34 (The convolution theorem in $\mathbb{F}_{17}$). In $\mathbb{F}_{17}$, the element $\omega = 4$ has $\omega^2 = 16 = -1$, so it is a primitive 4-th root of unity with domain $H = \{1, 4, 16, 13\}$. To multiply $f = 1 + 2X$ and $g = 3 + X$, pad both coefficient vectors to length 4 and transform: $\hat{a} = (3, 9, 16, 10)$ and $\hat{b} = (4, 7, 2, 16)$, the values of f and of g on H . Multiplying entrywise gives $(12, 12, 15, 7)$, the values of fg on H , and the inverse transform returns $(3, 7, 2, 0)$: the coefficients of $fg = 3 + 7X + 2X^2$, agreeing with the schoolbook product. At $n = 4$ nothing is saved; the example exhibits the three-step procedure, which saves work once the transforms themselves cost $O(n \log n)$ (§9.7).

9.7 The fast Fourier transform

Computing $\hat{a} = Va$ directly from Definition 9.28 costs $\Theta(n^2)$ field multiplications: each of the n outputs is a sum of n products. The fast Fourier transform (FFT)—here a number-theoretic FFT—computes the same vector in $O(n \log n)$ operations by a divide-and-conquer recursion, provided n is a power of two or, more generally, smooth (Remark 9.10).

Theorem 9.35 (Cooley–Tukey radix-2 step). *Suppose $n = 2m$ and ω is a primitive n -th root of unity in $\mathbb{F}$. Given $f(X) = \sum_{k=0}^{n-1} a_k X^k$, split its coefficients by parity into*

$$f_{\text{even}}(Y) = \sum_{k=0}^{m-1} a_{2k} Y^k, \quad f_{\text{odd}}(Y) = \sum_{k=0}^{m-1} a_{2k+1} Y^k,$$

so that $f(X) = f_{\text{even}}(X^2) + X f_{\text{odd}}(X^2)$. Then $\eta := \omega^2$ is a primitive m -th root of unity, and for $j = 0, \dots, m-1$,

$$f(\omega^j) = f_{\text{even}}(\eta^j) + \omega^j f_{\text{odd}}(\eta^j), \quad (7)$$

$$f(\omega^{j+m}) = f_{\text{even}}(\eta^j) - \omega^j f_{\text{odd}}(\eta^j). \quad (8)$$

Thus the length- n NTT of a reduces to two length- m NTTs (of the even and odd coefficient subsequences, over the domain $\langle \eta \rangle$) plus $O(n)$ extra operations.

Proof. The identity $f(X) = f_{\text{even}}(X^2) + X f_{\text{odd}}(X^2)$ regroups the sum $\sum_k a_k X^k$ into even-index and odd-index terms; in the odd terms one factor of X is pulled out, leaving powers X^{2k} . That $\eta = \omega^2$ is a primitive m -th root of unity follows from Lemma 9.5 and the order-of-a-power formula (Corollary 3.23): $\text{ord}(\omega^2) = n / \gcd(2, n) = n/2 = m$, using $2 \mid n$.

It remains to evaluate. For (7), put $X = \omega^j$; then $X^2 = \omega^{2j} = \eta^j$, so $f(\omega^j) = f_{\text{even}}(\eta^j) + \omega^j f_{\text{odd}}(\eta^j)$. For (8), put $X = \omega^{j+m}$; then $X^2 = \omega^{2j+2m} = \omega^{2j} \omega^n = \eta^j \cdot 1 = \eta^j$ (since $2m = n$ and $\omega^n = 1$), while the linear factor is $\omega^{j+m} = \omega^j \omega^m = -\omega^j$. Here $\omega^m = -1$, because ω^m has order $n / \gcd(m, n) = n/m = 2$ (Corollary 3.23), and -1 is the unique element of order 2 in a field: $x^2 = 1$

with $x \neq 1$ forces $(x - 1)(x + 1) = 0$ and so $x = -1$; and $-1 \neq 1$, since $\text{char } F \nmid n$ (Theorem 9.13) with n even forces $\text{char } F \neq 2$. Substituting, $f(\omega^{j+m}) = f_{\text{even}}(\eta^j) - \omega^j f_{\text{odd}}(\eta^j)$. $\square$

Remark 9.36 (The butterfly). Equations (7)–(8) constitute the *butterfly*: from the two subtransform outputs $E_j = f_{\text{even}}(\eta^j)$ and $O_j = f_{\text{odd}}(\eta^j)$ one forms the *twiddle* $t_j = \omega^j O_j$ and reads off the two outputs $E_j + t_j$ and $E_j - t_j$. In a diagram, E_j and O_j are two nodes on the left, each joined to both outputs on the right, the two E_j -edges with weight 1 and the two O_j -edges with weights $+\omega^j$ and $-\omega^j$; the pair of crossing edges forms the $\times$ after which the operation is named. Each butterfly costs one multiplication (the twiddle) and two additions, and there are $n/2$ of them per level.

Theorem 9.37 ($O(n \log n)$ cost of the FFT). *Let $n = 2^t$ and let $T(n)$ be the number of field additions and multiplications the radix-2 recursion of Theorem 9.35 uses to compute a length- n NTT. Then*

$$T(n) = 2T(n/2) + cn \quad (n > 1), \quad T(1) = O(1),$$

for a constant c , and therefore $T(n) = O(n \log n)$. The inverse NTT has the same cost, since it is the forward NTT with ω replaced by ω^{-1} , followed by scaling each entry by $1/n$ (Theorem 9.29).

Proof. The recursion is exactly Theorem 9.35: a length- n transform calls two length- $n/2$ transforms (on the even and odd coefficient subsequences) and combines them with $n/2$ butterflies of $O(1)$ operations each, for the cn overhead. Unrolling, with $n = 2^t$,

$$T(2^t) = 2T(2^{t-1}) + c2^t.$$

Divide by 2^t and set $S(t) = T(2^t)/2^t$: then $S(t) = S(t-1) + c$, so $S(t) = S(0) + ct = O(1) + ct$, and $T(2^t) = 2^t S(t) = O(2^t) + ct2^t = O(t2^t)$. With $t = \log_2 n$ this is $T(n) = O(n \log n)$. (Equivalently: the recursion tree has $\log_2 n$ levels, each performing $\Theta(n)$ total work across all subproblems at that level.) The inverse-transform claim is Theorem 9.29: the inverse is the same Cooley–Tukey recursion with ω^{-1} in place of ω (also a primitive n -th root of unity), plus a final pass of n divisions by n , all within $O(n \log n)$. $\square$

Remark 9.38 (Mixed radix and smoothness). Fast transforms apply when n is smooth and the field contains a primitive n -th root of unity, that is, $n \mid q - 1$ (Theorem 9.7); for a power-of-two n dividing $q - 1$, every level of the recursion is a radix-2 step.

Example 9.39 (A length-4 transform). Take $n = 4$ with primitive 4-th root ω (so $\omega^2 = -1$ and $\omega^4 = 1$, and $\eta = \omega^2 = -1$ is the primitive 2nd root). For $f = a_0 + a_1X + a_2X^2 + a_3X^3$, the two length-2 subtransforms over the domain $\{1, -1\}$ —each itself a single butterfly, with twiddle $\eta^0 = 1$ —are

$$E = (a_0 + a_2, a_0 - a_2), \quad O = (a_1 + a_3, a_1 - a_3),$$

and the butterflies give

$$\hat{a}_0 = E_0 + \omega^0 O_0, \quad \hat{a}_2 = E_0 - \omega^0 O_0, \quad \hat{a}_1 = E_1 + \omega^1 O_1, \quad \hat{a}_3 = E_1 - \omega^1 O_1.$$

(Outputs j and $j + m$ are paired, as in (7)–(8).) This uses 2 subtransforms of 2 additions each plus 2 butterflies of 2 additions each ($n/2 = 2$ per level, Remark 9.36)—together 2 twiddle multiplications and 8 additions—versus 16 coefficient-products and 12 additions for the direct 4×4 matrix multiply; the difference grows with n .

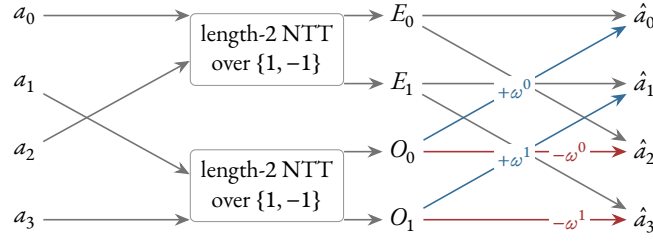


Figure 2. The length-4 transform of Example 9.39 as a circuit. The inputs split by parity into two length-2 subtransforms, whose outputs E_0, E_1 and O_0, O_1 feed the two butterflies of Remark 9.36: each E_j is carried to outputs $\hat{a}_j$ and $\hat{a}_{j+2}$ with weight 1, each O_j with weights $+\omega^j$ and $-\omega^j$ (Eq. (7)–Eq. (8)); the crossing pair of weighted edges is the $\times$ after which the butterfly is named.

9.8 The role of smooth multiplicative domains in polynomial proof systems

This subsection states how the preceding results are used in Halo 2, the proof system of Orchard, which encodes a computation as polynomial identities on the evaluation domain $H = \mu_n$: the n -step table described at the start of this section is indexed by the n points of H .

Remark 9.40 (The roles played by H, Z_H , the Lagrange basis, and the FFT).

- **Witnesses as evaluations on H .** A prover lays out the trace of a computation as the values of one or more polynomials on the n points of H . By Theorem 9.27 this is the same data as a polynomial of degree $< n$; the prover may move freely between the value (Lagrange) representation, suited to stating constraints row by row, and the coefficient representation, suited to committing and to low-degree testing. The change of representation is the (inverse) NTT.
- **Constraints as divisibility by Z_H .** “Constraint polynomial C holds on every row” means C vanishes on all of H , which by Theorem 9.13 means exactly that $Z_H = X^n - 1$ divides C . The prover demonstrates this by exhibiting a quotient t with $C(X) = t(X)(X^n - 1)$. The closed form $Z_H(X) = X^n - 1$, evaluated by one exponentiation, makes this check cheap at a random point. Proposition 9.14 extends the same technique to cosets γH , used to separate quotient and remainder domains.
- **Lagrange selectors.** The Lagrange polynomials L_i of Definition 9.20 are *selectors*: L_i is 1 on row i and 0 elsewhere (Proposition 9.21), so boundary and instance constraints (e.g. “the first row equals the public input”) become $L_0 \cdot (\dots)$. The closed form and the barycentric formula (Theorem 9.22, Remark 9.23) let the verifier evaluate these at a random challenge in $O(n)$ or even $O(\log n)$ field operations rather than reconstructing polynomials.
- **Group structure for permutations and shifts.** Because H is a cyclic group (Definition 9.11), multiplication by ω is the cyclic “next-row” shift $\omega^i \mapsto \omega^{i+1}$. Constraints relating a row to its neighbour become $f(\omega X)$ versus $f(X)$; permutation arguments, on which Halo 2’s copy constraints are built, rest on the action of the group on itself and on its cosets. An arbitrary set of n points has none of this structure.

- **Smoothness for speed.** Provers manipulate polynomials of size n (often after enlarging the domain by a small constant factor for low-degree testing). Transforms between representations, multiplications and coset evaluations are computed by NTTs, which by Theorem 9.37 cost $O(n \log n)$ instead of $O(n^2)$ (at the sizes used in practice, the difference between feasible and infeasible computation), provided n is smooth and $\mathbb{F}_q$ contains a primitive n -th root of unity. By Theorem 9.7 that requires $n \mid q - 1$; by Remark 9.10 this is why high-two-adicity primes are chosen for the scalar fields of the Pallas and Vesta curves (Section 10).

10 Elliptic curves

This section constructs elliptic-curve groups, the standard source of groups of roughly 2^{255} elements for which no discrete-logarithm attack faster than the generic $O(\sqrt{N})$ attacks of Remark 7.22 is known. The multiplicative groups $\mathbb{F}_p^\times$ lack this property: the *index-calculus* algorithms exploit the factorisation of the integers behind the residues and run in subexponential time, so p must have *thousands* of bits to push the cheapest known attack beyond 2^{128} operations. An *elliptic curve* is the solution set of a cubic equation in two variables together with one additional point, the “point at infinity”. The section proves that these points form an abelian group under a geometric rule (Theorem 10.16, associativity in sketch), derives explicit addition formulas, including complete ones valid for every input pair, bounds the group order by Hasse’s theorem (Theorem 10.22), and states the elliptic-curve discrete logarithm problem.

Throughout, K is a field whose characteristic (Definition 4.30) is neither 2 nor 3; in the applications K is a prime field $\mathbb{F}_p$ with p large. The running example is the pair of *Pasta curves*

$$E_p : y^2 = x^3 + 5 \quad \text{over } \mathbb{F}_p, \quad E_q : y^2 = x^3 + 5 \quad \text{over } \mathbb{F}_q,$$

where p and q are the two 255-bit primes of Remark 6.23 and Remark 7.18,

$$\begin{aligned} p &= 2^{254} + 0x224698fc094cf91b992d30ed00000001, \\ q &= 2^{254} + 0x224698fc0994a8dd8c46eb2100000001. \end{aligned}$$

The primes and the curve equations are the deployed constants, specified in §5.4.9.6 (“Pallas and Vesta”) of the Zcash protocol specification. The curve E_p is called *Pallas* and E_q *Vesta*. By design they form a *cycle*: the number of $\mathbb{F}_p$ -points of Pallas equals q , and the number of $\mathbb{F}_q$ -points of Vesta equals p . A toy curve over $\mathbb{F}_{11}$ is treated in parallel for hand computation.

10.1 Weierstrass equations

Definition 10.1 (Short Weierstrass equation). Let K be a field of characteristic $\neq 2, 3$, and let $a, b \in K$. The *short Weierstrass equation* with coefficients a, b is the equation

$$y^2 = x^3 + ax + b. \tag{9}$$

The elements a and b are the *Weierstrass coefficients*.

Over a field of characteristic $\neq 2, 3$, an invertible affine change of variables reduces the most general cubic of this shape, $y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6$, to the form (9): completing the square in y (possible since $2 \neq 0$) removes the xy and y terms but changes the quadratic coefficient to $a_2 + a_1^2/4$; translating with the substitution $x = X - (a_2 + a_1^2/4)/3$ (possible since $3 \neq 0$) removes the X^2 term. Characteristics 2 and 3 are excluded for this reason: those cases require the longer Weierstrass form, and they do not arise for the Pasta curves, which are defined over large prime fields.

Cubics whose graph has a self-intersection or a cusp are excluded, because the group law defined below fails at such a singular point.

Definition 10.2 (Discriminant and nonsingularity). The *discriminant* of the short Weierstrass equation (9) is

$$\Delta = -16(4a^3 + 27b^2) \in K. \quad (10)$$

The equation, and the cubic $f(x) = x^3 + ax + b$, is *nonsingular* (or *smooth*) if $\Delta \neq 0$, equivalently $4a^3 + 27b^2 \neq 0$. (The factor -16 is a convention inherited from the general Weierstrass form; over the fields considered here it is nonzero and does not affect the condition.)

The following lemma expresses $4a^3 + 27b^2$ through the roots of a *depressed cubic* (a cubic with no square term); the smoothness criterion is derived from it.

Lemma 10.3 (Discriminant of a depressed cubic). Let $f(x) = x^3 + ax + b = (x - r_1)(x - r_2)(x - r_3)$ with r_1, r_2, r_3 in some field containing K . Then

$$\prod_{i < j} (r_i - r_j)^2 = -(4a^3 + 27b^2).$$

Proof. Expanding $(x - r_1)(x - r_2)(x - r_3)$ and comparing coefficients with $x^3 + ax + b$ gives Vieta's relations for the depressed cubic:

$$r_1 + r_2 + r_3 = 0, \quad r_1r_2 + r_1r_3 + r_2r_3 = a, \quad r_1r_2r_3 = -b.$$

Differentiating $f(x) = \prod_i (x - r_i)$ by the product rule and evaluating at a root annihilates every term but one: $f'(r_i) = \prod_{j \neq i} (r_i - r_j)$. Multiplying the three evaluations pairs each factor $(r_i - r_j)$ with $(r_j - r_i)$:

$$\prod_i f'(r_i) = \prod_{i < j} (r_i - r_j)(r_j - r_i) = (-1)^3 \prod_{i < j} (r_i - r_j)^2.$$

It remains to compute $\prod_i f'(r_i) = \prod_i (3r_i^2 + a)$. Expanding the product and collecting the elementary symmetric functions of the squares r_i^2 ,

$$\prod_i (3r_i^2 + a) = 27 \prod_i r_i^2 + 9a \sum_{i < j} r_i^2 r_j^2 + 3a^2 \sum_i r_i^2 + a^3.$$

Vieta's relations evaluate each symmetric function: with $e_1 = 0$, $e_2 = a$, $e_3 = -b$,

$$\sum_i r_i^2 = e_1^2 - 2e_2 = -2a, \quad \sum_{i < j} r_i^2 r_j^2 = e_2^2 - 2e_1 e_3 = a^2, \quad \prod_i r_i^2 = e_3^2 = b^2.$$

Substituting, $\prod_i f'(r_i) = 27b^2 + 9a^3 - 6a^3 + a^3 = 4a^3 + 27b^2$, and the two displays together give $\prod_{i < j} (r_i - r_j)^2 = -(4a^3 + 27b^2)$. $\square$

Proposition 10.4 (Equivalent conditions for smoothness). *Let $f(x) = x^3 + ax + b \in K[x]$. The following are equivalent.*

1. *The quantity $4a^3 + 27b^2$ of Definition 10.2 is nonzero.*
2. *The cubic f has no repeated root in the algebraic closure $\overline{K}$ —the smallest extension field of K in which every nonconstant polynomial over K factors into linear factors.*
3. *The polynomials f and f' are coprime in $\overline{K}[x]$.*
4. *There is no point $(x_0, y_0) \in \overline{K}^2$ satisfying the curve equation $y^2 = f(x)$ together with the two partial-derivative equations $2y = 0$ and $f'(x) = 0$.*

Proof. Equivalence (1) $\Leftrightarrow$ (2): over $\overline{K}$ the monic cubic factors as $f = (x - r_1)(x - r_2)(x - r_3)$, and Lemma 10.3 gives $\prod_{i < j} (r_i - r_j)^2 = -(4a^3 + 27b^2)$. The left side vanishes exactly when two roots coincide, so f has a repeated root if and only if $4a^3 + 27b^2 = 0$.

Equivalence (2) $\Leftrightarrow$ (3): this is the derivative test of Theorem 5.40, applied over $\overline{K}$ where f splits: a repeated root of f is a common root of f and f' , hence a common factor $(x - \alpha)$, and conversely a nontrivial $\gcd(f, f')$ has a root in $\overline{K}$ which is then a repeated root of f .

Equivalence (4) $\Leftrightarrow$ (2): a singular point of the affine curve $y^2 = f(x)$ is a point where both partial derivatives of $g(x, y) = y^2 - f(x)$ vanish, namely $\partial g / \partial y = 2y = 0$ and $\partial g / \partial x = -f'(x) = 0$. Since $\text{char}(K) \neq 2$, the first equation forces $y_0 = 0$; the curve equation then gives $f(x_0) = 0$, which combined with $f'(x_0) = 0$ says x_0 is a repeated root of f (Theorem 5.40(1)). Conversely a repeated root x_0 of f yields the singular point $(x_0, 0)$. $\square$

Definition 10.5 (Elliptic curve and rational points). *An elliptic curve over K is a short Weierstrass equation (9) with nonzero discriminant, $4a^3 + 27b^2 \neq 0$. For any field extension $L \supseteq K$, the set of L -rational affine points is*

$$E^{\text{aff}}(L) = \{ (x, y) \in L \times L : y^2 = x^3 + ax + b \},$$

and the set of L -rational points is

$$E(L) = E^{\text{aff}}(L) \cup \{\mathcal{O}\},$$

where $\mathcal{O}$ is a single formal symbol called the *point at infinity*. When $L = K$ one writes E for the curve and $E(K)$ for its rational points.

Remark 10.6 (The Pasta curves are smooth). For the Pasta curves $a = 0$ and $b = 5$, so $4a^3 + 27b^2 = 27 \cdot 25 = 675 = 3^3 \cdot 5^2$. The curve is therefore nonsingular whenever the characteristic does not divide 675, i.e. whenever the prime is neither 3 nor 5; the Pasta primes are far larger than 5, so the condition holds. More generally, any curve $y^2 = x^3 + b$ with $b \neq 0$ is nonsingular away from characteristics 2 and 3.

10.2 The point at infinity, geometrically

The point $\mathcal{O}$ is the identity element of the group law defined below. To exhibit it as a point of the curve, embed the affine plane in the *projective plane*. A point of the projective plane $\mathbb{P}^2(K)$ is an equivalence class $[X : Y : Z]$ of nonzero triples $(X, Y, Z) \in K^3 \setminus \{0\}$, where $(X, Y, Z) \sim (\lambda X, \lambda Y, \lambda Z)$ for every $\lambda \in K^\times$. The affine points (x, y) embed as $[x : y : 1]$; the points with $Z = 0$ form the *line at infinity*, the extra “directions”. Homogenising the Weierstrass equation (9) by setting $x = X/Z, y = Y/Z$ and clearing denominators gives the *projective Weierstrass equation*

$$Y^2Z = X^3 + aXZ^2 + bZ^3. \quad (11)$$

Setting $Z = 0$ in (11) forces $X^3 = 0$, hence $X = 0$, leaving the *single* projective point $[0 : 1 : 0]$. This is the point at infinity $\mathcal{O}$. It is the point at infinity in the vertical direction, and every vertical line $x = \text{const}$ passes through it.

10.3 The chord-and-tangent group law: geometry

The geometric definition of addition rests on one fact: a line meets a cubic curve in exactly three points, counted with multiplicity, once the point at infinity is included.

Definition 10.7 (The group law, geometric form). Let $P, Q \in E(K)$. Define $P + Q$ as follows.

- If $P = \mathcal{O}$, set $P + Q = Q$; if $Q = \mathcal{O}$, set $P + Q = P$.
- Otherwise $P = (x_1, y_1)$ and $Q = (x_2, y_2)$ are affine. Draw the line ℓ through P and Q ; if $P = Q$, let ℓ be the tangent line to the curve at P . The line ℓ meets the curve in a third point R' , counted with multiplicity, where a vertical ℓ has third point $\mathcal{O}$. Define $P + Q$ to be the *reflection of R' across the x -axis*: if $R' = (x_3, -y_3)$ then $P + Q = (x_3, y_3)$, and if $R' = \mathcal{O}$ then $P + Q = \mathcal{O}$.

The reflection turns the rule that *the three intersection points of any line with the curve add up to $\mathcal{O}$* into a group law with identity $\mathcal{O}$: reflection across the x -axis sends an affine point (x, y) to $(x, -y)$, its additive inverse by the next subsection, and fixes $\mathcal{O}$. The rule is stated here as a heuristic; every use of it below is discharged by explicit computation with the formulas of §10.4, which obtain the third intersection point from Vieta’s relations, as in Lemma 10.3.

10.4 Explicit affine formulas

Fix an elliptic curve $E : y^2 = x^3 + ax + b$ over K , and let $P = (x_1, y_1)$ and $Q = (x_2, y_2)$ denote affine points.

Definition 10.8 (Negation). Define

$$-\mathcal{O} = \mathcal{O}, \quad -(x_1, y_1) = (x_1, -y_1).$$

The reflected point $(x_1, -y_1)$ is again on the curve, since the curve equation depends on y only through y^2 . Geometrically, the line through P and $-P$ is the vertical line $x = x_1$, which meets the curve at $P, -P$, and $\mathcal{O}$.

Construction 10.9 (Chord addition). Suppose $P, Q \neq \mathcal{O}$ and $x_1 \neq x_2$. The line through P and Q is $y = \lambda x + \mu$ with

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \quad \mu = y_1 - \lambda x_1.$$

Substituting the line into the curve equation, $(\lambda x + \mu)^2 = x^3 + ax + b$, and collecting terms gives

$$g(x) := x^3 - \lambda^2 x^2 + (a - 2\lambda\mu)x + (b - \mu^2) = 0.$$

Both x_1 and x_2 are roots of the monic cubic g : each point lies on the line and on the curve, so $g(x_i) = f(x_i) - (\lambda x_i + \mu)^2 = f(x_i) - y_i^2 = 0$. Since $x_1 \neq x_2$, factoring out $(x - x_1)(x - x_2)$ (Theorem 5.17, applied twice) leaves a monic linear factor $x - x_3$ with $x_3 \in K$: the x -coordinate of the third intersection point R' . By Vieta, the sum of the roots equals the negative of the x^2 -coefficient—so $x_1 + x_2 + x_3 = \lambda^2$, which determines the third root without factoring. Hence

$$x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1, \quad (12)$$

and $P + Q = (x_3, y_3)$. The y -formula already incorporates the reflection across the x -axis: the third intersection point is $(x_3, \lambda x_3 + \mu)$, and reflecting gives $y_3 = -(\lambda x_3 + \mu) = -(\lambda x_3 + y_1 - \lambda x_1) = \lambda(x_1 - x_3) - y_1$.

Construction 10.10 (Tangent doubling). Suppose $P = Q = (x_1, y_1)$ with $y_1 \neq 0$. Implicit differentiation of $y^2 = x^3 + ax + b$ yields the tangent slope: from $2y dy = (3x^2 + a) dx$,

$$\lambda = \frac{3x_1^2 + a}{2y_1}.$$

With $\mu = y_1 - \lambda x_1$, substituting $y = \lambda x + \mu$ into the curve equation produces the same monic cubic $g(x) = x^3 + ax + b - (\lambda x + \mu)^2$ as in Construction 10.9, and here x_1 is a *double* root. The statement that “tangency counts the intersection twice” is verified directly: $g(x_1) = f(x_1) - y_1^2 = 0$, and

$$g'(x_1) = 3x_1^2 + a - 2\lambda(\lambda x_1 + \mu) = 3x_1^2 + a - 2\lambda y_1 = 0$$

by the choice of λ , so x_1 is a repeated root of g (Theorem 5.40(1)). Factoring $(x - x_1)^2$ out of g leaves a monic linear factor $x - x_3$ with $x_3 \in K$, and Vieta now reads $x_1 + x_1 + x_3 = \lambda^2$, hence

$$x_3 = \lambda^2 - 2x_1, \quad y_3 = \lambda(x_1 - x_3) - y_1, \quad [2]P = (x_3, y_3). \quad (13)$$

The remaining cases are exactly those in which the line is vertical.

- If $P = \mathcal{O}$, then $P + Q = Q$; symmetrically if $Q = \mathcal{O}$.
- If $x_1 = x_2$ but $y_1 = -y_2$ (so $Q = -P$, including the subcase $y_1 = y_2 = 0$), the line is vertical, the third point is $\mathcal{O}$, and $P + Q = \mathcal{O}$.
- If $P = Q$ and $y_1 = 0$, the tangent is vertical (the slope formula in Construction 10.10 has zero denominator), so $[2]P = \mathcal{O}$. These are exactly the points of order two, studied in Section 10.9.

Every case is covered: either an input is $\mathcal{O}$; or both inputs are affine with $x_1 \neq x_2$ (use (12)); or both are affine with $x_1 = x_2$, in which case either $y_1 = -y_2$ (result $\mathcal{O}$) or $y_1 = y_2 \neq 0$ (use (13)).

Proposition 10.11 (Closure). *For all $P, Q \in E(K)$, the point $P + Q$ computed by the case analysis above lies in $E(K)$. In particular $E(K)$ is closed under $+$.*

Proof. If either input is $\mathcal{O}$, the result is the other input, which lies in $E(K)$ by hypothesis; if the result is $\mathcal{O}$, it lies in $E(K)$ by definition. Otherwise (x_3, y_3) is produced by (12) or (13), and in both constructions x_3 was exhibited as a root of the cubic $g(x) = x^3 + ax + b - (\lambda x + \mu)^2$ —as the third root beside x_1, x_2 in Construction 10.9, and beside the double root x_1 in Construction 10.10. The equation $g(x_3) = 0$ states precisely that the third intersection point $(x_3, \lambda x_3 + \mu) = (x_3, -y_3)$ satisfies the curve equation; and since the curve equation is invariant under $y \mapsto -y$, the reflected point (x_3, y_3) is on the curve too, so $P + Q \in E^{\text{aff}}(K) \subseteq E(K)$. For rationality: all coordinates are rational expressions in x_1, x_2, y_1, y_2, a whose denominators are $x_2 - x_1$ or $2y_1$, nonzero in the cases where each formula is invoked; so $x_3, y_3 \in K$. $\square$

Remark 10.12 (Complete versus incomplete addition). The chord formula (12) is *incomplete*: it fails whenever $x_1 = x_2$ —the doubling and inverse cases—and an implementation must branch into the case analysis above. *Complete* addition formulas, valid for *all* input pairs including $P = Q$ and $P = -Q$, exist for the curves used here, at the cost of more field operations per addition; Construction 10.13 exhibits the deployed set and Proposition 10.14 proves it complete. Later volumes depend on the distinction. Halo 2’s elliptic-curve gadgets provide complete addition and use incomplete gates only where their callers justify the exclusions. The Sinsemilla hash of Orchard deliberately uses the *incomplete* formulas nondeterministically: an exceptional case does not violate the constraints but leaves the affected output unconstrained (the prover may choose the intermediate slope freely), which the specification models by letting the hash return $\perp$ and weakening the proved statement accordingly; its Merkle path validity check may pass on the resulting sentinel values. Security follows not from the proof system but from Theorem 5.4.4 of the Zcash protocol specification: any input reaching an exceptional case of the incomplete addition yields a nontrivial discrete-logarithm relation among the Sinsemilla generators, so under discrete-log hardness on Pallas (Remark 10.31) such inputs are infeasible to find, for honest and adversarial provers alike.

A circuit cannot branch. A proof system checks the group law by a fixed list of polynomial identities in the input, output, and auxiliary cells of a row, evaluated on every row alike, so each branch of the case analysis appears as a factor that vanishes exactly when the branch is not taken. The deployed gate uses five auxiliary cells, four of them “inverse-or-zero” hints that let a polynomial test a field element for zero.

Construction 10.13 (Complete addition as deployed). Let $E : y^2 = x^3 + b$ over $\mathbb{F}_p$ with $p > 3$, and encode the point at infinity as the pair $(0, 0)$, so that every element of $E(\mathbb{F}_p)$ is a pair of field elements. For $z \in \mathbb{F}_p$ let $\text{inv}_0(z)$ be z^{-1} if $z \neq 0$ and 0 if $z = 0$. For encoded inputs $P = (x_p, y_p)$

and $Q = (x_q, y_q)$ the prover fills five auxiliary cells,

$$\alpha = \text{inv}_0(x_q - x_p), \quad \beta = \text{inv}_0(x_p), \quad \gamma = \text{inv}_0(x_q), \quad \delta = \begin{cases} \text{inv}_0(y_q + y_p) & \text{if } x_q = x_p, \\ 0 & \text{otherwise,} \end{cases}$$

$$\lambda = \begin{cases} (y_q - y_p) \alpha & \text{if } x_q \neq x_p, \\ 3x_p^2 (2y_p)^{-1} & \text{if } x_q = x_p \text{ and } y_p \neq 0, \\ 0 & \text{otherwise,} \end{cases}$$

and an output pair (x_r, y_r) : it is Q if $x_p = 0$; else P if $x_q = 0$; else $(0, 0)$ if $x_q = x_p$ and $y_q = -y_p$; else $x_r = \lambda^2 - x_p - x_q$ and $y_r = \lambda(x_p - x_r) - y_p$. The verifier accepts the row iff the twelve identities

$$(x_q - x_p)((x_q - x_p)\lambda - (y_q - y_p)) = 0, \quad (\text{C1})$$

$$(1 - (x_q - x_p)\alpha)(2y_p\lambda - 3x_p^2) = 0, \quad (\text{C2})$$

$$x_p x_q (x_q - x_p) (\lambda^2 - x_p - x_q - x_r) = 0, \quad (\text{C3a})$$

$$x_p x_q (x_q - x_p) (\lambda(x_p - x_r) - y_p - y_r) = 0, \quad (\text{C3b})$$

$$x_p x_q (y_q + y_p) (\lambda^2 - x_p - x_q - x_r) = 0, \quad (\text{C3c})$$

$$x_p x_q (y_q + y_p) (\lambda(x_p - x_r) - y_p - y_r) = 0, \quad (\text{C3d})$$

$$(1 - x_p \beta) (x_r - x_q) = 0, \quad (\text{C4a})$$

$$(1 - x_p \beta) (y_r - y_q) = 0, \quad (\text{C4b})$$

$$(1 - x_q \gamma) (x_r - x_p) = 0, \quad (\text{C5a})$$

$$(1 - x_q \gamma) (y_r - y_p) = 0, \quad (\text{C5b})$$

$$(1 - (x_q - x_p)\alpha - (y_q + y_p)\delta) x_r = 0, \quad (\text{C6a})$$

$$(1 - (x_q - x_p)\alpha - (y_q + y_p)\delta) y_r = 0 \quad (\text{C6b})$$

hold in $\mathbb{F}_p$. These are, in this order, the twelve constraints of the gate complete addition in the `halo2_gadgets` crate, each carrying there one further factor, a per-row cell equal to 1 on the rows where the gate applies and 0 elsewhere, which switches the list on or off; the crate assigns the cells as above, and the circuit field is the Pallas base field $\mathbb{F}_p$.

Proposition 10.14 (The gate is complete and sound). *Suppose no affine point of $E(\mathbb{F}_p)$ has $x = 0$ and none has $y = 0$ —for $y^2 = x^3 + b$, that b is a non-square and $-b$ a non-cube in $\mathbb{F}_p$. Then for all $P, Q \in E(\mathbb{F}_p)$, encoded as in Construction 10.13:*

1. (completeness) *the prover's cells satisfy (C1)–(C6b), and the output pair encodes $P + Q$;*
2. (soundness) *every $(\lambda, \alpha, \beta, \gamma, \delta, x_r, y_r) \in \mathbb{F}_p^7$ satisfying (C1)–(C6b) has (x_r, y_r) equal to the encoding of $P + Q$.*

This is what complete means for a gate: the twelve identities are satisfiable for every input pair—doublings, inverse pairs, and the identity included—and the output they admit is unique, so they

are the graph of the total function $+: E(\mathbb{F}_p) \times E(\mathbb{F}_p) \rightarrow E(\mathbb{F}_p)$, with no case left to a branch outside the constraint system.

Proof. Under the hypothesis an encoded pair has $x = 0$ iff it is $\mathcal{O}$, and two affine points with the same x -coordinate have $y_q = \pm y_p$ with $y_p \neq 0$, so they are equal or inverse and not both. Every case uses one observation: for $z \in \mathbb{F}_p$ and any cell value b , the factor $1 - zb$ equals 1 when $z = 0$, whatever b is, while for $z \neq 0$ the honest choice $b = z^{-1}$ makes it 0. A constraint of the shape $(1 - zb)t = 0$ therefore forces $t = 0$ exactly when $z = 0$ and is discharged by the honest hint otherwise. Five cases exhaust the input pairs.

Case $P = \mathcal{O}$. Then $x_p = 0$, so (C4a)–(C4b) force $(x_r, y_r) = (x_q, y_q)$, the encoding of $\mathcal{O} + Q = Q$; that is soundness. The prover outputs Q by the first clause, and its cells satisfy the rest: (C1) reads $x_q(x_q\lambda - y_q) = 0$ and holds for $\lambda = y_q \text{ inv}_0(x_q)$; (C2) and (C3a)–(C3d) carry a factor x_p or $2y_p\lambda - 3x_p^2$, both zero; (C5a)–(C5b) are discharged by $\gamma = x_q^{-1}$ if $Q \neq \mathcal{O}$, and read $(x_r, y_r) = (0, 0) = (x_p, y_p)$ if $Q = \mathcal{O}$; and (C6a)–(C6b) have first factor $1 - x_q\alpha = 0$, which is 0 for $\alpha = x_q^{-1}$ when $Q \neq \mathcal{O}$, while for $Q = \mathcal{O}$ the other factor $x_r = y_r = 0$ vanishes.

Case $Q = \mathcal{O}, P \neq \mathcal{O}$. Then $x_q = 0 \neq x_p$, so (C5a)–(C5b) force $(x_r, y_r) = (x_p, y_p)$, the encoding of P : soundness. The prover outputs P by the second clause, with $\alpha = (-x_p)^{-1}$ and $\lambda = (-y_p)\alpha = y_p/x_p$; then (C1) reads $(-x_p)(-x_p\lambda + y_p) = 0$ and holds; (C2) and (C6a)–(C6b) carry the factor $1 - (x_q - x_p)\alpha = 0$, the latter with $\delta = 0$; (C3a)–(C3d) carry $x_q = 0$; and (C4a)–(C4b) are discharged by $\beta = x_p^{-1}$.

Case P, Q affine with $x_q \neq x_p$. Constraint (C1) forces λ to be the chord slope $(y_q - y_p)/(x_q - x_p)$, and since $x_p x_q (x_q - x_p) \neq 0$, (C3a)–(C3b) force (x_r, y_r) to be the chord formula (12)—the encoding of $P + Q$, which is affine because $x_p \neq x_q$. That is soundness. The prover's λ is that slope and its output that formula, by the fourth clause; (C3c)–(C3d) contain the same two vanishing factors; (C2), (C4a)–(C4b), (C5a)–(C5b) are discharged by the honest α, β, γ ; and (C6a)–(C6b) have first factor $1 - 1 - 0 = 0$, since $\delta = 0$.

Case $Q = -P, P$ affine. Here $x_q = x_p$ and $y_q + y_p = 0$, so both hint-bearing products in (C6a)–(C6b) are 0 whatever α and δ are, and the constraints force $x_r = y_r = 0$: the encoding of $P + (-P) = \mathcal{O}$, which is soundness. The prover outputs $(0, 0)$ by the third clause; (C1) and (C3a)–(C3b) carry $x_q - x_p = 0$, and (C3c)–(C3d) carry $y_q + y_p = 0$; (C2), whose first factor is now 1, needs $2y_p\lambda = 3x_p^2$, which the honest $\lambda = 3x_p^2(2y_p)^{-1}$ supplies ($y_p \neq 0$); and (C4a)–(C5b) are discharged by $\beta = \gamma = x_p^{-1}$.

Case $Q = P$ affine. Again $x_q = x_p$ fixes $(x_q - x_p)\alpha = 0$, so (C2) forces λ to be the tangent slope $3x_p^2/(2y_p)$, and since $x_p x_q (y_q + y_p) = 2x_p^2 y_p \neq 0$, (C3c)–(C3d) force (x_r, y_r) to be the doubling formula (13)—the encoding of $[2]P$, affine because $y_p \neq 0$. That is soundness. The prover's λ is that slope and its output that formula, by the fourth clause; (C1) and (C3a)–(C3b) carry $x_q - x_p = 0$; (C4a)–(C5b) are discharged by $\beta = \gamma = x_p^{-1}$; and (C6a)–(C6b) have first factor $1 - 0 - 2y_p\delta = 0$ for the honest $\delta = (2y_p)^{-1}$. $\square$

Example 10.15 (The gate on the toy curve). Pallas satisfies both hypotheses of Proposition 10.14: 5 is a non-square in $\mathbb{F}_p$ (Example 7.16) and -5 is not a cube (Remark 10.26). The toy curve $y^2 = x^3 + 5$ over $\mathbb{F}_{11}$ of Example 10.38 satisfies neither— $5 = 4^2$ gives the points $(0, \pm 4)$, and $(8, 0)$ has order 2—so on it the gate can be examined both on inputs satisfying the hypotheses and on

inputs violating them. For each input pair considered below, a script determines every output pair (x_r, y_r) that *some* assignment of the five auxiliary cells in $\mathbb{F}_{11}$ makes the twelve identities accept—the search over $\alpha, \beta, \gamma, \delta$ factored by which identities mention each: β only (C4), γ only (C5), α and δ only (C2) and (C6)—so soundness is checked against all provers, not only the honest one. On the nine points $\mathcal{O}, (4, \pm 5), (5, \pm 3), (6, \pm 1), (10, \pm 2)$ with $x, y \neq 0$, all 81 ordered pairs—eight doublings, eight inverse pairs, seventeen involving $\mathcal{O}$ —admit exactly one output, the encoding of the true sum. Three of them, worked: for $[2](5, 3)$ the honest cells are $\lambda = 3 \cdot 5^2 \cdot 6^{-1} = 9 \cdot 2 = 7$ and $\delta = 6^{-1} = 2$, giving $(x_r, y_r) = (49 - 10, 7(5 - 6) - 3) = (6, 1)$, the value of (13); for the inverse pair $(5, 3) + (5, 8)$ the prover supplies the same $\lambda = 7$ but $\delta = \text{inv}_0(11) = 0$, and (C6a)–(C6b) force $(0, 0)$; for $\mathcal{O} + (5, 3)$ the cells are $\alpha = 5^{-1} = 9$, $\lambda = 3 \cdot 9 = 5$, $\beta = 0$, and (C4a)–(C4b) force $(5, 3)$. Off the admissible set the gate fails as the proof predicts. With $P = (0, 4)$ and $Q = (5, 3)$ the identities admit only the output $(5, 3)$ —the coordinate $x_p = 0$ is read as $\mathcal{O}$ —whereas $P + Q = (10, 9)$ (Example 10.38); the pair $(0, 4) + (0, 7)$ admits no output at all, (C4b) demanding $y_r = 7$ and (C5b) $y_r = 4$; and $(8, 0) + (8, 0)$ admits none either, since (C2) reduces to $3 \cdot 8^2 = 0$, false in $\mathbb{F}_{11}$. On Pallas none of the three inputs exists, and the gate is complete.

10.5 Identity, inverses, commutativity, and associativity

Theorem 10.16 (Group law). *Let E be an elliptic curve over a field K . The set $E(K)$ with the chord-and-tangent operation $+$ of Definition 10.7 is an abelian group with identity $\mathcal{O}$.*

Closure is Proposition 10.11. The next proposition establishes every remaining axiom except associativity.

Proposition 10.17 (Identity, inverses, commutativity). *The operation $+$ on $E(K)$ has the following properties.*

1. *The point $\mathcal{O}$ is a two-sided identity: $P + \mathcal{O} = \mathcal{O} + P = P$ for all P .*
2. *Every P has the two-sided inverse $-P$ of Definition 10.8: $P + (-P) = \mathcal{O}$.*
3. *The operation is commutative: $P + Q = Q + P$.*

Proof. Item (1) is the first clause of Definition 10.7. For item (2), let $P = (x, y)$ be affine. If $y \neq 0$, the line through P and $-P = (x, -y)$ is vertical, so its third point is $\mathcal{O}$ and $P + (-P) = \mathcal{O}$ by the vertical-line case; if $y = 0$, then $P = -P$ and $[2]P = \mathcal{O}$ directly by the vertical-tangent case; and $\mathcal{O} + (-\mathcal{O}) = \mathcal{O} + \mathcal{O} = \mathcal{O}$. For item (3), the chord through P and Q does not depend on the order in which the two points are named, and neither does the tangent in the doubling case; algebraically, the formulas (12) are symmetric under the swap $(x_1, y_1) \leftrightarrow (x_2, y_2)$ —the slope λ is unchanged, and $x_3 = \lambda^2 - x_1 - x_2$ is symmetric in x_1, x_2 . $\square$

The remaining axiom is *associativity*, $(P + Q) + R = P + (Q + R)$. A direct verification through the explicit formulas divides into many coordinate cases, each a lengthy rational-function identity; we sketch the conceptual proof instead.

Proof of associativity (sketch). The proof uses the classical *Cayley–Bacharach theorem* for plane cubics: if two cubics with no common component meet in nine points counted with intersection multiplicity, then any third cubic passing through eight of them with the required multiplicities also passes through the ninth. To prove $(P + Q) + R = P + (Q + R)$, one writes both sides via the chord-and-tangent construction and arranges the relevant intersections among the curve E and two degenerate cubics, each a product of three suitably chosen lines. Cayley–Bacharach forces the two candidate sums to coincide; its intersection-multiplicity formulation includes tangencies and coincident points. Replacing “reflect across the x -axis” by “the three points of any line sum to $\mathcal{O}$ ” makes the bookkeeping uniform.

A complete, case-free treatment identifies $E(K)$ with the degree-zero part $\text{Pic}^0(E)$ of the divisor class group of the curve via the map $P \mapsto [P] - [\mathcal{O}]$, a bijection that intertwines the two operations; the group law on $\text{Pic}^0(E)$ is manifestly associative, because it is inherited from addition of divisors modulo principal divisors. Either route yields a complete proof; both lie beyond the elementary scope of this volume, so we take associativity as established. $\square$

Remark 10.18 (The notation $[n]P$). For $n \in \mathbb{Z}$ and $P \in E(K)$, we write $[n]P$ for the n -fold sum $P + \dots + P$, with $[0]P = \mathcal{O}$ and $[-n]P = -([n]P)$. This makes the abelian group $E(K)$ a $\mathbb{Z}$ -module: integers act on points. The map $n \mapsto [n]P$ is the *scalar multiplication* of Definition 10.20—not any coordinate-wise product.

10.6 Projective and Jacobian coordinates

The affine formulas (12) and (13) each require a field *inversion*, by $x_2 - x_1$ or $2y_1$, which in a large prime field costs roughly as much as dozens of multiplications, whether computed by the extended Euclidean algorithm (Construction 6.1) or as z^{p-2} by Fermat (Construction 6.4). Over a long chain of group operations, as in scalar multiplication, it is cheaper to carry points in a redundant coordinate system whose group law uses only additions, subtractions, and multiplications, and to invert once at the end.

Remark 10.19 (Inversion-free coordinate systems). Two redundant systems serve this purpose. In *(homogeneous) projective coordinates*, a triple $[X : Y : Z]$ with $Z \neq 0$ represents the affine point $(X/Z, Y/Z)$ on the projective curve (11), with $\mathcal{O} = [0 : 1 : 0]$. In *Jacobian coordinates*, a triple $(X : Y : Z)$ with $Z \neq 0$ represents $(X/Z^2, Y/Z^3)$, under the equivalence $(X, Y, Z) \sim (\lambda^2 X, \lambda^3 Y, \lambda Z)$; the curve equation becomes $Y^2 = X^3 + aXZ^4 + bZ^6$. In either system addition and doubling need no divisions, at the cost of more multiplications per operation; the final recovery of affine coordinates computes Z^{-1} once, then the needed powers of it by multiplication. Jacobian coordinates usually yield the faster doubling and are the standard choice in cryptographic libraries; the `pasta_curves` crate used by Halo 2 carries its curve points in them. For curves $y^2 = x^3 + b$ with $a = 0$, the term aZ^4 in the Jacobian doubling formulas vanishes, which lowers the cost of doubling; this is one reason the Pasta curves take $a = 0$.

10.7 Scalar multiplication and double-and-add

Scalar multiplication computes, for a point P and an integer $n \geq 0$,

$$[n]P = \underbrace{P + P + \dots + P}_{n \text{ times}}.$$

Computing this by $n - 1$ successive additions is infeasible when n has hundreds of bits. Double-and-add uses the binary expansion of n instead; it is the additive form of square-and-multiply (Construction 6.2), anticipated in Remark 6.3.

Definition 10.20 (Double-and-add). If $n = 0$, return $\mathcal{O}$. Otherwise write $n = \sum_{i=0}^{k-1} n_i 2^i$ with digits $n_i \in \{0, 1\}$ and top digit $n_{k-1} = 1$. The *left-to-right double-and-add* algorithm computes $[n]P$ as follows: set $R \leftarrow P$; for $i = k - 2$ down to 0, set $R \leftarrow [2]R$, and if $n_i = 1$ set $R \leftarrow R + P$; return R .

Proposition 10.21. For $n \geq 1$, *double-and-add* returns $[n]P$ using exactly $k - 1$ doublings and at most $k - 1$ additions, where $k = \lfloor \log_2 n \rfloor + 1$ is the bit length of n . For $n = 0$, it returns $\mathcal{O}$ without a group operation.

Proof. The case $n = 0$ is immediate. Let $n \geq 1$, and let R_j denote the value of R after bits $n_{k-1}, \dots, n_j$ have been incorporated. We claim $R_j = \lfloor [n/2^j] \rfloor P$, by downward induction on j . At initialisation, $R_{k-1} = P = \lfloor [n/2^{k-1}] \rfloor P$ because the top bit is 1. Assuming $R_{j+1} = \lfloor [n/2^{j+1}] \rfloor P$, the next iteration computes

$$[2]R_{j+1} + [n_j]P = [2\lfloor [n/2^{j+1}] \rfloor + n_j]P = \lfloor [n/2^j] \rfloor P,$$

where the last step is the digit identity $2\lfloor [n/2^{j+1}] \rfloor + n_j = \lfloor [n/2^j] \rfloor$: stripping the last bit of $\lfloor [n/2^j] \rfloor$ and appending it again recovers the number. At $j = 0$ the invariant reads $R_0 = [n]P$, which is what the algorithm returns. For the counts, the loop has exactly $k - 1$ iterations, each performing one doubling and at most one addition. $\square$

Scalar multiplication by an n -bit scalar therefore costs $O(n)$ group operations— $O(n)$ field operations up to constant factors, or $O(n \log^2 p)$ bit operations with schoolbook field arithmetic. Elliptic-curve cryptography rests on the contrast between this *forward* map and the apparent hardness of inverting it (§10.11).

10.8 The group structure of $E(\mathbb{F}_p)$

Let $K = \mathbb{F}_p$ with p prime, $p > 3$. Each of the p values of x admits at most two values of y (Theorem 5.18, applied to $y^2 - f(x)$ as a polynomial in y), so with $\mathcal{O}$ there are at most $2p + 1$ points, and by Theorem 10.16 the set $E(\mathbb{F}_p)$ is a *finite abelian group*. Hasse's theorem bounds its order.

Theorem 10.22 (Hasse). Let E be an elliptic curve over $\mathbb{F}_p$. Write

$$\#E(\mathbb{F}_p) = p + 1 - t,$$

which defines the integer t , called the trace of Frobenius. Then

$$|t| \leq 2\sqrt{p};$$

| equivalently, $\#E(\mathbb{F}_p)$ lies in the Hasse interval $[p + 1 - 2\sqrt{p}, p + 1 + 2\sqrt{p}]$.

Proof (sketch). The count has an exact character-sum expression. For fixed x , the number of $y \in \mathbb{F}_p$ with $y^2 = f(x)$ is $1 + \left(\frac{f(x)}{p}\right)$ when $f(x) \neq 0$ —two solutions for a nonzero square, none for a non-square (Proposition 7.12)—and exactly $1 = 1 + \left(\frac{0}{p}\right)$ when $f(x) = 0$, using the Legendre symbol of Definition 7.14 with the convention $\left(\frac{0}{p}\right) = 0$. Summing over x ,

$$\#E^{\text{aff}}(\mathbb{F}_p) = \sum_{x \in \mathbb{F}_p} \left(1 + \left(\frac{x^3 + ax + b}{p} \right) \right) = p + \sum_{x \in \mathbb{F}_p} \left(\frac{x^3 + ax + b}{p} \right),$$

so $t = -\sum_x \left(\frac{x^3 + ax + b}{p} \right)$. Heuristically, as x ranges over $\mathbb{F}_p$ the value $f(x)$ is a nonzero square about half the time (two points), a non-square about half the time (none), and zero occasionally (one point); the p summands ± 1 behave like a random walk, and Hasse's bound $|t| \leq 2\sqrt{p}$ is exactly square-root cancellation in this character sum.

The conceptual proof realises t as the trace of the p -power *Frobenius endomorphism* $\pi: (x, y) \mapsto (x^p, y^p)$ of the curve—the curve-level counterpart of the field automorphism of Definition 6.14. One shows that π satisfies the characteristic equation $\pi^2 - [t]\pi + [p] = 0$ in the endomorphism ring of E , and that the associated quadratic form (the degree) is positive definite; the discriminant condition $t^2 - 4p \leq 0$ for the complex roots of the characteristic polynomial then yields $|t| \leq 2\sqrt{p}$. The full argument requires the theory of isogenies and the Weil pairing, beyond the present scope. $\square$

By Hasse's theorem the relative deviation of the order from p is of order $p^{-1/2}$, so the bound guarantees without a brute-force count that the group has about p elements, about 255 bits for the Pasta curves. The bound also underlies the point-counting algorithms (Schoof–Elkies–Atkin) that compute the *exact* order in polynomial time, by which curve designers certify their group orders.

Cryptographic use requires $E(\mathbb{F}_p)$ to contain a large subgroup of *prime* order. The general structure theory of $E(\mathbb{F}_p)$ is not needed in this series: the Pasta curves are designed so that $\#E(\mathbb{F}_p)$ is itself prime, and a group of prime order is cyclic with no proper nontrivial subgroups (Corollary 3.34).

10.9 Torsion, the cofactor, and prime-order subgroups

Definition 10.23 (Order of a point and torsion). The *order* of a point $P \in E(\mathbb{F}_p)$ is its order as a group element (Definition 3.11): the least integer $m \geq 1$ with $[m]P = \mathcal{O}$. It exists and divides $\#E(\mathbb{F}_p)$, by Lagrange's theorem (Theorem 3.29 via Corollary 3.30). For an integer $m \geq 1$, the *m-torsion subgroup* is

$$E[m] = \{ P \in E(\overline{\mathbb{F}_p}) : [m]P = \mathcal{O} \},$$

the points over the algebraic closure killed by m ; its rational part is $E(\mathbb{F}_p)[m] = E[m] \cap E(\mathbb{F}_p)$.

Definition 10.24 (Cofactor). Suppose $\#E(\mathbb{F}_p) = h \cdot r$, where r is the (large) prime to be used and h is the remaining factor. The integer h is the *cofactor*, and r is the order of the *prime-order subgroup*: a point P of order r generates a cyclic subgroup $\langle P \rangle \cong \mathbb{Z}/r\mathbb{Z}$, and cryptographic protocols operate in this subgroup.

A nontrivial cofactor $h > 1$ is undesirable: it admits *small-subgroup* points, of order dividing h , which can leak information or break protocol assumptions—the small-subgroup attacks of Remark 7.25. Implementations therefore either *clear the cofactor*, multiplying incoming points by h , or use prime-order curves with $h = 1$. The Pasta curves have cofactor 1: the orders $\#E_p(\mathbb{F}_p) = q$ and $\#E_q(\mathbb{F}_q) = p$ are themselves prime, so $E_p(\mathbb{F}_p) \cong \mathbb{Z}/q\mathbb{Z}$ and $E_q(\mathbb{F}_q) \cong \mathbb{Z}/p\mathbb{Z}$, and no cofactor clearing is needed.

Proposition 10.25 (Two-torsion). *The affine points of order 2 in $E(\mathbb{F}_p)$ are exactly the points $(x_0, 0)$ with $f(x_0) = x_0^3 + ax_0 + b = 0$. Hence $E(\mathbb{F}_p)$ has 0, 1, or 3 points of order 2 according as f has 0, 1, or 3 roots in $\mathbb{F}_p$; and $\#E(\mathbb{F}_p)$ is even if and only if f has a root in $\mathbb{F}_p$.*

Proof. A point $P = (x, y)$ has order 2 iff $P = -P$ and $P \neq \mathcal{O}$, i.e. $y = -y$, i.e. $y = 0$ (since $\text{char}(\mathbb{F}_p) \neq 2$); the curve equation then forces $f(x) = 0$. Conversely each root x_0 of f in $\mathbb{F}_p$ gives the order-2 point $(x_0, 0)$.

Next, the number N of roots of f in $\mathbb{F}_p$ can only be 0, 1, or 3: two roots force a third. Suppose f has distinct roots $r_1 \neq r_2$ in $\mathbb{F}_p$. The factor theorem (Theorem 5.17) gives $f = (x - r_1)h$; evaluating at r_2 yields $0 = (r_2 - r_1)h(r_2)$, so $h(r_2) = 0$, and a second application factors the monic cubic completely, $f = (x - r_1)(x - r_2)(x - r_3)$ with $r_3 \in \mathbb{F}_p$ —explicitly $r_3 = -r_1 - r_2$ by Vieta, the depressed cubic having no x^2 term. Nonsingularity says f has no repeated root even over $\overline{\mathbb{F}_p}$ (Proposition 10.4(2)), so $r_3 \notin \{r_1, r_2\}$, and f has exactly three roots (Theorem 5.18 caps the count). Hence $N \in \{0, 1, 3\}$, matching the trichotomy of the statement. Together with $\mathcal{O}$ the order-2 points form the rational 2-torsion subgroup $E(\mathbb{F}_p)[2]$, which is accordingly trivial, $\mathbb{Z}/2\mathbb{Z}$, or $\mathbb{Z}/2\mathbb{Z} \times \mathbb{Z}/2\mathbb{Z}$.

For the parity claim, consider negation $P \mapsto -P$, an *involution*—a self-inverse map—of the finite set $E(\mathbb{F}_p)$. Its fixed points are exactly $\mathcal{O}$ and the points $(x_0, 0)$; every other point pairs off with its distinct negative into a 2-element orbit. Counting the set by orbits,

$$\#E(\mathbb{F}_p) \equiv 1 + N \pmod{2},$$

where $N \in \{0, 1, 3\}$ is the number of roots of f in $\mathbb{F}_p$. Hence $\#E(\mathbb{F}_p)$ is even iff N is odd, i.e. iff $N \geq 1$, i.e. iff f has a root in $\mathbb{F}_p$. (In general, an involution of a finite set determines the cardinality of the set modulo 2 from its fixed points alone.) $\square$

Remark 10.26 (No two-torsion on the Pasta curves). For $y^2 = x^3 + 5$ over $\mathbb{F}_p$, the curve has a point of order 2 iff $x^3 + 5 = 0$ has a solution, i.e. iff -5 is a cube in $\mathbb{F}_p$. For the Pasta primes the order $\#E_p(\mathbb{F}_p) = q$ is odd (it is prime), so there are *no* rational 2-torsion points; consistently, -5 is not a cube in $\mathbb{F}_p$.

10.10 Base fields, scalar fields, and the Pasta cycle

An elliptic-curve computation involves two prime fields. Every operation of the group law on $E(\mathbb{F}_p)$ —the slope $\lambda = (y_2 - y_1)(x_2 - x_1)^{-1}$, the squarings, and the inversions needed to add and to double—is arithmetic in $\mathbb{F}_p$, the *base field* (or coordinate field), performed with the prime-field algorithms of Section 6.1. Keys and signatures reside in the subgroup of large prime order r (Definition 10.24), and a scalar multiplication $P \mapsto [s]P$ on a point of order r depends only on $s \bmod r$, since $[r]P = \mathcal{O}$ (Proposition 3.12); scalars are therefore residues modulo r , and since r is prime, they form a *field* (Corollary 4.26).

Definition 10.27 (Base field and scalar field). For an elliptic curve used in cryptography, with a prime-order subgroup of order r :

- the *base field* $\mathbb{F}_p$ is the field over which the curve is defined and in which point coordinates lie;
- the *scalar field* $\mathbb{F}_r = \mathbb{Z}/r\mathbb{Z}$ is the field of scalars by which one multiplies points, i.e. the integers modulo the subgroup order r .

Both are prime fields, but in general $p \neq r$.

Remark 10.28 (Two fields at once, and the Pasta cycle). Scalar arithmetic runs in $\mathbb{F}_r$ and coordinate arithmetic in $\mathbb{F}_p$. Halo 2’s Pallas and Vesta curves (the “Pasta” pair) form a *cycle of curves*: the base field of one is the scalar field of the other and vice versa,

$$\#E_p(\mathbb{F}_p) = q, \quad \#E_q(\mathbb{F}_q) = p,$$

so the scalar field of Pallas is $\mathbb{F}_q$, the base field of Vesta, and symmetrically (§5.4.9.6 of the Zcash protocol specification). This lets a proof system express the verifier of one curve’s arithmetic *natively* in the field of the other, which enables the recursive proof composition of Halo 2. Both primes are chosen $\equiv 1 \pmod{2^{32}}$, so by Corollary 6.22 each field contains a multiplicative subgroup of order 2^{32} , furnishing the high-order roots of unity that the number-theoretic transform of Section 9 needs for fast polynomial multiplication.

10.11 The elliptic-curve discrete logarithm problem

The discrete logarithm problem of Definition 7.21, stated there for an abstract cyclic group, reads as follows for elliptic curves in additive notation.

Definition 10.29 (ECDLP). Let $P \in E(\mathbb{F}_p)$ be a point of large prime order r , generating $G = \langle P \rangle$. The *elliptic-curve discrete logarithm problem* (ECDLP) is: given P and a point $Q \in G$, find the unique integer $n \in \{0, 1, \dots, r-1\}$ with $Q = [n]P$. One writes $n = \log_p Q$.

The map $n \mapsto [n]P$ is a group isomorphism $\mathbb{Z}/r\mathbb{Z} \rightarrow G$ (Theorem 3.43), so a discrete logarithm always exists and is unique; the difficulty of the problem is purely *computational*. The forward map is fast by double-and-add (Proposition 10.21); on a well-chosen curve the best known algorithms to invert it are the two generic attacks, which transfer verbatim from Remark 7.22 to the additive setting.

Proposition 10.30 (Square-root attacks). *Baby-step giant-step solves the ECDLP in a group of prime order r in $O(\sqrt{r})$ group operations and $O(\sqrt{r})$ space. Pollard's rho method matches the $O(\sqrt{r})$ bound in expectation with only $O(1)$ space, under a modelling assumption on its pseudo-random walk that is stated explicitly in the proof and is unproved for every concrete walk.*

Proof. For baby-step giant-step, set $m = \lceil \sqrt{r} \rceil$ and write the unknown logarithm as $n = im + j$ with $0 \leq i, j < m$; since $m^2 \geq r$, every $n \in \{0, \dots, r-1\}$ has such a decomposition. Tabulate the *baby steps* $[j]P$ for $j = 0, \dots, m-1$ (a table of m points, built with $m-1$ additions), compute $S = [m]P$ once, and walk the *giant steps* $Q - [i]S$ for $i = 0, 1, 2, \dots$, testing each against the table. At $i = \lfloor n/m \rfloor$ the walk hits $Q - [im]P = [n - im]P = [j]P$, a table entry; the collision reveals i and j , hence $n = im + j$. Both the table and the walk cost $O(m) = O(\sqrt{r})$ group operations, and the table holds $O(\sqrt{r})$ points.

Pollard's rho eliminates the table, at the price of an explicitly heuristic analysis. The algorithm takes a pseudo-random walk on G whose every position is maintained in the form $[a]P + [b]Q$ with known coefficients a, b , started from such a combination with random coefficients; the walk's next step depends only on the current point, so once the walk revisits a point it cycles from then on. A repeat is therefore detected with $O(1)$ memory: run a second copy of the walk at double speed, and the two copies meet at a common point inside the cycle within a constant factor of the first repeat time. *Modelling assumption:* the successive positions of the walk behave like independent uniform samples from G . No proof of this is known for any concrete walk—the assumption is what “well-designed” means, and experiment supports it—and the rest of the argument is conditional on it. Under the assumption, the first k positions are pairwise distinct with probability at most $e^{-k(k-1)/(2r)}$ by the birthday bound (a forward reference: Proposition 11.18 belongs to the closing section), so the number T of steps to the first repeat satisfies $\Pr[T > k] \leq e^{-k(k-1)/(2r)}$. The tail-sum formula (Lemma 11.33) converts the tail bound into an expectation bound: with $m = \lceil \sqrt{r} \rceil$ and $r \geq 4$,

$$\mathbb{E}[T] = \sum_{k \geq 1} \Pr[T \geq k] \leq m \sum_{j \geq 0} \Pr[T > jm] \leq m \left(1 + \sum_{j \geq 1} e^{-j^2/4} \right) = O(\sqrt{r}),$$

where the middle step groups the indices k into blocks of length m , bounding each block by its first term, and the last uses $jm(jm-1) \geq j^2 m(m-1) \geq j^2(r-\sqrt{r}) \geq j^2 r/2$ for $j \geq 1$, so that $\Pr[T > jm] \leq e^{-j^2/4} \leq e^{-j/4}$, a convergent geometric tail. A repeat presents one group element with two representations, $[a]P + [b]Q = [a']P + [b']Q$. If $b' \not\equiv b \pmod{r}$, this rearranges to $[a - a']P = [b' - b]Q = [(b' - b)n]P$, whence

$$n \equiv (a - a')(b' - b)^{-1} \pmod{r},$$

the inverse existing because r is prime. If instead $b' \equiv b$, then $[a]P = [a']P$ forces $a \equiv a'$: the two representations coincide, the collision reveals nothing, and the walk is restarted with fresh random starting coefficients; that such degenerate collisions are rare is a further part of the heuristic, borne out in practice. Both algorithms run in time exponential in the bit length $\log_2 r$. $\square$

Remark 10.31 (Best known attacks on well-chosen curves). For well-chosen curves, no unrestricted classical attack faster than $O(\sqrt{r})$ is known; this is not an unconditional lower bound for

all classical algorithms. In the classical generic-group model, however, Shoup's matching lower bound is proved (Remark 7.22). For $r \approx 2^{254}$ —the 255-bit Pasta orders—this is about 2^{127} operations, infeasible. Curves with special structure can be weaker, and the known weak classes are avoided by design. *Anomalous* curves, those with $\#E(\mathbb{F}_p) = p$, admit a polynomial-time attack through the formal group. The MOV/Frey–Rück attack transfers the ECDLP into a finite-field discrete logarithm when the *embedding degree*—the least $k \geq 1$ with $r \mid p^k - 1$, i.e. the degree (Example 8.4; the dimension of the extension over its base) of the smallest extension of $\mathbb{F}_p$ whose multiplicative group contains a subgroup of order r —is small, where index calculus applies. Cryptographically secure curves, the Pasta curves included, are chosen with a large prime subgroup order, a large embedding degree, and $\#E(\mathbb{F}_p) \neq p$, which excludes every known subexponential attack.

Remark 10.32 (The index-calculus contrast). In $\mathbb{F}_p^\times$ the index-calculus method solves the discrete logarithm in subexponential time $L_p[1/3]$ —shorthand for $2^{\Theta(\lambda^{1/3}(\log \lambda)^{2/3})}$ operations in the bit length $\lambda = \log_2 p$, a notation defined precisely in Definition 11.60 of the closing section—exploiting the integer arithmetic behind the residues (Remark 7.22); those groups must therefore be large. On a well-chosen elliptic curve no index calculus is known—the points offer no analogue of “factoring into small primes”—so the generic $O(\sqrt{r})$ bound stands, and much smaller groups suffice: about 256 bits for 128-bit security, versus thousands of bits for $\mathbb{F}_p^\times$. Smaller groups mean faster arithmetic and shorter keys, which is why elliptic curves dominate modern protocols, including Halo 2.

10.12 The j -invariant and the GLV endomorphism

The j -invariant classifies curves up to isomorphism; when it vanishes, as it does for the Pasta curves by design, the curve has an additional endomorphism.

Definition 10.33 (j -invariant). The j -invariant of the curve $y^2 = x^3 + ax + b$ (with $4a^3 + 27b^2 \neq 0$) is

$$j = 1728 \frac{4a^3}{4a^3 + 27b^2} \in K,$$

where $1728 = 12^3$ is a normalising constant of historical origin.

Proposition 10.34. Two elliptic curves over $\bar{K}$ are isomorphic over $\bar{K}$ if and only if they have the same j -invariant. The admissible coordinate changes preserving short Weierstrass form are $(x, y) \mapsto (u^2x, u^3y)$ for $u \in \bar{K}^\times$, under which $(a, b) \mapsto (u^4a, u^6b)$; the combination j is invariant.

Proof (sketch). A direct computation shows that the only isomorphisms between short Weierstrass curves fixing $\mathcal{O}$ are the scalings $(x, y) \mapsto (u^2x, u^3y)$. Substituting $x = u^{-2}x'$, $y = u^{-3}y'$ into $y^2 = x^3 + ax + b$ and clearing u^6 gives $y'^2 = x'^3 + u^4ax' + u^6b$, which is the stated action on coefficients. Under it, numerator and denominator of $4a^3/(4a^3 + 27b^2)$ both scale by u^{12} , so j is unchanged. Conversely, given two curves with equal j -invariants one solves for a suitable u transforming one coefficient pair into the other, treating separately the cases $j = 0$ (both a vanish) and $j = 1728$ (both b vanish). $\square$

Remark 10.35 (The Pasta curves have $j = 0$). For $y^2 = x^3 + b$ one has $a = 0$, so $j = 0$; both Pasta curves have $j = 0$. Curves with $j = 0$ admit an additional automorphism of order 3, which underlies the efficient endomorphism described next. (In the other exceptional case, $b = 0$ gives $j = 1728$ and an order-4 automorphism.) The Pasta design chooses $j = 0$ to make the Gallant–Lambert–Vanstone (GLV) speedup available, which roughly halves the doubling count in scalar multiplication.

Proposition 10.36 (The GLV endomorphism for $y^2 = x^3 + b$). *Suppose $\mathbb{F}_p$ contains a primitive cube root of unity ω , i.e. $p \equiv 1 \pmod{3}$ (Corollary 6.22), so $\omega \in \mathbb{F}_p$ satisfies $\omega^2 + \omega + 1 = 0$. On $E : y^2 = x^3 + b$, the map*

$$\phi(x, y) = (\omega x, y), \quad \phi(\mathcal{O}) = \mathcal{O},$$

is a group endomorphism of $E(\mathbb{F}_p)$. Let $G = \langle P \rangle$ be a subgroup of prime order r with $r \equiv 1 \pmod{3}$ and $\phi(G) \subseteq G$. Then ϕ acts on G as multiplication by a scalar: there is $\lambda \in \mathbb{Z}/r\mathbb{Z}$ with $\lambda^2 + \lambda + 1 \equiv 0 \pmod{r}$ and

$$\phi(Q) = [\lambda] Q \quad \text{for all } Q \in G.$$

For the Pasta curves the hypothesis $\phi(G) \subseteq G$ is automatic, since $E(\mathbb{F}_p)$ itself is the prime-order group G ; and $r \equiv 1 \pmod{3}$ holds for both orders.

Proof. The map lands on the curve. If $y^2 = x^3 + b$ then $y^2 = (\omega x)^3 + b$, because $\omega^3 = 1$; so $(\omega x, y) \in E(\mathbb{F}_p)$.

The map is a homomorphism. We check $\phi(P_1 + P_2) = \phi(P_1) + \phi(P_2)$ against the case analysis of §10.4. The $\mathcal{O}$ cases are immediate, and $\phi(-P) = -\phi(P)$ since negation touches only y ; in particular the inverse case $\phi(P) + \phi(-P) = \mathcal{O} = \phi(\mathcal{O})$ is respected, and ϕ preserves the case split, since it preserves equality of x -coordinates. For a chord $(x_1 \neq x_2)$, the slope transforms as

$$\lambda' = \frac{y_2 - y_1}{\omega x_2 - \omega x_1} = \omega^{-1} \lambda = \omega^2 \lambda,$$

using $\omega^{-1} = \omega^2$. Then, by (12) and $\omega^4 = \omega$,

$$x'_3 = \lambda'^2 - \omega x_1 - \omega x_2 = \omega^4 \lambda^2 - \omega x_1 - \omega x_2 = \omega (\lambda^2 - x_1 - x_2) = \omega x_3,$$

$$y'_3 = \lambda'(\omega x_1 - x'_3) - y_1 = \omega^2 \lambda \cdot \omega(x_1 - x_3) - y_1 = \lambda(x_1 - x_3) - y_1 = y_3,$$

so $\phi(P_1) + \phi(P_2) = (\omega x_3, y_3) = \phi(P_1 + P_2)$. The tangent case is identical, with $\lambda' = 3(\omega x_1)^2 / (2y_1) = \omega^2 \lambda$ in (13).

The relation $\phi^2 + \phi + \text{id} = 0$. Let $P = (x, y)$ be affine; we show $P + \phi(P) + \phi^2(P) = \mathcal{O}$ directly from the formulas of §10.4.

First let $x \neq 0$. The points $P = (x, y)$, $\phi(P) = (\omega x, y)$, $\phi^2(P) = (\omega^2 x, y)$ share their y -coordinate and have pairwise distinct x -coordinates, since $1, \omega, \omega^2$ are distinct and $x \neq 0$. The chord formula (12) applies to $P + \phi(P)$, with slope $\lambda = (y - y) / (\omega x - x) = 0$:

$$x_3 = 0 - x - \omega x = -(1 + \omega)x = \omega^2 x, \quad y_3 = 0 \cdot (x - x_3) - y = -y,$$

using $1 + \omega + \omega^2 = 0$. Hence $P + \phi(P) = (\omega^2 x, -y) = -\phi^2(P)$ (Definition 10.8), and $P + \phi(P) + \phi^2(P) = \mathcal{O}$.

If instead $x = 0$, the three points coincide: $\phi(P) = \phi^2(P) = P$. Here $y \neq 0$, since $x = y = 0$ on the curve would force $b = 0$, which nonsingularity $27b^2 \neq 0$ forbids; so the doubling formula (13) applies, with $a = 0$:

$$\lambda = \frac{3 \cdot 0^2 + 0}{2y} = 0, \quad x_3 = 0 - 2 \cdot 0 = 0, \quad y_3 = 0 \cdot (0 - 0) - y = -y,$$

so $[2]P = (0, -y) = -P$, hence $P + \phi(P) + \phi^2(P) = [3]P = \mathcal{O}$ in this case too. (The three points $P, \phi(P), \phi^2(P)$ are exactly the intersections of the horizontal line through P with the curve, the roots of $X^3 = y^2 - b$ being $x, \omega x, \omega^2 x$; the computation above proves the rule “three points of a line sum to $\mathcal{O}$ ” for this line, including the triple intersection at $x = 0$.)

With $\phi(\mathcal{O}) = \mathcal{O}$ the identity holds on all of $E(\mathbb{F}_p)$; as an identity of endomorphisms, $\phi^2 + \phi + \text{id} = 0$. Moreover $\phi^3 = \text{id}$, since $\omega^3 = 1$; and whenever $\#E(\mathbb{F}_p) > 3$ —as always at cryptographic sizes, the Pasta curves included— ϕ has order exactly 3, because at most two affine points, $(0, \pm y_0)$ with $y_0^2 = b$, share the coordinate $x = 0$, so some point of $E(\mathbb{F}_p)$ has $x \neq 0$ and ϕ moves it. The proviso is necessary: $y^2 = x^3 + 4$ over $\mathbb{F}_7$ has only the three points $\mathcal{O}, (0, 2), (0, 5)$, and there $\phi = \text{id}$.

Action on G as a scalar. The subgroup $G \cong \mathbb{Z}/r\mathbb{Z}$ is cyclic with generator P , and $\phi(G) \subseteq G$ by hypothesis, so $\phi(P) = [\lambda]P$ for some $\lambda \in \mathbb{Z}/r\mathbb{Z}$; then for any $Q = [m]P \in G$, the homomorphism property gives $\phi(Q) = [m]\phi(P) = [m\lambda]P = [\lambda]Q$. (This is the statement $\text{Hom}(\mathbb{Z}/r\mathbb{Z}, \mathbb{Z}/r\mathbb{Z}) \cong \mathbb{Z}/r\mathbb{Z}$: an endomorphism of a cyclic group is determined by, and equal to multiplication by, the image of a generator.) Applying the endomorphism relation to P ,

$$[\lambda^2 + \lambda + 1]P = \mathcal{O} \implies \lambda^2 + \lambda + 1 \equiv 0 \pmod{r},$$

since P has order r . Consistently, this congruence is solvable exactly when $\mathbb{F}_r$ contains a primitive cube root of unity, i.e. when $r \equiv 1 \pmod{3}$ (Corollary 6.22 again)—the hypothesis of the statement. $\square$

Remark 10.37 (How GLV halves the doubling count). Computing ϕ costs a single field multiplication $x \mapsto \omega x$, negligible against a group operation. A scalar $n \in \mathbb{Z}/r\mathbb{Z}$ is decomposed as

$$[n]P = [n_1]P + [n_2]\phi(P), \quad n \equiv n_1 + n_2\lambda \pmod{r},$$

where n_1, n_2 can each be taken about $\frac{1}{2} \log_2 r$ bits long; such a pair is found by lattice reduction in the lattice of pairs (c_1, c_2) with $c_1 + c_2\lambda \equiv 0 \pmod{r}$. The two half-length scalar multiplications are then run *simultaneously*, sharing their doublings (Straus–Shamir’s trick): one double-and-add loop of half the length, adding $P, \phi(P)$, or both according to the bits of n_1 and n_2 . The result costs about half the doublings of a single full-length double-and-add; this saving is one reason the Pasta curves take the form $y^2 = x^3 + b$ over primes $p \equiv 1 \pmod{3}$.

10.13 A worked toy example

Example 10.38 (The curve $y^2 = x^3 + 5$ over $\mathbb{F}_{11}$). Take $p = 11$: then $p > 3$, and $4 \cdot 0 + 27 \cdot 5^2 = 675 \equiv 4 \not\equiv 0 \pmod{11}$, so the curve $E : y^2 = x^3 + 5$ is nonsingular over $\mathbb{F}_{11}$ (Definition 10.5). Squaring the residues $0, 1, \dots, 10$ yields the squares $\{0, 1, 3, 4, 5, 9\}$, so the nonzero

quadratic residues are $\{1, 3, 4, 5, 9\}$. Tabulating $f(x) = x^3 + 5 \pmod{11}$ and reading off the y -values:

x	$x^3 \pmod{11}$	$f(x) = x^3 + 5$	y with $y^2 = f(x)$
0	0	5	± 4 ($4^2 = 16 \equiv 5$)
1	1	6	none ($6 \notin \text{QR}$)
2	8	2	none
3	5	10	none
4	9	3	± 5 ($5^2 = 25 \equiv 3$)
5	4	9	± 3
6	7	1	± 1
7	2	7	none
8	6	0	0
9	3	8	none
10	10	4	± 2

Counting: the five values $x \in \{0, 4, 5, 6, 10\}$ contribute two points each (10 points), $x = 8$ contributes the single point $(8, 0)$, and $\mathcal{O}$ is the remaining point:

$$\#E(\mathbb{F}_{11}) = 10 + 1 + 1 = 12.$$

Hasse's bound holds: $p + 1 = 12$, so the trace is $t = 0$, and $|t| = 0 \leq 2\sqrt{11} \approx 6.63$. The point $(8, 0)$ has order 2, consistent with Proposition 10.25 and the even order 12; here $-5 \equiv 6$ is a cube, namely $8^3 = 512 \equiv 6$.

An addition. Let $P = (0, 4)$ and $Q = (5, 3)$. Since $x_1 = 0 \neq 5 = x_2$, the chord formula (12) applies:

$$\lambda = \frac{3 - 4}{5 - 0} = (-1) \cdot 5^{-1} = (-1) \cdot 9 = -9 \equiv 2 \pmod{11},$$

using $5^{-1} = 9$ (check: $5 \cdot 9 = 45 \equiv 1$). Then

$$x_3 = \lambda^2 - x_1 - x_2 = 4 - 0 - 5 = -1 \equiv 10, \quad y_3 = \lambda(x_1 - x_3) - y_1 = 2(0 - 10) - 4 = -24 \equiv 9,$$

so $P + Q = (10, 9)$. The result is on the curve: $9^2 = 81 \equiv 4$ and $10^3 + 5 = 1005 \equiv 4 \pmod{11}$, as required.

A doubling. Take $P = (0, 4)$. With $a = 0$, formula (13) gives

$$\lambda = \frac{3x_1^2 + a}{2y_1} = \frac{0}{8} = 0, \quad x_3 = \lambda^2 - 2x_1 = 0, \quad y_3 = \lambda(x_1 - x_3) - y_1 = -4 \equiv 7,$$

so $[2](0, 4) = (0, 7) = -(0, 4)$. Hence $[3](0, 4) = \mathcal{O}$: the point $(0, 4)$ has order 3.

The GLV endomorphism. Since $11 \equiv 2 \pmod{3}$, the field $\mathbb{F}_{11}$ contains *no* primitive cube root of unity (Corollary 6.22 requires $3 \mid p - 1$), so the GLV map is not available over $\mathbb{F}_{11}$. The Pasta primes do satisfy $p \equiv q \equiv 1 \pmod{3}$, so GLV applies there. For a small GLV-friendly illustration take instead $p = 13 \equiv 1 \pmod{3}$, where $\omega = 3$ satisfies $3^2 + 3 + 1 = 13 \equiv 0 \pmod{13}$; then $\phi(x, y) = (3x, y)$ is a nontrivial endomorphism of $y^2 = x^3 + b$ over $\mathbb{F}_{13}$.

10.14 Pallas and Vesta assembled

Example 10.39 (The Pasta curves). For the running example $y^2 = x^3 + 5$, with p and q the 255-bit primes of the section opening:

- *Pallas*, over $\mathbb{F}_p$: the order is $\#E_p(\mathbb{F}_p) = q$, a prime, so the group is cyclic, $E_p(\mathbb{F}_p) \cong \mathbb{Z}/q\mathbb{Z}$ (Corollary 3.34), with cofactor $h = 1$ (Definition 10.24). The trace is $t = p + 1 - q$, an 87-bit (negative) integer, far inside the Hasse bound $2\sqrt{p} \approx 2^{128}$ of Theorem 10.22.
- *Vesta*, over $\mathbb{F}_q$: the order is $\#E_q(\mathbb{F}_q) = p$, a prime, so $E_q(\mathbb{F}_q) \cong \mathbb{Z}/p\mathbb{Z}$, again with cofactor 1.
- Both curves have $j = 0$ (Remark 10.35), and both primes satisfy $p \equiv q \equiv 1 \pmod{3}$, so each curve admits the order-3 endomorphism $\phi(x, y) = (\omega x, y)$ of Proposition 10.36 and supports the GLV speedup of Remark 10.37.
- Both fields have two-adicity 32 in the sense of Remark 9.10: 2^{32} divides $p - 1$ and $q - 1$, so each field carries the multiplicative subgroups of order 2^{32} asserted in Remark 10.28, the evaluation domains of the fast polynomial arithmetic of Section 9.
- The cycle property $\#E_p(\mathbb{F}_p) = q$ and $\#E_q(\mathbb{F}_q) = p$ means the scalar field of each curve is the base field of the other (Definition 10.27): the base field of Pallas is $\mathbb{F}_p$ and its scalar field is $\mathbb{F}_q$, matching the base field of Vesta, and symmetrically. This lets recursive proof systems chain proofs across the two curves without expensive non-native field arithmetic (Remark 10.28). The cycle is deployed: the `pasta_curves` crate defines the scalar field of Pallas to be $\mathbb{F}_q$ and that of Vesta to be $\mathbb{F}_p$, as specified in §5.4.9.6 of the Zcash protocol specification.

The discrete logarithm problem in $E_p(\mathbb{F}_p)$ and $E_q(\mathbb{F}_q)$ is believed to require about 2^{127} operations— $\sqrt{r}$ for the 255-bit orders $r \approx 2^{254}$ (Proposition 10.30)—with no faster attack known (Remark 10.31), slightly below the 128-bit security level targeted by Halo 2 and Zcash Orchard.

Each Pasta curve is thus a cyclic group of prime 255-bit order (Theorem 10.22 and Corollary 3.34), with a fast forward map (Proposition 10.21, accelerated by Remark 10.37) whose inversion, the ECDLP, resists every known attack except the generic $O(\sqrt{r})$ attacks of Proposition 10.30. Later volumes build on this pair.

11 Probability, asymptotics, and computation

This section fixes the language of security claims: probability, asymptotic notation, and efficient computation. It proves the union and birthday bounds, the properties of statistical distance, the bias bound for modular reduction, Wald’s identity, Chernoff bounds for the arrival counts of exponential clocks, and the closure of negligible functions under addition and multiplication by polynomials. The birthday bound sets hash output lengths, and the bias bound justifies the sampling of Orchard scalars; later volumes build their block-discovery and extraction analyses on the waiting-time and clock results. A signature scheme advertised as offering “128-bit security” cannot exclude forgery: a forger that guesses the secret key, a finite string, succeeds with positive

probability, so the claim bounds the success probability of every attacker within stated resources; §11.11 states what it asserts. Probability spaces are finite until §11.7; finite spaces suffice for keys, bounded-length messages, group elements, and the coins of an algorithm with a fixed running-time bound. The prerequisites are finite sets, sums, and elementary calculus: limits, the exponential function, and, for the density-defined laws of §11.7, the improper Riemann integral.

11.1 Finite probability spaces and events

Definition 11.1 (Finite probability space). A *finite probability space* is a pair $(\Omega, \Pr)$ where Ω is a nonempty finite set, called the *sample space*, and $\Pr: \Omega \rightarrow [0, 1]$ is a function, called the *probability mass function*, satisfying the normalisation condition

$$\sum_{\omega \in \Omega} \Pr[\omega] = 1.$$

An element $\omega \in \Omega$ is an *outcome* (or *elementary event*).

Definition 11.2 (Event). An *event* is a subset $A \subseteq \Omega$. Its *probability* is

$$\Pr[A] = \sum_{\omega \in A} \Pr[\omega],$$

with the convention $\Pr[\emptyset] = 0$ (an empty sum). The event A *occurs* on outcome ω if $\omega \in A$.

Because Ω is finite, every subset is an event. On an uncountable sample space the general theory specifies a family of admissible events together with its probability assignment; Remark 11.31 marks the one place this series requires measure theory. The following facts are used without further citation.

Proposition 11.3 (Basic axioms). For any finite probability space $(\Omega, \Pr)$ and events $A, B \subseteq \Omega$:

1. $0 \leq \Pr[A] \leq 1$, $\Pr[\emptyset] = 0$, and $\Pr[\Omega] = 1$.
2. (Complement.) $\Pr[\Omega \setminus A] = 1 - \Pr[A]$.
3. (Monotonicity.) If $A \subseteq B$ then $\Pr[A] \leq \Pr[B]$.
4. (Inclusion–exclusion for two events.) $\Pr[A \cup B] = \Pr[A] + \Pr[B] - \Pr[A \cap B]$.
5. (Finite additivity.) If $A_1, \dots, A_n$ are pairwise disjoint then $\Pr[\bigcup_{i=1}^n A_i] = \sum_{i=1}^n \Pr[A_i]$.

Proof. Every claim follows by manipulating the defining sums. For (1), each summand $\Pr[\omega]$ lies in $[0, 1]$ and the total is 1 by normalisation; an event's probability is a partial sum of nonnegative terms, hence lies in $[0, 1]$. For (2), the sets A and $\Omega \setminus A$ partition Ω , so $\Pr[A] + \Pr[\Omega \setminus A] = \sum_{\omega \in \Omega} \Pr[\omega] = 1$. For (3), splitting the sum over B gives $\Pr[B] = \sum_{\omega \in A} \Pr[\omega] + \sum_{\omega \in B \setminus A} \Pr[\omega] \geq \Pr[A]$, the second term being a sum of nonnegative numbers. For (5), disjointness means each ω in the union lies in exactly one A_i , so the double sum $\sum_i \sum_{\omega \in A_i} \Pr[\omega]$ counts each $\omega \in \bigcup_i A_i$ exactly

once. For (4), write $A \cup B = A \sqcup (B \setminus A)$ (disjoint), so by (5) $\Pr[A \cup B] = \Pr[A] + \Pr[B \setminus A]$; also $B = (A \cap B) \sqcup (B \setminus A)$, giving $\Pr[B \setminus A] = \Pr[B] - \Pr[A \cap B]$. Substituting yields the claim. $\square$

The distribution used most often in cryptography is the uniform distribution.

Definition 11.4 (Uniform distribution). Let S be a nonempty finite set. The *uniform distribution* on S is the probability space $(S, \Pr)$ with $\Pr[\omega] = 1/|S|$ for every $\omega \in S$. The notation $x \leftarrow S$ (or $x \xleftarrow{\$} S$) means that x is sampled according to the uniform distribution on S . Under the uniform distribution, for any event $A \subseteq S$,

$$\Pr[A] = \frac{|A|}{|S|}.$$

11.2 Conditional probability and independence

Conditioning updates the probability of one event on the information that another has occurred.

Definition 11.5 (Conditional probability). Let A, B be events with $\Pr[B] > 0$. The *conditional probability of A given B* is

$$\Pr[A \mid B] = \frac{\Pr[A \cap B]}{\Pr[B]}.$$

When $\Pr[B] = 0$ the quantity is left undefined.

Conditioning on B restricts the sample space to B and renormalises so that B has probability 1: the map $\omega \mapsto \Pr[\{\omega\} \mid B]$ is itself a probability mass function on B , and $\Pr[A \mid B]$ is the probability it assigns to $A \cap B$.

Proposition 11.6 (Chain rule and total probability). Let $A_1, \dots, A_n$ be events with $\Pr[A_1 \cap \dots \cap A_{n-1}] > 0$. Then

$$\Pr[A_1 \cap \dots \cap A_n] = \Pr[A_1] \Pr[A_2 \mid A_1] \Pr[A_3 \mid A_1 \cap A_2] \dots \Pr[A_n \mid A_1 \cap \dots \cap A_{n-1}].$$

Moreover, if $B_1, \dots, B_k$ partition Ω (pairwise disjoint with union Ω) and each $\Pr[B_i] > 0$, then for any event A ,

$$\Pr[A] = \sum_{i=1}^k \Pr[A \mid B_i] \Pr[B_i].$$

Proof. The chain rule follows by telescoping: each factor $\Pr[A_j \mid A_1 \cap \dots \cap A_{j-1}]$ equals $\Pr[A_1 \cap \dots \cap A_j] / \Pr[A_1 \cap \dots \cap A_{j-1}]$ by Definition 11.5, and the product collapses to $\Pr[A_1 \cap \dots \cap A_n] / 1$. (Monotonicity ensures that all the intermediate intersections have positive probability, so every factor is defined.) For total probability, the events $A \cap B_i$ are pairwise disjoint with union A (since the B_i partition Ω), so finite additivity and $\Pr[A \cap B_i] = \Pr[A \mid B_i] \Pr[B_i]$ give $\Pr[A] = \sum_i \Pr[A \cap B_i] = \sum_i \Pr[A \mid B_i] \Pr[B_i]$. $\square$

Independence is the formal statement that one event carries no information about another.

Definition 11.7 (Independence of events). Two events A, B are *independent* if

$$\Pr[A \cap B] = \Pr[A] \Pr[B].$$

A family $\{A_i\}_{i \in I}$ of events is *mutually independent* if for every finite subset $J \subseteq I$,

$$\Pr\left[\bigcap_{i \in J} A_i\right] = \prod_{i \in J} \Pr[A_i].$$

Remark 11.8. When $\Pr[B] > 0$, independence of A and B is equivalent to $\Pr[A \mid B] = \Pr[A]$. Mutual independence is strictly stronger than pairwise independence. The standard counterexample consists of two fair coin tosses, with A “the first is heads”, B “the second is heads”, and C “the two coins differ”. Each pair is independent, yet $\Pr[A \cap B \cap C] = 0 \neq \frac{1}{8} = \Pr[A] \Pr[B] \Pr[C]$, so the three are not mutually independent.

11.3 Random variables and expectation

Definition 11.9 (Random variable). Let $(\Omega, \Pr)$ be a finite probability space and T any set. A (T -valued) *random variable* is a function $X: \Omega \rightarrow T$. When $T \subseteq \mathbb{R}$, we call X *real-valued*. For $t \in T$,

$$\Pr[X = t] = \Pr[\{\omega \in \Omega : X(\omega) = t\}],$$

and more generally $\Pr[X \in B]$ denotes the probability that X takes a value in $B \subseteq T$. The function $t \mapsto \Pr[X = t]$ on the finite image of X is the *distribution* (or *law*) of X .

Definition 11.10 (Independent random variables). Random variables $X_1, \dots, X_n$ (with values in sets $T_1, \dots, T_n$) are *mutually independent* if for all $t_1 \in T_1, \dots, t_n \in T_n$,

$$\Pr[X_1 = t_1, \dots, X_n = t_n] = \prod_{i=1}^n \Pr[X_i = t_i].$$

Definition 11.11 (Expectation). Let $X: \Omega \rightarrow \mathbb{R}$ be a real-valued random variable on a finite space. Its *expectation* (or *mean*) is

$$\mathbb{E}[X] = \sum_{\omega \in \Omega} X(\omega) \Pr[\omega] = \sum_{t \in \text{im}(X)} t \cdot \Pr[X = t],$$

where $\text{im}(X)$ denotes the (finite) image of X . The two expressions agree by grouping outcomes according to their X -value.

Definition 11.12 (Indicator). For an event A , its *indicator random variable* $\mathbf{1}_A: \Omega \rightarrow \{0, 1\}$ takes the value $\mathbf{1}_A(\omega) = 1$ if $\omega \in A$ and 0 otherwise.

Proposition 11.13 (Indicator expectation). *For any event A , $\mathbb{E}[\mathbf{1}_A] = \Pr[A]$.*

Proof. By Definition 11.11, $\mathbb{E}[\mathbf{1}_A] = \sum_{\omega} \mathbf{1}_A(\omega) \Pr[\omega] = \sum_{\omega \in A} \Pr[\omega] = \Pr[A]$, since the indicator vanishes off A . $\square$

Theorem 11.14 (Linearity of expectation). *Let $X, Y: \Omega \rightarrow \mathbb{R}$ be random variables on a finite space and $a, b \in \mathbb{R}$. Then*

$$\mathbb{E}[aX + bY] = a \mathbb{E}[X] + b \mathbb{E}[Y].$$

More generally, for random variables $X_1, \dots, X_n$ and scalars $a_1, \dots, a_n$,

$$\mathbb{E}\left[\sum_{i=1}^n a_i X_i\right] = \sum_{i=1}^n a_i \mathbb{E}[X_i].$$

This holds whether or not the X_i are independent.

Proof. Directly from the definition,

$$\begin{aligned} \mathbb{E}[aX + bY] &= \sum_{\omega \in \Omega} (aX(\omega) + bY(\omega)) \Pr[\omega] \\ &= a \sum_{\omega} X(\omega) \Pr[\omega] + b \sum_{\omega} Y(\omega) \Pr[\omega] \\ &= a \mathbb{E}[X] + b \mathbb{E}[Y], \end{aligned}$$

splitting the finite sum and extracting the constants. The n -term statement follows by induction on n . $\square$

The following consequence underlies every averaging (“heavy-row”) argument in the series.

Proposition 11.15 (Markov’s inequality). *Let $X \geq 0$ be a real-valued random variable and $a > 0$. Then $\Pr[X \geq a] \leq \mathbb{E}[X]/a$.*

Proof. Every outcome contributes nonnegatively to $\mathbb{E}[X]$, and each outcome with $X(\omega) \geq a$ contributes at least $a \Pr[\omega]$; hence $\mathbb{E}[X] \geq a \Pr[X \geq a]$. $\square$

11.4 The union bound and a birthday calculation

This subsection proves the union bound, the inequality used most often in cryptographic proofs; it requires no independence. It gives the upper half of the *birthday bound*, which governs how soon collisions appear under uniform sampling: among roughly $\sqrt{N}$ uniform samples from a set of size N , two are likely to coincide. The birthday bound sets the output lengths of cryptographic hash functions.

Theorem 11.16 (Union bound / Boole’s inequality). *For any events $A_1, \dots, A_n$ in a finite probability space,*

$$\Pr\left[\bigcup_{i=1}^n A_i\right] \leq \sum_{i=1}^n \Pr[A_i].$$

Proof. Disjointify the union into “first occurrence” events: $B_1 = A_1$ and $B_i = A_i \setminus (A_1 \cup \dots \cup A_{i-1})$ for $i \geq 2$. The B_i are pairwise disjoint, $B_i \subseteq A_i$, and $\bigcup_i B_i = \bigcup_i A_i$. By finite additivity and monotonicity (Proposition 11.3),

$$\Pr\left[\bigcup_i A_i\right] = \Pr\left[\bigcup_i B_i\right] = \sum_i \Pr[B_i] \leq \sum_i \Pr[A_i]. \quad \square$$

The “first occurrence” disjointification recurs, in a telescoping form for increasing chains, in the proof of the continuity theorem of §11.7. Taking complements in the union bound gives the corresponding inequality for conjunctions.

Corollary 11.17 (Union bound for the good event). *If each event A_i holds except with probability at most ε_i (that is, $\Pr[\Omega \setminus A_i] \leq \varepsilon_i$), then all of them hold simultaneously except with probability at most $\sum_i \varepsilon_i$:*

$$\Pr\left[\bigcap_{i=1}^n A_i\right] \geq 1 - \sum_{i=1}^n \varepsilon_i.$$

Proof. By De Morgan, $\Omega \setminus \bigcap_i A_i = \bigcup_i (\Omega \setminus A_i)$. Apply the union bound to the complements and take the complement of both sides. $\square$

Proposition 11.18 (Birthday bound). *Sample $x_1, \dots, x_k$ independently and uniformly from a set of size N , and let C be the event that some two of them collide, i.e. $x_i = x_j$ for some $i < j$. Then*

$$\Pr[C] \leq \binom{k}{2} \frac{1}{N} = \frac{k(k-1)}{2N},$$

and, on the other side,

$$\Pr[C] = 1 - \prod_{i=0}^{k-1} \left(1 - \frac{i}{N}\right) \geq 1 - e^{-k(k-1)/(2N)}.$$

In particular a collision becomes likely once $k = \Theta(\sqrt{N})$.

Proof. Upper bound. For $i < j$ let A_{ij} be the event $x_i = x_j$. Since x_i and x_j are independent and uniform, $\Pr[A_{ij}] = \sum_v \Pr[x_i = v] \Pr[x_j = v] = N \cdot (1/N)^2 = 1/N$. Now $C = \bigcup_{i < j} A_{ij}$, a union of $\binom{k}{2}$ events, one per two-element subset of the indices (Lemma 6.12), so the union bound (Theorem 11.16) gives $\Pr[C] \leq \binom{k}{2}/N$.

Lower bound. The complement $\bar{C}$ is the event that all k samples are distinct. If $k > N$, the pigeonhole principle forces a collision, so $\Pr[\bar{C}] = 0$; the product also vanishes, through its factor $1 - N/N$, so the stated equality holds and the exponential bound is trivial. Let then $k \leq N$, and for $0 \leq i \leq k$ let D_i be the event that the first i samples are pairwise distinct ($D_0 = D_1 = \Omega$, and $D_k = \bar{C}$). We show $\Pr[D_i] = \prod_{j=0}^{i-1} (1 - j/N)$ by induction on $i \leq k$; every factor is positive, since $j \leq k-1 \leq N-1$, so in particular each D_i has positive probability and conditioning on it is legitimate (Definition 11.5). For the step, the events pinning the first i samples to specific distinct values partition D_i , each with probability $N^{-i} > 0$ by independence and uniformity; given

any one of them, the next sample avoids the i values taken with probability $1 - i/N$, again by independence and uniformity. Averaging over the cells (total probability, Proposition 11.6) gives $\Pr[D_{i+1} \mid D_i] = 1 - i/N$, and $\Pr[D_{i+1}] = \Pr[D_{i+1} \mid D_i] \Pr[D_i]$ completes the induction. Taking $i = k$ yields the stated equality, in which every factor is nonnegative, so the inequality $1 - x \leq e^{-x}$ applies factor by factor. (For $0 \leq x < 1$ this inequality follows from Bernoulli's inequality, $(1 - x/n)^n \geq 1 - x$ for $n > x$, by letting $n \rightarrow \infty$ in the limit definition of e^{-x} ; for $x \geq 1$ the left side is nonpositive.) Thus

$$\Pr[\overline{C}] = \prod_{i=0}^{k-1} \left(1 - \frac{i}{N}\right) \leq \prod_{i=0}^{k-1} e^{-i/N} = \exp\left(-\frac{1}{N} \sum_{i=0}^{k-1} i\right) = e^{-k(k-1)/(2N)},$$

summing the exponents via $\sum_{i=0}^{k-1} i = k(k-1)/2$. Taking complements gives $\Pr[C] = 1 - \Pr[\overline{C}] \geq 1 - e^{-k(k-1)/(2N)}$. $\square$

Example 11.19. With $N = 365$ and $k = 23$, the classical birthday problem, the exact product gives $\Pr[C] = 0.5073$, a probability above one half, and the proposition brackets it as $0.5000 \leq \Pr[C] \leq 0.6932$. The threshold appears at small scale as well: with $N = 16$, the fifth sample, the first beyond $\sqrt{16} = 4$, raises the collision probability above one half, $\Pr[C] = 0.5001$ at $k = 5$.

Remark 11.20. The threshold $k \approx \sqrt{N}$ is the reason an n -bit hash function (so $N = 2^n$) offers only about $2^{n/2}$ collision resistance: an adversary expects a collision after roughly $2^{n/2}$ random evaluations — for $N = 2^{256}$, that is $\sqrt{2^{256}} = 2^{128}$. Output lengths are therefore chosen twice the desired security level. The same square-root phenomenon recurs in random-walk discrete-log attacks (Pollard's rho, Proposition 10.30), so the curves used in Halo 2 and Orchard are sized so that $\sqrt{N}$ itself lies far beyond feasible computation.

11.5 Statistical distance

Statistical distance measures how far apart two distributions are, such as a real key-generation procedure and an idealised uniform one; it formalises “no statistical test can distinguish them well”.

Definition 11.21 (Statistical distance). Let P and Q be probability distributions on the same finite set S (i.e. probability mass functions $P, Q: S \rightarrow [0, 1]$). Their *statistical distance* (or *total variation distance*) is

$$\Delta(P, Q) = \frac{1}{2} \sum_{s \in S} |P(s) - Q(s)|.$$

For random variables X, Y with values in S , $\Delta(X, Y)$ denotes the statistical distance of their distributions.

The factor $\frac{1}{2}$ makes Δ range over $[0, 1]$ and makes it equal to the maximum advantage of a distinguisher, as part (3) of the next theorem establishes.

Theorem 11.22 (Properties of statistical distance). Let P, Q, R be distributions on a finite set S . Then:

1. $0 \leq \Delta(P, Q) \leq 1$; and $\Delta(P, Q) = 0$ if and only if $P = Q$.
2. (Metric.) Δ is symmetric and satisfies the triangle inequality $\Delta(P, R) \leq \Delta(P, Q) + \Delta(Q, R)$.
3. (Variational characterisation.) $\Delta(P, Q) = \max_{A \subseteq S} (P(A) - Q(A)) = \max_{A \subseteq S} |P(A) - Q(A)|$, where $P(A) = \sum_{s \in A} P(s)$.
4. (Data processing / post-processing.) For any (possibly randomised) function f applied to a sample, $\Delta(f(P), f(Q)) \leq \Delta(P, Q)$. In particular no algorithm can increase statistical distance.

Proof. (1) Nonnegativity holds termwise. For the upper bound, split S into $S^+ = \{s : P(s) \geq Q(s)\}$ and $S^- = S \setminus S^+$. Since $\sum_s P(s) = \sum_s Q(s) = 1$, we have $\sum_s (P(s) - Q(s)) = 0$, so $\sum_{s \in S^+} (P(s) - Q(s)) = \sum_{s \in S^-} (Q(s) - P(s)) =: D \geq 0$. Hence $\Delta(P, Q) = \frac{1}{2} (\sum_{S^+} (P - Q) + \sum_{S^-} (Q - P)) = D$. As $D = \sum_{S^+} (P(s) - Q(s)) \leq \sum_{S^+} P(s) \leq 1$, we get $\Delta \leq 1$. If $\Delta = 0$ then every $|P(s) - Q(s)| = 0$, so $P = Q$; the converse is immediate.

(2) Symmetry holds termwise. For the triangle inequality, $|P(s) - R(s)| \leq |P(s) - Q(s)| + |Q(s) - R(s)|$ termwise by the triangle inequality on $\mathbb{R}$; summing over s and halving gives the claim.

(3) With D as above, take $A = S^+$: then $P(A) - Q(A) = \sum_{S^+} (P - Q) = D = \Delta(P, Q)$, so the maximum is at least Δ . Conversely, for any $A \subseteq S$, $P(A) - Q(A) = \sum_{s \in A} (P(s) - Q(s)) \leq \sum_{s \in A \cap S^+} (P(s) - Q(s)) \leq D = \Delta$, since dropping the negative terms only increases the sum and extending from $A \cap S^+$ to all of S^+ increases it further. Thus the maximum over A of $P(A) - Q(A)$ is exactly Δ ; replacing A by its complement swaps the sign, giving the same value for $\max_A |P(A) - Q(A)|$.

(4) First take f deterministic, $f: S \rightarrow U$. For $u \in U$, $f(P)(u) = P(f^{-1}(u))$. Then

$$\Delta(f(P), f(Q)) = \frac{1}{2} \sum_u |P(f^{-1}(u)) - Q(f^{-1}(u))| \leq \frac{1}{2} \sum_u \sum_{s \in f^{-1}(u)} |P(s) - Q(s)| = \Delta(P, Q),$$

using $|\sum_s a_s| \leq \sum_s |a_s|$ on each fibre, and that the fibres $f^{-1}(u)$ partition S . A randomised f is a deterministic function of the sample s together with independent fresh coins r ; apply the deterministic case to the joint variable (s, r) , noting that the two joint distributions $P \times U_r$ and $Q \times U_r$ have the same statistical distance $\Delta(P, Q)$, the shared coin distribution contributing nothing. $\square$

Remark 11.23 (Distinguishing interpretation). Part (3) states that $\Delta(P, Q)$ is exactly the best advantage of any test — even a computationally unbounded one — that, given a single sample, must guess whether it came from P or Q : the optimal test outputs “ P ” on the set $A = S^+$, and its advantage $\Pr_P[A] - \Pr_Q[A]$ equals $\Delta(P, Q)$. Part (4) makes statistical distance compose in proofs: if $\Delta(P, Q) \leq \varepsilon$, then P and Q are interchangeable in any analysis at the cost of an additive ε in every probability. When ε is negligible (Definition 11.61 below), we call P and Q *statistically indistinguishable*.

11.6 Uniform sampling and the bias of modular reduction

The most tractable source of randomness is a stream of independent uniform bits, whereas cryptography requires uniform elements of other finite sets: a scalar in $\mathbb{Z}/q\mathbb{Z}$ (Section 2), a field element of $\mathbb{F}_p$ (Section 6), a point in a group. If the target set S has $|S| = 2^\ell$, reading ℓ uniform bits as a number in $\{0, \dots, 2^\ell - 1\}$ and indexing into S is exact. When $|S|$ is not a power of two — most importantly when $S = \mathbb{Z}/q\mathbb{Z}$ for a prime q , the scalar field of an elliptic curve (Section 10) — a common method is *reduce-modulo*: sample a long uniform integer and reduce it mod q . Some residues then receive one more preimage than others; the next proposition bounds the resulting bias.

Proposition 11.24 (Bias of modular reduction). *Let $q \geq 2$ be an integer and let $K = 2^L$ for a nonnegative integer L . Sample U uniformly from $\{0, 1, \dots, K - 1\}$ (equivalently, read L uniform bits as an integer), and set $R = U \bmod q \in \{0, 1, \dots, q - 1\}$. Let $\mathcal{U}_q$ denote the uniform distribution on $\{0, \dots, q - 1\}$. Then the statistical distance of R from uniform satisfies*

$$\Delta(R, \mathcal{U}_q) \leq \frac{q}{K} = \frac{q}{2^L}.$$

Consequently, if $q < 2^n$ and one takes $L = n + s$ bits, then $\Delta(R, \mathcal{U}_q) < 2^{-s}$.

Proof. Write $K = mq + t$ with $m = \lfloor K/q \rfloor$ and remainder $t = K \bmod q$, so $0 \leq t < q$. For a residue $r \in \{0, \dots, q - 1\}$, the number of integers in $\{0, \dots, K - 1\}$ congruent to r modulo q is $m + 1$ if $r < t$ and m if $r \geq t$. Hence

$$\Pr[R = r] = \begin{cases} (m + 1)/K, & r < t, \\ m/K, & r \geq t. \end{cases}$$

Comparing with the target probability $1/q$ and using $m/K \leq 1/q \leq (m + 1)/K$ (since $mq \leq K \leq (m + 1)q$),

$$\left| \Pr[R = r] - \frac{1}{q} \right| \leq \frac{m + 1}{K} - \frac{m}{K} = \frac{1}{K} \quad \text{for every } r.$$

Therefore

$$\Delta(R, \mathcal{U}_q) = \frac{1}{2} \sum_{r=0}^{q-1} \left| \Pr[R = r] - \frac{1}{q} \right| \leq \frac{1}{2} \cdot q \cdot \frac{1}{K} = \frac{q}{2K} \leq \frac{q}{K}.$$

(The computation yields the sharper bound $q/(2K)$; q/K is the usual stated form.) For the final claim, $q < 2^n$ and $K = 2^{n+s}$ give $q/K < 2^n/2^{n+s} = 2^{-s}$. $\square$

Example 11.25. Let $L = 6$ uniform bits be reduced modulo $q = 5$. Then $K = 64 = 12 \cdot 5 + 4$, so the four residues $r < 4$ receive 13 preimages each and the residue $r = 4$ receives 12; the statistical distance from uniform is exactly $1/80$, within the guarantee 2^{-3} of the proposition (as $q < 2^3$ and $s = 6 - 3 = 3$).

Remark 11.26 (Sampling scalars in practice). By the proposition, a scalar modulo a ~ 256 -bit prime q with statistical distance below 2^{-128} from uniform is obtained by reducing $256 + 128 = 384$ uniform bits modulo q . The bias 2^{-s} , with the slack s set to the security parameter, is negligible (Definition 11.61), so the reduce-modulo sample is statistically indistinguishable from a

perfectly uniform scalar, and by the data-processing inequality (Theorem 11.22(4)) substituting one for the other anywhere costs at most 2^{-128} per use. An alternative, *rejection sampling*, draws $\lceil \log_2 q \rceil$ bits, accepts if the result is $< q$, and otherwise retries; it gives *exactly* uniform output at the cost of a variable but (with overwhelming probability) small number of retries. For this series the negligible bias of reduce-modulo is acceptable and simpler to analyse. The Orchard specification employs this reduce-modulo sampling when deriving scalars from hash outputs: its ToScalar reduces the full 512-bit PRFexpand output modulo the 255-bit group order, a slack of 257 bits, well beyond the 128-bit slack computed above (Zcash protocol specification §4.2.3 and §5.4.2).

11.7 Beyond finite spaces: countable additivity, limits, and densities

Two uses require more than a finite sample space. A process observed until an event occurs, such as an unbounded retry loop or a random walk run until it first hits a barrier, has no finite horizon, even when it terminates with probability one; a waiting time measured in continuous units has no finite sample space. This subsection adds countable additivity, continuity along monotone events, and laws defined by densities, and states in Remark 11.31 the measure-theoretic results it assumes.

Definition 11.27 (Countable probability space). A *countable probability space* is a pair $(\Omega, \Pr)$ in which Ω is a countable set and $\Pr: \Omega \rightarrow [0, 1]$ satisfies $\sum_{\omega \in \Omega} \Pr[\omega] = 1$; the sum is independent of the enumeration order because its terms are nonnegative. Events are arbitrary subsets $A \subseteq \Omega$, with $\Pr[A] = \sum_{\omega \in A} \Pr[\omega]$, and Definitions 11.2–11.11 carry over with “finite” read as “countable” (a random variable’s image may now be countably infinite), with one exception: Definition 11.4 does not — equal weights cannot sum to 1 on a countably infinite set, so the uniform distribution remains a strictly finite notion. Expectation additionally requires its defining sum to converge absolutely.

Proposition 11.28 (Countable additivity). Let $(\Omega, \Pr)$ be a countable probability space and $A_1, A_2, \dots$ pairwise disjoint events. Then

$$\Pr\left[\bigcup_{n \geq 1} A_n\right] = \sum_{n \geq 1} \Pr[A_n],$$

and in particular the basic axioms of Proposition 11.3 hold unchanged.

Proof. Every outcome of the union lies in exactly one A_n , so both sides sum the same nonnegative terms $\Pr[\omega]$, grouped differently; a series of nonnegative terms may be summed in any order, or in groups, without changing its value. $\square$

Theorem 11.29 (Continuity along monotone events). Let $\Pr$ be a probability assignment, on any sample space, defined for a family of events closed under complement and countable union, and countably additive on it. If $A_1 \subseteq A_2 \subseteq \dots$ is an increasing chain of events, then

$$\Pr\left[\bigcup_{n \geq 1} A_n\right] = \lim_{n \rightarrow \infty} \Pr[A_n],$$

and dually $\Pr[\bigcap_n B_n] = \lim_n \Pr[B_n]$ for a decreasing chain $B_1 \supseteq B_2 \supseteq \dots$. On a countable space the hypothesis holds automatically, by Proposition 11.28.

Proof. Disjointify by telescoping: set $D_1 = A_1$ and $D_n = A_n \setminus A_{n-1}$ for $n \geq 2$. The D_n are pairwise disjoint, $\bigcup_{k \leq n} D_k = A_n$, and $\bigcup_n D_n = \bigcup_n A_n$. Countable additivity gives

$$\Pr\left[\bigcup_n A_n\right] = \sum_{n \geq 1} \Pr[D_n] = \lim_{n \rightarrow \infty} \sum_{k=1}^n \Pr[D_k] = \lim_{n \rightarrow \infty} \Pr[A_n],$$

the final step by finite additivity. The decreasing case follows by complementation. $\square$

Definition 11.30 (Infinite sequence of independent trials). Fix a finite outcome set T and a distribution p on T . An *infinite sequence of independent trials* with law p is a sequence of random variables $X_1, X_2, \dots$ such that every prefix $(X_1, \dots, X_n)$ consists of n mutually independent p -distributed trials in the sense of Definition 11.10 — equivalently, the prefix's law is that of the finite probability space T^n carrying the product weights $\Pr[(t_1, \dots, t_n)] = p(t_1) \cdots p(t_n)$. An event is *finitely determined* if membership in it is decided by some fixed finite prefix; its probability is the finite-space value. An event of the form “some prefix satisfies P ” is the union of an increasing chain of finitely determined events, and Theorem 11.29 assigns it the limit of the prefix probabilities.

Remark 11.31 (Assumed measure-theoretic results). Definition 11.30 presumes that a single, countably additive probability assignment on infinite sequences exists that restricts correctly to every finite prefix at once. That existence statement is the Kolmogorov extension theorem, and its proof belongs to measure theory, which lies off this series' critical path: the space of infinite sequences over at least two possible outcomes is uncountable, so it escapes Definition 11.27, and the extension specifies the admissible events. This series therefore takes it, with its countable additivity and hence Theorem 11.29, as an assumption rather than a theorem. On infinite-sequence spaces, only finitely determined events and their monotone limits appear in this series, and for those the values are determined by the finite theory together with continuity; the density-defined laws of Definition 11.32 rest on the analogous assumption, their interval and joint-event probabilities being taken to cohere as the stated integrals. The independent families of Definition 11.40 extend that assumption in two ways: joint events of several density-defined quantities are taken to be iterated integrals, in whichever order of integration is convenient — the exchange of order for nonnegative integrands is the Fubini–Tonelli theorem, likewise measure-theoretic — and the gap sequences of Definition 11.44 are infinite families, of which only finite prefixes enter any event this series uses, except through countable unions over the values of an arrival count — $N(t)$ of Definition 11.44, or the count X of Theorem 11.51, each event “count = x ” being a joint event of finitely many gaps. For any set B of values, the event “count $\in B$ ” — among them the event that the count is finite, and the tail events of Theorem 11.49 — is taken to have probability $\sum_{x \in B} \Pr[\text{count} = x]$, by the countable additivity assumed above.

Definition 11.32 (Law defined by a density). A real-valued random quantity X has *density* $f: \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}$ if $\int_{-\infty}^{\infty} f(t) dt = 1$ and

$$\Pr[a \leq X \leq b] = \int_a^b f(t) dt \quad \text{for all } a \leq b;$$

improper Riemann integrals suffice throughout this series. The *distribution function* is $F(t) = \Pr[X \leq t] = \int_{-\infty}^t f$, so $\Pr[X > t] = 1 - F(t)$, and single points carry probability zero. The expectation is $E[X] = \int_{-\infty}^{\infty} t f(t) dt$ when this integral converges absolutely. Two density-defined quantities X, Y are *independent* if $\Pr[X \in I, Y \in J] = \Pr[X \in I] \Pr[Y \in J]$ for all intervals I, J ; probabilities of joint events are then taken to factor through iterated integrals of the product $f_X(s) f_Y(t)$.

11.7.1 Waiting times: tail sums, geometric trials, and Wald's identity. The results of this unit need only countable expectation and finitely determined events. For a variable with values in $\mathbb{N} \cup \{\infty\}$, the defining sum of the expectation has nonnegative terms, and the value $+\infty$ is permitted when it diverges.

Lemma 11.33 (Tail-sum formula). *Let T be a random variable with values in $\mathbb{N} \cup \{\infty\}$ and $\Pr[T = \infty] = 0$. Then $E[T] = \sum_{k \geq 1} \Pr[T \geq k]$, either side being infinite exactly when the other is.*

Proof. Expanding $\Pr[T \geq k] = \sum_{m \geq k} \Pr[T = m]$ and regrouping the doubly indexed nonnegative series, each term $\Pr[T = m]$ is counted once for every $k \leq m$, that is, m times; nonnegative series may be regrouped freely. $\square$

Proposition 11.34 (Geometric waiting time). *In an infinite sequence of independent trials, each succeeding with probability $\varepsilon > 0$, let T be the index of the first success. Then $\Pr[T \geq k] = (1 - \varepsilon)^{k-1}$ and $E[T] = 1/\varepsilon$.*

Proof. The event $T \geq k$ says the first $k - 1$ trials all fail — a finitely determined event of probability $(1 - \varepsilon)^{k-1}$; in particular $\Pr[T = \infty] = \lim_k (1 - \varepsilon)^{k-1} = 0$ by continuity (Theorem 11.29). The tail-sum formula then gives $E[T] = \sum_{k \geq 1} (1 - \varepsilon)^{k-1} = 1/\varepsilon$, a geometric series. $\square$

Proposition 11.35 (Wald's identity). *Let $X_1, X_2, \dots$ be an infinite sequence of independent trials with common law, each $X_i \geq 0$ with finite mean, and let T be a stopping rule: a random index such that, for every k , whether $T = k$ is decided by the first k trials alone. If $E[T] < \infty$, then*

$$E\left[\sum_{i=1}^T X_i\right] = E[T] \cdot E[X_1].$$

Proof. Write $S = \sum_{i=1}^T X_i$ and $S_k = \sum_{i=1}^k X_i$, so that $S = S_k$ on the event $T = k$. The hypothesis $E[T] < \infty$ forces $\Pr[T = \infty] = 0$, by the convention preceding Lemma 11.33. The variable S takes countably many values — on $\{T = k\}$ it agrees with S_k , whose values run over a finite set, the trials having finitely many outcomes — and each event $\{S = s\}$ is the disjoint countable union $\bigcup_k \{T = k, S_k = s\}$ of finitely determined events, so its probability is assigned by countable additivity (Remark 11.31). By $E[S]$ we mean $\sum_s s \Pr[S = s]$, the expectation of this countably supported distribution, a sum of nonnegative terms.

Expanding each $\Pr[S = s]$ and regrouping the doubly indexed nonnegative series,

$$\mathbb{E}[S] = \sum_s s \sum_{k \geq 1} \Pr[T = k, S_k = s] = \sum_{k \geq 1} \mathbb{E}[S_k \mathbf{1}_{T=k}],$$

each inner expectation living on the finite prefix space of the first k trials (Definition 11.30), where linearity (Theorem 11.14) expands it as $\sum_{i=1}^k \mathbb{E}[X_i \mathbf{1}_{T=k}]$. Regrouping the nonnegative double series once more,

$$\mathbb{E}[S] = \sum_{i \geq 1} \sum_{k \geq i} \mathbb{E}[X_i \mathbf{1}_{T=k}].$$

Fix i and let $n \geq i$. The events $T = k$ (for $k \leq n$), $T > n$, and $T \geq i$ are all decided by the first n trials, and on that prefix space the pointwise identity $\mathbf{1}_{T \geq i} = \sum_{k=i}^n \mathbf{1}_{T=k} + \mathbf{1}_{T > n}$ holds, so linearity gives

$$\sum_{k=i}^n \mathbb{E}[X_i \mathbf{1}_{T=k}] = \mathbb{E}[X_i \mathbf{1}_{T \geq i}] - \mathbb{E}[X_i \mathbf{1}_{T > n}].$$

The trials take finitely many values, so $X_i \leq M$ for a constant M , and $0 \leq \mathbb{E}[X_i \mathbf{1}_{T > n}] \leq M \Pr[T > n] \rightarrow 0$ as $n \rightarrow \infty$: the terms $\Pr[T \geq k]$ of the convergent series $\mathbb{E}[T]$ tend to zero. Hence $\sum_{k \geq i} \mathbb{E}[X_i \mathbf{1}_{T=k}] = \mathbb{E}[X_i \mathbf{1}_{T \geq i}]$.

The remaining step is a factorisation. The event $T \geq i$ — the negation of $T \leq i - 1$ — is decided by the first $i - 1$ trials, so its indicator is a function $g(t_1, \dots, t_{i-1})$ of the first $i - 1$ trial values, and on the prefix space of the first i trials the expectation splits along the product weights of Definition 11.30: with p the common law,

$$\mathbb{E}[X_i \mathbf{1}_{T \geq i}] = \sum_{t_1, \dots, t_i} g(t_1, \dots, t_{i-1}) t_i p(t_1) \dots p(t_i) = \Pr[T \geq i] \cdot \mathbb{E}[X_i],$$

the double sum factoring into the sum over $(t_1, \dots, t_{i-1})$, which totals $\Pr[T \geq i]$, times $\sum_{t_i} t_i p(t_i) = \mathbb{E}[X_i]$. Assembling the pieces and applying the tail-sum formula (Lemma 11.33),

$$\mathbb{E}[S] = \sum_{i \geq 1} \mathbb{E}[X_i] \Pr[T \geq i] = \mathbb{E}[X_1] \cdot \mathbb{E}[T]. \quad \square$$

11.7.2 Exponential clocks, Poisson counts, and the Chernoff method. Block discovery is modelled by *clocks*: each party's next block arrives after a random delay whose remaining law does not depend on how long the delay has already run, and two parties race. A light client that samples a chain needs a tail bound on the probability that an adversary has found cL blocks while the honest chain found L , and a wallet that spaces its network queries needs a delay law with a prescribed mean and no memory. This unit constructs these clocks and proves the tail bounds from density-defined laws (Definition 11.32) and Markov's inequality with a free parameter.

Definition 11.36 (Exponential law). A random quantity X is *exponential with rate* $\lambda > 0$, written $X \sim \text{Exp}(\lambda)$, if it has the density $f(t) = \lambda e^{-\lambda t}$ for $t \geq 0$ and $f(t) = 0$ for $t < 0$.

Lemma 11.37 (Tail, mean, and memorylessness). Let $X \sim \text{Exp}(\lambda)$. Then $\Pr[X > t] = e^{-\lambda t}$ for all $t \geq 0$, $\mathbb{E}[X] = 1/\lambda$, and

$$\Pr[X > s + t \mid X > s] = \Pr[X > t] \quad \text{for all } s, t \geq 0,$$

conditional probability being the ratio of Definition 11.5: a wait without an arrival leaves the law of the remaining wait unchanged. Conversely, a nonnegative quantity with $\Pr[X > t] = e^{-\lambda t}$ for all $t \geq 0$ is exponential with rate λ .

Proof. The density integrates to 1, and $\Pr[X > t] = 1 - \int_0^t \lambda e^{-\lambda s} ds = e^{-\lambda t}$. Integrating by parts,

$$\mathbb{E}[X] = \int_0^\infty t \lambda e^{-\lambda t} dt = [-te^{-\lambda t}]_0^\infty + \int_0^\infty e^{-\lambda t} dt = \frac{1}{\lambda}.$$

Since $X > s + t$ implies $X > s$, the conditional probability is $e^{-\lambda(s+t)} / e^{-\lambda s} = e^{-\lambda t}$. For the converse, the events $X > a$ and $X > b$ ($a \leq b$) differ by $a < X \leq b$, so $\Pr[a < X \leq b] = e^{-\lambda a} - e^{-\lambda b} = \int_a^b \lambda e^{-\lambda t} dt$; and the point a carries probability at most $e^{-\lambda(a-\delta)} - e^{-\lambda a}$ for every $\delta > 0$, hence zero, so $\Pr[a \leq X \leq b]$ is the same integral. $\square$

Remark 11.38 (Exponential clocks as limits of geometric waiting times). The exponential law is the continuous limit of the geometric waiting time (Proposition 11.34). If trials succeed with a small probability ε and each occupies time ε/λ , the first success arrives at time $\varepsilon T/\lambda$, and for integer-valued T

$$\Pr[\varepsilon T/\lambda > t] = \Pr[T \geq \lfloor \lambda t/\varepsilon \rfloor + 1] = (1 - \varepsilon)^{\lfloor \lambda t/\varepsilon \rfloor} \longrightarrow e^{-\lambda t} \quad (\varepsilon \rightarrow 0),$$

because $(1 - \varepsilon)^{1/\varepsilon} \rightarrow e^{-1}$ and the floor alters the exponent by less than one. Whether a proof of work satisfies this model, with each candidate independent of the last, depends on the trials; the volumes on consensus treat that question.

Example 11.39 (A jittered schedule). A wallet that must poll each of n addresses about once a day waits $\Delta \sim \text{Exp}(1/T)$ between polls, with $T = 86400/n$ seconds: $\mathbb{E}[\Delta] = T$ by Lemma 11.37, one expected poll per address per day. For $n = 3$, $T = 28\,800$ seconds, eight hours; the gap exceeds a full day with probability $e^{-3} = 0.0498$, and exceeds twice its mean with probability $e^{-2} = 0.1353$ whatever T is. By memorylessness, an observer who has seen no poll for some time obtains no information about the time of the next one.

Clock races and clock restarts both involve joint events of several density-defined quantities, which Definition 11.32 provides only for pairs.

Definition 11.40 (Independent family of density-defined quantities). Density-defined quantities $X_1, \dots, X_n$ with densities $f_1, \dots, f_n$ form an *independent family* if $\Pr[X_1 \in I_1, \dots, X_n \in I_n] = \prod_i \Pr[X_i \in I_i]$ for all intervals $I_1, \dots, I_n$. As in Definition 11.32, the probability of a joint event $(X_1, \dots, X_n) \in R$, for a region $R \subseteq \mathbb{R}^n$ cut out by finitely many inequalities, is then taken to be the iterated integral of $f_1(s_1) \dots f_n(s_n)$ over R , in whichever order of integration is convenient (Remark 11.31). In particular a quantity that is a function of one block of the family is independent of a quantity that is a function of a disjoint block: integrate the two blocks separately. A sequence $X_1, X_2, \dots$ is an independent family if every finite prefix is one, in the manner of Definition 11.30.

Lemma 11.41 (Integrating out one quantity). *Let U be a density-defined quantity, with density f_U , that is a function of the first k members of an independent family $X_1, \dots, X_n$, and write $W = (X_{k+1}, \dots, X_n)$. Let R be a region in the (U, W) -coordinates whose section $\{s : (s, w) \in R\}$ is an interval for every fixed value w of W . Then*

$$\Pr[(U, W) \in R] = \int_{-\infty}^{\infty} f_U(s) \Pr[W \in R_s] ds, \quad R_s = \{w : (s, w) \in R\},$$

the inner probability being a joint event of $X_{k+1}, \dots, X_n$ alone.

Proof. Integrate the product density over the first k coordinates first, holding w fixed: the result is the probability that U lies in the interval $\{s : (s, w) \in R\}$, which by Definition 11.32 equals $\int f_U(s) \mathbf{1}[(s, w) \in R] ds$. What remains is the integral of $f_{k+1} \dots f_n$ against this function of w ; exchanging the order and integrating over w first leaves, for each s , the inner probability $\Pr[W \in R_s]$. $\square$

Lemma 11.42 (Density of a sum). *Let U, V be independent density-defined quantities, non-negative (their densities vanish on the negative axis). Then $U + V$ is density-defined, with density the convolution $(f_U * f_V)(x) = \int_0^x f_U(u) f_V(x - u) du$ for $x \geq 0$.*

Proof. For $0 \leq a \leq b$, the event $a \leq U + V \leq b$ is the region $\{(u, v) : u, v \geq 0, a \leq u + v \leq b\}$, so

$$\Pr[a \leq U + V \leq b] = \int_0^\infty f_U(u) \int_{\max(a-u, 0)}^{\max(b-u, 0)} f_V(v) dv du = \int_0^\infty f_U(u) \int_{\max(a, u)}^{\max(b, u)} f_V(x - u) dx du,$$

substituting $x = u + v$ in the inner integral. Exchanging the order, the region becomes $a \leq x \leq b$, $0 \leq u \leq x$, and the right-hand side is $\int_a^b (f_U * f_V)(x) dx$. With $a = 0$ and $b = \infty$ the left-hand side is 1, so $f_U * f_V$ is a density. $\square$

Lemma 11.43 (Two-clock race). *Let $X \sim \text{Exp}(\lambda)$ and $Y \sim \text{Exp}(\mu)$ be independent. Then, for every $u \geq 0$,*

$$\Pr[X < Y, X \leq u] = \frac{\lambda}{\lambda + \mu} (1 - e^{-(\lambda + \mu)u}),$$

symmetrically with the roles of (X, λ) and (Y, μ) exchanged, and $\Pr[X = Y] = 0$. Consequently $\Pr[X < Y] = \lambda/(\lambda + \mu)$; the winning time $\min(X, Y)$ is exponential with rate $\lambda + \mu$; and the winner is independent of the winning time,

$$\Pr[X < Y, \min(X, Y) \leq u] = \Pr[X < Y] \cdot \Pr[\min(X, Y) \leq u] \quad \text{for all } u \geq 0.$$

Proof. The event is the region $0 \leq s \leq u, t > s$ in the (X, Y) -plane. Integrating over t first (Definition 11.40),

$$\Pr[X < Y, X \leq u] = \int_0^u \lambda e^{-\lambda s} \Pr[Y > s] ds = \int_0^u \lambda e^{-(\lambda + \mu)s} ds = \frac{\lambda}{\lambda + \mu} (1 - e^{-(\lambda + \mu)u}).$$

The diagonal $s = t$ has inner integral zero, so $\Pr[X = Y] = 0$, and $u = \infty$ gives $\Pr[X < Y]$. On the event $X < Y$ the winning time is X , so the displayed formula is $\Pr[X < Y, \min(X, Y) \leq u]$; adding

its mirror for $Y < X$ gives $\Pr[\min(X, Y) \leq u] = 1 - e^{-(\lambda+\mu)u}$, whence $\min(X, Y)$ is exponential with rate $\lambda + \mu$ by the characterisation in Lemma 11.37, and the formula is exactly the product $\Pr[X < Y] \cdot \Pr[\min(X, Y) \leq u]$. Differencing at two values of u extends the factorisation from half-lines to intervals. $\square$

Definition 11.44 (Exponential clock process). An *exponential clock of rate λ* — a *Poisson process*, by Theorem 11.47 — is an independent family $G_1, G_2, \dots$ of $\text{Exp}(\lambda)$ quantities, the *gaps*. Its *arrival times* are the partial sums $S_n = G_1 + \dots + G_n$, with $S_0 = 0$, and its *arrival count* up to time $t \geq 0$ is $N(t) = \sup\{n : S_n \leq t\}$, the number of arrivals in $[0, t]$, a value in $\mathbb{N} \cup \{\infty\}$ that Theorem 11.47 shows finite with probability one. The event $N(t) = n$ is $S_n \leq t < S_{n+1}$, a joint event of the first $n + 1$ gaps.

Lemma 11.45 (Arrival-time density). *The n -th arrival time S_n of an exponential clock of rate λ has density $f_n(s) = \lambda^n s^{n-1} e^{-\lambda s} / (n-1)!$ for $s \geq 0$.*

Proof. Induction on n : f_1 is the exponential density, and $S_{n+1} = S_n + G_{n+1}$ with S_n a function of the first n gaps, hence independent of G_{n+1} (Definition 11.40), so Lemma 11.42 gives

$$f_{n+1}(s) = \int_0^s \frac{\lambda^n u^{n-1} e^{-\lambda u}}{(n-1)!} \lambda e^{-\lambda(s-u)} du = \frac{\lambda^{n+1} e^{-\lambda s}}{(n-1)!} \int_0^s u^{n-1} du = \frac{\lambda^{n+1} s^n e^{-\lambda s}}{n!}. \quad \square$$

Definition 11.46 (Poisson law). A random variable K with values in $\mathbb{N}$ is *Poisson with mean $\nu > 0$* if $\Pr[K = k] = e^{-\nu} \nu^k / k!$ for every $k \geq 0$. The weights sum to 1 by the exponential series $e^\nu = \sum_{k \geq 0} \nu^k / k!$, and $\mathbb{E}[K] = \sum_{k \geq 1} k e^{-\nu} \nu^k / k! = \nu e^{-\nu} \sum_{k \geq 1} \nu^{k-1} / (k-1)! = \nu$, so the parameter is the mean.

An arrival count is a function of the gaps of a clock family, so it lives on no finite or countable space, and the expectation of Definition 11.11 does not literally apply to it. The following convention covers $N(t)$ above and the count X of Theorem 11.51 below. Let X be a variable with values in $\mathbb{N} \cup \{\infty\}$ that is a function of the gaps, each event $X = x$ with x finite being a joint event of finitely many gaps, with the probability of Definition 11.40. Events $X \in B$, for sets B of finite values, have the probabilities $\sum_{x \in B} \Pr[X = x]$ (Remark 11.31), and when X is finite with probability 1 its *mean* is defined by the law,

$$\mathbb{E}[X] = \sum_{x \geq 0} x \Pr[X = x],$$

a sum of nonnegative terms, permitted to be $+\infty$ as in the convention preceding Lemma 11.33. On a finite or countable space this sum is the second expression of Definition 11.11, the expectation regrouped by value (Definition 11.27), so the two definitions agree whenever both apply.

Theorem 11.47 (Poisson arrival counts). *For an exponential clock of rate λ and every $t \geq 0$, the arrival count $N(t)$ is Poisson with mean λt :*

$$\Pr[N(t) = n] = e^{-\lambda t} \frac{(\lambda t)^n}{n!}, \quad n = 0, 1, 2, \dots$$

In particular $N(t)$ is finite with probability 1 and $\mathbb{E}[N(t)] = \lambda t$: a rate- λ clock delivers λ arrivals per unit time on average.

Proof. For $n = 0$ the event is $G_1 > t$, of probability $e^{-\lambda t}$. For $n \geq 1$ the event $S_n \leq t < S_n + G_{n+1}$ is, in the (S_n, G_{n+1}) -coordinates, the region $s \leq t, g > t - s$, whose section at fixed g is the interval $(t - g, t]$; the quantity S_n is a function of the first n gaps with the density of Lemma 11.45, so Lemma 11.41 gives

$$\Pr[N(t) = n] = \int_0^t \frac{\lambda^n s^{n-1} e^{-\lambda s}}{(n-1)!} \Pr[G_{n+1} > t - s] ds = \frac{\lambda^n e^{-\lambda t}}{(n-1)!} \int_0^t s^{n-1} ds = e^{-\lambda t} \frac{(\lambda t)^n}{n!}.$$

The event $N(t) < \infty$ is the disjoint union of the events $N(t) = n$, so its probability is the sum of the weights (Remark 11.31), which is 1 (Definition 11.46); and the mean — the sum over the law, by the convention above — is the one computed in Definition 11.46. $\square$

Markov's inequality applied to X itself gives a bound decaying only like $1/a$; applied to e^{tX} , it gives a bound decaying exponentially, with the free parameter t optimised for the threshold. This is the Chernoff method.

Definition 11.48 (Moment generating function). For a real-valued random variable X with finite or countable image — one on a finite or countable space, or an arrival count of a clock family in the sense of the convention preceding Theorem 11.47, restricted to its finite values (the value ∞ has probability zero there), whose law is countably additive over its values: $\Pr[X \in B] = \sum_{x \in B} \Pr[X = x]$ for every set B of values (Remark 11.31) — and a real t , the *moment generating function* is

$$M_X(t) = \sum_x e^{tx} \Pr[X = x] \in (0, \infty],$$

the sum running over the finite values of X ; its terms are positive, so it either converges or is $+\infty$. On a countable space it is $\mathbb{E}[e^{tX}]$, the expectation regrouped by value, and $M_X(0) = \sum_x \Pr[X = x] = 1$.

Theorem 11.49 (The Chernoff method). *Let X be as above, a real, and $t > 0$ with $M_X(t) < \infty$. Then*

$$\Pr[X \geq a] \leq e^{-ta} M_X(t),$$

and hence $\Pr[X \geq a] \leq \inf_t e^{-ta} M_X(t)$, the infimum over the $t > 0$ at which M_X is finite.

Proof. The events $X = x$ are disjoint, so $\Pr[X \geq a] = \sum_{x \geq a} \Pr[X = x]$ by countable additivity (Proposition 11.28, or the clause of Definition 11.48 for a clock family). Since $t > 0$, each term with $x \geq a$ satisfies $\Pr[X = x] \leq e^{t(x-a)} \Pr[X = x]$, and summing gives $\Pr[X \geq a] \leq e^{-ta} \sum_x e^{tx} \Pr[X = x]$, the terms with $x < a$ being nonnegative. This is Markov's inequality (Proposition 11.15) applied to e^{tX} , written on the law of X . $\square$

Corollary 11.50 (Chernoff bound for a Poisson variable). *Let K be Poisson with mean ν and let $a > \nu$. Then $M_K(t) = \exp(\nu(e^t - 1))$ for every real t , and*

$$\Pr[K \geq a] \leq e^{a-\nu} \left(\frac{\nu}{a}\right)^a.$$

In particular, for $\nu = \mu L$ and $a = cL$ with $c > \mu$, the bound reads $e^{L(c-\mu)} (c/\mu)^{-cL}$.

Proof. Expanding, $M_K(t) = \sum_k e^{tk} e^{-\nu} \nu^k / k! = e^{-\nu} \sum_k (\nu e^t)^k / k! = e^{-\nu} e^{\nu e^t}$. The exponent $\phi(t) = -ta + \nu(e^t - 1)$ of the Chernoff bound has $\phi'(t) = -a + \nu e^t$ and $\phi''(t) = \nu e^t > 0$, so it is minimised at $e^t = a/\nu$, a positive t exactly when $a > \nu$; there $\phi = -a \log(a/\nu) + a - \nu$, with $\log$ the natural logarithm — base-free $\log$ is natural throughout, base two being written $\log_2$ — and exponentiating gives the bound. Substituting $\nu = \mu L$ and $a = cL$ gives $e^{cL-\mu L}(\mu/c)^{cL}$. $\square$

Theorem 11.51 (Negative-binomial arrival count). *Let two exponential clocks be independent — their gaps together form one independent family — with rates λ , the honest clock, and μ , the adversary clock, and let X be the number of adversary arrivals strictly before the L -th honest arrival, for a fixed $L \geq 1$. Then X is finite with probability 1, with the negative binomial law*

$$\Pr[X = m] = \binom{m+L-1}{m} p^L q^m, \quad p = \frac{\lambda}{\lambda + \mu}, \quad q = \frac{\mu}{\lambda + \mu}, \quad m = 0, 1, 2, \dots,$$

of mean $Lq/p = L\mu/\lambda$, and, for every real t with $\mu e^t < \lambda + \mu$,

$$M_X(t) = \left(\frac{\lambda}{\lambda + \mu - \mu e^t} \right)^L;$$

with the honest rate normalised to 1 this is $M_X(t) = (1/(1 + \mu - \mu e^t))^L$. Only the ratio μ/λ enters.

Proof. Write S_L for the L -th honest arrival time, a function of the honest gaps with the density f_L of Lemma 11.45, and N_A for the adversary's arrival count. The event $X = m$ differs from $N_A(S_L) = m$ only on the events $S_L = A_1 + \dots + A_m$ and $S_L = A_1 + \dots + A_{m+1}$, with A_i the adversary gaps; each of these, in the coordinates $(S_L, A_1, \dots, A_{m+1})$, has a single point as its section at fixed adversary gaps, hence probability zero (Definition 11.32), so $\Pr[X = m] = \Pr[N_A(S_L) = m]$. The latter event is the region $A_1 + \dots + A_m \leq S_L < A_1 + \dots + A_{m+1}$, whose section at fixed adversary gaps is an interval. By Lemma 11.41 and Theorem 11.47,

$$\begin{aligned} \Pr[X = m] &= \int_0^\infty f_L(s) \Pr[N_A(s) = m] ds = \frac{\lambda^L \mu^m}{(L-1)! m!} \int_0^\infty s^{m+L-1} e^{-(\lambda+\mu)s} ds \\ &= \frac{\lambda^L \mu^m}{(L-1)! m!} \cdot \frac{(m+L-1)!}{(\lambda + \mu)^{m+L}}, \end{aligned}$$

using $\int_0^\infty s^k e^{-\alpha s} ds = k!/\alpha^{k+1}$ for $\alpha > 0$, which k integrations by parts establish. Rearranging the factorials gives the binomial coefficient and the powers of p and q .

For the generating function, set $x = qe^t$, so that $0 \leq x < 1$ exactly when $\mu e^t < \lambda + \mu$. Then $M_X(t) = p^L \sum_{m \geq 0} \binom{m+L-1}{m} x^m$, and we claim the series is $(1-x)^{-L}$, by induction on L . For $L = 1$ it is the geometric series. For the step, write $T_L(n) = \sum_{m=0}^n \binom{m+L-1}{m} x^m$ for the partial sums. Pascal's rule $\binom{m+L}{m} = \binom{m+L-1}{m-1} + \binom{m+L-1}{m}$ — sort the m -subsets of $m+L$ items, which the coefficients count (Lemma 6.12), by whether they contain a distinguished item — gives, on subtracting $x T_{L+1}(n)$ from $T_{L+1}(n)$ term by term,

$$(1-x) T_{L+1}(n) = T_L(n) - \binom{n+L}{n} x^{n+1}.$$

Hence $T_{L+1}(n) \leq T_L(n)/(1-x) \leq (1-x)^{-L-1}$ by the induction hypothesis: the partial sums $T_{L+1}(n)$ increase and are bounded, so the series converges, and its terms $\binom{n+L}{n}x^n$ tend to zero. Letting $n \rightarrow \infty$ in the display gives $(1-x) \sum_{m \geq 0} \binom{m+L}{m}x^m = (1-x)^{-L}$, the claim for $L+1$. Hence $M_X(t) = (p/(1-qe^t))^L$, which is the displayed form after multiplying numerator and denominator by $\lambda + \mu$. At $t = 0$ this gives $\sum_m \Pr[X = m] = 1$; the event $X < \infty$ is the disjoint union of the events $X = m$, so its probability is this sum (Remark 11.31), and X is finite with probability 1. For the mean — the sum over the law, by the convention preceding Theorem 11.47 — $m \binom{m+L-1}{m} = L \binom{m+L-1}{m-1}$, so $E[X] = Lq p^L \sum_{k \geq 0} \binom{k+L}{k} q^k = Lq p^L (1-q)^{-L-1} = Lq/p$ by the same series with $L+1$ in place of L . $\square$

Remark 11.52 (The coin-sequence form). The same law arises without clocks. In an infinite sequence of independent trials (Definition 11.30) succeeding with probability p , the number of failures strictly before the L -th success equals m exactly when the $(m+L)$ -th trial succeeds and, among the first $m+L-1$, precisely m fail: $\binom{m+L-1}{m}$ orders, one per choice of the m failing positions (Lemma 6.12), each of probability $p^{L-1}q^m \cdot p$. The two-clock race and memorylessness (Lemma 11.43 and Lemma 11.37) identify the clock model's attribution sequence — which clock fires next — with such a coin sequence of success probability $\lambda/(\lambda + \mu)$; later volumes carry out that identification.

Theorem 11.53 (Chernoff bound for the arrival count). *In the setting of Theorem 11.51 with honest rate 1 and adversary rate μ , for every $c > \mu$,*

$$\Pr[X \geq cL] \leq \left[\left(\frac{1+c}{1+\mu} \right)^{1+c} \left(\frac{\mu}{c} \right)^c \right]^L,$$

the Chernoff bound optimised at $e^t = c(1+\mu)/(\mu(1+c))$. The base in brackets is strictly less than 1, so the bound decays exponentially in L .

Proof. The count X is finite with probability one with values in $\mathbb{N}$ (Theorem 11.51) and has a law countably additive over its values (Remark 11.31), so Theorem 11.49 applies: with Theorem 11.51, $\Pr[X \geq cL] \leq e^{-tcL} M_X(t) = e^{L\phi(t)}$ with $\phi(t) = -tc - \log(1+\mu - \mu e^t)$, for $0 < t < \log((1+\mu)/\mu)$. On that interval

$$\phi'(t) = -c + \frac{\mu e^t}{1+\mu - \mu e^t}, \quad \phi''(t) = \frac{\mu e^t(1+\mu)}{(1+\mu - \mu e^t)^2} > 0,$$

so ϕ' is strictly increasing (positive derivative) and vanishes at most once, namely where $\mu e^t(1+\mu) = c(1+\mu)$, that is, at $e^{t^*} = c(1+\mu)/(\mu(1+c))$; this lies in the interval because $c > \mu$ gives $e^{t^*} > 1$ and $c/(1+c) < 1$ gives $e^{t^*} < (1+\mu)/\mu$. Thus ϕ decreases before t^* and increases after it, and $\phi(t^*)$ is its minimum on the interval. At t^* , $1+\mu - \mu e^{t^*} = (1+\mu)/(1+c)$, so $e^{\phi(t^*)} = (e^{t^*})^{-c} (1+c)/(1+\mu)$, which is the bracketed base. Finally $\phi(0) = 0$ and ϕ decreases on $[0, t^*]$, since $\phi'(0) = \mu - c < 0$, so $\phi(t^*) < 0$: the base is below 1. $\square$

Example 11.54 (The Poisson expression is not a bound for the arrival count). With the honest rate 1, the arrival count X has mean μL , the same as the Poisson variable of Corollary 11.50 with $\nu = \mu L$, which counts adversary arrivals in a window of fixed length L ; the two laws differ because

the window here ends at the random time S_L . Substituting one for the other is an error. At $\mu = 1/2$ and $c = 3/4$, the exact tail $\Pr[X \geq cL]$ (summing Theorem 11.51), the bound of Theorem 11.53, and the Poisson expression $e^{L(c-\mu)}(c/\mu)^{-cL}$ are

L	exact tail	clock bound	Poisson expression
40	$5.05 \cdot 10^{-2}$	$2.53 \cdot 10^{-1}$	$1.15 \cdot 10^{-1}$
200	$1.09 \cdot 10^{-4}$	$1.04 \cdot 10^{-3}$	$2.00 \cdot 10^{-5}$
400	$8.23 \cdot 10^{-8}$	$1.08 \cdot 10^{-6}$	$4.00 \cdot 10^{-10}$

At $L = 200$ and $L = 400$ the Poisson expression lies *below* the exact tail, so it is not an upper bound for the arrival count; its base per unit of L is 0.9473, against 0.9662 for the valid bound.

Later volumes build block-discovery and extraction analyses on these results: block clocks are the exponential clocks above, their races and arrival counts the lemmas of this unit, block-attribution sequences are infinite sequences of independent trials, extractor running times are the waiting times of the previous unit (the geometric waiting time and Wald's identity), and the limit arguments instantiate Theorem 11.29.

11.8 Asymptotic notation

Efficiency and hardness are stated by comparing how functions of an integer parameter grow as that parameter tends to infinity, in the Bachmann–Landau notation. Throughout, functions map $\mathbb{N}$ (or a cofinite subset of it) to the nonnegative reals $\mathbb{R}_{\geq 0}$, and “for sufficiently large n ” means “for all n beyond some threshold n_0 ”.

Definition 11.55 (Big-O, Omega, Theta, little-o, little-omega). Let $f, g: \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ with g eventually positive.

- $f = O(g)$ (read “ f is big-O of g ”, an asymptotic upper bound) if there exist constants $c > 0$ and n_0 with $f(n) \leq c g(n)$ for all $n \geq n_0$.
- $f = \Omega(g)$ (asymptotic lower bound) if there exist $c > 0$ and n_0 with $f(n) \geq c g(n)$ for all $n \geq n_0$; equivalently, $f = \Omega(g)$ if and only if $g = O(f)$.
- $f = \Theta(g)$ (tight bound) if $f = O(g)$ and $f = \Omega(g)$; i.e. there are $c_1, c_2 > 0$ and n_0 with $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for $n \geq n_0$.
- $f = o(g)$ (read “little-o”, strictly smaller order) if for every $c > 0$ there is n_0 with $f(n) \leq c g(n)$ for all $n \geq n_0$; equivalently $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$.
- $f = \omega(g)$ (strictly larger order) if $g = o(f)$; equivalently $\lim_{n \rightarrow \infty} f(n)/g(n) = \infty$.

Remark 11.56. The “=” in “ $f = O(g)$ ” is traditional but denotes set membership: $O(g)$ is the class of all functions bounded above by a constant multiple of g , and $f = O(g)$ means $f \in O(g)$. Such equations are not read right to left. We follow the customary abuse of notation.

Example 11.57. The polynomial $3n^2 + 10n + 7$ is $\Theta(n^2)$: for $n \geq 1$ one has $3n^2 \leq 3n^2 + 10n + 7 \leq 20n^2$, with equality on the right at $n = 1$, so the constant 20 is exact there. Also $n^2 = o(n^3)$, since

$n^2/n^3 = 1/n \rightarrow 0$; $\log_2 n = o(n)$; $n^{100} = o(2^n)$ (any polynomial is little-o of any exponential with base > 1); and $n! = \omega(2^n)$. A constant factor does not change the order: $2^n = o(2^{n+1})$ is *false*, but $2^n = \Theta(2^{n+1})$ is true, since $2^{n+1} = 2 \cdot 2^n$ is within a constant factor.

Proposition 11.58 (Useful closure facts). *Let $f_1 = O(g_1)$ and $f_2 = O(g_2)$. Then $f_1 + f_2 = O(\max(g_1, g_2)) = O(g_1 + g_2)$ and $f_1 f_2 = O(g_1 g_2)$. The same closure statements hold with O replaced uniformly by Ω or Θ . Moreover $O(\cdot)$ is transitive: $f = O(g)$ and $g = O(h)$ imply $f = O(h)$.*

Proof. Choose witnesses (c_1, n_1) and (c_2, n_2) for the two hypotheses and let $n_0 = \max(n_1, n_2)$. For $n \geq n_0$, $f_1(n) + f_2(n) \leq c_1 g_1(n) + c_2 g_2(n) \leq (c_1 + c_2) \max(g_1(n), g_2(n))$, and $\max(g_1, g_2) \leq g_1 + g_2 \leq 2 \max(g_1, g_2)$, so the two right-hand forms agree up to a constant; this proves the sum rule. For products, $f_1(n) f_2(n) \leq c_1 c_2 g_1(n) g_2(n)$. Transitivity: $f \leq c g$ and $g \leq c' h$ eventually give $f \leq cc' h$ eventually. The Ω statements follow by the equivalence $f = \Omega(g) \Leftrightarrow g = O(f)$, and Θ by combining the two. $\square$

11.9 Polynomial, exponential, and negligible functions

Fix a distinguished variable $\lambda \in \mathbb{N}$, the *security parameter* (§11.11); informally, larger λ means more security and larger keys. Resources (running time, key length) and success probabilities are functions of λ . This subsection classifies growth rates in λ and proves (Proposition 11.63) that the *negligible* functions, those shrinking faster than the reciprocal of every polynomial, are closed under the operations that security proofs perform, addition and multiplication by polynomials; this is why “negligible” formalises “ignorable”.

Definition 11.59 (Polynomial and super-polynomial). A function $f: \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is *polynomially bounded*, written $f = \text{poly}(\lambda)$, if there is a constant c (and a threshold) with $f(\lambda) \leq \lambda^c + c$ for all large λ ; equivalently, $f(\lambda) = O(\lambda^c)$ for some constant c . It is *super-polynomial* if it grows faster than every polynomial, i.e. $f(\lambda) = \omega(\lambda^c)$ for every constant c (equivalently, $\lambda^c = o(f(\lambda))$ for all c).

Definition 11.60 (Exponential and sub-exponential). A function f is *exponential* if $f(\lambda) = 2^{\Omega(\lambda)}$; the prototypical case is $f(\lambda) = 2^{c\lambda}$ for a constant $c > 0$. A super-polynomial function with $f(\lambda) = 2^{o(\lambda)}$ is *sub-exponential*; examples are $\lambda^{\log_2 \lambda} = 2^{(\log_2 \lambda)^2}$ and the index-calculus running time $L_p[1/3] = 2^{\Theta(\lambda^{1/3}(\log \lambda)^{2/3})}$ in the bit length $\lambda = \log_2 p$ (Remark 7.22). Every exponential function is super-polynomial; the converse fails by these examples.

Definition 11.61 (Negligible function). A function $\nu: \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is *negligible*, written $\nu = \text{negl}(\lambda)$, if for every positive constant c there exists λ_0 such that

$$\nu(\lambda) < \frac{1}{\lambda^c} \quad \text{for all } \lambda \geq \lambda_0.$$

Equivalently, $\nu(\lambda) = o(\lambda^{-c})$ for every constant $c > 0$; equivalently, $\lambda^c \nu(\lambda) \rightarrow 0$ as $\lambda \rightarrow \infty$ for every constant c . A function is *non-negligible* if it is not negligible, i.e. $f(\lambda) \geq \lambda^{-c}$ for infinitely many λ , for some constant c ; it is *noticeable* if there exist c and λ_0 with $f(\lambda) \geq \lambda^{-c}$ for all $\lambda \geq \lambda_0$ (noticeable implies non-negligible, but not conversely); it is *overwhelming* if $1 - f$ is negligible.

Example 11.62. The functions $2^{-\lambda}$, $2^{-\sqrt{\lambda}}$, and $\lambda^{-\log \lambda}$ are negligible. For $2^{-\lambda}$: given any c , the product $2^{-\lambda} \lambda^c = \lambda^c / 2^\lambda$ tends to 0 (a polynomial over an exponential), meeting the definition. By contrast, $1/\lambda^{100}$ is *not* negligible — taking $c = 101$ shows $1/\lambda^{100} > 1/\lambda^{101}$ for all $\lambda \geq 2$ — it is noticeable, even though it tends to 0. Tending to zero therefore does not suffice.

Proposition 11.63 (Closure properties of negligible functions). *Let ν, μ be negligible and let p be a polynomially bounded function. Then:*

1. $\nu + \mu$ is negligible (the class is closed under addition, hence under any fixed finite sum).
2. $p \cdot \nu$ is negligible (a polynomial times a negligible function is negligible).
3. If $f(\lambda) \leq \nu(\lambda)$ for all large λ , then f is negligible (domination by a negligible function).
4. Summing $p(\lambda)$ functions, each bounded by a single negligible function ν , yields a negligible function; in particular a union bound over polynomially many negligible events is negligible.

Proof. The proof rests on an exponent shift: to obtain a bound below λ^{-c} , negligibility is invoked at a strictly larger exponent, so that the surplus absorbs the combining factor. The same shift underlies the hybrid and union-bound steps of security proofs.

(1) Fix c . Applying the definition at the exponent $c + 1$, there is a threshold beyond which $\nu(\lambda), \mu(\lambda) < \lambda^{-(c+1)} \leq \frac{1}{2} \lambda^{-c}$ (the latter once $\lambda \geq 2$); adding the two bounds gives $\nu(\lambda) + \mu(\lambda) < \lambda^{-c}$ for all large λ .

(2) Suppose $p(\lambda) \leq \lambda^d$ for large λ (and some constant d). Fix c . By negligibility applied at the exponent $c + d$, $\nu(\lambda) < \lambda^{-(c+d)}$ for large λ , whence $p(\lambda)\nu(\lambda) < \lambda^d \lambda^{-(c+d)} = \lambda^{-c}$ for large λ . Hence $p\nu$ is negligible.

(3) Immediate: if $f \leq \nu$ eventually and $\nu < \lambda^{-c}$ eventually, then $f < \lambda^{-c}$ eventually, for every c .

(4) Write the sum $\Sigma(\lambda) = \sum_{i=1}^{p(\lambda)} \nu_i(\lambda)$, where each $\nu_i \leq \nu$ by hypothesis (in security proofs all the ν_i arise from one bound). Then $\Sigma(\lambda) \leq p(\lambda) \nu(\lambda)$, which is negligible by (2); conclude by (3). $\square$

Remark 11.64 (Why negligible is the threshold of security). Suppose an adversary wins a game with negligible probability $\nu(\lambda)$. If it repeats the attack $p(\lambda)$ times (polynomially many, the most an efficient adversary can afford), the probability that some repetition succeeds is, by the union bound, at most $p(\lambda)\nu(\lambda)$ — still negligible by Proposition 11.63(2). Thus negligible success probability remains negligible under any efficient amplification, and may therefore be ignored. Conversely, a noticeable success probability $\geq \lambda^{-c}$ can be boosted to a constant by λ^c repetitions. The negligible/non-negligible split formalises the phrase “the scheme is secure except with negligible probability”: security asserts a negligible advantage, and an attack means a non-negligible one. (A merely non-negligible — not noticeable — success probability is boosted to a constant only on the infinitely many λ where the inverse-polynomial bound holds, which already suffices to contradict a security definition.)

11.10 Algorithms, running time, and PPT

This subsection makes “efficient algorithm” and “infeasible” precise in the standard model: algorithms are Turing machines (equivalently, up to polynomial factors, programs in any reasonable model of computation), inputs and outputs are finite binary strings, and the measured resource is the number of elementary steps as a function of input length. Write $\{0, 1\}^*$ for the set of all finite binary strings and $|x|$ for the length of $x \in \{0, 1\}^*$.

Definition 11.65 (Deterministic polynomial-time algorithm). An algorithm M runs in (*deterministic*) *polynomial time* if there is a polynomial p such that, on every input $x \in \{0, 1\}^*$, M halts within at most $p(|x|)$ steps and produces an output. The class of decision problems solvable by such machines is P .

Key generation and sampling require randomness, which a probabilistic algorithm reads from an auxiliary tape of independent uniform random bits, its “coins”.

Definition 11.66 (Probabilistic algorithm and PPT). A *probabilistic* (or *randomised*) algorithm M is a Turing machine with an additional read-only *random tape* initialised with independent uniformly random bits $r \in \{0, 1\}^{\mathbb{N}}$. For a fixed input x , the output $M(x)$ is a random variable over the choice of r ; write $M(x; r)$ to display the coins explicitly, and then $M(x; \cdot)$ is a deterministic function. The algorithm runs in *probabilistic polynomial time* (it is *PPT*) if there is a polynomial p such that, for every input x and every choice of coins r , M halts within $p(|x|)$ steps. (Bounding the time for every r , not merely in expectation, is the standard “strict” notion and ensures M ever reads only $p(|x|)$ coins.)

Remark 11.67 (Non-uniformity and circuits). There are two standard ways to formalise an efficient adversary. The *uniform* model is a single PPT machine that works for all input lengths, as above. The *non-uniform* model allows a separate algorithm (equivalently, a Boolean circuit) of size $\text{poly}(\lambda)$ for each λ , which may be hard-wired with an arbitrary polynomial-length “advice” string depending on λ . Non-uniform adversaries are at least as powerful as uniform ones and are often more convenient in reduction proofs, because the advice can encode a “best” attack. The asymptotic theory of this section is identical for both models; security definitions in the Halo 2/Orchard setting typically state security against non-uniform PPT adversaries, the more conservative choice.

The hardness notion below rests on the Cobham–Edmonds thesis: “efficiently computable” means “computable in polynomial time”. Feasibility and infeasibility are opposite regimes but not logical negations: intermediate success probabilities can fall into neither class.

Definition 11.68 (Computational infeasibility). A computational task parameterised by the security parameter λ is (*computationally*) *feasible* if there is a PPT algorithm solving it with overwhelming success probability. It is *infeasible* if for every PPT algorithm $\mathcal{A}$ the probability that $\mathcal{A}$ solves it is negligible in λ :

$$\Pr[\mathcal{A} \text{ solves the task on parameter } \lambda] = \text{negl}(\lambda),$$

where the probability is over the task’s randomness and $\mathcal{A}$ ’s coins. Equivalently: no efficient algorithm succeeds with non-negligible probability.

Remark 11.69 (Reading a hardness assumption). Cryptographic assumptions are infeasibility statements. For instance, the discrete-logarithm assumption in a family of groups $\{\mathbb{G}_\lambda\}$ of order $q(\lambda)$ asserts: for every PPT $\mathcal{A}$,

$$\Pr[\mathcal{A}(g, g^x) = x] = \text{negl}(\lambda),$$

the probability taken over a uniformly chosen generator g of $\mathbb{G}_\lambda$, a uniform $x \leftarrow \mathbb{Z}/q\mathbb{Z}$, and $\mathcal{A}$'s coins. The security of Halo 2 and Orchard rests on such asymptotic infeasibility assumptions; choosing a parameter size does not prove them. Separately, concrete analysis asks how much work the best known attacks need at one fixed size. Generic discrete-log attacks take $\Theta(\sqrt{q})$ group operations by the birthday/rho phenomenon of Remark 11.20; absent a faster structure-specific attack, an order near $2^{2\lambda}$ therefore targets about λ bits of generic-group security. The Pasta orders lie slightly above 2^{254} , giving roughly 2^{127} generic work. The asymptotic language permits the terms “negligible” and “polynomial”; concrete estimates remain conditional on the attacks presently known.

11.11 The security parameter

This subsection fixes the parameter in which negligibility, efficiency, and infeasibility are measured, and states what the “128-bit security” claim of the section opening asserts.

Definition 11.70 (Security parameter). The *security parameter* is a positive integer $\lambda \in \mathbb{N}$ supplied to all algorithms of a cryptographic scheme, conventionally in unary as the string 1^λ (so that “polynomial in the input length” coincides with “polynomial in λ ”). It governs simultaneously:

- the *sizes* of objects — key lengths and encodings, each $\text{poly}(\lambda)$; in a generic discrete-log instantiation one commonly chooses a group order near $2^{2\lambda}$;
- the *running time* of honest algorithms, which must be $\text{poly}(\lambda)$; and
- the *security level*: every PPT adversary's advantage must be $\text{negl}(\lambda)$.

Remark 11.71 (Unary encoding of the security parameter). Passing 1^λ (a string of λ ones, of length λ) rather than the integer λ in binary (length $\log_2 \lambda$) is a deliberate convention: it forces “polynomial time in the input length” to mean polynomial in λ itself, so an honest key generator may take time $\text{poly}(\lambda)$ to produce λ -bit keys. In a scheme satisfying the security definition, increasing λ raises honest cost only polynomially while every PPT adversary's success remains negligible; in common concrete families, the best known attack cost grows exponentially. Asymptotic security is the statement that this gap holds for all sufficiently large λ ; the practitioner then fixes a single value (commonly $\lambda = 128$, targeting 2^{128} -operation attacks) and checks the concrete estimates against that target: the Pallas/Vesta group orders, slightly above 2^{254} , give roughly 2^{127} generic group operations (Section 10; Zcash protocol specification §5.4.9.6), hash outputs of ≥ 256 bits per the birthday bound of Proposition 11.18, and sampling slack of at least 128 extra bits per Proposition 11.24 (the deployed ToScalar of Remark 11.26 reduces 512 bits, a slack of 257). Every “negligible” in the remainder of the series is implicitly a function of this λ .

The “128-bit security” claim of the section opening has two separate layers. Asymptotic infeasibility (Definition 11.68) requires the success probability of every probabilistic polynomial-time forger, uniform or hard-wired with advice (Remark 11.67), to be negligible in λ , a bound that no polynomial number of repetitions amplifies to a noticeable probability (Remark 11.64); it holds conditionally on the stated cryptographic assumptions and does not exclude the positive success probability of a forger that guesses a key. A concrete “128-bit” estimate states that the best known applicable attack at a fixed parameter size costs on the order of 2^{128} operations; it is an attack estimate, not a theorem about every possible algorithm.

II

Crypto Guide

Cryptographic primitives from scratch

Abstract

This volume of *The Zcash Arboretum* constructs the cryptographic primitives from which a shielded protocol is built: each is defined, constructed, and proved secure against an explicitly defined adversary. It assumes the *Math Guide*, whose groups, finite fields, elliptic curves, probability, and model of computation are used throughout without re-derivation.

Every guarantee in the volume is a claim about an adversary, and every security definition is a game: a challenger, an efficient adversary, and an advantage that must be negligible. A proof is a reduction that converts any adversary with non-negligible advantage into a solver for a long-studied computational problem, with the loss of the reduction accounted for down to a concrete bit-security budget. Within this framework the volume treats, in turn, the following adversaries: one who recovers the exponent behind a public group element, and with it every key; one who finds two documents with a common digest, or reuses a digest outside its intended role; a receiver who learns a committed value before the commitment is opened, and a committer who opens a commitment to a value other than the one committed; one who, given oracle access, distinguishes a function keyed by a short secret from a uniformly random function; one who controls a channel and reads, splices, or forges the messages that pass over it; one who interposes between two parties and terminates both ends of a conversation that each party believes secure; one who produces a signature that the signer never made; a cheating prover who convinces a verifier of a false statement, and a curious verifier who extracts from a transcript information about the witness it hides; and one who proves membership of an element in a set that does not contain it. The primitives that meet these definitions—hash functions, commitments, pseudorandom functions, authenticated encryption, key agreement, signatures, zero-knowledge arguments, and Merkle trees—are the primitives on which the deployed protocol relies.

1 The provable-security model

This section makes “the scheme is secure” a statement with mathematical content; every later section states its security definitions and proofs in these terms. It defines the security parameter

and the adversary, indistinguishability of probability ensembles, security games and advantage, reductions and their loss, the standard and random oracle models, and concrete n -bit security, and proves the hierarchy of indistinguishability notions (Proposition 1.10) and the hybrid lemma (Lemma 1.13). A security claim is a universally quantified statement about a precisely delimited class of adversaries; the groups, fields and curves of the *Math Guide* carry no security guarantee by themselves. Unconditional proofs of security are almost never available, since one would typically imply $P \neq NP$ —that problems whose solutions can be checked in polynomial time are not all solvable in polynomial time—and considerably more. Security is instead *reduced* to the conjectured hardness of computational problems that have resisted decades of attack, such as factoring, discrete logarithm and decisional Diffie–Hellman: any adversary that breaks the scheme can be transformed mechanically into an algorithm that solves the underlying problem.

1.1 Adversaries and the security parameter

The vocabulary of efficiency is that of the closing section of the *Math Guide*, recalled here with the refinements cryptography needs.

The *security parameter* $\lambda \in \mathbb{N}$ is a positive integer that controls the sizes of all objects in a cryptographic scheme—key lengths, group orders, output lengths—and is conventionally supplied to every algorithm in unary, written 1^λ , the string of λ ones, so that an algorithm running in time polynomial in the length of its input automatically receives time polynomial in λ (*Math Guide*, §“*The security parameter*”). Every scheme in this monograph is accordingly not a single object but an infinite *family* of objects indexed by λ .

Schemes are families because security is asymptotic: no scheme with a fixed finite key length is unbreakable, since an adversary with enough time enumerates a finite key space. The requirement is that, as λ grows, the costs of the honest parties (key generation, encryption, signing) grow polynomially while the cost of the best attack grows super-polynomially, ideally exponentially; security is a statement about the limit $\lambda \rightarrow \infty$.

The *unary* encoding replaces the $\lceil \log_2 \lambda \rceil$ -bit binary representation of λ because complexity theory measures running time as a function of input *length*: in binary, of length $\Theta(\log \lambda)$, the parameter would allow an algorithm polynomial in its input length only $\text{poly}(\log \lambda)$ steps—too few to write down a λ -bit key. The convention is a normalisation only; no substantive result depends on it.

Remark 1.1. Every quantity in this volume—the adversary’s success probability, its running time, the key length—is a *function* of λ , even where the notation suppresses the dependence. A single number—“the advantage is at most 2^{-128} ”—either fixes a concrete λ or describes the function’s behaviour for the λ of interest.

An algorithm $\mathcal{A}$ is *probabilistic polynomial-time* (PPT) if it is a probabilistic Turing machine—one equipped with a read-only tape of independent uniformly random bits, its *random tape*—and there is a polynomial p such that on every input x , *and for every setting of the random tape*, $\mathcal{A}$ halts within $p(|x|)$ steps: the strict, worst-case notion of the *Math Guide* (§“*Algorithms, running time, and PPT*”). We write $y \leftarrow \mathcal{A}(x)$ for the random variable obtained by running $\mathcal{A}$ on x with freshly sampled randomness, and $y := \mathcal{A}(x; r)$ for the deterministic output when the random tape is fixed to the string r . The honest algorithms and the adversary alike are modelled as PPT machines.

Polynomial time. Polynomial time abstracts “feasible computation”: it is closed under composition and invariant across all reasonable deterministic computational models (the strong Church–Turing thesis). The *honest* parties run in polynomial time so that the scheme is usable; security asserts that *no* polynomial-time attack succeeds.

Randomness. Honest algorithms need randomness: deterministic encryption leaks whether two ciphertexts encrypt the same message, and key generation must sample secrets unpredictably. The adversary is granted randomness as well, both so that it has the resources of the honest parties and because a deterministic adversary is the special case in which the random tape is ignored.

This volume adopts as standard the non-uniform strengthening that the *Math Guide* mentions (§“Algorithms, running time, and PPT”).

Definition 1.2 (Non-uniform PPT). A *non-uniform* PPT adversary is a family $\{\mathcal{A}_\lambda\}_{\lambda \in \mathbb{N}}$ of probabilistic circuits (or, equivalently, Turing machines each supplied with an *advice string* z_λ of length $\text{poly}(\lambda)$) such that a single polynomial in λ bounds the size of $\mathcal{A}_\lambda$. The advice may depend arbitrarily on λ but not on the particular inputs drawn at run time.

Remark 1.3. The distinction between uniform adversaries (a single machine that reads 1^λ and works for all λ) and non-uniform ones (a separate circuit per λ , or a uniform machine with per- λ advice) is substantive: non-uniformity grants the adversary free precomputed advice tailored to each parameter size, for example a short hint about the group in use for that parameter. Security against non-uniform adversaries is the stronger and now-standard requirement, partly because non-uniform models compose more cleanly under reduction (one may “hard-wire” a fixed good choice of randomness or auxiliary input). All definitions below can be read in either model; hardness assumptions are stated against non-uniform adversaries where the distinction matters, and the choice is otherwise left implicit.

Remark 1.4 (Expected versus strict polynomial time, and quantum adversaries). Two refinements recur. First, some definitions allow *expected* polynomial-time adversaries, bounding the average rather than worst-case running time; this matters for certain extraction arguments in zero-knowledge, and is flagged where it arises. Second, for post-quantum security one replaces PPT by *quantum* polynomial-time (QPT)—polynomial-size circuits of quantum gates followed by a measurement, a model this volume does not formalise; the consequences for the assumptions this volume relies on are taken up under “Quantum adversaries: Shor’s algorithm” in the next section. The structural definitions of this section (advantage, indistinguishability, reduction) are agnostic to which class of adversaries one quantifies over; only the underlying hardness assumptions change.

1.2 Distribution ensembles and indistinguishability

The most basic security goal is that no adversary *distinguishes two situations*: a real protocol transcript from a simulated one, an encryption of m_0 from an encryption of m_1 , a pseudorandom string from a truly random one. The situations are formalised as *ensembles* of probability distributions.

Definition 1.5 (Probability ensemble). A *probability ensemble* is a sequence $X = \{X_\lambda\}_{\lambda \in \mathbb{N}}$ of probability distributions (equivalently, random variables), one for each value of the security parameter, where X_λ is supported on binary strings whose length is bounded by a polynomial in λ . There is often an additional index—an input a —written $\{X_\lambda(a)\}$; for clarity we suppress it unless needed.

The statistical distance of the *Math Guide* (§“Statistical distance”) is recalled for countable supports, which ensembles require; the *Math Guide* treats finite ones.

Definition 1.6 (Statistical distance). Let X, Y be discrete random variables over a common finite or countable set S . Their *statistical distance* (or total variation distance) is

$$\Delta(X, Y) := \frac{1}{2} \sum_{s \in S} |\Pr[X = s] - \Pr[Y = s]|.$$

Proposition 1.7 (Variational characterisation). For X, Y over S ,

$$\Delta(X, Y) = \max_{T \subseteq S} (\Pr[X \in T] - \Pr[Y \in T]) = \max_D (\Pr[D(X) = 1] - \Pr[D(Y) = 1]),$$

where the last maximum ranges over all (even computationally unbounded, deterministic) decision functions $D : S \rightarrow \{0, 1\}$. In particular, no procedure whatsoever—efficient or not—can distinguish X from Y with advantage exceeding $\Delta(X, Y)$.

Proof. The set form is the variational characterisation in the statistical-distance theorem of the *Math Guide* (§“Statistical distance”), whose argument is unchanged over a countable S . Only the reformulation over decision functions is new: a deterministic D is the indicator of the set $T = D^{-1}(1)$, so the maximum over D equals the maximum over T ; a randomised D is a convex combination of deterministic ones and cannot do better. $\square$

Proposition 1.8 (Properties of statistical distance). The statistical distance is a metric on probability distributions: $0 \leq \Delta(X, Y) \leq 1$, it is symmetric, $\Delta(X, Y) = 0$ iff X and Y are identically distributed, and it satisfies the triangle inequality $\Delta(X, Z) \leq \Delta(X, Y) + \Delta(Y, Z)$. Moreover it never increases under (possibly randomised) post-processing: for any function—more generally, any randomised algorithm— f ,

$$\Delta(f(X), f(Y)) \leq \Delta(X, Y).$$

Proof. These are the metric and data-processing parts of the statistical-distance theorem of the *Math Guide* (§“Statistical distance”), again with the argument unchanged over a countable S . $\square$

The grades of indistinguishability below use the asymptotic vocabulary of the *Math Guide* (§“Polynomial, exponential, and negligible functions”). A function $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is *negligible*, written $f = \text{negl}(\lambda)$, if for every $c \in \mathbb{N}$ there exists λ_0 such that $f(\lambda) < \lambda^{-c}$ for all $\lambda \geq \lambda_0$. A function is *non-negligible* if it is not negligible; it is *noticeable* if there exists c with $f(\lambda) \geq \lambda^{-c}$ for all

sufficiently large λ (noticeable functions are non-negligible, but not conversely); it is *overwhelming* if $1 - f$ is negligible. The class of negligible functions is closed under addition and under multiplication by any polynomial—the closure property, proved there, on which the hybrid argument of the next subsection depends.

Definition 1.9 (Perfect, statistical, and computational indistinguishability). Let $X = \{X_\lambda\}$ and $Y = \{Y_\lambda\}$ be probability ensembles.

- The ensembles X and Y are *perfectly indistinguishable*, written $X \equiv Y$, if X_λ and Y_λ are *identically distributed* for every λ ; equivalently $\Delta(X_\lambda, Y_\lambda) = 0$ for all λ .
- The ensembles X and Y are *statistically indistinguishable*, written $X \approx_s Y$, if their statistical distance is negligible:

$$\Delta(X_\lambda, Y_\lambda) = \text{negl}(\lambda).$$

- The ensembles X and Y are *computationally indistinguishable*, written $X \approx_c Y$, if for every PPT algorithm D (the *distinguisher*), the function

$$\text{Adv}_{X,Y}^{\text{dist}}(D, \lambda) := \left| \Pr[D(1^\lambda, X_\lambda) = 1] - \Pr[D(1^\lambda, Y_\lambda) = 1] \right|$$

is negligible in λ . The probabilities are over the draw from the ensemble and over D 's internal randomness.

The three notions form a strict hierarchy, whose second strictness example needs two primitives, defined here in the form the volume uses throughout. Write U_n for the uniform distribution on $\{0, 1\}^n$. A *one-way function* is a polynomial-time computable $h: \{0, 1\}^* \rightarrow \{0, 1\}^*$ that no PPT algorithm inverts on a random input: for every PPT $\mathcal{A}$,

$$\Pr[h(\mathcal{A}(1^\lambda, h(x))) = h(x)] = \text{negl}(\lambda) \quad (x \leftarrow U_\lambda).$$

A *pseudorandom generator* (PRG) is a polynomial-time computable $G: \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$ whose output on a uniform seed is computationally indistinguishable from uniform, $\{G(U_\lambda)\} \approx_c \{U_{2\lambda}\}$; the doubling stretch is a normalisation. Whether one-way functions exist is open, which is why the second strictness below is stated conditionally.

Proposition 1.10 (Hierarchy). *For all ensembles X, Y :*

$$X \equiv Y \implies X \approx_s Y \implies X \approx_c Y.$$

The first implication is strict: there exist ensembles that are statistically but not perfectly indistinguishable. The second is strict provided one-way functions exist: under that assumption there exist ensembles that are computationally but not statistically indistinguishable.

Proof. If $X \equiv Y$ then $\Delta(X_\lambda, Y_\lambda) = 0$, which is negligible, so $X \approx_s Y$. If $X \approx_s Y$, then for any—in particular any PPT—distinguisher D , Proposition 1.7 gives $\text{Adv}_{X,Y}^{\text{dist}}(D, \lambda) \leq \Delta(X_\lambda, Y_\lambda) = \text{negl}(\lambda)$, so $X \approx_c Y$.

For strictness of the first implication, let X_λ be uniform on $\{0, 1\}^\lambda$ and let Y_λ be uniform on $\{0, 1\}^\lambda \setminus \{0^\lambda\}$. They are not identically distributed (the point 0^λ has probabilities $2^{-\lambda}$ and 0), so not perfectly indistinguishable, yet $\Delta(X_\lambda, Y_\lambda) = 2^{-\lambda} = \text{negl}(\lambda)$, so they are statistically indistinguishable.

For strictness of the second, assume one-way functions exist; then a PRG exists—the Håstad–Impagliazzo–Levin–Luby theorem, together with the routine lemma that extends any stretch to the doubling one, both used without proof—and furnishes an ensemble: let G be a PRG, let $X_\lambda := G(U_\lambda)$, and let $Y_\lambda := U_{2\lambda}$. By definition of a PRG, $X \approx_c Y$. But X_λ is supported on at most 2^λ strings out of $2^{2\lambda}$, so a set T equal to the image of G has $\Pr[X_\lambda \in T] = 1$ while $\Pr[Y_\lambda \in T] \leq 2^\lambda / 2^{2\lambda} = 2^{-\lambda}$; hence $\Delta(X_\lambda, Y_\lambda) \geq 1 - 2^{-\lambda}$, which is not negligible. Thus $X \approx_c Y$ but $X \not\approx_s Y$. $\square$

Remark 1.11 (Cost and benefit of computational indistinguishability). In the strictness example the PRG output and a random string are at statistical distance nearly 1, because the PRG image is a vanishing fraction of the codomain; an unbounded distinguisher checks membership in the image and succeeds. Computational indistinguishability asserts only that *finding* an efficient test for membership in that image is infeasible. Information-theoretic security with short keys is impossible; computational security, which rests on conjectured hardness, replaces it and permits both public-key primitives and the encryption of long messages under short keys. Statistical and perfect security, when attainable, are unconditional.

1.3 The hybrid argument

Two lemmas make indistinguishability composable: efficient post-processing cannot create a distinguishing advantage, and a polynomially long chain of pairwise-indistinguishable steps cannot accumulate one. The proof of the first is the simplest instance of a reduction.

Lemma 1.12 (Closure under efficient post-processing). *If $X \approx_c Y$ and M is any PPT algorithm, then the ensembles $\{M(1^\lambda, X_\lambda)\}$ and $\{M(1^\lambda, Y_\lambda)\}$ satisfy $\{M(X_\lambda)\} \approx_c \{M(Y_\lambda)\}$.*

Proof. Suppose some PPT distinguisher D separates $\{M(X_\lambda)\}$ from $\{M(Y_\lambda)\}$ with advantage $\delta(\lambda)$. Define $D' := D \circ M$, i.e. $D'(1^\lambda, z) := D(1^\lambda, M(1^\lambda, z))$. Since M and D are both PPT, so is D' . Then

$$\text{Adv}_{X,Y}^{\text{dist}}(D', \lambda) = |\Pr[D(M(X_\lambda)) = 1] - \Pr[D(M(Y_\lambda)) = 1]| = \delta(\lambda).$$

By $X \approx_c Y$ the left side is negligible, so δ is negligible. $\square$

The hybrid argument compares two ensembles connected by a chain of pairwise-close intermediate ensembles. It is the triangle inequality with a refinement that imprecise statements omit: the uniformity hypothesis, which explains why the number of hybrids must be polynomial.

Lemma 1.13 (Hybrid lemma). *Let $t = t(\lambda)$ be polynomially bounded, and let $H_\lambda^0, H_\lambda^1, \dots, H_\lambda^t$ be a sequence of distributions (the hybrids). Suppose that for every PPT distinguisher D the neighbouring advantages*

$$\alpha_i(D, \lambda) := |\Pr[D(H_\lambda^i) = 1] - \Pr[D(H_\lambda^{i+1}) = 1]|$$

are uniformly negligible: a single negligible function ν (depending on D) satisfies $\alpha_i(D, \lambda) \leq \nu(\lambda)$ for all $i \in \{0, \dots, t(\lambda) - 1\}$ simultaneously. Equivalently, $\max_{0 \leq i < t(\lambda)} \alpha_i(D, \lambda) = \text{negl}(\lambda)$, i.e. $\alpha_{i(\lambda)}(D, \lambda)$ is negligible along every index function $i(\lambda)$ —arbitrary, not merely efficiently computable, since the maximising index need not be efficient. Then $\{H_\lambda^0\}$ and $\{H_\lambda^t\}$ are computationally indistinguishable, i.e. for every PPT D , $|\Pr[D(H_\lambda^0) = 1] - \Pr[D(H_\lambda^t) = 1]| = \text{negl}(\lambda)$.

Proof. Fix a PPT distinguisher D and let ν be the uniform bound the hypothesis supplies for D . By the triangle inequality applied to the telescoping sum,

$$|\Pr[D(H_\lambda^0) = 1] - \Pr[D(H_\lambda^t) = 1]| = \left| \sum_{i=0}^{t-1} (\Pr[D(H_\lambda^i) = 1] - \Pr[D(H_\lambda^{i+1}) = 1]) \right| \leq \sum_{i=0}^{t-1} \alpha_i(D, \lambda),$$

and the hypothesis bounds the sum by $t(\lambda) \cdot \nu(\lambda)$, which is negligible since t is a polynomial (closure of the negligible class under polynomial multiplication). Hence $\{H^0\} \approx_c \{H^t\}$. $\square$

Uniformity in i is essential, and no argument derives it from the weaker hypothesis that each α_i is negligible for every *fixed* i : there the threshold beyond which the i -th term is small can depend on i , and the dependence cannot be removed. For a countermodel, take $t(\lambda) = \lambda + 1$ and let H_λ^i be the point mass at 0 if $i \leq \lambda$ and the point mass at 1 otherwise. For every fixed i the advantage $\alpha_i(D, \lambda)$ is nonzero only at $\lambda = i$, hence negligible; yet H_λ^0 and $H_\lambda^{t(\lambda)}$ are the point masses at 0 and 1, perfectly distinguishable.

Applications usually obtain the conclusion by a stronger, structural route. Suppose the chain arises from a *single* underlying assumption $X \approx_c Y$: neighbouring hybrids H_λ^i and H_λ^{i+1} differ only in that one designated component is drawn from X_λ in the former and from Y_λ in the latter, and each hybrid is efficiently samplable given $(1^\lambda, i)$ and a sample of that component. Then the *random-index* distinguisher D' for the pair (X, Y) , on challenge z , samples i uniformly from $\{0, \dots, t - 1\}$, builds the hybrid with z in the designated position, and runs D ; writing $p_i := \Pr[D(H_\lambda^i) = 1]$, its distinguishing advantage equals $|\frac{1}{t} \sum_{i=0}^{t-1} (p_i - p_{i+1})| = |p_0 - p_t|/t$. Since D' is a single PPT machine, the assumption makes its advantage negligible, and $|p_0 - p_t| \leq t(\lambda) \cdot \text{Adv}(D') = \text{negl}(\lambda)$. This argument bounds the total advantage of D directly, bypassing the lemma's uniform per-step hypothesis rather than discharging it; the lemma itself covers the general case in which no such common structure is available.

Remark 1.14. The hybrid argument is where the restriction to “polynomially many” steps is used. A negligible per-step advantage multiplied by a polynomial number of steps remains negligible; a negligible per-step advantage multiplied by, say, 2^λ steps need not. Hence constructions that would require exponentially many hybrids yield no security proof.

1.4 Security as a game

The standard formulation expresses a security definition as a *game* (or experiment) between a *challenger* and an adversary $\mathcal{A}$: the challenger, following a fixed protocol, sets up the scheme, answers the adversary's queries, and finally declares the adversary a winner or loser by outputting a single bit.

Definition 1.15 (Security game and advantage). A *security game* (or *experiment*) G is a PPT interactive algorithm, the challenger, that takes 1^λ , interacts with an adversary $\mathcal{A}$, and finally outputs a bit $G^{\mathcal{A}}(\lambda) \in \{0, 1\}$, where 1 denotes “adversary wins”. The *advantage* of $\mathcal{A}$ is a measure of how much better than trivial its winning probability is; its precise form depends on the type of game:

- *Distinguishing (indistinguishability) games*, where the challenger secretly chooses a uniform bit $b \in \{0, 1\}$ and $\mathcal{A}$ outputs a guess b' ; the adversary wins iff $b' = b$. Here

$$\text{Adv}_G(\mathcal{A}, \lambda) := \left| \Pr[G^{\mathcal{A}}(\lambda) = 1] - \frac{1}{2} \right| = \frac{1}{2} |\Pr[b' = 1 \mid b = 1] - \Pr[b' = 1 \mid b = 0]|.$$

Trivial guessing achieves winning probability $1/2$, hence advantage 0.

- *Search (unforgeability/inversion) games*, where the adversary must produce a specific kind of object—a forgery, a preimage, a discrete logarithm—and wins iff it succeeds. Here the baseline winning probability is (essentially) 0, and one sets

$$\text{Adv}_G(\mathcal{A}, \lambda) := \Pr[G^{\mathcal{A}}(\lambda) = 1].$$

Definition 1.16 (Security relative to a game). A scheme is *secure* with respect to the game G if for every PPT adversary $\mathcal{A}$,

$$\text{Adv}_G(\mathcal{A}, \lambda) = \text{negl}(\lambda).$$

The negligible bound may differ from one adversary to another; the claim is universal over the class. The use of negl on the right makes the definition robust to the polynomial slack that reductions introduce.

Example 1.17 (Pseudorandomness, as a game). The distinguishing game for a pseudorandom generator $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$, the primitive that witnessed strictness in Proposition 1.10, is the following.

1. The challenger samples $b \leftarrow \{0, 1\}$ uniformly. If $b = 0$ it samples a seed $s \leftarrow \{0, 1\}^\lambda$ and sends $z := G(s)$; if $b = 1$ it sends a uniform $z \leftarrow \{0, 1\}^{2\lambda}$.
2. The adversary $\mathcal{A}$ outputs a guess b' ; the game outputs 1 iff $b' = b$.

The generator is secure iff $\text{Adv}(\mathcal{A}, \lambda) = |\Pr[b' = b] - \frac{1}{2}|$ is negligible for every PPT $\mathcal{A}$. In this case the game and ensemble formulations coincide: the two situations are the fixed ensembles $\{G(U_\lambda)\}$ and $\{U_{2\lambda}\}$, and the displayed identity in Definition 1.15 shows the game advantage of $\mathcal{A}$ is exactly half its distinguishing advantage in the sense of Definition 1.9—so the game is secure iff the ensembles are computationally indistinguishable. When the adversary chooses part of the challenge or queries oracles adaptively, the interaction no longer reduces to a comparison of two fixed ensembles and the game form is required; the security definitions of the later sections are games.

Remark 1.18 (Game-hopping). Game-based proofs are *sequences of games*: one rewrites the real security game through a series of games $G_0, G_1, \dots, G_k$, each differing from the last by a small, justifiable change, until reaching a final game in which the adversary provably has zero (or trivially negligible) advantage. Each hop is justified by exactly one of three moves: the two games are identical unless some negligible-probability “bad” event occurs, so their winning probabilities differ by at most $\Pr[\text{bad}]$ (on the complement of that event the two runs coincide); or distinguishing the two games would break a hardness assumption (a reduction, below); or the change is purely syntactic. Bounding the total advantage is then a telescoping sum across hops—a structured variant of the hybrid argument (Lemma 1.13) that makes a long proof auditable one hop at a time.

1.5 Reductions

A *reduction* is an efficient transformation that converts a successful adversary against a scheme into a successful algorithm against a problem believed to be hard. Its existence proves the implication “ Π is hard $\Rightarrow X$ is secure” by establishing the contrapositive “ X is broken $\Rightarrow \Pi$ is solved”.

Definition 1.19 (Computational problem and hardness). A *computational problem* Π consists of a PPT instance generator $\text{Inst}(1^\lambda)$ producing a problem instance together with any side information, and a relation deciding when a candidate output *solves* the instance. The *success probability* of an algorithm $\mathcal{B}$ is

$$\text{Succ}_\Pi(\mathcal{B}, \lambda) := \Pr[\mathcal{B}(1^\lambda, \text{Inst}(1^\lambda)) \text{ solves the instance}].$$

The problem Π is *hard* if $\text{Succ}_\Pi(\mathcal{B}, \lambda) = \text{negl}(\lambda)$ for every PPT $\mathcal{B}$ (for a search problem), or if every PPT $\mathcal{B}$ decides the instance with at most negligible advantage, $\text{Adv}(\mathcal{B}, \lambda) = \text{negl}(\lambda)$ in the sense of Definition 1.15 (for a decision problem).

Definition 1.20 (Black-box reduction). Let X be a scheme with security game G_X and Π a computational problem. A (*black-box*) *reduction* from breaking X to solving Π is a PPT oracle algorithm $\mathcal{R}$ with the following per-adversary guarantee: for *every* adversary $\mathcal{A}$, writing $\varepsilon_X(\lambda) := \text{Adv}_{G_X}(\mathcal{A}, \lambda)$ for its advantage, the algorithm $\mathcal{R}^\mathcal{A}$ —which runs $\mathcal{A}$ as a subroutine, simulating $\mathcal{A}$ ’s game internally—solves Π with success probability

$$\text{Succ}_\Pi(\mathcal{R}^\mathcal{A}, \lambda) \geq f(\varepsilon_X(\lambda), \lambda),$$

where f is a function fixed by the reduction (not by $\mathcal{A}$) that maps every non-negligible ε_X to a non-negligible bound. The running time of $\mathcal{R}$ is measured with each call to $\mathcal{A}$ counted as one step, so “PPT” bounds $\mathcal{R}$ ’s own work, and hence its number of calls to $\mathcal{A}$, by a polynomial in λ ; the running time of $\mathcal{R}^\mathcal{A}$ is then polynomial whenever $\mathcal{A}$ ’s is, however many times $\mathcal{R}$ runs $\mathcal{A}$. How far f degrades ε_X is the reduction’s *loss*, discussed below. The reduction is *black-box* because $\mathcal{R}$ uses $\mathcal{A}$ only through its input/output interface, making no use of $\mathcal{A}$ ’s code.

Every reduction has the same structure. To prove “if Π is hard then X is secure”:

1. Assume, towards the contrapositive, a PPT adversary $\mathcal{A}$ that breaks X with non-negligible advantage ε_X .

2. Construct a PPT algorithm $\mathcal{R}$ that, given a random instance of Π , *embeds* that instance into a simulated security game for $\mathcal{A}$. The simulation must be faithful: the view $\mathcal{R}$ presents to $\mathcal{A}$ must follow (perfectly, statistically, or computationally) the distribution of the real game, so that $\mathcal{A}$'s advantage carries over.
3. Run $\mathcal{A}$. From whatever $\mathcal{A}$ does to win its game, $\mathcal{R}$ *extracts* a solution to the Π -instance.
4. Conclude that $\mathcal{R}$ solves Π with non-negligible probability, contradicting the hardness of Π . Therefore no such $\mathcal{A}$ exists, and X is secure.

Theorem 1.21 (Hard-core security of the Diffie–Hellman-style key, schematically). *Let $\Pi = \text{CDH}$ be the computational Diffie–Hellman problem in a cyclic group $G = \langle g \rangle$ of prime order q , the setting of the Math Guide (§“The discrete logarithm problem”): given (g, g^a, g^b) for uniform $a, b \in \mathbb{Z}/q\mathbb{Z}$, compute g^{ab} . Consider the “key-recovery” game G_X for a key-exchange scheme in which the adversary, given the public transcript (g^a, g^b) , wins iff it outputs the shared key g^{ab} . If CDH is hard, then no PPT adversary wins G_X with noticeable probability.*

Proof. Suppose $\mathcal{A}$ is a PPT adversary that, on input a transcript (g^a, g^b) , outputs g^{ab} with probability $\varepsilon_X(\lambda) = \text{Adv}_{G_X}(\mathcal{A}, \lambda)$. Construct $\mathcal{R}$ solving CDH. On input a CDH instance $(g, A, B) = (g, g^a, g^b)$:

1. The reduction $\mathcal{R}$ forms the simulated transcript (A, B) and feeds it to $\mathcal{A}$ as the public transcript of G_X .
2. Because (a, b) are uniform in the CDH instance *and* in the real game, the view of $\mathcal{A}$ follows *identically* the distribution of its view in G_X (the simulation is perfect; no loss in faithfulness).
3. When $\mathcal{A}$ outputs a candidate key K , $\mathcal{R}$ outputs K as its CDH answer.

Since the simulated and real views are identically distributed, $\mathcal{A}$ outputs $K = g^{ab}$ with exactly probability $\varepsilon_X(\lambda)$; whenever it does, $\mathcal{R}$ returns the correct CDH solution. Hence

$$\text{Succ}_{\text{CDH}}(\mathcal{R}, \lambda) = \varepsilon_X(\lambda).$$

The running time of $\mathcal{R}$ is that of $\mathcal{A}$ plus a constant amount of group bookkeeping, hence polynomial. If ε_X were noticeable, $\mathcal{R}$ would solve CDH with noticeable probability, contradicting its hardness. Therefore $\varepsilon_X(\lambda)$ is not noticeable, as claimed. Moreover, since $\mathcal{R}$ is PPT and CDH is hard (Definition 1.19, search branch), $\varepsilon_X(\lambda) = \text{Succ}_{\text{CDH}}(\mathcal{R}, \lambda) = \text{negl}(\lambda)$. $\square$

This reduction is *tight*: it calls the adversary once and converts advantage ε_X into success probability exactly ε_X . Most reductions incur a loss, quantified as follows.

Definition 1.22 (Reduction loss and tightness). Consider a reduction $\mathcal{R}$ that transforms an adversary running in time t with advantage ε into a Π -solver running in time $t' = t + t_{\mathcal{R}}$ with success probability ε' . The *security loss* is the factor

$$L(\lambda) := \frac{\varepsilon/t}{\varepsilon'/t'} = \frac{\varepsilon}{\varepsilon'} \cdot \frac{t'}{t},$$

the ratio of the solver’s work-per-success to the adversary’s work-per-success. A reduction is *tight* if $L(\lambda) = O(1)$ (a small constant, ideally 1), and *loose* if $L(\lambda)$ grows with λ (e.g. $L = q_H$, the number of random-oracle queries (Definition 1.26 below), or $L = q_s$, the number of signature queries, or $L = \lambda$). Two common sources of loss are: an *advantage gap* $\epsilon' \ll \epsilon$ (the solver succeeds far less often than the adversary, e.g. because the reduction must *guess* where to embed its instance among q queries, succeeding only with probability $1/q$); and a *time blow-up* $t' \gg t$ (the reduction runs the adversary many times, as in rewinding/forking arguments).

Remark 1.23 (Why loss matters concretely). Asymptotically, loss does not affect the conclusion: a polynomial loss factor turns a negligible advantage into a negligible advantage and a hard problem into security. Concretely, however, loss carries a cost. Suppose a scheme reduces to a problem believed to offer 128 bits of security, but the reduction loses a factor $L = 2^{40}$ (not unusual: $q_H \approx 2^{60}$ random-oracle queries times a forking-lemma square-root loss can be much worse). Then the *proof* guarantees only $128 - 40 = 88$ bits of security for the scheme. To restore a 128-bit guarantee one must enlarge the group so that the *underlying* problem offers 168 bits—larger keys, slower operations. A tight reduction lets the scheme inherit the full strength of the assumption at the smallest parameters, and a loose one forces larger ones. Tightness is therefore a design goal.

Remark 1.24 (Rewinding and the forking lemma, in brief). A recurrent source of large loss is *rewinding*: running the adversary, recording its behaviour, then restarting it from a saved state with fresh randomness on some branch to obtain a second, correlated transcript. From two transcripts that share a prefix but diverge afterwards one can often extract a secret (e.g. two valid signatures on the same commitment yield the signing key, or two accepting proof transcripts yield a witness). The *forking lemma*, stated as the general forking lemma under “Security in the random oracle model and the forking lemma”, quantifies the success probability of obtaining such a pair: if the adversary succeeds with probability ϵ after at most q random-oracle queries whose answers are drawn from a set of size b , the forked run succeeds with probability at least $\epsilon(\epsilon/q - 1/b)$, i.e. on the order of ϵ^2/q . The square in ϵ is a quadratic security loss: to keep ϵ^2/q noticeable one needs ϵ noticeably large, roughly halving the bit-security extracted. Such arguments, which later sections of this volume use for Schnorr-type and Fiat–Shamir signatures, are the archetype of loose but valid reductions.

1.6 The standard model and the random oracle model

The assumptions a proof makes define the *model* in which it resides. Two models predominate.

Definition 1.25 (Standard model). A proof resides in the *standard model* if it assumes only the computational hardness of explicitly stated problems (factoring, discrete logarithm, the decisional Diffie–Hellman problem of the next section, and so on) and grants the adversary no idealised access to any component of the scheme. Every primitive, including every hash function, is a concrete algorithm whose code the adversary may inspect and exploit.

Definition 1.26 (Random oracle model). A proof resides in the *random oracle model* (ROM) if it idealises one or more hash functions as a *random oracle*: a publicly accessible function $\mathcal{O}$

that, on each distinct input, returns an *independent, uniformly random* output of the appropriate length, consistently answering repeated queries with the same value. The oracle is available to the honest parties, to the adversary, and, in particular, to the reduction. No party holds the “code” of $\mathcal{O}$; it can only be queried.

Real hash functions are fixed, public algorithms, not truly random functions. The idealisation is adopted because it gives the reduction two capabilities that the standard model withholds and that render many otherwise-intractable proofs tractable.

Remark 1.27 (The powers of the random oracle: observability and programmability). In the ROM the reduction *simulates* the oracle for the adversary. *Observability*: the reduction sees every query the adversary makes to $\mathcal{O}$. Since the adversary, to use a hash value meaningfully, must query it, the reduction learns the adversary’s intermediate computations—for instance the preimage the adversary is about to hash. *Programmability*: the reduction may choose each oracle answer on the fly, subject only to the answers appearing uniform and consistent. It can therefore *plant* its hard-problem instance inside an oracle response—e.g. set $\mathcal{O}(x) := B$ where B is part of a CDH challenge—so that the adversary’s success necessarily reveals the solution. Neither capability exists for a concrete standard-model hash function, which fixes its outputs in advance and hides its internal queries.

Example 1.28 (The benefit of the ROM: full-domain hash, sketched). In the “full domain hash” signature, the signer signs a message m as $\sigma = H(m)^d \bmod N$, where (N, d) is an RSA private key—a modulus $N = pq$ with secret prime factors and a secret exponent d paired with a public e so that $x^{ed} \equiv x \pmod{N}$ for x coprime to N , by Euler’s theorem (*Math Guide*, §“Fermat’s little theorem and Euler’s theorem”); thus $x \mapsto x^e$ is a permutation of the unit group $(\mathbb{Z}/N\mathbb{Z})^\times$ that only the holder of d can invert—and H is a hash function mapping messages into $(\mathbb{Z}/N\mathbb{Z})^\times$. (Restricting to units costs nothing: a nonzero non-unit of $\mathbb{Z}/N\mathbb{Z}$ shares a factor with N , so finding one factors N . Hash functions appear here informally, as this sketch needs them; their formal syntax and security games are given in the section after next.) To prove unforgeability one reduces to RSA inversion: given a challenge y (find y^d), the reduction programs the oracle so that for a random subset of messages $H(m) := y \cdot r_m^e$ (embedding the challenge) and for the rest $H(m) := r_m^e$ (where it knows the “signature” r_m); with each r_m drawn uniformly from $(\mathbb{Z}/N\mathbb{Z})^\times$, the programmed values are distributed exactly as honest oracle answers on H ’s range. When the forger outputs a forgery on a message where the reduction embedded the challenge, the reduction extracts y^d . This proof has *no known standard-model analogue* for the plain scheme: it relies essentially on programming H . The cost is a security loss (the reduction must guess which message the forger attacks, incurring a factor q_H unless the reduction is optimised)—a loss in the sense of Definition 1.22.

Remark 1.29 (The status of the ROM: heuristic, not theorem). A proof in the ROM is a proof about an *idealised* model: deploying the scheme instantiates $\mathcal{O}$ with a concrete hash function, and the proof does *not* formally transfer—a ROM proof rules out all attacks that treat the hash as a black box, while offering no guarantee against attacks on the specific structure of the chosen hash. The case for trusting it runs in three steps: prove the scheme secure against a perfect random oracle; instantiate $\mathcal{O}$ with a hash believed to have no exploitable structure; and observe that any remaining attack must exploit structure the random oracle lacks, so would be a direct break of the

presumed structure-free hash. The gap is nonetheless a theorem: Canetti, Goldreich, and Halevi exhibit contrived schemes provably secure in the ROM yet insecure under *every* efficiently computable instantiation, so no concrete hash soundly instantiates the oracle for all schemes. No natural deployed scheme is known to be affected; this volume treats a ROM proof as strong positive evidence, not a guarantee, and prefers a standard-model proof when one is available at comparable cost. A quantum adversary (Remark 1.4) queries the oracle *in superposition*, a single query addressing a weighted combination of all inputs at once; in the resulting quantum random oracle model (QROM) the two powers of Remark 1.27 become subtler and classical ROM theorems do not automatically carry over, so post-quantum claims for arguments made non-interactive by hashing the transcript (the Fiat–Shamir transform, developed under “The Fiat–Shamir transform: from interactive to non-interactive”) must be made in that model.

Remark 1.30 (Relatives: CRS, generic-group, and algebraic-group models). Further idealisations, between and beyond these two models, are relevant to the proof systems this volume targets. The *common reference string* (CRS) model posits a trusted public string sampled from a prescribed distribution, used by many succinct proof systems. The *generic group model* (GGM) idealises the group dual to the manner in which the ROM idealises a hash: the adversary may only perform group operations through an oracle on opaque “handles”, never exploiting the bit representation of group elements; it is the standard setting for lower bounds on discrete-log-type problems. The *algebraic group model* (AGM) is an intermediate, milder idealisation requiring the adversary to “explain” every group element it outputs as a known combination of its inputs. These models trade realism for provability along the same axis as the ROM; later constructions name the model wherever they require one.

1.7 Concrete security and the bit-security budget

Asymptotic security establishes that a scheme is eventually secure, not whether a deployment with given parameters resists an adversary with a given budget. Concrete (or exact) security settles the latter by explicit bounds on running time and advantage.

Definition 1.31 (Concrete security: (t, ϵ) -security). Fix the security parameter to a concrete value (so all sizes are fixed). A scheme is (t, ϵ) -secure for a game G if every adversary running in time at most t has advantage $\text{Adv}_G(\mathcal{A}) \leq \epsilon$. More refined statements bound the advantage as an explicit function of the adversary’s resources, e.g. time t , number of queries q , and memory s : $\text{Adv}_G(\mathcal{A}) \leq f(t, q, s)$.

Definition 1.32 (n -bit security). A scheme (or problem) is commonly said to offer n bits of security when the cheapest known attack achieving constant success probability (say at least $\frac{1}{2}$) costs on the order of 2^n elementary operations. A proof may justify the stronger, explicit claim that every adversary running in time t has advantage at most about $t/2^n$, but that linear advantage bound is not implied merely by the absence of a faster known attack. Thus “128-bit security” is a concrete estimate, relative to a specified attack model and state of knowledge, that breaking the scheme costs roughly 2^{128} work.

The security parameter λ is chosen so that the resulting concrete scheme delivers the desired

number of bits of security. In the cleanest case the two coincide, $n = \lambda$; in general the choice accounts for the actual cost of the best attack on the underlying problem.

Example 1.33 (Why $\lambda \neq n$ for discrete logarithm). For a generic-group or hash-based primitive, doubling the output length doubles the bit security: a hash with $2n$ -bit output offers n -bit collision resistance—by the birthday bound (*Math Guide*, §“The union bound and a birthday calculation”)—and $2n$ -bit preimage resistance, so the parameter and the security level are proportional. For discrete-logarithm-based primitives the accounting differs. In a group of prime order $q \approx 2^\ell$, generic attacks (baby-step/giant-step, Pollard’s rho) solve discrete logarithm in time $\approx \sqrt{q} = 2^{\ell/2}$ (*Math Guide*, §“The discrete logarithm problem”). Hence a prime-order elliptic-curve group whose order is 256 bits offers only about 128 bits of security—one halves the bit length of the group order to obtain the generic security level. For discrete logarithm in a finite-field multiplicative group, sub-exponential index-calculus attacks additionally force the ambient field modulus into the thousands of bits for 128-bit security. Elliptic-curve groups, for which no comparable sub-exponential classical attack is known, are preferred for this reason: they deliver n -bit security at $\ell \approx 2n$ rather than thousands of bits. The single number n thus conceals a problem-specific translation from parameter sizes to the security level. The same arithmetic assesses the working groups of this volume: the Pasta curves assembled in the *Math Guide* (§“Pallas and Vesta assembled”) have prime order close to 2^{254} , so the square-root rule puts the cost of generic discrete logarithm there at about 2^{127} operations; the next section audits them criterion by criterion under “Instantiation on elliptic curves; the Pasta curves”.

Remark 1.34 (Reading a bit-security budget end to end). The preceding components combine into a parameter-selection procedure. First, the target security level n is fixed (commonly 128). Second, the underlying hard problem and the cost of its best attack as a function of the parameter are identified (e.g. $2^{\ell/2}$ for elliptic-curve discrete log), and the parameter is set so the problem itself offers n bits (here $\ell = 2n = 256$). Third, the reduction’s security loss is subtracted (Definition 1.22): if the proof loses a factor $L = 2^k$ (that is, k bits), the problem must offer $n + k$ bits for the *scheme* to offer n , so one enlarges the parameter accordingly—or, with a tight reduction, one need not. Finally, the choice is checked against the ROM caveat (Remark 1.29): a ROM proof bounds only black-box attacks on the hash, so the concrete hash must additionally exhibit no exploitable structure. The asymptotic theorem “the scheme is secure if the problem is hard” thus becomes a concrete choice of parameter sizes.

Remark 1.35 (Scope of a security proof). A proof of security is a proof *relative to a model*: a definition of the goal (the game), a class of adversaries (PPT, possibly non-uniform or quantum), and a set of assumptions (the hard problems, and any idealisation such as the ROM). It guarantees nothing outside that model: side-channel leakage (timing, power), faulty randomness, implementation bugs, broken assumptions, and goals the game did not capture all lie beyond its reach. The framework does not render schemes unconditionally safe; it renders the *conditions* of safety explicit, falsifiable, and modular, so that progress on assumptions and attacks translates into recalibrated guarantees. Every construction in this monograph is analysed within this framework.

2 Hardness assumptions

This section states the hardness assumptions on which the group-based constructions of the volume rest: those of the discrete logarithm problem (DLP) and of the computational and decisional Diffie–Hellman problems (CDH, DDH), ordered by reduction. Each secret of these constructions—a spending key, the trapdoor of a commitment, the nonce of a signature—lies in $\mathbb{Z}/q\mathbb{Z}$ and is published only as g^x ; an adversary that computes discrete logarithms recovers signing keys from verification keys, opens commitments to values of its choice, and removes the blinding from every rerandomised key of later sections. The section proves the generic square-root bounds and the Pohlig–Hellman reduction, which together show that λ bits of security against generic attacks, λ the security parameter, require a group of *prime* order q with $\log_2 q \geq 2\lambda$; instantiates the assumptions on the Pasta curves; and records that Shor’s quantum algorithm breaks all of them. Throughout, $G = \langle g \rangle$ denotes a cyclic group of prime order q , written multiplicatively until §2.6 switches to the additive notation of elliptic curves.

2.1 The discrete logarithm problem

This subsection defines the discrete logarithm problem and proves that in prime order its average-case and worst-case hardness coincide. Recall from the *Math Guide* (§“The discrete logarithm problem”) that a group G is *cyclic* if $G = \langle g \rangle = \{g^k : k \in \mathbb{Z}\}$ for some *generator* g . If $|G| = q$ then $\mathbb{Z}/q\mathbb{Z} \cong G$ via $k \mapsto g^k$, an isomorphism efficient to evaluate—repeated squaring costs $O(\log k)$ group operations—whose inversion, the discrete logarithm problem, is conjecturally infeasible.

Definition 2.1 (Discrete logarithm). Let $G = \langle g \rangle$ be a cyclic group of order q , written multiplicatively. For $h \in G$, the *discrete logarithm* of h to the base g , denoted $\log_g h$, is the unique residue $x \in \mathbb{Z}/q\mathbb{Z}$ with $g^x = h$. Existence follows from $G = \langle g \rangle$ and $\text{ord}(g) = q$; uniqueness because $g^x = g^{x'}$ forces $g^{x-x'} = 1$, whence $q \mid x - x'$.

Definition 2.2 (Discrete logarithm problem, DLP). Fix a family $\{G_\lambda\}$ of cyclic groups, where $G_\lambda = \langle g_\lambda \rangle$ has *prime* order q_λ with $\log_2 q_\lambda = \Theta(\lambda)$, together with a sampler that on input 1^λ outputs a description of $(G_\lambda, g_\lambda, q_\lambda)$. The game $\text{DLOG}^A(\lambda)$ runs as follows.

1. The challenger runs the sampler, draws $x \xleftarrow{\$} \mathbb{Z}/q_\lambda\mathbb{Z}$ uniformly, and sends $(G_\lambda, g_\lambda, q_\lambda, h = g_\lambda^x)$ to the adversary A .
2. The adversary outputs $x' \in \mathbb{Z}/q_\lambda\mathbb{Z}$.
3. The game outputs 1 iff $g_\lambda^{x'} = h$.

The *DLP assumption* for the family states that for every PPT A ,

$$\Pr[\text{DLOG}^A(\lambda) = 1] \in \text{negl}(\lambda).$$

The order is required to be prime because a composite order splits the problem across its factors (§2.5).

Proposition 2.3 (Random self-reducibility of DLP). *In a cyclic group $G = \langle g \rangle$ of prime order q , the discrete logarithm problem is random self-reducible: an algorithm solving a fraction ε of instances, over uniformly random b , converts into one solving any fixed instance with probability ε per trial. Consequently worst-case and average-case hardness coincide.*

Proof. Let $\mathcal{A}$ succeed on a uniform target with probability ε . Given a fixed challenge $b = g^x$, draw $r \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$ and form $b' = b \cdot g^r = g^{x+r}$. Addition modulo q is a bijection and r is uniform, so b' is uniform in G and independent of b ; hence $\mathcal{A}(b')$ returns $x + r \bmod q$ with probability ε , and subtracting r recovers x . Each candidate is checked by testing $g^x \stackrel{?}{=} b$, so failures are recognised. Fresh randomness r per trial makes trials independent: t of them succeed with probability $1 - (1 - \varepsilon)^t$, overwhelming once $t = \omega(\varepsilon^{-1} \log \lambda)$. $\square$

Random self-reducibility does not make every instance hard: the logarithm of the identity is immediate.

2.2 Diffie–Hellman: computational and decisional

This subsection defines the computational and decisional Diffie–Hellman problems, which concern the *product* of two exponents. In Diffie–Hellman key exchange the parties publish g^a and g^b and share g^{ab} , which each computes from the other’s public value and its own exponent; an eavesdropper observes g, g^a, g^b and must be unable to obtain g^{ab} .

Definition 2.4 (Computational Diffie–Hellman, CDH). With notation as in Definition 2.2, the game $\text{CDH}^A(\lambda)$ runs as follows: the challenger draws $a, b \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$ independently and sends (g, g^a, g^b) ; the adversary outputs $z \in G$; the game outputs 1 iff $z = g^{ab}$. The *CDH assumption* states that for every PPT $\mathcal{A}$,

$$\Pr_{a,b}[\text{CDH}^A(\lambda) = 1] \in \text{negl}(\lambda).$$

An adversary may be unable to *compute* g^{ab} yet able to *distinguish* it from a random group element, and such a distinguisher already breaks, for example, the indistinguishability of an ElGamal ciphertext. The decisional variant excludes such distinguishers.

Definition 2.5 (Decisional Diffie–Hellman, DDH). The *decisional Diffie–Hellman problem* is to distinguish two distributions over G^3 ,

$$\mathcal{D}_{\text{dh}} = (g^a, g^b, g^{ab}), \quad \mathcal{D}_{\text{rand}} = (g^a, g^b, g^c),$$

with a, b, c independent and uniform in $\mathbb{Z}/q\mathbb{Z}$; this is the distinguishing game of Definition 1.15, the challenger handing over a sample of one or the other according to its hidden bit. The *DDH assumption* states that every PPT distinguisher D has negligible advantage

$$\text{Adv}_{\text{ddh}}(D) = \left| \Pr_{a,b}[D(g^a, g^b, g^{ab}) = 1] - \Pr_{a,b,c}[D(g^a, g^b, g^c) = 1] \right| \in \text{negl}(\lambda).$$

A triple (g^a, g^b, g^c) with $c \equiv ab \pmod{q}$ is a *Diffie–Hellman triple*; the assumption says no efficient distinguisher recognises such triples.

Remark 2.6 (DDH admits a random self-reduction). Rerandomisation applies to DDH as to DLP, by the following map. Given a target triple $(u, v, w) = (g^a, g^b, g^c)$, choose $\alpha, \gamma \xleftarrow{\$} (\mathbb{Z}/q\mathbb{Z})^\times$ and $\beta, \delta \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$, and form

$$(u', v', w') = (u^\alpha g^\beta, v^\gamma g^\delta, w^{\alpha\gamma} u^{\alpha\delta} v^{\beta\gamma} g^{\beta\delta}).$$

Writing $A = \alpha a + \beta$, $B = \gamma b + \delta$, and $E = c - ab$, the new exponents are A , B , and $AB + \alpha\gamma E$, with A and B uniform and independent. A DH triple ($E = 0$) therefore becomes a fresh uniform DH triple; a non-DH triple becomes uniform conditional on being non-DH, because $\alpha\gamma E$ is uniform in $(\mathbb{Z}/q\mathbb{Z})^\times$. The latter distribution lies at statistical distance (Definition 1.6) $1/q$ from $\mathcal{D}_{\text{rand}}$, whose third component completes a DH triple by accident with probability $1/q$. A noticeable distinguishing advantage on random instances thus transfers to every fixed triple, up to this negligible term.

2.3 The assumption hierarchy

This subsection orders the three problems by reduction. Write $A \leq B$ for “ A reduces to B ”: an oracle solving B yields an efficient solver for A , so B is at least as hard as A , and the assumption “ A is hard” is the *stronger* commitment, since it implies “ B is hard”. The reductions of this subsection give

$$\text{DDH} \leq \text{CDH} \leq \text{DLP},$$

so assuming DDH hard is the strongest commitment and assuming DLP hard the weakest.

Theorem 2.7 (DLP is at least as hard as CDH). *If DLP is solvable in PPT, then CDH is solvable in PPT. Equivalently, the CDH assumption implies the DLP assumption.*

Proof. Suppose $\mathcal{A}$ solves DLP. Given a CDH instance (g, g^a, g^b) , run $\mathcal{A}$ on (g, g^a) to recover a , then compute $(g^b)^a = g^{ab}$ by fast exponentiation. The cost is one DLP call plus $O(\log q)$ group operations; only one of the two exponents is inverted. $\square$

Theorem 2.8 (CDH is at least as hard as DDH). *If CDH is solvable in PPT with non-negligible success probability, then DDH is solvable in PPT with non-negligible advantage; if the success probability is noticeable, rerandomisation drives the advantage overwhelmingly close to 1. Equivalently, the DDH assumption implies the CDH assumption.*

Proof. Suppose $\mathcal{A}$ solves CDH with probability ε . Build a distinguisher D : on input (g^a, g^b, z) , call $\mathcal{A}(g, g^a, g^b)$ to obtain a candidate w , and output 1 iff $w = z$. On a Diffie–Hellman triple, $z = g^{ab}$, so D outputs 1 whenever $\mathcal{A}$ succeeds, that is, with probability ε . On a random triple, $z = g^c$ with c uniform and independent of a, b , while w depends only on (g^a, g^b) ; the event $w = z$ therefore has probability exactly $1/q$. Hence

$$\text{Adv}_{\text{ddh}}(D) \geq \varepsilon - \frac{1}{q},$$

non-negligible whenever ε is, since $1/q$ is negligible.

For the amplified form let ε be noticeable. The distinguisher D' rerandomises its input t times independently by the map of Remark 2.6, runs D with fresh coins on each copy, and outputs 1 iff *some* copy answered 1. A DH input becomes t independent uniform DH triples, each accepted with probability ε , so D' accepts with probability at least $1 - (1 - \varepsilon)^t$. A sample of $\mathcal{D}_{\text{rand}}$ fails to be a DH triple except with probability $1/q$, and then each copy's third component is uniform over the $q - 1$ non-completing values given the first two, so each copy is accepted with probability at most $1/(q - 1)$ and D' accepts with probability at most $1/q + t/(q - 1)$. With $t = \omega(\varepsilon^{-1} \log \lambda)$, polynomial because ε is noticeable, the advantage of D' is $1 - \text{negl}(\lambda)$. $\square$

Example 2.9 (DDH can be easy while CDH and DLP are hard). Some elliptic-curve groups carry an efficiently computable, non-degenerate *bilinear pairing* $e: G \times G \rightarrow G_T$ with $e(g^a, g^b) = e(g, g)^{ab}$. On such a group DDH is *easy*: to test whether (g^a, g^b, z) is a DH triple, check

$$e(g^a, g^b) \stackrel{?}{=} e(g, z),$$

which holds iff $\log_g z \equiv ab \pmod{q}$. Yet CDH—and a fortiori DLP—may remain hard, because the pairing reveals g^{ab} only inside the *target* group G_T , never as an element of G . Such groups, called *gap groups*, separate DDH from CDH and are the setting of pairing-based cryptography. The DDH assumption is therefore strictly stronger than the CDH assumption: a “DDH group” must admit no efficiently computable pairing of this kind on G itself.

Remark 2.10 (The converses are open). The reverse reductions are not known in general. Whether an oracle computing g^{ab} yields a is the longstanding Diffie–Hellman versus discrete log question; partial results of den Boer and Maurer establish equivalence when a suitable auxiliary group of *smooth* order—an order that is a product of small primes—is available, but no unconditional equivalence is known. In the other direction DDH may be strictly easier than CDH, as Example 2.9 shows. Each protocol therefore assumes exactly the strength it requires: signatures and key exchange often need only CDH, while ElGamal-style encryption needs DDH for its ciphertexts to hide the plaintext.

2.4 Generic algorithms and the square-root barrier

This subsection proves that two generic discrete-logarithm algorithms achieve $O(\sqrt{q})$ group operations and that no generic algorithm does asymptotically better. In a group with no exploitable structure beyond the group law, the best known algorithms are *generic*: they manipulate group elements only through multiplication, inversion, and equality testing, never inspecting the bit-representation of an element. Shoup’s idealisation makes the class precise: a random injective *labelling* $\sigma: \mathbb{Z}/q\mathbb{Z} \rightarrow S$ encodes the element g^k as the opaque string $\sigma(k)$, and the algorithm may only ask an oracle for $\sigma(i + j)$ given $\sigma(i)$ and $\sigma(j)$, and compare labels for equality.

Proposition 2.11 (Baby-step giant-step, BSGS). *In a cyclic group of order q , any discrete logarithm can be computed in $O(\sqrt{q})$ group operations and $O(\sqrt{q})$ stored group elements, deterministically and unconditionally.*

Baby-step giant-step.

Proof. Let $m = \lfloor \sqrt{q} \rfloor$. Since $m^2 \geq q$, every $x \in \{0, \dots, q-1\}$ decomposes as

$$x = i \cdot m + j, \quad 0 \leq i < m, \quad 0 \leq j < m,$$

and the target relation $g^x = h$ becomes $g^j = h \cdot (g^{-m})^i$. *Baby steps:* compute and store the table $\{(j, g^j) : 0 \leq j < m\}$, indexed by group element in a hash map— $m-1$ multiplications. *Giant steps:* set $u = g^{-m}$ (one inversion plus one exponentiation, $O(\log q)$ operations) and for $i = 0, 1, 2, \dots$ compute $h \cdot u^i$ incrementally, looking each value up in the table. The decomposition guarantees some pair (i, j) matches, yielding $x = im + j$ after at most m giant steps. Total: $O(\sqrt{q})$ operations and $O(\sqrt{q})$ memory. $\square$

Remark 2.12 (Memory is the bottleneck). The $O(\sqrt{q})$ memory of BSGS, not its time, is the binding constraint in practice: at the 128-bit security level no attacker can store $\sqrt{q} \approx 2^{128}$ table entries. Pollard’s rho method matches the time bound with negligible memory, which is why rho, rather than BSGS, is the realistic generic attack.

Proposition 2.13 (Pollard’s rho, sketch). *In a cyclic group of prime order q there is a randomised algorithm computing discrete logarithms in expected $O(\sqrt{q})$ group operations and $O(1)$ storage.*

Pollard’s rho.

Proof. Sketch. Walk pseudo-randomly on G , keeping with each visited element its known representation $g^\alpha h^\beta$. Partition G into three roughly equal sets S_1, S_2, S_3 by an easily computed function of the element’s label, and from $P = g^\alpha h^\beta$ step to

$$f(P) = \begin{cases} h \cdot P = g^\alpha h^{\beta+1}, & P \in S_1, \\ P^2 = g^{2\alpha} h^{2\beta}, & P \in S_2, \\ g \cdot P = g^{\alpha+1} h^\beta, & P \in S_3, \end{cases}$$

each step updating $(\alpha, \beta) \in (\mathbb{Z}/q\mathbb{Z})^2$ explicitly at $O(1)$ cost. The sequence eventually cycles: a tail leading into a loop, in the shape of the letter ρ . Modelling f as a random function on q elements, the birthday bound (*Math Guide*, §“The union bound and a birthday calculation”) produces a collision $P_i = P_j$, $i \neq j$, within $O(\sqrt{q})$ steps in expectation. A collision gives $g^{\alpha_i} h^{\beta_i} = g^{\alpha_j} h^{\beta_j}$, hence $g^{\alpha_i - \alpha_j} = h^{\beta_j - \beta_i}$; writing $h = g^x$,

$$\alpha_i - \alpha_j \equiv x(\beta_j - \beta_i) \pmod{q},$$

and if $\beta_j \neq \beta_i$ the difference is invertible modulo the *prime* q , so $x \equiv (\alpha_i - \alpha_j)(\beta_j - \beta_i)^{-1} \pmod{q}$. Floyd’s tortoise-and-hare cycle detection, or Brent’s improvement, finds the collision with $O(1)$ memory, never storing the trajectory. The exceptional case $\beta_j \equiv \beta_i$ occurs with probability $O(1/q)$; re-randomising the start point handles it. $\square$

Remark 2.14 (Cost, not time, and constant factors). Pollard rho parallelises with near-linear speedup via the van Oorschot–Wiener method of *distinguished points*: each of P processors walks independently and reports to a shared table only the points whose label satisfies a cheap

predicate, say a run of leading zero bits; two walks that ever meet coincide from then on and are detected at the next distinguished point, so the processors achieve $\sqrt{q}/P$ wall-clock time with modest communication. The security-relevant measure is therefore the *cost* of the attack—the total operation count $\sqrt{q}$, read as the attacker’s budget—not the time on any single machine. A further constant-factor saving applies in groups admitting the $\pm$ automorphism $P \mapsto -P$, as elliptic curves do: identifying P with $-P$ halves the effective search space and lowers the expected count from $\sqrt{\pi q/2}$ to $\sqrt{\pi q/4}$, a factor $\sqrt{2}$ that leaves the $\Theta(\sqrt{q})$ scaling untouched. The constant is the birthday integral: a random walk on N points is still collision-free after k steps with probability $\prod_{i < k} (1 - i/N) \approx e^{-k^2/2N}$ (the product in the birthday calculation cited above), and summing over k gives an expected first-collision index of $\int_0^\infty e^{-k^2/2N} dk = \sqrt{\pi N/2}$.

The generic lower bound. Shoup’s lower bound, the *square-root barrier*, shows that up to constant factors the two attacks are the best *possible* generic algorithms.

Theorem 2.15 (Shoup’s generic lower bound). *Let q be prime. Any generic algorithm that makes at most n queries to the group oracle outputs the discrete logarithm of a uniformly random challenge with probability at most*

$$\frac{\binom{n+2}{2} + 1}{q} = O\left(\frac{n^2}{q}\right).$$

In particular, constant success probability requires $n = \Omega(\sqrt{q})$ queries.

Proof. Sketch—the linear-polynomial technique. In the idealisation, the challenge logarithm is an indeterminate X drawn uniformly from $\mathbb{Z}/q\mathbb{Z}$. By induction on the queries, every element the algorithm forms is σ of a *known linear polynomial* $a_i + b_i X \bmod q$ whose coefficients the algorithm chose. Treat X formally: as long as the polynomials produced are pairwise distinct as functions of X , the labels seen are consistent with X taking any value, so the algorithm has learnt nothing about it and can do no better than guess, succeeding with probability $1/q$. Information leaks only through an accidental collision: two distinct polynomials agreeing at the secret x , that is, $(a_i - a_j) + (b_i - b_j)x \equiv 0 \pmod{q}$. For a fixed pair this nonzero linear equation has at most one root modulo the prime q —the degree-one case of the Schwartz–Zippel lemma (*Math Guide*, §“The Schwartz–Zippel lemma”)—so it holds with probability at most $1/q$ over uniform x . With n queries plus the two inputs g and h there are at most $\binom{n+2}{2}$ pairs, and the union bound (*Math Guide*, §“The union bound and a birthday calculation”) caps the collision probability at $\binom{n+2}{2}/q$. Adding the $1/q$ guess yields the bound; solving $n^2/q = \Omega(1)$ gives $n = \Omega(\sqrt{q})$. $\square$

Corollary 2.16 (Why $\log_2 q \geq 2\lambda$). *To force every generic attacker to spend at least 2^λ group operations— λ bits of security against generic attacks—the prime group order must satisfy*

$$\sqrt{q} \gtrsim 2^\lambda \iff \log_2 q \geq 2\lambda.$$

Proof. Propositions 2.11 and 2.13 show that $\Theta(\sqrt{q})$ operations suffice for a generic attacker, and Theorem 2.15 shows no generic attacker does asymptotically better. Equating the attacker’s cost

$\sqrt{q}$ with the target work factor 2^λ gives $q \approx 2^{2\lambda}$; demanding at least this much is $\log_2 q \geq 2\lambda$. $\square$

Example 2.17 (The 128-bit target). For $\lambda = 128$ the corollary demands a prime group order with $\log_2 q \geq 256$. This is the reason for the standardised 256-bit elliptic curves: NIST P-256 (secp256r1) has a 256-bit field and a 256-bit group order, so $\sqrt{q} \approx 2^{128.0}$. The Pasta curves of §2.6, with 255-bit orders slightly above 2^{254} and about 126 bits of security, fall slightly below this class. In the multiplicative group $\mathbb{F}_p^\times$, by contrast, the sub-exponential index calculus of Remark 2.18 forces $\log_2 p$ into the thousands of bits for the same λ . Elliptic curves are used precisely because no sub-exponential algorithm that beats the generic attacks is known for them.

Remark 2.18 (Why the barrier is special to elliptic curves). The square-root barrier governs *generic* groups only; a concrete representation can expose structure that beats it. The multiplicative group $\mathbb{F}_p^\times$ is the standard example: *index calculus* exploits the factorisation of integers into small primes—smooth numbers—to solve DLP in sub-exponential time

$$\exp(O((\log p)^{1/3}(\log \log p)^{2/3})),$$

far below $\sqrt{p}$. No analogue is known for the points of a well-chosen elliptic curve over $\mathbb{F}_p$: there is no useful notion of a “small” point over which to factor. This non-generic gap is why elliptic-curve groups suffice at 256 bits where $\mathbb{F}_p^\times$ requires about 3072 bits for the same security. It also means the rule $\log_2 q \geq 2\lambda$ protects against generic attacks *only*; a curve must additionally be screened against the special-purpose attacks of Definition 2.24.

2.5 Pohlig–Hellman and the necessity of large prime order

This subsection proves the Pohlig–Hellman reduction and derives from it the rule for choosing the group order. The square-root barrier puts the cost of an attack at $\sqrt{q}$ for *prime* q ; if the group order factors, the discrete logarithm splits into independent subproblems the size of the factors, and the cost falls to that of the *largest prime* dividing the order.

Theorem 2.19 (Pohlig–Hellman). *Let $G = \langle g \rangle$ have order $N = \prod_{i=1}^r p_i^{e_i}$ with the p_i distinct primes. Then a discrete logarithm in G can be computed in*

$$O\left(\sum_{i=1}^r e_i (\log N + \sqrt{p_i})\right)$$

group operations. The term $\sqrt{p_{\max}}$ dominates, where $p_{\max}$ is the largest prime dividing N .

Proof. We compute $x = \log_g h$ modulo each prime power $p_i^{e_i}$ and recombine by the Chinese Remainder Theorem, the moduli being pairwise coprime with product N .

Reduction to prime-power order. Fix $p = p_i$, $e = e_i$, and put $g_i = g^{N/p^e}$, $h_i = h^{N/p^e}$. Then g_i has order exactly p^e , and $\log_{g_i} h_i \equiv x \pmod{p^e}$: raising to N/p^e eliminates the part of x coprime to p 's power and leaves $x \bmod p^e$. It therefore suffices to solve DLP in $\langle g_i \rangle$ of order p^e .

Reduction to prime order, digit by digit. Write the unknown in base p :

$$x \equiv x_0 + x_1 p + \dots + x_{e-1} p^{e-1} \pmod{p^e}, \quad 0 \leq x_k < p.$$

Let $\gamma = g_i^{p^{e-1}}$, an element of order exactly p . Raising h_i to the power p^{e-1} gives

$$h_i^{p^{e-1}} = g_i^{x p^{e-1}} = \gamma^{x_0},$$

because every higher digit contributes a multiple of p^e to the exponent and annihilates γ . Thus x_0 is a single discrete logarithm in a group of prime order p , solvable in $O(\sqrt{p})$ operations by BSGS or rho (Propositions 2.11 and 2.13). Having x_0 , set $h_i^{(1)} = h_i \cdot g_i^{-x_0}$; its logarithm is divisible by p , and the same step at exponent p^{e-2} yields x_1 . Iterating recovers all e digits with e prime-order DLPs of cost $O(\sqrt{p})$ each, plus $O(e \log N)$ operations of exponentiation. Summing over i and applying the CRT gives the stated bound. $\square$

Example 2.20 (A composite-order trap). Suppose, contrary to good practice, a group has order $N = 2^{256} - 1$. The order factors completely into

$$\begin{aligned} 2^{256} - 1 = & 3 \cdot 5 \cdot 17 \cdot 257 \cdot 641 \cdot 65537 \cdot 274177 \cdot 6700417 \cdot 67280421310721 \\ & \cdot 59649589127497217 \cdot 5704689200685129054721, \end{aligned}$$

a product of small primes—it is *smooth*. Despite N having 256 bits, the largest prime factor is only about $2^{72.3}$, far below 2^{128} , so Pohlig–Hellman with rho in the largest subgroup solves DLP in roughly $\sqrt{p_{\max}} \approx 2^{36.1}$ operations, about 2^{36} , far below the nominal 128-bit level. A 256-bit order is necessary but *not* sufficient: it must be (almost) prime.

Corollary 2.21 (Design rule for the group order). *For DLP-based security at level λ , the group order must be divisible by a prime q with $\log_2 q \geq 2\lambda$, and the cryptographic group must be, or be restricted to, the subgroup of order q . When the ambient group has order $N = h \cdot q$ with small cofactor h and large prime q , one works in the order- q subgroup; clearing the cofactor—multiplying incoming elements by h , or checking subgroup membership—prevents small-subgroup attacks that would otherwise leak $x \bmod h$ via Pohlig–Hellman on the order- h part.*

Remark 2.22 (Prime order and Pohlig–Hellman). The “prime order” of Definition 2.2 makes Pohlig–Hellman vacuous (the only factor is q itself), makes every non-identity element a generator (no element lies in a weak proper subgroup), and eliminates cofactor handling. The Pasta curves below are constructed to have prime order, hence cofactor 1.

2.6 Instantiation on elliptic curves; the Pasta curves

This subsection fixes the concrete groups used in the remainder of the volume. Recall from the *Math Guide* (§“Elliptic curves”, with the group law in §“The group structure of $E(\mathbb{F}_p)$ ”) that an elliptic curve over $\mathbb{F}_p$, for $p > 3$ prime, in short Weierstrass form is

$$E: y^2 = x^3 + ax + b, \quad a, b \in \mathbb{F}_p, \quad 4a^3 + 27b^2 \neq 0,$$

and that its set of $\mathbb{F}_p$ -points,

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 : y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\},$$

equipped with chord-and-tangent addition and the point at infinity $\mathcal{O}$ as identity, forms a finite abelian group. The discrete logarithm problem in this group—given P and $Q = [x]P$, find x —is the *elliptic-curve discrete logarithm problem* (ECDLP), stated for elliptic curves in the *Math Guide* (§“The elliptic-curve discrete logarithm problem”); CDH and DDH transcribe verbatim with multiplicative notation replaced by the additive scalar multiplication $x \mapsto [x]P$, giving ECCDH and ECDDH.

Theorem 2.23 (Hasse bound, recalled). *For E over $\mathbb{F}_p$ (Math Guide, §“The group structure of $E(\mathbb{F}_p)$ ”),*

$$|\#E(\mathbb{F}_p) - (p + 1)| \leq 2\sqrt{p}, \quad \text{i.e.} \quad \#E(\mathbb{F}_p) = p + 1 - t, \quad |t| \leq 2\sqrt{p},$$

where t is the trace of Frobenius. The group order is thus $p + 1$ up to a deviation of order $\sqrt{p}$.

For λ -bit security the prime p is chosen with about 2λ bits, so that $\#E(\mathbb{F}_p) \approx p \approx 2^{2\lambda}$, and, following Corollary 2.21, $\#E(\mathbb{F}_p)$ is required to be prime—cofactor 1. ECDLP, ECCDH, and ECDDH are then assumed at the level Corollary 2.16 predicts, *provided* the curve avoids the known structural attacks that beat the generic bound; the following criteria screen for them.

Definition 2.24 (Secure-curve criteria). A curve $E/\mathbb{F}_p$ of prime order $q = \#E(\mathbb{F}_p)$ is admissible for λ -bit DLP security if:

1. (*Size / generic.*) The order satisfies $\log_2 q \geq 2\lambda$, so Pollard rho costs at least 2^λ operations (Corollary 2.16).
2. (*Prime order.*) The order q is prime, so Pohlig–Hellman is vacuous and the cofactor is 1 (Corollary 2.21).
3. (*Anti-MOV / large embedding degree.*) The multiplicative order k of p modulo q —the *embedding degree*—is large. The MOV/Frey–Rück attack uses a pairing to embed $E(\mathbb{F}_p)$ into $\mathbb{F}_{p^k}^\times$ and runs the sub-exponential index calculus of Remark 2.18 there; the attack is effective only for small k , as on *supersingular* curves—over a prime field $p > 3$, those with trace $t = 0$, so that $\#E(\mathbb{F}_p) = p + 1$ divides $p^2 - 1$ and $k \leq 2$. A curve with $t \neq 0$ is *ordinary*.
4. (*Anti-Smart / not anomalous.*) The order avoids $q = p$: on an *anomalous* curve, $\#E(\mathbb{F}_p) = p$ with trace $t = 1$, the Smart–Sato–Araki–Semaev attack solves ECDLP in polynomial time.
5. (*Twist security.*) The *quadratic twist* E' —the curve $dy^2 = x^3 + ax + b$ for a fixed non-square $d \in \mathbb{F}_p$; every $x \in \mathbb{F}_p$ is the x -coordinate of a point of E or of E' —should also have large prime order, so that a fault or an invalid-point injection cannot move the computation onto a weak twist.

Definition 2.25 (The Pasta curves). The *Pasta* curves are a pair, *Pallas* and *Vesta*, in short

Weierstrass form

$$y^2 = x^3 + 5$$

(so $a = 0$, $b = 5$, a j -invariant-0 shape), defined over two 255-bit primes p and q arranged in a cycle:

$$\#Pallas = q \text{ (Pallas defined over } \mathbb{F}_p), \quad \#Vesta = p \text{ (Vesta defined over } \mathbb{F}_q).$$

The order of each curve equals the field of definition of the other. Both primes are congruent to 1 mod 2^{32} , giving their multiplicative groups large 2-adic valuation for fast FFTs, and each curve has cofactor 1: the whole group is of prime order. The primes themselves are assembled digit by digit in the *Math Guide* (§“Pallas and Vesta assembled”), and the field/scalar bookkeeping of the cycle in §“Base fields, scalar fields, and the Pasta cycle” there.

Remark 2.26 (The role of the curve cycle). The cycle property $\#Pallas = q$, $\#Vesta = p$ makes (Pallas, Vesta) an *amicable*, or cyclic, pair: one curve’s scalar field is the other’s coordinate field. A proof system can therefore use Pallas-arithmetic to verify statements about Vesta-arithmetic and vice versa without non-native field emulation; this is the mechanism behind recursive proof composition in Halo 2, whose recursion alternates between the two curves. The cycle is an *efficiency* property, not a security one: it does not enter the hardness assumptions, and each curve is screened independently against Definition 2.24.

Proposition 2.27 (The Pasta curves against the criteria). *The curves Pallas and Vesta satisfy criteria (1)–(4) of Definition 2.24 at $\lambda \approx 126$. Criterion (5) fails for both curves—neither twist has prime order, and the Pallas twist falls below the SafeCurves 2^{100} twist-attack cost threshold—but mandatory point validation makes the failure inconsequential.*

Proof. Sketch, criterion by criterion.

(1) Each order is a 255-bit prime slightly above 2^{254} , so $\log_2 q \approx 254.0$ and the bare generic cost is $\sqrt{q} \approx 2^{127.0}$. Plain Pollard rho expects about $\sqrt{\pi q/2} \approx 2^{127.3}$ operations; the $\pm$ automorphism speedup of Remark 2.14 lowers this to $\sqrt{\pi q/4} \approx 2^{126.8}$; and the order-6 automorphism group of these j -invariant-0 curves—the factor $\sqrt{m}$ for an order- m automorphism group of Remark 2.28, here an extra $\sqrt{3}$ —lowers it to $\sqrt{\pi q/12} \approx 2^{126.0}$. Hence “about 126 bits” of security, the same range as the 128-bit curves of Example 2.17.

(2) Both orders are prime by construction: cofactor 1, Pohlig–Hellman vacuous.

(3) Both curves are ordinary: the trace $t = p + 1 - q$ is odd, so $t \neq 0$. The embedding degree k of each is large. For Pallas, $q - 1 = 2^{32} \cdot 3^2 \cdot 1709 \cdot 24859 \cdot C$ with C a 194-bit composite, and $p^{(q-1)/32} \not\equiv 1$ and $p^{(q-1)/\ell} \not\equiv 1 \pmod{q}$ for $\ell \in \{3, 1709, 24859\}$, so the order k of p modulo q is a multiple of $2^{28} \cdot 3^2 \cdot 1709 \cdot 24859 > 2^{56}$; for Vesta, $p - 1 = 2^{32} \cdot 3 \cdot 463 \cdot C'$, and $q^{(p-1)/4} \not\equiv 1$, $q^{(p-1)/\ell} \not\equiv 1 \pmod{p}$ for $\ell \in \{3, 463\}$ make k a multiple of $2^{31} \cdot 3 \cdot 463 > 2^{41}$. Either extension field lies far beyond the reach of any feasible index-calculus computation; MOV/Frey–Rück does not apply.

(4) Neither curve is anomalous: $\#Pallas = q \neq p$ and $\#Vesta = p \neq q$, so the trace $t = p + 1 - q \neq 1$ and the Smart attack does not apply.

(5) Twist security is the criterion the pair does not meet. The Pasta search deliberately ignored it—the curves were generated with the search flag `--ignoretwist` of the `amicable.sage` search recorded in the README of the `zcash/pasta` repository—and neither twist has prime order. Each $x \in \mathbb{F}_p$ supplies two affine points in total, on E or on E' (one on each when $x^3 + b = 0$), so $\#E + \#E' = 2(p + 1)$ and $\#E' = 2(p + 1) - \#E = p + 1 + t$; hence $\#Pallas' = 2(p + 1) - q$ and $\#Vesta' = 2(q + 1) - p$, which factor as

$$\begin{aligned}\#Pallas' &= 3^2 \cdot 7 \cdot 8191 \cdot 85021 \cdot 11540602760389 \cdot P_{176}, \\ \#Vesta' &= 3^3 \cdot 7 \cdot 13 \cdot 2851 \cdot 30097 \cdot P_{217},\end{aligned}$$

with P_{176} a 176-bit prime and P_{217} a 217-bit prime. The best attack on the Pallas twist therefore costs about $\sqrt{P_{176}} \approx 2^{87.6}$, below the SafeCurves 2^{100} twist-security threshold; the Vesta twist clears the threshold at about $2^{108.2}$. The failure has no consequence in this setting: twist attacks require an implementation that accepts an attacker-supplied x -coordinate without validation (an x -only ladder) or that skips its on-curve checks, and the deployed arithmetic does neither. The `pasta_curves` crate carries points in full coordinates and has no x -only code path; its deserialisation (`from_bytes`) reconstructs y by taking the square root of $x^3 + 5$, failing exactly when x lies on the twist, where $x^3 + 5$ is a non-square, and even `from_bytes_unchecked` delegates to the checked path because a compressed encoding cannot skip the curve check, while points built from raw coordinates are gated on `is_on_curve`. This is the behaviour protocol specification § 5.4.9.6 mandates: its decoding map returns $\perp$ (failure) when $x^3 + b$ has no square root, and an implementation must not assume that the root exists or that the encoding lies on the curve. Orchard deserialises its points through this routine, and cofactor 1 leaves no small subgroup on the curve itself. Invalid-curve and twist-point injection are therefore excluded by *validation*, not by twist structure.

The standard generic analysis therefore governs, and the security level is about 126 bits. $\square$

Remark 2.28 (j -invariant 0 and endomorphisms). The shape $y^2 = x^3 + b$ gives j -invariant 0, and with it an extra automorphism of order 3 (*Math Guide*, §“The j -invariant and the GLV endomorphism”): for ω a primitive cube root of unity in $\mathbb{F}_p$, the cheap endomorphism $\phi(x, y) = (\omega x, y)$ acts on the group as multiplication by a fixed scalar ζ with $\zeta^2 + \zeta + 1 \equiv 0 \pmod{q}$ (the λ of the *Math Guide*’s proposition), enabling the GLV speedup for honest scalar multiplication. The same endomorphism gives the attacker a slightly larger automorphism group, and with it the constant-factor Pollard-rho speedup used in Proposition 2.27—a factor $\sqrt{m}$ for an order- m automorphism group—but that is a small constant and does not change the $\Theta(\sqrt{q})$ scaling. Beyond that constant, the extra automorphism is not known to weaken ECDLP; the structural weaknesses at issue are small embedding degree and anomalousness, both excluded above.

2.7 Quantum adversaries: Shor’s algorithm

Every assumption above restricts the adversary to classical PPT computation. This subsection states which of them fail against a large fault-tolerant quantum computer and which primitives remain secure.

Theorem 2.29 (Shor, informal; stated without proof). *There is a quantum algorithm that, given a cyclic group $G = \langle g \rangle$ of order q with efficient group operations and a target $h = g^x$, outputs x in time polynomial in $\log q$, succeeding with overwhelming probability. Consequently DLP—and with it CDH and DDH—falls in quantum polynomial time in any group, elliptic-curve groups included.*

Discrete logarithm is an instance of the *hidden subgroup problem* over $(\mathbb{Z}/q\mathbb{Z})^2$: the function $f(\alpha, \beta) = g^\alpha h^{-\beta} = g^{\alpha-x\beta}$ is constant exactly on the cosets of

$$H = \{(\alpha, \beta) : \alpha \equiv x\beta \pmod{q}\} = \langle (x, 1) \rangle \subseteq (\mathbb{Z}/q\mathbb{Z})^2,$$

a line of slope x . The algorithm prepares a uniform superposition over $(\mathbb{Z}/q\mathbb{Z})^2$, evaluates f into a second register, and applies the quantum Fourier transform—the reversible change of basis into the characters of $(\mathbb{Z}/q\mathbb{Z})^2$, the homomorphisms $\chi_{u,v}(\alpha, \beta) = e^{2\pi i(u\alpha+v\beta)/q}$ into the unit circle; measurement then samples characters trivial on H , i.e. pairs (u, v) with $ux + v \equiv 0 \pmod{q}$, and a small number of samples determine the slope x by classical linear algebra modulo q . The whole procedure uses $O(\text{poly}(\log q))$ qubits and gates, and it accesses the group only through the black-box map $(\alpha, \beta) \mapsto g^\alpha h^{-\beta}$, so it is independent of the representation: elliptic-curve DLP falls exactly as $\mathbb{F}_p^\times$ DLP does, and integer factoring falls to the order-finding variant.

Remark 2.30 (No contradiction with the square-root barrier). Shoup’s lower bound (Theorem 2.15) assumes the generic model for *classical* algorithms making sequential, individual oracle queries. Shor’s algorithm is non-generic in exactly the relevant sense: it queries the group function in superposition and exploits the periodicity the Fourier transform exposes, abilities the classical generic model does not grant.

Remark 2.31 (Primitives broken and unbroken by Shor’s algorithm). Shor’s algorithm breaks the entire discrete-log family and integer factoring in polynomial time: RSA, finite-field Diffie–Hellman, and elliptic-curve cryptography are all quantum-broken. It does *not* break the following.

- *Symmetric primitives and hash functions.* Grover’s algorithm searches a space of size 2^n in about $2^{n/2}$ quantum queries—a quadratic, not exponential, speedup—so a cipher with a 2λ -bit key, or an $n = 2\lambda$ -bit hash against preimages, retains λ bits of *quantum* security. Generic quantum collision finding instead costs $\Theta(2^{n/3})$ queries, so in the query metric an output of about 3λ bits is needed for λ -bit quantum collision security—though the algorithm needs comparably much quantum memory, and whether it beats classical parallel collision search in realistic cost models is debated. Doubling therefore suffices for key search and preimages, but for collisions only under that contested cost reading.
- *Post-quantum assumptions.* Hardness assumptions based on neither discrete logarithms nor factoring are not known to fall to Shor and underlie post-quantum cryptography.

The discrete-log assumptions of this volume are therefore adopted under the standing premise that no cryptographically relevant quantum computer exists. Within that premise, and against all classical attacks, the Pasta curves provide the approximately 126-bit security of Proposition 2.27, on which the later Halo 2 constructions rely.

2.8 Standing assumptions

Remark 2.32 (Standing assumptions for the sequel). We collect the premises the rest of the volume relies on. For each Pasta curve $C \in \{\text{Pallas}, \text{Vesta}\}$ with prime order q_C and generator g_C :

1. ECDLP is hard in $\langle g_C \rangle$: no classical PPT adversary finds x from $[x]g_C$ with non-negligible probability—concretely, every known attack costs at least 2^{126} operations (Proposition 2.27).
2. ECCDH and, where needed, ECDDH hold in $\langle g_C \rangle$ (Definitions 2.4 and 2.5, transcribed additively).
3. The adversary is classical and probabilistic polynomial-time (Remark 2.31).

By the hierarchy of §2.3, assuming ECDDH is the strongest commitment and implies the rest; each later construction invokes, by name, the weakest assumption it requires.

3 Hash functions and the random oracle model

A hash function compresses arbitrary data into a short fixed-length tag. Digital signatures, commitment schemes, message authentication, key derivation, proof-of-work, Merkle trees, and the Fiat–Shamir transform behind Halo 2’s non-interactive arguments all use one, because an *ideal* hash acts as a deterministic source of fresh randomness keyed by its input. This section defines the three classical security notions and proves their hierarchy (Proposition 3.8), bounds generic collision finding at about the square root of the number of digests (Lemma 3.11), proves the two powers of the random oracle model on which Fiat–Shamir proofs rest (Proposition 3.14), and proves that domain separation keeps a digest computed for one use from serving in another (Proposition 3.17). It then constructs hashing into a finite field and onto an elliptic curve (from which the next section’s commitment generators derive) and the arithmetisation-friendly hash Poseidon that Orchard evaluates inside its circuit, and registers the transparent layer’s SHA-256-derived hashes.

3.1 Syntax

The set of all finite binary strings is $\{0, 1\}^*$; the strings of length exactly n form $\{0, 1\}^n$; the length of a string x is $|x|$; and concatenation is written $x \parallel y$.

Definition 3.1 (Hash function). A *hash function* with output length $n \in \mathbb{N}$ is a function

$$H: \{0, 1\}^* \longrightarrow \{0, 1\}^n.$$

Elements of the codomain are called *digests*, *hashes*, or *tags*. The function is *compressing* on a domain $D \subseteq \{0, 1\}^*$ if D contains strings longer than n ; when the domain is all of $\{0, 1\}^*$, the function is overwhelmingly many-to-one.

Practice deploys a single fixed function such as SHA-256, but collision resistance of a fixed function admits no meaningful asymptotic definition. For any fixed compressing H there *exist* inputs $x \neq x'$ with $H(x) = H(x')$, by the pigeonhole principle, so a short program that outputs

one such hardcoded pair exists, is efficient, and always succeeds; the asymptotic statement “no efficient adversary finds a collision” is therefore *false* for every fixed compressing function. (No collision for SHA-256 is known, but existence suffices to falsify the universally quantified claim.) The standard remedy indexes the function by a key.

Definition 3.2 (Keyed hash family). A *keyed hash family* (or *hash function ensemble*) is a function

$$H: \mathcal{K} \times \{0, 1\}^* \longrightarrow \{0, 1\}^n,$$

where $\mathcal{K}$ is a finite key space. A key $s \in \mathcal{K}$ is sampled *publicly* at random, fixed, and we write $H_s(\cdot) := H(s, \cdot)$ for the family member in force.

The key indexes the member of the family that the adversary must attack: a hardcoded collision for one member is of no use against an independently sampled member, so “no efficient adversary finds a collision for a random member” is a meaningful asymptotic statement.

Remark 3.3 (Convention: keyed definitions, unkeyed deployments). This volume follows the standard convention. Security notions are stated for the keyed family; every concrete construction deployed is unkeyed; and the key is identified, informally, with “the choice of standard”: the sampling of s occurred, once and in public, when the function was designed. No formal theorem connects the two readings; the gap is the one that separates the random oracle model of §3.4 from deployed hashes.

Definition 3.4 (Extendable-output function). An *extendable-output function* (XOF) is a primitive producing a digest of caller-chosen length: syntactically a function

$$H: \{0, 1\}^* \times \mathbb{N} \longrightarrow \{0, 1\}^*, \quad |H(x, \ell)| = \ell,$$

whose outputs are *prefix-consistent*: for $\ell \leq \ell'$, the string $H(x, \ell)$ is a prefix of $H(x, \ell')$, so a longer request extends a shorter one as a stream.

The canonical XOFs are SHAKE128 and SHAKE256 from the SHA-3 family. Hashing into a finite field or onto a curve (§3.6) needs a *controlled number* of uniform bits for reduction modulo a prime, which an XOF supplies directly.

3.2 Security notions: preimage, second-preimage, and collision resistance

Each of the three classical security notions is a game between a challenger and a PPT adversary $\mathcal{A}$, in the template of Definition 1.15; the keyed family H is secure in a given sense if every PPT adversary wins the corresponding game with probability negligible in the security parameter λ (*Math Guide*, §“Polynomial, exponential, and negligible functions”; §1.2). All sizes are functions of λ : the output length $n = n(\lambda)$, the key length, and a polynomially bounded challenge input length $m = m(\lambda)$ from which the games sample their random challenge $x \xleftarrow{\$} \{0, 1\}^m$.

Definition 3.5 (Preimage resistance). The game $\text{PRE}_H^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger samples $s \xleftarrow{\$} \mathcal{K}$ and $x \xleftarrow{\$} \{0, 1\}^m$, computes $y := H_s(x)$, and sends (s, y) to the adversary.
2. The adversary outputs $x' \in \{0, 1\}^*$.
3. The game outputs 1 iff $H_s(x') = y$.

The family H is *preimage resistant* (or *one-way*) if for every PPT $\mathcal{A}$,

$$\text{Adv}_H^{\text{pre}}(\mathcal{A}, \lambda) := \Pr[\text{PRE}_H^{\mathcal{A}}(\lambda) = 1] \in \text{negl}(\lambda).$$

Any preimage of y wins; the adversary need not recover the original x .

Definition 3.6 (Second-preimage resistance). The game $\text{SEC}_H^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger samples $s \xleftarrow{\$} \mathcal{K}$ and $x \xleftarrow{\$} \{0, 1\}^m$, and sends (s, x) to the adversary.
2. The adversary outputs $x' \in \{0, 1\}^*$.
3. The game outputs 1 iff $x' \neq x$ and $H_s(x') = H_s(x)$.

The family H is *second-preimage resistant* if for every PPT $\mathcal{A}$,

$$\text{Adv}_H^{\text{sec}}(\mathcal{A}, \lambda) := \Pr[\text{SEC}_H^{\mathcal{A}}(\lambda) = 1] \in \text{negl}(\lambda).$$

Definition 3.7 (Collision resistance). The game $\text{COL}_H^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger samples $s \xleftarrow{\$} \mathcal{K}$ and sends s to the adversary.
2. The adversary outputs a pair (x, x') .
3. The game outputs 1 iff $x \neq x'$ and $H_s(x) = H_s(x')$.

The family H is *collision resistant* if for every PPT $\mathcal{A}$,

$$\text{Adv}_H^{\text{col}}(\mathcal{A}, \lambda) := \Pr[\text{COL}_H^{\mathcal{A}}(\lambda) = 1] \in \text{negl}(\lambda).$$

Proposition 3.8 (Hierarchy of notions). *Let H be a keyed hash family that is compressing, with challenge input length $m > n$. Then:*

1. *collision resistance implies second-preimage resistance, with no loss; and*
2. *second-preimage resistance implies preimage resistance up to an additive loss of 2^{n-m} , which is negligible—so the implication holds outright—whenever H is sufficiently compressing, $m - n = \omega(\log \lambda)$.*

Neither implication reverses in general.

Proof. (1) From an adversary $\mathcal{A}$ against second-preimage resistance, construct $\mathcal{B}$ against collision resistance: on input the key s , the reduction $\mathcal{B}$ samples $x \xleftarrow{\$} \{0, 1\}^m$ itself, runs $x' \leftarrow \mathcal{A}(s, x)$ —the simulated view is exactly that of SEC, since $\mathcal{B}$ samples x from the same distribution as the challenger—and outputs the pair (x, x') . Whenever $\mathcal{A}$ succeeds, $x' \neq x$ and $H_s(x') = H_s(x)$, which is precisely a collision. Hence $\text{Adv}_H^{\text{col}}(\mathcal{B}, \lambda) \geq \text{Adv}_H^{\text{sec}}(\mathcal{A}, \lambda)$: the reduction is tight, and if the left side is negligible so is the right.

(2) Towards the contrapositive, let $\mathcal{A}$ be an inverter; we build a second-preimage finder $\mathcal{B}$. On input (s, x) , the reduction computes $y := H_s(x)$, runs $x' \leftarrow \mathcal{A}(s, y)$ —again a perfect simulation of PRE, since x is uniform and y is its digest—and outputs x' . Whenever $\mathcal{A}$ succeeds, $H_s(x') = y = H_s(x)$; it remains to show that $x' \neq x$ except with probability at most 2^{n-m} . Fix $\mathcal{A}$'s coins, so that its output on each digest is a single string $x' = \mathcal{A}(s, y)$. An input x can satisfy $x = \mathcal{A}(s, H_s(x))$ only if x is *the* output assigned to its own digest, so each of the 2^n digests contributes at most one such input, and at most 2^n inputs in all are “fixed points” of the composed map. Over uniform $x \xleftarrow{\$} \{0, 1\}^m$, therefore, $\Pr[x' = x] \leq 2^n / 2^m = 2^{n-m}$; averaging over $\mathcal{A}$'s coins preserves the bound. Hence

$$\text{Adv}_H^{\text{sec}}(\mathcal{B}, \lambda) \geq \text{Adv}_H^{\text{pre}}(\mathcal{A}, \lambda) - 2^{n-m}.$$

The loss is *additive*, not multiplicative: nothing bounds $\Pr[x' \neq x]$ *conditioned* on success, because $\mathcal{A}$'s successes might concentrate on small fibres where returning x itself is likely. Sufficient compression, $m - n = \omega(\log \lambda)$, makes 2^{n-m} negligible and the implication unconditional.

Non-reversal is witnessed by Example 3.9 below. $\square$

Example 3.9 (Preimage-resistant but not collision-resistant). Take any one-way family H and define $H'_s(x) := H_s(\bar{x})$, where $\bar{x}$ deletes the last bit of x . The family H' inherits one-wayness: a preimage challenge (s, y) for H is distributed exactly as one for H' at one bit greater challenge length, and an H' -inverter's output x'' satisfies $H_s(\bar{x}'') = y$, so deleting its last bit gives the reduction a preimage under H . Yet the pair $x \parallel 0, x \parallel 1$ is an immediate collision for every x , found without any computation. Hence preimage resistance does not imply collision resistance. The other separations are constructed similarly; the implications of Proposition 3.8 are strictly one-directional.

Remark 3.10 (Which application needs which notion). Password storage, which keeps a digest of each password so that a stolen table does not reveal them, requires *preimage* resistance: an attacker who obtains the table holds digests, and any preimage suffices to authenticate. (Preimage resistance is not the only requirement there: passwords are guessable, so deployments hash each together with a unique random string, its *salt*, through a deliberately slow function, to resist offline guessing.) A signature scheme that signs $H(m)$, where an adversary may later try to substitute a forged document for the honestly chosen m , needs *second-preimage* resistance: the message is fixed and honest, and the attacker must collide with it. A scheme in which the *signer* may be dishonest, preparing two contracts with the same digest, signing the benign one and presenting the malicious one, needs full *collision* resistance, because the adversary then controls both messages. Commitment schemes, Merkle trees, and the Fiat–Shamir transform all fall into this last category and therefore require collision resistance.

3.3 The birthday bound on generic collision finding

The generic collision attack evaluates the hash on many inputs until two digests coincide; the birthday calculation of the *Math Guide* bounds its cost.

Lemma 3.11 (Birthday bound for hashing). *Let $H: \{0, 1\}^* \rightarrow \{0, 1\}^n$ and $N = 2^n$. Evaluate H on q distinct inputs, in the idealised model where the q digests behave as independent uniform samples from $\{0, 1\}^n$, and let C be the event that two digests coincide. Then*

$$\Pr[C] = 1 - \prod_{i=0}^{q-1} \left(1 - \frac{i}{N}\right),$$

with the two-sided estimate

$$1 - e^{-q(q-1)/(2N)} \leq \Pr[C] \leq \frac{q(q-1)}{2N} \leq \frac{q^2}{2N},$$

and the lower bound gives $\Pr[C] \geq 1/2$ once $q \geq 1.18\sqrt{N}$. A collision therefore appears with constant probability after about $\sqrt{N} = 2^{n/2}$ evaluations.

Proof. The exact product and both estimates are the birthday-bound proposition of the *Math Guide* (§“The union bound and a birthday calculation”), instantiated with set size $N = 2^n$ and sample count $k = q$. Only the half-probability threshold is new. Setting the exponent of the lower bound equal to $-\ln 2$ gives $q(q-1) = 2N \ln 2$, i.e.

$$q \approx \sqrt{2 \ln 2} \sqrt{N} \approx 1.18\sqrt{N},$$

the constant being $\sqrt{2 \ln 2} = 1.1774 \dots$; for $q \geq 1.18\sqrt{N}$ the exponential falls below $1/2$ and $\Pr[C] \geq 1/2$. $\square$

Remark 3.12 (Reading the birthday bound). *The square-root law.* An n -bit digest yields only $n/2$ bits of collision security in the sense of Definition 1.32. The conventional 128-bit collision security requires $n = 256$; for this reason SHA-256 (there is no “SHA-128”) is the standard digest size, and the transcript hashes of Fiat–Shamir-style protocols must carry at least 256-bit output.

The attack is practical. The direct attack stores all $q \approx 2^{n/2}$ digests and sorts them, using $2^{n/2}$ memory, the same obstruction that makes baby-step giant-step impractical (Remark 2.12). Collision finding, however, like the discrete-logarithm walk of Proposition 2.13, admits cycle finding: one iterates $x \mapsto H(x)$ and detects the cycle with Floyd’s or Brent’s algorithm, in the manner of Pollard’s rho, or runs the van Oorschot–Wiener parallel collision search across many machines. These methods achieve the same $2^{n/2}$ time in essentially constant memory, so parameter choices must meet the birthday cost in practice, not only in theory.

Preimages cost the square. The best generic attack against preimage and second-preimage resistance is brute force: about 2^n evaluations to hit a prescribed digest. Those notions therefore have full n -bit generic security, twice the collision level, because the attacker cannot exploit birthday coincidences among self-chosen pairs.

3.4 The random oracle model

This subsection defines the random oracle precisely and *proves* the two powers stated informally in Remark 1.27; the random oracle model is that of Definition 1.26, one of the two idealisations under which proofs proceed, and a heuristic with the limits of Remark 1.29.

Definition 3.13 (Random oracle). A *random oracle* with output length n is a function $\mathcal{O}: \{0, 1\}^* \rightarrow \{0, 1\}^n$ drawn uniformly at random from the set of all such functions. Equivalently—and this is the form every proof uses—the oracle is realised *lazily*: a table, initially empty, is maintained; on query x , if x is in the table the stored value is returned, and otherwise a fresh $\mathcal{O}(x) \xleftarrow{\$} \{0, 1\}^n$ is sampled, recorded, and returned.

Access to the oracle is *only* by querying: it has no description shorter than its (exponentially long) truth table. A scheme is analysed in the random oracle model (ROM) when every party, honest or adversarial, queries a single shared $\mathcal{O}$ in place of a concrete hash, and security is proved relative to the random choice of $\mathcal{O}$; on deployment a concrete hash such as SHA-256 or BLAKE2b *instantiates* the oracle.

Proposition 3.14 (The two powers of the ROM). *In a ROM security proof, the reduction, which simulates $\mathcal{O}$ for the adversary, gains two capabilities that no concrete hash affords.*

1. *Observability (extractability). The reduction sees every input the adversary queries. Since querying is the only way to learn anything about $\mathcal{O}(x)$, if the adversary's output depends on $\mathcal{O}(x)$ then the reduction holds x .*
2. *Programmability. On a not-yet-queried input x , the reduction may choose the response $\mathcal{O}(x)$ to be any value it selects, provided the value is distributed uniformly; the simulation remains perfect, and the reduction can thereby plant a challenge—a value it must invert, say—inside an oracle answer.*

Proof. Both capabilities are immediate from the lazy realisation of Definition 3.13, because in the simulation the reduction *is* the table-keeper. Every query arrives at the reduction before an answer exists, which is observability. And on a fresh input the honest table-keeper would sample the answer uniformly and independently of the adversary's view so far; a reduction that instead inserts any value *with that same distribution* produces a view identical to the honest one, while the table keeps repeated queries consistent. That is programmability. $\square$

Remark 3.15 (Why Fiat–Shamir needs the ROM). The Fiat–Shamir transform uses both powers at once. It is the compiler that renders Halo 2's interactive protocol non-interactive, developed in this volume under “From identification to signature via the Fiat–Shamir transform” and in full generality under “The Fiat–Shamir transform: from interactive to non-interactive”. The transform replaces the verifier's random challenge e by $e = \mathcal{O}(\text{transcript so far})$. Soundness of the resulting non-interactive argument is proved by *rewinding*: the reduction observes (capability 1) the transcript the prover commits to, then *reprograms* (capability 2) the oracle to return a fresh independent challenge on that same transcript, extracting two accepting transcripts that share a

prefix and, from them, a witness—the forking technique previewed in Remark 1.24. Neither step is available against a concrete hash, whose outputs the reduction can neither observe selectively nor rewrite, so the argument holds *only* in the ROM.

3.5 Domain separation and personalisation

A deployed protocol uses one hash function for many purposes: it derives keys, commits to notes, tags spent notes, and seeds Fiat–Shamir challenges. Digests are interchangeable bit strings, so an output produced for one purpose can be *replayed* as another unless domain separation keeps the purposes apart.

Definition 3.16 (Domain separation). Let H be a hash modelled as a random oracle, and let $\{\tau_i\}_{i \in I}$ be a set of distinct, *prefix-free* domain tags (also called labels or personalisation strings), one per intended use: no τ_i is a prefix of another—as when all tags have equal length, or when each is length-prefixed. The *use- i hash* is

$$H^{(i)}(x) := H(\tau_i \parallel x).$$

Proposition 3.17 (Domain separation yields independent oracles). *If H is a random oracle and the tags $\{\tau_i\}$ are prefix-free, then the family $\{H^{(i)}\}_{i \in I}$ is distributed exactly as a collection of independent random oracles. Consequently a digest obtained from $H^{(i)}$ is—except with the negligible probability of a generic collision—useless against any use $j \neq i$.*

Proof. Prefix-freeness makes the domains $\tau_i \parallel \{0, 1\}^*$ pairwise disjoint: a string with prefix τ_i cannot also have prefix τ_j unless one tag is a prefix of the other. Inputs queried through different $H^{(i)}$ are therefore never equal. A random oracle assigns independent uniform values to distinct inputs, so restricting H to the disjoint slices yields, per slice, a uniformly random function, independent across slices—precisely a collection of independent random oracles. The replay conclusion follows: the value $H^{(i)}(x)$ reveals nothing about any $H^{(j)}(\cdot)$ with $j \neq i$. $\square$

Three idioms implement the technique in deployed practice; Zcash uses all three.

1. *Context tagging.* An ASCII string naming the use is bound into every call, either prepended, as in Definition 3.16, or in any framing that keeps the tag unambiguously delimited from the data. The Orchard tag inventory is of this kind: the strings `z.cash:Orchard-cv`, `z.cash:Orchard-NoteCommit`, and `z.cash:Orchard-CommitIvk` separate the value commitment, note commitment, and key commitment uses of one group-hash (protocol specification §§ 5.4.8.3–5.4.8.4), and `z.cash:Orchard-gd` separates diversified key generation (protocol specification § 5.4.1.6). The deployed framing differs from the definition’s: the curve hash of §3.7 *appends* its domain tag after the message, closed by a byte carrying the tag’s length (protocol specification § 5.4.9.8); the length byte makes the tag self-delimiting, so distinct tags still induce disjoint effective inputs, which is all that the proof of Proposition 3.17 uses. The tags enter that hash in two ways, and the commitment tags in both: `-cv` and `-gd` serve directly as its domain tag, while each commitment tag, suffixed `-M`, is hashed as the

message under the fixed domain tag `z.cash:SinsemillaQ` to derive one of the commitment’s two base points and, suffixed `-r`, serves directly as the domain tag deriving the other (protocol specification §§ 5.4.1.9 and 5.4.8.4).

2. *Native personalisation.* BLAKE2b’s parameter block carries a dedicated 16-byte personalisation field that is mixed into the initialisation vector, so distinct personalisations separate the primitive’s effective inputs before a single message byte is processed; this field is the reason Zcash prefers BLAKE2 for its oracles. With BLAKE2b modelled as a random oracle of the pair (personalisation, message), the fixed-length field is a prefix-free tag and Proposition 3.17 makes the resulting domains independent; a concrete BLAKE2b instantiation does not make that independence a fact “by construction”. The deployed personalisations include `Zcash_ExpandSeed` for key expansion (protocol specification § 5.4.2), `Zcash_OrchardKDF` for the note-encryption key-derivation function (KDF, the object of §6.6; protocol specification § 5.4.5.6), and `Zcash_Orchardock` for the outgoing cipher key (protocol specification § 5.4.2)—all exactly 16 bytes.
3. *Suffix and framing bits in the SHA-3 family.* FIPS 202 reserves domain-separation suffix bits appended to the message before padding: 01 for SHA-3 hashing and 1111 for the SHAKE XOFs. SHA3-256 and SHAKE256 share the same underlying permutation, yet the framed inputs differ, so their effective inputs are disjoint—though their finite digests can still coincide by chance.

An ad hoc tag that is a prefix of another merges two domains without any visible failure and readmits the replay attack, which is why Definition 3.16 demands prefix-freeness rather than mere distinctness. The Merkle trees of the final section apply the same technique *within* a single structure, tagging tree layers to prevent a forgery that confuses leaves with internal nodes; see “Layer and domain separation” there.

3.6 Hashing to a field element

Commitments over the Pasta curves, Fiat–Shamir challenges, and key derivations hash a message *to a field element*: a near-uniform element of $\mathbb{F}_p$ rather than a bit string. Reducing n uniform bits modulo a prime p is not uniform on $\mathbb{F}_p$, because 2^n is not a multiple of p and some residues receive one more preimage than others; the remedy is to hash to a *wide* output and reduce.

Construction 3.18 (Expand-then-reduce). Let p be a prime of $L = \lceil \log_2 p \rceil$ bits, and fix a XOF, or a domain-separated hash, expand, producing arbitrary-length near-uniform output. To map a message m to $\mathbb{F}_p$, with a domain tag τ and a security slack of k bits (commonly $k = 128$):

1. compute $b \leftarrow \text{expand}(\tau \parallel m, L + k)$, a string of $L + k$ near-uniform bits;
2. interpret b as an integer $B \in \{0, \dots, 2^{L+k} - 1\}$ and output $B \bmod p \in \mathbb{F}_p$.

Proposition 3.19 (Near-uniformity of expand-then-reduce). *If expand is a random oracle, the output of Construction 3.18 lies within statistical distance (Definition 1.6) at most*

$$\frac{p}{2^{L+k+1}} \leq 2^{-k}$$

of the uniform distribution on $\mathbb{F}_p$. With $k = 128$, no algorithm whatsoever—efficient or not—distinguishes the output from a uniform field element with advantage better than 2^{-128} .

Proof. The integer B is uniform on $\{0, \dots, K-1\}$ with $K = 2^{L+k}$, and the bias of reducing a uniform B modulo p is exactly the situation of the modular-reduction proposition of the *Math Guide* (§“Uniform sampling and the bias of modular reduction”): each residue’s probability differs from $1/p$ by at most $1/K$, because the preimage counts $\lfloor K/p \rfloor$ and $\lceil K/p \rceil$ differ by one, and summing over the p residues gives statistical distance at most $p/(2K) = p/2^{L+k+1}$ —the sharper form proved there. Since $p < 2^L$, this is less than $2^{-(k+1)} \leq 2^{-k}$. The final claim is the variational characterisation of statistical distance (Proposition 1.7): no decision procedure exceeds the distance. $\square$

The slack is necessary. With $k = 0$, a single L -bit block reduced modulo p , the bias can be a constant: for p slightly above a power of two, $K = 2^L$ is slightly less than twice p , the residues beyond the wrap-around point receive only one preimage while the rest receive two, and each residue with one preimage has as little as half its uniform probability.

Deployed standards package the construction. The IETF hash-to-curve specification (RFC 9380) defines `hash_to_field( $m, count$ )`, producing `count` field elements by calling an expansion routine—`expand_message_xmd` over a fixed-output hash, or `expand_message_xof` over a XOF—with a *mandatory* domain-separation tag, and reducing each chunk of

$$L_{\text{RFC}} = \left\lceil (\lceil \log_2 p \rceil + k) / 8 \right\rceil \text{ bytes}$$

modulo p , where k is the suite’s target security level in bits. The RFC’s own suites for 255-bit fields—the curve25519 family—set $k = 128$, hence 48-byte chunks. The Zcash suite for Pallas and Vesta takes the larger $k = 256$, reducing each element from a full 64-byte BLAKE2b-512 digest, so the slack of Proposition 3.19 is realised with margin (expansion is BLAKE2b in the XMD pattern under the suite tag `{tag}-{curve}_XMD:BLAKE2b-SSWU_R0_`, and each of the two 64-byte chunks is reduced modulo p ; protocol specification § 5.4.9.8). The same procedure maps identities to field elements and produces the field elements that the next subsection maps to a curve. Halo 2’s deployed Fiat–Shamir transcript uses the same wide-output-then-reduce principle: each challenge is the 64-byte digest of a BLAKE2b state personalised with `Hal02-Transcript`, reduced modulo the field order; the order in which that state absorbs messages and squeezes challenges is fixed by the Halo 2 protocol and treated in a later volume of the series.

3.7 Hashing to a curve point

Commitment generators with discrete logarithms known to no party are derived by a hash *onto an elliptic curve*, mapping a message to a point of $E(\mathbb{F}_p)$. The output must be near-uniform in the relevant subgroup, so that the map can stand in for a random oracle into the group in security arguments, and the map must run in *constant time*, because its input often depends on secret data.

The direct approach, *try-and-increment*, hashes the message to a candidate $x \in \mathbb{F}_p$ (Construction 3.18); tests whether $g(x) = x^3 + ax + b$ is a quadratic residue, so that a y -coordinate exists; and, if not, increments x and retries. It is correct and roughly uniform, but about half of all x fail the residue test, since the squares are half of $\mathbb{F}_p^\times$ (*Math Guide*, §“Quadratic residues and the Euler criterion”, whose Legendre symbol $\left(\frac{g(x)}{p}\right)$ is exactly the test); the number of trials depends on the message, and the running time leaks information about m to side-channel attacks. A constant-time map must succeed on the *first* try, for every input.

Construction 3.20 (Simplified Shallue–van de Woestijne–Ulas map). Let $E: y^2 = g(x) = x^3 + ax + b$ over $\mathbb{F}_p$ with $ab \neq 0$, and choose, as RFC 9380 requires, a constant $Z \in \mathbb{F}_p$ such that Z is a nonsquare, $Z \neq -1$, the polynomial $g(X) - Z$ is irreducible over $\mathbb{F}_p$, and $g(b/(Za))$ is a square. (The RFC selects Z mechanically, as the candidate of smallest absolute value meeting these conditions—a nothing-up-my-sleeve constant with no hidden design freedom.) The map $\text{SWU}: \mathbb{F}_p \rightarrow E(\mathbb{F}_p)$ sends u to a point as follows. Compute the two candidate x -coordinates

$$x_1 = \frac{-b}{a} \left(1 + \frac{1}{Z^2 u^4 + Zu^2} \right), \quad x_2 = Zu^2 x_1,$$

with the fallback $x_1 = b/(Za)$ when the denominator $Z^2 u^4 + Zu^2$ vanishes (as at $u = 0$). Set $x = x_1$ if $g(x_1)$ is a square and $x = x_2$ otherwise; let y be a square root of $g(x)$ (*Math Guide*, §“Square roots modulo a prime”), its sign fixed by a convention—the deployed map matches the parity of y to that of u —and output (x, y) .

The construction contains no trial loop, hence runs in constant time; it remains to prove that one of the two candidates always yields a point.

Proposition 3.21 (Correctness of the branch selection). *Under the parameter conditions of Construction 3.20, the map is well-defined for every $u \in \mathbb{F}_p$: the selected value $g(x)$ is a square, so $\text{SWU}(u)$ is a genuine point of $E(\mathbb{F}_p)$.*

Proof sketch. Suppose first that $D = Z^2 u^4 + Zu^2 \neq 0$. The argument rests on the algebraic identity

$$g(x_2) = Z^3 u^6 g(x_1),$$

a polynomial identity in u : substitute $x_2 = Zu^2 x_1$ into g and simplify using the formula for x_1 (the computation is omitted). If $g(x_1) = 0$, the first candidate yields a point, with $y = 0$. Otherwise apply multiplicativity of the Legendre symbol (*Math Guide*, §“Quadratic residues and the Euler criterion”):

$$\left(\frac{g(x_2)}{p}\right) = \left(\frac{Z^3 u^6}{p}\right) \left(\frac{g(x_1)}{p}\right) = \left(\frac{Z}{p}\right) \left(\frac{(Zu^3)^2}{p}\right) \left(\frac{g(x_1)}{p}\right) = \left(\frac{Z}{p}\right) \left(\frac{g(x_1)}{p}\right),$$

since $Z^2 u^6 = (Zu^3)^2$ is a nonzero square, contributing $+1$ (as $D \neq 0$ forces $u \neq 0$). Because Z is a nonsquare, $\left(\frac{Z}{p}\right) = -1$, so the two symbols are opposite and the branch selection picks the square. If $D = 0$, the prescribed fallback has $g(x_1) = g(b/(Za))$, a square by the last condition on Z . Thus every input produces a valid affine point. $\square$

A single SWU evaluation does not suffice: the map is not surjective onto $E(\mathbb{F}_p)$, and its output is not uniformly distributed on its image. The standardised pipeline repairs both defects.

Construction 3.22 (The hash-to-curve pipeline). The RFC 9380 map `hash_to_curve`: $\{0, 1\}^* \rightarrow E(\mathbb{F}_p)$ composes four steps:

1. hash to the field: `hash_to_field`($m, 2$) produces two independent near-uniform field elements u_0, u_1 (§3.6);
2. map each through SWU: $Q_0 = \text{SWU}(u_0)$, $Q_1 = \text{SWU}(u_1)$;
3. sum on the curve: $R = Q_0 + Q_1$. Summing two independent SWU images provably repairs the non-uniformity: the sum lies within negligible statistical distance of uniform on $E(\mathbb{F}_p)$, the theorem on which the pipeline’s random-oracle claim rests;
4. clear the cofactor: multiply R by $b = \#E(\mathbb{F}_p)/r$ to land in the prime-order subgroup of order r in which the protocol operates.

The result is a constant-time hash from messages to group elements which, under the hash model and suite conditions of RFC 9380, can instantiate a random oracle into the group. For the Pasta curves the last step is vacuous—their cofactor is 1 (Definition 2.25).

Remark 3.23 (The isogeny required by j -invariant 0). For the Pasta curves the SWU step cannot be applied directly. The SWU formula for x_1 divides by a , and its numerator carries the factor $-b$: with $a = 0$ the formula is undefined, while $b = 0$ makes x_1 identically zero with $g(x_1) = 0$, collapsing the map to the constant point $(0, 0)$. The construction therefore requires $ab \neq 0$ —and Pallas and Vesta have $a = 0$ (the j -invariant-0 shape $y^2 = x^3 + 5$ of Definition 2.25; curves with $b = 0$, $j = 1728$, are excluded symmetrically). The obstruction is intrinsic, not notational: the j -invariant is an isomorphism invariant, and a Weierstrass change of variables rescales (a, b) to $(u^4 a, u^6 b)$ (*Math Guide*, §“The j -invariant and the GLV endomorphism”), so *no* model of these curves attains $ab \neq 0$. The standardised remedy evaluates simplified SWU on an *isogenous* auxiliary curve $E' : y^2 = x^3 + a'x + b'$ with $a'b' \neq 0$, then transports the result through an *isogeny* $E' \rightarrow E$: a nonconstant map between curves given by polynomial coordinate formulas and respecting the group law. The deployed implementation applies a degree-3 isogeny, and—because an isogeny is a group homomorphism—sums $Q_0 + Q_1$ on E' first and transports the single sum (protocol specification § 5.4.9.8).

Construction 3.24 (Nothing-up-my-sleeve generators). To generate public generators $G_1, \dots, G_k$ of a vector commitment (the Pedersen vector commitment of Construction 4.22 in the next section) so that no one knows any discrete-logarithm relation among them, derive each as

$$G_i = \text{hash_to_curve}(\tau \parallel i)$$

from a fixed public domain tag τ —often a human-readable protocol name—and the index i . Because the seed is a transparent, auditable string with no hidden parameters, no party could have constructed a backdoor relation in advance. Halo 2 produces the generators of its Pedersen vector and inner-product commitments (the latter Construction 4.38) in exactly this way:

every generator G_i is `hash_to_curve("Halo2-Parameters")` applied to a five-byte message—a zero byte, then the four-byte little-endian encoding of i —the leading zero keeping the G_i disjoint from the two auxiliary generators the same tag derives from the one-byte messages 1 and 2, and Orchard’s value commitment derives its two bases from the tag `z.cash:0rchar-d-cv` with the one-byte messages “v” and “r”.

Remark 3.25 (Why NUMS generators are binding-safe). Knowledge of a discrete-logarithm relation $\sum_i a_i G_i = 0$ with not all a_i zero would let a committer open one commitment two ways, violating the binding property that the next section defines and relies on. The NUMS derivation excludes this: the generators are outputs of a function modelled as a random oracle into the group, so they are distributed as independent uniform group elements, and finding a discrete-logarithm relation among independent uniform elements *reduces* to computing a discrete logarithm on E , hard under the standing assumptions of Remark 2.32; the reduction, which embeds its challenge in every generator, is Theorem 4.23 of the next section. Hence no efficient party, the scheme designer included, can know such a relation.

3.8 Arithmetisation-friendly hashing: Poseidon

SHA-2 and BLAKE2 are *bit-oriented*: they rearrange and XOR 32- and 64-bit words. In an arithmetic circuit over a large prime field $\mathbb{F}_p$, as in Halo 2, whose cost model (developed in a later volume of the series) prices each field multiplication as a constraint, every Boolean operation of such a hash must be re-expressed through field arithmetic and range checks. An arithmetisation-friendly hash instead builds its round function from a few *low-degree field operations*, native to the circuit; Orchard employs Poseidon (Grassi, Khovratovich, Rechberger, Roy, and Schofnegger, 2019) through the mode of operation of Construction 3.26.

Construction 3.26 (The sponge construction). Fix a permutation $f: \mathbb{F}_p^t \rightarrow \mathbb{F}_p^t$ on a state of t field elements. Split the state into a *rate* of r elements—the only part input is added into and output is read from—and a *capacity* of $c = t - r$ elements, never accessed directly, which carries the security margin. To hash, *absorb* the message into the rate, r elements at a time, applying f between blocks; then *squeeze* output from the rate, applying f between reads. Collision and preimage resistance hold up to approximately $\sqrt{p^c}$ work, as for a random sponge: the birthday bound of Lemma 3.11 applied to the p^c capacity states the adversary cannot see.

Construction 3.27 (The Poseidon permutation). The permutation f is a sequence of rounds, each comprising three steps:

1. *add round constants*: add a fixed vector in $\mathbb{F}_p^t$ to the state;
2. *S-box*: apply $x \mapsto x^\alpha$ to state elements, where $\alpha \geq 3$ is the smallest integer with $\gcd(\alpha, p - 1) = 1$, so that the power map is a bijection on $\mathbb{F}_p$; over the Pasta fields $\alpha = 5$, because $3 \mid p - 1$ there rules out $\alpha = 3$;
3. *mix*: multiply the state by a fixed maximum-distance-separable (MDS) matrix $M \in \mathbb{F}_p^{t \times t}$ —one whose every square submatrix is invertible, giving the fastest possible diffusion of any element’s change across the whole state.

The cost-saving principle (the HADES strategy) distinguishes two round types: a few *full* rounds at the start and end apply the S-box to all t elements, while in the many *partial* rounds between them the S-box touches a single element—the MDS mix still diffusing it across the state. Round counts are chosen to resist the known algebraic (Gröbner-basis) attacks.

Remark 3.28 (Circuit cost). Inside a circuit, the only non-linear operation in all of Poseidon is $x \mapsto x^5$, verified by a couple of multiplication gates per S-box; the round-constant additions and the MDS multiplication are linear and add almost no cost. A 2-to-1 Poseidon hash costs a few hundred constraints, against tens of thousands for a bit-oriented hash; this difference determines whether such a circuit is practical.

Example 3.29 (The Orchard Poseidon instance). Orchard fixes $t = 3$ (rate 2, capacity 1), $\alpha = 5$, and 8 full plus 56 partial rounds, over the Pallas base field, at the 128-bit security level (protocol specification § 5.4.1.10). It is used in *constant-input-length* mode as a two-input hash:

$$\text{PoseidonHash}(x, y) = f([x, y, 2^{65}])_0,$$

the first component of the state after a single permutation of the input padded by the fixed capacity value $2^{65} = 2 \cdot 2^{64}$, which encodes the input length 2 into the capacity so that no message padding is required (an input of any other constant length is encoded the same way, with that length in place of the factor 2). Keyed by its first argument, this hash serves as the pseudorandom function

$$F_{\text{nk}}(\rho) = \text{PoseidonHash}(\text{nk}, \rho)$$

—the PRF notion is defined in “Pseudorandom functions and permutations” later in this volume—deployed as $\text{PRF}^{\text{nfOrchard}}$ (protocol specification § 5.4.2), from which Orchard constructs the nullifier—a construction developed in a later volume of the series; this volume supplies only the PRF. Unlike the Pedersen and Sinsemilla hashes constructed in the next section, whose collision resistance reduces to a standard assumption (discrete log), Poseidon’s security rests on cryptanalytic confidence in its round function: the same epistemic footing as SHA-256, not a reduction.

3.9 The transparent layer’s hashes

Where the Orchard shielded protocol hashes with BLAKE2b, Sinsemilla, and Poseidon, the transparent layer, inherited from Bitcoin, hashes with SHA-256 and two functions derived from it, none requiring results beyond §3.2 and the birthday bound of §3.3. This register fixes their names, domains, and deployed uses for the volumes above.

- *RIPEMD-160*. A 160-bit iterated (Merkle–Damgård, the construction of §5.3) hash, $\{0, 1\}^* \rightarrow \{0, 1\}^{160}$, older than Bitcoin and inherited with its address format, whose collision resistance is limited to the 80-bit birthday bound of Lemma 3.11. Zcash uses it only inside hash160 below and the script opcodes `OP_RIPEMD160` and `OP_HASH160`.
- *The address hash* $\text{hash160} := \text{RIPEMD160} \circ \text{SHA256}$. A pay-to-public-key-hash (P2PKH) address carries the 20-byte hash160 of the 33-byte compressed encoding of an ECDSA

public key (the scheme is registered in §8.10), and a pay-to-script-hash (P2SH) address the hash160 of a *redeem script*, the spending condition the spender later reveals and satisfies (protocol specification § 5.6.1.1). The P2PKH output script—the condition a spend of the output must satisfy—is `OP_DUP OP_HASH160  $\langle b \rangle$  OP_EQUALVERIFY OP_CHECKSIG`, which recomputes the hash from the public key the spender reveals; wallets compute it through the `zcash_transparent` crate, and a partially created transaction (PCZT), the multi-party construction format treated in a later volume of the series, carries on each transparent input preimage maps keyed by RIPEMD-160, SHA-256, hash160, and hash256 digests.

- *Double SHA-256 (SHA-256d, or hash256)*. The composition $x \mapsto \text{SHA256}(\text{SHA256}(x))$ (protocol specification § 5.4.1.1), whose outer application also closes the length-extension property of plain SHA-256 (§5.3). It hashes block headers; its leading four bytes are the checksum that `Base58Check`, the string encoding of transparent addresses, appends to the raw address bytes before converting to base 58 (protocol specification § 5.6.1.1) and the payload checksum in every peer-to-peer message header; and it is the legacy transaction identifier: the txid of a version-4-or-earlier transaction is SHA-256d of its serialisation (protocol specification § 7.1.1), whereas a version-5 txid is the BLAKE2b digest tree of ZIP 244.

4 Commitment schemes

A commitment scheme lets a sender fix a message in a value, the commitment, and later reveal the message to a receiver: once the commitment is delivered, the sender can no longer alter the message (*binding*), yet the receiver cannot learn it until it is opened (*hiding*). This section defines commitment schemes, their setup models, and hiding and binding as games in the sense of §1.4; proves that no nontrivial scheme is both perfectly hiding and perfectly binding (Theorem 4.8); and constructs hash-based, Pedersen, Pedersen vector and Sinsemilla commitments, the pairing-based and inner-product polynomial commitments of §4.9, and verifiable secret sharing. The Pedersen and Sinsemilla commitments require only the discrete-logarithm hardness of §2 and the nothing-up-my-sleeve generators of §3, and most later constructions use them. In Orchard, the value commitments checked by binding signatures, the note commitments accumulated in Merkle trees and the witness polynomials of the Halo 2 proof system are all commitments in this sense; polynomial commitments are the cryptographic component of the succinct argument systems of §9, and verifiable secret sharing serves the threshold signatures of a later volume.

4.1 Commit and reveal

The sender computes a commitment to a message and transmits it. Later, at a time fixed by the protocol, the sender opens the commitment by transmitting the message together with the randomness used to compute it, and the receiver checks the revealed message against the commitment. Hiding protects the *sender* against the *receiver* between commit and reveal; binding protects the *receiver* against the *sender* at reveal time. Without binding a commitment is no evidence of the message; without hiding it keeps nothing secret. The games of §4.3 formalise the two properties.

4.2 Syntax

Definition 4.1 (Commitment scheme). A *commitment scheme* Π for a message space $\mathcal{M}$ and randomness space $\mathcal{R}$ is a triple of algorithms (Setup, Commit, Open):

- The algorithm $\text{Setup}(1^\lambda)$ is PPT (§1.1) and outputs *public parameters* pp , which implicitly determine the message space $\mathcal{M}_{pp}$ and randomness space $\mathcal{R}_{pp}$ and are an implicit input to the remaining algorithms.
- The algorithm $\text{Commit}(pp, m; r)$, for $m \in \mathcal{M}_{pp}$ and $r \in \mathcal{R}_{pp}$, is deterministic given r and outputs a *commitment* c . We write $\text{Commit}(pp, m)$, without an explicit r , for the probabilistic algorithm that samples $r \xleftarrow{\$} \mathcal{R}_{pp}$ uniformly and returns r alongside c .
- The algorithm $\text{Open}(pp, c, m, r)$ is deterministic and outputs a bit: 1 (accept) or 0 (reject).

Correctness requires that honestly produced commitments open: for all pp in the support of $\text{Setup}(1^\lambda)$, all $m \in \mathcal{M}_{pp}$, and all $r \in \mathcal{R}_{pp}$,

$$\text{Open}(pp, \text{Commit}(pp, m; r), m, r) = 1.$$

The pair (m, r) is called the *opening* (or *decommitment*) of c . Unless stated otherwise, every scheme in this volume is *canonically opening*: $\text{Open}(pp, c, m, r)$ recomputes $\text{Commit}(pp, m; r)$ and checks equality with c . For such a scheme correctness holds automatically, and a valid opening of c is precisely a pair (m, r) with $\text{Commit}(pp, m; r) = c$.

Remark 4.2 (The two phases, and a third concern). In the *commit phase* the sender transmits c and nothing else; in the *reveal phase* it transmits the opening (m, r) and the receiver runs Open . A *setup phase* producing pp precedes both; who runs Setup , and whether that party can be trusted, is decisive for some of the constructions below (Remark 4.3).

Remark 4.3 (Three setup models). Commitment schemes divide by how much trust their parameters demand.

1. *Common reference string (CRS)*. A trusted party runs Setup and is assumed to *erase* the randomness behind it. The parameters have internal structure, and knowledge of the discarded randomness—the *trapdoor*—would break the scheme. Of the schemes treated in this section, only KZG (§4.9) requires this model.
2. *Uniform random string (URS), or transparent setup*. The parameters consist only of public coins: uniformly random group elements with no known relations among them. There is no trapdoor and no trusted party—anyone can re-derive the parameters from a public seed. This is the regime of the Pedersen and inner-product commitments below. A Pedersen generator derived by hashing to the curve (Construction 3.24) is transparent, although the textbook Setup samples and discards a secret.
3. *Plain*. No parameters at all, or only a standardised hash function—the home of the hash-based commitment below.

Transparent setup is preferable because a flawed or subverted trusted setup can silently destroy binding, as the KZG trapdoor shows (§4.9).

4.3 Hiding and binding

Both properties are games in the sense of Definition 1.15. In the hiding game the adversary is the receiver; in the binding game it is the sender.

Definition 4.4 (Hiding game and hiding). Let Π be a commitment scheme and $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ an adversary. For $b \in \{0, 1\}$, the game $\text{Hide}_{\Pi}^{\mathcal{A}}(\lambda, b)$ runs as follows.

1. The challenger runs $pp \leftarrow \text{Setup}(1^\lambda)$.
2. The adversary $\mathcal{A}_1(pp)$ outputs two messages $m_0, m_1 \in \mathcal{M}_{pp}$ and state st .
3. The challenger samples $r \xleftarrow{\$} \mathcal{R}_{pp}$ and computes $c = \text{Commit}(pp, m_b; r)$.
4. The adversary $\mathcal{A}_2(st, c)$ outputs a bit b' , which is the output of the experiment.

The *hiding advantage* of $\mathcal{A}$ is

$$\text{Adv}_{\Pi}^{\text{hide}}(\mathcal{A}, \lambda) := \left| \Pr[\text{Hide}_{\Pi}^{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{Hide}_{\Pi}^{\mathcal{A}}(\lambda, 1) = 1] \right|.$$

The scheme is *computationally hiding* if the advantage is negligible (§1.2) for every PPT $\mathcal{A}$; *statistically hiding* if it is negligible even for computationally unbounded $\mathcal{A}$; and *perfectly hiding* if it is exactly 0 for every adversary and every λ .

Proposition 4.5 (Distributional characterisation of hiding). *A commitment scheme Π is perfectly hiding if and only if for every pp in the support of Setup and every pair $m_0, m_1 \in \mathcal{M}_{pp}$, the random variables*

$$\text{Commit}(pp, m_0; r) \quad \text{and} \quad \text{Commit}(pp, m_1; r), \quad r \xleftarrow{\$} \mathcal{R}_{pp},$$

are identically distributed. It is statistically hiding if the statistical distance (Definition 1.6) between them is negligible. (Only the perfect clause is an equivalence: parameters of negligible Setup probability could exhibit large distance without granting any adversary noticeable advantage.)

Proof. Fix pp and m_0, m_1 , and write D_m for the distribution of $\text{Commit}(pp, m; r)$ over uniform r . By the variational characterisation of statistical distance (Proposition 1.7), $\Delta(D_{m_0}, D_{m_1})$ equals *exactly* the maximum advantage, over all decision procedures—efficient or not—of telling D_{m_0} from D_{m_1} . An adversary who outputs the pair (m_0, m_1) on parameters pp and applies the optimal test therefore achieves hiding advantage $\Delta(D_{m_0}, D_{m_1})$ whenever Setup produces pp ; conversely no adversary can exceed the expectation of this quantity over pp . Advantage exactly 0 for all adversaries is thus equivalent to $\Delta = 0$ —identical distributions—for every attainable pp and every pair, and negligible statistical distance yields negligible advantage for unbounded adversaries. This agreement of the game formulation with the distance formulation is the reason statistical hiding was *defined* through the unbounded-adversary game. $\square$

Definition 4.6 (Binding game and binding). For a commitment scheme Π and adversary $\mathcal{A}$, the game $\text{Bind}_{\Pi}^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger runs $pp \leftarrow \text{Setup}(1^\lambda)$.
2. The adversary $\mathcal{A}(pp)$ outputs a commitment c and two openings (m_0, r_0) and (m_1, r_1) .
3. The game outputs 1 iff $m_0 \neq m_1$ and $\text{Open}(pp, c, m_0, r_0) = \text{Open}(pp, c, m_1, r_1) = 1$.

The *binding advantage* of $\mathcal{A}$ is

$$\text{Adv}_{\Pi}^{\text{bind}}(\mathcal{A}, \lambda) := \Pr[\text{Bind}_{\Pi}^{\mathcal{A}}(\lambda) = 1],$$

a search game in the sense of Definition 1.15. The scheme is *computationally*, *statistically*, or *perfectly binding* according as the advantage is negligible for every PPT adversary, negligible for every unbounded adversary, or exactly 0 for every adversary and every λ .

For a canonically opening scheme and fixed pp , two openings collide, that is, win the binding game, precisely when $\text{Commit}(pp, m_0; r_0) = \text{Commit}(pp, m_1; r_1)$ with $m_0 \neq m_1$. Perfect binding is thus equivalent to the map $c \mapsto m$ being well defined on all attainable commitments, the formulation used in Remark 4.9.

Remark 4.7 (The asymmetry of the quantifiers). Hiding quantifies over what can be *extracted* from a single commitment that the challenger computes and the adversary only observes. Binding quantifies over what can be *manufactured*: an ambiguous commitment of the adversary's choosing, under no obligation to follow Commit. The two properties are in tension: the more commitment values a message can take, the better for hiding and the more opportunities for two messages to share one.

4.4 Incompatibility of perfect hiding and perfect binding

Theorem 4.8 (No scheme is perfectly hiding and perfectly binding). *Let Π be a canonically opening commitment scheme with $|\mathcal{M}_{pp}| \geq 2$. If Π is perfectly hiding, then it is not perfectly binding: indeed, a computationally unbounded adversary wins the binding game with probability 1. Hence no nontrivial commitment scheme is simultaneously perfectly hiding and perfectly binding.*

Proof. Fix any pp in the support of $\text{Setup}(1^\lambda)$ and any two distinct messages $m_0, m_1 \in \mathcal{M}_{pp}$. For a message m , let

$$C_m := \{ \text{Commit}(pp, m; r) : r \in \mathcal{R}_{pp} \}$$

be the set of attainable commitments to m , and let D_m be the distribution of $\text{Commit}(pp, m; r)$ for uniform r . Perfect hiding makes D_{m_0} and D_{m_1} identical (Proposition 4.5); identical distributions have identical supports, so $C_{m_0} = C_{m_1}$. Pick any c in this common support. By definition of the supports there exist $r_0, r_1 \in \mathcal{R}_{pp}$ with

$$\text{Commit}(pp, m_0; r_0) = c = \text{Commit}(pp, m_1; r_1),$$

and then $(c, (m_0, r_0), (m_1, r_1))$ wins the binding game: both openings are valid by canonical opening, and $m_0 \neq m_1$. The double opening *exists* for every attainable pp ; an unbounded adversary *finds* it by enumerating $\mathcal{R}_{pp}$, so it wins Bind with probability 1, giving $\text{Adv}_{\Pi}^{\text{bind}} = 1 \neq 0$. $\square$

Remark 4.9 (The impossibility as a counting fact). Information-theoretically the theorem counts the bits the commitment carries. Perfect binding forces the map $c \mapsto m$ to be well defined, so c determines m and carries at least $\log_2 |\mathcal{M}|$ bits of information about the message. Perfect hiding forces the distribution of c to be independent of m : the commitment carries *zero* bits. The two demands are compatible only when $\log_2 |\mathcal{M}| = 0$, that is, $|\mathcal{M}| = 1$. Every nontrivial scheme therefore makes one property perfect (or statistical) and the other computational. The Pedersen commitment of §4.6 is perfectly hiding and computationally binding; the hash-based commitment of §4.5, read in the random oracle model, is computationally binding (from collision resistance) and computationally hiding.

Remark 4.10 (Two regimes, two failure modes). Which property is left computational determines *how* the scheme fails if its assumption is broken. If binding is only computational, an adversary with enough power—one able to compute discrete logarithms, say—could *retroactively change* a committed value, and the soundness of any protocol built on top degrades to a computational assumption. If hiding is only computational, such an adversary could *retroactively learn* a committed value: privacy degrades the same way, and everlasting secrecy is lost once the assumption is broken, even for commitments published long before. Which failure is worse depends on the application: a ledger that publishes its commitments permanently has reason to care about the second, a protocol whose soundness protects funds in circulation about the first.

4.5 Hash-based commitments

The simplest construction hashes the message together with fresh randomness. It needs two properties from §3: a hash function $H: \{0, 1\}^* \rightarrow \{0, 1\}^n$ is *collision resistant* (Definition 3.7) if no PPT adversary finds $x \neq x'$ with $H(x) = H(x')$ except with negligible probability, and *preimage resistant* (one-way) if, given $H(x)$ for random x of appropriate length, no PPT adversary finds any x' with $H(x') = H(x)$ except with negligible probability.

Construction 4.11 (Hash commitment). Let $H: \{0, 1\}^* \rightarrow \{0, 1\}^n$ be a hash function. The *hash commitment* has message space $\mathcal{M} = \{0, 1\}^*$ and randomness space $\mathcal{R} = \{0, 1\}^\ell$ for a randomness length $\ell = \ell(\lambda)$ (for example $\ell = 2n$). Let $\text{enc}(m, r)$ be an injective, self-delimiting encoding of the pair—for example a canonical length encoding of m , followed by m , followed by the fixed-length string r . Define

$$\text{Commit}(m; r) := H(\text{enc}(m, r)),$$

opened canonically. There are no nontrivial public parameters beyond the choice of H —the scheme lives in the plain model of Remark 4.3.

Proposition 4.12 (Binding of the hash commitment). *If H is collision resistant, the hash commitment is computationally binding.*

Proof. Suppose $\mathcal{A}$ wins the binding game: it outputs c with valid openings (m_0, r_0) , (m_1, r_1) and $m_0 \neq m_1$, so $H(\text{enc}(m_0, r_0)) = c = H(\text{enc}(m_1, r_1))$. Injectivity of enc makes the two hash inputs distinct strings—this is the purpose of the self-delimiting encoding; without it, a boundary shift could present distinct pairs as the same string. The adversary has therefore produced an H -collision, and the reduction that runs $\mathcal{A}$ and outputs the pair $(\text{enc}(m_0, r_0), \text{enc}(m_1, r_1))$ wins the collision game with exactly the binding advantage of $\mathcal{A}$, in essentially the same time. $\square$

Proposition 4.13 (Hiding of the hash commitment, in the ROM). *Model H as a random oracle (Definition 3.13). For messages of length $\text{poly}(\lambda)$ and randomness length $\ell = \omega(\log \lambda)$, the hash commitment is computationally hiding: an adversary making at most q oracle queries has hiding advantage at most $q \cdot 2^{-\ell}$.*

Proof. The challenge commitment is $c = H(\text{enc}(m_b, r))$ for uniform $r \in \{0, 1\}^\ell$. Because the oracle's outputs are independent and uniform, c is a fresh uniform string—carrying no information about b —unless the adversary queries the oracle at the exact point $\text{enc}(m_b, r)$. Before any such query the randomness r is information-theoretically hidden: the adversary's view is independent of it. Each of the q queries therefore hits that point with probability at most $2^{-\ell}$, and the union bound (*Math Guide*, §“The union bound and a birthday calculation”) caps the probability of any hit by $q \cdot 2^{-\ell}$. Conditioned on no hit, the adversary's view is identically distributed in the two worlds $b = 0$ and $b = 1$, so the advantage is at most $q \cdot 2^{-\ell}$, negligible for $\ell = \omega(\log \lambda)$ and polynomial q . $\square$

Remark 4.14 (Why the randomness is essential). The deterministic commitment $c = H(m)$ fails hiding for every low-entropy message space: to distinguish a commitment to m_0 from one to m_1 , the adversary hashes its two candidates and compares. Neither collision resistance nor one-wayness prevents this, since the attack inverts nothing. The salt r contributes ℓ uniformly random bits, so the hash input is unpredictable even when the message is one of two known values. Deterministic encryption of low-entropy plaintexts fails in the same way, with the same remedy.

Remark 4.15 (Strength profile). The hash commitment is *transparent*: it has no parameters and no algebraic structure that a subverted setup could exploit. It is *plausibly post-quantum*: generic quantum collision finding takes $\Theta(2^{n/3})$ queries against an n -bit hash (Remark 2.31), so increasing the output length compensates for the loss. Its weakness is opacity: the commitment is an unstructured bit string, so no algebraic relation on commitments reflects any relation on the committed values; the sum of two hash commitments is not a commitment to the sum of the messages. The Pedersen construction removes this defect at the price of resting on discrete log.

4.6 The Pedersen commitment

The Pedersen commitment works in a prime-order group rather than on bit strings. Let G be a cyclic group of prime order q , written *additively*: for a point $P \in G$ and scalar $a \in \mathbb{F}_q$, the scalar multiple is $[a]P$, with $[0]P = \mathcal{O}$ the identity, and $a \mapsto [a]P$ is a group homomorphism $\mathbb{F}_q \rightarrow G$. Concretely, G is the group of $\mathbb{F}_p$ -points of a prime-order elliptic curve (*Math Guide*, §“The group structure of $E(\mathbb{F}_p)$ ”), and in this volume the Pallas curve of §2.6; the constructions use only that G is isomorphic to $\mathbb{F}_q$ as a group and that discrete logarithms in G are hard. Formally, a *group*

generator $\mathcal{G}$ on input 1^λ outputs $(\mathbb{G}, q, G)$ with q a $\Theta(\lambda)$ -bit prime and G a generator, and the DLog assumption for $\mathcal{G}$ states that no PPT adversary, given $(\mathbb{G}, q, G, [a]G)$ for uniform $a \xleftarrow{\$} \mathbb{F}_q$, outputs a except with negligible probability—Definition 2.2, transcribed additively as in §2.6.

Construction 4.16 (Pedersen commitment). The algorithm $\text{Setup}(1^\lambda)$ runs $(\mathbb{G}, q, G) \leftarrow \mathcal{G}(1^\lambda)$, samples $s \xleftarrow{\$} \mathbb{F}_q^\times$, sets $H := [s]G$, outputs $pp = (\mathbb{G}, q, G, H)$, and discards s . The message and randomness spaces are $\mathcal{M} = \mathcal{R} = \mathbb{F}_q$, and

$$\text{Commit}(pp, m; r) := [m]G + [r]H,$$

opened canonically.

The parameters are two generators G and H whose relative discrete logarithm $s = \log_G H$ is unknown to *all* parties, the committer included. Knowledge of s breaks binding (Remark 4.19); perfect hiding holds whether or not s is known. In deployment H is hashed to the curve (Construction 3.24), so nobody ever *holds* s and the setup is transparent (Remark 4.3).

Theorem 4.17 (Perfect hiding). *The Pedersen commitment is perfectly hiding.*

Proof. Fix any attainable $pp = (\mathbb{G}, q, G, H)$; since $s \in \mathbb{F}_q^\times$, the point H is not $\mathcal{O}$. Because $\mathbb{G}$ has prime order, the nonzero H generates $\mathbb{G}$, so $r \mapsto [r]H$ is a bijection $\mathbb{F}_q \rightarrow \mathbb{G}$. For uniform r , therefore, $[r]H$ is uniform on $\mathbb{G}$ —and so is its translate $[m]G + [r]H$, for *every* message m : adding the fixed element $[m]G$ permutes the group and leaves the uniform distribution invariant. The commitment distribution is uniform on $\mathbb{G}$ regardless of m , so for any m_0, m_1 the two distributions coincide exactly, and every adversary's advantage is 0 by Proposition 4.5. $\square$

Theorem 4.18 (Computational binding under DLog). *If the DLog assumption holds for $\mathcal{G}$, the Pedersen commitment is computationally binding. Concretely, from any adversary $\mathcal{A}$ that wins the binding game with probability ε one constructs $\mathcal{B}$ computing discrete logarithms with probability $(1 - 1/q) \varepsilon \geq \varepsilon - 1/q$, in essentially the same running time.*

Proof. The algebraic relation comes first. A double opening gives $[m_0]G + [r_0]H = [m_1]G + [r_1]H$ with $m_0 \neq m_1$, so

$$[m_0 - m_1]G = [r_1 - r_0]H.$$

The case $r_1 = r_0$ is impossible: it would force $[m_0 - m_1]G = \mathcal{O}$, and since G has order q this means $m_0 \equiv m_1 \pmod{q}$, contradicting $m_0 \neq m_1$ in $\mathbb{F}_q$. With $r_1 - r_0$ invertible modulo the prime q , the relation solves for the hidden discrete logarithm:

$$s = \log_G H = (m_0 - m_1)(r_1 - r_0)^{-1} \pmod{q}.$$

A double opening therefore determines the discrete log of H .

The reduction is as follows. On a DLog challenge $(\mathbb{G}, q, G, Q)$, the algorithm $\mathcal{B}$ sets $H := Q$, passes $pp = (\mathbb{G}, q, G, Q)$ to $\mathcal{A}$, and, when $\mathcal{A}$ returns a double opening, outputs $(m_0 - m_1)(r_1 - r_0)^{-1} \pmod{q} = \log_G Q$. One difference of distributions must be accounted for: in the real scheme $H = [s]G$ with $s \xleftarrow{\$} \mathbb{F}_q^\times$, so H is uniform on $\mathbb{G} \setminus \{\mathcal{O}\}$, whereas the challenge $Q = [a]G$ has a

uniform on all of $\mathbb{F}_q$, so Q is uniform on all of $\mathbb{G}$. The two distributions differ only on the event $Q = \mathcal{O}$, of probability $1/q$ —their statistical distance is exactly $1/q$ —and on that event $\mathcal{A}$ cannot win in any case (with $H = \mathcal{O}$ the relation above forces $m_0 = m_1$). Conditioned on $Q \neq \mathcal{O}$, the simulation is perfect, so

$$\text{Adv}^{\text{dlog}}(\mathcal{B}) = \left(1 - \frac{1}{q}\right) \text{Adv}_{\Pi}^{\text{bind}}(\mathcal{A}) \geq \varepsilon - \frac{1}{q},$$

and $\mathcal{B}$'s overhead over $\mathcal{A}$ is a constant number of field operations. $\square$

Remark 4.19 (Equivocation with the trapdoor). Any party who knows $s = \log_G H$ can open a Pedersen commitment to *any* message. Given $c = [m]G + [r]H$ and a target m' , set

$$r' := r + (m - m') s^{-1} \bmod q;$$

then $[m']G + [r']H = [m']G + [r]H + [(m - m')]G = c$. This trapdoor makes discarding s in Setup mandatory (Remark 4.3), and zero-knowledge simulators use the equivocation to open commitments consistently with a transcript produced out of order.

Theorem 4.20 (Additive homomorphism). *For all $m_0, m_1, r_0, r_1 \in \mathbb{F}_q$,*

$$\text{Commit}(m_0; r_0) + \text{Commit}(m_1; r_1) = \text{Commit}(m_0 + m_1; r_0 + r_1),$$

and more generally, for all scalars $\alpha, \beta \in \mathbb{F}_q$,

$$[\alpha] \text{Commit}(m_0; r_0) + [\beta] \text{Commit}(m_1; r_1) = \text{Commit}(\alpha m_0 + \beta m_1; \alpha r_0 + \beta r_1).$$

Proof. Expand and regroup, using commutativity of $+$ in $\mathbb{G}$, the identity $[a]P + [b]P = [a + b]P$, and the linearity $[\alpha]([m]G + [r]H) = [\alpha m]G + [\alpha r]H$ of scalar multiplication:

$$\begin{aligned} [\alpha]([m_0]G + [r_0]H) + [\beta]([m_1]G + [r_1]H) &= [\alpha m_0]G + [\beta m_1]G + [\alpha r_0]H + [\beta r_1]H \\ &= [\alpha m_0 + \beta m_1]G + [\alpha r_0 + \beta r_1]H. \end{aligned} \quad \square$$

Remark 4.21 (Value balancing: the homomorphism deployed). Zcash uses the homomorphism for value balancing. Each transaction value v_i is committed as $C_i = [v_i]G + [r_i]H$; a transaction balances iff the sum of its input values equals the sum of its output values. By Theorem 4.20, the verifier can check this *without learning any v_i* : the value components cancel in $\sum C_{\text{in}} - \sum C_{\text{out}}$ precisely when the value sums agree *modulo the group order q* , in which case the difference equals $[r]H$ for the net randomness r —a commitment to 0. Cancellation alone certifies balance only modulo q , and the deployed protocol closes the gap with range checks carried inside the proofs: the values of the notes (the shielded value records) spent and created by each Action (Orchard's combined spend-and-output unit) are proved to lie in $\{0, \dots, 2^{64} - 1\}$ (the Action statement, protocol specification § 4.18.4), the committed net value is the difference of two such values, and the number of summands is bounded, so the sums cannot wrap (§ 4.14). The prover does not reveal r ; it proves *knowledge* of r by using $[r]H$ as the verification key of a signature over the transaction—the *binding signature*, treated with the completed balance argument in § 8.9. In deployed Orchard

the instantiation is $\text{ValueCommit}(v; \text{rcv}) = [v]V + [\text{rcv}]R$ with hashed-to-curve generators V, R (protocol specification § 5.4.8.3); each Action publishes a commitment to its *net* value change, and the verifier folds the published commitments with a commitment to the transparent value balance to obtain the binding verification key. The homomorphism turns an arithmetic relation on hidden values into a group equation on public commitments, a pattern used throughout the rest of the volume.

4.7 Pedersen vector commitments

A single group element can commit to a vector of scalars rather than a single scalar.

Construction 4.22 (Pedersen vector commitment). On input 1^λ and a length $n = \text{poly}(\lambda)$, Setup outputs (G, q) together with generators $G_1, \dots, G_n, H \xleftarrow{\$} G \setminus \{\mathcal{O}\}$, sampled so that no nontrivial discrete-log relation among them is known to any party—in practice each generator is derived from a public seed by hashing into the curve, rejecting the identity if it occurs, a nothing-up-my-sleeve procedure (Construction 3.24) that makes the setup transparent. The message space is $\mathbb{F}_q^n$, the randomness space $\mathbb{F}_q$, and for $\mathbf{m} = (m_1, \dots, m_n)$,

$$\text{Commit}(pp, \mathbf{m}; r) := \sum_{i=1}^n [m_i]G_i + [r]H = \langle \mathbf{m}, \mathbf{G} \rangle + [r]H,$$

where $\langle \mathbf{m}, \mathbf{G} \rangle := \sum_i [m_i]G_i$ is the multiscalar (mixed) inner product of the message vector with the generator vector $\mathbf{G} = (G_1, \dots, G_n)$. Opening is canonical.

Theorem 4.23 (Properties of the vector commitment). *The Pedersen vector commitment is perfectly hiding, and computationally binding under the DLog assumption in G .*

Proof. Hiding. Exactly as in Theorem 4.17: for uniform r the blinding term $[r]H$ is uniform on G , so the commitment is uniform on G regardless of the message vector.

Binding. A double opening with $\mathbf{m} \neq \mathbf{m}'$ gives $\langle \mathbf{m}, \mathbf{G} \rangle + [r]H = \langle \mathbf{m}', \mathbf{G} \rangle + [r']H$, that is, a *nontrivial discrete-log relation*

$$\sum_{i=1}^n [m_i - m'_i] G_i + [r - r'] H = \mathcal{O}$$

with not all coefficients zero. The reduction $\mathcal{B}$ must extract a single logarithm from a relation among $n + 1$ generators, and it does so by embedding its challenge in *every* generator. On challenge $Q = [s]G$, it samples $u_i, w_i \xleftarrow{\$} \mathbb{F}_q$ for each i and $u_H, w_H \xleftarrow{\$} \mathbb{F}_q$, sets

$$G_i := [u_i]G + [w_i]Q, \quad H := [u_H]G + [w_H]Q,$$

and resamples any pair whose generator lands on $\mathcal{O}$. Each result is uniform on $G \setminus \{\mathcal{O}\}$ —the $[u_i]G$ term alone makes the pre-rejection point uniform on G —so the simulated parameters follow the real distribution. Moreover, the parameters reveal nothing about the w -coordinates: for every candidate value of w_i there is exactly one u_i consistent with the observed G_i , so conditioned

on the adversary's entire view the w_i and w_H remain independent and uniform. Write the recovered relation as $\sum_i [a_i]G_i + [a_H]H = \mathcal{O}$ with coefficient vector $(a_1, \dots, a_n, a_H) \neq \mathbf{0}$. Substituting the embeddings and collecting the G - and Q -components gives

$$c_0 + c_1 s \equiv 0 \pmod{q}, \quad c_0 = \sum_i a_i u_i + a_H u_H, \quad c_1 = \sum_i a_i w_i + a_H w_H.$$

Since the coefficient vector is nonzero and the w -coordinates are uniform and independent of it, c_1 is uniform on $\mathbb{F}_q$, hence nonzero with probability $1 - 1/q$; in that case $s = -c_0 c_1^{-1} \pmod{q}$, and $\mathcal{B}$ outputs the challenge logarithm. Altogether $\text{Adv}^{\text{dlog}}(\mathcal{B}) \geq (1 - 1/q) \text{Adv}^{\text{bind}}(\mathcal{A})$. $\square$

Remark 4.24 (Compression and the cardinality trade-off). The vector commitment is *compressing*: n field elements (plus the randomness) map to a single group element, exponentially shorter than the message for large n . Compression is incompatible with statistical binding for a counting reason: there are q^n message vectors and only q commitment values, so many colliding openings exist. Binding is therefore only computational—an unbounded adversary finds a colliding opening by solving the underlying discrete-log relation—and the trade-off of Theorem 4.8 appears at the level of cardinalities: a compressing commitment *cannot* determine its message, so perfect hiding costs it nothing further.

Remark 4.25 (Linearity of the vector commitment). The vector commitment remains additively homomorphic in both message and randomness, componentwise: $\text{Commit}(\mathbf{m}; r) + \text{Commit}(\mathbf{m}'; r') = \text{Commit}(\mathbf{m} + \mathbf{m}'; r + r')$, by the same regrouping as Theorem 4.20. This linearity, applied to a *committed* vector against *public* coefficient vectors, underlies the inner-product arguments of §4.9, which prove statements about $\langle \mathbf{m}, \mathbf{b} \rangle$ for public $\mathbf{b}$ while revealing only a single committed group element.

4.8 Sinsemilla: an algebraic hash-based commitment

Between the opaque hash commitment of §4.5 and the linear Pedersen commitment lie the *algebraic hash functions*: collision-resistant hashes whose internal operations are elliptic-curve group operations. A zero-knowledge circuit verifying a hash pays for every internal operation, and curve additions in the field over which the proof system works are cheap, while collision resistance still reduces to the discrete logarithm. Sinsemilla, deployed in Zcash Orchard, is a Pedersen-style hash of this kind over a sequence of message chunks, constructed below from the Pedersen hash.

Construction 4.26 (Pedersen hash). Split the message M into k chunks $m_1, \dots, m_k$, each interpreted as a scalar in a bounded range, and fix k independent generators $P_1, \dots, P_k \in \mathbb{G}$ with unknown mutual discrete logarithms. Define

$$\text{PedersenHash}(M) := \sum_{j=1}^k [m_j]P_j \in \mathbb{G}.$$

This is the unblinded Pedersen vector commitment map used as a hash. The output is a group element; composing with a fixed encoding of group elements as bit strings yields an ordinary hash.

Proposition 4.27 (Collision resistance of the Pedersen hash). *Suppose no efficient algorithm finds a nontrivial relation $\sum_j [a_j]P_j = \mathcal{O}$ with not all $a_j = 0$ and each $|a_j|$ within the allowed chunk range—hardness that follows from DLog when the P_j are independent uniform generators. Then PedersenHash is collision resistant.*

Proof. A collision $M \neq M'$, with chunk decompositions (m_j) and (m'_j) , gives $\sum_j [m_j - m'_j]P_j = \mathcal{O}$ with some coefficient $m_j - m'_j \neq 0$ —a nontrivial relation with coefficients in the allowed range. The reduction from relation-finding to DLog is the every-generator embedding of Theorem 4.23: set each $P_j = [u_j]G + [w_j]Q$ for the challenge Q , and read the unknown logarithm off the recovered relation. $\square$

The Pedersen hash needs one independent generator per chunk, so its generator table grows linearly with the maximum message length. Sinsemilla iterates a two-operand step over a small fixed generator table shared by all chunk positions, and keeps the discrete-log collision reduction.

Construction 4.28 (Sinsemilla hash). Split the message M into pieces $m_1, \dots, m_k$, each of fixed width $w \leq 32$ bits, and identify a piece $v \in \{0, 1\}^w$ with its integer value, bits read little-endian (the specification’s LEBS2IP). The public parameters are a base point $Q \in \mathbb{G}$ and a table $S: \{0, 1\}^w \rightarrow \mathbb{G}$ assigning to each possible piece value its own generator, both obtained by hashing to the curve. Write $\text{GroupHash}(D, M)$ for the hash-to-curve pipeline of Construction 3.22 applied to the byte string M under the domain-separation tag built from the byte string D —for Pallas the RFC 9380 suite tag $D \parallel \text{"-pallas_XMD:BLAKE2b_SSWU_R0_"} of §3.6 (protocol specification § 5.4.9.8)—and write $\text{LE}_{32}(j)$ for the four-byte little-endian encoding of an integer $0 \leq j < 2^{32}$. The table is$

$$S(v) := \text{GroupHash}(\text{"z.cash:SinsemillaS"}, \text{LE}_{32}(v)), \quad v \in \{0, 1\}^w,$$

with 2^w entries shared by every instance, and the base point is personalised (§3.5) to the enclosing use—a byte string D naming it, such as a note-commitment or Merkle-hash domain—by taking that string as the message:

$$Q := \text{GroupHash}(\text{"z.cash:SinsemillaQ"}, D).$$

Both derivations are nothing-up-my-sleeve (Construction 3.24), so no party knows any discrete-log relation among $\{S(v)\}_v \cup \{Q\}$ (Remark 3.25). The deployed instantiation takes $w = 10$ (the specification’s k) over Pallas, derives Q and the table as above with LE_{32} as the index encoding, and loads the table into the circuit as a lookup (protocol specification § 5.4.1.9). The hash iterates over the pieces with a running accumulator $\text{Acc} \in \mathbb{G}$:

$$\text{Acc}_0 := Q, \quad \text{Acc}_j := \text{Acc}_{j-1} + (\text{Acc}_{j-1} + S(m_j)) = [2]\text{Acc}_{j-1} + S(m_j),$$

and outputs the point $\text{SinsemillaHashToPoint}(M) := \text{Acc}_k$. The step costs two *incomplete* curve additions—first $\text{Acc}_{j-1} + S(m_j)$, then the result plus Acc_{j-1} ; no doubling is ever performed, and the addition formula is undefined when its operands are equal or are mutual negatives

(*Math Guide*, §“Elliptic curves”)—and this pair of additions is the operation the circuit verifies in few constraints. Unrolling the recurrence gives the closed form

$$\text{SinsemillaHashToPoint}(\mathcal{M}) = \sum_{j=1}^k [2^{k-j}]S(m_j) + [2^k]Q :$$

the integer coefficients are powers of two determined by position alone, and the message enters only through *which* generator $S(m_j)$ each piece selects.

Proposition 4.29 (Collision resistance of Sinsemilla). *Model the table entries $S(v)$ and the base point Q as independent uniform generators (the nothing-up-my-sleeve hash-to-curve derivation covers Q as well as the table). Then any collision in $\text{SinsemillaHashToPoint}$, on inputs for which its incomplete additions are defined, yields a nontrivial discrete-log relation among $\{S(v)\}_v \cup \{Q\}$, and any input on which an incomplete addition is undefined yields one directly; either way a discrete logarithm follows via the every-generator embedding of Theorem 4.23 extended to Q . The point hash is thus collision resistant under the DLog assumption in $\mathbb{G}$, the exceptional cases of incomplete addition covered by the same reduction rather than excluded.*

Proof sketch. Let $M \neq M'$ collide. Suppose first that they have the same piece count k . Subtracting the unrolled forms cancels the $[2^k]Q$ terms and leaves

$$\sum_{j=1}^k [2^{k-j}](S(m_j) - S(m'_j)) = \mathcal{O}.$$

Regroup over the distinct table generators: for each value $v \in \{0, 1\}^w$, the net coefficient on $S(v)$ is

$$c_v = \sum_{j: m_j=v} 2^{k-j} - \sum_{j: m'_j=v} 2^{k-j},$$

and the relation reads $\sum_v [c_v] S(v) = \mathcal{O}$. Each c_v is a difference of two subset-sums of the distinct powers $2^{k-1}, \dots, 2^0$, hence an integer with $|c_v| < 2^k$. For *admissible* message lengths—those with 2^k below the group order q , a finite design-time check—such a coefficient vanishes modulo q only if it vanishes as an integer; and $c_v = 0$ forces, by uniqueness of binary expansions, the two subset-sums to be equal as sets of powers, that is, the positions carrying value v in M and in M' to coincide. Coincidence for every v would force $M = M'$; since $M \neq M'$, some $c_v \neq 0$ and the relation is nontrivial.

If instead the piece counts differ, say $k > k'$, the Q -constants do not cancel: the relation acquires the coefficient $2^k - 2^{k'}$ on Q , nonzero and below the group order for admissible lengths, so the relation is again nontrivial. In either case, embedding a DLog challenge in every table generator *and* in Q recovers the challenge logarithm exactly as in Proposition 4.27.

It remains to treat the exceptional cases. An incomplete addition is undefined only when its operands are equal or are mutual negatives, so an input reaching one presents a coincidence $[\alpha]\text{Acc}_{j-1} + S(m_j) = \mathcal{O}$ with $\alpha \in \{-1, 1, 2\}$. Substituting the unrolled form of Acc_{j-1} makes this an explicit relation among Q and the table generators, with coefficient $\alpha 2^{j-1} \neq 0$ on Q and all magnitudes at most 2^k —below the group order for admissible lengths. An input that

merely triggers an exceptional case therefore yields a nontrivial relation directly, and the same embedding extracts the challenge logarithm from it; the protocol specification proves exactly this for the deployed instantiation (§ 5.4.1.9). $\square$

The deployed hash differs from the point hash in two ways. First, the deployed `SinsemillaHash` applies the Pallas coordinate extractor to `SinsemillaHashToPoint( $\mathcal{M}$ )`, returning its x -coordinate. Collision resistance survives the extraction: equal x -coordinates mean the two points are equal or negatives of one another, and either signed equality yields the same kind of nontrivial generator relation as in the proof above. Second, the message bits are zero-padded to a multiple of the piece width before chunking, so two bit strings of different lengths within the same final piece present identical piece sequences, and the deployed hash claims collision resistance only *between inputs of a fixed length for a given personalisation*, exactly as the protocol specification § 5.4.1.9 requires (“between inputs of fixed length, for a given personalization input”). Each personalisation domain hashes inputs of a single length, as the Merkle-tree hash `MerkleCRH` of §10 illustrates. The variable-length argument in the proof above applies to the piece-aligned point presentation, where the piece sequence is the message.

The commitment is obtained from the hash by adding the Pedersen blinding term unchanged.

Construction 4.30 (`SinsemillaCommit`). Fix one further independent generator H' , hashed to the curve with its own domain string, so that no discrete-log relation between H' and $\{S(v)\}_v \cup \{Q\}$ is known. Define, for $r \xleftarrow{\$} \mathbb{F}_q$,

$$\text{SinsemillaCommit}(m; r) := \text{SinsemillaHashToPoint}(m) + [r]H'.$$

Hiding is inherited unchanged from Pedersen: by the argument of Theorem 4.17, $[r]H'$ is uniform on $\mathbb{G}$ for uniform r , so the commitment distribution is independent of m , and the scheme is perfectly hiding. Binding rests on discrete log: two valid openings to distinct messages yield either a `SinsemillaHashToPoint` collision—excluded by Proposition 4.29—or a nontrivial discrete-log relation between H' and the hash generators.

Orchard’s note commitment `NoteCommit` instantiates this map (protocol specification § 5.4.8.4). Its key commitment `CommitLvk` composes the same map with x -coordinate extraction—the specification’s `SinsemillaShortCommit`—and returns a base-field element.

Remark 4.31 (In-circuit cost of `Sinsemilla`). Outside a circuit ordinary hashes are faster than `Sinsemilla`; its advantage is verification cost *inside* one. In a SNARK (§9) the prover re-expresses every computational step as polynomial constraints; a bit-oriented hash such as SHA-256, built from Boolean operations and modular additions foreign to the proof field, costs tens of thousands of constraints per evaluation, whereas `Sinsemilla`’s per-piece cost is a handful of curve operations in the field over which the proof system works. Orchard uses it for note commitments and Merkle trees: it is cheap to prove and rests on the same discrete-log hardness as the rest of the protocol. A later volume treats the concrete Orchard instantiation: its per-use personalisation strings and the in-circuit handling of incomplete addition.

4.9 Polynomial commitment schemes

A polynomial commitment scheme commits to a polynomial and supports proofs of statements about its values; the succinct argument systems of §9 are built on it.

Definition 4.32 (Polynomial commitment scheme). Fix a field $\mathbb{F}$ and a degree bound d . A *polynomial commitment scheme* (PCS) is a tuple (Setup, Commit, Open, Eval, Verify):

- The algorithm $\text{Setup}(1^\lambda, d)$ outputs public parameters pp supporting degree at most d ;
- the algorithm $\text{Commit}(pp, f; r)$ outputs a commitment C to a polynomial $f \in \mathbb{F}[X]_{\leq d}$, optionally with hiding randomness r ;
- the algorithm $\text{Open}(pp, C, f, r)$ outputs a bit, verifying that C commits to the *whole* polynomial f ;
- the algorithm $\text{Eval}(pp, f, r, z)$, for an evaluation point $z \in \mathbb{F}$, outputs a claimed value $y = f(z)$ together with an *evaluation proof* π ;
- the algorithm $\text{Verify}(pp, C, z, y, \pi)$ outputs a bit, checking the evaluation proof against the commitment.

Beyond the hiding and binding of the underlying commitment (Definitions 4.4 and 4.6), a PCS must satisfy *evaluation binding*: no PPT prover can produce a commitment C , a point z , and two accepting proofs for distinct claimed values $y \neq y'$ at z , except with negligible probability. Stronger schemes additionally satisfy *extractability* (proof of knowledge): from any successful prover one can extract an actual polynomial f of degree at most d consistent with all the evaluations it gets accepted.

Remark 4.33 (Rigidity, and why evaluation binding is separate). A single short commitment can support queries at arbitrary points because polynomials are *rigid*: a nonzero polynomial of degree at most d has at most d roots (*Math Guide*, §“The Schwartz–Zippel lemma”), so two distinct polynomials of degree at most d agree on at most d points. A prover that answers a different value at even a single point behaves as one holding a *different polynomial*. However, an accepting evaluation proof is *not* an opening of C , so evaluation binding does *not* follow from the binding of Commit; each construction below proves it from its own assumption—a strong Diffie–Hellman assumption for KZG, discrete log via a folding extractor for the inner-product scheme. The design problem of a PCS is to make the evaluation proof π short and fast to verify; committing to the coefficient list with an arbitrary commitment scheme does not solve it.

KZG: evaluation in the exponent. The pairing-based scheme of Kate, Zaverucha, and Goldberg achieves constant-size proofs by encoding a secret evaluation point in the parameters.

Construction 4.34 (KZG commitment). Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be groups of prime order $|\mathbb{F}|$ equipped with a nondegenerate bilinear pairing $e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ (Example 2.9), with generators $G \in \mathbb{G}_1$ and $\hat{G} \in \mathbb{G}_2$. The algorithm Setup samples a secret $\tau \xleftarrow{\$} \mathbb{F}$, publishes the

structured reference string

$$pp = ([\tau^0]G, [\tau^1]G, \dots, [\tau^d]G, \hat{G}, [\tau]\hat{G}),$$

and discards τ . The commitment to $f = \sum_{i=0}^d a_i X^i$ is evaluation in the exponent:

$$\text{Commit}(pp, f) := \sum_{i=0}^d [a_i]([\tau^i]G) = [f(\tau)]G,$$

computable from pp by anyone, without knowledge of τ .

Construction 4.35 (KZG evaluation proof). To prove $f(z) = y$, the prover uses the factor theorem: z is a root of $f(X) - y$, so the quotient

$$q(X) := \frac{f(X) - y}{X - z} \in \mathbb{F}[X]_{\leq d-1}$$

is a genuine polynomial *precisely because* $f(z) - y = 0$. The proof is the commitment to the quotient, $\pi := [q(\tau)]G$, computed from the reference string. The verifier checks the single pairing equation

$$e(C - [y]G, \hat{G}) \stackrel{?}{=} e(\pi, [\tau]\hat{G} - [z]\hat{G}),$$

which, by bilinearity, is exactly the in-the-exponent form of the polynomial identity $f(\tau) - y = q(\tau)(\tau - z)$.

Proposition 4.36 (KZG evaluation binding, sketched). *Evaluation binding of KZG rests on the d -strong Diffie–Hellman assumption in the bilinear group: given the powers $[\tau^i]G$, it is infeasible to produce $[(\tau - z)^{-1}]G$ for any z of one’s choosing.*

Proof idea. Two accepting proofs π, π' for distinct values $y \neq y'$ at the same point z satisfy the two pairing equations; subtracting them (in the exponent) gives $\pi - \pi' = [(y' - y)(\tau - z)^{-1}]G$, so the forger’s output yields

$$[(\tau - z)^{-1}]G = [(y' - y)^{-1}](\pi - \pi'),$$

exactly the element the assumption declares infeasible. $\square$

Remark 4.37 (Costs and benefits of KZG). The KZG scheme provides commitments and evaluation proofs of *constant* size (one group element each) and *constant* verification time (two pairings), all independent of the degree d . Its setup is trusted: the secret τ is a global trapdoor. Anyone who learns τ can forge accepting evaluation proofs for *false* statements—for any commitment C , any point z , and any desired value y , the element $\pi := [(\tau - z)^{-1}](C - [y]G)$ passes the pairing check, no knowledge of any polynomial required. This is the silent loss of binding under a subverted setup announced in Remark 4.3; evaluation binding survives only if τ is provably destroyed, in practice by a multi-party *powers-of-tau* ceremony whose trapdoor stays secret as long as at least one participant is honest. The construction also requires pairing-friendly curves, a more constrained and less efficient setting than the ordinary curves of §2.6. Binding is computational either way, resting on the bilinear assumption—consistent with Theorem 4.8.

The inner-product alternative. The transparent alternative commits to the coefficient vector with the Pedersen vector commitment and replaces the pairing check by a recursive argument.

Construction 4.38 (IPA-style polynomial commitment). Let $\mathbf{G} = (G_0, \dots, G_d)$ and H be transparent generators with unknown mutual discrete logs, derived from a public seed (Construction 4.22). Commit to $f = \sum_{i=0}^d a_i X^i$ by the Pedersen vector commitment of its coefficient vector $\mathbf{a} = (a_0, \dots, a_d)$:

$$\text{Commit}(pp, f; r) := \sum_{i=0}^d [a_i] G_i + [r] H = \langle \mathbf{a}, \mathbf{G} \rangle + [r] H.$$

The construction uses no pairings and no trusted setup.

Evaluation is an inner product: writing $\mathbf{z} = (1, z, z^2, \dots, z^d)$ for the public vector of powers of the query point,

$$f(z) = \sum_{i=0}^d a_i z^i = \langle \mathbf{a}, \mathbf{z} \rangle,$$

so proving an evaluation reduces to proving an inner-product relation about a committed vector against a public vector. The *inner-product argument* (IPA) proves it recursively: each round folds the two halves of the coefficient vector—and of the generator vector—into vectors of half the dimension by taking a random linear combination dictated by a verifier challenge, sending two group elements per round (the cross-term commitments L_j and R_j) to account for the mixed terms; after $\log_2(d+1)$ rounds a single scalar remains, and the verifier checks one final equation tying it to the folded commitment and the folded generators.

Proposition 4.39 (Properties of the IPA-based PCS). *The inner-product polynomial commitment has: transparent setup (public-coin parameters, no trapdoor); evaluation proofs of $O(\log d)$ group elements; and $O(d)$ -group-operation verification, batchable across proofs. Its binding and evaluation binding reduce to the discrete logarithm assumption in $\mathbb{G}$.*

Proof idea. Binding of the commitment itself is Theorem 4.23. For evaluation binding and extractability, one shows the folding protocol is *tree special-sound*: from a suitable tree of accepting transcripts, branching over the verifier’s challenge at each round, an extractor works back up the recursion and computes a coefficient vector $\mathbf{a}$ consistent with the commitment and the claimed inner product—a notion this sketch uses only informally and §9 defines precisely. Two distinct extracted openings of the same commitment would yield a nontrivial discrete-log relation among $\mathbf{G}, H$, contradicting DLog by the every-generator embedding. The $O(\log d)$ proof size is the halving recursion; the $O(d)$ verification is the final multiscalar multiplication over the unfolded generators. $\square$

Remark 4.40 (KZG versus IPA, and the Halo lineage). KZG gives constant proof size and constant verification, at the price of a structured trusted setup and pairing-friendly curves; the IPA gives transparent setup over a plain curve and rests on nothing beyond discrete log, at the price of logarithmic proofs and linear-time verification. The Halo 2 proof system deployed in Zcash

Orchard builds on the inner-product family to avoid trusted setup, and recovers practical verification cost through *batching*: the deployed verifier folds the final multiscalar multiplications of many proofs into one shared multiexponentiation—each proof’s deferred check is scaled by a fresh random factor and accumulated, and a single combined check is evaluated once. This amortises group operations across a batch, though per-proof verifier work remains linear in d . The Halo lineage’s accumulation technique extends the same deferral across *recursively composed* proofs, one proof attesting to another’s deferred check—the capability the Pasta cycle of Remark 2.26 was designed to serve—but deployed Zcash does not use recursion: each Orchard proof is verified (in batch) directly.

Remark 4.41 (Hiding variants). As stated, the KZG commitment is binding but not hiding: it is a deterministic function of f , so committing twice to the same polynomial is visible, and a low-entropy polynomial can be found by search—the failure of Remark 4.14. That is acceptable when the polynomial encodes only public data. When it encodes secrets, as the witness polynomials of a zero-knowledge proof do, randomness is required: KZG is augmented with a blinding summand $[r][\gamma]G$ using an extra setup element $[\gamma]G$, while the IPA commitment of Construction 4.38 already carries its $[r]H$ and is perfectly hiding by Theorem 4.23. The resulting regime is the perfect-hiding/computational-binding profile of Pedersen (Theorems 4.17 and 4.18), lifted from field elements to polynomials.

From commitments to arguments. A polynomial commitment scheme turns an information-theoretic proof object—a polynomial identity holding over a large field—into a succinct cryptographic argument. The prover commits to its witness polynomials, the verifier challenges at random points, and evaluation proofs show that the committed polynomials satisfy the required identities without revealing them. Soundness comes from Schwartz–Zippel (*Math Guide*, §“The Schwartz–Zippel lemma”): a nonzero polynomial of low degree is almost surely nonzero at a random point, so an identity that holds at the challenge almost surely holds identically. Binding and succinctness come from the commitment: the prover is bound to its polynomials *before* seeing the challenge, and the verifier handles only short commitments and proofs. The full compilation, and the precise meaning of “argument of knowledge”, are the subject of §9.

4.10 Secret sharing

Verifiable secret sharing is constructed here because the threshold signatures of a later volume depend on it and no earlier volume provides it. The setting is a prime field $\mathbb{F}_q$ —in the sequel the scalar field of the group G of §4.6—and the tool is Lagrange interpolation (*Math Guide*, §“Lagrange interpolation”): a polynomial of degree at most $t - 1$ over $\mathbb{F}_q$ is determined by its values at any t distinct points, and any t prescribed values at distinct points are attained by exactly one such polynomial.

Construction 4.42 (Shamir t -of- n secret sharing). Fix a threshold t and a party count n with $1 \leq t \leq n < q$. To share a secret $s \in \mathbb{F}_q$, the dealer samples coefficients $a_1, \dots, a_{t-1} \xleftarrow{\$} \mathbb{F}_q$ independently and uniformly and forms

$$f(X) := s + a_1X + a_2X^2 + \dots + a_{t-1}X^{t-1} \in \mathbb{F}_q[X], \quad f(0) = s,$$

a polynomial of degree at most $t - 1$ with constant term the secret. Party $i \in \{1, \dots, n\}$ receives the *share* $s_i := f(i)$, sent privately; the identifiers $1, \dots, n$ are distinct nonzero elements of $\mathbb{F}_q$ because $n < q$. *Reconstruction* from the shares of any set I of t parties is interpolation at 0:

$$s = f(0) = \sum_{i \in I} \lambda_{I,i} s_i, \quad \lambda_{I,i} := \prod_{j \in I, j \neq i} \frac{j}{j - i},$$

where $\lambda_{I,i} = \ell_i(0)$ is the Lagrange basis polynomial for the nodes I evaluated at 0, computable from the identifiers alone.

Correctness of reconstruction is the interpolation theorem: the t points $(i, s_i)_{i \in I}$ determine a unique polynomial of degree at most $t - 1$, which is f , and the Lagrange formula $f(X) = \sum_{i \in I} s_i \ell_i(X)$ evaluated at $X = 0$ gives the displayed sum. The coalition may be chosen after the shares are dealt, since the weights depend only on I .

Theorem 4.43 (Perfect secrecy below the threshold). *Let I be any set of at most $t - 1$ identifiers. For every secret $s \in \mathbb{F}_q$, the shares $(s_i)_{i \in I}$ of Construction 4.42 are jointly uniform on $\mathbb{F}_q^{|I|}$. In particular their distribution does not depend on s : fewer than t shares are independent of the secret, and an adversary of unbounded power holding them has advantage exactly 0 at telling any two candidate secrets apart.*

Proof. It suffices to treat $|I| = t - 1$, since fewer shares are a projection of these. Fix s and I , and consider the map

$$\Sigma_s: \mathbb{F}_q^{t-1} \rightarrow \mathbb{F}_q^{t-1}, \quad (a_1, \dots, a_{t-1}) \mapsto (f(i))_{i \in I},$$

from the dealer's coefficient vector to the coalition's shares. It is injective: two coefficient vectors with the same shares give two polynomials of degree at most $t - 1$ that agree at the t distinct points 0 (where both equal s) and I , hence are equal by uniqueness of interpolation, so the coefficient vectors coincide. It is surjective: for any target $(y_i)_{i \in I}$, Lagrange interpolation through the t points $(0, s)$ and $(i, y_i)_{i \in I}$ produces a polynomial of degree at most $t - 1$ with constant term s , whose remaining coefficients are a preimage. So Σ_s is a bijection—for every s —and it carries the uniform distribution on coefficient vectors to the uniform distribution on $\mathbb{F}_q^{t-1}$. The share vector is therefore uniform whatever the secret, and for any $s_0 \neq s_1$ the two share distributions coincide exactly; by the variational characterisation of statistical distance (Proposition 1.7) no procedure, efficient or not, distinguishes them. $\square$

The theorem is the perfect hiding of a commitment, in the form of Proposition 4.5. Shamir sharing is not binding: a dishonest dealer can distribute values that lie on no common polynomial of degree at most $t - 1$, so that different coalitions of t parties reconstruct different “secrets”, and a party receiving a share cannot detect this. Verifiable secret sharing has the dealer commit to the polynomial.

Construction 4.44 (Feldman verifiable secret sharing). Let $(\mathbb{G}, q, G)$ be as in §4.6, so that $\mathbb{F}_q$ is the scalar field of $\mathbb{G}$. The dealer shares s as in Construction 4.42, with $f(X) = \sum_{k=0}^{t-1} a_k X^k$ and $a_0 = s$, and additionally *broadcasts* the coefficient commitments

$$\Phi_k := [a_k] G \in \mathbb{G}, \quad k = 0, \dots, t - 1,$$

the coefficients in the exponent. Party i , on receiving its private share s_i , accepts it iff

$$[s_i]G \stackrel{?}{=} \sum_{k=0}^{t-1} [i^k] \Phi_k.$$

An honest share passes: since $a \mapsto [a]G$ is a homomorphism $\mathbb{F}_q \rightarrow \mathbb{G}$, $\sum_k [i^k] \Phi_k = \sum_k [a_k i^k]G = [f(i)]G$.

Proposition 4.45 (What the Feldman check certifies). *Because G generates the prime-order group $\mathbb{G}$, the map $a \mapsto [a]G$ is a bijection $\mathbb{F}_q \rightarrow \mathbb{G}$, so the broadcast vector $(\Phi_0, \dots, \Phi_{t-1})$ —however the dealer chose it—determines a unique polynomial $f^*(X) := \sum_k a_k^* X^k$ with $a_k^* = \log_G \Phi_k$, of degree at most $t - 1$. Party i 's check passes iff $s_i = f^*(i)$. Consequently every coalition of t parties holding accepted shares reconstructs the same polynomial f^* and the same secret $s^* = f^*(0) = \log_G \Phi_0$, and a party handed an inconsistent share detects it alone and with certainty. The check certifies nothing about the value of s^* , which the dealer chooses freely.*

Proof. Both claims are injectivity of $a \mapsto [a]G$. The right-hand side of the check equals $[f^*(i)]G$ by the homomorphism, so the equation $[s_i]G = [f^*(i)]G$ holds iff $s_i = f^*(i)$. Accepted shares are thus evaluations of the one polynomial f^* , and t of them interpolate to it by uniqueness. $\square$

The vector $(\Phi_k)_k$ is a *perfectly binding* commitment to the polynomial f —the coefficient list committed entry by entry under the bijection $a \mapsto [a]G$ —and the share check is an evaluation check against it, verifiable by anyone because it is linear in the coefficients. Perfect binding comes at the expense of hiding, as Theorem 4.8 requires.

Remark 4.46 (Leakage of Feldman sharing). Feldman sharing publishes $\Phi_0 = [s]G$. Secrecy below the threshold is no longer perfect: from the broadcast alone an unbounded adversary computes $s = \log_G \Phi_0$. For an efficient coalition of $t - 1$ parties the leak is exactly $[s]G$ and nothing more: its entire view—shares and broadcast—is computable from $[s]G$ and $t - 1$ uniformly random field elements, because the shares are uniform by Theorem 4.43 and the Φ_k are then determined by interpolating in the exponent through $(0, [s]G)$ and the points $(i, [s_i]G)$. Recovering s from that view is the discrete-logarithm problem (Definition 2.2). The leak is acceptable exactly when s is itself a discrete-logarithm secret key whose public key $[s]G$ is published regardless; then Φ_0 is the public key and nothing new is revealed. When the secret must stay perfectly hidden, Pedersen verifiable secret sharing replaces each Φ_k by the Pedersen commitment $[a_k]G + [b_k]H$ of Construction 4.16, restoring perfect hiding at the price of computational binding; this volume neither uses nor constructs it. Threshold signing uses the Feldman construction: each signer holds a Shamir share of a signing key, the coefficient commitments broadcast during key generation are Feldman's, and the Lagrange weights $\lambda_{i,t}$ recombine per-signer contributions into a single signature.

5 Pseudorandom functions, block ciphers, and format-preserving encryption

This section defines pseudorandom functions (PRFs), pseudorandom permutations, and strong pseudorandom permutations, each by a game in which an adversary queries a black box adaptively and outputs one bit guessing whether a short secret key drives the box or the box is a uniformly random function or permutation, in the strong case queried in both directions. The switching lemma (Lemma 5.9) bounds the cost of exchanging the two ideal objects. The section then constructs the instances the protocol uses: the block cipher AES; keyed and personalised BLAKE2, which supplies the PRFs inventoried in Remark 5.14; and the format-preserving cipher FF1, which builds a keyed bijection from a PRF that is not invertible and, on 11-byte strings, derives Orchard’s address diversifiers. The next section builds from these primitives symmetric encryption, authentication, and the authenticated encryption that carries every shielded note.

Notation is that of the provable-security model. Strings of length ℓ form $\{0, 1\}^\ell$, all finite strings form $\{0, 1\}^*$, and $|x|$ is the length of x ; for a finite set S , the notation $x \xleftarrow{\$} S$ draws x uniformly from S , independently of everything else. Adversaries are PPT machines in the security parameter λ (§1.1), a negligible function is $\text{negl}(\lambda)$ (recalled in §1.2), and every security notion below is a distinguishing game in the sense of Definition 1.15, scored by the difference

$$\text{Adv}(\mathcal{A}) = |\Pr[\mathcal{A} \text{ outputs } 1 \text{ in } \text{Exp}_1] - \Pr[\mathcal{A} \text{ outputs } 1 \text{ in } \text{Exp}_0]|$$

between the two experiments the game distinguishes. Oracle access formalises the black box.

Definition 5.1 (Oracle access). For a function f , the notation $\mathcal{A}^f$ denotes an adversary granted *oracle access* to f : a black box to which $\mathcal{A}$ may submit inputs x of its choice, receiving $f(x)$ in a single computation step, and from which it learns nothing about f beyond the answers to the queries it makes. The adversary’s running time always bounds the number of its oracle queries, which is hence polynomial in λ .

5.1 Pseudorandom functions and permutations

Pseudorandomness is measured against an ideal object, the “truly random function”.

Definition 5.2 (Random function). Let $\text{Func}(D, R)$ denote the set of all functions from a finite set D to a finite set R . A *random function* from D to R is an element $F \xleftarrow{\$} \text{Func}(D, R)$, chosen uniformly at random. Equivalently, the values $\{F(x)\}_{x \in D}$ are independent and uniformly distributed over R .

There are $|R|^{|D|}$ functions in $\text{Func}(D, R)$, an infeasibly large set for the domains of interest: for $D = R = \{0, 1\}^{128}$ the count is $2^{128 \cdot 2^{128}}$. Such a function has no description shorter than its truth table and cannot be stored; a pseudorandom function replaces it by a short key while preserving its observable behaviour. The ideal object is realised by *lazy sampling*, as for the random oracle of Definition 3.13: the oracle keeps a table, initially empty; on a fresh query x it samples $F(x) \xleftarrow{\$} R$, records the pair, and returns it; on a repeated query it returns the recorded value. This produces exactly the distribution of Definition 5.2 while materialising only the queried points.

Definition 5.3 (Keyed function). A *keyed function* is a deterministic, efficiently computable map

$$F: \{0, 1\}^\lambda \times \{0, 1\}^{\ell(\lambda)} \longrightarrow \{0, 1\}^{m(\lambda)}, \quad (k, x) \mapsto F_k(x),$$

where the input length ℓ and output length m are polynomially bounded functions of the key length λ . Fixing the key k yields the function $F_k: \{0, 1\}^{\ell(\lambda)} \rightarrow \{0, 1\}^{m(\lambda)}$.

Definition 5.4 (Pseudorandom function). Let F be a keyed function. The distinguishing game $\text{PRF}_F^A(\lambda)$ runs as follows.

1. The challenger samples a bit $b \xleftarrow{\$} \{0, 1\}$. If $b = 1$ it samples $k \xleftarrow{\$} \{0, 1\}^\lambda$ and sets $\mathcal{O} := F_k$; if $b = 0$ it sets $\mathcal{O} := R$ for $R \xleftarrow{\$} \text{Func}(\ell(\lambda), m(\lambda))$, realised by lazy sampling, where $\text{Func}(\ell, m) := \text{Func}(\{0, 1\}^\ell, \{0, 1\}^m)$.
2. The adversary $\mathcal{A}(1^\lambda)$, receiving the security parameter in unary to convey the parameter size, queries $\mathcal{O}$ adaptively.
3. The adversary outputs a bit b' ; the game outputs 1 iff $b' = b$.

The family F is a *pseudorandom function* (PRF) if for every PPT adversary $\mathcal{A}$,

$$\text{Adv}_F^{\text{prf}}(\mathcal{A}, \lambda) := \left| \Pr_{k \xleftarrow{\$} \{0, 1\}^\lambda} [\mathcal{A}^{F_k}(1^\lambda) = 1] - \Pr_{R \xleftarrow{\$} \text{Func}(\ell(\lambda), m(\lambda))} [\mathcal{A}^R(1^\lambda) = 1] \right| \in \text{negl}(\lambda).$$

The two probabilities are the adversary's output distributions in the worlds $b = 1$ and $b = 0$, so this difference equals twice the bit-guessing advantage $|\Pr[\text{PRF}_F^A(\lambda) = 1] - \frac{1}{2}|$ of Definition 1.15; the factor of two is immaterial to negligibility, and the difference form is the one every bound below is stated in.

The algorithm defining F is public; only the sampled key is hidden.

The restriction to efficient adversaries cannot be removed. As k ranges over its 2^λ values, the keyed function F_k takes at most 2^λ distinct values in $\text{Func}(D, R)$ —a vanishing fraction of all $|R|^{|D|}$ functions—so the two distributions of Definition 5.4 have statistical distance close to the maximum, and an *unbounded* adversary could distinguish them. Security therefore holds only against computationally bounded adversaries, as for the PRG separation in the proof of Proposition 1.10.

Remark 5.5 (Necessity of adaptivity). The adversary's access is *adaptive*: later queries may depend on earlier answers, and downstream constructions rely on this. A non-adaptive definition, in which the adversary fixes all queries before seeing any answer, is strictly weaker: families exist that pass every non-adaptive test yet fall to an adaptive distinguisher.

Decryption requires each key to yield an invertible map, so a block cipher is a keyed *permutation*, measured against a random permutation.

Definition 5.6 (Keyed permutation). A keyed function $F: \{0, 1\}^\lambda \times \{0, 1\}^\ell \rightarrow \{0, 1\}^\ell$ with equal input and output length is a *keyed permutation* if for every key k the map F_k is a bijection of $\{0, 1\}^\ell$, and both F_k and F_k^{-1} are efficiently computable given k . The set of all permutations of $\{0, 1\}^\ell$ is $\text{Perm}(\ell)$, of cardinality $(2^\ell)!$.

Definition 5.7 (Pseudorandom permutation). Let F be a keyed permutation. The game $\text{PRP}_F^A(\lambda)$ is the game of Definition 5.4 with the ideal world replaced: for $b = 0$ the challenger sets $\mathcal{O} := P$ for $P \xleftarrow{\$} \text{Perm}(\ell)$, a uniformly random permutation. The family F is a *pseudorandom permutation* (PRP) if for every PPT adversary $\mathcal{A}$,

$$\text{Adv}_F^{\text{PRP}}(\mathcal{A}, \lambda) := \left| \Pr_k [\mathcal{A}^{F_k}(1^\lambda) = 1] - \Pr_{P \xleftarrow{\$} \text{Perm}(\ell)} [\mathcal{A}^P(1^\lambda) = 1] \right| \in \text{negl}(\lambda).$$

Definition 5.8 (Strong pseudorandom permutation). A keyed permutation F is a *strong pseudorandom permutation* (SPRP) if no PPT adversary distinguishes even when granted oracle access to *both* the permutation and its inverse: for every PPT $\mathcal{A}$,

$$\text{Adv}_F^{\text{SPRP}}(\mathcal{A}, \lambda) := \left| \Pr_k [\mathcal{A}^{F_k, F_k^{-1}}(1^\lambda) = 1] - \Pr_{P \xleftarrow{\$} \text{Perm}(\ell)} [\mathcal{A}^{P, P^{-1}}(1^\lambda) = 1] \right| \in \text{negl}(\lambda).$$

A block cipher deployed where the adversary can obtain decryptions of chosen ciphertexts must be an SPRP, the standard security target for a block cipher.

On q distinct queries, a random function $R \in \text{Func}(\ell, \ell)$ returns q independent uniform strings, which may collide, and a random permutation $P \in \text{Perm}(\ell)$ returns q *distinct* uniform strings. Output collisions are the *only* statistical difference between the two, and the switching lemma bounds it.

Lemma 5.9 (PRP/PRF switching lemma). Let $\ell \geq 1$ and let $\mathcal{A}$ be any adversary—even a computationally unbounded one—making at most q queries to its oracle. Then

$$\left| \Pr_{P \xleftarrow{\$} \text{Perm}(\ell)} [\mathcal{A}^P = 1] - \Pr_{R \xleftarrow{\$} \text{Func}(\ell, \ell)} [\mathcal{A}^R = 1] \right| \leq \frac{q(q-1)}{2^{\ell+1}}.$$

We omit the proof, a game-hopping argument (Remark 1.18) that bounds the probability of an output collision among the q answers by the birthday count $\binom{q}{2}/2^\ell$ of Lemma 3.11, computed in the *Math Guide* (§“The union bound and a birthday calculation”).

Consequently, a secure PRP on a large domain is also a secure PRF while the number of queries stays well below the birthday bound $2^{\ell/2}$: the two advantages differ by at most $q(q-1)/2^{\ell+1}$. For AES, with $\ell = 128$, the switching term $q^2/2^{129}$ is negligible for any realistic query budget, so the FF1 construction of §5.4 uses AES—a permutation—as the PRF its round function requires.

5.2 Block ciphers and AES

Definition 5.10 (Block cipher). A *block cipher* with block length b and key length κ is a keyed permutation

$$E: \{0, 1\}^\kappa \times \{0, 1\}^b \longrightarrow \{0, 1\}^b,$$

together with its inverse $D = E^{-1}$ satisfying $D_k(E_k(x)) = x$ for all k, x . Here E_k is *encryption* and D_k *decryption* under key k . The security goal is that E be a strong pseudorandom permutation (Definition 5.8).

The asymptotic PRP definition concerns a family indexed by λ ; a block cipher fixes b and κ — $b = 128$ and $\kappa \in \{128, 192, 256\}$ for AES—and so offers *concrete* security in the style of Definition 1.31: an explicit function of the adversary’s resources bounds the advantage of the best known attacks.

Definition 5.11 (Concrete (t, q, ε) -security). A block cipher E is a (t, q, ε) -secure PRP if every adversary running in time at most t and making at most q oracle queries has $\text{Adv}_E^{\text{prp}}(\mathcal{A}) \leq \varepsilon$; analogously for SPRP and PRF security.

Construction 5.12 (AES). The Rijndael cipher of Daemen and Rijmen, standardised as AES in FIPS 197, has block length $b = 128$ bits and key lengths $\kappa \in \{128, 192, 256\}$, with $N_r \in \{10, 12, 14\}$ rounds respectively. Structurally it is a *substitution–permutation network* (SPN): the 128-bit state, viewed as a 4×4 array of bytes—elements of $\mathbb{F}_{2^8}$ —is first whitened by an initial round-key XOR, then passes through N_r rounds built from four layers:

1. **SubBytes**, a nonlinear byte substitution—the *S-box*—built on field inversion $a \mapsto a^{-1}$ in $\mathbb{F}_{2^8}$ (composed with a fixed affine map), chosen so that every input difference maps to any given output difference with only small probability, and every linear approximation of it holds with probability close to one half;
2. **ShiftRows**, a transposition of bytes within the array;
3. **MixColumns**, an invertible column-wise linear mix; and
4. **AddRoundKey**, the XOR of a 128-bit round key that a key schedule expands from k .

The first $N_r - 1$ rounds compose all four layers in this order; the final round omits MixColumns. The two permutation layers together guarantee that after two rounds every output byte depends on every input byte. Each layer—the initial whitening included—is individually invertible, so E_k is a permutation as Definition 5.10 requires, and decryption applies the inverses in reverse order.

Remark 5.13 (Confusion, diffusion, and the status of AES security). The SPN design realises Shannon’s two principles. The nonlinear S-box supplies *confusion*: making the relationship between key and ciphertext as complex as possible. The ShiftRows and MixColumns layers supply *diffusion*: spreading the influence of each input bit over the whole output. Iterating the two for ten or more rounds gives AES its empirical resistance to all known attacks. No proof that AES is a PRP exists; like every practical block cipher, its security is a *conjecture*, corroborated by decades of failed cryptanalysis but resting on no reduction to a cleaner problem. Every downstream theorem therefore takes “ E is a PRP/SPRP” as an assumption, the status Remark 1.29 assigned to the concrete hash functions.

5.3 PRFs from hash functions: length extension and keyed BLAKE2

Hash functions are a second source of PRFs. The naive keying of a hash H by prepending the key,

$$F_k(m) = H(k \parallel m),$$

is unsafe for the iterated Merkle–Damgård hashes (MD5, SHA-1, SHA-256)—which pad the message into blocks, fold the blocks one at a time into a fixed-width chaining state through a compression function, and emit the final state—because of the *length-extension attack*: from $H(k \parallel m)$ and $|m|$ alone, the adversary computes $H(k \parallel m \parallel \text{pad} \parallel m')$ for a suffix m' of its choosing, *without knowing k* , since a Merkle–Damgård digest is the internal chaining state after absorbing $k \parallel m$, from which the adversary resumes the iteration. Used as a message authentication code (MAC)—a keyed function whose output tag certifies a message, defined under “Message authentication codes” in the next section—the naive construction thus yields a valid tag on $m \parallel \text{pad} \parallel m'$, a message the key holder never authenticated.

The classical remedy is HMAC, the nested double call

$$\text{HMAC}_k(m) = H((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel m))$$

for two fixed pad constants: the inner hash absorbs the message under one key-derived value, and the outer hash re-keys the fixed-length result, so the emitted digest is no longer a resumable chaining state. Bellare’s analysis shows that HMAC is a PRF whenever the underlying compression function is, up to a birthday term $q^2/2^\ell$ in the chaining-state length ℓ ; as a PRF it is in particular a secure message authentication code, and it is the building block of the standardised key-derivation function HKDF, noted under “Key-derivation functions” in the next section. The shielded protocol does not use HMAC, because its hash admits direct keying.

Hash functions of the sponge family (Construction 3.26) and of the HAIFA family—the Merkle–Damgård variant whose compression function also takes the count of bits hashed so far, the lineage of BLAKE2—are not subject to length extension and so admit direct keying. The BLAKE2 design is built to be keyed *natively*: the construction loads the key length into its parameter block and prepends the key, padded to a full block, as the first message block, and its compression function mixes in a block counter and finalisation flags. (Its successor BLAKE3 is natively keyed too, by a different mechanism: the key words replace the initial state, and a dedicated flag marks the keyed mode.) The keyed function

$$F_k(m) = \text{BLAKE2}(m; \text{key} = k)$$

is then a PRF directly, without a nested double call. No length-extension attack applies: the finalisation flag, XORed into the state for the last block, means the emitted digest is *not* the raw chaining value and cannot be used to resume the computation.

Remark 5.14 (Zcash’s use of keyed and personalised BLAKE2). The Zcash protocol uses BLAKE2b and BLAKE2s through two parameters of the BLAKE2 design: the *key*, for PRF and MAC use as above, and the *personalisation* string, a domain-separation tag written into the initial state—16 bytes for BLAKE2b, 8 bytes for BLAKE2s. Personalisation makes the same input hashed for two purposes—nullifier derivation versus note-commitment derivation, for

example—yield independent-looking outputs, so each use site is effectively an independent PRF. This realises the independent oracle per context that Proposition 3.17 proved for tagged random oracles; the separation takes effect before any message byte is processed and, since BLAKE2 builds in keying and personalisation as parameters, costs less than HMAC. The deployed inventory spans both widths. On the BLAKE2b side, the key-expansion PRF $\text{PRF}^{\text{expand}}$ is BLAKE2b-512 personalised with `Zcash_ExpandSeed` (protocol specification §“Pseudo Random Functions”), evaluated on the spending key to derive Orchard’s key tree; the note-encryption KDF is personalised with `Zcash_OrchardKDF` (§“Orchard Key Derivation”) and the outgoing cipher key with `Zcash_Orchardock` (§“Pseudo Random Functions”)—all 16-byte tags. On the BLAKE2s side, the nullifier PRF and fixed-base generators of Sapling—the shielded protocol preceding Orchard—use 8-byte personalisations such as `Zcash_nf` and `Zcash_PH` (§“Pseudo Random Functions” and §“Pedersen Hash Function”).

5.4 Format-preserving encryption and FF1

Orchard derives the *diversifiers* behind its shielded addresses—one 11-byte value per address, many addresses per spending key—by encrypting a sequential address index under a keyed bijection on the set of 11-byte strings; an adversary who could invert the map, or distinguish it from a random permutation of that set, would link a wallet’s addresses to one another. The requirement is a block cipher whose “block” is an arbitrary finite set, as in encrypting a 16-digit card number into a 16-digit card number that legacy databases, field-length constraints, and checksum formats continue to accept. *Format-preserving encryption* (FPE) meets it.

Definition 5.15 (Format-preserving encryption). Let $\mathcal{X}$ be a finite set, the *format*—for example $\mathcal{X} = \{0, 1, \dots, 9\}^{16}$, the 16-digit strings. A *format-preserving encryption scheme* on $\mathcal{X}$ is a keyed permutation $\mathcal{E}: \{0, 1\}^k \times \mathcal{X} \rightarrow \mathcal{X}$ such that for every key k the map $\mathcal{E}_k: \mathcal{X} \rightarrow \mathcal{X}$ is a bijection with efficiently computable inverse $\mathcal{D}_k$. Security requires $\mathcal{E}$ to be a (strong) pseudorandom permutation on $\mathcal{X}$ —Definition 5.7 with $\text{Perm}(\mathcal{X})$ in place of $\text{Perm}(\{0, 1\}^\ell)$: no efficient adversary distinguishes $\mathcal{E}_k$ from a uniformly random permutation of $\mathcal{X}$, even with access to the inverse.

Since $|\mathcal{X}|$ need not be a power of two, a plain block cipher does not apply. FPE is necessarily deterministic, unlike the randomised encryption modes of the next section: the format $\mathcal{X}$ has no room to store a per-message random value. FPE therefore leaks equality of plaintexts, as any deterministic cipher must; this cost of preserving format with no expansion is acceptable for the tokenisation use cases FPE targets and immaterial for diversifier derivation, where each index is encrypted once.

Any format of the shape “strings of length ν over an alphabet of radix ρ ” is in bijection with $\{0, 1, \dots, \rho^\nu - 1\}$ by reading the string as a base- ρ numeral, so it suffices to build a keyed pseudorandom permutation of $\mathbb{Z}/M\mathbb{Z}$ for an arbitrary modulus $M = \rho^\nu$. The FF1 construction below splits $M = a \cdot b$ into two factors of nearly equal size and runs, with halves in $\mathbb{Z}/a\mathbb{Z}$ and $\mathbb{Z}/b\mathbb{Z}$, a *Feistel network*, which builds a permutation from round functions that need not be invertible.

Definition 5.16 (Balanced Feistel network). Let $F_1, \dots, F_r: \{0, 1\}^w \rightarrow \{0, 1\}^w$ be arbitrary round functions. The r -round Feistel network $\Psi[F_1, \dots, F_r]$ is the map on $\{0, 1\}^{2w}$ defined as follows: write the input as (L_0, R_0) with $L_0, R_0 \in \{0, 1\}^w$, iterate

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F_i(R_{i-1}), \quad i = 1, \dots, r,$$

and output (L_r, R_r) .

Proposition 5.17 (Feistel is a bijection regardless of the round functions). *For any round functions $F_1, \dots, F_r$ —injective or not—the map $\Psi[F_1, \dots, F_r]$ is a bijection of $\{0, 1\}^{2w}$, inverted by running the rounds backwards:*

$$R_{i-1} = L_i, \quad L_{i-1} = R_i \oplus F_i(L_i), \quad i = r, \dots, 1.$$

Proof. It suffices to show each round is invertible, the composition of bijections being a bijection. The i -th round sends $(L_{i-1}, R_{i-1}) \mapsto (R_{i-1}, L_{i-1} \oplus F_i(R_{i-1}))$. Given the output (L_i, R_i) , one recovers $R_{i-1} = L_i$ directly, and then $L_{i-1} = R_i \oplus F_i(R_{i-1}) = R_i \oplus F_i(L_i)$, using that XOR is its own inverse. The recovery uses only *forward* evaluations of F_i —at no point is an inverse of the round function required, and this permits a permutation to be built from a PRF that need not be invertible. The displayed backward recursion is exactly this inversion applied round by round. $\square$

In FF1 the round functions derive from a PRF—AES iterated in cipher-block-chaining fashion, as specified below—so AES is evaluated only in the forward direction, in encryption and decryption alike.

Bijectivity holds for any round functions; pseudorandomness does not, but follows from PRF round functions and enough rounds.

Theorem 5.18 (Luby–Rackoff). *Let the round functions $F_1, \dots, F_r$ be independent PRFs on $\{0, 1\}^w$. Against an adversary making q queries, the Feistel network Ψ is:*

1. *with $r = 3$ rounds, a pseudorandom permutation (forward queries only); and*
2. *with $r = 4$ rounds, a strong pseudorandom permutation (forward and inverse queries);*

in each case with distinguishing advantage $O(q^2/2^w)$ above the PRF advantage of the round functions.

We omit the proof; its outline accounts for the birthday term and the round counts. One first replaces each PRF round function by a truly random function, at the cost of r PRF advantage terms. For three rounds one then shows that, unless two of the at most q queries collide in an intermediate Feistel value—a birthday event of probability $O(q^2/2^w)$ —the middle round’s random function is evaluated at distinct inputs, so it produces independent uniform values that perfectly mask the relationship between input and output halves, and the network’s answers are distributed exactly as a random function’s—*independent and uniform on $\{0, 1\}^{2w}$* . A final application of the switching lemma (Lemma 5.9), at a further cost of $q(q-1)/2^{2w+1}$, carries the comparison from the random function to the random permutation; the term is absorbed by the same $O(q^2/2^w)$

bound. The fourth round adds the symmetry needed to resist inverse queries as well, upgrading PRP to SPRP. The full argument is a transcript analysis bounding the probability of any internal collision.

Remark 5.19 (Why FF1 uses ten rounds rather than four). Theorem 5.18 gives security only up to $q \approx 2^{w/2}$ queries, and in the FPE setting w is small—one half of a 16-digit number—so the birthday bound $q^2/2^w$ is weak. The standardised FF1 therefore prescribes *ten* rounds rather than the minimal four, as a security margin against the stronger attacks known for small-domain Feistel networks, which can sometimes beat the generic birthday bound. The round count is *specified*—NIST SP 800-38G fixes ten rounds—while a security proof matching the margin is an *open problem*: the existing analysis (Luby–Rackoff and its small-domain refinements) does not certify the full strength the margin is intended to provide. Invertibility does not depend on the round count (Proposition 5.17).

Construction 5.20 (The FF1 mode of NIST SP 800-38G). The FF1 mode instantiates an *unbalanced, modular, tweaked* Feistel network of ten rounds to build a PRP on $\mathbb{Z}/M\mathbb{Z}$ with $M = \rho^v$, the radix- ρ numeral reading of a length- v string. It supports a *tweak* T : a non-secret string that selects, in effect, one permutation from a family indexed by T , so that the same key encrypts the same plaintext differently under different tweaks—useful for binding a token to its context. Its operation:

1. **Split.** Set $u = \lfloor v/2 \rfloor$ and $v = v - u$. Split the plaintext numeral string X into a left part A of u symbols and a right part B of v symbols, with integer values in $\{0, \dots, \rho^u - 1\}$ and $\{0, \dots, \rho^v - 1\}$ respectively. The two halves live in the moduli $a = \rho^u$ and $b = \rho^v$; the network is unbalanced when v is odd.
2. **Round function from a PRF.** For each round $i = 0, 1, \dots, 9$, assemble a byte string Q encoding the tweak T , the round index i , and the current right half B ; prepend a fixed parameter block P encoding the radix, the lengths, and the tweak length; and apply $\text{PRF}(P \parallel Q)$, where PRF is AES in CBC-MAC mode: set $Y_0 = 0^{128}$, iterate $Y_j = \text{AES}_k(Y_{j-1} \oplus X_j)$ over the 16-byte blocks X_j of $P \parallel Q$, and output the final block Y_m (SP 800-38G, Algorithm 6). The iteration chains AES through an evolving state exactly as the Merkle–Damgård iteration of §5.3 chains a compression function; emitting the final state is safe *here* because the PRF’s inputs form a *prefix-free* set: the leading block P encodes the lengths of everything that follows, so two distinct inputs either differ already in their first block or agree on all lengths, and in neither case is one a proper prefix of the other. Prefix-free inputs are precisely the setting in which plain CBC-MAC is proved to be a PRF (a result of Petrank and Rackoff), and no length-extension query can arise. Further AES calls then expand the result to a long enough digest, which is read as an integer y .
3. **Modular Feistel step.** Update the halves by the modular analogue of Definition 5.16:

$$C = (A + y) \bmod m, \quad A \leftarrow B, \quad B \leftarrow C,$$

where the modulus m alternates between $a = \rho^u$ and $b = \rho^v$ with the parity of the round, so that the sum lands in the correct half’s domain; modular addition replaces XOR.

4. **Output.** After ten rounds, concatenate the final A and B and render the result back as a length- ν radix- ρ string.

Decryption runs the rounds in reverse, replacing modular addition by modular subtraction, $A \leftarrow (C - y) \bmod m$, exactly mirroring the backward recursion of Proposition 5.17. In step 2 the standard sizes the expanded digest at least four bytes beyond what the larger half requires, so the integer y ranges over at least 2^{32} multiples of the modulus, and the bias of the reduction $y \bmod m$ away from uniform is bounded by the modular-reduction bias of the *Math Guide* (§“Uniform sampling and the bias of modular reduction”) at below 2^{-33} per round.

Proposition 5.21 (FF1 is a keyed bijection on $\mathcal{X}$). *For every key k and tweak T , the FF1 map $\mathcal{E}_{k,T}: \mathcal{X} \rightarrow \mathcal{X}$ is a bijection of the format set $\mathcal{X}$, and $\mathcal{D}_{k,T}$ is its two-sided inverse.*

Proof. Each round is the map $(A, B) \mapsto (B, (A + y) \bmod m)$, where $y = y(i, T, B)$ depends only on the round index, the tweak, and the right half B —not on A . Fix B, i , and T ; then y is a constant and $A \mapsto (A + y) \bmod m$ is a translation of $\mathbb{Z}/m\mathbb{Z}$, hence a bijection, inverted by subtracting the same constant. The full round is therefore invertible: from the output (B, C) recover B directly, recompute $y = y(i, T, B)$ by a *forward* PRF evaluation, and set $A = (C - y) \bmod m$. Each even-indexed round is a bijection of $\mathbb{Z}/a\mathbb{Z} \times \mathbb{Z}/b\mathbb{Z}$ onto $\mathbb{Z}/b\mathbb{Z} \times \mathbb{Z}/a\mathbb{Z}$ and each odd-indexed round a bijection back, the two products coinciding when ν is even; the composition of the ten rounds, an even number, is a bijection of $\mathbb{Z}/a\mathbb{Z} \times \mathbb{Z}/b\mathbb{Z}$. The numeral map $(A, B) \mapsto A \cdot b + B$ identifies $\mathbb{Z}/a\mathbb{Z} \times \mathbb{Z}/b\mathbb{Z}$ with $\mathbb{Z}/M\mathbb{Z}$ as a bijection of *sets only*, not of groups: the factors satisfy $\gcd(a, b) = \rho^{\min(u, v)} > 1$, so the Chinese remainder theorem does not apply, and only bijectivity is used. Transporting through this map and the numeral bijection $\mathcal{X} \cong \mathbb{Z}/M\mathbb{Z}$, the composition $\mathcal{E}_{k,T}$ is a bijection of $\mathcal{X}$. Running the rounds backwards with subtraction inverts it on both sides, so $\mathcal{D}_{k,T} = \mathcal{E}_{k,T}^{-1}$. $\square$

Remark 5.22 (Bijectivity from algebra, security from the PRF). Feistel networks are the standard route to FPE because they separate two obligations. Invertibility follows from the structure alone: Propositions 5.17 and 5.21 hold for *any* round function, including a constant one. The PRF, AES, supplies *pseudorandomness*: making $\mathcal{E}_{k,T}$ look like a uniformly random permutation of $\mathcal{X}$ in the sense of Definition 5.15, per the Luby–Rackoff *heuristic*—Theorem 5.18 adapted, beyond what it literally proves, to the modular, multi-round, tweaked setting.

Remark 5.23 (The domain-size caveat, and the single Orchard use). The published SP 800-38G requires $\rho^\nu \geq 100$ and recommends $\rho^\nu \geq 10^6$ —its draft Revision 1 would make 10^6 mandatory—because for small domains an attacker can recover or distinguish the keyed permutation by exhaustively cataloguing input–output pairs, and known message-recovery attacks on small-domain FPE (Bellare–Hoang–Tessaro and successors) erode the security margin.

The single use of FF1 in Orchard is diversifier derivation: FF1-AES256 maps an address index to the 11-byte diversifier embedded in a shielded address, so that one spending key yields many mutually unlinkable addresses. The deployed instance (protocol specification §“Pseudo Random Permutations”) fixes radix $\rho = 2$, length $\nu = 88$, and the empty tweak, and reads the 11-byte index as a little-endian bit string, so the domain is the 2^{88} strings of 88 bits, far above both thresholds.

The address built on the diversifier, and the case for a keyed permutation rather than a hash in this role, are treated in a later volume of the series.

6 Symmetric encryption, AEAD, and key derivation

This section defines symmetric encryption, message authentication codes, authenticated encryption with associated data (AEAD), and key-derivation functions, each with its security notion. It proves that encrypt-then-MAC composes a cipher secure against chosen-plaintext attack and a strongly unforgeable MAC with independent keys into an AEAD (Theorem 6.16), and that ChaCha20-Poly1305, the AEAD of Zcash note encryption, is secure for a uniform key and non-repeating nonces if ChaCha20 is a pseudorandom function (Theorem 6.20); a key-derivation function derives that key from a Diffie–Hellman shared secret. The adversary controls the channel: it sees every ciphertext, and may modify, splice, or replay ciphertexts in transit.

Throughout, an *adversary* is a probabilistic algorithm $\mathcal{A}$; the notation $y \leftarrow \mathcal{A}(x)$ denotes running $\mathcal{A}$ on input x and assigning its output to y , and $x \xleftarrow{\$} S$ denotes sampling x uniformly from the finite set S . Every adversary in the computational definitions is probabilistic polynomial time (PPT) in the security parameter λ , and $\text{negl}(\lambda)$ denotes a negligible function (*Math Guide*, §“Polynomial, exponential, and negligible functions”; recalled in §1.2).

The one-time pad and EAV security. The information-theoretic baseline is the *one-time pad*: to encrypt $m \in \{0, 1\}^L$ under a uniformly random key $k \in \{0, 1\}^L$ used exactly once, output $c = m \oplus k$. Since $m \oplus k$ is uniform for every m , the ciphertext distribution is independent of the message, and even an unbounded eavesdropper learns nothing—Shannon’s *perfect secrecy*. The cost is twofold: the key must be as long as the message, and it must never be reused, for two ciphertexts under one pad reveal $c \oplus c' = m \oplus m'$ to any observer of both—the *two-time-pad* failure. The computational relaxation is *EAV security* (security against an eavesdropper): an adversary who chooses two equal-length messages m_0, m_1 and sees a single ciphertext $\text{Enc}_k(m_b)$ for a hidden uniform bit b guesses b with advantage at most $\text{negl}(\lambda)$ —the one-ciphertext, no-oracle restriction of the chosen-plaintext game formalised in §6.2. Replacing the truly random pad by the output of a pseudorandom generator stretched from a short seed—the *pseudo-one-time pad*—achieves EAV security with a fixed-length key, at the cost of resting on a pseudorandomness assumption.

6.1 Stream ciphers and ChaCha20

A *stream cipher* is the practical instantiation of the pseudo-one-time pad: a keyed, seekable generator producing a *keystream* that XORs into the plaintext. To encrypt many messages under one key without violating the one-time-pad no-reuse rule, modern stream ciphers take an additional public input called a *nonce* (number used once) and behave like a pseudorandom function of the nonce, in the sense of Section 5.

Definition 6.1 (Nonce-based stream cipher). A *nonce-based stream cipher* is a deterministic function

$$\text{KS} : \mathcal{K} \times \mathcal{N} \times \mathbb{N} \rightarrow \{0, 1\}^*$$

producing, for key k , nonce ν , and length L , a keystream $\text{KS}(k, \nu, L) \in \{0, 1\}^L$. Encryption of $m \in \{0, 1\}^L$ is $\text{Enc}_k(\nu, m) = m \oplus \text{KS}(k, \nu, |m|)$ with ciphertext (ν, c) ; decryption recomputes the keystream and XORs it off. The security goal is that for distinct nonces the keystreams are jointly indistinguishable from independent uniform strings, provided no nonce repeats under a fixed key.

The stream cipher Zcash deploys—inside the AEAD of §6.5, in the note-encryption layer—is ChaCha20, designed by Bernstein and standardised in RFC 8439. Its core is a reversible mixing of four 32-bit words built only from modular addition, XOR, and rotation (an “ARX” design), which avoids data-dependent table lookups and hence the timing side channels that table-based ciphers must otherwise mitigate.

Construction 6.2 (ChaCha20). The ChaCha20 state is a 4×4 matrix of 32-bit words, viewed as a vector $(x_0, \dots, x_{15}) \in (\mathbb{Z}/2^{32}\mathbb{Z})^{16}$; the symbol $+$ denotes addition modulo 2^{32} , $\oplus$ XOR, and $x \lll r$ left rotation by r bits. The *quarter-round* $\text{QR}(a, b, c, d)$ updates four words by

$$\begin{aligned} a &+= b; & d &\oplus= a; & d &\lll 16; \\ c &+= d; & b &\oplus= c; & b &\lll 12; \\ a &+= b; & d &\oplus= a; & d &\lll 8; \\ c &+= d; & b &\oplus= c; & b &\lll 7. \end{aligned}$$

The state is initialised from the 128-bit constant “expand 32-byte k” (words x_0 – x_3), a 256-bit key (x_4 – x_{11}), a 32-bit block counter (x_{12}), and a 96-bit nonce (x_{13} – x_{15}). One *double round* applies QR to the four columns and then to the four diagonals; the 20 rounds of ChaCha20 are ten double rounds. Writing X for the initial state and $R(X)$ for the state after the rounds, the 64-byte keystream block is the *feed-forward* sum $R(X) + X$ (word-wise, modulo 2^{32}), serialised in little-endian order.

Remark 6.3 (Feed-forward and counter mode). The rounds R form a bijection—each quarter-round is invertible, being a composition of invertible word operations—so without the final addition the block function would be an invertible permutation, and an adversary holding a keystream block could run the rounds backwards. The feed-forward sum $R(X) + X$ destroys invertibility; it is the standard Davies–Meyer-style construction of a one-way mixing function from a permutation. Incrementing the block counter x_{12} produces successive 64-byte keystream blocks, so $\text{KS}(k, \nu, L)$ is the concatenation of the blocks for counters 0, 1, 2, ...; this *counter mode* is what makes the cipher seekable and parallelisable. Like every concrete cipher of Section 5, ChaCha20 is only *conjectured* to be a secure PRF of the pair (nonce, counter); no attack better than brute force over the 256-bit key is known. Reusing a (k, ν) pair reproduces the same keystream and re-creates the two-time-pad failure, so nonce uniqueness is a hypothesis of the security claim.

6.2 Chosen-plaintext security

Security against *chosen-plaintext attack* (CPA) extends EAV security to an adversary that influences which messages are encrypted—by triggering payments, injecting plaintext into a session,

or guessing—and observes many ciphertexts. It is modelled by oracle access to the encryption algorithm in the game template of Definition 1.15.

Definition 6.4 (CPA security). For a symmetric encryption scheme Π with key generator Gen , adversary $\mathcal{A}$, and bit $b \in \{0, 1\}$, the game $\text{CPA}_{\Pi}^{\mathcal{A}}(\lambda, b)$ runs as follows.

1. The challenger samples $k \leftarrow \text{Gen}(1^\lambda)$ and gives $\mathcal{A}$ oracle access to $\text{Enc}_k(\cdot)$.
2. At some point $\mathcal{A}$ submits a challenge pair (m_0, m_1) of equal-length messages and receives $c^* \leftarrow \text{Enc}_k(m_b)$.
3. The adversary may continue querying the oracle and finally outputs a bit b' , which is the output of the game.

The scheme Π is *CPA-secure* if for every PPT $\mathcal{A}$,

$$\text{Adv}_{\Pi}^{\text{cpa}}(\mathcal{A}, \lambda) := |\Pr[\text{CPA}_{\Pi}^{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{CPA}_{\Pi}^{\mathcal{A}}(\lambda, 1) = 1]| \in \text{negl}(\lambda).$$

Proposition 6.5 (Determinism precludes CPA security). *No scheme whose Enc is deterministic and stateless is CPA-secure.*

Proof. The adversary queries the oracle on two distinct messages $m_0 \neq m_1$, recording $c_0 = \text{Enc}_k(m_0)$ and $c_1 = \text{Enc}_k(m_1)$. Decryption correctness forces $c_0 \neq c_1$: a single ciphertext cannot decrypt to both messages. The adversary submits the challenge (m_0, m_1) , receives $c^* = \text{Enc}_k(m_b)$, and outputs 0 if $c^* = c_0$ and 1 otherwise. Since Enc is deterministic, $c^* = c_b$, and since $c_0 \neq c_1$ the comparison identifies b : the guess is always correct and the advantage is 1. $\square$

A CPA-secure scheme must therefore be randomised or nonce-based. A nonce-based stream cipher used with a fresh nonce per encryption—random, or drawn from a counter—attains CPA security under the PRF conjecture for its keystream generator, because each encryption is then effectively a fresh one-time pad; for ChaCha20 the argument is the confidentiality part of the proof of Theorem 6.20.

Remark 6.6 (Semantic security). Goldwasser and Micali’s original notion, *semantic security*, demands that whatever an efficient adversary can compute about the plaintext from the ciphertext, the adversary could have computed without the ciphertext, from the message length alone. Semantic security is provably equivalent to the indistinguishability notions above; we use indistinguishability because it is easier to manipulate in reductions.

6.3 Message authentication codes

CPA security implies nothing about integrity. A stream-cipher ciphertext is XOR-malleable: flipping a ciphertext bit flips the corresponding plaintext bit, so an adversary on the channel can turn a payment of one amount into a payment of another without the key. Detecting modification requires a second keyed primitive, directed against forgery rather than distinguishing: the shared-key *message authentication code*, which the AEAD requires.

Definition 6.7 (Message authentication code). A *message authentication code* (MAC) is a pair $(\text{Mac}, \text{Vrfy})$ with key space $\mathcal{K}$: algorithm $\text{Mac}_k(m)$ outputs a tag t , and $\text{Vrfy}_k(m, t) \in \{0, 1\}$ accepts (1) or rejects (0). Correctness requires $\text{Vrfy}_k(m, \text{Mac}_k(m)) = 1$ for all k, m . When Mac is deterministic, canonical verification recomputes the tag and checks equality.

Definition 6.8 (Existential and strong unforgeability). For a MAC Π and adversary $\mathcal{A}$, the game $\text{Forge}_\Pi^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger samples $k \xleftarrow{\$} \mathcal{K}$ and gives $\mathcal{A}$ oracle access to $\text{Mac}_k(\cdot)$; let Q be the set of messages queried.
2. The adversary outputs a pair (m^*, t^*) ; the game outputs 1 iff $\text{Vrfy}_k(m^*, t^*) = 1$ and $m^* \notin Q$.

The MAC is *existentially unforgeable under chosen-message attack* (EUF-CMA) if for every PPT $\mathcal{A}$,

$$\text{Adv}_\Pi^{\text{forge}}(\mathcal{A}, \lambda) := \Pr[\text{Forge}_\Pi^{\mathcal{A}}(\lambda) = 1] \in \text{negl}(\lambda).$$

A *strongly unforgeable* (sUF-CMA) MAC additionally forbids producing a new valid tag on an already-queried message: the adversary wins on (m^*, t^*) as long as the oracle never returned that exact *pair*. We write $\text{Adv}_\Pi^{\text{s-forge}}(\mathcal{A}, \lambda)$ for the advantage in this variant.

The MAC Zcash uses, Poly1305, is not a pseudorandom-function MAC but a *one-time, universal-hash* MAC: it provides information-theoretic security per nonce and is fast, at the cost of a key that authenticates only a single message. Its security rests on a hash family whose *differences* are unpredictable.

Definition 6.9 (ε -almost- Δ -universal hash family). Let $(\mathcal{G}, +)$ be a finite abelian group. A family of functions $\{h_r : \mathcal{X} \rightarrow \mathcal{G}\}_{r \in \mathcal{R}}$ is *ε -almost- Δ -universal* (ε -A Δ U) if for all distinct $x, x' \in \mathcal{X}$ and every $\delta \in \mathcal{G}$,

$$\Pr_{r \xleftarrow{\$} \mathcal{R}} [h_r(x) - h_r(x') = \delta] \leq \varepsilon.$$

Taking $\delta = 0$ recovers ordinary ε -universality—low collision probability—while the Δ condition controls *additive differences*, which is what makes the one-time MAC below secure. With $\mathcal{G} = \{0, 1\}^n$ under XOR this is the classical almost-XOR-universality; Poly1305 takes $\mathcal{G} = \mathbb{Z}/2^{128}\mathbb{Z}$ under addition modulo 2^{128} .

The Carter–Wegman paradigm turns any A Δ U family into a one-time MAC by masking the hash value with a fresh uniform pad.

Theorem 6.10 (Carter–Wegman one-time MAC). Let $\{h_r\}$ be ε -A Δ U into $(\mathcal{G}, +)$. Define a MAC with key (r, s) , where $r \xleftarrow{\$} \mathcal{R}$ and the pad $s \xleftarrow{\$} \mathcal{G}$, by $\text{Mac}_{r,s}(m) = h_r(m) + s$. If (r, s) authenticates at most one message, then any adversary—even a computationally unbounded one—who sees one message–tag pair (m, t) forges a valid pair (m^*, t^*) with $m^* \neq m$ with probability at most ε .

Proof. The adversary observes $t = h_r(m) + s$ and must output (m^*, t^*) with $m^* \neq m$ and $t^* = h_r(m^*) + s$. Subtracting the two tag equations eliminates the pad: the forgery succeeds iff

$$h_r(m^*) - h_r(m) = t^* - t.$$

Condition on the adversary's view. The pad s is uniform and independent of r , so given the single observed tag t , every value of $h_r(m)$ is equally consistent—each candidate value determines a unique s —and the observation therefore leaks nothing about r : the posterior on r remains uniform on $\mathcal{R}$. The adversary's difference $\delta := t^* - t$ and distinct pair (m, m^*) are thus fixed independently of r , and the AΔU property gives $\Pr[h_r(m^*) - h_r(m) = \delta] \leq \varepsilon$. $\square$

The MAC Poly1305 instantiates the paradigm with a polynomial-evaluation hash over the prime field $\mathbb{F}_p$ for $p = 2^{130} - 5$ (prime-field arithmetic as in the *Math Guide*, §“The prime field and two inversion routes”).

Construction 6.11 (Poly1305). Fix the prime $p = 2^{130} - 5$ and work in $\mathbb{F}_p$. The key is a pair (r, s) : r is a 128-bit value subjected to a fixed bit-clamping (the top four bits of bytes 3, 7, 11, 15 and the bottom two bits of bytes 4, 8, 12 are forced to zero), and s is a 128-bit pad. To authenticate a message, split it into ℓ blocks $c_1, \dots, c_\ell$ of at most 16 bytes; encode each block as an integer in $[0, 2^{128})$ and add 2^{8b} , where b is the block's byte length—this sets a 1 bit immediately above the block's bytes, making every coefficient nonzero and the block encoding injective in both content and byte length. Interpreting r and each c_i as elements of $\mathbb{F}_p$, the hash is the polynomial evaluation

$$h_r(m) = \sum_{i=1}^{\ell} c_i r^{\ell-i+1} \in \mathbb{F}_p,$$

computed by Horner's rule as $(\dots((c_1 r + c_2)r + c_3)r \dots)r$. The tag is

$$\text{Mac}_{r,s}(m) = ((h_r(m) \bmod p) + s) \bmod 2^{128},$$

the field hash reduced and added to the pad modulo 2^{128} .

Proposition 6.12 (Poly1305 is almost-Δ-universal). *Restricted to messages of at most L bytes—hence at most $\ell_{\max} = \lceil L/16 \rceil$ blocks—the reduced Poly1305 hash family $\{m \mapsto h_r(m) \bmod p\}$, read into $\mathbb{Z}/2^{128}\mathbb{Z}$, is ε -almost-Δ-universal with respect to addition modulo 2^{128} , with*

$$\varepsilon \leq \frac{8 \ell_{\max}}{2^{106}}.$$

The loss over the ideal $\ell_{\max} \cdot p^{-1}$ accounts for the clamping of r —which restricts r to a subset of size 2^{106} —and for the final reduction modulo 2^{128} .

Proof. Sketch. Fix two distinct messages m, m' with block encodings (c_i) and (c'_i) , padded to a common length $\ell \leq \ell_{\max}$ by leading zero coefficients without changing their values. The block encoding guarantees that $m \neq m'$ yields distinct coefficient vectors: genuine coefficients are nonzero, so differing block counts set a nonzero coefficient against a zero padding one, while equal block

counts leave two blocks differing in content or in final-block byte length, hence in encoding. The difference of hashes is

$$h_r(m) - h_r(m') = \sum_{i=1}^{\ell} (c_i - c'_i) r^{\ell-i+1} =: P(r) \in \mathbb{F}_p,$$

a nonzero polynomial in r of degree at most ℓ . It remains to count the values of r that attain a target difference $\delta \in \mathbb{Z}/2^{128}\mathbb{Z}$. The reduced hash values lie in $[0, p)$, so their integer difference lies in the interval $(-p, p)$, which has length $2p < 8 \cdot 2^{128}$; each residue class modulo 2^{128} therefore contains at most $\lceil 2p/2^{128} \rceil = 8$ candidate integer differences u . For each such u , the equation $P(r) = u$ in $\mathbb{F}_p$ has at most ℓ roots—a polynomial of degree $\leq \ell$ over a field has at most ℓ roots (*Math Guide*, §“Roots and the factor theorem”)—so at most 8ℓ values of r produce the difference δ . The clamped r is uniform over a set of 2^{106} values (clamping zeroes 22 of the 128 bits), so it equals one of those roots with probability at most $8\ell/2^{106} \leq 8\ell_{\max}/2^{106}$. $\square$

For messages of 16 384 bytes ($\ell_{\max} = 1024$ blocks) the bound gives $\varepsilon \leq 8 \cdot 1024/2^{106} = 2^{-93}$, against the ideal $\ell_{\max}/p \approx 2^{-120}$; the clamping and reduction losses cost 27 bits. By Proposition 6.12 and the Carter–Wegman Theorem 6.10, Poly1305 with a fresh, uniformly random pad s per message is a one-time MAC with forgery probability at most $8\ell_{\max}/2^{106}$ per verification attempt; the pad must be uniform, not merely unpredictable, because the pad-elimination step of the theorem uses uniformity. In the AEAD below, ChaCha20 generates the one-time key (r, s) pseudorandomly from the nonce—uniformly once the cipher is idealised as a random function—so the guarantee holds per nonce.

6.4 Authenticated encryption with associated data

Authenticated encryption combines the two guarantees: the adversary learns nothing about plaintexts (CPA security) *and* cannot produce a new ciphertext that decrypts to anything other than the rejection symbol $\perp$, a value distinct from every message (ciphertext integrity). The *associated data* is public context—packet headers, version bytes, an ephemeral public key—that travels in the clear but is authenticated with the ciphertext, so that the ciphertext cannot be replayed under a different context.

Definition 6.13 (AEAD scheme). An *authenticated encryption with associated data* (AEAD) scheme is a pair (Enc, Dec) with key space $\mathcal{K}$, nonce space $\mathcal{N}$, and associated-data space $\mathcal{D}$: algorithm $\text{Enc}_k(v, a, m)$ returns a ciphertext c , and $\text{Dec}_k(v, a, c)$ returns a message or $\perp$. Correctness: $\text{Dec}_k(v, a, \text{Enc}_k(v, a, m)) = m$ for all k, v, a, m . The scheme authenticates but does not encrypt the associated data a .

Definition 6.14 (Ciphertext integrity, INT-CTXT). For an AEAD scheme Π and adversary $\mathcal{A}$, the game $\text{Ctxt}_{\Pi}^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger samples $k \xleftarrow{\$} \mathcal{K}$ and gives $\mathcal{A}$ two oracles: an encryption oracle $\text{Enc}_k(\cdot, \cdot, \cdot)$, recording the set Q of returned triples (v, a, c) , and a verification oracle that on (v, a, c) returns 1 if $\text{Dec}_k(v, a, c) \neq \perp$ and 0 otherwise. The adversary must never repeat a nonce to

the *encryption* oracle (the *nonce-respecting* model); verification queries may reuse nonces freely.

2. The game outputs 1 iff some verification query $(\nu^*, a^*, c^*) \notin Q$ is answered 1.

The scheme Π has *ciphertext integrity* if for every PPT $\mathcal{A}$, $\Pr[\text{Ctxt}_\Pi^A(\lambda) = 1] \in \text{negl}(\lambda)$.

Definition 6.15 (Authenticated encryption). An AEAD scheme is *secure* (is an *AEAD*) if it is both CPA-secure and has ciphertext integrity (Definition 6.14), where the CPA game of Definition 6.4 is run with $k \xleftarrow{\$} \mathcal{K}$ and extended so that the adversary supplies the nonce and associated data of every oracle query and of the challenge, subject to the *nonce-respecting* restriction—a restriction on the adversary, not a property of the scheme—that no nonce is used twice across oracle queries and challenge. The restriction is necessary: an AEAD may encrypt deterministically given (ν, a, m) —ChaCha20-Poly1305 does—and then querying the oracle at (ν, a, m_0) and submitting the challenge (ν, a, m_0, m_1) would identify the bit by comparing ciphertexts.

The standard route from a CPA-secure cipher and a strongly unforgeable MAC to an AEAD is *encrypt-then-MAC*: encrypt the message, compute a MAC over the ciphertext together with the associated data and nonce, and append the tag; decryption verifies the tag *before* decrypting, and any tag failure returns $\perp$. The other orders, MAC-then-encrypt and encrypt-and-MAC, do *not* generically achieve authenticated encryption.

Theorem 6.16 (Encrypt-then-MAC yields authenticated encryption). *Let $\Pi_E = (\text{Enc}, \text{Dec})$ be a CPA-secure encryption scheme and $\Pi_M = (\text{Mac}, \text{Vrfy})$ a strongly unforgeable (sUF-CMA) MAC, with independent keys k_E, k_M . Define the composite scheme Π' by*

$$\text{Enc}'_{k_E, k_M}(\nu, a, m) = (c, t), \quad c \leftarrow \text{Enc}_{k_E}(\nu, m), \quad t \leftarrow \text{Mac}_{k_M}(\nu \| a \| c),$$

with Dec' returning $\perp$ unless $\text{Vrfy}_{k_M}(\nu \| a \| c, t) = 1$, in which case it returns $\text{Dec}_{k_E}(\nu, c)$. Then Π' is a secure AEAD: it is CPA-secure and has ciphertext integrity.

Proof. Ciphertext integrity. Suppose a PPT $\mathcal{A}$ making at most q_v verification queries wins the game: some fresh query $(\nu^*, a^*, (c^*, t^*)) \notin Q$ decrypts successfully, i.e. $\text{Vrfy}_{k_M}(\nu^* \| a^* \| c^*, t^*) = 1$. Each encryption-oracle call on (ν, a, m) returned (c, t) and corresponds to one MAC query on the string $w = \nu \| a \| c$ returning tag t . Because the framing $\nu \| a \| c$ parses uniquely (the component lengths are fixed or prepended), the freshness of (ν^*, a^*, c^*) means the MAC oracle never produced the string-tag pair (w^*, t^*) with $w^* := \nu^* \| a^* \| c^*$: either w^* was never queried, or it was queried and returned a different tag. Either way (w^*, t^*) is a fresh valid pair—a *strong* forgery against Π_M . The reduction $\mathcal{B}$ runs $\mathcal{A}$, answering encryption queries with its own MAC oracle and a self-chosen k_E . Lacking k_M , the reduction $\mathcal{B}$ cannot answer verification queries, so it guesses: it samples a uniform index $i \leq q_v$ in advance, answers each fresh verification query before the i -th with 0 (queries on triples in Q are answered 1, correctly), halts at the i -th fresh query, and outputs that query's (w^*, t^*) as its forgery. On the event that $\mathcal{A}$ wins and i indexes the *first* successful fresh query of $\mathcal{A}$ —the guess is right with probability $1/q_v$ —every earlier 0 answer was correct, the simulation

is perfect, and $\mathcal{B}$ wins, so

$$\Pr[\text{Ctxt}_{\Pi'}^A(\lambda) = 1] \leq q_v \cdot \text{Adv}_{\Pi_M}^{\text{s-forge}}(\mathcal{B}, \lambda) \in \text{negl}(\lambda),$$

the factor q_v being polynomial.

CPA security. The tag is computed from c (together with k_M, ν, a , and the MAC's own coins) and so carries no information about the challenge bit beyond what c already carries. Formally, the reduction $\mathcal{B}'$ samples k_M itself, forwards $\mathcal{A}$'s oracle and challenge queries to its own Π_E oracle to obtain the c -components, and appends tags it computes with k_M . This simulates Π' perfectly, so $\text{Adv}_{\Pi'}^{\text{cpa}}(\mathcal{A}, \lambda) = \text{Adv}_{\Pi_E}^{\text{cpa}}(\mathcal{B}', \lambda) \in \text{negl}(\lambda)$. $\square$

Remark 6.17 (Why the ordering matters). Encrypt-then-MAC rejects forged ciphertexts *without decrypting*: a rejected ciphertext never reaches the decryption logic, which therefore cannot leak, and this blocks chosen-ciphertext and padding-oracle attacks. MAC-then-encrypt verifies only after decrypting, exposing the decryption routine to attacker-chosen ciphertexts, which has historically been the source of many padding-oracle attacks. An authenticated-encryption scheme is also secure against chosen-ciphertext attack (CCA), whose game is the CPA game of Definition 6.4 with a decryption oracle added that refuses only the challenge ciphertext: by ciphertext integrity, every ciphertext the adversary did not legitimately obtain decrypts to $\perp$ except with negligible probability, so the oracle is of no use.

6.5 The ChaCha20-Poly1305 AEAD

The scheme ChaCha20-Poly1305 (RFC 8439) is the AEAD by which Zcash encrypts every shielded note ciphertext and outgoing ciphertext (protocol specification, §“Symmetric Encryption”). It is an encrypt-then-MAC composition of the ChaCha20 stream cipher with the Poly1305 one-time MAC, in which the cipher derives *both* the keystream and the one-time MAC key from the single (k, ν) input.

Construction 6.18 (ChaCha20-Poly1305). Fix a 256-bit key k and a 96-bit nonce ν . Encryption of a message m with associated data a proceeds as follows.

1. *One-time MAC key.* Run ChaCha20 with key k , nonce ν , and block counter 0; take the first 32 bytes of the keystream block as the Poly1305 one-time key (r, s) —clamp the first 16 bytes to form r ; the next 16 are s . Discard the rest of this block.
2. *Encrypt.* Run ChaCha20 with key k and nonce ν starting at block counter 1, producing the keystream KS_1 (the keystream of Remark 6.3 taken from counter 1 onward), and set $c = m \oplus \text{KS}_1(k, \nu, |m|)$.
3. *Authenticate.* Form the Poly1305 input by concatenating: a padded with zeros to a 16-byte boundary, c padded with zeros to a 16-byte boundary, the 8-byte little-endian length of a , and the 8-byte little-endian length of c . Compute the tag $t = \text{Poly1305}_{r,s}(\text{that input})$.

The ciphertext is (c, t) . Decryption recomputes (r, s) and the tag from (k, ν, a, c) , compares it to t in *constant time*, returns $\perp$ on mismatch, and otherwise outputs $m = c \oplus \text{KS}_1(k, \nu, |c|)$.

Remark 6.19 (Length encoding prevents splicing). Without the trailing 8-byte lengths of a and c , the zero-padding that aligns a and c to 16-byte boundaries would let an adversary shift bytes between the associated data and the ciphertext, or pad-extend one of them, while leaving the Poly1305 input string unchanged—breaking the binding between a and c even though every authenticated byte is intact. Appending the explicit lengths makes the MAC input an injective encoding of the pair (a, c) , the same injectivity that the per-block length encoding supplies within one message in the AΔU analysis (Proposition 6.12).

Theorem 6.20 (Security of ChaCha20-Poly1305). *Model ChaCha20 as a pseudorandom function $F_k(\nu, j)$ from (nonce, block counter) pairs to 64-byte blocks. In the nonce-respecting model, ChaCha20-Poly1305 is a secure AEAD. Concretely, consider a PPT adversary making q encryption queries and q_v verification queries, each of whose Poly1305 inputs—padded associated data, padded ciphertext, and the length block—comprises at most $\ell_{\max}$ sixteen-byte blocks, $\ell_{\max} = \lceil |a|/16 \rceil + \lceil |c|/16 \rceil + 1$. The adversary’s CPA advantage is at most twice the best PRF-distinguishing advantage against F at comparable cost, and the adversary forges—wins the INT-CTXT game—with probability at most*

$$\text{Adv}_F^{\text{prf}}(\mathcal{B}, \lambda) + \frac{8 q_v \ell_{\max}}{2^{106}},$$

where $\mathcal{B}$ is a PRF distinguisher of comparable cost.

Proof. Sketch. Replace F_k by a truly random function Φ from (nonce, counter) pairs to 64-byte blocks. Any noticeable change in the adversary’s success probability yields a PRF distinguisher of comparable cost—it simulates the whole game using its function oracle—so each replacement costs one $\text{Adv}_F^{\text{prf}}$ term; the CPA game is played twice (once per challenge bit), whence the factor two there. We analyse the idealised scheme.

Confidentiality. Since the adversary is nonce-respecting, every encryption query uses a fresh ν , and the keystream blocks $\Phi(\nu, 1), \Phi(\nu, 2), \dots$ are uniform and independent of everything else in the adversary’s view; the ciphertext $c = m \oplus \text{KS}$ is a one-time pad encryption and reveals nothing about m beyond its length. The tag is a deterministic function of c, a, ν and the (independent) key (r, s) , and adds no information. The idealised CPA advantage is therefore zero.

Integrity. For each nonce ν , the block $\Phi(\nu, 0)[0:32]$ is uniform and independent across distinct nonces; clamping its first half samples r from Poly1305’s prescribed restricted set, while the second half s stays uniform, so the resulting one-time keys are independent across nonces. Because counter 0 is reserved for this key while encryption consumes counters ≥ 1 , each one-time key is also independent of the keystream that produced its ciphertext. Per nonce, the setting is exactly the Carter–Wegman one-time MAC of Theorem 6.10 instantiated with the ε -AΔU Poly1305 hash of Proposition 6.12, $\varepsilon \leq 8\ell_{\max}/2^{106}$. Consider any single fresh verification query (ν^*, a^*, c^*, t^*) . If ν^* was never used for encryption, the pad s for ν^* is uniform and entirely unknown, so the tag is correct with probability 2^{-128} . If ν^* equals the nonce of some encryption query—nonce reuse is allowed in *verification* queries, only encryption is nonce-respecting—then the adversary has seen exactly one tag under that (r, s) , and freshness of (a^*, c^*) makes this a forgery on a new message under a one-time key, succeeding with probability at most ε by Theorem 6.10. The union

bound over the q_v verification queries (*Math Guide*, §“The union bound and a birthday calculation”) gives forgery probability at most $8q_v\ell_{\max}/2^{106}$ in the idealised scheme; adding back the PRF-switching cost yields the stated bound. $\square$

Remark 6.21 (One key for encryption and authentication). The scheme reuses one 256-bit key for both encryption and authentication, seemingly violating the independent-keys hypothesis of Theorem 6.16. Reserving block counter 0 for the Po1y1305 key and counters ≥ 1 for the keystream restores independence: once the cipher is idealised as a truly random function—the switch the proof of Theorem 6.20 pays for with a PRF-distinguishing term—outputs at distinct counters are independent, so (r, s) is independent of the keystream and the two-key analysis applies. For the PRF itself the independence is computational, not literal. Domain separation by a counter or label, the discipline of §3.5, is the general pattern by which one master key safely serves several roles.

Remark 6.22 (Wrong-key rejection and key commitment). Later volumes use this AEAD in a *trial-decryption* pattern: a receiver detects the notes addressed to it by attempting decryption of every ciphertext under each of its keys, relying on decryption under a wrong key to return $\perp$ (protocol specification, §“Symmetric Encryption”). The scheme provides this for *honestly produced* ciphertexts. Fix a ciphertext $(c, t) = \text{Enc}_k(v, a, m)$ and a key k' independent of k , and idealise the cipher under k and under k' as independent random functions, as in the proof of Theorem 6.20. The one-time key (r', s') that decryption under k' derives from block $(v, 0)$ is then uniform and independent of (c, t) : the pad s' alone makes the recomputed tag match t with probability exactly 2^{-128} , and even conditioned on one exposed tag under (r', s') , Theorem 6.10 bounds acceptance by the AΔU slack $8\ell_{\max}/2^{106}$ of Proposition 6.12, with $\ell_{\max}$ as in Theorem 6.20. Decryption under an independent key therefore rejects except with probability at most $2^{-128} + 8\ell_{\max}/2^{106}$ per attempt, plus two PRF-distinguishing terms for the two idealisations.

The guarantee does not extend to malicious senders: CPA security and INT-CTXT are *single-key* notions, and nothing in Definition 6.15 constrains a ciphertext crafted by a sender who knows several keys. The scheme is not *key-committing*—the separate notion demanding that no ciphertext verify under two distinct keys—and a sender who chooses keys k_1, k_2 can craft a single (c, t) that decrypts validly, to different plaintexts, under both: with (r_i, s_i) the one-time key each k_i derives, the hash h_{r_i} of Construction 6.11 is linear in the blocks of c , so fixing t , leaving two blocks of c free, and solving over $\mathbb{F}_p$ the two linear equations that pin the hash under r_i to $t - s_i$ (retrying until both solutions encode valid blocks) makes both keys accept (c, t) —the multi-key collision of Len, Grubbs, and Ristenpart’s partitioning-oracle attack. The trial-decryption arguments of later volumes therefore never rest on the tag alone against adversarial senders: their note-plaintext consistency checks—recomputing the note commitment from the decrypted plaintext and comparing it against the commitment the transaction carries—close this gap.

6.6 Key-derivation functions

Theorem 6.20 assumes a uniformly random 256-bit key; its proof does not apply to a non-uniform key, whose structure the adversary may exploit. A Diffie–Hellman shared secret is *high-entropy but not uniform*: a structured group encoding, not a uniform bit string; a hardware noise source may likewise spread its unpredictability unevenly across its bits. A *key-derivation function* (KDF)

converts such a source secret into one or more pseudorandom keys; the entropy of the source is measured worst-case, not average-case. (A passphrase is a usually *low*-entropy source and needs a deliberately costly password KDF instead.)

Definition 6.23 (Min-entropy). The *min-entropy* of a random variable X is

$$H_\infty(X) = -\log_2 \max_x \Pr[X = x];$$

equivalently, the best chance of guessing X in one try is $2^{-H_\infty(X)}$. Given side information Z , the *conditional min-entropy* is

$$\tilde{H}_\infty(X | Z) = -\log_2 \mathbb{E}_{z \leftarrow Z} \max_x \Pr[X = x | Z = z],$$

so that $2^{-\tilde{H}_\infty(X|Z)}$ is the best chance of guessing X from Z . Recall the *statistical distance* $\Delta(X, Y) = \frac{1}{2} \sum_u |\Pr[X = u] - \Pr[Y = u]|$ of Definition 1.6 (*Math Guide*, §“Statistical distance”); we say X is ϵ -close to *uniform* on a set S if $\Delta(X, U_S) \leq \epsilon$.

Definition 6.24 (Key-derivation function and its security). A *key-derivation function* is a function

$$\text{KDF} : \mathcal{X} \times \mathcal{I} \times \{0, 1\}^* \times \mathbb{N} \rightarrow \{0, 1\}^*$$

taking a source secret (a sample of a random variable Σ over $\mathcal{X}$), a non-secret *salt* $\text{slt} \in \mathcal{I}$, an *info* or context string ctx , and a desired output length L . The security goal must specify the source model. For an arbitrary source of conditional min-entropy at least γ given the adversary’s side information, an extractor-style guarantee uses a salt sampled uniformly and independently of the source and requires the pair $(\text{slt}, \text{KDF}(\Sigma, \text{slt}, \text{ctx}, L))$ to be computationally indistinguishable from (slt, U_L) , even given ctx and that side information. No fixed deterministic unsalted function can provide this guarantee for *every* high-min-entropy source; an unsalted KDF therefore rests on a narrower premise, such as input keying material that is already computationally pseudorandom, or that is hard to query in a random-oracle model.

The notion a KDF targets is that of a seeded randomness extractor whose output need only be *computationally* indistinguishable from uniform.

Definition 6.25 (Strong computational extractor). A keyed function $\text{Ext} : \mathcal{I} \times \mathcal{X} \rightarrow \{0, 1\}^n$ is a *strong* (γ, ϵ) -*computational extractor* if for every source Σ over $\mathcal{X}$ of conditional min-entropy at least γ given auxiliary information Z , and a uniformly random salt $\text{slt} \xleftarrow{\$} \mathcal{I}$, the pair $(\text{slt}, \text{Ext}_{\text{slt}}(\Sigma))$ is computationally indistinguishable from (slt, U_n) , even given Z , with distinguishing advantage at most ϵ . “Strong” means the salt is revealed to the distinguisher.

The leftover hash lemma realises the notion information-theoretically; the deployed mechanism below rests on random-oracle modelling instead.

Theorem 6.26 (Leftover hash lemma; stated without proof). Let $\{g_r : \mathcal{X} \rightarrow \{0, 1\}^n\}$ be a 2^{-n} -universal hash family with a uniform public seed R . For any X with $H_\infty(X) \geq \gamma$,

$$\Delta((R, g_R(X)), (R, U_n)) \leq \frac{1}{2} \sqrt{2^{n-\gamma}}.$$

Thus $\gamma \geq n + 2 \log_2(1/\epsilon)$ bits of min-entropy yield n output bits that are ϵ -close to uniform—a strong (γ, ϵ) -extractor in the statistical, not merely computational, sense.

HKDF. The standardised general-purpose KDF is Krawczyk’s HKDF (RFC 5869), built from HMAC (§5.3) in an *extract-then-expand* architecture: a salted HMAC call first distils suitable non-uniform input keying material into a short pseudorandom key (the extractor stage, justified along leftover-hash lines), and a counter-chained sequence of HMAC calls then expands that key, under a context string, up to RFC 5869’s limit of 255 hash-output-length blocks (the expander stage, justified by HMAC’s PRF security). The guarantee of HKDF depends on assumptions about the source and about HMAC. Zcash does not use it, because the protocol’s requirements are narrower and a single hash call meets them in the random-oracle analysis. The shielded protocol uses HMAC nowhere: neither the orchard nor the `zcash_note_encryption` crate carries an HMAC dependency (HMAC enters the wallet stack through the transparent BIP-32 derivation and BIP-39 mnemonic layers and, transitively, through the optional Tor client, never through a shielded crate).

The Zcash KDF. Orchard derives the note-encryption key by one personalised BLAKE2b call:

$$K_{\text{enc}} = \text{KDF}(\text{sharedSecret}, \text{epk}) = \text{BLAKE2b-256}(\text{"Zcash_OrchardKDF"; sharedSecret} \parallel \text{epk}),$$

where `sharedSecret` is the Diffie–Hellman shared point of the note-encryption key agreement, `epk` is the sender’s ephemeral public key, and the quoted string occupies BLAKE2b’s 16-byte personalisation field (the input is the serialised shared point followed by the ephemeral-key bytes, and the output is 32 bytes; protocol specification, §“Orchard Key Derivation”). The analysis models this personalised hash as a random oracle: by the domain-separation principle of §3.5 (Proposition 3.17, via the native-personalisation idiom catalogued there), the personalisation makes it an oracle independent of every other hash use in the protocol, and the derived key is uniform in the adversary’s view *unless* the adversary queries that oracle at the hidden `sharedSecret`—a query whose argument the adversary must first compute from the two public keys, an instance of the computational Diffie–Hellman problem of §2.2. Including `epk` in the input binds the derived key to this ephemeral key, the role the context string plays in HKDF. No salt is used because the claim is this narrower Diffie–Hellman-plus-random-oracle one—the unsalted premise of Definition 6.24—not extraction from every high-min-entropy source.

Remark 6.27 (The role of the KDF in the larger protocol). In the Zcash note-encryption layer the shared secret is a point of the Pallas curve, and the key derived above is the uniform key that Theorem 6.20 assumes for ChaCha20-Poly1305. Section 7 composes the key agreement, this KDF, and the AEAD into hybrid public-key encryption.

7 Key agreement

This section constructs Diffie–Hellman key agreement between two parties that share no prior secret and develops it into the hybrid public-key encryption by which shielded notes are delivered. It proves that the passive security of Diffie–Hellman equals the DDH advantage exactly (Theorem 7.11), that no hardness assumption on the group prevents a man-in-the-middle attack on

the unauthenticated protocol (Proposition 7.14), that hybrid encryption is passively secure under CDH, with the key-derivation function modelled as a random oracle and the AEAD secure (Theorem 7.24), and that ElGamal encryption is key-private under DDH (Theorem 7.28), a property that carries over to note encryption (Remark 7.29) and hides the recipient of a ciphertext published permanently on the ledger.

7.1 The Diffie–Hellman protocol

The protocol rests on the asymmetry of Section 2. In a cyclic group $G = \langle g \rangle$ of order q , the map

$$\exp_g : \mathbb{Z}/q\mathbb{Z} \rightarrow G, \quad x \mapsto g^x,$$

is a group isomorphism evaluated in $O(\log q)$ group operations by repeated squaring but believed infeasible to invert in the groups this volume uses; inverting it is the discrete logarithm problem (Definition 2.2). The section depends only on the group structure and the hardness of problems in it, not on the presentation of the group; the reference instances are a large prime-order subgroup of $\mathbb{F}_p^\times$ and the elliptic-curve point groups of §7.6. Diffie and Hellman observed that this asymmetry suffices for two parties with no prior shared secret to establish one over a public channel.

Construction 7.1 (Diffie–Hellman key exchange). Fix public parameters (G, g, q) with $G = \langle g \rangle$ of order q , known to all parties (and to the adversary).

1. Alice samples $a \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$ uniformly, computes $A = g^a$, and sends A to Bob.
2. Bob samples $b \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$ uniformly, computes $B = g^b$, and sends B to Alice.
3. Alice computes $k_A = B^a$; Bob computes $k_B = A^b$.

The value $k_A = k_B$ is the *shared secret*; the transmitted elements A and B are the *public shares* (public keys), and the exponents a and b are the *private keys*, never sent.

Proposition 7.2 (Correctness of Diffie–Hellman). *In Construction 7.1 both parties compute the same group element, $k_A = k_B = g^{ab}$.*

Proof. Unwinding the definitions,

$$k_A = B^a = (g^b)^a = g^{ba} = g^{ab} = (g^a)^b = A^b = k_B,$$

where $ba = ab$ because integer multiplication is commutative, and exponents may be read modulo q because g has order q . □

The shared secret g^{ab} is a function of the transcript (A, B) , computable by anyone who knows *one* of the two private keys and, conjecturally, by no one who knows only the public shares; §7.4 makes the conjecture precise.

Remark 7.3 (Small subgroups and the prime-order rule). If G has small subgroups, an adversary who substitutes an element of small order for a public share confines the victim’s computed secret

to a small subgroup and learns the victim's exponent modulo a small factor of q —the Pohlig–Hellman leakage of Corollary 2.21 in protocol form. The remedy is the standing rule of Section 2: prefer G of *prime* order q . Then the only subgroups are $\{1\}$ and G itself, every non-identity element is a generator, and no nontrivial confinement is possible. When the ambient group does not have prime order— $\mathbb{F}_p^\times$ has composite order $p - 1$ —one works inside a prime-order subgroup and has each recipient *validate* incoming elements as members: check $B^q = 1$ and $B \neq 1$ before exponentiating. The elliptic-curve analogue of this validation appears in Remark 7.19.

7.2 Static and ephemeral keys

Construction 7.1 leaves the lifetime of a key pair unspecified, and the properties of the protocol depend on it.

Definition 7.4 (Static and ephemeral keys). A Diffie–Hellman key pair (x, g^x) is *static* (long-term) if it is generated once and reused across many sessions, typically published and bound to an identity; it is *ephemeral* if it is generated freshly for a single session and discarded immediately afterwards.

Three protocol shapes result; all three are used below.

- *Ephemeral–ephemeral (DHE)*. Both parties use fresh keys. Once a and b are erased, no one—the parties included—can recover g^{ab} : this is *forward secrecy*, the guarantee that a later compromise of all stored long-term state cannot decrypt recorded past sessions, because the only secrets that ever existed were the deleted ephemerals.
- *Static–ephemeral*. The recipient Bob holds a static pair $(b, B = g^b)$, with B known in advance; Alice contributes a fresh ephemeral (a, A) . The shared secret g^{ab} then requires *no interaction from Bob at agreement time*: Alice computes $g^{ab} = B^a$ from the published B , attaches A , and sends everything in a single flow; Bob recovers $g^{ab} = A^b$ whenever he next comes online. This is the shape of hybrid encryption (§7.7) and of shielded-note delivery (§7.9): sender online once, recipient possibly offline, no round trip.
- *Static–static*. Both keys are long-term, so g^{ab} is a fixed function of the two identities—the same in every session. There is no forward secrecy, and the fixed secret must be combined with fresh per-session randomness (a nonce) before it may key any symmetric scheme; using it directly would reuse one key across all time.

In the static–ephemeral shape a static public key $B = g^b$ functions as an encryption public key: anyone can derive a secret shared with its owner by sending a single ephemeral share.

7.3 The key-agreement scheme abstraction

We package the non-interactive (static–ephemeral) pattern as a scheme with named algorithms, so that later constructions, including deployed note encryption, invoke key agreement independently of the underlying group.

Definition 7.5 (Key-agreement scheme). A (non-interactive) *key-agreement scheme* over a public parameter set pp consists of a private-key space $\mathcal{S}$, a public-key space $\mathcal{P}$, a shared-secret space $\mathcal{K}$, and three algorithms:

- $\text{KeyGen}(\text{pp}) \rightarrow s$, randomised, outputting a private key $s \in \mathcal{S}$;
- $\text{DerivePublic}(\text{pp}, s) \rightarrow p$, deterministic, mapping a private key to its public key $p \in \mathcal{P}$;
- $\text{Agree}(\text{pp}, s, p') \rightarrow k$, deterministic, mapping one's own private key and a counterparty's public key to a shared secret $k \in \mathcal{K}$.

Correctness demands that for all pp and all s, t in the support of $\text{KeyGen}(\text{pp})$,

$$\text{Agree}(\text{pp}, s, \text{DerivePublic}(\text{pp}, t)) = \text{Agree}(\text{pp}, t, \text{DerivePublic}(\text{pp}, s)). \quad (1)$$

Construction 7.6 (Diffie–Hellman as a key-agreement scheme). Fix $\text{pp} = (G, g, q)$ with $G = \langle g \rangle$ of prime order q . The scheme $\text{DH}_{G,g}$ takes $\mathcal{S} = \mathbb{Z}/q\mathbb{Z}$ (or $(\mathbb{Z}/q\mathbb{Z}) \setminus \{0\}$, to avoid the degenerate case whose public key is the identity), $\mathcal{P} = \mathcal{K} = G$, and

$$\text{KeyGen}: a \xleftarrow{\$} \mathcal{S}, \quad \text{DerivePublic}(\text{pp}, a) = g^a, \quad \text{Agree}(\text{pp}, a, B) = B^a.$$

Correctness is exactly Proposition 7.2: $\text{Agree}(\text{pp}, s, g^t) = (g^t)^s = g^{ts} = g^{st} = (g^s)^t = \text{Agree}(\text{pp}, t, g^s)$.

Remark 7.7 (The deployed interface generalises the base point). Zcash note encryption is specified against this abstraction with one generalisation: the deployed DerivePublic takes the base point as an *explicit argument* rather than fixing a protocol-wide generator, so that Sapling and Orchard derive keys over per-address *diversified bases* (protocol specification §“Key Agreement”). Those bases are used in §7.9; the address construction that produces them belongs to a later volume. Because the interface never inspects the underlying group, the same construction has carried over across three groups: Sprout’s Curve25519, Sapling’s Jubjub, and Orchard’s Pallas.

Remark 7.8 (The abstraction boundary, and what falls outside it). Any concrete scheme satisfying equation (1) can be swapped in without changing the surrounding logic: a prime-field group, an elliptic curve, or a post-quantum non-interactive key agreement such as the CSIDH group action. A *key-encapsulation mechanism* (KEM), by contrast, cannot satisfy the correctness equation: encapsulation requires the recipient’s public key before it can produce its ciphertext, so no counterparty-independent DerivePublic exists. The static–ephemeral one-flow *pattern* generalises to any KEM, with the encapsulation ciphertext in the role of the ephemeral public key, but only through a different interface.

7.4 Passive security and the Diffie–Hellman assumptions

A passive adversary, or eavesdropper, reads the channel without altering it; its view of an honest session is the transcript (g^a, g^b) , and §2.2 formalises its task in two ways. The *computational* problem is to produce the shared secret: writing $\text{Adv}_{\text{cdh}}(\mathcal{A})$ for the success probability of $\mathcal{A}$ in the

CDH game of Definition 2.4,

$$\text{Adv}_{\text{cdh}}(\mathcal{A}) = \Pr_{a,b}[\mathcal{A}(g^a, g^b) = g^{ab}].$$

The *decisional* problem is to distinguish (g^a, g^b, g^{ab}) from (g^a, g^b, g^c) with advantage $\text{Adv}_{\text{ddh}}(\mathcal{A})$ (Definition 2.5). The hierarchy $\text{DDH} \leq \text{CDH} \leq \text{DLP}$ is proved in §2.3 (Theorems 2.7 and 2.8), together with the pairing-based separation of DDH from CDH (Example 2.9).

Remark 7.9 (A concrete DDH break: the Legendre symbol). Take $G = \mathbb{F}_p^\times$, the *full* multiplicative group, with generator g . The Legendre symbol $\left(\frac{b}{p}\right)$ is efficiently computable by Euler’s criterion $b^{(p-1)/2} \bmod p$ and is multiplicative (*Math Guide*, §“Quadratic residues and the Euler criterion”). A generator of $\mathbb{F}_p^\times$ is a nonsquare, so $\left(\frac{g^a}{p}\right) = (-1)^a$: the symbol of a public share reveals the *parity* of its exponent. The parities of a and b determine the parity of ab , hence $\left(\frac{g^{ab}}{p}\right) = (-1)^{ab}$ is determined by the two observed symbols. The distinguisher computes the predicted symbol of the shared secret from (g^a, g^b) , compares it with the symbol of the third component, and outputs “Diffie–Hellman” on a match. On a genuine triple the match is certain; on a random triple the third component’s symbol is an independent uniform sign, matching with probability $1/2$. The distinguishing advantage is the constant $1/2$ —even though CDH in $\mathbb{F}_p^\times$ is believed hard. The CDH/DDH gap therefore does not depend on pairings; it already occurs in the multiplicative group of a prime field. The remedy is the prime-order rule of Remark 7.3: in a subgroup of odd prime order q , every element b is a square ($b = (b^{(q+1)/2})^2$), the symbol is identically $+1$, and it carries no information. This is one of the two reasons—the other being Pohlig–Hellman—that DDH is only ever assumed in prime-order groups.

Definition 7.10 (Passive security of a key-agreement scheme). For a key-agreement scheme Σ over pp and an adversary $\mathcal{A}$, the game $\text{KA-IND}_\Sigma^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger samples $s, t \leftarrow \text{KeyGen}(\text{pp})$ independently and computes the transcript $(\text{DerivePublic}(\text{pp}, s), \text{DerivePublic}(\text{pp}, t))$.
2. It sets $k_0 = \text{Agree}(\text{pp}, s, \text{DerivePublic}(\text{pp}, t))$, samples $k_1 \xleftarrow{\$} \mathcal{K}$ uniformly, draws a hidden bit $\beta \xleftarrow{\$} \{0, 1\}$, and sends the transcript together with k_β .
3. The adversary outputs a bit β' .

The advantage is $\text{Adv}_\Sigma^{\text{ka-ind}}(\mathcal{A}) = |\Pr[\beta' = 1 \mid \beta = 0] - \Pr[\beta' = 1 \mid \beta = 1]|$; the scheme is *passively secure* if every PPT adversary’s advantage is negligible in λ .

Theorem 7.11 (Passive security of Diffie–Hellman). *For the scheme $\text{DH}_{G,g}$ of Construction 7.6 with KeyGen uniform on $\mathbb{Z}/q\mathbb{Z}$, every adversary $\mathcal{A}$ satisfies*

$$\text{Adv}_{\text{DH}_{G,g}}^{\text{ka-ind}}(\mathcal{A}) = \text{Adv}_{\text{ddh}}(\mathcal{A}),$$

identically and with running time preserved exactly. Under the DDH assumption the shared secret of an honest ephemeral–ephemeral session is therefore pseudorandom given the transcript:

a passive adversary learns nothing about it that it could not have guessed about a random group element.

Proof. The two games are the same game. In KA-IND for $\text{DH}_{G,g}$, the transcript is (g^a, g^b) with a, b uniform and independent; the real key is $k_0 = g^{ab}$; and the uniform alternative $k_1 \xleftarrow{\$} G$ is exactly g^c for uniform independent c , because $\exp_g$ is a bijection. The challenge (g^a, g^b, k_β) is thus verbatim a sample of $\mathcal{D}_{\text{dh}}$ or $\mathcal{D}_{\text{rand}}$ according to β —the DDH distinguishing game of Definition 2.5. The identity map converts each adversary into the other, so the advantages coincide and no running time is added. $\square$

Remark 7.12 (Why pseudorandomness, not mere unpredictability). Every use of a Diffie–Hellman secret in this volume feeds it into a key-derivation function to produce a symmetric key. Hardness of CDH guarantees that the secret is hard to produce *in full*, not that it lacks efficiently predictable structure such as the Legendre leak of Remark 7.9. Hardness of DDH guarantees that the secret is indistinguishable from a random group element (Theorem 7.11): no predicate of it is efficiently computable from the transcript. In practice one often assumes only CDH and models the hash-based KDF as a random oracle: the derived key is then uniform unless the adversary queries the oracle at the hidden shared secret (Theorem 7.24). Either way the goal is a derived symmetric key indistinguishable from uniform, the hypothesis every theorem of Section 6 consumes.

7.5 Active attacks and the role of authentication

Passive security is the strongest guarantee the bare protocol offers; the following attack by an adversary who controls the channel shows that it does not suffice for a secure channel.

Definition 7.13 (Man-in-the-middle attack on unauthenticated Diffie–Hellman). An active adversary $\mathcal{M}$ controlling the channel between Alice and Bob runs two independent Diffie–Hellman sessions, one with each victim. It samples $m \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$ and computes g^m . When Alice sends $A = g^a$ intended for Bob, $\mathcal{M}$ intercepts it and forwards g^m to Bob in its place; when Bob replies $B = g^b$, the adversary intercepts it and forwards g^m to Alice. Alice computes $k_A = (g^m)^a = g^{am}$, which $\mathcal{M}$ also computes as A^m ; symmetrically Bob computes $k_B = (g^m)^b = g^{bm}$, which $\mathcal{M}$ computes as B^m .

Proposition 7.14 (The attack defeats unauthenticated Diffie–Hellman). *After the attack of Definition 7.13, Alice and Bob each hold a secret they believe is shared with the other but is shared instead with $\mathcal{M}$; and $\mathcal{M}$, knowing both g^{am} and g^{bm} , can decrypt, read, modify, and re-encrypt every subsequent message in each direction, transparently to both parties. No assumption on G —not even ideal hardness of the discrete logarithm problem—prevents this.*

Proof. Correctness of Diffie–Hellman (Proposition 7.2), applied once per session, gives agreement between Alice and $\mathcal{M}$ on g^{am} and between Bob and $\mathcal{M}$ on g^{bm} . The adversary knows its own exponent m and received both A and B , so it computes both secrets directly—no problem instance is ever inverted, which is why no hardness assumption is relevant. Neither victim verifies the *origin* of the group element they received; each transcript is a syntactically valid run of

Construction 7.1, so nothing in either party’s view distinguishes the attacked execution from an honest one. With k_A and k_B in hand, $\mathcal{M}$ decrypts traffic from Alice under k_A , re-encrypts it under k_B for Bob, and symmetrically in reverse, modifying it arbitrarily in between. $\square$

Passive security concerns the secrecy of the shared secret; active security additionally requires the authenticity of the public shares, since confidentiality gives no protection on a channel established with the wrong party. The bare protocol (Construction 7.1) provides no authentication—nothing in g^a or g^b identifies its sender—so the remedy must bind each public share to its sender’s identity, letting the recipient reject a substituted share.

Remark 7.15 (Three mechanisms supply authentication). Deployed remedies fall into three families, each external to the bare exchange.

1. *Signed Diffie–Hellman*. Each party signs its ephemeral share with a long-term signing key whose verification key is certified—by a certificate authority (a third party whose signature on the key the parties already trust) or by trust-on-first-use (accepting the first key seen and rejecting later changes). The adversary of Proposition 7.14 cannot forge Alice’s signature on g^m , so the substitution is rejected. Digital signatures are the subject of Section 8; this composition is the structure of authenticated TLS.
2. *Static keys that are themselves authenticated*. If Bob’s static public key $B = g^b$ reaches Alice authentically—a trusted directory, a pinned key—then a static–ephemeral exchange towards B already authenticates Bob to Alice: only the holder of b can complete it. This authenticates the *recipient*, not the sender. It is the model relevant to public-key encryption, and to shielded-note delivery, where the recipient’s address *is* an authenticated public key (§7.9).
3. *Out-of-band verification*. The parties compare a short fingerprint of the exchanged public keys over a channel that is authentic even if low-bandwidth—digits read aloud on a call, a QR code scanned in person.

In every case the key-agreement goal remains the passive guarantee of Theorem 7.11; authentication is a separate layer composed around the exchange, not a property of the group operation.

7.6 Elliptic-curve Diffie–Hellman

This subsection instantiates the preceding constructions on the elliptic-curve groups of the deployed protocol. Recall from the *Math Guide* (§“Elliptic curves”) that an elliptic curve E over a field F of characteristic neither 2 nor 3 is given in short Weierstrass form $y^2 = x^3 + ax + b$ with nonzero discriminant, and that its F -rational points $E(F)$ form a finite abelian group under chord-and-tangent addition with the point at infinity $\mathcal{O}$ as identity (*Math Guide*, §“The group structure of $E(\mathbb{F}_p)$ ”). The group is written additively, so exponentiation $g \mapsto g^x$ becomes *scalar multiplication* $P \mapsto [x]P$, computable in $O(\log x)$ point additions by double-and-add.

Construction 7.16 (ECDH). Fix an elliptic curve E/F , a base point $P \in E(F)$ of large prime order q generating $G = \langle P \rangle$, and cofactor $h = |E(F)|/q$. In the abstraction of Definition 7.5:

$$\text{KeyGen: } a \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}, \quad \text{DerivePublic}(a) = [a]P, \quad \text{Agree}(a, Q) = [a]Q.$$

The shared secret of parties with private keys a and b is $[a]([b]P) = [ab]P = [b]([a]P)$. The associated hard problems are the discrete-logarithm, CDH, and DDH definitions of Section 2 (Definitions 2.2, 2.4, and 2.5) read with $[x]P$ in place of g^x —the problems ECDLP (*Math Guide*, §“The elliptic-curve discrete logarithm problem”), ECCDH, and ECDDH of §2.6.

Proposition 7.17 (Correctness of ECDH). *The scheme of Construction 7.16 satisfies the key-agreement correctness condition (1).*

Proof. Scalar multiplication is cyclic-group exponentiation in additive notation: the group is abelian, and $\mathbb{Z}$ acts on $\langle P \rangle$ through $\mathbb{Z}/q\mathbb{Z}$ because $[q]P = \mathcal{O}$, so $[x]([y]P) = [xy]P$ for all integers x, y . Hence $\text{Agree}(a, [b]P) = [ab]P = [ba]P = \text{Agree}(b, [a]P)$, which is condition (1). $\square$

Remark 7.18 (Why elliptic curves). Well-chosen elliptic-curve groups admit only the generic $O(\sqrt{q})$ attacks of §2.4, whereas index calculus solves the discrete logarithm in $\mathbb{F}_p^\times$ in subexponential time (Remark 2.18); a 256-bit curve therefore delivers the security level that a multi-thousand-bit prime field requires (Example 2.17). That efficiency, together with the availability of curves engineered for cheap in-circuit arithmetic, is why the Pallas curve of §2.6 underlies Orchard.

Remark 7.19 (Subgroup and curve validation). A recipient computing $[a]Q$ on an attacker-supplied point Q must guard against two attacks, the elliptic-curve forms of the validation duty in Remark 7.3.

1. *Invalid-curve attack.* The received coordinates may satisfy not E ’s equation but that of a different, weaker curve which shares the same addition formulas (the chord and tangent slopes of the *Math Guide*, §“Explicit affine formulas”, involve a but never b); scalar multiplication then proceeds without error in the wrong group, whose order may be smooth. The implementation must check the curve equation on every incoming point.
2. *Small-subgroup attack.* When the cofactor $h > 1$, an adversary submits a point Q of small order dividing h , confining $[a]Q$ to a tiny subgroup and learning a modulo a small factor. The defences are *cofactor clearing*—multiply incoming points by h —or choosing a curve with $h = 1$.

Prime-order curves such as Pallas have $h = 1$, hence no small subgroup, and only the on-curve check remains. The deployed arithmetic enforces that check at deserialisation (Proposition 2.27): the compressed encoding cannot represent a point off the curve.

7.7 Hybrid public-key encryption

Hybrid public-key encryption (HPKE) composes three parts: key agreement in the static-ephemeral flow of §7.2, so that the recipient need not be online; a key-derivation function, which

turns the shared secret, a *structured group element* rather than a uniform string of key bits, into a symmetric key; and an authenticated cipher, which transports the message. The two symmetric ingredients are those of Section 6, recalled below in the shape the composition needs.

Definition 7.20 (KDF, recalled and specialised). In this section a *key-derivation function* is a deterministic function

$$\text{KDF}: \mathcal{K} \times \{0, 1\}^* \rightarrow \{0, 1\}^\ell,$$

mapping input keying material $k \in \mathcal{K}$ together with a context string to an ℓ -bit key. This specialises Definition 6.24 (§6.6) by omitting the salt and fixing the output length ℓ . For Theorem 7.24 we model the function as a random oracle (§3.4): distinct inputs then have independent uniform outputs, and security follows provided the adversary never queries the oracle at the hidden Diffie–Hellman secret and context. This is a source-specific random-oracle guarantee, not a generic claim that a deterministic function extracts uniform bits from every high-min-entropy source.

Definition 7.21 (AEAD, recalled). An *authenticated encryption scheme with associated data* with key space $\{0, 1\}^\ell$ —matching the KDF output—is a pair (Enc, Dec) with $\text{Enc}_K(N, ad, m) = c$ and $\text{Dec}_K(N, ad, c) \in \{m\} \cup \{\perp\}$, where N is a nonce, ad is associated data authenticated but not encrypted, and $\perp$ denotes rejection (Definition 6.13). Correctness: $\text{Dec}_K(N, ad, \text{Enc}_K(N, ad, m)) = m$. Security is authenticated-encryption security (Definition 6.15): encryptions of any two equal-length messages are CPA-indistinguishable (Definition 6.4), and no adversary can produce a fresh ciphertext decrypting to anything but $\perp$ (ciphertext integrity, Definition 6.14)—confidentiality and integrity simultaneously.

Construction 7.22 (DH-based hybrid public-key encryption). Fix ECDH over $G = \langle P \rangle$ of prime order q (Construction 7.16, in the interface of Definition 7.5), a secure KDF, and a secure AEAD as above. The recipient holds a static key pair $(b, B = [b]P)$ with B published. To *encrypt* a message m to B :

1. sample an ephemeral $e \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$ and set $E_{\text{pk}} = [e]P$;
2. compute the shared secret $S = \text{Agree}(e, B) = [eb]P$;
3. derive the symmetric key $K = \text{KDF}(S, \text{context})$, where *context* includes E_{pk} and the protocol’s labels;
4. output (E_{pk}, c) with $c = \text{Enc}_K(N, ad, m)$ for a fixed or derived nonce N —fixed is safe because each K is used once.

To *decrypt* (E_{pk}, c) with b : compute $S' = [b]E_{\text{pk}}$, derive $K' = \text{KDF}(S', \text{context})$ with E_{pk} read from the ciphertext, and output $\text{Dec}_{K'}(N, ad, c)$, which is m on an honest ciphertext; on a tampered one, ciphertext integrity makes the output $\perp$ except with negligible probability. Anyone holding B can encrypt; only the holder of b can decrypt.

Theorem 7.23 (Correctness of hybrid encryption). *For every message m and every honestly generated ciphertext (E_{pk}, c) of Construction 7.22, decryption returns m .*

Proof. Correctness of ECDH (Proposition 7.17) gives $S' = [b]([e]P) = [be]P = [eb]P = [e]B = S$. The KDF is deterministic and both sides supply the same context—the recipient recovers E_{pk} from the ciphertext itself—so $K' = K$. Correctness of the AEAD (Definition 7.21) finishes: $\text{Dec}_K(N, ad, \text{Enc}_K(N, ad, m)) = m$. $\square$

Theorem 7.24 (Passive security of hybrid encryption). *Model the KDF as a random oracle (Definition 3.13) and let the AEAD be secure in the sense of Definition 6.15. Consider a passive adversary $\mathcal{A}$ who sees the public key B and a single challenge ciphertext: it names two equal-length messages m_0, m_1 , receives the encryption of m_β for a hidden uniform bit β , and guesses β , with CPA advantage $\text{Adv}_{\text{cpa}}(\mathcal{A})$ defined as in Definition 6.4. Under CDH in G no PPT adversary has non-negligible advantage; concretely, if $\mathcal{A}$ makes at most q_H random-oracle queries, then*

$$\text{Adv}_{\text{cpa}}(\mathcal{A}) \leq 2q_H \cdot \text{Adv}_{\text{cdh}}(\mathcal{B}) + \text{Adv}_{\text{ae}}(\mathcal{C}),$$

for explicit reductions $\mathcal{B}$ against CDH and $\mathcal{C}$ against the AEAD's authenticated-encryption security (of which the proof consumes only the confidentiality half, so $\text{Adv}_{\text{ae}}(\mathcal{C})$ is the CPA advantage of Definition 6.4 read against the AEAD), each running in essentially the same time as $\mathcal{A}$.

Proof. The proof is a game hop.

Game 0 is the real CPA game: the challenge ciphertext is (E_{pk}, c) with $E_{pk} = [e]P$, $S = [eb]P$, $K = \text{KDF}(S, \text{context})$, and $c = \text{Enc}_K(N, ad, m_\beta)$. Say $\mathcal{A}$ wins a game when its guess β' equals β .

Game 1 replaces K by an independent uniform ℓ -bit string, leaving everything else unchanged.

The hop. In the random oracle model the two games proceed identically unless the adversary queries the oracle at exactly the point $(S, \text{context}) = ([eb]P, \text{context})$: until that query, the oracle's value there is a uniform string it has never seen, which is precisely what Game 1 supplies. Producing $S = [eb]P$ from the view $(B, E_{pk}) = ([b]P, [e]P)$ is exactly solving CDH. The reduction $\mathcal{B}$ embeds its CDH challenge as (B, E_{pk}) and simulates Game 1 around it: it draws β and a uniform ℓ -bit key K itself, encrypts m_β under K to form the challenge ciphertext, and implements the oracle by lazy sampling—a fresh uniform string for each new query, the stored answer on a repeated one, so that the oracle remains a consistent function. The simulation is a perfect run of Game 1, and the critical query occurs in it with the same probability as in Game 0, the two games being identical until it happens. One obstruction remains: in a pairing-free group $\mathcal{B}$ cannot recognise which of the queries carries $[eb]P$ (Example 2.9 shows recognising it is the DDH-test a gap group would supply). So $\mathcal{B}$ picks one of the at most q_H queries uniformly at random and outputs its first component; whenever the critical query occurs, $\mathcal{B}$'s choice is correct with probability at least $1/q_H$. Hence

$$|\Pr[\mathcal{A} \text{ wins Game 0}] - \Pr[\mathcal{A} \text{ wins Game 1}]| \leq q_H \cdot \text{Adv}_{\text{cdh}}(\mathcal{B}).$$

The factor q_H is a guessing-type security loss in the tightness taxonomy of Definition 1.22. Assuming *gap*-CDH—CDH hardness even given a DDH-decision oracle, which would let $\mathcal{B}$ test each query—restores tightness; the loose bound is stated because it is the bound plain CDH yields.

Game 1 analysed. In Game 1 the AEAD key is uniform and independent of everything else in the adversary's view, which is exactly the hypothesis of the AEAD's confidentiality game. The reduction $\mathcal{C}$ forwards (m_0, m_1) to its AEAD challenger, embeds the returned ciphertext as c alongside a self-generated E_{pk} , and relays $\mathcal{A}$'s guess; the AEAD challenger's hidden bit plays β , so $|\Pr[\mathcal{A} \text{ wins Game 1}] - \frac{1}{2}| = \frac{1}{2} \text{Adv}_{ae}(\mathcal{C})$.

Accounting. The difference-form advantage of Definition 6.4 is twice the bit-guessing advantage $|\Pr[\mathcal{A} \text{ wins}] - \frac{1}{2}|$ of Definition 1.15, so the triangle inequality across the hop gives

$$\text{Adv}_{cpa}(\mathcal{A}) \leq 2(q_H \cdot \text{Adv}_{cdh}(\mathcal{B}) + \frac{1}{2} \text{Adv}_{ae}(\mathcal{C})) = 2q_H \cdot \text{Adv}_{cdh}(\mathcal{B}) + \text{Adv}_{ae}(\mathcal{C}),$$

the stated bound.

Alternative hypothesis. The random oracle can be replaced, but not by Definition 6.24 alone: that guarantee is stated for a uniform independent salt, which the specialised KDF of Definition 7.20 omits, and the definition denies any generic unsalted guarantee. The replacement needs an explicit hypothesis on the deployed function—the narrower premise Definition 6.24 allows for unsalted use: for a *uniform* group element S , the output $\text{KDF}(S, \text{context})$ is computationally indistinguishable from a uniform ℓ -bit string given the context. Under that hypothesis and DDH in place of CDH, Theorem 7.11 first replaces the shared secret by a uniform group element at cost Adv_{ddh} , and the stated indistinguishability then replaces K by a uniform string—no oracle and no q_H factor, at the price of the stronger decisional assumption on the group and an explicit pseudorandomness assumption on the KDF. $\square$

Remark 7.25 (Necessity of each component). Omitting any component of Construction 7.22 breaks the composition.

- *Without the KDF*, one would key the cipher with the raw group element S . Its bit-representation is non-uniform—curve points do not fill the encoding space—and in non-DDH groups it is partially predictable, the Legendre-symbol leakage of Remark 7.9 being the classical instance. The KDF standardises the format, extracts uniform bits, and, by folding E_{pk} into its context, ties the derived key to this particular ephemeral share.
- *Without AEAD integrity*, a passively confidential but malleable cipher would let an active adversary tamper with c undetectably—the stream ciphers of Section 6 are fully XOR-malleable. Ciphertext integrity (Definition 6.14) makes any modification decrypt to $\perp$ except with negligible probability, giving the symmetric payload its chosen-ciphertext robustness.
- *Without the ephemeral key*—a static-static variant—every message to B would begin from the same shared secret. The fresh per-message ephemeral, with E_{pk} bound into the KDF context, gives each ciphertext separate key material, so nothing relies on nonce uniqueness under a reused key. What it does *not* supply is forward secrecy against later compromise of the recipient's static key b : from a recorded E_{pk} , an attacker who learns b recomputes $[b]E_{pk}$ and with it the ciphertext's key. Erasing e provides only the sender-side guarantee that the key derives from no stored secret of the sender.

7.8 Key privacy

Theorem 7.24 bounds what an eavesdropper learns about the message, not about the recipient: a scheme may hide the message perfectly and still reveal the recipient's key in every ciphertext, which an observer of the ledger can test against every public key it can enumerate. *Key privacy*, due to Bellare, Boldyreva, Desai, and Pointcheval, rules this out. We define it for a generic public-key scheme and prove it for ElGamal encryption, the bare static-ephemeral flow with the message folded in, on whose key privacy later volumes rely.

Definition 7.26 (Key privacy under chosen-plaintext attack, IK-CPA). A *public-key encryption scheme* Π over pp consists of a randomised $\text{KeyGen}(\text{pp}) \rightarrow (\text{sk}, \text{pk})$, a randomised $\text{Enc}_{\text{pk}}(m) \rightarrow c$, and a deterministic $\text{Dec}_{\text{sk}}(c)$ with $\text{Dec}_{\text{sk}}(\text{Enc}_{\text{pk}}(m)) = m$ for every key pair in the support of KeyGen and every message m . For an adversary $\mathcal{A}$ the game $\text{IK-CPA}_{\Pi}^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger samples $(\text{sk}_0, \text{pk}_0), (\text{sk}_1, \text{pk}_1) \leftarrow \text{KeyGen}(\text{pp})$ independently and sends $(\text{pk}_0, \text{pk}_1)$.
2. The adversary outputs a message m .
3. The challenger draws a hidden bit $\beta \xleftarrow{\$} \{0, 1\}$ and sends $c \leftarrow \text{Enc}_{\text{pk}_{\beta}}(m)$.
4. The adversary outputs a bit β' .

The advantage is $\text{Adv}_{\Pi}^{\text{ik-cpa}}(\mathcal{A}) = |\Pr[\beta' = 1 \mid \beta = 0] - \Pr[\beta' = 1 \mid \beta = 1]|$; the scheme is *key-private* (IK-CPA) if every PPT adversary's advantage is negligible in λ . The message is the adversary's own, so nothing about it is hidden; what the game hides is the key.

Construction 7.27 (ElGamal encryption). Fix $\text{pp} = (G, P, q)$ as in Construction 7.16, written additively, and take group elements as messages; write EG for the scheme. A key pair carries its own base H : in the *fixed-base* variant $H = P$; in the *per-key-base* variant KeyGen samples $H \xleftarrow{\$} G \setminus \{\mathcal{O}\}$, the generalised interface of Remark 7.7.

- KeyGen : choose H as above and $x \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$; output $\text{sk} = x$ and $\text{pk} = (H, Y)$ with $Y = [x]H$.
- $\text{Enc}_{(H,Y)}(M)$: sample $r \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}$; output $(R, C) = ([r]H, [r]Y + M)$.
- $\text{Dec}_x(R, C)$: output $C - [x]R$.

Correctness is the scalar identity in the proof of Proposition 7.17, valid for any base in G : $[x]R = [x]([r]H) = [r]([x]H) = [r]Y$, so $C - [x]R = M$. The scheme is the static-ephemeral flow of §7.2 with the shared secret $[rx]H$ spent as a one-time pad on a group element; the fixed-base variant is ElGamal's original scheme transposed from $\mathbb{F}_p^*$ to a prime-order group.

Theorem 7.28 (ElGamal is key-private under DDH). *For ElGamal over G (Construction 7.27, either base variant), every adversary $\mathcal{A}$ yields two distinguishers $\mathcal{B}_0, \mathcal{B}_1$ against DDH in G (Definition 2.5, read with $[x]P$ in place of g^x as in Construction 7.16), each running in the time of $\mathcal{A}$ plus one key generation and four scalar multiplications, with*

$$\text{Adv}_{\text{EG}}^{\text{ik-cpa}}(\mathcal{A}) \leq \text{Adv}_{\text{ddh}}(\mathcal{B}_0) + \text{Adv}_{\text{ddh}}(\mathcal{B}_1).$$

Under the DDH assumption ElGamal is therefore IK-CPA: a ciphertext is computationally independent of the public key it was made for.

Proof. The proof is a hybrid through a game that uses no key at all. Write H_0 and H_1 for the IK-CPA game with β fixed to 0 and to 1, and p_0, p_1 for the probability that $\mathcal{A}$ outputs 1 in each, so that $\text{Adv}_{\text{EG}}^{\text{ik-cpa}}(\mathcal{A}) = |p_0 - p_1|$. Let H_* be the game whose challenger, after the same key generation and the same message M from $\mathcal{A}$, answers with $(R, C) = (U_1, U_2 + M)$ for $U_1, U_2 \xleftarrow{\$} G$ independent and uniform, and let p_* be the corresponding probability. The challenge of H_* is computed without reference to either key, and the triangle inequality $|p_0 - p_1| \leq |p_0 - p_*| + |p_* - p_1|$ reduces the claim to bounding each hop by a DDH advantage.

The hop $H_0 \rightarrow H_$.* The distinguisher $\mathcal{B}_0$ receives a triple $(X, Y, Z) = ([a]P, [b]P, [c]P)$ with $c = ab$ or c uniform. It samples $s \xleftarrow{\$} (\mathbb{Z}/q\mathbb{Z})^\times$ in the per-key-base variant and sets $s = 1$ in the fixed-base variant, plants the challenge in key 0 as

$$H_0 = [s]P, \quad Y_0 = [s]X = [a]H_0,$$

generates $(x_1, (H_1, Y_1))$ honestly, and hands $\mathcal{A}$ the pair $((H_0, Y_0), (H_1, Y_1))$. For invertible s the base $[s]P$ is uniform on $G \setminus \{\mathcal{O}\}$, and $Y_0 = [a]H_0$ with a uniform is an honest public key whose private key a the distinguisher never learns and does not need, since the game offers no decryption. On receiving M it replies with

$$(R, C) = ([s]Y, [s]Z + M)$$

and outputs $\mathcal{A}$'s bit. If $c = ab$, then $R = [sb]P = [b]H_0$ and $[s]Z = [sab]P = [b]([a]H_0) = [b]Y_0$, so (R, C) is an honest encryption under key 0 with ephemeral $r = b$ uniform: the view of $\mathcal{A}$ is exactly H_0 . If c is uniform, then $[s]Z = [sc]P$ is uniform in G and independent of everything else because s is invertible, and $R = [b]H_0$ is uniform in G and independent of everything else because H_0 generates G and b is independent of a, c, s , of key 1, and of M , which $\mathcal{A}$ chose before seeing R : the view is exactly H_* . Hence $|p_0 - p_*| = \text{Adv}_{\text{ddh}}(\mathcal{B}_0)$.

The hop $H_ \rightarrow H_1$* is the same with the challenge planted in key 1 and key 0 generated honestly, giving $\mathcal{B}_1$ with $|p_* - p_1| = \text{Adv}_{\text{ddh}}(\mathcal{B}_1)$. Each distinguisher performs one honest key generation and the four scalar multiplications $[s]P, [s]X, [s]Y, [s]Z$ beyond running $\mathcal{A}$. Summing the two hops gives the bound. $\square$

Remark 7.29 (Key privacy of note encryption and diversified addresses). The hybrid argument carries over unchanged to Construction 7.22: planting the challenge in the recipient's key and the ephemeral share exactly as above makes the shared secret S uniform, and once it is, the pair (E_{pk}, c) with $c = \text{Enc}_{\text{KDF}(S, \text{context})}(N, \text{ad}, m)$ is computed without reference to the recipient's key—*provided* the KDF context names no recipient key, which it must not. The deployed

KDF takes $(\text{sharedSecret}, \text{epk})$ (§6.6): the shared secret is the keying material, the context is epk under a personalisation label, and no key of the recipient enters. The protocol specification requires the composed note-encryption scheme to be key-private (§“Key Derivation”), so that a note ciphertext reveals nothing about the address to which it was sent. Key privacy also supplies the group-element part of diversified-address unlinkability. A diversified address $(G_d, \text{pk}_d) = (G_d, [\text{ivk}] G_d)$ (Remark 7.30) is a per-key-base ElGamal public key, and a fresh address of the same ivk over a new base G_d' has, when that base is modelled as a uniform group element, the distribution of an encryption of $\mathcal{O}$ under that key; telling which of two authorities a third address belongs to is then the IK-CPA game, whose advantage Theorem 7.28 bounds. The protocol specification sets up that correspondence, modelling the group hash behind the base as a random oracle (§“DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions”); the resulting unlinkability requirement is stated in a later volume.

7.9 Shielded-note delivery as deployed hybrid encryption

Shielded-note delivery in the deployed protocol is the composition assembled in this section, run in band on the public ledger.

Remark 7.30 (Note delivery is HPKE on the ledger). A shielded address embeds a static public key $\text{pk}_d = [\text{ivk}] G_d$ on the protocol’s curve (Pallas, in Orchard), where the private scalar ivk is the recipient’s incoming viewing key. The base G_d is not a protocol-wide generator but a per-address *diversified base*, the hash-to-curve image (§3.7) of the address’s diversifier. The generalised interface of Remark 7.7 accommodates such per-address bases; the key hierarchy and address encoding behind them belong to a later volume. The sender runs Construction 7.22 over that base, with one deployed refinement: the ephemeral scalar esk is not sampled directly but derived by $\text{PRF}^{\text{expand}}$ (Remark 5.14) from the note’s random seed (ZIP 212), so that the uniform-ephemeral hypothesis of Theorem 7.24 is met through the PRF’s pseudorandomness and the recipient can later re-derive esk from the delivered seed. The sender publishes $\text{epk} = [\text{esk}] G_d$ in the transaction, derives

$$K = \text{KDF}([\text{esk} \cdot \text{ivk}] G_d, \text{epk})$$

by the personalised BLAKE2b call of §6.6, and AEAD-encrypts the note plaintext—a version byte, the recipient’s diversifier, the value, the note’s random seed, and the memo—under K with ChaCha20-Poly1305 (Construction 6.18), attaching the ciphertext to the transaction. The nonce is fixed to zero and the associated data is empty, safe because each K keys exactly one encryption (protocol specification §“Encryption (Sapling and Orchard)”).

Chain scanners see (epk, c) in every shielded output, but lacking ivk they learn nothing (Theorem 7.24). The recipient *trial-decrypts*: for each incoming transaction it derives $K' = \text{KDF}([\text{ivk}] \text{epk}, \text{epk})$ and keeps, as the notes addressed to it, those outputs whose AEAD tag verifies *and* whose plaintext passes two consistency checks: the note commitment recomputed from it must match the commitment on chain, and the esk re-derived from its random seed must reproduce the published epk (protocol specification §“Decryption using an Incoming Viewing Key (Sapling and Orchard)”). By the static-ephemeral flow of §7.2, delivery is non-interactive: the recipient need never have been online when the note was sent. Authenticity of the static key—the concern of §7.5—is supplied by mechanism (2) of Remark 7.15: the address is part

of the recipient’s verified payment credential, so no man-in-the-middle substitution of the key is possible; and, as noted there, this authenticates the recipient rather than the sender. The complementary guarantees on the note’s *contents*—that the value is well-formed and conserved, that the commitment on chain matches the plaintext delivered—come not from the encryption but from the accompanying zero-knowledge proof.

The signatures that authorise the transaction carrying the note ciphertext are the subject of the next section.

8 Digital signatures

This section defines digital signatures and their security against a *forger*: an adversary that may obtain the signer’s signature on messages of its choosing and succeeds if it then exhibits, without the secret key, a valid signature the signer never made. In Zcash a forged spend authorisation spends coins the forger does not control, and a forged balance certificate creates value. The section derives the Schnorr signature from an identification protocol, the first *proof of knowledge* of this volume, proves it unforgeable in the random oracle model (Theorem 8.18), and specialises it to the two RedDSA signatures of Orchard (protocol specification § 5.4.7): the *spend authorisation signature*, whose key pair can be *re-randomised* so that successive uses are unlinkable (protocol specification § 4.15), and the *binding signature*, which certifies the consistency of a transaction’s value commitments and proves knowledge of a discrete logarithm (protocol specification § 4.14). Signatures are the last primitive of this volume built directly on the discrete logarithm, and lead to the proof systems of Section 9.

8.1 Signatures versus MACs

The message authentication codes of §6.3 prevent forgery only between parties who share a secret key: verification runs Vrfy_k , so whoever can *check* a tag can also *compute* one, and a tag is no evidence to a third party. A transaction’s authorisation must instead be checkable by every full node, none of whom may be able to forge it, and the check must be transferable: a block, once validated, convinces every other node. A *digital signature* provides this with two keys: signing uses a secret key sk , verification a matching *public* key pk . Anybody can verify; only the holder of sk can sign; and because verification needs no secret, a valid signature is *non-repudiable* evidence that the holder of sk signed. The cost is a computational assumption: the Carter–Wegman MAC of §6.3 is information-theoretically secure, whereas every signature scheme in this section rests on the hardness of the discrete logarithm.

Conventions for this section. Throughout, G is a cyclic group of prime order q , written additively, with a fixed generator G ; scalar multiplication is $[x]P$ for $x \in \mathbb{F}_q$ and $P \in G$. Concretely G is the Pallas group of §2.6, whose order is the 255-bit prime q , so scalars are elements of the field $\mathbb{F}_q$. Because G has prime order and G generates it, the map $x \mapsto [x]G$ is a bijection $\mathbb{F}_q \rightarrow G$: the discrete logarithm of any $P \in G$ to base G always *exists* and is *unique*. Hardness of computing it is the DLog assumption of §2.1, transcribed additively.

8.2 Syntax and security goal

Definition 8.1 (Digital signature scheme). A *digital signature scheme* is a triple of PPT algorithms $\Pi = (\text{KeyGen}, \text{Sign}, \text{Verify})$ with, for each security parameter λ , a message space $\mathcal{M}_\lambda$:

- $\text{KeyGen}(1^\lambda)$ outputs a key pair (pk, sk) ;
- $\text{Sign}(sk, m)$, for $m \in \mathcal{M}_\lambda$, outputs a signature σ ;
- $\text{Verify}(pk, m, \sigma)$ is deterministic and outputs a bit.

Correctness requires that for every λ , every (pk, sk) in the support of $\text{KeyGen}(1^\lambda)$, and every $m \in \mathcal{M}_\lambda$,

$$\Pr[\text{Verify}(pk, m, \text{Sign}(sk, m)) = 1] = 1,$$

the probability over the coins of Sign . Probability-1 correctness is called *perfect*; a scheme achieving only overwhelming-probability correctness tolerates a $\text{negl}(\lambda)$ failure rate. Every scheme in this section is perfectly correct.

The security goal transcribes the MAC forgery game of Definition 6.8 into the public-key setting: the adversary additionally *receives* the verification key, and its oracle access models a signer who can be induced to sign arbitrary messages.

Definition 8.2 (Existential unforgeability under chosen-message attack). For a signature scheme Π and adversary $\mathcal{A}$, the game $\text{Forge}_\Pi^{\mathcal{A}}(\lambda)$ runs as follows.

1. The challenger runs $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$.
2. The adversary $\mathcal{A}$ runs on input pk with oracle access to $\text{Sign}(sk, \cdot)$; let Q be the set of messages it submits to the oracle, and let (m^*, σ^*) be its output.
3. The game outputs 1 iff $\text{Verify}(pk, m^*, \sigma^*) = 1$ and $m^* \notin Q$.

Scheme Π is *existentially unforgeable under chosen-message attack* (EUF-CMA) if for every PPT $\mathcal{A}$,

$$\text{Adv}_\Pi^{\text{forge}}(\mathcal{A}, \lambda) := \Pr[\text{Forge}_\Pi^{\mathcal{A}}(\lambda) = 1] \in \text{negl}(\lambda).$$

The oracle access is *adaptive* and bounded only by the adversary's polynomial running time: the adversary may choose each query after seeing all previous signatures. The forgery is *existential*: the adversary wins with a valid signature on *any* message never submitted to the oracle, however meaningless.

Definition 8.3 (Strong unforgeability). The *strong* variant, sUF-CMA, replaces the winning condition $m^* \notin Q$ by: the pair (m^*, σ^*) was not among the message–signature pairs returned by the oracle.

Plain EUF-CMA permits a scheme in which, given one valid signature, anyone can cheaply compute a second, different valid signature on the same message. Such *malleability* is harmless

when a signature only certifies origin, but breaks any mechanism that uses a signature as a unique identifier, such as an anti-replay check that identifies a message by its signature.

8.3 The hash-and-sign paradigm

The standard schemes—RSA, Schnorr, ECDSA—sign only fixed-size inputs: an element of $\mathbb{Z}/N\mathbb{Z}$ for RSA, a scalar in $\mathbb{F}_q$ for Schnorr and ECDSA. Arbitrary messages are hashed first.

Theorem 8.4 (Hash-and-sign). *Let Π' be a signature scheme with message space $\mathcal{Y}_\lambda$, and let $H: \{0, 1\}^* \rightarrow \mathcal{Y}_\lambda$ be drawn from a collision-resistant family (Definition 3.7). Define Π with message space $\{0, 1\}^*$ by*

$$\text{Sign}(sk, m) := \text{Sign}'(sk, H(m)), \quad \text{Verify}(pk, m, \sigma) := \text{Verify}'(pk, H(m), \sigma).$$

If Π' is EUF-CMA and H is collision resistant, then Π is EUF-CMA; concretely, for every adversary $\mathcal{A}$ against Π there are comparably efficient $\mathcal{B}_1$ against H and $\mathcal{B}_2$ against Π' with

$$\Pr[\mathcal{A} \text{ wins}] \leq \text{Adv}_H^{\text{col}}(\mathcal{B}_1) + \text{Adv}_{\Pi'}^{\text{forge}}(\mathcal{B}_2).$$

Proof. Let (m^*, σ^*) be a valid forgery of $\mathcal{A}$ on a fresh message $m^* \notin Q$, and distinguish two cases according to its digest. *Case 1: the digest is reused*, i.e. $H(m^*) = H(m)$ for some queried $m \in Q$. Then $m^* \neq m$ but their digests agree, so the pair (m^*, m) is a collision in H ; the reduction $\mathcal{B}_1$, which generates a key pair with KeyGen' itself and answers $\mathcal{A}$'s signing queries by signing digests under that secret key—the collision game grants no signing oracle—outputs it. *Case 2: the digest is fresh*, i.e. $H(m^*)$ differs from the digest of every queried message. Then $(H(m^*), \sigma^*)$ is a valid forgery against Π' on a message never submitted to its signing oracle, and $\mathcal{B}_2$ outputs it. The two cases are exhaustive, so the union bound on the winning event gives the stated inequality. $\square$

Remark 8.5 (Inputs of the hash in hash-and-sign). In all the Schnorr-type schemes below, the hash absorbs more than the message: the public key and the per-signature commitment value enter the hash alongside m . Hashing the public key prevents key-substitution (duplicate-signature) attacks, in which a signature valid under one key is exhibited as valid under another; hashing the commitment is exactly what the Fiat–Shamir transform of §8.5 requires. “Hash-and-sign” in practice thus hashes more than the message, and Theorem 8.4 captures only its message-compression part.

8.4 The Schnorr identification protocol

Every Schnorr-type scheme in the rest of this section derives from an interactive protocol in which a prover convinces a verifier that it *knows* the discrete logarithm of a public point without revealing anything about it. The protocol is the canonical *Sigma-protocol*: a three-move, public-coin, honest-verifier zero-knowledge (HVZK) proof of knowledge, a notion developed for arbitrary relations in Section 9.

The object of the protocol is the discrete-log relation

$$R_{\text{dl}} = \{ (X, x) \in \mathbb{G} \times \mathbb{F}_q : X = [x]G \} :$$

the *statement* is a public point X , the *witness* its discrete logarithm x .

Construction 8.6 (Schnorr identification). Fix $G = \langle G \rangle$ of prime order q . The prover holds a secret key $x \in \mathbb{F}_q$; both parties hold the public key $X = [x]G$. The protocol has three moves.

1. *Commitment*. The prover samples $r \xleftarrow{\$} \mathbb{F}_q$ uniformly and sends $R = [r]G$. We call R the *commitment* (or *nonce point*) and r the *nonce*.
2. *Challenge*. The verifier samples $c \xleftarrow{\$} \mathbb{F}_q$ uniformly and sends c .
3. *Response*. The prover sends $s = r + cx$ in $\mathbb{F}_q$.

The verifier accepts iff

$$[s]G = R + [c]X. \quad (2)$$

A triple (R, c, s) satisfying (2) is an *accepting transcript* for X . The verifier's only message is a uniformly random value chosen independently of R , so the protocol is *public-coin*: the verifier keeps no secrets and exercises no discretion.

Proposition 8.7 (Perfect completeness). *If prover and verifier are honest, the verifier accepts with probability 1.*

Proof. With $R = [r]G$, $X = [x]G$ and $s = r + cx$, linearity of scalar multiplication gives

$$[s]G = [r + cx]G = [r]G + [cx]G = R + [c]([x]G) = R + [c]X,$$

which is exactly (2). The computation holds identically for every choice of r and c . $\square$

Theorem 8.8 (2-special soundness). *There is an efficient extractor that, given two accepting transcripts (R, c, s) and (R, c', s') for the same statement X with the same commitment R but distinct challenges $c \neq c'$, outputs the discrete logarithm x of X .*

Proof. Both transcripts satisfy (2); subtracting the two equations in G cancels R :

$$[s - s']G = [c - c']X = [(c - c')x]G.$$

Since G generates a group of prime order q , equality of the two multiples forces the equality of scalars $s - s' = (c - c')x$ in $\mathbb{F}_q$. The hypothesis $c \neq c'$ makes $c - c'$ a nonzero element of the field $\mathbb{F}_q$, hence invertible, so the extractor outputs

$$x = (s - s') (c - c')^{-1} \in \mathbb{F}_q.$$

Verification that $X = [x]G$ is direct, and the whole computation costs a single field inversion. $\square$

Remark 8.9 (From special soundness to knowledge, by rewinding). Suppose a (possibly dishonest) prover makes the verifier accept with probability ε noticeably larger than $1/q$ —the probability of guessing in advance the single challenge for which a prepared response works. A rewinding extractor runs the prover once: it records the commitment R , supplies a uniform challenge c , and

records whether the response is accepted. It then *rewinds* the prover to the moment immediately after it sent R —restoring its internal state, which is possible because the rewinding extractor runs the prover as a subroutine—and supplies a fresh independent challenge c' . A standard averaging argument shows that with probability roughly ε^2 , both runs accept and $c \neq c'$; the argument is the *heavy-row lemma*, an instance of Markov's inequality applied to the matrix whose rows are the prover's random coins and whose columns are challenges (*Math Guide*, §“Random variables and expectation”). The two accepting transcripts share R , so the extractor of Theorem 8.8 recovers x . Making the verifier accept with non-trivial probability is therefore, up to rewinding, the same as *knowing* x . The same rewinding argument, sharpened into the forking lemma, carries the signature analysis of §8.6.

Theorem 8.10 (Special honest-verifier zero knowledge). *There is an efficient simulator Sim that, on input a statement $X \in \mathbb{G}$ and a challenge $c \in \mathbb{F}_q$, outputs an accepting transcript (R, c, s) distributed identically to an honest execution conditioned on the challenge being c . Consequently, for a uniform challenge, the distribution of simulated transcripts coincides exactly with the distribution of real ones.*

Proof. The simulator computes the transcript *in reverse*: it samples $s \xleftarrow{\$} \mathbb{F}_q$ uniformly, sets

$$R := [s]G - [c]X,$$

and outputs (R, c, s) —accepting by construction, since (2) rearranges to exactly this definition of R .

Fix the challenge to c and compare distributions. In a real honest transcript, r is uniform on $\mathbb{F}_q$, $R = [r]G$, and $s = r + cx$; the map $r \mapsto r + cx$ is a translation of $\mathbb{F}_q$, hence a bijection, so s is uniform on $\mathbb{F}_q$, and $R = [s - cx]G = [s]G - [c]X$ is the same deterministic function of s that the simulator uses. The real transcript *is*: sample s uniform, set $R = [s]G - [c]X$ —precisely the simulated distribution. The two distributions are identical, not merely statistically close, and averaging over a uniform c preserves the identity. $\square$

Remark 8.11 (Why honest-verifier zero knowledge suffices). The simulator of Theorem 8.10 matches the real distribution only when the challenge is sampled independently of R . A cheating verifier could instead choose c as a function of R , and against such a verifier the argument for the simulator—which commits to R only *after* seeing c —no longer applies. For the signature schemes of this section honest-verifier zero knowledge suffices: after the Fiat–Shamir transform the challenge is a *hash* of R , and in the random oracle model that hash behaves like an honest, R -independent coin. The simulator also *proves* privacy: since anyone, without x , can produce transcripts with the true distribution, a transcript carries no information about x beyond what the statement X already reveals.

8.5 From identification to signature via the Fiat–Shamir transform

An identification protocol convinces one verifier in one interactive execution; a signature must be verifiable by any party, at any later time, without interaction. The *Fiat–Shamir transform* makes the prover compute the challenge itself, as a hash of everything the verifier would have seen before

issuing it and of the message m . It converts *any* Sigma-protocol into a non-interactive proof, and, with a message in the hash, into a signature. This subsection treats the Schnorr instance; the general transform and its failure modes are treated in Section 9 (§“The Fiat–Shamir transform: from interactive to non-interactive”).

Construction 8.12 (Schnorr signature). Fix $G = \langle G \rangle$ of prime order q and a hash function $H: \{0, 1\}^* \rightarrow \mathbb{F}_q$. Messages are arbitrary bit strings.

- $\text{KeyGen}(1^\lambda)$: sample $x \xleftarrow{\$} \mathbb{F}_q$, set $X = [x]G$, output $(pk, sk) = (X, x)$.
- $\text{Sign}(x, m)$: sample $r \xleftarrow{\$} \mathbb{F}_q$, set $R = [r]G$, compute the challenge

$$c = H(R \parallel X \parallel m),$$

set $s = r + cx$ in $\mathbb{F}_q$, and output $\sigma = (R, s)$.

- $\text{Verify}(X, m, (R, s))$: recompute $c = H(R \parallel X \parallel m)$ and accept iff $[s]G = R + [c]X$.

Proposition 8.13 (Perfect correctness). *The Schnorr signature scheme is perfectly correct.*

Proof. For an honest signature (R, s) with $R = [r]G$, $c = H(R \parallel X \parallel m)$ and $s = r + cx$, the verifier recomputes the same c —the challenge is a deterministic function of (R, X, m) , all of which the verifier holds—and the check $[s]G = R + [c]X$ holds by the completeness computation of Proposition 8.7. $\square$

Remark 8.14 (Two conventions). Construction 8.12 makes two presentational choices.

1. The signature transmits (R, s) and omits c , which the verifier recomputes. The space-saving variant transmits (c, s) instead: the verifier recovers $R = [s]G - [c]X$ (the simulator’s equation, read as reconstruction) and checks $H(R \parallel X \parallel m) = c$. The two are interchangeable; the deployed schemes below use the (R, s) form.
2. The hash takes the public key X alongside R and m —*key-prefixing*. The single-key unforgeability proof of §8.6 does not require it, but it forecloses attacks that relate signatures under different keys (Remark 8.5), it is standard in modern instantiations such as EdDSA, and it matters in the re-randomised setting of §8.8.

8.6 Security in the random oracle model and the forking lemma

A Schnorr forgery (R^*, s^*) on a fresh message is an accepting transcript of the identification protocol whose challenge is $H(R^* \parallel X \parallel m^*)$ —a value the forger cannot choose freely if the hash is honest. The security argument makes “honest” precise by idealising H , then turns the rewinding of Remark 8.9 into a quantitative bound.

Definition 8.15 (Random oracle, general range). The random oracle of Definition 3.13 generalises verbatim to an arbitrary finite range: a random oracle with range Y is a function $\mathcal{O}: \{0, 1\}^* \rightarrow Y$ drawn uniformly from the set of all such functions, realised lazily by sampling each new answer $\mathcal{O}(z) \xleftarrow{\$} Y$ on first query. For Schnorr signatures the range is the challenge space $Y = \mathbb{F}_q$, and the ROM for signatures models $H(\cdot) = \mathcal{O}(\cdot)$, with every party—signer, verifier, forger, and reduction—sharing oracle access (Definition 1.26).

The model remains a heuristic, with the uninstantiability caveat and the working stance of Remark 1.29: a ROM proof rules out all attacks that treat the hash as a black box—the overwhelming majority—and is the standard yardstick for Fiat–Shamir signatures. The reductions below exploit both powers of Proposition 3.14: *programmability* (the reduction chooses oracle answers on the fly, provided they appear uniform) and *observability* (the reduction sees every query the adversary makes).

The first step removes the signing oracle with the simulator of Theorem 8.10.

Lemma 8.16 (Signature simulation). *In the ROM there is an efficient simulator that, given only the public key X and the ability to program H , answers signing queries with signatures distributed identically to genuine ones, except that it aborts with probability at most $q_s(q_s + q_b)/q$ over an attack making q_s signing and q_b hash queries.*

Proof. To sign m without x : sample $c \xleftarrow{\$} \mathbb{F}_q$ and $s \xleftarrow{\$} \mathbb{F}_q$ uniformly, set $R := [s]G - [c]X$ —the HVZK simulator of Theorem 8.10—then *program*

$$H(R \parallel X \parallel m) := c,$$

and output (R, s) . The signature verifies by construction, and by Theorem 8.10 its distribution matches a real signature’s exactly: in both cases s is uniform, and c is uniform because a fresh oracle answer is.

Programming fails only if the entry $H(R \parallel X \parallel m)$ is already defined—by an earlier adversary hash query or an earlier simulated signature. The point $R = [s]G - [c]X$ is a fixed translate of $[s]G$ for uniform s , hence uniform over the q elements of $\mathbb{G}$; at most $q_b + q_s$ oracle entries exist at any point of the attack, so a given signing query collides with probability at most $(q_b + q_s)/q$, and the union bound over q_s signing queries gives the stated abort probability. Conditioned on no abort, the adversary’s view is identical to the real game. $\square$

With the EUF-CMA game simulated from X alone (Lemma 8.16), it remains to convert one forgery into *two* accepting transcripts on the same commitment, as Theorem 8.8 requires. The tool is the forking lemma, in the game-based form due to Bellare and Neven.

Theorem 8.17 (General forking lemma). *Fix an integer $Q \geq 1$, a finite set C with $|C| = b \geq 2$, and a randomised input generator Gen . Let B be a randomised algorithm that on input y and challenges $c_1, \dots, c_Q \in C$ returns a pair (I, out) with $I \in \{0, 1, \dots, Q\}$, where $I = 0$ means failure, and let*

$$\text{acc} := \Pr[I \geq 1 : y \leftarrow \text{Gen}; c_1, \dots, c_Q \xleftarrow{\$} C; (I, \text{out}) \leftarrow B(y, c_1, \dots, c_Q)].$$

Define the forking algorithm F_B on input y : pick coins ρ for B and $c_1, \dots, c_Q \xleftarrow{\$} C$; run $(I, \text{out}) \leftarrow B(y, c_1, \dots, c_Q; \rho)$; if $I = 0$, return $\perp$; otherwise resample $c'_1, \dots, c'_Q \xleftarrow{\$} C$ afresh and run $(I', \text{out}') \leftarrow B(y, c_1, \dots, c_{I-1}, c'_1, \dots, c'_Q; \rho)$ with the same coins ρ ; if $I' = I$ and $c_I \neq c'_I$, return $(I, \text{out}, \text{out}')$, else $\perp$. Then

$$\text{frk} := \Pr[F_B(y) \neq \perp : y \leftarrow \text{Gen}] \geq \text{acc} \left(\frac{\text{acc}}{Q} - \frac{1}{b} \right),$$

the probability also over the coins of F_B .

Proof idea. Condition on the coins ρ and the prefix $c_1, \dots, c_{I-1}$ up to the forking point. For each fixed prefix there is (averaging over the suffix) a set S of *good* values of c_I on which B succeeds with index I ; write $p = |S|/b$. The fork succeeds at this prefix iff the first draw c_I and the independent re-draw c'_I both land in S and differ: two independent uniform draws land in S together with probability p^2 , and subtracting the diagonal of equal draws costs at most $p \cdot 1/b$, so the per-prefix fork probability is at least $p(p - 1/b)$. Taking expectations over prefixes, $\mathbb{E}[p^2] \geq (\mathbb{E}[p])^2$, since $\mathbb{E}[p^2] - (\mathbb{E}[p])^2 = \mathbb{E}[(p - \mathbb{E}[p])^2] \geq 0$ by linearity of expectation (*Math Guide*, §“Random variables and expectation”); this squares the average success probability. The index I ranging over Q possible positions introduces the factor $1/Q$, and collecting terms yields $\text{frk} \geq \text{acc}^2/Q - \text{acc}/b$. The squaring step is where the *squared* probability—and hence the tightness loss examined below—enters. $\square$

Theorem 8.18 (EUF-CMA security of Schnorr signatures in the ROM). *Model H as a random oracle with range $\mathbb{F}_q$. If an adversary $\mathcal{A}$ runs in time t , makes at most q_b hash queries and q_s signing queries, and forges with probability ε , then there is an algorithm $\mathcal{D}$ running in time approximately $2t$ that computes discrete logarithms in $\mathbb{G}$ with probability*

$$\varepsilon' \geq \frac{\varepsilon^2}{q_b + q_s + 1} - \frac{q_b + q_s + 1}{q} - \frac{q_s(q_b + q_s)}{q}.$$

In particular, if DLog is hard in $\mathbb{G}$, the Schnorr signature scheme is EUF-CMA in the ROM.

Proof structure. Construct the algorithm B of Theorem 8.17 around $\mathcal{A}$, with input $y = X$ —the DLog challenge, so Gen outputs $X = [x]G$ for uniform x —challenge set $C = \mathbb{F}_q$ (so $b = q$), and $Q = q_b + q_s + 1$ prepared challenges. Algorithm B runs $\mathcal{A}$ on $pk = X$, answering the j -th *distinct* hash query with c_j from its challenge list and answering signing queries by the simulator of Lemma 8.16, whose programming aborts cost the $q_s(q_b + q_s)/q$ term.

Suppose $\mathcal{A}$ outputs a valid fresh forgery $(m^*, (R^*, s^*))$, and let $c^* = H(R^* \| X \| m^*)$. The critical hash input $R^* \| X \| m^*$ must actually have been queried during the run: otherwise c^* is a uniform value $\mathcal{A}$ has never seen, and the verification equation $[s^*]G = R^* + [c^*]X$ then holds only with probability $1/q$ over the oracle’s choice—the $(q_b + q_s + 1)/q$ term collects these guessing events (the $+1$ covering the query B itself makes to verify the forgery). So B outputs the index I of the critical query together with $\text{out} = (R^*, c^*, s^*, m^*)$, and $\text{acc} \geq \varepsilon$ minus the loss terms above.

It remains to fork. Both runs of F_B share the coins ρ and the challenge prefix $c_1, \dots, c_{I-1}$, which together determine $\mathcal{A}$ ’s behaviour up to the moment it produces the critical query; both runs

therefore query the *same* critical input $R^* \| X \| m^*$ at index I , but receive different answers $c^* \neq c'^*$. Success of the fork yields two accepting transcripts (R^*, c^*, s^*) and (R^*, c'^*, s'^*) with the same commitment and distinct challenges, whereupon the extractor of Theorem 8.8 computes

$$x = (s^* - s'^*)(c^* - c'^*)^{-1},$$

the discrete logarithm of X . Substituting $acc, b = q$ and $Q = q_b + q_s + 1$ into $frk \geq acc^2 / Q - acc / b$ gives the stated bound; the running time is two runs of $\mathcal{A}$ plus bookkeeping. $\square$

Example 8.19 (The tightness loss, in numbers). The dominant term of Theorem 8.18 is $\varepsilon' \geq \varepsilon^2 / q_b$: the reduction runs the forger twice and *squares* its success probability, divided by the number of hash queries. The reduction is therefore far from tight (Definition 1.22). Concretely, a forger succeeding with probability $\varepsilon = 2^{-40}$ after $q_b = 2^{60}$ hash queries yields a guaranteed DLog-solver advantage of only about

$$\varepsilon^2 / q_b = 2^{-80} / 2^{60} = 2^{-140},$$

a factor 2^{100} weaker than ε itself. The guarantee runs in the useful direction—any forger yields *some* DLog solver, so DLog hardness still implies security—but the quantitative content is weak. Two consequences follow, and they must be distinguished. The first is conservative (Remark 1.23): a designer who wants *this theorem* to underwrite a target signature-security level must provision $\mathbb{G}$ with roughly twice as many bits of DLog security as the target, since the reduction hands back only the square of the forger’s advantage; and the squaring is intrinsic to the rewinding route—it enters at the squaring step of Theorem 8.17, so any analysis that runs the forger twice and extracts by special soundness incurs it. The second is that the theorem does *not* establish a matching attack: the loss belongs to the reduction, not provably to the scheme, so doubled parameters are a cost of this proof, not a demonstrated necessity, and parameter selection in practice weighs concrete attack models and query bounds as well. Tighter reductions exist once the rewinding route is abandoned—under the *one-more* discrete logarithm assumption (that no efficient adversary, given $\ell + 1$ random elements of $\mathbb{G}$ and ℓ queries to a discrete-logarithm oracle, outputs all $\ell + 1$ logarithms), under DDH (Definition 2.5) together with key-prefixing, or in the idealised algebraic models of Remark 1.30.

Proposition 8.20 (Nonce reuse reveals the secret key). *Suppose a signer produces two Schnorr signatures (R, s) and (R, s') on distinct messages $m \neq m'$ with the same commitment R —a nonce r reused outright, or drawn twice from a faulty source. Then, except with probability $1/q$ over the challenge hash, anyone can compute the secret key from the two signatures.*

Proof. The challenges $c = H(R \| X \| m)$ and $c' = H(R \| X \| m')$ differ except with probability $1/q$, the chance that the random oracle answers the two distinct inputs identically. Given $c \neq c'$, the pairs (R, c, s) and (R, c', s') are two accepting transcripts sharing a commitment with distinct challenges, and the extractor of Theorem 8.8—run in this case by the *adversary*—outputs $x = (s - s')(c - c')^{-1}$. $\square$

The security of the scheme therefore rests on the nonce being fresh and unpredictable for every signature. Nonce reuse is the classic operational failure of deployed Schnorr-family schemes, and motivates deriving nonces *deterministically* from the secret key and message, the defining idea of EdDSA.

8.7 RedDSA: re-randomisable Schnorr for Orchard

This subsection defines EdDSA and its generalisation RedDSA, the scheme that Orchard instantiates for both of its signatures.

Construction 8.21 (EdDSA, schematically). EdDSA retains the Schnorr signature algebra of Construction 8.12 unchanged and hardens two implementation surfaces.

1. *Deterministic nonces.* The nonce is derived as a hash of a secret prefix (part of the expanded secret key) and the message, rather than sampled fresh: no faulty random-number generator can repeat a nonce across distinct messages and trigger Proposition 8.20.
2. *Key-prefixing.* The challenge hash absorbs the public key alongside R and m , as in Construction 8.12 and for the reasons of Remark 8.14.

In the ROM the deterministic nonce is, to anyone ignorant of the secret prefix, a fresh uniform value per message, so the unforgeability proof of Theorem 8.18 carries over essentially unchanged.

RedDSA generalises EdDSA in two directions, both used by Orchard: the base point becomes a parameter, and the key pair can be shifted by a scalar.

Construction 8.22 (RedDSA). RedDSA is the Schnorr/EdDSA scheme with the base point $P \in \mathbb{G}$ a *parameter* of the scheme: different uses sign with respect to different generators.

- **KeyGen:** sample $x \xleftarrow{\$} \mathbb{F}_q$, output secret key x and public key $X = [x]P$.
- **Sign(x, m):** sample a fresh random byte string T and derive the nonce $r = H(T \parallel X \parallel m)$. This is *randomised*, hash-based nonce generation: unlike EdDSA's derivation, it is not deterministic from the signing key and message, so its safety rests on T being fresh and unpredictable, the hashing ensuring only that a repeated T cannot repeat a nonce across distinct messages. Set $R = [r]P$, $c = H(R \parallel X \parallel m) \in \mathbb{F}_q$, $s = r + cx$, and output $\sigma = (R, s)$.
- **Verify($X, m, (R, s)$):** accept iff $[s]P = R + [c]X$ with $c = H(R \parallel X \parallel m)$.

For a fixed base point, RedDSA *is* Schnorr/EdDSA, and its EUF-CMA security in the ROM is Theorem 8.18 with G replaced by P .

Remark 8.23 (RedDSA as deployed). The deployed scheme follows Construction 8.22 line for line (protocol specification § 5.4.7). The randomiser T is 80 uniform bytes, and both hashes are the function H° : BLAKE2b-512 personalised with `Zcash_RedPallasH`, its 64-byte output reduced to $\mathbb{F}_q$ by wide reduction—the expand-then-reduce discipline of Construction 3.18, with the 512-bit width making the modular bias negligible. The nonce hash absorbs $T \parallel X \parallel m$ and the challenge hash $R \parallel X \parallel m$, key-prefixed in both cases.

Orchard instantiates RedDSA *twice*, both times as *RedPallas*—RedDSA over the Pallas curve—and the two instantiations differ *only* in the base point:

- the *spend authorisation* base point, derived by hashing to the curve with domain `z.cash:Orchard` and input `G`, for the re-randomisable signature of §8.8; and
- the *value-commitment randomness* base point, derived with domain `z.cash:Orchard-cv` and input `r`, for the binding signature of §8.9, which relies on this base being byte-for-byte the randomness base of Orchard’s value commitment.

The curve varies only across protocol generations: Sapling’s *RedJubjub* is the same scheme over the Jubjub curve; the curve, its two base points and the $H^\otimes$ personalisation (`Zcash_RedJubjubH` against `Zcash_RedPallasH`) are all that distinguish the two.

8.8 Key re-randomisation and unlinkability

This subsection defines key re-randomisation, which Orchard uses to make spend authorisations unlinkable. Publishing the spender’s public key with each spend would link all spends by the same wallet; because a RedDSA public key is a homomorphic image of the secret, it can instead be *shifted* to a fresh, unlinkable key for every spend.

Definition 8.24 (Key re-randomisation). Fix a base point P . Given a secret key $x \in \mathbb{F}_q$ with public key $X = [x]P$, and a *randomiser* $\alpha \in \mathbb{F}_q$, the re-randomised signing and verification keys are

$$rsk := x + \alpha \in \mathbb{F}_q, \quad rk := X + [\alpha]P \in \mathbb{G}.$$

Proposition 8.25 (Re-randomisation consistency). *The pair (rsk, rk) is a valid RedDSA key pair: $rk = [rsk]P$, so a RedDSA signature under rsk verifies under rk .*

Proof. Linearity of scalar multiplication gives

$$[rsk]P = [x + \alpha]P = [x]P + [\alpha]P = X + [\alpha]P = rk.$$

The pair is thus exactly KeyGen-shaped, and the correctness of signing and verification (Proposition 8.13) applies verbatim. $\square$

Shifting the secret by α shifts the public key by $[\alpha]P$, which is computable from α and P alone, *without* the secret key. A verifier, or a circuit, can therefore check the relationship between X and rk given α , while only the spender who knows x can sign under the shifted key.

Theorem 8.26 (Perfect unlinkability of re-randomised keys). *Fix a base point P . For any public key $X \in \mathbb{G}$, if $\alpha \xleftarrow{\$} \mathbb{F}_q$ is uniform then $rk = X + [\alpha]P$ is uniformly distributed over $\mathbb{G}$. Consequently, in the game where an adversary—of unbounded computational power, and permitted to choose two public keys X_0, X_1 itself—receives $rk = X_b + [\alpha]P$ for a challenger’s uniform bit b and fresh uniform α , and must guess b : the distributions of rk for $b = 0$ and $b = 1$ are identical, and no adversary guesses b with probability better than $1/2$.*

Proof. The point P generates $\mathbb{G}$, which has prime order q , so $\alpha \mapsto [\alpha]P$ is a bijection $\mathbb{F}_q \rightarrow \mathbb{G}$; for uniform α , the point $[\alpha]P$ is uniform over $\mathbb{G}$. Translation by the fixed element X is a bijection of

$\mathbb{G}$ and carries the uniform distribution to itself, so $rk = X + [\alpha]P$ is uniform over $\mathbb{G}$ —for *every* X , hence independently of X . The two conditional distributions of rk coincide exactly, so the adversary’s view is statistically independent of b ; no test distinguishes identical distributions, and the best any distinguisher can do is guess, succeeding with probability exactly $1/2$. $\square$

The guarantee is information-theoretic but holds only if α is sampled uniformly and freshly per use and kept secret. In Orchard the transaction builder samples $\alpha \xleftarrow{\$} \mathbb{F}_q$ afresh for each spend and reveals it only to the proving circuit and the signer.

Remark 8.27 (Scope of unlinkability). Theorem 8.26 covers only the re-randomised *key*. Full unlinkability of a spend further requires that the accompanying signature and zero-knowledge proof leak nothing beyond their statements, that the randomiser α stays secret, and that no other transaction field correlates two spends. In Orchard the protocol publishes rk with each Action (the unit that spends one note and creates one) while proving, inside the Halo 2 circuit, knowledge of a witness for $rk = X + [\alpha]P$ with X a public key whose holder is authorised to spend the note (Orchard’s *spend validating key*)—the circuit constrains rk as a public input—and the network verifies the spend authorisation signature against the public rk . Signing under rk requires $rsk = x + \alpha$, hence knowledge of *both* x and α .

Remark 8.28 (Unforgeability under re-randomised keys, and the key-prefixing subtlety). Unforgeability must be re-established under re-randomised keys. A forgery (R, s) under rk satisfies $[s]P = R + [c]rk$ with $c = H(R \parallel rk \parallel m)$. Subtracting the contribution of $[\alpha]P$ from the verification equation gives $[s - c\alpha]P = R + [c]X$, apparently a forgery under the original key X . But the identity holds with $c = H(R \parallel rk \parallel m)$, while the base verifier recomputes $H(R \parallel X \parallel m)$ —the plain subtraction converts forgeries only for the *non-key-prefixed* variant of Schnorr. For key-prefixed RedDSA one argues in the ROM by either of two routes. (i) The pair $(rsk, rk) = (x + \alpha, X + [\alpha]P)$ is itself a base-scheme key pair (Proposition 8.25) whose public key is uniform (Theorem 8.26), so a forger against rk is *literally* a forger against the base scheme at the key rk ; the reduction, which knows α , recovers $x = rsk - \alpha$ from the extracted rsk . (ii) When the adversary sees signatures under several randomisers α_i , one re-runs the forking argument of Theorem 8.18 on the critical query $H(R^* \parallel rk \parallel m^*)$, simulating signing under the remaining keys by oracle programming (Lemma 8.16). Both routes avoid a pitfall: a naive translation that programs $H(R \parallel rk \parallel m) := H_{\text{base}}(R \parallel X \parallel m)$ must be a carefully injective domain swap, and it fails outright with multiple randomisers—distinct rk_i -prefixed inputs would collide on a single X -prefixed point, a detectable deviation from a random oracle.

8.9 Binding signatures as a proof of knowledge of a discrete logarithm

The second RedPallas instantiation certifies that value is conserved: the commitments hiding a shielded transaction’s amounts must balance, or the transaction creates money. The binding signature enforces this with the homomorphism of §4.6 and the fact that a Schnorr signature is a proof of knowledge of a discrete logarithm.

The value-commitment setup. Shielded protocols commit to value with the homomorphic Pedersen commitment

$$\text{cm}(v, \rho) = [v]V + [\rho]P,$$

where V and P are *independent* generators of $\mathbb{G}$ —independent meaning that no party knows the discrete logarithm of one to the base of the other, arranged by deriving both by hashing to the curve (Construction 3.24)—and ρ is a random blinding scalar. (In the notation of Remark 4.21, the randomness base here called P is the generator R ; this section reserves R for the signature nonce point. The base P is exactly the binding base point of Remark 8.23.) By Theorem 4.20, the group sum of commitments commits to the sum of values with the sum of blindings.

Sapling commits *per note*: a transaction carries input commitments $\text{cm}(v_i^{\text{in}}, \rho_i^{\text{in}})$ and output commitments $\text{cm}(v_j^{\text{out}}, \rho_j^{\text{out}})$, and the net commitment is

$$B = \sum_i \text{cm}(v_i^{\text{in}}, \rho_i^{\text{in}}) - \sum_j \text{cm}(v_j^{\text{out}}, \rho_j^{\text{out}}).$$

Orchard has *no* per-note value commitments: each Action—the unit that simultaneously spends a note of value v^{old} and creates a note of value v^{new} —commits once to its *net* value change, publishing

$$\text{cv}^{\text{net}} = \text{cm}(v^{\text{old}} - v^{\text{new}}, \rho) = [v^{\text{old}} - v^{\text{new}}]V + [\rho]P,$$

and $B = \sum_i \text{cv}^{\text{net}, i}$ over the transaction's Actions (protocol specification § 5.4.8.3). The structure of Actions and their assembly into a transaction are treated in a later volume. In both protocols the homomorphism reduces the sum to

$$B = [\Delta_v]V + [\bar{\rho}]P,$$

where Δ_v is the net value— $\sum_i v_i^{\text{in}} - \sum_j v_j^{\text{out}}$ in Sapling, $\sum_i (v_i^{\text{old}} - v_i^{\text{new}})$ in Orchard—and $\bar{\rho}$ is the net blinding. The transaction balances precisely when Δ_v equals the public balancing value v^{bal} ; for a transaction with internal balance $\Delta_v = 0$, the V -component vanishes and $B = [\bar{\rho}]P$. Since P generates the prime-order group $\mathbb{G}$, every element of $\mathbb{G}$ is some multiple of P , so what distinguishes a balanced transaction is not the *existence* of a representation $B = [\beta]P$ but the ability to compute one. When the values balance, the signer knows the multiplier: it is the net blinding $\bar{\rho}$, assembled from the individual blindings. When they do not, computing any P -multiple representation of B from the published commitments would require knowing $\log_p V$, assumed hard.

Definition 8.29 (Binding signature). With B and $\bar{\rho}$ as above and the transaction balanced, so that $B = [\bar{\rho}]P$, set the *binding verification key* $\text{bvk} := B$ and the *binding signing key* $\text{bsk} := \bar{\rho}$. The binding signature is a RedDSA signature over the base point P , with signing key $\bar{\rho}$ and verification key B , on a message that is the hash of the transaction (the *sighash*). The verifier *recomputes* B from the publicly listed value commitments and the public balancing value, then checks $[s]P = R + [c]B$ with $c = H(R \parallel B \parallel \text{sighash})$.

Theorem 8.30 (The binding signature is a proof of knowledge of $\log_p B$). *In the ROM, a valid binding signature on a transaction is a proof of knowledge of the discrete logarithm of the net commitment B to the base P ; combined with the soundness of the accompanying zero-knowledge*

proofs (per Action in Orchard, per note in Sapling), which fix and range-check the committed values, it forces the transaction to balance.

Proof. Part 1: proof of knowledge. The binding signature is exactly a Fiat–Shamir Schnorr proof for the relation R_{dl} with base P and statement B . By special soundness (Theorem 8.8) together with the forking argument of Theorem 8.18—specialised to a single statement and *no* signing oracle—any algorithm producing a valid signature under B with non-negligible probability can be rewound to two accepting transcripts (R, c, s) , (R, c', s') with $c \neq c'$, from which the extractor recovers $\bar{\rho} = (s - s')(c - c')^{-1}$ with $B = [\bar{\rho}]P$. Producing a valid binding signature is therefore equivalent, up to the standard rewinding loss, to *knowing* $\log_p B$.

Part 2: knowledge forces balance. Write $B = [\Delta_v]V + [\bar{\rho}']P$ for the *true* net value and net blinding that the commitments imply; these are well defined once the accompanying proofs fix the committed openings—in Orchard the per-Action circuit fixes $v^{\text{old}}, v^{\text{new}}$ and the opening of cv^{net} , in Sapling the per-note proofs fix each v_i, ρ_i . If the signer knows $\bar{\rho}$ with $B = [\bar{\rho}]P$, subtracting the two expressions for B gives

$$[\Delta_v]V = [\bar{\rho} - \bar{\rho}']P,$$

where Δ_v enters as a scalar, that is, as its residue in $\mathbb{F}_q$. Whenever $\Delta_v \not\equiv 0 \pmod{q}$, that residue is a nonzero element of $\mathbb{F}_q$, hence invertible, so

$$V = [(\bar{\rho} - \bar{\rho}') \Delta_v^{-1}]P$$

—exhibiting $\log_p V$ and contradicting the independence of V and P . Therefore $\Delta_v \equiv 0 \pmod{q}$.

Part 3: from congruence to equality. Balance in $\mathbb{F}_q$ alone is not yet economic balance: an integer net value equal to a nonzero multiple of q would pass the test above while creating q units in a single transaction. The gap is closed by the *range checks* in the same accompanying proofs: in Orchard the per-Action circuit constrains v^{old} and v^{new} to $\{0, \dots, 2^{64} - 1\}$ (the Action statement, protocol specification § 4.18.4), and in Sapling the per-note proofs constrain each v_i likewise. A transaction with n Actions (count notes instead for Sapling) therefore has $|\Delta_v| \leq n(2^{64} - 1)$ as an integer, far below the 255-bit q for any transaction that can be serialised, and the only multiple of q in that range is 0. Hence $\Delta_v = 0$ over $\mathbb{Z}$, not merely in $\mathbb{F}_q$. $\square$

In deployment the transaction is not internally balanced: it declares a public balancing value v^{bal} , and the verifier checks the signature against $B - [v^{\text{bal}}]V$, folding the public value into the verification key (protocol specification § 4.14). The argument of Theorem 8.30 applies verbatim to the folded key and forces $\Delta_v \equiv v^{\text{bal}} \pmod{q}$; the balancing value is encoded as a signed 64-bit integer, so the range checks promote the congruence to the integer equality $\Delta_v = v^{\text{bal}}$ exactly as in Part 3.

Remark 8.31 (Functions of the binding signature). The single pair (R, s) accomplishes three things at once. First, it proves knowledge of $\bar{\rho}$, certifying balance, by Theorem 8.30. Second, because the Fiat–Shamir challenge hashes the transaction sighash, the same signature *authenticates* the transaction: altering any signed field changes c and invalidates (R, s) , so the binding signature doubles as an integrity check binding the shielded components together—the origin of the name. Third, the construction needs no trusted setup and no separate circuit for the balance check: it

is pure elliptic-curve arithmetic that anyone can verify. The homomorphism of the commitment turns the arithmetic statement “the values balance” into the algebraic statement “ B is a known multiple of P ”, and the Schnorr signature is the standard proof of knowledge for precisely that statement.

Remark 8.32 (Spend authorisation and binding signatures compared). The spend authorisation signature verifies under a re-randomised key $rk = X + [\alpha]P$, chosen to *hide* which long-term key authorised the spend: the signing key $rsk = x + \alpha$ encodes authority, and α provides unlinkability (Theorem 8.26). The binding signature verifies under a key $B = [\bar{\rho}]P$ that the spender does *not* choose: the transaction’s own commitments force it, the “secret key” $\bar{\rho}$ is the net blinding factor, and signing proves balance (Theorem 8.30). Both are the same Schnorr arithmetic and differ only in the *meaning* of the discrete logarithm: authority in one case, value conservation in the other.

8.10 The transparent layer’s schemes: ECDSA and BIP-340 over secp256k1

Two signature schemes in the series live outside the Pallas group: the transparent layer authorises spends with ECDSA, and the issuance bundle of Zcash Shielded Assets is authorised with a BIP-340 Schnorr signature. Both run over secp256k1 and are recorded, not constructed from first principles, at the level their uses in the series require: syntax, deployed conventions, and the status of their security.

The curve secp256k1 is the short Weierstrass curve $y^2 = x^3 + 7$ over the prime field $\mathbb{F}_p$ with $p = 2^{256} - 2^{32} - 977$, with a fixed base point G of prime order n (a 256-bit prime) and cofactor 1; every constant is a named parameter of SEC 2. In SEC 2’s terminology it is a *Koblitz curve*, one with $a = 0$, chosen for implementation speed; the choice plays no role in what follows. The group law, point encodings, and the discrete-logarithm problem on such a curve are the general machinery of the *Math Guide*, §“Elliptic curves” and §“The elliptic-curve discrete logarithm problem”; the hardness assumption is Definition 2.1 in the group $\langle G \rangle$.

Construction 8.33 (ECDSA over secp256k1, schematically). Let $e \in \mathbb{Z}/n\mathbb{Z}$ be the integer read from the 32-byte digest of the message, and write $x(R)$ for the x -coordinate of a point, read as an integer; the secret scalar is written d , since x is already in use.

- KeyGen: sample $d \xleftarrow{\$} [1, n - 1]$, set $X = [d]G$, output (X, d) .
- Sign(d, e): sample $k \xleftarrow{\$} [1, n - 1]$, set $R = [k]G$ and $r = x(R) \bmod n$, then $s = k^{-1}(e + rd) \bmod n$; restart if $r = 0$ or $s = 0$; output (r, s) .
- Verify($X, e, (r, s)$): require $r, s \in [1, n - 1]$, compute $R' = [es^{-1}]G + [rs^{-1}]X$, and accept iff R' is not the identity and $x(R') \equiv r \pmod{n}$.

Correctness: $R' = [s^{-1}(e + rd)]G = [k]G = R$. The nonce k must be fresh and secret, for the reason of Proposition 8.20 in different algebra: two signatures (r, s) and (r, s') under one k on distinct digests $e \neq e'$ give $s - s' = k^{-1}(e - e')$, hence $k = (e - e')(s - s')^{-1}$ and then $d = (sk - e)r^{-1}$.

Where Schnorr’s challenge is a hash of R , ECDSA’s is the coordinate r itself, so the scheme is not a Fiat–Shamir transform and the forking-lemma route to Theorem 8.18 does not apply. Unforgeability analyses exist—in the generic group model, and in the ROM under additional assumptions on the map $R \mapsto x(R) \bmod n$ —but lie outside this volume: the transparent layer relies for ECDSA’s security on its deployment record in Bitcoin rather than on a reduction.

Remark 8.34 (Deployment conventions: the sighash, DER, and low s). Three conventions fix how the transparent layer uses Construction 8.33, which protocol specification § 4.1.7 names as the script signature scheme.

1. *The message is the sighash.* The digest e is the 32-byte SIGHASH transaction hash of protocol specification § 4.10 (ZIP 244 for version-5 transactions), so the sighash algorithm plays the role of the hash in Theorem 8.4.
2. *DER encoding and the sighash-type byte.* On the wire a signature is the DER encoding of (r, s) —an ASN.1 SEQUENCE of two minimally encoded INTEGERS—followed by one trailing byte, the *sighash type*, selecting which parts of the transaction the sighash commits to (SIGHASH_ALL for every signature the transaction builder emits; a signer working from a partially created transaction honours whichever valid type the input records). The signer appends the byte; the verifier splits it off and enforces strict DER unconditionally, the BIP-66 rule, so a malformed encoding fails script evaluation. The public key travels as its 33-byte compressed encoding, the bytes that hash160 commits to (§3.9).
3. *Low- s normalisation.* If (r, s) verifies then so does $(r, n - s)$, so a third party can re-encode a valid signature and change a pre-ZIP-244 transaction id; the normalisation takes the representative $s \leq (n - 1)/2$ (BIP-62), which the deployed signer always emits (the secp256k1 crate, whose libsecp256k1 backend documents every created signature as lower- s) and which script evaluation checks only under the LOWS flag, one that the consensus check of the zcash_script and zebra-script crates does not set.

Construction 8.35 (BIP-340 Schnorr over secp256k1, schematically). Write $H_\tau(m) = \text{SHA256}(\text{SHA256}(\tau) \parallel \text{SHA256}(\tau) \parallel m)$ for the *tagged hash* with ASCII tag τ —domain separation in the sense of Definition 3.16, the doubled tag digest making the prefix a whole 64-byte SHA-256 block—and read digests as integers modulo n .

- *Keys.* Sample $d \xleftarrow{\$} [1, n - 1]$ and set $X = [d]G$. The public key is the 32-byte coordinate $x(X)$ alone (*x-only*): the verifier reconstructs X as the point with that abscissa and *even* y , and the signer replaces d by $n - d$ when $y(X)$ is odd, keeping the pair consistent.
- *Sign(d, m):* derive $k = H_{\text{BIP0340}/\text{nonce}}(t \parallel x(X) \parallel m) \bmod n$ with $t = d \oplus H_{\text{BIP0340}/\text{aux}}(a)$ for auxiliary randomness a ; set $R = [k]G$, negating k when $y(R)$ is odd; compute $e = H_{\text{BIP0340}/\text{challenge}}(x(R) \parallel x(X) \parallel m) \bmod n$ and $s = k + ed \bmod n$; output the 64 bytes $(x(R), s)$.
- *Verify($x(X), m, (r, s)$):* lift $x(X)$ to the even- y point X , recompute e , set $R' = [s]G - [e]X$, and accept iff R' is not the identity, $y(R')$ is even, and $x(R') = r$.

Remark 8.36 (BIP-340 against Construction 8.12; the ZSA consumer). The algebra is the Schnorr signature of Construction 8.12 in its (R, s) form with key-prefixing (Remark 8.14), under $\mathbb{G} = \langle G \rangle$ of order n and $H = H_{\text{BIP0340}/\text{challenge}}$; the unforgeability proof of §8.6 carries over with that tagged hash modelled as the random oracle. What differs is convention: points travel by abscissa alone with parity fixed by the even- y rule, saving a byte per key and the parity of R per signature and letting the same 32 bytes serve as key encoding and as hash input; the challenge hash is domain-separated by tag; and the nonce is deterministic with optional auxiliary randomness, as in Construction 8.21. In the series, BIP-340 is the issuance authorisation signature of Zcash Shielded Assets: ZIP 227 (*specified*, in a draft ZIP) instantiates IssueAuthSig as BIP-340 over secp256k1, prefixes both the validating-key encoding and the signature with a scheme byte $0x00$, and signs the transaction’s sighash. The reference implementation, not yet deployed, is the QED-it fork of the orchard crate (commit cf801a5), which signs through the BIP-340 routine of the secp256k1 crate with zeroed auxiliary randomness. The issuance protocol itself is treated in a later volume.

9 Interactive proofs, zero knowledge, and SNARKs

This section defines interactive proofs and arguments, proofs of knowledge, zero knowledge, Sigma-protocols, the Fiat–Shamir transform, and SNARKs. It proves that special soundness yields knowledge soundness (Theorem 9.12) and that, under a min-entropy condition, the Fiat–Shamir transform of a Sigma-protocol is a knowledge-sound, zero-knowledge non-interactive argument in the random oracle model (Theorem 9.20), and it states the recipe on which the Halo 2 proving system deployed in Orchard is built: a *polynomial interactive oracle proof*, compiled into an argument by a *polynomial commitment scheme* (§4.9) and made non-interactive by the Fiat–Shamir transform (Theorem 9.28).

9.1 Soundness, zero knowledge, and succinctness

Soundness requires that no *cheating prover* make the verifier accept a false statement. *Interactive proofs* and *arguments* provide soundness; their refinement, *proofs of knowledge*, entitles the verifier to conclude that the prover *possesses* a witness, not merely that one exists, so that a prover holding none—no spendable note, no authorisation, no satisfying assignment—cannot mint value, spend another’s coins, or certify a false computation to validators that accept the proof in place of the hidden data.

Zero knowledge requires that the verifier, whether it follows the protocol or deviates from it arbitrarily, learn nothing beyond the truth of the statement: neither the witness—the factorisation, the colouring, the discrete logarithm, the transcript of the computation—nor what it would reveal, such as the identity of the spender, the links between transactions, or the amounts a shielded protocol hides. The two requirements are in tension, soundness asking that the transcript bind the prover and zero knowledge that it reveal nothing; a single protocol achieves both.

Succinctness requires a proof much shorter than the statement it certifies and cheap to check. The *SNARK*, the succinct non-interactive argument of knowledge, is knowledge-sound and succinct; a *zk-SNARK*, a SNARK that is also zero knowledge, has all three properties, so every val-

idator can check a private transaction and learn from it only its validity.

The section draws on the games and reductions of §1, the discrete logarithm of §2, the random oracle of §3, the commitments of §4, and the Schnorr protocol of §8.4, its running example.

Conventions are those of §1: λ is the security parameter, adversaries are PPT (non-uniform where it matters), and negl denotes a negligible function. Proof systems are indexed by an *instance* x rather than by 1^λ , and asymptotic statements are measured in the instance length $|x|$. For the instances this volume considers, $|x|$ and λ are polynomially related, so the two scales agree up to the polynomial factors the definitions already absorb.

9.2 Languages, relations, and witnesses

A statement, and the reason for its truth, are formalised as an instance and a witness of a relation over bit strings.

Definition 9.1 (Language, relation, witness). Fix the alphabet $\{0, 1\}$. A *language* is a set $L \subseteq \{0, 1\}^*$. A *binary relation* is a set $R \subseteq \{0, 1\}^* \times \{0, 1\}^*$ of pairs (x, w) , where x is called the *instance* (or *statement*) and w a *witness* for x . The relation *induces* the language

$$L_R := \{x : \exists w \text{ with } (x, w) \in R\},$$

and for each instance x the (possibly empty) *witness set* is $R(x) := \{w : (x, w) \in R\}$. The relation R is an *NP-relation* if it is

1. *polynomially balanced*: there is a polynomial p with $|w| \leq p(|x|)$ for every $(x, w) \in R$; and
2. *polynomial-time decidable*: a deterministic algorithm decides, in time polynomial in $|x|$, whether a given pair (x, w) belongs to R .

The class NP is exactly the class of languages induced this way: a language L lies in NP if and only if $L = L_R$ for some NP-relation R . We take this certificate characterisation of NP from complexity theory without proof; through it, every statement with a short checkable reason becomes an instance of an NP-relation. Membership in L_R is existentially quantified—*some* witness exists—while holding an element of $R(x)$ is possession of the reason itself.

Example 9.2 (Three running relations). Three relations recur through the section, in increasing order of expressive power.

1. *Discrete logarithm*. In the setting of §8.4—a cyclic group $G = \langle G \rangle$ of prime order q , written additively—the relation is

$$R_{\text{dl}} = \{(X, x) \in G \times \mathbb{F}_q : X = [x]G\}.$$

Because $x \mapsto [x]G$ is a bijection $\mathbb{F}_q \rightarrow G$, every instance $X \in G$ has exactly one witness: the induced language is *trivial*—all of G —and membership is vacuous. What is hard, under the DLP assumption of §2.1, is to *produce* the witness.

2. *Graph three-colouring*. With instances encoding finite graphs $G = (V, E)$,

$$R_{\text{3col}} = \{ (G, c) : c: V \rightarrow \{1, 2, 3\} \text{ and } c(u) \neq c(v) \text{ for all } \{u, v\} \in E \},$$

the relation of *proper* three-colourings. The induced language L_{3col} is NP-complete—every NP language reduces to it in polynomial time, a notion and fact we take from complexity theory without proof—so a zero-knowledge protocol for it yields one for *every* NP language by reduction (§9.8).

3. *Circuit satisfiability*. With instances encoding circuits,

$$R_{\text{sat}} = \{ (C, w) : C \text{ a Boolean (or arithmetic) circuit and } C(w) = 1 \}.$$

This is the universal target of practical proof systems: a computation is first *arithmetised* into the satisfiability of an arithmetic circuit over a finite field F , and the prover proves knowledge of a satisfying assignment (§9.8).

The discrete-logarithm example shows that “ $x \in L$ ” is too weak a target for cryptography: every group element is some multiple of G , so a protocol establishing mere membership establishes nothing. The content of the Schnorr protocol is that the prover *knows* the scalar, and §9.4 formalises “knows” for an algorithm.

9.3 Interactive protocols, proofs, and arguments

Definition 9.3 (Interactive protocol). An *interactive protocol* is a pair (P, V) of algorithms, the *prover* and the *verifier*. Both receive a common input x ; the prover additionally receives a private input w (a witness, or an arbitrary auxiliary string); each party holds its own private random tape. The parties alternate sending messages $a_1, a_2, \dots$ over a number of rounds polynomial in $|x|$; after the last message the verifier outputs a single bit, written

$$\langle P(w), V \rangle(x) \in \{0, 1\},$$

with 1 for *accept*—a random variable over both parties’ coins. The *transcript* of an execution is the full sequence of exchanged messages (together with x , and with the verifier’s coins when relevant). The protocol is *public-coin* if every message the verifier sends is a fresh uniformly random string that it also reveals—the verifier’s messages *are* its coins—and its final decision is a deterministic function of the transcript. The verifier is required to run in probabilistic polynomial time, always.

The verifier is the party the protocol *protects*, so its power is fixed at PPT: a guarantee that holds only for verifiers with super-polynomial resources protects no real verifier. The power of the prover varies, and distinguishes proofs from arguments.

Definition 9.4 (Interactive proof and interactive argument). Let $L \subseteq \{0, 1\}^*$ be a language, and let $c, s: \mathbb{N} \rightarrow [0, 1]$. The protocol (P, V) is an *interactive proof system* for L with *completeness error* c and *soundness error* s if:

1. *Completeness.* For every $x \in L$ there is a prover input w with

$$\Pr[\langle P(w), V \rangle(x) = 1] \geq 1 - c(|x|).$$

2. *Soundness.* For every $x \notin L$ and every prover strategy P^* —of *unbounded* computational power, with arbitrary auxiliary input z —

$$\Pr[\langle P^*(z), V \rangle(x) = 1] \leq s(|x|).$$

The soundness clause is a game in the sense of §1.4: the honest verifier is the challenger, the prover strategy is the adversary, and the win condition is an accept on a false instance. The two thresholds must be bounded apart,

$$c(|x|) + s(|x|) \leq 1 - \frac{1}{\text{poly}(|x|)},$$

and in the protocols of this volume both errors are negligible (often $c = 0$). The class of languages admitting such proof systems—with the honest prover unbounded—is IP. Every protocol this volume constructs is for a language $L = L_R$ induced by an NP-relation R , with the honest prover required to run in polynomial time given a witness $w \in R(x)$ as its private input; the unbounded honest prover matters only for the complexity-theoretic statements below.

An *interactive argument system* (or *computationally sound* proof) relaxes only the soundness quantifier: the guarantee holds against every *PPT* cheating prover P^* , that is, for $x \notin L$ and PPT P^* with arbitrary auxiliary input z , $\Pr[\langle P^*(z), V \rangle(x) = 1] \leq s(|x|)$, where s is required to be negligible. Soundness of an argument typically rests on a computational hardness assumption.

Remark 9.5 (Proofs versus arguments: neither dominates). A proof resists an unbounded cheating prover: its soundness is information-theoretic and unconditional. An argument resists only efficient adversaries, and an unbounded prover can typically break one outright—by breaking the binding of a commitment the protocol relies on (Theorem 4.8 showed perfectly hiding commitments have this exposure), or by brute-forcing a discrete logarithm. In exchange, arguments achieve properties provably impossible for proofs, chief among them *succinctness*: a sound interactive proof for an NP-complete language whose communication is much smaller than the witness would let an unbounded prover compress NP certificates below what complexity theory is believed to allow, collapsing classes believed distinct, whereas succinct *arguments* exist under standard assumptions (§9.8). In cryptographic applications, the SNARK literature included, the prover is resource-bounded, so the argument notion is the appropriate one.

Without interaction and verifier randomness—with a deterministic verifier and a single prover message—a proof system proves exactly the class NP: the one message is a certificate, the verifier’s check is the relation’s decision procedure, and zero knowledge is impossible, since the certificate is essentially the witness itself and the verifier can repeat the prover’s analysis offline. Verifier randomness forces a cheating prover to commit to its messages *before* knowing the challenge, so a random spot-check detects what a predictable one does not. Interaction lets the verifier probe adaptively, round after round. Together they add expressive power: the equality $\text{IP} = \text{PSPACE}$ —interactive

proofs capture exactly the languages decidable with polynomial memory and unrestricted running time, a class believed far larger than NP—is a theorem of complexity theory (Lund, Fortnow, Karloff, and Nisan; Shamir) that we cite without proof. Interaction and randomness are also what make zero knowledge and knowledge extraction possible: a single static certificate can be neither hidden nor rewound.

Proposition 9.6 (Soundness amplification by repetition). *Let (P, V) be an interactive proof (or argument) for L with completeness error c and soundness error s , and let $k = k(|x|)$ be polynomially bounded. The protocol that runs k independent instances and accepts only if all k accept has completeness error at most $k \cdot c$ and soundness error at most s^k . Sequential repetition achieves this always; parallel repetition achieves it for proofs and, with care, for many arguments. In particular, if $s \leq 1 - \delta$ for a noticeable function δ , then choosing k with $\delta(|x|) \cdot k(|x|) = \omega(\log|x|)$ —always possible with k polynomial—drives the soundness error below every inverse polynomial, so a base protocol needs only a noticeable soundness gap.*

Proof. For completeness, the honest prover fails at least one of the k runs with probability at most $k \cdot c$ by the union bound (*Math Guide*, §“The union bound and a birthday calculation”). For soundness under sequential repetition, fix $x \notin L$ and any cheating strategy; conditioned on the transcripts of the first $i - 1$ runs, the i -th run is an execution against a fresh independently-random verifier, so it accepts with probability at most s whatever state the prover carries forward; multiplying the conditional bounds gives s^k . (For parallel repetition of proofs the same bound holds by a more careful argument, which we omit; for arguments parallel repetition can fail to amplify in general, and we use only the sequential form.) Finally $s^k \leq (1 - \delta)^k \leq e^{-\delta k}$, which is negligible once $\delta k = \omega(\log|x|)$. $\square$

9.4 Knowledge soundness and extractors

For the relation R_{dl} every instance $X \in \mathbb{G}$ lies in the induced language, so the soundness clause of Definition 9.4—quantified over $x \notin L$ —holds *vacuously*: a verifier that always accepts is “sound” for the trivial language. The property wanted is that a convincing prover *possesses* the discrete logarithm. Possession is not an observable property of an algorithm, so the definition is operational: a prover “knows” a witness if an efficient procedure, the *extractor*, granted full control over the prover—the ability to run it, feed it messages, and *rewind* it to an earlier state to try different continuations—can compute the witness from it.

Definition 9.7 (Knowledge soundness; proof of knowledge). Let (P, V) be an interactive protocol and R an NP-relation. The protocol is a *proof of knowledge* for R with *knowledge error* $\kappa: \mathbb{N} \rightarrow [0, 1]$ if there exist a probabilistic oracle algorithm $\mathcal{E}$ (the *extractor*) and a polynomial q such that for every prover strategy P^* —possibly cheating, possibly computationally unbounded—and every instance x , writing

$$\varepsilon(x) := \Pr[\langle P^*, V \rangle(x) = 1]$$

for the probability that P^* convinces the honest verifier, the following holds whenever $\varepsilon(x) > \kappa(x)$: the extractor $\mathcal{E}$, given x and oracle access to P^* *with the ability to rewind*—to run P^* from

any point with the same coins, feeding verifier messages of $\mathcal{E}$'s choosing—outputs a witness $w \in R(x)$ within expected running time

$$\frac{q(|x|)}{\varepsilon(x) - \kappa(x)}.$$

If the guarantee is required only for *PPT* prover strategies P^* , the protocol is an *argument of knowledge*.

The extraction cost may grow without bound as ε approaches κ , so the bound is useful when the prover's success is noticeably above the knowledge error. A prover succeeding with probability at most κ need yield nothing: the knowledge error is the “free” success rate, typically the probability of blind guessing, below which acceptance carries no evidence of knowledge. Through the expected-time clause, extractors are the reason this volume's adversary class admits expected polynomial time (Remark 1.4).

Remark 9.8 (The rewinding justification). A counterparty in a live execution cannot rewind the prover: the extractor is an *analytical device*, never executed during an honest run of the protocol. In a reduction the extractor *is* the reduction: the cheating prover is a subroutine the reduction runs through its input/output interface alone (Definition 1.20), and black-box access already includes choosing the subroutine's random tape and restarting it from the top with the same coins, so replaying it under different challenges requires no reading of its code. The *simulator* of §9.5 rewinds a verifier in the same way to produce transcripts without a witness.

Proposition 9.9 (Knowledge soundness implies soundness). *A proof of knowledge for R with knowledge error κ is an interactive proof for L_R with soundness error $s(x) \leq \kappa(x)$. The converse fails: a protocol can be sound without being a proof of knowledge.*

Proof. Fix $x \notin L_R$, so that $R(x) = \emptyset$. If some prover strategy P^* achieved $\varepsilon(x) > \kappa(x)$, the extractor run on P^* would halt in finite expected time and output some $w \in R(x)$ —impossible, as the witness set is empty. Hence $\varepsilon(x) \leq \kappa(x)$ for every P^* , which is the soundness bound. For the failure of the converse: soundness constrains only false instances, so a protocol for a trivial language—such as R_{all} 's, where no false instances exist—is automatically sound however it behaves, while nothing forces a witness to be extractable from it. A verifier may be convinced of a true statement by a prover that does not know why it is true; knowledge soundness is strictly stronger, and cryptographic applications rely on it, not on soundness alone (§9.8). $\square$

9.5 The simulation paradigm and zero knowledge

Under the *simulation paradigm*, the verifier learned nothing beyond the truth of the statement if it could have produced, without interacting with the prover, a transcript indistinguishable from the real one. The three grades of indistinguishability—perfect, statistical, computational—carry over from §1.2 (Definitions 1.6 and 1.9, with the unbounded-distinguisher bound of Proposition 1.7 and the hierarchy of Proposition 1.10), with ensembles indexed by instances x (and auxiliary inputs) rather than by the security parameter alone, and negligibility measured in $|x|$.

Definition 9.10 (Zero knowledge). Let (P, V) be an interactive proof or argument for $L = L_R$. For a verifier strategy V^* , an instance x , a witness $w \in R(x)$, and an auxiliary input z , the *view*

$$\text{View}_{V^*}(P(w), V^*(z))(x)$$

is the random variable consisting of x , the random tape of V^* , and every message V^* receives—everything the verifier sees. The protocol is *zero knowledge* if there exists a PPT simulator Sim such that for every $x \in L$, every witness $w \in R(x)$, and every auxiliary input z , the ensembles

$$\{\text{Sim}(x, z)\} \quad \text{and} \quad \{\text{View}_{V^*}(P(w), V^*(z))(x)\}$$

are indistinguishable; the grade of indistinguishability—perfect, statistical, computational—gives *perfect* (PZK), *statistical* (SZK), or *computational* (CZK) zero knowledge. Two regimes for the quantifier over V^* :

1. *Honest-verifier zero knowledge* (HVZK): the guarantee is required only for $V^* = V$, the verifier that follows the protocol, with z empty; the simulator must reproduce the distribution of honest-execution transcripts.
2. *Malicious-verifier (auxiliary-input) zero knowledge*: the guarantee holds for *every* PPT verifier strategy V^* , deviating arbitrarily, with arbitrary auxiliary input z . Here Sim may use V^* as a subroutine *and rewind it*, and is permitted expected polynomial running time (Remark 1.4).

The simulator receives x —and z —but *not* the witness w . If its output is nevertheless indistinguishable from the real view, then the real view carries no efficiently extractable information about w beyond the fact $x \in L$: any property of w an efficient verifier could compute from the interaction, it could compute equally well from the simulator’s output—otherwise the computation would itself be a distinguisher. Unlike the prover, the simulator may rewind V^* , retrying until the verifier’s challenges align with a transcript preparable without the witness (Remark 9.8).

The auxiliary input z models the prior knowledge a malicious verifier may bring to the interaction—the history of earlier protocol runs, partial information about the witness, or any other string. Quantifying over all z makes zero knowledge *compose*: a protocol that is zero knowledge for every auxiliary input remains zero knowledge when run as a subroutine of a larger system, whose surrounding state becomes the z of the analysis. Practice and this volume adopt the auxiliary-input formulation.

9.6 Sigma-protocols

Sigma-protocols realise the preceding definitions with three moves and public coins, every property following from algebraic relations among the messages; the name refers to the capital Σ that the message flow traces.

Definition 9.11 (Sigma-protocol). Let R be an NP-relation. A *Sigma-protocol* for R with *challenge space* C is a three-move public-coin protocol on common input x , the prover holding w

with $(x, w) \in R$:

1. *Commitment.* The prover sends a first message a (the *commitment*, or announcement), computed from (x, w) and fresh randomness.
2. *Challenge.* The verifier sends $e \xleftarrow{\$} C$, uniformly random.
3. *Response.* The prover sends z , computed from (x, w, a, e) and its randomness.

The verifier decides by a deterministic predicate $\text{Verify}(x, a, e, z) \in \{0, 1\}$. Three properties are required.

- *Completeness.* An honest prover holding $(x, w) \in R$ convinces the honest verifier with probability 1.
- *Special soundness.* There is a PPT algorithm that, given x and any two accepting transcripts (a, e, z) and (a, e', z') with the *same* commitment a but *distinct* challenges $e \neq e'$, outputs a witness $w \in R(x)$.
- *Special honest-verifier zero knowledge* (sHVZK). There is a PPT simulator that, on input x and *any* challenge $e \in C$, outputs a pair (a, z) such that $\text{Verify}(x, a, e, z) = 1$ and the triple (a, e, z) is distributed exactly—or statistically, or computationally, close to—a real honest transcript conditioned on the challenge being e . The simulator chooses a *after* fixing e ; reversing the honest order is what makes simulation possible without w .

Two accepting transcripts sharing a commitment but differing in challenge are called a *collision*. Special soundness says that a witness is efficiently computable from a collision: it is the local, two-transcript reformulation of knowledge extraction, and the basis of the theorem that follows.

Theorem 9.12 (Special soundness yields knowledge soundness). *Let (P, V) be a Sigma-protocol for R with challenge space C of size $N := |C|$. Then (P, V) is a proof of knowledge for R with knowledge error $\kappa = 1/N$.*

Proof. We construct an extractor establishing the slightly weaker knowledge error $2/N$, with expected running time $O(1/(\varepsilon - 2/N))$ prover invocations whenever the prover's success probability satisfies $\varepsilon > 2/N$; the closing paragraph addresses the gap to the optimal $1/N$.

The prover as a matrix. Fix x and a prover strategy P^* ; fold any auxiliary input into its code. The behaviour of P^* is a deterministic function of its random tape ρ : fixing ρ fixes the commitment $a(\rho)$, and then each challenge e determines the response $z(\rho, e)$. Form the Boolean matrix

$$H[\rho, e] := \begin{cases} 1 & \text{if } \text{Verify}(x, a(\rho), e, z(\rho, e)) = 1, \\ 0 & \text{otherwise,} \end{cases}$$

with one row per random tape and one column per challenge; then $\varepsilon = \Pr_{\rho, e}[H[\rho, e] = 1]$, and a collision is two 1-entries in a single row.

The extractor. Step 1. Run P^* with fresh ρ and a uniform challenge e , repeatedly, until an accepting transcript (a, e, z) appears. The number of trials is geometric with success probability ε , so this costs $1/\varepsilon$ invocations in expectation—the geometric waiting time (*Math Guide*, §“Beyond finite spaces: countable additivity, limits, and densities”). *Step 2.* Alternate two kinds of trial: (i) rewind P^* to the moment just after it emitted a —the tape ρ stays fixed—and feed one fresh uniform challenge $e' \neq e$; (ii) make one entirely fresh probe, as in Step 1. If a kind-(i) trial accepts, a collision $(a, e, z), (a, e', z')$ is in hand: go to Step 3. If a kind-(ii) trial accepts, *abandon* the current row and continue Step 2 from the new accepting transcript (a fresh *phase*). *Step 3.* Apply the special-soundness algorithm of Definition 9.11 to the collision and output the witness.

Analysis. Call a row ρ *heavy* if its acceptance density $\delta_\rho := \Pr_e[H[\rho, e] = 1]$ is at least $\varepsilon/2$. The heavy-row lemma—a Markov-style averaging—states that an accepting entry lies in a heavy row with probability at least $1/2$: the total accepting mass contributed by light rows is less than $\varepsilon/2$ (each light row contributes less than $\varepsilon/2$ of its own row measure), so conditioned on acceptance the light rows carry less than half the mass. Every phase opens with a fresh accepting transcript distributed exactly as Step 1’s (the opening transcript is either Step 1’s or a kind-(ii) success, both conditioned draws from the accepting entries), so the lemma applies at every phase, not only the first.

Each alternation of Step 2 contains a kind-(ii) trial, which accepts with probability ε ; the number of alternations in a phase is therefore dominated by a geometric variable with mean $1/\varepsilon$, and a phase costs $O(1/\varepsilon)$ invocations in expectation. This interleaving is necessary, not an optimisation: a row whose only accepting challenge is the already-used e —any row with $\delta_\rho \leq 1/N$ —contains no collision at all, and an extractor that only rewound would remain in such a row for unbounded expected time. The fresh probes bound the expected cost of every phase.

Now condition on the phase’s row being heavy. A kind-(i) trial draws e' uniformly from $C \setminus \{e\}$ and accepts with probability at least

$$\frac{\delta_\rho N - 1}{N - 1} \geq \delta_\rho - \frac{1}{N} \geq \frac{\varepsilon}{2} - \frac{1}{N},$$

which is positive precisely when $\varepsilon > 2/N$. The phase ends at its first success of either kind, and the chance that this first success is a collision rather than a row-switch is at least

$$\frac{\varepsilon/2 - 1/N}{(\varepsilon/2 - 1/N) + \varepsilon} = \Omega\left(\frac{\varepsilon - 2/N}{\varepsilon}\right).$$

Un-conditioning costs the heavy-row factor $1/2$, so each phase yields a collision with probability $\Omega((\varepsilon - 2/N)/\varepsilon)$, and the number of phases is dominated by a geometric variable with mean $O(\varepsilon/(\varepsilon - 2/N))$. By Wald’s identity—the expected total is the expected number of phases times the expected cost per phase (*Math Guide*, same section)—the extractor’s expected number of prover invocations is

$$O\left(\frac{\varepsilon}{\varepsilon - 2/N}\right) \cdot O\left(\frac{1}{\varepsilon}\right) = O\left(\frac{1}{\varepsilon - 2/N}\right),$$

of the form $q(|x|)/(\varepsilon - 2/N)$ demanded by Definition 9.7 with $\kappa = 2/N$.

Closing the factor of two. The threshold $\delta_\rho \geq \varepsilon/2$ defining heaviness is an artefact of this exposition, and it is exactly what costs the extra $1/N$ in the knowledge error. Sharper accounting removes

it: Damgård’s refined heavy-row analysis in his Sigma-protocol lecture notes, and, in full generality, the k -special-soundness extractor of Attema, Cramer, and Kohl, which achieves knowledge error exactly $(k - 1)/N$ —here $k = 2$, giving the optimal $\kappa = 1/N$ that matches the soundness error. We cite these rather than reproduce them; every use in this volume tolerates the factor of two. $\square$

Remark 9.13 (Knowledge error equals soundness error, and how to size the challenge space). The knowledge error $1/N$ of Theorem 9.12 is not an artefact of the analysis: an explicit attack attains it. A prover with no witness guesses the challenge in advance: it picks $e^* \in C$, runs the sHVZK simulator on (x, e^*) to obtain an accepting pair (a, z) —for a protocol such as Schnorr’s the simulator’s algebra works for *every* instance—and sends a . If the verifier’s challenge happens to equal e^* , which occurs with probability exactly $1/N$, the prepared z is accepted. One challenge is thus always answerable without a witness, so $1/N$ is also the protocol’s soundness error, and Theorem 9.12 says the two coincide. The lower bound carries one caveat: under Definition 9.7 an extractor facing this guessing prover ($\varepsilon = 1/N$) is granted expected time $q(|x|)/(1/N - \kappa)$, so for exponential N a claimed κ below $1/N$ is not formally refuted—the definition would hand the extractor a super-polynomial budget, enough to brute-force a witness unaided. The attack does rule out any noticeably smaller κ once the extractor is required to run in strict polynomial time and witnesses are genuinely hard to compute: such an extractor, run on this witness-free prover, would otherwise compute a witness from a prover that holds none—for R_{dl} , solving the discrete-logarithm problem itself. There are two ways to make the error negligible: take C exponentially large—for instance $C = \mathbb{F}_q$ for a λ -bit prime q , so $N = q$ —or repeat a small-challenge protocol $\omega(\log \lambda)$ times (Proposition 9.6).

The succinct arguments of §9.8 run many challenge rounds, and their extractors need a tree of transcripts in place of a collision.

Definition 9.14 (Tree special soundness). Let (P, V) be a $(2\mu + 1)$ -move public-coin protocol for R in which the prover speaks first and the verifier issues μ uniform challenges from a space C , and let $k_1, \dots, k_\mu$ be positive integers. A $(k_1, \dots, k_\mu)$ -tree of accepting transcripts for x is a tree of depth μ in which every node at level i has k_i children labelled by *pairwise distinct* challenges for round i , every root-to-leaf path carries the prover messages of one accepting transcript, and transcripts passing through a common node share their prefix up to that node. The protocol is $(k_1, \dots, k_\mu)$ -special-sound if a PPT algorithm computes a witness $w \in R(x)$ from any such tree. A Sigma-protocol is the case $\mu = 1, k_1 = 2$.

Extraction generalises accordingly: under the hypotheses of a tree-extraction theorem—the Attema–Cramer–Kohl analysis cited in the proof of Theorem 9.12 is the general form—rewinding the prover round by round assembles the required tree of $\prod_i k_i$ accepting transcripts: the protocol is a proof of knowledge with knowledge error growing from $1/|C|$ to the order of $(\sum_i (k_i - 1))/|C|$, and with extraction cost polynomial provided the tree size $\prod_i k_i$ is. Both degradations are benign in the regime the compiled protocols of this section occupy: C exponentially large, μ logarithmically bounded, and the k_i constant, so the error stays negligible and $\prod_i k_i$ polynomial. This is the notion under which the inner-product argument’s extractor operates (Proposition 4.39): each of the $\log_2 d$ folding rounds contributes one level of branching

to the tree, three accepting children at every node. Each round's accepting relation is linear in three unknown parent terms with coefficients that are powers of the challenge, so three suitably distinct challenges make that system invertible. A later volume of the series carries out that extraction for the deployed protocol, whose IPA normalisation involves each round's challenge through the coefficients $u^{-1}, 1, u$: the extractor accordingly needs pairwise distinct *nonzero* challenges at every node—a negligible-fraction exclusion absorbed into the knowledge error—and not the pairwise distinct squares ($u \neq \pm u'$) that the alternative $u^{-2}, 1, u^2$ normalisation would demand.

Proposition 9.15 (sHVZK implies HVZK). *A Sigma-protocol satisfying special honest-verifier zero knowledge is honest-verifier zero knowledge in the sense of Definition 9.10, with the same grade (perfect, statistical, or computational).*

Proof. The honest view is a triple (a, e, z) in which e is uniform on C . The HVZK simulator draws $e \xleftarrow{\$} C$ itself, invokes the sHVZK simulator on (x, e) to obtain (a, z) , and outputs (a, e, z) . For each fixed e , the simulated pair (a, z) is distributed as a real run conditioned on that challenge—exactly, or up to the stated grade of closeness—and the simulator's e has the same uniform marginal as the honest verifier's; averaging the conditional distributions over uniform e therefore reproduces the honest view, with any statistical or computational slack preserved under the averaging. $\square$

Example 9.16 (Schnorr: the canonical Sigma-protocol). The Schnorr identification protocol of §8.4 is a Sigma-protocol for the discrete-logarithm relation R_{dl} of Example 9.2, and the properties proved there, self-contained, are exactly the clauses of Definition 9.11. The commitment is $R = [r]G$; the challenge is a uniform $c \in \mathbb{F}_q$; the response is $s = r + cx$; the predicate is $[s]G = R + [c]X$. Perfect completeness is Proposition 8.7; 2-special soundness is Theorem 8.8, whose collision algebra yields the witness $x = (s - s')(c - c')^{-1}$; special HVZK is Theorem 8.10, whose simulator samples s and solves $R = [s]G - [c]X$ —choosing the commitment after the challenge, as the definition prescribes. The challenge space is $\mathbb{F}_q$, so $N = q$ and the knowledge error $1/q$ is negligible in a single round (Remark 9.13). By Theorem 9.12 and Proposition 9.15, Schnorr's protocol is a perfect-HVZK proof of knowledge of a discrete logarithm.

Remark 9.17 (From honest to malicious verifiers: three remedies). Sigma-protocols are honest-verifier zero knowledge only. A malicious V^* may choose e as a function of the commitment a , and the sHVZK simulator—which fixes e before producing a —does not directly apply: nothing lets it predict which challenge the verifier's function will return for a commitment it has yet to manufacture. Three standard remedies close the gap. (1) *Guess and rewind*, for small challenge spaces: the simulator guesses $e^* \xleftarrow{\$} C$, runs the sHVZK simulator on (x, e^*) , and feeds the resulting a to V^* ; if the verifier's challenge equals e^* , the round is simulated, and otherwise the simulator rewinds V^* and guesses afresh. The honest prover fixes a before any challenge exists, so the simulated a carries (almost) no information about the guess and each attempt succeeds with probability close to $1/N$: for polynomially large N the expected $O(N)$ attempts stay polynomial, and sequential repetition of the small-challenge protocol—simulated round by round, the auxiliary input carrying the history—is full malicious-verifier zero knowledge, at the price of many rounds. For exponentially

large N the guess succeeds with negligible probability, and repetition does not generically upgrade honest-verifier to malicious-verifier zero knowledge. (2) *Challenge commitment*: the verifier commits to e before seeing a (a coin-tossing step using the commitments of §4), so e cannot depend on a ; under the additional conditions such a compiler imposes for simulation, the simulator rewinds past the opening. (3) *The Fiat–Shamir transform*: remove the verifier’s choice altogether by deriving e from a hash of the transcript—under the hypotheses of Theorem 9.20, in the programmable random oracle model, the resulting *non-interactive* protocol is zero knowledge against *all* verifiers, since no verifier-chosen challenge remains to be biased. The SNARK construction uses route (3), and honest-verifier zero knowledge is the property the transform requires.

9.7 The Fiat–Shamir transform: from interactive to non-interactive

In a public-coin protocol the verifier’s messages are coin tosses and its decision a deterministic function of the transcript (Definition 9.3), so each message can be replaced by a value that *anyone* can compute—a hash of everything sent so far—and the interaction reduces to a single proof string, verifiable by recomputing the hashes. The hash must be idealised: the analysis lives in the random oracle model, with $H: \{0, 1\}^* \rightarrow C$ a random oracle whose range is the challenge space (Definition 8.15), and it uses both properties established in Proposition 3.14: *observability*—the reduction sees every query the adversary makes—and *programmability*—the reduction may fix the oracle’s output at a freshly queried point to any value, provided answers stay consistent and uniformly distributed.

Definition 9.18 (The Fiat–Shamir transform). Let (P, V) be a public-coin protocol for R with challenge space C , and let H be a random oracle with range C . The *Fiat–Shamir transform* $\text{FS}[(P, V), H]$ is the non-interactive system defined, in the Sigma case, by:

- *Prover*. Compute the commitment a as in (P, V) ; set $e := H(x, a)$; compute the response z ; output the proof $\pi := (a, z)$ (equivalently (a, e, z)).
- *Verifier*. Given x and $\pi = (a, z)$, recompute $e := H(x, a)$ and accept iff $\text{Verify}(x, a, e, z) = 1$.

A multi-round public-coin protocol applies the rule per round: the i -th challenge is

$$e_i := H(x, a_1, e_1, \dots, a_i),$$

each challenge hashing the *entire* transcript prefix. To bind a message m to the proof—turning it into a signature—one includes m in the hash: $e := H(x, a, m)$.

Remark 9.19 (Weak Fiat–Shamir). The definition’s insistence that each challenge hash *all* prior messages, the instance x included, is essential, and omitting any part of it has broken deployed systems. Hashing $e := H(a)$ without x —“weak Fiat–Shamir”—lets an attacker fix (a, e, z) first and choose the instance *afterwards*, searching for an x that the fixed challenge validates: the interactive protocol’s soundness quantified over provers who commit before the challenge, and an unhashed x re-opens exactly that order of quantifiers, yielding proofs of false statements. In multi-round protocols, hashing only the latest message rather than the running prefix breaks soundness the

same way, one round at a time.

Theorem 9.20 (Fiat–Shamir security in the ROM). *Let (P, V) be a Sigma-protocol for R with challenge space C of size N , special soundness, and special HVZK; assume further that, conditioned on (x, e) , the first message output by the sHVZK simulator has min-entropy $\omega(\log \lambda)$ (min-entropy H_∞ : Definition 6.23). In the random oracle model, $\Pi = \text{FS}[(P, V), H]$ is a non-interactive argument for R that is:*

1. perfectly complete, inheriting the protocol’s completeness;
2. knowledge sound: from any PPT prover P^* that makes at most Q random-oracle queries and outputs an accepting proof for x with probability ε , an extractor obtains a witness $w \in R(x)$ with probability at least $\varepsilon(Q - 1/N)$ up to constant factors—non-negligible whenever ε is non-negligible and N is super-polynomial; and
3. non-interactive zero knowledge: a PPT simulator that controls the random oracle produces, without the witness, proofs indistinguishable from real ones—for all verifiers, including malicious ones.

Proof. Completeness. The honest prover computes $e = H(x, a)$ and an honest response; the verifier recomputes the same e from the same inputs and runs the same predicate, so the protocol’s probability-1 completeness transfers verbatim.

Zero knowledge, by programming. The simulator, on input x , draws $e \xleftarrow{\$} C$, runs the sHVZK simulator on (x, e) to obtain an accepting pair (a, z) , and then *programs* the oracle:

$$H(x, a) := e.$$

Programming fails only if (x, a) was queried before this moment; by hypothesis the simulated first message has, conditioned on (x, e) , min-entropy $\omega(\log \lambda)$, so a given prior query collides with a with probability $2^{-H_\infty(a)}$, and a union bound over the adversary’s prior queries makes the failure probability negligible. Conditioned on no failure, the programmed answer is a fresh uniform value—exactly what an unprogrammed oracle would have returned—so the oracle’s joint distribution is untouched, and by sHVZK the output (a, z) is distributed as a real proof. No verifier can detect the programming, and no verifier-chosen challenge exists for a malicious verifier to bias, so its deviation has no effect and honest-verifier simulation yields full non-interactive zero knowledge.

Knowledge soundness, by forking. Let P^* output an accepting proof (a, z) for x with probability ε . With overwhelming probability a successful P^* actually queried H at (x, a) : otherwise the challenge $e = H(x, a)$ used by the verifier is a uniform value the prover never saw, and the proof verifies with probability at most $1/N$. Say the critical query is the j -th of the at most Q queries. The extractor runs P^* once, recording the transcript of oracle answers and the index j of the critical query; it then *rewinds* P^* to just before the j -th query and re-runs it with the same coins, answering the first $j - 1$ queries identically—so the commitment a is unchanged—and the j -th and subsequent queries with fresh uniform values, the j -th being $e' \xleftarrow{\$} C$. This is the experiment of

the general forking lemma (Theorem 8.17, with $b = N$): if P^* succeeds with probability ε over Q query slots, then with probability at least $\varepsilon(\varepsilon/Q - 1/N)$ both runs succeed at the same index *and* $e \neq e'$. Conditioned on that event, the two runs yield accepting transcripts (a, e, z) and (a, e', z') with a common commitment and distinct challenges—a collision—and the special-soundness algorithm of Definition 9.11 outputs $w \in R(x)$. $\square$

Remark 9.21 (Caveats, and the refinements SNARKs need). The random oracle is an idealisation: a real hash function is fixed, public, and not random, and there exist contrived protocols secure in the ROM yet insecure under *every* concrete instantiation. The transform is nonetheless trusted in practice, with the working stance of Remark 1.29, and it underlies essentially every deployed non-interactive proof and signature—Fiat–Shamir-compiled SNARKs included. Two refinements matter for the compiled arguments of §9.8. First, Theorem 9.20 covers the one-challenge Sigma case only, and does not by itself prove security for a multi-round protocol such as Halo 2’s: there the extractor must fork the transcript at *each* round, its success probability degrading with the round count and with the protocol’s soundness structure, and special soundness must generalise to tree special soundness (Definition 9.14)—a separate multi-round theorem, whose hypotheses must match the protocol at hand. Second, when the underlying interactive protocol is itself only computationally sound—an argument—the ROM extraction *layers on top of* the computational soundness reduction rather than replacing it. There is also a gap between proofs that merely observe the oracle and proofs that program it (the non-programmable ROM is strictly weaker as a proof technique); deployed systems adopt the programmable model—the zero-knowledge clause above, which programs the oracle outright, already shows why—and an analysis must state which programmable-oracle (and, where used, algebraic) model it works in.

Example 9.22 (Schnorr signatures, recovered). Applying Definition 9.18 to Schnorr’s Sigma-protocol (Example 9.16) with a message m folded into the challenge hash yields, symbol for symbol, the Schnorr signature scheme of Construction 8.12; and its EUF-CMA security in the ROM (Theorem 8.18) is established by exactly the forking argument in the proof of Theorem 9.20, augmented with a signing-oracle simulation. In general, the Fiat–Shamir transform of a Sigma-protocol proof of knowledge, with the message included in the challenge hash, is a signature scheme.

9.8 SNARKs: succinct non-interactive arguments of knowledge

The Sigma-protocol paradigm reaches all of NP: a commit–challenge–reveal proof of knowledge for the NP-complete relation $R_{3\text{col}}$ of Example 9.2—zero knowledge under one-way functions alone, a result of Goldreich, Micali, and Wigderson taken without proof—extends to every NP language by reduction. The generic reduction is impractical, however: its proofs are polynomially *longer* than the computation they certify, so practical systems abandon it and arithmetise the computation directly.

A transaction’s proof is a string stored in a block and checked, possibly years later, by every full node, none of which communicates with its author; an accepting proof *authorises* a transfer of value. The proof system must therefore be non-interactive, short and verifiable at a cost orders of magnitude below that of the computation it certifies, and sound against a prover with every incentive to cheat. The SNARK meets all three constraints.

Definition 9.23 (Non-interactive argument). A *non-interactive argument* for an NP-relation R is a triple of PPT algorithms:

- $\text{Setup}(1^\lambda, R) \rightarrow srs$, producing a *structured reference string*—the public parameters. In the random oracle model all three algorithms additionally access one globally sampled oracle; the oracle is not part of the srs —a PPT setup cannot output an infinite object—though public parameters may be derived through it. A setup holding no secret trapdoor is *transparent*: no secret is ever held, so none must be trusted to have been discarded (Remark 4.3).
- $\text{Prove}(srs, x, w) \rightarrow \pi$, for $(x, w) \in R$.
- $\text{Verify}(srs, x, \pi) \rightarrow \{0, 1\}$.

The system is *complete* if honestly generated proofs always verify. A *preprocessing* argument lets Setup depend on the circuit or relation and output, alongside the proving parameters, a short *verification key*, amortising the verifier’s per-statement work across all statements of that shape.

Definition 9.24 (SNARK). A *succinct non-interactive argument of knowledge* (SNARK) for an NP-relation R is a non-interactive argument that is additionally:

1. *Knowledge sound*. For every PPT prover P^* there is a PPT extractor $\mathcal{E}$ —in the ROM, observing P^* ’s oracle queries and rewinding or forking it—such that, on the same setup,

$$\Pr[\text{Verify}(srs, x, \pi) = 1 \wedge (x, \mathcal{E}^{P^*}) \notin R] \leq \text{negl}(\lambda),$$

the probability over the setup and both algorithms’ coins: an accepting proof implies an extractable witness, except with negligible probability.

2. *Succinct*. The proof size $|\pi|$ and the verifier’s running time are *sublinear* in the size of the computation proved—typically $|\pi| = O(\text{poly}(\lambda))$ or $O(\text{poly}(\lambda) \log|C|)$, and verification $O(\text{poly}(\lambda) \log|C|)$ for a circuit C , independent of, or polylogarithmic in, the witness and circuit sizes. (Some of the literature reserves “succinct” for polylogarithmic proofs; the constant-size proofs of pairing-based schemes are the strongest form.)

With zero knowledge—the non-interactive, ROM form of Definition 9.10, as delivered in the Sigma case by Theorem 9.20, clause 3—it is a *zk-SNARK*.

Remark 9.25 (Why these systems are arguments). The “AR” in SNARK, *argument*, reflects a necessity, not a choice. A succinct *proof* for an NP-complete language is essentially ruled out (Remark 9.5): a transcript shorter than the witness cannot information-theoretically pin the witness down. Succinct *arguments* avoid this obstruction because a computationally bounded prover cannot *find* a convincing-but-false short proof even though one may exist. In the constructions below the computational assumption enters through the polynomial commitments, which are binding only against *efficient* adversaries, under cryptographic assumptions; an unbounded prover could equivocate on a commitment and defeat extraction.

The statement of a private transaction reads: *there exists a note I am authorised to spend, and in-*

put and output values balance. Plain soundness guarantees only that *some* satisfying witness exists (Proposition 9.9), and over a large anonymity set such witnesses abound: other people’s notes satisfy the existential quantifier. Soundness alone would therefore not prevent a prover holding no spendable note from proving the statement, should it obtain a proof by some means. *Knowledge* soundness guarantees that whoever produced the accepting proof *holds* a concrete witness—a real spendable note, a real authorisation—that the extractor could recover from that prover, so the application may act—release funds, grant access—exactly as though the witness had been presented in the clear, while it stays hidden.

The pairing-based route. The first SNARKs to reach deployment, and the ones the deployed Sapling protocol still uses, verify with a bilinear pairing.

Definition 9.26 (Bilinear pairing). Let G_1, G_2, G_T be cyclic groups of prime order r , the first two written additively with generators G and $\hat{G}$. A *bilinear pairing* is a map $e: G_1 \times G_2 \rightarrow G_T$ that is

1. *bilinear*: $e([a]P, [b]Q) = e(P, Q)^{ab}$ for all $P \in G_1, Q \in G_2$, and $a, b \in \mathbb{F}_r$;
2. *non-degenerate*: $e(G, \hat{G}) \neq 1$, so that $e(G, \hat{G})$ generates G_T ; and
3. *efficiently computable*.

Bilinearity lets a verifier test one multiplication of hidden scalars—whether $c = ab$, given $[a]G$, $[b]\hat{G}$, and $[c]G$ —without learning any of them (Example 2.9); the KZG check of Construction 4.35 is such a test. The groups come from *pairing-friendly* elliptic curves, chosen with a small embedding degree (the quantity of Definition 2.24, there required large) so that G_T lives in a manageable extension field; the curve of the deployed Sapling protocol is BLS12-381, a Barreto–Lynn–Scott curve of embedding degree twelve (protocol specification § 5.4.9.2). The deployed system treats the pairing as a black box, and so does this volume.

The pairing-based lineage has three generations. The BCTV14 family (Ben-Sasson, Chiesa, Tromer, and Virza) is a preprocessing argument in the sense of Definition 9.23: the setup encodes one fixed arithmetic circuit into a structured reference string of group elements, the prover’s work is a handful of multiscalar multiplications over that string, and the proof is a constant number of group elements—eight, in the deployed encoding (protocol specification § 5.4.10.1)—checked by a constant number of pairing equations, whatever the circuit’s size. Groth’s refinement (Groth16) cuts the proof to *three* group elements, $\pi_A, \pi_C \in G_1$ and $\pi_B \in G_2$, and verification to a single pairing-product equation: three pairings— π_A against π_B , π_C against a fixed verification-key element, and a public-input-weighted combination of verification-key elements against another—compared with one pairing precomputed once per circuit. Deployed Zcash uses Groth16 over BLS12-381 for every Sapling spend and output proof (protocol specification § 5.4.10.2). PLONK, the third generation, is the recipe of Theorem 9.28 below with the KZG scheme of Construction 4.34 as its commitment: the gate-and-permutation arithmetisation Halo 2 also uses, each polynomial opening checked by two pairings, so proof size and verification cost stay constant—the strongest form of the succinctness clause of Definition 9.24.

The cost of this family is its setup. The reference string is produced from secret scalars—the evaluation point τ of Construction 4.34 and, in Groth16, further secrets that tie the proof elements to one circuit—which must be *destroyed* once the parameters are published: they are the *toxic waste*. Whoever retains them forges accepting proofs of false statements at will, exactly as the trapdoor τ forges KZG openings (Remark 4.37), and no verifier can distinguish such forgeries from honest proofs, so the failure of knowledge soundness is undetectable. For Groth16 the waste is *per circuit*—every new statement shape needs a fresh setup—whereas PLONK’s KZG string is universal, one powers-of- τ string serving every circuit up to a size bound, but still trusted. The mitigation is a *multiparty ceremony*: participants contribute secret randomness in turn, each re-randomising the running parameters and destroying its own share, so that the waste stays unknown as long as *one* participant was honest. Zcash ran such ceremonies for the parameters of its earlier proof systems—the BCTV14 Sprout circuit and the Groth16 Sapling circuits (protocol specification §§ 5.7–5.8)—and each remains sound only on that one-honest-participant assumption.

The other route rests on the discrete logarithm alone. The inner-product commitment of Construction 4.38 needs no pairing and no trusted setup—its generators are derived by hashing, with no secret behind them—at the price of logarithmic proofs and a linear-time opening check (Proposition 4.39); the Halo 2 proving system deployed in Orchard takes this route.

Polynomial interactive oracle proofs. Modern SNARKs are built in two layers, one information-theoretic and one cryptographic; the information-theoretic layer is formulated in terms of polynomials.

Definition 9.27 (Polynomial interactive oracle proof). Fix a finite field F . A *polynomial interactive oracle proof* (Poly-IOP) for a relation R over F is a public-coin interactive protocol in which, in place of ordinary messages, the prover in each round sends an *oracle* for a polynomial $p_i \in F[X]$ of some bounded degree; the verifier sends uniformly random field-element challenges, and may *query* each polynomial oracle at points of its choice, receiving the evaluation $p_i(\zeta)$. After the interaction the verifier accepts or rejects as a function of its challenges and the few evaluations it queried. Completeness and (knowledge) soundness are as for interactive proofs, with the soundness analysis purely algebraic—resting, invariably, on the Schwartz–Zippel lemma: a nonzero polynomial of degree at most d over F vanishes at a uniformly random point with probability at most $d/|F|$ (*Math Guide*, §“The Schwartz–Zippel lemma”). The Poly-IOPs used in practice have verifiers that make only $O(1)$ or $O(\log)$ queries, each a single evaluation.

The oracle is an idealisation—no real prover can supply an opaque object answering evaluation queries honestly—under which soundness of the Poly-IOP layer is *unconditional*; the cryptography enters when the oracles are instantiated.

Arithmetisation is how a computation becomes a Poly-IOP statement. The claim “I know w with $C(w) = 1$ ” (the relation R_{sat} of Example 9.2) is encoded as a small set of polynomial identities over F that hold if and only if the computation is correct. The computation is first expressed as a *constraint system*—the customisable *PLONKish* gate-and-permutation form used by Halo 2—whose satisfaction is equivalent to a handful of polynomials satisfying identities on a chosen eval-

uation domain, a fixed finite set of points with one point per constraint row. In the PLONKish form, polynomials encoding the wire values a, b, c and fixed selector polynomials q_L, q_R, q_M, q_O, q_C chosen at circuit-definition time must satisfy the gate identity

$$q_L \cdot a + q_R \cdot b + q_M \cdot ab + q_O \cdot c + q_C = 0$$

at every point of the domain—selectors switching each row among addition, multiplication, and custom behaviours—while a *permutation argument* enforces the copy constraints wiring one gate’s output to another’s input. A further step turns the requirement on the domain into one checkable at a single point: the identity is demanded only on the domain H , and a polynomial vanishes on all of H exactly when it is divisible by the vanishing polynomial $Z_H := \prod_{b \in H} (X - b)$ (*Math Guide*, §“Roots and the factor theorem”), so the prover also sends an oracle for the quotient t with

$$q_L \cdot a + q_R \cdot b + q_M \cdot ab + q_O \cdot c + q_C = t \cdot Z_H$$

as an identity of polynomials—one that must now hold at *every* point of $\mathbb{F}$. Checking it at a single random point is the Poly-IOP: the verifier queries the oracles at a random ζ , checks the displayed equation there, and by Schwartz–Zippel a false statement survives with probability at most $d/|\mathbb{F}|$, with d bounding the degree of either side.

The polynomial commitment scheme is the cryptographic half. The PCS of Definition 4.32 supplies what the idealised oracle provided: a short, binding commitment to a polynomial, with short proofs—verifiable against the commitment alone—that $p(\zeta) = v$; evaluation binding forbids two accepted values at one point, and the *extractability* the definition names is what SNARK compilation consumes: a prover whose evaluation proofs verify must “know” an underlying polynomial. Three instantiations span the design space: KZG (Construction 4.34)—pairing-based, constant-size, trusted setup; the inner-product-argument commitment over a single elliptic curve (Construction 4.38)—transparent, no trusted setup, logarithmic-size openings, the choice made by Halo 2; and the hash-based transparent schemes, outside the deployed path.

Theorem 9.28 (The compilation recipe, informal). *Let PIOP be a public-coin polynomial IOP for R , complete and knowledge-sound, and let PC be a polynomial commitment scheme that is correct, evaluation-binding, and extractable. Compile as follows: wherever the prover would send a polynomial oracle p_i , it sends $\text{Commit}(p_i)$ instead; wherever the verifier would query $p_i(\zeta)$, the prover sends the claimed value together with a PC evaluation proof, which the verifier checks against the commitment. Under compatible composition hypotheses the result is a public-coin interactive argument of knowledge for R , and a suitable multi-round Fiat–Shamir theorem in the ROM (Definition 9.18 gives the transform: every challenge a hash of the entire transcript so far) makes it non-interactive. Its proof is succinct; under the strict Definition 9.24 it is a SNARK only if verification is also sublinear in the proved computation. If, additionally, the Poly-IOP is properly blinded—random masking terms on the prover’s polynomials, hiding commitments (Remark 4.41)—and the corresponding simulation theorem applies, the result is zero knowledge.*

Proof sketch: layered extraction and the succinctness account. Completeness transfers verbatim: honest commitments open to honest evaluations, and the honest Poly-IOP verifier accepts.

Knowledge soundness is layered: the output of one extractor is the input of another. Given a successful prover against the compiled argument, the SNARK extractor first invokes PC *extractability* on each commitment, recovering actual polynomials $\hat{p}_i$ of the right degrees consistent with every evaluation the prover got accepted; this step is where the cryptographic assumption enters, and why the compiled object is an *argument*—a prover that could equivocate on its commitments would defeat it, and only computational binding rules that out. The extractor then hands the $\hat{p}_i$ to the Poly-IOP’s *unconditional* knowledge extractor, which is sound against *any* polynomials—in particular the extracted ones—and outputs $w \in R(x)$; evaluation binding guarantees the prover could not have answered queries inconsistently with the committed $\hat{p}_i$, so the algebraic soundness analysis applies to the extracted polynomials as run. Fiat–Shamir then converts the public-coin interactive argument to a non-interactive one in the ROM by the multi-round generalisation of Theorem 9.20: challenges become transcript hashes (Remark 9.19 governing what they must absorb), observability and forking supply the rewinding that the layered extractor needs—tree-style, per Definition 9.14—and the honest-verifier zero knowledge of the blinded protocol becomes full non-interactive zero knowledge by oracle programming.

Succinctness is a matter of counting, and it depends entirely on PC. The proof consists of a constant or logarithmic number of commitments and evaluation proofs—short by construction, sublinear in the circuit, independent of the witness—so the compiled *proof* is succinct whenever the commitment scheme’s objects are. The *verifier* is sublinear only when the evaluation checks are themselves succinct, as for KZG’s two pairings; with the inner-product commitment each evaluation check costs $O(d)$ group operations (Proposition 4.39), so the compiled verifier runs in time linear in the circuit while the proof stays succinct. The compiled object therefore satisfies every clause of Definition 9.24 when the PC openings verify succinctly, and otherwise yields succinct proofs with linear-time verification—the trade-off Halo 2 accepts and then offsets through batching. $\square$

Remark 9.29 (Halo 2 is the recipe, instantiated). The proving system deployed in Orchard (protocol specification § 5.4.10.3) instantiates the recipe of Theorem 9.28. The Poly-IOP is a PLONKish arithmetisation—custom gates declared as polynomial expressions, lookup arguments asserting that witness values lie in a declared table, and a permutation argument enforcing copy constraints. The polynomial commitment is the inner-product-argument scheme over a single elliptic curve, *transparent*—the generators are derived by hashing to the curve from the fixed domain string `Hal02-Parameters`, so the setup holds no trapdoor—with logarithmic-size openings. Fiat–Shamir over a concrete hash makes the argument non-interactive: the transcript is a running BLAKE2b state into which every prover message is absorbed, domain-separated by type, and out of which every challenge is squeezed—the entire-prefix discipline of Definition 9.18 enforced by construction.

The linear cost of each IPA opening check (Proposition 4.39) makes the deployed verifier’s work linear in the commitment dimension: Halo 2 has succinct *proofs* but does not meet the strict sublinear-verifier clause of Definition 9.24, and it is called a zk-SNARK under the broader convention that asks succinctness of the proof alone. The system’s characteristic technique, from which *Halo* takes its name, is *accumulation*: the one expensive step, the $O(d)$ final check of each IPA opening, is deferred and carried forward rather than performed inside a circuit, one proof attest-

ing to the verification of another—which is what makes *recursive* composition possible without a trusted setup. Deployed Zcash does not use recursion: each Orchard proof is verified directly, and what the deployed verifier takes from the Halo idea is the deferral alone, used for *batching*—many proofs’ deferred final checks combined into one multiscalar multiplication (consumed for Orchard bundles as described in Remark 4.40). These claims rest on model and composition hypotheses beyond the generic recipe (Remark 9.21).

The later volumes rely on rewinding (Remark 9.8), on knowledge soundness, and on the layered split of Theorem 9.28, which is the architecture of every modern proof system and the frame for auditing one, since algebraic and cryptographic defects reside in different layers.

10 Merkle trees and commitments to sets

This final section constructs, from the collision-resistant hash of §3 alone, the Merkle tree: a short commitment to a list of values with membership proofs of logarithmic size. It proves, by reduction to collision resistance, that the root is *binding*—no party can publish one root and later open it to two different lists (*equivocation*)—and that membership proofs are *sound*—no party can prove membership of a value absent from the committed list, which in a payment system would spend a note the ledger never recorded (Proposition 10.5, Theorems 10.12 and 10.25). It constructs the append-only tree of a shielded pool, updated in logarithmic time (Proposition 10.20), and makes membership *private* by hiding the path from leaf to root in a zero-knowledge argument of §9: an observer of an honest proof learns nothing about *which* member it concerns, information that would trace every coin (Theorem 10.29). The commitments of §4 are the leaves, and the algebraic hashes of §3 and §4 compute the internal nodes; the section ends with an Orchard shielded spend assembled end to end.

10.1 Two statements about a set

A prover holds a large collection of values—transactions in a block, public keys in a directory; in this volume, the growing set of note commitments of a shielded payment system. A verifier must be convinced of two statements about it without storing it or seeing most of it.

- *Membership*. “Value v occurs at position i of the list”—or, forgetting the position, “ v belongs to the set.”
- *Integrity*. “The collection is exactly the one committed to earlier, unaltered.”

Both are served by a single short *commitment* that determines the whole collection and short *membership proofs*, where “short” means of size independent of the number of elements, or at most logarithmic in it: integrity becomes one comparison, and membership a small certificate sent with the value. The *Merkle tree*, due to Ralph Merkle, achieves this from a collision-resistant hash function alone. It is the membership mechanism of Orchard: every shielded spend proves, against a published root, that some note commitment lies in the tree.

Notation follows §3.1: $\{0, 1\}^*$ is the set of finite binary strings, $\{0, 1\}^n$ the strings of length exactly n , $x \parallel y$ concatenation, and λ the security parameter. All algorithms are PPT in λ , and

$\text{negl}(\lambda)$ denotes a negligible function (*Math Guide*, §“Polynomial, exponential, and negligible functions”).

10.2 The compression function and collision resistance

The tree uses a single primitive, recalled in the keyed form in which its security is a meaningful asymptotic statement.

Definition 10.1 (Compression function). Let $H: \mathcal{K} \times \{0, 1\}^* \rightarrow \{0, 1\}^\ell$ be a keyed hash family exactly as in Definition 3.2: a key $s \in \mathcal{K}$ is sampled publicly at random and fixed, the digest length is $\ell = \ell(\lambda)$, and $H_s := H(s, \cdot)$ is the family member in force. When the input length is restricted to a fixed multiple of the output length, the family is a *compression function*; a *two-to-one* compression function has signature

$$h_s: \{0, 1\}^{2\ell} \rightarrow \{0, 1\}^\ell,$$

taking two digests to one.

The key s models the choice of a concrete function from a family. For a single fixed function a colliding pair exists and some small program outputs it (the hardcoded-collision triviality discussed in §3.1), so only the keyed formulation admits a non-trivial quantification over adversaries. Deployments use a fixed standardised function or, inside arithmetic circuits, an algebraic hash in the sense of §3.8 and §4.8; we suppress s and write h and H .

The tree needs collision resistance (Definition 3.7, §3.2): given the key s , every PPT adversary outputs $x \neq x'$ with $H_s(x) = H_s(x')$ only with probability $\text{negl}(\lambda)$. This is the strongest of the three classical hash notions (Proposition 3.8), and the generic birthday attack caps an ℓ -bit digest at $\ell/2$ bits of collision security (Lemma 3.11); hence the 256-bit outputs used for the 128-bit level in this section. Merkle trees require *full* collision resistance, not merely second-preimage resistance, because the party that assembles the tree may itself be the adversary and choose every leaf (Theorem 10.12 makes the reason exact); this is the regime with both inputs adversarial, which Remark 3.10 assigns to collision resistance.

10.3 Binary Merkle trees

Definition 10.2 (Binary Merkle tree). Fix a two-to-one compression function $h: \{0, 1\}^{2\ell} \rightarrow \{0, 1\}^\ell$, a depth $d \in \mathbb{N}$, and an ordered list $L = (L_0, \dots, L_{2^d-1})$ of 2^d leaf values in $\{0, 1\}^\ell$. The *Merkle tree of depth d over L* is the complete binary tree with 2^d leaves whose nodes carry labels in $\{0, 1\}^\ell$, defined level by level from the leaves (level 0) to the root (level d) by

$$N_{0,j} = L_j \quad (0 \leq j < 2^d), \quad N_{t+1,j} = h(N_{t,2j} \parallel N_{t,2j+1}) \quad (0 \leq t < d, 0 \leq j < 2^{d-t-1}). \quad (3)$$

Level t has 2^{d-t} nodes, and $N_{t,j}$ is the j -th node from the left at level t . The unique level- d node is the *root*: $\text{root}(L) := N_{d,0}$.

Example 10.3 (A depth-two tree). With $d = 2$ and leaves L_0, L_1, L_2, L_3 , the tree has seven nodes: the four leaves, the two level-one nodes

$$N_{1,0} = b(L_0 \parallel L_1), \quad N_{1,1} = b(L_2 \parallel L_3),$$

and the root

$$\text{root}(L) = N_{2,0} = b(b(L_0 \parallel L_1) \parallel b(L_2 \parallel L_3)).$$

The single ℓ -bit root depends on the whole list of four ℓ -bit strings.

A list whose length m is not a power of two is handled by fixing the depth d with $2^d \geq m$ and padding the remaining $2^d - m$ leaf slots with a canonical value, for instance the all-zero string or the hash of the empty string. Padding must not create ambiguity: two different lists must never yield the same padded leaf sequence, since otherwise the root fails to determine the list whatever the properties of the hash. Binding the true length m into the root's computation—one further compression of the top node with an encoding of m —removes any such ambiguity. In the alternative *unbalanced* tree, a node with a single child passes its child's label upward unchanged; Remark 10.16 treats it together with second-preimage attacks.

That the root *commits* to the list is a security property, stated as a game in the style of Definition 1.15.

Definition 10.4 (Merkle commitment; binding). The *Merkle commitment* to a list L of 2^d leaves is $\text{root}(L) \in \{0, 1\}^\ell$. The game $\text{BIND}_{H,d}^A(\lambda)$ runs as follows.

1. The challenger samples $s \xleftarrow{\$} \mathcal{K}$, fixing the compression function $b := H_s$, and sends s to the adversary.
2. The adversary outputs two lists L, L' , each of 2^d leaves in $\{0, 1\}^\ell$.
3. The game outputs 1 iff $L \neq L'$ and $\text{root}(L) = \text{root}(L')$.

The commitment is *binding* if every PPT adversary wins with probability $\text{negl}(\lambda)$.

Proposition 10.5 (Binding of the root). *If b is collision resistant, the Merkle commitment is binding. Concretely, there is a deterministic procedure that, from any two distinct equal-length lists $L \neq L'$ with $\text{root}(L) = \text{root}(L')$, extracts a collision for b in time linear in the tree size—one root-to-leaf walk of $O(d)$ hash comparisons. The reduction is tight: an adversary winning BIND with probability ε yields a collision finder succeeding with the same probability ε .*

Proof. Build both trees. The lists differ at some leaf index j_0 , so $N_{0,j_0} \neq N'_{0,j_0}$, writing N and N' for the labels of the two trees. Follow the leaf-to-root path

$$N_{0,j_0}, N_{1,\lfloor j_0/2 \rfloor}, N_{2,\lfloor j_0/4 \rfloor}, \dots, N_{d,0}$$

in both trees. The labels differ at level 0 and agree at level d —the common root—so there is a *smallest* level t at which they first agree: with $j = \lfloor j_0/2^t \rfloor$,

$$N_{t,j} = N'_{t,j}, \quad \text{yet} \quad N_{t-1,2j} \neq N'_{t-1,2j} \quad \text{or} \quad N_{t-1,2j+1} \neq N'_{t-1,2j+1},$$

since one of the two children at level $t - 1$ is the path node, at which the labels still disagree by minimality of t . By the recurrence (3),

$$h(N_{t-1,2j} \parallel N_{t-1,2j+1}) = N_{t,j} = N'_{t,j} = h(N'_{t-1,2j} \parallel N'_{t-1,2j+1}),$$

while the two 2ℓ -bit inputs differ in at least one half: a collision for h , located after at most d comparisons along one path. The reduction built on this extractor runs the BIND adversary once and extracts whenever it wins, so collision resistance forces the winning probability to be negligible. $\square$

10.4 Authentication paths and membership proofs

Membership needs a certificate relating one leaf to the root: the siblings along the leaf's path, which are exactly the data needed to recompute the root from the leaf.

Definition 10.6 (Authentication path). Let a depth- d tree over L be given, and let i be a leaf index, $0 \leq i < 2^d$, with binary expansion $i = b_{d-1}b_{d-2} \dots b_0$, so that bit b_t records whether the level- t node on the leaf-to-root path is a left child ($b_t = 0$) or a right child ($b_t = 1$). The *authentication path* (also *Merkle proof* or *membership witness*) for index i is

$$\pi_i = ((s_0, b_0), (s_1, b_1), \dots, (s_{d-1}, b_{d-1})),$$

where s_t is the label of the *sibling* of the level- t node on the path. Explicitly, with $j_t = \lfloor i/2^t \rfloor$ the path node's index at level t ,

$$s_t = \begin{cases} N_{t,j_t+1} & \text{if } b_t = 0 \quad (\text{sibling on the right}), \\ N_{t,j_t-1} & \text{if } b_t = 1 \quad (\text{sibling on the left}). \end{cases}$$

The path has exactly d entries.

Definition 10.7 (Path verification). The verifier $\text{Verify}(rt, i, v, \pi_i)$, on a root rt , an index i , a claimed leaf value $v \in \{0, 1\}^\ell$, and a path $\pi_i = ((s_0, b_0), \dots, (s_{d-1}, b_{d-1}))$, proceeds in two stages. First the *index-consistency check*: reject unless, for every t , the bit b_t carried in π_i equals bit t of the binary expansion of i . Then set $a_0 := v$ and fold the path upward: for $t = 0, \dots, d-1$,

$$a_{t+1} := \begin{cases} h(a_t \parallel s_t) & \text{if } b_t = 0, \\ h(s_t \parallel a_t) & \text{if } b_t = 1. \end{cases} \quad (4)$$

Accept iff $a_d = rt$. We call a_d the *root recomputed from* (i, v, π_i) . The bit b_t places the running value on the side matching the argument order of the tree-building recurrence: a left child is hashed on the left.

Proposition 10.8 (Completeness). For every list L , every index i , and π_i the genuine authentication path for i in the tree over L ,

$$\text{Verify}(\text{root}(L), i, L_i, \pi_i) = \text{accept}.$$

Proof. The index-consistency check passes by construction, the genuine path carrying exactly the bits of i . We show by induction on t that $a_t = N_{t, \lfloor i/2^t \rfloor}$, the label of the level- t node on the leaf-to-root path. The base is $a_0 = L_i = N_{0,i}$. For the step, the level- t path node and its sibling s_t are precisely the two children of the level- $(t+1)$ path node, and the fold (4) places a_t on the side dictated by b_t with s_t on the other side—reproducing exactly the two arguments of h in the recurrence (3), so $a_{t+1} = N_{t+1, \lfloor i/2^{t+1} \rfloor}$. At $t = d$ this reads $a_d = N_{d,0} = \text{root}(L)$, and the verifier accepts. $\square$

Proposition 10.9 (Logarithmic proof size). *An authentication path in a depth- d tree consists of d sibling labels and d direction bits, $d\ell + d$ bits in all, and verification performs exactly d evaluations of h . Choosing the minimal sufficient depth $d = \lceil \log_2 m \rceil$ for a list of $m \geq 2$ leaves (any $2^d \geq m$ suffices) yields proofs of $\lceil \log_2 m \rceil (\ell + 1)$ bits—logarithmic in the length of the list.*

Proof. One sibling label, one direction bit, and one compression per level, over the d levels above the leaves. $\square$

For a set of a billion elements, $m = 2^{30} = 1\,073\,741\,824$, a membership proof consists of 30 hash values and 30 direction bits. With a 256-bit hash the sibling labels occupy $30 \cdot 256 = 7680$ bits, which is 960 bytes; with the direction bits, 964 bytes, less than a kilobyte for any arrangement of the billion elements. (The byte counts are the output of the volume’s companion script.) The verifier stores one digest, and the certificate fits in a single network packet.

Example 10.10 (A depth-two membership proof). Continue Example 10.3, and prove membership of L_2 at index $i = 2 = (10)_2$, so $b_0 = 0$ and $b_1 = 1$. At level 0 the path node $N_{0,2} = L_2$ is a left child, with sibling $s_0 = N_{0,3} = L_3$. At level 1 the path node $N_{1,1} = h(L_2 \parallel L_3)$ is a right child, with sibling $s_1 = N_{1,0} = h(L_0 \parallel L_1)$. The path is

$$\pi_2 = ((L_3, 0), (h(L_0 \parallel L_1), 1)).$$

Verification folds upward:

$$a_1 = h(L_2 \parallel L_3), \quad \text{then} \quad a_2 = h(h(L_0 \parallel L_1) \parallel a_1) = \text{root}(L),$$

and the verifier accepts. The verifier never receives L_0 or L_1 , only their digest $h(L_0 \parallel L_1)$, and is nevertheless convinced that L_2 occupies index 2 of the committed list.

10.5 Soundness: forging a path implies a collision

Soundness states that, against an honest root, a path for a value other than the genuine leaf verifies only with negligible probability.

Definition 10.11 (Path forgery). Fix a list L of 2^d leaves with root $rt = \text{root}(L)$. The game $\text{PATHFORGE}_{H,L}^A(\lambda)$ runs as follows.

1. The challenger samples $s \xleftarrow{\$} \mathcal{K}$ to fix $h := H_s$, builds the tree over L , and sends s together with L —equivalently, the whole tree—to the adversary.

2. The adversary outputs an index i and a pair (v^*, π^*) .
3. The game outputs 1 iff $\text{Verify}(rt, i, v^*, \pi^*) = \text{accept}$ and $v^* \neq L_i$.

A winning output is a *path forgery at index i* : the verifier accepts, against the honestly computed root, a leaf value different from the genuine one at position i .

Theorem 10.12 (Soundness of Merkle membership proofs). *Let h be collision resistant. Then no PPT adversary, given the list L and the whole tree, produces a path forgery except with probability $\text{negl}(\lambda)$. In constructive form: there is a deterministic extractor that from any list L , index i , and accepting forgery (v^*, π^*) with $v^* \neq L_i$ outputs a collision for h , in time linear in the tree size—recomputing the genuine leaf-to-root labels from L dominates; given the built tree, one verifier run plus a level-by-level comparison, $O(d)$ work in all, completes the extraction.*

Proof. Run the verifier's fold (4) on (i, v^*, π^*) , obtaining the chain $a_0^* = v^*, a_1^*, \dots, a_d^* = rt$ (the final equality because the forgery is accepting). Alongside it place the genuine chain $a_t = N_{t, \lfloor i/2^t \rfloor}$, the labels of the leaf-to-root path in the honest tree; by the induction of Proposition 10.8 this is the chain the fold produces on the genuine inputs, and $a_d = rt$.

The two chains disagree at the bottom, $a_0^* = v^* \neq L_i = a_0$, and agree at the top, $a_d^* = a_d = rt$. Hence

$$t^* := \min \{ t : a_t^* = a_t \} \in \{1, \dots, d\}$$

exists, and $a_{t^*-1}^* \neq a_{t^*-1}$ by minimality. Because the forgery is accepting, the index-consistency check of Definition 10.7 forces the direction bits carried in π^* to equal the bits of i —the same bits the genuine path carries—so at level $t^* - 1$ both chains place their running value on the same side of the compression call: both compute $h(\cdot \parallel \text{sibling})$ if $b_{t^*-1} = 0$, both $h(\text{sibling} \parallel \cdot)$ if $b_{t^*-1} = 1$. Form the two 2ℓ -bit preimages accordingly,

$$X^* = \begin{cases} a_{t^*-1}^* \parallel s_{t^*-1}^* & \text{if } b_{t^*-1} = 0, \\ s_{t^*-1}^* \parallel a_{t^*-1}^* & \text{if } b_{t^*-1} = 1, \end{cases} \quad X = \begin{cases} a_{t^*-1} \parallel s_{t^*-1} & \text{if } b_{t^*-1} = 0, \\ s_{t^*-1} \parallel a_{t^*-1} & \text{if } b_{t^*-1} = 1, \end{cases}$$

where $s_{t^*-1}^*$ is the sibling claimed in π^* and s_{t^*-1} the genuine sibling. Then

$$h(X^*) = a_{t^*}^* = a_{t^*} = h(X), \quad X^* \neq X,$$

the inequality because the two preimages differ in the half occupied by the running value. The pair (X^*, X) is a collision for h , found deterministically in $O(d)$ work once the genuine chain is at hand—a read-off from the built tree, or $2^d - 1$ hashes from the bare list L . $\square$

If a PPT adversary $\mathcal{A}$ produces path forgeries with probability $\varepsilon(\lambda)$, the collision finder $\mathcal{B}$ that receives the key, samples or receives L , runs $\mathcal{A}$, and applies the extractor outputs a collision whenever $\mathcal{A}$ succeeds—hence with probability $\varepsilon(\lambda)$, at an overhead of building the tree over L , which $\mathcal{B}$ does in any case to play the challenger, plus one verifier run and $O(d)$ comparisons. Collision resistance of h therefore forces $\varepsilon(\lambda) \in \text{negl}(\lambda)$, and the reduction is tight.

Remark 10.13 (Scope: a fixed honest root). Theorem 10.12 is a statement about a *fixed honest root*: relative to a root that genuinely is $\text{root}(L)$, the only leaf value admitting an accepting path at position i is L_i , up to a collision in h . It does not prevent an adversary free to *choose* the root from committing to a maliciously crafted tree; binding (Proposition 10.5) rules that out. With an untrusted party supplying the root, both properties are necessary: binding ensures the root determines at most one list, and path soundness ensures openings are faithful to it. The vector-commitment theorem of §10.8 establishes both by a single argument.

10.6 Layer and domain separation

The soundness theorem does not cover the following structural confusion. Suppose the leaves are arbitrary-length data, hashed into $\{0, 1\}^\ell$ by the *same* function used internally. Then an internal label $h(A \parallel B)$ is itself a well-formed leaf value—it has the right length. An adversary presents the internal node $N_{1,0} = h(L_0 \parallel L_1)$ as a *leaf* near the top of a shallower tree, with the correspondingly shorter authentication path, and a verifier that does not know the depth accepts. The root then has a second list that produces it: a second-preimage attack that involves no collision of h . Unless each label determines whether it is a leaf or an internal node, the leaf-to-root structure on which Theorem 10.12 relies fails.

The remedy applies the domain-separation discipline of §3.5 *within* a single structure.

Definition 10.14 (Layer and domain separation). A Merkle construction has *domain separation* if the function applied at distinct structural positions is forced to differ, by prepending a position-dependent tag to every hash input. Concretely, from a variable-input-length hash H define

$$\text{LeafHash}(v) := H(0x00 \parallel v), \quad \text{NodeHash}_t(A, B) := H(0x01 \parallel \langle t \rangle \parallel A \parallel B),$$

where the byte tags $0x00$ and $0x01$ distinguish the *layer kind*—leaf versus internal node—and $\langle t \rangle$ is a fixed-length encoding of the level index, distinguishing internal nodes by depth. The tree recurrence becomes

$$N_{0,j} = \text{LeafHash}(L_j), \quad N_{t+1,j} = \text{NodeHash}_{t+1}(N_{t,2j}, N_{t,2j+1}),$$

and the verifier’s fold applies the same tagged functions at the same positions.

Proposition 10.15 (Separation forces within-domain collisions). *With the tagging of Definition 10.14, any collision extracted by the binding proof (Proposition 10.5) or the soundness proof (Theorem 10.12) is a collision for H between two inputs sharing the same leading tag: a leaf preimage against a leaf preimage, or a level- t node preimage against a level- t node preimage. In particular, no accepting forgery can substitute an internal node for a leaf, or a node at one level for a node at another, without exhibiting a within-domain collision of H —which collision resistance forbids.*

Proof sketch. The extractors of both proofs locate the smallest level at which two disagreeing computations agree, and collide the two preimages of the compression call at that position. Under

the tagged recurrence, the verifier’s recomputation applies the position’s prescribed function exactly as the honest tree does: at the leaves both preimages pass through `LeafHash`, at level $t \geq 1$ both pass through `NodeHasht`. Both colliding preimages therefore begin with the same tag—`0x00`, or `0x01 || (t)`—and the collision lies within a single domain. A cross-domain confusion, by contrast, would require two inputs with *different* leading tags and equal H -images, which is again an H -collision; in either case the adversary has broken collision resistance, and the shallower-tree attack above is excluded because a leaf preimage and a node preimage can no longer be the same string. $\square$

Remark 10.16 (Unbalanced trees, padding, and a deployed failure). The handling of lists whose length is not a power of two has failed in deployment. Bitcoin’s transaction Merkle tree handles an odd level by *duplicating* its last node before hashing pairwise, and the duplication rule is not injective: a list of transactions and the same list with its tail duplicated can yield the same root. This is CVE-2012-2459, a consensus-splitting vulnerability—an attacker could present two *different* transaction lists sharing a root, one valid and one invalid, and nodes could be made to cache the invalid one as permanently rejected (documented in Zebra: the zebra-chain crate reproduces the upstream warning that the flawed algorithm must not be reused, and the deployed defence in zebra-consensus rejects any block whose transaction hashes are not unique). The failure yields three general rules. First, layer-separate leaves from internal nodes, as Definition 10.14 prescribes. Second, make every padding or duplication rule *injective* on admissible lists—for instance by binding the true leaf count into the root’s computation. Third, feed `NodeHash` only fixed-length inputs, so that no length-extension structure of the underlying hash (§3.5) can intervene between layers.

Remark 10.17 (Algebraic hashes in the tree; Orchard’s MerkleCRH). When the Merkle path is verified inside a zero-knowledge circuit (§10.9), arithmetic gates over $\mathbb{F}_p$ evaluate the compression function, and the choice of h dominates the cost. A bit-oriented hash such as SHA-256 costs tens of thousands of constraints per call; one therefore uses an *algebraic* hash with compression $h: \mathbb{F}_p^2 \rightarrow \mathbb{F}_p$. Poseidon (§3.8) is a low-degree permutation-based map computable in few constraints; Sinsemilla (§4.8) is cheap in constraints through lookups and incomplete curve additions, *not* through low degree. For Poseidon, layer and domain separation come not from byte tags but from distinct field constants—initial capacity values or round constants—fed into the permutation; the tree’s security argument is identical, with “collision” read as a collision of the field-valued compression function.

Orchard’s Merkle hash MerkleCRH retains the tag mechanism of Definition 10.14 in encoded form. Each internal call prepends the 10-bit encoded layer index to the two 255-bit child encodings—a single fixed input length of 520 bits (a total the volume’s companion script checks), exactly the fixed-length-per-personalisation regime in which the deployed Sinsemilla hash is collision resistant (§4.8)—with *all* layers sharing one Sinsemilla instance (protocol specification § 5.4.1.3). Separation between *distinct* Sinsemilla instances comes from the personalisation string—here `z.cash:Orchard-MerkleCRH`—fed to hash-to-curve to derive the instance-specific base point Q , while the generator table S is shared across all instances (the base point is derived from the personalisation under the hash-to-curve domain `z.cash:SinsemillaQ`, the table under the fixed `z.cash:SinsemillaS`). Sinsemilla is collision resistant under a discrete-logarithm assumption (Proposition 4.29), so the tree’s hashing and its commitment role coincide in one Pedersen-like

map.

10.7 Append-only and incrementally updatable trees

The set of note commitments of a shielded pool only grows, one leaf at a time. Rebuilding a depth-32 tree on every insertion would cost 2^{32} hashes; the frontier defined below permits the update in d hashes and $O(d)$ state.

Definition 10.18 (Append-only Merkle tree). Fix a maximum depth d , giving capacity 2^d , and the level-indexed NodeHash functions of Definition 10.14. Leaf values here enter as level-0 labels directly, with no LeafHash: the leaves are already digests, and the fixed depth together with the level tags (Remark 10.17) carries the layer separation—the convention of deployed Orchard, whose leaf is the extracted coordinate itself (Example 10.31). An *append-only* (or *incremental*) *Merkle tree* maintains a position counter m —the number of leaves inserted so far, $0 \leq m \leq 2^d$ —and supports one mutating operation $\text{Append}(v)$, which places v at leaf index m and increments m . Empty leaf slots $m, \dots, 2^d - 1$ carry a fixed *empty-leaf* value $\perp_0$, and the labels of entirely empty subtrees follow by precomputation:

$$\perp_{t+1} := \text{NodeHash}_{t+1}(\perp_t, \perp_t), \quad 0 \leq t < d.$$

The *root after m appends*, written root_m , is the root of the depth- d tree, under this recurrence, whose first m level-0 labels are the inserted values and whose remaining ones are $\perp_0$.

Definition 10.19 (Frontier). The *frontier* of an append-only tree holding m leaves is the minimal state needed to (i) compute the current root and (ii) update incrementally on the next Append. For each level $t \in \{0, \dots, d-1\}$ it holds at most one *carry*: the label of the left child of the next node to be completed at level $t+1$, retained precisely when leaf index m has a 1 bit at position t —a completed left sibling awaiting its right neighbour. Equivalently, writing m in binary as $b_{d-1} \dots b_0$, the frontier stores, for each t with $b_t = 1$, the already-finalised left-sibling label f_t at level t along the rightmost filled path. Its size is at most d labels—one per set bit of m , hence $O(\log m)$ state.

The binary representation of m decomposes the inserted leaves into completed perfect subtrees of distinct heights, one of height t for each set bit b_t , and the frontier holds exactly their roots. An append is the Merkle analogue of incrementing a binary counter: a carry at level t combines two height- t subtrees into one of height $t+1$, and the chain of carries is as long as the run of trailing 1-bits of m .

Proposition 10.20 (Incremental append in logarithmic time). *From the frontier of an append-only tree holding $m < 2^d$ leaves, $\text{Append}(v)$ computes the new frontier and the new root root_{m+1} using at most d evaluations of NodeHash and $O(d)$ additional work, while storing only $O(d)$ labels throughout.*

Proof. The algorithm is carry propagation followed by an upward fold. Set the running label $c \leftarrow v$ at level $t \leftarrow 0$. While bit t of m is 1, the new path node at level t is a right child whose left sibling

is the stored carry f_t : set $c \leftarrow \text{NodeHash}_{t+1}(f_t, c)$, clear the carry f_t , and increment t . When bit t of m is 0, store c as the new carry f_t and stop. Writing z for the number of trailing 1-bits of m , the carry phase costs exactly z hashes and deposits f_z at level z —the root of the just-completed height- z subtree containing the new leaf, matching the binary increment $m \mapsto m + 1$, which clears z trailing ones and sets bit z .

To obtain root_{m+1} , fold upward from level z with the just-deposited f_z as the running value—it is the path label at level z , not a sibling. For $t = z, \dots, d - 1$, combine the running value with the stored carry f_t as left sibling when bit t of m is 1—those carries lie above the cleared run and survive the append untouched—and with the precomputed empty-subtree label $\perp_t$ as right sibling otherwise; at level z itself bit z of m is 0, so the first combination is with $\perp_z$, never with the deposited carry. One hash per level makes $d - z$ hashes in all (zero if $z = d$, when the tree has just filled). The total is exactly $z + (d - z) = d$ evaluations of NodeHash , and the retained state is the at-most- d carries: precisely the frontier for $m + 1$ leaves. $\square$

Example 10.21 (The frontier as a binary counter). Take depth $d = 3$ (capacity 8) and insert v_0, v_1, v_2 in turn; the frontier evolves exactly as the binary counter of the leaf count m .

- After v_0 , $m = 1 = (001)_2$: one completed height-0 subtree; the frontier is $\{f_0 = v_0\}$.
- After v_1 , $m = 2 = (010)_2$: leaf v_1 at index 1 is a right child, triggering one carry $c = \text{NodeHash}_1(v_0, v_1)$ —a completed height-1 subtree; the frontier is $\{f_1 = c\}$, with f_0 cleared.
- After v_2 , $m = 3 = (011)_2$: leaf v_2 at index 2 is a left child and becomes a new height-0 carry; the frontier is $\{f_1 = \text{NodeHash}_1(v_0, v_1), f_0 = v_2\}$ —exactly the two set bits of 3.

Each append touched only the carry chain to the new leaf, never the whole tree. (The trace involves no numeric hash values and follows mechanically from the algorithm of Proposition 10.20; the volume’s companion script replays it symbolically and checks the occupied frontier levels against the set bits of m at every step.)

Appending changes the root; the following definition fixes the root against which a proof is valid.

Definition 10.22 (History binding). An append-only tree is *history-binding* when the surrounding protocol treats every historical root as an acceptable commitment in its own right: each membership proof verifies against the root it was formed for, and appending further leaves never invalidates a previously valid $(\text{root}, \text{leaf}, \text{path})$ triple for that root. The soundness of each individual proof is exactly Theorem 10.12 applied to the corresponding fixed root.

Deployed Orchard is history-binding by construction. The chain records each block’s note-commitment-tree root—its *anchor*—and a spend may prove membership against *any* recorded historical anchor: consensus requires the anchor named by a transaction to be the root of the Orchard note-commitment tree as it stood at the end of some earlier block, checked by membership in the set of recorded anchors (protocol specification § 3.7, ‘Action Transfers and their Descriptions’). The frontier is likewise the deployed representation of the growing tree (in the `incrementalmerkletree` crate; the `orchard` crate precomputes the empty-subtree labels).

Remark 10.23 (Why append-only, and what it costs). Deletion or in-place update could be supported at $O(\log m)$ per operation, given the relevant siblings; the restriction to appends is a design choice, made for monotonicity. Once a leaf is inserted it is never removed, so a membership proof valid against the root at insertion time remains valid against *that historical root* forever. A shielded pool requires this property: a note, once added, can be spent later by proving against some past anchor, without the spender following the current root as other parties' insertions change it. The cost falls on the verifier, which must accept a window of historical roots rather than a single current one; in the deployed protocol that window is the set of recorded anchors.

10.8 Vector commitments

The Merkle construction instantiates the following abstraction: a commitment to an *ordered vector* that opens one position at a time. Its unordered counterpart, the *cryptographic accumulator*, certifies set membership with no notion of position; Orchard does not use it, since notes occupy positions and the position is part of the spend circuit's witness.

Definition 10.24 (Vector commitment). A *vector commitment* is a tuple of PPT algorithms $(\text{Setup}, \text{Commit}, \text{Open}, \text{Vf})$:

- $\text{Setup}(1^\lambda, n) \rightarrow pp$ produces public parameters for vectors of length n ;
- $\text{Commit}(pp, (v_0, \dots, v_{n-1})) \rightarrow (C, aux)$ outputs a short commitment and auxiliary state;
- $\text{Open}(pp, i, aux) \rightarrow \pi_i$ produces a proof that position i holds v_i ;
- $\text{Vf}(pp, C, i, v, \pi) \rightarrow \{\text{accept}, \text{reject}\}$ verifies an opening.

The scheme is *correct* if honest openings always verify, and *succinct* if $|C|$ and $|\pi_i|$ are bounded by a fixed polynomial in λ and $\log n$. It is *position-binding* if every PPT adversary wins the following game with probability $\text{negl}(\lambda)$: given pp , the adversary outputs (C, i, v, v', π, π') and wins iff $v \neq v'$ while both $\text{Vf}(pp, C, i, v, \pi)$ and $\text{Vf}(pp, C, i, v', \pi')$ accept. The adversary chooses C itself: binding must hold even for maliciously formed commitments, with no honest committer in the game.

Theorem 10.25 (Merkle trees are succinct position-binding vector commitments). *Instantiate the syntax by: Setup samples h (and fixes d with $n = 2^d$); Commit computes the tree and outputs $C = \text{root}$ with aux the full tree; Open returns the authentication path; and Vf = Verify. If h is collision resistant, this is a correct, succinct, position-binding vector commitment, with commitment size ℓ and opening size $O(\ell \log n)$.*

Proof. Correctness is completeness (Proposition 10.8), and the size bounds are Proposition 10.9. For position-binding, suppose an adversary outputs two accepting openings (v, π) and (v', π') at the same index i under the same root C , with $v \neq v'$. Run the verifier's fold (4) on each, obtaining two recomputed chains that disagree at the bottom ($v \neq v'$) and agree at the top (both reach C , since both openings accept). The index-consistency check forces both paths to carry the direction bits of i , so at every level both chains place their running value on the same side of the compres-

sion call. The first-level-of-agreement extraction of Theorem 10.12 applies verbatim to this pair of chains and yields two distinct 2ℓ -bit inputs with the same h -image: a collision. The argument uses *no honest reference tree*—it compares the two forged openings with each other—so it applies even to a maliciously chosen root, which is the strong form of position-binding the definition requires. $\square$

Remark 10.26 (Position-binding versus path soundness). The two theorems serve different verifiers. Path soundness (Theorem 10.12) fixes an *honest* root and forbids opening a position to anything but its true value, for a verifier that trusts the origin of the root. The vector-commitment theorem (Theorem 10.25) forbids opening a single position two *different* ways even under an *adversarial* root, for a verifier that received the root from an untrusted source; it expresses the intended meaning of “commitment”: a commitment C determines at most one value per position, whoever produced C . Both proofs collide two chains at their first level of agreement, forged against genuine in the first case and forged against forged in the second.

10.9 Privacy-preserving membership via zero-knowledge paths

The membership proof of §10.4 is succinct but not private: it discloses the leaf value (here, the note commitment being spent) and its index i , and the sibling labels can leak structural information about the neighbourhood of the leaf. An observer correlating spends with the appends that created their leaves would link payments end to end. The remedy proves the *existence* of a valid path in zero knowledge, with the entire path a private witness and only the root public.

Definition 10.27 (The membership relation). For the domain-separated Merkle construction of depth d , with leaves entering as level-0 labels under the convention of Definition 10.18, define

$$R_{\text{mem}} := \{ (rt; (v, i, \pi)) : \text{Verify}(rt, i, v, \pi) = \text{accept} \},$$

with the *statement* the public root rt and the *witness* comprising the leaf v , its index i , and the authentication path π . A zero-knowledge argument for R_{mem} lets a prover convince a verifier that *some* leaf lies in the tree with root rt while revealing none of v , i , or π . We assume a zk-SNARK as §9 develops it (Definition 9.24): *completeness* (honest provers convince), *knowledge soundness* (an accepting prover “knows” a witness, formalised by an extractor), and *zero knowledge* (the proof reveals nothing beyond the truth of the statement, formalised by a simulator).

Construction 10.28 (In-circuit path verification). The Merkle-path circuit C_{mem} of depth d takes as public input the root rt , and as private (witness) inputs the leaf v , the direction bits $b_0, \dots, b_{d-1}$, and the siblings $s_0, \dots, s_{d-1}$. It implements the d compression calls of the fold (4)—the level- t call being the position’s prescribed function, NodeHash_{t+1} in the domain-separated construction, with no LeafHash call since the leaf enters as the level-0 label (Definition 10.18)—each preceded by the conditional swap that selects the argument order (a_t, s_t) or (s_t, a_t) according to b_t , and enforces the single output constraint $a_d = rt$. Its size is d times the cost of one compression call plus $O(d)$ constraints for the swaps. The position enters *only* through the bits $b_0, \dots, b_{d-1}$ —the binary decomposition of i —so the index-consistency check of Definition 10.7 is enforced structurally: there is no separate index to disagree with its bits. Deployed Orchard

matches this shape exactly: the circuit’s Merkle gadget decomposes the witnessed position into little-endian bits and applies a conditional swap per layer (protocol specification § 4.9, ‘Merkle Path Validity’).

Theorem 10.29 (Privacy-preserving membership). *Let h be collision resistant and let (P, V) be a zk-SNARK for R_{mem} —equivalently, for satisfiability of C_{mem} . Then the protocol “the prover sends a SNARK proof for the statement rt ; the verifier checks it” is a membership argument that is:*

1. *complete: a prover holding a genuine leaf and its path produces an accepting proof;*
2. *sound: any prover producing an accepting proof for an honest root $rt = \text{root}(L)$ knows—via the SNARK extractor—a witness (v, i, π) with $\text{Verify}(rt, i, v, \pi) = \text{accept}$, and v equals the genuine leaf L_i , unless a collision in h is thereby found: the proven leaf is a real member of the committed list;*
3. *private: the proof reveals nothing about v, i , or π beyond the fact that some valid leaf exists under rt .*

Proof idea. Completeness composes SNARK completeness with tree completeness (Proposition 10.8): the genuine (L_i, i, π_i) satisfies C_{mem} , and the honest prover proves it. Soundness composes the SNARK knowledge extractor with the path-forgery-to-collision extractor of Theorem 10.12: from an accepting prover, the SNARK extractor produces (v, i, π) accepted by Verify ; if $v \neq L_i$, that output is precisely a path forgery, hence yields a collision for h , hence occurs with probability $\text{negl}(\lambda)$. Zero knowledge is inherited directly from the SNARK simulator, which produces from rt alone a proof indistinguishable from the real one; hence the real proof conveys nothing about v, i , or π . $\square$

Remark 10.30 (Public root, private path; deployment trade-offs). In R_{mem} the root must be *public* so that the verifier checks the proof against a state everyone agrees on—a previously published note-commitment-tree anchor—and soundness is relative to that public root, as Remark 10.13 delimits. The path must be *private* so that observers cannot link the spend to a particular leaf. SNARK succinctness adds a third property: the verifier checks a proof whose size is essentially independent of d —polylogarithmic—even though the witness contains d siblings, so the construction *compresses* the membership proof as well as *hiding* it.

Deployments differ in where they bear the cost of that verification. Pairing-based schemes such as KZG (§4.9) achieve constant-size proofs with fast verification, at the price of a trusted setup. Halo 2’s IPA-based proofs grow logarithmically in size, but their verification is *linear* in the circuit size; the deployment amortises it by batch-verifying many proofs in a single multiexponentiation. The accumulation technique of the Halo lineage—which would make per-proof verifier work sublinear by deferring the linear check across recursively composed proofs—is not deployed: each Orchard proof is verified, in batch, directly.

10.10 A shielded spend, end to end

This subsection composes the constructions of the volume into an Orchard shielded spend; every primitive it uses is constructed in this volume, and the description follows deployed Orchard.

Example 10.31 (An Orchard shielded spend). Each shielded output of a transaction appends a *note commitment* cm —itself a hiding, binding Sinsemilla commitment to the note’s value, recipient, and randomness (§4.8)—as a leaf of an append-only Merkle tree built with the Sinsemilla compression function of Remark 10.17. Strictly, the leaf is the extracted x -coordinate

$$\text{cm}_x = \text{ExtractP}(\text{cm}),$$

a 255-bit Pallas base-field element (*Math Guide*, §“Pallas and Vesta assembled”) matching the child encodings of Remark 10.17. The protocol periodically publishes the current root—the *anchor*—on chain, one per block.

To spend a note, its owner:

1. locates the leaf cm_x in the public tree and reads off its authentication path π and position i ;
2. chooses a recent anchor $rt = \text{root}_m$ whose tree already contains the leaf—any recorded historical anchor serves, per §10.7;
3. proves in zero knowledge, per Theorem 10.29, the statement rt with witness (cm_x, i, π) : membership of the note commitment under the anchor, well-formedness of the note opening, and correct derivation of a *nullifier*—a deterministic tag whose on-chain uniqueness prevents double-spending, computed from the note and the spender’s nullifier key on $\text{PRF}^{\text{nfOrchard}}$ of §3.8 by a construction of a later volume (the recomputed root is constrained to equal the public anchor, and the instance carries the anchor and the nullifier; protocol specification § 4.9, ‘Merkle Path Validity’, and § 4.18.4, ‘Action Statement’);
4. publishes the proof and the nullifier—but neither cm , nor i , nor π .

By Theorem 10.29, the spend convinces the network that the spender owns *some* note genuinely added to the tree—soundness resting on the collision resistance of Sinsemilla (Proposition 4.29) and the knowledge soundness of the proof system—while the network learns nothing about *which* note. (In the deployed circuit the root-equals-anchor constraint is enforced for notes of nonzero value, while zero-valued dummy spends may witness an arbitrary path—by design, so that every action carries a spend whether or not a real note backs it.) The Merkle root is thus a succinct, binding, privacy-preserving commitment to the set of all notes ever created.

In the spend, the note commitment is the hiding and binding commitment of §4; the tree hash and its personalised domains apply the domain-separation discipline of §3, instantiated by Sinsemilla; the nullifier is derived from a PRF of §5 instantiated by Poseidon; the proof is a zk-SNARK of §9 for the relation R_{mem} extended with the note’s opening; and the authorising signatures over the spend—re-randomised RedDSA for spend authority, the binding signature for value balance—are those of §8. How the pieces sit inside a transaction, per Action, with the key hierarchy and note-delivery encryption around them, is treated in a later volume of the series.

III

Halo 2 Guide

The Halo 2 proof system, from arithmetisation to recursion

Abstract

This volume of *The Zcash Arboretum* develops the proof system that Orchard uses: Halo 2, the argument by which the author of a shielded Action convinces every validator that the Action is well formed while disclosing nothing about the note, the key, or the Merkle path behind it. It assumes the *Math Guide*, whose finite fields, polynomials and evaluation domains, elliptic curves, and probability are used throughout without re-derivation, and the *Crypto Guide*, whose definitions it instantiates rather than restates: arguments of knowledge and zero knowledge, the Fiat–Shamir transform, Pedersen vector commitments, and the polynomial-commitment interface with its compilation recipe.

The running example is knowledge of x with $x^3 + x + 5 = 35$ over the field $\mathbb{F}_{97}$, a size at which every number of the example can be checked by hand; every number is computed by script. The volume reduces the example to a grid of gates, the grid to polynomials, the polynomials to a single identity, and the identity to one evaluation at a random point, at which the prover must open a commitment without sending the committed polynomial. The inner-product argument (IPA) that performs the opening has a section of its own: the problem stated without machinery, one fold written out step by step, the recursion from the prover’s and the verifier’s side, the verifier’s structured scalars, the extraction from three transcripts that proves the argument knowledge sound under the discrete-logarithm assumption on Vesta, with the hash that derives the generators modelled as a random oracle, the blinders and what they do not hide, and the running example opened on a curve of prime order 97. The deployed proof system follows: the Action circuit’s arithmetisation, the binding of the public statement, the Fiat–Shamir transcript, the reduction of many openings to one, the complete protocol, and its zero knowledge. The volume then analyses the knowledge soundness of the Fiat–Shamir-compiled argument, lists the verifier’s checks on the running example in the order he performs them, and closes with recursive composition and accumulation, which Orchard does not deploy.

1 Introduction

1.1 Scope and layering

This is the third volume of *The Zcash Arboretum*, and it develops the cryptography behind Zcash’s Orchard protocol: the proof system Halo 2, with which every Orchard Action carries a zero-knowledge proof of its own well-formedness. The protocol specification fixes that use: Halo 2 is used with the Vesta curve (the *Math Guide*, §“Pallas and Vesta assembled”) to prove and verify the Orchard Action statement (protocol specification § 4.1.13, “Zero-Knowledge Proving System”), and the proving system is defined by reference to the halo2 book (§ 5.4.10.3, “Halo 2”). The statement itself (§ 4.18.4, “Action Statement (Orchard)”) is read in §1.2. Actions of the Orchard pool and of the Ironwood pool, the second shielded pool of Orchard-protocol notes, prove this statement, extended at NU6.3 by one public bit, disableCrossAddress, that when set forces the new note’s address to equal the spent note’s, with one circuit and one verifying key (protocol specification, § 4.6, “Action Descriptions”; ZIP 229; ZIP 258), so the proof system and proof format described here serve both pools.

The two volumes below this one built its material from the ground up. The *Math Guide* constructed the mathematics: finite fields, elliptic curves, polynomials and evaluation domains, probability. The *Crypto Guide* constructed the cryptographic primitives: hash functions, commitments, Σ -protocols and the Fiat–Shamir transform, zero knowledge, polynomial commitment schemes. This volume assumes all of these and defines none of them afresh; it assembles them into a proof system. In particular the *Crypto Guide* owns every definition this volume instantiates: the non-interactive argument and the SNARK, together with the polynomial interactive oracle proof and the compilation recipe that turns one into the other (§“SNARKs: succinct non-interactive arguments of knowledge”); the Fiat–Shamir transform (§“The Fiat–Shamir transform: from interactive to non-interactive”); the Pedersen vector commitment (§“Pedersen vector commitments”); and the polynomial-commitment interface, with its sketch of an inner-product opening (§“Polynomial commitment schemes”). Where a lower volume’s result is needed it is cited at the point of use, by italic volume name and section title: the volumes are separate documents, so there is no cross-document reference, and this volume forward-references only its own later sections.

1.2 The running example and the security notions

The running example is the statement

$$\text{knowledge of } x \in \mathbb{F}_{97} \text{ with } x^3 + x + 5 = 35,$$

one of whose witnesses (the *Crypto Guide*, §“Languages, relations, and witnesses”) is $x = 3$, since $27 + 3 + 5 = 35$; the others in $\mathbb{F}_{97}$ are 18 and 76, and the prover of the example holds $x = 3$. It is arithmetised in §2, committed and opened in §3, bound to its public input 35 and compiled into a proof string in §4, and verified in §6. Its field arithmetic, that of the claim, the grid, the polynomials and every scalar, is in the prime field $\mathbb{F}_{97}$ (the *Math Guide*, §“The prime field and two inversion routes”), a size at which every product can be checked by hand; the commitments of §3 are points of a curve of order 97 whose coordinates lie in a second prime field, $\mathbb{F}_{79}$. Every number printed is computed by script, and the script’s output is the source of the text.

A proof system for such a statement is required to have three properties, each a definition of the *Crypto Guide* (§“Interactive proofs, zero knowledge, and SNARKs”) used here without re-statement. *Overwhelming completeness*: an honest prover holding a witness is accepted with overwhelming probability, in the sense of the *Math Guide* (§“Polynomial, exponential, and negligible functions”). *Knowledge soundness* (the *Crypto Guide*, §“Knowledge soundness and extractors”): an efficient extractor, given rewindable access to any prover the verifier accepts with non-negligible probability, outputs a valid witness. *Zero knowledge* (the *Crypto Guide*, §“The simulation paradigm and zero knowledge”): a simulator given only the public input produces transcripts indistinguishable from those of real executions.

The volume establishes these properties to different extents. Design-stage claims are classified in three grades: *specified*, when the protocol specification or a ZIP states the property; *designed-but-unspecified*, when the design intends it but no specification, ZIP or theorem of this volume states it for the deployed system; and *open problem*, when no design is known to achieve it. The Halo 2 protocol is complete except with negligible probability: for each random challenge, a number of values at most proportional to the size of the circuit, out of the $|\mathbb{F}|$ possible, makes undefined a division that the verifier or the honest prover performs; §4.6 lists these degenerate challenges once the protocol is defined. The inner-product opening argument is knowledge sound under the discrete-logarithm assumption on Vesta, with the hash that derives its generators modelled as a random oracle (Theorem 3.5 and, for the deployed opening, Corollary 3.13). Knowledge soundness of the deployed non-interactive argument rests on an analysis in the algebraic group model and the random-oracle model whose instantiation to the deployed transcript is designed-but-unspecified (§5). Zero knowledge is recorded in §4.6 from the halo2 book’s claim for the interactive protocol, under the composition and Fiat–Shamir hypotheses stated there.

The running example has the form of the Orchard Action statement. The prover of an Action shows knowledge of a note, the keys that control it, and a Merkle path such that the note’s commitment is well formed, the path, unless the spent value is zero, leads from that commitment to the published tree root (the *Crypto Guide*, §“Merkle trees and commitments to sets”), the net value commitment is a Pedersen commitment to the old value minus the new value (the *Crypto Guide*, §“The Pedersen commitment”), the nullifier, the unique tag whose publication marks the note spent, is computed correctly, and the key that authorises the spend is a re-randomisation of the owner’s key (the *Crypto Guide*, §“Key re-randomisation and unlinkability”). The protocol specification states this as nine conditions on a public input and a private witness (§ 4.18.4, “Action Statement (Orchard)”), the five just paraphrased and four further: that the spent note’s address is derived from the same keys, that the new note’s commitment is well formed, and that two public bits, when zero, force the spent value and the created value, respectively, to zero. From NU6.3 the circuit checks a tenth condition on a further public bit, which restricts cross-address transfers and is set for every Orchard-pool Action (§ 4.6; ZIP 258). Each condition unfolds into small arithmetic facts over the field $\mathbb{F}_{p_{\text{Pallas}}}$ (the *Math Guide*, §“Base fields, scalar fields, and the Pasta cycle”): the note commitment alone is a Sinsemilla commitment, a Sinsemilla hash to a point plus a blinding multiple of a fixed generator (the *Crypto Guide*, §“Sinsemilla: an algebraic hash-based commitment”), whose hash is evaluated in the circuit with the help of a lookup table. Together the conditions are laid out on the $2^{11} = 2048$ rows of the deployed Action circuit; the lookup table and the row count are given in §4, “From statement to circuit: arithmetisation in practice”. The

running example itself is a conjunction of *four* arithmetic facts, $x \cdot x = x^2$, $x^2 \cdot x = x^3$, $x^3 + x = s$ and $s + 5 = 35$, one per row of the grid that §2 builds. Both the toy and the Action circuit are instances of the constraint system that §2 defines; the toy’s constraint on each row reads that row alone and consults no table.

The chain of reductions. Five reductions connect the claim to the checks the verifier performs.

1. **Computation $\rightarrow$ grid** (§2, “From computation to a grid of constraints”). The claim is rewritten as a table of rows, each row one elementary gate ($\times$ or $+$), with copy constraints between rows forcing shared values.
2. **Grid $\rightarrow$ polynomials** (§2, “From the grid to polynomials”). Each column of the table is interpolated into a polynomial (the *Math Guide*, §“Lagrange interpolation”). The condition that every row obeys its gate becomes the condition that one polynomial $G(X)$ is zero at every row.
3. **Vanishing $\rightarrow$ identity** (§2, “From vanishing to a single identity”). The statement that G is zero at every row becomes the algebraic identity $G(X) = t(X) Z_H(X)$ for some quotient t , where Z_H is the vanishing polynomial of the set H of row names (the *Math Guide*, §“The evaluation domain H and its vanishing polynomial”). A trace that satisfies every gate yields such an identity, and a trace that violates a gate cannot. The copy constraints are not part of G ; the permutation argument enforces them (item 5).
4. **Identity $\rightarrow$ one random check** (§2, “Why checking one random point is a proof”). The verifier tests the identity at a single random point z . A false identity holds at a uniformly random point with probability at most its degree divided by the field size (the *Math Guide*, §“The Schwartz–Zippel lemma”).
5. **Random check $\rightarrow$ committed openings.** A binding commitment (the *Crypto Guide*, §“Hiding and binding”) prevents the prover from changing her polynomials after z is drawn (§2, “Why the prover cannot wriggle: binding commitments”), and the inner-product argument (IPA) of §3 opens a commitment at z without sending the polynomial; the permutation argument enforces the copy constraints (§2, “Enforcing the wiring: the permutation argument”); a dedicated public column, the instance column, fixes the public input 35 (§4, “Instance columns and binding the public statement”); and blinding hides the witness (§4, “Zero knowledge: hiding the witness in Halo 2”).

Section 6 composes the five reductions into one chain of implications, from the verifier’s accepted checks back to the claim.

The scale caveat, stated once. The field $\mathbb{F}_{97}$ is a toy. Over Orchard’s field $\mathbb{F}_{p_{\text{Pallas}}}$, whose prime has 255 bits, a *fixed* false identity of degree around 2^{14} (the degree of the deployed constraint polynomial, bounded in §4, “The complete Halo 2 protocol”) survives one fresh uniform evaluation challenge with probability around $2^{14}/2^{254} = 2^{-240}$, in place of the visible $3/97$ that §2 exhibits for the toy’s cheat. Every such number in this volume is one Schwartz–Zippel term: the probability that one random point hides one fixed lie. It is not the security level of the complete proof

system, which also rests on the binding of the commitments, the extractor of the opening argument, and the loss of the Fiat–Shamir transform, treated in §3, §4 and §5. The caveat is stated here and not re-derived; every probability printed later is read with it.

1.3 Outline

The sections follow the running example; Figure 1 shows the section in which each stage is built.

Section 2 takes the example from the computation to committed polynomials: the grid, the column polynomials, the single identity, the random point, and then the general form of the constraint system, PLONKish arithmetisation, of which the toy is an instance, followed by the permutation argument for the copy constraints and the lookup argument for facts that no low-degree gate states economically. It ends with the problem a vector commitment alone does not solve: proving the value of a committed polynomial at z without sending the polynomial, in fewer group elements than the polynomial has coefficients.

Section 3 solves that problem with the inner-product argument. It states the inner-product problem without machinery, works one fold with every algebra step written out, gives the recursion with both parties’ computations side by side, makes the verifier’s work explicit, proves the argument knowledge sound by extraction from three transcripts under the discrete-logarithm assumption with uniform generators, separates what the blinders hide from what they do not, and opens the example’s own polynomial through every round on a curve of prime order 97, by script. Its last subsection gives Halo 2’s parameters, the deployed form of the argument as a change of variables, the bytes of a proof, the deferred multi-scalar multiplication and the batching that amortises it, and accumulation, which Orchard does not deploy.

Section 4 gives the deployed proof system: the Action circuit’s arithmetisation, the instance column that binds the public statement, the Fiat–Shamir transform that replaces the verifier’s challenges by hash outputs, the reduction of many openings to one, the complete protocol in seven phases, its zero knowledge, and the compiled-polynomial-IOP pattern the protocol instantiates.

Section 5 treats knowledge soundness of the non-interactive argument: the extractor of §3 when the challenges are hash outputs, the algebraic group model in which the halo2 book proves its tight bound for the analysed protocol, and the gap between concrete and asymptotic security.

Section 6 lists the verifier’s checks on the running example in the order he performs them, the chain of implications back to the prover’s knowledge of advice satisfying every gate and every copy constraint, and the same chain at Orchard scale.

Section 7 treats recursive proof composition, accumulation schemes, the Halo trick, and the cycle of curves that makes native recursion efficient. Orchard proves with Halo 2 on the Pasta curves but does not recurse, and the section states at the outset that Orchard does not deploy its material.

2 Arithmetisation: from a computation to committed polynomials

This section carries the running example of §1 — the prover knows $x = 3$ with $x^3 + x + 5 = 35$ over $\mathbb{F}_{97}$ — from a computation to a handful of polynomials that a verifier can query at one point,

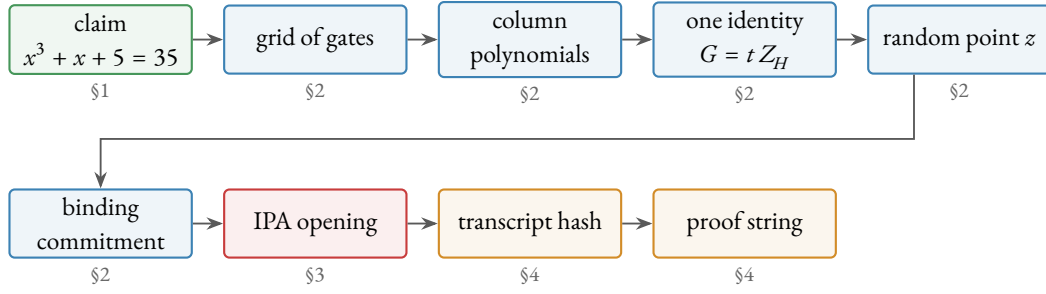


Figure 1. The running example’s path through the volume, each stage tagged with the section that owns it. The claim is fixed in this section (green). The grid of gates, the column polynomials, the single identity $G = t Z_H$, the random point z and the binding commitment are built in §2 (blue). The commitment is opened by the inner-product argument in §3 (red). The transcript hash, whose outputs replace the verifier’s challenges, and the proof string are produced in §4 (amber). Section 6 reads the chain backwards, from the accepted proof string to the claim.

toy first and general second. It ends with the property the argument has not yet provided: that the queried polynomials are fixed before the evaluation point is chosen.

2.1 From computation to a grid of constraints

The verifier cannot evaluate the computation without the witness. Arithmetisation dismantles the computation into *local* facts, each involving only a handful of values, arranged so that “every local fact holds and the values are consistently shared” is equivalent to “the whole computation ran correctly”. The dialect of arithmetisation Halo 2 uses is called *PLONKish*; §2.5 defines it once the toy has shown what it generalises.

Execution trace. Evaluate $x^3 + x + 5$ the way a machine would, naming every intermediate value:

$$x^2 = x \cdot x, \quad x^3 = x^2 \cdot x, \quad s = x^3 + x, \quad s + 5 = 35.$$

Four elementary operations, two multiplications and two additions, and each becomes one row of a table. A row has three *wires* a, b, c (two inputs and an output) and a bank of five *selectors* q_M, q_L, q_R, q_O, q_C that switch on whichever gate the row performs. The universal gate equation, imposed at every row, is

$$q_M a b + q_L a + q_R b + q_O c + q_C + \text{PI} = 0, \quad (1)$$

where PI carries the negated public input on that row (here -35 , for the target 35). A multiplication gate $a \cdot b = c$ is the choice $q_M = 1, q_O = -1$ and the rest zero; an addition gate $a + b = c$ is $q_L = q_R = 1, q_O = -1$. Filling in the four operations gives the *execution trace* of Table 1. Definition 2.7 generalises this fixed selector bank to arbitrary custom gates and lookup arguments (the halo2 book’s *PLONKish Arithmetization* chapter); §4 gives the form of the Action circuit’s gates, and one in full, under “From statement to circuit: arithmetisation in practice”.

Table 1. The execution trace for $x = 3$, all entries in $\mathbb{F}_7$ (so $-1 = 96$ and $-35 = 62$). Every row satisfies the gate equation (1): row 0 reads $1 \cdot 3 \cdot 3 + (-1) \cdot 9 = 0$ and row 3 reads $1 \cdot 30 + 5 + (-35) = 0$. The selector columns q_i are fixed circuit data and PI is the negated public input, both known to the verifier; the wires a, b, c are the prover's *advice*, her secret trace.

row	a	b	c	q_M	q_L	q_R	q_O	q_C	PI	gate performed
0	3	3	9	1	0	0	-1	0	0	$x \cdot x = x^2$
1	9	3	27	1	0	0	-1	0	0	$x^2 \cdot x = x^3$
2	27	3	30	0	1	1	-1	0	0	$x^3 + x = s$
3	30	0	0	0	1	0	0	5	-35	$s + 5 = 35$

Copy constraints. The table as printed does not yet encode the computation. Nothing forces the x in row 0 to equal the x in rows 1 and 2, nor the output $x^2 = 9$ of row 0 to be the input of row 1. A cheating prover could put $x = 3$ in row 0 and $x = 8$ in row 2 and satisfy each gate *in isolation* without the rows forming one computation. The missing assertions are that certain cells are *equal*, the *copy constraints* (also called *equality constraints*):

$$\underbrace{a_0 = b_0 = b_1 = b_2}_{\text{all are } x=3}, \quad \underbrace{c_0 = a_1}_{x^2=9}, \quad \underbrace{c_1 = a_2}_{x^3=27}, \quad \underbrace{c_2 = a_3}_{s=30}$$

where a_j denotes the cell of column a in row j . Figure 2 draws them as wires. They turn a list of unrelated gates into a single dataflow; enforcing them is an argument in its own right, the permutation argument of §2.6.

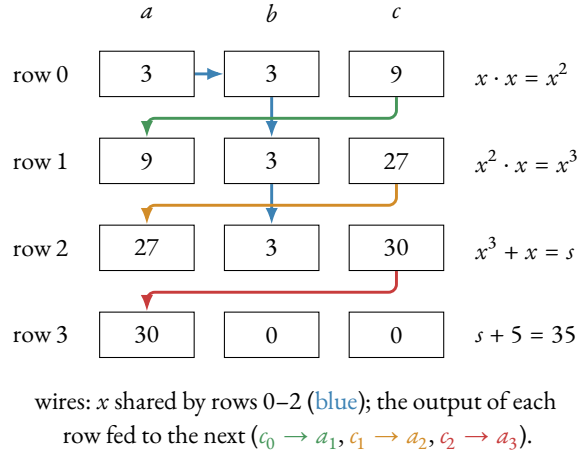


Figure 2. The four-row trace of Table 1 with the copy constraints drawn as wires. Each wire colour is one class of cells that must hold a single value: the blue wire ties the four occurrences of x , and the three coloured wires carry each row's output c_j into the next row's input a_{j+1} . The gates check rows; the wires make the rows one computation.

Satisfiability of the grid. The claim is now equivalent to a purely combinatorial assertion: *there exist values for the advice cells a, b, c such that (i) every row satisfies the gate equation (1) with the fixed selectors, and (ii) every copy constraint holds.* Knowing x is exactly knowing how to fill

the advice columns. The verifier knows the fixed columns and the public input; the prover must convince him that satisfying advice exists while revealing none of it. Everything that follows is machinery for doing precisely that.

2.2 From the grid to polynomials

The grid has finitely many rows, and the aim is a statement about all rows at once that the verifier can check at *one* point. A polynomial of degree less than n is determined by its values at n distinct points (*Math Guide*, §“Lagrange interpolation”), and the rest of the section relies on this.

An index set for the rows. The table has $n = 4$ rows, and four distinct field elements must name them. The chosen four are the 4th roots of unity. A *primitive n -th root of unity* is an element ω with $\omega^n = 1$ and no smaller positive power equal to 1 (*Math Guide*, §“Roots of unity”), and the *evaluation domain* it generates is $H = \{\omega^0, \dots, \omega^{n-1}\}$, a multiplicative subgroup of $\mathbb{F}^\times$ of order n (*Math Guide*, §“The evaluation domain H and its vanishing polynomial”). In $\mathbb{F}_{97}$ take $\omega = 22$: then $\omega^2 = 96 = -1$ and $\omega^4 = 1$, so

$$H = \{\omega^0, \omega^1, \omega^2, \omega^3\} = \{1, 22, 96, 75\} = \{1, 22, -1, -22\},$$

and row i lives at the point ω^i . Roots of unity are chosen because they form a cyclic group: the row after ω^i is $\omega \cdot \omega^i$, so reading a column at the next row is evaluating its polynomial at ωX ; the polynomial vanishing on H is $X^n - 1$; and interpolation over H is a fast Fourier transform (*Math Guide*, §“The fast Fourier transform”). The gate check of this subsection alone would hold over any four distinct points.

Columns become polynomials. Each column of the table is four numbers, one per row. Interpolate: for the advice column a there is a unique polynomial $a(X)$ of degree < 4 with

$$a(\omega^0) = 3, \quad a(\omega^1) = 9, \quad a(\omega^2) = 27, \quad a(\omega^3) = 30$$

(*Math Guide*, §“Lagrange interpolation”), and likewise $b(X)$, $c(X)$, the publicly known selector polynomials q_M, q_L, q_R, q_O, q_C and the public-input polynomial PI, each of degree < 4 . Computed over $\mathbb{F}_{97}$, the nine interpolants are

$$\begin{aligned} a(X) &= 90 + 61X + 22X^2 + 24X^3, & q_M(X) &= 49 + 19X + 30X^3, \\ b(X) &= 75 + 32X + 25X^2 + 65X^3, & q_L(X) &= 49 + 78X + 67X^3, \\ c(X) &= 65 + 16X + 3X^2 + 22X^3, & q_R(X) &= 73 + 24X + 73X^2 + 24X^3, \\ \text{PI}(X) &= 64 + 50X + 33X^2 + 47X^3, & q_O(X) &= 72 + 54X + 24X^2 + 43X^3, \\ & & q_C(X) &= 74 + 76X + 23X^2 + 21X^3, \end{aligned}$$

and each returns its column at the rows: b takes the values $(3, 3, 3, 0)$ on H , q_O the values $(96, 96, 96, 0)$, and PI the values $(0, 0, 0, 62)$. Interpolation is thus a bijection between columns of four values and polynomials of degree < 4 : the encoding loses no information.

The gate polynomial. Assemble the left-hand side of the gate equation (1) with the polynomials in place of the per-row numbers.

Theorem 2.1 (The gate polynomial). *Let*

$$G(X) := q_M(X) a(X) b(X) + q_L(X) a(X) + q_R(X) b(X) + q_O(X) c(X) + q_C(X) + \text{PI}(X). \quad (2)$$

Then $\deg G \leq 9$, and $G(\omega^i) = 0$ for every row i if and only if every gate of the trace is satisfied.

Proof. The term $q_M ab$ multiplies three polynomials of degree at most 3, and every other term has degree at most 6, so $\deg G \leq 9$. Evaluate G at the label ω^i of row i : each column polynomial returns that row's value there, so $G(\omega^i)$ is exactly the left-hand side of (1) for row i . It is zero precisely when gate i holds, and the conjunction over i gives the claim. $\square$

For the honest trace,

$$G(X) = 36 + 27X + 54X^2 + 50X^3 + 77X^4 + 24X^5 + 43X^6 + 47X^7 + 81X^8 + 46X^9,$$

of degree exactly 9, and it vanishes at all four points of H by construction. “Every gate is satisfied” has become “ G vanishes on H ”; the copy constraints remain separate. Four separate checks are now one statement about one polynomial.

2.3 From vanishing to a single identity

The condition “ G vanishes on H ” is still a statement about four points. This subsection replaces it by a polynomial identity that refers to no individual row.

The vanishing polynomial. The *vanishing polynomial* of H is the monic polynomial (leading coefficient 1) whose roots are exactly the points of H ,

$$Z_H(X) := \prod_{i=0}^3 (X - \omega^i) = X^4 - 1,$$

where the closed form $X^n - 1$ holds for every evaluation domain of order n (*Math Guide*, §“The evaluation domain H and its vanishing polynomial”): each ω^i is a root of $X^n - 1$, there are n distinct ones, and comparing leading coefficients finishes. The closed form lets both parties evaluate Z_H at any point with $O(\log n)$ multiplications.

Theorem 2.2 (Gate satisfaction is a quotient). *Every gate of the trace holds if and only if there is a polynomial $t(X)$ with $G(X) = t(X) Z_H(X)$.*

Proof. By Theorem 2.1, every gate holds if and only if G vanishes on H . The factor theorem (*Math Guide*, §“Roots and the factor theorem”) gives $G(\omega^i) = 0$ if and only if $(X - \omega^i) \mid G$; and because the four roots are distinct, G vanishes at all of them if and only if the product of the

four linear factors divides G (*Math Guide*, §“Lagrange interpolation”, its remark on the vanishing polynomial: a polynomial vanishes on a set of distinct nodes if and only if the product of the corresponding linear factors divides it). Thus

$$G \text{ vanishes on } H \iff (X - \omega^i) \mid G \text{ for each } i \iff Z_H(X) \mid G(X), \quad (3)$$

and divisibility is the existence of the quotient t . $\square$

Satisfaction of the *gate* constraints is precisely the existence of t ; correct wiring still requires the copy constraints. For the honest trace the division comes out even: G has degree 9, Z_H has degree 4, and

$$t(X) = \frac{G(X)}{X^4 - 1} = 61 + 70X + 43X^2 + 47X^3 + 81X^4 + 46X^5$$

is a polynomial of degree 5 with remainder zero, as it must be.

Example 2.3 (Why a cheat cannot produce t). Suppose the advice is wrong: some gate fails. Then G does not vanish on all of H , so by (3) Z_H does not divide G , and *no* polynomial t satisfies $G = t Z_H$. The dividing line is exact: the quotient exists if and only if every gate is satisfied. Concretely, take a prover who claims $x^2 = 10$ and writes $c_0 = 10$ in row 0 of Table 1 instead of 9. Row 0’s gate now reads $3 \cdot 3 - 10 = -1 \neq 0$, so the tampered gate polynomial G^* has $G^*(\omega^0) = -1 = 96$ while still vanishing at the other three rows:

$$G^*(X) = 54 + 10X + 43X^2 + 74X^3 + 83X^4 + 65X^5 + 78X^6 + 47X^7 + 81X^8 + 46X^9.$$

Dividing G^* by Z_H with remainder (*Math Guide*, §“Division with remainder”) gives the quotient $t^*(X) = 67 + 14X + 78X^2 + 47X^3 + 81X^4 + 46X^5$ and a nonzero remainder. This Euclidean quotient is an illustrative choice for t^* , not a claim that it maximises the cheat’s chances. Whatever t^* she offers as the quotient, the *gap polynomial* $D(X) := G^*(X) - t^*(X) Z_H(X)$ is nonzero, since Z_H does not divide G^* . For the Euclidean quotient D is the remainder,

$$D(X) = 24(1 + X + X^2 + X^3), \quad (4)$$

of degree 3. The identity $G^* = t^* Z_H$ is false, D is the difference of its two sides, and §2.4 shows how the verifier detects $D \neq 0$.

2.4 Why checking one random point is a proof

The prover wishes to convince the verifier that $G(X) = t(X) Z_H(X)$ as polynomials, with G built from her secret advice. The verifier does not read the polynomials: they encode the witness, and reading them would cost time at least linear in the number of rows. Instead, the two run a four-step protocol. A *commitment* is, for now, a short string that fixes a value without revealing it (*Crypto Guide*, §“Commitment schemes”); §2.9 says what a commitment to a polynomial must do and which one Halo 2 uses.

Construction 2.4 (The random-point check). 1. The prover commits to her polynomials a, b, c and to the quotient t ; the selector and public-input polynomials the verifier already knows.

2. The verifier picks a point $z \in \mathbb{F}$ *uniformly at random* and sends it.
3. The prover *opens* her commitments at z : she returns the field elements $a(z), b(z), c(z), t(z)$ together with short proofs that these are the true evaluations of the committed polynomials.
4. The verifier forms $G(z)$ from the openings and the self-evaluated public polynomials via (2), computes $Z_H(z) = z^4 - 1$, and checks the single field equation

$$G(z) \stackrel{?}{=} t(z) Z_H(z). \quad (5)$$

He accepts if and only if (5) holds.

The honest case (completeness). If the prover is honest then $G = t Z_H$ holds as polynomials, hence at every point, hence at z . With the honest trace and the draw $z = 20$,

$$Z_H(20) = 20^4 - 1 = 46, \quad t(20) = 67, \quad G(20) = 75 = 67 \cdot 46 = t(20) Z_H(20),$$

and the verifier accepts. He accepts every time, for every z .

The dishonest case (soundness). Soundness rests on the fact that a nonzero polynomial of low degree has few roots.

Theorem 2.5 (Soundness of the random-point check). *Suppose the trace is invalid, so that the prover's gate polynomial G^* does not vanish on all of H , and let t^* be any polynomial she commits to as the quotient. Set $D(X) := G^*(X) - t^*(X) Z_H(X)$. Then D is a nonzero polynomial, and if the prover's polynomials a, b, c (which determine G^*) and t^* are fixed before z is drawn and the verifier receives their true evaluations at z ,*

$$\Pr_z[\text{the verifier accepts}] = \Pr_z[D(z) = 0] \leq \frac{\deg D}{|\mathbb{F}|}.$$

Proof. Were D the zero polynomial, $G^* = t^* Z_H$ would be divisible by Z_H and so, by (3), vanish on all of H , which it does not. Hence $D \neq 0$ for every choice of t^* . By hypothesis a, b, c and t^* , and with them G^* and D , are fixed before z is drawn (the verifier never receives G^* itself; from the evaluations at z he reconstructs only the single number $G^*(z)$). The check (5) passes at z exactly when $G^*(z) - t^*(z) Z_H(z) = D(z) = 0$, that is, exactly when z is a root of D . A nonzero polynomial of degree at most $\deg D$ has at most $\deg D$ roots, so a uniformly random z is one of them with probability at most $\deg D / |\mathbb{F}|$ (*Math Guide*, §“The Schwartz–Zippel lemma”). $\square$

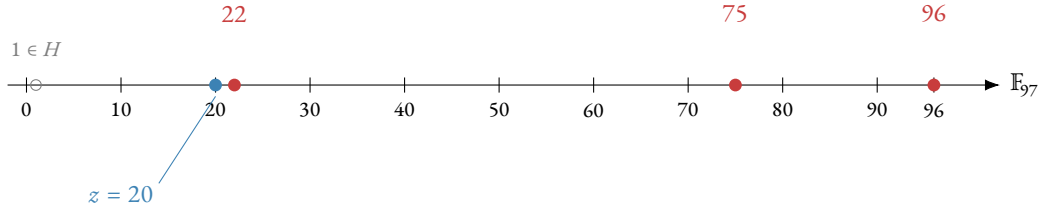
The bound $\deg D / |\mathbb{F}|$ is negligible when $|\mathbb{F}|$ is exponential in the security parameter and $\deg D$ is polynomial in it. The verifier does not recompute the trace; he tests one polynomial identity at one random point.

Example 2.6 (The random-point check on the toy). For the $x^2 = 10$ cheat of Example 2.3, $D(X) = 24(1 + X + X^2 + X^3)$ has degree 3 and so at most 3 roots; in $\mathbb{F}_{97}$ its roots are exactly $\{22, 96, 75\}$,

the three rows where the tampered gate still held (Figure 3). This particular quotient choice slips past only if the random z lands in that three-element set:

$$\Pr[\text{cheat accepted}] = \frac{3}{97}.$$

At the actual draw $z = 20 \notin \{22, 96, 75\}$ the openings give $G^*(20) = 20$ but $t^*(20) Z_H(20) = 64$; since $20 \neq 64$, the verifier rejects.



filled: the roots of the gap polynomial D , the set $H \setminus \{1\}$, where the tampered identity still holds; $D(z) = 0$ on 3 of the 97 points only. **blue:** the draw $z = 20$, where $D(20) \neq 0$ and the check rejects, as it does at every unmarked point.

Figure 3. The gap polynomial $D = 24(1 + X + X^2 + X^3)$ of the $x^2 = 10$ cheat over $\mathbb{F}_{97}$: its three roots 22, 75, 96 marked in red, the point $1 \in H$ (not a root) in grey, and the verifier’s draw $z = 20$ landing elsewhere. The check (5) passes only at a root of D .

Scale. The fraction $3/97$ is large only because the field is a toy, and it concerns one particular quotient; the next paragraph bounds every cheat by $11/97$. What such a term means at the deployed field size is the scale caveat of §1.2.

Bounding the degree. The argument needs $\deg D$ small relative to $|\mathbb{F}|$, otherwise “ $\deg D$ roots out of $|\mathbb{F}|$ ” is no constraint at all. The commitment scheme enforces a degree bound on every committed polynomial (*Crypto Guide*, §“Polynomial commitment schemes”), so the prover cannot commit to a t^* of unbounded degree. The bound follows the degree the commitment admits, not the honest degree. The wires are committed at degree below $n = 4$, so $\deg G^* \leq 9$; committed as the two pieces of degree below n that the deployed rule below prescribes, t^* has degree at most 7, so $\deg(t^* Z_H) \leq 11$ and $\deg D \leq 11$. Every cheat is therefore accepted with probability at most $11/97$, and some cheat attains it, where the illustrative quotient reaches $3/97$: the t^* of degree 7 that agrees with G^*/Z_H at the eight points $2, \dots, 9$ gives D the eleven roots $2, \dots, 9, 22, 75, 96$. This is why a circuit’s gates are kept to low degree, and why the quotient t , of degree 5 here and up to a few times n in general, is split into degree-bounded pieces. In the deployed system the quotient is cut into pieces of degree below n , each committed separately (the halo2 book’s *Vanishing argument* chapter); “The complete Halo 2 protocol” in §4 gives the count.

2.5 PLONKish arithmetisation

The toy has met every ingredient concretely: an evaluation domain H , a generator ω , the vanishing polynomial Z_H , columns interpolated into polynomials of degree $< n$, and a gate expressed as a

polynomial identity on H . The general object can now be named.

Definition 2.7 (PLONKish constraint system). Fix a prime field $\mathbb{F}$ and an integer k with $2^k \mid |\mathbb{F}| - 1$, so that $\mathbb{F}^\times$ contains a primitive 2^k -th root of unity ω (*Math Guide*, §“Existence over finite fields: the condition $n \mid q - 1$ ”), and let $H := \{\omega^0, \dots, \omega^{n-1}\} \subset \mathbb{F}^\times$ be the multiplicative subgroup of size $n := 2^k$ that ω generates. Its vanishing polynomial is $Z_H(X) := X^n - 1$, zero exactly on H , so a congruence $p(X) \equiv 0 \pmod{Z_H(X)}$ means “ p is zero at every row ω^i ”, that is, $Z_H \mid p$. A *PLONKish circuit* C over H consists of:

1. a finite set Col of columns, each of n cells, one per row, partitioned into *fixed* columns (pre-processed circuit data, known to both parties), *advice* columns (the prover’s witness) and *instance* columns (public input); a *cell* is an element of $H \times \text{Col}$, and the circuit assigns every cell of a fixed column its value;
2. a maximum constraint degree $d_{\max}$;
3. a finite set $R \subset \mathbb{Z}$ of *rotations* containing 0; a *column expression* is a polynomial over $\mathbb{F}$ in the values $v(\omega^r X)$ for $v \in \text{Col}$ and $r \in R$, where the value of v at rotation r at row i is the cell of v in row $i + r$ taken modulo n ;
4. a finite sequence of *custom gates* $g_j = 0$, each g_j a column expression of degree at most $d_{\max}$;
5. a set $H_{\text{act}} \subseteq H$ of *active* rows: all of H in the toy, and in the deployed system the first $u < n$ rows $\omega^0, \dots, \omega^{u-1}$, the trailing rows being reserved for a boundary row and blinding (Remark 2.17 fixes u);
6. a subset $E \subseteq H_{\text{act}} \times \text{Col}$ of *equality-constrained* cells together with a permutation σ of E (a bijection $E \rightarrow E$) whose cycles, the orbits $\{\kappa, \sigma(\kappa), \sigma^2(\kappa), \dots\}$ of the cells $\kappa \in E$, are the classes of cells required to hold a common value;
7. a set of *lookup arguments*, each an input tuple $(A_0, \dots, A_{m-1})$ of column expressions and a table tuple $(S_0, \dots, S_{m-1})$ of columns.

An *assignment* gives every cell a value in $\mathbb{F}$, the fixed cells their circuit values; write v_κ for the value of cell κ . It is *satisfying* if every gate vanishes at every row of H (in the deployed system the selectors are zero outside H_{act}), $v_\kappa = v_{\sigma(\kappa)}$ for every $\kappa \in E$, and for every lookup and every active row ω^i the tuple $(A_0(\omega^i), \dots, A_{m-1}(\omega^i))$ equals $(S_0(\omega^i), \dots, S_{m-1}(\omega^i))$ for some active row $\omega^{i'}$. The *relation* $\mathcal{R}_C$ of the circuit is the set of pairs (instance-column values, advice-column values) of the satisfying assignments.

The letter d is reserved, from §3 on, for the length of a committed vector; the constraint degree is therefore written $d_{\max}$ throughout this volume.

Vanilla PLONK as the special case. The original PLONK system of Gabizon, Williamson and Ciobotaru uses a single fixed five-selector universal gate,

$$q_M(X) a(X) b(X) + q_L(X) a(X) + q_R(X) b(X) + q_O(X) c(X) + q_C(X) + \text{PI}(X) \equiv 0 \pmod{Z_H(X)},$$

with three wire polynomials a, b, c , selector polynomials q_M, q_L, q_R, q_O, q_C and the public-input polynomial PI, carrying the negated public inputs; the toy's gate (1) is exactly this gate, with PI held in an instance column. PLONKish generalises it by permitting designer-chosen gates of higher arity and degree, each switched by its own selector $q_i(X)$, a fixed column that vanishes on the rows where the gate is inactive. In the toy the five selectors are fixed columns, a, b, c are advice, PI is the instance column, the single gate has degree 3 and reads rotation 0 only, and the copy constraints of §2.1 are the equality set E with σ cycling each class; there are no lookups until §2.7. Figure 4 shows the general shape.

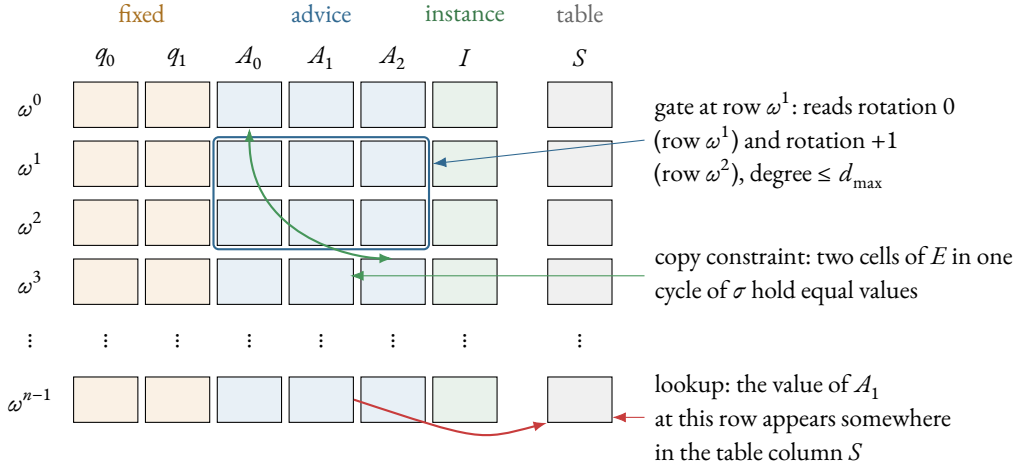


Figure 4. The PLONKish grid of Definition 2.7: columns by kind (fixed q_0, q_1 ; advice A_0, A_1, A_2 ; instance I ; a fixed table column S), rows indexed by the domain H . The outlined gate at row ω^1 reads its columns at rotations 0 and +1; the green double arrow is one cycle of the equality permutation σ joining two advice cells; the red arrow is a lookup asserting that an advice value appears in S .

Columns as committed polynomials. With the Lagrange basis $\ell_j(X)$ of H , defined by $\ell_j(\omega^j) = 1$ and $\ell_j(\omega^{j'}) = 0$ for $j' \neq j$ (*Math Guide*, §“The Lagrange basis on H ”), a column with cells $v_0, \dots, v_{n-1}$ is the polynomial $v(X) = \sum_j v_j \ell_j(X)$ of degree $< n$, exactly as the toy's columns were interpolated in §2.2. The prover commits to each advice polynomial, and the fixed polynomials are committed once for all proofs, with the inner-product polynomial commitment of §3. Every argument that follows — gates, permutation, lookup, vanishing — is a polynomial identity on these committed polynomials, verified by querying them at random points exactly as §2.4 queried G and t .

2.6 Enforcing the wiring: the permutation argument

The copy constraints of §2.1 remain to be enforced. The gate identity of §§2.2–2.4 checks each row separately; it does not assert that a_0, b_0, b_1, b_2 are the same value x , or that row 1's input is row 0's output, and a prover could satisfy every gate with rows that do not form one computation. The *permutation argument* reduces the equalities to the equality of two products, which is checked at random challenges.

Equalities as a permutation. Group the cells into the classes that must share a value, $\{a_0, b_0, b_1, b_2\}$, $\{c_0, a_1\}$, $\{c_1, a_2\}$, $\{c_2, a_3\}$, and let σ be the permutation of the twelve cells that cycles each class in the order written and fixes b_3 and c_3 :

$$a_0 \rightarrow b_0 \rightarrow b_1 \rightarrow b_2 \rightarrow a_0, \quad c_0 \leftrightarrow a_1, \quad c_1 \leftrightarrow a_2, \quad c_2 \leftrightarrow a_3.$$

Give every cell a distinct *label*: the cell in row j of the a -, b -, c -column gets ω^j , $k_1 \omega^j$, $k_2 \omega^j$ respectively, where k_1, k_2 place the three columns in disjoint cosets of H so that no two labels collide. Two cosets of a subgroup are equal or disjoint (*Math Guide*, §“Cosets and Lagrange’s theorem”), so it suffices that k_1, k_2 and k_2/k_1 all lie outside H ; in $\mathbb{F}_{97}$ take $k_1 = 2, k_2 = 3$:

$$H = \{1, 22, 96, 75\}, \quad 2H = \{2, 44, 95, 53\}, \quad 3H = \{3, 66, 94, 31\}.$$

Write λ_{cell} for a cell’s label and v_{cell} for its value. Asserting “cells in a class are equal” is then equivalent to asserting that the *multiset* (a finite collection in which an element may occur more than once; two multisets are equal when every element occurs equally often in both) of pairs $\{(v_{\text{cell}}, \lambda_{\text{cell}})\}$ is unchanged when each label is replaced by the label of the cell’s image under σ : a pair (v, λ) moves to (v, λ_σ) , and the moved pair was already present exactly when the cell at λ_σ carried the same value v . Figure 5 draws the cycles with their labels. Because labels are distinct, comparing two multisets of pairs is a job for a *product*: compress each pair to the linear form $v + \beta \lambda + \gamma$ with verifier challenges β, γ , and compare

$$\prod_{\text{cells}} (v_{\text{cell}} + \beta \lambda_{\text{cell}} + \gamma) \quad \text{against} \quad \prod_{\text{cells}} (v_{\text{cell}} + \beta \lambda_{\sigma(\text{cell})} + \gamma). \quad (6)$$

If the wiring is honest the two products run over the same multiset of factors and agree identically in β, γ . If a copy constraint is false the difference is a nonzero polynomial in β, γ , which can still vanish at one particular challenge pair, but only rarely; the converse is probabilistic, not an “if and only if” for fixed challenges. Theorem 2.11 below makes both halves precise.

The running product. A product over all cells is not a local fact, so the prover commits to a *running product*: a column Z_{perm} with $Z_{\text{perm}}(\omega^0) = 1$ whose row $j + 1$ equals row j times the ratio of row j ’s factors in (6), numerator over denominator. After the last row the product has run over every cell, and because $\omega^n = \omega^0$ the row-to-row rule at the last row forces the full product to equal $Z_{\text{perm}}(\omega^0) = 1$ again: the grand product is 1 exactly when the running product closes its loop. The deployed circuit imposes the rule cross-multiplied, “next row times denominator equals this row times numerator”, not as a field division, so a zero factor never makes the constraint undefined; the prover commits to Z_{perm} and the verifier checks the boundary value and the row-to-row update at the random point, exactly as he checked the gate identity (the halo2 book’s *Permutation argument* chapter). The identities are written out in Construction 2.10.

A single wired pair. Strip the product down to a single wired pair, $c_0 = a_1$ (both should be 9), with labels $\lambda_{c_0} = 3$ and $\lambda_{a_1} = 22$ that σ swaps. Their factors satisfy

$$(9 + \beta \lambda_{c_0} + \gamma)(9 + \beta \lambda_{a_1} + \gamma) = (9 + \beta \lambda_{a_1} + \gamma)(9 + \beta \lambda_{c_0} + \gamma),$$

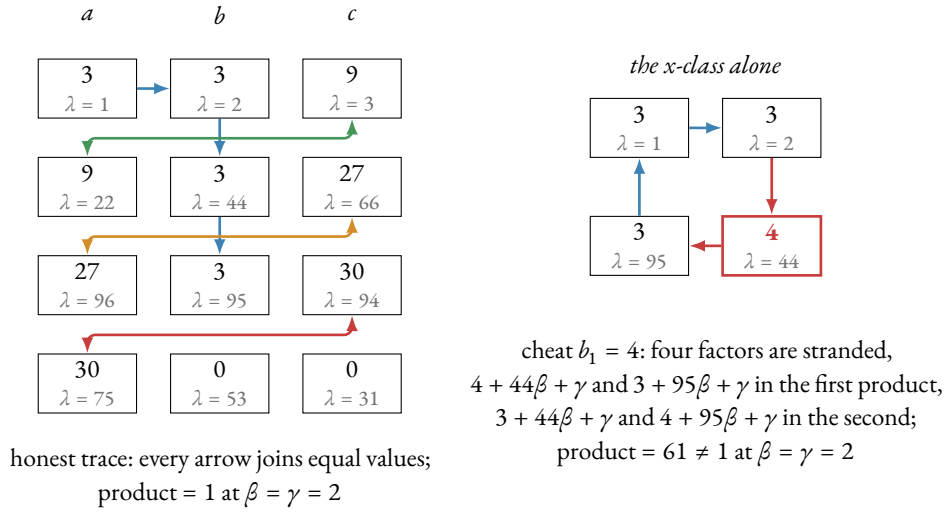


Figure 5. Left: the four copy classes of the toy as cycles of σ on the grid, every cell showing its value and its label λ (column a in H , column b in $2H$, column c in $3H$); the arrow closing the x -class cycle, from b_2 back to a_0 , is omitted. Right: the x -class with b_1 tampered to 4; the two arrows leaving and entering the tampered cell now join unequal values, so four factors of (6), two in each product, have no partner in the other product.

because the shared value 9 makes the two left-hand factors a mere reordering of the two right-hand ones: the labels may permute under σ precisely because the values agree. Now let the prover cheat the copy, $a_1 = 8 \neq 9$. The factor $8 + \beta \lambda_{a_1} + \gamma$ on the left is matched on the right only by $8 + \beta \lambda_{c_0} + \gamma$, which differs since the labels differ; the two products differ as polynomials in β, γ . A single unequal value thus leaves a factor without a partner, and the two products differ.

Example 2.8 (The permutation check on the toy). For the honest trace with $\beta = \gamma = 2$, the ratio of the two products in (6) over all twelve cells is exactly 1. Tamper one wire, setting $b_1 = 4$ and so breaking the copy $b_1 = x = 3$, and the same ratio at the same challenges becomes $61 \neq 1$: the inconsistent dataflow is exposed without any value being revealed.

The general argument. Halo 2 states the argument for any number of columns, and the statement needs one lemma on two-variable polynomials. The Schwartz–Zippel lemma of the *Math Guide* (§“The Schwartz–Zippel lemma”) is univariate; that section’s remark on the multivariate generalisation states the several-variable bound without proof, as a consequence of the univariate lemma applied once per variable, and Lemma 2.9 carries out that derivation for two variables.

Lemma 2.9 (Two variables). *Let $P \in \mathbb{F}[\beta, \gamma]$ be nonzero, of degree at most N in each variable separately. For β, γ drawn independently and uniformly from $\mathbb{F}$, $\Pr[P(\beta, \gamma) = 0] \leq 2N/|\mathbb{F}|$.*

Proof. Write $P = \sum_e c_e(\beta) \gamma^e$ with coefficients $c_e \in \mathbb{F}[\beta]$ of degree at most N . Since $P \neq 0$, some $c_{e_0} \neq 0$, and $c_{e_0}(\beta) = 0$ with probability at most $N/|\mathbb{F}|$ (*Math Guide*, §“The Schwartz–Zippel lemma”). When $c_{e_0}(\beta) \neq 0$, the polynomial $P(\beta, \cdot) \in \mathbb{F}[\gamma]$ is nonzero of degree at most N , so it vanishes at the independent uniform γ with probability at most $N/|\mathbb{F}|$. The union bound (*Math Guide*, §“The union bound and a birthday calculation”) adds the two. $\square$

Construction 2.10 (Permutation argument over m columns). Let $v_0(X), \dots, v_{m-1}(X)$ be the interpolants of the m columns that participate in equality constraints, and let $N = mn$ be the number of participating cells. Label the cell (i, j) (column i , row j) by $\delta_i \omega^j$, where $\delta_0, \dots, \delta_{m-1} \in \mathbb{F}^\times$ are chosen so that the cosets $\delta_i H$ are pairwise disjoint and all N labels are distinct; the deployed choice is $\delta_i = \delta^i$ for an element δ whose multiplicative order is the odd part of $|\mathbb{F}^\times|$ (the halo2 book's *Permutation argument* chapter). Preprocessing fixes the *permuted-label polynomials* $s_i(X)$ with $s_i(\omega^j) = \delta_{i'}$ whenever $\sigma(i, j) = (i', j')$; they are preprocessed like the fixed columns but are not circuit columns, and the verifying key — the verifier's preprocessed description of the circuit, holding commitments to every fixed column and every permuted-label polynomial — contains their commitments. After the columns are committed the verifier sends challenges $\beta, \gamma \in \mathbb{F}$, and the prover commits to a running-product polynomial $Z_{\text{perm}}(X)$ and proves the two identities, both modulo $Z_H(X)$,

$$\begin{aligned} \ell_0(X) (1 - Z_{\text{perm}}(X)) &\equiv 0, \\ Z_{\text{perm}}(\omega X) \prod_{i=0}^{m-1} (v_i(X) + \beta s_i(X) + \gamma) - Z_{\text{perm}}(X) \prod_{i=0}^{m-1} (v_i(X) + \beta \delta_i X + \gamma) &\equiv 0, \end{aligned} \quad (7)$$

which together assert

$$\prod_{i,j} (v_i(\omega^j) + \beta \delta_i \omega^j + \gamma) = \prod_{i,j} (v_i(\omega^j) + \beta s_i(\omega^j) + \gamma). \quad (8)$$

The verifier opens Z_{perm} at the random point z and at ωz , and checks the boundary and the cross-multiplied update.

The first identity fixes $Z_{\text{perm}}(\omega^0) = 1$; the second, holding at every row j , gives $Z_{\text{perm}}(\omega^{j+1}) = Z_{\text{perm}}(\omega^j) \cdot \prod_i (v_i + \beta \delta_i \omega^j + \gamma) / \prod_i (v_i + \beta s_i(\omega^j) + \gamma)$ whenever the denominator is nonzero, and at $j = n - 1$ it returns to row ω^0 , whence (8). The toy is the case $m = 3$ with $(\delta_0, \delta_1, \delta_2) = (1, 2, 3)$, which is not of the deployed form δ^i .

Theorem 2.11 (Soundness of the permutation argument). *The assignment satisfies every equality constraint (cells related by σ carry equal values) if and only if the two sides of (8) are equal as polynomials in β and γ . Consequently the honest prover always satisfies (8), and if some constraint is violated the two sides agree at uniformly random β, γ with probability at most $2N/|\mathbb{F}|$, where N is the number of participating cells.*

Proof. Index cells by κ , write v_κ and λ_κ for value and label, and set

$$P(\beta, \gamma) := \prod_{\kappa} (\gamma + v_\kappa + \beta \lambda_\kappa) - \prod_{\kappa} (\gamma + v_\kappa + \beta \lambda_{\sigma(\kappa)}) \in \mathbb{F}[\beta][\gamma].$$

Each side is a product of monic linear polynomials in γ over the ring $\mathbb{F}[\beta]$, with roots $r_\kappa := -(v_\kappa + \beta \lambda_\kappa)$ and $r'_\kappa := -(v_\kappa + \beta \lambda_{\sigma(\kappa)})$; two such roots are equal in $\mathbb{F}[\beta]$ exactly when their value-label pairs coincide, since a polynomial in β is determined by its coefficients.

If every constraint holds then $v_{\sigma(\kappa)} = v_\kappa$ for all κ , so $r'_\kappa = -(v_{\sigma(\kappa)} + \beta \lambda_{\sigma(\kappa)}) = r_{\sigma(\kappa)}$, and the second product is the first with its factors reindexed by the bijection σ : $P = 0$.

If some constraint fails, pick κ_0 with $v_{\sigma(\kappa_0)} \neq v_{\kappa_0}$. The pair $(v_{\kappa_0}, \lambda_{\sigma(\kappa_0)})$ occurs among the roots r' ; among the roots r the label $\lambda_{\sigma(\kappa_0)}$ occurs exactly once, paired with $v_{\sigma(\kappa_0)} \neq v_{\kappa_0}$, so the multisets of roots differ. Cancel every common factor from both products; a factor $(\gamma - r)$ then remains on one side, say the first, and r is not a root of the other side. Substitute $\gamma := r$, a ring homomorphism $\mathbb{F}[\beta][\gamma] \rightarrow \mathbb{F}[\beta]$ (*Math Guide*, §“Roots and the factor theorem”, evaluation is a ring homomorphism; its proof holds verbatim over the coefficient ring $\mathbb{F}[\beta]$): the first product becomes 0 and the second becomes a product of nonzero elements of $\mathbb{F}[\beta]$, which is nonzero because $\mathbb{F}[\beta]$ is an integral domain (*Math Guide*, §“Degree”, Corollary “ $F[X]$ is an integral domain”). Hence $P \neq 0$. Its degree in each variable is at most N , and Lemma 2.9 bounds the probability that the random challenges hit a zero of P by $2N/|\mathbb{F}|$. $\square$

The honest prover’s ratio is 1 whenever the denominator is nonzero. The deployed boundary rule of Remark 2.17 admits the final value 0, and the error bound changes accordingly.

Corollary 2.12 (Permutation argument under the deployed boundary rule). *Let the columns be fixed before β, γ , drawn uniformly, and let the running products be constrained on the active rows $\omega^0, \dots, \omega^{u-1}$ only, with $Z_{\text{perm}}(\omega^0) = 1$ and the value at the next row ω^u permitted to be 0 or 1 (the deployed rule; Remark 2.17 fixes $u < n$ and gives the identities). If some copy constraint fails, the probability over β, γ that some running products satisfy the update and boundary identities on the active rows is at most $3N/|\mathbb{F}|$, with N the number of participating cells.*

Proof. Suppose some running products satisfy the identities and no numerator factor $v_{\kappa} + \beta \lambda_{\kappa} + \gamma$ vanishes. Then induction from $Z_{\text{perm}}(\omega^0) = 1$ keeps every running value nonzero, so the boundary value is 1, the updates telescope, and the two products over the active rows are equal; by Theorem 2.11 that happens with probability at most $2N/|\mathbb{F}|$. For each β at most N values of γ make a numerator factor vanish, which adds $N/|\mathbb{F}|$, and the union bound gives $2N/|\mathbb{F}| + N/|\mathbb{F}| = 3N/|\mathbb{F}|$. $\square$

Cost. Excluding the final commitment check, the verifier’s work for the argument is $O(m + \log n)$: evaluating the two identities (7) at the random point z needs the $2m$ linear factors, and $\ell_0(z) = (z^n - 1)/(z - 1)$ costs $O(\log n)$ multiplications by the closed form of the Lagrange basis (*Math Guide*, §“The Lagrange basis on H ”). The complete deployed verifier still carries one check linear in n , the multi-scalar multiplication (MSM; *Math Guide*, §“The mixed inner product with group elements”) that closes every opening; §3 develops it under “The verifier’s work: structured scalars and the final generator”.

The deployed chunking. The update identity in (7) has degree $m + 1$ in the committed polynomials, and the active-row factor of Remark 2.17, which masks the rows reserved for blinding, adds one more; so a single product accommodates at most $m \leq d_{\text{max}} - 2$ columns. Columns beyond that are split into chunks of at most $d_{\text{max}} - 2$, each with its own running product, and a boundary identity $\ell_0(X)(Z_{\text{perm},a}(X) - Z_{\text{perm},a-1}(\omega^u X)) \equiv 0$ copies the value at the boundary row ω^u , the first row after the active rows $\omega^0, \dots, \omega^{u-1}$, from one product to the start of the next (the halo2 book’s *Permutation argument* chapter). The column layout is implementation-specific: the protocol specification fixes the Action statement, not its circuit. The deployed Action circuit, whose

implementation is orchard, has fifteen equality-enabled columns (ten advice, one instance, four fixed) and maximum constraint degree $d_{\max} = 9$, and therefore uses three running products of 7, 7 and 1 columns, not one.

2.7 Proving a value sits in a table: lookups

Some constraints admit no low-degree gate on the constrained cells alone. “This cell is a ten-bit number” or “this chunk indexes an entry of the Sinsemilla generator table” (Sinsemilla is the hash of the *Crypto Guide*, §“Sinsemilla: an algebraic hash-based commitment”) has no low-degree formula in the cell itself: written as one polynomial gate, “ $v \in \{0, \dots, 2^{10} - 1\}$ ” is $\prod_{i=0}^{2^{10}-1} (v - i) = 0$, a gate of degree 2^{10} , and a table of arbitrary entries does no better. A decomposition into ten boolean-constrained bits keeps the degree at 2 but spends ten further cells on every such value. The single gate would exceed the degree bound of §2.4: in the toy field its degree 2^{10} exceeds $|\mathbb{F}| = 97$, so the bound $\deg D / |\mathbb{F}|$ is vacuous, and at any field size it multiplies the degree of the quotient the prover must split, commit and open (§2.3). The Action circuit uses range and table constraints throughout.

Membership of every entry of a column A in a table column S is, like a copy constraint, a statement about *multisets*, but it is not a permutation: A may repeat one table entry and omit others. The prover therefore commits to A' , a rearrangement of A in which equal values occupy contiguous *runs*, and to S' , a rearrangement of S in which the first row of each run of A' holds the matching table value. Two *row-local* conditions then give membership: each row of A' either begins a run, $A'_i = S'_i$, so that its value occurs in S , or continues one, $A'_i = A'_{i-1}$. The rearrangements are certified by the running product of §2.6 without labels: labelled factors fix one wiring σ , whereas here the rearrangements are the prover’s choice, so a single running product with unlabelled factors, row by row $(A'_i + \beta)(S'_i + \gamma)$ against $(A_i + \beta)(S_i + \gamma)$, certifies that each primed column is a rearrangement of its original. Induction on the row index then gives membership for every row.

Example 2.13 (A two-bit range check on the toy grid). Constrain cells to two-bit values with the table $S = (0, 1, 2, 3)$ and the column $A = (2, 3, 2, 2)$. Grouped, $A' = (2, 2, 2, 3)$; align $S' = (2, 0, 1, 3)$, a rearrangement of S placing 2 beside the first run and 3 beside the second. Row 0 starts a run ($2 = 2$), rows 1 and 2 continue it (2 above), row 3 starts a run ($3 = 3$): every check is row-local and every polynomial low-degree, however large the table (Figure 6). The unlabelled product certifies the rearrangements: at the challenges $(\beta, \gamma) = (5, 7)$ both $\prod_i (A'_i + \beta)(S'_i + \gamma)$ and $\prod_i (A_i + \beta)(S_i + \gamma)$ equal 82. The forged column $A = (2, 7, 2, 2)$ fails the row checks: its 7 must start a run in any arrangement A' , and no rearrangement of a table without a 7 can put a 7 beside it. Over all 4 arrangements of the forged column and all 24 rearrangements of the table, the row checks pass for none.

The Action circuit uses lookups of both kinds: it range-constrains each ten-bit word of a decomposition by a lookup into the table $\{0, \dots, 2^{10} - 1\}$ (the halo2 book’s *Decomposition* chapter), and it evaluates Sinsemilla by a lookup into a fixed table whose 2^{10} rows hold an index i and the two coordinates of the i -th Sinsemilla generator (the halo2 book’s *Sinsemilla* chapter; the Orchard book’s *Circuit* chapter records the deployed uses of both). A table fact costs a few committed columns and one more running product, and the gates stay low-degree. The form of the

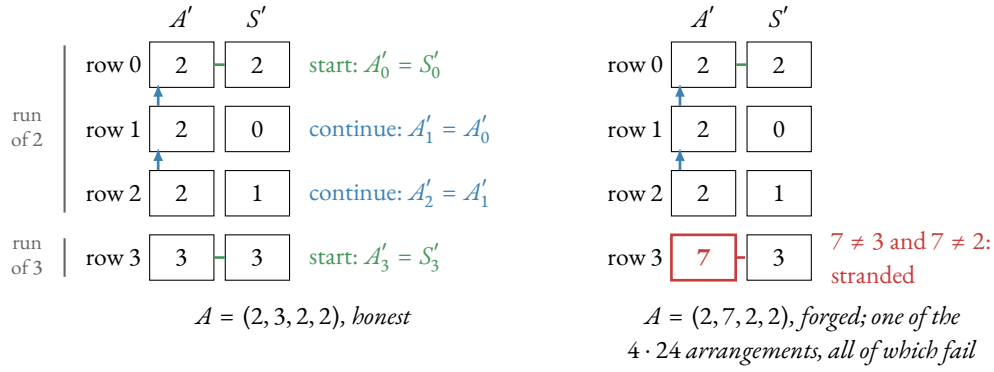


Figure 6. The lookup of Example 2.13. Left: the grouped column A' beside the aligned table S' ; a run starts where $A'_i = S'_i$ (green bar) and continues where $A'_i = A'_{i-1}$ (blue arrow to the row above). Right: the forged column with a 7; the 7 starts a run, but no entry of S' equals 7 and the row above holds 2, so both row-local checks fail at row 3.

Action circuit's gates, one gate in full, and its three lookups are given in §4 under “From statement to circuit: arithmetisation in practice”; the exact identities, the boundary row, and multi-column tables come next.

2.8 The lookup argument

Construction 2.14 (Lookup argument). The objective is that every entry of the column $A(X)$ on the active rows appears as some entry of the column $S(X)$ (the halo2 book's *Lookup argument* chapter). The prover supplies polynomials $A'(X)$ and $S'(X)$ where (i) A' is a permutation of A with like values grouped into contiguous runs, and (ii) S' is a permutation of S in which the first row of each run of like values in A' holds the matching value. The prover commits to A' and S' ; after receiving challenges β, γ she commits to a running-product column $Z_{\text{lookup}}(X)$ and proves the four identities, all modulo $Z_H(X)$,

$$\begin{aligned}
 \ell_0(X) (1 - Z_{\text{lookup}}(X)) &\equiv 0, \\
 Z_{\text{lookup}}(\omega X) (A'(X) + \beta)(S'(X) + \gamma) - Z_{\text{lookup}}(X) (A(X) + \beta)(S(X) + \gamma) &\equiv 0, \\
 \ell_0(X) (A'(X) - S'(X)) &\equiv 0, \\
 (A'(X) - S'(X))(A'(X) - A'(\omega^{-1}X)) &\equiv 0,
 \end{aligned} \tag{9}$$

the second and fourth additionally masked by the active-row factor of Remark 2.17.

Theorem 2.15 (Soundness of the lookup argument). *Let the columns A, S, A', S' be fixed before β, γ are drawn. If the four identities (9) hold on the active rows, with the boundary conditions of Remark 2.17, then except with probability at most $4n/|\mathbb{F}|$ over the challenges β, γ , every value of A on the active rows equals the value of S at some active row.*

Proof. Let the active rows be $0, \dots, u-1$ (Remark 2.17), and write $A_i := A(\omega^i)$, likewise for S, A', S' , and $Z_i := Z_{\text{lookup}}(\omega^i)$. Suppose first that no numerator factor $A_i + \beta$ or $S_i + \gamma$ with $i < u$

vanishes. The columns are fixed before the challenges, so each of the u values $-A_i$ equals β with probability $1/|\mathbb{F}|$, and likewise for γ ; by the union bound the supposition fails with probability at most $2u/|\mathbb{F}|$. Under it, the second identity at row $i < u$ reads $Z_{i+1}(A'_i + \beta)(S'_i + \gamma) = Z_i(A_i + \beta)(S_i + \gamma)$, and induction from $Z_0 = 1$ gives $Z_{i+1} \neq 0$ and $(A'_i + \beta)(S'_i + \gamma) \neq 0$ for every $i < u$. Hence $Z_u \neq 0$, the boundary condition forces $Z_u = 1$, and multiplying the identities over $i < u$ and cancelling the nonzero Z_i gives

$$\prod_{i < u} (A'_i + \beta)(S'_i + \gamma) = \prod_{i < u} (A_i + \beta)(S_i + \gamma).$$

Set $f(\beta) := \prod_i (A'_i + \beta)$, $g(\beta) := \prod_i (A_i + \beta)$, $f'(\gamma) := \prod_i (S'_i + \gamma)$, $g'(\gamma) := \prod_i (S_i + \gamma)$ and $P := ff' - gg' \in \mathbb{F}[\beta, \gamma]$. If A' is a permutation of A and S' of S then $f = g$ and $f' = g'$, so $P = 0$. If instead, say, A' is not a permutation of A , then the multisets of roots $\{-A'_i\}$ and $\{-A_i\}$ differ, so $f \neq g$ by the cancel-and-evaluate argument in the proof of Theorem 2.11 (now over the field $\mathbb{F}$ itself); comparing the coefficients of the top power of γ in $ff' = gg'$, both f' and g' being monic of the same degree, would force $f = g$, so $P \neq 0$, of degree at most u in each variable, and Lemma 2.9 bounds the probability that the challenges hide the difference by $2u/|\mathbb{F}|$. The case of S' is symmetric. The two terms together are at most $2u/|\mathbb{F}| + 2u/|\mathbb{F}| \leq 4n/|\mathbb{F}|$.

So, with that probability excepted, A' is a rearrangement of A and S' of S on the active rows. The third identity fixes row 0: $A'_0 = S'_0$, a table value. The fourth, at every later active row i , says $(A'_i - S'_i)(A'_i - A'_{i-1}) = 0$: either $A'_i = S'_i$, a table value, or $A'_i = A'_{i-1}$. (At row 0 the reference $A'(\omega^{-1}X)$ wraps around to the last row and carries no information, which is why the third identity is needed there.) By induction on i , every A'_i equals the entry of S' at some active row, hence that of S at some active row; and since every entry of A on the active rows occurs in A' there, the claim follows. $\square$

Multi-column tables. A lookup of m input expressions $A_0, \dots, A_{m-1}$ against m table columns $S_0, \dots, S_{m-1}$ is reduced to a single column by a random linear combination with a challenge θ drawn after the input and table columns are committed:

$$A_c(X) := \sum_{j=0}^{m-1} \theta^{m-1-j} A_j(X), \quad S_c(X) := \sum_{j=0}^{m-1} \theta^{m-1-j} S_j(X), \quad (10)$$

and Construction 2.14 runs on A_c and S_c .

Proposition 2.16 (Compression of a multi-column lookup). *Let θ be drawn uniformly after all input and table columns are fixed. If the input tuple at some active row equals the table tuple at no active row, then the compressed input A_c at that row equals S_c at some active row with probability at most $n(m-1)/|\mathbb{F}|$.*

Proof. For each active table row the difference of the compressed values is a polynomial in θ of degree at most $m-1$ whose coefficients are the differences of the tuple entries, not all zero; it is therefore nonzero, and the random θ zeroes it with probability at most $(m-1)/|\mathbb{F}|$ (*Math Guide*, §“The Schwartz–Zippel lemma”). A union bound over the at most n active rows gives $n(m-1)/|\mathbb{F}|$. $\square$

Variable tables, where S is itself an advice column, and range checks, where S enumerates the allowed range, are special cases of the same argument.

Remark 2.17 (Active rows and blinding). The lookup identities, like the permutation identities of §2.6, hold on the *active* rows only. The deployed system withholds the trailing rows of every column from them: a boundary row and, after it, n_{blind} blinding rows, which in every witness column (advice, and the permuted and running-product columns of the arguments) hold uniform random values, for the reason “Zero knowledge: hiding the witness in Halo 2” in §4 gives; fixed and instance columns carry no random values. Let $u := n - n_{\text{blind}} - 1$. Rows $0, \dots, u - 1$ are the active rows, row u is the boundary row, and rows $u + 1, \dots, n - 1$ are the blinding rows. (The letter u indexes a row here; in §3 it denotes a round challenge.) Let $\ell_{\text{blind}}(X)$ be the sum of the Lagrange basis polynomials of the n_{blind} blinding rows, and let $\ell_{\text{last}}(X) := \ell_u(X)$ select the boundary row. Both are computed directly from the domain by both parties; they are not committed circuit columns. Each product-update identity, and the run identity of (9), is multiplied by the factor $(1 - (\ell_{\text{last}}(X) + \ell_{\text{blind}}(X)))$, which is 1 on the active rows and 0 elsewhere, so the random rows do not violate the active-row constraints. The wrap-around that closed the loop in Construction 2.10 is gone, so the running product is pinned at the boundary row instead, by $\ell_{\text{last}}(X) (Z(X)^2 - Z(X)) \equiv 0$, which allows $Z(\omega^u) \in \{0, 1\}$ (the halo2 book’s *Lookup argument* and *Permutation argument* chapters). The value 0 is permitted so that a vanishing *numerator* factor ($A_i + \beta$ or $S_i + \gamma$; in the permutation argument $v_k + \beta \lambda_k + \gamma$) still leaves the honest prover a satisfying assignment: she sets the product to 0 from the next row on. The prover cannot force this event, because β, γ are drawn after her columns are committed. A *denominator* factor that vanishes at an earlier row than every vanishing numerator factor admits no satisfying running product: the update at that row j reads $0 = Z(\omega^j)$ times a nonzero numerator, and $Z(\omega^j) \neq 0$. This happens with probability at most $2u/|F|$ for a lookup and $N/|F|$ for the permutation argument, so completeness is overwhelming, not perfect. A boundary value of 0 is not a certificate of multiset equality, so soundness also excludes the challenges at which a numerator factor vanishes: the bound of Theorem 2.15 includes that term, and Corollary 2.12 adds it for the permutation argument.

2.9 Why the prover cannot wriggle: binding commitments

The random-point argument of §2.4 rests on a hypothesis: Theorem 2.5 assumed the prover’s polynomials were fixed before she saw z . If she could choose a, b, c, t *after* learning z , she would pick numbers making (5) hold and reveal nothing real. A polynomial commitment enforces that hypothesis.

What a commitment must do. A commitment to a polynomial f is a short string $C = \text{Commit}(f)$ with two properties, both developed in the *Crypto Guide*, §“Hiding and binding”:

- **Binding.** Having published C , the prover cannot later open it as two different polynomials; C pins down f . Committing first, then receiving z , then opening, leaves her answering for the *one* polynomial she fixed in advance, the situation Theorem 2.5 assumed.

- **Hiding.** The string C reveals nothing about f . This is what will let the proof be zero-knowledge, under “Zero knowledge: hiding the witness in Halo 2” in §4.

A *polynomial* commitment adds an evaluation proof: given C and a point z , the prover can prove “ $f(z) = v$ ” for the committed f without exposing f , and no prover can produce accepting proofs of two different values at the same point (*evaluation binding*, *Crypto Guide*, §“Polynomial commitment schemes”). Theorem 2.5 concerns the idealised protocol, in which the verifier receives the true evaluations of polynomials fixed in advance. Compiled with a polynomial commitment scheme that is evaluation-binding and extractable, the four-step protocol of §2.4 becomes, under compatible composition hypotheses, an argument of knowledge (*Crypto Guide*, §“SNARKs: succinct non-interactive arguments of knowledge”, Theorem “The compilation recipe, informal”): by its layered extraction, a prover who passes either answers with the true evaluations of polynomials fixed before z , where Theorem 2.5 applies, or breaks the scheme’s evaluation binding or extractability. This volume asserts no bound on the compiled error as a sum of $\deg D/|F|$ and a commitment term (Remark 4.11, §5). For Halo 2 the extractability of the openings is Theorem 3.5 and, for the deployed opening, Corollary 3.13, under the discrete-logarithm assumption on Vesta, with the generator hash modelled as a random oracle.

How Halo 2 commits. Encode f of degree $< n$ by its coefficient vector $\mathbf{f} = (f_0, \dots, f_{n-1})$; the running example’s advice polynomial becomes $\mathbf{a} = (90, 61, 22, 24)$. Let $\mathbb{G}$ be a group of prime order $|F|$ (deployed: Vesta, of order p_{Pallas}), and let $G_0, \dots, G_{n-1}, H \in \mathbb{G}$ be generators derived from a public hash, with $\mathbf{G} := (G_0, \dots, G_{n-1})$; here H is a group element, not the evaluation domain. The commitment is the single group element

$$C = \langle \mathbf{f}, \mathbf{G} \rangle + [r]H = \sum_i [f_i] G_i + [r]H,$$

the Pedersen vector commitment of the coefficient vector with blinder $r \in F$ (*Crypto Guide*, §“Pedersen vector commitments”; the mixed inner product of a scalar vector with a vector of group elements is the *Math Guide*’s, §“The mixed inner product with group elements”). The commitment is perfectly hiding for uniform r and computationally binding under the discrete-logarithm assumption in $\mathbb{G}$, with the hash modelled as a random oracle into $\mathbb{G}$ (*Crypto Guide*, §“Pedersen vector commitments”, Theorem “Properties of the vector commitment”): two openings of one C would exhibit a nontrivial discrete-logarithm relation among the G_i and H . Proving $f(z) = v$ is proving a statement about an inner product, since

$$f(z) = \sum_i f_i z^i = \langle \mathbf{f}, (1, z, z^2, \dots, z^{n-1}) \rangle,$$

and the *inner-product argument* does this in a logarithmic number of rounds, each halving the vectors, until the claim is trivial: the evaluation proof is $O(\log n)$ group elements, and it needs *no trusted setup*, because the generators come from a public hash rather than from anyone’s secret (*Crypto Guide*, §“Polynomial commitment schemes”, the inner-product alternative). Halo 2 uses exactly this one polynomial commitment scheme and no other: the inner-product argument over a group with no pairing (the halo2 book, which the protocol specification names as the definition of the system, §5.4.10.3, “Halo 2”; its *Inner product argument* chapter), on the Vesta curve, where

the protocol specification places the Action statement’s proofs (protocol specification, § 5.4.9.6, “Pallas and Vesta”). The parameters, the generators and the deployed byte layout are given at the end of §3.

Opening a vector commitment. For the argument of this section the commitment must bind, so that the prover answers for polynomials fixed before z is drawn. A vector commitment alone does not provide step 3 of the random-point check. The only opening the Pedersen construction offers is the whole vector together with its blinder, $n + 1$ field elements: sending it reveals f , and it is no shorter than f . The verifier needs $\langle \mathbf{f}, (1, z, \dots, z^{n-1}) \rangle = v$ established for a committed $\mathbf{f}$ he never sees, in fewer than n group elements. That is the inner-product argument of §3.

3 The inner-product polynomial commitment

This section constructs the inner-product polynomial commitment that opens the commitments of §2.9 at the verifier’s point. It states the problem, performs one halving step, recurses it into the full argument with both parties’ tables, gives the verifier’s closing computation, proves knowledge soundness under the discrete-logarithm assumption, proves honest-verifier zero knowledge of the masked opening, runs the running example’s polynomial through every round on a curve of prime order 97, and in the last subsection gives the parameters, the normalisation, the opening, the bytes and the deferred check of the deployed Halo 2. Beyond the *Math Guide*, the section relies on these parts of the *Crypto Guide*: the Pedersen vector commitment (§“Pedersen vector commitments”), extractors and the tree extraction of Sigma-protocols (§“Knowledge soundness and extractors”, §“The Schnorr identification protocol”, §“Sigma-protocols”), the simulation paradigm (§“The simulation paradigm and zero knowledge”), hashing to a curve (§“Hashing to a curve point”) and the standing assumptions (§“Standing assumptions”). From this volume, the generic argument takes only the cue of §2.9; the toy reuses the running example’s advice polynomial and random point (§2.2, §2.4), restated where used; the last subsection takes the deployed circuit’s field from §1.2 and the advice, permutation, lookup and quotient objects from §2, and forward-references §4 for the row count and the remaining objects of the proof; the other references to §2 mark where the argument is used.

3.1 The problem: an inner product in fewer than d group elements

The prover holds a vector $\mathbf{a} = (a_0, \dots, a_{d-1})$ of d field elements. She has published one group element,

$$P = \langle \mathbf{a}, \mathbf{G} \rangle + [r]H = \sum_{i=0}^{d-1} [a_i] G_i + [r]H,$$

the Pedersen vector commitment of $\mathbf{a}$ with blinder r under the public generators $\mathbf{G} = (G_0, \dots, G_{d-1})$ and H (*Crypto Guide*, §“Pedersen vector commitments”; the mixed inner product $\langle \mathbf{a}, \mathbf{G} \rangle$ of a scalar vector with a vector of group elements is the *Math Guide*’s, §“The mixed inner product with group elements”). The verifier holds P , a public vector $\mathbf{b} = (b_0, \dots, b_{d-1})$ of his own,

and a number v the prover claims. He wants to be sure that

$$\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{i=0}^{d-1} a_i b_i = v.$$

There is a trivial protocol: the prover sends $\mathbf{a}$ and r ; the verifier recomputes P from them, which the binding of the commitment makes a meaningful check, and then computes $\langle \mathbf{a}, \mathbf{b} \rangle$ himself. It costs $d + 1$ field elements and reveals $\mathbf{a}$. Neither is acceptable here: the vector is the prover's secret advice, and the verifier of §2.4 asks for evaluations rather than polynomials because a polynomial is long, so a proof as long as the vector gains nothing. The problem is therefore:

Convince the verifier that $\langle \mathbf{a}, \mathbf{b} \rangle = v$ for the vector $\mathbf{a}$ committed in P , using fewer than d group elements, and without revealing $\mathbf{a}$.

Why this is polynomial evaluation. Let $p(X) = a_0 + a_1X + \dots + a_{d-1}X^{d-1}$ be the polynomial with coefficient vector $\mathbf{a}$, and let x be a point. Then

$$p(x) = \sum_{i=0}^{d-1} a_i x^i = \langle \mathbf{a}, (1, x, x^2, \dots, x^{d-1}) \rangle.$$

Evaluating a committed polynomial is therefore *exactly* the problem above, with the public vector $\mathbf{b}(x) := (1, x, x^2, \dots, x^{d-1})$ of powers of the point. The running example makes it concrete. The advice polynomial of §2.2 is $a(X) = 90 + 61X + 22X^2 + 24X^3$ over $\mathbb{F}_{97}$, so $\mathbf{a} = (90, 61, 22, 24)$ and $d = 4$; the random point of §2.4 is $z = 20$, so $\mathbf{b}(20) = (1, 20, 20^2, 20^3) = (1, 20, 12, 46)$ in $\mathbb{F}_{97}$; and the value the verifier needs is $a(20) = \langle \mathbf{a}, \mathbf{b}(20) \rangle = 59$. Everything the random-point check of §2.4 asks of the prover, “open a, b, c and t at z ”, is four instances of this one problem.

Where the difficulty sits. The commitment is linear in the vector: adding two commitments commits to the sum of their vectors, and scaling one by λ commits to $\lambda\mathbf{a}$ (*Crypto Guide*, §“Pedersen vector commitments”, the remark that linearity survives). The inner product is linear in $\mathbf{a}$ too. But the two linear maps land in different places, one in the group and one in the field, and by the hiding of the commitment no public operation takes P to $\langle \mathbf{a}, \mathbf{b} \rangle$. A claim about a vector of length one is checked by sending the scalar and its blinder. The argument that follows reduces the length by *halving*. Each halving step turns a claim about vectors of length 2^j into a claim of the same shape about vectors of length 2^{j-1} , at a price of two group elements, and after $k = \log_2 d$ steps the claim is about a single scalar. The argument uses exactly this linearity.

The relation. Throughout, $\mathbb{G}$ is a cyclic group of prime order p and $\mathbb{F} = \mathbb{F}_p$ its scalar field, so that $\mathbb{G}$ is a one-dimensional $\mathbb{F}$ -vector space (*Math Guide*, §“The mixed inner product with group elements”).

Definition 3.1 (The opening relation). Fix public generators $\mathbf{G} = (G_0, \dots, G_{d-1})$ and H in $\mathbb{G} \setminus \{\mathcal{O}\}$. To open the committed polynomial p at $x \in \mathbb{F}$ with claimed value v is to prove membership

in

$$\mathcal{R}_{\text{IPA}} := \left\{ ((P, x, v); (\mathbf{a}, r)) : P = \langle \mathbf{a}, \mathbf{G} \rangle + [r]H, v = \langle \mathbf{a}, \mathbf{b}(x) \rangle, \mathbf{a} \in \mathbb{F}^d \right\},$$

$$\mathbf{b}(x) := (1, x, \dots, x^{d-1}),$$

where (P, x, v) is the public statement and $(\mathbf{a}, r)$ the witness. The condition $\mathbf{a} \in \mathbb{F}^d$ is the degree bound $\deg p \leq d - 1$: a commitment under d generators can only hold d coefficients.

Notation for the section. The following conventions hold from here to the end of §3.

- The vector length is $d = 2^k$, and d is also the exclusive degree bound and the number of generators. The round count is $k = \log_2 d$. The letter n , the row count of Definition 2.7, is not used until the deployed parameters at the end of the section, where $d = n$; the constraint degree stays $d_{\max}$ as §2.5 fixed it. The *Crypto Guide*'s sketch of this argument indexes its generators from 0 to d and bounds the degree by d inclusively; nothing changes but the labels.
- Generators are indexed from 0, $\mathbf{G} = (G_0, \dots, G_{d-1})$, so that the coefficient a_i of X^i multiplies G_i ; the *Crypto Guide*'s "Pedersen vector commitments" indexes its generators $G_1, \dots, G_n$ instead, a shift of labels only.
- The letter H names the blinding generator $H \in \mathbb{G}$ of the Pedersen commitment; the evaluation domain $H \subset \mathbb{F}$ of §2 does not appear in this section. The letter p names the group order, and with an argument, $p(X)$, the committed polynomial.
- The letter z is the running example's opening point, $z = 20$, and x the generic one. The deployed protocol has a second challenge which its description also calls z ; this volume writes it ζ .
- The structured scalars $s_i \in \mathbb{F}$ of §3.4 are scalars; the permuted-label polynomials $s_i(X)$ of Construction 2.10 are polynomials. Type and argument disambiguate. The mask $s(X)$ of §3.6 is one further polynomial, with coefficient vector $\mathbf{m}$.
- In the byte count of §3.8, a is the number of Actions, an integer, not the vector $\mathbf{a}$.
- A vector of length 2^j splits into its *low half*, the first 2^{j-1} entries, and its *high half*, the last 2^{j-1} : $\mathbf{a} = (\mathbf{a}_{\text{lo}}, \mathbf{a}_{\text{hi}})$, and likewise for $\mathbf{G}$ and $\mathbf{b}$.

3.2 One fold, every step written out

One halving step is called a *fold*. This subsection performs one fold on a vector of length d .

Carry the claim inside the group element. The commitment P says nothing about $\mathbf{b}$ or v . Bring them in: after P is fixed, the verifier samples a further generator U uniformly from $\mathbb{G} \setminus \{\mathcal{O}\}$ and sends it. Both parties form

$$C := P + [v] U. \quad (11)$$

If the prover is honest, $C = \langle \mathbf{a}, \mathbf{G} \rangle + [r]H + [\langle \mathbf{a}, \mathbf{b} \rangle] U$: a single group element carrying three kinds of information on three independent sets of generators, the vector on $\mathbf{G}$, the blinder on H , and the inner product on U . The claim " P commits to $\mathbf{a}$ and $\langle \mathbf{a}, \mathbf{b} \rangle = v$ " has become the claim

that C has this one *shape*, for some $\mathbf{a}$ and r , with the U -coefficient tied to the $\mathbf{G}$ -coefficients by the inner product with the public $\mathbf{b}$. The fold preserves exactly this shape while halving the vectors.

Split into halves. Write $\mathbf{a} = (\mathbf{a}_{\text{lo}}, \mathbf{a}_{\text{hi}})$, $\mathbf{G} = (\mathbf{G}_{\text{lo}}, \mathbf{G}_{\text{hi}})$ and $\mathbf{b} = (\mathbf{b}_{\text{lo}}, \mathbf{b}_{\text{hi}})$, each half of length $d/2$. An inner product splits along the halves:

$$\langle \mathbf{a}, \mathbf{G} \rangle = \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{lo}} \rangle + \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{hi}} \rangle, \quad \langle \mathbf{a}, \mathbf{b} \rangle = \langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{lo}} \rangle + \langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{hi}} \rangle.$$

The fold. Let $u \in \mathbb{F}^\times$ be a nonzero challenge, and define the *folded* vectors of length $d/2$,

$$\mathbf{a}' := u \mathbf{a}_{\text{lo}} + u^{-1} \mathbf{a}_{\text{hi}}, \quad \mathbf{G}' := u^{-1} \mathbf{G}_{\text{lo}} + u \mathbf{G}_{\text{hi}}, \quad \mathbf{b}' := u^{-1} \mathbf{b}_{\text{lo}} + u \mathbf{b}_{\text{hi}}, \quad (12)$$

entry by entry: $a'_i = u a_i + u^{-1} a_{i+d/2}$, $G'_i = [u^{-1}]G_i + [u]G_{i+d/2}$ and $b'_i = u^{-1}b_i + u b_{i+d/2}$ for $0 \leq i < d/2$. The coefficient vector is folded with the weights (u, u^{-1}) and the generator and evaluation vectors with the *inverse* weights (u^{-1}, u) ; this is the symmetric fold of the Halo paper and of Bulletproofs, and the reason for the inverse weights appears in the next computation.

The inner product of the folded vectors. Expand $\langle \mathbf{a}', \mathbf{G}' \rangle$ using bilinearity of the mixed inner product (*Math Guide*, §“The mixed inner product with group elements”), one term per pair of halves:

$$\begin{aligned} \langle \mathbf{a}', \mathbf{G}' \rangle &= \langle u \mathbf{a}_{\text{lo}} + u^{-1} \mathbf{a}_{\text{hi}}, u^{-1} \mathbf{G}_{\text{lo}} + u \mathbf{G}_{\text{hi}} \rangle \\ &= u u^{-1} \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{lo}} \rangle + u u \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + u^{-1} u^{-1} \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle + u^{-1} u \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{hi}} \rangle \\ &= \underbrace{\langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{lo}} \rangle + \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{hi}} \rangle}_{= \langle \mathbf{a}, \mathbf{G} \rangle} + u^2 \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + u^{-2} \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle. \end{aligned}$$

The two *diagonal* terms, low with low and high with high, have weight $u u^{-1} = 1$ and reassemble the original $\langle \mathbf{a}, \mathbf{G} \rangle$: that is what the inverse weights were for. The two *cross* terms, low with high and high with low, survive with the weights u^2 and u^{-2} . The same expansion for the scalar inner product, with $\mathbf{b}'$ folded by the same weights as $\mathbf{G}'$, gives

$$\langle \mathbf{a}', \mathbf{b}' \rangle = \langle \mathbf{a}, \mathbf{b} \rangle + u^2 \langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle + u^{-2} \langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle.$$

The cross terms do not depend on the challenge. The four cross inner products $\langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle$, $\langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle$, $\langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle$ and $\langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle$ involve only the halves, not u . The prover can therefore commit to them *before* the challenge is drawn. She samples two fresh blinders $\ell, \rho \in \mathbb{F}$ and sends the two group elements

$$L := \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + [\ell]H + [\langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle] U, \quad R := \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle + [\rho]H + [\langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle] U, \quad (13)$$

each of the same three-part shape as C : a vector part on the generators, a blinder on H , and the matching scalar inner product on U . Only then does the verifier draw u .

The fold identity. Now add the cross terms to C with the weights the expansion produced and regroup by generator set:

$$\begin{aligned}
C + [u^2]L + [u^{-2}]R &= \langle \mathbf{a}, \mathbf{G} \rangle + [r]H + [\langle \mathbf{a}, \mathbf{b} \rangle] U \\
&\quad + u^2 \left(\langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + [\ell]H + [\langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle] U \right) \\
&\quad + u^{-2} \left(\langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle + [\rho]H + [\langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle] U \right) \\
&= \left(\langle \mathbf{a}, \mathbf{G} \rangle + u^2 \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + u^{-2} \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle \right) \\
&\quad + [r + u^2 \ell + u^{-2} \rho]H + \left[\langle \mathbf{a}, \mathbf{b} \rangle + u^2 \langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle + u^{-2} \langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle \right] U \\
&= \langle \mathbf{a}', \mathbf{G}' \rangle + [r']H + [\langle \mathbf{a}', \mathbf{b}' \rangle] U,
\end{aligned}$$

where $r' := r + u^2 \ell + u^{-2} \rho$ and the last line substitutes the two expansions. Written as one equation, the *fold identity* is

$$C' := C + [u^2]L + [u^{-2}]R = \langle \mathbf{a}', \mathbf{G}' \rangle + [r']H + [\langle \mathbf{a}', \mathbf{b}' \rangle] U. \quad (14)$$

What has been achieved. Compare the two sides of (14) with (11). The element C' has the same shape as C , over vectors of half the length: its $\mathbf{G}'$ -part is the folded coefficient vector, its H -part a blinder, and its U -coefficient the inner product of the folded vectors. The verifier can compute C' , $\mathbf{G}'$ and $\mathbf{b}'$ entirely on his own from C, L, R, u and the public data. The prover computes $\mathbf{a}'$ and r' . The claim about $(\mathbf{a}, \mathbf{b}, \mathbf{G})$ of length d has been replaced by a claim about $(\mathbf{a}', \mathbf{b}', \mathbf{G}')$ of length $d/2$, and the exchange cost exactly two group elements and one field element of challenge. Figure 7 shows the fold and identity (14). Only completeness is claimed here; that a prover who cannot open C cannot open C' either is proved in §3.5.

The evaluation vector folds into a power vector. When $\mathbf{b} = \mathbf{b}(x)$, the high half is the low half scaled: $\mathbf{b}_{\text{hi}} = x^{d/2} \mathbf{b}_{\text{lo}}$, since $b_{i+d/2} = x^{i+d/2} = x^{d/2} x^i$. Hence

$$\mathbf{b}' = u^{-1} \mathbf{b}_{\text{lo}} + u x^{d/2} \mathbf{b}_{\text{lo}} = (u^{-1} + u x^{d/2}) (1, x, \dots, x^{d/2-1}) :$$

the folded evaluation vector is again a vector of powers of x , scaled by the scalar $u^{-1} + u x^{d/2}$. The verifier never needs to store $\mathbf{b}'$; he needs one scalar per round. This is the seed of the closed form g of §3.4.

The fold at $d = 4$, symbolically. For the running example's length, the halves are $\mathbf{a}_{\text{lo}} = (a_0, a_1)$, $\mathbf{a}_{\text{hi}} = (a_2, a_3)$, $\mathbf{G}_{\text{lo}} = (G_0, G_1)$, $\mathbf{G}_{\text{hi}} = (G_2, G_3)$, $\mathbf{b}_{\text{lo}} = (1, x)$ and $\mathbf{b}_{\text{hi}} = (x^2, x^3)$, so the cross terms of (13) read

$$\begin{aligned}
L &= [a_0]G_2 + [a_1]G_3 + [\ell]H + [a_0x^2 + a_1x^3] U, \\
R &= [a_2]G_0 + [a_3]G_1 + [\rho]H + [a_2 + a_3x] U,
\end{aligned}$$

and the folded vectors are $\mathbf{a}' = (ua_0 + u^{-1}a_2, ua_1 + u^{-1}a_3)$, $\mathbf{G}' = ([u^{-1}]G_0 + [u]G_2, [u^{-1}]G_1 + [u]G_3)$ and $\mathbf{b}' = (u^{-1} + ux^2) (1, x)$. Section 3.7 fills these in with numbers.

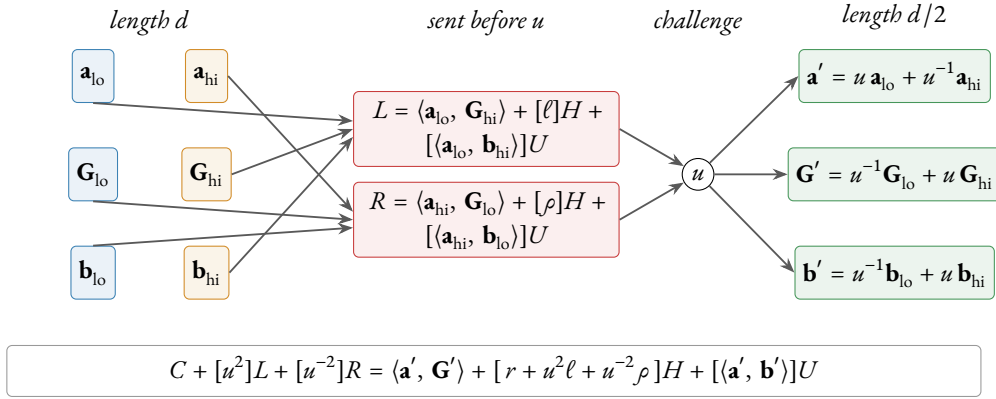


Figure 7. One fold. The three vectors of length d (left; low halves in blue, high halves in amber) determine the two cross terms L and R (red): L from $\mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}}$ and $\mathbf{b}_{\text{hi}}$, R from the complementary halves, both sent before the challenge u is drawn. The challenge then folds all three vectors to length $d/2$ (green). The identity beneath is equation (14): the folded commitment has the same three-part shape as the original, over the folded vectors, with the cross terms absorbed at weights u^2 and u^{-2} .

3.3 The recursion: k rounds to a single scalar

The folded claim of §3.2 has the shape of the original claim, so it can be folded again. After $k = \log_2 d$ folds the vectors have length one, and a length-one claim is checked by sending the scalar and its blinder. That is the whole protocol; this subsection writes it out as a construction, numbers the rounds, and lays the two parties' work side by side.

Construction 3.2 (IPA-based polynomial commitment). Let G be a cyclic group of prime order p in which the discrete logarithm problem is hard (*Crypto Guide*, §“Standing assumptions”), and let $F = \mathbb{F}_p$. Choose $d = 2^k$ and sample the generators $\mathbf{G} = (G_0, \dots, G_{d-1})$ and H independently and uniformly from $G \setminus \{\mathcal{O}\}$, as in the *Crypto Guide*’s Construction “Pedersen vector commitment” (§“Pedersen vector commitments”). Deployed generators are instead hash-to-curve outputs on public labels, uniform when hash-to-curve is modelled as a random oracle into G (*Crypto Guide*, §“Hashing to a curve point”, Remark “Why NUMS generators are binding-safe”).

Commitment. For $p(X) = \sum_{i=0}^{d-1} a_i X^i \in F[X]$ and a blinder $r \in F$,

$$\text{Commit}(p; r) := \langle \mathbf{a}, \mathbf{G} \rangle + [r]H \in G.$$

Opening. To prove $((P, x, v); (\mathbf{a}, r)) \in \mathcal{R}_{\text{IPA}}$, the two parties proceed as follows.

1. **Setup.** The verifier samples U uniformly from $G \setminus \{\mathcal{O}\}$ and sends it. Both parties set $C^{(k)} := P + [v]U$, $\mathbf{G}^{(k)} := \mathbf{G}$ and $\mathbf{b}^{(k)} := \mathbf{b}(x)$; the prover sets $\mathbf{a}^{(k)} := \mathbf{a}$ and $r^{(k)} := r$.
2. **Rounds.** For $j = k, k-1, \dots, 1$, the vectors $\mathbf{a}^{(j)}, \mathbf{G}^{(j)}, \mathbf{b}^{(j)}$ have length 2^j and are split into halves as in §3.2.

(a) The prover samples independent uniform $\ell_j, \rho_j \in \mathbb{F}$ and sends

$$\begin{aligned} L_j &:= \langle \mathbf{a}_{\text{lo}}^{(j)}, \mathbf{G}_{\text{hi}}^{(j)} \rangle + [\ell_j]H + [\langle \mathbf{a}_{\text{lo}}^{(j)}, \mathbf{b}_{\text{hi}}^{(j)} \rangle] U, \\ R_j &:= \langle \mathbf{a}_{\text{hi}}^{(j)}, \mathbf{G}_{\text{lo}}^{(j)} \rangle + [\rho_j]H + [\langle \mathbf{a}_{\text{hi}}^{(j)}, \mathbf{b}_{\text{lo}}^{(j)} \rangle] U. \end{aligned}$$

(b) The verifier sends a uniform $u_j \in \mathbb{F}^\times$.

(c) Both parties fold by (12):

$$\begin{aligned} \mathbf{G}^{(j-1)} &:= u_j^{-1} \mathbf{G}_{\text{lo}}^{(j)} + u_j \mathbf{G}_{\text{hi}}^{(j)}, & \mathbf{b}^{(j-1)} &:= u_j^{-1} \mathbf{b}_{\text{lo}}^{(j)} + u_j \mathbf{b}_{\text{hi}}^{(j)}, \\ C^{(j-1)} &:= C^{(j)} + [u_j^2]L_j + [u_j^{-2}]R_j; \end{aligned}$$

the prover also folds $\mathbf{a}^{(j-1)} := u_j \mathbf{a}_{\text{lo}}^{(j)} + u_j^{-1} \mathbf{a}_{\text{hi}}^{(j)}$ and $r^{(j-1)} := r^{(j)} + u_j^2 \ell_j + u_j^{-2} \rho_j$.

3. **Final message.** After round 1 every vector has length one. Write $a := \mathbf{a}^{(0)}$, $G^{(0)} := \mathbf{G}^{(0)}$ and $b^{(0)} := \mathbf{b}^{(0)}$ for the single entries. The prover sends the scalar a and the aggregated blinder

$$r^* := r^{(0)} = r + \sum_{j=1}^k (u_j^2 \ell_j + u_j^{-2} \rho_j).$$

4. **Check.** The verifier accepts if and only if

$$P + [v]U + \sum_{j=1}^k ([u_j^2]L_j + [u_j^{-2}]R_j) \stackrel{?}{=} [a]G^{(0)} + [r^*]H + [a \cdot b^{(0)}]U, \quad (15)$$

where the left-hand side is $C^{(0)}$ and $G^{(0)}$, $b^{(0)}$ are computed as §3.4 describes.

The round index descends so that round j begins with vectors of length 2^j ; round k is the first round and round 1 the last. The scheme instantiates the polynomial-commitment interface of the *Crypto Guide* (§“Polynomial commitment schemes”, the inner-product alternative), and the opening protocol is the inner-product argument of Bootle, Cerulli, Chaidos, Groth and Petit as refined in Bulletproofs by Bünz, Bootle, Boneh, Poelstra, Wuille and Maxwell, in the symmetric normalisation those papers use.

Completeness. The honest prover always passes. By induction on the rounds,

$$C^{(j)} = \langle \mathbf{a}^{(j)}, \mathbf{G}^{(j)} \rangle + [r^{(j)}]H + [\langle \mathbf{a}^{(j)}, \mathbf{b}^{(j)} \rangle] U \quad \text{for } j = k, k-1, \dots, 0 : \quad (16)$$

at $j = k$ this is (11) with the honest P , and the step from j to $j-1$ is the fold identity (14) applied to the round's vectors, challenge and blinders. At $j = 0$ the vectors have one entry each, the inner products are products of scalars, and (16) reads $C^{(0)} = [a]G^{(0)} + [r^*]H + [a b^{(0)}]U$, which is (15). The only way an honest run can fail is a challenge $u_j = 0$, which the verifier excludes.

Proof size. The prover sends two group elements per round and two field elements at the end: $2k$ group elements plus $O(1)$, against the $d+1 = 2^k + 1$ field elements of the trivial protocol. The

verifier keeps only the challenges $u_1, \dots, u_k$ and the received points; he need not fold $\mathbf{G}$ round by round, as the next subsection shows. Figure 8 draws the k rounds, and Tables 2 and 3 list, one row per message, what each party holds, computes and sends, so that a reader can play either role; §3.7 refills the same rows with numbers.

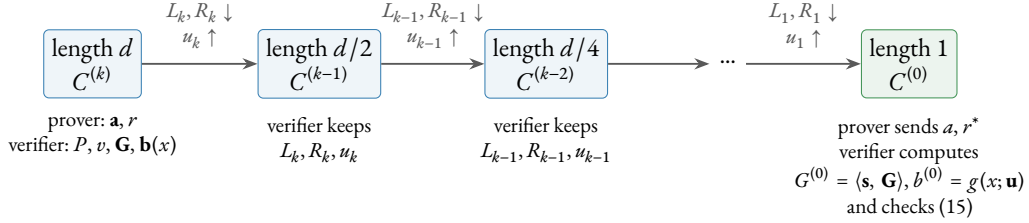


Figure 8. The k rounds of Construction 3.2. Each round halves the vector length: the prover sends two points ($\downarrow$), the verifier answers with one challenge ($\uparrow$) and keeps the three values. After k rounds one scalar and one aggregated blinder close the argument; the verifier’s closing work is the subject of §3.4.

Table 2. The prover’s side of Construction 3.2, one row per message. The round row repeats for $j = k, \dots, 1$.

step	receives / holds	computes and sends
setup	holds $\mathbf{a}, r, x, v$; receives U	sets $\mathbf{a}^{(k)} := \mathbf{a}, \mathbf{G}^{(k)} := \mathbf{G}, \mathbf{b}^{(k)} := \mathbf{b}(x), r^{(k)} := r$
round j	vectors of length 2^j , split into halves receives u_j	samples ℓ_j, ρ_j ; sends L_j, R_j as in Construction 3.2 folds $\mathbf{a}^{(j-1)} = u_j \mathbf{a}_{\text{lo}}^{(j)} + u_j^{-1} \mathbf{a}_{\text{hi}}^{(j)}, \mathbf{G}^{(j-1)}, \mathbf{b}^{(j-1)}$; $r^{(j-1)} = r^{(j)} + u_j^2 \ell_j + u_j^{-2} \rho_j$
final	one entry each: $a = \mathbf{a}^{(0)}, r^* = r^{(0)}$	sends a, r^*

3.4 The verifier’s work: structured scalars and the final generator

The check (15) needs two quantities the verifier has not yet been told how to obtain cheaply: the final generator $G^{(0)}$ and the final evaluation scalar $b^{(0)}$. Folding $\mathbf{G}$ round by round costs $d + d/2 + \dots + 2 = 2d - 2$ scalar multiplications (two per entry of each folded vector), and folding $\mathbf{b}$ likewise many field operations. Both have closed forms in the challenges.

Theorem 3.3 (Structured scalars and the polynomial g). *For $0 \leq i < d$ write the binary expansion $i = \sum_{\ell=0}^{k-1} \beta_\ell 2^\ell$ with digits $\beta_\ell \in \{0, 1\}$, and define the structured scalars*

$$s_i := \prod_{\ell=1}^k u_\ell^{(-1)^{1-\beta_{\ell-1}}} \quad (\text{the factor is } u_\ell \text{ when } \beta_{\ell-1} = 1 \text{ and } u_\ell^{-1} \text{ when } \beta_{\ell-1} = 0). \quad (17)$$

Then, after the k folds of Construction 3.2,

$$G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle = \sum_{i=0}^{d-1} [s_i] G_i, \quad b^{(0)} = \langle \mathbf{s}, \mathbf{b}(x) \rangle = g(x; u_1, \dots, u_k), \quad (18)$$

Table 3. The verifier's side of Construction 3.2, one row per message, with the closing computations of §3.4. The round row repeats for $j = k, \dots, 1$; folding C round by round is optional because (15) sums the cross terms directly.

step	receives / holds	computes and sends
setup	holds $P, x, v, \mathbf{G}, H$	samples U and sends it; sets $C^{(k)} := P + [v]U$
round j	receives L_j, R_j	samples $u_j \in \mathbb{F}^*$ and sends it; keeps (L_j, R_j, u_j) ; optionally $C^{(j-1)} := C^{(j)} + [u_j^2]L_j + [u_j^{-2}]R_j$
final	receives a, r^*	computes $\mathbf{s}$ and $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle$ (one length- d multi-scalar multiplication), $b^{(0)} = g(x; u_1, \dots, u_k)$ ($O(k)$ field operations), and checks (15)

where

$$g(X; u_1, \dots, u_k) := \prod_{\ell=1}^k (u_\ell^{-1} + u_\ell X^{2^{\ell-1}}), \quad (19)$$

a polynomial of degree $d - 1$ in X .

Proof. Follow one original generator G_m through the folds. Claim: after the rounds $k, k-1, \dots, j+1$ have been performed, G_m contributes to exactly one entry of $\mathbf{G}^{(j)}$, the entry at position $m \bmod 2^j$, with coefficient $\prod_{\ell=j+1}^k u_\ell^{(-1)^{1-\beta_{\ell-1}}}$, where $\beta_{\ell-1}$ is the digit of m at position $\ell-1$. At $j = k$ nothing has happened and the claim says G_m sits at position m with coefficient 1. For the step from j to $j-1$: round j folds the length- 2^j vector by $G_i^{(j-1)} = [u_j^{-1}]G_i^{(j)} + [u_j]G_{i+2^{j-1}}^{(j)}$ for $0 \leq i < 2^{j-1}$. The position $m \bmod 2^j$ lies in the low half $[0, 2^{j-1})$ exactly when the digit β_{j-1} of m is 0, in which case it moves to position $m \bmod 2^{j-1}$ with the extra factor u_j^{-1} ; and it lies in the high half exactly when $\beta_{j-1} = 1$, in which case it is the entry $i + 2^{j-1}$ with $i = m \bmod 2^{j-1}$ and moves to position i with the extra factor u_j . Either way the new coefficient is the old one times $u_j^{(-1)^{1-\beta_{j-1}}}$, which is the claim for $j-1$. At $j = 0$ every G_m sits in the single entry $G^{(0)}$ with coefficient s_m , which is the first half of (18). The vector $\mathbf{b}$ is folded with the same weights, so the same argument gives $b^{(0)} = \sum_m s_m b_m = \langle \mathbf{s}, \mathbf{b}(x) \rangle$.

For the closed form, expand the product (19): choosing from the ℓ -th factor either u_ℓ^{-1} (when $\beta_{\ell-1} = 0$) or $u_\ell X^{2^{\ell-1}}$ (when $\beta_{\ell-1} = 1$) produces, for each m , the monomial $s_m X^{\sum_{\ell=1}^k \beta_{\ell-1} 2^{\ell-1}} = s_m X^m$, and every $m \in [0, d)$ arises from exactly one choice. Hence $g(X) = \sum_m s_m X^m$, whose value at x is $\langle \mathbf{s}, \mathbf{b}(x) \rangle$, and whose degree is $d-1$ since the top monomial has coefficient $s_{d-1} = \prod_\ell u_\ell \neq 0$. $\square$

At $d = 4$ the scalars are $s_0 = u_1^{-1}u_2^{-1}$, $s_1 = u_1u_2^{-1}$, $s_2 = u_1^{-1}u_2$ and $s_3 = u_1u_2$, and $g(x; u_1, u_2) = (u_1^{-1} + u_1x)(u_2^{-1} + u_2x^2)$: the digit of weight 1 selects the exponent of u_1 , the round that folded pairs of neighbours, and the digit of weight 2 selects the exponent of u_2 , the round that folded the two halves of the whole vector.

The final equation, and why it checks. The verifier’s closing work is now explicit. He evaluates $b^{(0)} = g(x; \mathbf{u})$ from the challenges; he computes $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle$, one multi-scalar multiplication of length d ; he adds up the left-hand side of (15) from P , U , the $2k$ received points and the challenges, $2k + 1$ scalar multiplications; and he compares it with the right-hand side, three more scalar multiplications. For the honest prover the two sides agree by the invariant (16). That a dishonest prover cannot make them agree except by knowing an opening is §3.5.

The cost of the final check. Evaluating g at x costs $3k - 2$ field multiplications: k products $u_\ell x^{2^{\ell-1}}$, $k - 1$ squarings to produce the powers $x, x^2, x^4, \dots$, and $k - 1$ multiplications of the factors together; at $k = 2$ that is four multiplications, and at Orchard’s $k = 11$ it is 31. But $G^{(0)}$ is a linear combination of *all* d generators, and computing it is a multi-scalar multiplication of length d , at Orchard’s $d = 2^{11}$ some 2048 scalar multiplications: the one part of the verifier’s work that is linear in the size of the committed vector, and the check “linear in n ” that §2.6 deferred to this subsection. Everything else the verifier does is logarithmic. The asymmetry has a name in the closed form: $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle$ is nothing but $\text{Commit}(g; 0)$, the unblinded commitment to the polynomial g whose value the verifier just computed in $O(k)$ operations. Checking a *value* of g is cheap; checking its *commitment* is not. This asymmetry is the seed of the accumulation scheme of §7, and of the deferral described at the end of this section.

3.5 Why it is binding: extraction from three transcripts

This subsection shows that *any* prover that makes the verifier of Construction 3.2 accept knows an opening $(\mathbf{a}, r)$ of P with $\langle \mathbf{a}, \mathbf{b}(x) \rangle = v$, in the operational sense of the *Crypto Guide* (§“Knowledge soundness and extractors”): an extractor that runs the prover as a subroutine, rewinds it to an earlier state and feeds it fresh challenges computes the pair from its answers. The mechanism is the one that makes Schnorr’s identification protocol a proof of knowledge (*Crypto Guide*, §“The Schnorr identification protocol”): a single transcript reveals nothing, but two accepting transcripts with the same first message and distinct challenges are two linear equations, and subtracting them cancels the unknown commitment and leaves the witness. Here each round has three unknown group elements instead of one, so three transcripts are needed per round, and the k rounds stack the three-way branching into a tree, the tree special soundness of the *Crypto Guide* (§“Sigma-protocols”). The single-round step is pure linear algebra and is proved in full; the tree is an induction on it; the discrete-logarithm assumption enters once, at the root.

Representations. Fix a level j with generator vector $\mathbf{G}^{(j)}$ of length 2^j . A *representation* of an element $Q \in G$ over $(\mathbf{G}^{(j)}, H, U)$ is a triple $(\mathbf{x}; y; w) \in \mathbb{F}^{2^j} \times \mathbb{F} \times \mathbb{F}$ with

$$Q = \langle \mathbf{x}, \mathbf{G}^{(j)} \rangle + [y]H + [w]U.$$

Call the representation *well formed* at level j if its U -coordinate is the inner product of its $\mathbf{G}$ -coordinates with the round’s evaluation vector, $w = \langle \mathbf{x}, \mathbf{b}^{(j)} \rangle$. The invariant (16) says exactly that the honest prover holds a well-formed representation $(\mathbf{a}^{(j)}; r^{(j)}; \langle \mathbf{a}^{(j)}, \mathbf{b}^{(j)} \rangle)$ of $C^{(j)}$ at every level. A representation over folded generators lifts to one over the generators of the level above: if $Q =$

$\langle \mathbf{a}', \mathbf{G}' \rangle + [\gamma]H + [w]U$ with $\mathbf{G}' = u^{-1}\mathbf{G}_{\text{lo}} + u\mathbf{G}_{\text{hi}}$, then by bilinearity

$$Q = \langle (u^{-1}\mathbf{a}', u\mathbf{a}'), \mathbf{G} \rangle + [\gamma]H + [w]U, \quad (20)$$

and the lift preserves well-formedness, since $\langle (u^{-1}\mathbf{a}', u\mathbf{a}'), \mathbf{b} \rangle = \langle \mathbf{a}', u^{-1}\mathbf{b}_{\text{lo}} + u\mathbf{b}_{\text{hi}} \rangle = \langle \mathbf{a}', \mathbf{b}' \rangle$. Under the discrete-logarithm assumption a prover can know at most one representation of any Q : two of them differ by a nontrivial relation among $\mathbf{G}, H$ and U , which is the binding argument of the *Crypto Guide* (§“Pedersen vector commitments”). Nothing below uses this until the root is reached.

Lemma 3.4 (Single-round recovery). *Fix one round of Construction 3.2 at level j : the verifier’s element $C := C^{(j)}$, the generators $\mathbf{G} := \mathbf{G}^{(j)}$ and evaluation vector $\mathbf{b} := \mathbf{b}^{(j)}$, and the prover’s cross terms L, R . Let u, u', u'' be three challenges for this round, and suppose that for each $v \in \{u, u', u''\}$ a representation $(\mathbf{x}_v; \gamma_v; w_v)$ of the folded element $C + [\nu^2]L + [\nu^{-2}]R$ over $(\mathbf{G}, H, U)$ is known. If the squares u^2, u'^2, u''^2 are pairwise distinct, then representations $(\mathbf{x}_L; \gamma_L; w_L)$, $(\mathbf{x}_C; \gamma_C; w_C)$ and $(\mathbf{x}_R; \gamma_R; w_R)$ of L, C and R over $(\mathbf{G}, H, U)$ are obtained by solving one 3×3 linear system with rows $(\nu^2, 1, \nu^{-2})$, and they satisfy, for each ν ,*

$$\mathbf{x}_\nu = \nu^2 \mathbf{x}_L + \mathbf{x}_C + \nu^{-2} \mathbf{x}_R, \quad \gamma_\nu = \nu^2 \gamma_L + \gamma_C + \nu^{-2} \gamma_R, \quad w_\nu = \nu^2 w_L + w_C + \nu^{-2} w_R. \quad (21)$$

If the three given representations are well formed at level j , so is the recovered representation of C .

Proof. Write the three known representations as vectors $\mathbf{Y}_\nu := (\mathbf{x}_\nu, \gamma_\nu, w_\nu) \in \mathbb{F}^{2^j+2}$ and the three unknown ones as $\mathbf{X}_L, \mathbf{X}_C, \mathbf{X}_R$. The three group equations

$$[\nu^2]L + C + [\nu^{-2}]R = \langle \mathbf{x}_\nu, \mathbf{G} \rangle + [\gamma_\nu]H + [w_\nu]U, \quad \nu \in \{u, u', u''\},$$

suggest the linear system

$$M \begin{pmatrix} \mathbf{X}_L \\ \mathbf{X}_C \\ \mathbf{X}_R \end{pmatrix} = \begin{pmatrix} \mathbf{Y}_u \\ \mathbf{Y}_{u'} \\ \mathbf{Y}_{u''} \end{pmatrix}, \quad M := \begin{pmatrix} u^2 & 1 & u^{-2} \\ u'^2 & 1 & u'^{-2} \\ u''^2 & 1 & u''^{-2} \end{pmatrix},$$

to be solved coordinate by coordinate. Multiplying the row of ν by the nonzero scalar ν^2 turns it into $(\nu^4, \nu^2, 1)$, which does not change the kernel. If $M(c_0, c_1, c_2)^\top = 0$, the polynomial $c_0 W^2 + c_1 W + c_2$ of degree at most 2 vanishes at $W = u^2, u'^2, u''^2$; when these squares are pairwise distinct it is zero (*Math Guide*, §“Roots and the factor theorem”, Theorem “At most d roots”), so M is invertible. When two squares coincide, two rows of M coincide and M is singular. Define $(\mathbf{X}_L, \mathbf{X}_C, \mathbf{X}_R)$ as the unique solution. Because it solves the system exactly, $\mathbf{Y}_\nu = \nu^2 \mathbf{X}_L + \mathbf{X}_C + \nu^{-2} \mathbf{X}_R$ for each ν , which is (21) read coordinate by coordinate. That the solutions are representations of L, C and R follows by taking the same linear combinations of the group equations: let $(\alpha, \alpha', \alpha'')$ be the middle row of M^{-1} , so that $\alpha(u^2, 1, u^{-2}) + \alpha'(u'^2, 1, u'^{-2}) + \alpha''(u''^2, 1, u''^{-2}) = (0, 1, 0)$ and $\mathbf{X}_C = \alpha \mathbf{Y}_u + \alpha' \mathbf{Y}_{u'} + \alpha'' \mathbf{Y}_{u''}$. Adding α times the first group equation, α' times the second and α'' times the third gives, on the left, $[0]L + [1]C + [0]R = C$, and on the right, by bilinearity, $\langle \mathbf{x}_C, \mathbf{G} \rangle + [\gamma_C]H + [w_C]U$. The first and third rows of M^{-1} do the same for L and R . For well-formedness, set $\delta_\nu := w_\nu - \langle \mathbf{x}_\nu, \mathbf{b} \rangle$ and $\delta_L := w_L - \langle \mathbf{x}_L, \mathbf{b} \rangle$, $\delta_C := w_C - \langle \mathbf{x}_C, \mathbf{b} \rangle$,

$\delta_R := w_R - \langle \mathbf{x}_R, \mathbf{b} \rangle$. Subtracting the inner product of the first identity of (21) with $\mathbf{b}$ from the third gives $\delta_\nu = \nu^2 \delta_L + \delta_C + \nu^{-2} \delta_R$ for each ν , that is, $M(\delta_L, \delta_C, \delta_R)^\top = (\delta_u, \delta_{u'}, \delta_{u''})^\top$. If the given representations are well formed the right-hand side is zero, and M invertible forces $\delta_L = \delta_C = \delta_R = 0$; in particular $w_C = \langle \mathbf{x}_C, \mathbf{b} \rangle$. $\square$

Theorem 3.5 (Knowledge soundness of the opening protocol). *Let $\mathbf{G}$ be a group of prime order in which the discrete logarithm problem is hard, and let $\mathbf{G}$ and H be uniform in $\mathbf{G} \setminus \{\mathcal{O}\}$ as in Construction 3.2 (for hashed generators: hash-to-curve modelled as a random oracle into $\mathbf{G}$). The opening protocol of Construction 3.2 is knowledge sound for $\mathcal{R}_{\text{IPA}}$: there is an extractor which, given rewindable access to any prover that makes the verifier accept with non-negligible probability, runs in expected polynomial time and outputs either a witness $(\mathbf{a}, r)$ with $P = \langle \mathbf{a}, \mathbf{G} \rangle + [r]H$ and $\langle \mathbf{a}, \mathbf{b}(x) \rangle = v$, or a nontrivial discrete-logarithm relation among $\mathbf{G}$, H and the verifier's generators U, U' . Its knowledge error is $O(\log d / |\mathbb{F}|)$.*

Proof. The proof is given at the level of this volume: the single-round step is Lemma 3.4, the tree is an induction on it, and the rewinding that supplies the sibling transcripts is the *Crypto Guide*'s (§“The Schnorr identification protocol”, the remark on extraction by rewinding, and §“Sigma-protocols”, the tree-extraction theorem cited there, which also gives the expected running time). The trees it assembles have children with pairwise distinct challenges (the *Crypto Guide*'s Definition “Tree special soundness”), while Lemma 3.4 needs pairwise distinct squares. A square has at most two square roots, so any five pairwise distinct challenges contain three with pairwise distinct squares: the rewinding assembles five children per node, and the extractor uses three of them. From a tree the extractor computes a witness or, at the root, a discrete-logarithm relation; that definition asks for a witness alone, and the difference is where the assumption enters.

Leaves. Rewind the prover to the moment after it sent (L_1, R_1) and obtain three accepting final messages (a, r^*) for three challenges u_1 with pairwise distinct squares. An accepting final message is, by (15), a representation of $C^{(0)}$ over $(G^{(0)}, H, U)$, namely $(a; r^*; a b^{(0)})$, and it is well formed at level 0: its U -coordinate is the inner product of a with the length-one vector $b^{(0)}$. Lift it to level 1 by (20): $\mathbf{x} = (u_1^{-1} a, u_1 a)$.

One node. The three lifted leaves are well-formed representations of the three folded elements $C^{(1)} + [u_1^2]L_1 + [u_1^{-2}]R_1$ over $(\mathbf{G}^{(1)}, H, U)$. Lemma 3.4 at the round-1 node returns a well-formed representation of $C^{(1)}$ (and representations of L_1 and R_1 , which are not needed further).

Induction. Rewind to the moment after (L_2, R_2) , obtain three challenges u_2 with pairwise distinct squares, and under each of them repeat the two steps above. That yields, after lifting to level 2, three well-formed representations of the elements $C^{(2)} + [u_2^2]L_2 + [u_2^{-2}]R_2$, and the lemma at the round-2 node returns a well-formed representation of $C^{(2)}$. Continuing upward, after k levels the extractor holds a well-formed representation $(\mathbf{x}; y; w)$ of $C^{(k)} = P + [v]U$ over $(\mathbf{G}, H, U)$, with $w = \langle \mathbf{x}, \mathbf{b}(x) \rangle$. The tree of transcripts consumed has three children at every node and depth k , hence $3^k = d^{\log_2 3}$ leaves (Figure 9).

The root, where the assumption enters. The root representation says $P = \langle \mathbf{x}, \mathbf{G} \rangle + [y]H + [w - v]U$. If $w = v$ then $(\mathbf{x}, y)$ is a witness: $P = \langle \mathbf{x}, \mathbf{G} \rangle + [y]H$ and $\langle \mathbf{x}, \mathbf{b}(x) \rangle = w = v$. If $w \neq v$, the extractor rewinds once more, to the setup step at which the verifier sampled U , and repeats the whole extraction under an independent sample U' , obtaining $P = \langle \mathbf{x}', \mathbf{G} \rangle + [y']H + [w' - v]U'$.

If $w' = v$ the witness is $(\mathbf{x}', y')$. Otherwise subtracting the two expressions for P gives

$$[w - v]U - [w' - v]U' + \langle \mathbf{x} - \mathbf{x}', \mathbf{G} \rangle + [\gamma - \gamma']H = \mathcal{O}$$

with the nonzero coefficient $w - v$ on U : a nontrivial relation among the independent uniform generators $\mathbf{G}, H, U, U'$, which the every-generator embedding of the *Crypto Guide* (§“Pedersen vector commitments”, the proof of binding) turns into a discrete logarithm, the reduction embedding its challenge in the public generators and in the verifier’s samples alike. This is the only place the assumption is used.

The error. The figure does not come from a union bound over the tree. At each node the extractor needs three siblings whose challenges avoid $O(1)$ values: $\pm v$ for each earlier sibling’s challenge v , so that the squares stay pairwise distinct. Each of the $k = \log_2 d$ rounds therefore contributes $O(1/|\mathbb{F}|)$, and the rounds together with the resampling of U give $O(\log d/|\mathbb{F}|)$, the theorem’s figure. Collisions among the challenges resampled inside the 3^k -leaf tree cost the extractor a rewind and a fresh sample; they affect its expected running time, not the knowledge error. A naive union bound over the roughly 3^k sibling draws would bound the tree-wide collision probability by $O(d^{\log_2 3}/|\mathbb{F}|)$, still negligible, but that is not the source of the figure. The weaker $O(d/|\mathbb{F}|)$ sometimes quoted for this argument is implied by the proved bound; a larger protocol whose own error is already linear in d loses nothing by quoting it. $\square$

What the theorem does not say. The theorem concerns the interactive protocol; the compiled argument is treated in §5. In the algebraic group model the extractor does not rewind for the representations at all: an algebraic prover hands them over with every group element it sends, and the tree collapses to a single transcript (§5, “The algebraic group model”). After the Fiat–Shamir transform has replaced the verifier’s challenges by hashes, the rewinding above must be re-proved against a prover who can query the hash adaptively and resume from any earlier transcript state (for one round, the *Crypto Guide*’s forking lemma, §“Security in the random oracle model and the forking lemma”; the multi-round case in §5), and its cost cannot be obtained by multiplying every node of the tree by an independent $1/\varepsilon$ factor. Nor does the theorem restate the binding of the commitment itself: that a single P cannot be opened as two vectors is the *Crypto Guide*’s theorem, cited in §2.9 and not reproved here.

The assumption, and what breaks it. The extractor’s alternative output is a discrete-logarithm relation among $\mathbf{G}, H$ and the verifier’s U, U' , so the argument is binding as long as discrete logarithms are hard in $\mathbb{G}$ and the generators are uniform; hashed generators are uniform when hash-to-curve is modelled as a random oracle into $\mathbb{G}$ (*Crypto Guide*, §“Hashing to a curve point”, Remark “Why NUMS generators are binding-safe”), a hypothesis of Theorem 3.5. For the deployed instance the group is the Vesta curve (§3.8), and the assumption that carries the argument is the discrete logarithm on Vesta, the first of the *Crypto Guide*’s standing assumptions (§“Standing assumptions”), against which the generic attacks cost about 2^{126} operations (§“Instantiation on elliptic curves; the Pasta curves”); a quantum adversary running Shor’s algorithm lies outside those assumptions, which fix a classical adversary. Suppose the assumption fails: an algorithm computes discrete logarithms on Vesta. Then it computes the logarithms $\eta_0, \dots, \eta_{d-1}, \eta_H$ of the hashed generators to one base, and every commitment $P = [\pi]$ base opens

to every vector: for any $\tilde{\mathbf{a}}$ whatsoever, the blinder $\tilde{r} := (\pi - \langle \tilde{\mathbf{a}}, \boldsymbol{\eta} \rangle) / \eta_H$ satisfies $\langle \tilde{\mathbf{a}}, \mathbf{G} \rangle + [\tilde{r}]H = P$, and the honest protocol run on $(\tilde{\mathbf{a}}, \tilde{r})$ convinces the verifier of $\langle \tilde{\mathbf{a}}, \mathbf{b}(x) \rangle$, a value the cheat chose. One commitment then opens to two vectors and to two values at one point; binding and evaluation binding fall together, and with them the premise of Theorem 2.5 that the polynomials were fixed before z was drawn.

Example 3.6 (The extractor run on the toy). The extractor is run against an honest prover for the running example’s vector $\mathbf{a} = (90, 61, 22, 24)$ opened at $z = 20$ with $v = 59$, on the prime-order-97 curve and generators of §3.7, with the blinder $r = 88$, so $P = (51, 31)$ and $C^{(2)} = P + [59]U = (43, 6)$. It fixes (L_2, R_2) , branches over three round-2 challenges $u_2 \in \{18, 81, 21\}$, and under each fixes (L_1, R_1) and branches over three round-1 challenges: $\{85, 19, 84\}$, $\{37, 43, 46\}$ and $\{38, 43, 93\}$, nine leaves in all. At the node $u_2 = 18$ the 3×3 system has determinant 67 and returns the representation $\mathbf{a}^{(1)} = (80, 0)$ with blinder 3 and U -coordinate $40 = \langle \mathbf{a}^{(1)}, \mathbf{b}^{(1)} \rangle$, well formed as the lemma promises; the other two nodes return $(50, 41)$ and $(85, 35)$ likewise. At the root the system in u_2 has determinant 75 and returns $\mathbf{a} = (90, 61, 22, 24)$, $r = 88$ and U -coordinate $59 = v$: the extracted witness is the committed vector, its blinder, and the true value. Figure 9 draws the tree with the node $u_2 = 18$ called out.

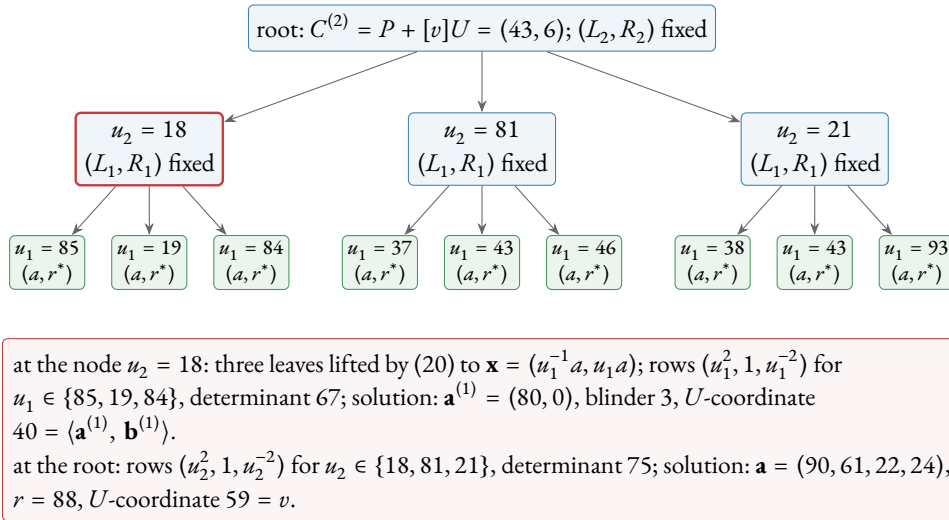


Figure 9. The extractor’s tree of accepting transcripts for $d = 4$ (Example 3.6): every node branches over three challenges, every root-to-leaf path is one accepting run of Construction 3.2, and each node’s 3×3 system of Lemma 3.4 turns its three children’s representations into its own. The red node’s system and the root’s are written out beneath.

3.6 Zero knowledge: what the blinders hide and what they do not

The transcript of one opening consists of group elements, challenges and two final scalars. The group elements are hidden by the blinders: the commitment blinder r makes P uniform in $\mathbf{G}$ whatever $\mathbf{a}$ is (perfect hiding, *Crypto Guide*, §“The Pedersen commitment” and §“Pedersen vector commitments”), and the round blinders ℓ_j, ρ_j make every L_j and R_j uniform and independent of

everything else for the same reason, since each carries its own fresh $[\ell_j]H$ or $[\rho_j]H$. The aggregated blinder r^* is uniform too, being r plus a combination of the fresh ℓ_j and ρ_j . The final scalar is not hidden by these blinders.

Remark 3.7 (Folding alone is not zero knowledge). The final scalar $a = \mathbf{a}^{(0)}$ is a linear form in the committed coefficients whose coefficients the verifier knows. Follow one coefficient a_i through the k folds exactly as the proof of Theorem 3.3 followed G_m : the coefficient side folds with the weights (u_j, u_j^{-1}) , the *inverse* of the generator side's (u_j^{-1}, u_j) , so a_i reaches the single final entry multiplied by $\prod_{\ell} u_{\ell}^{(-1)^{\beta_{\ell-1}}}$, the factor u_{ℓ} when the digit $\beta_{\ell-1}$ of i is 0 and u_{ℓ}^{-1} when it is 1. Writing $\mathbf{t}$ for this vector, $t_i = s_i^{-1}$ entry by entry, and

$$\mathbf{a}^{(0)} = \langle \mathbf{a}, \mathbf{t} \rangle, \quad t_i := \prod_{\ell=1}^k u_{\ell}^{(-1)^{\beta_{\ell-1}}} = s_i^{-1}; \quad \text{at } d = 4 : \mathbf{t} = (u_1 u_2, u_1^{-1} u_2, u_1 u_2^{-1}, u_1^{-1} u_2^{-1}). \quad (22)$$

In the toy of §3.7, with challenges $u_2 = 52$ and $u_1 = 20$, the vector is $\mathbf{t} = (70, 22, 75, 79)$, the entrywise inverse of the structured scalars $\mathbf{s} = (79, 75, 22, 70)$, and the final scalar 33 is

$$\langle (90, 61, 22, 24), (70, 22, 75, 79) \rangle.$$

One opening thus hands the verifier one linear equation in the four secret coefficients; every further opening of the same commitment under fresh challenges hands him another, and d of them determine $\mathbf{a}$ outright. Four openings of the same vector, with challenge pairs $(u_2, u_1) = (63, 29)$, $(79, 7)$, $(34, 1)$ and $(49, 74)$, end in the scalars 88, 22, 40 and 14, and solving the 4×4 system recovers $(90, 61, 22, 24)$. Blinding the commitment and the round points hides those points, not this scalar. The masked opening below therefore does not fold $\mathbf{a}$ itself: it first masks the polynomial by an independently random polynomial vanishing at the opening point, so that the linear form is taken of a vector the verifier cannot relate to $\mathbf{a}$. The mask's commitment S and the challenge that follows it are essential to the zero-knowledge argument below.

The masked opening. The masked opening runs Construction 3.2 on a masked statement.

1. **Mask.** Before any folding, the prover samples a polynomial $s(X) = \sum_{i=0}^{d-1} m_i X^i$ of degree below d with $s(x) = 0$: she draws the coefficient vector $\mathbf{m}$ uniformly from $\mathbb{F}^d$ and then replaces m_0 by $m_0 - s(x)$, which makes $\mathbf{m}$ uniform on the hyperplane $\{\mathbf{m} : \langle \mathbf{m}, \mathbf{b}(x) \rangle = 0\}$. She samples a blinder r_s and sends

$$S := \text{Commit}(s; r_s) = \langle \mathbf{m}, \mathbf{G} \rangle + [r_s]H.$$

2. **Challenge.** The verifier sends a uniform $\xi \in \mathbb{F}$.
3. **Opening.** By linearity of the commitment (*Crypto Guide*, §“Pedersen vector commitments”, the remark that linearity survives),

$$P + [\xi]S = \text{Commit}(p + \xi s; r + \xi r_s), \quad (23)$$

a commitment to the coefficient vector $\mathbf{a} + \xi \mathbf{m}$ of the polynomial $p + \xi s$, whose value at x is $p(x) + \xi s(x) = v$. The parties run Construction 3.2 on the statement $(P + [\xi]S, x, v)$ with the witness $(\mathbf{a} + \xi \mathbf{m}, r + \xi r_s)$, the verifier sampling U in its setup step, after S and ξ .

4. **Final message and check.** The prover sends the final scalar $c := (\mathbf{a} + \xi \mathbf{m})^{(0)}$ and the aggregated blinder $f := r + \xi r_s + \sum_{j=1}^k (u_j^2 \ell_j + u_j^{-2} \rho_j)$. The verifier accepts if and only if

$$P + [\xi]S + [v]U + \sum_{j=1}^k ([u_j^2]L_j + [u_j^{-2}]R_j) \stackrel{?}{=} [c]G^{(0)} + [f]H + [c \cdot b^{(0)}]U, \quad (24)$$

which is (15) for the masked statement.

Completeness is that of Construction 3.2, since the witness satisfies $\mathcal{R}_{\text{IPA}}$ for the masked statement; the mask leaves the opened value unchanged because $s(x) = 0$. The transcript of one opening is

$$(S; L_k, R_k, \dots, L_1, R_1; c, f) \quad \text{with the challenges} \quad (\xi, U, u_k, \dots, u_1).$$

The hiding argument. The final scalar is the linear form (22), taken of the masked vector:

$$c = \langle \mathbf{a} + \xi \mathbf{m}, \mathbf{t} \rangle = \langle \mathbf{a}, \mathbf{t} \rangle + \xi \langle \mathbf{m}, \mathbf{t} \rangle. \quad (25)$$

In the toy, with $\mathbf{m} = (66, 23, 23, 88)$, for which $s(20) = 0$, and $\xi = 29$, the masked vector is $(64, 49, 10, 54)$, its inner product with $\mathbf{b}(20)$ is 59, and under the challenges $u_2 = 52, u_1 = 20$ of the unmasked run the final scalar is $c = 1 = \langle (64, 49, 10, 54), \mathbf{t} \rangle$ with $\mathbf{t} = (70, 22, 75, 79)$; a second mask $\mathbf{m}' = (32, 68, 64, 88)$ under the same challenges ends in $c' = 75$, and both runs are accepted (§3.7). The same form, on a different masked vector, gives an unrelated scalar.

Proposition 3.8 (Honest-verifier zero knowledge of the masked opening). *Let G have prime order p , and let the verifier draw ξ uniformly from $\mathbb{F}$, U uniformly from $G \setminus \{\mathcal{O}\}$ and each u_j uniformly from $\mathbb{F}^\times$. There is a simulator which, given (P, x, v) but not $(\mathbf{a}, r)$, outputs challenges and a transcript whose distribution lies within statistical distance $1/|\mathbb{F}| + (2/(|\mathbb{F}| - 1))^k$ (Math Guide, §“Statistical distance”) of the honest one. The masked opening is therefore statistical honest-verifier zero knowledge, not perfect, in the sense of the Crypto Guide (§“The simulation paradigm and zero knowledge”).*

Proof. Call a challenge tuple *good* if $\xi \neq 0$ and $\mathbf{t}$ is not a multiple of $\mathbf{b}(x)$. The vector $\mathbf{m}$ is uniform on the hyperplane $\langle \mathbf{m}, \mathbf{b}(x) \rangle = 0$, a subspace of dimension $d - 1$. The linear functional $\mathbf{m} \mapsto \langle \mathbf{m}, \mathbf{t} \rangle$ (Math Guide, §“Bilinear forms and the inner product on F^n ”) restricted to that subspace is nonzero unless $\mathbf{t}$ is a scalar multiple of $\mathbf{b}(x)$. If $\mathbf{t} = \lambda \mathbf{b}(x)$, the first entries force $\lambda = t_0 = \prod_{\ell} u_{\ell}$, and the entry at $i = 2^{\ell-1}$, whose only nonzero digit is $\beta_{\ell-1}$, reads $t_0 u_{\ell}^{-2} = t_0 x^{2^{\ell-1}}$, that is, $u_{\ell}^{-2} = x^{2^{\ell-1}}$; each of these k equations has at most two solutions u_{ℓ} , so at most 2^k challenge tuples among $(|\mathbb{F}| - 1)^k$ are excluded. A nonzero linear functional on a vector space is onto $\mathbb{F}$, and its fibres are the cosets of its kernel, all of equal size, so it carries the uniform distribution to the uniform distribution on $\mathbb{F}$. For good challenges $\xi \langle \mathbf{m}, \mathbf{t} \rangle$ is therefore uniform, and by (25) so is c , independently of $\mathbf{a}$.

In an honest run with good challenges, S and every L_j, R_j are uniform and independent (fresh blinders), c is uniform and independent of them (the mask), and f is the unique scalar that makes (24) hold once everything else is fixed, H generating G . The simulator produces the same distribution in the reverse order, the way the Schnorr simulator chooses its commitment after the challenge (Crypto Guide, §“Sigma-protocols”): it draws the challenges as the verifier does, draws S ,

all L_j and all R_j but R_1 uniformly from $\mathbb{G}$, draws c and f uniformly from $\mathbb{F}$, and sets R_1 to the one point that satisfies (24), namely $R_1 := [u_1^2](\text{right-hand side} - \text{every other term of the left})$. Both distributions are uniform on the solution set of the one equation: the honest run parametrises that set by the $2k + 1$ points and c , with f determined; the simulator parametrises it by $2k$ points and the two scalars, with R_1 determined; and each parametrisation is a bijection onto that set, with p^{2k+2} points on each side since $\mathbb{G}$ and $\mathbb{F}$ both have p elements, because the determined coordinate is unique in each case. For good challenges the simulated transcript is therefore distributed exactly as the real one. The challenges have the same distribution in both, and the tuples that are not good have probability at most $1/|\mathbb{F}| + 2^k/(|\mathbb{F}| - 1)^k$; the statistical distance is at most that probability. $\square$

The gap, stated. This argues one opening in isolation. The proof system opens many committed polynomials at once, under shared challenges, and the mask is one of several blinding devices it uses; whether the joint transcript reveals nothing is a separate question, addressed under “Zero knowledge: hiding the witness in Halo 2” in §4, where the mask, in the deployed form of §3.8, reappears as the last of those devices.

3.7 A complete toy opening, carried by script

The toy opening commits the running example’s advice polynomial, $a(X) = 90 + 61X + 22X^2 + 24X^3$ of §2.2, and opens it at the random point $z = 20$ of §2.4 with $v = a(20) = 59$: the opening the proof system needs, of a column polynomial at the verifier’s random point, never of the statement polynomial at the running example’s secret witness 3. Every value below is computed by script and can be recomputed by hand in $\mathbb{F}_{97}$ and on a curve with 97 points.

A genuine group of order 97. The vector $\mathbf{a}$ lives in $\mathbb{F}_{97}^4$, so the commitment group must have order 97: scalars of a prime-order group live in the field of its order (*Math Guide*, §“Base fields, scalar fields, and the Pasta cycle”), exactly as the deployed circuit’s field is the scalar field of the deployed commitment curve. The group $(\mathbb{F}_{97}, +)$ has order 97 but trivially computable discrete logarithms, a division, so the toy uses an elliptic curve. For a curve over a prime field $\mathbb{F}_q$ to have exactly 97 points, the Hasse bound (*Math Guide*, §“The group structure of $E(\mathbb{F}_p)$ ”) confines the prime q to $|q + 1 - 97| \leq 2\sqrt{q}$: $q \in \{79, 83, 89, 97, 101, 103, 107, 109, 113\}$. The case $q = 97$ is excluded on purpose: a curve with as many points as its base field has elements is *anomalous*, and its discrete logarithm is easy (*Math Guide*, §“The elliptic-curve discrete logarithm problem”). The first short Weierstrass curve (*Math Guide*, §“Weierstrass equations”) in lexicographic order of (q, A, B) with exactly 97 points is

$$E: y^2 = x^3 + 3 \quad \text{over } \mathbb{F}_{97}, \quad \#E(\mathbb{F}_{97}) = 97,$$

counted point by point; 97 is prime, so the group is cyclic and every point but $\mathcal{O}$ generates it, and its trace $79 + 1 - 97 = -17$ is not 1. It is the same kind of object as the *Math Guide*’s worked curve $y^2 = x^3 + 5$ over $\mathbb{F}_{11}$ (§“A worked toy example”), one size up. With 97 points every discrete logarithm on it is a short search; the toy exhibits the algebra of the argument, never its hardness.

Generators from a hash. The generators are derived from public labels by hashing to the curve, as the deployed ones are: the label and a counter are hashed to a candidate x -coordinate, and the counter is incremented until $x^3 + 3$ is a square in $\mathbb{F}_{79}$. This is a variant of the try-and-increment loop of the *Crypto Guide* (§“Hashing to a curve point”), which increments x itself; its constant-time refinement is irrelevant to a toy. Indexed from 0 so that the coefficient a_i multiplies G_i , the generators are

$$\begin{aligned} G_0 &= (12, 25), & G_1 &= (17, 27), & G_2 &= (71, 26), \\ G_3 &= (49, 10), & H &= (78, 9), & U &= (53, 26). \end{aligned}$$

The hashed point U stands in for the verifier’s uniform sample of Construction 3.2. No party chose any of them, so no relation among them is known in advance; a single discrete logarithm between two of them would give one, since $H = [b]G_0$ yields $[b]G_0 - H = \mathcal{O}$. On this curve such a logarithm is a short search; on Vesta its infeasibility is the assumption of §3.5.

The instance. The prover holds $\mathbf{a} = (90, 61, 22, 24)$, which takes the values $(3, 9, 27, 30)$ on the domain of §2.2, and the blinder $r = 10$; her commitment is

$$P = [90]G_0 + [61]G_1 + [22]G_2 + [24]G_3 + [10]H = (39, 25).$$

The verifier holds P , the point $z = 20$, hence $\mathbf{b}(20) = (1, 20, 20^2, 20^3) = (1, 20, 12, 46)$ in $\mathbb{F}_{97}$, and the claimed value $v = 59 = \langle \mathbf{a}, \mathbf{b}(20) \rangle$. With $d = 4$ there are $k = 2$ rounds, numbered 2 then 1.

The symmetric opening, both sides. Tables 4 and 5 refill the rows of Tables 2 and 3 with numbers, listing each party’s inputs and computations; the verifier’s table never uses $\mathbf{a}$.

The verifier’s equation (15) closes: the left-hand side $P + [59]U + [52^2]L_2 + [28^2]R_2 + [20^2]L_1 + [34^2]R_1$ is $(54, 39)$, and so is the right-hand side $[33]G^{(0)} + [40]H + [33 \cdot 19]U$. Each fold identity holds along the way: after round 2, $(72, 23) = \langle \mathbf{a}^{(1)}, \mathbf{G}^{(1)} \rangle + [95]H + [\langle \mathbf{a}^{(1)}, \mathbf{b}^{(1)} \rangle]U$, and after round 1, $(54, 39) = [33](12, 25) + [40]H + [33 \cdot 19]U$. The closed forms of Theorem 3.3 agree with the folds: $\langle \mathbf{s}, \mathbf{G} \rangle = (12, 25)$ is the $G^{(0)}$ the prover reached by folding, and $g(20; u_1, u_2) = (u_1^{-1} + u_1 z)(u_2^{-1} + u_2 z^2) = 19$ is her $b^{(0)}$. In a group of 97 elements collisions among the printed points are frequent, and nothing follows from any of them. Here $G^{(0)}$ happens to equal G_0 and R_1 happens to equal P ; the masked run below and the runs of §3.8 show more.

A false value is rejected. Let the prover claim $v' = 60$ instead of 59. The prover’s messages are those of the honest run; only $C^{(2)} = P + [60]U$ changes, by one more U . The left-hand side becomes $(62, 15)$ while the right-hand side stays $(54, 39)$, and the verifier rejects. By Theorem 3.5, a prover accepted on $v' = 60$ would yield an opening of P with value 60 and hence, beside the honest opening, a discrete-logarithm relation among the generators; on a curve of 97 points such relations are a short search, and the theorem gives no security there.

Table 4. The prover's side of the toy opening (Table 2 with numbers): $\mathbf{a} = (90, 61, 22, 24)$ opened at $z = 20$ on the 97-point curve $y^2 = x^3 + 3$ over $\mathbb{F}_{79}$, all scalars in $\mathbb{F}_{79}$. Points are given as (x, y) with coordinates in $\mathbb{F}_{79}$.

step	receives / holds	computes and sends
setup	holds $\mathbf{a} = (90, 61, 22, 24)$, $r = 10$, $z = 20$, $v = 59$; receives $U = (53, 26)$	$\mathbf{a}^{(2)} := \mathbf{a}$, $\mathbf{G}^{(2)} := (G_0, G_1, G_2, G_3)$, $\mathbf{b}^{(2)} := (1, 20, 12, 46)$, $r^{(2)} := 10$
round 2	halves $\mathbf{a}_{\text{lo}} = (90, 61)$, $\mathbf{a}_{\text{hi}} = (22, 24)$; $\mathbf{b}_{\text{lo}} = (1, 20)$, $\mathbf{b}_{\text{hi}} = (12, 46)$ receives $u_2 = 52$ ($u_2^{-1} = 28$)	samples $\ell_2 = 44$, $\rho_2 = 16$; $\langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle = 6$, $\langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle = 17$; sends $L_2 = [90]G_2 + [61]G_3 + [44]H + [6]U = (74, 6)$, $R_2 = [22]G_0 + [24]G_1 + [16]H + [17]U = (38, 72)$ $\mathbf{a}^{(1)} = 52(90, 61) + 28(22, 24) = (58, 61)$; $\mathbf{b}^{(1)} = 28(1, 20) + 52(12, 46) = (70, 42)$; $\mathbf{G}^{(1)} = ([28]G_0 + [52]G_2, [28]G_1 + [52]G_3) =$ $((54, 40), (30, 71))$; $r^{(1)} = 10 + 52^2 \cdot 44 + 28^2 \cdot 16 = 95$
round 1	halves $a_{\text{lo}} = 58$, $a_{\text{hi}} = 61$; $b_{\text{lo}} = 70$, $b_{\text{hi}} = 42$ receives $u_1 = 20$ ($u_1^{-1} = 34$)	samples $\ell_1 = 12$, $\rho_1 = 37$; $a_{\text{lo}}b_{\text{hi}} = 11$, $a_{\text{hi}}b_{\text{lo}} = 2$; sends $L_1 = [58]G_1^{(1)} + [12]H + [11]U = (28, 25)$, $R_1 = [61]G_0^{(1)} + [37]H + [2]U = (39, 25)$ $a^{(0)} = 20 \cdot 58 + 34 \cdot 61 = 33$; $b^{(0)} = 34 \cdot 70 + 20 \cdot 42 = 19$; $G^{(0)} = [34]G_0^{(1)} + [20]G_1^{(1)} = (12, 25)$; $r^{(0)} = 95 + 20^2 \cdot 12 + 34^2 \cdot 37 = 40$
final	$a = 33$, $r^* = 40$	sends $a = 33$, $r^* = 40$

Table 5. The verifier's side of the same opening (Table 3 with numbers). He never sees $\mathbf{a}$, r , ℓ_j or ρ_j ; the structured scalars and $g(20)$ come from the two challenges alone, and $G^{(0)}$ is his one length-4 multi-scalar multiplication.

step	receives / holds	computes and sends
setup	holds $P = (39, 25)$, $z = 20$, $v = 59$, $\mathbf{G}$, $H = (78, 9)$	sends $U = (53, 26)$, a hashed point standing in for his sample; $C^{(2)} := P + [59]U = (47, 39)$
round 2	receives $L_2 = (74, 6)$, $R_2 = (38, 72)$	samples $u_2 = 52$ and sends it; keeps $(L_2, R_2, 52)$; $C^{(1)} := C^{(2)} + [52^2]L_2 + [28^2]R_2 = (72, 23)$
round 1	receives $L_1 = (28, 25)$, $R_1 = (39, 25)$	samples $u_1 = 20$ and sends it; keeps $(L_1, R_1, 20)$; $C^{(0)} := C^{(1)} + [20^2]L_1 + [34^2]R_1 = (54, 39)$
final	receives $a = 33$, $r^* = 40$	$\mathbf{s} = (34 \cdot 28, 20 \cdot 28, 34 \cdot 52, 20 \cdot 52) = (79, 75, 22, 70)$; $G^{(0)} = [79]G_0 + [75]G_1 + [22]G_2 + [70]G_3 = (12, 25)$; $b^{(0)} = g(20; 20, 52) = (34 + 20 \cdot 20)(28 + 52 \cdot 12) = 19$; right-hand side $[33]G^{(0)} + [40]H + [33 \cdot 19]U = (54, 39) = C^{(0)}$: accept

The masked opening of the same polynomial. The same P is then opened at the same z with the masked opening of §3.6. The prover samples $\mathbf{m} = (66, 23, 23, 88)$, for which $s(20) = 0$, and $r_s = 95$, and sends $S = \langle \mathbf{m}, \mathbf{G} \rangle + [95]H = (70, 71)$; the verifier answers $\xi = 29$. The masked vector $\mathbf{a} + 29\mathbf{m} = (64, 49, 10, 54)$ has inner product 59 with $\mathbf{b}(20)$, and the rounds run on it with the point U , the challenges $u_2 = 52, u_1 = 20$ and the round blinders of the unmasked run: the prover sends $L_2 = (36, 7), R_2 = (12, 25), L_1 = (51, 31)$ and $R_1 = (72, 23)$, and ends in $c = 1$ and $f = 79$. The final generator $G^{(0)} = (12, 25)$ and the final evaluation scalar $b^{(0)} = 19$ are those of the unmasked run, since they depend on the challenges alone. The verifier’s equation (24) closes at $(46, 13)$ on both sides, and the false value $v' = 60$ moves the left-hand side to $(56, 70)$: rejected. A second mask $\mathbf{m}' = (32, 68, 64, 88)$ under the same challenges ends in $c' = 75$ and is accepted as well.

3.8 Halo 2’s specifics: parameters, generators from a hash, the deferred multi-scalar multiplication, and the proof bytes

Everything above holds for any prime-order group and any $d = 2^k$. This subsection, and only this one, fixes the group, the size, the generators and the normalisation of the deployed Halo 2, states its opening, and counts the bytes an Orchard proof spends on the argument. Nothing earlier in the section depends on it.

The group and the size. The Orchard Action circuit does its arithmetic in $\mathbb{F}_{p_{\text{Pallas}}}$, the base field of the Pallas curve, because the statement manipulates Pallas points and their coordinates live there (§1.2; *Math Guide*, §“Base fields, scalar fields, and the Pasta cycle”). Its column polynomials therefore have coefficients in $\mathbb{F}_{p_{\text{Pallas}}}$, and a Pedersen commitment to such a vector needs a group whose scalar field is $\mathbb{F}_{p_{\text{Pallas}}}$: that group is the Vesta curve, whose order is p_{Pallas} (protocol specification, §5.4.9.6, “Pallas and Vesta”). The specification uses Halo 2 with the Vesta curve to prove and verify the Action statement (§4.1.13, “Zero-Knowledge Proving System”), so every P, L_j, R_j, S and $G^{(0)}$ of this section is a Vesta point and every scalar a Pallas base-field element. The other half of the Pasta cycle, Pallas’s scalar field being Vesta’s base field, is needed only by the optional native recursion of §7; Orchard’s verifier never uses it. The committed polynomials have degree below the row count $n = 2^{11} = 2048$ of the Action circuit (Definition 2.7; the count is given under “From statement to circuit: arithmetisation in practice” in §4), so here $d = n = 2^{11}$ and the argument runs $k = 11$ rounds.

Generators from a hash. The 2048 vector generators, the blinding generator H and the inner-product generator U are all outputs of hash-to-curve on one fixed domain-separation label, the vector generators indexed by their position and H and U by two further tags (*Crypto Guide*, §“Hashing to a curve point”, Construction “Nothing-up-my-sleeve generators”). With hash-to-curve modelled as a random oracle into $\mathbb{G}$ they are independent and uniform, and a relation among them yields a discrete logarithm on Vesta (the same section, Remark “Why NUMS generators are binding-safe”). No secret is generated in producing them, unlike the trapdoor of KZG (*Crypto Guide*, §“Polynomial commitment schemes”, under “KZG: evaluation in the exponent”), the point the closing subsubsection returns to. Because U is a fixed parameter

rather than a verifier's sample drawn after each P , the deployed opening re-randomises it per opening with a challenge ζ ("The deployed opening in full" below).

Challenges from the transcript. The interactive verifier of Construction 3.2 draws u_j uniformly from $\mathbb{F}^\times$, and the halo2 book's *Protocol Description* excludes zero challenges. The deployed verifier is a hash ("Removing the verifier: the Fiat–Shamir transform" in §4), and the transcript of the halo2_proofs implementation squeezes every challenge, ξ , ζ and each u_j , as a full field element with no rejection of zero. A zero u_j would make the fold's inversion fail, so the honest prover succeeds unless one of the k challenges is zero, an event of probability $1/|\mathbb{F}|$ per challenge, about 2^{-254} : completeness of the deployed argument is overwhelming, not literally perfect, in the sense fixed in §1.2.

Remark 3.9 (The deployed IPA normalisation). The deployed opening, stated in full below, is the masked opening of §3.6 with three changes. It folds with the weights $(1, u_j^{-1})$ on the coefficient vector and $(1, u_j)$ on $\mathbf{b}$ and $\mathbf{G}$, with cross terms $L_j = \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle + \dots$ and $R_j = \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + \dots$, the halves swapped relative to (13), and the verifier's accumulation $[u_j^{-1}]L_j + [u_j]R_j$; it scales the cross terms' U -coordinates by a further challenge ζ drawn after ξ ; and it moves the claimed value into the commitment as $-[v]G_0$ in place of $+ [v]U$. Its final scalars are c and f , and its check is (29). The halo2 book, which the protocol specification (§ 5.4.10.3, "Halo 2") cites for the proving system, states this opening in its chapter *Protocol Description*. Its blinding generator $\mathcal{W}$ is this volume's H , its U is U , and its second challenge z is ζ , renamed because $z = 20$ is the running example's opening point; the book's comparison with the scheme of Bünz, Chiesa, Mishra and Spooner (chapter *Comparison to other work*) writes H , U and z for the same three objects. The chapter numbers the round challenges from the first round, $u_0, \dots, u_{k-1}$, where this volume's descending index gives $u_k, \dots, u_1$, and states the verifier's evaluation polynomial (its κ) as

$$\begin{aligned} g_{\text{dep}}(X) &= \prod_{i=0}^{k-1} (1 + u_{k-1-i} X^{2^i}) \quad \text{in its numbering,} \\ g_{\text{dep}}(X) &= \prod_{\ell=1}^k (1 + u_\ell X^{2^{\ell-1}}) \quad \text{in this volume's,} \end{aligned} \tag{26}$$

with structured scalars $s_i = \prod_{\ell: \beta_{\ell-1}=1} u_\ell$, beside the symmetric $g(X) = \prod_{\ell=1}^k (u_\ell^{-1} + u_\ell X^{2^{\ell-1}})$ of (19) with $s_i = \prod_{\ell} u_\ell^{(-1)^{1-\beta_{\ell-1}}}$.

The change of variables. The two folds are one argument. Run both on the same vector, the same generators and the same evaluation vector, with the deployed challenge of every round the square of the symmetric one, $u_{\text{dep}} := u_{\text{sym}}^2$. In one round, writing u for u_{sym} ,

$$\begin{aligned} \mathbf{a}'_{\text{sym}} &= u \mathbf{a}_{\text{lo}} + u^{-1} \mathbf{a}_{\text{hi}} = u (\mathbf{a}_{\text{lo}} + u^{-2} \mathbf{a}_{\text{hi}}) = u \mathbf{a}'_{\text{dep}}, \\ \mathbf{G}'_{\text{sym}} &= u^{-1} \mathbf{G}_{\text{lo}} + u \mathbf{G}_{\text{hi}} = u^{-1} (\mathbf{G}_{\text{lo}} + u^2 \mathbf{G}_{\text{hi}}) = u^{-1} \mathbf{G}'_{\text{dep}}, \\ \mathbf{b}'_{\text{sym}} &= u^{-1} \mathbf{b}_{\text{lo}} + u \mathbf{b}_{\text{hi}} = u^{-1} (\mathbf{b}_{\text{lo}} + u^2 \mathbf{b}_{\text{hi}}) = u^{-1} \mathbf{b}'_{\text{dep}}, \end{aligned}$$

so $\mathbf{a}'_{\text{dep}} = u^{-1} \mathbf{a}'_{\text{sym}}$, $\mathbf{G}'_{\text{dep}} = u \mathbf{G}'_{\text{sym}}$ and $\mathbf{b}'_{\text{dep}} = u \mathbf{b}'_{\text{sym}}$, and the scalings cancel in every inner product: $\langle \mathbf{a}'_{\text{dep}}, \mathbf{G}'_{\text{dep}} \rangle = \langle \mathbf{a}'_{\text{sym}}, \mathbf{G}'_{\text{sym}} \rangle$ and $\langle \mathbf{a}'_{\text{dep}}, \mathbf{b}'_{\text{dep}} \rangle = \langle \mathbf{a}'_{\text{sym}}, \mathbf{b}'_{\text{sym}} \rangle$. The cross terms agree, up to their

H -blindness, once the naming is unwound: the deployed L carries $\langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle$, which the symmetric presentation calls R , and its weight $u_{\text{dep}}^{-1} = u^{-2}$ is the symmetric weight on R ; likewise for the other cross term. The single-round form holds for one round on identical inputs; over several rounds the change of variables is *cumulative*, because each round's inputs already carry the previous rounds' factors:

$$\mathbf{a}_{\text{dep}}^{(j)} = \left(\prod_{\ell > j} u_{\ell} \right)^{-1} \mathbf{a}_{\text{sym}}^{(j)}, \quad \mathbf{G}_{\text{dep}}^{(j)} = \left(\prod_{\ell > j} u_{\ell} \right) \mathbf{G}_{\text{sym}}^{(j)}, \quad \mathbf{b}_{\text{dep}}^{(j)} = \left(\prod_{\ell > j} u_{\ell} \right) \mathbf{b}_{\text{sym}}^{(j)}, \quad (27)$$

with u_{ℓ} the symmetric challenges of the rounds already run. Each deployed factor of (26) is a symmetric factor scaled, $1 + u^2 X^m = u (u^{-1} + u X^m)$, so after k rounds

$$g_{\text{dep}} = \left(\prod_{\ell=1}^k u_{\ell} \right) g_{\text{sym}}, \quad \mathbf{s}_{\text{dep}} = \left(\prod_{\ell=1}^k u_{\ell} \right) \mathbf{s}_{\text{sym}}, \quad a_{\text{dep}}^{(0)} = \left(\prod_{\ell=1}^k u_{\ell} \right)^{-1} a_{\text{sym}}^{(0)},$$

while $G^{(0)}$ and $b^{(0)}$ carry the product itself, so that $[a^{(0)}]G^{(0)}$ and $a^{(0)} b^{(0)}$ agree in both normalisations, and the two verifier equations coincide once each run attaches ℓ_j and ρ_j to the same cross product. Table 6 prints both runs on the toy's data and keeps the blinder names instead, so the deployed run attaches each blinder to the other cross product, and its equation differs from the symmetric one only in the H -term ($r^* = 33$ against 40); with the blinders exchanged, the deployed run also closes at (54, 39) with $r^* = 40$. What the substitution does *not* do is identify the challenge distributions: a uniform deployed challenge is not the square of a uniform symmetric one ("Extraction in the deployed rows" below), so the deployed rows are analysed with their own lemma, and the deployed opening is knowledge sound by the argument of §3.5 with Lemma 3.10 at every node and two branchings, over ξ and ζ , in place of the resampled U at the root: Corollary 3.13.

Table 6. The two folds on the same $\mathbf{a} = (90, 61, 22, 24)$, the same generators and the same $\mathbf{b}(20)$, with $u_{\text{dep}} = u_{\text{sym}}^2$ in every round and the same round blinders. The last column is the cumulative factor of (27): after round 2 it is 52, after round 1 it is $52 \cdot 20 = 70$ in $\mathbb{F}_{97}$. The cross terms differ as points because the deployed naming attaches each blinder to the other cross product; every folded quantity differs by the stated factor only.

	symmetric fold	deployed fold	relation
round 2: challenge	$u_2 = 52$	$u_2 = 52^2 = 85$	
cross terms sent	$L_2 = (74, 6), R_2 = (38, 72)$	$L_2 = (78, 70), R_2 = (71, 26)$	halves swapped
$\mathbf{a}^{(1)}$	(58, 61)	(72, 59)	52^{-1} .
$\mathbf{G}^{(1)}$	((54, 40), (30, 71))	((53, 53), (41, 6))	$52 \cdot$
$\mathbf{b}^{(1)}$	(70, 42)	(51, 50)	$52 \cdot$
round 1: challenge	$u_1 = 20$	$u_1 = 20^2 = 12$	
cross terms sent	$L_1 = (28, 25), R_1 = (39, 25)$	$L_1 = (43, 6), R_1 = (70, 71)$	halves swapped
$a^{(0)}$	33	85	$(52 \cdot 20)^{-1} = 70^{-1}$.
$G^{(0)}$	(12, 25)	(34, 53)	$70 \cdot$
$b^{(0)}$	19	69	$70 \cdot$
structured scalars $\mathbf{s}$	(79, 75, 22, 70)	(1, 12, 85, 50)	$70 \cdot$
$g(20)$	$(u_1^{-1} + u_1 z)(u_2^{-1} + u_2 z^2) = 19$	$(1 + u_1 z)(1 + u_2 z^2) = 69$	$70 \cdot$
aggregated blinder r^*	40	33	
verifier's equation	$(54, 39) = (54, 39)$	$(5, 72) = (5, 72)$	same $[a^{(0)}]G^{(0)}, a^{(0)}b^{(0)}$

Extraction in the deployed rows. The deployed fold changes the rows of the extractor’s linear system.

Lemma 3.10 (Single-round recovery in the deployed rows). *Fix one round at level j as in Lemma 3.4, but let the round fold the coefficient vector with the weights $(1, u^{-1})$ and the generator and evaluation vectors with the weights $(1, u)$,*

$$\mathbf{a}' := \mathbf{a}_{\text{lo}} + u^{-1}\mathbf{a}_{\text{hi}}, \quad \mathbf{G}' := \mathbf{G}_{\text{lo}} + u\mathbf{G}_{\text{hi}}, \quad \mathbf{b}' := \mathbf{b}_{\text{lo}} + u\mathbf{b}_{\text{hi}},$$

with the cross terms named with the halves swapped relative to (13),

$$L := \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle + [\ell]H + [\langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle] U, \quad R := \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + [\rho]H + [\langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle] U,$$

and the folded commitment $C' := C + [u^{-1}]L + [u]R$, so that the fold identity reads $C' = \langle \mathbf{a}', \mathbf{G}' \rangle + [r + u^{-1}\ell + u\rho]H + [\langle \mathbf{a}', \mathbf{b}' \rangle]U$ by the same expansion as in §3.2. Then the conclusions of Lemma 3.4 hold with rows $(v^{-1}, 1, v)$ and the identities $\mathbf{x}_v = v^{-1}\mathbf{x}_L + \mathbf{x}_C + v\mathbf{x}_R$ (likewise for y, w), provided only that u, u', u'' are pairwise distinct and nonzero. The deployed cross terms also scale their U -coordinates by a challenge $\zeta \neq 0$ drawn before the rounds (“The deployed opening in full” below), which the lemma covers verbatim with U replaced by $[\zeta]U$.

Proof. The fold identity is the expansion of §3.2 with the weights changed: the diagonal terms of $\langle \mathbf{a}_{\text{lo}} + u^{-1}\mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} + u\mathbf{G}_{\text{hi}} \rangle$ carry the weights $1 \cdot 1$ and $u^{-1}u = 1$, and the cross terms $\langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle$ and $\langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle$ carry u^{-1} and u , which is why they are absorbed as $[u^{-1}]L + [u]R$ with L holding the high-with-low product. The lift (20) becomes $\mathbf{x} = (\mathbf{a}', u\mathbf{a}')$, and preserves well-formedness since $\langle (\mathbf{a}', u\mathbf{a}'), \mathbf{b} \rangle = \langle \mathbf{a}', \mathbf{b}_{\text{lo}} + u\mathbf{b}_{\text{hi}} \rangle$. The matrix now has rows $(v^{-1}, 1, v)$; multiplying the row of v by $v \neq 0$ gives $(1, v, v^2)$, and the same root count, now for a polynomial of degree at most 2 in v itself, shows that the matrix is invertible exactly when the challenges are pairwise distinct. Everything else is as in the proof of Lemma 3.4. $\square$

The two row shapes impose different hypotheses: two challenges $u' = -u$ have equal squares, so a triple containing them makes the symmetric matrix singular, while the deployed matrix remains invertible whenever the three challenges are distinct and nonzero. Over $\mathbb{F}_{97}$ the triple $(3, 94, 5)$, in which $94 = -3$, has squares $(9, 9, 25)$: the symmetric rows are $(9, 1, 54)$, $(9, 1, 54)$ and $(25, 1, 66)$, two of them equal, with determinant 0; the deployed rows are $(65, 1, 3)$, $(32, 1, 94)$ and $(39, 1, 5)$, with determinant $28 \neq 0$. Remark 3.9 shows that the deployed fold is the symmetric fold under the substitution $u_{\text{dep}} = u_{\text{sym}}^2$, but that relabelling does not carry the hypotheses across: the squaring map $\mathbb{F}^\times \rightarrow \mathbb{F}^\times$ is two-to-one onto the squares, so a uniform deployed challenge is not the square of a uniform symmetric one, and the condition “pairwise distinct squares” is a condition on three symmetric challenges that has no counterpart for three uniform deployed ones. Each shape is therefore analysed with its own rows. In the proof of Theorem 3.5 with Lemma 3.10 in place of Lemma 3.4, three pairwise distinct nonzero challenges suffice at each node, and each sibling avoids the earlier siblings’ challenges and 0.

Example 3.11 (The extractor in the deployed rows). On the data of Example 3.6, the extractor with rows $(u^{-1}, 1, u)$ and the fold of Lemma 3.10 branches over its own challenges:

$u_2 \in \{78, 83, 56\}$ at the root, and under them the round-1 triples $\{31, 71, 92\}$, $\{91, 50, 47\}$ and $\{95, 14, 96\}$. The node systems have determinants 70, 41 and 33 and the root system 54, and the extractor recovers the same $\mathbf{a} = (90, 61, 22, 24)$, $r = 88$ and U -coordinate $59 = v$.

The deployed opening in full. The deployed opening is the masked opening of §3.6 with the deployed fold, a further challenge ξ , and the claimed value moved into the commitment. Let $\mathbf{e}_0 := (1, 0, \dots, 0)$, and write $\mathbf{t}$ for the coefficient-side vector and $\mathbf{s}$ for the structured scalars of the deployed fold throughout the rest of this subsection.

1. **Mask.** The prover samples $s(X)$ with $s(x) = 0$ and r_s , and sends $S := \text{Commit}(s; r_s) = \langle \mathbf{m}, \mathbf{G} \rangle + [r_s]H$, as in §3.6.
2. **Two challenges.** The verifier sends a uniform ξ , and then a uniform ζ .
3. **The masked commitment.** Both parties replace P by

$$P' := P + [\xi]S - [v]G_0, \quad (28)$$

which, by linearity of the commitment (*Crypto Guide*, §“Pedersen vector commitments”, the remark that linearity survives), commits with blinder $r + \xi r_s$ to the coefficient vector

$$\mathbf{a}' := \mathbf{a} + \xi \mathbf{m} - v \mathbf{e}_0$$

of the polynomial $p + \xi s - v$, whose value at x is $p(x) + \xi s(x) - v = 0$. The claim “ $p(x) = v$ ” has become the claim “ P' commits to a vector whose inner product with $\mathbf{b}(x)$ is 0”: the value is folded into the commitment, and the opening claims the value 0.

4. **Rounds.** The parties run the k rounds on $(\mathbf{a}', \mathbf{G}, \mathbf{b}(x))$ with the fold of Lemma 3.10, weights $(1, u_j^{-1})$ on the coefficients and $(1, u_j)$ on generators and evaluation vector, and with the U -coordinates of the cross terms scaled by ζ :

$$\begin{aligned} L_j &:= \langle \mathbf{a}'_{\text{hi}}^{(j)}, \mathbf{G}_{\text{lo}}^{(j)} \rangle + [\ell_j]H + [\zeta \langle \mathbf{a}'_{\text{hi}}^{(j)}, \mathbf{b}_{\text{lo}}^{(j)} \rangle] U, \\ R_j &:= \langle \mathbf{a}'_{\text{lo}}^{(j)}, \mathbf{G}_{\text{hi}}^{(j)} \rangle + [\rho_j]H + [\zeta \langle \mathbf{a}'_{\text{lo}}^{(j)}, \mathbf{b}_{\text{hi}}^{(j)} \rangle] U; \end{aligned}$$

the verifier accumulates $[u_j^{-1}]L_j + [u_j]R_j$, and the prover’s blinder accumulates $f := r + \xi r_s + \sum_j (u_j^{-1} \ell_j + u_j \rho_j)$.

5. **Final message and check.** The prover sends the final scalars $c := \mathbf{a}'^{(0)}$ and f . The verifier accepts if and only if

$$P + [\xi]S - [v]G_0 + \sum_{j=1}^k ([u_j^{-1}]L_j + [u_j]R_j) \stackrel{?}{=} [c]G^{(0)} + [c \cdot b^{(0)} \cdot \zeta]U + [f]H, \quad (29)$$

where $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle$ and $b^{(0)} = \langle \mathbf{s}, \mathbf{b}(x) \rangle = g_{\text{dep}}(x)$ for the structured scalars $s_i = \prod_{\ell: \beta_{\ell-1}=1} u_\ell$ of (26).

Completeness is the invariant (16) again, with $\mathbf{a}'$ in place of $\mathbf{a}$, the value 0 in place of v , and every U -coordinate scaled by ζ ; the final U -coordinate is $\zeta \langle \mathbf{a}'^{(0)}, \mathbf{b}^{(0)} \rangle = c b^{(0)} \zeta$. Subtracting $[v]G_0$ moves the claimed value into the commitment, so that the argument opens to 0 and the check carries no term in v on U ; the mask leaves the opened value unchanged in either form because $s(x) = 0$. The halo2 book gives the reason in its comparison with the scheme of Bünz, Chiesa, Mishra and Spooner, which adds the value on the inner-product generator instead (chapter *Comparison to other work*): G_0 is a fixed base, so $[v]G_0$ is the cheaper multiplication inside a recursive verifier. The factor ζ has a reason of its own. The deployed U is a fixed public generator rather than a verifier's sample drawn after P ("Generators from a hash" above): a prover who knew U in advance could plant a U -component in P or S , and drawing ζ after both re-randomises the generator that carries the inner product, so that a planted component would have to have anticipated ζ ; Corollary 3.13 makes this precise.

Hiding in the deployed opening. The final scalar is the linear form of (22) for this fold, taken of the masked vector:

$$c = \langle \mathbf{a} + \xi \mathbf{m} - v \mathbf{e}_0, \mathbf{t} \rangle = \langle \mathbf{a} - v \mathbf{e}_0, \mathbf{t} \rangle + \xi \langle \mathbf{m}, \mathbf{t} \rangle, \quad t_i = \prod_{\ell: \beta_{\ell-1}=1} u_\ell^{-1}, \quad (30)$$

the coefficient side folding with the inverse weights of the generator side as always. Unmasked, the deployed fold leaks this form as the symmetric one does (Remark 3.7): four deployed openings of (90, 61, 22, 24) without the mask, with challenge pairs $(u_2, u_1) = (16, 85), (8, 16), (69, 95)$ and (11, 41), end in the scalars 70, 24, 73 and 63, and solving the 4×4 system recovers the vector. Masked, with $\mathbf{m} = (66, 23, 23, 88)$, $\xi = 29$ and the challenges $u_2 = 85, u_1 = 12$, the masked vector is (5, 49, 10, 54), its inner product with $\mathbf{b}(20)$ is 0, and the final scalar is $c = 20 = \langle (5, 49, 10, 54), \mathbf{t} \rangle$ with $\mathbf{t} = (1, 89, 8, 33)$; a second mask $\mathbf{m}' = (32, 68, 64, 88)$ under the same challenges ends in $c' = 46$, and both runs are accepted. The transcript of one opening is

$$(S; L_k, R_k, \dots, L_1, R_1; c, f) \quad \text{with the challenges} \quad (\xi, \zeta, u_k, \dots, u_1).$$

Corollary 3.12 (Honest-verifier zero knowledge of the deployed opening). *Let $\mathbb{G}$ have prime order p , and let the verifier draw ξ, ζ uniformly from $\mathbb{F}$ and each u_j uniformly from $\mathbb{F}^\times$. There is a simulator which, given (P, x, v) but not $(\mathbf{a}, r)$, outputs challenges and a transcript of the deployed opening whose distribution lies within statistical distance $1/|\mathbb{F}| + (|\mathbb{F}| - 1)^{-k}$ of the honest one.*

Proof. The proof of Proposition 3.8, with the deployed $\mathbf{t}$ and with (29) in place of (24). Call a challenge tuple *good* if $\xi \neq 0$ and $\mathbf{t}$ is not a multiple of $\mathbf{b}(x)$; now $\mathbf{t} = \lambda \mathbf{b}(x)$ forces $\lambda = t_0 = 1$ and then $u_\ell^{-1} = x^{2^{\ell-1}}$ for every ℓ , at most one challenge tuple among $(|\mathbb{F}| - 1)^k$. For good challenges c is uniform and independent of $\mathbf{a}$ by (30) and the argument of that proof. The simulator draws the challenges, S , every L_j and every R_j but R_1 , and c and f uniformly, and sets R_1 to the one point that satisfies (29), namely $R_1 := [\mu_1^{-1}](\text{right-hand side} - \text{every other term of the left})$; the counting of that proof shows that the two distributions agree for good challenges, and the tuples that are not good have probability at most $1/|\mathbb{F}| + (|\mathbb{F}| - 1)^{-k}$. $\square$

The deployed opening of the toy polynomial. The toy's P of §3.7 is opened at $z = 20$ with the deployed opening, using the round challenges $u_2 = 85$ and $u_1 = 12$ of Table 6; Table 7 lists both sides with every intermediate value. The verifier's equation (29) closes at $(5, 72)$ on both sides, with $g(20) = (1 + 12 \cdot 20)(1 + 85 \cdot 20^2) = 69$ and $\mathbf{s} = (1, 12, 85, 12 \cdot 85) = (1, 12, 85, 50)$ in the deployed shape, and the false value $v' = 60$ moves the left-hand side to $(67, 31)$ against $(5, 72)$ on the right: rejected once more. The deployed run and the masked symmetric run of §3.7 open the same commitment to the same value with entirely different numbers; Remark 3.9 relates the two folds. The unmasked deployed run of Table 6 and this masked opening both close at $(5, 72)$, and that run's $R_1 = (70, 71)$ equals the mask commitment S : further collisions of the kind noted in §3.7, accidents of a 97-element group, not consequences of the mask.

Corollary 3.13 (Knowledge soundness of the deployed opening). *Let $\mathbf{G}$, $\mathbf{H}$ and H be as in Theorem 3.5, and let U be a fixed generator, uniform in $\mathbf{G} \setminus \{O\}$ like them. The deployed opening, with the check (29), is knowledge sound for $\mathcal{R}_{\text{IPA}}$: there is an extractor which, given rewindable access to any prover that makes the verifier accept with non-negligible probability, runs in expected polynomial time and outputs either a witness $(\mathbf{a}, r)$ for (P, x, v) or a nontrivial discrete-logarithm relation among $\mathbf{G}$, H and U . Its knowledge error is $O(\log d / |\mathbb{F}|)$.*

Proof. The extractor branches over two values $\xi_1 \neq \xi_2$ after S , under each over two nonzero values $\zeta_1 \neq \zeta_2$, and under each of those over the rounds as in the proof of Theorem 3.5, with three pairwise distinct nonzero challenges per node; the tree has $4 \cdot 3^k$ leaves, and the rewinding and running time are as in that proof.

Rounds. Fix ξ and $\zeta \neq 0$, and write $U_\zeta := [\zeta]U$. With U_ζ in place of U the rounds are those of Lemma 3.10, so the induction in the proof of Theorem 3.5 returns a representation $(\mathbf{x}; y; w)$ of $P' = P + [\xi]S - [v]G_0$ over $(\mathbf{G}, H, U_\zeta)$ with $w = \langle \mathbf{x}, \mathbf{b}(x) \rangle$, that is, the representation $(\mathbf{x}; y; \zeta w)$ over $(\mathbf{G}, H, U)$.

The challenge ζ . Under one ξ , the values ζ_1 and ζ_2 give representations $(\mathbf{x}_1; y_1; \zeta_1 w_1)$ and $(\mathbf{x}_2; y_2; \zeta_2 w_2)$ of the same P' over $(\mathbf{G}, H, U)$. If they differ, their difference is a nontrivial relation among $\mathbf{G}$, H and U . Otherwise $\mathbf{x}_1 = \mathbf{x}_2 =: \mathbf{x}$, so $w_1 = w_2 =: w$ and $(\zeta_1 - \zeta_2)w = 0$, whence $w = 0$: $P' = \langle \mathbf{x}, \mathbf{G} \rangle + [y]H$ with $\langle \mathbf{x}, \mathbf{b}(x) \rangle = 0$.

The challenge ξ . The values ξ_1 and ξ_2 follow the same S and give $P + [\xi_i]S - [v]G_0 = \langle \mathbf{x}_i, \mathbf{G} \rangle + [y_i]H$ with $\langle \mathbf{x}_i, \mathbf{b}(x) \rangle = 0$ for $i = 1, 2$. Subtracting, $S = \langle \mathbf{m}, \mathbf{G} \rangle + [r_s]H$ with $\mathbf{m} := (\mathbf{x}_1 - \mathbf{x}_2) / (\xi_1 - \xi_2)$, $r_s := (y_1 - y_2) / (\xi_1 - \xi_2)$ and $\langle \mathbf{m}, \mathbf{b}(x) \rangle = 0$. Substituting back, $P = \langle \mathbf{a}, \mathbf{G} \rangle + [r]H$ with $\mathbf{a} := \mathbf{x}_1 - \xi_1 \mathbf{m} + v \mathbf{e}_0$ and $r := y_1 - \xi_1 r_s$, and $\langle \mathbf{a}, \mathbf{b}(x) \rangle = 0 - 0 + v \cdot 1 = v$, the first entry of $\mathbf{b}(x)$ being 1. This is the witness.

The error. Beyond the $O(\log d / |\mathbb{F}|)$ of the rounds, the two branchings at the root and the exclusion of $\zeta = 0$ each cost $O(1 / |\mathbb{F}|)$. A relation among $\mathbf{G}$, H and U yields a discrete logarithm by the every-generator embedding, as at the root of Theorem 3.5. $\square$

The bytes. An Orchard proof is a byte string. The proofs of the $a \geq 1$ Action descriptions that one transaction carries in one pool, here called a *bundle*, are aggregated into a single proof (protocol specification, § 4.6, “Action Descriptions”); a transaction with Actions in both the Orchard and the Ironwood pool carries one such proof for each. Its length is $2720 + 2272a$ bytes (ZIP 225,

Table 7. The deployed opening of the same $P = (39, 25)$ at $z = 20$ with the deployed mask: the fold of Lemma 3.10, the cross terms named with the halves swapped, the U -coordinates scaled by ζ , the value 0 opened for $P + [\xi]S - [v]G_0$, and the final scalars c and f . The round challenges are those of Table 6.

step	prover	verifier
mask	samples $\mathbf{m} = (66, 23, 23, 88)$, for which $s(20) = 0$, and $r_s = 95$; sends $S = \langle \mathbf{m}, \mathbf{G} \rangle + [95]H = (70, 71)$	receives S ; sends $\xi = 29$, then $\zeta = 12$
masked vector	$\mathbf{a}' = \mathbf{a} + 29\mathbf{m} - 59\mathbf{e}_0 = (5, 49, 10, 54)$; $\langle \mathbf{a}', \mathbf{b}(20) \rangle = 0$; blinder $f := 10 + 29 \cdot 95$	$P' := P + [29]S - [59]G_0$
round 2	halves $\mathbf{a}'_{\text{lo}} = (5, 49)$, $\mathbf{a}'_{\text{hi}} = (10, 54)$; $\ell_2 = 44$, $\rho_2 = 92$; cross terms $\langle \mathbf{a}'_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle = [10]G_0 + [54]G_1 = (34, 53)$, $\langle \mathbf{a}'_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle = 23$, $\langle \mathbf{a}'_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle = [5]G_2 + [49]G_3 = (54, 39)$, $\langle \mathbf{a}'_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle = 83$; sends $L_2 = (34, 53) + [44]H + [12 \cdot 23]U = (17, 52)$, $R_2 = (54, 39) + [92]H + [12 \cdot 83]U = (47, 40)$ folds $\mathbf{a}^{(1)} = (5, 49) + 8(10, 54) = (85, 93)$, $\mathbf{b}^{(1)} = (1, 20) + 85(12, 46) = (51, 50)$, $\mathbf{G}^{(1)} = (G_0 + [85]G_2, G_1 + [85]G_3)$ $= ((53, 53), (41, 6))$; $f = 10 + 29 \cdot 95 + 8 \cdot 44 + 85 \cdot 92 = 73$	sends $u_2 = 85$ ($85^{-1} = 8$); keeps $(L_2, R_2, 85)$; $P^{(1)} := P' + [8]L_2 + [85]R_2 = (64, 5)$
round 1	halves $a'_{\text{lo}} = 85$, $a'_{\text{hi}} = 93$; $\ell_1 = 95$, $\rho_1 = 56$; cross terms $[93]G_0^{(1)} = (58, 8)$, $93 \cdot 51 = 87$, $[85]G_1^{(1)} = (24, 9)$, $85 \cdot 50 = 79$; sends $L_1 = (58, 8) + [95]H + [12 \cdot 87]U = (46, 13)$, $R_1 = (24, 9) + [56]H + [12 \cdot 79]U = (24, 70)$ folds $c = a'^{(0)} = 85 + 89 \cdot 93 = 20$, $b^{(0)} = 51 + 12 \cdot 50 = 69$, $G^{(0)} = G_0^{(1)} + [12]G_1^{(1)} = (34, 53)$; $f = 73 + 89 \cdot 95 + 12 \cdot 56 = 82$	sends $u_1 = 12$ ($12^{-1} = 89$); keeps $(L_1, R_1, 12)$; $P^{(0)} := P^{(1)} + [89]L_1 + [12]R_1 = (5, 72)$
final	sends $c = \mathbf{a}'^{(0)} = 20$, $f = 82$	$g(20) = (1 + 12 \cdot 20)(1 + 85 \cdot 20^2) = 69$; $\mathbf{s} = (1, 12, 85, 50)$, $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle = (34, 53)$; left: $P' + [85^{-1}]L_2 + [85]R_2 + [12^{-1}]L_1 + [12]R_1 =$ $(5, 72)$; right: $[20]G^{(0)} + [20 \cdot 69 \cdot 12]U + [82]H = (5, 72)$; accept ; with $v' = 60$: left $(67, 31) \neq (5, 72)$; reject

the `sizeProofsOrchard` field; ZIP 229, the fields `sizeProofsOrchard` and `sizeProofsIronwood`; the consensus rule is the protocol specification’s, § 7.5, “Action Description Encoding and Consensus”), 4992 bytes for a single Action. Every Vesta point and every field element in it occupies 32 bytes. Table 8 lists by kind, from the per-object counts of the Action circuit whose verifying key the consensus rules fix (protocol specification, § 4.6, “Action Descriptions”), every one of those bytes: a per-Action block of 22 points and 49 scalars, repeated for each Action, and a shared block of 33 points and 52 scalars. The inner-product argument’s share is the last three rows of the shared block: the blinding commitment S , the $k = 11$ pairs (L_j, R_j) , which are 22 points, and the two scalars c and f : 23 points, 736 bytes, or 800 bytes with the scalars, once per bundle, because the complete protocol collapses every opening of every polynomial into one opening (“The multipoint opening argument” in §4). This is the only part of the proof that grows with n , and it grows as $2 \log_2 n + 1$ points. Everything else grows with the circuit’s structure and not with n : one commitment per advice column, two permuted-column commitments and one running product per lookup, the permutation running products and quotient chunks, the multipoint commitment, and one field element per claimed evaluation.

3.8.1 The “Halo” in Halo 2: deferring the expensive check. The verifier of this section is logarithmic everywhere but in one place. He reads $2k + 1$ points and two scalars, evaluates g in $3k - 2$ field multiplications, and forms the left-hand side of (29) with $2k + 2$ scalar multiplications ($[\xi]S$, $[v]G_0$ and the $2k$ round terms); then he needs $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle$, one combination of *all* 2048 generators, group arithmetic proportional to the size of the whole circuit (§3.4). Two ways of amortising that cost follow, batch verification and accumulation, the second not deployed by Orchard; between them, the paragraph “Transparency: no trusted setup” records why the linear cost is accepted.

Batch verification. The consensus rules require each Action proof to be valid for the verifying key of the Orchard circuit (protocol specification, § 4.6, “Action Descriptions”); validity includes the check (29) with its multi-scalar multiplication of length d , and the specification composes no proofs, so the verifier pays that multiplication itself rather than passing it on. Several proofs can share it (*Crypto Guide*, § “Polynomial commitment schemes”, Remark “KZG versus IPA, and the Halo lineage”). Written as one multi-scalar multiplication that must evaluate to $\mathcal{O}$, equation (29) has $d + 2k + 4$ terms: the generators $G_0, \dots, G_{d-1}, U, H, P, S$ and the $2k$ points L_j, R_j . The equations $E_1, \dots, E_b$ of b proofs are checked together by testing $\sum_i [\mu_i]E_i = \mathcal{O}$ for fresh uniform $\mu_i \in \mathbb{F}$ drawn by the verifier.

Lemma 3.14 (Randomised batch check). *Let $\mathbb{G}$ have prime order, let $E_1, \dots, E_b \in \mathbb{G}$, and let $\mu_1, \dots, \mu_b \in \mathbb{F}$ be independent and uniform. If some $E_i \neq \mathcal{O}$, then $\Pr[\sum_j [\mu_j]E_j = \mathcal{O}] = 1/|\mathbb{F}|$.*

Proof. Fix i with $E_i \neq \mathcal{O}$ and condition on every μ_j with $j \neq i$. Since $\mathbb{G}$ has prime order, E_i generates it, so $\mu \mapsto [\mu]E_i$ is a bijection $\mathbb{F} \rightarrow \mathbb{G}$, and the sum equals $\mathcal{O}$ for exactly one value of μ_i . $\square$

Because $\mathbf{G}$, U and H are shared, their terms merge, and b equations combine into one multi-scalar multiplication of $d + 2 + b(2k + 2)$ terms, against $b(d + 2k + 4)$ when checked separately.

Table 8. The bytes of an Orchard proof, $2720 + 2272a$ for a Actions (ZIP 225, ZIP 229), laid out as points and scalars of 32 bytes each. The three rows marked **IPA** are this section’s: 23 points and 2 scalars, 800 bytes, shared by the whole bundle. The advice, permutation, lookup and quotient rows are the objects of §2; the random polynomial is constructed under “The complete Halo 2 protocol” and the multipoint polynomial under “The multipoint opening argument”, both in §4.

kind	count	bytes	what it is
<i>Per-Action block, once per Action</i>			
point	10	320	advice column commitments, one per advice column
point	3	96	permuted lookup input commitments A' , one per lookup (Construction 2.14)
point	3	96	permuted lookup table commitments S' , one per lookup (Construction 2.14; unrelated to the mask commitment S)
point	3	96	permutation running-product commitments: the 15 equality-enabled columns in chunks of 7 (§2.6)
point	3	96	lookup running-product commitments, one per lookup
scalar	1	32	instance column evaluation at x
scalar	25	800	$25 = 5 \cdot 2 + 5 \cdot 3$ advice evaluations, one per queried rotation of each advice column (§4.4)
scalar	8	256	permutation product evaluations: 3 products at x and ωx , the first 2 also at $\omega^{-6}x$, the rotation to the boundary row $n - n_{\text{blind}} - 1$ of Remark 2.17, with $n_{\text{blind}} = 5$ blinding rows (§4.6)
scalar	15	480	lookup evaluations: product at x and ωx ; A' at x and $\omega^{-1}x$; S' at x
total		2272	22 points and 49 scalars
<i>Shared block, once per bundle</i>			
point	1	32	random polynomial commitment: a random polynomial added to blind the quotient (the complete protocol, §4)
point	8	256	quotient chunk commitments: constraint degree $d_{\text{max}} = 9$, so $d_{\text{max}} - 1 = 8$ chunks of degree below n (Theorem 4.7)
scalar	29	928	fixed column evaluations at x , one per fixed column
scalar	15	480	permuted-label polynomial evaluations at x , one per equality-enabled column (Construction 2.10)
scalar	1	32	random polynomial evaluation at x
point	1	32	multipoint polynomial commitment: the one polynomial into which every query collapses (Construction 4.4)
scalar	5	160	multipoint evaluations: the value of each query group’s combined polynomial at the reduction’s fresh challenge point, one per distinct set of query points; five sets (§4.4)
point	1	32	IPA : the blinding commitment S
point	22	704	IPA : the $k = 11$ pairs (L_j, R_j)
scalar	2	64	IPA : the final scalars c and f
total		2720	33 points and 52 scalars

In the complete protocol P is itself a combination of commitments (§4); those of the verifying key are shared as well, and each further proof adds its own points, the $22a + 33$ of Table 8, and the instance commitments computed from its public inputs, one per Action (§4.2). The $d = 2048$ generator terms are paid once per batch. A batch check is not accumulation: it emits nothing that a later proof could consume, and it verifies the batch and stops. Both take linear combinations of deferred equations; their interfaces and guarantees differ, and the accumulation scheme of Bünz, Chiesa, Mishra and Spooner, sketched below under “Accumulation (not deployed by Orchard)”, is not part of the specified protocol.

Transparency: no trusted setup. The generators are outputs of a public hash-to-curve map on fixed inputs (this subsection, “Generators from a hash”); the scheme has no trapdoor and requires no trusted setup, unlike KZG, whose trapdoor permits forgeries (*Crypto Guide*, §“Polynomial commitment schemes”, under “KZG: evaluation in the exponent”). The price is the $2k + 1$ points of Table 8, against the constant-size opening proof of KZG, and the linear verifier. ZIP 224’s motivation names the generation of the structured reference string of Zcash’s earlier proving system a point of risk within the protocol and asks for a proving system that does not require one (ZIP 224, Motivation).

Accumulation (not deployed by Orchard). In a recursive verifier, a circuit that verifies a proof, the multi-scalar multiplication $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle = \text{Commit}(g; 0)$ of §3.4 would cost d scalar multiplications inside the circuit. Instead the claim is combined with a running claim of the same shape, $G'^{(0)} = \text{Commit}(g'; 0)$, by a random linear combination with a challenge ρ drawn after both (a verifier challenge, not a round blinder ρ_j of this section; §7 keeps the letter): $G^{(0)} + [\rho] G'^{(0)} = \text{Commit}(g + \rho g'; 0)$ holds for all ρ when both inputs hold and for at most one ρ otherwise. The combined claim, returned to a single tuple by a prover-assisted opening, is the *accumulator* carried along the chain of proofs, and a single multi-scalar multiplication outside any circuit at the end discharges all combined claims. The soundness of an accumulation scheme for a closely related inner-product commitment is cited in §7 from Bünz, Chiesa, Mishra and Spooner, under the extractability of its openings, which they conjecture from the binding of the Pedersen commitment; the cycle of curves that keeps the recursive arithmetic in the right fields is the other half of Pasta. Both are developed in §7, under “Accumulation and the Halo trick” and “A cycle of curves for efficient native recursion”, where recursion and accumulation over the cycle are classified designed-but-unspecified in the sense of §1.2: the protocol specification uses Halo 2 only to prove and verify Action statements and composes no proofs.

4 The Halo 2 proof system

This section defines the Halo 2 proof system as deployed for the Orchard Action statement: the Action circuit, the instance column, the transcript, the multipoint opening, the protocol, and its zero knowledge. It combines the arithmetisation of §2, which reduces a circuit to committed polynomials and identities among them (the gates, the permutation argument for the wiring, the lookup argument for table membership), with the polynomial commitment of §3, which opens a committed polynomial at a point in logarithmically many group elements and is binding un-

der the discrete logarithm on Vesta. The abstractions the *Crypto Guide* owns, the non-interactive argument and the SNARK, the polynomial interactive oracle proof, the Fiat–Shamir transform and the polynomial-commitment interface, are recalled at the point of use. Halo 2 is a *transparent* PLONKish argument: the parameters of its commitment scheme are derived by hashing to the curve and carry no trapdoor (§3.8), and for the Orchard Action circuit its polynomial commitment works over Vesta. The protocol specification defines Action proofs as proofs of the Halo 2 proving system described in the halo2 book (protocol specification, § 5.4.10.3, “Halo 2”); this section cites the book, by chapter, as the specification of the protocol, and names an implementation only for a behaviour that neither document fixes.

The subsections treat, in order, the Action circuit as a PLONKish circuit (its gates, one gate in full, its lookup arguments and its column polynomials); the instance column; the Fiat–Shamir transform and the deployed transcript; the multipoint opening argument; the complete protocol in seven phases; its zero knowledge; and the compiled-polynomial-IOP pattern the protocol instantiates. Recursion is not deployed by Orchard and is treated in §7.

Notation. The constraint degree is $d_{\max}$, as Definition 2.7 fixed it; the letter d is the committed vector length of §3, which equals the row count n throughout this section. The letter H carries three meanings; two come from the sections this one joins: §2 writes H for the evaluation domain and §3 for the blinding generator of the Pedersen commitment. Here the domain appears only through its vanishing polynomial $Z_H(X) = X^n - 1$, its Lagrange basis ℓ_j and its rows ω^j ; a bare H inside a commitment formula is the blinding generator, as in §3. In §4.3 and the security statements the letter H also names the transcript hash, modelled as a random oracle, as in the *Crypto Guide*; there it is applied to arguments, $H(\cdot)$, or called the hash or the oracle. The letter x is the evaluation challenge of phase 5; the running example’s witness is written by its value 3 here and in §6.

4.1 From statement to circuit: arithmetisation in practice

Definition 2.7 says what a PLONKish circuit is; the deployed instance is the Orchard Action circuit. The protocol specification fixes the statement it proves, the Action statement (protocol specification, § 4.18.4, “Action Statement (Orchard)”), and requires every Action proof to verify under the verifying key identified by the current Orchard circuit version (protocol specification, § 4.6, “Action Descriptions”, consensus rules). No specification or ZIP fixes the parameters of the circuit behind that key; they are those of its implementation, orchard. As a PLONKish circuit in the sense of Definition 2.7 it has the field $\mathbb{F}_{p_{\text{Pallas}}}$, $n = 2^{11}$ rows, ten advice columns, one instance column and twenty-nine fixed columns, among them its selector columns and the three columns of its lookup table; the constraint-degree bound $d_{\max} = 9$; and three lookup arguments into that table, of 2^{10} rows. Its custom gates encode the conditions of the Action statement, using among others complete and incomplete addition on Pallas, the distinction the *Math Guide* draws in §“Explicit affine formulas” (its remark on complete versus incomplete addition). This subsection gives the form of the gates, one gate in full, the three lookup arguments, and the column polynomials on which every argument of §2 operates.

Key and witness. The verifying key is a deterministic function of the commitment parameters and the circuit description: the values of the fixed columns, the gates, the lookup arguments and the permutation σ of Definition 2.7. For the Action circuit it is a fixed consensus parameter, the key the consensus rule above identifies. The prover’s witness is the content of the advice columns. A copy constraint places two cells in one cycle of σ (Construction 2.10), and the public inputs are bound to advice cells in this way (§4.2).

4.1.1 Custom gates. Every custom gate of the deployed circuit, a gate in the sense of Definition 2.7, has the form

$$q \cdot c(v_1(\omega^{r_1} X), \dots, v_s(\omega^{r_s} X)) = 0,$$

where $v_1, \dots, v_s$ are columns read at rotations $r_1, \dots, r_s$, c is a polynomial, and the *selector* q is a polynomial in the fixed columns that is zero on every row where the gate does not apply (the halo2 book’s *PLONKish Arithmetization* chapter). The gate is required to vanish at every row; on the rows where q is zero, the blinding rows among them, it holds whatever the cells contain. The universal gate of vanilla PLONK is the special case of §2.5.

4.1.2 The incomplete-addition gate. The incomplete-addition gate of the Action circuit constrains $R = P + Q$ for affine points $P = (x_p, y_p)$ and $Q = (x_q, y_q)$ of Pallas with $x_p \neq x_q$. It has one selector q_{add} and four equality-enabled advice columns x_p, y_p, x_{qr} and y_{qr} ; it reads x_p, y_p, x_q, y_q at the current row and x_r, y_r at the next row of the *same* two columns x_{qr}, y_{qr} , so that Q occupies the selector row and R the row below it. Its two constraints are

$$q_{\text{add}} \cdot ((x_r + x_q + x_p)(x_p - x_q)^2 - (y_p - y_q)^2) = 0, \quad (31)$$

$$q_{\text{add}} \cdot ((y_r + y_q)(x_p - x_q) - (y_p - y_q)(x_q - x_r)) = 0 \quad (32)$$

(the halo2 book’s *Incomplete and complete addition* chapter).

Theorem 4.1 (The gate is the chord formula with the slope eliminated). *Let $P = (x_p, y_p)$ and $Q = (x_q, y_q)$ be affine points of a short Weierstrass curve with $x_p \neq x_q$, and set $\lambda := (y_p - y_q)/(x_p - x_q)$. For a pair $(x_r, y_r) \in \mathbb{F}^2$ the following are equivalent:*

1. *the bracketed polynomials of (31) and (32) both vanish;*
2. *$x_r + x_q + x_p = \lambda^2$ and $y_r + y_q = \lambda(x_q - x_r)$;*
3. *$(x_r, y_r) = P + Q$ by the chord formulas of the Math Guide (§“Explicit affine formulas”), $x_r = \lambda^2 - x_p - x_q$ and $y_r = \lambda(x_p - x_r) - y_p$.*

Proof. (i) $\iff$ (ii). Since $x_p - x_q \neq 0$, dividing the first bracket by $(x_p - x_q)^2$ and the second by $(x_p - x_q)$ are invertible operations: the first bracket vanishes exactly when $x_r + x_q + x_p = (y_p - y_q)^2 / (x_p - x_q)^2 = \lambda^2$, and the second exactly when $y_r + y_q = \lambda(x_q - x_r)$. The gate stores the cleared forms because a constraint must be a polynomial in the cells, and a division is not.

(ii) $\iff$ (iii). The x -equations are the same equation rearranged. For the y -equations, the chord formula uses the slope of the line through P and Q , which is λ whichever endpoint is subtracted

from which, and the identity $\lambda(x_p - x_q) = y_p - y_q$ gives $\lambda x_p - y_p = \lambda x_q - y_q$; hence $\lambda(x_p - x_r) - y_p = \lambda x_q - y_q - \lambda x_r = \lambda(x_q - x_r) - y_q$, which is (ii)'s y -equation with y_q moved across. $\square$

On the 97-point toy curve of §3.7, $y^2 = x^3 + 3$ over $\mathbb{F}_{79}$, the chord formulas give $P + Q = (58, 71)$ for $P = (12, 25)$ and $Q = (71, 53)$, and both brackets are 0 at $(x_r, y_r) = (58, 71)$. On Pallas, with $P = (-1, 2)$ and $Q = [2]P$, both brackets are again 0 at the chord sum.

Degrees. The bracket of (31) has degree three in the cells and that of (32) degree two; with a selector column taking values in $\{0, 1\}$ the constraints have degrees four and three. The deployed key combines selectors into shared fixed columns (the halo2 book's *Selector combining* chapter), which can raise both degrees, keeping them within the deployed bound $d_{\max} = 9$.

Layout. The selector q_{add} is 1 on row i , which holds P and Q , each coordinate copy-constrained to the cell it came from; row $i + 1$ holds R (Table 9). The slope λ occupies no cell: the identities (31)–(32) verify the sum without it, which is what Theorem 4.1 says. By that theorem the two constraints force $(x_r, y_r) = P + Q$, a point of the curve, and $R \neq \mathcal{O}$ because $x_p \neq x_q$ excludes $Q = -P$.

Table 9. The cells of one incomplete addition: two rows of four advice columns and the selector. The four input cells x_p, y_p, x_q, y_q on the selector row i are copy-constrained to the cells they came from; the result cells x_r, y_r occupy the x_{qr}, y_{qr} columns of row $i + 1$, where the gate's next-rotation queries find them. Dashes mark cells the gate does not use. The gate does not read q_{add} on row $i + 1$. When additions are chained, as in fixed-base scalar multiplication, row $i + 1$ is the selector row of the next addition, and R is that addition's Q .

row	x_p	y_p	x_{qr}	y_{qr}	q_{add}
i	x_p	y_p	x_q	y_q	1
$i + 1$	–	–	x_r	y_r	–

The excluded inputs. If $Q = P$, both brackets vanish identically, since every term carries a factor $x_p - x_q$ or $y_p - y_q$, and (x_r, y_r) is unconstrained. If $Q = -P \neq \mathcal{O}$, the bracket of (31) reduces to $-(2y_p)^2$, which is nonzero because Pallas, of prime order, has no point of order two, and no assignment satisfies the gate. The Action circuit applies this gate only to non-identity inputs with $x_p \neq x_q$: it sums the window multiples of fixed-base scalar multiplication for every window but the last, an offset in the window table excluding the identity and the doubling case there, and adds the last window, whose sum can be the identity or a doubling, with the complete-addition gate (the halo2 book's *Fixed-base scalar multiplication* chapter).

4.1.3 Lookup arguments of the Action circuit. A gate asserts only polynomial identities. “This value has ten bits” and “this point is the j -th Sinsemilla generator” are set memberships, and §2.7 showed why no low-degree gate on the cell alone states them; the circuit states them as lookup arguments. Each is one instance of Construction 2.14, the input expressions playing $A_0, \dots, A_{m-1}$ and the table columns $S_0, \dots, S_{m-1}$, compressed by the challenge θ as in (10) whenever $m > 1$.

The table. The Action circuit has three lookup arguments, one range check and two Sinsemilla generator lookups, and all three use one table. It holds, for $0 \leq j < 2^{10}$,

$$\text{row } j : \quad (j, X(S[j]), Y(S[j])), \quad S[j] := \text{GroupHash}(\text{"z.cash:SinsemillaS"}, \text{LE}_{32}(j)),$$

in three table columns $(t_{\text{idX}}, t_x, t_y)$, where $\text{LE}_\ell(v)$ is the ℓ -bit little-endian encoding of the integer v , the generator table S is defined by the protocol specification (§ 5.4.1.9, “Sinsemilla Hash Function”), and the hash GroupHash that derives it is the *Crypto Guide*’s (§ “Sinsemilla: an algebraic hash-based commitment”; the hash-to-curve pipeline is § “Hashing to a curve point”). The first two rows are:

row	coordinates of $S[j]$, hexadecimal
$j = 0$	$X = 0\text{db}5218\text{b } 6\text{e}881\text{f}0\text{f } 1431\text{d}4\text{ea } 7\text{d}4\text{af}7\text{b } 29\text{a}05\text{baf } \text{bede}62\text{b}5 \text{ } 5\text{a}91\text{eb}91 \text{ } 2044\text{ea}5\text{f}$ $Y = 2\text{f}0\text{f}40\text{c}2 \text{ } \text{f}152\text{a}01\text{c } 9\text{caf}6629 \text{ } 8493\text{d}5\text{d}0 \text{ } 944\text{a}041\text{c } 2\text{e}65\text{ba}01 \text{ } 17\text{c}24\text{f}76 \text{ } \text{bf}8\text{e}6483$
$j = 1$	$X = 211112\text{b}4 \text{ } \text{b}3\text{e}19518 \text{ } \text{c}8\text{fd}33\text{eb } 39175404 \text{ } \text{e}67700\text{ca } 24649\text{b}8\text{f } \text{ce}4\text{ae}33\text{e } \text{a}108\text{af}91$ $Y = 06\text{c}59939 \text{ } 93\text{adb}03\text{b } \text{a}38\text{a}3\text{e}79 \text{ } \text{cd}5\text{a}35\text{fe } \text{b}43\text{c}744\text{a } 670\text{e}19\text{bc } 1\text{d}83\text{c}293 \text{ } \text{f}810\text{c}5\text{ee}$

Unused table rows. On every active row $j \geq 2^{10}$ (Remark 2.17), each of t_{idX} , t_x and t_y holds its row-0 value, so that every active row of the table is a genuine entry, $(0, X(S[0]), Y(S[0]))$ or $(j, X(S[j]), Y(S[j]))$. Rows left at 0 would add the tuple $(0, 0, 0)$, which is no entry, since row 0 is the only entry with index 0 and $X(S[0]) \neq 0$, and which an input tuple could then match.

The range check. The range-check lookup is

$$q_{\text{lookup}} \cdot \left(q_{\text{running}} \cdot (z_i - 2^{10} z_{i+1}) + (1 - q_{\text{running}}) \cdot z_i \right) \mapsto t_{\text{idX}}, \quad (33)$$

with z_i and z_{i+1} the running-sum advice column at the current and next rotations. With $q_{\text{running}} = 1$ the input is a running-sum step: the prover witnesses $z_{i+1} = (z_i - a_i) \cdot 2^{-10}$ on successive rows, so the expression recovers the limb $a_i = z_i - 2^{10} z_{i+1}$, and membership in $t_{\text{idX}} = \{0, \dots, 2^{10} - 1\}$ asserts that each limb has ten bits. With $q_{\text{running}} = 0$ the cell itself is looked up. Every range check of the Action circuit is either an instance of this lookup, a limb of fewer than ten bits being looked up together with its shift to ten bits, or, for a range of $R \leq 8$ values, the gate $q \cdot w(w-1) \dots (w-(R-1)) = 0$; the three-bit windows of fixed-base scalar multiplication and the single bits use the gate (the halo2 book’s *Decomposition* chapter).

The Sinsemilla generator lookup. The Sinsemilla lookup is the three-column instance

$$\left(q_{S1} \cdot m_{i+1}, \quad q_{S1} \cdot x_p + (1 - q_{S1}) \cdot X(S[0]), \quad q_{S1} \cdot y_p + (1 - q_{S1}) \cdot Y(S[0]) \right) \mapsto (t_{\text{idX}}, t_x, t_y), \quad (34)$$

where $m_{i+1} = z_i - 2^{10} \cdot q_{\text{run}} \cdot z_{i+1}$ recovers the ten-bit message word from the hash’s own running-sum decomposition of its message, and $q_{\text{run}} := q_{S2} - q_{S2}(q_{S2} - 1) = q_{S2}(2 - q_{S2})$ is an expression in a fixed column $q_{S2} \in \{0, 1, 2\}$ that evaluates to 1 on the interior rows of a message piece and to 0 on its final row, where the trailing running sum is implicitly zero. The coordinate y_p is not a cell but a polynomial expression in the queried cells equal to the y -coordinate of the generator added (the halo2 book’s *Sinsemilla* chapter).

Expression-valued inputs. Lookup inputs may be arbitrary expressions over queried cells, not merely cells. The Sinsemilla lookup asserts that the generator added is $S[m_{i+1}]$ with no advice cell holding that generator’s y -coordinate. The $(1 - q_{S1})$ branches set the input tuple of every row with $q_{S1} = 0$ to table row 0, so the assertion of Construction 2.14, that every active row’s input tuple appears in the table, holds with nothing to exempt.

4.1.4 Column polynomials and the quotient. Every column, advice, fixed and instance, is a vector of $n = 2^k$ field elements, and its polynomial is its interpolant over the rows, of degree $< n$: the column polynomials of §2.5. The trailing rows of each advice column hold uniformly random field elements, the blinding rows of §4.6, on which every selector is zero and which Remark 2.17 exempts from the permutation and lookup arguments.

The quotient. Each gate polynomial is evaluated with the committed column polynomials substituted for the queried cells, a query at rotation r reading $v(\omega^r X)$, and the resulting polynomials, together with the permutation identities (7) and the lookup identities (9), are folded with powers of a challenge y and divided by $Z_H(X) = X^n - 1$. The result is the quotient $b(X)$ of phase 4 of “The complete Halo 2 protocol” (§4.5), where the folding and the division are written out (the halo2 book’s *Vanishing argument* chapter).

The running example. The running example of §2.2 stands at the same point: it interpolated a four-row trace into a, b, c , the five selector polynomials and PI, and assembled the one polynomial $G(X)$ of degree 9 whose divisibility by $Z_H = X^4 - 1$, with the quotient t of degree 5, the rest of the protocol certifies.

The deployed parameters. For the Action circuit $k = 11$, $n = 2^{11} = 2048$ and $d_{\max} = 9$, the bound holding for its gates and for its permutation and lookup identities. The quotient therefore splits into $d_{\max} - 1 = 8$ chunks of degree below n , derived in §4.5. The two arguments this construction leans on are the ones §2 proved: the permutation argument of §2.6 stands behind every copy constraint, and the lookup argument of §2.8 behind the lookups (33) and (34).

4.2 Instance columns and binding the public statement

Definition 2.7 lists instance columns beside fixed and advice columns, and the arguments of §2 treat them like any other column. This subsection states how the instance column binds a proof to its public statement.

The public input of the running example. In the running example the target 35 is public, so it is not advice but *instance* data: it sits in the public-input polynomial $PI(X)$ of (2), which the verifier builds himself from the column $(0, 0, 0, -35)$; its interpolant $64 + 50X + 33X^2 + 47X^3$ of §2.2 takes the value $-35 = 62$ on row 3 and 0 elsewhere. A prover who claimed that $X^3 + X + 5 = 36$ has a root would face a *different* G , one with -36 in place of -35 , and her honest trace would no longer vanish on the rows: the fourth gate would read $30 + 5 - 36 \neq 0$. The public input enters the polynomial G whose divisibility Theorem 2.5 tests, and an accepted proof attests the statement for the verifier’s public values.

Definition 4.2 (Instance polynomial). An *instance column* is a public-input column: its n entries $\pi = (\pi_0, \dots, \pi_{n-1}) \in \mathbb{F}^n$ are not part of the witness, and both parties agree on them before verification. Like every other column it is interpolated over the rows in the Lagrange basis,

$$\pi(X) := \sum_{j=0}^{n-1} \pi_j \ell_j(X) \in \mathbb{F}[X], \quad \deg \pi < n,$$

with $\ell_j(\omega^j) = 1$ and $\ell_j(\omega^{j'}) = 0$ for $j' \neq j$ (§2.5; the *Math Guide*, §“The Lagrange basis on H ”). The public values occupy designated rows, and every other π_j is 0.

Binding by copy constraints. The deployed circuit binds the public values through the permutation argument of §2.6, not through a gate. The instance column is equality-enabled, so it participates in the permutation σ ; it is one of the fifteen columns counted under “The deployed chunking” there, beside the ten advice and four fixed ones. Each designated instance cell is copy-constrained to an advice cell that the circuit’s constraints read (the halo2 book’s *Permutation argument* chapter for the mechanism), so a satisfying assignment exists only when those advice cells hold the public values the verifier supplied (Figure 10). The alternative, a dedicated gate $q(X)(a(X) - \pi(X)) \equiv 0 \pmod{Z_H(X)}$ with a selector q active exactly on the designated rows, is not used: the copy constraint reuses the permutation argument, at no extra gate degree.

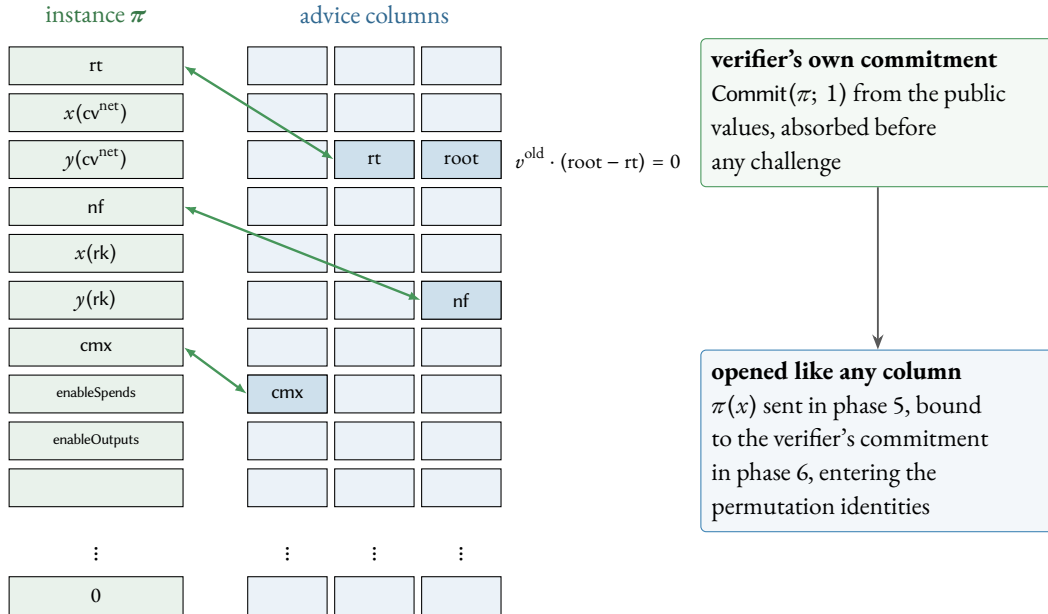


Figure 10. The instance column of the Action circuit. Rows 0 to 8 hold the primary input in the encoding of the protocol specification (§ 4.18.4), the curve points cv^{net} and rk as two coordinates each; the further input `disableCrossAddress` of § 4.6 occupies a row that no normative text fixes, drawn empty; every other row is 0. Each designated cell is copy-constrained to an advice cell the circuit’s constraints read; three such constraints are drawn (green double arrows): `nf` and `cmx` are copied to the cells where the circuit computes them, `rt` to a cell that a gate ties to the computed Merkle root. The verifier commits to the column himself, with the fixed blinding factor 1, and absorbs that commitment before the first challenge.

The instance commitment. The verifier does not take $\pi(X)$ from the prover. *Both* parties compute the commitment to each instance polynomial from the instance values (the halo2 book’s *Circuit commitments* chapter), with the public blinding factor 1 in place of a random one so that the two computations agree, and absorb it into the transcript before the first challenge is drawn, in phase 1 of §4.5; every Fiat–Shamir challenge thereby depends on the public input. The book fixes the factor 1 for the fixed columns only; for the instance columns it is the choice of the implementation, `halo2_proofs`. Thereafter the instance column is treated like any other: its claimed evaluation $\pi(x)$ appears among the phase-5 evaluations, enters the permutation identities as one of the v_i of Construction 2.10, and the multipoint opening of phase 6 binds it to the instance commitment. Because the verifier computed that commitment himself, from the instance values he verifies against, the prover has no freedom over π : an evaluation inconsistent with the verifier’s own values fails the opening argument, by the evaluation binding recalled in §2.9.

The public input of the Action statement. The Action statement’s public input is the tuple $(rt, cv^{\text{net}}, nf, rk, cmx, \text{enableSpends}, \text{enableOutputs})$ (protocol specification, § 4.18.4, “Action Statement (Orchard)”: the anchor rt , the root of the note-commitment tree the spent note is proved against (the *Crypto Guide*, §“Append-only and incrementally updatable trees”); the net value commitment cv^{net} ; the nullifier nf ; the randomised verification key rk ; the extracted commitment $cmx = \text{Extract}_p(cm)$ of the output note (the *Crypto Guide*, §“A shielded spend, end to end”); and the flags enableSpends and enableOutputs , which permit non-zero-valued spends and outputs. The primary-input note of the same section encodes the tuple as the sequence

$$[rt, x(cv^{\text{net}}), y(cv^{\text{net}}), nf, x(rk), y(rk), cmx, \text{enableSpends}, \text{enableOutputs}] \in \mathbb{F}^9,$$

each integer read as an element of $\mathbb{F} = \mathbb{F}_{p_{\text{Pallas}}}$. The Action description specifies the proof with an eighth component, $\text{disableCrossAddress} := 1 - \text{enableCrossAddress}$, where $\text{enableCrossAddress}$ permits the output note’s receiver to differ from the input note’s (protocol specification, § 4.6, “Action Descriptions”); from NU6.3 the proof of every Action of either pool is verified with this further public input, which is 1 for every Orchard-pool Action (ZIP 258). Rows 0 to 8 of the instance column hold the nine values above; no normative text fixes the row of the further input, and every other row is 0 (Figure 10). Each is copy-constrained to an advice cell the circuit’s constraints read: cv^{net} , nf , rk and cmx to the cells where the circuit computes them; rt to a cell tied to the computed Merkle root by the gate $v^{\text{old}}(\text{root} - rt) = 0$, which encodes the statement’s condition that $v^{\text{old}} = 0$ or the Merkle path is valid; the two enable flags to cells read by the gates $v^{\text{old}}(1 - \text{enableSpends}) = 0$ and $v^{\text{new}}(1 - \text{enableOutputs}) = 0$, which encode, for flags in $\{0, 1\}$, the conditions $v^{\text{old}} = 0$ or $\text{enableSpends} = 1$, and $v^{\text{new}} = 0$ or $\text{enableOutputs} = 1$; and $\text{disableCrossAddress}$ to a cell read by a gate that, when it is nonzero, equates the coordinates of the receivers (g_d, pk_d) of the two notes. That condition is the meaning § 4.6 gives the flag; the list of conditions in § 4.18.4 does not yet include it.

Binding to the instance. The verifier computes the instance commitment from the public values π itself, and the copy constraints tie the designated cells to the advice cells the constraints read; so a witness extracted from an accepting proof satisfies the relation at π , not at other public

values π' the prover may target. The probabilistic conclusion, that a prover who knows no witness for π , whatever witnesses she holds for other instances, convinces the verifier with at most negligible probability, holds under the knowledge soundness whose status §5 classifies: designed-but-unspecified for the deployed transcript, resting on the discrete logarithm on Vesta or on the algebraic group model, with the transcript hash modelled as a random oracle. Under that hypothesis an Action proof that verifies against an anchor, a nullifier and a value commitment certifies knowledge of a witness consistent with those values.

4.3 Removing the verifier: the Fiat–Shamir transform

The protocols of §§2–3 are public-coin: every verifier message is uniformly random and independent of the prover’s messages, a field element (the point z of the random-point check, the pair β, γ of the permutation and lookup arguments, the challenges u_j of each opening round) or, in Construction 3.2, the group element U ; and the verifier’s decision is a deterministic function of the public input and the transcript. Their soundness requires only that each challenge be drawn after the commitments it tests are fixed.

The transform. The Fiat–Shamir transform replaces the i -th challenge c_i by

$$c_i := H(x, m_1, c_1, \dots, c_{i-1}, m_i),$$

the hash of the public statement x and of the transcript prefix, the prover’s messages $m_1, \dots, m_i$ and the earlier challenges, under a fixed hash function H (the *Crypto Guide*, §“The Fiat–Shamir transform: from interactive to non-interactive”). The ordering of §2.9 is enforced because each commitment is an input to the hash that derives the challenge testing it. The toy of §2.4 drew $z = 20$ as an interactive challenge; the deployed prover derives every challenge from a hash. The two kinds of challenge are kept apart in what follows: the interactive one is uniform by assumption, the hashed one only in the model in which the hash is a random oracle.

Grinding. A prover against the compiled argument may vary the inputs of H and inspect the resulting challenges. The bound on that freedom uses round-by-round soundness (Canetti, Chen, Holmgren, Lombardi, Rothblum, Rothblum and Wichs, *Fiat–Shamir: From Practice to Theory*), which Remark 4.11 recalls.

Proposition 4.3 (Fiat–Shamir of a round-by-round sound protocol). *Let Π be a public-coin interactive protocol whose partial transcripts for a false statement carry a doomed labelling with error ε : the empty transcript is doomed, a doomed complete transcript is rejected, and from a doomed partial transcript, whatever the prover’s next message, the next challenge leaves it doomed except with probability at most ε . With H modelled as a random oracle, a prover making at most Q distinct oracle queries, the verifier’s recomputations counted among them, makes the verifier of the Fiat–Shamir transform of Π accept with probability at most $Q\varepsilon$.*

Proof. Each query is a transcript prefix followed by a prover message, and its answer is uniform and independent of everything before the query. The verifier accepts only a transcript that is not doomed, and the empty transcript is doomed, so acceptance requires some query on a doomed

prefix whose answer leaves the doomed set; each query does so with probability at most ϵ , and the union bound over the at most Q queries gives $Q\epsilon$. $\square$

The proposition has the form of the halo2 book’s extraction theorem (*Protocol Description* chapter, “Witness-extended Emulation”, its term $q\epsilon$) and of the state-restoration analysis of Ghoshal and Tessaro that §5 cites, where its application to the deployed transcript is classified. For the toy’s identity check, a single challenge with error $11/97$ (§2.4), ten queries exhaust the bound. This volume establishes no round-by-round error for Halo 2, whose binding is computational. At Orchard scale the identity check’s term is about 2^{-240} (one Schwartz–Zippel term, in the sense §1.2 fixed), and budgets of 2^{40} , 2^{64} and 2^{80} queries multiply that term alone to 2^{-200} , 2^{-176} and 2^{-160} ; these agree with the $Q_{\epsilon_{\text{bad}}}$ of the halo2 book’s algebraic-group-model bound quoted in §5.2, proved for the abstract protocol, whose transfer to the deployed transcript §5 classifies. The permutation argument’s term over (β, γ) , $3N/|F|$ with $N = 15 \cdot 2042$ participating cells (Corollary 2.12), is larger, about $2^{-237.5}$.

The hash must take the whole transcript prefix as input: a statement or commitment left out can be chosen after the challenge is seen, the “weak Fiat–Shamir” failure the *Crypto Guide* records in §“The Fiat–Shamir transform: from interactive to non-interactive”. The bound holds with H modelled as a random oracle, a heuristic. The *Crypto Guide* proves the one-round theorem, with its forking-lemma extractor and its programmed-oracle simulator, and states that a multi-round protocol such as this one needs a multi-round theorem whose hypotheses match the protocol at hand; §5 weighs that theorem against the deployed transcript.

Non-interactivity. The compiled argument is non-interactive: the proof is one static object, commitments, evaluations and the opening’s scalars, and verification recomputes the challenges by the same hash and runs the same checks. The protocol specification defines a Halo 2 proof as a byte sequence (protocol specification, § 5.4.10.3, “Halo 2”, and its “Encoding of Halo 2 Proofs”), so a full node verifies an Action proof from the transaction alone.

The deployed transcript. Neither the protocol specification nor the halo2 book fixes the transcript that instantiates the transform; it is that of the implementation, `halo2_proofs`, and is as follows. The state is a BLAKE2b-512 hash state with the sixteen-byte personalisation `HalO2-Transcript` (the *Crypto Guide*, §“Domain separation and personalisation”). Absorbing a point $P = (x, y) \neq \mathcal{O}$ of Vesta appends the bytes `0x01 || LE256(x) || LE256(y)`; the identity is not absorbable. Absorbing a scalar $s \in \mathbb{F}$ appends `0x02 || LE256(s)`. Squeezing a challenge appends `0x00` to the state, which keeps it, and returns the 64-byte digest of the state read as a little-endian integer and reduced modulo p_{Pallas} , with no rejection of any value (§3.8). The proof carries each point as its 32-byte compressed encoding, the x -coordinate with the parity of y in the top bit (protocol specification, § 5.4.9.6, “Pallas and Vesta”), and each scalar as $\text{LE}_{256}(s)$. Before any prover message both parties absorb, as a scalar, a digest of the verifying key, its BLAKE2b hash reduced into $\mathbb{F}$, and then the instance commitments of §4.2, which therefore never appear in the proof. Every prover message is then absorbed in the order the protocol fixes, and each challenge is squeezed at its prescribed point, after everything it must depend on. The proof is the sequence of the prover’s messages in the phase order of Table 10, counted by kind in Table 8; the verifier replays the absorptions and recomputes every challenge. Figure 12 in §4.5 draws the timeline.

4.4 The multipoint opening argument

When the verifier has drawn his evaluation point x , the prover has committed to several dozen polynomials and claimed their values at x and at rotations of x : a running product at x and ωx , a permuted lookup input at x and $\omega^{-1}x$, an advice column at every rotation its gates read. Opening each claim with the argument of §3 would cost $2k + 1 = 23$ points per claim, and a one-Action proof makes 95 claims about 69 committed polynomials, the 94 evaluations at x and its rotations in Table 8 and the quotient's ($49a + 46$ claims for a Actions). Halo 2 reduces all of them to a single opening of a single polynomial at a single point. The reduction combines random linear combinations, division by vanishing polynomials and one random-point check. The letters q and f below follow the halo2 book's *Multipoint opening argument* chapter and are unrelated to the selector polynomials $q_\bullet$ of §2, which are among the polynomials being folded; nor is this f the final blinder f of the deployed opening (§3.8), which phase 7 sends. The book's *Protocol Description* chapter writes the values $q_i(x_3)$ as u , the letter this volume reserves for the round challenges of the inner-product argument; they are written $\bar{q}_i$ here. The index i ranges over sets of query points, not over columns.

Construction 4.4 (Multipoint opening). Let $p_1, \dots, p_N$ be polynomials of degree $< n$ with commitments $C_1, \dots, C_N$ under Construction 3.2, each p_ν queried at a finite set $T_\nu \subset \mathbb{F}$ of points, and let $\hat{p}_\nu(t)$ for $t \in T_\nu$ be the values the prover has claimed. Let $T^{(1)}, \dots, T^{(m)}$ be the distinct sets among the T_ν and $I_i := \{\nu : T_\nu = T^{(i)}\}$ the i -th group, enumerated as $\nu_{i,0}, \nu_{i,1}, \dots$; write $Z_i(X) := \prod_{t \in T^{(i)}} (X - t)$.

1. **Fold each group (x_1).** The verifier sends $x_1 \in \mathbb{F}$. For each group i the prover forms $q_i(X) := \sum_e x_1^e p_{\nu_{i,e}}(X)$, the verifier forms the same combination of commitments, $Q_i := \sum_e [x_1^e] C_{\nu_{i,e}}$, which commits to q_i by linearity (*Crypto Guide*, §“Pedersen vector commitments”), and both form the combined claimed values $\hat{q}_i(t) := \sum_e x_1^e \hat{p}_{\nu_{i,e}}(t)$ for $t \in T^{(i)}$. Both interpolate $r_i(X)$, the unique polynomial of degree $< |T^{(i)}|$ with $r_i(t) = \hat{q}_i(t)$ on $T^{(i)}$ (the *Math Guide*, §“Lagrange interpolation”); the verifier can do this from the claimed values alone.
2. **Divide out the points (x_2).** The verifier sends $x_2 \in \mathbb{F}$. The prover forms the quotients $f_i(X) := (q_i(X) - r_i(X))/Z_i(X)$, which are polynomials exactly when the combined values $\hat{q}_i(t)$ equal $q_i(t)$ for every $t \in T^{(i)}$, in particular when every claimed value is true, and combines them into

$$f(X) := \sum_{i=1}^m x_2^{i-1} f_i(X),$$

samples a fresh blinder r_f and sends $F := \text{Commit}(f; r_f)$.

3. **A fresh point (x_3).** The verifier sends $x_3 \in \mathbb{F}$. The prover sends $\bar{q}_i := q_i(x_3)$ for $i = 1, \dots, m$.
4. **The value f must take.** The verifier computes

$$v_f := \sum_{i=1}^m x_2^{i-1} \frac{\bar{q}_i - r_i(x_3)}{Z_i(x_3)},$$

the value of $f(x_3)$ if every f_i is a polynomial and every $\bar{q}_i$ is honest; he can, since r_i and Z_i are his to evaluate, and he rejects if some $Z_i(x_3)$ is 0.

5. **One polynomial** (x_4). The verifier sends $x_4 \in \mathbb{F}$. Both parties form

$$f^*(X) := f(X) + \sum_{i=1}^m x_4^i q_i(X), \quad F^* := F + \sum_{i=1}^m [x_4^i] Q_i, \quad v^* := v_f + \sum_{i=1}^m x_4^i \tilde{q}_i,$$

and run one opening of Construction 3.2 on the statement (F^*, x_3, v^*) , the prover's witness being the coefficient vector of f^* and the blinder $r_f + \sum_i x_4^i r_{Q_i}$, where r_{Q_i} is the blinder of Q_i , the x_1 -combination of the blinders of the polynomials in group i . The verifier accepts the original claims if and only if that opening accepts.

Two remarks on the shape. Nothing is sent between x_1 and x_2 , so the deployed transcript squeezes them back to back; they are two challenges because they play two roles, one keeping the polynomials of a group linearly independent and one keeping the quotients of different groups independent. And the reduction is free for the verifier in group operations except for the combinations of commitments, which are a few scalar multiplications per committed polynomial; the one expensive step, the final multi-scalar multiplication, is paid once, inside the single opening. Figure 11 draws the data flow.

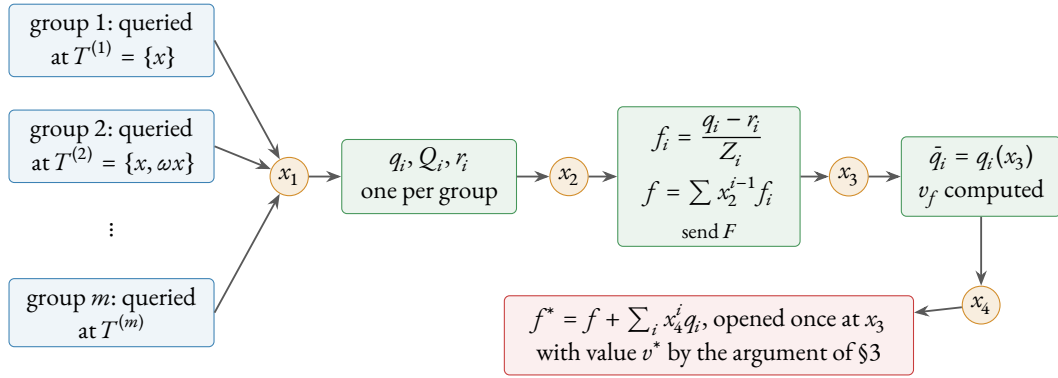


Figure 11. The multipoint reduction of Construction 4.4. Committed polynomials are grouped by the set of points at which they are queried (blue), folded within each group by x_1 into q_i , quotiented by the group's vanishing factor into f_i , combined across groups by x_2 into f , which is committed as F ; after the fresh point x_3 the prover sends the $q_i(x_3)$, and x_4 folds f and the q_i into f^* (red), whose single opening at x_3 discharges every original claim. Challenges are amber.

Completeness. If every claimed value is true, then $q_i - r_i$ vanishes on $T^{(i)}$ and is divisible by Z_i (the *Math Guide*, §“Roots and the factor theorem”), each f_i is a polynomial of degree $< n$, so is f , and $f(x_3) = v_f$ whenever no $Z_i(x_3)$ is 0; $f^*(x_3) = v^*$ then holds, and the honest opening of §3 accepts. The verifier's rejection when x_3 is one of the query points, an event of probability at most $|\bigcup_i T^{(i)}|/|\mathbb{F}|$, is the reduction's only completeness defect.

Lemma 4.5 (Soundness of the multipoint reduction). *In Construction 4.4, let the committed polynomials p , and the claimed values be fixed before x_1 is drawn, let $\hat{f}$ be a polynomial of degree $< n$ committed in F , fixed before x_3 is drawn, let $x_1, \dots, x_4$ be independent and uniform in $\mathbb{F}$, and*

let $Z(X) := \prod_{t \in \bigcup_i T^{(i)}} (X - t)$. If some claimed value is false, $\hat{p}_{v_0}(t_0) \neq p_{v_0}(t_0)$ with v_0 in group i_0 , then the probability that the verifier computes v_f and the polynomial $\hat{f} + \sum_i x_4^i q_i$ committed in F^* takes the value v^* at x_3 is at most

$$\frac{|I_{i_0}| + 2m + n - 1 + \deg Z}{|\mathbb{F}|}.$$

Proof. The argument takes one Schwartz–Zippel term per challenge (the *Math Guide*, §“The Schwartz–Zippel lemma”). *The fold.* The difference $\hat{q}_{i_0}(t_0) - q_{i_0}(t_0) = \sum_e x_1^e (\hat{p}_{v_{i_0,e}}(t_0) - p_{v_{i_0,e}}(t_0))$ is a nonzero polynomial in x_1 of degree below $|I_{i_0}|$, fixed before x_1 is drawn, so except with probability $|I_{i_0}|/|\mathbb{F}|$ the folded claim is false: $r_{i_0}(t_0) \neq q_{i_0}(t_0)$. *The combination.* Each $T^{(i)}$ is contained in $\bigcup_{i'} T^{(i')}$, so each Z/Z_i is a polynomial. The polynomial

$$M(X) := \hat{f}(X) Z(X) - \sum_{i=1}^m x_2^{i-1} (q_i(X) - r_i(X)) \frac{Z(X)}{Z_i(X)}$$

is fixed once x_2 is drawn and F is sent, before x_3 . At t_0 the term $\hat{f}(t_0) Z(t_0)$ and every term with $t_0 \notin T^{(i)}$ vanish, and

$$M(t_0) = - \sum_{i: t_0 \in T^{(i)}} x_2^{i-1} (q_i(t_0) - r_i(t_0)) \prod_{t \notin T^{(i)}} (t_0 - t),$$

the product over $t \in \bigcup_{i'} T^{(i')} \setminus T^{(i)}$: a polynomial in x_2 of degree below m , fixed before x_2 is drawn, whose coefficient at $x_2^{i_0-1}$ is nonzero. Except with probability $m/|\mathbb{F}|$ over x_2 , then, $M(t_0) \neq 0$ and M is a nonzero polynomial of degree at most $n - 1 + \deg Z$. *The last fold.* Acceptance means $(\hat{f}(x_3) - v_f) + \sum_i x_4^i (q_i(x_3) - \bar{q}_i) = 0$, a polynomial equation in x_4 of degree at most m whose coefficients are fixed before x_4 is drawn. Unless every coefficient is zero, that is, unless $\hat{f}(x_3) = v_f$ and $\bar{q}_i = q_i(x_3)$ for every i , a uniform x_4 satisfies it with probability at most $m/|\mathbb{F}|$. *The fresh point.* If $\hat{f}(x_3) = v_f$ and every $\bar{q}_i = q_i(x_3)$, then, since v_f is computed only when $Z(x_3) \neq 0$, multiplying the first equation by $Z(x_3)$ gives $M(x_3) = 0$, which a uniform x_3 achieves for a nonzero M with probability at most $(n - 1 + \deg Z)/|\mathbb{F}|$. A union bound over the four terms gives the stated bound. $\square$

The bound is one more Schwartz–Zippel allowance, of the order of $n/|\mathbb{F}|$. The opening’s own knowledge error, $O(\log d/|\mathbb{F}|)$ by Corollary 3.13, lies below it, and that corollary’s witness-or-relation form supplies the binding opening the lemma assumes.

The deployed instance. In the Action circuit the point sets are five. Every fixed, instance and permuted-label polynomial, the random polynomial of phase 4 and the collapsed quotient are queried at $\{x\}$; the advice columns, five at $\{x, \omega x\}$ and five at $\{\omega^{-1}x, x, \omega x\}$, according to the rotations their gates read; the permuted lookup input $\mathcal{A}'$ at $\{x, \omega^{-1}x\}$ and the permuted table $\mathcal{S}'$ at $\{x\}$; the lookup running products and the last permutation running product at $\{x, \omega x\}$; and the first two permutation running products at $\{x, \omega x, \omega^{-6}x\}$, the last rotation reaching the boundary row of Remark 2.17 that chains one product into the next. The sets $\{x\}$, $\{x, \omega x\}$, $\{x, \omega^{-1}x\}$,

$\{\omega^{-1}x, x, \omega x\}$ and $\{x, \omega x, \omega^{-6}x\}$ give the five values $\bar{q}_i$, the multipoint evaluations of Table 8, and the whole reduction adds one point and five scalars to the proof.

4.5 The complete Halo 2 protocol

The pieces are now assembled into the Halo 2 protocol (the halo2 book’s *Protocol Description* chapter, instantiated with the arguments of its *Proving system* chapters), in seven phases. Throughout, $\mathbb{F}$ is the constraint field, $\mathbb{F}_{p_{\text{Pallas}}}$ for Orchard; $\mathbb{G}$ is the commitment group, the Vesta curve, whose order is p_{Pallas} (§3.8); the rows are $\omega^0, \dots, \omega^{n-1}$ for a primitive n -th root of unity ω , with $n = 2^k$; and every commitment is $\text{Commit}(p; r) = \langle \mathbf{p}, \mathbf{G} \rangle + [r]H$ of Construction 3.2, H the blinding generator. The protocol has a preprocessed setup and seven phases: (1) advice commitments; (2) lookup compression, challenge θ ; (3) running products, challenges β, γ ; (4) the quotient, challenge γ ; (5) the evaluation challenge x ; (6) the multipoint opening, challenges x_1, x_2, x_3, x_4 ; (7) the inner-product opening. Every challenge is squeezed from the transcript of §4.3 at the point named; “the verifier sends” below means that. Table 10 lists the messages with their Orchard counts and Figure 12 draws the timeline.

Setup. The circuit fixes the columns, gates, lookup arguments and the equality permutation σ , as §4.1 described; its selectors and lookup tables are fixed columns. Key generation interpolates every fixed column and every permuted-label polynomial $s_i(X)$ of Construction 2.10 and commits to them with the fixed blinding factor 1 (for the fixed columns, the halo2 book’s *Circuit commitments* chapter); these commitments, with the circuit description, form the verifying key. The transparent parameters of the commitment scheme, the n generators $\mathbf{G}$, the blinding generator H and the inner-product generator U hashed to the curve as §3.8 described, together with the domain data, are held separately. All of it is reproducible from public algorithms and the circuit; no secret setup trapdoor exists. Verification is a deterministic function of the parameters, the verifying key, the instance values and the proof.

Phase 1: advice commitments. Both parties first absorb the digest of the verifying key and the instance commitments of §4.2 into the transcript, so that every subsequent challenge depends on the circuit and on the public input. The prover constructs the advice columns $\mathbf{w}_1, \dots, \mathbf{w}_{n_d} \in \mathbb{F}^n$, which with the fixed and instance columns form a satisfying assignment in the sense of Definition 2.7, fills the reserved trailing rows of each with uniformly random field elements, the blinding rows of §4.6, interpolates each to $w_i(X)$ of degree $< n$, and sends $\text{Commit}(w_i; r_i)$ with fresh blinders r_i . The verifier absorbs the commitments.

Phase 2: lookup compression, challenge θ . The verifier sends θ . For each lookup the prover compresses the input expressions and table columns by powers of θ as in (10), forms the permuted columns A' and S' of Construction 2.14, and sends their two commitments; the verifier absorbs them.

Phase 3: running products, challenges β, γ . The verifier sends β and γ . The prover commits to the permutation running products, one per chunk of $d_{\text{max}} - 2$ columns as “The deployed

chunking” in §2.6 described, and to one lookup running product Z_{lookup} per lookup, and sends the commitments; the verifier absorbs them. The order is not a convenience. The permuted columns A' and S' must be fixed *before* the challenges that randomise the products are drawn, since Theorem 2.15 bounds the prover’s success probability only over challenges drawn after her columns; a prover who saw β, γ first could choose the permutation against them.

Phase 4: the quotient, challenge γ . Before γ is drawn the prover commits to a uniformly random polynomial of degree $< n$, sends the commitment, and the verifier absorbs it; its role is the zero-knowledge one explained in §4.6. The verifier then sends γ . The prover assembles the *gate equation*, one polynomial

$$C(X) := \sum_{t=0}^{T-1} \gamma^t g_t(X),$$

the γ -weighted combination of the T constraint polynomials g_t of the circuit: each constraint of each custom gate with its selector factor, the permutation identities (7) and the chunk-boundary identities, the lookup identities (9), and the boundary identities of Remark 2.17, each masked by its active-row factor where that remark requires. If every constraint holds then every g_t vanishes on the rows, so C does, and $Z_H \mid C$ by (3): the *quotient* $b(X) := C(X)/Z_H(X) = C(X)/(X^n - 1)$ is a polynomial. If some constraint fails at some row, Lemma 4.6 bounds the probability that C nonetheless vanishes on the rows, since γ is drawn after every polynomial in the g_t is committed and after θ, β and γ .

Lemma 4.6 (Batching by γ). *Let $g_0, \dots, g_{T-1} \in \mathbb{F}[X]$ be fixed, and let γ be uniform in $\mathbb{F}$. If some g_t does not vanish on the rows, then $C = \sum_{t=0}^{T-1} \gamma^t g_t$ vanishes on the rows with probability at most $(T-1)/|\mathbb{F}|$.*

Proof. Let $g_{t_0}(\omega^j) \neq 0$. Then $C(\omega^j) = \sum_t \gamma^t g_t(\omega^j)$ is a polynomial in γ of degree at most $T-1$ with the nonzero coefficient $g_{t_0}(\omega^j)$, so it vanishes for at most $T-1$ values of γ (the *Math Guide*, §“The Schwartz–Zippel lemma”). $\square$

Theorem 4.7 (Degree of the quotient). *Let every column polynomial and every permuted-label polynomial s_i have degree $< n$, and every constraint polynomial degree at most $d_{\max}$, counting as degree one each column polynomial at a rotation, each s_i , each of $\ell_0, \ell_{\text{last}}$ and ℓ_{blind} , and X (as in “The deployed chunking” of §2.6). Then*

$$\deg b \leq d_{\max}(n-1) - n = (d_{\max} - 1)(n-1) - 1.$$

Proof. A constraint of degree at most $d_{\max}$ is a sum of products of at most $d_{\max}$ such factors, each of degree at most $n-1$ (rotations do not change degrees); so $\deg g_t \leq d_{\max}(n-1)$ and $\deg C \leq d_{\max}(n-1)$. Division by Z_H , of degree n , subtracts n . The two forms agree since $(d_{\max} - 1)(n-1) - 1 = d_{\max}(n-1) - (n-1) - 1 = d_{\max}(n-1) - n$. $\square$

For the deployed $d_{\max} = 9$ and $n = 2048$ the bound is $\deg b \leq 16375$, far above the degree bound $n-1 = 2047$ that the commitment of Construction 3.2 supports with $d = n$ generators,

so b cannot be committed as one polynomial. The prover therefore uses a fixed proof shape of $d_{\max} - 1$ chunks $b_0, \dots, b_{d_{\max}-2}$, each of degree $< n$, with

$$b(X) = \sum_{i=0}^{d_{\max}-2} X^{ni} b_i(X),$$

retaining leading zero chunks when the actual degree is smaller, and commits to each chunk separately with its own blinder; there are 8 chunks for Orchard. The count is right because b has at most $(d_{\max} - 1)(n - 1) = (d_{\max} - 1)n - (d_{\max} - 1) < (d_{\max} - 1)n$ coefficients. Once the evaluation point x is drawn, the verifier recombines the chunk commitments, weighted by powers of x^n , into one commitment $\sum_i [x^{ni}] \text{Commit}(b_i)$ to the polynomial $\sum_i x^{ni} b_i(X)$, whose value at x is $b(x)$; no chunk is ever opened on its own (the halo2 book's *Protocol Description* chapter, steps 5 to 8, and note 1 of its "Zero-knowledge and Completeness").

Phase 5: the evaluation challenge x . The verifier sends x . The prover evaluates every committed polynomial the verifier's identities query, *except* the quotient chunks, at x and at the rotated points the identities reference, ωx for the running products and for any gate reading the next row, $\omega^{-1}x$ for the permuted lookup input A' and for gates reading the previous row, and $\omega^{-6}x$, the deployed boundary row, for all but the last permutation product, and sends the claimed values as field elements. The deployed proof also carries the evaluations at x of the fixed columns and of the permuted-label polynomials, which the same opening binds to the verifying key's commitments, and of the random polynomial, bound to its phase-4 commitment. No evaluation of b is sent: the verifier computes the expected quotient value $C(x)/(x^n - 1)$ from the sent evaluations, evaluating $\ell_0, \ell_{\text{last}}, \ell_{\text{blind}}$ and Z_H at x himself, and enforces it against the collapsed chunk commitment inside the multipoint opening of phases 6 and 7, where the claimed evaluations themselves are bound to their commitments as well. The gate equation is thus checked as the identity $C(x) = b(x) Z_H(x)$, the deployed form of (5).

Phase 6: the multipoint opening, challenges $x_1, \dots, x_4$. The parties run Construction 4.4 on every committed polynomial with a claimed evaluation: the advice columns, the instance column, the fixed and permuted-label polynomials, the permuted lookup columns, the running products, the random polynomial, and the collapsed quotient with its expected value, the last two joining the $\{x\}$ group. The prover sends the commitment F after x_1, x_2 and the five values $\bar{q}_i$ after x_3 ; after x_4 both parties hold the statement (F^*, x_3, v^*) .

Phase 7: the inner-product opening. The parties execute the deployed opening of §3.8, the masked argument in the normalisation of Remark 3.9, on (F^*, x_3, v^*) . The prover sends the mask commitment S ; the verifier sends ξ and ζ ; then for $k = 1$ rounds the prover sends the pair (L_j, R_j) and the verifier answers with u_j ; finally the prover sends the scalars c and f . The verifier accepts if and only if the deployed equation (29) holds with $P := F^*, v := v^*$ and $x := x_3$, its one multi-scalar multiplication of length n included. Its symmetric counterpart is the masked check (24), which differs from it by the three changes of that remark: the fold normalisation, a change of variables; the factor ζ on U ; and the value carried as $+ [v]U$ rather than $- [v]G_0$. Neither it nor (15) is the

deployed equation. This is the proof's end; Table 8 counts its bytes by kind, and nothing else is sent.

Accumulation, not deployed. In the recursive variant of Halo 2 the verifier is itself a circuit, and that circuit defers the one expensive step of phase 7, the claim $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle = \text{Commit}(g_{\text{dep}}; 0)$ of (29), the deployed form (Remark 3.9) of the claim $G^{(0)} = \text{Commit}(g; 0)$ of §3.4, into an accumulator folded with the accumulators of earlier proofs; a single decider at the end verifies all accumulated claims. Orchard does not do this. The protocol specification uses Halo 2 only to prove and verify Action statements and composes no proofs (protocol specification, § 4.1.13, “Zero-Knowledge Proving System”), the Orchard design proposal states that it makes no use of the recursion (ZIP 224), and the verifier pays every deferred multiplication itself; several proofs may share it by the randomised batch check of Lemma 3.14. The variant is developed as the coda of §7.

Costs. The prover computes one multi-scalar multiplication of length n per commitment she sends before the opening, $22a + 10$ for a bundle of a Actions: per Action the ten advice columns, the six permuted lookup columns and the three permutation and three lookup products; shared, the random polynomial, the eight quotient chunks and the multipoint polynomial f . She also computes the a instance commitments, as the verifier does (§4.2). The opening adds one of length n , for its mask commitment S , and the $2k$ cross terms of its rounds, two in each, of lengths $n/2, n/4, \dots, 1$ in successive rounds. The verifier performs $O(a + \log n)$ field and group operations for a proof of a Actions: he commits to the a instance columns (at most ten nonzero values each), recomputes the challenges, evaluates each Action's gate equation at x , combines the $23a + 46$ commitments of phase 6, and runs the k rounds of phase 7; to this he adds the single deferred multi-scalar multiplication of length n , which is the linear term that §3.8.1 discussed and that a batch verifier amortises across proofs with fresh random scalars; that batching is distinct from the accumulation above.

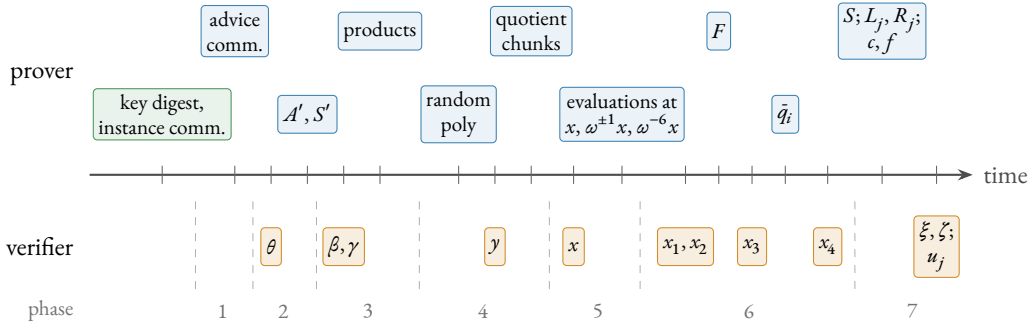


Figure 12. The transcript of §4.3 as a timeline. Prover messages (blue, staggered for legibility) are absorbed above the line; the data both parties hold in advance (green) is absorbed but never sent; challenges (amber) are squeezed below the line, each after everything it must depend on; dashed lines separate the phases of Table 10. The proof is the sequence of blue boxes read left to right.

Table 10. The seven phases as absorb-and-squeeze events, with the Orchard counts of Table 8. Each challenge is listed in the phase that draws it, and the prover column states which messages follow it. The per-Action messages repeat for each of the a Actions of a bundle and the shared ones appear once, giving $2720 + 2272 a$ bytes. The phase-5 scalars per Action are the instance evaluation, 25 advice evaluations, 8 permutation-product and 15 lookup evaluations; the shared 45 are 29 fixed, 15 permuted-label and 1 random-polynomial evaluation.

phase	prover sends	verifier squeezes	Orchard count
before 1	— (both absorb the key digest and the instance commitments)	—	0 bytes
1	advice commitments	—	10 points per Action
2	after θ , the A' and S' commitments, per lookup	θ	3 + 3 points per Action
3	after β, γ , the permutation products and the lookup products	β, γ	3 + 3 points per Action
4	random polynomial commitment; then, after y , the quotient chunks	y	1 + 8 points, shared
5	after x , the claimed evaluations at x and its rotations	x	49 scalars per Action; 45 shared
6	after x_1, x_2 , the commitment F ; then, after x_3 , the values $\tilde{q}_i$	$x_1, x_2; x_3$; then x_4	1 point, 5 scalars, shared
7	S ; then, after ξ, ζ , the pairs (L_j, R_j) for $j = k, \dots, 1$, each followed by u_j ; then c, f	$\xi, \zeta; u_j$ after each pair	1 + 22 points, 2 scalars, shared

4.6 Zero knowledge: hiding the witness in Halo 2

Knowledge soundness, classified in §5, concerns what a convincing prover knows; zero knowledge concerns what the proof reveals. Halo 2 is zero-knowledge through two devices, hiding commitments and blinded polynomials, assembled into a simulator. This subsection states what each device achieves on its own, and states the claim for the whole protocol as a cited proposition with its hypotheses.

Evaluations of the running example. The prover of the running example sends commitments and the evaluations $a(20), b(20), c(20), t(20)$ at the random point. An evaluation is a linear equation in the secret coefficients: $a(20) = 90 + 61 \cdot 20 + 22 \cdot 20^2 + 24 \cdot 20^3 = 59$ is one equation in the four unknowns (90, 61, 22, 24), and evaluations at four distinct points determine the column, hence the secret witness 3. Halo 2 therefore blinds: the witness trace reserves independent random rows, and every commitment carries an independent random blinding term. Adding random coefficients to a column polynomial would not do: a column polynomial is determined by its n row values, and a random coefficient on X^i changes every row, breaking the gates; a random multiple of the Lagrange polynomial ℓ_j of a reserved row changes row j alone, which is what a blinding row is. The random values violate no constraint: every selector is zero on the reserved rows, so the gates hold there, and the permutation and lookup identities carry the active-row factor of Remark 2.17. Each column’s openings are then uniform (Lemma 4.8); the joint statement is the proposition of “The simulator” below, whose simulator is one in the sense of the *Crypto Guide* (§“The simulation paradigm and zero knowledge”).

Hiding commitments. The commitment of Construction 3.2 is *perfectly hiding*. The prover commits a polynomial p as

$$\text{Commit}(p; r) = \langle \mathbf{p}, \mathbf{G} \rangle + [r] H, \quad r \text{ uniform in } \mathbb{F},$$

with $\mathbf{p}$ the coefficient vector, $\mathbf{G}$ the generator vector, and H the blinding generator of §3, of no known discrete-logarithm relation to $\mathbf{G}$; the term $[r]H$ makes the commitment a uniform group element for every p , since H generates $\mathbb{G}$, so a commitment by itself leaks nothing about what was committed. This is the perfect hiding of the Pedersen commitment, proved in the *Crypto Guide* (§“The Pedersen commitment” and §“Hiding and binding”) and recalled in §2.9. Every commitment the prover sends in phases 1 to 6, the advice columns, the permuted lookup columns A' and S' , the permutation and lookup running products, the random polynomial, the quotient chunks and the multipoint polynomial f , has this form with a fresh r (the halo2 book’s *Protocol Description* chapter, which blinds every group element of the protocol, its note 4). The verifying key’s commitments and the instance commitments carry the fixed blinding factor 1 instead: they commit public data both parties recompute (§4.5, §4.2), so there is nothing to hide, and a random blinder would prevent the two computations from agreeing.

Blinding rows. The prover opens commitments, and an opening reveals evaluations. To keep those evaluations from revealing the witness, the last n_{blind} rows of the table, the *blinding rows*, carry no circuit data: no gate applies there, and each advice column holds uniformly random field elements on them. By Lemma 4.8 below, a column opened at m distinct points outside the rows needs $n_{\text{blind}} \geq m$. The halo2 book’s *Protocol Description* chapter accordingly requires $n_e + 1$ random evaluations of a polynomial opened at n_e rotations of x , the extra one covering its opening at the point x_3 of the multipoint reduction. In the Action circuit a polynomial is opened at most four times: at three rotations of x , for some advice columns and the first two permutation products, and at x_3 . The deployed circuit takes $n_{\text{blind}} = 5$, one more than that, a parameter fixed by the implementation, `halo2_proofs`; with the boundary row of Remark 2.17 its usable rows number $n - (n_{\text{blind}} + 1) = 2042$. The permutation and lookup running products are blinded the same way, their trailing rows random, and the remark’s active-row factor exempts every one of those rows from every argument, so the random values do not violate a constraint (Figure 13; the halo2 book’s *Permutation argument* and *Lookup argument* chapters, their “Zero-knowledge adjustment” sections).

Lemma 4.8 (Openings of a column with blinding rows). *Let $v(X) = \sum_{j=0}^{n-1} v_j \ell_j(X)$ be a column polynomial over the rows $\omega^0, \dots, \omega^{n-1}$, let B be a set of n_{blind} row indices, the blinding rows, and let $z_1, \dots, z_m \in \mathbb{F}$ be distinct points, none of them a row. With the values on the other rows held fixed, the map $(v_j)_{j \in B} \mapsto (v(z_1), \dots, v(z_m))$ from $\mathbb{F}^{n_{\text{blind}}}$ to $\mathbb{F}^m$ is affine, and its linear part has rank $\min(m, n_{\text{blind}})$ (Math Guide, §“Bases, dimension, and linear maps”). Consequently, if $m \leq n_{\text{blind}}$ and the blinding values are uniform and independent, the m evaluations are jointly uniform on $\mathbb{F}^m$ and independent of the values on the other rows.*

Proof. The map is $v(z_i) = \sum_{j \notin B} v_j \ell_j(z_i) + \sum_{j \in B} v_j \ell_j(z_i)$, a constant plus the linear map with matrix $L := [\ell_j(z_i)]_{i \leq m, j \in B}$. The Lagrange basis on the rows has the closed form $\ell_j(X) = \omega^j (X^n -$

$1)/(n(X - \omega^j))$ (the *Math Guide*, §“The Lagrange basis on H ”), so $L = D_1 K D_2$ with $D_1 := \text{diag}(z_i^n - 1)$, nonzero on the diagonal because no z_i is a row, $D_2 := \text{diag}(\omega^j/n)$, nonzero because n is invertible in $\mathbb{F}$, and $K := [1/(z_i - \omega^j)]$. Hence $\text{rank } L = \text{rank } K$. Every square submatrix of K is nonsingular: take rows I and columns J with $|I| = |J| = s$, and suppose coefficients c_j , $j \in J$, not all zero, satisfy $\sum_{j \in J} c_j/(z_i - \omega^j) = 0$ for all $i \in I$. Over the common denominator, $\sum_{j \in J} c_j/(X - \omega^j) = N(X)/\prod_{j \in J} (X - \omega^j)$ with $N(X) := \sum_{j \in J} c_j \prod_{j' \in J, j' \neq j} (X - \omega^{j'})$ of degree $< s$; the polynomial N vanishes at the s distinct points z_i , so it is the zero polynomial (the *Math Guide*, §“Roots and the factor theorem”), and then $N(\omega^j) = c_j \prod_{j' \neq j} (\omega^j - \omega^{j'}) = 0$ forces $c_j = 0$ for every $j \in J$, since the rows are distinct: a contradiction. Thus K has an invertible square submatrix of size $\min(m, n_{\text{blind}})$, and its rank, at most $\min(m, n_{\text{blind}})$, equals $\min(m, n_{\text{blind}})$.

If $m \leq n_{\text{blind}}$ the linear part is onto $\mathbb{F}^m$, so the affine map is onto, and every fibre is a coset of the same kernel, of the same size $|\mathbb{F}|^{n_{\text{blind}}-m}$. A uniform point of $\mathbb{F}^{n_{\text{blind}}}$ therefore lands in each fibre with the same probability: the image is uniform on $\mathbb{F}^m$. The constant term depends on the other rows but not the distribution, so the evaluations are independent of them. $\square$

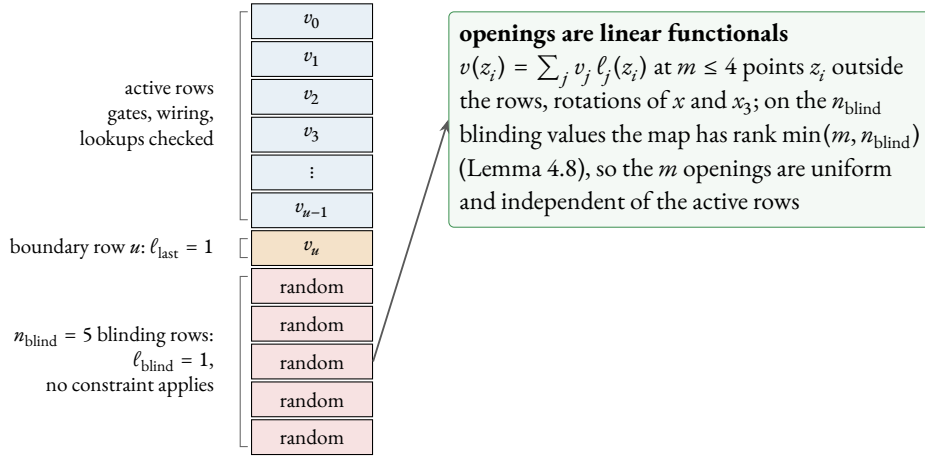


Figure 13. One column of the deployed table: the $u = n - 6 = 2042$ active rows $0, \dots, u - 1$ (blue) on which every argument is checked, the boundary row u selected by ℓ_{last} (amber) that pins the running products, and the $n_{\text{blind}} = 5$ blinding rows (red) selected by ℓ_{blind} , filled with uniform random values and exempted from every constraint by the active-row factor of Remark 2.17. Each opening of the column at a point outside the rows is a linear functional of all n row values, and Lemma 4.8 says the blinding rows alone make at most n_{blind} such openings jointly uniform.

On the toy domain $\{1, 22, 96, 75\}$ of §2.2, the rank of the lemma’s map was computed for every choice of $m \leq 3$ distinct points among the 93 outside it and for 2000 random choices of four: with $n_{\text{blind}} = 2$ blinding rows the rank is 1 for one point and 2 for two, and the map is onto; for three points the rank stays 2 and it is not; with $n_{\text{blind}} = 3$ the ranks are 1, 2, 3 for one, two and three points, onto each time, and 3 for four points, no longer onto. The threshold is exact: the map is a bijection at $m = n_{\text{blind}}$ and fails first at $m = n_{\text{blind}} + 1$, so $n_{\text{blind}} \geq m$ is the condition, and the deployed $n_{\text{blind}} = 5$ exceeds the at most $m = 4$ openings of any column by one. One hypothesis of the lemma is that the points lie outside the rows: an opening *at* a row would return a cell, witness or blinder, and the halo2 book’s *Protocol Description* chapter accordingly excludes challenges in

the domain (its note 5). The deployed transcript squeezes full field elements with no rejection, and Corollary 4.10 bounds the effect.

Scope of the lemma. It is a *marginal* statement about one column. The proof opens many related polynomials under shared challenges, and the joint distribution of all their evaluations, subject to the public relations among them, the gate equation, the running-product updates, the lookup identities, is not a consequence of the lemma applied column by column. The composition is addressed under “The simulator” below, and its status is stated there.

The quotient and the random polynomial. The quotient chunks admit no blinding-row treatment: C determines them entirely, and the rows of b mean nothing. Their commitment blinders, together with the blinding-row randomness they inherit through C , protect them instead, and their single collapsed opening at x is the publicly computable value $C(x)/(x^n - 1)$, which reveals nothing the verifier did not already hold. There is one more place the quotient could leak: the multipoint reduction evaluates its $\{x\}$ -group polynomial q_1 at the fresh point x_3 , and q_1 contains the collapsed quotient, whose value at x_3 is not a public function of anything. This is why phase 4 commits a uniformly random polynomial before y and opens it at x alongside the quotient: it joins the same group with the same x_1 -weighting, so $q_1(x_3)$ carries a uniform summand, and the value $\bar{q}_1$ the prover sends is uniform whatever the quotient’s value at x_3 is. The protocol commits that random polynomial for this reason (the halo2 book’s *Protocol Description* chapter, step 3 and note 2).

Composition. In each phase the verifier sees only two kinds of object: hiding commitments, which are uniform group elements, and a bounded number of evaluations of the committed polynomials at the transcript’s challenge points. The blinding rows supply randomness which, by Lemma 4.8, makes the openings of each column uniform on its own; a full composition proof must show that this randomness masks the *joint* evaluations subject to the public relations, and must account for the correlations among advice, products, quotient and multipoint polynomials. The final inner-product opening uses the deployed mask of §3.8, the random polynomial vanishing at x_3 whose commitment S opens phase 7, in addition to the commitment and round blinders, because Remark 3.7 showed that a final commitment blinder alone would not hide the folded coefficient; the opening of one polynomial was proved statistically honest-verifier zero knowledge there, and that proof is one ingredient of the composition, not the composition.

The simulator. Zero knowledge requires a simulator for the *complete joint transcript*, given only the public input. The usual proof samples suitably blinded commitments and evaluations satisfying the verifier’s equations and then programs the random oracle so that the Fiat–Shamir challenges come out as required, in the manner of the *Crypto Guide*’s one-round simulator (§“The Fiat–Shamir transform: from interactive to non-interactive”). For the interactive protocol the halo2 book exhibits such a simulator and claims the following; this volume records the claim and does not reprove it.

Proposition 4.9 (Zero knowledge of the interactive protocol, cited). *The interactive Halo 2 protocol is perfect special honest-verifier zero knowledge, the simulator being given the verifier’s challenges in advance (the Crypto Guide, §“Sigma-protocols”), for challenge tuples in which every challenge is nonzero and outside the rows and every factor $1 + u_\ell x_3^{2^{\ell-1}}$ of $g_{\text{dep}}(x_3)$ in (26) is nonzero (the halo2 book’s Protocol Description chapter, “Zero-knowledge and Completeness”). The simulator commits to uniformly random polynomials in place of every polynomial the prover commits in phases 1 to 3 (the book’s a_i : the advice columns, the permuted lookup columns and the running products), the quotient chunks and the multipoint polynomial, and uses its foreknowledge of ξ to choose the mask $s(X)$ so that $p + \xi s - v$ vanishes at x_3 .*

Corollary 4.10 (Unrestricted challenges). *Let every challenge be uniform in $\mathbb{F}$ with no value excluded, as the deployed transcript draws them. Then the transcripts of the interactive protocol and of the simulator of Proposition 4.9, run on the same challenges, are within statistical distance $N_{\text{ch}}(n+2)/|\mathbb{F}|$, where $N_{\text{ch}} = 22$ is the number of challenges, $\theta, \beta, \gamma, y, x, x_1, \dots, x_4, \xi, \zeta$ and the $k = 11$ values u_ℓ ; for $n = 2^{11}$ this is $45100/p_{\text{Pallas}} \approx 2^{-238.5}$.*

Proof. Each challenge has at most $n+2$ excluded values: 0, the n rows, and, for u_ℓ , the value that makes its factor of $g_{\text{dep}}(x_3)$ zero. The challenges have the same distribution in both experiments; on the event that none takes an excluded value the two transcripts are identically distributed by Proposition 4.9, and by a union bound that event fails with probability at most $N_{\text{ch}}(n+2)/|\mathbb{F}|$, which therefore bounds the statistical distance. $\square$

Perfect hiding of each commitment and Lemma 4.8 are ingredients of that argument, not by themselves a proof that the joint distribution matches a real execution. For the compiled argument, the *Crypto Guide*’s compilation recipe (Theorem “The compilation recipe, informal”, §“SNARKs: succinct non-interactive arguments of knowledge”) yields zero knowledge when the polynomial IOP is properly blinded and the corresponding simulation theorem applies; for a multi-round protocol that is a multi-round simulation theorem in the programmable random-oracle model, which this volume neither proves nor cites from a source that proves it for this protocol. Non-interactive zero knowledge of the deployed proof is therefore stated, not proved.

The three properties. Subject to the hypotheses just named, Halo 2 targets three guarantees for the relation $\mathcal{R}_C$ of a PLONKish circuit (Definition 2.7), the three that §1.2 demanded of a proof. *Overwhelming completeness*: an honest prover holding a satisfying witness convinces the verifier except on negligible transcript degeneracies, a zero inner-product challenge (§3.8, probability $1/|\mathbb{F}|$ per challenge), a point x_3 among the query points of the multipoint reduction (“Completeness” in §4.4), an evaluation point x on a row, where the verifier’s division by $x^n - 1$ in phase 5 is undefined (probability $n/|\mathbb{F}|$), or challenges β, γ at which a running-product denominator factor vanishes at an earlier row than any numerator factor does (Remark 2.17; probability at most $N/|\mathbb{F}|$ for the permutation argument on N participating cells and $2u/|\mathbb{F}|$ per lookup on u active rows). *Knowledge soundness*: an efficient extractor, given rewindable access to any prover the verifier accepts with non-negligible probability, outputs a satisfying witness, so that an accepted proof certifies that one is *known*. Corollary 3.13 supplies the extractor for the opening layer; §5 lifts it to the non-interactive argument, whose challenges are transcript hashes, and classifies what

that lifting establishes. *Zero knowledge*: a witness-free simulator reproduces the proof, exactly for restricted challenges (Proposition 4.9) and within the distance of Corollary 4.10 for the deployed ones. Knowledge soundness is the property of Action proofs that the Orchard balance and nullifier arguments consume, together with the binding signature (protocol specification, § 4.14, “Balance and Binding Signature (Orchard)”) and the consensus nullifier rules (protocol specification, § 3.9, “Nullifier Sets”); zero knowledge is the property the privacy argument consumes.

4.7 Halo 2 as a compiled polynomial IOP

The protocol of §4.5 factors into two layers: a *polynomial interactive oracle proof* for the relation, and a polynomial commitment scheme, in the sense of the *Crypto Guide* (§“Polynomial commitment schemes”), that realises the proof’s oracles. Every phase of the protocol belongs to one of the two layers.

The polynomial IOP, recalled. The *Crypto Guide* defines the object (§“SNARKs: succinct non-interactive arguments of knowledge”, under “Polynomial interactive oracle proofs”): a public-coin interactive protocol in which, in place of ordinary messages, the prover in each round sends one or more *oracles* for polynomials $p_i \in \mathbb{F}[X]$ of bounded degree, the verifier answers with uniformly random field elements, and at the end the verifier *queries* each oracle at a small number of points, receiving the evaluations, and accepts or rejects as a deterministic function of the challenges, the evaluations and the public input. Completeness, soundness and knowledge soundness are those of interactive arguments, with oracle access in place of reading the messages, and the soundness analysis is purely algebraic, resting on the Schwartz–Zippel lemma. The one respect in which a polynomial IOP differs from a vanilla interactive argument is that the verifier never reads a prover message in full: it queries a few evaluations of each oracle. That is what renders the verifier succinct even when the polynomials have large degree; the message length is never read, only as many field elements as there are queries.

The compiler, recalled. The *Crypto Guide*’s compilation recipe (the same section) turns a polynomial IOP into a non-interactive argument with a polynomial commitment scheme (Setup, Commit, Open, Eval, Verify) and a random oracle H . Setup runs the commitment scheme’s setup. The prover simulates the IOP, replacing each oracle message p_i by its commitment C_i , derives every verifier challenge from the transcript by the Fiat–Shamir transform of §4.3, and supplies, for each of the IOP’s queries, the claimed evaluation together with an evaluation proof (Eval). The verifier parses the transcript, recomputes the challenges, checks each evaluation proof (Verify), and runs the IOP’s accept-or-reject check on the opened evaluations (Figure 14).

PLONK as a polynomial IOP. The IOP that Halo 2 compiles is PLONK’s, in the PLONKish generality of §2.5. Its oracles, sent by the prover, are one polynomial per advice column; for each lookup, the two permuted columns A' and S' ; the permutation running products, one per chunk, and one lookup running product per lookup; and the quotient b , split into chunks below the degree bound. The fixed columns, the selectors among them, and the permuted-label polynomials are *preprocessed*, part of the verifying key rather than prover messages, which is why they are absent from the list. The verifier’s challenges, in order, are θ for the lookup compression, β and γ for the

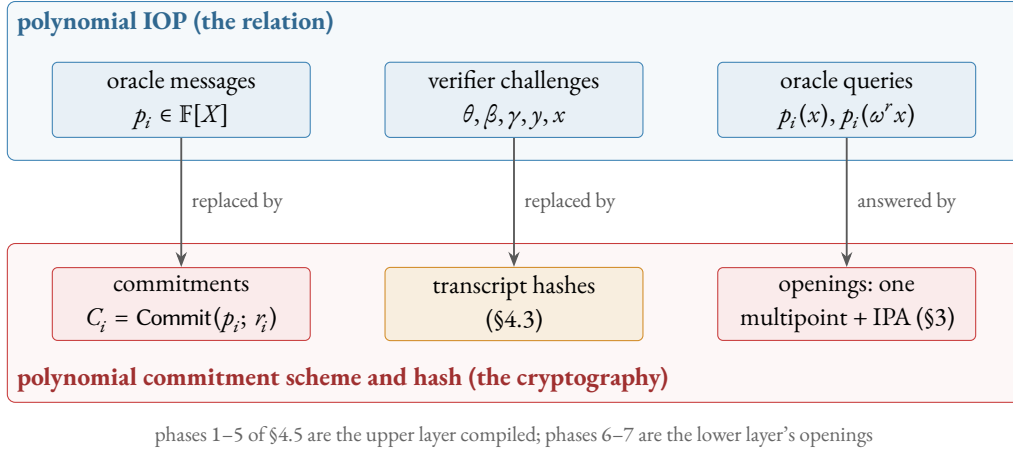


Figure 14. The compiler. The polynomial IOP (blue) is the information-theoretic layer: oracles, public-coin challenges and evaluation queries, sound by the Schwartz–Zippel lemma alone. The cryptographic layer (red and amber) realises it: each oracle becomes a commitment, each challenge a transcript hash, each query an opening, and Halo 2 collapses every opening into one. The two layers are separable: the same IOP compiles with any commitment scheme meeting the hypotheses of Remark 4.11.

permutation and lookup products, y for the combination of all constraints into one gate equation, and x for the evaluation point. The final check is the gate equation $C(x) = b(x) Z_H(x)$, with C the y -combination of every gate, permutation and lookup constraint polynomial and Z_H the vanishing polynomial of the rows, which the verifier evaluates from the oracles' values at x and at the rotations its constraints read: ωx or $\omega^{-1}x$ for an adjacent row, and $\omega^{-6}x$, the rotation to the boundary row of Remark 2.17, for the chunk-boundary identities of the permutation argument.

The map onto the phases. Phases 1 to 5 of §4.5 are this IOP compiled, with one addition: the commitments to the advice, the permuted lookup columns, the running products and the quotient chunks are the oracle messages replaced by commitments, and the random polynomial of phase 4, committed and evaluated at x , is the compiler's addition for zero knowledge (§4.6), not an IOP message; the challenges are the IOP's challenges squeezed from the transcript, and the claimed evaluations of phase 5 answer the IOP's oracle queries. Phases 6 and 7 bind those answers to the commitments: the multipoint reduction of §4.4, with its challenges $x_1, \dots, x_4$ and combined polynomial f^* , followed by the single inner-product opening of f^* . That is the bridge between the abstract compiler and the concrete protocol, and the phase numbering is the one Table 10 uses. Under the hypotheses of the *Crypto Guide*'s compilation recipe (Theorem “The compilation recipe, informal”) and of Remark 4.11, the compiled protocol has the IOP's relation and the commitment scheme's assumptions, transparency, proof size and verifier cost.

Remark 4.11 (The compilation principle, and what it does not say). Soundness of the compiled argument is not the sum of two numbers, and the notions under which it is proved deserve their names. A polynomial IOP is *round-by-round* knowledge sound, informally, when its partial transcripts admit a “doomed” labelling that a prover message cannot escape except with bounded probability over the next verifier challenge; a related *state-restoration* notion lets a prover restore an earlier verifier state and redraw its challenge, which is precisely what a prover

attacking a transcript hash can do. Precise equivalence or implication theorems between the two notions require additional conditions on the protocol and on the extractor, and this volume does not identify them. The *Crypto Guide*'s compilation recipe (Theorem “The compilation recipe, informal”, §“SNARKs: succinct non-interactive arguments of knowledge”) assumes a complete, knowledge-sound public-coin polynomial IOP; a correct, evaluation-binding and extractable commitment scheme; compatible composition hypotheses; and a suitable multi-round Fiat–Shamir theorem in the random-oracle model. For a multi-round protocol such as Halo 2 this volume reads those hypotheses as three: the IOP has a round-by-round or state-restoration knowledge-soundness theorem; the commitment scheme is extractable and evaluation-binding in a form compatible with it; and the transcript satisfies the hypotheses of an appropriate multi-round Fiat–Shamir theorem. The conclusion is that the compiled protocol is a non-interactive argument of knowledge in the random-oracle model, by layered extraction: the commitment scheme's extractor recovers the committed polynomials, converting the argument's prover into an IOP prover; the multi-round Fiat–Shamir theorem transfers the round-by-round or state-restoration guarantee to the non-interactive setting; and evaluation binding ensures that the opened evaluations are those of the extracted polynomials. What it does *not* say is that the knowledge error is determined by two scalars, the IOP's knowledge error and the commitment scheme's extraction error, added together: the exact error and the query loss depend on the definitions and on the extraction model, and §5 names the theorems that fix them for the analysed protocol and classifies their instantiation to the deployed transcript.

Delegated results. The subsection proves nothing afresh. The commitment interface (Setup, Commit, Open, Eval, Verify) with its hiding, binding and extraction properties is the *Crypto Guide*'s (§“Polynomial commitment schemes”); the Fiat–Shamir transform and its random-oracle security are the same volume's (§“The Fiat–Shamir transform: from interactive to non-interactive”); the definitions of completeness, soundness and knowledge soundness for interactive arguments used above are its (§“Interactive proofs, zero knowledge, and SNARKs” and §“Knowledge soundness and extractors”); and the Schwartz–Zippel lemma and the vanishing polynomial of the rows are the *Math Guide*'s (§“The Schwartz–Zippel lemma” and §“The evaluation domain H and its vanishing polynomial”). The volume adds the instantiation: which oracles, which challenges, which openings, and the one commitment scheme that realises them.

5 A rigorous treatment of knowledge soundness

Theorem 3.5 and Corollary 3.13 proved the inner-product opening knowledge sound for a verifier who draws his challenges: an extractor that runs the prover as a subroutine, rewinds it and feeds it fresh challenges recovers the committed vector or a nontrivial discrete-logarithm relation among the generators, which the hardness of the discrete logarithm on Vesta excludes. The deployed verifier draws nothing; every challenge is a hash of the transcript (§4.3), and Remark 4.11 named the hypotheses under which a compiled argument inherits its layers' guarantees without fixing them. This section determines what the extractor establishes when every challenge is a hash of the transcript. It combines the rewinding-extractor framework of the *Crypto Guide*, the definition of knowledge soundness (§“Knowledge soundness and extractors”), the tree extraction of Sigma-

protocols (§“Sigma-protocols”) and the Fiat–Shamir theorem with its stated caveats (§“The Fiat–Shamir transform: from interactive to non-interactive”), with the algebraic group model, named among the idealisations of §“The standard model and the random oracle model”. It lifts the interactive extractor to the non-interactive argument in two layers and names the loss of the naive lifting; it states the model in which the halo2 book proves its extraction theorem and classifies what that theorem does and does not establish for the deployed transcript; and it closes on the gap between concrete and asymptotic security. The running-time vocabulary, polynomial, negligible and PPT, is the *Math Guide*’s (§“Polynomial, exponential, and negligible functions” and §“Algorithms, running time, and PPT”); expected polynomial time is the *Crypto Guide*’s (§“Adversaries and the security parameter”, the remark on expected versus strict polynomial time); tightness and loss are the *Crypto Guide*’s (§“Reductions”, §“Concrete security and the bit-security budget”); the assumption is its standing one (§“Standing assumptions”).

5.1 From the interactive extractor to the non-interactive argument

What the interactive extractor costs. Theorem 3.5 extracts from a tree of accepting transcripts with three children at every node and one level per round: 3^k leaves for $k = \log_2 d$ rounds, that is $d^{\log_2 3}$ with $\log_2 3 = 1.58496 \dots$, polynomial in d though more than a binary tree would cost. At the deployed $d = 2^{11} = 2048$ the tree has $3^{11} = 177147$ leaves against the $2^{11} = 2048$ a binary tree would have; the extractor runs in expected polynomial time, the rewinding that assembles the tree being the *Crypto Guide*’s (§“Sigma-protocols”, the tree-extraction theorem cited there). Its knowledge error is the $O(\log d/|\mathbb{F}|)$ of Theorem 3.5, one $O(1/|\mathbb{F}|)$ term per round for the challenges the siblings must avoid. The scale of these terms is worth fixing once: the proof system’s own allowances are linear in $n = d$, the $18431/|\mathbb{F}| \approx 2^{-239.8}$ of the identity check (§2.4), whose difference polynomial $C - bZ_H$ has degree at most $d_{\max} n - 1 = 18431$ at the deployed $d_{\max} = 9$ ($\deg C \leq d_{\max}(n - 1)$ by the proof of Theorem 4.7, and the $d_{\max} - 1$ committed chunks of the quotient have degree $< n$ each, §4.5); the order- $n/|\mathbb{F}|$ term of the multipoint reduction (Lemma 4.5); and $d/|\mathbb{F}| = 2^{11}/2^{254} = 2^{-243}$ is that linear allowance for the opening. The opening’s proved error sits below it, and quoting the linear figure for the opening loses nothing in a system whose other terms are already linear in d , as the proof of Theorem 3.5 noted. Every one of these is a Schwartz–Zippel term, not a security level, in the sense fixed in §1.2.

Layered extraction. The compiled argument of §4.7 was assembled in two layers, an information-theoretic polynomial IOP and a cryptographic commitment scheme, and its extractor is assembled in the same two layers, run backwards (Figure 15).

The lower layer recovers polynomials. The prover has sent commitments C_y and, in phases 6 and 7, one opening of the combined $F^* = F + \sum_i [x_4^i] Q_i$ with $Q_i = \sum_e [x_1^e] C_{y_{i,e}}$ (Construction 4.4). Phase 7 runs the deployed opening of §3.8, and the extractor of Corollary 3.13, the tree extraction of Theorem 3.5 with two further levels, ξ and ζ , the latter in the root role of the resampled U , returns the coefficient vector behind F^* , a representation of F^* over $(\mathbf{G}, H)$, or a nontrivial discrete-logarithm relation among $\mathbf{G}$, H and U . To reach the individual commitments the extractor rewinds over the combining challenges as well: the vectors behind F^* for $m + 1$ distinct values of x_4 are $m + 1$ linear equations in the $m + 1$ unknown vectors behind $F, Q_1, \dots, Q_m$, whose coefficient matrix has the rows $(1, x_4, \dots, x_4^m)$ and is invertible because a nonzero polynomial of degree

at most m has at most m roots (*Math Guide*, §“Roots and the factor theorem”), the argument of the proof of Lemma 3.4; the vectors behind Q_i for $|I_i|$ distinct values of x_1 likewise yield the vectors behind each C_v in group i . One C_v is the collapsed quotient $\sum_i [x^{ni}] \text{Commit}(b_i)$ of §4.5; its vectors for $d_{\max} - 1$ values of x with pairwise distinct x^n yield the vectors behind the chunks b_i , by the same argument with x^n in place of x_4 . The output is one polynomial of degree $< n$ per commitment the prover sent, and two different vectors extracted for one commitment would be a nontrivial discrete-logarithm relation among $\mathbf{G}$ and H , exactly as at the root of Theorem 3.5. This layer is the only place the discrete logarithm on Vesta enters, and it enters in only one way: the extractor either recovers the polynomials or exhibits a relation.

The upper layer recovers the witness. With the polynomials in hand, the prover’s claimed evaluations are the true evaluations of fixed polynomials except with the Schwartz–Zippel allowance of Lemma 4.5, since a claimed value that differs from the extracted polynomial’s value survives the reduction only with that probability; and “the true evaluations of fixed polynomials” is precisely the hypothesis under which §2 proved its theorems. Theorem 2.5 turns the accepted identity $C(x) = b(x) Z_H(x)$ into divisibility of C , Lemma 4.6 splits it into the vanishing of every constraint polynomial on the rows, Theorem 2.11 with Corollary 2.12 turns the running products into the wiring, and Theorem 2.15 with Proposition 2.16 turns the lookup identities into table membership, each with its stated term over $|\mathbb{F}|$; Theorem 2.2 reads gate satisfaction off divisibility. The witness is then read off directly: the advice columns are the values of the extracted advice polynomials on the rows. Nothing in this layer uses an assumption; the IOP’s extractor outputs the oracles, which the layer below supplies. The consequence runs both ways. A break of the discrete logarithm on Vesta breaks the lower layer and nothing else, in the way §3.5 described, one commitment opening to any vector; and no assumption can rescue an identity that the upper layer does not actually enforce, because the upper layer’s errors are counting statements about polynomials.

What is proved and what is described. In the interactive setting the composed extractor is a tree extractor in the sense of the *Crypto Guide* with a wider tree: three children at each of the k inner-product levels, two at the ζ level and two at the ξ level, $m + 1$ at the x_4 level, $\max_i |I_i|$ at the x_1 level, one x_1 serving every group, and, at the x level, $(d_{\max} - 2)n + 1 = 14337$ children: each value of x^n has at most n preimages, so only that many distinct challenges guarantee the $d_{\max} - 1$ pairwise distinct x^n the quotient step needs, as Theorem 3.5 took five children for three distinct squares. The two ζ children must moreover be nonzero, as in Corollary 3.13. Each polynomial the upper layer consumes is fixed by a binding commitment sent before every challenge that tests it: the advice before θ , the permuted lookup columns A' and S' before β and γ , the running products and the random polynomial before y , the quotient chunks before x (§4.5); the polynomial behind F and the mask behind S , committed after x , serve only inside the lower layer; and all of the consumed ones precede x , the first challenge at which the tree branches, so the upper layer’s Schwartz–Zippel arguments apply on every branch without further rewinding. The inner-product part of that tree is Corollary 3.13, proved; the theorems of the upper layer are proved in §2; the accounting that joins them into one tree-extraction statement with one knowledge error is the compilation principle of Remark 4.11, whose hypotheses this volume names and does not discharge. In particular the compiled error is not asserted to be the sum of the layers’ errors.

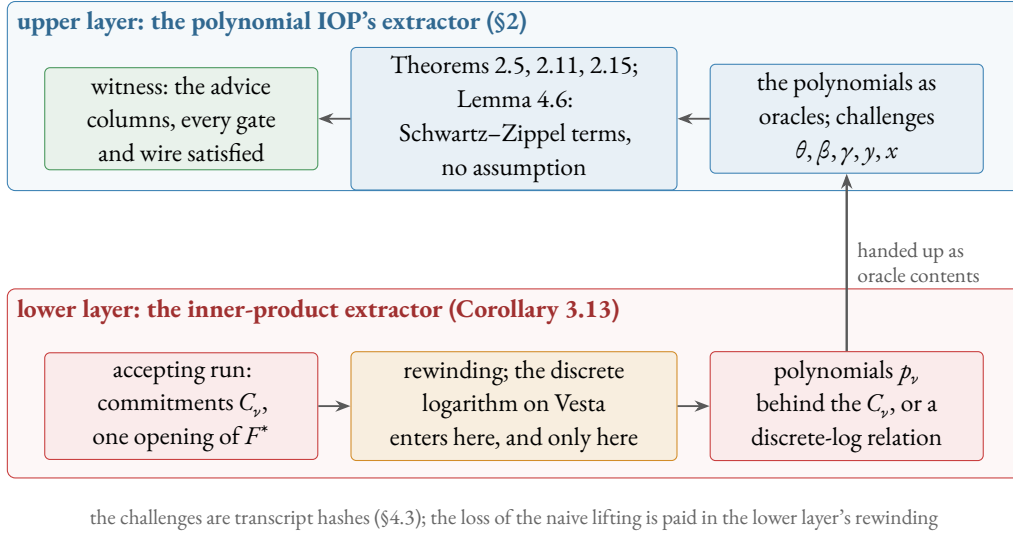


Figure 15. Layered extraction. The lower layer (red) is the cryptographic one: from an accepting run, the extractor of Corollary 3.13, rewinding over the opening’s challenges, the combining challenges of the multipoint reduction and the evaluation challenge x , recovers the polynomial behind every commitment or exhibits a discrete-logarithm relation; the assumption (amber) lives here and nowhere else. The upper layer (blue) is the information-theoretic one: with the polynomials as oracle contents, the theorems of §2 read the witness (green) off them with Schwartz–Zippel terms only.

The naive lifting, and its loss. Everything above is interactive: the extractor chose the challenges it fed the prover. In the deployed argument the challenges are hashes, and the extractor of the *Crypto Guide*’s Fiat–Shamir theorem obtains its second transcript by *forking*: it finds, among the prover’s at most Q hash queries, the critical one whose answer became the challenge, rewinds to it, and answers it afresh, succeeding with probability about $\varepsilon(\varepsilon/Q - 1/|\mathbb{F}|)$, so that the extractor’s success is the prover’s squared and divided by Q (§“The Fiat–Shamir transform: from interactive to non-interactive”, the knowledge-soundness clause of its theorem). That theorem covers one challenge. The tree of Theorem 3.5 needs three siblings at each of k levels, and a sibling at level j must share the transcript prefix through level $j - 1$, which in the hashed setting means agreeing with the prover’s prefix and its earlier hash answers: in a transcript hash, changing one message redraws *all* later challenges, so siblings at round j can be found only among the prover’s own queries. Let μ be the number of verifier challenges of the protocol being lifted. The simplest lifting guesses, among the at most Q queries, the μ that answered them, and runs the interactive extractor against the interactive prover that the guess defines; the guess is right with probability at least $Q^{-\mu}$, so the knowledge error is multiplied by Q^{μ} . Attema, Fehr and Kloofß (*Fiat–Shamir Transformation of Multi-Round Interactive Proofs*) record this Q^{μ} as the loss for a general $(2\mu + 1)$ -move protocol; nesting the one-round forking level by level fares worse, since each level squares the success probability before dividing by Q . For the opening alone μ exceeds $k = \log_2 d$, the choice of U (in the deployed opening, ξ and ζ) being a challenge too, and μ grows once the proof system’s own challenges are counted. The loss belongs to this analysis. For $(k_1, \dots, k_{\mu})$ -special-sound protocols (*Crypto Guide*, §“Sigma-protocols”, the definition of tree special soundness) the same authors prove, in the random oracle model and with no idealisation of the group, a knowledge er-

ror that grows only linearly in Q . The rounds of the opening’s extraction tree have that shape, but its root step, which rewinds the choice of U (in the deployed opening, ξ and ζ), and its alternative output, a discrete-logarithm relation, are not checked here against that theorem’s hypotheses, and this volume does not assess that route.

With the deployed $k = \log_2 d = 11$, so $\mu > 11$ for the opening alone, and a query budget of $Q = 2^{80}$ hash evaluations, the factor Q^μ exceeds $Q^{11} = 2^{880}$; multiplied by the linear allowance $d/|\mathbb{F}| = 2^{-243}$ it exceeds 2^{637} , a bound above 1, which is no bound at all. Nor does the proved $O(\log d/|\mathbb{F}|)$ help: the challenge space is a 255-bit field with fewer than 2^{255} elements, so 2^{880} exceeds $|\mathbb{F}|$ itself, and a factor larger than $|\mathbb{F}|$ turns every term of the form $c/|\mathbb{F}|$ with $c \geq 1$ into a number above 1. Asymptotically the loss falls elsewhere. With Q and d polynomial in the security parameter λ (*Crypto Guide*, §“Adversaries and the security parameter”) and $|\mathbb{F}| = 2^{\Theta(\lambda)}$, the factor $Q^\mu = 2^{O(\log^2 \lambda)}$ is super-polynomial and sub-exponential in the *Math Guide*’s classification (§“Polynomial, exponential, and negligible functions”). The Schwartz–Zippel terms survive it: $Q^\mu O(\log d)/|\mathbb{F}| = 2^{O(\log^2 \lambda) - \Theta(\lambda)}$ is negligible. The reduction does not survive it: the guessing extractor succeeds only with probability reduced by Q^μ , or runs Q^μ times longer to compensate, and so is not PPT, and the prover’s success is then bounded only by Q^μ times the discrete-logarithm advantage, which need not be negligible when that advantage is merely negligible. The interactive extractor is polynomial, $d^{\log_2 3}$ transcripts in expected polynomial time; the looseness arises in the naive analysis of the passage to the non-interactive argument.

What the tighter notions are for. Remark 4.11 named the two notions built to avoid this: *round-by-round* knowledge soundness, which labels partial transcripts as doomed and bounds the probability that one verifier challenge lifts the doom, and *state-restoration* soundness, which lets the prover restore an earlier verifier state and redraw its challenge, precisely the freedom a prover attacking a transcript hash has. Each exists to obtain a bound on the compiled argument that loses a factor linear in Q rather than Q^μ ; no equivalence between them is asserted here, as none was there. Claiming a single factor Q for a compiled protocol nevertheless requires the precise theorem’s state-restoration, transcript, challenge and extraction hypotheses to hold of that protocol, and this volume does not assert a factor- Q bound for Halo 2’s compiled protocol. The next subsection names the model in which the halo2 book proves such a bound, in the state-restoration form, for its abstract interactive protocol, what that analysis assumes, and why its instantiation to the deployed transcript remains unwritten.

5.2 The algebraic group model

The nested forking of the previous subsection rewinds because the extractor has no other way to learn what is behind a group element: three accepting continuations are the price of one representation. The *algebraic group model* of Fuchsbauer, Kiltz and Loss (*The Algebraic Group Model and its Applications*) removes the need by restricting the adversary’s group operations, and it enables tighter analyses for protocols that suit it. It is not by itself a theorem eliminating every Fiat–Shamir loss; what it eliminates is the rewinding, and what remains must still be proved.

Definition 5.1 (Algebraic adversary). An *algebraic adversary* against a group G is a PPT algorithm that, whenever it outputs a group element $P \in G$, additionally outputs a vector $\mathbf{a}$ of scalars with

$$P = \langle \mathbf{a}, \mathbf{Y} \rangle,$$

where $\mathbf{Y}$ is the vector of group elements it has previously seen: the generators it was given and the elements it has received. The vector $\mathbf{a}$ is the *representation* of P .

Where the model sits. The algebraic group model sits between the standard model, in which the adversary is any PPT algorithm and every primitive is concrete code it may inspect (*Crypto Guide*, §“The standard model and the random oracle model”), and the generic group model, in which the adversary may perform group operations only through an oracle on opaque handles and never exploits the representation of an element (the same section, the remark on the generic and algebraic group models). An algebraic adversary is less restricted than a generic one, since it may compute on the bit representation of group elements however it likes; it must only be able to explain each element it outputs as a combination of elements it has seen. The model is an idealisation (the same remark): a proof in it says nothing about an adversary that outputs a group element it cannot explain. Figure 16 draws the three models as nested restrictions on the adversary and places this section’s analyses on them; the random oracle model, which idealises the hash rather than the group, is an independent axis, and an analysis may combine either group model with it.

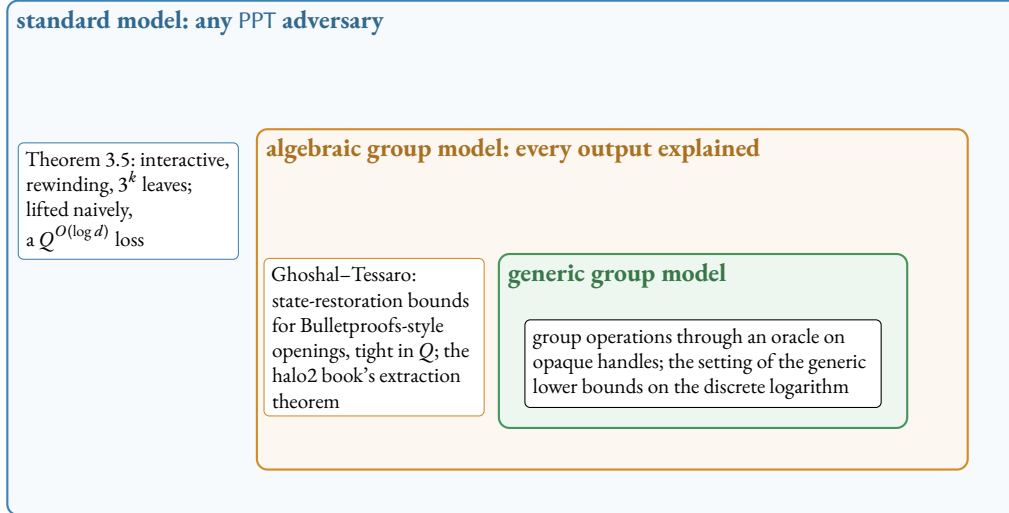


Figure 16. Three models as nested restrictions on the adversary. Every adversary admitted by an inner ring is admitted by the rings around it, so a proof in an outer ring is the stronger statement and a bound in an inner ring is the easier to make tight. The interactive extractor of Theorem 3.5 lives in the standard model for uniform generators; the tight state-restoration bounds of Ghoshal and Tessaro, and the halo2 book’s extraction theorem that invokes them, live in the algebraic group model (amber). The generic group model (green) is where the discrete logarithm’s own lower bounds are proved.

Online extraction in the algebraic group model. In this model the prover outputs every group element with its representation, and the extractor reads the witness off the representations of a single accepting transcript instead of assembling it from rewound ones; no rewinding occurs, and the price is the assumption that the prover is algebraic. For the deployed opening, equation (29), the halo2 book’s *Protocol Description* chapter carries this out (“Witness-extended Emulation”, the definition of the bad-challenge sets): comparing the two sides of the accepting equation coordinate by coordinate either exhibits a nontrivial discrete-logarithm relation among the generators or, unless some challenge falls in a set of *bad challenges* fixed by the messages before it, forces the representation of P to open to the claimed value. The analysed protocol draws every challenge from one space $\mathcal{C}$ of field elements, none zero or a row, and each round challenge u_j further so that its factor $1 + u_j x_3^{2^{j-1}}$ of $g_{\text{dep}}(x_3)$ in (26) is nonzero. Each round challenge u_j has at most $4n$ bad values in $\mathcal{C}$, and ζ and ξ have one each; the book bounds the bad sets of the proof system’s other challenges, $x_1, \dots, x_4, x$ and those of the committed rounds, by the degrees of the identities they test, the largest being $n_g(n-1)$, the bound on the constraint polynomial’s degree in each indeterminate, provided $n_g(n-1) \geq 4n$, as at the deployed $n_g = 9$, where $18423 > 4n = 8192$; the book admits $n_g \geq 4$, for which this can fail.

Ghoshal and Tessaro, and the deployed stance. Ghoshal and Tessaro (*Tight State-Restoration Soundness in the Algebraic Group Model*) carry this analysis out for Bulletproofs-style inner-product arguments: they prove tight state-restoration soundness in the algebraic group model and derive Fiat–Shamir security bounds from it under their stated hypotheses, the transfer from the state-restoration game to the compiled argument being their theorem. The deployed analysis follows them. The protocol specification defines the proving system by reference to the halo2 book (protocol specification, § 5.4.10.3, “Halo 2”), and the book’s *Protocol Description* chapter fixes the framework: it adopts state-restoration soundness because the concrete protocol is realised through the Fiat–Shamir transformation and a cheating prover can fork the transcript at will, and it invokes Ghoshal and Tessaro’s theorem for the passage from that game to soundness of the compiled argument. Its abstract interactive protocol is stated for a relation of the form “a fixed multivariate constraint polynomial in the column polynomials and the challenges vanishes on the rows”, with the column polynomials as the witness, over public parameters of $n+2$ group elements, $\mathbf{G}$, U and the blinding generator H (written W there).

Definition 5.2 (State-restoration witness-extended emulation). Let Π be a public-coin interactive argument for a relation $\mathcal{R}$ with r challenges from a space $\mathcal{C}$. A *state-restoration prover* $\mathcal{P}^*$, given the public parameters, outputs a statement ϕ and then queries an oracle on pairs (τ, a) , where τ is a transcript prefix the oracle has recorded (initially only the empty one) and a is a next prover message; it may thus extend any earlier prefix. In the *real* game the oracle answers a message of rounds 1 to r with a fresh uniform $c \in \mathcal{C}$ and records (τ, a, c) , and answers a final message with the verifier’s decision on (τ, a) , recording (τ, a) . In the *ideal* game an algorithm $\mathcal{E}$, given the public parameters and ϕ and keeping state across queries, receives each query (τ, a) , for an algebraic prover together with the representations (Definition 5.1) of the group elements in a , returns the answer, which is recorded with the query, and at the end outputs w . For a dis-

tinguisher $\mathcal{D}$ given the recorded transcripts,

$$\text{Adv}_{\Pi, \mathcal{R}}^{\text{srwee}}(\mathcal{P}^*, \mathcal{D}, \mathcal{E}) := \Pr[E_{\text{real}}] - \Pr[E_{\text{ideal}}],$$

where E_{real} is the event that $\mathcal{D}$ outputs 1 in the real game, and E_{ideal} the event that $\mathcal{D}$ outputs 1 in the ideal game and, if an accepting transcript was recorded, $(\phi, w) \in \mathcal{R}$.

Theorem 5.3 (Extraction for the abstract protocol; interface). *Let Π be the halo2 book’s abstract interactive argument for its relation $\mathcal{R}$, over a group $\mathbb{G}$ of prime order with scalar field $\mathbb{F}$, for n rows, a positive integer n_g with $n_g(n-1) \geq 4n$ and a constraint polynomial of degree at most $n_g(n-1)$ in each indeterminate, with challenges from $\mathcal{C}$. There is an extractor $\mathcal{E}$ such that for every non-uniform algebraic state-restoration prover $\mathcal{P}^*$ making at most Q queries to its oracle there is an adversary $\mathcal{B}$ with*

$$\text{Adv}_{\Pi, \mathcal{R}}^{\text{srwee}}(\mathcal{P}^*, \mathcal{D}, \mathcal{E}) \leq Q \varepsilon_{\text{bad}} + \text{Adv}_{\mathbb{G}, n+2}^{\text{dlrel}}(\mathcal{B}), \quad \varepsilon_{\text{bad}} \leq \frac{n_g(n-1)}{|\mathcal{C}|},$$

for every computationally unbounded distinguisher $\mathcal{D}$. Here ε_{bad} bounds the fraction of bad challenges after every partial transcript, and $\text{Adv}_{\mathbb{G}, n+2}^{\text{dlrel}}(\mathcal{B})$ is the probability that $\mathcal{B}$, given $n+2$ uniform elements of $\mathbb{G}$, outputs a nontrivial discrete-logarithm relation among them.

The theorem is quoted as an interface and not proved in this volume: it is the halo2 book’s (Protocol Description, “Witness-extended Emulation”), proved there by instantiating Theorem 1 of Ghoshal and Tessaro with the bad-challenge sets described above. Its extractor works online, from the representations of a single transcript, and never rewinds. At the deployed $n_g = d_{\text{max}} = 9$ and $n = 2^{11}$, $\varepsilon_{\text{bad}} \leq 18423/|\mathcal{C}| \approx 2^{-239.8}$, and $Q \varepsilon_{\text{bad}} \approx 2^{-159.8}$ at $Q = 2^{80}$ queries.

The instantiation gap, classified. Applying those bounds to the deployed Halo 2 transcript requires matching the precise folding, masking, challenge distribution and composition assumptions of the analysed protocol to the deployed one, and this volume does not prove that instantiation theorem. Four mismatches are visible from the earlier sections. The book’s abstract relation sees the permutation and lookup arguments of §2 only through the constraint polynomial and its committed columns; the Orchard circuit’s actual gates and table columns (§4.1) and its chunked running products (“The deployed chunking” in §2.6) instantiate that abstraction and must be checked against its degree hypotheses. The book’s challenges exclude zero, the rows and, for each u_j , the value that zeroes its factor of $g_{\text{dep}}(x_3)$, while the deployed transcript squeezes full field elements with no rejection (§3.8), so the deployed challenge distribution differs from the analysed one by the excluded events. The analysed folding is the deployed normalisation of Remark 3.9, and the masking is the deployed mask of §3.8, so those two match; the hash, however, is BLAKE2b (§4.3), and the transfer theorem’s random oracle must be instantiated by it. And the book’s theorem is for the interactive argument against a state-restoration prover, the compiled bound following by the transfer theorem under its hypotheses. In the classification of §1.2, the status of the transfer is *designed-but-unspecified*: the bounds exist for the analysed protocol and the design intends them for the deployed one, but the theorem that carries them across is not written, here or in the specification.

5.3 Concrete versus asymptotic security

The *Crypto Guide* distinguishes a reduction's existence from its tightness (§“Reductions”, the definition of loss and the remark on why loss matters concretely) and reads security as a budget of bits (§“Concrete security and the bit-security budget”): a scheme inherits the strength of its assumption only up to the loss of the reduction that connects them. Two tightness gaps separate the proved statements from the deployment.

The first gap: the naive lifting. The naive Fiat–Shamir knowledge-soundness analysis of the opening loses the factor $Q^u > Q^{\log d}$ of §5.1, exponential in the round count $\log d$ with base the query count Q : above 2^{880} at the deployed size for a 2^{80} -query prover, against a challenge space of fewer than 2^{255} elements. The interactive extraction itself is polynomial, $d^{\log_2 3} = 177147$ transcripts at $d = 2048$ in expected polynomial time, with the proved knowledge error $O(\log d / |\mathbb{F}|)$ of Theorem 3.5. The gap lies in the naive analysis of the lifting, and the notions of §5.1 and the model of §5.2 exist to close it.

The second gap: the model. Each analysis cited in this section that avoids the naive loss relies on an idealisation: the halo2 book's extraction theorem (Theorem 5.3) on the algebraic group model, and its passage to the compiled argument on the random oracle model as well; the special-soundness analysis of Attema, Fehr and Klooß on the random oracle model alone. No standard-model tight bound is claimed for the deployment, by this volume or by the analyses it cites; this is a statement about the design's analyses, not a specification claim, and the specification makes none. Read against the bit-security budget, the consequence is concrete. The discrete-logarithm layer offers about 2^{126} operations of classical security (§3.5; *Crypto Guide*, §“Standing assumptions”); a reduction tight in Q lets the argument inherit that strength, less the algebraic term $Q \varepsilon_{\text{bad}}$ of Theorem 5.3, about $2^{-159.8}$ at 2^{80} queries, which is not the bottleneck; a reduction losing 2^{880} lets it inherit nothing. For the analysed protocol the number is therefore the assumption's, in the algebraic group and random oracle models; for the deployed argument it is so only through the instantiation that §5.2 classifies designed-but-unspecified.

The stance. Proved without idealisation of the adversary or of the transcript hash are the interactive statement, Theorem 3.5, for uniform generators (the deployed hashed generators add its hypothesis that hash-to-curve is a random oracle), and the information-theoretic layer of §2; described, with the hypotheses named, are the layered extraction that joins them and the naive lifting's loss; cited, with its instantiation to the deployed transcript classified designed-but-unspecified, is the halo2 book's bound in the algebraic group model.

6 The verifier's decision and its implication chain

This section adds no theorem: it lists the verifier's checks in the order he performs them, assembles the implication chain from an accepted proof back to the claim, each link labelled by the statement that proves it and by its error term, repeats the chain for the Orchard Action circuit, and maps the five reductions of §1.2 to the formal statements that own them. Nothing here depends on the recursion of §7, which Orchard does not deploy.

6.1 The verifier's checks, read in order

What he holds. The verifier of the toy holds the fixed data of the circuit, the five selector polynomials and the permuted-label polynomials of Construction 2.10; the public input, the column $(0, 0, 0, -35)$ interpolated to $\text{PI}(X)$ (§4.2); and the proof. The proof is a byte string (§4.3): commitments, claimed evaluations, and one opening, in the order of the phases of §4.5 (Table 10), counted by kind in Table 8. He never sees a column polynomial, a blinder or the advice; he sees curve points and field elements, and he performs the checks below in this order.

1. **The challenges.** He first commits to the instance column himself, from the public input and with the fixed blinder 1 (§4.2), so that the public input never comes from the prover. He then replays the transcript (§4.3): he absorbs the key digest, that instance commitment and each prover message in turn, and squeezes every challenge at its prescribed point. In the toy the point $z = 20$ was drawn by decree; in the deployed protocol the evaluation point x is a hash of everything before it.
2. **The identity at the point.** From the claimed evaluations and the fixed polynomials he evaluates himself, he forms both sides of the gate identity. In the toy this is the one field equation (5): $G(20) = 75$ against $t(20) Z_H(20) = 67 \cdot 46 = 75$, accepted; the $c_0 = 10$ cheat of Example 2.3 gives 20 against 64, rejected. In the deployed protocol the identity is $C(x) = b(x) Z_H(x)$, with C the y -combination of every constraint polynomial and b the chunked quotient, and it is enforced by opening the collapsed quotient commitment at x to the value $C(x)/Z_H(x)$ that the verifier computes himself from the evaluations of phase 5, inside the multipoint opening (phases 6 and 7 of §4.5).
3. **The wiring.** He evaluates the boundary and update identities of the permutation argument (7) at the point and, in the deployed form, the chunk-boundary identities of §2.6 and the boundary identity $\ell_{\text{last}}(Z^2 - Z)$ of Remark 2.17. In the toy the ratio of the two products (6) at $\beta = \gamma = 2$ is 1 for the honest trace and 61 for the tampered $b_1 = 4$ (Example 2.8).
4. **The tables.** He evaluates the four lookup identities (9) and, in the deployed form, the boundary identity $\ell_{\text{last}}(Z^2 - Z)$ of Remark 2.17 at the point, five for each lookup argument. The toy's grid has none; the two-bit check of Example 2.13 showed the shape, with both products equal to 82 and the forged 7 stranded in every arrangement. In the deployed protocol items 3 and 4 are not separate checks: their identities are summands of C and are enforced by the one check of item 2; only the toy checks them separately.
5. **The openings.** Every evaluation he received for the three checks above was *claimed*. He now binds each claim to its commitment: the multipoint reduction of Construction 4.4 folds every claim into one statement, and the inner-product equation (29) discharges it with one multi-scalar multiplication of length n . In the toy, the opening of $a(X)$ at $z = 20$ to the value 59 closes at $(54, 39)$ on both sides (Table 5) and the false value 60 moves the left-hand side to $(62, 15)$; the deployed opening of the same commitment closes at $(5, 72)$ and rejects 60 likewise (Table 7). Among the claims is $\pi(x)$, which the opening binds to his own instance commitment (§4.2).

If every check passes he accepts. The accepted checks imply the following chain, read backwards from the last check to the first; Figure 17 draws it.

The chain, read backwards.

1. *The identity check passed at his random point.* Because the commitment binds, the polynomials behind the openings were fixed before the point was chosen: the Pedersen vector commitment is binding under the discrete logarithm on Vesta (the *Crypto Guide*, §“Pedersen vector commitments”, recalled in §2.9), and the opening argument is knowledge sound (Theorem 3.5 and Corollary 3.13), so from any prover who passes the opening an extractor recovers the committed vector and the value it opens to, or else a discrete-logarithm relation among the hashed generators; its knowledge error is $O(\log d/|\mathbb{F}|)$, and the multipoint reduction that fed it adds one Schwartz–Zippel allowance of order $n/|\mathbb{F}|$ (Lemma 4.5). With a hash in place of coins the ordering holds by construction, every commitment being an input of the hash that produced the challenge testing it (§4.3). Because the point was random and a false identity disagrees with the true one at all but $\deg D$ points, passing supports $G(X) = t(X) Z_H(X)$ as polynomials except with the error $\deg D/|\mathbb{F}|$ of Theorem 2.5 for this fixed check: 11/97 for the toy’s degree budget, 3/97 for the particular cheat of Example 2.6, about 2^{-240} at Orchard scale, one term in the sense §1.2 fixed. In the deployed form the identity is $C = b Z_H$, and the challenge y that combined every constraint into C costs one more term, $(T - 1)/|\mathbb{F}|$ for T constraint polynomials (Lemma 4.6).
2. *The identity holds as polynomials.* Then $Z_H \mid G$, so G vanishes on every row (Theorem 2.2, equivalence (3)), so every gate equation holds at every row (Theorem 2.1). Both steps are exact equivalences with no error term: in the toy, the four rows of Table 1 each satisfy (1). In the deployed form every constraint polynomial combined into C vanishes on the active rows: every custom gate, and the permutation and lookup identities that the next two links consume.
3. *The permutation identities passed.* Except with the error $2N/|\mathbb{F}|$ of Theorem 2.11 over β, γ , with N the number of participating cells, twelve in the toy, and, in the deployed form, $N/|\mathbb{F}|$ more for the challenges at which a product factor vanishes (Corollary 2.12), the copy constraints hold: the per-row gates are wired into one coherent computation, the witness 3 of row 0 is the witness of rows 1 and 2, and each row’s output is the next row’s input. The lookup identities passed likewise: except with the error $4n/|\mathbb{F}|$ of Theorem 2.15, and $n(m - 1)/|\mathbb{F}|$ more for the θ -compression of (10) (Proposition 2.16) when a lookup has $m > 1$ columns, every looked-up value is in its table.
4. *The public input fixed the target at 35.* In the toy, 35 enters the gate polynomial G through $\text{PI}(X)$, which the verifier interpolates himself (§4.2), so it enters the identity of link 1. In the deployed circuit the instance polynomial $\pi(X)$ of Definition 4.2 is the verifier’s own: he committed to it from the values he intends to verify against, and its designated cells are copy-constrained to the advice cells that carry the same values, so the permutation of link 3 forces agreement. There is no error term of its own. The relation certified is the one at the public input the verifier supplied (§4.2).

Together with the observation that closed §2.1, that filling the advice columns *is* knowing the witness, because those cells are the wires carrying it and its powers, the four links say: there exists an assignment of the advice columns, a value of the witness, making the entire arithmetisation of $X^3 + X + 5 = 35$ hold, at the public 35.

From existence to knowledge. The chain so far establishes that satisfying advice *exists*. That the prover *knows* it is the extraction question. The extractor of Theorem 3.5 and Corollary 3.13 recovers, from any prover who passes the opening, the committed vector and its blinder: it establishes knowledge of committed polynomial data, and nothing more. It does not by itself prove that those polynomials encode a satisfying circuit witness; that is the upper layer's task in the layered extraction of Remark 4.11, where the commitment scheme's extractor turns commitments into polynomials and the polynomial IOP's extractor turns polynomials into a witness. Under the composed hypotheses named there, a round-by-round or state-restoration knowledge-soundness theorem for the IOP, extractability and evaluation binding for the commitment scheme, and a multi-round Fiat–Shamir theorem for the transcript, an extractor for the *full* protocol obtains such a witness; §5 weighs those hypotheses against the deployed transcript, and in particular states in which model the tight bound is proved.

What he has not learnt. For the interactive protocol and an honest verifier the transcript reveals nothing beyond the truth of the statement (Proposition 4.9, cited, and Corollary 4.10); for the non-interactive deployed proof this is stated, not proved (§4.6). Every commitment the prover sent is a uniform group element (perfect hiding, recalled in §4.6); the verifying key's and the instance commitments carry the fixed blinding factor 1 and commit only public data; the evaluations of each blinded column are uniform by Lemma 4.8; the final scalar of the opening, which folding alone would have leaked as a linear form of the coefficients (Remark 3.7), is masked by the random polynomial $s(X)$ vanishing at the opening point (§3.6, in the deployed form of §3.8); and the joint transcript is simulated by the witness-free simulator of Proposition 4.9, within the distance of Corollary 4.10 for the deployed challenges. The toy has no blinding rows and is not zero knowledge: its claimed value $a(20) = 59$ alone singles out the witness 3 among the three roots 3, 18, 76 of $X^3 + X + 5 = 35$ in $\mathbb{F}_{97}$, and indeed among all 97 candidates; the blinding rows of Lemma 4.8 are what make the deployed evaluations uniform. The deployed mask makes the opening's own scalar a fresh uniform value: $c = 20$ under one mask (Table 7) and $c' = 46$ under another (§3.8), both accepted.

The error budget. The verifier has checked a small transcript, $2720 + 2272a$ bytes for a Actions (Table 8), and a logarithmic opening of $2k + 1 = 23$ points. Each link of the chain carries its own error term, which bounds the failure of that link alone: the identity's $\deg D/|\mathbb{F}|$ (Theorem 2.5); the y -combination's $(T - 1)/|\mathbb{F}|$ (Lemma 4.6); the permutation's $2N/|\mathbb{F}|$ (Theorem 2.11) and $N/|\mathbb{F}|$ for the challenges at which a product factor vanishes (Corollary 2.12); the lookup's $4n/|\mathbb{F}|$ (Theorem 2.15) and the θ -compression's $n(m - 1)/|\mathbb{F}|$ (Proposition 2.16); the multipoint reduction's allowance of order $n/|\mathbb{F}|$ (Lemma 4.5); and the opening's $O(\log d/|\mathbb{F}|)$ (Theorem 3.5 and Corollary 3.13). Combining them into one knowledge error for the protocol is the compilation principle of Remark 4.11, whose hypotheses §5 names and does not discharge; no

sum of the terms is asserted. Beyond these terms lie the loss of replacing coins by a hash, of which the grinding route alone multiplies a term by the query budget Q , so that 2^{-240} becomes 2^{-176} after 2^{64} attempts (§4.3, Proposition 4.3), while the multi-round transform's full loss is weighed in §5, which asserts no factor- Q bound for the compiled protocol; and the assumptions beneath the chain, the discrete logarithm on Vesta that makes the commitments bind and the extractor's alternative output infeasible (Theorem 3.5 and Corollary 3.13), the random-oracle model for the transcript hash, and the extraction model in which the non-interactive bound is proved (§5). The isolated 2^{-240} is not the verdict's error: it is one Schwartz–Zippel term for one fixed lie, read as §1.2 fixed.

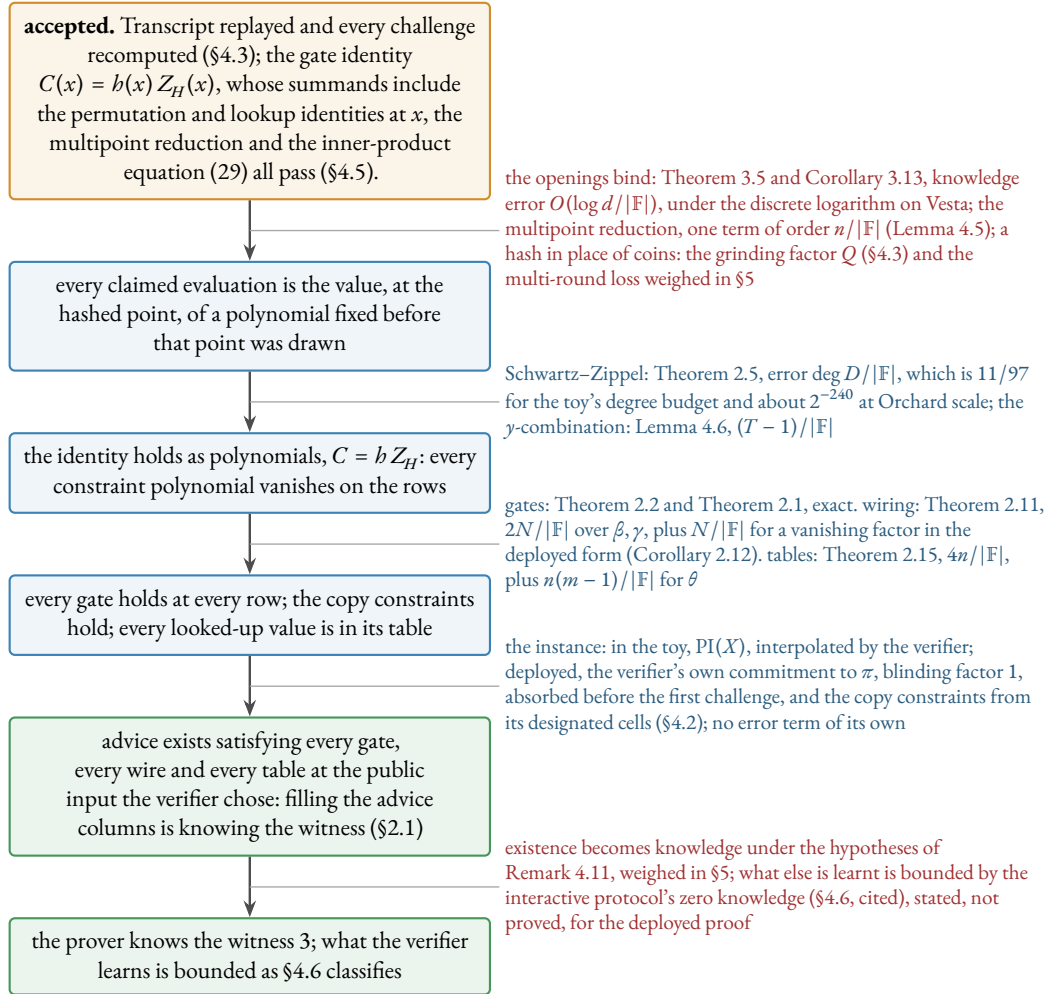


Figure 17. The implication chain from the verifier's accepted checks (amber) back to the claim (green), read downwards; each arrow carries the statement that proves it and its error term. Blue labels are algebraic links whose only losses are Schwartz–Zippel terms or none; red labels are the two links that rest on an assumption or a model hypothesis, the binding of the openings at the top and the extraction of the witness at the bottom.

6.2 The Action circuit at deployed scale

Nothing in the chain used that the statement was small. Replace the four gates by the constraints of the Orchard Action statement, the nine conditions the protocol specification imposes on the public input and the witness (protocol specification, § 4.18.4, “Action Statement (Orchard)”: the integrity of the old and new note commitments, the Merkle path from the spent commitment to the anchor unless the spent value is 0 (the *Crypto Guide*, § “Merkle trees and commitments to sets”), the opening of the net value commitment, a Pedersen commitment to the value difference (the *Crypto Guide*, § “The Pedersen commitment”), the nullifier’s derivation, the spend authority, that the randomised verification key rk is the re-randomisation of the spend validating key ak by a witnessed randomiser (the *Crypto Guide*, § “Key re-randomisation and unlinkability”), the integrity of the diversified address, that the spent note’s address (g_d, pk_d) satisfies $pk_d = [ivk] g_d$ for the incoming viewing key ivk derived in the witness from ak and further key material (the *Crypto Guide*, § “Shielded-note delivery as deployed hybrid encryption”) unless that derivation fails, and the two enable flags, which force the spent value to 0 unless $enableSpends = 1$ and the created value to 0 unless $enableOutputs = 1$. From NU6.3 the circuit checks a tenth condition on the further public bit $disableCrossAddress$ (§ 4.6; ZIP 258): when it is set, the receivers (g_d, pk_d) of the spent and created notes coincide (§4.2). Each condition unfolds into the gates of the deployed circuit (§4.1), and the grid grows accordingly (Table 11): the four rows become $2^{11} = 2048$; the one universal gate of degree 3 becomes custom gates reading rotations, under the degree bound $d_{\max} = 9$, the incomplete addition of Theorem 4.1 among them; the quotient t of degree 5 becomes b of degree up to 16375, cut into 8 chunks (Theorem 4.7); the four-class permutation on twelve cells becomes fifteen equality-enabled columns in three running products; a Sinsemilla generator table of 2^{10} rows, reached by three lookup arguments, joins the permutation (§4.1.3; the hash is the *Crypto Guide*’s, § “Sinsemilla: an algebraic hash-based commitment”); the four openings at z become 49 claimed evaluations per Action and 45 shared (Table 10), collapsed into one opening of 23 points; and the point $z = 20$ drawn by decree becomes an x squeezed from the BLAKE2b transcript. The spine is identical: arithmetise into a grid, interpolate the columns, compress “all rows correct” into one identity, $G = t Z_H$ in the toy and $C = b Z_H$ in the deployed protocol, commit, and test at a random point.

The algebraic core of Action verification. For the verification of an Action proof that the consensus rules require (protocol specification, § 4.6, “Action Descriptions”), the randomised polynomial-identity test of link 1 is the algebraic core: a Schwartz–Zippel argument on one identity at one hashed point. The full verifier of §4.5 evaluates that identity’s permutation and lookup summands from the claimed evaluations, binds every claimed evaluation through the multipoint reduction, recomputes every transcript challenge, and closes the single inner-product equation with its one multi-scalar multiplication of length n , before concluding that the circuit’s constraints hold at the public input it verified against, the anchor, the net value commitment, the nullifier, the randomised verification key, the extracted note commitment and the flags of §4.2. Under the knowledge soundness whose status §5 classifies (designed-but-unspecified for the deployed transcript), an accepted Action proof certifies, for a known witness, that the constraints of the circuit whose verifying key the consensus rules fix (protocol specification, § 4.6) hold. That these imply the conditions of the Action statement (protocol specification, § 4.18.4) is a property of that cir-

Table 11. The toy and the deployed Action circuit, object by object. Every row is the same object at two scales; every number is one already printed in the section cited, where it is computed by script or, for the 2^{10} rows of the Sinsemilla table, recorded from the definition of Sinsemilla.

	the toy	the Orchard Action
field	$\mathbb{F}_{97}$	$\mathbb{F}_{p_{\text{Pallas}}}$, a prime of 255 bits
commitment group	$y^2 = x^3 + 3$ over $\mathbb{F}_{97}$, 97 points (§3.7)	Vesta, of order p_{Pallas} (§3.8)
rows	$n = 4$, $H = \{1, 22, 96, 75\}$	$n = 2^{11} = 2048$
columns	3 advice, 5 selectors, 1 instance	10 advice, 29 fixed, 1 instance (§4.1)
gates	one universal gate (1), degree 3, rotation 0	custom gates at rotations, $d_{\text{max}} = 9$ (Definition 2.7; the deployed value, §4.1.4)
identity	$G = t Z_H$; $\deg G = 9$, $\deg t = 5$	$C = b Z_H$ with C the γ -combination; $\deg b \leq 16375$, 8 chunks (Theorem 4.7)
wiring	a permutation of twelve cells, four copy classes (b_3, c_3 fixed), one running product (§2.6)	fifteen equality-enabled columns, three running products
table facts	none on the grid; Example 2.13	three lookup arguments into a 2^{10} -row Sinsemilla table (§4.1.3)
the point	$z = 20$, by decree	x squeezed from the BLAKE2b transcript (§4.3)
one-term error	11/97; 3/97 for the $c_0 = 10$ cheat	about 2^{-240} ; 2^{-176} after 2^{64} grinding attempts
openings	a, b, c, t at z ; four cross-term points and S per opening (§3.7)	49 claimed evaluations per Action and 45 shared, one opening of 23 points (Tables 10 and 8; the collapse, Construction 4.4)
proof	—	$2720 + 2272 a$ bytes, 4992 for one Action (Table 8)

cuit, which this volume does not verify and which the specification records as having failed for the Orchard circuit deployed before NU6.2 (the same section); value balance additionally requires the binding signature (protocol specification, § 4.14, “Balance and Binding Signature (Orchard)”), and privacy rests also on components outside the proof, such as note encryption.

What the formal treatment supplies. Each of the five reductions of §1.2, “The chain of reductions”, is owned by a formal statement. Table 12 lists, reduction by reduction, what the toy exhibited and which statement carries it; the last two rows point beyond the spine, to the extractor that turns existence into knowledge and to the recursion, which Orchard does not deploy and on which nothing above depends.

7 Recursive proof composition and accumulation schemes

This section states recursive proof composition and accumulation for the inner-product argument; Orchard deploys neither. The protocol specification uses Halo 2 for one purpose, to prove and verify Action statements (protocol specification, § 4.1.13, “Zero-Knowledge Proving System”), and requires every Action proof to be valid (§ 4.6, “Action Descriptions”); the design proposal that introduced the pool states that it makes no use of the proof system’s support for recursive proofs (ZIP 224, under “Proving system”). A verifier holding several proofs may still check them together. The complete verification equation E_i of each proof is a multi-scalar multiplication over all d generators that should evaluate to $\mathcal{O}$ (§3.8.1, “Batch verification”); for independent uniform $\mu_1, \dots, \mu_b \in \mathbb{F}$ the verifier of b proofs checks $\sum_i [\mu_i] E_i = \mathcal{O}$ instead. If some $E_j \neq \mathcal{O}$, then

Table 12. The map from the worked example to the formal statements that own it, in the order of the reductions of §1.2. The fifth reduction, from the random check to committed openings, is split into its parts.

reduction	the toy exhibits	the statement that owns it
computation → grid	Table 1, Figure 2; knowing the witness is filling the advice	Definition 2.7 (§2.1, §2.5)
grid → polynomials	the nine interpolants; G of degree 9 vanishing on H	Theorem 2.1 (§2.2)
vanishing → identity	$t = G/Z_H$ of degree 5; the gap $D = 24(1 + X + X^2 + X^3)$	Theorem 2.2 (§2.3)
identity → one random check	$z = 20$ accepted; 3/97 and 11/97; Figure 3	Theorem 2.5 (§2.4); Lemma 4.6 and Theorem 4.7 (§4.5)
random check → committed openings		
binding	$\mathbf{a} = (90, 61, 22, 24)$ sealed in $P = (39, 25)$	the <i>Crypto Guide</i> , §“Pedersen vector commitments”, recalled in §2.9
the opening	Tables 4 and 5; $v' = 60$ rejected	Construction 3.2, Theorem 3.3, Lemma 3.4, Theorem 3.5, Corollary 3.13 (§3)
the wiring	product 1 honest, 61 with $b_1 = 4$	Theorem 2.11 (§2.6)
the tables	$82 = 82$; the forged 7 stranded	Theorem 2.15 (§2.8)
the public 35	$\text{PI} = (0, 0, 0, -35)$	Definition 4.2 (§4.2)
the removed verifier	z by decree against x by hash; $Q\varepsilon$	the <i>Crypto Guide</i> , §“The Fiat–Shamir transform: from interactive to non-interactive”, applied in §4.3
many openings, one hiding	— the leaked linear form; $c = 20$ against $c' = 46$	Construction 4.4 and Lemma 4.5 (§4.4) Remark 3.7 and the mask (§3.6, §3.8); Lemma 4.8 (§4.6)
the pattern	—	Remark 4.11 and “The map onto the phases” (§4.7)
“the prover knows the witness”	—	the extractor in the algebraic group model and the Fiat–Shamir loss (§5, “The algebraic group model”)
beyond the spine	—	the accumulation fold, Proposition 7.5, the accumulation scheme cited as Theorem 7.4, and recursion over the cycle of curves, none deployed by Orchard, which uses one direction of the cycle only (§7.3, “Accumulation and the Halo trick”; §7.4, “The half Orchard uses”)

E_j generates the group of prime order, so for any fixed values of the other factors exactly one μ_j satisfies the combined check, and a false batch passes with probability $1/|F|$ (Lemma 3.14). Batch verification verifies the batch and emits nothing; it is not the accumulation developed here. In the classification of §1.2, the Pasta curves and the use Orchard makes of them (§7.4) are *specified*; recursion and accumulation over those curves are *designed-but-unspecified*: their constructions are published and the halo2 book describes them (*Recursion* chapter), and ZIP 224 leaves them to future protocol updates, but no specification defines a recursive statement or an accumulation rule for Zcash. Theorem 7.4 and Proposition 7.5 below are theorems, the first cited and the second proved.

7.1 Recursion and its two classical obstacles

The setting. Consider a computation iterated many times,

$$z_m = f^{(m)}(z_0) := f(f(\dots f(z_0) \dots)),$$

one step f applied m times to an initial state, and suppose a verifier wants to be sure that z_m is the honest result without recomputing the m steps. Valiant’s *incrementally verifiable computation* asks for a proof of z_m that is updated step by step, each update costing about one step’s worth of proving and the finished proof no larger than the proof of a single step. The more general setting, in which computations branch and merge and each step consumes the proofs of its predecessors, is the *proof-carrying data* of Chiesa and Tromer: every message in a distributed computation carries a proof that it was produced correctly from messages that themselves carried such proofs.

The construction. The natural construction is *recursive proof composition*: proving, inside one proof, that another proof verifies. Write $\mathcal{V}$ for the verifier of a non-interactive argument (*Crypto Guide*, §“SNARKs: succinct non-interactive arguments of knowledge”), and suppose its algorithm is itself arithmetised, in the sense of §2, into a PLONKish circuit of Definition 2.7, the *verifier circuit*: its instance columns hold the statement and its advice columns hold a proof, and its gates are satisfiable exactly when $\mathcal{V}$ accepts that proof of that statement. At step $i + 1$ the prover proves the compound statement

“ $z_{i+1} = f(z_i)$, and there exists a proof π_i that the verifier circuit accepts for the statement of step i .”

The proof π_i is advice of the step- $(i + 1)$ circuit, never sent onward; the proof π_{i+1} that results attests to step $i + 1$ and, through the embedded verifier, to π_i , hence to step i , and so on down to z_0 . One proof attests to the entire history, and the cost of producing π_{i+1} is the cost of one step of f plus one run of the verifier circuit, independent of how long the history is. The requirement on which everything rests is the one just used: the argument’s verifier must be efficiently expressible as a circuit. A verifier whose circuit is as large as the computation it checks would make each step as expensive as the whole history, and the construction would buy nothing.

Two obstacles. The classical instantiations meet that requirement badly, in two different ways.

1. *Pairings in a circuit, and a ceremony.* The pairing-based systems, Groth16, which Sapling deploys, and PLONK compiled with the KZG commitment (*Crypto Guide*, §“SNARKs: succinct non-interactive arguments of knowledge” for Groth16 and the bilinear pairing; §“Polynomial commitment schemes”, under “KZG: evaluation in the exponent”), have constant-size proofs and verifiers that run in time independent of the circuit. But those verifiers evaluate pairings, and a pairing lands in a target group that lives in an extension field $\mathbb{F}_{p^e}$ of the curve’s base field, of degree e equal to the curve’s embedding degree (*Crypto Guide*, §“Instantiation on elliptic curves; the Pasta curves”, the large-embedding-degree criterion). A circuit whose native arithmetic is one prime field must emulate arithmetic in another field with many gates per operation, so a pairing is costly to express inside a circuit. These particular systems also need a trusted setup, the ceremony that §3.8.1 contrasted with hashed generators.
2. *A linear-time verifier.* The transparent systems built on the inner-product argument of §3 need no pairing and no ceremony, but their verifier is linear in the size of the committed vectors: the one multi-scalar multiplication $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle$ of length d that §3.4 singled out. Inside a verifier circuit that multiplication is d scalar multiplications of curve points, each a double-and-add with $\ell - 1$ doublings for an ℓ -bit scalar (*Math Guide*, §“Scalar multiplication and double-and-add”), so the verifier circuit grows linearly in d , the length of the committed vectors of the proof it verifies. A linear-time check dominates the verifier circuit and negates the per-step economy that recursion was meant to deliver.

Deferred verification. Halo, the construction of Bowe, Grigg and Hopwood (*Halo: Recursive Proof Composition without a Trusted Setup*) from which Halo 2 descends, *defers* the linear-time part of verification instead of recomputing it at every step, and uses the inner-product argument, in which no pairing occurs. Bünz, Chiesa, Mishra and Spooner formalised the deferral as an *accumulation scheme* (*Proof-Carrying Data from Accumulation Schemes*; BCMS20), the notion the next subsection defines; their Theorem 7.1 is quoted below, without proof, as Theorem 7.4.

7.2 Accumulation schemes

The idea is to separate what a verifier must check at each step from what can be checked once, at the end of a chain. Fix a *predicate* Φ , a property of an input q decidable in polynomial time, for example “this inner-product opening is correct” of a transcript of Construction 3.2. An accumulation scheme replaces a sequence of checks of Φ by a sequence of accumulation steps and one run of a decider.

Definition 7.1 (Accumulation scheme). Let λ be the security parameter (*Crypto Guide*, §“Adversaries and the security parameter”) and Φ a predicate on inputs q . An *accumulation scheme* for Φ is a triple (P, V, D) of polynomial-time algorithms with access to a common random oracle, whose *accumulators* are bit strings.

1. The *accumulation prover* P , on input q and an old accumulator acc_{old} , outputs a new accumulator acc and an accumulation proof π_v .
2. The *accumulation verifier* V , on input $(q, \text{acc}_{\text{old}}, \text{acc}, \pi_v)$, outputs a bit; the bit 1 asserts that acc correctly accumulates q and acc_{old} .

3. The *decider* D , on input an accumulator, outputs a bit; an accumulator is *valid* if D outputs 1 on it.

The scheme is *complete* if, whenever $\Phi(q) = 1$, $D(\text{acc}_{\text{old}}) = 1$ and (acc, π_v) is an output of P on $(q, \text{acc}_{\text{old}})$, both $V(q, \text{acc}_{\text{old}}, \text{acc}, \pi_v) = 1$ and $D(\text{acc}) = 1$. It is *sound* if for every adversary of size polynomial in λ that outputs $(q, \text{acc}_{\text{old}}, \text{acc}, \pi_v)$ the probability, over the random oracle and the adversary's coins, of

$$V(q, \text{acc}_{\text{old}}, \text{acc}, \pi_v) = 1 \wedge D(\text{acc}) = 1 \wedge (\Phi(q) = 0 \vee D(\text{acc}_{\text{old}}) = 0)$$

is negligible in λ .

The definition is that of BCMS20 (Section 4.1) with one input and one old accumulator per step; theirs admits any number of each and adds a generator and an indexer, which fix the public parameters and keys left implicit here. For recursion the size of an accumulator must also be bounded independently of the number of inputs accumulated into it (BCMS20, Definition 5.1).

Soundness carries the checks along a chain.

Lemma 7.2 (Soundness along a chain). *Let (P, V, D) be sound in the sense of Definition 7.1, let acc_0 be valid, and let m be polynomial in λ . For any efficient algorithm producing $q_1, \dots, q_m$ and $\text{acc}_1, \dots, \text{acc}_m$, where acc_i accumulates q_i and acc_{i-1} , such that V accepts every step and $D(\text{acc}_m) = 1$, the probability that $\Phi(q_i) = 0$ for some i is at most m times the soundness error of the adversary that outputs step i for a uniform $i \in \{1, \dots, m\}$; it is therefore negligible.*

Proof. If some q_i violates Φ , then either acc_i is valid and step i exhibits the event of the soundness definition, or some later step $j > i$ turns an invalid acc_{j-1} into a valid acc_j and exhibits it. Let $\mathcal{A}$ be the algorithm that produces the chain and outputs step i for a uniform $i \in \{1, \dots, m\}$; since the bad event forces some step to exhibit the soundness event, its probability is at most m times the soundness error of $\mathcal{A}$, which is negligible because m is polynomial (*Math Guide*, §“Polynomial, exponential, and negligible functions”, the closure properties). $\square$

The verifier of each step runs the accumulation verifier and never the decider. In a recursive chain the accumulation verifier runs inside the circuit of the step's proof, so the argument also needs the knowledge soundness of each step's proof, to recover what that circuit checked; BCMS20 prove the resulting proof-carrying data secure for computations of constant depth, which for a chain means constant length, because the extractor is applied recursively once per level (Theorem 5.2 and Remark 5.3).

For an inner-product polynomial commitment such a scheme is known under the hypothesis of Theorem 7.4 below, and it resolves the second obstacle of §7.1 where it arose: the verifier circuit runs the $O(\log d)$ part of the opening check, the rounds, the scalars and the small combinations of points, and accumulates the $O(d)$ part, the claim about $G^{(0)}$, into a running accumulator; the decider pays that multi-scalar multiplication once, at the end of the chain. The next subsection defines the accumulator, states the scheme as an interface theorem, and works the fold on the toy curve.

7.3 Accumulation and the Halo trick

The asymmetry, recalled. Everything the verifier of Construction 3.2 does to decide the check (15), or its deployed form (29), is logarithmic in d : $3k - 2$ field multiplications for $g(x; \mathbf{u})$, a combination of the $2k$ received points and a handful of fixed ones with $O(k)$ scalar multiplications, and a comparison of two curve points. The single exception is the evaluation

$$G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle = \text{Commit}(g(X; u_1, \dots, u_k); 0),$$

a multi-scalar multiplication of length d over *all* the generators, which §3.4 named the one linear step and recognised as the unblinded commitment to the polynomial g whose *value* the verifier had just computed cheaply. Verifying an Action proof as the consensus rules require includes that step for every proof; batch verification, recalled at the start of this section, evaluates one combined multiplication for the proofs of a batch, and the batch is verified and nothing survives it. Accumulation takes the same kind of linear combination and does something else with it: it carries the combined claim forward as data, unverified, and pays for it once at the end of a chain. Figure 18 sets the two side by side; the rest of this subsection is the right-hand panel, and none of it is deployed by Orchard.

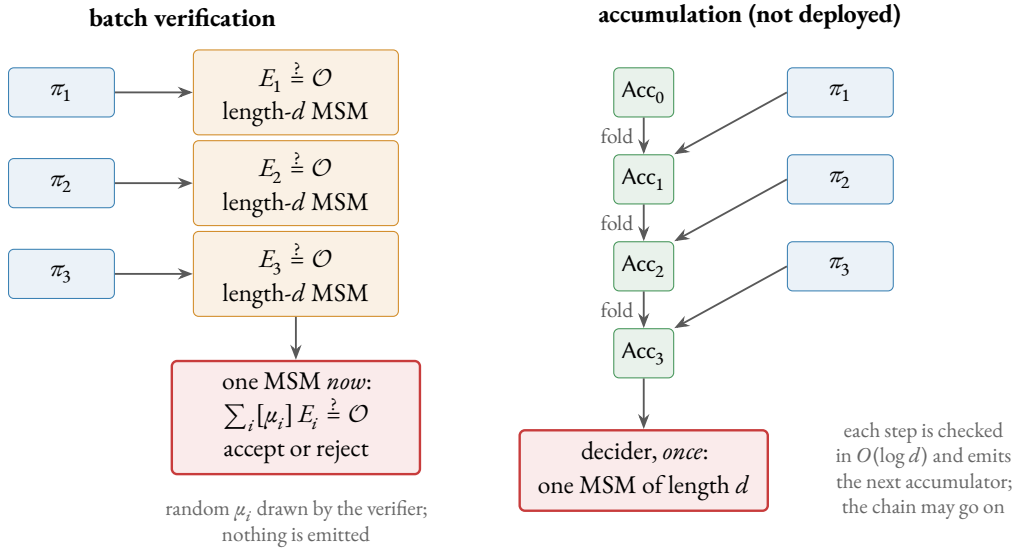


Figure 18. Left: batch verification. Each proof's complete verification equation E_i is a multi-scalar multiplication over the d generators (amber); the verifier combines them with fresh random factors and evaluates one multiplication immediately (red), then accepts or rejects. Right: accumulation. Each proof's deferred claim is accumulated into a running accumulator (green), each step being checked at logarithmic cost; the accumulator is emitted for the next step, and a decider evaluates one length- d multiplication at the end of the chain. Both take random linear combinations; only the right-hand scheme carries a claim forward.

The atomic accumulator. The claim to be deferred has a fixed shape, and the accumulator is that shape written down.

Definition 7.3 (Atomic accumulator for the inner-product argument). An *accumulator instance* is a tuple

$$\text{Acc} := (G^{(0)}, u_1, \dots, u_k) \in \mathbb{G} \times (\mathbb{F}^\times)^k,$$

a curve point and the k round challenges of one opening. It is *valid* if and only if

$$G^{(0)} = \langle \mathbf{s}(\mathbf{u}), \mathbf{G} \rangle = \text{Commit}(g(X; u_1, \dots, u_k); 0), \quad (35)$$

with $\mathbf{s}(\mathbf{u})$ and g the structured scalars and the polynomial of Theorem 3.3, in the symmetric normalisation of Construction 3.2. The deferred claim concerns the *unblinded* commitment: the blinder of the opened commitment was discharged by the $[r^*]H$ term of (15), or the $[f]H$ term of (29), and plays no part in it. From here on the unary $\text{Commit}(\cdot)$ abbreviates $\text{Commit}(\cdot; 0)$.

The deployed analogue replaces $\mathbf{s}(\mathbf{u})$ and g by their forms in Remark 3.9; Proposition 7.5 and, with verifier-drawn challenges, the closure argument below hold for it verbatim, with Corollary 3.13 in place of Theorem 3.5, since they use only the linearity of the commitment, the degree of g and the knowledge soundness of the opening.

A verifier who is handed a point $G^{(0)}$ together with the challenges can complete the check (15) in $O(\log d)$ operations *provided* the tuple is valid; validity is precisely what he has not paid for. The scheme quoted next lets him defer that payment.

Theorem 7.4 (Accumulation for the inner-product argument; interface). *Let PC_{DL} be the inner-product polynomial commitment of BCMS20 (Appendix A.2), for polynomials of degree below d over a group $\mathbb{G}$ of prime order, and suppose that PC_{DL} is a polynomial commitment scheme in the random oracle model, its openings in particular being extractable. Then (BCMS20, Theorem 7.1) there is an accumulation scheme in the sense of Definition 7.1, in the random oracle model, for the predicate “this PC_{DL} opening is correct”, whose accumulators are themselves PC_{DL} openings, such that:*

1. *the accumulation prover $\mathbf{P}$ runs in time dominated by $O(d)$ scalar multiplications in $\mathbb{G}$;*
2. *the accumulation verifier $\mathbf{V}$ runs in time dominated by $O(\log d)$ scalar multiplications in $\mathbb{G}$;*
3. *the decider $\mathbf{D}$ is the opening check of PC_{DL} : a succinct part, which yields a tuple of the form of Definition 7.3 in the normalisation of Remark 3.9, followed by the validity check (35) of that tuple, one multi-scalar multiplication of length d .*

The theorem is quoted as an interface and not proved in this volume. Its hypothesis, the extractability of PC_{DL} openings, is stated by BCMS20 as a conjecture from the binding of the Pedersen commitment, the extractor they record for the Fiat–Shamir-compiled opening running in superpolynomial time (BCMS20, Appendix A.3.2). The scheme PC_{DL} is neither Construction 3.2, whose fold is symmetric, nor the deployed opening of Remark 3.9, which blinds every commitment and subtracts $[v] G_0$ where PC_{DL} adds a multiple of its inner-product generator (the halo2 book’s *Comparison to other work* chapter); no theorem transferring it to either is proved here or cited, and in the classification of §1.2 that transfer, like accumulation over these curves, is designed-but-unspecified. What an accumulation step does for Construction 3.2, and why its random com-

bination is sound, is worked out below at the level the volume proves things.

The accumulation step in the verifier circuit. Consider a Halo 2 verifier circuit, expressed over the scalar field of one Pasta curve and verifying a proof produced over the other curve, an arrangement whose necessity §7.4 explains. At step i of a chain the circuit receives the previous proof π_{i-1} , whose opening is an instance of Construction 3.2, and the accumulator Acc_{i-1} exposed as a public input of π_{i-1} , of which it uses the instance on the curve π_{i-1} is committed on (see below), and does three things.

1. *Run the succinct part.* It recomputes the challenges $\mathbf{u}'$ of the opening from the transcript, evaluates $g(x; \mathbf{u}')$ with $O(k)$ scalar operations, and forms the left-hand side of (15) and every term of the right-hand side except the one on the final generator, here written $G'^{(0)}$, with $O(k)$ point operations. The point operations are native, since the points of π_{i-1} have coordinates in the circuit's field. The challenges, $g(x; \mathbf{u}')$ and the scalar products lie in the scalar field of the curve π_{i-1} is committed on, the other Pasta field; they are computed non-natively, or exposed as public inputs and checked by the next circuit of the chain, which is native to that field (“What the cycle does and does not buy” in §7.4; the halo2 book’s *Recursion* chapter).
2. *Witness the expensive part.* It does not compute $G'^{(0)}$. The prover of the step supplies the claimed point as advice, and the circuit uses it to complete the check, which is then passed *conditionally on the validity of the fresh tuple* $(G'^{(0)}, \mathbf{u}')$ of that opening.
3. *Accumulate.* The step now holds two claims of the shape (35): the inherited instance $(G^{(0)}, \mathbf{u})$ of Acc_{i-1} and the fresh $(G'^{(0)}, \mathbf{u}')$. For a challenge ρ derived by the random oracle from both tuples, the two reduce to the single claim

$$G^{(0)} + [\rho] G'^{(0)} = \text{Commit}(g(X; \mathbf{u}) + \rho g(X; \mathbf{u}')) = \langle \mathbf{s}(\mathbf{u}) + \rho \mathbf{s}(\mathbf{u}'), \mathbf{G} \rangle, \quad (36)$$

whose coefficient vector on the right anyone holding both challenge tuples can recompute. Proposition 7.5 shows that a false input survives the combination for at most one ρ . The step’s prover then opens the combined point at a fresh point, outside the circuit, and the circuit checks that opening succinctly as in item 1; the tuple the opening leaves behind is the new accumulator Acc_i (over a 2-cycle, its instance on that curve), exposed as a public input of the proof π_i the step produces. The paragraph “Closure: the prover-assisted step” below describes that opening.

Over a 2-cycle of curves (§7.4) the proofs alternate between the two curves, and the accumulator is a pair of instances, one per curve. The step that verifies a proof committed on one curve folds that proof’s fresh tuple into the instance on the same curve, which was last updated two steps earlier and forwarded unchanged by the intervening step, and it forwards the other curve’s instance in the same way, without point arithmetic. Figure 19 draws the chain. At its end a single multi-scalar multiplication of length d , run outside any circuit, checks the last accumulator and with it every opening in the history: for the scheme of Theorem 7.4 by Lemma 7.2.

Why the fold is sound. The soundness of the fold (36) is one line of linear algebra in the group.

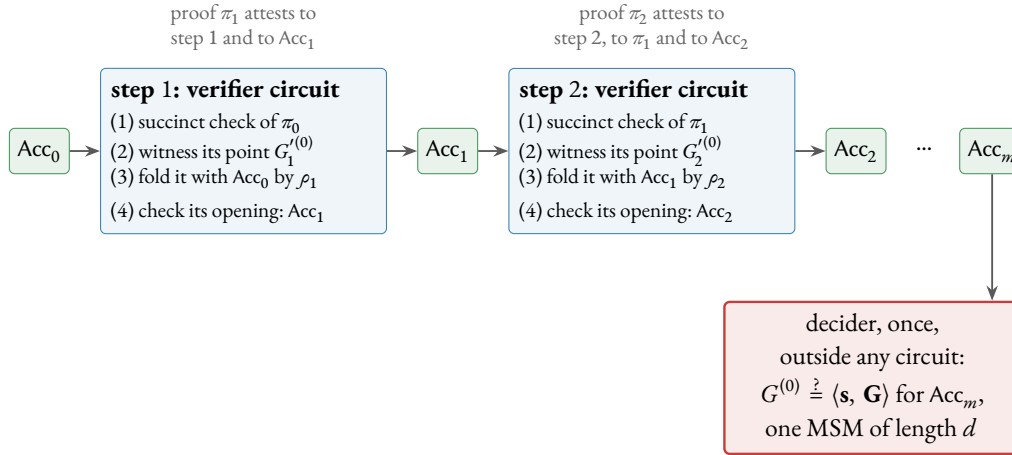


Figure 19. A chain of recursively composed proofs. Each step’s verifier circuit (blue) runs the logarithmic part of the previous proof’s opening check, witnesses the expensive point instead of computing it, combines the resulting tuple with the accumulator (green) it inherited by (36), and checks succinctly the opening of the combined point at a fresh point, whose tuple is the new accumulator, a public input of the step’s own proof. Over a cycle of curves each Acc_i is a pair of instances, one per curve: each step folds into the instance on the curve of the proof it verifies and passes the other through unchanged. The decider (red) pays one length- d multi-scalar multiplication at the end of the chain, and no circuit ever contains one.

Proposition 7.5 (Soundness of the ρ -fold). *Let $(G^{(0)}, \mathbf{u})$ and $(G'^{(0)}, \mathbf{u}')$ be two accumulator instances and ρ uniform in $\mathbb{F}$, drawn after both are fixed. If both instances are valid, the folded equation (36) holds for every ρ . If at least one is invalid, it holds for at most one value of ρ , hence with probability at most $1/|\mathbb{F}|$.*

Proof. Set $\Delta := G^{(0)} - \text{Commit}(g(X; \mathbf{u}))$ and $\Delta' := G'^{(0)} - \text{Commit}(g(X; \mathbf{u}'))$, the two errors. By linearity of the commitment (*Crypto Guide*, §“Pedersen vector commitments”, the remark that linearity survives), $\text{Commit}(g + \rho g') = \text{Commit}(g) + [\rho] \text{Commit}(g')$, so the left-hand side of (36) minus the right is $\Delta + [\rho] \Delta'$, and the equation holds exactly when

$$\Delta + [\rho] \Delta' = \mathcal{O}. \quad (37)$$

If both instances are valid, $\Delta = \Delta' = \mathcal{O}$ and the equation holds for all ρ . Otherwise two cases. If $\Delta' \neq \mathcal{O}$, then, $\mathbb{G}$ being cyclic of prime order, Δ' generates $\mathbb{G}$ and the map $\rho \mapsto [\rho] \Delta'$ is a bijection $\mathbb{F} \rightarrow \mathbb{G}$ (*Math Guide*, §“The mixed inner product with group elements”, the one-dimensional vector space $\mathbb{G}$ over $\mathbb{F}$), so exactly one ρ sends Δ' to $-\Delta$: the discrete logarithm of $-\Delta$ to the base Δ' , which exists whether or not anyone can compute it. If $\Delta' = \mathcal{O}$, then $\Delta \neq \mathcal{O}$ and (37) holds for no ρ . A uniform ρ hits the at most one bad value with probability at most $1/|\mathbb{F}|$. $\square$

Example 7.6 (The fold on the toy curve). The fold is run on the 97-point curve and the four generators $G_0, \dots, G_3$ of §3.7, with $d = 4$ and the symmetric structured scalars of Theorem 3.3, on two challenge tuples, $(u_1, u_2) = (52, 44)$ with $\mathbf{s} = (80, 10, 68, 57)$ and $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle = (21, 10)$, and $(u'_1, u'_2) = (41, 39)$ with $\mathbf{s}' = (64, 11, 53, 47)$ and $G'^{(0)} = (23, 2)$, both valid by construction, for every $\rho \in \mathbb{F}_{97}$. With both instances valid the folded equation (36) holds for all 97 values of ρ .

Spoiling an instance by adding a nonzero multiple of G_0 to its point makes its error Δ a nonzero point, and the surviving values of ρ are counted again. With the first instance false and the second valid, no ρ survives: this is the case $\Delta' = \mathcal{O} \neq \Delta$ of the proof. With the second false and the first valid, exactly one survives, $\rho = 0$, the value that discards the second instance altogether. With both false exactly one survives, $\rho = 50$, the discrete logarithm of $-\Delta$ to the base Δ' on this curve, where such logarithms are a short search. In every spoiled case the count is at most one of 97: the bound $1/|\mathbb{F}|$ of Proposition 7.5, attained.

Closure: the prover-assisted step. The folded claim (36) is not an accumulator instance. Its coefficient vector $\mathbf{s}(\mathbf{u}) + \rho \mathbf{s}(\mathbf{u}')$ is in general not the structured vector of any single challenge tuple, so carrying it means carrying both tuples, and after m folds all m of them: the decider would still pay one multiplication of length d , but the accumulator would grow with the chain, against the size bound that recursion requires (§7.2). The tuple type of Definition 7.3 is therefore *not* closed under verifier-side linear combination alone. It is closed under one more step, which the prover assists. Write $A := G^{(0)} + [\rho] G'^{(0)}$ for the combined point and $a(X) := g(X; \mathbf{u}) + \rho g(X; \mathbf{u}')$ for the combined polynomial, of degree below d . The combined claim says exactly that $A = \text{Commit}(a)$. The step's prover opens A by the opening protocol of Construction 3.2 at a fresh point x' , derived by the random oracle from A and both tuples, claiming the value $a(x')$, which the verifier computes for himself in $O(k)$ field operations from the two tuples, since $g(x'; \mathbf{u})$ and $g(x'; \mathbf{u}')$ are each a product of k factors. That opening does two things at once. First, it carries the combined claim up to a blinder.

Proposition 7.7 (Prover-assisted step, verifier-drawn challenges). *Under the hypotheses of Theorem 3.5, let ρ, x' and the opening's challenges be drawn uniformly by the verifier, in this order, ρ after the two tuples are fixed. There is an extractor which, given rewindable access to a step prover accepted with non-negligible probability, runs in expected polynomial time and outputs either blinders δ, δ' with $G^{(0)} = \text{Commit}(g(X; \mathbf{u}); \delta)$ and $G'^{(0)} = \text{Commit}(g(X; \mathbf{u}'); \delta')$, or a nontrivial discrete-logarithm relation among $\mathbf{G}, H, U, U'$. Its knowledge error is $O(\log d/|\mathbb{F}|) + (d - 1)/|\mathbb{F}| + 1/|\mathbb{F}|$.*

Proof. By Theorem 3.5 the opening yields a representation $A = \langle \mathbf{a}^*, \mathbf{G} \rangle + [r_A]H$, unique since a second one would be a discrete-logarithm relation among $\mathbf{G}$ and H , and certifies $a^*(x') = a(x')$ for the polynomial a^* with coefficient vector $\mathbf{a}^*$; two polynomials of degree below d that differ agree at a uniform x' , drawn after A was fixed, with probability at most $(d - 1)/|\mathbb{F}|$ (*Math Guide*, §“The Schwartz–Zippel lemma”), so except with that probability $\mathbf{a}^*$ is the coefficient vector of a and $A = \text{Commit}(a; r_A)$. The blinder r_A is the prover's and need not be 0. Rewinding the choice of ρ and extracting again at a second value $\tilde{\rho} \neq \rho$ gives representations of A and of $\tilde{A} := G^{(0)} + [\tilde{\rho}] G'^{(0)}$, whose difference is $[\rho - \tilde{\rho}] G'^{(0)}$; solving yields $G'^{(0)} = \text{Commit}(g(X; \mathbf{u}'); \delta')$ and $G^{(0)} = \text{Commit}(g(X; \mathbf{u}); \delta)$ for blinders δ, δ' that the extractor computes. This is the argument of Proposition 7.5 applied to the $\mathbf{G}$ -coordinates of the representations: a combination correct at two values of ρ has both errors zero. It is all the chain needs: a tuple whose point is $\text{Commit}(g(X; \mathbf{u}); \delta)$ still certifies the opening it came from, because, writing c for the final scalar that (15) calls a ,

$$[c] \text{Commit}(g; \delta) + [r^*]H = [c] \text{Commit}(g) + [r^* + c\delta]H,$$

so that opening satisfies the honest check with the blinder $r^* + c\delta$ in place of r^* , and Theorem 3.5 applies to it; in (29) the offset joins the term $[f]H$ likewise. The chain thus carries the claim that each deferred point's $\mathbf{G}$ -coordinates are $\mathbf{s}(\mathbf{u})$; the exact equality (35), blinder 0 included, is required only of the last accumulator, which the decider checks and an honest prover satisfies. An intermediate tuple may miss (35) by such a multiple $[\delta]H$ without any opening it serves being false. $\square$

Second, the opening leaves behind a deferred claim of the atomic form: it ends, as every opening does, in a point $G^{(0)}$ and challenges $\mathbf{u}''$ that the verifier circuit witnesses rather than computes. The accumulator after the step is again one tuple. This is the accumulation step of BCMS20 (Section 7.1), which likewise derives the combining challenge and the fresh point by the random oracle from the tuples and the combined point; there the accumulator is the opening itself, and its succinct part runs at the next step. With ρ, x' and the opening's challenges drawn by a verifier, the step's soundness is Proposition 7.7. With random-oracle challenges, the step's soundness is BCMS20's Theorem 7.1 for PC_{DL} under its conjectured extractability (Theorem 7.4); for Construction 3.2 it is designed-but-unspecified.

What the accumulator carries. The deferred equation (35) is a claim of *commitment equality*, equivalently a claim about the value of one multi-scalar multiplication, for a coefficient vector $\mathbf{s}(\mathbf{u})$ that is public and structured: anyone holding the challenges can write it down. It is not itself an evaluation opening; no polynomial is being evaluated at any point when the decider runs, only a point recomputed and compared. The accumulation protocol reduces this claim, and carries it along the chain, through the prover-assisted opening just described, which turns each commitment equality into one evaluation claim at a fresh point and each opening back into one commitment equality. The Orchard verifier, once more, does none of this: it computes $G^{(0)}$ and compares.

7.4 A cycle of curves for efficient native recursion

The verifier circuit of §7.3 was “expressed over the scalar field of one Pasta curve and verifying a proof produced over the other”. This subsection explains why the two curves are needed, what the arrangement buys, and where Orchard already uses half of it.

Two fields per curve, recalled. A curve E over a prime field has two fields attached to it (*Math Guide*, §“Base fields, scalar fields, and the Pasta cycle”): the *base field*, in which its points' coordinates lie and in which the group law's slopes, squarings and inversions are computed, and the *scalar field*, the integers modulo the prime order of its group, in which the scalars of $[k]P$ live. Write E_p for a curve over $\mathbb{F}_p$ and $q := \#E_p(\mathbb{F}_p)$ for its prime order, so that its base field is $\mathbb{F}_p$ and its scalar field $\mathbb{F}_q$. A PLONKish circuit of Definition 2.7 is defined over one prime field $\mathbb{F}$, its *native field*: its cells hold elements of $\mathbb{F}$ and its gates are polynomial identities over $\mathbb{F}$, so an addition or multiplication in $\mathbb{F}$ costs one cell or one gate. Arithmetic in any *other* prime field is *non-native*: an element of the other field must be carried as several $\mathbb{F}$ -cells, each range-checked by the lookups of §2.7, and each of its operations becomes many gates with carries and reductions. Which field a circuit is native to is not a free choice. Its column polynomials are committed with the inner-product commitment of §3 on some curve, and a Pedersen commitment to a vector of $\mathbb{F}$ -elements needs a

group whose scalar field is $\mathbb{F}$; so *the native field of a circuit is the scalar field of the curve its proof is committed on*, exactly as the Action circuit, native to $\mathbb{F}_{P_{\text{Pallas}}}$, is committed on Vesta, whose order is p_{Pallas} (§3.8).

Why one curve cannot host its own verifier. Take a proof committed on E_p . Its group elements, the column and quotient commitments, the cross terms (L_j, R_j) and the deferred point of its opening, are points of E_p with coordinates in $\mathbb{F}_p$. The in-circuit folding of §7.3 adds and scales such points, which is base-field arithmetic in $\mathbb{F}_p$, so a verifier circuit handles them natively only if its native field is $\mathbb{F}_p$. But a circuit committed on E_p is native to the *scalar* field $\mathbb{F}_q$: its challenges, its structured scalars and every field operation of the succinct part are $\mathbb{F}_q$ -arithmetic. One curve would host both jobs only if $\mathbb{F}_p = \mathbb{F}_q$, that is, $\#E_p(\mathbb{F}_p) = p$. Curves with exactly p points exist; by Hasse’s theorem (*Math Guide*, §“The group structure of $E(\mathbb{F}_p)$ ”), which writes $\#E_p(\mathbb{F}_p) = p + 1 - t$ with the trace t , they are the curves with $t = 1$, called *anomalous*. On an anomalous curve the discrete logarithm is computable in polynomial time, the attack of Smart, of Semaev and of Satoh and Araki, and the *Crypto Guide* lists “not anomalous” among the criteria a curve must meet to be used at all (§“Instantiation on elliptic curves; the Pasta curves”, the anti-Smart criterion). So no *secure* curve has $\mathbb{F}_p = \mathbb{F}_q$, and no one curve can host both the coordinates of its proofs and its own verifier circuit natively.

The 2-cycle. The efficient resolution uses two curves that trade fields. A *2-cycle* of elliptic curves is a pair (E_p, E_q) with

$$\#E_p(\mathbb{F}_p) = q \quad \text{and} \quad \#E_q(\mathbb{F}_q) = p,$$

both primes: the scalar field of each curve is the base field of the other. Then:

1. a circuit committed on E_q has native field $\mathbb{F}_p$, the scalar field of E_q ; and $\mathbb{F}_p$ is the coordinate field of E_p , so that circuit manipulates the points of a proof produced over E_p natively, and verifies such a proof at the cost §7.3 counted, the proof’s scalars aside;
2. the proof that circuit produces is committed on E_q , and its points have coordinates in $\mathbb{F}_q$, the native field of a circuit committed on E_p ; so a circuit over E_q handles its points natively in turn;
3. the proofs of a chain alternate between the two curves, each step handling its predecessor’s points natively (Figure 20).

What the cycle does and does not buy. The cycle makes the *preceding* curve’s coordinate arithmetic native; it does not identify the two scalar fields, which remain distinct primes. Challenges squeezed in the previous proof’s field and the scalars derived from them are $\mathbb{F}_p$ -elements that the $\mathbb{F}_q$ -native circuit must handle: they need compatible encodings across the two fields, range checks where an element of the larger field is carried in the smaller, and, where a genuine $\mathbb{F}_p$ multiplication is unavoidable, non-native scalar arithmetic. Nor is the cycle a logical necessity. A single curve could verify its own proofs with non-native coordinate arithmetic throughout, at a cost of many gates per point operation multiplied by the $O(\log d)$ point operations of the succinct part; that is possible and substantially more expensive. The cycle is the efficient native design, not the only correct one.

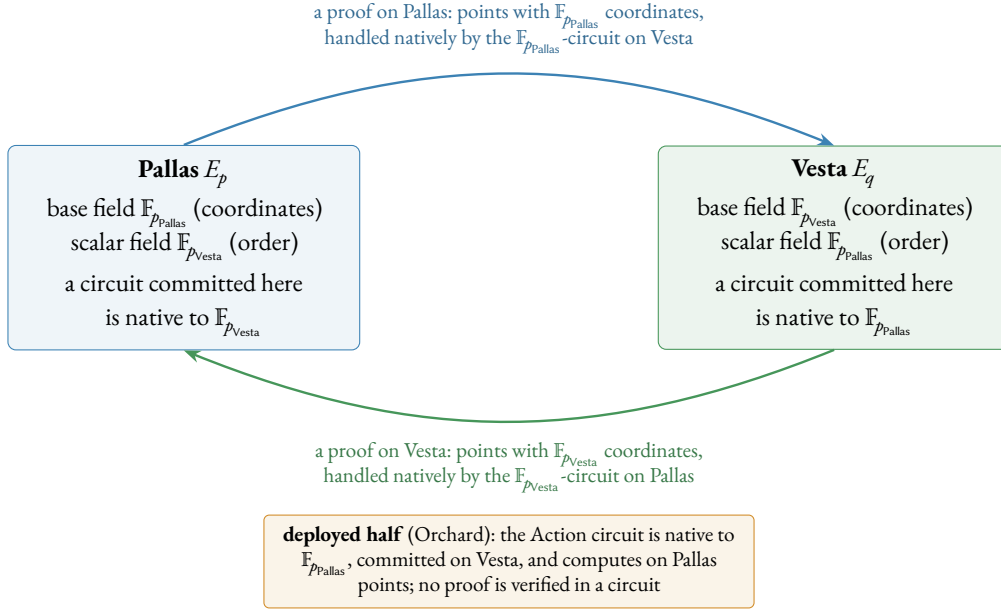


Figure 20. The Pasta 2-cycle. The base field of each curve (its coordinate field) is the scalar field of the other, so a circuit committed on Vesta (green) is native to $\mathbb{F}_{p_{\text{Pallas}}}$ and handles Pallas points (blue) natively, and symmetrically. In a recursive chain the proofs alternate along the two arrows. Orchard (amber) uses one direction of the cycle only: Pallas arithmetic inside a Vesta-committed circuit, with no verifier circuit at all.

The deployed instance. The Pasta cycle of the *Math Guide* (§“Base fields, scalar fields, and the Pasta cycle”; §“Pallas and Vesta assembled”) is a 2-cycle: Pallas is defined over $\mathbb{F}_{p_{\text{Pallas}}}$ with group order p_{Vesta} , and Vesta over $\mathbb{F}_{p_{\text{Vesta}}}$ with group order p_{Pallas} , both curves $y^2 = x^3 + 5$ (protocol specification, § 5.4.9.6, “Pallas and Vesta”). The cycle follows from the specification’s two primes and nothing else. With

$$\begin{aligned} p_{\text{Pallas}} &= 0x40000000\ 00000000\ 00000000\ 00000000\ 224698fc\ 094cf91b\ 992d30ed\ 00000001, \\ p_{\text{Vesta}} &= 0x40000000\ 00000000\ 00000000\ 00000000\ 224698fc\ 0994a8dd\ 8c46eb21\ 00000001, \end{aligned}$$

the point $(-1, 2)$ lies on Pallas, $[p_{\text{Vesta}}](-1, 2) = \mathcal{O}$ there, and exactly one multiple of p_{Vesta} lies in the Hasse interval $[p + 1 - 2\sqrt{p}, p + 1 + 2\sqrt{p}]$ for $p = p_{\text{Pallas}}$. That settles the order: the order of the point $(-1, 2)$ divides the prime p_{Vesta} and is not 1, so it is p_{Vesta} , which therefore divides $\#\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ by Lagrange’s theorem (*Math Guide*, §“Cosets and Lagrange’s theorem”); the group order lies in an interval of length $4\sqrt{p}$, far shorter than p_{Vesta} , which contains one multiple of p_{Vesta} ; hence $\#\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) = p_{\text{Vesta}}$. The symmetric check on Vesta shows that the Vesta group has order p_{Pallas} . The traces are

$$\begin{aligned} t_{\text{Pallas}} &= p_{\text{Pallas}} + 1 - p_{\text{Vesta}} = -86663725065984043395317759, \\ t_{\text{Vesta}} &= p_{\text{Vesta}} + 1 - p_{\text{Pallas}} = 86663725065984043395317761, \end{aligned}$$

each far inside the Hasse bound and neither equal to 1: neither curve is anomalous, as the *Crypto Guide*’s criterion requires. The two traces sum to 2, as they must for any 2-cycle, since $(p + 1 - q) + (q + 1 - p) = 2$; the toy curve of §3.7, $y^2 = x^3 + 3$ over $\mathbb{F}_9$ with 97 points, has trace $79 + 1 - 97 =$

−17, and the case excluded in that subsection, a curve over $\mathbb{F}_{97}$ with 97 points and so trace 1, is exactly the anomalous one.

The cycle in the accumulation chain. The Pasta cycle renders each step of §7.3 native. A proof committed on Vesta is produced by a circuit native to $\mathbb{F}_{p_{\text{Pallas}}}$; that circuit verifies the previous proof of the chain, committed on Pallas, whose group elements, the column and quotient commitments, the pairs (L_j, R_j) and the point of the Pallas accumulator instance it forwards, have coordinates in exactly $\mathbb{F}_{p_{\text{Pallas}}}$, so the point arithmetic of the succinct checks and of the fold (36) is native. The next proof, committed on Pallas by a circuit native to $\mathbb{F}_{p_{\text{Vesta}}}$, verifies the Vesta proof by the symmetric argument. This cross-curve alternation avoids non-native emulation of the preceding curve’s coordinate field; as “What the cycle does and does not buy” said, it does not eliminate the scalar-field conversions.

The half Orchard uses. This is the part of the cycle in consensus. The Action circuit is native to $\mathbb{F}_{p_{\text{Pallas}}}$ and committed on Vesta (§3.8), and the statement it proves manipulates Pallas points: the note commitment cm , a Pallas point whose x -coordinate cm_x is a public input; the randomised verification key rk ; and the net value commitment cv^{net} , the last two exposed by their coordinates in the instance column of §4.2. Their coordinates lie in $\mathbb{F}_{p_{\text{Pallas}}}$, the circuit’s native field, by the very coincidence the cycle is built on, so the circuit computes on them natively, with gates such as the incomplete-addition gate of §4.1.2. The protocol specification records the arrangement: Vesta for the proof system, Pallas for the application circuit, both curves designed to be efficiently implementable inside a circuit though only Pallas is so used (protocol specification, § 5.4.9.6, “Pallas and Vesta”); ZIP 224 says the same in the language of this subsection, that the proposal uses half of the cycle, Pallas being the curve embedded in Vesta’s circuits, and that the full cycle is left for future use (ZIP 224, under “Curves”). One direction of Figure 20 is thus in consensus today; the return arrow, and everything in this section that depends on it, is not.

IV

Consensus Guide

Nakamoto consensus: the double-spend race, the arithmetic of confirmations, and the deployed most-work rule

Abstract

Assuming the *Math Guide*'s probability foundations and the hash and commitment foundations of the *Crypto Guide*, this volume of *The Zcash Arboretum* constructs Nakamoto consensus from first principles and quantifies the security it buys. It states the block tree and the most-work rule as the protocol specification defines them and as the deployed zebra implements them; builds, on the *Math Guide*'s foundations, the race-specific probability the analysis needs (memoryless clocks, the biased random walk, the negative binomial law); develops the exact first-seen-model double-spending analysis of Rosenfeld's *Analysis of hashrate-based double-spending* — the catch-up theorem, the confirmation formula, and the economics of the attack — with every derived number recomputed by script and the results instantiated for Zcash's proposed 25-second blocks; explains the draft NU7 transition to 25-second blocks and halving-preserving reissuance; and explains the protocol and implementation safety rails (difficulty adjustment, reorg windows, coinbase maturity, wallet confirmation policy) that frame the model's assumptions. Where the specification, the paper, and the deployed nodes disagree, the divergence is stated.

1 Scope, sources, and the two double-spends

The protocol layers above this one take a working ledger for granted. This volume documents the mechanism that selects that ledger: Nakamoto consensus — proof of work, the most-work rule, and the probabilistic finality they purchase. The treatment is quantitative throughout. Its spine is Meni Rosenfeld's *Analysis of hashrate-based double-spending* (11 December 2012; latest version 12 February 2014; arXiv:1402.2009), which gives an exact counterpart to the approximate analysis in Satoshi Nakamoto's whitepaper; we reprove its results in full, recompute its tables by script, and instantiate its conclusions for the proposed NU7 parameters.

Two distinct attacks are both called “double-spending”, and this volume treats exactly one of them. Two spends of the same shielded note reveal the same nullifier, and the protocol specification's §“Nullifier Sets” forbids a repeated nullifier within a transaction or valid block chain; such

a same-chain attempt is therefore invalid under the consensus rules. What consensus must prevent is the chain-level attack: paying a merchant on the chain everyone sees, then replacing that chain wholesale with another in which the payment never happened. No zero-knowledge proof (*Crypto Guide*, §“Interactive proofs, zero knowledge, and SNARKs”) rules this out; under the model developed here, the proof-of-work race gives the attack its probabilistic bound.

The quantitative argument assumes the *Math Guide*’s probability section (§“Probability, asymptotics, and computation”), which constructs probability from its definition — finite spaces, conditioning, independence, random variables, expectation — and extends it to the three rungs used here: countable additivity, continuity along monotone events, and density-defined laws (§“Beyond finite spaces: countable additivity, limits, and densities”). The block and anchor vocabulary uses the *Crypto Guide*’s hash and commitment sections. On that base, Section 3 constructs the race-specific instruments no lower volume owns — memoryless clocks, random walks, the negative binomial law; the algebra used is elementary. Readers wanting only the results can read Sections 2, 6, and 8 and take the theorems on faith.

1.1 Sources and their standing

Rosenfeld’s analysis supplies the probability and economic model, not consensus rules; its two printed tables are reproduced by exact arithmetic before instantiating the model here. The protocol specification and the relevant ZIPs define the protocol.

This volume targets the *draft NU7 rules* as of 23 September 2026: ZIP-259 selects ZIP-218, ZIP-237, and ZIP-2003. Activation heights remain unassigned, and some proposed limits and issuance rules are incomplete. These boundaries are stated where they affect a calculation; no unsigned constant is treated as a deployed rule.

Remark 1.1 (The paper’s own condition). The arXiv v1 PDF of the paper is lightly broken: it contains exactly two citations, both rendered as “[?]” (in the abstract and in §1), and no References section at all. Both plainly intend the Nakamoto whitepaper, and we read them so. The paper numbers only two of its displayed equations — its equation (1) is our Theorem 5.2, its equation (2) our equation (3) — and sets its two model assumptions as a bare numbered list; the *Assumption* environments of Section 4 are ours.

2 The block tree and the most-work rule

Zcash transactions are grouped into blocks, and every block names its parent by including the parent header’s hash (*Crypto Guide*, §“Hash functions and the random oracle model”); the genesis block alone has no parent. Blocks therefore form a tree rooted at genesis, and each root-to-leaf path — each *branch* — is one internally consistent version of history. Branches need not be consistent with one another: one branch can contain a transaction that conflicts with a transaction in a sibling branch, and that freedom is exactly the attack surface this volume quantifies.

2.1 Work, and the best chain

A single coherent history requires a rule every node can apply locally and use to converge once one valid branch is strictly preferred. The rule is not simply “longest”: it weighs each block by the

computational effort represented by its header target.

Definition 2.1 (Work of a block). Let `ToTarget` be the conversion from the header’s `nBits` field to its 256-bit target threshold (specification §7.7.4). The *work* of a block is

$$\text{work} = \left\lfloor \frac{2^{256}}{\text{ToTarget}(\text{nBits}) + 1} \right\rfloor$$

(specification §7.7.5, “Definition of Work”). A lower target admits fewer valid headers, so a block meeting it certifies proportionally more expected hashing; work is that certificate made additive.

The deployed Zebra validator computes this quantity with a 256-bit in-word trick: zebra evaluates $(\text{!expanded.0} / (\text{expanded.0} + 1)) + 1$; the retired `zcashd` computed the identical expression on `arith_uint256`, with `~` spelling the complement. The form agrees with Definition 2.1 exactly: writing t for the target and $A = 2^{256}$, the complement is $\neg t = A - 1 - t$, and

$$\left\lfloor \frac{A - 1 - t}{t + 1} \right\rfloor + 1 = \left\lfloor \frac{A - (t + 1)}{t + 1} \right\rfloor + 1 = \left\lfloor \frac{A}{t + 1} \right\rfloor,$$

so the in-word computation equals the out-of-word quotient (checked by script over random targets).

Definition 2.2 (Best valid block chain). Specification §3.3 (“The Block Chain”): a node “sums the work, as defined in §7.7.5, of all blocks in each valid block chain, and considers the valid block chain with greatest total work to be best. To break ties between leaf blocks, a node will prefer the block that it received first.”

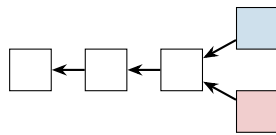
Rosenfeld’s paper states the same rule for Bitcoin — “the branch representing the most proof of work”, with first-seen tie-breaking — so the model uses the same selection rule. The shorthand “longest chain” is exact only while difficulty is constant, which is precisely the paper’s Assumption 4.2 and not a property of the deployed chain (Section 8.2). Note also what the rule does *not* say: nothing marks a chain as permanently chosen. A node that learns of a heavier branch switches to it, however deep the fork point — subject only to the node-local rails of Section 8.4. Consensus is a standing competition, not an election with a winner.

Definition 2.3 (Confirmations). A transaction has n *confirmations* if it is included in a block of the best chain and there are n blocks on the path from that block to the chain’s leaf, inclusive. The block containing the transaction is its first confirmation.

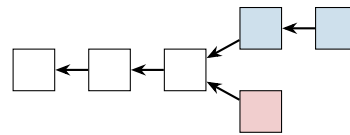
Different nodes may briefly disagree on the best chain when two blocks of equal cumulative work arrive in different orders. Absent a concurrent extension of the other branch, a later block makes one branch strictly heavier, and nodes converge on it once that block propagates (Figure 1). Such ties are the benign case. The attack of Section 5 is the malicious one: a fork manufactured in private and released only when it wins.

2.2 The attack

The double-spend attack, as the paper lays it out (§3):



(a) tied branches: nodes disagree



(b) one more block makes the upper branch heavier

Figure 1. Branch selection. Arrows point from child to parent. In panel (a) two blocks extend the same parent and the branches carry equal work; each node keeps the one it saw first. In panel (b) a new block lands on the upper branch, which now has strictly more cumulative work; during propagation, every node that knows this state selects the upper branch, unless the lower branch has meanwhile been extended. At constant difficulty “more work” and “longer” coincide, which is how the paper, and this volume’s figures, may draw block counts.

1. broadcast a transaction paying the merchant;
2. secretly mine a branch from the last pre-transaction block, containing instead a conflicting transaction paying the attacker himself;
3. wait until the merchant’s transaction has n confirmations and the merchant, satisfied, hands over the goods;
4. in the paper’s first-seen tie model, keep extending the secret branch until it carries more work than the public one, then broadcast it. Every node that sees it switches to it, the payment to the merchant is no longer part of history, and the attacker has both the goods and the coins.

Nothing in step 4 violates any validity rule — the released branch is a perfectly well-formed chain, merely a different one. The only question is probabilistic: with what probability does the secret branch ever get ahead? Answering it exactly in that first-seen model is the business of Sections 4 and 5; the toolkit comes first.

3 A probability toolkit

The analysis ahead needs four tools: memoryless clocks (to model block discovery), a reduction from continuous time to a coin sequence, the negative binomial distribution (to count the attacker’s progress), and one normalisation lemma about races. Their foundations are the *Math Guide*’s: probability spaces, conditioning, independence, random variables, and expectation are constructed there from the definition of probability onward (§“Probability, asymptotics, and computation”), and the extension beyond finite spaces — countable additivity, continuity along monotone events, density-defined laws, and infinite sequences of independent trials — is its §“Beyond finite spaces: countable additivity, limits, and densities”. What no lower volume owns are the four instruments themselves; each is constructed here on that base, only as far as the critical path requires.

3.1 Memoryless clocks and the attribution sequence

Definition 3.1 (Exponential clock). A random variable X is *exponential with rate* $\lambda > 0$ if $\Pr[X > t] = e^{-\lambda t}$ for all $t \geq 0$ — a law defined by the density $\lambda e^{-\lambda t}$ on $t \geq 0$ in the sense of the *Math Guide*’s density definition, with mean $1/\lambda$.

Lemma 3.2 (Memorylessness). *An exponential X satisfies $\Pr[X > s + t \mid X > s] = \Pr[X > t]$ for all $s, t \geq 0$: having waited without success teaches nothing about the remaining wait.*

Proof. $\Pr[X > s + t \mid X > s] = e^{-\lambda(s+t)} / e^{-\lambda s} = e^{-\lambda t}$. $\square$

Lemma 3.3 (Race of two clocks). *Let X and Y be independent exponentials with rates λ and μ . Then*

$$\Pr[X < Y] = \frac{\lambda}{\lambda + \mu},$$

and the winning time $\min(X, Y)$ is exponential with rate $\lambda + \mu$.

Proof. Integrating over the time at which X fires,

$$\Pr[X < Y] = \int_0^\infty \lambda e^{-\lambda t} \Pr[Y > t] dt = \int_0^\infty \lambda e^{-(\lambda+\mu)t} dt = \frac{\lambda}{\lambda + \mu}.$$

For the minimum, $\Pr[\min(X, Y) > t] = \Pr[X > t] \Pr[Y > t] = e^{-(\lambda+\mu)t}$. Moreover, the joint density that X wins at time t factors as

$$\lambda e^{-(\lambda+\mu)t} = \frac{\lambda}{\lambda + \mu} (\lambda + \mu) e^{-(\lambda+\mu)t},$$

and similarly for Y . Thus the winner is independent of the winning time. $\square$

Proposition 3.4 (The attribution sequence). *Model the honest network and attacker as independent exponential clocks with rates p/T_0 and q/T_0 , restarting when either fires (Section 4). Successive block attributions form an infinite sequence of independent, identically distributed Bernoulli trials in the *Math Guide*’s sense: attacker with probability q , honest network with probability p , independent of all earlier attributions and all elapsed times.*

Proof. By Lemma 3.3 the first block is the attacker’s with probability $(q/T_0) / (p/T_0 + q/T_0) = q$. When a block is found, the finder’s clock restarts by construction, and the loser’s remaining wait is distributed as a fresh clock by Lemma 3.2; the state after each block is therefore probabilistically identical to the start, independent of the past. The density factorisation in Lemma 3.3 also makes each winner independent of that race’s elapsed time. The claim follows by induction. $\square$

Proposition 3.4 is the paper’s bridge (its §3, with footnote 1 supplying the clock rates) from physical time to combinatorics: since the question “does the attacker ever get ahead?” concerns only the *order* of block discoveries, not their times, every result below is a statement about a p/q coin sequence, and the time constant T_0 disappears from the answers. Where the exponential model itself comes from — and how well Zcash’s Equihash fits it — is a deployment question, taken up in Section 8.3.

3.2 The negative binomial law

Proposition 3.5 (Progress of the loser). *In an attribution sequence with attacker probability $q = 1 - p$, let m be the number of attacker blocks found strictly before the honest network's n -th block. Then*

$$P(m) = \binom{m+n-1}{m} p^n q^m, \quad m = 0, 1, 2, \dots$$

— the negative binomial distribution.

Proof. The event fixes the first $m+n$ trials exactly this far: the $(m+n)$ -th trial is honest (the n -th honest success), and among the first $m+n-1$ trials exactly m are the attacker's, in any of $\binom{m+n-1}{m}$ orders. Each such order has probability $p^{n-1} q^m \cdot p$. $\square$

Lemma 3.6 (First-to- n race). *In the same sequence, let H_n be the event that the honest network reaches n blocks before the attacker does, and A_n the reverse. Then H_n and A_n partition the sample space, and*

$$\begin{aligned} \Pr[H_n] &= \sum_{m=0}^{n-1} \binom{m+n-1}{m} p^n q^m, & \Pr[A_n] &= \sum_{b=0}^{n-1} \binom{b+n-1}{b} q^n p^b, \\ \Pr[H_n] + \Pr[A_n] &= 1. \end{aligned}$$

Proof. Among the first $2n-1$ trials one side already has n successes, so the race terminates; a tie is impossible because block counts advance one at a time. The event H_n says the n -th honest block arrives while the attacker holds $m \leq n-1$; summing Proposition 3.5 over those m gives the first sum, and A_n is the same computation with the roles of p and q swapped. $\square$

Remark 3.7 (From hash trials to exponential clocks). The exponential model is itself a limit, not an axiom. A miner's work is modelled as a stream of candidate evaluations, each an independent Bernoulli trial with a tiny success probability; the number of trials to the first success is geometric, and a geometric law with small success probability, viewed at the scale of its mean, converges to the exponential of Definition 3.1. The step that matters is *progress-freeness*: a candidate that fails must carry no information usable by the next one. Whether Zcash's Equihash satisfies this is examined with the deployed parameters in Section 8.3.

4 Playing catch-up

The attacker of Section 2.2 finds himself, at the moment the merchant ships, some number of blocks behind the public chain, and wins if his branch ever overtakes it. This section computes the probability of that event as a function of the deficit; the next section supplies the distribution of the deficit itself.

Both computations run under the paper's two standing assumptions, stated here nearly verbatim (the environment form is ours, Remark 1.1):

Assumption 4.1 (Constant hashrates). The total hashrate of the honest network and the attacker is constant. Combined they have a hashrate of H , of which pH belongs to the honest network and qH to the attacker, where $0 < p, q < 1$ and $p + q = 1$.

Assumption 4.2 (Constant difficulty). The mining difficulty is constant, and such that with hashrate H the average time to find a block is T_0 .

Under Assumption 4.2 every block carries equal work, so “most work” and “longest” coincide and the race can be scored in block counts. Section 8.2 explains how the deployed chain departs from these assumptions — difficulty adjusts after every block and hashrate drifts — and why the formulas should be read as an equal-difficulty baseline rather than an exact deployment model.

Definition 4.3 (The deficit walk). Let z denote the honest chain’s lead in blocks over the attacker’s branch. By Proposition 3.4, each newly found block moves z up by 1 with probability p (honest block) or down by 1 with probability q (attacker block), independently of the past:

$$z_{i+1} = \begin{cases} z_i + 1 & \text{with probability } p, \\ z_i - 1 & \text{with probability } q. \end{cases}$$

The attack succeeds the moment z reaches -1 : the secret branch is then strictly ahead, and releasing it rewrites history.

Theorem 4.4 (Catch-up probability; Rosenfeld §3). *Let a_z be the probability that the walk of Definition 4.3, started at deficit z , ever reaches -1 . Then*

$$a_z = \min(q/p, 1)^{\max(z+1, 0)} = \begin{cases} 1 & \text{if } z < 0 \text{ or } q \geq p, \\ (q/p)^{z+1} & \text{if } z \geq 0 \text{ and } q < p. \end{cases} \quad (1)$$

Proof. For $z < 0$ the walk has already succeeded, so $a_z = 1$; assume $z \geq 0$. Fix an integer $N > z$ and absorb the walk at both -1 and N ; let $h_N(z)$ be the probability of reaching -1 before N . Conditioning on the first step,

$$h_N(z) = p h_N(z+1) + q h_N(z-1) \quad (0 \leq z \leq N-1), \quad h_N(-1) = 1, \quad h_N(N) = 0,$$

a second-order linear recurrence. The trial solution $h_N(z) = t^z$ satisfies it exactly when t is a root of the *characteristic polynomial* $pt^2 - t + q = p(t-1)(t-q/p)$.

If $q \neq p$ the roots 1 and q/p are distinct, so $h_N(z) = A + B(q/p)^z$; the boundary conditions give

$$h_N(z) = \frac{(q/p)^z - (q/p)^N}{(q/p)^{-1} - (q/p)^N}.$$

If $q = p$ the root is double and $h_N(z) = A + Bz$, giving $h_N(z) = (N-z)/(N+1)$.

Now let $N \rightarrow \infty$. The events $E_N = \{\text{reach } -1 \text{ before } N\}$ are nondecreasing in N , and their union is the event that the walk ever reaches -1 : a walk that does so does it at a finite time, having visited finitely many states, so E_N holds for every N above the largest state visited. By continuity

along monotone events (*Math Guide*, §“Beyond finite spaces: countable additivity, limits, and densities”), $a_z = \lim_{N \rightarrow \infty} h_N(z)$. For $q < p$ the term $(q/p)^N$ vanishes and the quotient tends to $(q/p)^{z+1}$; for $q = p$, $(N - z)/(N + 1) \rightarrow 1$; for $q > p$, dividing numerator and denominator by $(q/p)^N$ sends both to -1 , so the quotient tends to 1. $\square$

Corollary 4.5 (Half or more always catches up). *When $q \geq p$, the attacker overtakes the public chain with probability 1 from any finite deficit. No confirmation count defends when the attacker controls at least half of the hashrate.*

Corollary 4.6 (Geometric decay in the deficit). *When $q < p$, each additional block of deficit multiplies the catch-up probability by q/p . Concretely (script-computed): at $q = 10\%$, $a_0 = 1/9 \approx 0.1111$ and $a_5 \approx 1.88 \times 10^{-6}$; at $q = 30\%$, $a_0 = 3/7 \approx 0.4286$ and $a_5 \approx 6.20 \times 10^{-3}$.*

Corollary 4.7 (Blocks, not time). *Equation (1) contains neither T_0 nor any other time scale: the security of a deficit is a function of block counts alone. Two chains with different block spacings but the same q offer identical security per confirmation — and therefore different security per minute (Section 6.1).*

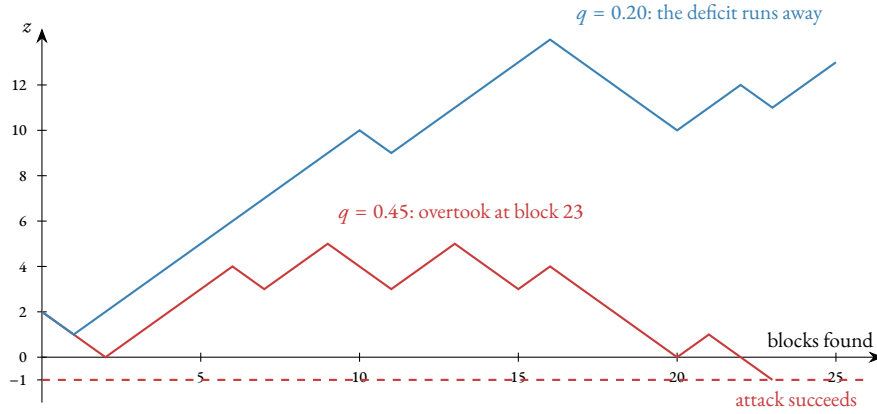


Figure 2. Two runs of the deficit walk from $z = 2$, simulated by script with fixed seeds. At $q = 0.45$ the drift is weak and this run reaches -1 after 23 blocks — the attack succeeds; at $q = 0.20$ the deficit grows roughly linearly, and by Theorem 4.4 the chance of ever returning from deficit 13 is $(1/4)^{14} \approx 3.7 \times 10^{-9}$.

5 Waiting for confirmations

The merchant’s defence is patience: ship only after the paying transaction has n confirmations (Definition 2.3). This section computes exactly how much that patience buys in the paper’s first-seen model. The paper’s model of the attack timeline has one convention worth surfacing before the theorem.

Remark 5.1 (The pre-mined block). The paper assumes the attacker banks one block before the race is scored: “we assume one block was pre-mined by the attacker before commencing the attack” (§4). The attacker forks from the public leaf at the moment the merchant’s transaction is

broadcast, and a patient attacker times the broadcast to coincide with a private block he has just found but not announced. At the moment the honest chain reaches n confirmations, the attacker who has found m further blocks therefore holds $m + 1$, and the deficit walk of Definition 4.3 starts at

$$z = n - m - 1.$$

The convention is a modelling choice, not a law: an attacker with no banked block starts at $z = n - m$, and every probability below shrinks accordingly. It is also, as Theorem 5.2 shows, precisely the choice that makes the final formula collapse into a symmetric race.

By Proposition 3.5, the number m of attacker blocks found while the honest network finds its n is negative binomial. Averaging the catch-up probability of Theorem 4.4 over m gives the central result for the specification and Rosenfeld first-seen tie rule. Section 8.1 explains the additional success opportunity supplied by Zebra's tie-break. The model is not an exact deployed rollback probability.

Theorem 5.2 (Double-spend success probability; Rosenfeld eq. (1)). *Let the merchant wait for $n \geq 1$ confirmations, under Assumptions 4.1 and 4.2 and the convention of Remark 5.1, with a private branch accepted only after it becomes strictly heavier. The probability that the attacker's branch eventually overtakes the public chain is $r(q, n) = 1$ if $q \geq p$, and otherwise*

$$r(q, n) = 1 - \sum_{m=0}^{n-1} \binom{m+n-1}{m} (p^n q^m - p^m q^n) = 2 \sum_{m=0}^{n-1} \binom{m+n-1}{m} p^m q^n. \quad (2)$$

Proof. Conditioning on m and applying Theorem 4.4 at $z = n - m - 1$,

$$r = \sum_{m=0}^{\infty} P(m) a_{n-m-1} = \underbrace{\sum_{m=0}^{n-1} P(m) \left(\frac{q}{p}\right)^{n-m}}_{\text{behind: must catch up}} + \underbrace{\sum_{m=n}^{\infty} P(m)}_{\text{already ahead}},$$

valid for $q < p$; when $q \geq p$ every $a_{n-m-1} = 1$ and $r = \sum_m P(m) = 1$ immediately. For the first sum, each term transforms exactly:

$$\binom{m+n-1}{m} p^n q^m \left(\frac{q}{p}\right)^{n-m} = \binom{m+n-1}{m} p^m q^n,$$

which by Lemma 3.6 is the probability that the attacker's n -th block arrives while the honest network holds m ; summed over $m \leq n - 1$, the first sum is $\Pr[A_n]$, the probability that the attacker wins the first-to- n race. The second sum is $\Pr[m \geq n] = 1 - \Pr[H_n] = \Pr[A_n]$ by the same lemma. Hence $r = 2\Pr[A_n]$, the right-hand form of (2); the middle form follows from $\Pr[A_n] = 1 - \Pr[H_n]$ applied to one of the two copies. The printed upper limit in the paper's equation (1) is n rather than $n - 1$; the $m = n$ term is identically zero, so the forms agree. $\square$

Remark 5.3 (The race doubling). The proof exposes a structure the paper's algebra passes through silently: under the one-pre-mined-block convention, the *catch-up half* — the honest chain reaches n first and the attacker later overtakes — has exactly the same probability as the *already-ahead*

half, in which the attacker reaches n first. A first-to- n winner may have fallen behind and recovered along the way; neither half asserts otherwise. Thus the double-spend probability is twice the chance of winning a first-to- n race. The identity also absorbs the boundary case: at $q = p$ each race half is exactly $\frac{1}{2}$, giving $r = 1$ with no case split, continuously with the $q > p$ regime. The algebra here is the paper's; reading its two sums as the two halves of one race, and the observation that the pre-mine convention is what makes them equal, are this volume's presentational additions (verified by script over random (q, n) ; the two sums agree to floating-point precision).

Example 5.4 (A full race, exactly). Take $q = 1/5$ (a twenty-percent attacker) and $n = 6$. The deficit starts at $z = 5 - m$, the catch-up factor is $(q/p)^{6-m} = 4^{-(6-m)}$, and every quantity below is an exact rational, printed to six decimals (script-computed):

m	$P(m)$	$a_{5-m} = 4^{-(6-m)}$	product
0	0.262144	0.000244	0.000064
1	0.314573	0.000977	0.000307
2	0.220201	0.003906	0.000860
3	0.117441	0.015625	0.001835
4	0.052848	0.062500	0.003303
5	0.021139	0.250000	0.005285
catch-up half (sum of products)			0.011654
already-ahead half, $\Pr[m \geq 6]$			0.011654
$r(0.2, 6)$			0.023308

The two halves agree to every printed digit — they are equal exactly, by Remark 5.3 — and the closed form (2) evaluates to the same 0.023308. A merchant facing a possible twenty-percent adversary who ships after six confirmations is assigned a 2.33% strict-overtake probability by this model.

Remark 5.5 (Satoshi's approximation). The whitepaper's §11 computes the same quantity with one approximation and two offsetting conventions: the attacker's progress is approximated as Poisson with mean $\lambda = nq/p$ (the law $\Pr[k] = e^{-\lambda} \lambda^k / k!$ on the nonnegative integers); no block is pre-mined, but success is scored at *breakeven* rather than strictly ahead, and those two conventions cancel exactly — the whitepaper's catch-up factor for m attacker blocks is $(q/p)^{n-m}$, term for term the factor of Remark 5.1. The Poisson approximation is the entire gap. In each of the following script-computed comparisons it pushes the estimate down: at $q = 10\%$, $n = 6$, the whitepaper's formula gives 0.0243% against the exact 0.0591% — an understatement by a factor of about 2.4; at $q = 30\%$, $n = 6$: 13.211% against 15.645%; at $q = 10\%$, $n = 10$: 0.0001% against 0.0008%. The direction is not uniform over all (q, n) ; the reason to prefer the negative binomial model is that it is exact under the stated assumptions, not that the approximation is always one-sided.

Table 1. The double-spend success probability $r(q, n)$, in percent, computed by script from Theorem 5.2. On the rows the paper’s Table 1 also prints ($q \in \{2, 10, 20, 30, 40, 48, 50\}\%$), the script reproduces its values cell-for-cell; the 5% and 45% rows are this volume’s additions.

q	$n=1$	2	3	4	5	6	7	8	9	10
2%	4.000	0.237	0.016	0.0011	$8 \cdot 10^{-5}$			$< 10^{-5}$		
5%	10.000	1.450	0.232	0.039	0.0066	0.0012	$2.1 \cdot 10^{-4}$	$4 \cdot 10^{-5}$	$< 10^{-5}$	
10%	20.000	5.600	1.712	0.546	0.178	0.059	0.020	0.0067	0.0023	0.0008
20%	40.000	20.800	11.584	6.669	3.916	2.331	1.401	0.848	0.516	0.316
30%	60.000	43.200	32.616	25.207	19.762	15.645	12.475	10.003	8.055	6.511
40%	80.000	70.400	63.488	57.958	53.314	49.300	45.769	42.621	39.787	37.218
45%	90.000	85.050	81.375	78.342	75.716	73.375	71.252	69.299	67.487	65.793
48%	96.000	94.003	92.508	91.264	90.177	89.201	88.307	87.478	86.703	85.972
50%					100 for every n					

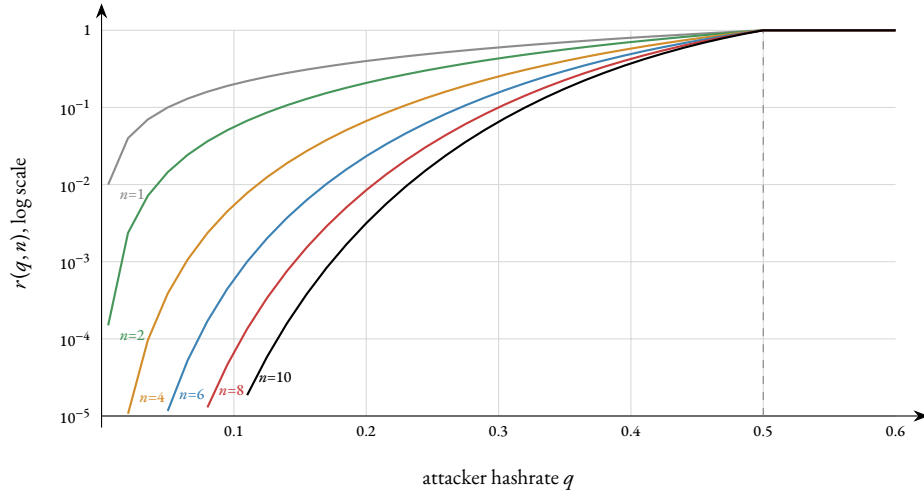


Figure 3. The success probability $r(q, n)$ against the attacker’s hashrate share q , for $n \in \{1, 2, 4, 6, 8, 10\}$ confirmations, logarithmic vertical scale clipped at 10^{-5} ; coordinates computed by script from Theorem 5.2. Every curve reaches 1 at $q = \frac{1}{2}$ (dashed) and stays there: beyond the majority line, confirmations are worthless. Below it, each added confirmation drops r by a roughly constant factor — the curves are near-equally spaced in log scale — and the factor improves as q falls.

6 What the numbers say

Theorem 5.2 compresses the security of Nakamoto consensus into one two-parameter function, and its qualitative lessons are all visible in Table 1 and Figure 3. This section states them precisely, then instantiates the one lesson that is specific to Zcash.

Table 2. The least number of confirmations pushing $r(q, n)$ below three targets, computed by script. The growth is logarithmic in the inverse target risk but explosive in q : a 10% attacker is answered by 6 confirmations, a 45% attacker by 539, and at 50% by no number at all.

q	$r < 10\%$	$r < 1\%$	$r < 0.1\%$
2%	1	2	3
5%	2	3	4
8%	2	3	5
10%	2	4	6
15%	3	6	9
20%	4	8	13
25%	5	12	20
30%	9	20	32
35%	15	36	58
40%	34	82	133
45%	135	331	539

Proposition 6.1 (No finite count is final). *For every $0 < q < p$ and every n , $r(q, n) \geq 2q^n > 0$.*

Proof. By Theorem 5.2, $r = 2\Pr[A_n]$, and one way for the attacker to win the first-to- n race is to find the first n blocks outright, an event of probability q^n . $\square$

Proposition 6.2 (Divergence at the majority line). *Fix a target $0 < \varepsilon < 1$. For $0 < q < \frac{1}{2}$, let $n_\varepsilon(q)$ be the least n with $r(q, n) < \varepsilon$. Then $n_\varepsilon(q) \rightarrow \infty$ as $q \uparrow \frac{1}{2}$.*

Proof. First, $n_\varepsilon(q)$ exists. The event A_n that the attacker reaches n blocks first is the event that at least n of the first $2n - 1$ block discoveries are the attacker's. Since $q < p$, each such length- $(2n - 1)$ sequence has probability at most $q^n p^{n-1}$, and there are fewer than 2^{2n-1} sequences. Hence

$$r(q, n) = 2\Pr[A_n] \leq \frac{(4pq)^n}{p} \longrightarrow 0,$$

because $4pq < 1$.

For fixed n the function $q \mapsto r(q, n)$ is a polynomial on $[0, \frac{1}{2}]$, hence continuous, and $r(\frac{1}{2}, n) = 1$: at $q = p$ each half of the race of Remark 5.3 has probability exactly $\frac{1}{2}$ by Lemma 3.6 and symmetry. Given any N , for each $1 \leq n \leq N$ there is a $\delta_n > 0$ with $r(q, n) > \varepsilon$ whenever $q > \frac{1}{2} - \delta_n$. Taking $\delta = \min_{1 \leq n \leq N} \delta_n > 0$ excludes every $n \leq N$ throughout that neighbourhood, so $n_\varepsilon(q) > N$. Since N was arbitrary, the claimed divergence follows. $\square$

Remark 6.3 (Nothing is special about six). The paper (§5): “There is nothing special about the default, often-cited figure of 6 confirmations. It was chosen based on the assumption that an attacker is unlikely to amass more than 10% of the hashrate, and that a negligible risk of less than 0.1% is acceptable. Both these figures are arbitrary” — six confirmations are “overkill for casual attackers, and at the same time powerless against more dedicated attackers”. Table 2 is the honest replacement: pick the adversary you defend against and the risk you accept, and read off n ; the pair (q, ϵ) is a policy, not a law of nature.

6.1 Zcash time: 25-second blocks

Corollary 4.7 says security is purchased in blocks. The wall-clock price is set by the target spacing: 25 seconds in ZIP-218. Since public discoveries arrive at rate p/T_0 , waiting for n confirmations takes nT_0/p seconds in expectation in this model. Three confirmations cost a nominal 75 seconds and ten cost 250 seconds when all hashpower publishes on the public chain. With a 10% withholding attacker, the expected wait is instead divided by 0.9.

For example, a nominal thirty-minute wait buys 72 confirmations at 25-second spacing, compared with three at Bitcoin’s 600-second spacing. The corresponding strict-overtake probabilities against a 10% attacker are approximately 9.35×10^{-34} and 1.712%, respectively; either count takes $30/0.9 \approx 33.3$ minutes in expectation under withholding. These are model calculations, not guarantees about either network.

The comparison holds q fixed. It contains no network and therefore does not price honest blocks racing one another during propagation. Quantifying that lost work requires a propagation distribution and mining topology. Moreover, a policy waiting for a fixed amount of accumulated miner reward asks a different question: decreasing the reward per block in proportion to the target spacing requires proportionally more blocks to reach that amount. Ignoring transaction fees (value paid for transaction inclusion) and rounding, the shorter spacing then cancels out of the mean wait. The monetary and fixed-hashrate models are not interchangeable.

Remark 6.4 (Where the model bends). Three assumptions do not survive contact with the deployed chain; they determine how cautiously the calculations may be transferred to it. (i) *Difficulty is not constant*: Zcash retunes the target after every block (Section 8.2). The race is scored in work, not block counts, and q may be interpreted approximately as the attacker’s share of work production while the branches’ block weights remain close; once their difficulty schedules diverge, the equal-step random-walk formula is no longer exact. (ii) *Hashrate is not constant*: q in the theorems is whatever share the attacker sustains for the attack’s duration; the paper’s own caveat is that T_0 re-enters only if the attacker cannot sustain his hashrate long enough, which it judges unlikely for a serious attacker. (iii) *Propagation is not instantaneous*: honest self-orphaning wastes honest work and thereby raises the attacker’s effective share; its size is not quantified here. A fourth gap is not the model’s: the attacker of this volume follows the protocol’s validity rules perfectly and attacks only the *choice* between valid histories. Attacks on the rules themselves are other volumes’ subjects.

7 The economics of the attack

The probabilities of Section 5 answer the adversarial merchant, who assumes the customer is an attacker. The paper’s §6 answers the realistic one: when is the attack worth mounting at all? Throughout this section $q < p$ — “[o]therwise, all bets are off” (§6) — and the model is deliberately stylised; its own caveats close the section.

The attacker targets k merchants simultaneously with payments of v each, all invalidated by the same secret branch; the goods are worth αv to him, $0 < \alpha \leq 1$; he gives up after mining o blocks in vain; and each of his blocks would earn the block income B if it ended up in the accepted chain. If the attack succeeds his blocks are all accepted (he keeps goods *and* coins *and* rewards); if it fails he has paid kv , keeps goods worth $k\alpha v$, and his o orphaned blocks forfeit oB . For an exact finite-stop profit calculation, r would have to be the success probability of that stopping policy. Following the paper, the tables below instead insert the unlimited-horizon $r(q, n)$ as a heuristic proxy. They are not an exact evaluation of a strategy that always stops after $o = 20$ private blocks.

Proposition 7.1 (Profitability; Rosenfeld §6). *Given success probability r and these stipulated payoffs, the attacker’s expected profit over not attacking is*

$$k\alpha v - (1 - r)(kv + oB) = kv(\alpha + r - 1) - (1 - r) oB,$$

positive if and only if

$$v > \frac{(1 - r) oB}{k(\alpha + r - 1)}$$

(when $\alpha + r > 1$; when $\alpha + r \leq 1$ the modeled attack is never strictly profitable). This modeled attack is not strictly profitable whenever

$$v \leq \frac{o(1 - r)B}{k(\alpha + r - 1)} \stackrel{\alpha=1}{=} \frac{o}{k} \cdot \frac{B(1 - r)}{r}, \quad (3)$$

which with the paper’s choices $o = 20$, $k = 5$, $\alpha = 1$, $B = 25$ BTC is its equation (2), $v \leq 100(1/r - 1)$ BTC.

Proof. The gain $k\alpha v$ is certain (the goods are obtained either way); the loss $kv + oB$ is paid exactly when the attack fails, with probability $1 - r$. Rearranging the positivity condition gives the threshold; substituting the paper’s constants gives $20 \cdot 25 \cdot (1 - r)/(5r) = 100(1 - r)/r$. $\square$

7.1 Reward-normalised exposure

The *block subsidy* is value issued by the protocol in a block, distinct from fees transferred by its transactions. Its *scheduled* component is base issuance determined by height, before any reserve payout. A *funding stream* is a required allocation of a share of that subsidy to a designated recipient. The *coinbase* is the block’s first transaction, which distributes the available subsidy and fees according to the applicable rules. The *reserve* is the accounting balance of funds removed from circulation and awaiting reissuance.

The income B forfeited by an orphaned block is the amount its miner would have retained: miner subsidy plus retained transaction fees, not the subsidy allocated to other recipients. Under

NU7 this depends on scheduled issuance, the prior reserve balance, and the applicable fee-removal rule (Section 8.8). A universal ZEC figure would therefore hide assumptions about both chain state and unresolved draft rules.

Instead, Table 3 measures transaction value in units of the chosen block income B . With the paper's $o = 20$, $k = 5$, and $\alpha = 1$, equation (3) becomes

$$\frac{v}{B} \leq 4 \frac{1-r}{r}.$$

Once a particular threat model supplies B , multiplying by it gives the corresponding monetary threshold. Treating B as constant over the attack remains a modelling assumption.

Table 3. The largest whole-number multiple of block income B with non-positive heuristic expected profit: $\lfloor 4(1-r)/r \rfloor$, computed by script. The table is dimensionless: it neither fixes a ZEC subsidy nor predicts future fee income. Values inherit every assumption of the model; the caveats of Remark 7.3 apply in full.

q	$n=1$	2	4	6	8	10
5%	36	271	10,327	344,743	10,931,506	336,737,801
10%	16	67	729	6,759	59,475	508,917
20%	6	15	55	167	467	1,262
30%	2	5	11	21	35	57
40%	1	1	2	4	5	6

Within the model, a handful of confirmations makes the attack unprofitable against small attackers at any plausible payment size; against a 30% attacker, six confirmations make modeled expected profit non-positive only up to about 21 times the assumed block income B . The required count grows only logarithmically in the value at stake, by the geometric decay of Theorem 5.2. These are neither exact finite-stop profit calculations nor bounds on deployed rollback risk; the deployed tie-break, changing difficulty, and rollback rails require separate modelling.

Remark 7.2 (Majority is a different problem). At $q = p$, eventual strict overtake has probability 1, but the walk has zero drift; that fact alone does not give the attacker a lasting monopoly. When $q > p$, the economics change character entirely: the paper notes a strict majority attacker can double-spend at no cost beyond normal mining, reject every other miner's blocks to take the whole issuance, and exclude transactions at will, driving honest miners out and entrenching himself — so a majority attack “is best seen as an attempt to destroy” the currency, motivated from outside it (a short position, a competing system), not an attempt to profit inside it. Confirmation policy is not the defence at that point; the defence is the cost of assembling the majority, which lies outside this model.

Remark 7.3 (The model's own caveats). The paper flags each stylisation itself. The give-up point $o = 20$ “is a significant simplification” (an optimal attacker balances completion against compounding losses); the safe values “should be taken with a grain of salt, because of the many modeling assumptions”; and a model resting on mining *costs* rather than forfeited *rewards* would make the required confirmations linear, not logarithmic, in the transaction value — “very poor security, hence in a situation where this is relevant, we have already lost anyway” (§6). The bound's role is the shape it reveals — logarithmic patience buys exponential safety — not its third significant digit.

8 NU7: 25-second blocks and bounded shielded work

The ZIP-259 draft combines three changes: shorter blocks with limits on shielded work (ZIP-218), reissuance that preserves the halving schedule — periodic divisions of scheduled subsidy by two (ZIP-237) — and rejection of version 4 transactions (ZIP-2003). This section describes that proposal, not a deployed upgrade. Its activation height, denoted A , is unassigned on each network at the source boundary in Section 1.1.

NU7 introduces no new transaction format: versions 5 and 6 remain available. Their signatures nevertheless commit to a *consensus branch identifier*, a value identifying the network upgrade’s rules. An unchanged encoding therefore does not make a pre-NU7 signature valid under NU7. Disabling version 4 also disables spending funds in Sprout, an older shielded pool, because the newer formats cannot represent its transfers; it does not transfer their value to a reserve or remove it from the supply.

8.1 Chain selection and the tie-break

The most-work rule remains Definition 2.2: compare the sum of the blocks’ work, not their count. Changing target spacing changes neither the work definition nor this ordering.

Remark 8.1 (The implementation tie-break). The specification and Rosenfeld’s model prefer the first-seen block at equal cumulative work. The examined Zebra implementation instead uses the larger tip hash. This distinction concerns local chain choice, not the validity of a block. The divergence matters to the attack model. At equal work, an attacker can compare its private tip hash with the public tip and reveal immediately when its hash wins the deployed deterministic tie-break; the strict-overtake boundary $z = -1$ used above does not count that success opportunity. Therefore $r(q, n)$ is exact for the stated strict-overtake model and is a lower bound if only its tie-break is replaced by Zebra’s, keeping equal difficulty, unlimited rollback, and the other assumptions fixed. It is not thereby a lower bound on the full deployed process: changing difficulty and rollback limits also change the success event. Quantifying the extra probability, including any scope to choose among candidate private tips, requires a different stopping rule and is outside this volume.

8.2 Difficulty in motion

Assumption 4.2 freezes difficulty; Zcash retargets after every block. In ZIP-218, the NU7 target spacing is 25 seconds and the target-averaging window is 102 blocks. The timestamps determining the adjustment still come from the branch being evaluated, including on an attacker’s private branch.

Write $N = 102$ for the target-averaging window and $T = N \cdot 25 = 2,550$ seconds for its nominal timespan.

The specification’s §7.7.3 defines the rest of the calculation. For a block height b , let $m(b)$ be the median timestamp of the eleven preceding blocks: their sixth-smallest timestamp. Let $\bar{t}(b)$ be the mean target of the N blocks preceding b : the sum of their targets divided by N . Define the

observed and damped timespans by

$$\begin{aligned} a(b) &= m(b) - m(b - N), \\ d(b) &= T + \text{trunc}\left(\frac{a(b) - T}{4}\right), \end{aligned}$$

where trunc discards the fractional part towards zero. The bounded timespan $c(b)$ is $d(b)$ clamped between $[0.84T]$ and $[1.32T]$: values below the lower endpoint are replaced by that endpoint, and similarly above the upper endpoint. The maximum target PoWLimit is $2^{243} - 1$ on Mainnet, the production network, and $2^{251} - 1$ on Testnet, the test network (§5.3). The target calculation is

$$t(b) = \min\left(\text{PoWLimit}, \left\lfloor \frac{\bar{t}(b)}{T} \right\rfloor c(b)\right),$$

followed by the compact encoding defined in the specification’s §7.7.4. The order of the floor and multiplication matters. At NU7 the clamp endpoints are 2,142 and 3,366 seconds; the eleven-timestamp median, fourfold damping, and target cap are unchanged.

Activation does not reset the accumulated work or multiply the target by three in a single operation. The first NU7 blocks still look back over pre-NU7 history and adjust through the same bounded rule. Likewise, the timestamp rules do not shrink with the target spacing: a timestamp must exceed the median of the preceding eleven and be at most 90 minutes above it (§7.6). A further non-consensus receipt check rejects blocks more than two hours ahead of the validator’s clock. Testnet’s exceptional minimum-difficulty rule continues to use six target spacings: its delay threshold becomes $6 \cdot 25 = 150$ seconds. This exception does not apply to Mainnet.

Remark 8.2 (What per-block retargeting changes). The averaging window, damping, and clamp constrain the adjustment but do not prove that two forked branches retain close targets. Work per block moves inversely with the target; a branch’s timestamps can change both its future block rate and the work carried by each block. Equal expected work production does not make the probability of an eventual overtake independent of these jump sizes. Consequently, the probability tables are an equal-work baseline, not an exact analysis of private branches whose difficulty schedules diverge.

8.3 Equihash and the memoryless model

Zcash’s proof of work is Equihash with parameters $n = 200$, $k = 9$ (specification §7.7, §7.7.1; the algorithm is Biryukov and Khovratovich’s, ePrint 2015/946), but the quantity the race counts is gated one step later: “[t]he difficulty filter is unchanged from Bitcoin, and is calculated using SHA-256d on the whole block header (including solutionSize and solution)”, required to be at most $\text{ToTarget}(n\text{Bits})$ (§7.7.2). Each candidate Equihash solution therefore faces a fresh SHA-256d lottery. Treating distinct headers as independent trials is the usual random-function model for SHA-256d, not a property asserted by the specification; under that model these are the Bernoulli trials of Remark 3.7.

The specification nowhere asserts progress-freeness — the argument is this volume’s, not a citation. What Equihash changes is the granularity: candidates arrive in batches, one batch per

memory-hard solver run. Progress within a run is discarded when a competing block arrives. Under independent identically distributed runs, the number of runs until an accepted header is geometric; replacing those discrete completion times by an exponential clock is a further approximation, coarse within one run and good only when a run is short relative to the target spacing: 25 seconds in the NU7 draft. Run duration is implementation- and hardware-dependent and is not quantified here. A proof-of-work whose solve time were comparable to the spacing would make block discovery visibly non-exponential and pull the attribution sequence away from Proposition 3.4.

8.4 Reorg rails, maturity, and the finality floor

Consensus does not impose a maximum reorganisation depth. ZIP-218 recommends a node-local 600-block rollback window: at the target spacing this is $600 \cdot 25 = 15,000$ seconds, or 250 minutes. Such a limit bounds the history that a node automatically reverses; it is not a proof that older blocks cannot be replaced by a higher-work valid chain. Following a branch outside that retained history requires different local state or operator intervention. Actual node implementations may choose a different window.

Remark 8.3 (What the rails buy). A finite rollback policy truncates some paths to eventual overtake, so the unlimited-rollback probability $r(q, n)$ is not the exact failure probability of a node enforcing that policy. A limit is also not global agreement: different nodes may stop on different sides of a long split. The specification's §3.3 adds a separate requirement for validators that risk Mainnet funds or display transaction information: their chain must include the activation block of the most recent settled network upgrade. That socially anchored checkpoint is distinct from probabilistic finality.

A *transparent output* records its value and destination in cleartext. The consensus maturity rule prevents spending such an output of a coinbase until the spending block is at least 100 heights later (§7.1.2). ZIP-218 keeps that count, a nominal $100 \cdot 25 = 2,500$ seconds. Coinbase outputs placed directly in a shielded pool do not have this transparent maturity restriction. A 600-block rollback window can therefore reverse an already mature coinbase: maturity permits spending, not finality.

8.5 Wallet confirmation policy

A wallet can distinguish *trusted* outputs that its own transaction handling can recreate after a rollback from an *untrusted* external payment that may require the sender to pay again. The examined wallet backend uses three confirmations for the former and ten for the latter, with zero-confirmation shielding of transparent funds allowed by its default policy. These choices are not consensus rules.

Table 1 quantifies their modelled trade-off: three confirmations assign a 10% attacker risk 1.712%, while ten assign the same attacker about 0.0008% and a 20% attacker about 0.316%. The count remains a (q, ε) policy, not a guarantee.

The *anchor* is the note-commitment-tree root against which a spend proves membership (*Crypto Guide*, §Privacy-preserving membership via zero-knowledge paths). ZIP-218 keeps the recommended anchor depth at three blocks. Its recommended transaction-expiry delta is 120

blocks, a nominal $120 \cdot 25 = 3,000$ seconds, without overriding an explicitly configured expiry delta. There is no instruction to divide every confirmation count by three.

8.6 A block budget for shielded work

A byte limit does not directly bound the computational work inside a block. The 2,000,000-byte limit must therefore be considered alongside the 25-second target spacing and the costs of shielded validation and wallet scanning. ZIP-218 adds limits on the shielded components that drive this work.

An Orchard *Action* pairs one spend slot with one output slot; dummy slots allow either side to carry no real transfer. Ironwood is the newer shielded pool using the same Action structure (ZIP-229). The older Sapling shielded pool counts spends and outputs separately; its cryptographic construction is not needed for this accounting.

Let O and I count Orchard and Ironwood Actions in a block, and let S count Sapling spends plus outputs. With the Ironwood coverage proposed in PR 1361, the complete shielded budget is

$$O \leq 330, \quad I \leq 330, \quad S \leq 300, \quad O + I + S \leq 330.$$

The final inequality is shared across pools: their separate maxima cannot all be filled in one block. Sprout transfers cannot occur because NU7 rejects the version 4 format that represents them.

The Ironwood term remains under review rather than already specified by the merged ZIP-218 draft. For example, $O = 100, I = 200, S = 30$ exhausts the shared budget; increasing I to 201 violates it even though each per-pool limit still holds. At two Actions per transaction, a block devoted to one Orchard-protocol pool fits at most 165 such transactions under this budget, or a nominal $165/25 = 6.6$ transactions per second. This is a capacity calculation under a specified transaction shape, not a measured throughput guarantee. Transparent components remain subject to the byte limit rather than this shielded-action budget.

8.7 Scheduled issuance with shorter blocks

A *zatoshi* is the atomic currency unit: 1 ZEC equals 10^8 zatoshi. A *halving* divides the scheduled subsidy by two. Retaining its nominal wall-clock interval while tripling block frequency requires both three times as many blocks per interval and one third of the subsidy per block. ZIP-218 therefore changes the full interval from 1,680,000 to $L = 5,040,000$ blocks. These earlier intervals are needed to compute progress at activation, not to supply an alternative rule. The change preserves expected timing, not exact calendar dates, because actual block times remain random.

The progress already made towards the next halving must survive activation. Let H_b be the Blossom activation height: 653,600 on Mainnet and 584,000 on Testnet. Before Blossom, halvings were separated by 840,000 blocks; the initial slow-start schedule, which ramps up the subsidy during the first 20,000 blocks, contributes a shift of 10,000 blocks. These constants and the maximum subsidy $M = 1,250,000,000$ zatoshi are defined in the specification's §5.3 and §7.8. For a

height $b \geq A$, the draft's halving count and scheduled subsidy s_b , measured in zatoshi, are

$$k(b) = \left\lfloor \frac{H_B - 10,000}{840,000} + \frac{A - H_B}{1,680,000} + \frac{b - A}{5,040,000} \right\rfloor, \quad (4)$$

$$s_b = \left\lfloor \frac{M}{2 \cdot 3 \cdot 2^{k(b)}} \right\rfloor. \quad (5)$$

The factors 2 and 3 account separately for Blossom's and NU7's spacing changes. If activation occurs during the second-halving era, $k(b) = 2$ gives $s_b = 52,083,333$ zatoshi, or 0.52083333 ZEC, until the next halving; $k(b) = 3$ gives 26,041,666 zatoshi.

The old Mainnet third-halving height $H_3 = 4,406,400$ is therefore not the NU7 third-halving height if $A < H_3$. Converting the remaining interval gives

$$H_3^{\text{NU7}} = A + 3(H_3 - A) = 13,219,200 - 2A.$$

No numerical activation height is assigned here. Nor does retiming the halving function automatically retime a funding stream whose endpoint is specified as a literal block height. The draft does not provide such a change, so the funding-stream endpoints and the rescaled halving heights must be treated separately.

8.8 The NSM reserve and delayed reissuance

The *Network Sustainability Mechanism* (NSM) separates temporarily removed funds from the still-unissued scheduled subsidy. In the merged ZIP-237 draft, the *issued supply* V_b is the total value recorded in the chain's value pools after block b . Each pool is a separately tracked balance for one part of the protocol; tracking a shielded pool's total does not reveal the values of its individual notes. The *NSM reserve* R_b records value removed from circulation and not yet reissued. The reserve is public consensus state, not a spendable pool or an account with a spending key. In particular, it is not 21 million ZEC minus the issued supply: that larger difference also includes future scheduled issuance.

The initial reserve recovers the historical subsidy and fees that miners left unclaimed before NU6. Since ZIP-236, a coinbase must account for its full available value, so that historical shortfall is fixed. The draft defines the seed R_{A-1} as total scheduled issuance through the last pre-NU6 block minus issued supply there. The exact integer seeds for Mainnet and Testnet remain unspecified.

Let $D \geq A$ be the separately assigned height at which reissuance starts, and let d_b be the non-negative number of zatoshi removed from circulation in block b by whatever removal mechanisms have been deployed. With $L = 5,040,000$ blocks per halving interval, the draft selects

$$f = \frac{\lfloor 6,931,680,000/L \rfloor}{10^{10}} = \frac{1375}{10^{10}}.$$

Its additional subsidy u_b , total block subsidy b_b , and reserve update are

$$u_b = \begin{cases} 0, & b < D, \\ \lfloor fR_{b-1} \rfloor, & b \geq D, \end{cases} \quad (6)$$

$$b_b = s_b + u_b, \quad R_b = R_{b-1} - u_b + d_b. \quad (7)$$

All amounts are integers in zatoshi. Reissuance uses the previous block's reserve, so a removal in the current block cannot finance its own additional subsidy. Before D , removals accumulate without payout. After D , a positive reserve pays at least one zatoshi: because R is an integer and $0 < f < 1$, one has $1 \leq \lceil fR \rceil \leq R$ for $R > 0$. The draft rejects a negative reserve; it does not silently clamp it to zero.

For a small exact example, take $R_{b-1} = 100,000,000$ zatoshi (1 ZEC) and $b \geq D$. The additional subsidy is $\lceil 13.75 \rceil = 14$ zatoshi. Without a new removal the reserve becomes 99,999,986; with a same-block removal of another 1 ZEC it becomes 199,999,986, but the current payout remains 14. This example does not use the unassigned historical seed.

Why this preserves supply and halvings. Transfers between ordinary value pools cancel in their total. A block increases that total by its subsidy and reduces it by its removals:

$$V_b - V_{b-1} = s_b + u_b - d_b, \quad (V_b + R_b) - (V_{b-1} + R_{b-1}) = s_b.$$

The second identity follows by adding the reserve update; reissuance moves value between the two ledgers without creating a second copy. Scheduled issuance retains its halvings, with the reserve paying an additional amount on top. Funding streams, wherever active, take their prescribed fractions of the *total* subsidy b_b , not only s_b . The resulting subsidy depends on prior chain state as well as height.

To understand the chosen rate, temporarily ignore integer rounding and new removals. The recurrence becomes $R_{b+j} = (1 - f)^j R_b$, so keeping about half the reserve after L blocks requires $f = 1 - 2^{-1/L} \approx (\ln 2)/L$. The draft implements an integer-scaled approximation with numerator 1375. Indeed, $(1 - 1375/10^{10})^{5,040,000} \approx 0.50007$. This is a half-life of the reserve without new contributions, not a new halving rule for the scheduled subsidy; small reserves depart from the smooth curve because every positive payout rounds up.

Unresolved deployment and fee details. ZIP-259 selects a reissuance date corresponding to February 2031, but the network-specific height D remains unassigned. The drafts do not yet specify a complete, consistent NU7 removal rule: ZIP-235 requires removal of at least 60% of fees through an explicit coinbase amount, while ZIP-259 describes a block-level contribution requiring no new transaction data. The reserve equations therefore leave d_b as a defined input. Neither a particular fee-removal mechanism nor an exact NU7 miner income is inferred from these incomplete rules.

9 Layer boundary

Every theorem above rests on the *Math Guide*'s probability section: the definition of probability, its extension beyond finite spaces, and the continuity theorem used in the catch-up proof. This volume supplies the best-chain model and the quantitative bounds in Tables 1, 2, and 3. Policies that decide which chain view to trust or when to treat a transaction as final lie above that boundary.

V

Ironwood Guide

The Zcash Orchard protocol and its Ironwood pool

Abstract

This volume of *The Zcash Arboretum* constructs the Orchard protocol for a shielded payment in its Ironwood pool: how the payment is constructed, delivered to its recipient and verified; why a valid payment cannot create value, consume a note twice or consume a note without the authority of its owner; and what its public data hide. Every public field of the payment is constructed, in dependency order, as a fixed function of private notes, keys and randomness; the relation that each Action of the payment proves is stated over those objects; and each security property is a theorem under named assumptions. The sealed Orchard pool enters only where a rule shared with the Ironwood pool, or a rule that distinguishes the two pools, is needed to state the Ironwood-pool payment correctly. The volume assumes the *Math Guide* and the *Crypto Guide* and cites the *Halo 2 Guide* for the proof system. It is non-normative: the Zcash Protocol Specification and the Zcash Improvement Proposals are authoritative.

1 Introduction

This section fixes the subject of the volume, its status and its vocabulary before any construction: the scope; in plain language, the structure of a shielded payment and the division of its verification; the Orchard protocol and its two value pools; and the requirements that a shielded payment must meet.

1.1 Scope and status

This volume of *The Zcash Arboretum* constructs the Orchard protocol for a shielded payment in the Ironwood pool, both terms being defined in §1.3: how the payment is constructed, delivered and verified; why a valid payment cannot create value, consume a note twice, or consume a note without the authority of its owner; and what it hides. The sealed Orchard pool enters only where a rule shared with the Ironwood pool, or a rule that distinguishes the two pools, is needed to state the Ironwood-pool payment correctly. The volume assumes the *Math Guide* and the *Crypto Guide*,

and cites the *Halo 2 Guide* for the proof system. It is non-normative: the Zcash Protocol Specification, cited as the protocol specification with the title of a section, and the Zcash Improvement Proposals are authoritative.

Definition 1.1 (Block chain, transaction, consensus rules). A *block chain* is a sequence of blocks that starts at the *genesis block*, the header of each block referring to the preceding block; the *height* of a block is its number of predecessors, so that the genesis block has height 0 (ZIP 200, Terminology; protocol specification, §“The Block Chain”). Each block contains an ordered sequence of one or more *transactions*, the units in which the chain records transfers of value (protocol specification, §“Transactions and Treestates”). The *consensus rules* are the validation rules that determine which block chains, and hence which blocks and transactions, are valid (ZIP 200, Terminology, “Consensus rule set”).

Definition 1.2 (ZIP). A *Zcash Improvement Proposal* (ZIP) is a design document that describes a feature of Zcash, or a change to it, with a concise technical specification and a rationale. Each ZIP carries a status. Two statuses are named in this volume: *Draft*, the status that every ZIP receives on submission, and *Reserved*, the status of a ZIP number that the ZIP editors have reserved while no ZIP text has been published (ZIP 0, Abstract and “ZIP Status Field”). A deployed rule is cited to a section of the protocol specification by its title, or to a ZIP and its section.

Definition 1.3 (Network upgrade). A *network upgrade* is an intentional change of the consensus rules that takes effect at a block height fixed for it, its *activation height*: blocks below that height are validated under the preceding rules, and blocks at or above it under the new rules. Network upgrades are named and activate in a fixed order (ZIP 200, Terminology and “Activation mechanism”). This volume names two: NU6.3, whose deployment ZIP is ZIP 258, and NU7, the upgrade that follows NU6.3, whose deployment ZIP is ZIP 259 (ZIP 258, Abstract; ZIP 259, Abstract and Rationale, “Rationale for no new transaction format”).

The volume describes the protocol as specified for NU7, whose deployment ZIP has status Draft (ZIP 259, preamble). It names a network upgrade only where the name fixes which rule applies, gives no activation parameter, such as an activation height or a date, and describes no upgrade by what it adds or changes.

Definition 1.4 (Epistemic classes). Where the status of a claim about the deployed protocol needs stating, the volume places it in one of exactly two classes. The claim is *specified* when normative text of the protocol specification or of a ZIP fixes it. It is *designed but unspecified* when the deployed construction is designed to satisfy it but neither the protocol specification, nor a ZIP, nor a published proof establishes it for the deployed instance. An example of each class is given where its object is introduced: knowledge soundness of the Action proof in §1.2, and the cross-address restriction in §1.3.

1.2 The structure of a shielded payment

This subsection names, in plain language, the objects of a shielded payment and the checks that verify it. Each term receives one defining clause here and its construction in the section named.

Definition 1.5 (Notes, note commitments, nullifiers). A shielded payment transfers value held in *notes*. A note records, among other fields, an amount of value and the *address* of its recipient, a public value derived from the recipient’s keys (§3.3); no note is published in the clear (§4.1). For each note that a payment creates, the chain records a commitment to the note in the sense of the *Crypto Guide*, §“Hiding and binding”, its *note commitment*, in the extracted form constructed in §4.2. For each note that a payment consumes, the chain records a tag, the note’s *nullifier*, which marks the note as consumed (§6.1).

Definition 1.6 (Value pool). In this volume a *value pool* is a separately accounted collection of notes with its own record of note commitments, the *note commitment tree* (§5.1); its own record of nullifiers, the *nullifier set* (§6.2); and its own *chain value pool balance*, the total value that the chain accounts as held in the pool (§11.4).

Definition 1.7 (Action, dummy note). A shielded payment is carried by *Actions*. An Action consumes one note of a pool and creates one note of the same pool. A side of an Action that the payment does not use carries a *dummy note*, a note of value zero that pads the Action (§4.4; protocol specification, §“Action Transfers and their Descriptions” and §“Dummy Notes (Orchard)”).

Definition 1.8 (Bundle, balancing value). The Actions of one transaction in one pool, together with data common to them, form the transaction’s *bundle* for that pool; a transaction carries at most one bundle per pool (§4.4; ZIP 229, Abstract and “Transaction Format”). The *balancing value* of a bundle is the net value, public, that the bundle moves out of its pool into the rest of the transaction; it is negative when value enters the pool (§8.2).

Definition 1.9 (Note ciphertext). A created note reaches its recipient through the chain: the Action that creates it also carries a *note ciphertext*, an encryption of the note to the recipient’s address, from which the recipient recovers the note (§10).

Remark 1.10 (Division of verification). A *verifier*, a party that checks transactions against the consensus rules, accepts a bundle, without learning its notes, on one proof per bundle, two kinds of signature and two checks against the state of the chain (protocol specification, §“Action Transfers and their Descriptions” and §“Action Descriptions”).

- (1) One zero-knowledge proof per bundle, the *Action proof* (*Crypto Guide*, §“SNARKs: succinct non-interactive arguments of knowledge”; *Halo 2 Guide*, §“The Halo 2 proof system”): an argument that the prover knows, for each Action, private data that together with the Action’s public data satisfy one fixed relation, in the sense of the *Crypto Guide*, §“Languages, relations, and witnesses”, namely the Action statement, defined in “The Action statement” (§9.2). The proof system has no notion of value or ownership; the relation supplies both.
- (2) Per Action, a *spend-authorisation signature* (*Crypto Guide*, §“Syntax and security goal”), made with a signing key obtained from the *spend authorising key*, the signing secret associated with the consumed note’s address (§3.1), by which the holder of that key authorises the consumption (§7).

- (3) Per bundle, one *binding signature*, by which balance is checked: the total value of the notes that the bundle consumes, minus that of the notes it creates, equals its balancing value (§8.3).
- (4) Two checks against the state of the chain. The proof refers to one earlier state of the pool’s note commitment tree, through a digest of that state, the bundle’s *anchor* (§5.2), and the verifier checks that the anchor is one that the chain recorded for an earlier block. The verifier also checks that every nullifier that the bundle publishes is new to the pool (§6.2).

Both kinds of signature sign one digest of the transaction, constructed in “Transaction digests and signatures” (§11.3). The two checks of (4) lie outside the proof because they concern the history of the whole chain, which a proof over the data of one Action cannot attest.

Knowledge soundness of the Action proof as deployed (*Crypto Guide*, §“SNARKs: succinct non-interactive arguments of knowledge”), the property under which an accepted proof establishes the knowledge claimed in (1), is designed but unspecified: the volume takes it as Assumption 9.11 (§9.3), not as a result.

1.3 The Orchard protocol and its two pools

Definition 1.11 (Orchard protocol). The *Orchard protocol* is the cryptographic design shared by two value pools: its primitive instances, among them the Pallas and Vesta curves and the Sinsemilla hash (§2); its keys and addresses; its notes, note commitments and nullifiers; its note encryption; and the one relation that every Action proves, the Action statement, defined in “The Action statement” (§9.2), with the circuit designed to encode it (§9.3). ZIP 229 fixes this vocabulary (ZIP 229, Terminology), and the volume adopts it.

Definition 1.12 (Orchard pool, Ironwood pool). The *Orchard pool* and the *Ironwood pool* are the two value pools of the Orchard protocol. Each has its own note commitment tree, and hence its own anchors, its own nullifier set and its own chain value pool balance (ZIP 229, Terminology, and Rationale, “Separate state”).

The Ironwood pool admits net inflow, value moved into it by a negative balancing value (§11.4). The Orchard pool is *sealed*, in two respects (ZIP 258, “Consensus rules from NU6.3 activation”); each rule is stated where its object is constructed. First, no transaction increases the value held in the Orchard pool (§11.4). Second, every Orchard-pool Action creates its note at the address of the note it consumes (ZIP 229, Rationale, “enableCrossAddress polarity”), a rule made exact in “Bundle flags” (§9.1) and with the Action statement, defined in “The Action statement” (§9.2). ZIP 229, Abstract, states the effect of the second rule: outputs to the Orchard pool go to an address for which the creator of the transaction can authorise spends, which discourages transfers between users within the Orchard pool. Consensus does not prevent such transfers: any party able to authorise spends for an address can create an Orchard-pool note to that address, since the consumed side of the Action can be a dummy note at that address (ZIP 326, “Rationale for key-generation restrictions”; the witness of such an Action is exhibited in §9.2).

The restriction of an Action’s created note to the address of its consumed note, in force for every Orchard-pool Action and for every Ironwood-pool Action whose bundle does not enable

cross-address transfers (§9.1), is specified: the protocol specification, §“Action Descriptions”, fixes the meaning of the flag that controls it, although ZIP 2006, which that section cites for it, has status Reserved (ZIP 2006, preamble).

Notes of the two pools have the same form and the same note commitment. They differ only in the note plaintext format, the encoding in which note encryption carries a note: an Ironwood-pool note uses the format whose first byte, the *lead byte* that names the format, is $0x03$, under which the note’s *commitment trapdoor*, the randomness of its note commitment, is derived by hashing a random seed, which the plaintext carries, together with every field that the commitment binds (the format and the lead-byte rule in §4.3, the plaintext in §10.1; ZIP 258, “ZIP 2005 activation”). Actions of both pools prove the same relation, the Action statement, defined in “The Action statement” (§9.2), with one circuit and one *verifying key*, the public parameter against which a proof is checked (§9.3; protocol specification, §“Action Descriptions”; ZIP 258, “Consensus rules from NU6.3 activation”). The pools are distinguished by their note commitment trees, nullifier sets and chain value pool balances, and by the component of the transaction that carries their bundles (§11.2), not by separate circuits (ZIP 229, Abstract, and Rationale, “Reuse of the Orchard protocol with minimal changes”).

Orchard-pool notes are spendable, in version-5 and version-6 transactions (§11.2), subject to the rules that seal the Orchard pool (ZIP 258, “Consensus rules from NU6.3 activation”; ZIP 259, Abstract).

The name “Orchard” alone qualifies the shared cryptography of the Orchard protocol; a statement that depends on the pool names the pool, the Orchard pool or the Ironwood pool.

1.4 Requirements on a shielded payment

Definition 1.13 (Transparent and shielded payments, observer). A *transparent payment* is one whose consumed and created amounts and addresses are public. The goal of a shielded payment is to achieve the effects of a transparent payment, namely transfer of value, exclusion of *double-spending*, the consumption of one note twice, and conservation of value, while hiding the contents of its notes, and the links between the notes it consumes and the notes it creates, from an *observer*: any party that reads the chain but holds no key of the notes concerned. Requirements R1 to R5 below make this precise. Each of R1 to R4 has a *soundness part*, a property of every valid payment, and a *hiding part*, a bound on what the observer learns; R5 has a soundness part only.

Definition 1.14 (Requirements R1 to R5).

- R1. *Hidden representation of value*. Soundness: the chain’s record of a created note binds that note, so that its owner can later prove possession of it and no party can present the record as the record of a different note. Hiding: the record reveals neither the recipient’s address nor the value, and does not distinguish a real note from a dummy note.
- R2. *Membership without identification*. Soundness: every consumed note of non-zero value was previously committed in the same pool, at the state of the pool’s record of commitments to which the payment refers. Hiding: the payment does not identify which com-

mitted note it consumes. A note of value zero, such as a dummy note, transfers no value, and R2 does not require it to have been committed.

- R3. *No second consumption*. Soundness: no note of a pool is consumed twice. Hiding: this holds without a public list of consumed notes; the data published when a note is consumed do not identify which committed note it is.
- R4. *Conservation*. Soundness: in every valid bundle the total value of the consumed notes equals the total value of the created notes plus the balancing value, so that no payment creates value. Hiding: no note's value is disclosed; of the amounts, only the balancing value is public.
- R5. *Spend authority*, a soundness part only. Only the holder of a note's spending authority, the spend authorising key associated with its address (§3.1), can consume the note; and no other party can alter what an authorised payment transfers, namely the notes it consumes and creates and the value it moves into or out of the pool, without invalidating it. Requirement R5 requires nothing of data that do not change what is transferred; which data the signatures fix is stated in “Transaction digests and signatures” (§11.3).

The requirement map, Table 5, given with the Action statement (§9.2), pairs the soundness and hiding parts of each requirement with the results that discharge them; “Security” (§12) states those results as theorems with their assumptions.

2 Notation and primitive instances

This section fixes the encodings and the primitive instances on which the later constructions rest, each with its Orchard parameters and domain separators. The underlying constructions belong to the *Math Guide* and the *Crypto Guide* and are cited, not re-derived; each named assumption is stated at the first construction whose security needs it.

2.1 Fields, groups, and encodings

The Pallas group $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ is the group of points of the curve $y^2 = x^3 + 5$ over the prime field $\mathbb{F}_{p_{\text{Pallas}}}$, with identity $\mathcal{O}$. It has prime order p_{Vesta} and cofactor 1, and $\mathbb{F}_{p_{\text{Vesta}}}$ is both its scalar field and the base field of the Vesta curve (*Math Guide*, §“Base fields, scalar fields, and the Pasta cycle” and §“Pallas and Vesta assembled”, Example “The Pasta curves”; *Crypto Guide*, §“Instantiation on elliptic curves; the Pasta curves”, Definition “The Pasta curves”). The group, its order and the cycle with Vesta are cited, not re-derived; the moduli are those of the protocol specification, §“Pallas and Vesta”. Table 1 maps this notation, once, to the names used by the specification and by the lower volumes. Those names appear in the table only; later text writes only the column headed “This volume”. Maps defined on the Pallas group carry the specification's subscript $\mathbb{P}$, as in $\text{Extract}_{\mathbb{P}}$.

Scalar multiplication is written $[k]P$, for $k \in \mathbb{F}_{p_{\text{Vesta}}}$ or an integer read modulo p_{Vesta} . The symbol $\parallel$ denotes concatenation of bit strings or of byte strings, and ε the empty string. Byte-string literals and tags are set in monospace, as in `z.cash:orchard`, and single bytes as `0x...`. The symbol $\perp$ is the

Table 1. Notation for the fields, the Pallas group and the encodings, with the names of the protocol specification and of the *Math* and *Crypto Guides*. The spend validating key is constructed in §3.1; the point $\text{ak}^{\mathbb{P}}$ and the field element $\text{ak} = \text{Extract}_{\mathbb{P}}(\text{ak}^{\mathbb{P}})$ are kept apart throughout.

Object	This volume	Specification	Lower volumes
Pallas base-field modulus	p_{Pallas}	$q_{\mathbb{P}}$	p
Pallas scalar-field modulus, Vesta	p_{Vesta}	$r_{\mathbb{P}}$	q
base-field modulus, order of the Pallas group			
Pallas base field and scalar field	$\mathbb{F}_{p_{\text{Pallas}}}, \mathbb{F}_{p_{\text{Vesta}}}$	$\mathbb{F}_{q_{\mathbb{P}}}, \mathbb{F}_{r_{\mathbb{P}}}$	$\mathbb{F}_p, \mathbb{F}_q$
Pallas group	$\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$	$\mathbb{P}$	$E_p(\mathbb{F}_p)$
k -bit little-endian encoding of an integer	LE_k	l2LEBSP_k	—
Encoding of a point P	P^*	$\text{repr}_{\mathbb{P}}(P)$	—
Sinsemilla short commitment	ShortCommit	SinsemillaShortCommit	—
Spend validating key, as a point and as its x -coordinate	$\text{ak}^{\mathbb{P}}, \text{ak}$	$\text{ak}^{\mathbb{P}}, \text{ak}$	—

failure value of a partial map; a map that may fail takes values in $X \cup \{\perp\}$ for its codomain X .

Both moduli have bit length 255, and $p_{\text{Pallas}} < p_{\text{Vesta}}$:

$$2^{254} < p_{\text{Pallas}} < p_{\text{Vesta}} < 2^{255}.$$

Two consequences are used later. First, the integer representative of an element of either field lies in $\{0, \dots, 2^{255} - 1\}$ and has a 255-bit encoding, which leaves the top bit of a 256-bit string free. Second, the integer representative of an element of $\mathbb{F}_{p_{\text{Pallas}}}$ lies in $\{0, \dots, p_{\text{Vesta}} - 1\}$; reading it as a Pallas scalar is therefore an injective, not a surjective, map $\mathbb{F}_{p_{\text{Pallas}}} \rightarrow \mathbb{F}_{p_{\text{Vesta}}}$. This map is used wherever a base-field element multiplies a point (§3.3, §6.1).

Definition 2.1 (Integer and byte encodings). For $k \geq 0$ and an integer $m \in \{0, \dots, 2^k - 1\}$, the k -bit little-endian encoding $\text{LE}_k(m) \in \{0, 1\}^k$ is the bit sequence $b_0 b_1 \dots b_{k-1}$ with $m = \sum_{i=0}^{k-1} b_i 2^i$. The map $\text{LE}_k: \{0, \dots, 2^k - 1\} \rightarrow \{0, 1\}^k$ is a bijection with inverse LE_k^{-1} . A field element is encoded through its integer representative in $\{0, \dots, p - 1\}$, for p the modulus. In the conversion names below, l stands for integer, LE for little-endian, BS for bit string, and OS for octet string (a sequence of eight-bit bytes). The digit 2 is read “to”, and P stands for primitive, a basic conversion operation. The subscript k counts bits, not bytes. Three maps relate integers, bit strings and byte strings:

- the map l2LEOSP_k sends $m \in \{0, \dots, 2^k - 1\}$ to its $\lceil k/8 \rceil$ -byte little-endian encoding;
- for k a multiple of 8, the map LEOS2IP_k sends a $k/8$ -byte string $(s_0, \dots, s_{k/8-1})$ to the integer $\sum_i s_i 256^i \in \{0, \dots, 2^k - 1\}$;
- the map $\text{LEBS2OSP}_k: \{0, 1\}^k \rightarrow \{0, \dots, 255\}^{\lceil k/8 \rceil}$ pads its input on the right with zero bits to a multiple of 8 bits, packs each group of 8 bits into a byte, least significant bit first, and keeps the order of the groups.

Hence $\text{LEBS2OSP}_k(\text{LE}_k(m)) = \text{I2LEOSP}_k(m)$. The byte form is needed wherever a bit string enters BLAKE2b or GroupHash (protocol specification, §“Integers, Bit Sequences, and Endianness”).

The widths used later are LE_{10} (the Merkle layer tag, §5.1), LE_{64} (the note value, §4.2), LE_{88} (the diversifier index, §3.3), LE_{255} (field elements) and LE_{256} (the star encoding below; the key encoding of r_{ivk} , §3.2).

Definition 2.2 (Coordinate extraction). Let $\text{Extract}_P: \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \rightarrow \mathbb{F}_{p_{\text{Pallas}}}$ be

$$\text{Extract}_P(\mathcal{O}) := 0, \quad \text{Extract}_P((x, y)) := x.$$

Its lifting $\text{Extract}_P^\perp: \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \cup \{\perp\} \rightarrow \mathbb{F}_{p_{\text{Pallas}}} \cup \{\perp\}$ maps $\perp$ to $\perp$ and agrees with Extract_P on points (protocol specification, §“Coordinate Extractor for Pallas”).

The map Extract_P supplies a field element wherever a later construction needs one in place of a point: the field element ak (§3.1), the extracted note commitment cmx (§4.2), the node values of the note commitment tree (§5.1) and the nullifier nf (§6.1). It is two-to-one on non-identity points, by Lemma 2.4 and because $P \neq -P$ for $P \neq \mathcal{O}$ in a group of odd order; every binding argument over an extracted coordinate treats the opposite point explicitly.

Lemma 2.3 (Extract vanishes only at the identity). For $P \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, $\text{Extract}_P(P) = 0$ if and only if $P = \mathcal{O}$.

Proof. If $P = \mathcal{O}$, then $\text{Extract}_P(P) = 0$ by definition. A point $(0, y)$ would satisfy $y^2 = 5$ in $\mathbb{F}_{p_{\text{Pallas}}}$, but 5 is a quadratic non-residue modulo p_{Pallas} (*Math Guide*, §“Quadratic residues and the Euler criterion”, Example “5 is a non-square in the Pallas base field”). Hence no point other than $\mathcal{O}$ has extracted coordinate 0. $\square$

Lemma 2.4 (Coordinate extraction identifies opposite points). For $P, P' \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, $\text{Extract}_P(P) = \text{Extract}_P(P')$ if and only if $P' = P$ or $P' = -P$.

Proof. Negation is $-\mathcal{O} = \mathcal{O}$ and $-(x, y) = (x, -y)$ (*Math Guide*, §“Explicit affine formulas”, Definition “Negation”), so $\text{Extract}_P(-P) = \text{Extract}_P(P)$ for every P , which is the reverse implication. For the forward implication let $\text{Extract}_P(P) = \text{Extract}_P(P')$. If $P = \mathcal{O}$, then $\text{Extract}_P(P') = 0$, and Lemma 2.3 gives $P' = \mathcal{O} = -P$. If $P = (x, y)$, then $x \neq 0$ by Lemma 2.3, so $P' \neq \mathcal{O}$ and $P' = (x, y')$ with $y'^2 = x^3 + 5 = y^2$; hence $y' = \pm y$ and $P' = \pm P$. $\square$

Definition 2.5 (Star encoding). For $P \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ the *star encoding* $P^* \in \{0, 1\}^{256}$ is

$$\mathcal{O}^* := \text{LE}_{256}(0), \quad (x, y)^* := \text{LE}_{256}(x + 2^{255}(y \bmod 2)),$$

with x and y read as integer representatives: the 255-bit x -coordinate fills the low bits and the parity of y the top bit. The partial inverse $\text{abst}_P: \{0, 1\}^{256} \rightarrow \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \cup \{\perp\}$ reads the first 255 bits of s as $\tilde{s} := \text{LE}_{255}^{-1}(s_0 \dots s_{254})$ and the last bit as the sign bit $\tilde{y} := s_{255}$, and returns

1. the failure value $\perp$ if $\tilde{x} \geq p_{\text{Pallas}}$; otherwise let $x := \tilde{x} \in \mathbb{F}_{p_{\text{Pallas}}}$;
2. the identity $\mathcal{O}$ if $x = 0$ and $\tilde{y} = 0$;
3. the failure value $\perp$ if $x^3 + 5$ is not a square in $\mathbb{F}_{p_{\text{Pallas}}}$;
4. otherwise the point (x, y) with $y^2 = x^3 + 5$ and $y \bmod 2 = \tilde{y}$

(protocol specification, §“Pallas and Vesta”).

The encoding is canonical: $\text{abst}_P(P^*) = P$ for every point P , and $\text{abst}_P(s) \neq \perp$ implies $\text{abst}_P(s)^* = s$. For the first claim, $\mathcal{O}^*$ decodes to $\mathcal{O}$ at step 2. A point (x, y) has $x \neq 0$ (Lemma 2.3), and $y \neq 0$ because a point with $y = 0$ has order 2, which a group of odd order does not contain (*Math Guide*, §“Torsion, the cofactor, and prime-order subgroups”, Remark “No two-torsion on the Pasta curves”). The two square roots $\pm y$ are therefore non-zero and, p_{Pallas} being odd, of opposite parities, so step 4 returns (x, y) . For the second claim, a string decoded at step 2 is $\text{LE}_{256}(0) = \mathcal{O}^*$, and a string decoded at step 4 to (x, y) has low bits $\text{LE}_{255}(x)$ and top bit $y \bmod 2$, so it equals $(x, y)^*$. Consequently $P \mapsto P^*$ is injective, each point has exactly one valid encoding, and every other string decodes to $\perp$, among them $\text{LE}_{256}(2^{255})$ (whose x -part is 0 and sign bit 1) and every string whose x -part is at least p_{Pallas} . The star encoding is the form in which a point enters a hash or a commitment as a bit string.

2.2 GroupHash, domain separation, and nothing-up-my-sleeve generators

Construction 2.6 (GroupHash on Pallas). For byte strings D (the domain separator) and M (the message), the point $\text{GroupHash}(D, M) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ is computed as follows.

1. Let

$$\text{DST} := D \parallel - \parallel \text{pallas} \parallel \text{_XMD:BLAKE2b_SSWU_RO_};$$

the map is defined when DST has at most 255 bytes, which holds for every domain separator of Table 2.

2. The map `hash_to_field` expands (M, DST) by the function `expand_message_xmd` over BLAKE2b-512 into two 64-byte strings. BLAKE2b runs with zero personalisation; the string DST, followed by a byte carrying its length, is part of the hashed input. Each string is read as a big-endian integer and reduced modulo p_{Pallas} , giving $u_0, u_1 \in \mathbb{F}_{p_{\text{Pallas}}}$.
3. The simplified Shallue–van de Woestijne–Ulas map sends each u_i to a point Q_i of a curve 3-isogenous to Pallas.
4. The output is the image of $Q_0 + Q_1$ under the 3-isogeny onto Pallas.

No cofactor is cleared, since the Pallas group has prime order. The isogenous curve, the isogeny coefficients and the constant of the map are those of the protocol specification, §“Group Hash into Pallas and Vesta”, and are not reproduced. The map is deterministic, public and efficiently computable (*Crypto Guide*, §“Hashing to a field element”, Construction

“Expand-then-reduce”; §“Hashing to a curve point”, Constructions “Simplified Shallue–van de Woestijne–Ulas map” and “The hash-to-curve pipeline”, Remark “The isogeny required by j -invariant 0”).

Definition 2.7 (Domain separator and fixed generator). A *domain separator* is a fixed ASCII byte string naming a use site; it is the first argument of GroupHash, so that distinct uses draw distinct generators. A *fixed generator* is the value of GroupHash on a public, fixed pair (D, M) . It is nothing-up-my-sleeve: the public pair determines it, and no party chose it (*Crypto Guide*, §“Hashing to a curve point”, Construction “Nothing-up-my-sleeve generators”). The strings are fixed by the protocol specification.

Assumption 2.8 (GroupHash as a random oracle). In security arguments GroupHash is modelled as a random oracle on pairs (D, M) with values in $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ (*Crypto Guide*, §“The random oracle model”, Definition “Random oracle”, with the group in place of bit strings): distinct pairs receive independent uniform points. In particular the fixed generators of Table 2 are, in the model, independent uniform points. The assumption is a heuristic about the concrete map, supported by its design for indistinguishability from a random oracle (protocol specification, §“Group Hash into Pallas and Vesta”, note); it is not a theorem. Every later binding, collision, uniqueness or unlinkability result that needs it names it.

Table 2 lists every GroupHash input of the Orchard protocol, with the object it defines and the section that uses it. The messages G , K , v and r are one-byte ASCII strings, and the Sinsemilla messages are the ASCII strings shown. The rows are those of the protocol specification, §“Spend Authorization Signature (Sapling and Orchard)”, §“Computing ρ values and Nullifiers”, §“DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions”, §“Sinsemilla Hash Function”, §“Sinsemilla commitments”, §“MerkleCRH^{Orchard} Hash Function” and §“Homomorphic Pedersen commitments (Sapling and Orchard)”.

Table 2. Every input of GroupHash in the Orchard protocol. The hash base of MerkleCRH is derived from its domain without the suffix -M, and MerkleCRH has no blinding base. The table entries $S(j)$ are shared by every Sinsemilla domain.

Domain separator D	Message M	Object	Used in
z.cash:Orchard	G	$G^{\text{SpendAuth}}$, spend-authorisation base	§3.1, §7.1
z.cash:Orchard	K	K^{Orchard} , nullifier base	§6.1
z.cash:Orchard-gd	$\text{LEBS2OSP}_{88}(d)$	g_d for a diversifier d	§3.3
z.cash:Orchard-gd	ε	g_d if the row above gives $\mathcal{O}$	§3.3
z.cash:SinsemillaQ	z.cash:Orchard-NoteCommit-M	hash base of NoteCommit	§4.2
z.cash:Orchard-NoteCommit-r	ε	blinding base of NoteCommit	§4.2
z.cash:SinsemillaQ	z.cash:Orchard-CommitIvk-M	hash base of Commit ^{ivk}	§2.4
z.cash:Orchard-CommitIvk-r	ε	blinding base of Commit ^{ivk}	§2.4
z.cash:SinsemillaQ	z.cash:Orchard-MerkleCRH	hash base of MerkleCRH	§5.1
z.cash:Orchard-cv	v	V^{Orchard} , value base	§8.1
z.cash:Orchard-cv	r	R^{Orchard} , randomness base	§8.1
z.cash:SinsemillaS	$\text{I2LEOSP}_{32}(j)$	table entry $S(j)$, $0 \leq j < 2^{10}$	§2.4

Remark 2.9 (Domain separation). The pairs of Table 2 are pairwise distinct, and the map from

(D, M) to the input hashed by `hash_to_field` is injective, because DST is closed by a byte carrying its length. Under Assumption 2.8 the table’s generators are therefore independent, and a relation found in one context transfers to no other (*Crypto Guide*, §“Domain separation and personalisation”, Proposition “Domain separation yields independent oracles”). The *Crypto Guide*’s inventory of Orchard tags and personalisations in the same section is cited, not repeated.

2.3 Pseudorandom functions and field reductions

Definition 2.10 (Pseudorandom function). A keyed family $(F_k)_k$ of functions is a *pseudorandom function* (PRF) if no efficient adversary with adaptive oracle access distinguishes F_k , for a secret uniform key k , from a uniformly random function with the same domain and range, except with negligible advantage (*Crypto Guide*, §“Pseudorandom functions and permutations”, Definition “Pseudorandom function”).

Construction 2.11 (Expansion function). For a key $k \in \{0, 1\}^{256}$ and a byte string t ,

$$\text{PRFexpand}_k(t) := \text{BLAKE2b-512}(\text{zcash_ExpandSeed}, \text{LEBS2OSP}_{256}(k) \parallel t),$$

a 64-byte string, identified with $\{0, 1\}^{512}$; here $\text{BLAKE2b-512}(P, x)$ is unkeyed BLAKE2b with a 64-byte output, the 16-byte personalisation P and the input x . A key given as a 32-byte string enters as is. The key is thus carried in the message, not in BLAKE2b’s native keyed mode (protocol specification, §“Pseudo Random Functions”), so the *Crypto Guide*’s PRF argument for native keying (§“PRFs from hash functions: length extension and keyed BLAKE2”) does not cover this construction; Assumption 2.12 states the property directly. The key is written as a subscript; the leading byte of t names the derived quantity (protocol specification, §“Pseudo Random Functions”).

Assumption 2.12 (Pseudorandomness of PRF expansion). For k uniform on $\{0, 1\}^{256}$, the function $t \mapsto \text{PRFexpand}_k(t)$ from byte strings to $\{0, 1\}^{512}$ is a PRF in the sense of the preceding definition. This is the security requirement that the protocol specification, §“Pseudo Random Functions”, places on `PRFexpand`.

Definition 2.13 (Field reductions). For $x \in \{0, 1\}^{512}$, read as 64 bytes,

$$\begin{aligned} \text{ToBase}(x) &:= \text{LEOS2IP}_{512}(x) \bmod p_{\text{Pallas}} \in \mathbb{F}_{p_{\text{Pallas}}}, \\ \text{ToScalar}(x) &:= \text{LEOS2IP}_{512}(x) \bmod p_{\text{Vesta}} \in \mathbb{F}_{p_{\text{Vesta}}} \end{aligned}$$

(protocol specification, §“Orchard Key Components”).

For x uniform on $\{0, 1\}^{512}$, the statistical distance of $\text{ToBase}(x)$ from the uniform distribution on $\mathbb{F}_{p_{\text{Pallas}}}$ is at most $p_{\text{Pallas}}/2^{512}$, and that of $\text{ToScalar}(x)$ from the uniform distribution on $\mathbb{F}_{p_{\text{Vesta}}}$ at most $p_{\text{Vesta}}/2^{512}$ (*Math Guide*, §“Uniform sampling and the bias of modular reduction”, Proposition “Bias of modular reduction”, with $L = 512$). Both bounds are below 2^{-257} .

Construction 2.14 (Domain bytes of PRF expansion). Every later use of PRFexpand derives one quantity as

$$y := f(\text{PRFexpand}_\kappa(b \parallel t')),$$

where the lead byte b , the key κ , the remainder t' of the input and the reduction $f \in \{\text{ToBase}, \text{ToScalar}\}$, or no reduction, form one row of Table 3. Each row's objects are defined in the section it names (protocol specification, §“Orchard Key Components” for $0x06$, $0x07$, $0x08$ and $0x82$; §“Sending Notes (Orchard)” for $0x04$, $0x09$ and $0x0B$).

Table 3. Every lead byte of PRFexpand used in this volume. The row $0x82$ applies no reduction: its 64-byte output splits into dk , the first 32 bytes, and ovk , the last 32 bytes.

b	Key κ	Remainder t'	f	Output	Section
$0x06$	sk	ε	ToScalar	ask	§3.1
$0x07$	sk	ε	ToBase	nk	§3.1
$0x08$	sk	ε	ToScalar	r_{ivk}	§3.1
$0x82$	$LE_{256}(r_{ivk})$	$I2LEOSP_{256}(ak) \parallel I2LEOSP_{256}(nk)$	none	$dk \parallel ovk$	§3.2
$0x04$	$rseed$	$I2LEOSP_{256}(\rho)$	ToScalar	esk	§4.3
$0x09$	$rseed$	$I2LEOSP_{256}(\rho)$	ToBase	ψ	§4.3
$0x0B$	$rseed$	$LEBS2OSP_{256}(g_d^*) \parallel LEBS2OSP_{256}(pk_d^*) \parallel$ $I2LEOSP_{64}(v) \parallel I2LEOSP_{256}(\rho) \parallel$ $I2LEOSP_{256}(\psi)$	ToScalar	rcm	§4.3

Lemma 2.15 (Independence of domain-separated expansions). *Let k be uniform on $\{0, 1\}^{256}$ and used only as a PRFexpand key. Let an efficient algorithm choose, possibly adaptively, pairwise distinct inputs $t_1, \dots, t_n$, in particular inputs with pairwise distinct leading bytes, and receive the answers. Under Assumption 2.12, the answers $(\text{PRFexpand}_k(t_1), \dots, \text{PRFexpand}_k(t_n))$ are computationally indistinguishable from n independent uniform elements of $\{0, 1\}^{512}$. Consequently, for reductions $f_i \in \{\text{ToBase}, \text{ToScalar}\}$, the answers $(f_i(\text{PRFexpand}_k(t_i)))_i$ are computationally indistinguishable from independent uniform elements of the target fields, up to an additional statistical distance of at most the sum of the reduction distances. For a key within statistical distance δ of uniform, both conclusions hold with δ added.*

Proof. One hybrid replaces PRFexpand_k by a uniformly random function; the distinguishing advantage changes by at most the PRF advantage of the algorithm that runs the given one and forwards its queries to its own oracle (Assumption 2.12; adaptive queries are within the definition, *Crypto Guide*, §“Pseudorandom functions and permutations”, Remark “Necessity of adaptivity”). A random function, sampled lazily, answers each new input with a fresh uniform value, so its answers on pairwise distinct inputs are independent and uniform, also when each input depends on earlier answers. For the reduced answers, replacing each f_i of a fresh uniform string by a fresh uniform field element changes the distribution of the whole interaction by at most the reduction distance of f_i ; summing over i bounds the total by the triangle inequality, and the post-processing by the algorithm does not increase it (*Math Guide*, §“Statistical distance”, Theorem “Properties

of statistical distance”). A key within statistical distance δ of uniform changes the interaction, a function of the key and the algorithm’s coins, by at most δ , by the same theorem. $\square$

The hypothesis of Lemma 2.15 holds for the spending key sk , and for $rseed$ towards an adversary that sees $rseed$ only through PRFexpand outputs; the note plaintext, which contains $rseed$, is treated in §10.2. It does not hold for the row $0x82$, whose key encodes the trapdoor r_{ivk} , which is also used in $Commit^{ivk}$; that row is covered by Assumption 3.13 in §3.2. The lemma is used in §3.1, in Lemma “Rejection in key generation” (§3.2), in Propositions 3.18, 3.17, 4.8, 6.10 and 10.6, in Proposition “Confidentiality of the outgoing ciphertext” (§10.3), in Theorems 12.9 and 12.12, and in Remark “Hypothesis (H) for honest keys” (§11.3).

Remark 2.16 (Hash roles). Each hash is defined at its construction site. BLAKE2b-512 with personalisation `Zcash_ExpandSeed` serves PRFexpand (this subsection); BLAKE2b-512 without personalisation serves `hash_to_field` inside `GroupHash` (§2.2); personalised BLAKE2b-512 serves the RedPallas hash H (§7.1); BLAKE2b-256 serves the note-encryption key derivation and the outgoing cipher key (§10.2, §10.3); BLAKE2b serves the transaction and signature digests (§11.3); Sinsemilla serves MerkleCRH, NoteCommit and $Commit^{ivk}$ (§2.4); Poseidon serves the nullifier PRF, constructed with the nullifier (§6.1). The personalisations of PRFexpand, the note-encryption key derivation and the outgoing cipher key are those of the *Crypto Guide*’s inventory (§“PRFs from hash functions: length extension and keyed BLAKE2”, Remark “Zcash’s use of keyed and personalised BLAKE2”); every personalisation is stated at its construction.

2.4 Sinsemilla: hash, commitment, and short forms

Construction 2.17 (Sinsemilla hash on Pallas). The *Crypto Guide*, §“Sinsemilla: an algebraic hash-based commitment”, Construction “Sinsemilla hash”, fixes the chunk width $k = 10$ (written w there, where k counts the pieces), the table and the domain base

$$\begin{aligned} S(j) &:= \text{GroupHash}(\text{z.cash:SinsemillaS}, \text{l2LEOSP}_{32}(j)) & (0 \leq j < 2^k), \\ Q(D) &:= \text{GroupHash}(\text{z.cash:SinsemillaQ}, D) \end{aligned}$$

for a byte string D , and the accumulator; it writes $\text{LE}_{32}(j)$ for the four-byte encoding $\text{l2LEOSP}_{32}(j)$. The Orchard instance adds the following (protocol specification, §“Sinsemilla Hash Function”).

1. The *chunk bound* c is the largest integer with $2^c \leq (p_{\text{Vesta}} - 1)/2$; thus $c = 253$, and a message has at most $kc = 2530$ bits.
2. *Incomplete addition* $+$ on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \cup \{\perp\}$ returns $\perp$ if either operand is $\perp$; for points it returns $\perp$ in an *exceptional case*, when an operand is $\mathcal{O}$ or the operands have equal x -coordinates (equal or opposite points), and their group sum otherwise (*Math Guide*, §“Explicit affine formulas”, Remark “Complete versus incomplete addition”).
3. For a byte string D and a bit string M of length at most kc , let $n := \lceil |M|/k \rceil \leq c$, pad M with zero bits to nk bits, split the result into k -bit pieces, and let m_i be LE_k^{-1} of the i -th

piece, $1 \leq i \leq n$. Then

$$\text{Acc}_0 := Q(D), \quad \text{Acc}_i := (\text{Acc}_{i-1} \dot{+} S(m_i)) \dot{+} \text{Acc}_{i-1} \quad (1 \leq i \leq n),$$

and $\text{SinsemillaHashToPoint}(D, M) := \text{Acc}_n$. The output is $\perp$ exactly when some incomplete addition meets an exceptional case.

4. The extracted hash is

$$\text{SinsemillaHash}(D, M) := \text{Extract}_{\mathbb{P}}^{\perp}(\text{SinsemillaHashToPoint}(D, M)) \in \mathbb{F}_{\mathcal{P}_{\text{Pallas}}} \cup \{\perp\}.$$

Construction 2.18 (Sinsemilla commitment and short commitment). For a byte string D , a bit string M of length at most $k\epsilon$ and a trapdoor $r \in \mathbb{F}_{\mathcal{P}_{\text{Vesta}}}$, let

$$\hat{M} := \text{SinsemillaHashToPoint}(D \parallel -\mathbb{M}, M), \quad H_D := \text{GroupHash}(D \parallel -r, \epsilon).$$

Then, with complete group addition,

$$\text{SinsemillaCommit}_r(D, M) := \begin{cases} \hat{M} + [r]H_D & \text{if } \hat{M} \neq \perp, \\ \perp & \text{otherwise,} \end{cases}$$

$$\text{ShortCommit}_r(D, M) := \text{Extract}_{\mathbb{P}}^{\perp}(\text{SinsemillaCommit}_r(D, M)) \in \mathbb{F}_{\mathcal{P}_{\text{Pallas}}} \cup \{\perp\}$$

(protocol specification, §“Sinsemilla commitments”). The domain D is routed to the hash as $D \parallel -\mathbb{M}$ and to the blinding base as $D \parallel -r$ with the empty message. The *Crypto Guide* constructs the commitment with a generic independent blinding generator (§“Sinsemilla: an algebraic hash-based commitment”, Construction “SinsemillaCommit”); the routing and the $\perp$ case are the Orchard additions.

The unblinded SinsemillaHash is deterministic and therefore not hiding; ShortCommit is its blinded counterpart. Proposition 2.24 and Lemma 2.26 state the security of both.

Construction 2.19 (Orchard instances). The Orchard protocol uses three Sinsemilla instances, each with one fixed message length per domain, all within ϵ chunks (Table 4): MerkleCRH, a SinsemillaHash instance whose layout is given in §5.1; NoteCommit, a SinsemillaCommit instance whose layout is given in Definition 4.4; and Commit^{ivk}, a ShortCommit instance defined next.

Table 4. The Orchard Sinsemilla instances. Each domain hashes messages of the single length shown, of at most $\epsilon = 253$ chunks of $k = 10$ bits.

Instance	Form	Domain	Message bits	Chunks
MerkleCRH	SinsemillaHash	z.cash:Orchard-MerkleCRH	$10 + 2 \cdot 255 = 520$	52
NoteCommit	SinsemillaCommit	z.cash:Orchard-NoteCommit	$2 \cdot 256 + 64 + 2 \cdot 255 = 1086$	109
Commit ^{ivk}	ShortCommit	z.cash:Orchard-CommitIvk	$2 \cdot 255 = 510$	51

Definition 2.20 (The key commitment). For $x, y \in \mathbb{F}_{p_{\text{Pallas}}}$ and $r_{\text{ivk}} \in \mathbb{F}_{p_{\text{Vesta}}}$,

$$\text{Commit}_{r_{\text{ivk}}}^{\text{ivk}}(x, y) := \text{ShortCommit}_{r_{\text{ivk}}}(\text{z.cash:Orchard-CommitIvk}, \text{LE}_{255}(x) \parallel \text{LE}_{255}(y)),$$

a 510-bit message (protocol specification, §“Sinsemilla commitments”). Its output is $\perp$ exactly when

$$\text{SinsemillaHashToPoint}(\text{z.cash:Orchard-CommitIvk-M}, \cdot)$$

returns $\perp$; it is 0 exactly when the commitment point is $\mathcal{O}$ (Lemma 2.3); otherwise it is the x -coordinate of a non-identity point, an element of $\{1, \dots, p_{\text{Pallas}} - 1\}$ and hence, since $p_{\text{Pallas}} < p_{\text{Vesta}}$, a non-zero Pallas scalar. The x -coordinates of non-identity points form a set of $(p_{\text{Vesta}} - 1)/2$ elements, by Lemma 2.4 and because a group of odd prime order has no point with $y = 0$. The incoming viewing key of §3.2 is this instance applied to (ak, nk) with trapdoor r_{ivk} ; the exclusion of the outputs 0 and $\perp$ by key generation is stated there.

Remark 2.21 (Cost inside a proof). Sinsemilla is used for its cost inside an arithmetic circuit with lookups, one table lookup and two incomplete additions over $\mathbb{F}_{p_{\text{Pallas}}}$ per k -bit chunk (*Crypto Guide*, §“Sinsemilla: an algebraic hash-based commitment”, Remark “In-circuit cost of Sinsemilla”; *Halo 2 Guide*, §“The lookup argument” and §“From statement to circuit: arithmetisation in practice”), and its collision resistance reduces to discrete logarithms on Pallas together with GroupHash modelled as a random oracle, not to discrete logarithms alone.

Assumption 2.22 (Discrete logarithms on Pallas). For every classical probabilistic polynomial-time algorithm, given a generator G of $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ and $[s]G$ for s uniform in $\mathbb{F}_{p_{\text{Vesta}}}$, the probability of outputting s is negligible (*Crypto Guide*, §“The discrete logarithm problem”, Definition “Discrete logarithm problem, DLP”, instantiated on Pallas; *Math Guide*, §“The elliptic-curve discrete logarithm problem”). Concretely, the best known classical attack costs about 2^{126} group operations (*Crypto Guide*, §“Instantiation on elliptic curves; the Pasta curves”, Proposition “The Pasta curves against the criteria”); the level is cited, not recomputed.

Lemma 2.23 (Relations among fixed generators). *Under Assumptions 2.8 and 2.22, no efficient algorithm outputs, except with negligible probability, distinct inputs of GroupHash with values $P_1, \dots, P_t$ and coefficients $a_1, \dots, a_t \in \mathbb{F}_{p_{\text{Vesta}}}$, not all zero, with $\sum_i [a_i]P_i = \mathcal{O}$.*

Proof. In the model of Assumption 2.8 a reduction receiving a challenge $Y = [s]G$ answers each new GroupHash input with $[u]G + [w]Y$ for fresh uniform $u, w \in \mathbb{F}_{p_{\text{Vesta}}}$, a uniform point and hence distributed as the oracle’s answer. For every value of w exactly one u is consistent with the answer, so the w remain uniform and independent of the algorithm’s view. A relation $\sum_i [a_i]P_i = \mathcal{O}$ gives $c_0 + c_1 s = 0$ with $c_1 = \sum_i a_i w_i$, which is non-zero except with probability $1/p_{\text{Vesta}}$, and then $s = -c_0 c_1^{-1}$ (*Crypto Guide*, §“Pedersen vector commitments”, Theorem “Properties of the vector commitment”, proof; §“Hashing to a curve point”, Remark “Why NUMS generators are binding-safe”). $\square$

Every later argument that ends in a non-trivial discrete-logarithm relation among values of GroupHash ends in this lemma.

Proposition 2.24 (Collision resistance of the Sinsemilla instances). *Consider an Orchard instance of Table 4 with message length l , hash domain D' (z.cash:Orchard-MerkleCRH for MerkleCRH; $D \parallel -M$ for a commitment instance with domain D) and, for a commitment instance, blinding base H_D . Under Assumptions 2.22 and 2.8, no efficient algorithm outputs, except with negligible probability:*

(i) messages $M, M' \in \{0, 1\}^l$ whose hash points

$$P := \text{SinsemillaHashToPoint}(D', M) \quad \text{and} \quad P' := \text{SinsemillaHashToPoint}(D', M')$$

are not $\perp$ and satisfy $P = P'$ with $M \neq M'$, or $P = -P'$; hence, by Lemma 2.4, no $M \neq M'$ with equal values of the extracted form SinsemillaHash other than $\perp$;

(ii) for a commitment instance, openings (M, r) and (M', r') in $\{0, 1\}^l \times \mathbb{F}_{p_{\text{Vesta}}}$ whose commitments

$$C := \text{SinsemillaCommit}_r(D, M) \quad \text{and} \quad C' := \text{SinsemillaCommit}_{r'}(D, M')$$

are not $\perp$ and satisfy $C = C'$ with $M \neq M'$, or $C = -C'$; hence no $M \neq M'$ with equal values of ShortCommit other than $\perp$;

(iii) a message in $\{0, 1\}^l$ on which $\text{SinsemillaHashToPoint}(D', \cdot)$ returns $\perp$, equivalently on which the SinsemillaHash or commitment form returns $\perp$ for every trapdoor.

Each such output yields efficiently a non-trivial discrete-logarithm relation among $Q(D')$, $S(0)$, ..., $S(2^k - 1)$ and, in (ii), H_D .

Proof. The proof is by citation, with the Orchard steps made in place. The cited arguments are, in the *Crypto Guide*, §“Sinsemilla: an algebraic hash-based commitment”, Proposition “Collision resistance of Sinsemilla”, with the paragraph after it on the extracted coordinate and the fixed length, and Construction “SinsemillaCommit”; and the Security argument of the protocol specification, §“Sinsemilla Hash Function”: Lemma “An injectivity property for Sinsemilla”, the theorem “Collision resistance of SinsemillaHash and SinsemillaHashToPoint” with its note extending it to added terms with independent bases, which covers SinsemillaCommit and ShortCommit, and the theorem that a $\perp$ output of SinsemillaHashToPoint yields a non-trivial discrete-logarithm relation. These arguments write a non- $\perp$ output on the pieces $m = (m_1, \dots, m_n)$ as

$$\text{SinsemillaHashToPoint}(D', M) = [2^n]Q(D') + \sum_{j=0}^{2^k-1} [\chi_j(m)]S(j), \quad (1)$$

where $\chi_j(m) := \sum_{i=1}^n 2^{n-i} \delta_{m_i, j}$ with δ the Kronecker delta, and with $[r]H_D$ added for a commitment. Equal outputs on distinct messages give

$$\sum_j [\chi_j(m) - \chi_j(m')]S(j) + [r - r']H_D = \mathcal{O},$$

non-trivial because $m \mapsto \chi(m)$ is injective. Opposite outputs give

$$[2^{n+1}]Q(D') + \sum_j [\chi_j(m) + \chi_j(m')]S(j) + [r + r']H_D = \mathcal{O},$$

non-trivial because its coefficient on $Q(D')$ is non-zero. An exceptional case at step i has the form $[\alpha]\text{Acc}_{i-1} + S(m_i) = \mathcal{O}$ with $\alpha \in \{-1, 1, 2\}$; substituting (1) for Acc_{i-1} gives a relation with coefficient $\alpha 2^{i-1}$ on $Q(D')$. For the hash forms the terms in H_D are absent.

Three steps are made in place. (1) Each instance hashes one fixed length l per domain, and l is admissible: $n \leq c$, with $n = 52, 109$ and 51 (Table 4). Admissibility is exactly the hypothesis under which no coefficient wraps modulo p_{Vesta} . Because $2^n \leq 2^c \leq (p_{\text{Vesta}} - 1)/2$, each difference $\chi_j(m) - \chi_j(m')$ has absolute value below $2^n < p_{\text{Vesta}}$ and vanishes modulo p_{Vesta} only if it vanishes as an integer, so uniqueness of binary expansions makes χ injective; and $0 < 2^{n+1} \leq p_{\text{Vesta}} - 1$ and $0 < |\alpha 2^{i-1}| \leq 2^n$ keep the coefficient on $Q(D')$ non-zero. Padding to nk bits is injective on messages of the fixed length l , so $M \neq M'$ gives $m \neq m'$. (2) By the definition of incomplete addition, an instance returns $\perp$ exactly when some incomplete addition meets an exceptional case, the case the cited $\perp$ theorem treats. An operand $\mathcal{O}$ arises only if $Q(D')$ or some $S(j)$ is $\mathcal{O}$, itself a non-trivial relation, because an incomplete addition that meets no exceptional case never returns $\mathcal{O}$. The commitment forms return $\perp$ exactly when the point hash does, for every trapdoor, since the blinding addition is complete. (3) The generators $Q(D'), S(0), \dots, S(2^k - 1)$ and H_D are values of GroupHash on pairwise distinct inputs (Table 2), so the lemma on relations among fixed generators excludes each relation except with negligible probability. The consequences for the extracted forms follow from Lemma 2.4: equal extracted values other than $\perp$ come from equal or opposite points, which are the cases of (i) and (ii). $\square$

Remark 2.25 (Fixed length). The fixed-length hypothesis cannot be dropped. The point hash `SinsemillaHashToPoint` zero-pads its input to a multiple of k bits, so for a message M whose length is not a multiple of k the messages M and $M \parallel 0$ collide with no discrete-logarithm relation. The specification requires collision resistance only between inputs of one fixed length for a given domain (protocol specification, §“Sinsemilla Hash Function”), and every Orchard domain hashes a single length.

Lemma 2.26 (Hiding and binding of `SinsemillaCommit`). *Let D be a byte string and $l \leq kc$ a message length.*

- (i) *Perfect hiding: if $H_D \neq \mathcal{O}$, then for every $M \in \{0, 1\}^l$ with hash point $\hat{M} \neq \perp$ and r uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and independent of M , the commitment $\text{SinsemillaCommit}_r(D, M)$ is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$; hence the distribution of neither it nor $\text{ShortCommit}_r(D, M)$, a fixed function of it, depends on M .*
- (ii) *Computational binding: under Assumptions 2.22 and 2.8, no efficient algorithm outputs, except with negligible probability, openings (M, r) and (M', r') with $M \neq M'$ in $\{0, 1\}^l$ whose commitments $\text{SinsemillaCommit}_r(D, M)$ and $\text{SinsemillaCommit}_{r'}(D, M')$ are equal and not $\perp$.*

Proof. The lemma is the *Crypto Guide*’s Construction “SinsemillaCommit” (§“Sinsemilla: an algebraic hash-based commitment”) for the Orchard blinding base. (i) Since $H_D \neq \mathcal{O}$ and the group has prime order p_{Vesta} , the map $r \mapsto [r]H_D$ is a bijection from $\mathbb{F}_{p_{\text{Vesta}}}$ onto the group, so $[r]H_D$

is uniform, and so is its translate by the fixed point $\hat{M}$ (*Crypto Guide*, §“The Pedersen commitment”, Theorem “Perfect hiding”). (ii) Two such openings give the first relation in the proof of Proposition 2.24, whose coefficient vector on the $S(j)$ is non-zero for $n = \lfloor l/k \rfloor \leq c$ by step (1) of that proof. The only Orchard addition is the blinding base $H_D = \text{GroupHash}(D \parallel -r, \varepsilon)$, whose input differs from those of $Q(D \parallel -M)$ and of every $S(j)$, since its domain separator ends in $-r$; the lemma on relations among fixed generators excludes the relation. The specification states the same properties, with hiding conditional on no $\perp$ output (protocol specification, §“Sinsemilla commitments”). $\square$

3 Keys and addresses

This section constructs the key components of the protocol specification, §“Orchard Key Components”, from the instances of §2: the spending key and the spend-side secrets, the viewing keys, and the diversified addresses. It states each assumption that their security needs at its first use and proves, in Proposition 3.18, four one-way separations between key tiers.

3.1 The spending key and the spend-side secrets

Remark 3.1 (Scope of the key derivations). Every derivation of this section is that of a key generated with $\text{use_qsk} = \text{false}$, where use_qsk is the Boolean key-generation flag of ZIP 2005. ZIP 2005, §“Changes to the Protocol Specification”, item §4.2.3, specifies, as a change to the specification’s §“Orchard Key Components” and not in that section itself, an alternative source of ak^P and r_{ivk} , with nk derived as here; it yields components $(\text{ak}, \text{nk}, r_{\text{ivk}})$ of the same types, and ak^P again of even y -coordinate. The volume does not construct the alternative and names none of its intermediate keys. Assumption 3.13, Propositions 3.18, 3.17, 6.10 and 10.6, Proposition “Confidentiality of the outgoing ciphertext” (§10.3), and the hypotheses on honest key generation of Theorems 12.9 and 12.12 are stated for $\text{use_qsk} = \text{false}$ only. For such keys every key component is a deterministic function of the spending key sk .

Definition 3.2 (Spending key and key generation). The *spending key* sk is a uniformly random 256-bit string, identified with a 32-byte string (protocol specification, §“Constants”). Key generation draws sk , derives from it the components of this section, and discards sk and draws a new one when $\text{ask} = 0$ (below), when $\text{ivk} \in \{0, \perp\}$ (§3.2), or on the one further condition of the protocol specification, §“Orchard Key Components”, which concerns the internal key of ZIP 32, a key this volume does not construct, and is stated in §3.2. A spending key that key generation does not discard is *accepted*; it determines every other component of a key generated with $\text{use_qsk} = \text{false}$.

Construction 3.3 (Spend-side secrets). For a spending key sk , the *spend authorising key* ask , the *nullifier deriving key* nk and the trapdoor r_{ivk} of $\text{Commit}^{\text{ivk}}$ are

$$\begin{aligned} \text{ask} &:= \text{ToScalar}(\text{PRFexpand}_{\text{sk}}([0x06])) \in \mathbb{F}_{p_{\text{Vesta}}}, \\ \text{nk} &:= \text{ToBase}(\text{PRFexpand}_{\text{sk}}([0x07])) \in \mathbb{F}_{p_{\text{Pallas}}}, \\ r_{\text{ivk}} &:= \text{ToScalar}(\text{PRFexpand}_{\text{sk}}([0x08])) \in \mathbb{F}_{p_{\text{Vesta}}} \end{aligned}$$

(protocol specification, §“Orchard Key Components”). The three lead bytes are rows of Table 3. Key generation discards sk if $ask = 0$.

The scalars ask and r_{ivk} lie in $\mathbb{F}_{p_{Vesta}}$, the scalar field of the Pallas group, because each multiplies a Pallas point: ask the generator $G^{SpendAuth}$ below, r_{ivk} the blinding base of $Commit^{ivk}$ (§3.2). The key nk is an element of the base field $\mathbb{F}_{p_{Pallas}}$.

The three inputs of $PRFexpand_{sk}$ have distinct lead bytes. By Lemma 2.15, under Assumption 2.12, the triple (ask, nk, r_{ivk}) computed from a uniform sk , before rejection and before the normalisation below, is therefore computationally indistinguishable from independent uniform elements of $\mathbb{F}_{p_{Vesta}}$, $\mathbb{F}_{p_{Pallas}}$ and $\mathbb{F}_{p_{Vesta}}$, up to an additional statistical distance of three reduction distances, each below 2^{-257} (§2.3).

Construction 3.4 (Spend validating key and sign normalisation). Let $G^{SpendAuth} := \text{GroupHash}(z.cash:Orchard, G)$ (Table 2), the generator of the spend-authorisation signature scheme (protocol specification, §“Spend Authorization Signature (Sapling and Orchard)”). For $ask \neq 0$:

1. compute the point $P := [ask] G^{SpendAuth}$;
2. if its y -coordinate is odd, that is, if bit 255 of the star encoding P^* is 1, replace ask by $-ask$;
3. let $ak^P := [ask] G^{SpendAuth}$ for the updated ask , and $ak := \text{Extract}_P(ak^P) \in \mathbb{F}_{p_{Pallas}}$.

The *spend validating key* is the point ak^P together with its x -coordinate ak , a field element (protocol specification, §“Orchard Key Components”).

After step 2 the point ak^P has even y -coordinate. The point P is not $\mathcal{O}$, since $ask \neq 0$ and $G^{SpendAuth}$ generates a group of prime order, so $P = (x, y)$ with $y \neq 0$ (§2.1, on the star encoding). Replacing ask by $-ask$ replaces P by $-P = (x, -y)$, and the representatives y and $p_{Pallas} - y$ have opposite parities because p_{Pallas} is odd.

For an accepted key, $ak \in \{1, \dots, p_{Pallas} - 1\}$, and ak determines ak^P . Indeed $ak^P \neq \mathcal{O}$, so $ak \neq 0$ by Lemma 2.3. By Lemma 2.4 the points with x -coordinate ak are ak^P and $-ak^P$, whose y -coordinates have opposite parities; the point ak^P is the one with even y -coordinate, that is, $ak^P = \text{abst}_P(\text{LE}_{256}(ak))$, the *even- y lift* of ak . Theorem 12.9 cites this normalisation.

3.2 Viewing keys

Definition 3.5 (Full viewing key). The *full viewing key* of an accepted key is the triple

$$\text{fvk} := (ak, nk, r_{ivk}) \in \{1, \dots, p_{Pallas} - 1\} \times \mathbb{F}_{p_{Pallas}} \times \mathbb{F}_{p_{Vesta}},$$

with ak the field element of §3.1; the point ak^P is recomputed from ak by the even- y lift.

Construction 3.6 (Incoming viewing key scalar). With the key commitment of §2.4 and $D := \text{z.cash:Orchard-CommitIvk}$,

$$\text{ivk} := \text{Commit}_{\text{r}_{\text{ivk}}}^{\text{ivk}}(\text{ak}, \text{nk}) = \text{ShortCommit}_{\text{r}_{\text{ivk}}}(D, \text{LE}_{255}(\text{ak}) \parallel \text{LE}_{255}(\text{nk})) \in \mathbb{F}_{p_{\text{Pallas}}} \cup \{\perp\}.$$

Explicitly, let $\hat{M}$ be the Sinsemilla hash point of the 510-bit message $\text{LE}_{255}(\text{ak}) \parallel \text{LE}_{255}(\text{nk})$ under the domain $\text{z.cash:Orchard-CommitIvk-M}$, and let

$$H_D := \text{GroupHash}(\text{z.cash:Orchard-CommitIvk-r}, \varepsilon)$$

be the blinding base; then $\text{ivk} = \text{Extract}_p(\hat{M} + [\text{r}_{\text{ivk}}] H_D)$ if $\hat{M} \neq \perp$, and $\text{ivk} = \perp$ otherwise. Key generation discards sk when $\text{ivk} \in \{0, \perp\}$, so an accepted key has $\text{ivk} \in \{1, \dots, p_{\text{Pallas}} - 1\}$. Because $p_{\text{Pallas}} < p_{\text{Vesta}}$, this integer is a non-zero element of $\mathbb{F}_{p_{\text{Vesta}}}$, the scalar that multiplies the diversified base in §3.3.

Let $X_{p_{\text{Pallas}}} := \{\text{Extract}_p(P) : P \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}\} \subset \{1, \dots, p_{\text{Pallas}} - 1\}$. The set $X_{p_{\text{Pallas}}}$ has $(p_{\text{Vesta}} - 1)/2$ elements (§2.4, Definition “The key commitment”), and an accepted ivk lies in it. The blinding base H_D is not $\mathcal{O}$: the protocol specification, §“Sinsemilla commitments”, states that the commitment is distributed as a uniform point for a uniform trapdoor, which holds only for a blinding base other than $\mathcal{O}$.

Lemma 3.7 (Distribution of ivk). *Let r_{ivk} be uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and independent of (ak, nk) , and let $\hat{M} \neq \perp$. Then $\text{Commit}_{\text{r}_{\text{ivk}}}^{\text{ivk}}(\text{ak}, \text{nk})$ is 0 with probability $1/p_{\text{Vesta}}$ and otherwise uniform on $X_{p_{\text{Pallas}}}$, independently of (ak, nk) .*

Proof. By Lemma 2.26(i) the point $\hat{M} + [\text{r}_{\text{ivk}}] H_D$ is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, whatever $\hat{M}$ is. It is $\mathcal{O}$, of extracted coordinate 0, with probability $1/p_{\text{Vesta}}$; each $x \in X_{p_{\text{Pallas}}}$ is the extracted coordinate of exactly the two points $\pm P$ with $P \neq -P$ (Lemma 2.4), and so has probability $2/p_{\text{Vesta}}$. $\square$

In particular ivk is not uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ (protocol specification, §“Orchard Key Components”, note).

The further rejection condition of key generation concerns the internal key of ZIP 32, §“Orchard internal key derivation”. That key has the same ak and nk and a trapdoor $t \in \mathbb{F}_{p_{\text{Vesta}}}$ that is an efficiently computable function of $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$, and key generation discards sk when its incoming viewing key $\text{Commit}_t^{\text{ivk}}(\text{ak}, \text{nk})$ is 0 or $\perp$ (protocol specification, §“Orchard Key Components”).

Lemma 3.8 (Rejection in key generation). *Under Assumptions 2.22 and 2.8, a single draw of key generation from independent uniform $(\text{ask}, \text{nk}, \text{r}_{\text{ivk}})$ is rejected with negligible probability. Under Assumptions 2.12, 2.22 and 2.8, the output of key generation is within negligible statistical distance of a single draw from a uniform sk without rejection.*

Proof. A single draw is rejected only if $\text{ask} = 0$, of probability $1/p_{\text{Vesta}}$; or $\hat{M} = \perp$; or $\text{ivk} = 0$, of probability $1/p_{\text{Vesta}}$ by Lemma “Distribution of ivk ”; or $\text{Commit}_t^{\text{ivk}}(\text{ak}, \text{nk}) = 0$. The value $\text{Commit}_t^{\text{ivk}}(\text{ak}, \text{nk})$ commits to the message of ivk , so it is $\perp$ exactly when $\hat{M} = \perp$. The event $\hat{M} = \perp$ has negligible probability under Assumptions 2.22 and 2.8: an algorithm that samples (ask, nk) and outputs the message $\text{LE}_{255}(\text{ak}) \parallel \text{LE}_{255}(\text{nk})$ finds a $\perp$ input with that

probability, which Proposition 2.24(iii) bounds. For $\hat{M} \neq \perp$, $\text{Commit}_t^{\text{ivk}}(\text{ak}, \text{nk}) = 0$ exactly when $\hat{M} + [t] H_D = \mathcal{O}$ (Lemma 2.3). By (1), with $n = 51$ (Table 4), this is a relation among $Q(\text{z.cash:Orchard-CommitIvk-M})$, the $S(j)$ and H_D whose coefficient on the first is 2^{51} , non-zero modulo p_{Vesta} . An algorithm that samples $(\text{ask}, \text{nk}, r_{\text{ivk}})$, computes t and outputs this relation succeeds with the probability of the event, which the lemma on relations among fixed generators (§2.4) bounds under the same assumptions. The rejection event is efficiently decidable from the three outputs of $\text{PRFexpand}_{\text{sk}}$, so by Lemma 2.15 a first draw from a uniform sk is also rejected with negligible probability. Key generation outputs its first draw whenever that draw is accepted; its output is therefore within that negligible probability, in statistical distance, of a single draw without rejection. $\square$

The proofs of this section analyse a single draw and add the statistical distance bounded by this lemma, under Assumptions 2.12, 2.22 and 2.8.

Construction 3.9 (Diversifier key and outgoing viewing key). Let $K := \text{LE}_{256}(r_{\text{ivk}})$ and

$$R := \text{PRFexpand}_K([0 \times 82] \parallel \text{l2LEOSP}_{256}(\text{ak}) \parallel \text{l2LEOSP}_{256}(\text{nk})),$$

a 64-byte string, where l2LEOSP_{256} encodes the field elements ak and nk as 32-byte little-endian integers. The *diversifier key* dk is the first 32 bytes of R and the *outgoing viewing key* ovk the last 32 bytes (protocol specification, §“Orchard Key Components”; row 0×82 of Table 3). One output of PRFexpand , keyed by the encoding of r_{ivk} , yields both.

Definition 3.10 (Incoming and outgoing viewing keys). The *incoming viewing key* is the pair (dk, ivk) ; the *outgoing viewing key* is ovk . The diversifier key dk derives diversifiers (§3.3). What the holders of ivk and of ovk can compute from note ciphertexts is stated in “Trial decryption and note acceptance” (§10.4) and “The outgoing ciphertext” (§10.3).

Proposition 3.11 (Binding of ivk to (ak, nk)). Under Assumptions 2.22 and 2.8, no efficient algorithm outputs, except with negligible probability, triples $(\text{ak}, \text{nk}, r_{\text{ivk}}) \neq (\text{ak}', \text{nk}', r_{\text{ivk}}')$ with $\text{Commit}_{r_{\text{ivk}}}^{\text{ivk}}(\text{ak}, \text{nk}) = \text{Commit}_{r_{\text{ivk}}'}^{\text{ivk}}(\text{ak}', \text{nk}') \neq \perp$.

Proof. Equal extracted values come from commitment points C, C' with $C = \pm C'$ (Lemma 2.4). If the messages differ, both cases are excluded by Proposition 2.24(ii), since LE_{255} is injective on integer representatives. If the messages agree, then $r_{\text{ivk}} \neq r_{\text{ivk}}'$, the case $C = C'$ gives $[r_{\text{ivk}} - r_{\text{ivk}}'] H_D = \mathcal{O}$, impossible for $H_D \neq \mathcal{O}$ in a group of prime order, and the case $C = -C'$ is excluded by the same proposition. $\square$

Hence ivk is bound to the pair (ak, nk) , and so is every address of the key (§3.3): its transmission key $\text{pk}_d = [\text{ivk}] g_d$ with $g_d \neq \mathcal{O}$ determines ivk , an integer below p_{Vesta} . The binding is used in Lemma 12.5 (“Double-spend resistance”, §12.2) and Theorem 12.9 (“Authorisation of spends”, §12.3). Binding alone would follow from any collision-resistant hash of (ak, nk) ; the trapdoor r_{ivk} adds hiding.

Remark 3.12 (Hiding of ivk and the double use of r_{ivk}). For r_{ivk} uniform and independent of every other quantity, ivk is independent of (ak, nk) by Lemma “Distribution of ivk ”. The derivation of

(dk, ovk) is keyed by the same r_{ivk} , so once dk or ovk is known that argument no longer applies, and modelling PRFexpand only as a PRF does not restore it: the key of row 0x82 is also the trapdoor of Commit^{ivk} , outside the hypothesis of Lemma 2.15 (§2.3; protocol specification, §“Orchard Key Components”, note on the use of r_{ivk} as both trapdoor and key). The volume states the joint assumption on Pallas scalar multiplication and PRFexpand that the specification proposes as Assumption 3.13.

Assumption 3.13 (Joint hiding of the viewing keys). For keys with $\text{use_qsk} = \text{false}$: for every $(ak, nk) \in \{1, \dots, p_{\text{Pallas}} - 1\} \times \mathbb{F}_{p_{\text{Pallas}}}$, given to the distinguisher, and r_{ivk} uniform on $\mathbb{F}_{p_{\text{Vesta}}}$, the triple (ivk, dk, ovk) computed from (ak, nk, r_{ivk}) by the constructions of this subsection is computationally indistinguishable from

$$(\text{Commit}_{r_{ivk}'}^{ivk}(ak, nk), u_1, u_2),$$

where r_{ivk}' is uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and u_1, u_2 are uniform 32-byte strings, all independent. It is a precise form of the joint assumption that the protocol specification proposes (§“Orchard Key Components”, note); it is not perfect hiding. Results apply it after Lemma 2.15 (Assumption 2.12) has made r_{ivk} uniform and independent of (ak, nk) , averaging over the key-generation distribution of (ak, nk) . It is used by Propositions 3.18, 3.17, 6.10 and 10.6, by Proposition “Confidentiality of the outgoing ciphertext” (§10.3) and by Theorem 12.12.

3.3 Diversified addresses

Construction 3.14 (Diversified address). For an index $j \in \{0, \dots, 2^{88} - 1\}$ chosen by the key holder:

1. the *diversifier* is $d := \text{FF1-AES256}_{dk}(\text{LE}_{88}(j)) \in \{0, 1\}^{88}$, the FF1 mode of NIST SP 800-38G over AES-256 with key dk , radix 2, length 88 and the empty tweak (*Crypto Guide*, §“Format-preserving encryption and FF1”, Construction “The FF1 mode of NIST SP 800-38G”; protocol specification, §“Pseudo Random Permutations”);
2. the *diversified base* is $g_d := \text{DiversifyHash}(d)$ (protocol specification, §“DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions”), where

$$T := \text{GroupHash}(z.\text{cash}:\text{Orchard-gd}, \text{LEBS2OSP}_{88}(d))$$

and

$$\text{DiversifyHash}(d) := \begin{cases} T & \text{if } T \neq \mathcal{O}, \\ \text{GroupHash}(z.\text{cash}:\text{Orchard-gd}, \epsilon) & \text{otherwise;} \end{cases}$$

3. the *diversified transmission key* is $pk_d := [ivk] g_d$, with ivk read as a scalar (§2.1).

The *address* is $(d, pk_d) \in \{0, 1\}^{88} \times (\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\})$ (protocol specification, §“Orchard Key Components”).

The map LEBS2OSP_{88} packs d into 11 bytes, eight bits at a time, least significant bit first (§2.1). Every index yields an address. The map GroupHash returns a point for every input, and DiversifyHash replaces the identity by the fixed fallback point, which is not the identity: the protocol specification types $\text{DiversifyHash}^{\text{Orchard}}$ into the non-identity points. The fallback case has negligible probability under Assumption 2.8. Hence $g_d \neq \mathcal{O}$ for every d , and $pk_d \neq \mathcal{O}$ because ivk is a non-zero scalar and the group has prime order. For each key dk the map $j \mapsto d$ is a permutation of the 88-bit strings (*Crypto Guide*, §“Format-preserving encryption and FF1”, Proposition “FF1 is a keyed bijection on $\mathcal{X}$ ”), so distinct indices give distinct diversifiers: one incoming viewing key (dk, ivk) yields 2^{88} distinct addresses, all with transmission keys under the same ivk . No index is excluded.

Assumption 3.15 (FF1-AES256 as a pseudorandom permutation). For a uniform 256-bit key, FF1-AES256 with the empty tweak on $\{0, 1\}^{88}$ is a pseudorandom permutation: no efficient adversary with oracle access distinguishes it from a uniformly random permutation of $\{0, 1\}^{88}$ (*Crypto Guide*, §“Pseudorandom functions and permutations”, Definition “Pseudorandom permutation”, and §“Format-preserving encryption and FF1”, Definition “Format-preserving encryption”). It is the security requirement of the protocol specification, §“Pseudo Random Permutations”, whose note records that the distinguishing attack of Dunkelman, Kumar, Lambaooij and Sanadhya at this domain size and round count is no better than exhaustive search for the 256-bit key. It is used so that a diversifier reveals neither its index nor the key that produced it, in Proposition 3.17 (“Unlinkability of diversified addresses”).

Assumption 3.16 (DDH on Pallas). Let P be a generator of $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, let b and c be uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and a uniform on X_{Pallas} (§3.2), all independent. The triples $([a]P, [b]P, [ab]P)$ and $([a]P, [b]P, [c]P)$ are computationally indistinguishable. With a uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ instead, this is the DDH assumption of the *Crypto Guide* (§“Diffie–Hellman: computational and decisional”, Definition “Decisional Diffie–Hellman, DDH”). The volume assumes the form with a uniform on X_{Pallas} because ivk is so distributed, and claims no reduction between the two forms; the protocol specification expects the non-uniformity of ivk to cause no security issue (§“Orchard Key Components”, note). It is used by Propositions 3.17 and 10.6 (“Confidentiality and key privacy of note encryption”).

Proposition 3.17 (Unlinkability of diversified addresses). *Let two keys be generated independently with $\text{use_qsk} = \text{false}$. An adversary holds ak, nk and ovk of both keys, and none of r_{ivk}, dk and ivk of either. It chooses indices j_0, j_1 and j' with $j' \notin \{j_0, j_1\}$, and receives the address of key 0 at index j_0 , the address of key 1 at index j_1 , and the address of key β at index j' , for a uniform hidden bit β ; it outputs a bit β' . Conditioned on the event E that the three diversifiers are pairwise distinct, its advantage $|\Pr[\beta' = 1 \mid \beta = 0] - \Pr[\beta' = 1 \mid \beta = 1]|$ is negligible under Assumptions 2.12 (through Lemma 2.15), 3.13, 3.15, 2.8 and 3.16, together with Assumption 2.22 for Lemma “Rejection in key generation”. This is the unlinkability requirement of the protocol specification, §“DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions”, which stipulates distinct diversifiers.*

Proof. For each β the game is modified by a sequence of hops, each changing the adversary’s output distribution by a negligible amount (*Crypto Guide*, §“The hybrid argument”, Lemma “Hy-

brid lemma”). Each hop is justified by a reduction that runs the game; the event E is efficiently decidable from the diversifiers and has probability at least $1/2$ in every game below, so conditioning on E at most doubles each cost.

1. Each key is replaced by a single draw without rejection (Lemma “Rejection in key generation”, §3.2).
2. For each key in turn, $(\text{ask}, \text{nk}, \text{r}_{\text{ivk}})$ is replaced by independent uniform elements of the three fields (Lemma 2.15); the reduction derives the remaining components and the addresses.
3. For each key in turn, $(\text{ivk}, \text{dk}, \text{ovk})$ is replaced by

$$(\text{Commit}_{\text{r}_{\text{ivk}}}^{\text{ivk}}(\text{ak}, \text{nk}), u_1, u_2)$$

(Assumption 3.13); the reduction is given (ak, nk) , generates the other key itself, and computes the game from the triple. Now dk and ovk are uniform and independent of all else, and, except on the negligible events $\text{ivk} \in \{0, \perp\}$, the scalar ivk is uniform on X_{Pallas} and independent of (ak, nk) (Lemmas “Distribution of ivk ” and “Rejection in key generation”, §3.2).

4. For $b \in \{0, 1\}$ in turn, $\text{FF1-AES256}_{\text{dk}_b}$ is replaced by a uniformly random permutation π_b of $\{0, 1\}^{88}$ (Assumption 3.15); the reduction queries its oracle at the indices it needs. Then $d' = \pi_b(\text{LE}_{88}(j')) \neq d_\beta$ because $j' \neq j_\beta$, and on E the triple (d_0, d_1, d') is uniform on pairwise distinct triples, for either value of β . The event E fails only if $d_{1-\beta}$ equals d_β or d' , of probability $2/2^{88}$.
5. On E the inputs $\text{LEBS2OSP}_{88}(d)$ of GroupHash are distinct, since LEBS2OSP_{88} is injective, so under Assumption 2.8 the bases g_0, g_1, g' are independent uniform points, each $\mathcal{O}$, which invokes the fallback, with probability $1/p_{\text{Vesta}}$ only.

In the last game the adversary’s view consists of $\text{ak}, \text{nk}, \text{ovk}$ of both keys and (d_0, d_1, d') , independent of the rest and identically distributed for $\beta = 0, 1$, together with

$$(g_0, [\text{ivk}_0] g_0), \quad (g_1, [\text{ivk}_1] g_1), \quad (g', [\text{ivk}_\beta] g'),$$

where $\text{ivk}_0, \text{ivk}_1$ are independent and uniform on X_{Pallas} . The first two pairs are public keys of the per-key-base ElGamal scheme with private keys ivk_0 and ivk_1 . For $g_\beta \neq \mathcal{O}$ the uniform point g' equals $[r] g_\beta$ for a uniform scalar r , so the third pair is $([r] g_\beta, [r]([\text{ivk}_\beta] g_\beta))$, an encryption of $\mathcal{O}$ under key β (*Crypto Guide*, §“Key privacy”, Construction “ElGamal encryption”). Distinguishing β is therefore the game of Definition “Key privacy under chosen-plaintext attack, IK-CPA” of the same subsection, for the message $\mathcal{O}$. The proof of its Theorem “ElGamal is key-private under DDH” bounds the advantage by two DDH advantages. That proof plants the first exponent of the DDH triple as a private key; with private keys uniform on X_{Pallas} , the same proof, with the other key’s private key drawn uniformly from X_{Pallas} (the x -coordinate of a uniform non-identity point), reduces to Assumption 3.16. Its reduction samples (d_0, d_1, d') itself and programs GroupHash at their three inputs with the planted bases before running the adversary. The *Crypto Guide*’s Remark “Key privacy of note encryption and diversified addresses” records this correspondence for diversified addresses. Summing the costs of the hops, the advantage is negligible. $\square$

3.4 Key capabilities and scope

Figure 1 gives the derivation graph, each edge labelled by its operation, and each one-way edge also by the assumption that makes it one-way. Proposition 3.18 proves four separations: the full viewing key yields no ask, the incoming viewing key neither nk nor ask, the outgoing viewing key no ivk, and an address no ivk.

Proposition 3.18 (Capability separation). *Let a key be generated honestly with $\text{use_qsk} = \text{false}$. No efficient algorithm, given the public parameters and only the listed key material of the key, outputs the listed secret, except with negligible probability:*

- (a) *given the full viewing key $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$, the scalar ask;*
- (b) *given the incoming viewing key (dk, ivk) , the key nk or the scalar ask;*
- (c) *given ovk , the scalar ivk;*
- (d) *given an address (d, pk_d) of the key, the scalar ivk.*

The hypotheses are: for (a), Assumptions 2.12 (through Lemma 2.15) and 2.22; for (b), those of (a) and, for nk, Assumption 3.13; for (c), Assumptions 2.12 and 3.13; for (d), Assumptions 2.12, 3.13, 2.8 and 2.22 (“Discrete logarithms on Pallas”). Every part also uses Lemma “Rejection in key generation” (§3.2), under Assumptions 2.12, 2.22 and 2.8. These are the only separations claimed.

Proof. By Lemma “Rejection in key generation” (§3.2), a single draw from a uniform sk replaces the key at a negligible cost. By Lemma 2.15, its triple $(\text{ask}, \text{nk}, \text{r}_{\text{ivk}})$ is then replaced by independent uniform elements of $\mathbb{F}_{p_{\text{Vesta}}}$, $\mathbb{F}_{p_{\text{Pallas}}}$ and $\mathbb{F}_{p_{\text{Vesta}}}$, at a further negligible cost: in each part the success of an algorithm is efficiently decidable from the three values, by deriving the remaining components, running the algorithm and comparing its output with the secret. The result is called the idealised key.

(a) Let an algorithm $\mathcal{A}$ output ask from $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$ with probability ϵ for the idealised key. An algorithm $\mathcal{B}$ receives $Y = [s] G^{\text{SpendAuth}}$ for s uniform on $\mathbb{F}_{p_{\text{Vesta}}}$, draws nk and r_{ivk} uniformly, and runs $\mathcal{A}$ on $(\text{Extract}_p(Y), \text{nk}, \text{r}_{\text{ivk}})$. The input is distributed as for the idealised key, because the sign normalisation replaces ask by $\pm \text{ask}$ and does not change ak (Lemma 2.4). A correct output a satisfies $[a] G^{\text{SpendAuth}} = \text{ak}^P$, the even- γ lift of $\text{Extract}_p(Y)$, which is Y or $-Y$; $\mathcal{B}$ outputs whichever of a and $-a$ maps to Y . Hence $\mathcal{B}$ computes discrete logarithms to the generator $G^{\text{SpendAuth}}$ with probability ϵ , and ϵ is negligible by Assumption 2.22.

(b) An algorithm that computes ask from (dk, ivk) computes it from $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$ by first deriving (dk, ivk) , so (a) applies. Let $\mathcal{A}$ output nk from (dk, ivk) with probability ϵ . A distinguisher for Assumption 3.13, given (ak, nk) of the idealised key and a triple (x, y, z) , runs $\mathcal{A}$ on (y, x) and outputs 1 exactly when $\mathcal{A}$ outputs nk. On the real triple it outputs 1 with probability ϵ . On the ideal triple $\mathcal{A}$ receives u_1 and $\text{Commit}_{\text{r}_{\text{ivk}}}^{\text{ivk}}(\text{ak}, \text{nk})$, which is independent of nk unless $\hat{\mathcal{M}} = \perp$, a negligible event (Lemma “Rejection in key generation”); since nk is uniform on $\mathbb{F}_{p_{\text{Pallas}}}$, $\mathcal{A}$ then succeeds with probability at most $1/p_{\text{Pallas}}$ plus a negligible term. So ϵ exceeds the distinguisher’s advantage by a negligible amount at most.

(c) Let $\mathcal{A}$ output ivk from ovk with probability ϵ . The distinguisher given (ak, nk) and (x, y, z)

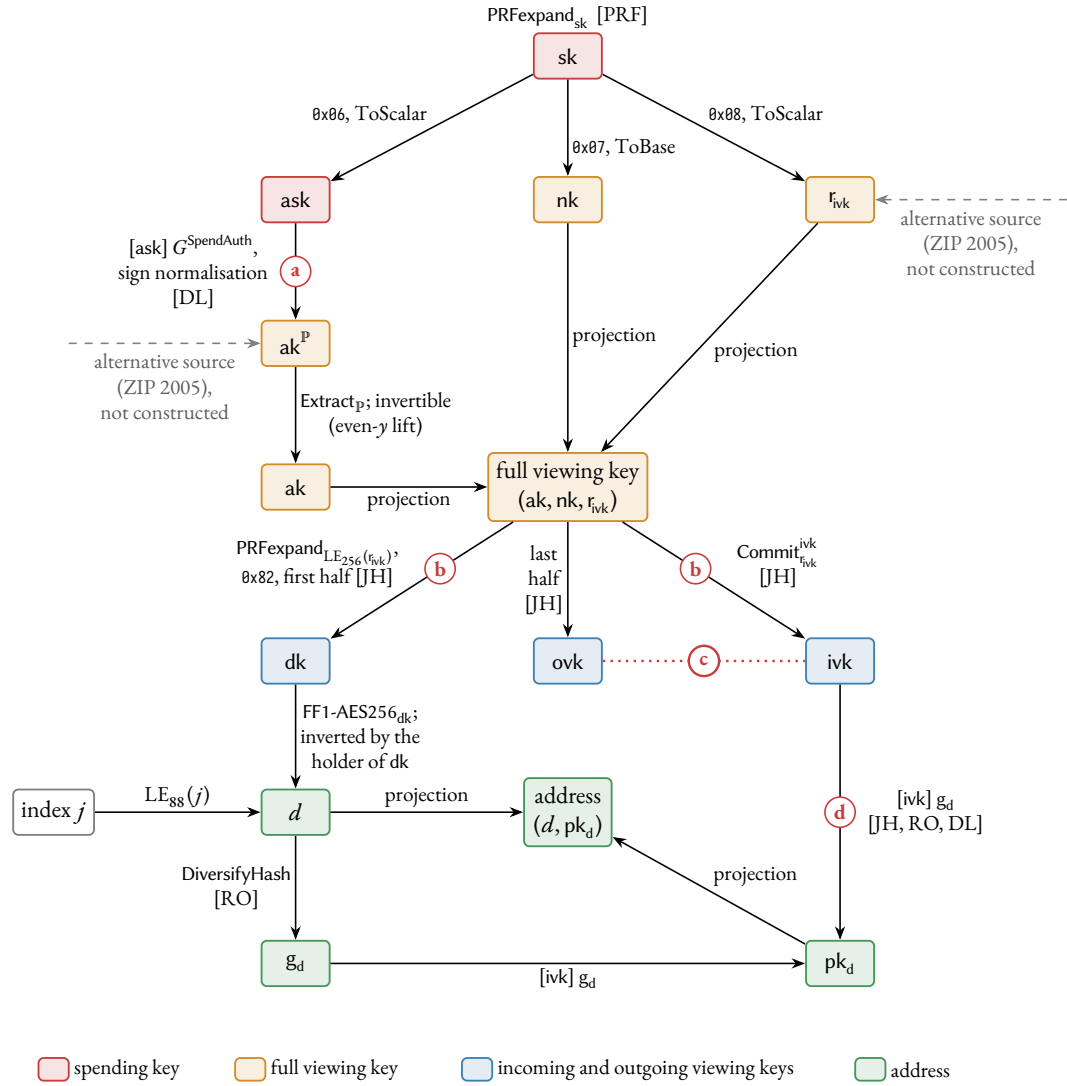


Figure 1. Derivation graph of a key generated with `use_ksk = false`, from the spending key sk to the address (d, pk_d) . Each edge carries its operation, and each one-way edge also carries, in brackets, the assumption under which it is one-way: PRF, Assumption 2.12; DL, Assumption 2.22; JH, Assumption 3.13; RO, Assumption 2.8. Each one-way edge is a PRF call, a scalar multiplication, the joint hiding assumption or GroupHash as a random oracle; bundling edges invert by projection, Extract_p by the even- γ lift, and the FF1 edge is inverted by the holder of dk . The edges out of sk are one-way under Assumption 2.12, since an algorithm recovering sk from (ask, nk, r_{ivk}) distinguishes PRFexpand_{sk} from a random function. Colours give the lowest key tier that computes a node; the index j is uncoloured. The dashed edges into ak^p and r_{ivk} mark the alternative source of ZIP 2005, not constructed. Markers (a) to (d) are the four separations that Proposition 3.18 proves; no other separation is claimed.

runs $\mathcal{A}$ on z and outputs 1 exactly when $\mathcal{A}$ outputs x . On the ideal triple, $z = u_2$ is independent of $x = \text{Commit}_{\text{ivk}}^{\text{ivk}}(\text{ak}, \text{nk})$, which, for $\hat{M} \neq \perp$, takes each value with probability at most $2/p_{\text{Vesta}}$ by Lemma “Distribution of ivk”. Hence ϵ is at most the advantage under Assumption 3.13 plus $2/p_{\text{Vesta}}$ and a negligible term.

(d) Let $\mathcal{A}$ output ivk from the address at an index j , possibly of its choice, with probability ϵ . The distinguisher given (ak, nk) and (x, y, z) computes $d := \text{FF1-AES256}_y(\text{LE}_{88}(j))$, $g_d := \text{DiversifyHash}(d)$ and $\text{pk}_d := [x] g_d$, runs $\mathcal{A}$ on (d, pk_d) , and outputs 1 exactly when $\mathcal{A}$ outputs x . On the ideal triple, $\mathcal{A}$ succeeds with probability $\epsilon' \geq \epsilon - \delta$, for δ the advantage under Assumption 3.13; there, outside the negligible event $\hat{M} = \perp$, the value $\text{ivk} := x$ is 0 with probability $1/p_{\text{Vesta}}$ and otherwise uniform on X_{Pallas} , independently of (ak, nk) and of u_1 , hence of d . An algorithm $\mathcal{B}$ receives a generator G and $Y = [s] G$ for s uniform on $\mathbb{F}_{p_{\text{Vesta}}}$. It draws t uniform on $\mathbb{F}_{p_{\text{Vesta}}} \setminus \{0\}$ and u_1 uniform, computes d from u_1 and j , programs GroupHash at $(z.\text{cash}:\text{Orchard-gd}, \text{LEBS2OSP}_{88}(d))$ with the uniform non-identity point $[t] G$ (Assumption 2.8), answers other queries with fresh uniform points, and runs $\mathcal{A}$ on $(d, [t] Y) = (d, [s]([t] G))$. With probability $(p_{\text{Vesta}} - 1)/(2p_{\text{Vesta}})$ the exponent s lies in X_{Pallas} ; it is then uniform on X_{Pallas} , and the view of $\mathcal{A}$ is that of the ideal triple conditioned on $\text{ivk} \neq 0$, up to statistical distance $1/p_{\text{Vesta}}$ for the programmed point. Algorithm $\mathcal{B}$ outputs the output of $\mathcal{A}$ and so computes s with probability at least

$$\frac{p_{\text{Vesta}} - 1}{2p_{\text{Vesta}}} \left(\epsilon' - \frac{2}{p_{\text{Vesta}}} \right),$$

a loss factor of $2p_{\text{Vesta}}/(p_{\text{Vesta}} - 1)$, about 2, against Assumption 2.22. Hence ϵ' , and with it ϵ , is negligible. $\square$

Of the objects of this section only an address (d, pk_d) , with the base g_d computed from d , is public; sk , ask , ak^{P} , ak , nk , r_{ivk} , the full viewing key, dk , ivk , ovk and the index j are secret key material. One incoming viewing key yields 2^{88} addresses (§3.3), unlinkable in the sense of Proposition 3.17 (“Unlinkability of diversified addresses”).

What a holder of each key can compute from the objects of later sections is stated where those objects are constructed, each with a back-reference to Proposition 3.18: from nullifiers in “Nullifier uniqueness and unlinkability” (§6.4), and from note ciphertexts in “The outgoing ciphertext” (§10.3) and “Trial decryption and note acceptance” (§10.4). This section states no such power.

Keys and addresses are objects of the Orchard protocol, not of a pool: one key hierarchy and one address format serve both the Orchard pool and the Ironwood pool (ZIP 229, Abstract).

4 Notes and note commitments

This section defines the note, its commitment and its extracted commitment, and proves their hiding and binding; it constructs the derivation of all randomness of a created note from one seed under the lead byte $0x03$, and defines the Action and the bundle as units of a transaction.

4.1 Notes

Definition 4.1 (Unit of value). The *zatoshi* is the smallest unit of value; one ZEC, the unit of the native currency of Zcash, is 10^8 zatoshi (protocol specification, §“Shielded Pools and Notes” and §“Mainnet and Testnet”). Every value in this volume, note values included, is an integer number of zatoshi.

Definition 4.2 (Note). A *note* is a tuple $n = (\text{addr}, v, \rho, \psi, \text{rcm})$, where

- the address $\text{addr} = (d, \text{pk}_d)$ of the recipient, of the type constructed in §3.3: a diversifier $d \in \{0, 1\}^{88}$ and a transmission key $\text{pk}_d \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}$;
- the value $v \in \{0, \dots, 2^{64} - 1\}$, in zatoshi;
- the elements $\rho \in \mathbb{F}_{p_{\text{Pallas}}}$ and $\psi \in \mathbb{F}_{p_{\text{Pallas}}}$;
- the commitment trapdoor $\text{rcm} \in \mathbb{F}_{p_{\text{Vesta}}}$

(protocol specification, §“Shielded Pools and Notes”; the lengths 88 and 64 are those of §“Constants”). The specification types rcm as an integer in $\{0, \dots, 2^{255} - 1\}$ (§“Commitment”). This volume types it in $\mathbb{F}_{p_{\text{Vesta}}}$, because every derivation of rcm yields a reduced value and rcm acts only as the scalar of $[\text{rcm}]$, which depends on rcm modulo p_{Vesta} alone; this typing makes the binding of Proposition 4.5 a statement about notes. For a created note, ψ and rcm are derived from its note seed, for an Ironwood-pool note in §4.3, and ρ is the nullifier of the note consumed in the same Action, a rule constructed in “Nullifier chaining” (§6.3).

A note is never published. A transaction that creates a note publishes its extracted note commitment cmx (Definition 4.4) and a ciphertext of its note plaintext for the recipient, constructed in “Encryption to the recipient” (§10.2) (protocol specification, §“Note Commitments” and §“Sending Notes (Orchard)”). Of the fields of a created note, only ρ appears on chain in the clear: it equals the nullifier that the same Action publishes for its consumed note (“Nullifier chaining”, §6.3). This representation of value serves requirement R1 of §1.4; Propositions 4.5 and 4.8 state what the commitment hides.

Remark 4.3 (Roles of the fields). The address and the value are the content of the note, its recipient and its amount; the trapdoor rcm randomises its commitment; the elements ρ and ψ , which also enter the commitment, are the note-specific inputs of its nullifier, constructed in “The nullifier” (§6.1).

4.2 Note commitments

Definition 4.4 (Note commitment). Let $n = ((d, \text{pk}_d), v, \rho, \psi, \text{rcm})$ be a note, $\text{g}_d := \text{DiversifyHash}(d)$ (§3.3) and

$$\mathcal{M}(n) := \text{g}_d^* \parallel \text{pk}_d^* \parallel \text{LE}_{64}(v) \parallel \text{LE}_{255}(\rho) \parallel \text{LE}_{255}(\psi),$$

with ρ and ψ read as integers in $\{0, \dots, p_{\text{Pallas}} - 1\}$; the string $M(n)$ has the fixed length 1086 of Table 4. With $D := \text{z.cash:Orchard-NoteCommit}$, the *note commitment* of n is

$$\text{cm} := \text{NoteCommit}_{\text{rcm}}(M(n)) := \text{SinsemillaCommit}_{\text{rcm}}(D, M(n)) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \cup \{\perp\}.$$

By the routing of §2.4, with the hash point and the blinding base

$$\begin{aligned}\hat{M} &:= \text{SinsemillaHashToPoint}(\text{z.cash:Orchard-NoteCommit-M}, M(n)), \\ H_D &:= \text{GroupHash}(\text{z.cash:Orchard-NoteCommit-r}, \varepsilon),\end{aligned}$$

the commitment is $\text{cm} = \hat{M} + [\text{rcm}] H_D$ if $\hat{M} \neq \perp$, and $\text{cm} = \perp$ otherwise. The *extracted note commitment* is $\text{cmx} := \text{Extract}_{\text{P}}^{\perp}(\text{cm}) \in \mathbb{F}_{p_{\text{Pallas}}} \cup \{\perp\}$ (§2.1). The specification’s `NoteCommit` takes the five encoded fields as separate arguments, in the order of their encodings in $M(n)$ (protocol specification, §“Sinsemilla commitments”, §“Sending Notes (Orchard)” and §“Coordinate Extractor for Pallas”).

A sender never publishes $\perp$: it redraws the note seed (step (vi) of the derivation of §4.3). The extracted commitment cmx is the value an Action publishes for its created note, and the leaf of that note in the pool’s note commitment tree of “The Merkle hash and the tree” (§5.1), whose nodes are elements of $\mathbb{F}_{p_{\text{Pallas}}}$.

The blinding base H_D is not $\mathcal{O}$: the protocol specification, §“Sinsemilla commitments”, states that `SinsemillaCommit` is perfectly hiding for a uniform trapdoor, for every domain, which holds only for a blinding base other than $\mathcal{O}$ (as for `Commit`^{ivk}, §3.2).

Proposition 4.5 (Hiding and binding of note commitments).

- (a) *Perfect hiding.* Let rcm be uniform on $\mathbb{F}_{p_{\text{Vesta}}}$. For every note content $(d, \text{pk}_d, v, \rho, \psi)$ whose message $M(n)$ has hash point $\hat{M} \neq \perp$, the commitment cm is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$. Hence the distributions of cm and of cmx do not depend on $(g_d, \text{pk}_d, v, \rho, \psi)$, and `NoteCommit` is perfectly hiding on messages without a $\perp$ output, in the sense of the *Crypto Guide*’s Definition “Hiding game and hiding” (§“Hiding and binding”).
- (b) *Binding of the extracted commitment.* Under Assumptions 2.22 and 2.8, no efficient algorithm outputs, except with negligible probability, two notes $n \neq n'$ with $\text{cmx}(n) = \text{cmx}(n') \neq \perp$. In particular two distinct notes share no note commitment other than $\perp$. With Proposition 2.24(iii), which excludes messages with output $\perp$, part (b) gives computational binding in the sense of the *Crypto Guide*’s Definition “Binding game and binding” (same section), with the note content as the message and rcm as the randomness, stated for notes and for the extracted value.

Proposition 4.8 states hiding for the trapdoor that a sender derives.

Proof. (a) This is Lemma 2.26(i) for the domain D and the message length 1086, whose hypothesis $H_D \neq \mathcal{O}$ holds by the paragraph above; the value cmx is a fixed function of cm . Equal distributions for every pair of contents without a $\perp$ output are perfect hiding by the *Crypto Guide*’s Proposition “Distributional characterisation of hiding” (§“Hiding and binding”).

(b) Step 1 reduces distinct notes to distinct openings. The map $(g_d, pk_d, v, \rho, \psi) \mapsto M(n)$ is injective: the string $M(n)$ concatenates fields of fixed lengths; the star encoding is injective (§2.1); the encoding LE_{64} is injective on $\{0, \dots, 2^{64} - 1\}$; and the encoding LE_{255} is injective on the representatives of $\mathbb{F}_{p_{Pallas}}$, since $p_{Pallas} < 2^{255}$ (§2.1). The map $n \mapsto (M(n), rcm)$ is therefore injective except on pairs of notes with $d \neq d'$ and $\text{DiversifyHash}(d) = \text{DiversifyHash}(d')$. Let T and T' be the values of GroupHash at $(z.\text{cash}:\text{Orchard-gd}, \text{LEBS2OSP}_{88}(d))$ and at the corresponding input for d' , distinct inputs because LEBS2OSP_{88} is injective. If $T, T' \neq \mathcal{O}$, equal diversified bases give $T - T' = \mathcal{O}$; otherwise $T = \mathcal{O}$ or $T' = \mathcal{O}$. Each is a non-trivial relation among values of GroupHash on distinct inputs, which the lemma on relations among fixed generators (§2.4) excludes except with negligible probability.

Step 2 treats distinct openings. Let $\text{cmx}(n) = \text{cmx}(n') \neq \perp$. The lifting $\text{Extract}_{\perp}^{\perp}$ maps only $\perp$ to $\perp$, so cm and cm' are points, and by Lemma 2.4 $\text{cm}' = \text{cm}$ or $\text{cm}' = -\text{cm}$. The pairs $(M(n), rcm)$ and $(M(n'), rcm')$ are openings of the instance NoteCommit in $\{0, 1\}^{1086} \times \mathbb{F}_{p_{Vesta}}$. By Proposition 2.24(ii), outputs with $\text{cm}' = -\text{cm}$, or with $\text{cm}' = \text{cm}$ and $M(n) \neq M(n')$, have negligible probability (in the opposite case the relation in its proof has coefficient 2^{109+1} on the hash base, for a message of 109 chunks). In the remaining case $\text{cm}' = \text{cm}$ and $M(n) = M(n')$, and outside the event of Step 1 the openings differ, so $rcm \neq rcm'$ in $\mathbb{F}_{p_{Vesta}}$; then $[rcm - rcm'] H_D = \mathcal{O}$, impossible for $H_D \neq \mathcal{O}$ in a group of prime order p_{Vesta} . A note commitment other than $\perp$ shared by $n \neq n'$ gives equal extracted commitments other than $\perp$, which proves the particular case. $\square$

4.3 The note seed: Ironwood derivations (lead byte 0x03)

Definition 4.6 (Note seed and lead byte). A created note reaches its recipient as a *note plaintext*, the byte string constructed in “The note plaintext” (§10.1) that the sender encrypts. Its first byte, the *lead byte*, names the format of the plaintext and selects the derivation of the note’s randomness. The sender draws one *note seed* $rseed$, a uniform 32-byte string, the key type of PRFexpand (§2.3), which the plaintext carries. The values ψ and rcm of the note and the ephemeral secret key $\text{esk} \in \mathbb{F}_{p_{Vesta}}$ of “Encryption to the recipient” (§10.2) are all derived from it (protocol specification, §“Note Plaintexts and Memo Fields” and §“Sending Notes (Orchard)”).

Construction 4.7 (Derivations under lead byte 0x03). The input is an address (d, pk_d) , a value $v \in \{0, \dots, 2^{64} - 1\}$ and $\rho \in \mathbb{F}_{p_{Pallas}}$. Let $g_d := \text{DiversifyHash}(d)$ and $\underline{\rho} := \text{I2LEOSP}_{256}(\rho)$. For a point $P = (x, y)$ the 32-byte string $\text{LEBS2OSP}_{256}(P^*)$ equals $\text{I2LEOSP}_{256}(x + 2^{255}(y \bmod 2))$ (§2.1). The sender

- (i) checks that $pk_d \neq \mathcal{O}$;
- (ii) draws $rseed$ uniformly from the 32-byte strings;
- (iii) derives $\text{esk} := \text{ToScalar}(\text{PRFexpand}_{rseed}([0x04] \parallel \underline{\rho})) \in \mathbb{F}_{p_{Vesta}}$, and returns to (ii) if $\text{esk} = 0$;
- (iv) derives $\psi := \text{ToBase}(\text{PRFexpand}_{rseed}([0x09] \parallel \underline{\rho})) \in \mathbb{F}_{p_{Pallas}}$;

(v) derives $\text{rcm} := \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}(\text{pre_rcm})) \in \mathbb{F}_{p_{\text{Vesta}}}$, where

$$\begin{aligned} \text{pre_rcm} := & [\text{0x0B}] \parallel \text{LEBS2OSP}_{256}(\text{g}_d^*) \parallel \text{LEBS2OSP}_{256}(\text{pk}_d^*) \\ & \parallel \text{I2LEOSP}_{64}(v) \parallel \underline{\rho} \parallel \text{I2LEOSP}_{256}(\psi), \end{aligned}$$

a string of $1 + 32 + 32 + 8 + 32 + 32 = 137$ bytes;

(vi) computes cmx of the note $n := ((d, \text{pk}_d), v, \rho, \psi, \text{rcm})$ by Definition 4.4, and returns to (ii) if $\text{cmx} = \perp$.

The output is the note n , its seed rseed and esk (protocol specification, §“Sending Notes (Orchard)”; ZIP 2005, “Changes to the Protocol Specification”, §4.7.3).

The order is forced: the string pre_rcm contains ψ , so rcm is derived after ψ . The strings g_d^* and pk_d^* in pre_rcm are those of the message $M(n)$, packed into bytes, so rcm is a function of rseed and of every committed field $(\text{g}_d, \text{pk}_d, v, \rho, \psi)$. The leading bytes 0x04 , 0x09 and 0x0B are rows of Table 3.

An Ironwood-pool note has the type of Definition “Note” (§4.1) and is committed by the same NoteCommit (Definition 4.4). Every note created by an Ironwood-pool Action, a dummy created note included, has a plaintext with lead byte 0x03 and the derivations above; no note created by an Orchard-pool Action does. This is the only note-level distinction between the two pools (ZIP 229, “Transaction Format”; ZIP 258, “ZIP 2005 activation”).

Proposition 4.8 (Hiding under the derived trapdoor). *An adversary chooses two tuples $(d^{(i)}, \text{pk}_d^{(i)}, v^{(i)}, \rho^{(i)})$, $i \in \{0, 1\}$, of the input type of the construction above, with $\text{pk}_d^{(i)} \neq \mathcal{O}$. The challenger draws a uniform bit b , runs steps (ii) to (vi) of the construction on tuple b , and returns $(\text{cm}_b, \text{esk}_b)$, the note commitment of the note it computes and its esk ; the adversary receives no other function of rseed (the ciphertext that carries rseed is treated in Theorem 12.12). It outputs a bit b' . Let the number ϵ_{PRF} bound the advantage against Assumption 2.12 of each algorithm built from the adversary in the proof, each of which queries $\text{PRFexpand}_{\text{rseed}}$ on three inputs with pairwise distinct leading bytes; let $\delta < 2^{-257}$ bound the distance of ToScalar from uniform (§2.3); and let $\epsilon_{\perp}$ be the largest, over i , of the sum of two probabilities that one pass through steps (ii) to (vi) on tuple i returns to step (ii), one with $\text{PRFexpand}_{\text{rseed}}$ as specified and one with its three outputs replaced by independent uniform strings. Then*

$$|\Pr[b' = 1 \mid b = 0] - \Pr[b' = 1 \mid b = 1]| \leq 2(\epsilon_{\text{PRF}} + \delta + \epsilon_{\perp}),$$

and ϵ_{PRF} and $\epsilon_{\perp}$ are negligible under Assumptions 2.12, 2.22 and 2.8. The same bound holds with cmx_b in place of cm_b .

Proof. Fix the adversary’s tuples. One pass through steps (ii) to (vi) with a fresh rseed is a *trial*; its output is (cm, esk) , and it is *accepted*, event $\mathcal{A}$, when $\text{esk} \neq 0$ and $\hat{M} \neq \perp$, since $\text{cmx} = \perp$ exactly when $\hat{M} = \perp$. Trials are independent and identically distributed, and the challenger returns the output of the first accepted one; so its answer X_i on tuple i is distributed as the output Y_i of one trial conditioned on $\mathcal{A}$. For a random variable Y and an event $\mathcal{A}$ with $\Pr[\neg \mathcal{A}] = p < 1$, the statistical

distance between Y and Y conditioned on A is at most p : for every set S ,

$$\Pr[Y \in S] - \Pr[Y \in S \mid A] = \Pr[Y \in S, \neg A] - \Pr[Y \in S, A] \frac{p}{1-p},$$

and both terms lie in $[0, p]$.

Hybrid 1 replaces the outputs of $\text{PRFexpand}_{\text{rseed}}$ on $[0x04] \parallel \underline{\rho}$, $[0x09] \parallel \underline{\rho}$ and pre_rcm by independent uniform $u_1, u_2, u_3 \in \{0, 1\}^{512}$, so that $\text{esk} = \text{ToScalar}(u_1)$, $\psi = \text{ToBase}(u_2)$ and $\text{rcm} = \text{ToScalar}(u_3)$. The three inputs have distinct leading bytes, the input pre_rcm is chosen after the answer that determines ψ , and rseed is used only as a key of PRFexpand . By Lemma 2.15 (Assumption 2.12), the probability that any efficient test outputs 1 on the trial's output changes by at most ϵ_{PRF} . Hybrid 2 replaces rcm by a uniform element of $\mathbb{F}_{p_{\text{Vesta}}}$, independent of all else, at statistical cost δ . The event A does not involve rcm , so its probability is the same in both hybrids.

In Hybrid 2 the values esk and ψ are independent, A is the intersection of an event on esk and an event on ψ , and rcm is uniform and independent of both. Conditioned on A , therefore, $\hat{M} \neq \perp$ and $\text{cm} = \hat{M} + [\text{rcm}] H_D$ is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ by Proposition 4.5(a), independently of esk , and esk is distributed as $\text{ToScalar}(u_1)$ conditioned on being non-zero. This distribution $\mathcal{W}$ does not depend on i .

For each i , the adversary's output probability on X_i differs from that on $\mathcal{W}$ by at most the rejection probability of a trial as specified (X_i against Y_i), plus $\epsilon_{\text{PRF}} + \delta$ (Y_i against its form in Hybrid 2), plus the rejection probability in Hybrid 2, which equals that with uniform strings (the unconditioned form in Hybrid 2 against $\mathcal{W}$). The two rejection probabilities sum to at most $\epsilon_{\perp}$. Adding the bounds for $i = 0$ and $i = 1$ gives the stated bound. For cmx_b , an adversary against cmx_b is one against cm_b that first applies the fixed map $\text{Extract}_P^{\perp}$.

Negligibility. With uniform strings, $\text{esk} = 0$ has probability at most $1/p_{\text{Vesta}} + \delta$. The event $\hat{M} = \perp$ has the probability with which an efficient algorithm finds a message of hash point $\perp$: the algorithm that runs the adversary's choice of tuples, draws u_2 , and outputs $\mathcal{M}(n)$ for tuple i with $\psi = \text{ToBase}(u_2)$. That probability is negligible by Proposition 2.24(iii) under Assumptions 2.22 and 2.8. Rejection is efficiently decidable from the three outputs of $\text{PRFexpand}_{\text{rseed}}$, so the rejection probability as specified exceeds that with uniform strings by at most ϵ_{PRF} , negligible under Assumption 2.12. $\square$

Remark 4.9 (Purpose of the hashed trapdoor). Because rcm is a hash of rseed and of every committed field, Ironwood-pool notes are recoverable in the sense of ZIP 2005. That ZIP proposes, with details subject to change, a Recovery Protocol for recovering Ironwood-pool funds if the Orchard protocol must be disabled because discrete logarithms become computable; a recovery protocol whose statement recomputes rcm by this derivation can bind a note even against an adversary able to compute discrete logarithms (ZIP 2005, Abstract; Rationale, “Proposed Recovery Protocol” and “Repairing note commitments”). Against such an adversary NoteCommit alone is not binding: the discrete-logarithm relations between its bases give a linear equation over $\mathbb{F}_{p_{\text{Vesta}}}$ with several solutions, hence two distinct notes that open one commitment (ZIP 2005, Rationale, “Attacks against binding of note commitments”). What the deployed protocol checks of the derivation is stated with the Action statement, defined in “The Action statement” (§9.2).

The specification defines, for each pool, the set $\text{allowedLeadBytes}^{\text{pool}}$ of lead bytes allowed in a note plaintext of that pool. For every transaction that contains Actions of the pool, the set is

$\{0x03\}$ for the Ironwood pool and $\{0x02\}$ for the Orchard pool (protocol specification, §“Note Plaintexts and Memo Fields”; ZIP 2005, “Changes to the Protocol Specification”, §3.2.1; ZIP 258, “ZIP 2005 activation”). The lead byte travels encrypted, and each decrypting party enforces the rule: decryption with an incoming viewing key, by the recipient, and decryption with an outgoing viewing key, by its holder, both return $\perp$ when the lead byte is not allowed (§“Decryption using an Incoming Viewing Key (Sapling and Orchard)” and §“Decryption using an Outgoing Viewing Key (Sapling and Orchard)”). For an Ironwood-pool output this is check (i) of the acceptance procedure of “Trial decryption and note acceptance” (§10.4). The one consensus check of the lead byte is stated in “Chain state and pool rules” (§11.4).

4.4 Actions, bundles, and dummy notes

Definition 4.10 (Action). An *Action* of a pool consumes one note n^{old} and creates one note n^{new} of that pool; it is the specification’s Action transfer, and every Action has both sides (protocol specification, §“Action Transfers and their Descriptions”). Its public data carry the extracted commitment cmx of the created note (Definition 4.4), the nullifier of the consumed note (“The nullifier”, §6.1), a randomised validating key with a spend-authorisation signature (“Randomised validating keys”, §7.2), a net value commitment (“Value commitments”, §8.1), and the ciphertexts of “Encryption to the recipient” (§10.2). “Action descriptions and bundles” (§11.1) assembles them and redefines neither the Action nor the bundle. The relation that each Action must satisfy is the Action statement, defined in “The Action statement” (§9.2).

Definition 4.11 (Dummy notes). Every Action has one consumed and one created note, so that its public fields are the same whether a side is real or not; a side that a payment does not use carries a *dummy note*, a note of value 0. A *dummy consumed note* is built as follows (protocol specification, §“Dummy Notes (Orchard)”; ZIP 2005, “Changes to the Protocol Specification”, §4.8.3):

- (i) draw a fresh spending key as in §3.1, and derive its key components and an address (d, pk_d) (§3.2, §3.3);
- (ii) set $v := 0$, draw rseed uniformly, and set $\rho := \text{Extract}_P(P)$ for a uniform point P of $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$;
- (iii) derive ψ and rcm under the lead byte chosen as for a created note of the pool (ZIP 2005, “Changes to the Protocol Specification”, §4.8.3); in the Ironwood pool the lead byte is $0x03$ and the derivations are steps (iv) and (v) of §4.3;
- (iv) compute cm by Definition 4.4, and repeat from the draw of rseed with a fresh seed if $\text{cm} = \perp$.

Its nullifier is computed as for any note (“The nullifier”, §6.1). For step (iii) the two sources differ: the specification’s section chooses the lead byte with the pool fixed to the Orchard pool, which gives $0x02$ also in the Ironwood pool, whereas ZIP 2005, §4.8.3, makes the choice identical for dummy and non-dummy notes. The volume follows the ZIP. No condition of the Action

statement checks the derivation of rcm^{old} (Definition 9.2, Remark “What the statement does not check”).

A *dummy created note* is a created note of value 0, built as any created note; in the Ironwood pool it therefore carries lead byte $0x03$. Its address depends on the flag `enableCrossAddress` of the bundle (“Bundle flags”, §9.1).

- When `enableCrossAddress` = 1, the dummy created note goes to a random address (protocol specification, §“Dummy Notes (Orchard)”).
- When `enableCrossAddress` = 0, as in every Orchard-pool bundle, condition A10 of Definition 9.2 places the note that an Action creates at the expanded receiver, defined in “Bundle flags”, of the note that the same Action consumes. A dummy created note then goes to the expanded receiver of that consumed note, real or dummy. In the Orchard pool, beside a real consumed note, the sender fills the component C_{enc} of the note ciphertext of the dummy created note (“Encryption to the recipient”, §10.2) with random bytes, not with an encryption of its plaintext (ZIP 326, “Fabricated same-address outputs and randomized note ciphertexts”). A dummy consumed note beside a real created note lies at the expanded receiver of that created note: in place of step (i) it takes the created note’s address and the key components of the key from which that address derives, and the Action’s spend-authorisation signature requires that key’s ask (Remark “Meaning of A10”, §9.2). A fresh key as in step (i) remains admissible when both sides of the Action are dummies at its address.

That the public data do not reveal which sides of an Action are real is part of Theorem 12.12 (§12.4).

Definition 4.12 (Bundle). The *bundle* of a pool in a transaction, which ZIP 229 calls the pool’s component, is the sequence of that pool’s Actions in the transaction together with the bundle-level data that later sections attach: one anchor (“Anchors”, §5.2), one balancing value (“The balancing value”, §8.2), one binding signature (“The binding signature”, §8.3), the flags (“Bundle flags”, §9.1) and one proof for all its Actions (“The Action circuit and the Halo 2 proof”, §9.3). A transaction carries at most one bundle per pool (ZIP 229, “Transaction Format”).

5 The note commitment tree

This section constructs the note commitment tree of a pool and its Merkle hash, defines the anchors, proves in Proposition 5.9 that a valid authentication path to an anchor exhibits a leaf of the anchored tree unless it yields a collision of the hash’s Sinsemilla instance or an input on which that instance returns $\perp$, and proves in Lemma 5.15 that the authentication path of a leaf is a function of public data.

5.1 The Merkle hash and the tree

Remark 5.1 (Requirement R2). Requirement R2 of “Requirements on a shielded payment” (§1.4) asks the consumer of a note of non-zero value to show that the note was previously committed, without identifying it. In the Orchard protocol the claim to be shown is that the extracted commitment cmx of the note (Definition 4.4) is a leaf of the note commitment tree of the note’s pool, and the leaf is not to be disclosed. Membership of a private leaf under a public root is provable in zero knowledge, with the root public and the leaf, its position and its authentication path private (*Crypto Guide*, §“Privacy-preserving membership via zero-knowledge paths”, Theorem “Privacy-preserving membership”). We construct the tree, its roots, called anchors, and its authentication paths; the relation proved for each Action is the Action statement, defined in “The Action statement” (§9.2).

Definition 5.2 (Note commitment tree). Let

$$\text{MerkleDepth}^{\text{Orchard}} := 32, \quad \text{Uncommitted}^{\text{Orchard}} := 2 \in \mathbb{F}_{p_{\text{Pallas}}}$$

(protocol specification, §“Constants”). Each pool of the Orchard protocol has its own *note commitment tree*, an append-only Merkle tree (*Crypto Guide*, §“Binary Merkle trees” and §“Append-only and incrementally updatable trees”) of fixed depth $\text{MerkleDepth}^{\text{Orchard}} = 32$, hence with 2^{32} leaf positions. Its *layers* are numbered from layer 0, the root, to layer 32, the leaves; layer b has 2^b nodes $M_i^b \in \mathbb{F}_{p_{\text{Pallas}}}$, $0 \leq i < 2^b$.

1. *Appending*. Each Action of the pool in a transaction publishes the extracted commitment cmx of its created note (§4.4). When the transaction enters the chain, each such cmx is appended to the pool’s tree: it occupies the leaf M_i^{32} for the least unused index i , and i is the *note position* pos of that commitment. Every pool’s tree has no appended leaf before the first block of the chain.
2. *Leaves*. After n appends, the leaf M_i^{32} is the value appended at position i for $0 \leq i < n$, and $M_i^{32} := \text{Uncommitted}^{\text{Orchard}} = 2$ for $n \leq i < 2^{32}$.
3. *Internal nodes*. For $0 \leq b < 32$ and $0 \leq i < 2^b$,

$$M_i^b := \text{MerkleCRH}(b, M_{2i}^{b+1}, M_{2i+1}^{b+1}),$$

with MerkleCRH the Merkle hash constructed below; the *root* is M_0^0 .

(Protocol specification, §“Note Commitment Trees”, §“Merkle Path Validity”, §“Note Commitments” and §“Transactions and Treestates”).

Construction 5.3 (Merkle hash). Let $D := \text{z.cash:Orchard-MerkleCRH}$. The Merkle hash

$$\text{MerkleCRH}: \{0, \dots, 31\} \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{F}_{p_{\text{Pallas}}} \rightarrow \mathbb{F}_{p_{\text{Pallas}}}$$

is

$$\text{MerkleCRH}(l, \text{left}, \text{right}) := \begin{cases} 0 & \text{if } \eta = \perp, \\ \eta & \text{otherwise,} \end{cases}$$

where

$$\eta := \text{SinsemillaHash}(D, \text{LE}_{10}(31 - l) \parallel \text{LE}_{255}(\text{left}) \parallel \text{LE}_{255}(\text{right})) \in \mathbb{F}_{p_{\text{Pallas}}} \cup \{\perp\},$$

with SinsemillaHash the extracted Sinsemilla hash of §2.4 and field elements encoded through their integer representatives. The protocol specification names the map $\text{MerkleCRH}^{\text{Orchard}}$ and the width 255 $\text{MerkleHashLength}^{\text{Orchard}}$ (§“MerkleCRH^{Orchard} Hash Function” and §“Constants”).

The prefix $\text{LE}_{10}(31 - l) = \text{LE}_{10}(\text{MerkleDepth}^{\text{Orchard}} - 1 - l)$ is defined because $31 < 2^{10}$. It encodes 0 for the combination of two leaves into a node of layer $l = 31$, and 31 for the combination that yields the root, $l = 0$. Every message has the single length $10 + 255 + 255 = 520$ bits, 52 chunks of $k = 10$ bits, within the bound of $c = 253$ chunks of §2.4 (Table 4). The encoding LE_{255} is injective on $\mathbb{F}_{p_{\text{Pallas}}}$ because $p_{\text{Pallas}} < 2^{255}$ (§2.1), so for a fixed layer distinct argument pairs give distinct messages. The protocol specification requires that SinsemillaHash be collision resistant and that no input of 520 bits yielding $\perp$ can feasibly be found (§“MerkleCRH^{Orchard} Hash Function”, security requirements). For the domain D and 520-bit messages these are parts (i) and (iii) of Proposition 2.24, under Assumptions 2.22 and 2.8.

Remark 5.4 (Nodes as field elements). Because MerkleCRH ends in $\text{Extract}_{\mathbb{P}}$, each layer’s output has the type of its inputs, and one hash composes from the leaves to the root. Proofs of the Action statement, defined in “The Action statement” (§9.2), are Halo 2 proofs with the Vesta curve (protocol specification, §“Zero-Knowledge Proving System”), whose scalar field is $\mathbb{F}_{p_{\text{Pallas}}}$ (*Math Guide*, §“Base fields, scalar fields, and the Pasta cycle”). A node that is one element of $\mathbb{F}_{p_{\text{Pallas}}}$ is one value of that proof’s arithmetic, whereas a point-valued node would carry two coordinates and a curve-equation condition at each of the 32 layers.

Remark 5.5 (Dropping the y -coordinate). A node serves only as a digest, never to recover a point, and $\text{Extract}_{\mathbb{P}}(P) = \text{Extract}_{\mathbb{P}}(P')$ only if $P' = P$ or $P' = -P$ (Lemma 2.4). A collision of $\text{SinsemillaHash}(D, \cdot)$ on 520-bit messages is therefore either a collision of the underlying hash to a point or a pair of messages whose hash points are opposite. Proposition 2.24(i) excludes both, and part (iii) excludes a $\perp$ output, under Assumptions 2.22 and 2.8.

Remark 5.6 (Layer separation). The prefix $\text{LE}_{10}(31 - l)$ separates the inputs of different layers (protocol specification, §“MerkleCRH^{Orchard} Hash Function”, note). The 32 layer functions $\text{MerkleCRH}(l, \cdot, \cdot)$ evaluate one instance $\text{SinsemillaHash}(D, \cdot)$ on message sets with distinct prefixes. Every collision that the proof of Proposition 5.9 extracts therefore lies between two messages with the same prefix, and a node computed at one layer is accepted at another only through such a collision or a $\perp$ input (*Crypto Guide*, §“Layer and domain separation”, Proposition “Separation forces within-domain collisions”).

Lemma 5.7 (Uncommitted leaves). *No point P of the Pallas group satisfies $\text{Extract}_{\mathbb{P}}(P) = 2$. Consequently $\text{Uncommitted}^{\text{Orchard}} = 2$ equals the extracted commitment $\text{cmx} = \text{Extract}_{\mathbb{P}}^{\perp}(\text{cm})$ of no note, and a leaf position holding 2 holds the commitment of no note.*

Proof. By definition $\text{Extract}_{\mathbb{P}}(\mathcal{O}) = 0 \neq 2$ in $\mathbb{F}_{p_{\text{Pallas}}}$. A point $(x, y) \neq \mathcal{O}$ with $x = 2$ on the curve $y^2 = x^3 + 5$ over $\mathbb{F}_{p_{\text{Pallas}}}$ would satisfy $y^2 = 2^3 + 5 = 13$. By Euler’s criterion $13^{(p_{\text{Pallas}}-1)/2} \equiv -1$

(mod p_{Pallas}) (*Math Guide*, §“Quadratic residues and the Euler criterion”, Example “13 is a non-square in both Pasta fields”), so 13 is a non-square modulo p_{Pallas} and no such y exists. Only the base-field modulus p_{Pallas} enters. For the consequence, cmx is $\perp$ when $\text{cm} = \perp$ and otherwise Extract_p of a point, so it is not 2. The protocol specification states the consequence as the theorem “Uncommitted^{Orchard} is not in the range of NoteCommit^{Orchard}” (§“Sinsemilla commitments”). $\square$

The protocol specification, §“Note Commitment Trees”, and ZIP 258, “Changes to the Protocol Specification”, state the capacity rule: a block must not add Ironwood-pool note commitments that would make the Ironwood-pool note commitment tree exceed its capacity of $2^{\text{MerkleDepth}^{\text{Orchard}}} = 2^{32}$ leaves; the same rule binds the Orchard-pool tree. The Ironwood-pool tree is a tree of its own, built by the construction above with the same depth, the same value Uncommitted^{Orchard} at unused positions and the same hash; only the values cmx published by Ironwood-pool Actions are appended to it.

5.2 Anchors

Definition 5.8 (Treestate and anchor). A pool’s *treestate* comprises its note commitment tree and its nullifier set, constructed in “Nullifier sets” (§6.2). Treestates are chained: the input treestate of the first block is empty; the input treestate of the first transaction of a block is the final treestate of the preceding block, and that of each later transaction the output treestate of the preceding transaction; the *final treestate* of a block is the output treestate of its last transaction (protocol specification, §“Transactions and Treestates”). The *anchor* of a block in a pool is the root M_0^0 of the pool’s tree in the block’s final treestate, an element of $\mathbb{F}_{p_{\text{Pallas}}}$; every block has exactly one anchor per pool. A bundle carries one anchor field, `anchorIronwood` in the Ironwood pool and `anchorOrchard` in the Orchard pool, shared by all its Actions; an Action carries no anchor of its own (“Action descriptions and bundles”, §11.1; protocol specification, §“Action Transfers and their Descriptions”).

The protocol specification, §“Action Transfers and their Descriptions”, and ZIP 258, “Changes to the Protocol Specification”, state the anchor rule: whenever a transaction has Ironwood-pool Actions, its field `anchorIronwood` must refer to some earlier block’s final Ironwood-pool treestate, that is, equal the Ironwood-pool anchor of a block that precedes the transaction’s block in the chain. The field `anchorOrchard` is bound likewise to the Orchard pool. The rule sets no bound on how far back that block lies.

Proposition 5.9 (Membership soundness). *Let rt be the anchor of a block in a pool, n the number of leaves appended to the pool’s tree through that block, and $L_0, \dots, L_{2^{32}-1}$ the leaves of the tree that rt roots, so that $L_i = \text{Uncommitted}^{\text{Orchard}}$ for $i \geq n$. For $v, s_0, \dots, s_{31} \in \mathbb{F}_{p_{\text{Pallas}}}$ and $\text{pos} = \sum_{t=0}^{31} b_t 2^t$ with $b_t \in \{0, 1\}$, let*

$$a_0 := v, \quad a_{t+1} := \begin{cases} \text{MerkleCRH}(31 - t, a_t, s_t) & \text{if } b_t = 0, \\ \text{MerkleCRH}(31 - t, s_t, a_t) & \text{if } b_t = 1 \end{cases} \quad (0 \leq t < 32). \quad (2)$$

The triple $(v, \text{pos}, (s_0, \dots, s_{31}))$ is a valid path to rt if $a_{32} = rt$. This is the path verification of the Crypto Guide (§“Authentication paths and membership proofs”, Definition “Path verification”),

with level- t compression $\text{MerkleCRH}(31 - t, \cdot, \cdot)$ and the bits of pos as direction bits; “Authentication paths” (§5.3) constructs the path of an appended leaf.

- (i) From a valid path to rt with $v \neq L_{\text{pos}}$ and the leaves $L_0, \dots, L_{n-1}$ one computes efficiently either two distinct 520-bit messages with a common 10-bit prefix and equal values of $\text{SinsemillaHash}(D, \cdot)$, for $D = \text{z.cash:Orchard-MerkleCRH}$, or a 520-bit message on which $\text{SinsemillaHash}(D, \cdot)$ returns $\perp$.
- (ii) Under Assumptions 2.22 and 2.8, an efficient adversary, even one that chooses the appended leaves, outputs a valid path to rt with $v \neq L_{\text{pos}}$ with at most negligible probability.
- (iii) In particular, if $v = \text{Extract}_P(P)$ for a point P of the Pallas group, as the extracted commitment $\text{cmx} \neq \perp$ of every note is, then, except with negligible probability, $\text{pos} < n$ and v is the value appended at position pos , in the anchoring block or earlier.

Proof. The argument is the *Crypto Guide*’s Theorem “Soundness of Merkle membership proofs” (§“Soundness: forging a path implies a collision”), with layer-dependent compressions.

(i) Let $j_t := \lfloor \text{pos}/2^t \rfloor$, and write $j \oplus 1$ for the integer j with its lowest bit complemented. The genuine chain is $\bar{a}_t := M_{j_t}^{32-t}$ for $0 \leq t \leq 32$, with genuine siblings $\bar{s}_t := M_{j_t \oplus 1}^{32-t}$ for $0 \leq t < 32$. Since $j_t = 2j_{t+1} + b_t$, the node $\bar{a}_{t+1} = M_{j_{t+1}}^{31-t}$ is $\text{MerkleCRH}(31 - t, \cdot, \cdot)$ of its two children, of which $\bar{a}_t$ is the left one if $b_t = 0$ and the right one if $b_t = 1$, the other being $\bar{s}_t$. The genuine chain therefore satisfies (2) with $\bar{s}_t$ in place of s_t , and $\bar{a}_0 = L_{\text{pos}}$, $\bar{a}_{32} = M_0^0 = rt$ (*Crypto Guide*, §“Authentication paths and membership proofs”, Proposition “Completeness”, whose induction this is). The fold on the given path has $a_0 = v \neq \bar{a}_0$ and $a_{32} = rt = \bar{a}_{32}$. Let t^* be the least $t \geq 1$ with $a_t = \bar{a}_t$; it exists, and $a_{t^*-1} \neq \bar{a}_{t^*-1}$ by minimality. At level $t^* - 1$ both chains evaluate $\text{MerkleCRH}(32 - t^*, \cdot, \cdot)$ with the running value on the side fixed by b_{t^*-1} : the first on a_{t^*-1} and s_{t^*-1} , the second on $\bar{a}_{t^*-1}$ and $\bar{s}_{t^*-1}$. The two 520-bit messages hashed there share the prefix $\text{LE}_{10}(31 - (32 - t^*)) = \text{LE}_{10}(t^* - 1)$, the layer tag of the *Crypto Guide*’s Proposition “Separation forces within-domain collisions” (§“Layer and domain separation”). They differ in the half that encodes the running value, LE_{255} being injective on $\mathbb{F}_{p_{\text{Pallas}}}$, and their values under MerkleCRH , $a_{t^*} = \bar{a}_{t^*}$, agree. If $\text{SinsemillaHash}(D, \cdot)$ returns $\perp$ on either message, that message is the second output. Otherwise both values of MerkleCRH are values of $\text{SinsemillaHash}(D, \cdot)$, and the pair is the first output. The genuine chain is computed from $L_0, \dots, L_{n-1}$ by the recurrence of the tree. A node whose leaves all lie at positions at least n depends only on its layer, so the computation takes a number of evaluations of MerkleCRH linear in $n + 32$; the fold takes 32 more.

(ii) The extraction of (i) is deterministic and defined for every list of leaves, so it applies to leaves that an adversary chose. An adversary that outputs a valid path with $v \neq L_{\text{pos}}$ with probability ϵ therefore yields an algorithm that outputs, with probability ϵ , a collision of $\text{SinsemillaHash}(D, \cdot)$ on 520-bit messages or a 520-bit message with output $\perp$. Equal values of SinsemillaHash other than $\perp$ come from hash points that are equal on distinct messages or opposite (Lemma 2.4). Proposition 2.24, parts (i) and (iii), excludes both outputs except with negligible probability, under Assumptions 2.22 and 2.8; hence ϵ is negligible.

(iii) If $\text{pos} \geq n$, then $L_{\text{pos}} = 2$, and $v \neq 2$ by Lemma 5.7, so the path is one with $v \neq L_{\text{pos}}$,

which (ii) excludes except with negligible probability. Otherwise $\text{pos} < n$, and outside the same event $v = L_{\text{pos}}$, the value appended at position pos ; the n leaves of the tree that rt roots are those appended through the anchoring block. $\square$

Remark 5.10 (Old anchors). Appending leaves changes the final treestates of later blocks and of no earlier one. Once a block B is in the chain, its anchor therefore roots one fixed tree: a valid path to it stays valid after later appends, the anchor stays admissible under the anchor rule while B remains in the chain, and Proposition 5.9 applies to it at any later time. A valid path to the anchor of B for an extracted note commitment shows that the commitment was appended in B or earlier, which is what R2 requires. Excluding a second consumption of a note is not the task of the tree, which is append-only (protocol specification, §“Note Commitment Trees”), but that of the nullifier, constructed in “Nullifiers” (§6).

Definition 5.11 (Reorganisation). A *reorganisation* is the replacement of the chain’s most recent blocks by a competing branch.

With the height of a block as in §1.1, the *tip* is the last block of the chain. A reorganisation that removes the block whose final treestate a bundle’s anchor refers to leaves the transaction in violation of the anchor rule on the new chain, unless a block of the new chain that precedes the transaction’s block has the same anchor; to be included in the new chain, the transaction must then be rebuilt against an anchor of that chain. The block d blocks below a tip of height H , the block at height $H - d$, is removed only by a reorganisation that replaces at least $d + 1$ blocks, since a reorganisation that replaces k blocks removes the blocks at heights $H - k + 1$ to H (ZIP 315, “Rationale for anchor selection”; ZIP 218, “Anchor selection depth”).

The *block target spacing*, the intended mean interval between consecutive blocks, is 25 seconds (ZIP 218, “Block target spacing”).

Remark 5.12 (Recommended anchor depth). ZIP 315 recommends that a transaction built when the tip is at height H use as its anchor the final treestate of the block at height $H - 3$, three blocks below the tip (ZIP 315, “Anchor selection”); ZIP 218 recommends the same depth of 3 blocks (“Anchor selection depth”). At the 25-second target spacing this depth is $3 \times 25 = 75$ seconds. The depth is a recommendation to the builder of a transaction, not a consensus rule: the anchor rule admits the final treestate of any earlier block.

Remark 5.13 (Anchor choice as public data). The anchor is published with the bundle. Call the set $\{0, \dots, n - 1\}$ of positions of the leaves appended to the tree that an anchor roots the *candidate set* of the anchor; by Proposition 5.9(iii), a valid path to the anchor for an extracted note commitment has its position in this set, except with negligible probability. A later anchor roots a tree that contains every appended leaf of an earlier one, so its candidate set is at least as large. An anchor at a depth other than the one that other builders of transactions use distinguishes its builder. These two observations are not ZIP 315’s stated rationale for a fixed depth, which is that too small a depth risks invalidation by a reorganisation, that too large a depth prevents recently received notes from being spent, and that a depth varying with the notes spent leaks information about them (ZIP 315, “Rationale for anchor selection”).

5.3 Authentication paths

Construction 5.14 (Authentication path). Let the pool's tree hold n leaves, namely the tree that the anchor rt of a block roots, for a block at or after the one that appended the note's commitment, so that the note position satisfies $\text{pos} < n$. Write $\text{pos} = \sum_{t=0}^{31} b_t 2^t$ with $b_t \in \{0, 1\}$, and $j_t := \lfloor \text{pos}/2^t \rfloor$. Levels $t = 0, \dots, 31$ count from the leaves: the *level- t route node* is $M_{j_t}^{32-t}$, a left child if $b_t = 0$ and a right child if $b_t = 1$. The *authentication path* of pos is

$$(s_0, \dots, s_{31}) \in \mathbb{F}_{p_{\text{Pallas}}}^{32}, \quad s_t := M_{j_t \oplus 1}^{32-t},$$

the sibling of the level- t route node in the tree of size n , with $j \oplus 1$ the integer j with its lowest bit complemented. Verification of $(v, \text{pos}, (s_0, \dots, s_{31}))$ against rt is the fold (2): the level- t step evaluates $\text{MerkleCRH}(31 - t, \cdot, \cdot)$, whose message carries the prefix $\text{LE}_{10}(t)$, with the running value on the left if $b_t = 0$ and on the right if $b_t = 1$; verification accepts if and only if $a_{32} = rt$, that is, if the triple is a valid path to rt in the sense of Proposition 5.9. In the protocol specification, §“Merkle Path Validity”, the entry s_t is the node $M_{\text{MerkleSibling}(b_t, \text{pos})}^b$ with $b = 32 - t$.

The genuine path of an appended leaf is accepted: the fold on $(L_{\text{pos}}, \text{pos}, (s_0, \dots, s_{31}))$ reproduces the route nodes, $a_t = M_{j_t}^{32-t}$ for every t , and ends at $M_0^0 = rt$ (*Crypto Guide*, §“Authentication paths and membership proofs”, Proposition “Completeness”, whose induction applies with the level- t compression $\text{MerkleCRH}(31 - t, \cdot, \cdot)$, as in the proof of Proposition 5.9). A note's commitment therefore has a valid path to the anchor of every block from the one that appended it onward.

Lemma 5.15 (Authentication paths from public data). *Let the tree hold n leaves, let $\text{pos} < n$, and let*

$$\begin{aligned} \text{Empty}_0 &:= \text{Uncommitted}^{\text{Orchard}} = 2, \\ \text{Empty}_{t+1} &:= \text{MerkleCRH}(31 - t, \text{Empty}_t, \text{Empty}_t) \quad (0 \leq t < 32). \end{aligned}$$

For each level t the entry s_t of the authentication path of pos is as follows.

- (i) *If $b_t = 1$, then s_t is the root of the complete subtree over the positions $[(j_t - 1) 2^t, j_t 2^t]$, all below pos ; it is a function of the leaves at those positions alone, and the same in every tree of size greater than pos .*
- (ii) *If $b_t = 0$, then s_t is the root of the subtree over the positions $[(j_t + 1) 2^t, (j_t + 2) 2^t]$, a function of the leaves appended in that range, with $\text{Uncommitted}^{\text{Orchard}}$ at its unused positions; it equals Empty_t when $(j_t + 1) 2^t \geq n$.*

Hence the authentication path is a deterministic function of pos and of the first n leaves, which are the values cmx published by the pool's Actions in the chain through the anchoring block. A holder of the note who has these leaves computes the path without communicating pos , and verification needs, besides the path, only the public anchor.

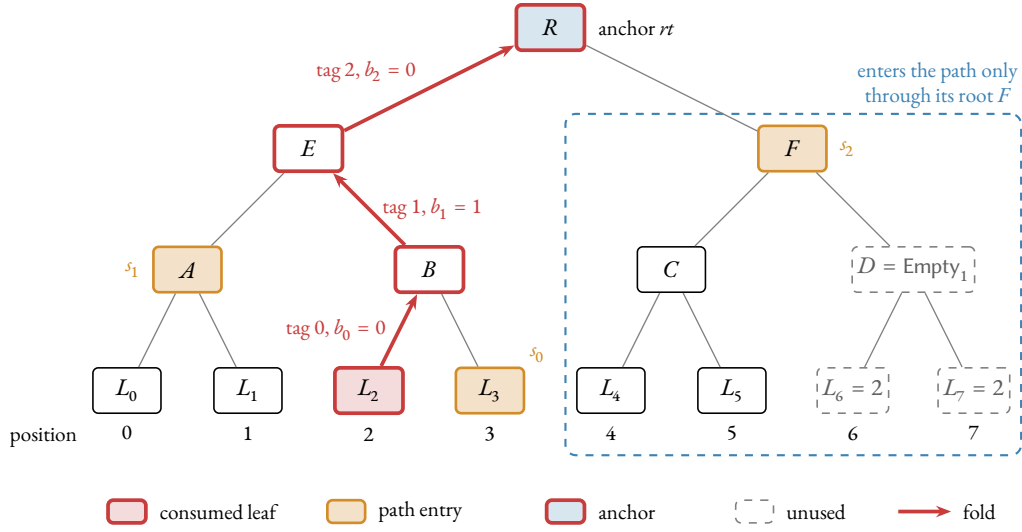


Figure 2. A depth-3 instance of the note commitment tree of §5.1; the protocol’s tree has depth 32. Leaves L_0 to L_5 are appended ($n = 6$); positions 6 and 7 hold $\text{Uncommitted}_{\text{Orchard}} = 2$, so $D = \text{Empty}_1$. The consumed note’s leaf is L_2 , at position 2 with bits $b_0 = 0, b_1 = 1, b_2 = 0$. Its authentication path is $(s_0, s_1, s_2) = (L_3, A, F)$, each entry the sibling of the route node at its level. The bold fold $L_2 \rightarrow B \rightarrow E \rightarrow R$ evaluates MerkleCRH at each level with the prefix $\text{LE}_{10}(t)$, shown as tag t , and the running value on the side given by b_t ; it ends at the root R , the anchor. By Lemma 5.15, the entry A , left of the route, is the root of a complete subtree and the same at every later anchor; the entry F , right of the route and the root over positions 4 to 7, is a function of the leaves appended at the anchor, with 2 at unused positions, and changes when L_6 or L_7 is appended; the entry L_3 , right of the route but already appended, is fixed at every later anchor. The boxed subtree enters the path only through its root F . Every entry is computed from pos and the first n leaves.

Proof. By the recurrence of the tree, the node M_i^b is a function of the leaves at positions $[i \cdot 2^{32-b}, (i+1) \cdot 2^{32-b})$, by induction on $32 - b$. Apply this with $b = 32 - t$ and $i = j_t \oplus 1$, which equals $j_t - 1$ when $b_t = 1$ and $j_t + 1$ when $b_t = 0$, since b_t is the parity of j_t . When $b_t = 1$ the range ends at $j_t \cdot 2^t \leq \text{pos} < n$, so every position in it is appended, and its leaf is fixed once appended because the tree is append-only. When the range starts at or beyond n , every leaf in it is 2, and its root is Empty_t by induction on t : a level- $(t+1)$ node over an unused range is $\text{MerkleCRH}(31 - t, \cdot, \cdot)$ of two level- t nodes over unused ranges. Positions below n hold the appended values cmx, published in the chain. $\square$

Figure 2 shows the construction and Lemma 5.15 on a tree of depth 3.

Remark 5.16 (Scope of hiding). The anchor is public. The position, the consumed note’s cmx and its authentication path are not fields of the bundle, and the holder of the note computes the path locally (Lemma 5.15): obtaining the path of a particular position from another party would disclose pos to that party. The candidate set for the consumed note is the candidate set of the anchor (§5.2), the positions of the n appended leaves of the anchored tree. Leaves that an observer knows by other means to be commitments of dummy created notes (§4.4) or of notes already consumed, and other data of the transaction, reduce the effective set, so n alone does not measure it. What an Action reveals is stated in Theorem 12.12 (“Privacy”, §12.4).

6 Nullifiers

This section constructs the nullifier of a note, the nullifier sets that consensus maintains per pool, and the rule that makes the nullifier of each consumed note the element ρ of the note created in the same Action. It proves two properties of the construction: uniqueness, from discrete logarithms on Pallas with GroupHash modelled as a random oracle; and, for keys generated with `use_qsk = false`, unlinkability, from these two assumptions, the pseudorandomness of Poseidon and of PRF expansion, and the joint hiding of the viewing keys. It also states what each key tier can compute from nullifiers.

6.1 The nullifier

Requirement R3 of “Requirements on a shielded payment” (§1.4) excludes a second consumption of a note without a public list of consumed notes. Each note therefore determines a *nullifier*, a tag published when the note is consumed, and a verifier (§1.2) rejects a nullifier already in the nullifier set of its pool (§6.2). The construction must make the nullifier

- (a) *deterministic*: a function of the note and of the nullifier deriving key nk of the key that owns it, so that a second consumption publishes the same value; and
- (b) *unlinkable*: to a party without nk , the published value reveals nothing about the note’s commitment or address.

Construction 6.1 (Nullifier PRF). Let $f: \mathbb{F}_{\text{Pallas}}^3 \rightarrow \mathbb{F}_{\text{Pallas}}^3$ be the Poseidon permutation of width $t = 3$ over the Pallas base field, used as a sponge of rate 2 and capacity 1, with S-box $x \mapsto x^5$, 8 full and 56 partial rounds (*Crypto Guide*, §“Arithmetisation-friendly hashing: Poseidon”, Constructions “The sponge construction” and “The Poseidon permutation”, Example “The Orchard Poseidon instance”). Its round constants and MDS matrix are those of the protocol specification, §“PoseidonHash Function”, and are not reproduced. In constant-input-length mode the hash PoseidonHash: $\mathbb{F}_{\text{Pallas}} \times \mathbb{F}_{\text{Pallas}} \rightarrow \mathbb{F}_{\text{Pallas}}$ is

$$\text{PoseidonHash}(x, y) := \pi_1(f(x, y, 2^{65})),$$

where π_1 is the projection onto the first component; the capacity element 2^{65} encodes the input length 2, so no padding is needed (same citations). The *nullifier PRF* is

$$\text{PRF}^{\text{nfOrchard}}: \mathbb{F}_{\text{Pallas}} \times \mathbb{F}_{\text{Pallas}} \rightarrow \mathbb{F}_{\text{Pallas}}, \quad \text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho) := \text{PoseidonHash}(\text{nk}, \rho),$$

keyed by the nullifier deriving key $\text{nk} \in \mathbb{F}_{\text{Pallas}}$ of “The spending key and the spend-side secrets” (§3.1; protocol specification, §“Pseudo Random Functions”).

Assumption 6.2 (Poseidon is a PRF). For nk uniform on $\mathbb{F}_{\text{Pallas}}$, no efficient adversary with adaptive oracle access distinguishes $\rho \mapsto \text{PoseidonHash}(\text{nk}, \rho)$ from a uniformly random function $\mathbb{F}_{\text{Pallas}} \rightarrow \mathbb{F}_{\text{Pallas}}$, except with negligible advantage: this is the game of Definition “Pseudorandom function” (§2.3; *Crypto Guide*, §“Pseudorandom functions and permutations”) with key, input and output sets $\mathbb{F}_{\text{Pallas}}$. It is the security requirement that the protocol specification, §“Pseudo

Random Functions”, places on PoseidonHash keyed by its first argument. Its status is cryptanalytic: it is supported by the analysis of the Poseidon round function (protocol specification, §“PoseidonHash Function”, notes) and has no reduction to a standard problem (*Crypto Guide*, Example “The Orchard Poseidon instance”). Of the results of this section only Proposition 6.10 uses it; Proposition 6.8 uses no property of PoseidonHash, and the volume proves no unlinkability result for nullifiers without this assumption.

Definition 6.3 (Nullifier). Let

$$K^{\text{Orchard}} := \text{GroupHash}(z.\text{cash}:\text{Orchard}, k),$$

the nullifier base, a fixed generator of Table 2. For $nk, \rho, \psi \in \mathbb{F}_{p_{\text{Pallas}}}$ and a point $cm \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$,

$$\text{DeriveNullifier}_{nk}(\rho, \psi, cm) := \text{Extract}_p([s] K^{\text{Orchard}} + cm) \in \mathbb{F}_{p_{\text{Pallas}}},$$

where

$$s := (\text{PRF}_{nk}^{\text{nfOrchard}}(\rho) + \psi) \bmod p_{\text{Pallas}} \in \{0, \dots, p_{\text{Pallas}} - 1\} :$$

the sum of the two field elements is taken as its representative in $\{0, \dots, p_{\text{Pallas}} - 1\}$, and that integer multiplies K^{Orchard} . The *nullifier* of a note $n = (\text{addr}, v, \rho, \psi, \text{rcm})$ whose note commitment cm (Definition 4.4) is not $\perp$, under the nullifier deriving key nk of the key that owns addr , is

$$nf := \text{DeriveNullifier}_{nk}(\rho, \psi, cm)$$

(protocol specification, §“Computing ρ values and Nullifiers”).

Remark 6.4 (Reduction modulo p_{Pallas}). The protocol specification attaches two notes to the definition (§“Computing ρ values and Nullifiers”). First, the sum is reduced modulo p_{Pallas} intentionally, although the scalar field of the Pallas group is $\mathbb{F}_{p_{\text{Vesta}}}$. Since $p_{\text{Pallas}} < p_{\text{Vesta}}$ (§2.1), the representative in $\{0, \dots, p_{\text{Pallas}} - 1\}$ is a scalar without further reduction, by the injective reading $\mathbb{F}_{p_{\text{Pallas}}} \rightarrow \mathbb{F}_{p_{\text{Vesta}}}$ of §2.1, and the multiplier of K^{Orchard} ranges over p_{Pallas} of the p_{Vesta} scalars. Second, the inputs ρ and ψ must be those of the note committed to by cm .

Remark 6.5 (Determinism and masking). For a fixed nk the nullifier is a function of the fields ρ and ψ of the note and of its commitment cm , so a note has exactly one nullifier under the nk of its owner. That every accepted consumption of the note publishes this value is Lemma 12.5, proved in “Double-spend resistance” (§12.2) from the Action statement, defined in “The Action statement” (§9.2). The point $[s] K^{\text{Orchard}}$ is added to cm before extraction. For notes of one key with pairwise distinct ρ , the key generated with $\text{use_qsk} = \text{false}$, the multipliers are jointly pseudorandom to an efficient party that holds the incoming viewing key and ak^p but not nk , so the extracted values give it no usable information on the commitments or on the address; Proposition 6.10 states this exactly, under Assumption 6.2 and the further assumptions it names. The element ρ enters as the input of the PRF and ψ as an additive term of the multiplier. Both are fixed when the note is created: ρ by “Nullifier chaining” (§6.3), and ψ by the derivation of §4.3.

6.2 Nullifier sets

Each verifier maintains, for each shielded pool, a *nullifier set*, part of that pool’s treestate (“Anchors”, §5.2). Every Action publishes one nullifier, that of its consumed note, real or dummy (§4.4), and the nullifiers published by the Actions of valid transactions are inserted into the nullifier set of their pool (protocol specification, §“Nullifiers” and §“Nullifier Sets”). The consensus rule of §“Nullifier Sets” is that a nullifier must not repeat, either within a transaction or across transactions in a valid block chain; a nullifier is compared only with nullifiers of its own pool, so equal values in two pools do not repeat a nullifier (the specification treats the pools’ nullifiers as disjoint).

Both pools use the one derivation of Definition 6.3, with the same PRF and the same nullifier base: the nullifier construction belongs to the Orchard protocol, not to a pool (ZIP 229, Terminology). The Orchard pool and the Ironwood pool keep separate nullifier sets, as they keep separate note commitment trees, anchors and value pool balances (ZIP 229, Rationale, “Separate state”; protocol specification, §“Nullifier Sets”). The nullifier of an Ironwood-pool Action is inserted into, and checked against, the nullifier set of the Ironwood pool only.

Remark 6.6 (Pool locality). Equal nullifier values in the two pools do not violate the consensus rule, so every uniqueness property derived from it holds within one pool. In particular the uniqueness of the elements ρ proved in “Double-spend resistance” (Corollary 12.7) is uniqueness within a pool, not across the two pools.

6.3 Nullifier chaining

Construction 6.7 (Chaining rule). In every Action the sender sets the element ρ of the created note to the nullifier that the same Action publishes for its consumed note,

$$\rho^{\text{new}} := \text{nf}^{\text{old}} \in \mathbb{F}_{P_{\text{Pallas}}},$$

and encodes it as the 32-byte string $\text{I2LEOSP}_{256}(\rho^{\text{new}})$. When the Action consumes no real note, its consumed side is a dummy note (Definition “Dummy notes”, §4.4), and the nullifier of that dummy note is ρ^{new} (protocol specification, §“Sending Notes (Orchard)”, §“Dummy Notes (Orchard)” and §“Faerie Gold attack and fix”).

Every Action has a consumed side (Definition “Action”, §4.4), so every note that an Action creates is constructed with ρ equal to the nullifier of that Action’s consumed note. The leaves that enter a pool’s note commitment tree are the extracted commitments of notes created by that pool’s Actions (“The Merkle hash and the tree”, §5.1), so the rule covers every note whose extracted commitment enters the tree. The consumed dummy note is not created by an Action: its ρ is Extract_P of a uniformly random point of the Pallas group (§4.4).

The derivations of §4.3 take the chained ρ as input: with $\underline{\rho} := \text{I2LEOSP}_{256}(\rho)$,

$$\text{esk} = \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}([\text{0x04}] \parallel \underline{\rho})), \quad \psi = \text{ToBase}(\text{PRFexpand}_{\text{rseed}}([\text{0x09}] \parallel \underline{\rho})),$$

and rcm is derived from the string pre_rcm , which contains $\underline{\rho}$ and the encoding of ψ . Hence esk and ψ are functions of rseed and of the nullifier that the note’s own Action publishes, and rcm is

a function of these and of the remaining note fields.

A *Faerie Gold attack* is one in which a sender creates several notes to one recipient that share a nullifier, so that only one of them can be spent (protocol specification, §“Faerie Gold attack and fix”). What a verifier can rely on, the uniqueness of the elements ρ within a pool and resistance to the Faerie Gold attack, is proved in “Double-spend resistance” (Corollary 12.7) from the nullifier-set rule and the Action statement, defined in “The Action statement” (§9.2). This subsection states the rule that an honest sender follows and draws no conclusion about accepted notes from it.

6.4 Nullifier uniqueness and unlinkability

Proposition 6.8 (Nullifier uniqueness). *Under Assumptions 2.22 and 2.8, no efficient adversary outputs, except with negligible probability, two notes $n = (\text{addr}, v, \rho, \psi, \text{rcm})$ and $n' = (\text{addr}', v', \rho', \psi', \text{rcm}')$ and two nullifier deriving keys $\text{nk}, \text{nk}' \in \mathbb{F}_{p_{\text{Pallas}}}$, all of its choice, whose note commitments cm and cm' are not $\perp$ and satisfy $\text{cm} \neq \text{cm}'$ and*

$$\text{DeriveNullifier}_{\text{nk}}(\rho, \psi, \text{cm}) = \text{DeriveNullifier}_{\text{nk}'}(\rho', \psi', \text{cm}').$$

The adversary may hold every nullifier deriving key, and $\text{nk} = \text{nk}'$ and $\rho = \rho'$ are admitted. Since two distinct notes have distinct commitments except with negligible probability (Proposition 4.5(b)), the same holds for two distinct notes in place of two distinct commitments.

Proof. The adversary supplies both openings, so no knowledge extraction is used. Let its output satisfy the conditions, and write

$$\begin{aligned} K &:= K^{\text{Orchard}}, \\ Q &:= Q(\text{z.cash:Orchard-NoteCommit-M}), \\ H_D &:= \text{GroupHash}(\text{z.cash:Orchard-NoteCommit-r}, \epsilon) \end{aligned}$$

for the nullifier base and for the hash base and the blinding base of NoteCommit (Definition 4.4). A reduction that runs the adversary computes $s := (\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho) + \psi) \bmod p_{\text{Pallas}}$ and s' likewise; the pieces $m = (m_1, \dots, m_{109})$ and m' of the messages $\mathcal{M}(n)$ and $\mathcal{M}(n')$, which have 109 chunks of $k = 10$ bits (Table 4); and rcm, rcm' . Every coefficient below is therefore known to it, the PRF outputs included, and no property of PoseidonHash is used.

Equal nullifiers are equal extracted coordinates, so by Lemma 2.4

$$[s] K + \text{cm} = \theta([s'] K + \text{cm}') \quad \text{for some } \theta \in \{1, -1\}.$$

Since $\text{cm} \neq \perp$, its hash point is not $\perp$, and by the unrolled form (1) of the proof of Proposition 2.24,

$$\text{cm} = [2^{109}] Q + \sum_{j=0}^{2^k-1} [\chi_j(m)] S(j) + [\text{rcm}] H_D, \quad \chi_j(m) = \sum_{i=1}^{109} 2^{109-i} \delta_{m_i, j},$$

and likewise for cm' . Substituting gives, with coefficients in $\mathbb{F}_{p_{\text{Vesta}}}$ and s, s' read as scalars,

$$[s - \theta s'] K + [(1 - \theta) 2^{109}] Q + \sum_j [\chi_j(m) - \theta \chi_j(m')] S(j) + [\text{rcm} - \theta \text{rcm}'] H_D = \mathcal{O}. \quad (3)$$

The relation is non-trivial in each case.

- If $\theta = -1$, the coefficient of Q is 2^{110} , non-zero modulo p_{Vesta} because $0 < 2^{110} < 2^{253} \leq (p_{\text{Vesta}} - 1)/2$.
- If $\theta = 1$ and $M(n) \neq M(n')$, then $m \neq m'$, and $\chi(m) \neq \chi(m')$ modulo p_{Vesta} by step (1) of the proof of Proposition 2.24 ($109 \leq c = 253$); some coefficient of an $S(j)$ is non-zero.
- If $\theta = 1$ and $M(n) = M(n')$, then $\text{rcm} \neq \text{rcm}'$, since equal messages and equal trapdoors give $\text{cm} = \text{cm}'$; the coefficient of H_D is non-zero.

The points $K, Q, S(0), \dots, S(2^k - 1)$ and H_D are values of GroupHash on pairwise distinct inputs (Table 2). The reduction outputs these inputs with the coefficients of (3), a non-trivial relation with known coefficients. By the lemma on relations among fixed generators (§2.4), the reduction behind Proposition 2.24, this happens with negligible probability under Assumptions 2.22 and 2.8. The specification records that its Sinsemilla collision argument extends to additional terms with independent bases for this purpose (protocol specification, §“Sinsemilla Hash Function”, Security argument, note; §“Faerie Gold attack and fix”).

For distinct notes whose commitments are not $\perp$, equal commitments have negligible probability by Proposition 4.5(b), and distinct commitments are the case above. $\square$

Remark 6.9 (Scope of Proposition 6.8). The proposition uses no pseudorandomness of PoseidonHash, and none could be used: the adversary holds the PRF keys, and the reduction knows every coefficient. It does not require distinct elements ρ , which the specification’s security requirement assumes (§“Computing ρ values and Nullifiers”). Two notes with equal commitments are equal except with negligible probability (Proposition 4.5(b)) and so have equal nullifiers under one nk. Excluding equal commitments among the notes of a pool is the role of the chaining rule of §6.3, as Corollary 12.7 proves.

Proposition 6.10 (Nullifier unlinkability). *Let a key be generated with $\text{use_qsk} = \text{false}$ from a uniform spending key, as in §3.1 and §3.2, with nullifier deriving key nk. An adversary receives the incoming viewing key (dk, ivk) and the point ak^P of the key, but not nk. It then submits, adaptively, notes $n_i = (\text{addr}_i, v_i, \rho_i, \psi_i, \text{rcm}_i)$, $1 \leq i \leq q$, of its choice, with pairwise distinct ρ_i and note commitments $\text{cm}_i \neq \perp$, and after each submission receives an answer:*

- *in the real game, $\text{nf}_i := \text{DeriveNullifier}_{\text{nk}}(\rho_i, \psi_i, \text{cm}_i)$;*
- *in the ideal game, $\text{Extract}_P([u_i] K^{\text{Orchard}} + \text{cm}_i)$ for u_i uniform on $\mathbb{F}_{p_{\text{Vesta}}}$, independent of each other and of all else.*

It outputs a bit. Let $\delta := (p_{\text{Vesta}} - p_{\text{Pallas}})/p_{\text{Vesta}}$, which is below 2^{-167} , the statistical distance between the uniform distribution on $\{0, \dots, p_{\text{Pallas}} - 1\}$, read in $\mathbb{F}_{p_{\text{Vesta}}}$, and the uniform distribution on $\mathbb{F}_{p_{\text{Vesta}}}$. Then the difference of the probabilities that it outputs 1 in the two games is at most

$$2(\epsilon_{\text{key}} + \epsilon_{\text{expand}} + 3 \cdot 2^{-257} + \epsilon_{\text{vk}}) + \epsilon_{\text{Pos}} + q \delta,$$

where ϵ_{expand} , ϵ_{vk} and ϵ_{Pos} are the largest advantages against Assumptions 2.12 (through Lemma 2.15), 3.13 and 6.2 of the algorithms built from the adversary in the proof, and ϵ_{key}

is the probability that a single draw of key generation from a uniform spending key is rejected plus the probability that $\text{ask} = 0$ or $\hat{M} = \perp$ for independent uniform (ask, nk) , with $\hat{M}$ the hash point of $\text{Commit}^{\text{ivk}}$ on (ak, nk) (§3.2). The term ϵ_{key} is negligible under Assumptions 2.12, 2.22 and 2.8 (Lemma “Rejection in key generation”, §3.2); the bound is therefore negligible for every efficient adversary, whose number q of queries is polynomial, under these assumptions and Assumptions 3.13 and 6.2. When $K^{\text{Orchard}} \neq \mathcal{O}$, the point $[u_i] K^{\text{Orchard}} + \text{cm}_i$ is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ whatever cm_i is, so the ideal answers are independent of the notes and of each other; in the model of Assumption 2.8, $K^{\text{Orchard}} = \mathcal{O}$ has probability $1/p_{\text{Vesta}}$.

Proof. Write $K := K^{\text{Orchard}}$. The proof passes from the real game H_0 through hybrid games $H_1, \dots, H_5$; each hop changes the probability of the output 1 by at most the amount stated (Crypto Guide, §“The hybrid argument”, Lemma “Hybrid lemma”), and each reduction runs the adversary and the rest of the game.

Hybrid H_1 : idealised key. The key is generated by a single draw from a uniform sk , without rejection, at the cost of the probability that the draw is rejected (Lemma “Rejection in key generation”, §3.2). Then $(\text{ask}, \text{nk}, r_{\text{ivk}})$ is replaced by independent uniform elements of $\mathbb{F}_{p_{\text{Vesta}}}$, $\mathbb{F}_{p_{\text{Pallas}}}$ and $\mathbb{F}_{p_{\text{Vesta}}}$, from which the other components are derived as in §3.1 and §3.2. By Lemma 2.15, this costs at most ϵ_{expand} plus the three reduction distances, each below 2^{-257} (§2.3): the three inputs of $\text{PRF}_{\text{expand}}^{\text{sk}}$ have distinct lead bytes, and the game is an efficient function of the triple, which derives ak^{p} , ivk and dk , runs the adversary and answers each query with the nullifier under nk .

Hybrid H_2 : joint hiding. When $\text{ask} \neq 0$, the pair (ivk, dk) is replaced by $(\text{Commit}_{r_{\text{ivk}}}^{\text{ivk}}(\text{ak}, \text{nk}), u)$, for r_{ivk} uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and u a uniform 32-byte string, independent of all else. The cost is at most ϵ_{vk} . The reduction for Assumption 3.13 draws ask and nk itself; for $\text{ask} \neq 0$ the pair (ak, nk) lies in the domain of the assumption (§3.1). It receives a triple for that pair, passes its first two components to the adversary as ivk and dk , together with ak^{p} , and answers every query with the nullifier under its own nk . Giving ak^{p} costs nothing, because after H_1 the scalar ask is independent of $(\text{nk}, r_{\text{ivk}})$ and the assumption holds for every fixed (ak, nk) .

Hybrid H_3 : independent viewing keys. The pair (ivk, dk) is replaced, whatever ask is, by $(\text{Extract}_{\text{p}}(W), u)$ for a uniform point W and a uniform 32-byte string u , independent of all else. When $\text{ask} \neq 0$ and the hash point $\hat{M}$ of the message $\text{LE}_{255}(\text{ak}) \parallel \text{LE}_{255}(\text{nk})$ is not $\perp$, the commitment point $\hat{M} + [r_{\text{ivk}}'] H_D$ is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ by Lemma 2.26(i), its blinding base H_D not being $\mathcal{O}$ (§3.2), so the two games are identically distributed conditioned on that event, which depends on (ask, nk) only. The cost is at most the probability that $\text{ask} = 0$ or $\hat{M} = \perp$, the remaining part of ϵ_{key} . In H_3 the key nk enters the game only through the answers: dk , ivk and ak^{p} are independent of it.

Hybrid H_4 : random function. The function $\text{PRF}_{\text{nk}}^{\text{nfOrchard}}$ is replaced by a uniformly random function $F: \mathbb{F}_{p_{\text{Pallas}}} \rightarrow \mathbb{F}_{p_{\text{Pallas}}}$. The cost is at most ϵ_{pos} : the reduction for Assumption 6.2 runs H_3 with its oracle in place of $\text{PRF}_{\text{nk}}^{\text{nfOrchard}}$, since nk , uniform on $\mathbb{F}_{p_{\text{Pallas}}}$, is used nowhere else. In H_4 the i -th answer is $\text{Extract}_{\text{p}}([s_i] K + \text{cm}_i)$ with $s_i := (F(\rho_i) + \psi_i) \bmod p_{\text{Pallas}}$. The ρ_i are pairwise distinct, so, with F sampled lazily, $F(\rho_i)$ is uniform on $\mathbb{F}_{p_{\text{Pallas}}}$ and independent of everything the adversary has received when it submits n_i . Translation by ψ_i is a bijection modulo p_{Pallas} , so s_i is uniform on $\{0, \dots, p_{\text{Pallas}} - 1\}$ and independent of that view, whatever ψ_i is.

Hybrid H_5 : uniform scalars. For $i = 1, \dots, q$ in turn, the scalar s_i , read in $\mathbb{F}_{p_{\text{Vesta}}}$ (§2.1), is replaced

by u_i uniform on $\mathbb{F}_{p_{\text{Vesta}}}$. The two distributions differ by $1/p_{\text{Pallas}} - 1/p_{\text{Vesta}}$ on each of the p_{Pallas} residues and by $1/p_{\text{Vesta}}$ on each of the other $p_{\text{Vesta}} - p_{\text{Pallas}}$ scalars, so their statistical distance is

$$\frac{1}{2} \left(p_{\text{Pallas}} \left(\frac{1}{p_{\text{Pallas}}} - \frac{1}{p_{\text{Vesta}}} \right) + \frac{p_{\text{Vesta}} - p_{\text{Pallas}}}{p_{\text{Vesta}}} \right) = \frac{p_{\text{Vesta}} - p_{\text{Pallas}}}{p_{\text{Vesta}}} = \delta.$$

The rest of the game is a randomised function of the view before the i -th answer and of the replaced value, which is independent of that view; each step therefore costs at most δ , and the q steps at most $q\delta$ (*Math Guide*, §“Statistical distance”, Theorem “Properties of statistical distance”, parts (2) and (4)).

In H_5 the answers are those of the ideal game, and the key material is that of H_3 . Neither in H_5 nor in the ideal game do the answers depend on the key, so the hops of H_3 , H_2 and H_1 , taken in reverse order with the answers of the ideal game, lead from H_5 to the ideal game at the same costs. Summing the costs of both passes gives the bound. The last statement of the proposition holds because, for $K \neq \mathcal{O}$, the map $u \mapsto [u]K$ is a bijection from $\mathbb{F}_{p_{\text{Vesta}}}$ onto the group of prime order p_{Vesta} , and translation by cm_i is a bijection of the group; under Assumption 2.8 the point K is uniform. $\square$

Remark 6.11 (Design statement, not proved). The specification describes the nullifier as combining elliptic-curve cryptography with the Poseidon-based PRF so as to give privacy defence in depth against a weakness in either (protocol specification, §“Faerie Gold attack and fix”). It states that, under the weaker assumption that PoseidonHash preserves sufficient entropy from nk and ρ , the construction would remain unlinkable under a decisional Diffie–Hellman assumption on Pallas even if the Poseidon-based PRF were distinguishable from an ideal PRF (§“PoseidonHash Function”, note; *Crypto Guide*, §“Diffie–Hellman: computational and decisional”, Definition “Decisional Diffie–Hellman, DDH”; the form assumed in this volume is Assumption 3.16). The volume records this as the specification’s design statement and derives no result from it; Proposition 6.10 relies on Assumption 6.2 instead.

For keys with `use_qsk = false`, the key tiers of Proposition 3.18 (“Capability separation”) have the following powers over nullifiers. The full viewing key $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$ contains nk . Its holder computes $\text{DeriveNullifier}_{\text{nk}}(\rho, \psi, \text{cm})$ for every note of the key whose fields it knows, and tests whether that value is in the nullifier set of the note’s pool; that every accepted consumption of the note publishes this value is Lemma 12.5. It does so without spending authority, since the full viewing key yields no `ask` (Proposition 3.18(a)). A holder of only the incoming viewing key (dk, ivk) cannot link a nullifier to its note, for notes with pairwise distinct ρ , by Proposition 6.10.

Remark 6.12 (Limits). Proposition 6.10 gives nothing against a holder of nk , which includes every holder of the full viewing key: such a holder recomputes, and so links, the nullifier of every note of the key whose fields it knows. It does not concern links through other public data, such as timing or the structure of the transaction; the privacy of an Action, with its preconditions, is Theorem 12.12, in “Privacy” (§12.4).

7 Spend authorisation

This section instantiates RedDSA, the re-randomisable Schnorr signature scheme of the *Crypto Guide*, on the Pallas group; constructs the randomised validating key and the spend-authorisation

signature of an Action; and proves two properties of them: the published key is, with the signature scheme’s hash modelled as a random oracle, within negligible statistical distance of a key independent of the spend validating key; and a new valid signature under a randomised key yields the key’s discrete logarithm. The signed message is a parameter throughout; “Transaction digests and signatures” (§11.3) fixes it.

7.1 The RedPallas signature scheme

Remark 7.1 (Spending authority and its signature). Requirement R5 of “Requirements on a shielded payment” (§1.4) admits only the holder of a note’s spending authority as a party that consumes the note. For keys generated with `use_ksk = false`, the spending authority over the notes of every address derived from a spending key is knowledge of its spend authorising key ask of “The spending key and the spend-side secrets” (§3.1), the discrete logarithm of $ak^P = [ask] G^{\text{SpendAuth}}$ to the base $G^{\text{SpendAuth}}$. No lower key tier yields ask , since every lower tier is computed from the full viewing key (Proposition 3.18(a)). Each Action therefore carries a signature that only a holder of ask can produce, under a validating key that does not link Actions consuming notes of the same key: a fresh re-randomisation of ak^P (§7.2), never ak^P itself. Theorem 12.9, in “Authorisation of spends” (§12.3), proves for keys with `use_ksk = false` that no efficient party without ask produces an accepted Action that consumes a note of an honestly generated address and whose spend-authorisation signature is on a digest that the key holder did not sign under the Action’s randomised key, except with negligible probability.

Definition 7.2 (RedPallas hash). The *RedPallas hash* is the map H from byte strings to 64-byte strings

$$H(z) := \text{BLAKE2b-512}(\text{Zcash_RedPallasH}, z),$$

with the notation of the expansion function (§2.3) and the 16-byte personalisation `Zcash_RedPallasH`. Its reduction to scalars is

$$H^\circ(z) := \text{LEOS2IP}_{512}(H(z)) \bmod p_{\text{Vesta}} = \text{ToScalar}(H(z)) \in \mathbb{F}_{p_{\text{Vesta}}}$$

(protocol specification, §“RedDSA, RedJubjub, and RedPallas” and §“BLAKE2 Hash Functions”). The symbol H° is that of the specification and of the *Crypto Guide*’s Remark “RedDSA as deployed” (§“RedDSA: re-randomisable Schnorr for Orchard”); it is distinct from the star encoding P^* of a point.

Assumption 7.3 (The RedPallas hash as a random oracle). In security arguments the RedPallas hash H is modelled as a random oracle with output length 512, that is, with range the 64-byte strings, queried by every party (*Crypto Guide*, §“The random oracle model”, Definition “Random oracle”). Consequently H° is a random function into $\mathbb{F}_{p_{\text{Vesta}}}$: its values at distinct inputs are independent, and each is distributed as $\text{ToScalar}(U)$ for U uniform on the 64-byte strings. That distribution is at statistical distance at most

$$\delta_{512} := p_{\text{Vesta}}/2^{512} < 2^{-257}$$

from the uniform distribution on $\mathbb{F}_{p_{\text{Vesta}}}$ (*Math Guide*, §“Uniform sampling and the bias of modular reduction”, Proposition “Bias of modular reduction”, with $L = 512$), and it gives each scalar probability at most $1/p_{\text{Vesta}} + 2^{-512}$, since each residue has at most $\lfloor 2^{512}/p_{\text{Vesta}} \rfloor + 1$ preimages among the 2^{512} strings. The assumption is a heuristic about BLAKE2b, not a theorem. Every later result on RedPallas signatures or on the randomiser generator below names it.

Construction 7.4 (RedPallas). RedPallas is the *Crypto Guide*’s Construction “RedDSA” (§“RedDSA: re-randomisable Schnorr for Orchard”) on the Pallas group $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, of prime order p_{Vesta} , with both hashes $H^{\otimes}$ and with a base point $B \neq \mathcal{O}$ as parameter; the group having prime order, B generates it. For a point P let $\underline{P} := \text{LEBS2OSP}_{256}(P^*)$, its 32-byte encoding; the map $P \mapsto \underline{P}$ is injective, as the star encoding is (§2.1) and LEBS2OSP_{256} is a bijection onto the 32-byte strings. Messages are byte strings.

1. *Keys*. A signing key is a scalar $x \in \mathbb{F}_{p_{\text{Vesta}}}$; its validating key is $X := [x] B$.
2. *Signing* a message M under x : draw T uniformly from the 80-byte strings, where $80 = (512 + 128)/8$ is the specification’s randomness length for a 512-bit hash; let

$$\begin{aligned} r &:= H^{\otimes}(T \parallel \underline{X} \parallel M), & R &:= [r] B, \\ c &:= H^{\otimes}(\underline{R} \parallel \underline{X} \parallel M), & S &:= r + c \cdot x \in \mathbb{F}_{p_{\text{Vesta}}}, \end{aligned}$$

and output the 64-byte signature $\sigma := \underline{R} \parallel \text{I2LEOSP}_{256}(S)$.

3. *Validation* of a 64-byte string σ under a point X on $\mathcal{M}$: let $\bar{R}$ be the first 32 bytes of σ and $\bar{S}$ the last 32; let $R := \text{abst}_{\mathbb{P}}(\text{LEBS2OSP}_{256}^{-1}(\bar{R}))$, $S := \text{LEOS2IP}_{256}(\bar{S})$ and $c := H^{\otimes}(\bar{R} \parallel \underline{X} \parallel M)$. Accept if and only if $R \neq \perp$, $S < p_{\text{Vesta}}$ and

$$[S] B = R + [c] X. \tag{4}$$

(Protocol specification, §“RedDSA, RedJubjub, and RedPallas”; its validation equation carries the cofactor, which is 1 on Pallas.)

The partial inverse $\text{abst}_{\mathbb{P}}$ returns $\perp$ on every 256-bit string that is not the star encoding of a point (§2.1); validation therefore rejects a non-canonical $\bar{R}$, and an accepted signature has $\bar{R} = \underline{R}$. An honest signature is accepted: its first half $\underline{R}$ decodes to R , its S is an integer representative below p_{Vesta} , the verifier recomputes the signer’s c , and $[S] B = [r] B + [c \cdot x] B = R + [c] X$ (*Crypto Guide*, §“From identification to signature via the Fiat–Shamir transform”, Proposition “Perfect correctness”). Both the nonce hash and the challenge hash take the encoded validating key $\underline{X}$: the scheme is *key-prefixed*. The message M is a parameter: the message of every RedPallas signature in this volume is the signature digest constructed in “Transaction digests and signatures” (§11.3), and no argument before that subsection depends on its value.

Definition 7.5 (Spend-authorisation and binding-signature instances). The volume uses two instances of RedPallas, which share the signing and validation algorithms and differ in the base B and in the use of key re-randomisation (protocol specification, §“Spend Authorization Sig-

nature (Sapling and Orchard)” and §“Binding Signature (Sapling and Orchard)”):

1. the *spend-authorisation instance*, with key re-randomisation (§7.2) and base

$$B = G^{\text{SpendAuth}} = \text{GroupHash}(\text{z.cash:Orchard}, \mathfrak{g});$$

2. the *binding-signature instance*, without re-randomisation, used in “The binding signature” (§8.3), with base the value-commitment randomness base

$$B = R^{\text{Orchard}} = \text{GroupHash}(\text{z.cash:Orchard-cv}, r).$$

Both bases are rows of Table 2 (§2.2); the specification takes each as the generator parameter of RedDSA.

Definition 7.6 (Randomiser generator and key re-randomisation). The *randomiser generator* GenRandom draws T uniformly from the 80-byte strings and returns $H^{\otimes}(T) \in \mathbb{F}_{p_{\text{Vesta}}}$ (protocol specification, §“RedDSA, RedJubjub, and RedPallas”); its distance from uniform is bounded in §7.2. *Re-randomisation* by a randomiser $\alpha \in \mathbb{F}_{p_{\text{Vesta}}}$ maps a key pair (x, X) to $(x + \alpha, X + [\alpha] B)$ (*Crypto Guide*, §“Key re-randomisation and unlinkability”, Definition “Key re-randomisation”).

Remark 7.7 (Key prefixing). Let σ be valid under X on M , with decoded components R and S and challenge $c = H^{\otimes}(R \parallel \underline{X} \parallel M)$, and let $X' := X + [\beta] B \neq \mathcal{O}$ for a scalar $\beta \neq 0$. The shifted string $\sigma' := R \parallel \text{I2LEOSP}_{256}(S + c\beta)$ satisfies $[S + c\beta] B = R + [c] X'$. Validation of σ' under X' , however, recomputes the challenge as $c' := H^{\otimes}(R \parallel \underline{X'} \parallel M)$, the value of the oracle at an input other than that of c , since $\underline{X'} \neq \underline{X}$; and (4) holds for σ' under X' only if $[c - c'] X' = \mathcal{O}$, that is, only if $c' = c$. Under Assumption 7.3, for each β the value c' is independent of c and equals it with probability at most $1/p_{\text{Vesta}} + 2^{-512}$. A signature is therefore not transported from one key to a related key by shifting its response. With a challenge that omitted the key, $c' = c$ always, and the shift would turn every signature under X into one under X' (*Crypto Guide*, §“From identification to signature via the Fiat–Shamir transform”, Remark “Two conventions”; §“Key re-randomisation and unlinkability”, Remark “Unforgeability under re-randomised keys, and the key-prefixing subtlety”). The proof of Proposition 7.12, which admits randomisers chosen by the adversary, uses key prefixing in its steps (3) and (5).

7.2 Randomised validating keys

Construction 7.8 (Randomised validating key). For each Action the spender draws a *spend-authorisation randomiser* $\alpha := \text{GenRandom}()$, independently of every other draw, and sets

$$\text{rsk} := \text{ask} + \alpha \in \mathbb{F}_{p_{\text{Vesta}}}, \quad \text{rk} := \text{ak}^{\text{P}} + [\alpha] G^{\text{SpendAuth}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}),$$

with ask the sign-normalised spend authorising key of the spending key of the consumed note and ak^{P} its spend validating key (§3.1). Since $\text{ak}^{\text{P}} = [\text{ask}] G^{\text{SpendAuth}}$, it follows that $\text{rk} = [\text{rsk}] G^{\text{SpendAuth}}$ (*Crypto Guide*, §“Key re-randomisation and unlinkability”, Proposition

“Re-randomisation consistency”), so (rsk, rk) is a key pair of the spend-authorisation instance. The Action publishes the *randomised validating key* rk and its *spend-authorisation signature*, the RedPallas signature under rsk on the signature digest constructed in “Transaction digests and signatures” (§11.3); it publishes none of ask , α , rsk , ak^{P} and ak (protocol specification, §“Spend Authorization Signature (Sapling and Orchard)”).

Lemma 7.9 (Distance of GenRandom from uniform). *Under Assumption 7.3, let an algorithm, the observer, make at most q_b queries to H and, at one point, choose a byte string w and receive $\alpha := H^{\otimes}(T \parallel w)$, for T uniform on the 80-byte strings and independent of all else. The pair formed by the observer’s view and α is within statistical distance*

$$\delta_{512} + q_b \cdot 2^{-640} \quad (5)$$

of the pair obtained when α is replaced by a uniform element of $\mathbb{F}_{p_{\text{Vesta}}}$ independent of all else. A randomiser from GenRandom is the case $w = \varepsilon$; the nonce of a RedPallas signature under X on M is the case $w = X \parallel M$. The first term of (5) is set by the 512-bit output of H , not by the 640-bit length of T ; the length of T enters only the second term.

Proof. Let E be the event that the observer queries H at $T \parallel w$. In a second experiment, $H(T \parallel w)$ is replaced, in the computation of α only, by a uniform 64-byte string U independent of all else. In the lazy realisation of the random oracle (*Crypto Guide*, §“The random oracle model”, Definition “Random oracle”) the answers at inputs other than $T \parallel w$ are independent of $H(T \parallel w)$, so the two experiments are identical until E occurs; E has the same probability in both, and their outputs are within statistical distance $\Pr[E]$ (*Crypto Guide*, §“Security as a game”, Remark “Game-hopping”). In the second experiment the observer’s view is independent of T , so each query agrees with $T \parallel w$ in its first 80 bytes with probability at most 2^{-640} , and $\Pr[E] \leq q_b \cdot 2^{-640}$ by the union bound. There U is independent of H , of T and of the observer’s coins, and the pair of view and α is a function of $\alpha = \text{ToScalar}(U)$ and of them. Replacing α by a uniform scalar changes the pair by at most δ_{512} (Assumption 7.3; *Math Guide*, §“Statistical distance”, Theorem “Properties of statistical distance”, part (4)). The triangle inequality, part (2) of the same theorem, gives (5). $\square$

Two consensus rules of the protocol specification, §“Action Descriptions”, concern spend authorisation:

1. the randomised validating key rk of an Action must not be the identity $\mathcal{O}$;
2. the spend-authorisation signature of the Action must be valid under rk on the signature digest of “Transaction digests and signatures” (§11.3).

Validation rejects a non-canonical encoding of the signature’s point component (§7.1). An honestly computed rk equals $\mathcal{O}$ only if $\alpha = -\text{ask}$; for ask fixed before α is drawn, this event has probability at most $1/p_{\text{Vesta}} + 2^{-512}$ (Assumption 7.3).

Proposition 7.10 (Unlinkability of randomised validating keys). *Let $\text{ak}_1^{\text{P}}, \dots, \text{ak}_n^{\text{P}}$ be points of the Pallas group, equal or distinct, and let $\text{rk}_i := \text{ak}_i^{\text{P}} + [\alpha_i] G^{\text{SpendAuth}}$ for randomisers $\alpha_1, \dots, \alpha_n$ independent of each other and of the points.*

(i) If each α_i is uniform on $\mathbb{F}_{p_{\text{Vesta}}}$, the tuple $(\text{rk}_1, \dots, \text{rk}_n)$ is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})^n$, whatever the points ak_i^{P} , and hence independent of them. For each i and the scalar ask_i with $\text{ak}_i^{\text{P}} = [\text{ask}_i] G^{\text{SpendAuth}}$, the pair $(\text{ask}_i + \alpha_i, \text{rk}_i)$ is distributed exactly as a fresh key pair $(\gamma, [\gamma] G^{\text{SpendAuth}})$ with γ uniform, independently of ask_i . The distribution of the tuple is thus the same for every assignment of the points, and no observer, whatever its running time, distinguishes two assignments, for instance one in which two Actions share a key from one in which they do not, with non-zero advantage.

(ii) If each α_i is drawn by GenRandom, then under Assumption 7.3, for an observer that receives the tuple and makes at most q_b queries to H, the pair of its view and the tuple is within statistical distance

$$n(\delta_{512} + (q_b + n) \cdot 2^{-640})$$

of the pair obtained from a uniform tuple independent of H; the pairs arising from two assignments of the points are therefore within statistical distance $2n(\delta_{512} + (q_b + n) \cdot 2^{-640})$.

Proof. (i) For each i this is the *Crypto Guide*'s Theorem "Perfect unlinkability of re-randomised keys" (§"Key re-randomisation and unlinkability"). The base $G^{\text{SpendAuth}}$ generates the Pallas group, of prime order p_{Vesta} (§7.1), so $\alpha \mapsto [\alpha] G^{\text{SpendAuth}}$ is a bijection from $\mathbb{F}_{p_{\text{Vesta}}}$ onto the group, and translation by ak_i^{P} is a bijection of the group; hence rk_i is uniform whatever ak_i^{P} is, and the independence of the α_i gives the product distribution. The scalar $\text{ask}_i + \alpha_i$ is uniform for every fixed ask_i because α_i is, and $\text{rk}_i = [\text{ask}_i + \alpha_i] G^{\text{SpendAuth}}$. Equal distributions give every distinguisher advantage zero (*Math Guide*, §"Statistical distance", Theorem "Properties of statistical distance", part (3)).

(ii) The randomisers are replaced by independent uniform scalars one at a time, for $i = 1, \dots, n$. At step i the lemma on the distance of GenRandom from uniform applies, with $w = \varepsilon$, to the observer extended by the rest of the experiment: it holds the points and $\alpha_1, \dots, \alpha_{i-1}$, already uniform; it draws $T_{i+1}, \dots, T_n$ itself and queries H at them; it receives α_i , computes the tuple and runs the given observer. It makes at most $q_b + n - i$ queries, and the given observer's view and the tuple are a function of its view and α_i . By (5) and part (4) of the same theorem, step i changes the pair of view and tuple by at most $\delta_{512} + (q_b + n - i) \cdot 2^{-640}$. By the triangle inequality, the n steps together change it by at most

$$n\delta_{512} + \left(nq_b + \frac{n(n-1)}{2}\right)2^{-640} \leq n(\delta_{512} + (q_b + n) \cdot 2^{-640}).$$

After the last step the α_i are uniform, independent and independent of H, so by (i) the tuple is uniform and independent of H, for every assignment of the points. The triangle inequality through that common distribution gives the factor 2 for two assignments. $\square$

Remark 7.11 (Scope of Proposition 7.10). The proposition concerns rk alone, for randomisers drawn by the honest spender; a randomiser chosen by an adversary is the subject of Proposition 7.12 (§7.3). The spend-authorisation signature published with rk adds no information about ak^{P} : under Assumption 7.3 it is simulated from rk and the message M alone by programming H

(*Crypto Guide*, §“Security in the random oracle model and the forking lemma”, Lemma “Signature simulation”). The simulator draws u uniform on the 64-byte strings and S uniform on $\mathbb{F}_{p_{\text{Vesta}}}$, sets $c := \text{ToScalar}(u)$ and $R := [S] G^{\text{SpendAuth}} - [c] \text{rk}$, programs H at $R \parallel \text{rk} \parallel M$ to u , and outputs $R \parallel \text{I2LEOSP}_{256}(S)$. Beyond the abort probability of the lemma, a real signature differs from the simulated one only through its nonce: for a uniform nonce the two are identically distributed, since $S = r + c \cdot \text{rsk}$ is then uniform given u and $R = [S] G^{\text{SpendAuth}} - [c] \text{rk}$. The difference is therefore at most the bound (5) per signature, with q_b there the number of all other queries to H in the experiment. The remaining public fields of an Action are treated in Theorem 12.12, in “Privacy” (§12.4).

7.3 RedPallas unforgeability

Proposition 7.12 (Unforgeability under re-randomisation). *Assume that discrete logarithms on Pallas are hard (Assumption 2.22) and that H is a random oracle (Assumption 7.3). Let x be uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and $\text{ak}^{\text{P}} := [x] G^{\text{SpendAuth}}$. An adversary $\mathcal{A}$ receives ak^{P} , makes at most q_b queries to H and at most q_s queries to a signing oracle that answers a pair (α, M) of its choice, $\alpha \in \mathbb{F}_{p_{\text{Vesta}}}$ and M a byte string, with the spend-authorisation signature under $x + \alpha$ on M , whose validating key is $\text{ak}^{\text{P}} + [\alpha] G^{\text{SpendAuth}}$; it outputs a triple $(\text{rk}^*, M^*, \sigma^*)$ with rk^* a point. It forges if σ^* is valid under rk^* on M^* and $(\text{rk}^*, M^*, \sigma^*)$ is not the triple of validating key, message and signature of an oracle answer. Let ϵ be its forging probability, $Q := q_b + q_s + 1$ and*

$$\eta := Q \delta_{512} + q_s(q_b + 2q_s) \cdot 2^{-640}.$$

- (i) Extraction. *There is an algorithm $\mathcal{D}$ that receives ak^{P} but not x , runs $\mathcal{A}$ twice on shared coins, answers its signing queries without x , and outputs a scalar rsk^* with $\text{rk}^* = [\text{rsk}^*] G^{\text{SpendAuth}}$, for the rk^* of the first run, with probability at least*

$$\frac{(\epsilon - \eta)^2}{Q} - \frac{Q}{p_{\text{Vesta}}} - \frac{q_s(q_b + q_s)}{p_{\text{Vesta}}} \quad (\epsilon \geq \eta),$$

the bound of the Crypto Guide’s Theorem “EUFCMA security of Schnorr signatures in the ROM” (§“Security in the random oracle model and the forking lemma”) with ϵ replaced by $\epsilon - \eta$ and the group order by p_{Vesta} .

- (ii) Unforgeability under re-randomisation. *If $\mathcal{A}$ also outputs $\alpha^* \in \mathbb{F}_{p_{\text{Vesta}}}$ with $\text{rk}^* = \text{ak}^{\text{P}} + [\alpha^*] G^{\text{SpendAuth}}$, of its own choice and possibly dependent on ak^{P} , then $\mathcal{D}$ outputs $x = \text{rsk}^* - \alpha^*$, the discrete logarithm of ak^{P} , with the same probability. By Assumption 2.22, ϵ is then negligible for every efficient $\mathcal{A}$.*

With every randomiser zero, part (ii) is the existential unforgeability of RedPallas under chosen-message attack (Crypto Guide, §“Syntax and security goal”, Definition “Existential unforgeability under chosen-message attack”) in its strong form (Definition “Strong unforgeability” there), since freshness is required of the triple rather than of the message. Part (ii) implies the SURK-CMA requirement of the protocol specification, §“Signature with Re-Randomizable Keys”, whose forgery is a message–signature pair not returned by the oracle, hence a triple not returned by it.

Proof. The argument is route (ii) of the *Crypto Guide*’s Remark “Unforgeability under re-randomised keys, and the key-prefixing subtlety” (§“Key re-randomisation and unlinkability”), with the bound of its Theorem “EUF-CMA security of Schnorr signatures in the ROM” (§“Security in the random oracle model and the forking lemma”), which applies to RedPallas with the generator replaced by the base $G^{\text{SpendAuth}}$ (Construction “RedDSA”). The Orchard steps are made in place; write $G := G^{\text{SpendAuth}}$.

(1) *Idealisation.* First, the nonce r_i of each oracle answer is replaced by a uniform scalar. By the lemma on the distance of GenRandom from uniform, with $w = \text{rk}_i \parallel M_i$ and the rest of the experiment as observer, each of the at most q_s replacements costs at most $\delta_{512} + (q_b + 2q_s) \cdot 2^{-640}$: apart from this nonce query, the experiment queries H at most q_b times for $\mathcal{A}$, $q_s - 1$ times for other nonces, q_s times for challenges and once for validating the forgery. Second, the answers of H to the queries of $\mathcal{A}$ and to the validation query are replaced by uniform preimages, under ToScalar, of uniform scalars, so that their values under $H^\oplus$ are uniform on $\mathbb{F}_{p_{\text{Vesta}}}$. Each of these at most $q_b + 1$ replacements costs at most δ_{512} , because in both distributions an answer, conditioned on its reduction, is uniform on the preimages of that reduction. The forging probability thus drops by at most η (*Math Guide*, §“Statistical distance”, Theorem “Properties of statistical distance”, part (3)).

(2) *Embedding.* The reduction receives a discrete-logarithm challenge $Y = [x] G$ with x uniform and gives $\mathcal{A}$ the key $\text{ak}^P := Y$.

(3) *Signing.* It answers the i -th query (α_i, M_i) without x , under $\text{rk}_i := \text{ak}^P + [\alpha_i] G$, by the simulator of the *Crypto Guide*’s Lemma “Signature simulation” (§“Security in the random oracle model and the forking lemma”): it draws u_i uniform on the 64-byte strings and S_i uniform on $\mathbb{F}_{p_{\text{Vesta}}}$, sets $c_i := \text{ToScalar}(u_i)$ and $R_i := [S_i] G - [c_i] \text{rk}_i$, programs H at $R_i \parallel \text{rk}_i \parallel M_i$ to u_i , aborting if that entry is already defined, and returns $\sigma_i := R_i \parallel \text{I2LEOSP}_{256}(S_i)$. After step (1) a real answer has a uniform nonce r_i and, at a fresh input, the challenge $c_i = \text{ToScalar}(u_i)$ for a uniform u_i ; then $S_i = r_i + c_i(x + \alpha_i)$ is uniform given u_i , and $R_i = [S_i] G - [c_i] \text{rk}_i$, which is the simulated distribution (*Crypto Guide*, §“The Schnorr identification protocol”, Theorem “Special honest-verifier zero knowledge”). The point R_i is uniform, so the reduction aborts with probability at most $q_s(q_b + q_s)/p_{\text{Vesta}}$, as in the lemma. Key prefixing places rk_i in every programmed input: entries programmed under distinct keys have distinct inputs, and the simulation needs no relation among the rk_i .

(4) *Critical query.* Let $(\text{rk}^*, M^*, \sigma^*)$ be a forgery, with $\sigma^* = \bar{R}^* \parallel \bar{S}^*$ decoding to $R^* \neq \perp$ and $S^* < p_{\text{Vesta}}$. If $\text{rk}^* = \mathcal{O}$, the scalar $\text{rsk}^* := 0$ satisfies (i), and in (ii) $x = -\alpha^*$; the reduction outputs these. Let $\text{rk}^* \neq \mathcal{O}$. The validation input $\bar{R}^* \parallel \text{rk}^* \parallel M^*$ is not a programmed entry. Its first 32 bytes, its next 32 bytes and its remainder would otherwise give $\bar{R}^* = \underline{R}_i$, $\text{rk}^* = \text{rk}_i$ and $M^* = M_i$ for some i , the second by injectivity of $P \mapsto \underline{P}$, and the challenge would be c_i . Then $R^* = R_i$, and (4) gives $[S^*] G = R_i + [c_i] \text{rk}_i = [S_i] G$. Since G generates the group, of prime order p_{Vesta} , and $S^*, S_i < p_{\text{Vesta}}$, this forces $S^* = S_i$, so $\sigma^* = \sigma_i$ and the triple is an oracle answer. The input was queried by $\mathcal{A}$, except with probability at most $1/p_{\text{Vesta}}$: otherwise its value under $H^\oplus$ is a uniform scalar drawn at validation, and, since $\text{rk}^* \neq \mathcal{O}$ generates the group, at most one scalar satisfies (4). This guessing term is part of the term Q/p_{Vesta} of the cited theorem.

(5) *Fork.* The reduction wraps $\mathcal{A}$ as the algorithm of the *Crypto Guide*’s Theorem “General forking lemma” (§“Security in the random oracle model and the forking lemma”), with challenge

set $\mathbb{F}_{p_{\text{Vesta}}}$, Q prepared challenges $c_1, \dots, c_Q$, and coins that include those of $\mathcal{A}$ and of the simulator. The wrapper answers the j -th distinct unprogrammed query to H , of $\mathcal{A}$ or of validation, with a uniform preimage of c_j under ToScalar , and returns the index of the critical query with $\mathcal{A}$'s output. The two runs of the forking algorithm share the coins and the challenges before that index, so $\mathcal{A}$ makes the same critical query in both, with the same $\bar{R}^*$, the same rk^* , since key prefixing makes rk^* part of the critical input, and the same M^* . In (ii) they also share α^* , which rk^* determines because G generates the group. The runs receive distinct challenges $c \neq c'$ at the critical index and return responses S^* and S'^* .

(6) *Extraction*. The transcripts (R^*, c, S^*) and (R^*, c', S'^*) are accepting for rk^* with $c \neq c'$, so the *Crypto Guide*'s Theorem “2-special soundness” (§“The Schnorr identification protocol”) gives

$$\text{rsk}^* = (S^* - S'^*)(c - c')^{-1},$$

the discrete logarithm of rk^* , which is (i). For (ii) the reduction outputs $\text{rsk}^* - \alpha^* = x$. The probability bound follows as in the cited theorem, from the forking bound

$$\text{frk} \geq \text{acc} \left(\frac{\text{acc}}{Q} - \frac{1}{p_{\text{Vesta}}} \right),$$

with acc at least $\epsilon - \eta$ less the abort and guessing terms of steps (3) and (4). $\square$

Remark 7.13 (Uses of Proposition 7.12). Proposition 7.12 holds for every message: it places no condition on M^* beyond the freshness of the triple. Theorem 12.9 applies part (i), and Proposition 11.10 applies part (ii) to a signature on a new digest.

8 Value commitments and the binding signature

This section constructs the net value commitment of an Action, defines the transparent transaction value pool, the balancing value of each pool and the fee, and constructs the binding signature. Proposition 8.10 keeps apart an algebraic identity, under which the binding validating key is a known multiple of the randomness base exactly when a pool balances modulo p_{Vesta} , and a knowledge statement, under which a valid binding signature forces balance modulo p_{Vesta} .

8.1 Value commitments

Remark 8.1 (Requirement R4). Requirement R4 of “Requirements on a shielded payment” (§1.4) is met through the additive homomorphism of the Pedersen commitment (*Crypto Guide*, §“The Pedersen commitment”, Theorem “Additive homomorphism”). Each Action publishes a commitment to its net value; the group sum of a bundle's commitments commits to the sum of the net values under the sum of the trapdoors, and a verifier compares that sum with the bundle's public balancing value (§8.2) without learning any single value.

Definition 8.2 (Net value commitment). Let an Action consume a note of value v^{old} and create a note of value v^{new} , both in $\{0, \dots, 2^{64} - 1\}$, the value type of Definition “Note” (§4.1); a dummy side has value 0 (§4.4). The *net value* of the Action is the integer

$$v^{\text{net}} := v^{\text{old}} - v^{\text{new}} \in \{-2^{64} + 1, \dots, 2^{64} - 1\}.$$

The *value base* and the *randomness base* are

$$\begin{aligned} V^{\text{Orchard}} &:= \text{GroupHash}(z.\text{cash}:\text{Orchard-cv}, v), \\ R^{\text{Orchard}} &:= \text{GroupHash}(z.\text{cash}:\text{Orchard-cv}, r), \end{aligned}$$

the value-commitment rows of Table 2. For a trapdoor $\text{rcv} \in \mathbb{F}_{p_{\text{Vesta}}}$, the *net value commitment* of the Action is the point

$$\text{cv}^{\text{net}} := \text{ValueCommit}_{\text{rcv}}^{\text{Orchard}}(v^{\text{net}}) := [v^{\text{net}} \bmod p_{\text{Vesta}}] V^{\text{Orchard}} + [\text{rcv}] R^{\text{Orchard}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$$

(protocol specification, §“Homomorphic Pedersen commitments (Sapling and Orchard)”).

Both bases have order p_{Vesta} : the protocol specification types them as points of order p_{Vesta} (§“Balance and Binding Signature (Orchard)”), a set that excludes $\mathcal{O}$ (§“Pallas and Vesta”). The specification defines $\text{ValueCommit}^{\text{Orchard}}$ on the signed integers

$$\left\{ -\frac{p_{\text{Vesta}} - 1}{2}, \dots, \frac{p_{\text{Vesta}} - 1}{2} \right\}$$

(§“Commitment”), and a signed integer enters the scalar multiplication as its residue modulo p_{Vesta} (§2.1). Reduction modulo p_{Vesta} is injective on this domain, since two of its elements differ by at most $p_{\text{Vesta}} - 1$. The domain contains every net value, and every signed 64-bit integer, because $p_{\text{Vesta}} > 2^{254}$ (§2.1) gives $(p_{\text{Vesta}} - 1)/2 \geq 2^{253}$.

Construction 8.3 (Value-commitment trapdoor). For each Action the author of the transaction draws rcv uniformly from $\mathbb{F}_{p_{\text{Vesta}}}$, by $\text{ValueCommit}^{\text{Orchard}}.\text{GenTrapdoor}$, independently of every other value (protocol specification, §“Sending Notes (Orchard)” and §“Homomorphic Pedersen commitments (Sapling and Orchard)”). Unlike rcm , ψ and esk , which are derived from the note seed rseed (§4.3), the trapdoor rcv is derived from no note, and no note field, note plaintext (§10.1) or transaction field (§11.2) contains it. The author retains every rcv of the bundle; together they determine the binding signing key (§8.3).

The specification types the trapdoor as an integer in $\{0, \dots, 2^{255} - 1\}$ (§“Commitment”). As for rcm (§4.1), this volume types it in $\mathbb{F}_{p_{\text{Vesta}}}$, the set that GenTrapdoor samples, since rcv acts only as the scalar of $[\text{rcv}]$.

Lemma 8.4 (Hiding and binding of net value commitments). *With $G := V^{\text{Orchard}}$ and $H := R^{\text{Orchard}}$, the map $\text{ValueCommit}^{\text{Orchard}}$ is the Pedersen commitment of the Crypto Guide (§“The Pedersen commitment”, Construction “Pedersen commitment”), with messages the residues modulo p_{Vesta} .*

- (i) *Perfect hiding: for rcv uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and independent of v^{net} , the commitment cv^{net} is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, whatever v^{net} is.*
- (ii) *Computational binding: under Assumptions 2.22 and 2.8, no efficient algorithm outputs, except with negligible probability, pairs (v, rcv) and (v', rcv') of an integer and a trapdoor with $v \not\equiv v' \pmod{p_{\text{Vesta}}}$ and $[v] V^{\text{Orchard}} + [\text{rcv}] R^{\text{Orchard}} = [v'] V^{\text{Orchard}} + [\text{rcv}'] R^{\text{Orchard}}$.*

On the domain of $\text{ValueCommit}^{\text{Orchard}}$ distinct integers are distinct residues, so the same bound excludes openings of one point to two distinct net values.

Proof. (i) This is the *Crypto Guide*’s Theorem “Perfect hiding” (§“The Pedersen commitment”). Since $R^{\text{Orchard}} \neq \mathcal{O}$ and the group has prime order, $\text{rcv} \mapsto [\text{rcv}] R^{\text{Orchard}}$ is a bijection from $\mathbb{F}_{p_{\text{Vesta}}}$ onto $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$; the uniform point $[\text{rcv}] R^{\text{Orchard}}$ stays uniform under translation by the fixed point $[v^{\text{net}} \bmod p_{\text{Vesta}}] V^{\text{Orchard}}$. (ii) The double-opening algebra of the *Crypto Guide*’s Theorem “Computational binding under DLog” (same section) gives

$$[v - v'] V^{\text{Orchard}} - [\text{rcv}' - \text{rcv}] R^{\text{Orchard}} = \mathcal{O},$$

a relation with non-zero coefficient $v - v'$ on V^{Orchard} . The two bases are values of GroupHash at distinct inputs; under Assumption 2.8 they replace the independent generators that the cited construction samples, and the lemma on relations among fixed generators (§2.4) excludes the relation except with negligible probability. The specification requires computational binding and states that the scheme is unconditionally hiding (protocol specification, §“Homomorphic Pedersen commitments (Sapling and Orchard)”). $\square$

8.2 The balancing value

Definition 8.5 (Transparent transaction value pool). *Transparent inputs* and *transparent outputs* are the cleartext value transfers of a transaction outside every shielded pool, with public amounts. Each transaction has a *transparent transaction value pool*: its transparent inputs add their values to it, and its transparent outputs remove theirs. The balancing value of each shielded pool, defined next for the two pools of the Orchard protocol, is also added to it (protocol specification, §“Transactions and Treestates”).

Definition 8.6 (Balancing value). For each pool of the Orchard protocol, the Orchard pool or the Ironwood pool, the *balancing value* $v_{\text{pool}}^{\text{balance}}$ of a transaction is its declared net value, in zatoshi, of the notes of the pool that it consumes minus those that it creates: a signed integer in $\{-2^{63}, \dots, 2^{63} - 1\}$, encoded in cleartext as the two’s-complement 64-bit field `valueBalanceOrchard` or `valueBalanceIronwood`, of type `int64`. It is 0 when the transaction has no Action of the pool, and the field is then absent. A positive $v_{\text{pool}}^{\text{balance}}$ takes net value out of the pool and adds it to the transparent transaction value pool; a negative one takes value from the transparent transaction value pool into the pool (protocol specification, §“Balance and Binding Signature (Orchard)”; ZIP 229, “Transaction Format”).

In a correctly constructed transaction, $v_{\text{pool}}^{\text{balance}}$ equals the sum of the net values v^{net} of the pool’s Actions. A verifier cannot check this equation directly, since each v^{net} is hidden in its cv^{net} ; the binding signature (§8.3) enforces it.

The protocol specification, §“Transactions and Treestates”, states the consensus rule: the value remaining in the transparent transaction value pool must be non-negative. For a transaction other than a coinbase transaction (Definition “Coinbase transaction”, §11.2) the remaining value is the

transaction fee (ibid.); no transaction field encodes it (ZIP 229, “Transaction Format” and “Non-requirements”). The pool of a coinbase transaction also receives, as implicit inputs, the block subsidy and the fees of the block’s other transactions, and its outputs must consume all of the value in it (protocol specification, §“Transactions and Treestates” and §“Transaction Consensus Rules”). ZIP 317, “Fee calculation”, specifies the *conventional fee*, which wallets SHOULD pay and which is not a consensus requirement:

$$\begin{aligned} \text{marginal_fee} \cdot \max(\text{grace_actions}, \text{logical_actions}) &\text{ zatoshi,} \\ \text{marginal_fee} &= 5000, \quad \text{grace_actions} = 2, \end{aligned}$$

where *logical_actions* is a sum of per-component contributions, in which each Ironwood-pool Action contributes one (the contribution $n\text{ActionsIronwood}$). The fee of the worked payment is computed in Example 11.16 (§11.5).

Remark 8.7 (Disclosure by the balancing value). The balancing value is cleartext, so a negative one discloses the amount that enters the pool; the field names no receiving note or address.

8.3 The binding signature

Fix a pool of the Orchard protocol and a transaction whose bundle for that pool has $n \geq 1$ Actions, with net values v_i^{net} , trapdoors rcv_i and net value commitments cv_i^{net} ($i = 1, \dots, n$), and write v^{balance} for the pool’s balancing value.

Definition 8.8 (Binding signing key). The *binding signing key* of the bundle is

$$\text{bsk} := \text{rcv}_1 + \dots + \text{rcv}_n \in \mathbb{F}_{p_{\text{Vesta}}}$$

(protocol specification, §“Balance and Binding Signature (Orchard)”). By the additive homomorphism (*Crypto Guide*, §“The Pedersen commitment”, Theorem “Additive homomorphism”),

$$\sum_{i=1}^n \text{cv}_i^{\text{net}} = \left[\sum_{i=1}^n v_i^{\text{net}} \right] V^{\text{Orchard}} + [\text{bsk}] R^{\text{Orchard}}, \quad (6)$$

the integer sum entering as its residue modulo p_{Vesta} .

Definition 8.9 (Binding validating key). The verifier computes, from public fields only, the *binding validating key*

$$\text{bvk} := \left(\sum_{i=1}^n \text{cv}_i^{\text{net}} \right) - [v^{\text{balance}}] V^{\text{Orchard}},$$

where the subtracted point is $\text{ValueCommit}_0^{\text{Orchard}}(v^{\text{balance}})$, the commitment with zero trapdoor, and v^{balance} , an element of the domain of $\text{ValueCommit}^{\text{Orchard}}$, enters as its residue modulo p_{Vesta} . No transaction field encodes bvk (protocol specification, §“Balance and Binding Signature (Orchard)”).

An *opening* of a net value commitment cv_i^{net} is a pair (v'_i, r'_i) of an integer v'_i and a scalar $r'_i \in \mathbb{F}_{p_{\text{Vesta}}}$ with $\text{cv}_i^{\text{net}} = [v'_i \bmod p_{\text{Vesta}}] V^{\text{Orchard}} + [r'_i] R^{\text{Orchard}}$. In the next proposition a *binding signature*

under a point X is a signature valid under X in the binding-signature instance of RedPallas, with base R^{Orchard} (§7.1); Construction “Binding signature” below fixes its use.

Proposition 8.10 (Balance and key knowledge). *For the bundle fixed above:*

(i) *Identity. If $\text{cv}_i^{\text{net}} = \text{ValueCommit}_{\text{rcv}_i}^{\text{Orchard}}(v_i^{\text{net}})$ for every i and $\text{bsk} := \sum_i \text{rcv}_i$, then*

$$\text{bvk} - [\text{bsk}] R^{\text{Orchard}} = \left[\sum_{i=1}^n v_i^{\text{net}} - v^{\text{balance}} \right] V^{\text{Orchard}},$$

hence $\text{bvk} = [\text{bsk}] R^{\text{Orchard}}$ if and only if $\sum_i v_i^{\text{net}} \equiv v^{\text{balance}} \pmod{p_{\text{Vesta}}}$.

(ii) *Knowledge. Let (v'_i, r'_i) be an opening of cv_i^{net} for every i , and let $b \in \mathbb{F}_{p_{\text{Vesta}}}$ satisfy $\text{bvk} = [b] R^{\text{Orchard}}$. If $D := \sum_i v'_i - v^{\text{balance}}$ is non-zero modulo p_{Vesta} , then*

$$\left(b - \sum_{i=1}^n r'_i \right) D^{-1} \pmod{p_{\text{Vesta}}}$$

is the discrete logarithm of V^{Orchard} to the base R^{Orchard} , a relation between two values of GroupHash.

Consequently, under Assumptions 2.22, 2.8 and 7.3, an efficient party that outputs the net value commitments and the balancing value of a bundle, openings (v'_i, r'_i) of every cv_i^{net} and a binding signature under the resulting bvk on a message of its choice satisfies $\sum_i v'_i \equiv v^{\text{balance}} \pmod{p_{\text{Vesta}}}$, except with negligible probability. The proposition holds for every message.

Proof. (i) Expanding each cv_i^{net} by Definition 8.2 and applying the additive homomorphism gives (6); subtracting $[v^{\text{balance}}] V^{\text{Orchard}}$ gives the identity. Since V^{Orchard} has prime order p_{Vesta} , the point $[c] V^{\text{Orchard}}$ is $\mathcal{O}$ if and only if $c \equiv 0 \pmod{p_{\text{Vesta}}}$.

(ii) By the homomorphism, $\text{bvk} = [D] V^{\text{Orchard}} + [\sum_i r'_i] R^{\text{Orchard}}$. Equating with $[b] R^{\text{Orchard}}$ gives

$$[D] V^{\text{Orchard}} = \left[b - \sum_{i=1}^n r'_i \right] R^{\text{Orchard}},$$

and D is invertible modulo the prime p_{Vesta} . This is Part 2 of the proof of the *Crypto Guide*’s Theorem “The binding signature is a proof of knowledge of $\log_p B$ ” (§“Binding signatures as a proof of knowledge of a discrete logarithm”), with $P = R^{\text{Orchard}}$ and the balancing value folded into the key as that section describes after the theorem.

Consequence. Let an efficient party $\mathcal{A}$ output, with probability ϵ , a bundle’s net value commitments and balancing value, openings of every commitment, a message and a binding signature on it under the resulting bvk , with D non-zero modulo p_{Vesta} ; this event is efficiently decidable from the output. If $\text{bvk} = \mathcal{O}$, then $b := 0$ satisfies $\text{bvk} = [b] R^{\text{Orchard}}$. Otherwise Part 1 of the proof of the same theorem, special soundness with forking in the random-oracle model for H (Assumption 7.3), rewinds $\mathcal{A}$ to two accepting transcripts with a common first component and distinct challenges and extracts b with $\text{bvk} = [b] R^{\text{Orchard}}$. In this volume the extraction is steps (1) and (4) to (6) of the proof of Proposition 7.12, with no signing queries ($q_i = 0$), the base R^{Orchard} in place of

$G^{\text{SpendAuth}}$ and bvk in place of rk^* . Key prefixing places the encoding of bvk in the critical query, so both runs sign under the same bvk , and the openings and the balancing value of the first run satisfy the hypothesis of part (ii) for it. If ϵ is non-negligible, the extraction succeeds with non-negligible probability, and part (ii) then outputs the discrete logarithm of V^{Orchard} to the base R^{Orchard} : a non-trivial relation between values of GroupHash at distinct inputs, which the lemma on relations among fixed generators (§2.4) excludes except with negligible probability under Assumptions 2.8 and 2.22. No argument uses the value of the message, and no concrete bound is claimed. The security argument of the protocol specification, §“Balance and Binding Signature (Orchard)”, is the deployed counterpart. $\square$

The openings are obtained, and the congruence is promoted to an equality of integers, in Lemma 12.3 and Theorem 12.4 (§12.1).

Construction 8.11 (Binding signature). The binding signature scheme $\text{BindingSig}^{\text{Orchard}}$ is the binding-signature instance of RedPallas (§7.1): RedPallas on the base R^{Orchard} , without key re-randomisation; its base differs from the spend-authorisation base $G^{\text{SpendAuth}}$. The author of the transaction computes bsk from the trapdoors it retains and signs with it; the verifier recomputes bvk from the public fields and validates the signature under it (protocol specification, §“Binding Signature (Sapling and Orchard)” and §“Balance and Binding Signature (Orchard)”). The signed message is the signature digest SigHash constructed in “Transaction digests and signatures” (§11.3); which transaction data the signatures fix is stated there, in Proposition 11.10.

For an honestly constructed bundle with $\sum_i v_i^{\text{net}} = v^{\text{balance}}$, Proposition 8.10(i) gives $\text{bvk} = [\text{bsk}] R^{\text{Orchard}}$, so (bsk, bvk) is a key pair of the instance and the honest signature is accepted (§7.1). Under Assumption 7.3 a valid signature is a proof of knowledge of the discrete logarithm of bvk to the base R^{Orchard} (*Crypto Guide*, §“Binding signatures as a proof of knowledge of a discrete logarithm”, Theorem “The binding signature is a proof of knowledge of $\log_p B$ ”), so, for a party that also outputs openings of every net value commitment, the consequence stated in Proposition 8.10, under Assumptions 2.22, 2.8 and 7.3, makes it certify balance modulo p_{Vesta} without taking any net value as input: it is computed from bsk , the message and fresh randomness. What the public data of a bundle hide is stated in Theorem 12.12 (§12.4).

The construction runs once for each pool of the Orchard protocol. A transaction carries a balancing value and a binding signature, bindingSigOrchard or $\text{bindingSigIronwood}$, for each pool whose bundle has at least one Action. Each pool’s bvk sums only that pool’s net value commitments and subtracts only that pool’s balancing value; both pools use $\text{ValueCommit}^{\text{Orchard}}$ with the bases V^{Orchard} and R^{Orchard} , and $\text{BindingSig}^{\text{Orchard}}$. The consensus rule of the protocol specification, §“Transaction Consensus Rules” (ZIP 229, “Consensus Rules”), is: if a pool’s bundle has Actions, its binding signature must be a valid signature of SigHash under that pool’s bvk . Validation rejects a non-canonical encoding of the signature’s point component (§7.1; protocol specification, §“RedDSA, RedJubjub, and RedPallas”).

9 The Action statement and its proof

This section states the relation that is proved for every Action. It fixes the bundle flags that enter the relation as public inputs, states the Action statement over the objects constructed in §2 to §8, pairs each condition with the requirement it serves and with the check outside the proof that completes it, and states the proof system together with the two assumptions on the Action proof that “Security” (§12) uses.

9.1 Bundle flags

Each bundle carries one *flags byte*, present exactly when the bundle has at least one Action: `flagsOrchard` for the Orchard-pool bundle of a version 5 or version 6 transaction, and `flagsIronwood` for the Ironwood-pool bundle of a version 6 transaction (ZIP 229, “Transaction Format”). Bits are numbered from the least significant, bit 0. In both bytes bit 0 is `enableSpends` and bit 1 is `enableOutputs`. The consensus rules of ZIP 229, “Consensus Rules”, fix the remaining bits:

1. in `flagsIronwood`, bit 2 is `enableCrossAddress`, and bits 3 to 7 are reserved and must be 0;
2. in `flagsOrchard`, bits 2 to 7 are reserved and must be 0, in version 5 and version 6 transactions alike.

ZIP 229, “Transaction Format”, lists `enableCrossAddress` at bit 2 of `flagsOrchard` in its format table; this contradicts the consensus rules of the same ZIP, which reserve that bit, and the volume follows the consensus rules. The two texts differ in the role of the bit, not in its value in a valid transaction: ZIP 258, “Consensus rules from NU6.3 activation”, requires every Orchard-pool Action to be created with `enableCrossAddress = 0`.

Definition 9.1 (Flag values and expanded receiver). Let a bundle have flags byte $f = \sum_{i=0}^7 f_i 2^i$ with $f_i \in \{0, 1\}$. Its *flag values*, elements of $\{0, 1\}$, are

$$\begin{aligned} \text{enableSpends} &:= f_0, & \text{enableOutputs} &:= f_1, \\ \text{enableCrossAddress} &:= \begin{cases} f_2 & \text{for an Ironwood-pool bundle,} \\ 0 & \text{for an Orchard-pool bundle,} \end{cases} \end{aligned}$$

and $\text{disableCrossAddress} := 1 - \text{enableCrossAddress}$. For an Orchard-pool bundle bit 2 is reserved and 0, so every Orchard-pool Action has `enableCrossAddress = 0` by encoding. The *expanded receiver* of a payment address (d, pk_d) is the pair (g_d, pk_d) with $g_d = \text{DiversifyHash}(d)$, the diversified base of “Diversified addresses” (§3.3); the expanded receiver of a note is that of its address. The flags have the following meaning (protocol specification, §“Action Descriptions”):

- the value `enableSpends = 1` enables non-zero-valued spends in the bundle’s Actions;
- the value `enableOutputs = 1` enables non-zero-valued outputs in the bundle’s Actions;
- the value `enableCrossAddress = 1` enables the expanded receiver of an Action’s created note to differ from that of the note the same Action consumes.

The Action statement takes `disableCrossAddress` as its eighth primary input (same section). No normative text fixes the position of this input in the encoded instance, and none is asserted here.

ZIP 229 calls `enableCrossAddress = 1` the normal case for Ironwood-pool Actions (Rationale, “`enableCrossAddress` polarity”); the value is not required. No consensus rule requires bit 2 of `flagsIronwood` to be 1. A bundle in which that bit is 0 is admitted, and each of its Actions is then restricted, by condition A10 of Definition 9.2, to create its note at the expanded receiver of the note it consumes.

The flags are properties of the bundle. Encoded once in its flags byte, they are the same for every Action of the bundle (protocol specification, §“Action Descriptions”, note on the encoding of these components once per pool). The verifier reads them from the bundle and supplies `enableSpends`, `enableOutputs` and `disableCrossAddress` as public inputs of each Action; they are not computed inside the proof. They enter the relation only through conditions A8, A9 and A10 of Definition 9.2.

9.2 The Action statement

The Action statement is a relation in the sense of the *Crypto Guide*’s Definition “Language, relation, witness” (§“Languages, relations, and witnesses”), with the primary input as instance and the auxiliary input as witness, each a tuple of the typed objects below, encoded as bit strings by the encodings of §2.1. Every condition is an equation over objects constructed in §2 to §8.

Definition 9.2 (Orchard Action statement). In this definition $\mathcal{E}$ abbreviates the Pallas group $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, with identity $\mathcal{O}$, and $\mathcal{E}^* := \mathcal{E} \setminus \{\mathcal{O}\}$. For $P = (x, y) \in \mathcal{E}^*$ let $x(P) := x$ and $y(P) := y$, and let $x(\mathcal{O}) := y(\mathcal{O}) := 0$, so that $\text{Extract}_P(P) = x(P)$ for every $P \in \mathcal{E}$ (protocol specification, §“Coordinate Extractor for Pallas”). An integer that multiplies a point acts as its residue modulo p_{Vesta} (§2.1).

The relation $\mathcal{R}_{\text{Action}}$ is the set of pairs (pub, aux) of a *primary input*

$$\text{pub} = (rt, cv^{\text{net}}, nf^{\text{old}}, rk, cmx, \text{enableSpends}, \text{enableOutputs}, \text{disableCrossAddress})$$

and an *auxiliary input*

$$\text{aux} = (\text{path}, \text{pos}, g_d^{\text{old}}, pk_d^{\text{old}}, v^{\text{old}}, \rho^{\text{old}}, \psi^{\text{old}}, rcm^{\text{old}}, cm^{\text{old}}, \alpha, ak^P, nk, r_{\text{ivk}}, \\ g_d^{\text{new}}, pk_d^{\text{new}}, v^{\text{new}}, \psi^{\text{new}}, rcm^{\text{new}}, rcv, c)$$

that have the following types and satisfy conditions A1 to A10 below.

- Primary input: $rt, nf^{\text{old}}, cmx \in \{0, \dots, p_{\text{Pallas}} - 1\}$, read as elements of $\mathbb{F}_{p_{\text{Pallas}}}$; $cv^{\text{net}}, rk \in \mathcal{E}$; and `enableSpends`, `enableOutputs`, `disableCrossAddress` $\in \{0, 1\}$.
- Auxiliary input: $\text{path} = (s_0, \dots, s_{31})$ with every $s_t \in \{0, \dots, p_{\text{Pallas}} - 1\}$; $\text{pos} \in \{0, \dots, 2^{32} - 1\}$; $g_d^{\text{old}}, pk_d^{\text{old}}, g_d^{\text{new}}, pk_d^{\text{new}}, ak^P \in \mathcal{E}^*$; $v^{\text{old}}, v^{\text{new}} \in \{0, \dots, 2^{64} - 1\}$; $\rho^{\text{old}}, \psi^{\text{old}}, \psi^{\text{new}}, nk \in \mathbb{F}_{p_{\text{Pallas}}}$; $cm^{\text{old}} \in \mathcal{E}$; $rcm^{\text{old}}, rcm^{\text{new}}, r_{\text{ivk}}, rcv, \alpha \in \{0, \dots, 2^{255} - 1\}$; and $c = (c_0, \dots, c_{63}) \in \{0, 1\}^{64}$.

Let $\rho^{\text{new}} := \text{nf}^{\text{old}} \in \mathbb{F}_{p_{\text{Pallas}}}$, let $\delta := \text{disableCrossAddress}$, and let

$$\begin{aligned} M^{\text{old}} &:= (g_d^{\text{old}})^* \parallel (pk_d^{\text{old}})^* \parallel \text{LE}_{64}(v^{\text{old}}) \parallel \text{LE}_{255}(\rho^{\text{old}}) \parallel \text{LE}_{255}(\psi^{\text{old}}), \\ M^{\text{new}} &:= (g_d^{\text{new}})^* \parallel (pk_d^{\text{new}})^* \parallel \text{LE}_{64}(v^{\text{new}}) \parallel \text{LE}_{255}(\rho^{\text{new}}) \parallel \text{LE}_{255}(\psi^{\text{new}}), \end{aligned}$$

the 1086-bit messages of Definition 4.4, with the diversified bases witnessed. The conditions, all equations in $\mathbb{F}_{p_{\text{Pallas}}}$ or in $\mathcal{E}$, are:

A1. Old note commitment integrity: $\text{NoteCommit}_{\text{rcm}^{\text{old}}}(M^{\text{old}}) \in \{\text{cm}^{\text{old}}, \perp\}$.

A2. New note commitment integrity:

$$\text{Extract}_{\mathbb{P}}^{\perp}(\text{NoteCommit}_{\text{rcm}^{\text{new}}}(M^{\text{new}})) \in \{\text{cmx}, \perp\}.$$

A3. Merkle path validity: $v^{\text{old}} \cdot (\text{root} - \text{rt}) = 0$ or some evaluation of SinsemillaHash in the fold, the root included, returns $\perp$. Here $\text{root} := a_{32}$ is the result of the fold (2) of Proposition 5.9 with $a_0 := \text{Extract}_{\mathbb{P}}(\text{cm}^{\text{old}})$, siblings $s_0, \dots, s_{31}$ and the bits of pos as direction bits, in which the evaluation of MerkleCRH at level t encodes its left argument x as $\text{LE}_{255}(\tilde{x})$, where $\tilde{x} := x + c_{2t} p_{\text{Pallas}}$ if this integer is below 2^{255} and $\tilde{x} := x$ otherwise, and its right argument likewise with c_{2t+1} .

A4. Value commitment integrity:

$$\text{cv}^{\text{net}} = \text{ValueCommit}_{\text{rcv}}^{\text{Orchard}}(v^{\text{old}} - v^{\text{new}}) = [v^{\text{old}} - v^{\text{new}}] V^{\text{Orchard}} + [\text{rcv}] R^{\text{Orchard}}.$$

A5. Nullifier integrity: $\text{nf}^{\text{old}} = \text{DeriveNullifier}_{\text{nk}}(\rho^{\text{old}}, \psi^{\text{old}}, \text{cm}^{\text{old}})$, that is,

$$\text{nf}^{\text{old}} = \text{Extract}_{\mathbb{P}}([\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho^{\text{old}}) + \psi^{\text{old}}] \bmod p_{\text{Pallas}}] K^{\text{Orchard}} + \text{cm}^{\text{old}}).$$

A6. Spend authority: $\text{rk} = \text{ak}^{\mathbb{P}} + [\alpha] G^{\text{SpendAuth}}$.

A7. Diversified address integrity: with

$$\text{ivk} := \text{Commit}_{t_{\text{ivk}}}^{\text{ivk}}(\text{Extract}_{\mathbb{P}}(\text{ak}^{\mathbb{P}}), \text{nk}),$$

$$\text{either } \text{ivk} = \perp \text{ or } \text{pk}_d^{\text{old}} = [\text{ivk}] g_d^{\text{old}}.$$

A8. Enable spend flag: $v^{\text{old}} \cdot (1 - \text{enableSpends}) = 0$.

A9. Enable output flag: $v^{\text{new}} \cdot (1 - \text{enableOutputs}) = 0$.

A10. Cross-address restriction: the four equations

$$\begin{aligned} \delta \cdot (x(g_d^{\text{old}}) - x(g_d^{\text{new}})) &= 0, & \delta \cdot (\gamma(g_d^{\text{old}}) - \gamma(g_d^{\text{new}})) &= 0, \\ \delta \cdot (x(pk_d^{\text{old}}) - x(pk_d^{\text{new}})) &= 0, & \delta \cdot (\gamma(pk_d^{\text{old}}) - \gamma(pk_d^{\text{new}})) &= 0. \end{aligned}$$

(Protocol specification, §“Action Statement (Orchard)”, for the typed inputs and conditions A1 to A9; §“Action Descriptions” for the eighth primary input and the meaning of $\text{enableCrossAddress}$ that A10 formalises.)

The anchor rt is the bundle’s anchor (“Anchors”, §5.2). The rule $rk \neq \mathcal{O}$ of “Randomised validating keys” (§7.2) is checked outside the statement. The typing is part of the relation: the spend validating key ak^P , both diversified bases and both transmission keys are non-identity points (protocol specification, §“Action Statement (Orchard)”, note on types). The trapdoors rcm^{old} , rcm^{new} , r_{ivk} and rcv and the randomiser α act only as scalars on points of order p_{Vesta} , and the statement does not require them to be below p_{Vesta} (same note). The created note’s element ρ is not an auxiliary input; condition A2 fixes it. The relation is an NP-relation in the sense of the cited definition: its instances and witnesses have fixed length, and membership is decided by evaluating the typing and the ten conditions.

Points of the primary input enter the proof by their coordinates. The specification’s encoding note fixes the first seven primary inputs as the nine elements

$$rt, x(cv^{net}), y(cv^{net}), nf^{old}, x(rk), y(rk), cmx, enableSpend, enableOutputs$$

of $\mathbb{F}_{p_{Pallas}}$, in this order (protocol specification, §“Action Statement (Orchard)”, note on the encoding of primary inputs); the convention $x(\mathcal{O}) = y(\mathcal{O}) = 0$ encodes $cv^{net} = \mathcal{O}$. The eighth input, `disableCrossAddress`, is one further element of $\mathbb{F}_{p_{Pallas}}$ (§“Action Descriptions”); no normative text fixes its position, and the volume asserts none.

Conditions and the checks that complete them. Each condition is paired below with the requirement of “Requirements on a shielded payment” (§1.4) that it serves and, where one exists, with the check outside the proof that completes it; Table 5 collects the pairing and Figure 3 draws it.

Conditions A1 and A2 bind the consumed and the created note to the commitments cm^{old} and cmx under `NoteCommit`, the Sinsemilla commitment with domain `z.cash:Orchard-NoteCommit` (Definition 4.4), whose binding is Proposition 4.5; they serve R1. Condition A2 fixes the created note’s ρ to the nullifier that the same Action publishes for the note it consumes, the chaining rule of “Nullifier chaining” (§6.3). Its consequences for the uniqueness of ρ and for the Faerie Gold attack are proved in “Double-spend resistance” (Corollary 12.7).

Condition A3 serves R2. Since $0 \leq v^{old} < 2^{64} < p_{Pallas}$ (§2.1), the value v^{old} is zero in $\mathbb{F}_{p_{Pallas}}$ only if it is the integer 0, and a field has no zero divisors. Hence, outside its $\perp$ case and with canonical encodings ($c = 0$), A3 holds if and only if $v^{old} = 0$ or $root = rt$, the specification’s form: either $v^{old} = 0$, or $(\text{Extract}_p(cm^{old}), pos, path)$ is a valid path to rt in the sense of Proposition 5.9, with the layer-tagged hash of “The Merkle hash and the tree” (§5.1) and the paths of depth 32 of “Authentication paths” (§5.3). A consumed note of value zero is therefore exempt from membership. The exemption lets an Action whose consumed side is a dummy note (“Actions, bundles, and dummy notes”, §4.4) create a real note without consuming a committed one. For $v^{old} \neq 0$ the condition is membership of $\text{Extract}_p(cm^{old})$ under rt , up to the two relaxations below; Proposition 5.9 gives its soundness together with the anchor rule of “Anchors”, the check outside the proof that completes A3.

The two relaxations are the circuit’s: it does not check that the encoding of each layer’s input is canonical, and its path check may pass when a hash on the path, the root included, is 0, which occurs only when `SinsemillaHash` returns $\perp$ (protocol specification, §“Merkle Path Validity”, notes on Orchard; §“Action Statement (Orchard)”, notes on the Merkle path validity check). Remark “ $\perp$ -weakened conditions” below treats the $\perp$ case. The bits c , absent from the specification’s auxiliary input, record the encodings that the circuit witnesses. Since p_{Pallas} is odd and has bit length 255

(§2.1), $2p_{\text{Pallas}} > 2^{255}$, so the integers below 2^{255} congruent to an argument $x \in \{0, \dots, p_{\text{Pallas}} - 1\}$ are x and, when it is below 2^{255} , $x + p_{\text{Pallas}}$: the bits range over every encoding the circuit admits, and $c = 0$ gives the canonical ones. Non-canonical encodings leave the extraction in the proof of Proposition 5.9(i) valid. At the least $t^* \geq 1$ at which the fold agrees with the genuine chain of that proof, the running values at level $t^* - 1$ differ as elements of $\mathbb{F}_{p_{\text{Pallas}}}$, so no representative of the one equals a representative of the other, and the two 520-bit messages hashed there remain distinct with a common prefix. Parts (ii) and (iii) of the proposition therefore hold for the fold of A3.

Condition A4 serves R4. It is Definition 8.2: the integer $v^{\text{old}} - v^{\text{new}} \in \{-2^{64} + 1, \dots, 2^{64} - 1\}$, written as $\text{sign} \cdot \text{magnitude}$ with $\text{magnitude} \in \{0, \dots, 2^{64} - 1\}$ and $\text{sign} \in \{-1, +1\}$, acts as a scalar modulo p_{Vesta} . The specification requires the scalar multiplication to be correct on this whole signed range, which differs from the range of the balancing value (protocol specification, §“Action Statement (Orchard)”, note on the signed range). The binding signature of “The binding signature” (§8.3) is the check outside the proof that completes value conservation.

Condition A5 serves R3. It is Definition 6.3; the multiplier of K^{Orchard} is the integer in $\{0, \dots, p_{\text{Pallas}} - 1\}$ obtained by reducing the sum modulo p_{Pallas} . The nullifier-set rule of “Nullifier sets” (§6.2) is the check outside the proof that completes it.

Conditions A6 and A7 serve R5. Condition A6 is the re-randomisation of “Randomised validating keys” (§7.2), with $G^{\text{SpendAuth}} = \text{GroupHash}(z.\text{cash}:\text{Orchard}, \mathbb{G})$. Condition A7 recomputes the incoming viewing key as the $\text{Commit}^{\text{ivk}}$ instance of “Viewing keys” (§3.2), under the domain $z.\text{cash}:\text{Orchard}:\text{CommitIvk}$, from the field element $\text{Extract}_p(\text{ak}^P)$, not from the point; it ties the witnessed key components ak^P , nk and r_{ivk} to the expanded receiver $(g_d^{\text{old}}, \text{pk}_d^{\text{old}})$ of the consumed note. Outside its $\perp$ case it forces $\text{ivk} \neq 0$, since $\text{pk}_d^{\text{old}} \neq \mathcal{O}$, and the receiver determines ivk , an integer below $p_{\text{Pallas}} < p_{\text{Vesta}}$, since g_d^{old} is a non-identity point of a group of prime order p_{Vesta} . The tie is the binding of ivk to (ak, nk) (Proposition “Binding of ivk to (ak, nk) ”, §3.2): under Assumptions 2.22 and 2.8, an efficient algorithm outputs two witnesses that satisfy A7 outside its $\perp$ case for one receiver, with distinct triples $(\text{Extract}_p(\text{ak}^P), \text{nk}, r_{\text{ivk}} \bmod p_{\text{Vesta}})$, only with negligible probability. The tie fixes the point ak^P only up to sign (Lemma 2.4); Lemma 12.5 and Theorem 12.9 use it. The spend-authorisation signature under rk , verified outside the proof (§7.2), completes both conditions.

Conditions A8 and A9 have the product form of A3. The values are below $2^{64} < p_{\text{Pallas}}$ and the flags are bits, so A8 holds if and only if $v^{\text{old}} = 0$ or $\text{enableSpend} = 1$, and A9 if and only if $v^{\text{new}} = 0$ or $\text{enableOutput} = 1$, the specification’s forms. They tie the hidden values to the public flags of “Bundle flags” (§9.1).

Condition A10 also has the product form of A3: each of its equations is a public field element times a difference of witnessed values. The four points lie in $\mathcal{E}^*$, where the affine coordinates determine the point. For $\delta = 1$, A10 therefore holds if and only if $(g_d^{\text{new}}, \text{pk}_d^{\text{new}}) = (g_d^{\text{old}}, \text{pk}_d^{\text{old}})$, equality of the expanded receivers as points and not only of their x -coordinates; for $\delta = 0$ it is vacuous. Condition A10 formalises the meaning of $\text{enableCrossAddress}$ stated in the protocol specification, §“Action Descriptions”. Every Orchard-pool Action has $\delta = 1$; an Ironwood-pool Action has $\delta = 1 - \text{enableCrossAddress}$, normally 0 (ZIP 229, Rationale, “enableCrossAddress polarity”). Conditions A8 to A10 serve no requirement of their own.

Remark 9.3 (Meaning of A10). Condition A10 constrains each Action separately: when $\text{disableCrossAddress} = 1$, the note an Action creates is at the expanded receiver of the note that

same Action consumes. The consumed note may be a dummy of value 0, which A3 exempts from membership. Hence a bundle with `disableCrossAddress = 1` can still create a note at any expanded receiver (g_d, pk_d) for which the transaction creator produces a dummy-spend Action in the same bundle. Such an Action has a witness: let its consumed note be a dummy of value 0 at (g_d, pk_d) and its created note, of any value v , be at the same receiver, and let (ak^P, nk, r_{ivk}) be the key components of that receiver. Condition A3 holds because $v^{old} = 0$, A8 for either value of `enableSpends`, A10 with $\delta = 1$ because the two receivers agree, and A9 with `enableOutputs = 1` when $v \neq 0$. Conditions A1, A2, A4 and A5 hold when cm^{old} , cmx , cv^{net} and nf^{old} are computed from the witness by their definitions, and A6 and A7 hold for the key components of the receiver, whose address satisfies $pk_d = [ivk] g_d$ (§3.3). The Action also carries a valid spend-authorisation signature under its `rk`. By Theorem 12.9, in “Authorisation of spends”, for an honestly generated address with `use_qsk = false` this requires that address’s spend authorising key `ask`, except with negligible probability. Consensus enforces no more than this per-Action restriction (ZIP 326, “Rationale for key-generation restrictions”).

Remark 9.4 (Effect on the Orchard pool). For the Orchard pool, whose flag encoding forces `disableCrossAddress = 1`, the pool-level effect is the one that ZIP 229, Abstract, states: outputs to the Orchard pool go to an address for which the transaction creator can authorise spends, which discourages economic activity between users within that pool. By the dummy-spend construction of the preceding remark, consensus does not prevent transfers between users within the Orchard pool (ZIP 326, “Rationale for key-generation restrictions”; ZIP 258, “Consensus rules from NU6.3 activation”, for the rule on Orchard-pool Actions).

Remark 9.5 (Status). The expanded receiver (g_d, pk_d) , the meaning of `enableCrossAddress` that A10 formalises, and the eight-input primary input with `disableCrossAddress = 1 - enableCrossAddress` are specified in the protocol specification, §“Action Descriptions”; the flag encoding in ZIP 229; the Orchard-pool rule and the selection of the verifying key by upgrade in ZIP 258. The specification’s §“Action Statement (Orchard)” and the proof-validity rule of §“Action Descriptions” list the first seven primary inputs and conditions A1 to A9 only; Definition 9.2 follows §“Action Descriptions” for the rest. The specification names the conditions; the numbering A1 to A10 is this volume’s. ZIP 2006, to which the specification refers for the restriction, is a reserved number without content and is not cited for content.

Remark 9.6 (Sign of ak^P). The statement does not constrain the sign of the witnessed point ak^P . Condition A7 reads only $\text{Extract}_P(ak^P)$, which is the same for ak^P and $-ak^P$ (Lemma 2.4), so the even- γ normalisation of key generation (“The spending key and the spend-side secrets”, §3.1) is not enforced by the proof (protocol specification, §“Action Statement (Orchard)”, note on the sign of the spend validating key). Theorem 12.9, in “Authorisation of spends”, treats both signs.

Remark 9.7 ($\perp$ -weakened conditions). Conditions A1, A2, A3 and A7 are the specification’s $\perp$ -weakened forms: each holds when the Sinsemilla instance it evaluates, `NoteCommit`, the hash of `MerkleCRH` or `Commit^{ivk}`, returns $\perp$ on the witnessed input, for A3 on some input of the fold, because the circuit evaluates Sinsemilla with incomplete addition and cannot exclude its exceptional cases (protocol specification, §“Action Statement (Orchard)”, note on $\perp$ outputs and note on the Merkle path validity check; §“Merkle Path Validity”, notes on Orchard). The relation is therefore the disjunction, not the plain conjunction. The weakening

costs a negligible term. A witness that satisfies A1 or A2 only through $\perp$ contains a 1086-bit message, M^{old} or M^{new} , on which $\text{SinsemillaHashToPoint}(\text{z.cash:Orchard-NoteCommit-M}, \cdot)$ returns $\perp$; one that satisfies A3 only through $\perp$ contains a 520-bit message on which $\text{SinsemillaHash}(\text{z.cash:Orchard-MerkleCRH}, \cdot)$ returns $\perp$; one that satisfies A7 only through $\perp$ contains the 510-bit message $\text{LE}_{255}(\text{Extract}_P(\text{ak}^P)) \parallel \text{LE}_{255}(\text{nk})$ with the same property for the domain $\text{z.cash:Orchard-CommitIvk-M}$ (§2.4). An efficient algorithm that runs an adversary together with the extractor of Assumption 9.11 and outputs such a message succeeds with negligible probability by Proposition 2.24(iii), which turns the message into a non-trivial discrete-logarithm relation among the GroupHash generators, under Assumptions 2.22 and 2.8. For the witnesses extracted from accepted proofs, A1, A2, A3 and A7 therefore hold in their unweakened forms, without the $\perp$ case, except with negligible probability.

Remark 9.8 (What the statement does not check).

- (i) No condition involves ask . The point $\text{ak}^P = [\text{ask}] G^{\text{SpendAuth}}$ is computed at key generation, outside the statement, and ask is not a witness. Conditions A6 and A7 relate rk and the address to the witnessed ak^P , and the spend-authorisation signature under rk supplies knowledge of the signing key; Theorem 12.9 combines the two.
- (ii) The trapdoors rcm^{old} and rcm^{new} and the elements ψ^{old} and ψ^{new} are free witnesses: no condition checks that rcm and ψ were derived from rseed as in “The note seed” (§4.3). The sender computes that derivation; the recipient checks it, in “Trial decryption and note acceptance” (§10.4), and consensus checks it only for coinbase outputs (“Chain state and pool rules”, §11.4). Hence, in the deployed protocol, the hashed rcm adds no binding: against an adversary able to compute discrete logarithms among the bases of NoteCommit, NoteCommit is not binding, and such an adversary can open one commitment to two distinct notes (ZIP 2005, Rationale, “Attacks against binding of note commitments”). The volume claims nothing further in that setting.
- (iii) The statement reads none of the Action’s note-encryption fields, which “Note encryption” (§10) constructs; the specification records that the absence of a check on the ephemeral key is intentional (§“Action Statement (Orchard)”, note).

Remark 9.9 (Key source). The statement takes the key components $(\text{ak}^P, \text{nk}, \text{r}_{\text{ivk}})$ as witnesses and checks them only through A5, A6 and A7. It therefore applies equally to key components whose ak^P and r_{ivk} come from the alternative source of ZIP 2005 (“Changes to the Protocol Specification”, §4.2.3 “Orchard Key Components”) cited in “The spending key and the spend-side secrets” (§3.1). The key-derived results of the volume, namely Proposition 3.18, Assumption 3.13, Propositions 3.17, 6.10 and 10.6, and the hypotheses on honest key generation of Theorems 12.9 and 12.12, remain restricted to keys with $\text{use_qsk} = \text{false}$.

Table 5 is the requirement map of the volume. For each requirement of “Requirements on a shielded payment” (§1.4) it lists the soundness content, with the conditions, the checks outside the proof and the results that discharge it, and the hiding content, with the results that discharge it.

Table 5. The requirement map: for each requirement of §1.4, the conditions of Definition 9.2, the checks outside the proof and the results that discharge its soundness part, and the results that discharge its hiding part. It is the only requirement map of the volume.

Requirement	Soundness	Hiding
R1, hidden representation of value	A1 and A2 bind the consumed and the created note to their commitments (Proposition 4.5)	commitment hiding (Propositions 4.5 and 4.8); value-commitment hiding (§8.1); indistinguishability of real and dummy Actions (Theorem 12.12)
R2, membership without identification	A3 with the anchor rule of “Anchors” (Proposition 5.9)	zero knowledge of the proof (Assumption 9.14) with a locally computed authentication path (Lemma 5.15), composed in Theorem 12.12
R3, no second consumption	A5 with the nullifier-set rule (Theorem 12.6)	nullifier unlinkability (Proposition 6.10)
R4, conservation	A4 with the binding signature (Theorem 12.4)	value-commitment hiding (Theorem 12.12)
R5, spend authority	A6 and A7 with the spend-authorisation signature (Theorem 12.9) and bundle binding (Proposition 11.10)	none: R5 has a soundness part only
Flags	A8 to A10 serve no requirement of their own: they tie the hidden values and receivers to the public flags of §9.1	—

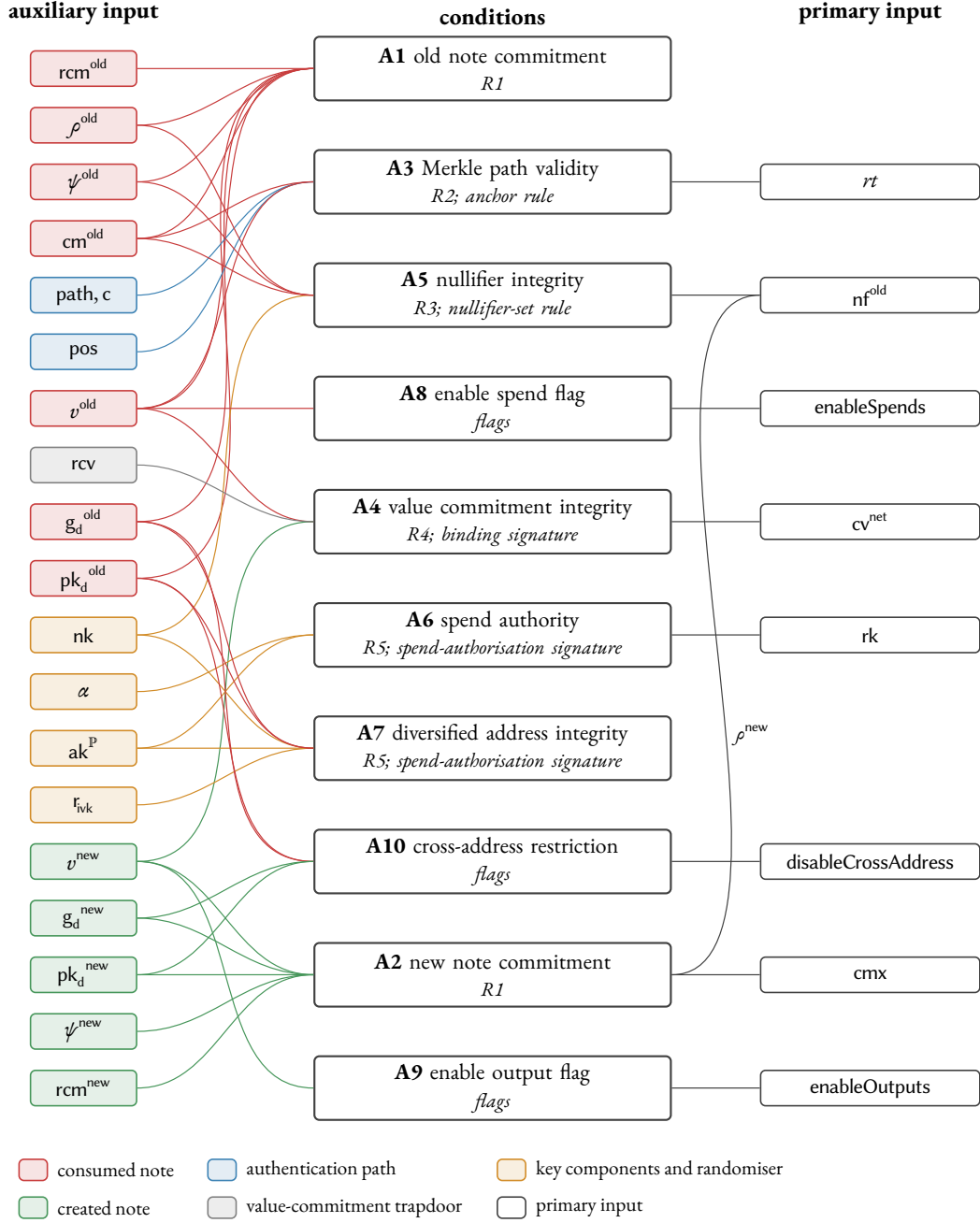


Figure 3. The Action statement of Definition 9.2. The auxiliary input is on the left, coloured by group as in the legend; the primary input is on the right; conditions A1 to A10 are in the centre, ordered by the side of the Action they constrain rather than by number. Each condition has an edge to every variable it reads; the edge from nf^{old} to A2 carries $\rho^{new} := nf^{old}$. Each condition is tagged with the requirement of §1.4 that it serves, or with “flags” for A8 to A10, and, where one exists, with the check outside the proof that completes it: the anchor rule for A3, the nullifier-set rule for A5, the binding signature for A4, and the spend-authorisation signature for A6 and A7.

9.3 The Action circuit and the Halo 2 proof

The Action statement of Definition 9.2 is a relation and prescribes no computation. The *post-NU6.3 Action circuit* is one fixed PLONKish circuit (*Halo 2 Guide*, §“PLONKish arithmetisation”) whose constraints are designed to encode the typing and conditions A1 to A10; that every satisfying assignment yields a witness of $\mathcal{R}_{\text{Action}}$ is part of Assumption 9.11 below. The same circuit serves every Action. Its instance is the encoding of the eight primary inputs of Definition 9.2, bound to the proof as in the *Halo 2 Guide*, §“Instance columns and binding the public statement”; this instance is the only Orchard-specific part of the proof system constructed in this volume. Proofs are Halo 2 proofs (*Halo 2 Guide*, §“The Halo 2 proof system” and §“The complete Halo 2 protocol”; protocol specification, §“Zero-Knowledge Proving System”; ZIP 229, Abstract): a PLONKish polynomial interactive oracle proof compiled with the inner-product polynomial commitment and made non-interactive by the Fiat–Shamir transform, with no trusted setup.

The circuit’s native field is $\mathbb{F}_{p_{\text{Pallas}}}$, the base field of the Pallas group, so the Pallas point operations of A1 to A10 are arithmetic in the native field. The proof’s polynomial commitments lie in the Vesta group G_{Vesta} , the group of points of the curve $y^2 = x^3 + 5$ over $\mathbb{F}_{p_{\text{Vesta}}}$, which has prime order p_{Pallas} (§2.1; *Math Guide*, §“Base fields, scalar fields, and the Pasta cycle”); the committed polynomials therefore have coefficients in $\mathbb{F}_{p_{\text{Pallas}}}$. The Pallas group carries the protocol’s keys, commitments, signatures and note encryption; the Vesta group carries only the proof’s commitments.

The circuit has 2^{11} rows and a constraint-degree bound of 9 across its gates and its permutation and lookup identities (*Halo 2 Guide*, §“From statement to circuit: arithmetisation in practice”). Both are cited constants; in this volume they enter only the Schwartz–Zippel term of the first supporting analysis below.

Every Action of either pool, whatever it consumes or creates, dummy sides included, is an instance of the same relation and circuit. The proofs of a bundle’s Actions are aggregated into one proof, encoded once per bundle, which the verifier checks against the primary inputs of all the bundle’s Actions and the verifying key; each Action carries its own spend-authorisation signature (protocol specification, §“Action Transfers and their Descriptions” and §“Action Descriptions”, note on proof aggregation; ZIP 229, “Transaction Format”). The verification procedure is stated in “Verification of a transaction” (§11.6).

Both pools use the post-NU6.3 circuit and its proving and verifying keys; the pools are distinguished by their note commitment trees, nullifier sets, value pool balances and component position in the transaction, not by circuits (ZIP 229, Abstract, and Rationale, “Reuse of the Orchard protocol with minimal changes”). Consensus selects the Action verifying key by network upgrade (protocol specification, §“Action Descriptions”, consensus rules). The key in force for every Action, in either pool and in version 5 and version 6 transactions alike, is the post-NU6.3 verifying key, named by upgrade and not by pool (ZIP 258, “Consensus rules from NU6.3 activation”).

Assumption 9.10 (Discrete logarithms on Vesta). For every classical probabilistic polynomial-time algorithm, given a generator G of G_{Vesta} and $[s]G$ for s uniform in $\mathbb{F}_{p_{\text{Pallas}}}$, the probability of outputting s is negligible (*Crypto Guide*, §“Standing assumptions”, Remark “Standing assumptions for the sequel”, item 1, for the Vesta curve). Concretely, the best known classical attack costs about 2^{126} group operations (*Crypto Guide*, §“Instantiation on elliptic curves; the Pasta curves”, Proposition “The Pasta curves against the criteria”); the level is cited, not recom-

puted. The volume uses this assumption only in the extraction analysis of the inner-product commitment that supports Assumption 9.11 (the second supporting analysis below).

Assumption 9.11 (Knowledge soundness of the Action proof). Let the hash of the Fiat–Shamir transform of the proof system be modelled as a random oracle. For every efficient adversary $\mathcal{A}$ that outputs primary inputs $\text{pub}_1, \dots, \text{pub}_n$ and an aggregate proof π , there is an efficient extractor $\mathcal{E}$, with access to $\mathcal{A}$ and to its random-oracle queries, that outputs auxiliary inputs $\text{aux}_1, \dots, \text{aux}_n$ such that the probability of the event

$$\begin{aligned} &\pi \text{ is accepted for } (\text{pub}_1, \dots, \text{pub}_n) \text{ under the post-NU6.3 verifying key} \\ &\text{and } (\text{pub}_i, \text{aux}_i) \notin \mathcal{R}_{\text{Action}} \text{ for some } i \end{aligned}$$

is negligible. This is the knowledge-soundness clause of the *Crypto Guide*’s Definition “SNARK” (§“SNARKs: succinct non-interactive arguments of knowledge”) for the relation $\mathcal{R}_{\text{Action}}$ of Definition 9.2. It includes that every satisfying assignment of the post-NU6.3 circuit yields a witness of $\mathcal{R}_{\text{Action}}$, for which the specification requires canonical decompositions of the scalar of A5 and of the scalar ivk in $[\text{ivk}] \text{g}_d^{\text{old}}$ (protocol specification, §“Action Statement (Orchard)”, note on canonical scalar decompositions).

Assumption 9.11 is an assumption, not a proved result. Its status is designed but unspecified: bounds exist for the analysed protocol, and no theorem carries them to the deployed Fiat–Shamir-compiled protocol (*Halo 2 Guide*, §“The algebraic group model”). The volume assumes the property, proves no bound and asserts no loss factor. The two supporting analyses that follow are cited, not re-proved, and not combined into a bound.

Remark 9.12 (Supporting analysis (i): a Schwartz–Zippel term). A non-zero gap polynomial D over $\mathbb{F}_{p_{\text{Pallas}}}$, fixed by the prover’s commitments before the challenge is drawn, vanishes at a uniform challenge with probability at most $\deg D / p_{\text{Pallas}}$ (*Halo 2 Guide*, §“Why checking one random point is a proof”). At the Orchard parameters, 2^{11} rows and constraint-degree bound 9, the constraint expression has degree below $9 \cdot 2^{11}$, and so does the committed quotient, of 8 chunks of degree below 2^{11} , times the vanishing polynomial of degree 2^{11} (*Halo 2 Guide*, §“The complete Halo 2 protocol”, Phase 4). Hence $\deg D < 9 \cdot 2^{11} = 18432$, and the term is below $18432 / p_{\text{Pallas}} < 2^{-239}$. The term bounds one check at one challenge and is not a security level of the proof: the complete protocol has further randomised checks, and under the Fiat–Shamir transform an adversary may retry challenges, one hash query per retry (*Halo 2 Guide*, §“Removing the verifier: the Fiat–Shamir transform”).

Remark 9.13 (Supporting analysis (ii): extraction for the inner-product commitment). The inner-product polynomial commitment, whose commitments lie in $\mathbb{G}_{\text{Vesta}}$, has an interactive opening protocol that is knowledge-sound under the discrete-logarithm assumption in that group, Assumption 9.10, with an expected-polynomial-time tree extractor (*Halo 2 Guide*, §“The inner-product polynomial commitment”, Theorem “Knowledge soundness of the opening protocol”). The Fiat–Shamir-compiled argument inherits the interactive extraction bound only up to a loss in the adversary’s random-oracle query count (§“From the interactive extractor to the non-interactive argument”), and the tighter analyses in the algebraic group model are proved for

the analysed protocol, not for the deployed transcript (§“The algebraic group model”). No loss factor is asserted.

Assumption 9.14 (Zero knowledge of the Action proof). Let the hash of the Fiat–Shamir transform be modelled as a programmable random oracle. There is an efficient simulator that, given only the primary inputs $\text{pub}_1, \dots, \text{pub}_n$ of a bundle and programming the random oracle, outputs an aggregate proof whose distribution is statistically indistinguishable from that of an honestly generated proof for any $\text{aux}_1, \dots, \text{aux}_n$ with $(\text{pub}_i, \text{aux}_i) \in \mathcal{R}_{\text{Action}}$ for every i : every distinguisher that makes polynomially many random-oracle queries, whatever its running time, has negligible advantage. This is zero knowledge of statistical grade in the non-interactive, random-oracle form of the *Crypto Guide*’s Definition “Zero knowledge” (§“The simulation paradigm and zero knowledge”).

Assumption 9.14 is an assumption, not a proved result. The *Halo 2 Guide*, §“Zero knowledge: hiding the witness in Halo 2”, records honest-verifier zero knowledge of the interactive protocol and zero knowledge of its Fiat–Shamir compilation in the programmable random-oracle model, subject to its composition and Fiat–Shamir hypotheses, without reproving it. Assumptions 9.11 and 9.14 are the two assumptions on the Action proof that “Security” (§12) composes with the propositions of the preceding sections.

10 Note encryption

This section constructs the in-band delivery of a created note: its plaintext, its encryption to the recipient by one-shot Diffie–Hellman on the diversified base, the outgoing ciphertext keyed from ovk and the Action’s own public fields, and the trial decryption and acceptance checks by which a recipient obtains a note consistent with the published cmx . It states the two assumptions that note encryption adds, proves decryption correctness, confidentiality and key privacy of the encryption to the recipient, confidentiality of the outgoing ciphertext, and the consistency of accepted notes, and states what each key tier can decrypt.

10.1 The note plaintext

Every Action carries, beside the extracted commitment cmx of its created note, a *note ciphertext* $(\text{epk}^*, C_{\text{enc}}, C_{\text{out}})$. The component C_{enc} encrypts the plaintext of the created note so that the holder of the recipient’s incoming viewing key ivk recovers the note and its memo from the chain alone (protocol specification, §“In-band secret distribution (Sapling and Orchard)” and §“Action Descriptions”). The component C_{out} lets the sender, or any party that holds the outgoing viewing key ovk under which it was formed, recover the same plaintext (§10.3). The fields epk^* , C_{enc} and C_{out} are not inputs of the Action statement (Definition 9.2), which therefore does not constrain them; the checks that make a decrypted note consistent with cmx are made by the recipient (§10.4).

Definition 10.1 (Note plaintext). Let a created note have recipient address (d, pk_d) and value $v \in \{0, \dots, 2^{64} - 1\}$. Its *note plaintext* is the tuple $\text{np} := (\text{leadByte}, d, v, \text{rseed}, \text{memo})$, encoded

as the byte string

$$\text{leadByte} \parallel \text{LEBS2OSP}_{88}(d) \parallel \text{I2LEOSP}_{64}(v) \parallel \text{rseed} \parallel \text{memo},$$

where leadByte is one byte, the format version of “The note seed” (§4.3), equal to $0x03$ for an Ironwood-pool output; d is the 88-bit diversifier of the address (11 bytes); v occupies 8 bytes; rseed is the 32-byte note seed; and memo is a 512-byte string chosen by the sender, whose use is by agreement between sender and recipient (protocol specification, §“Note Plaintexts and Memo Fields” and §“Encodings of Note Plaintexts and Memo Fields”). The encoding has the fixed length $1 + 11 + 8 + 32 + 512 = 564$ bytes, so the length of a ciphertext carries no information about the plaintext.

The plaintext omits pk_d, ρ, ψ and rcm . The recipient computes $\text{pk}_d = [\text{ivk}] g_d$, takes $\rho = \text{nf}^{\text{old}}$ from the same Action (“Nullifier chaining”, §6.3), and derives ψ and rcm from rseed (§4.3).

For each output the sender draws rseed uniformly from the 32-byte strings, independently of every other output. The ephemeral secret key of this section is

$$\text{esk} = \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}([0x04] \parallel \text{I2LEOSP}_{256}(\text{nf}^{\text{old}}))),$$

derived in “The note seed” (§4.3) together with ψ and rcm . The sender redraws rseed when $\text{esk} = 0$ or $\text{cmx} = \perp$, so $\text{esk} \in \mathbb{F}_{\mathcal{P}_{\text{Vesta}}}^*$ (protocol specification, §“Sending Notes (Orchard)”; ZIP 2005, “Changes to the Protocol Specification”, §4.7.3).

10.2 Encryption to the recipient

Construction 10.2 (Encryption to the recipient). Let (d, pk_d) be the recipient’s address, $g_d = \text{DiversifyHash}(d)$ its diversified base (“Diversified addresses”, §3.3), and esk as in §10.1. A star encoding P^* enters a byte string as the 32 bytes $\text{LEBS2OSP}_{256}(P^*)$ (§2.1), with which it is identified below. The sender computes the *ephemeral public key*, the *shared secret* and the *encryption key*

$$\begin{aligned} \text{epk} &:= [\text{esk}] g_d, & s &:= [\text{esk}] \text{pk}_d, \\ K_{\text{enc}} &:= \text{KDF}(s, \text{epk}) := \text{BLAKE2b-256}(\text{Zcash_OrchardKDF}, s^* \parallel \text{epk}^*), \end{aligned}$$

with $\text{BLAKE2b-256}(P, x)$ unkeyed BLAKE2b with a 32-byte output and the 16-byte personalisation P , and the ciphertext $C_{\text{enc}} := \text{Enc}_{K_{\text{enc}}}(\text{np})$. The pair $\text{Enc}_K, \text{Dec}_K$ denotes ChaCha20-Poly1305 encryption and decryption under the 256-bit key K with the all-zero 96-bit nonce and empty associated data; decryption returns $\perp$ when the tag does not verify (*Crypto Guide*, §“The ChaCha20-Poly1305 AEAD”, Construction “ChaCha20-Poly1305”). The Action publishes the 32-byte field epk^* and C_{enc} , of $564 + 16 = 580$ bytes, the plaintext and the 16-byte tag (protocol specification, §“Encryption (Sapling and Orchard)”, §“Orchard Key Agreement”, §“Orchard Key Derivation” and §“Symmetric Encryption”).

The key agreement is Diffie–Hellman on Pallas: public keys are non-identity points, private keys lie in $\mathbb{F}_{\mathcal{P}_{\text{Vesta}}}^*$, and both the derivation of a public key from a base and the agreement are scalar multiplication (protocol specification, §“Orchard Key Agreement”). A consensus rule requires epk^*

to be the canonical encoding of a non-identity Pallas point (protocol specification, §“Action Descriptions”). The fixed nonce is safe because every key encrypts once: K_{enc} depends on the fresh esk of its output, and the key of C_{out} is derived separately (§10.3). The scheme is the *Crypto Guide*’s Construction “DH-based hybrid public-key encryption” (§“Hybrid public-key encryption”) over the per-address base g_d in place of a fixed generator, as the same volume records for deployed note delivery (§“Shielded-note delivery as deployed hybrid encryption”).

Proposition 10.3 (Decryption correctness). *Let (d, pk_d) be an address of the incoming viewing key ivk , so $\text{pk}_d = [\text{ivk}] g_d$ (§3.3), and let $\text{epk} = [\text{esk}] g_d$ with $\text{esk} \in \mathbb{F}_{p_{\text{Vesta}}}^*$. Then*

$$[\text{ivk}] \text{epk} = [\text{esk}] \text{pk}_d = s.$$

Hence the holder of ivk computes $K_{\text{enc}} = \text{KDF}([\text{ivk}] \text{epk}, \text{epk})$ from the published epk^ and obtains $\text{Dec}_{K_{\text{enc}}}(C_{\text{enc}}) = \text{np}$, and a party that knows $(\text{pk}_d, \text{esk})$ obtains the same shared secret as $[\text{esk}] \text{pk}_d$.*

Proof. In the cyclic Pallas group $[a]([b]P) = [ab]P = [b]([a]P)$, so $[\text{ivk}]([\text{esk}] g_d) = [\text{esk}]([\text{ivk}] g_d) = [\text{esk}] \text{pk}_d$. The recipient decodes $\text{epk} = \text{abst}_p(\text{epk}^*)$, and because the star encoding is canonical (§2.1) it re-encodes to the published bytes. The function KDF is deterministic and both parties apply it to (s^*, epk^*) , so they derive the same K_{enc} ; correctness of ChaCha20-Poly1305 gives $\text{Dec}_{K_{\text{enc}}}(\text{Enc}_{K_{\text{enc}}}(\text{np})) = \text{np}$. This is the *Crypto Guide*’s Theorem “Correctness of hybrid encryption” (§“Hybrid public-key encryption”) over the base g_d . $\square$

Assumption 10.4 (The note-encryption hashes as random oracles). In security arguments the key-derivation function KDF (BLAKE2b-256 personalised `Zcash_OrchardKDF`), the function PRF^{ock} (BLAKE2b-256 personalised `Zcash_Orchardock`, constructed in §10.3), and PRFexpand on inputs with leading byte `0x04`, the derivation of esk from rseed , are modelled as independent random oracles (*Crypto Guide*, §“The random oracle model”, Definition “Random oracle”). The PRF assumption (Assumption 2.12) does not suffice for the last: it requires a key unknown to the adversary, whereas the plaintext that K_{enc} encrypts contains rseed , the key from which esk , and hence K_{enc} , is derived. On its other inputs PRFexpand is used only under Assumption 2.12. The assumption is a heuristic about BLAKE2b, not a theorem; every result that uses KDF , PRF^{ock} or the derivation of esk names it.

Assumption 10.5 (Confidentiality and integrity of ChaCha20-Poly1305). For a uniform 256-bit key that encrypts a single plaintext, ChaCha20-Poly1305 with the all-zero nonce and empty associated data is

- (a) *confidential*: the encryptions of any two plaintexts of equal length are computationally indistinguishable; and
- (b) *integral*: no efficient adversary without the key outputs a ciphertext, other than one it was given, that decrypts to a value other than $\perp$.

The *Crypto Guide*’s Theorem “Security of ChaCha20-Poly1305” (§“The ChaCha20-Poly1305 AEAD”) derives both, with explicit bounds, from the pseudorandomness of ChaCha20. The protocol specification requires one-time INT-CTXT and IND-CPA security of the scheme,

properties (b) and (a) above (*Crypto Guide*, Definitions “Ciphertext integrity, INT-CTXT” and “CPA security”, §“Authenticated encryption with associated data” and §“Chosen-plaintext security”; protocol specification, §“Symmetric Encryption”). The scheme is not key-committing: a party that chooses two keys can construct one ciphertext that decrypts validly under both (*Crypto Guide*, §“The ChaCha20-Poly1305 AEAD”, Remark “Wrong-key rejection and key commitment”). No result of this volume relies on the tag against a sender who chooses the key.

Proposition 10.6 (Confidentiality and key privacy of note encryption). *Let a key be generated honestly with $\text{use_sk} = \text{false}$. An efficient adversary receives the address (d, pk_d) of the key at an index of its choice and data z that an efficient algorithm computes independently of the key and of rseed ; it chooses a value v , a memo and $\rho \in \mathbb{F}_{\text{Pallas}}$. An output to (d, pk_d) with these values and $\rho = \text{nf}^{\text{old}}$ is formed as in §10.1 and the construction above, with rseed uniform, and cmx is the extracted commitment of its note. Then the triple $(\text{epk}^*, C_{\text{enc}}, \text{cmx})$ is computationally indistinguishable from (E^*, C', cmx) , where E is a uniform non-identity Pallas point and C' is the encryption of a fixed 564-byte string under an independent uniform key. The hypotheses are Assumptions 3.16, 10.4, 10.5, 2.8 (for g_d), 3.13 (for the joint distribution of ivk and d) and 2.12 (for the dependence of cmx on rseed), together with Assumption 2.22 for Lemma “Rejection in key generation” (§3.2).*

Since E and C' are computed from neither the plaintext nor the address, the pair $(\text{epk}^*, C_{\text{enc}})$ reveals neither the plaintext, rseed included, nor the recipient’s address: given two addresses of independent keys, the data z may contain the second, and the ciphertext is indistinguishable from one independent of both. The value cmx is included so that the result applies within an Action. Of the hypotheses only Assumption 3.16, used through the computational Diffie–Hellman problem that it implies, concerns the one-shot key agreement between esk and pk_d ; Assumptions 10.4 and 10.5 concern the hash functions and the cipher. The adversary is passive: the volume does not claim indistinguishability of encryptions against an adversary that may also obtain decryptions of ciphertexts of its choice other than the challenge, before and after receiving it (IND-CCA2), which the protocol specification, §“Key Derivation”, requires of the scheme.

Proof. The proof is a sequence of games (*Crypto Guide*, §“Security as a game”, Remark “Game-hopping”); each reduction below runs the adversary and the rest of the game. Let Q be the event that the adversary queries the oracle of Assumption 10.4 for PRFexpand at an input keyed by rseed .

Preliminary step. As in the proof of Proposition 3.18, the key is replaced by a single draw without rejection and its triple $(\text{ask}, \text{nk}, \text{r}_{\text{ivk}})$ by independent uniform elements (Lemma 2.15, Assumption 2.12; Lemma “Rejection in key generation”, §3.2, Assumptions 2.22 and 2.8). Then (ivk, dk) is replaced by $(\text{Commit}_{\text{ivk}}^{\text{ivk}}, (\text{ak}, \text{nk}), u_1)$ (Assumption 3.13; the reduction draws ask and nk itself). Now $d = \text{FF1-AES256}_{u_1}(\text{LE}_{\text{gg}}(j))$, and ivk is, independently of d , 0 with probability $1/p_{\text{Vesta}}$ and otherwise uniform on the set X_{Pallas} of x -coordinates of non-identity points, of $(p_{\text{Vesta}} - 1)/2$ elements, except on the negligible event $\hat{M} = \perp$ (Lemma “Distribution of ivk ”, §3.2). The redraw of rseed is replaced by a single trial, at the cost of the redraw probability, negligible as in the proof of Proposition 4.8.

(1) By Assumption 10.4, esk is ToScalar of a random-oracle output at an input keyed by rseed . It is replaced by an independent uniform element of $\mathbb{F}_{p_{\text{Vesta}}}^*$. The oracle’s value at that input is used

nowhere else, so the games are identical until Q occurs; they differ by at most $\Pr[Q]$ in the new game plus the reduction distance of ToScalar, below 2^{-257} , and $1/p_{\text{Vesta}}$ for the value 0 (*Crypto Guide*, §“Security as a game”, Remark “Game-hopping”; §2.3).

(2) The key K_{enc} is replaced by an independent uniform key. The games differ only on the event C that the adversary queries the KDF oracle at $s^* \parallel \text{epk}^*$, and $s = [\text{esk}] \text{pk}_d$ solves the computational Diffie–Hellman instance $(g_d, \text{pk}_d, \text{epk})$. This is the game hop of the *Crypto Guide*’s Theorem “Passive security of hybrid encryption” (§“Hybrid public-key encryption”), with loss factor the number q_K of KDF queries, run over g_d . A distinguisher $\mathcal{D}$ for Assumption 3.16 receives $([a] P, [b] P, [c] P)$ with a uniform on X_{Pallas} and b, c uniform. It answers each new query of GroupHash under the domain `z.cash:0rchain-gd` with $[t] P$ for a fresh uniform t , recorded, a uniform point as in the model of Assumption 2.8, so that $g_d = [t] P$ with t known. It sets $\text{pk}_d := [t]([a] P) = [a] g_d$, which embeds a as ivk , and $\text{epk} := [t]([b] P) = [b] g_d$, which embeds b as esk ; it draws u_1 and rseed , computes d , cmx and C_{enc} under a uniform key itself, and answers the other oracles lazily. It then decodes the first 32 bytes of a uniformly chosen KDF query to a point S and outputs 1 exactly when $S = [t]([c] P)$. The simulation differs from the game only when $\text{ivk} = 0$ in the game, or $b = 0$ or $t = 0$ in the simulation, of total probability at most $3/p_{\text{Vesta}}$. On a Diffie–Hellman triple $\mathcal{D}$ therefore outputs 1 with probability at least $\Pr[C]/q_K - 3/p_{\text{Vesta}}$; on a random triple the point $[t]([c] P)$ is, for $t \neq 0$, uniform and independent of the view, and $\mathcal{D}$ outputs 1 with probability at most $4/p_{\text{Vesta}}$. An algorithm that computes Diffie–Hellman secrets on Pallas thus decides DDH, and $\Pr[C] \leq q_K (\epsilon_D + 7/p_{\text{Vesta}})$ for the advantage ϵ_D of $\mathcal{D}$. The exponent embedded as ivk is uniform on X_{Pallas} in Assumption 3.16. Against the computational Diffie–Hellman problem with a uniform exponent on $\mathbb{F}_{p_{\text{Vesta}}}$, which lies in X_{Pallas} with probability $(p_{\text{Vesta}} - 1)/(2p_{\text{Vesta}})$ and on those instances gives an exact simulation, the same reduction loses a factor $2p_{\text{Vesta}}/(p_{\text{Vesta}} - 1)$, about 2 (proof of Proposition 3.18, part (d)).

(3) Now $\text{epk} = [\text{esk}] g_d$ with esk uniform on $\mathbb{F}_{p_{\text{Vesta}}}^*$ is a uniform non-identity point for every generator g_d , and the KDF context contains no key of the recipient (*Crypto Guide*, §“Key privacy”, Remark “Key privacy of note encryption and diversified addresses”). The ciphertext C_{enc} is an encryption under a uniform key used once, and Assumption 10.5(a) replaces np by the fixed string; the reduction obtains C_{enc} from its challenger and computes the rest. The last game outputs (E^*, C', cmx) .

(4) In the last game rseed enters the view only through cmx , that is through $\text{PRFexpand}_{\text{rseed}}$ under the leading bytes `0x09` and `0x0B`. A distinguisher with oracle access to $\text{PRFexpand}_{\text{rseed}}$ or to a random function computes cmx from its oracle, runs the adversary, and outputs 1 when some key k of a query at a leading byte `0x04` satisfies $\text{PRFexpand}_k([0x09] \parallel \text{l2LEOSP}_{256}(\rho))$ equal to its oracle’s answer at that input. It outputs 1 with probability at least $\Pr[Q]$ in the first case. In the second the view depends on that answer only through cmx , and the trapdoor rcm , ToScalar of the independent answer at the `0x0B` input, is within 2^{-257} of uniform; by Proposition 4.5(a), cmx is therefore independent of that answer up to $2^{-257} + \epsilon_{\perp}$, for $\epsilon_{\perp}$ the probability that the hash point of $\mathcal{M}(n)$ is $\perp$, negligible as in the proof of Proposition 4.8. The distinguisher thus outputs 1 with probability at most $q \cdot 2^{-512} + 2^{-257} + \epsilon_{\perp}$, for q such queries, and $\Pr[Q]$ in the last game is negligible under Assumption 2.12 (with Lemma 2.15). Every reduction of hops (2) and (3) draws rseed itself and detects Q , so each hop changes the probability of Q by at most its own cost. Hence $\Pr[Q]$ is negligible in the game of hop (1) as well, and the sum of the costs of all hops is negligible.

The DDH hybrid of the *Crypto Guide*'s Theorem “ElGamal is key-private under DDH” (§“Key privacy”) is not used: as stated it embeds a uniform exponent of $\mathbb{F}_{p_{\text{Vesta}}}$ as the recipient's key, whereas ivk is not uniform on $\mathbb{F}_{p_{\text{Vesta}}}$; and the factor-2 argument above transfers a search problem such as computational Diffie–Hellman, not a decision problem, since on instances outside X_{Pallas} a distinguisher's output is unconstrained. $\square$

10.3 The outgoing ciphertext

Construction 10.7 (Outgoing ciphertext). The sender chooses an outgoing viewing key ovk of its own, such as that of the address whose note the Action consumes (“Viewing keys”, §3.2), or $\text{ovk} = \perp$. If $\text{ovk} \neq \perp$, the *outgoing cipher key* is

$$\begin{aligned} \text{ock} &:= \text{PRF}_{\text{ovk}}^{\text{ock}}(\text{cv}^{\text{net}}, \text{cmx}, \text{epk}) \\ &:= \text{BLAKE2b-256}(\text{Zcash_Orchardock}, \text{ovk} \parallel (\text{cv}^{\text{net}})^* \parallel \text{I2LEOSP}_{256}(\text{cmx}) \parallel \text{epk}^*), \end{aligned}$$

where cv^{net} is the Action's net value commitment (Definition 8.2) and cmx its extracted note commitment (Definition 4.4), and the *outgoing plaintext* is the $32 + 32 = 64$ -byte string $\text{op} := \text{pk}_d^* \parallel \text{I2LEOSP}_{256}(\text{esk})$. If $\text{ovk} = \perp$, the sender draws ock uniformly from the 256-bit strings and op uniformly from the 64-byte strings. In both cases $C_{\text{out}} := \text{Enc}_{\text{ock}}(\text{op})$, of $64 + 16 = 80$ bytes, and the Action publishes it; with $\text{ovk} = \perp$ no outgoing viewing key recovers it (protocol specification, §“Encryption (Sapling and Orchard)”, §“Sending Notes (Orchard)” and §“Pseudo Random Functions”).

Construction 10.8 (Recovery with ovk). For an Ironwood-pool output, the holder of ovk

- (i) computes ock from ovk and the Action's public cv^{net} , cmx and epk^* , and decrypts C_{out} under ock to the outgoing plaintext op , rejecting $\perp$;
- (ii) parses op as a 32-byte string b_1 followed by a 32-byte string b_2 , and reads esk as the integer $\text{LEOSP}_{256}(b_2)$, rejecting $\text{esk} \geq p_{\text{Vesta}}$, and pk_d as the point $\text{abst}_p(b_1)$, rejecting $\perp$ and $\mathcal{O}$;
- (iii) computes $s := [\text{esk}] \text{pk}_d$, $K_{\text{enc}} := \text{KDF}(s, \text{epk})$ and $\text{np} := \text{Dec}_{K_{\text{enc}}}(C_{\text{enc}})$, rejecting $\perp$;
- (iv) rejects unless $\text{leadByte} = 0x03$;
- (v) with $\rho := \text{nf}^{\text{old}}$ of the same Action, rejects unless

$$\text{esk} = \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}([0x04] \parallel \text{I2LEOSP}_{256}(\rho)));$$

- (vi) computes $\text{g}_d := \text{DiversifyHash}(d)$, then ψ , then rcm under $0x03$ from g_d , pk_d , v , ρ and ψ , by steps (iv) and (v) of the derivation of §4.3;
- (vii) rejects if the note commitment of $((d, \text{pk}_d), v, \rho, \psi, \text{rcm})$ (Definition 4.4) is $\perp$ or its x -coordinate differs from cmx , or if $([\text{esk}] \text{g}_d)^* \neq \text{epk}^*$;
- (viii) otherwise returns the note $((d, \text{pk}_d), v, \rho, \psi, \text{rcm})$ and memo.

The order g_d and pk_d , then ψ , then rcm is forced because rcm depends on all three. The specification's decryption sections still use the legacy derivation of rcm ; ZIP 2005 specifies the reordered procedure (protocol specification, §“Decryption using an Outgoing Viewing Key (Sapling and Orchard)”); ZIP 2005, “Changes to the Protocol Specification”, §4.20.2 and §4.20.3). The specification's further check that b_1 re-encodes pk_d holds for every string that $abst_p$ decodes, the star encoding being canonical (§2.1). For pk_d the specification's procedure rejects only $\perp$; the rejection of $\mathcal{O}$ in step (ii) makes the returned pk_d a transmission key of Definition “Note” (§4.1).

Remark 10.9 (Transplant resistance). The key ock is a function of the Action's own cv^{net} , cmx and epk . A ciphertext C_{out} that a party without ovk copies into an Action whose (cv^{net}, cmx, epk) differ is decrypted by the holder of ovk under a key that, by Assumption 10.4, is uniform and independent of the copied ciphertext; by Assumption 10.5(b) that decryption returns $\perp$ except with negligible probability. A copied C_{out} is therefore rejected, not attributed to the other Action.

Proposition 10.10 (Confidentiality of the outgoing ciphertext). *Let the sending key be generated honestly with $use_qsk = \text{false}$. To an efficient party that holds neither its ovk nor its r_{ivk} , and may hold its ak , nk , ivk and dk , each ciphertext C_{out} that the holder of the key forms is computationally indistinguishable, jointly with every other public field of its Action, from the encryption of a fixed 64-byte string under an independent uniform key. The hypotheses are Assumptions 2.12, 3.13, 10.4, 10.5, 9.14, 2.22 and 2.8.*

Proof. As in the preliminary step of the proof of Proposition 10.6, the key is replaced by a single draw without rejection and its triple (ask, nk, r_{ivk}) by independent uniform elements (Lemma 2.15, Assumption 2.12; Lemma “Rejection in key generation”, §3.2, Assumptions 2.22 and 2.8). After this step, Assumption 3.13 makes ovk indistinguishable from a uniform 32-byte string independent of ivk and dk ; the reduction draws ask and nk itself, obtains (ivk, dk, ovk) from its challenger, computes every other field, and simulates the Action proof, whose witness contains r_{ivk} , under Assumption 9.14. Then ock is a random-oracle output (Assumption 10.4) at an input that the party queries with negligible probability, hence uniform, and distinct Actions give distinct inputs, their epk differing except with negligible probability, so each ock encrypts once; Assumption 10.5(a) then applies. With $ovk = \perp$, ock is uniform by construction. The ciphertext C_{out} thus reveals nothing about (pk_d, esk) . $\square$

A holder of ovk recovers, for every output sent under that ovk , the pair (pk_d, esk) , hence the shared secret $[esk] pk_d$ (Proposition 10.3), K_{enc} , the note plaintext and the recipient's address (d, pk_d) ; in this way the sender, or a party to whom the sender discloses ovk , reads outgoing notes (protocol specification, §“Decryption using an Outgoing Viewing Key (Sapling and Orchard)”). By this route it reads only outputs sent under that ovk . Detecting the notes sent to the key's own addresses by trial decryption requires ivk (§10.4), and ovk yields no ivk (Proposition 3.18, “Capability separation”, part (c), for keys generated with $use_qsk = \text{false}$).

10.4 Trial decryption and note acceptance

Construction 10.11 (Trial decryption). No public field of an Action names its recipient, so the holder of an incoming viewing key attempts every output: it computes $\text{epk} := \text{abst}_p(\text{epk}^*)$, rejecting $\perp$, which consensus excludes, then $s := [\text{ivk}] \text{epk}$, $K_{\text{enc}} := \text{KDF}(s, \text{epk})$ and $\text{np} := \text{Dec}_{K_{\text{enc}}}(C_{\text{enc}})$ (protocol specification, §“Decryption using an Incoming Viewing Key (Sapling and Orchard)”). A result other than $\perp$ is a *candidate*, not an accepted note.

The same ivk serves the outputs of both pools, since keys and addresses are defined for the Orchard protocol, not for a pool (§3.4), and an address does not name a pool (ZIP 326, “Scanning for Ironwood-pool and Orchard-pool notes”); the pool of an output is that of the bundle that carries it. For an honestly produced ciphertext under a key independent of the trial key, decryption returns $\perp$ except with negligible probability (Assumption 10.5(b); *Crypto Guide*, §“The ChaCha20-Poly1305 AEAD”, Remark “Wrong-key rejection and key commitment”); against a sender who chooses the keys the tag gives no such guarantee.

Construction 10.12 (Acceptance of an Ironwood-pool output). Given a candidate $\text{np} = (\text{leadByte}, d, v, \text{rseed}, \text{memo})$ of an Ironwood-pool output, the recipient accepts only if every check passes:

- (i) $\text{leadByte} = 0x03$, the recipient’s enforcement of the lead-byte rule of “The note seed” (§4.3);
- (ii) with $\rho := \text{nf}^{\text{old}}$ of the same Action and $\underline{\rho} := \text{l2LEOSP}_{256}(\rho)$, the note is re-derived in the order

$$\begin{aligned} g_d &:= \text{DiversifyHash}(d), & \text{pk}_d &:= [\text{ivk}] g_d, \\ \psi &:= \text{ToBase}(\text{PRFexpand}_{\text{rseed}}([0x09] \parallel \underline{\rho})), \\ \text{rcm} &:= \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}(\text{pre_rcm})), \end{aligned}$$

with the 137-byte string pre_rcm of §4.3 formed from g_d , pk_d , v , ρ and ψ ;

- (iii) $([\text{esk}] g_d)^* = \text{epk}^*$ for

$$\text{esk} := \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}([0x04] \parallel \underline{\rho})),$$

a check that ZIP 2005 places directly after the computation of g_d ;

- (iv) the note commitment cm' of the re-derived note (Definition 4.4) is not $\perp$ and $\text{Extract}_p(\text{cm}') = \text{cmx}$.

The accepted note is $n = ((d, \text{pk}_d), v, \rho, \psi, \text{rcm})$, with memo (ZIP 2005, “Changes to the Protocol Specification”, §4.20.2 and §4.20.3; protocol specification, §“Decryption using an Incoming Viewing Key (Sapling and Orchard)” and §“Note Plaintexts and Memo Fields”).

The order in (ii) is forced because rcm depends on g_d , pk_d and ψ ; the recovery with ovk of §10.3 follows the same order. The specification’s decryption sections still use the legacy derivation of

rcm; ZIP 2005 specifies both reordered procedures. The specification's check $\text{rcm} < p_{\text{Vesta}}$ holds for every output of ToScalar.

Remark 10.13 (Role of check (iii)). The Action statement (Definition 9.2) does not constrain epk. Check (iii) makes the recipient accept only an epk formed on the base g_d of the diversifier in the plaintext. Without it a sender could agree the key with one address of a recipient while naming the diversifier of another, and link the two addresses by observing whether the recipient accepts (ZIP 212, “Motivation”).

Remark 10.14 (Where the Ironwood derivation is checked). The Action statement takes ψ and rcm as witnesses and does not check their derivation from rseed (Definition 9.2). An accepted Ironwood-pool note has, by checks (i), (ii) and (iv), ψ and rcm derived under $0x03$ and the commitment cmx. For a coinbase output, consensus runs the recovery with ovk of §10.3 under the all-zero ovk (“Chain state and pool rules”, §11.4), which checks the lead byte and the whole $0x03$ derivation. For every other output, these recipient checks are the only check of the $0x03$ note format.

Orchard-pool outputs carry lead byte $0x02$ and use the legacy trapdoor derivation of the protocol specification, §“Sending Notes (Orchard)”; the recipient decrypts them with the same ivk under that byte and derivation (§“Note Plaintexts and Memo Fields”), on which this volume does not rely.

Proposition 10.15 (Consistency of accepted notes). *Let the acceptance procedure accept an Ironwood-pool output with note $n = ((d, \text{pk}_d), v, \rho, \psi, \text{rcm})$.*

- (a) *The note n is addressed to $(d, [\text{ivk}] g_d)$, an address of the recipient's key; it has $\rho = \text{nf}^{\text{old}}$ of the same Action; and its ψ and rcm are derived from its rseed under $0x03$.*
- (b) *The extracted commitment of n satisfies*

$$\text{Extract}_p(\text{NoteCommit}_{\text{rcm}}(M(n))) = \text{cmx}.$$

- (c) *For every efficient sender, which may choose epk^* , C_{enc} , C_{out} and the keys under which they are formed, the probability that the recipient accepts n while the sender outputs a note $n' \neq n$ with $\text{cmx}(n') = \text{cmx}$ is negligible, under the binding of note commitments (Proposition 4.5), hence under Assumptions 2.22 and 2.8. In particular, under Assumption 9.11, the committed values and the trapdoor of the created note that an extractor recovers from the Action proof, which satisfy condition A2 of the Action statement (Definition 9.2), are $(g_d, \text{pk}_d, v, \rho, \psi)$ and rcm modulo p_{Vesta} , those of n , except with negligible probability.*

Proof. Parts (a) and (b) hold by construction: acceptance requires checks (i), (ii) and (iv), and check (iv) recomputes the commitment of n deterministically and compares its x -coordinate with cmx, whatever the ciphertext and whichever key decrypted it. The address $(d, [\text{ivk}] g_d)$ is an address of the key because FF1-AES256_{dk} is a permutation of the 88-bit strings, so every d is the diversifier of one index (§3.3).

(c) Consider the algorithm that generates the recipient's key, runs the sender, runs the acceptance procedure on its output, and outputs n and the sender's n' . When the recipient

accepts, n opens $\text{cmx} \neq \perp$ by (b), so an $n' \neq n$ with $\text{cmx}(n') = \text{cmx}$ gives two distinct notes with equal extracted commitments other than $\perp$, which Proposition 4.5(b) excludes except with negligible probability. For the particular case, the extractor of Assumption 9.11 run on the sender is such a sender-side algorithm, with an opening of the committed values and a trapdoor in place of a note. Outside the negligible $\perp$ -weakened case of condition A2, which yields an input on which NoteCommit returns $\perp$ (Proposition 2.24(iii)), its opening (M', rcm') satisfies $\text{Extract}_p(\text{NoteCommit}_{\text{rcm}'}(M')) = \text{cmx}$. Step 2 of the proof of Proposition 4.5(b), which uses Proposition 2.24(ii) and the fact that $H_D \neq \mathcal{O}$, then gives $M' = M(n)$ and $\text{rcm}' \equiv \text{rcm} \pmod{p_{\text{Vesta}}}$ except with negligible probability, and $M(n)$ determines $(g_d, \text{pk}_d, v, \rho, \psi)$ because the encoding is injective. The proof uses neither the integrity of ChaCha20-Poly1305 nor key commitment, which the scheme lacks (Assumption 10.5): a ciphertext valid under two keys yields under each a candidate that must still pass check (iv). $\square$

On acceptance the recipient records n with its note position, the index of cmx among the leaves of the note commitment tree of the output's pool, fixed by the order in which the chain appends extracted commitments ("The Merkle hash and the tree", §5.1; protocol specification, §"Note Commitment Trees"). The position determines the authentication path, which the recipient computes from public data (Lemma 5.15) and which, with the opening n of the leaf cmx (Proposition 10.15), a later Action consuming n requires (conditions A1 and A3 of Definition 9.2).

The key tiers of Proposition 3.18 ("Capability separation") have the following powers over note ciphertexts, for keys generated with $\text{use_qsk} = \text{false}$.

Incoming tier. A holder of the incoming viewing key (dk, ivk) , of which trial decryption and acceptance use only ivk , detects and reads every note sent to any address of the key, value and memo included. It cannot compute the nullifiers of these notes, which require nk , and (dk, ivk) yields neither nk nor ask (Proposition 3.18(b)). It cannot link a published nullifier to one of them, for notes with pairwise distinct ρ (Proposition 6.10), so the nullifier sets alone do not show it which are consumed; and it cannot authorise a spend.

Full-viewing tier. A holder of the full viewing key $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$ derives (dk, ivk) and ovk ("Viewing keys", §3.2). It detects and reads incoming notes as above, and reads the outputs sent under its ovk (§10.3), not those sent under $\text{ovk} = \perp$ or another outgoing viewing key. With nk it computes the nullifier $\text{nf} = \text{DeriveNullifier}_{\text{nk}}(\rho, \psi, \text{cm})$ of each accepted note (Definition 6.3) and detects its consumption by the membership of nf in the nullifier set of the note's pool ("Nullifier sets", §6.2; protocol specification, §"Decryption using an Incoming Viewing Key (Sapling and Orchard)", notes). It cannot spend, since it yields no ask (Proposition 3.18(a)). For each unspent note the owner thus holds the note, its position, its authentication path and its nullifier, the consumed-note inputs of an Action (Definition 9.2); spending authority is ask .

11 Transactions and consensus rules

This section assembles the public data of an Action and of a bundle, states the transaction format that carries them, constructs the digest that both kinds of signature sign and proves in Proposition 11.10 which data the signatures fix, states the chain state and the rules of each pool, and gives the procedures of the sender and of the verifier, with a worked two-Action payment.

Table 6. The fields of an Action description, in the order of their encoding, with the type that parsing enforces and the byte length of the encoding.

Field	Type enforced on parsing	Encoding	Bytes	Constructed in
cv^{net}	$\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, the identity admitted	$(cv^{\text{net}})^*$	32	Definition 8.2
nf	$\{0, \dots, p_{\text{Pallas}} - 1\}$	$\text{l2LEOSP}_{256}(\text{nf})$	32	Definition 6.3
rk	$\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$; the identity excluded by a separate rule	rk^*	32	§7.2
cmx	$\{0, \dots, p_{\text{Pallas}} - 1\}$	$\text{l2LEOSP}_{256}(\text{cmx})$	32	Definition 4.4
epk	$\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}$	epk^*	32	§10.2
C_{enc}	580-byte string	itself	580	§10.2
C_{out}	80-byte string	itself	80	§10.3
Action description			820	

11.1 Action descriptions and bundles

The objects of this subsection are constructed in §4 to §10. The subsection fixes their encodings and their grouping; it redefines neither the Action nor the bundle (Definitions “Action” and “Bundle”, §4.4). As in §10.2, a star encoding P^* enters a byte string as the 32 bytes $\text{LEBS2OSP}_{256}(P^*)$ and is identified with them.

Definition 11.1 (Action description). The *Action description* of an Action is the byte string

$$(cv^{\text{net}})^* \parallel \text{l2LEOSP}_{256}(\text{nf}) \parallel rk^* \parallel \text{l2LEOSP}_{256}(\text{cmx}) \parallel epk^* \parallel C_{\text{enc}} \parallel C_{\text{out}}$$

of $5 \cdot 32 + 580 + 80 = 820$ bytes, whose fields, in this order, are those of Table 6: the net value commitment of Definition 8.2, the nullifier of the consumed note (Definition 6.3), the randomised validating key of “Randomised validating keys” (§7.2), the extracted commitment of the created note (Definition 4.4), and the ephemeral public key and the two ciphertexts of “Encryption to the recipient” (§10.2) and “The outgoing ciphertext” (§10.3) (protocol specification, §“Action Descriptions” and §“Action Description Encoding and Consensus”).

Every field of an Action description must be the canonical encoding of its type (protocol specification, §“Action Descriptions”; for cmx also §“Action Description Encoding and Consensus”). Hence nf and cmx are integers below p_{Pallas} , and neither is checked to be the x -coordinate of a Pallas point; epk is a non-identity point by its type; rk is a point, and the rule of “Randomised validating keys” (§7.2) excludes the identity; and cv^{net} may be the identity. The Action’s spend-authorisation signature, of 64 bytes (“The RedPallas signature scheme”, §7.1), and its proof are not part of the Action description. The bundle carries the signatures, one per Action, and one aggregate proof, whose per-Action instances are in the order of the Action descriptions (ZIP 229, “Consensus Rules”).

A bundle of one pool with $n \geq 1$ Actions publishes:

- (i) its n Action descriptions;
- (ii) one anchor rt (“Anchors”, §5.2), shared by all n Actions: no Action description carries

- an anchor, and the transaction carries one anchor field for each pool with Actions, anchorOrchard and anchorIronwood;
- (iii) one flags byte (“Bundle flags”, §9.1), whose bit 0 is enableSpends and bit 1 enableOutputs, and whose bit 2 is enableCrossAddress in flagsIronwood and reserved and 0 in flagsOrchard;
- (iv) the balancing value v^{balance} of “The balancing value” (§8.2);
- (v) one aggregate proof over its n Actions (“The Action circuit and the Halo 2 proof”, §9.3);
- (vi) n spend-authorisation signatures, one per Action;
- (vii) one binding signature (“The binding signature”, §8.3).

The anchor and the flags are common to all Actions of the bundle and are encoded once (protocol specification, §“Action Transfers and their Descriptions” and §“Action Descriptions”, note). A transaction carries at most one bundle per pool, hence one binding signature for each pool with Actions (ZIP 229, “Transaction Format” and “Consensus Rules”). Figure 4 shows each public field with the checks that consume it.

A bundle that consumes s real notes and creates o real notes has $n \geq \max(s, o)$ Actions, since every Action consumes exactly one note and creates exactly one; its $n - s$ consumed sides and $n - o$ created sides without a real note carry dummy notes of value 0 (Definition “Dummy notes”, §4.4; protocol specification, §“Dummy Notes (Orchard)”). When the consumed value exceeds the payment and the fee, the difference is *change*: an ordinary created note to an address of the sender, constructed as every other created note.

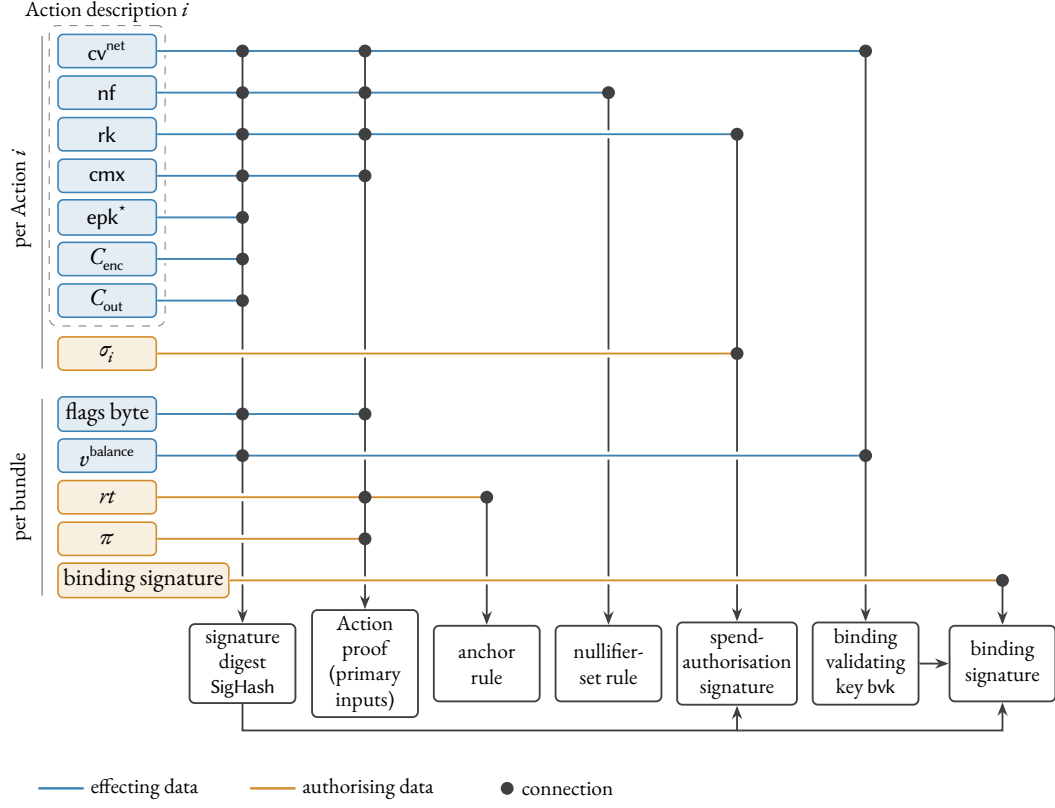


Figure 4. The public data of one bundle and the checks that consume them. Rows are the fields: per Action i , the net value commitment cv^{net} , the nullifier nf , the randomised validating key rk , the extracted commitment cmx , the ephemeral key epk^* and the ciphertexts C_{enc} and C_{out} of the Action description, and the spend-authorisation signature σ_i ; per bundle, the flags byte, the balancing value v^{balance} , the anchor rt , the aggregate proof π and the binding signature. Each column ends in the check or derived key that consumes it. A dot joins a field to a column; a row that crosses a column without a dot is not joined to it. Blue rows are effecting data and amber rows authorising data (“Transaction digests and signatures”, §11.3). Every effecting field, the ciphertexts included, joins the signature digest SigHash, which feeds every spend-authorisation signature and the binding signature; the anchor and the proof do not join it (Proposition 11.10). The fields nf , cmx , cv^{net} , rk , the flags and rt are primary inputs of the Action proof; rt also feeds the anchor rule, nf the nullifier-set rule, rk the spend-authorisation signature, and cv^{net} and v^{balance} the binding validating key bvk.

11.2 The transaction format

The transaction formats in force are version 5, specified in ZIP 225, and version 6, specified in ZIP 229; a transaction of any other version, version 4 included, is invalid (protocol specification, §“Transaction Consensus Rules”, as amended by ZIP 2003, “Specification”; ZIP 259, Abstract). Of the two Orchard-protocol bundles a version-5 transaction can carry only the Orchard-pool bundle; a transaction with Ironwood-pool Actions is of version 6 (protocol specification, §“Action Descriptions”). The Orchard-pool rules of “Chain state and pool rules” (§11.4) apply to Orchard-pool Actions in both versions (ZIP 229, “Consensus Rules”). *Sprout*, Zcash’s first shielded protocol, spends its notes only through *JoinSplit descriptions*, its transfer descriptions, which only transaction versions 2 to 4 carry (protocol specification, §“Transaction Encoding and Consensus”); Sprout funds are therefore unspendable, and their value remains counted as issued, in the sense of the total issued supply defined in “Chain state and pool rules” (§11.4) (ZIP 2003, Abstract and “Interaction with the proposed Network Sustainability Mechanism”; ZIP 259, Abstract). This subsection describes version 6.

Construction 11.2 (Version-6 transaction). A version-6 transaction consists of the fields of the version-5 format up to and including the Orchard component (ZIP 225), followed by the Ironwood component. Its first five fields form the header:

- (i) header, whose bit 31, the *fOverwintered* flag, is set and whose bits 0 to 30 hold the version, 6;
- (ii) *nVersionGroupId*, which must be `0xD884B698`;
- (iii) *nConsensusBranchId*, which must equal the consensus branch identifier used by the signature digest of “Transaction digests and signatures” (§11.3), the identifier of the consensus branch under which the transaction is validated, where a *consensus branch* is a sequence of consecutive blocks under one consensus rule set whose first block is the genesis block or follows a block validated under older rules, and each network upgrade fixes for the branch that it begins a globally unique non-zero 32-bit *consensus branch identifier* (ZIP 200, Terminology and “Specification”);
- (iv) *lock_time*, the transparent lock time, which no rule of this volume uses;
- (v) *nExpiryHeight*, the *expiry height*.

In version 6, bits 0 and 1 of *flagsOrchard* are named *enableSpends* and *enableOutputs*, as in *flagsIronwood*, and bits 2 to 7 of *flagsOrchard* are reserved and must be 0, in version 5 and version 6 alike (“Bundle flags”, §9.1). The Ironwood component has the structure of the Orchard component, with its own flags, balancing value, anchor, proof and signatures; both encode each Action as the 820-byte Action description of §11.1 (ZIP 229, “Transaction Format” and “Consensus Rules”; protocol specification, §“Transaction Consensus Rules”).

Definition 11.3 (Coinbase transaction). A *coinbase transaction* is the first transaction of a block; it collects the block subsidy, the value newly issued in that block, and the fees of the

Table 7. The Ironwood component of a version-6 transaction with n Ironwood-pool Actions (ZIP 229, “Transaction Format”). The fields after the count are present if and only if $n > 0$.

Field	Bytes	Content
nActionsIronwood	compactSize	the count n
vActionsIronwood	$820n$	the n Action descriptions of §11.1
flagsIronwood	1	bit 0 enableSpends, bit 1 enableOutputs, bit 2 enableCrossAddress, bits 3 to 7 reserved and 0
valueBalanceIronwood	8	the balancing value, signed 64-bit little-endian
anchorIronwood	32	the anchor rt
sizeProofsIronwood	compactSize	the proof length, $2720 + 2272n$
proofsIronwood	$2720 + 2272n$	the aggregate proof
vSpendAuthSigIronwood	$64n$	one spend-authorisation signature per Action, in Action order
bindingSigIronwood	64	the binding signature

block’s other transactions. Every block has exactly one (protocol specification, §“Coinbase Transactions”).

The field nExpiryHeight is 0, which disables expiry, or a block height. The *expiry rule* is: for a transaction other than a coinbase transaction, nExpiryHeight is at most 499 999 999, and such a transaction with nExpiryHeight $\neq 0$ is invalid in every block of height greater than nExpiryHeight; the nExpiryHeight of a coinbase transaction equals the height of its block (protocol specification, §“Transaction Consensus Rules”; ZIP 203, “Specification”; ZIP 229, “Transaction Format”). The sender’s choice of the expiry height is stated in “Construction of a transaction” (§11.5).

Definition 11.4 (Ironwood component). Let n be the number of Ironwood-pool Actions of a version-6 transaction. The *Ironwood component* is the sequence of the fields of Table 7, in that order. The fields after nActionsIronwood are present if and only if $n > 0$, and an absent valueBalanceIronwood is taken to be 0. A count and a proof length are encoded as *compactSize*, the variable-length integer encoding of Bitcoin (ZIP 229, “Transaction Format”).

The consensus rules on the component are: $n < 2^{16}$; if $n > 0$, at least one of enableSpends and enableOutputs of flagsIronwood is 1; bits 3 to 7 of flagsIronwood are 0; valueBalanceIronwood lies in $\{-\text{MAX_MONEY}, \dots, \text{MAX_MONEY}\}$, where $\text{MAX_MONEY} = 2.1 \cdot 10^{15}$ satoshi is a protocol constant (protocol specification, §“Constants”); and proofsIronwood has exactly $2720 + 2272n$ bytes (ZIP 229, “Consensus Rules”; protocol specification, §“Transaction Consensus Rules” and §“Action Description Encoding and Consensus”). The flags byte differs from flagsOrchard only in bit 2, which is enableCrossAddress in flagsIronwood and reserved and 0 in flagsOrchard (“Bundle flags”, §9.1). The proof length is the one that ZIP 229 fixes; the *Halo 2 Guide*, §“Halo 2’s specifics: parameters, generators from a hash, the deferred multi-scalar multiplication, and the proof bytes”, paragraph “The bytes”, describes what its constant and its per-Action term cover. For one Action the proof has $2720 + 2272 = 4992$ bytes.

11.3 Transaction digests and signatures

Write $h(P, x) := \text{BLAKE2b-256}(P, x)$, unkeyed BLAKE2b with a 32-byte output, the 16-byte personalisation P and the input x (§10.2); the personalisation is mixed into the initialisation vector (*Crypto Guide*, §“Domain separation and personalisation”). For a byte string C and $0 \leq a \leq b \leq |C|$, the string $C[a:b]$ consists of the bytes a to $b - 1$ of C , counted from 0. Every field enters a digest in its encoding of Table 6 or Table 7, and β denotes the 4-byte little-endian encoding of the consensus branch identifier.

Definition 11.5 (Transaction identifier). The *transaction identifier* of a version-6 transaction is

$$\text{txid} := h(\text{ZcashTxHash_} \parallel \beta, d_{\text{hdr}} \parallel d_{\text{tr}} \parallel d_{\text{sap}} \parallel d_{\text{orc}} \parallel d_{\text{lrw}}),$$

the Ironwood digest last, where:

- (i) the *header digest* d_{hdr} is h under ZTxIdHeadersHash of the 4-byte little-endian encodings of header, $n\text{VersionGroupId}$, $n\text{ConsensusBranchId}$, lock_time and $n\text{ExpiryHeight}$;
- (ii) the *transparent digest* d_{tr} and the *Sapling digest* d_{sap} are those of ZIP 244, “T.2: transparent_digest” and “T.3: sapling_digest”, the latter with the version-6 node of ZIP 229; they cover the transparent inputs and outputs and the Sapling component, the component of Zcash’s second shielded protocol;
- (iii) for $n \geq 1$ Ironwood-pool Actions, with the fields of Action i indexed by i , the *Ironwood digest* is

$$\begin{aligned} d_{\text{lrw}} &:= h(\text{ZTxIdIronwd_H_v6}, d_C \parallel d_M \parallel d_N \parallel \text{flagsIronwood} \parallel \text{valueBalanceIronwood}), \\ d_X &:= h(P_X, r_1^X \parallel \dots \parallel r_n^X) \quad (X \in \{C, M, N\}), \end{aligned}$$

with the *compact*, *memo* and *non-compact* records and personalisations

$$\begin{aligned} r_i^C &:= \text{nf}_i \parallel \text{cmx}_i \parallel \text{epk}_i^* \parallel C_{\text{enc},i}[0:52], & P_C &:= \text{ZTxIdIrActCH_v6}, \\ r_i^M &:= C_{\text{enc},i}[52:564], & P_M &:= \text{ZTxIdIrActMH_v6}, \\ r_i^N &:= (\text{cv}_i^{\text{net}})^* \parallel \text{rk}_i^* \parallel C_{\text{enc},i}[564:580] \parallel C_{\text{out},i}, & P_N &:= \text{ZTxIdIrActNH_v6}; \end{aligned}$$

for $n = 0$ it is $d_{\text{lrw}} := h(\text{ZTxIdIronwd_H_v6}, \varepsilon)$;

- (iv) the *Orchard digest* d_{orc} has the same structure over the Orchard-pool Actions, with flagsOrchard and $\text{valueBalanceOrchard}$, under the personalisation ZTxIdOrchardH_v6 , with the Action digests personalised

$$P_C = \text{ZTxIdOrcActCHash}, \quad P_M = \text{ZTxIdOrcActMHash}, \quad P_N = \text{ZTxIdOrcActNHash};$$

without Orchard-pool Actions it is $h(\text{ZTxIdOrchardH_v6}, \varepsilon)$.

(ZIP 244, “Digests”, “txid_digest”, “T.1: header_digest” and “T.4: orchard_digest”; ZIP 229, “Ironwood component digest”, “Anchor commitment (version 6)” and “Summary of the resulting digest structure”).

The ciphertext C_{enc} is the ChaCha20 encryption of the 564-byte note plaintext, byte for byte, followed by the 16-byte tag (*Crypto Guide*, §“The ChaCha20-Poly1305 AEAD”, Construction “ChaCha20-Poly1305”; “The note plaintext”, §10.1). The offsets therefore split it into the encryption of the 52-byte prefix of the plaintext, the lead byte, d , v and rseed ($1 + 11 + 8 + 32 = 52$), the encryption of the 512-byte memo, ending at $52 + 512 = 564$, and the tag. The empty Ironwood digest differs from the empty Orchard digest, since the two personalisations differ. Neither pool digest contains the anchor.

Definition 11.6 (Authorising-data digest; effecting and authorising data). The *authorising-data digest* of a version-6 transaction is

$$\text{auth} := h(\text{ZTxAuthHash}_- \parallel \beta, a_{\text{tr}} \parallel a_{\text{Sap}} \parallel a_{\text{Orc}} \parallel a_{\text{Irw}}),$$

where a_{tr} is the transparent scripts digest and a_{Sap} the Sapling authorising-data digest (ZIP 244, “Authorizing Data Commitment”; ZIP 229), and, for $n \geq 1$ Ironwood-pool Actions,

$$a_{\text{Irw}} := h(\text{ZTxAuthIrnwdH}_v6, \text{proofsIronwood} \parallel \text{vSpendAuthSigIronwood} \parallel \text{bindingSigIronwood} \parallel \text{anchorIronwood}),$$

and $a_{\text{Irw}} := h(\text{ZTxAuthIrnwdH}_v6, \varepsilon)$ for $n = 0$; the Orchard digest a_{Orc} has the same structure under ZTxAuthOrchaH_v6 . Each Orchard-protocol anchor is committed if and only if its component has Actions. The *effecting data* of a transaction are the data under the transaction identifier digest; its *authorising data* are the proofs, the signatures, the anchors and the transparent scripts, the data under the authorising-data digest (ZIP 229, “Ironwood component digest” and “Anchor commitment (version 6)”; ZIP 244, “Authorizing Data Commitment”).

ZIP 244, “Authorizing Data Commitment”, describes the pair of the transaction identifier and the authorising-data digest as a commitment to all data of the transaction. In version 6 the anchors are authorising data so that a transaction can be signed before its anchor is chosen and its proofs are computed; replacing a bundle’s anchor and proof together does not change the transaction identifier (ZIP 229, Motivation and “Anchor commitment (version 6)”). Below the two roots, a node whose directly hashed input differs between versions 5 and 6 has a version-6 personalisation of its own; the roots keep theirs and hash one more child in version 6. The nodes that omit or include an anchor in version 6 are personalised ZTxIdSSpendNH_v6 , ZTxAuthSap1iH_v6 , ZTxIdOrchardH_v6 and ZTxAuthOrchaH_v6 (ZIP 229, “Ironwood component digest” and “Anchor commitment (version 6)”).

Definition 11.7 (Signature digest). The *signature digest* of a transaction is its digest not associated with a transparent input, under the hash type SIGHASH_ALL, which covers all transparent inputs and outputs:

$$\text{SigHash} := h(\text{ZcashTxHash}_- \parallel \beta, d_{\text{hdr}} \parallel s_{\text{tr}} \parallel d_{\text{Sap}} \parallel d_{\text{Orc}} \parallel d_{\text{Irw}}),$$

whose header and shielded children are those of txid and whose child s_{tr} is the transparent signature digest of ZIP 244, “S.2: transparent_sig_digest”. For a coinbase transaction and for a transaction without transparent inputs, $s_{\text{tr}} = d_{\text{tr}}$, so $\text{SigHash} = \text{txid}$; otherwise s_{tr} commits in addition

to the values and scripts of the coins that the transparent inputs spend. The digest SigHash is the message of every spend-authorisation signature and of each pool's binding signature, the message left as a parameter in "The RedPallas signature scheme" (§7.1) (ZIP 244, "Signature Digest"; ZIP 229, "Transaction Identifiers, Auth Digests, and Signature Digests"; protocol specification, §"Action Descriptions" and §"Balance and Binding Signature (Orchard)").

Proposition 8.10, which holds for every message, applies to the binding signature on SigHash.

Remark 11.8 (Version 5). In version 5 the root of the transaction identifier has four children, the header, transparent, Sapling and Orchard digests, under the same root personalisation. The Orchard digest, under `ZTxIdOrchardHash`, hashes

$$d_C \parallel d_M \parallel d_N \parallel \text{flagsOrchard} \parallel \text{valueBalanceOrchard} \parallel \text{anchorOrchard},$$

with the Action digests of version 6, and the Orchard authorising-data digest, under `ZTxAuthOrchardHash`, covers the proofs and the signatures only. In version 5 the Orchard anchor is therefore effecting data and fixed by the signatures; the proofs are authorising data in both versions. A version-5 transaction carries Orchard-pool Actions only (ZIP 244, "txid_digest", "T.4: orchard_digest" and "A.3: orchard_auth_digest").

Figure 5 draws both trees. The binding of the signatures needs one further assumption, stated at its first use.

Assumption 11.9 (Collision resistance of BLAKE2b). No efficient algorithm outputs, except with negligible probability, two distinct pairs (P, x) and (P', x') , each of a 16-byte personalisation and an input string, with $h(P, x) = h(P', x')$. The notion is the *Crypto Guide's* Definition "Collision resistance" (§"Security notions: preimage, second-preimage, and collision resistance"), read for the unkeyed deployment as that volume's Remark "Convention: keyed definitions, unkeyed deployments" (§"Hash functions and the random oracle model", §"Syntax") prescribes; the personalisation is mixed into the initialisation vector (§"Domain separation and personalisation"), so the pair (P, x) is the hashed input. It is used by Proposition 11.10 and by the consensus-branch binding below.

Proposition 11.10 (Bundle binding). *Let T be a version-6 transaction, and for a bundle of T , of either pool, let $\text{rk}_1, \dots, \text{rk}_n$ be the randomised validating keys of its Actions, $\text{rk}_i = [\text{rsk}_i] G^{\text{SpendAuth}}$ with $\text{rsk}_i = \text{ask}_i + \alpha_i$ as in "Randomised validating keys" (§7.2), where ask_i is the spend authorising key of the spending key that authorises Action i . Assume:*

- (H) *each ask_i is uniform on the non-zero scalars a for which $[a] G^{\text{SpendAuth}}$ has even y -coordinate (§3.1), and every value of the experiment other than a signature by the holder of ask_i is computed from $\text{ak}_i^P = [\text{ask}_i] G^{\text{SpendAuth}}$ and from values drawn independently of ask_i ;*
- (S) *the holder of ask_i signs under rsk_i only $\text{SigHash}(T)$, and signs any other message only under $\text{ask}_i + \alpha$ for a fresh randomiser α drawn by `GenRandom`.*

An adversary receives T and all other public data, and obtains on request, for messages of its choice, signatures of the key holders under fresh randomisers as in (S); it receives no ask_i and no rsk_i .

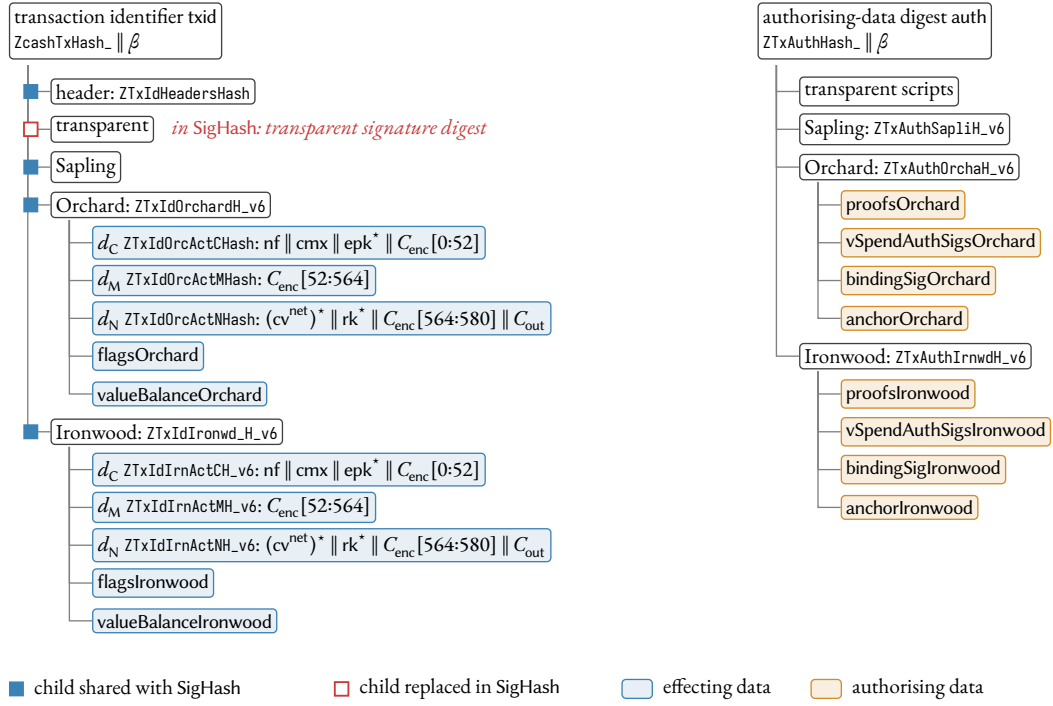


Figure 5. The version-6 transaction identifier (left) and authorising-data digest (right) as trees of personalised BLAKE2b-256 nodes, with the Orchard and Ironwood components expanded (ZIP 244; ZIP 229). Each node hashes the concatenation of its children, in order, under the personalisation shown; the string β encodes the consensus branch identifier. The Action digests d_C , d_M and d_N hash the listed fields of every Action of their component, Action after Action; the byte flagsIronwood is the Ironwood component's own flags byte. The header, transparent and Sapling digests are not expanded. The signature digest SigHash has the root personalisation and the children of txid (filled squares), except the transparent child (open square), which it replaces by the transparent signature digest. The anchors and the proofs lie under the authorising-data digest only. By Proposition 11.10, the signatures fix every field under txid and no field under the authorising-data digest.

Under Assumptions 11.9, 2.22 and 7.3, the probability that it outputs an accepted transaction T' that contains an Action description with validating key rk_i for some i and whose effecting data differ from those of T is negligible.

The signatures thus fix the effecting data: every Action's nf , cmx , cv^{net} , rk , epk , C_{enc} and C_{out} , the number and order of the Actions, each bundle's flags and balancing value, the header, with the consensus branch identifier and the expiry height, and the transparent data. They fix neither the anchor nor the proof of a bundle: for a pair (rt', π') such that π' verifies on the primary inputs formed with rt' , replacing (rt, π) by (rt', π') changes neither the transaction identifier nor any signed message, so every signature remains valid, and π' binds rt' as a primary input (Definition 9.2).

Proof. Let E be the event that the adversary outputs such a T' . On E , let i be the least index for which T' contains an Action description with key rk_i , and let σ' be the spend-authorisation signature that T' carries for it. Since T' is accepted, σ' is valid under rk_i on $m' := \text{SigHash}(T')$ (“Randomised validating keys”, §7.2). Let $m := \text{SigHash}(T)$, and split E into E_+ , where $m' = m$, and E_- , where $m' \neq m$.

Case E_- : a collision. The digest SigHash is computed by the tree of its definition from the effecting data and, when the transaction has transparent inputs, from the values and scripts of the coins that they spend. Each node hashes a string from which its children and its fields are recovered uniquely: its input consists of fixed-length fields and digests, of concatenations of fixed-length per-Action records whose number the length determines, or, for transparent outputs, of records whose variable-length script carries its length; the two nodes that share a personalisation, the transparent digest and the transparent signature digest, hash strings of different lengths (ZIP 244, “TxId Digest” and “Signature Digest”). Compare the trees of T and T' from the root, whose outputs are equal. At a node with equal outputs, either the two (personalisation, input) pairs differ, a collision of h , or they agree, and then the node's children have equal outputs and its fields are equal. The effecting data of T and T' differ, so some field, or some number of records, differs, and descending along the nodes that cover it ends in a collision. An algorithm that runs the experiment, which it can since it samples every key, and performs this descent outputs a collision with probability $\Pr[E_-]$, which is negligible by Assumption 11.9.

Case E_+ : a forgery. An algorithm $\mathcal{B}$ plays the adversary of Proposition 7.12. It receives $Y = [x] G^{\text{SpendAuth}}$ for x uniform on $\mathbb{F}_{p_{\text{Vesta}}}$, with access to H and to the signing oracle, and draws i^* uniformly from $\{1, \dots, n\}$. If $Y = \mathcal{O}$ or Y has odd y -coordinate, it stops; otherwise, an event of probability $(p_{\text{Vesta}} - 1)/(2p_{\text{Vesta}})$, the point Y has the distribution that (H) gives to $ak_{i^*}^P$, and $\mathcal{B}$ sets $ak_{i^*}^P := Y$. It computes every other value of the experiment as (H) permits, in particular the proof of T , whose witness (Definition 9.2) contains ak^P and α but not ask ; it draws every randomiser by GenRandom itself; it obtains each signature by the holder of ask_{i^*} , on m and on the adversary's requests, as the oracle's answer on the pair (α, M) ; and it produces the other signatures with keys it holds. The adversary's view is then that of the experiment. On E_+ with $i = i^*$, $\mathcal{B}$ outputs (rk_{i^*}, m', σ') with the randomiser α_{i^*} , of its own choice, as part (ii) of Proposition 7.12 admits. The oracle answered under the key rk_{i^*} only on m , except when a randomiser drawn for a request equals α_{i^*} , which has negligible probability by the lemma on the distance of GenRandom from uniform (§7.2); outside that event the output is a forgery. The index i^* is independent of

the adversary’s view, so $\mathcal{B}$ forges with probability at least

$$\frac{p_{\text{Vesta}} - 1}{2p_{\text{Vesta}}} \cdot \frac{\Pr[E_{\#}]}{n}$$

less a negligible term, where $n < 2^{16}$. By Proposition 7.12(ii), under Assumptions 2.22 and 7.3, $\Pr[E_{\#}]$ is negligible.

Anchor and proof. By the definitions above, the anchor and the proof of each bundle are inputs of the authorising-data digest and of no node of txid or SigHash. Replacing them changes neither, and every signature, whose message is SigHash, remains valid. The verifier checks the bundle’s proof against the primary inputs formed with the bundle’s anchor field (protocol specification, §“Action Descriptions”), now rt' , so the modified bundle is accepted only if π' verifies for rt' , and the anchor rule then applies to rt' . $\square$

Remark 11.11 (Hypothesis (H) for honest keys). For keys generated with `use_qsk = false` from uniform spending keys, hypothesis (H) holds up to a negligible change in the probability of every efficiently decidable event, in particular of the event of Proposition 11.10, which an algorithm holding the keys decides by verifying T' and comparing its effecting data with those of T . By Lemma 2.15 (Assumption 2.12) the triple $(ask, nk, r_{\text{ivk}})$ is indistinguishable from independent uniform elements; the sign normalisation of “The spending key and the spend-side secrets” (§3.1) maps a uniform non-zero ask to the uniform distribution on the set of (H); and rejection, by Lemma “Rejection in key generation” (§3.2), costs a negligible term under Assumptions 2.12, 2.22 and 2.8. A dummy spend uses such a key (Definition “Dummy notes”, §4.4).

Remark 11.12 (Consensus-branch binding). The consensus branch identifier enters SigHash twice, in the root personalisation and in the header digest, and a consensus rule requires `nConsensusBranchId` to equal the identifier of the consensus branch under which the transaction is validated (protocol specification, §“Transaction Consensus Rules”; ZIP 244, “txid_digest” and “T.1: header_digest”). Under any other branch a signed transaction T either fails that rule or carries another `nConsensusBranchId`, hence other effecting data, and Proposition 11.10 excludes valid signatures on such a transaction that keeps an Action of T , except with negligible probability. A signed transaction is therefore valid under one consensus branch only and cannot be carried across the activation of a network upgrade (ZIP 259, “Backward compatibility”). That the anchors and the proofs are authorising data does not affect this binding, since the header is effecting data.

11.4 Chain state and pool rules

The Orchard pool and the Ironwood pool each have their own chain state: a note commitment tree (“The Merkle hash and the tree”, §5.1) with its anchors (“Anchors”, §5.2), a nullifier set (“Nullifier sets”, §6.2) and a chain value pool balance, defined below. An Action is checked only against the state of its own pool (ZIP 229, Terminology and Rationale, “Separate state”; protocol specification, §“Nullifier Sets” and §“Note Commitment Trees”).

Definition 11.13 (Chain value pool balances). The *Ironwood chain value pool balance* of a block chain is the negation of the sum of the `valueBalanceIronwood` fields of all its transactions, and the *Orchard chain value pool balance* the negation of the sum of their `valueBalanceOrchard` fields, an absent field counting as 0. The *total issued supply* is the sum of the balances of all chain value pools of the protocol specification, §“Chain Value Pool Balances”, the transparent pool and the other shielded pools included, computed without overflow (ZIP 209, Terminology and “Specification”).

The consensus rules on the balances are: a block that would make the chain value pool balance of any pool negative is invalid, and a block that would make the total issued supply exceed `MAX_MONEY` (§11.2) is invalid (ZIP 209, “Specification”; protocol specification, §“Chain Value Pool Balances”).

Remark 11.14 (Outflow bound). Fix a pool P and write each of its balancing values as $v = v^+ - v^-$ with $v^+ := \max(v, 0)$ and $v^- := \max(-v, 0)$. The chain value pool balance of P is $\sum v^- - \sum v^+$, summed over the transactions of the chain, so the rule that it is non-negative at every block states that at every block the net value that has left P through positive balancing values, $\sum v^+$, is at most the net value that entered it through negative ones, $\sum v^-$. Value created inside P by a failure of soundness can therefore leave P only up to the value recorded as having entered it (ZIP 209, “Specification”).

The following Orchard-pool rules apply to every transaction, whatever its version (ZIP 258, “Consensus rules from NU6.3 activation”; ZIP 229, “Consensus Rules”).

- (i) Cross-address transfers are disabled for every Orchard-pool Action: bits 2 to 7 of `flagsOrchard` are 0, so every Orchard-pool Action has `enableCrossAddress` = 0 and primary input `disableCrossAddress` = 1 (“Bundle flags”, §9.1), and the verifying key that consensus selects for every Action enforces condition A10 of Definition 9.2. The meaning of A10 and its pool-level effect are those stated at A10 in “The Action statement” (§9.2).
- (ii) For every transaction, `valueBalanceOrchard` ≥ 0 : net value may leave the Orchard pool and never enter it, so the Orchard chain value pool balance never increases.

Of the two Orchard-protocol pools, therefore, only the Ironwood pool accepts net inflow: a *shielding transaction*, one whose transparent inputs fund a negative balancing value, can place that negative balancing value, among the Orchard-protocol pools, only in its Ironwood component (ZIP 258, same section; protocol specification, §“Shielded Pools and Notes”).

The coinbase transaction of “The transaction format” (§11.2) is subject to the following rules on the shielded path (protocol specification, §“Transaction Consensus Rules”; ZIP 229, “Consensus Rules”; ZIP 258, “Consensus rules from NU6.3 activation”).

- (iii) A coinbase transaction has no Orchard-pool Actions.
- (iv) The bit `enableSpends` is 0 in `flagsOrchard` of a coinbase transaction of version 5 or 6 and in `flagsIronwood` of a coinbase transaction of version 6. By condition A8 of Definition 9.2, under Assumption 9.11, every Action of a coinbase transaction has, except with negligible probability, an extracted witness with $v^{\text{old}} = 0$: a coinbase transaction consumes no note of

non-zero value. The rule concerns coinbase transactions only; every other transaction may spend Ironwood-pool notes.

- (v) Every Ironwood-pool output of a coinbase transaction must decrypt to a note plaintext by the recovery with ovk of “The outgoing ciphertext” (§10.3) under the all-zero outgoing viewing key, the string of 32 zero bytes, so its plaintext is public and its lead byte is $0x03$, the only lead byte that recovery accepts (ZIP 213, “Specification”; ZIP 258, “Changes to the Protocol Specification”).

Rule (v) is the only check by consensus of the lead byte of an Orchard-protocol output. Every Ironwood-pool output uses the note format of lead byte $0x03$ (“The note seed”, §4.3), and the Action statement does not check the derivation of rcm (Definition 9.2, Remark “What the statement does not check”); for every other output the lead byte is encrypted and the recipient enforces it, decryption returning $\perp$ when the byte is not allowed for the pool (“Trial decryption and note acceptance”, §10.4; protocol specification, §“Decryption using an Incoming Viewing Key (Sapling and Orchard)”).

The rules of ZIP 218, “Shielded pool action limits”, bound per block the number of Orchard-pool Actions and a shielded budget that counts them together with the descriptions of the other shielded protocols; they do not mention Ironwood-pool Actions, which a version-6 transaction counts in the separate field $nActionsIronwood$ (ZIP 229, “Transaction Format”). These are block-level rules, outside the Action statement.

11.5 Construction of a transaction

The following procedure extends the single spend of the *Crypto Guide*’s Example “An Orchard shielded spend” (§“A shielded spend, end to end”) to a bundle with signatures and balance. Its steps follow the order of the preceding sections.

Construction 11.15 (Ironwood-pool bundle). The input is the sender’s keys; the Ironwood-pool notes to consume, with their leaf positions; each recipient’s address, which its holder derives from an incoming viewing key (“Diversified addresses”, §3.3) and hands to the sender; the amounts; and the fee. The sender performs the following steps.

- (1) *Pairing*. For s consumed and o created notes it forms $n \geq \max(s, o)$ Actions (§11.1). A dummy consumed note comes from a fresh uniformly random spending key, with value 0, a uniform rseed, $\rho = \text{Extract}_p$ of a uniform Pallas point and an arbitrary authentication path; a dummy created note has value 0 and a random address (Definition “Dummy notes”, §4.4).
- (2) *Anchor*. It takes as rt the root of the Ironwood tree’s final treestate at a recent block, at the depth that “Anchors” (§5.2) recommends, and computes the authentication path of each real consumed note from public data (Lemma 5.15).
- (3) *Created notes*. For each Action it computes nf^{old} of the consumed note by Definition 6.3, sets $\rho^{\text{new}} := nf^{\text{old}}$ (“Nullifier chaining”, §6.3), draws a fresh uniform 32-byte rseed, derives esk , ψ^{new} and rcm^{new} under lead byte $0x03$ as in “The note seed” (§4.3), redrawing

rseed when $\text{esk} = 0$ or $\text{cmx} = \perp$, and computes cm^{new} by Definition 4.4 and $\text{cmx} := \text{Extract}_p(\text{cm}^{\text{new}})$.

- (4) *Value commitments.* For each Action it draws rcv uniformly from $\mathbb{F}_{p_{\text{Vesta}}}$ and sets $\text{cv}^{\text{net}} := [v^{\text{old}} - v^{\text{new}}] V^{\text{Orchard}} + [\text{rcv}] R^{\text{Orchard}}$ (Definition 8.2).
- (5) *Randomised keys.* For each Action it draws α by GenRandom and sets $\text{rsk} := \text{ask} + \alpha$ and $\text{rk} := \text{ak}^P + [\alpha] G^{\text{SpendAuth}}$ (“Randomised validating keys”, §7.2), with ask and ak^P of the consumed note’s spending key, the dummy key for a dummy spend.
- (6) *Flags and balance.* It sets $\text{enableSpend} = \text{enableOutput} = 1$, and $\text{enableCrossAddress} = 1$ whenever some Action’s created note pays an expanded receiver other than that of its consumed note (condition A10 of Definition 9.2); it sets $v^{\text{balance}} := \sum (v^{\text{old}} - v^{\text{new}})$ over the Actions, and chooses the created values so that the fee is the value left in the transparent transaction value pool (“The balancing value”, §8.2).
- (7) *Proof.* It computes one Halo 2 proof over the n instances

$$(\text{rt}, \text{cv}^{\text{net}}, \text{nf}^{\text{old}}, \text{rk}, \text{cmx}, \text{enableSpend}, \text{enableOutput}, 1 - \text{enableCrossAddress})$$

with the witnesses of Definition 9.2 (“The Action circuit and the Halo 2 proof”, §9.3). By Assumption 9.11 a prover of an accepted proof knows, for each Action, a witness satisfying A1 to A10; under Assumption 9.14 the proof reveals nothing beyond the instances.

- (8) *Encryption.* For each Action it computes $\text{epk} := [\text{esk}] g_d$, C_{enc} and C_{out} as in “Encryption to the recipient” (§10.2) and “The outgoing ciphertext” (§10.3).
- (9) *Header.* It sets the version to 6, $n\text{ConsensusBranchId}$ to the identifier of the consensus branch in force, and $n\text{ExpiryHeight}$ (§11.2).
- (10) *Digest.* It computes SigHash (“Transaction digests and signatures”, §11.3).
- (11) *Spend authorisation.* For each Action it computes the spend-authorisation signature under rsk on SigHash.
- (12) *Binding signature.* It sets $\text{bsk} := \sum \text{rcv} \bmod p_{\text{Vesta}}$ and $\text{bvk} := \sum \text{cv}^{\text{net}} - [v^{\text{balance}}] V^{\text{Orchard}}$, checks $\text{bvk} = [\text{bsk}] R^{\text{Orchard}}$, which the protocol specification recommends to the signer, and computes the binding signature under bsk on SigHash (“The binding signature”, §8.3).

(Protocol specification, §“Sending Notes (Orchard)”, §“Dummy Notes (Orchard)”, §“Spend Authorization Signature (Sapling and Orchard)” and §“Balance and Binding Signature (Orchard)”.)

Steps (10) to (12) depend on neither rt nor the proof, which are authorising data, so signing can precede proving (ZIP 229, Motivation).

The sender sets $n\text{ExpiryHeight}$. ZIP 218, “Default expiry delta”, recommends to node implementations that create transactions a default expiry height 120 blocks above the current height (ZIP 203, “Specification”), which at the 25-second target spacing stated in “Anchors” (§5.2) is $120 \times 25 = 3000$ seconds, 50 minutes. The consensus rule is the expiry rule of “The transaction format” (§11.2).

Example 11.16 (A two-Action payment). The example is numeric only in amounts, fees, value differences, the balancing value, sizes and counts, each the output of a script; every curve point, field element, hash, ciphertext and signature is named and its derivation shown, not evaluated. The subscripts S, R and D mark the sender’s key, the recipient’s key and the dummy spending key.

Amounts. The sender holds one Ironwood-pool note n_1^{old} of 7 ZEC, 700 000 000 zatoshi, and pays 5 ZEC, 500 000 000 zatoshi, to a recipient’s address addr_R . One real consumed note and two created notes need $\max(1, 2) = 2$ Actions, hence two logical actions, and the conventional fee of “The balancing value” (§8.2) is $5000 \cdot \max(2, 2) = 10\,000$ zatoshi. The change is $700\,000\,000 - 500\,000\,000 - 10\,000 = 199\,990\,000$ zatoshi. The following table pairs the notes into Actions.

	Action 1	Action 2
consumed note	n_1^{old} , the sender’s	n_2^{old} , a dummy
spending key	the sender’s: $\text{ask}_S, \text{ak}_S^{\text{P}}, \text{nk}_S$	the dummy key: $\text{ask}_D, \text{ak}_D^{\text{P}}, \text{nk}_D$
v_i^{old}	700 000 000	0
created note	n_1^{new} to addr_R	n_2^{new} , the change, to addr_S
v_i^{new}	500 000 000	199 990 000
$v_i^{\text{old}} - v_i^{\text{new}}$	200 000 000	−199 990 000

The net values sum to $\text{valueBalancelronwood} = 200\,000\,000 + (-199\,990\,000) = 10\,000$ zatoshi, the fee. The transaction has no transparent inputs or outputs, so the fee is the whole value remaining in its transparent transaction value pool.

Public fields. The recipient derives $\text{addr}_R = (d_R, \text{pk}_{d,R})$ from its incoming viewing key (“Diversified addresses”, §3.3) and gives it to the sender; addr_S is an address of the sender’s own key. Write $(g_{d,1}, \text{pk}_{d,1})$ and $(g_{d,2}, \text{pk}_{d,2})$ for the expanded receivers of addr_R and addr_S , and $\text{nk}_1 := \text{nk}_S$, $\text{nk}_2 := \text{nk}_D$. For $i = 1, 2$ the sender computes, from the consumed note’s fields $(\rho_i^{\text{old}}, \psi_i^{\text{old}}, \text{cm}_i^{\text{old}})$,

$$\text{nf}_i^{\text{old}} := \text{DeriveNullifier}_{\text{nk}_i}(\rho_i^{\text{old}}, \psi_i^{\text{old}}, \text{cm}_i^{\text{old}}) \quad (\text{Definition 6.3}),$$

$$\rho_i^{\text{new}} := \text{nf}_i^{\text{old}} \quad (\S 6.3),$$

draws a uniform rseed_i , derives $\text{esk}_i, \psi_i^{\text{new}}$ and $\text{rcm}_i^{\text{new}}$ from rseed_i under lead byte $0x03$ (“The note seed”, §4.3), and sets

$$\begin{aligned} \text{cm}_i^{\text{new}} &:= \text{NoteCommit}_{\text{rcm}_i^{\text{new}}}(\text{g}_{d,i}^* \parallel \text{pk}_{d,i}^* \parallel \text{LE}_{64}(v_i^{\text{new}}) \parallel \text{LE}_{255}(\rho_i^{\text{new}}) \parallel \text{LE}_{255}(\psi_i^{\text{new}})), \\ \text{cmx}_i &:= \text{Extract}_{\text{p}}(\text{cm}_i^{\text{new}}) \end{aligned}$$

(Definition 4.4). With rcv_i uniform on $\mathbb{F}_{\text{pVesta}}$ and α_i drawn by GenRandom ,

$$\begin{aligned} \text{cv}_1^{\text{net}} &:= [200\,000\,000] V^{\text{Orchard}} + [\text{rcv}_1] R^{\text{Orchard}}, & \text{rk}_1 &:= \text{ak}_S^{\text{P}} + [\alpha_1] G^{\text{SpendAuth}}, \\ \text{cv}_2^{\text{net}} &:= [-199\,990\,000] V^{\text{Orchard}} + [\text{rcv}_2] R^{\text{Orchard}}, & \text{rk}_2 &:= \text{ak}_D^{\text{P}} + [\alpha_2] G^{\text{SpendAuth}}, \end{aligned}$$

the negative difference read in $\mathbb{F}_{p_{\text{Vesta}}}$ (Definition 8.2; “Randomised validating keys”, §7.2). The anchor rt is the anchor of a recent Ironwood-pool block, with the authentication path of $\text{Extract}_p(\text{cm}_1^{\text{old}})$; the path of the dummy is arbitrary, since A3 exempts $v_2^{\text{old}} = 0$. The flags are $\text{enableSpend} = \text{enableOutput} = \text{enableCrossAddress} = 1$, because both created notes pay expanded receivers other than those of their Actions’ consumed notes (A10). One proof π covers the instances $(rt, \text{cv}_i^{\text{net}}, \text{nf}_i^{\text{old}}, \text{rk}_i, \text{cmx}_i, 1, 1, 0)$ for $i = 1, 2$. Finally $\text{epk}_i := [\text{esk}_i]_{\text{gd},i}, C_{\text{enc},i}$ encrypts the plaintext of n_i^{new} to its receiver, and $C_{\text{out},i}$ is formed under the sender’s ovk (“Encryption to the recipient”, §10.2; “The outgoing ciphertext”, §10.3).

Signatures, balance and size. The transaction has no transparent inputs, so SigHash equals its transaction identifier (“Transaction digests and signatures”, §11.3). The spend-authorisation signatures are σ_1 under $\text{rsk}_1 = \text{ask}_S + \alpha_1$ and σ_2 under $\text{rsk}_2 = \text{ask}_D + \alpha_2$, both on SigHash . The binding signing key is $\text{bsk} := \text{rcv}_1 + \text{rcv}_2 \bmod p_{\text{Vesta}}$, and

$$\begin{aligned} \text{bvk} &= \text{cv}_1^{\text{net}} + \text{cv}_2^{\text{net}} - [10\,000] V^{\text{Orchard}} \\ &= [200\,000\,000 - 199\,990\,000 - 10\,000] V^{\text{Orchard}} + [\text{bsk}] R^{\text{Orchard}} = [\text{bsk}] R^{\text{Orchard}}, \end{aligned}$$

$\text{bindingSigIronwood}$ is the binding signature under bsk on SigHash . By Table 7 the Ironwood component has

$$1 + 2 \cdot 820 + 1 + 8 + 32 + 3 + (2720 + 2272 \cdot 2) + 2 \cdot 64 + 64 = 9141$$

bytes: the count, the Action descriptions, the flags, the balancing value, the anchor, the proof length, the proof of 7264 bytes, the spend-authorisation signatures and the binding signature.

The two views. The verifier receives only the public data: for each i the Action description $(\text{cv}_i^{\text{net}}, \text{nf}_i^{\text{old}}, \text{rk}_i, \text{cmx}_i, \text{epk}_i, C_{\text{enc},i}, C_{\text{out},i})$ and σ_i , and rt , the flags byte with bits 0, 1 and 2 set, $\text{valueBalanceIronwood} = 10\,000$, π and $\text{bindingSigIronwood}$. It runs the procedure of “Verification of a transaction” (§11.6). What these data reveal is the subject of Theorem 12.12 in “Privacy” (§12.4). The recipient trial-decrypts every output with its incoming viewing key. On output 1 it recovers the plaintext $(\text{txid}, d_R, 500\,000\,000, \text{rseed}_1, \text{memo})$, re-derives the note with $\rho = \text{nf}_1^{\text{old}}$ of the same Action, checks epk_1 and cmx_1 , and records the leaf position of cmx_1 (“Trial decryption and note acceptance”, §10.4); the sender recovers the change note of output 2 in the same way. Either note is later consumed by the procedure of this subsection.

11.6 Verification of a transaction

Each step of the following procedure is tagged with the requirements whose soundness part it serves and with the conditions of Definition 9.2 that it completes, in the pairing of the requirement map, Table 5.

Construction 11.17 (Verification). Given the chain state of each pool, a transaction T in a block is accepted, as far as its Orchard-protocol components are concerned, if and only if the following checks pass.

- (1) *Parsing.* The version of T is 5 or 6, with the version group identifier of its version, and $\text{nConsensusBranchId}$ equals the identifier of the consensus branch in force. Every field

of every Action description is the canonical encoding of its type (Table 6): cv^{net} and rk are points, epk is a non-identity point, and nf and cmx are integers below p_{Pallas} ; $rk \neq \mathcal{O}$; every signature encoding is canonical as RedPallas validation requires (§7.1); and each component satisfies the rules of “The transaction format” (§11.2) on its count, its reserved flag bits, its flags, the range of its balancing value and the length of its proof. The transaction T has a transparent input, a Spend description in its Sapling component or an Orchard-protocol bundle with $\text{enableSpend} = 1$, and it has a transparent output, an Output description in its Sapling component or an Orchard-protocol bundle with $\text{enableOutput} = 1$, where Spend and Output descriptions are the Sapling component’s descriptions of a consumed and of a created note.

- (2) *Proof.* For each pool with Actions, the aggregate proof verifies under the post-NU6.3 verifying key of “The Action circuit and the Halo 2 proof” (§9.3) on the per-Action primary inputs

$$(rt, cv^{\text{net}}, nf, rk, cmx, \text{enableSpend}, \text{enableOutput}, 1 - \text{enableCrossAddress}),$$

with rt and the flags read from the bundle. [R1 to R5, through A1 to A10.]

- (3) *Anchor rule.* Each bundle’s rt is the root of its pool’s final treestate at an earlier block of the chain (“Anchors”, §5.2). [R2, with A3.]
- (4) *Nullifier rule.* No nullifier repeats within T or across the transactions of the chain; the nullifiers of each pool are checked against that pool’s nullifier set (“Nullifier sets”, §6.2). [R3, with A5.]
- (5) *Spend authorisation.* Each Action’s spend-authorisation signature is valid under its rk on SigHash (“Randomised validating keys”, §7.2). [R5, with A6 and A7.]
- (6) *Balance.* For each pool with Actions, the verifier computes $bvk := \sum cv^{\text{net}} - [v^{\text{balance}}] V^{\text{Orchard}}$ over the pool’s Actions and checks the binding signature under bvk on SigHash (“The binding signature”, §8.3; *Crypto Guide*, §“Binding signatures as a proof of knowledge of a discrete logarithm”); and the fee, the value remaining in the transparent transaction value pool, is non-negative (“The balancing value”, §8.2), a coinbase transaction satisfying instead the coinbase rule of the protocol specification, §“Transaction Consensus Rules”, which balances its outputs against the block subsidy and the fees of the block’s other transactions. [R4, with A4.]
- (7) *Pool rules.* The rules of “Chain state and pool rules” (§11.4), the expiry rule of “The transaction format” (§11.2) and the capacity rule of each pool’s note commitment tree (“The Merkle hash and the tree”, §5.1) hold.

(Protocol specification, §“Transaction Consensus Rules”, §“Action Descriptions”, §“Action Description Encoding and Consensus”, §“RedDSA, RedJubjub, and RedPallas”, §“Action Transfers and their Descriptions”, §“Nullifier Sets”, §“Note Commitment Trees”, §“Balance and Binding Signature (Orchard)” and §“Transactions and Treestates”; ZIP 229, “Consensus

Rules”; ZIP 2003, “Specification”; ZIP 259, Abstract.) These are the consensus checks on the Orchard-protocol components of T ; the remaining rules of §“Transaction Consensus Rules” concern its other components.

Remark 11.18 (Division of the checks). Under Assumption 9.11 the proof yields, for each Action, a witness that satisfies Definition 9.2 for its primary input. Whether rt is a root that the chain recorded and whether nf is new are facts about the chain’s history, which no proof over one Action’s data attests; the verifier checks them against its state, in steps (3) and (4). Beyond the parsing of step (1), the fee rule of step (6) and the rules of step (7), validity is thus the conjunction of one proof per bundle, two kinds of signature and these two checks against chain state; apart from the coinbase rule (v) of §11.4, no step decrypts or inspects a note.

Construction 11.19 (State update). On accepting a block, for each pool: every nullifier of the pool’s Actions enters the pool’s nullifier set, so that a later Action that publishes the same nullifier fails step (4); every cmx is appended to the pool’s note commitment tree, in the order of the transactions and, within a transaction, of the Actions, so that the note’s holder can later prove membership against an anchor whose tree contains it; and the chain value pool balance of the pool decreases by each transaction’s balancing value (protocol specification, §“Nullifier Sets”, §“Transactions and Treestates” and §“Chain Value Pool Balances”).

The verifier holds no viewing key and decrypts no Orchard-protocol ciphertext, except for the decryption of Ironwood-pool coinbase outputs under the all-zero outgoing viewing key that “Chain state and pool rules” (§11.4) requires, whose plaintexts are public by that rule (protocol specification, §“Transaction Consensus Rules”; ZIP 213, “Specification”). Its input is the transaction’s public data and the chain state; what those data reveal about the private inputs is the subject of Theorem 12.12 in “Privacy” (§12.4).

12 Security

This section composes the propositions of the preceding sections with the two assumptions on the Action proof, Assumptions 9.11 and 9.14. Theorem 12.4 states conservation per pool; its proof uses condition A4, the binding signature and Assumption 9.11. Lemma 12.5 and Theorem 12.6 exclude a second consumption of a note, Corollary 12.7 the Faerie Gold attack and Theorem 12.9 a consumption without the spend authorising key; Theorem 12.12 states what the public data of a bundle hide. Theorem 12.14 collects, under one list of assumptions, membership soundness, Theorems 12.6, 12.4 and 12.9 with Proposition 11.10, and Theorem 12.12.

Efficient adversaries and negligible functions are those of the *Crypto Guide*, §“Adversaries and the security parameter”, and computational indistinguishability is that of its Definition “Perfect, statistical, and computational indistinguishability” (§“Distribution ensembles and indistinguishability”). An adversary interacts with the honest parties, namely key holders, senders and signers, and with the random oracles of Assumptions 2.8, 7.3, 9.11, 9.14 and 10.4, which are independent of one another; every reduction simulates the honest parties and the random oracles. Every result of the section holds except with negligible probability, and every result that uses a property

of key generation is claimed for keys generated with $\text{use_qsk} = \text{false}$ (“The spending key and the spend-side secrets”, §3.1).

Definition 12.1 (Pools, accepted Actions and witnesses).

- (a) A *pool* is the Orchard pool or the Ironwood pool. A statement made for each pool uses that pool’s note commitment tree, anchors, nullifier set and bundle, its balancing value, $\text{valueBalanceOrchard}$ or $\text{valueBalanceIronwood}$, and its binding signature, bindingSigOrchard or $\text{bindingSigIronwood}$.
- (b) An adversary outputs a *block chain*: a sequence of blocks each of whose transactions passes the verification of “Verification of a transaction” (§11.6) against the chain state that the state update of that subsection produces from the preceding transactions. An Action, or a bundle, of a transaction of that chain is *accepted*.
- (c) The *witness* of an accepted Action is the auxiliary input that the extractor of Assumption 9.11 outputs for it, applied to the Action’s bundle as stated below. The Action *consumes* the note $(g_d^{\text{old}}, pk_d^{\text{old}}, v^{\text{old}}, \rho^{\text{old}}, \psi^{\text{old}}, rcm^{\text{old}})$ of its witness and *creates* the note $(g_d^{\text{new}}, pk_d^{\text{new}}, v^{\text{new}}, \rho^{\text{new}}, \psi^{\text{new}}, rcm^{\text{new}})$, with $\rho^{\text{new}} = \text{nf}^{\text{old}}$ (Definition 9.2); the values v^{old} and v^{new} are read from the witness.
- (d) In this section a note is given by its expanded receiver (g_d, pk_d) (Definition “Flag values and expanded receiver”, §9.1) in place of its address, and its trapdoor rcm acts through its residue modulo p_{Vesta} ; two notes are equal when these six components are. Its note commitment is $\text{NoteCommit}_{rcm}(M)$ and its extracted commitment is $\text{Extract}_p^\perp(\text{NoteCommit}_{rcm}(M))$, with M the message of Definition 4.4 formed from $(g_d, pk_d, v, \rho, \psi)$.
- (e) A *committed note* of a pool is a note whose extracted commitment is a leaf of that pool’s note commitment tree.

Assumption 9.11 is stated for one aggregate proof. For a block chain it is applied to each bundle, to the algorithm that runs the adversary, with every honest party and random oracle of the experiment, and outputs that bundle’s primary inputs and proof; the extractors of distinct bundles are run on the same execution. An oracle that a reduction does not simulate itself, such as the random oracle H of Assumption 7.3 or a signing oracle, receives each new query of a rewind run, and a query repeated on the same prefix of a run is answered as before. By the union bound over the polynomially many bundles, every witness satisfies Definition 9.2, except with negligible probability. By Remark “ $\perp$ -weakened conditions” (§9.2), conditions A1, A2, A3 and A7 then hold in their unweakened forms, without the $\perp$ case, except with negligible probability, under Assumptions 2.22 and 2.8. The results below include these events in their negligible terms.

Remark 12.2 (Binding for notes given by expanded receivers). Step 1 of the proof of Proposition 4.5(b) shows that the map $(g_d, pk_d, v, \rho, \psi) \mapsto M$ is injective, and Step 2 reads a note only through M and its trapdoor. For notes in the sense of (d) the two steps therefore show, under Assumptions 2.22 and 2.8: no efficient algorithm outputs, except with negligible probability, two

distinct notes with equal extracted commitments other than $\perp$. In particular a note that opens an extracted commitment is the only opening that an efficient algorithm finds. The proof of Proposition 6.8 likewise reads a note only through M , rcm , ρ and ψ , and applies to such notes.

12.1 Value conservation

Lemma 12.3 (Modular balance is integer balance). *Let $n < 2^{16}$; for $1 \leq i \leq n$ let v_i^{old} and v_i^{new} be integers in $\{0, \dots, 2^{64} - 1\}$; and let v^{balance} be an integer in $\{-\text{MAX_MONEY}, \dots, \text{MAX_MONEY}\}$. If*

$$\sum_{i=1}^n (v_i^{\text{old}} - v_i^{\text{new}}) \equiv v^{\text{balance}} \pmod{p_{\text{Vesta}}},$$

then the two sides are equal as integers. The hypotheses hold for every accepted bundle of either pool: the bound on n and the range of the balancing value are consensus rules (“The transaction format”, §11.2; protocol specification, §“Transaction Consensus Rules”, for both pools), and v_i^{old} and v_i^{new} are 64-bit values of the auxiliary input of Definition 9.2.

Proof. Let $v^{\text{sum}} := \sum_i (v_i^{\text{old}} - v_i^{\text{new}}) - v^{\text{balance}}$. Each difference has absolute value at most $2^{64} - 1$, and $\text{MAX_MONEY} = 2.1 \cdot 10^{15}$ (protocol specification, §“Constants”), so

$$|v^{\text{sum}}| \leq (2^{16} - 1)(2^{64} - 1) + \text{MAX_MONEY} = 1\,208\,907\,374\,970\,555\,465\,089\,025 < 2^{80}.$$

The hypothesis makes v^{sum} a multiple of p_{Vesta} , and $p_{\text{Vesta}} > 2^{254}$ (§2.1); the only such multiple of absolute value below 2^{80} is 0. $\square$

The security argument of the protocol specification, §“Balance and Binding Signature (Orchard)”, bounds the balancing value by its signed 64-bit encoding instead, which places v^{sum} in

$$\{-1\,208\,916\,596\,242\,592\,319\,864\,832, \dots, 1\,208\,916\,596\,242\,592\,319\,864\,833\};$$

that bound is also below 2^{80} , so the conclusion holds without the range rule.

Theorem 12.4 (Balance). *Under Assumptions 9.11, 2.22, 2.8 and 7.3, for each pool, the Ironwood pool included, every accepted bundle of that pool output by an efficient adversary, with n Actions and balancing value v^{balance} , satisfies*

$$\sum_{i=1}^n (v_i^{\text{old}} - v_i^{\text{new}}) = v^{\text{balance}}$$

as integers, the values read from the Actions’ witnesses, except with negligible probability. The binding signature checked is that of the same pool, under the binding validating key bvk recomputed from that bundle’s net value commitments and balancing value; both pools use the bases V^{Orchard} and R^{Orchard} .

Proof. Fix a pool and an efficient adversary $\mathcal{A}$. The adversary outputs polynomially many bundles, and the union bound reduces the claim to the j -th bundle of the pool, for each j . Let $\mathcal{B}_j$ be the

algorithm that runs $\mathcal{A}$ with the extractors of Assumption 9.11 and outputs that bundle's net value commitments $cv_1^{\text{net}}, \dots, cv_n^{\text{net}}$, its balancing value, the pairs

$$(v_i^{\text{old}} - v_i^{\text{new}}, \text{rcv}_i \bmod p_{\text{Vesta}}) \quad (1 \leq i \leq n)$$

read from the witnesses, the digest SigHash of its transaction and the pool's binding signature. Algorithm $\mathcal{B}_j$ is efficient and forwards the queries of $\mathcal{A}$ to H .

(1) *Openings*. By condition A4 of Definition 9.2, $cv_i^{\text{net}} = [v_i^{\text{old}} - v_i^{\text{new}}] V^{\text{Orchard}} + [\text{rcv}_i] R^{\text{Orchard}}$, so each output pair is an opening of cv_i^{net} in the sense of “The binding signature” (§8.3), except with the negligible probability that a witness fails the definition.

(2) *Signature*. The bundle is accepted, so its binding signature is valid on SigHash under

$$\text{bvk} = \sum_{i=1}^n cv_i^{\text{net}} - [v^{\text{balance}}] V^{\text{Orchard}}$$

in the binding-signature instance of RedPallas, with base R^{Orchard} (step (6) of the verification of §11.6). This key belongs to the pool's own bundle: each pool's bvk sums only that pool's net value commitments and subtracts only its balancing value (§8.3).

(3) *Congruence*. Algorithm $\mathcal{B}_j$ is therefore an efficient party of the consequence of Proposition 8.10: it outputs the net value commitments and the balancing value of a bundle, openings of every commitment, and a binding signature under the resulting bvk on a message of its choice. The consequence is proved in two steps. The extraction of Proposition 7.12, in the random-oracle model for H (Assumption 7.3), yields b with $\text{bvk} = [b] R^{\text{Orchard}}$; if the openings did not balance modulo p_{Vesta} , part (ii) of Proposition 8.10 would turn the residual V^{Orchard} component into the discrete logarithm of V^{Orchard} to the base R^{Orchard} , which Assumptions 2.22 and 2.8 exclude. Hence $\sum_i (v_i^{\text{old}} - v_i^{\text{new}}) \equiv v^{\text{balance}} \pmod{p_{\text{Vesta}}}$, except with negligible probability.

(4) *Integers*. The bundle is accepted, so it has $n < 2^{16}$ Actions, its balancing value lies in the range $\{-\text{MAX_MONEY}, \dots, \text{MAX_MONEY}\}$, and the witnessed values lie in $\{0, \dots, 2^{64} - 1\}$. Lemma 12.3 turns the congruence into an equality of integers.

The argument uses no property of the pool beyond its own bundle and binding signature, and applies to the Ironwood pool as to the Orchard pool. $\square$

12.2 Double-spend resistance

Lemma 12.5 (One nullifier per note). *Under Assumptions 9.11, 2.22 and 2.8, except with negligible probability: whenever two accepted Actions, of either pool, have witnesses whose consumed notes have the same extracted commitment $\text{Extract}_P(\text{cm}^{\text{old}})$, the two witnesses consume the same note with the same nullifier deriving key nk , and the two Actions publish the same nullifier nf^{old} . The lemma uses neither v^{old} nor condition A3.*

Proof. Let w and w' be the two witnesses, primed components belonging to w' . Outside the negligible events of the paragraph after Definition “Pools, accepted Actions and witnesses”, both satisfy Definition 9.2 with A1 and A7 unweakened. Every step below is carried out by the efficient algorithm that runs the adversary with the extractors and searches the accepted Actions for such a pair.

(1) *The note.* By A1, $\text{cm}^{\text{old}} = \text{NoteCommit}_{\text{rcm}^{\text{old}}}(\mathcal{M}^{\text{old}}) \neq \perp$, and likewise for w' . The two consumed notes thus have equal extracted commitments other than $\perp$, and by Remark “Binding for notes given by expanded receivers” they are equal, except with negligible probability. Equal notes have equal messages and trapdoors modulo p_{Vesta} , hence $\text{cm}^{\text{old}} = \text{cm}'^{\text{old}}$.

(2) *The key.* By A7, $\text{pk}_d^{\text{old}} = [\text{ivk}] \text{g}_d^{\text{old}}$ with

$$\text{ivk} := \text{Commit}_{\text{r}_{\text{ivk}}}^{\text{ivk}}(\text{Extract}_{\text{p}}(\text{ak}^{\text{p}}), \text{nk}) \neq \perp,$$

and, the notes being equal, $\text{pk}_d^{\text{old}} = [\text{ivk}'] \text{g}_d^{\text{old}}$ likewise. Both values of $\text{Commit}^{\text{ivk}}$ are integers below $p_{\text{Pallas}} < p_{\text{Vesta}}$, read as scalars (§2.1), and g_d^{old} is a non-identity point of a group of prime order p_{Vesta} ; hence $\text{ivk} = \text{ivk}'$. By the binding of ivk to (ak, nk) (Proposition “Binding of ivk to (ak, nk) ”, §3.2), under Assumptions 2.22 and 2.8,

$$(\text{Extract}_{\text{p}}(\text{ak}^{\text{p}}), \text{nk}, \text{r}_{\text{ivk}} \bmod p_{\text{Vesta}}) = (\text{Extract}_{\text{p}}(\text{ak}'^{\text{p}}), \text{nk}', \text{r}_{\text{ivk}}' \bmod p_{\text{Vesta}}),$$

except with negligible probability; in particular $\text{nk} = \text{nk}'$.

(3) *The nullifier.* By A5, $\text{nf}^{\text{old}} = \text{DeriveNullifier}_{\text{nk}}(\rho^{\text{old}}, \psi^{\text{old}}, \text{cm}^{\text{old}})$ (Definition 6.3), a function of its four arguments, which are equal for w and w' by (1) and (2). The two Actions therefore publish the same nullifier. No step uses v^{old} or A3. $\square$

Theorem 12.6 (No double-spending). *Under Assumptions 9.11, 2.22 and 2.8, for each pool, except with negligible probability, no committed note of non-zero value is consumed by two accepted Actions of that pool. The guarantee is per pool: the two pools’ nullifiers are checked against separate sets.*

Proof. Let two distinct accepted Actions of one pool consume the same note. Their consumed notes have the same extracted commitment, so by Lemma 12.5, except with negligible probability, the two Actions publish the same nullifier. Both belong to transactions of the adversary’s block chain, and the nullifier rule of “Nullifier sets” (§6.2), step (4) of the verification of §11.6, rejects a nullifier of the pool that repeats within a transaction or across the transactions of the chain. The event therefore lies in the negligible event of the lemma. The argument uses neither the value nor the membership of the note and holds for every note; the case of a committed note of non-zero value is the one that moves value. The nullifier sets of the two pools are separate (§6.2), so the argument concerns two Actions of one pool only. $\square$

Corollary 12.7 (Faerie Gold resistance). *Under Assumptions 9.11, 2.22 and 2.8, except with negligible probability:*

- (a) *the notes created by distinct accepted Actions of one pool have pairwise distinct elements ρ , and are therefore pairwise distinct;*
- (b) *two distinct notes have distinct nullifiers, whatever their nullifier deriving keys: no efficient algorithm outputs two distinct notes, with note commitments other than $\perp$, and two keys nk , nk' , equal or not, under which the two notes have equal nullifiers.*

Hence the Faerie Gold attack, as defined in “Nullifier chaining” (§6.3), fails within a pool: (a) excludes two Actions creating the same note, which would carry one nullifier, and (b) excludes distinct notes sharing a nullifier, so that consuming one note accepted by a recipient never blocks another. The uniqueness of ρ is pool-local: the pools’ nullifier sets are separate, so one value of ρ may occur once in each pool.

Proof. (a) The note that the witness of an accepted Action creates has $\rho^{\text{new}} = \text{nf}^{\text{old}}$ of that Action: the Action statement fixes $\rho^{\text{new}} := \text{nf}^{\text{old}}$ and binds the created note to the published cmx by A2 (Definition 9.2), so the chaining rule of §6.3 is enforced through Assumption 9.11. Distinct accepted Actions of one pool publish distinct nullifiers, by the nullifier rule (§6.2). Their created notes therefore have distinct ρ , and notes with a distinct component are distinct. The created note is the only opening of cmx that an efficient algorithm finds (Remark “Binding for notes given by expanded receivers”); a recipient that accepts a note from the Action obtains that note (Proposition 10.15(c)) and checks $\rho = \text{nf}^{\text{old}}$ itself (Proposition 10.15(a)).

(b) Two distinct notes have distinct note commitments, except with negligible probability, by the same remark; Proposition 6.8, which admits every pair of nullifier deriving keys, then excludes equal nullifiers.

Faerie Gold. Let a recipient accept a note n' from an accepted Action of a pool, and let nf' be its nullifier under the recipient’s nk. By (a) no other accepted Action of the pool creates n' , so the notes that the recipient accepts from distinct Actions are distinct. An accepted Action of the pool whose witness consumes a note $n \neq n'$ publishes, by A5, the nullifier of n under the witnessed key, which differs from nf' by (b). The value nf' therefore enters the pool’s nullifier set only through an Action that consumes n' itself, and the consumption of another note does not make n' unspendable.

Pool locality. The argument of (a) uses the nullifier set of one pool. An Orchard-pool Action and an Ironwood-pool Action may publish the same nullifier and create notes with the same ρ ; a consumption in one pool inserts its nullifier into that pool’s set only, and so blocks no note of the other pool (Remark “Pool locality”, §6.2). $\square$

12.3 Authorisation of spends

Definition 12.8 (Spend-authority experiment). A key holder generates a spending key sk with $\text{use_qsk} = \text{false}$ and derives $(\text{ask}, \text{nk}, \text{r}_{\text{vk}})$, ak^{P} and $\text{ak} = \text{Extract}_{\text{p}}(\text{ak}^{\text{P}})$ as in “The spending key and the spend-side secrets” (§3.1), and ivk as in “Viewing keys” (§3.2). The adversary receives the full viewing key $(\text{ak}, \text{nk}, \text{r}_{\text{vk}})$ but not ask, sees every public field, may create notes to any address of the key, and may obtain spend-authorisation signatures: on a query (α, m) , with $\alpha \in \mathbb{F}_{p_{\text{Vesta}}}$ and m a byte string, it receives a RedPallas signature on m under $\text{rsk} = \text{ask} + \alpha$ in the spend-authorisation instance, the signing model of Proposition 7.12. A *note of the key* is a note whose expanded receiver satisfies $\text{pk}_d = [\text{ivk}] g_d$ for the key’s ivk . The adversary *wins* if it outputs a block chain with an accepted Action whose witness consumes a note of the key, of any value, a dummy of value zero included, and whose spend-authorisation signature is valid under its rk on a signature digest m , where no query (α, m) had $\text{rk} = \text{ak}^{\text{P}} + [\alpha] G^{\text{SpendAuth}}$.

Theorem 12.9 (Spend authority). *For keys generated with $\text{use_qsk} = \text{false}$, and under Assumptions 9.11, 2.22, 2.8 (through the binding of $\text{Commit}^{\text{ivk}}$), 7.3 and 2.12, no efficient adversary wins the spend-authority experiment, except with negligible probability.*

Proof. Let an efficient adversary $\mathcal{A}$ win with probability ϵ , and write $G := G^{\text{SpendAuth}}$.

(1) *Idealised key.* By Lemma “Rejection in key generation” (§3.2), the key is replaced by a single draw from a uniform sk without rejection, at a cost negligible under Assumptions 2.12, 2.22 and 2.8. By Lemma 2.15 (Assumption 2.12), its triple $(\text{ask}, \text{nk}, \text{r}_{\text{ivk}})$, before the sign normalisation, is then replaced by independent uniform elements of $\mathbb{F}_{p_{\text{Vesta}}}$, $\mathbb{F}_{p_{\text{Pallas}}}$ and $\mathbb{F}_{p_{\text{Vesta}}}$, at a further negligible cost. Both hops apply because the event that $\mathcal{A}$ wins is efficiently decidable from the triple: an algorithm derives the key’s components, runs $\mathcal{A}$, answers its signing queries with ask , runs the extractors on its output and checks the winning condition. After this step nk and r_{ivk} are independent of ask , and, when $\text{ask} \neq 0$, the sign normalisation makes ak^{P} a uniform non-identity point of even y -coordinate (Remark “Hypothesis (H) for honest keys”, §11.3). This independence lets the reduction below simulate the full viewing key from ak^{P} alone.

(2) *Reduction.* An algorithm $\mathcal{B}$ plays the adversary of Proposition 7.12. It receives $Y = [x] G$ for x uniform on $\mathbb{F}_{p_{\text{Vesta}}}$, with access to H and to the signing oracle. If $Y = \mathcal{O}$ or Y has odd y -coordinate, it stops. Otherwise, an event of probability $(p_{\text{Vesta}} - 1)/(2p_{\text{Vesta}})$, since exactly half of the $p_{\text{Vesta}} - 1$ non-identity points have even y -coordinate (§2.1, on the star encoding), the point Y is distributed as ak^{P} in step (1) conditioned on $\text{ask} \neq 0$. Algorithm $\mathcal{B}$ sets $\text{ak}^{\text{P}} := Y$ and $\text{ak} := \text{Extract}_{\text{P}}(Y)$, draws nk and r_{ivk} uniformly, computes ivk , and gives $\mathcal{A}$ the full viewing key $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$. It answers each query (α, m) of $\mathcal{A}$ with the oracle’s signature under $x + \alpha$, forwards the queries of $\mathcal{A}$ to H , and simulates every other honest party and random oracle. When $\mathcal{A}$ outputs a block chain, $\mathcal{B}$ runs the extractors of Assumption 9.11, as stated after Definition “Pools, accepted Actions and witnesses”, and searches for an Action that meets the winning condition, all of whose parts it can check; failing one, it stops. For the Action found, with witness w , it determines $\theta \in \{1, -1\}$ with $\text{ak}_w^{\text{P}} = [\theta] Y$, stopping if there is none, and outputs the Action’s triple (rk, m, σ) together with the witnessed randomiser α_w and θ .

(3) *The witnessed key.* Except with negligible probability, $\text{ak}_w^{\text{P}} \in \{Y, -Y\}$. By A7, in its unweakened form, $\text{pk}_d^{\text{old}} = [\text{ivk}_w] \text{g}_d^{\text{old}}$ with $\text{ivk}_w := \text{Commit}_{\text{r}_{\text{ivk}_w}}^{\text{ivk}}(\text{Extract}_{\text{P}}(\text{ak}_w^{\text{P}}), \text{nk}_w) \neq \perp$, and the consumed note is a note of the key, so $\text{pk}_d^{\text{old}} = [\text{ivk}] \text{g}_d^{\text{old}}$. As in step (2) of the proof of Lemma 12.5, $\text{ivk}_w = \text{ivk}$, and the binding of ivk to (ak, nk) gives $\text{Extract}_{\text{P}}(\text{ak}_w^{\text{P}}) = \text{ak} = \text{Extract}_{\text{P}}(Y)$, under Assumptions 2.22 and 2.8. By Lemma 2.4, $\text{ak}_w^{\text{P}} \in \{Y, -Y\}$. The statement does not constrain the sign of the witnessed point (Remark “Sign of ak^{P} ”, §9.2), and the proof treats both signs. By A6,

$$\text{rk} = \text{ak}_w^{\text{P}} + [\alpha_w] G = [\theta] Y + [\alpha_w] G.$$

(4) *Forgery.* An answer of the signing oracle to a query (α, m') has validating key $Y + [\alpha] G$ and message m' . The winning condition excludes a query (α, m) with $\text{rk} = Y + [\alpha] G$, so the output triple is not the triple of an oracle answer: it is a forgery in the sense of Proposition 7.12. Algorithm $\mathcal{B}$ is efficient, and forges with probability at least

$$\frac{p_{\text{Vesta}} - 1}{2p_{\text{Vesta}}} (\epsilon - \nu)$$

for a negligible ν , the costs of step (1) and of the events excluded in step (3).

(5) *Discrete logarithm*. By Proposition 7.12(i), an algorithm $\mathcal{D}$ runs $\mathcal{B}$ twice on shared coins and outputs, with at least the probability that part states for the forging probability of $\mathcal{B}$, a scalar rsk^* with $\text{rk} = [\text{rsk}^*] G$ for the rk of the first run. It outputs $x' := \theta (\text{rsk}^* - \alpha_w)$, with θ and α_w of the first run. By step (3), $[\theta] Y = [\text{rsk}^* - \alpha_w] G$, so $x' = x$. For $\theta = 1$ this is part (ii) of the proposition with $\alpha^* = \alpha_w$; for $\theta = -1$ the same relation gives $-x = \text{rsk}^* - \alpha_w$, so a witnessed point $-\text{ak}^P$ is no easier than ak^P : either sign yields ask , the discrete logarithm of the honest ak^P . By Assumption 2.22 the probability that $\mathcal{D}$ outputs x is negligible, hence so is the forging probability of $\mathcal{B}$, and with it ϵ ; the loss factor $2p_{\text{Vesta}} / (p_{\text{Vesta}} - 1)$ is about 2. $\square$

Remark 12.10 (Authority without linkability). An Action publishes rk and a signature under $\text{rsk} = \text{ask} + \alpha$, never ak^P or ak . For α drawn by GenRandom afresh per Action, Proposition 7.10 (Assumption 7.3) makes the keys rk of n Actions independent of their spend validating keys up to statistical distance $n (\delta_{512} + (q_b + n) \cdot 2^{-640})$, with $\delta_{512} < 2^{-257}$, and makes (rsk, rk) distributed as a fresh key pair; the signature is a function of rsk , the digest and fresh randomness. The keys rk of distinct spends of one key are therefore unlinkable, while Theorem 12.9 shows that an accepted Action with a valid signature under its rk still demonstrates knowledge of ask : from an adversary that produces one for an honestly generated key without its ask , the reduction of its proof computes ask .

12.4 Privacy

Definition 12.11 (Public leakage and privacy preconditions). The *public leakage* of a bundle is its pool, its number of Actions, its anchor, its flags and its balancing value; the rest of the transaction, namely its transparent components, the other pool's bundle and its header with the expiry height, is public as well. The *private inputs* of a bundle are, per Action, its witness (Definition 9.2), the plaintext of its created note ("The note plaintext", §10.1), the outgoing viewing key under which its C_{out} is formed, or $\perp$, and whether each side is real or a dummy. The *privacy preconditions* are:

- (P1) every key involved, namely the key of each consumed note, of each recipient, and the key whose ovk the sender uses, is generated honestly with $\text{use_qsk} = \text{false}$;
- (P2) the bundle is an Ironwood-pool bundle generated by the procedure of "Construction of a transaction" (§11.5): the seed rseed of each created note fresh and uniform, the trap-door rcv uniform, the randomiser α drawn by GenRandom, each real consumed note a note of the pool that the procedure of "Trial decryption and note acceptance" (§10.4) accepted from an output of an accepted Action, and dummies as in the protocol specification, §"Dummy Notes (Orchard)": a dummy consumed note has a random spending key, value 0, ρ the x -coordinate of a uniform point, a uniform rseed and an unchecked path, and a dummy created note is an ordinary note of value 0 to an address of a freshly generated key;
- (P3) the authentication path is computed locally from public data (Lemma 5.15);

(P4) the adversary holds no viewing key of a key involved (no nk of a spender, no ivk of a recipient, no ovk of the sender), and no output is a coinbase output.

Within these preconditions the adversary may choose which notes are consumed and the recipients' addresses, and may know the consumed notes' contents.

Under (P2) the real consumed notes of one key have pairwise distinct elements ρ . An accepted note has $\rho = \text{nf}^{\text{old}}$ of the Action whose output carries it (Proposition 10.15(a)); one output yields at most one note under one ivk; and distinct Actions of the pool publish distinct nullifiers (§6.2). The experiment of the next theorem imposes this property directly.

Theorem 12.12 (Privacy of an Action). *Assume preconditions (P1) to (P4), and let two sequences of private inputs of one Ironwood-pool bundle have equal public leakage, both satisfy Definition 9.2 with the common anchor and flags, and each Action consume either a leaf of the tree at that anchor not consumed before or a dummy, and create either a note to an honestly generated address or a dummy. Under Assumptions 9.14, 2.12, 3.13, 6.2, 3.16, 2.8, 10.4, 10.5 and 7.3, with Assumption 2.22 for Lemma “Rejection in key generation” (§3.2) and the redraw of the note seed (§4.3), the public data of the two bundles (per Action cv^{net} , nf, rk, cmx, epk*, C_{enc} , C_{out} and the spend-authorisation signature; the proof and the binding signature) are computationally indistinguishable. In particular, beyond its public leakage an Action reveals nothing about its private inputs, and an Action whose spend or output side is a dummy is indistinguishable from one whose sides are real.*

Precisely, in the following experiment every efficient adversary has negligible advantage. Keys are generated as in (P1), and the adversary receives addresses of them at indices of its choice. It outputs the leaves of the Ironwood-pool tree through an anchoring block, the rest of a transaction, flags, and two specifications of n Actions with equal balancing values. A specification gives, per Action, a consumed side, either “dummy” or a note at one of these addresses whose extracted commitment is a leaf; a created side, either “dummy” or one of these addresses with a value and a memo; and the sender's ovk, that of one of the keys or $\perp$. The consumed leaves of a specification are pairwise distinct, the consumed notes of one key have pairwise distinct ρ , and the construction of (P2) yields witnesses of Definition 9.2 with the anchor and flags. A challenger builds the bundle of specification β , for a uniform bit β , by that construction and gives the adversary its public data; the adversary outputs a bit β' , and its advantage is $|\Pr[\beta' = 1 \mid \beta = 0] - \Pr[\beta' = 1 \mid \beta = 1]|$.

Proof. For $\beta \in \{0, 1\}$ let G_0^β be the experiment with specification β . Each of the games G_1^β to G_7^β below changes the probability that the adversary outputs 1 by a negligible amount (Crypto Guide, §“The hybrid argument”, Lemma “Hybrid lemma”; each game is reached in polynomially many steps, one key or one Action at a time), and G_7^0 and G_7^1 are identically distributed. Every reduction runs the experiment with the adversary and samples every value that its hop does not concern. The keys involved are those of (P1), the spending keys of the dummy consumed notes and the keys of the dummy created notes.

Game G_1 (*proof*). The aggregate proof is replaced by the output of the simulator of Assumption 9.14 on the primary inputs, which are public; the witnesses satisfy Definition 9.2 by hypothesis. The cost is the statistical distance of that assumption. From G_1 on, a witness enters the game

only through the public fields computed from its components.

Game G_2 (*keys*). Each key involved is replaced in turn, as in the preliminary step of the proof of Proposition 10.6 and hybrids H_1 to H_3 of the proof of Proposition 6.10: by a single draw without rejection (Lemma “Rejection in key generation”, §3.2; Assumptions 2.12, 2.22 and 2.8); its triple $(\text{ask}, \text{nk}, r_{\text{ivk}})$ by independent uniform elements (Lemma 2.15); and its triple $(\text{ivk}, \text{dk}, \text{ovk})$ by $(\text{Extract}_p(W), u_1, u_2)$, for a uniform point W and uniform 32-byte strings u_1 and u_2 (Assumption 3.13, whose reduction draws ask and nk itself, and the negligible event that $\text{ask} = 0$ or that the hash point of $\text{Commit}^{\text{ivk}}$ is $\perp$). The hops apply because after G_1 the trapdoor r_{ivk} enters the game only through $(\text{ivk}, \text{dk}, \text{ovk})$. From G_2 on, the components ask , nk , ivk , dk and ovk of each key are independent, and dk and ovk are uniform.

Game G_3 (*outgoing ciphertexts*). Each C_{out} is replaced by the encryption of a fixed 64-byte string under an independent uniform key, by Proposition “Confidentiality of the outgoing ciphertext” (§10.3). For $\text{ovk} \neq \perp$, the key ock is a value of the random oracle PRF^{ock} (Assumption 10.4) at an input that begins with the uniform secret ovk , which q queries of the adversary hit with probability at most $q \cdot 2^{-256}$. Distinct Actions give distinct inputs, their epk differing except with negligible probability, so each ock is uniform and encrypts once, and Assumption 10.5(a) replaces the plaintext. For $\text{ovk} = \perp$, the key is uniform by construction and Assumption 10.5(a) applies directly.

Game G_4 (*randomised keys*). Each randomiser α_i is replaced by a uniform scalar, at a cost of at most $\delta_{512} + q_b \cdot 2^{-640}$ each, $\delta_{512} < 2^{-257}$, for q_b queries to H (Lemma “Distance of GenRandom from uniform”, §7.2; Assumption 7.3). Then $y_i := \text{rsk}_i = \text{ask} + \alpha_i$ is uniform and independent of ask and of every other value, and $\text{rk}_i = [y_i] G^{\text{SpendAuth}}$ (Proposition 7.10(i)); the signature σ_i is computed under y_i . From G_4 on, no ask enters the game.

Game G_5 (*nullifiers*). For each key that consumes a note in the bundle, real or dummy, the key nk enters G_4 only through the nullifiers of those notes: G_1 removed it from the proof and G_2 from the viewing keys. As in hybrids H_4 and H_5 of the proof of Proposition 6.10, $\text{PRF}_{\text{nk}}^{\text{nfOrchard}}$ is replaced by a uniformly random function (Assumption 6.2), and each multiplier of K^{Orchard} by a uniform scalar $u_i \in \mathbb{F}_{p_{\text{Vesta}}}$, at a statistical cost below 2^{-167} each; the elements ρ of the consumed notes of one key are pairwise distinct by the definition of the experiment, and a dummy consumed note has a key of its own. Each nullifier is then $\text{Extract}_p([u_i] K^{\text{Orchard}} + \text{cm}_i)$, the x -coordinate of a uniform point whatever cm_i is, since $K^{\text{Orchard}} \neq \mathcal{O}$ except with probability $1/p_{\text{Vesta}}$ (Assumption 2.8).

Game G_6 (*ephemeral keys and note ciphertexts*). For each created note in turn, real or dummy, epk is replaced by a uniform non-identity point and C_{enc} by the encryption of a fixed 564-byte string under an independent uniform key, by steps (1) to (4) of the proof of Proposition 10.6 (Assumptions 10.4, 3.16, 10.5, 2.8 and 2.12), which keep cmx . The other public data, the data z of that proposition, depend after G_2 to G_5 on the recipient’s key only through the addresses of that key given to the adversary and the outputs to them. The reduction of step (2) of that proof embeds the exponent a as the ivk of the recipient’s key in every such address, computing its transmission key as $[t]([a] P)$ for the diversified base $[t] P$ that it programs, and computes every other output to that key from its own esk ; the proof is otherwise as written.

Game G_7 (*note commitments*). For each created note in turn, cmx is replaced by the x -coordinate of a uniform point. After G_6 the seed rseed of the note enters the game only through ψ and rcm , that is through cm : step (1) of G_6 replaced esk , and the ciphertexts no longer depend on the plaintext. The proof of Proposition 4.8 then places cm within $\epsilon_{\text{PRF}} + \delta + \epsilon_{\perp}$ of a uniform point, with

$\delta < 2^{-257}$ and $\epsilon_{\text{PRF}}, \epsilon_{\perp}$ negligible under Assumptions 2.12, 2.22 and 2.8.

Value commitments and the binding signature. In G_7^β every field other than the net value commitments and the binding signature is distributed independently of β : the nullifiers and the extracted commitments are x -coordinates of uniform points, each epk is a uniform non-identity point, the ciphertexts encrypt fixed strings under fresh keys, each rk_i and σ_i come from a fresh key y_i , the proof is simulated from the primary inputs, and the rest of the transaction is common. The net values v_i^β of specification β enter only

$$\text{cv}_i^{\text{net}} = [v_i^\beta] V^{\text{Orchard}} + [\text{rcv}_i] R^{\text{Orchard}}, \quad \text{bsk} = \sum_{i=1}^n \text{rcv}_i,$$

and, through these, the digest, the primary inputs of the simulator and bvk . Let λ be the discrete logarithm of V^{Orchard} to the base R^{Orchard} , which exists because R^{Orchard} generates the group (§8.1). The map

$$(\text{rcv}_i)_i \mapsto (\text{rcv}_i + (v_i^0 - v_i^1) \lambda)_i$$

is a bijection of $\mathbb{F}_{p_{\text{Vesta}}}^n$. It carries the net value commitments of specification 0 to those of specification 1, and it preserves $\sum_i \text{rcv}_i$, because $\sum_i v_i^0 = \sum_i v_i^1$, the common balancing value (step (6) of the construction of §11.5). The rcv_i being uniform and independent, the tuple $(\text{cv}_1^{\text{net}}, \dots, \text{cv}_n^{\text{net}}, \text{bsk})$ has the same distribution for both β , and G_7^0 and G_7^1 are identically distributed. This step is perfect and needs no reduction; the game is not required to compute λ .

The advantage of the adversary is therefore at most the sum of the costs of the hops for $\beta = 0$ and $\beta = 1$, which is negligible. In G_7 a dummy side differs from a real side only through the value in cv^{net} , so two sequences that differ in whether a side is a dummy, with equal public leakage, are covered by the same argument. $\square$

Remark 12.13 (Non-claims). Theorem 12.12 hides nothing of the public leakage: each bundle’s balancing value, so the net amount that each transaction moves across a pool’s boundary is public; each pool’s chain value pool balance, the negation of the sum of its balancing values (“Chain state and pool rules”, §11.4), so a pool hides which notes are held by whom, never the aggregate value it holds; the number of Actions and the presence of each component; the flags; the transparent components; the fee; the anchor, which confines the consumed note to the leaves of the anchored tree (“Authentication paths”, §5.3); and the transaction’s timing and expiry height. Per Action the chain gains one nullifier and one extracted note commitment, which by the theorem reveal neither sender, recipient nor amount. The theorem claims nothing for coinbase outputs, whose outgoing ciphertexts decrypt under the all-zero ovk (§11.4); nothing against a holder of a viewing key of a key involved, whose powers are stated in “Nullifier uniqueness and unlinkability” (§6.4), “The outgoing ciphertext” (§10.3) and “Trial decryption and note acceptance” (§10.4); nothing for an Orchard-pool bundle, whose created notes use lead byte $0x02$ and a derivation on which the volume does not rely (§10.4); and nothing about linkage through information outside the transaction, such as an address given to several parties, voluntary disclosures, or wallet and network metadata.

12.5 The end-to-end theorem

Theorem 12.14 (End-to-end security). *Assume, listed once: Assumption 2.22; Assumptions 9.11 and 9.14, with Assumption 9.10 in the supporting analysis of Assumption 9.11; Assumption 3.16; Assumptions 2.12 and 6.2; Assumption 3.13; Assumptions 10.4 and 10.5; Assumption 11.9; and Assumptions 2.8 and 7.3, for GroupHash and H as random oracles. Clauses (iv) and (v) are key-derived and are claimed for keys generated with `use_qsk = false`; clauses (i) to (iii) use no property of key generation. For each pool, no efficient adversary achieves any of the following, except with negligible probability:*

- (i) *an accepted Action for which the extractor of Assumption 9.11 outputs no witness satisfying Definition 9.2; or a second opening, other than its witnessed created note, of the cmx of an accepted Action; or an accepted Action whose witnessed v^{old} is non-zero and whose consumed note is not a committed note of the pool, a leaf of the tree at the Action's anchor (membership soundness) [R1 and R2 soundness];*
- (ii) *two accepted Actions of the pool consuming one committed note of non-zero value [R3 soundness];*
- (iii) *an accepted bundle whose balancing value differs, as an integer, from the sum of its Actions' $v^{\text{old}} - v^{\text{new}}$ [R4 soundness];*
- (iv) *winning the spend-authority experiment of "Authorisation of spends", that is, an accepted Action consuming a note of an honestly generated key without a signature of the key holder, the adversary holding the key's full viewing key but not its ask; or an accepted transaction that changes the effecting data of a transaction T while retaining one of its Actions, where the Actions of T are authorised by holders of honestly generated keys who sign as hypothesis (S) of Proposition 11.10 states [R5];*
- (v) *for Ironwood-pool bundles, distinguishing the public data of two bundles with equal public leakage under preconditions (P1) to (P4) of "Privacy", in particular a real Action from a dummy one [R1 hiding; R2 hiding, without identifying the note; R3 hiding, without a public list of consumed notes; R4 hiding, without disclosed amounts].*

The bracketed tags refer to the requirement map of "The Action statement" (§9.2, Table 5), which is not repeated.

Proof. Each clause is a result of this section or of an earlier one.

(i) The first part is Assumption 9.11, applied to each bundle as stated after Definition "Pools, accepted Actions and witnesses", with Remark " $\perp$ -weakened conditions" (§9.2) for the unweakened forms of A1, A2, A3 and A7. By A2 the created note opens cmx, and by Remark "Binding for notes given by expanded receivers" it is the only opening that an efficient algorithm finds. For $v^{\text{old}} \neq 0$, condition A3, outside its $\perp$ case, makes the fold of A3 on $(\text{Extract}_p(\text{cm}^{\text{old}}), \text{pos}, \text{path})$, with the witnessed encodings c, reach the bundle's anchor rt , and the anchor rule, step (3) of the verification of §11.6, makes rt the anchor of an earlier block in the pool. By A1, cm^{old} is a point, so Proposition 5.9(iii), which applies to that fold (§9.2), gives, under Assumptions 2.22 and 2.8,

that $\text{Extract}_p(\text{cm}^{\text{old}})$ is the leaf appended at position pos , in the anchoring block or earlier: the consumed note is a committed note of the pool.

(ii) Theorem 12.6.

(iii) Theorem 12.4, which adds Assumption 7.3.

(iv) The first part is Theorem 12.9. The second is Proposition 11.10, under Assumptions 11.9, 2.22 and 7.3; its hypothesis (H) holds for honestly generated keys up to a negligible change in the probability of every efficiently decidable event, the event of that proposition included, under Assumptions 2.12, 2.22 and 2.8 (Remark “Hypothesis (H) for honest keys”, §11.3).

(v) Theorem 12.12.

Clauses (i) to (iii) use Assumptions 9.11, 2.22, 2.8 and 7.3 only, and no property of key generation; clauses (iv) and (v) use honest key generation with $\text{use_qsk} = \text{false}$. Each assumption of the list is used by some clause, and Assumption 9.10 enters only through the supporting analysis of Assumption 9.11 (“The Action circuit and the Halo 2 proof”, §9.3). $\square$

Remark 12.15 (Nature of the guarantees). Beyond Assumption 9.11, every soundness reduction ends in a discrete-logarithm problem on Pallas: either a non-trivial relation among values of GroupHash, modelled as a random oracle (Assumption 2.8), namely Sinsemilla collisions and $\perp$ outputs (Proposition 2.24), with membership soundness (Proposition 5.9), the binding of NoteCommit and of Commit^{ivk}, nullifier uniqueness (Proposition 6.8) and the independence of V^{Orchard} and R^{Orchard} (Proposition 8.10); or the discrete logarithm of a given point, in the spend-authority reduction through the RedPallas extraction of Proposition 7.12. Every extraction from a RedPallas signature, the binding signature included, holds in the random-oracle model for H (Assumption 7.3). The digest step of bundle binding ends instead in a BLAKE2b collision (Assumption 11.9). Soundness is therefore computational and conditional on Assumption 9.11, which “The Action circuit and the Halo 2 proof” (§9.3) classifies as designed but unspecified, with discrete logarithms on Vesta entering only its supporting analysis; no concrete bound is asserted, since that assumption carries no loss factor. In clause (v), the contribution of the proof is statistical (Assumption 9.14; *Halo 2 Guide*, §“Zero knowledge: hiding the witness in Halo 2”); that of the value commitments with the binding signature is perfect (the final step of the proof of Theorem 12.12); that of rk and the spend-authorisation signatures is statistical in the random-oracle model for H (game G_4 there); and the unlinkability of every other published field is computational. Assumption 6.2 is cryptanalytic, with no reduction to a standard problem.

VI

Wallet Guide

The Zcash wallet layer: keys, transactions, and light-client synchronisation

Abstract

This volume of *The Zcash Arboretum* states the protocols that a Zcash wallet runs: the derivation of its keys and addresses from one seed; the discovery of its notes from the chain through an untrusted light-client server, and what that server learns; and the construction, authorisation and submission of its payments. Each protocol is stated as a definition, construction or rule, and its correctness, security or privacy guarantee as a lemma, proposition or theorem under named assumptions. The subject is the wallet of a user of the Orchard protocol who holds funds in the Ironwood pool, the sealed Orchard pool, the Sapling pool and the transparent pool; the light-client service is an abstract query and response interface, and no implementation is described. The volume assumes the *Math Guide*, the *Crypto Guide*, the *Halo 2 Guide* and the *Consensus Guide*, and cites the *Ironwood Guide* for every construction of the shielded protocol. It is non-normative: the Zcash Protocol Specification and the Zcash Improvement Proposals are authoritative.

1 Introduction

This section fixes the subject of the volume, its status and the vocabulary that every later section uses.

1.1 Scope and status

This volume states the protocols by which a wallet, in the sense of “Parties, pools, and classifications” (§1.2), performs three tasks. It derives and holds keys and addresses: hierarchical key derivation (ZIP 32), and unified addresses and unified viewing keys (ZIP 316). It discovers and tracks its notes: the compact-block format (ZIP 307) as an abstract message format, trial decryption, nullifier tracking, the retention of the note commitment tree information that its witnesses need, and what an untrusted light-client server learns. It pays: anchors and confirmation depths (ZIP 315), expiry (ZIP 203), the conventional fee (ZIP 317), transaction construction (note selection, change, dummy Actions), partially created transactions (ZIP 374), the transaction lifecycle from submission to a terminal state, and payment requests (ZIP 321).

The subject is the wallet of a user of the Orchard protocol who holds funds in the Ironwood pool, the sealed Orchard pool, the Sapling pool and the transparent pool. Sapling constructions and transparent scripts are cited to the protocol specification and to the BIPs where a wallet protocol needs them, and are never constructed; constructions of the shielded protocol are cited to the *Ironwood Guide* by section title, and are never re-derived. The light-client service is an abstract query and response interface, and tree storage is the information that a wallet retains; no implementation is described. The volume is non-normative: the protocol specification and the ZIPs are authoritative.

The volume describes the protocols as specified for NU7, whose deployment ZIP, ZIP 259, has status Draft.

ZIP 307 specifies the compact format for the Sapling pool only: a compact block of height, block hash and transactions, and a compact transaction of index, transaction identifier, and compact Sapling spends and outputs. The parent hash, the tree sizes, the compact Actions of both Orchard-protocol pools, the transparent part and pool selection are designed but unspecified in the sense of the *Ironwood Guide*, Definition “Epistemic classes” (“Scope and status”), and the volume states them abstractly in “Compact blocks (ZIP 307)” (§4).

Rules of the Ironwood pool and of NU6.3 are cited to the NU6.3 edition of the protocol specification, labelled “NU6.3 proposal”, whose section numbers equal those of the default edition; the text on `use_ysk` for protocol specification, §“Orchard Key Components”, is in no edition and is cited to ZIP 2005, “Changes to the Protocol Specification”, directly.

Table 1 gives the status, and only the status, of each ZIP that the volume cites. Every later citation of these ZIPs relies on the table for status. ZIP 258 activates ZIP 2005 with NU6.3 (ZIP 258, “ZIP 2005 activation”).

1.2 Parties, pools, and classifications

A *wallet* is a party that holds a seed and the keys derived from it, and no copy of the chain. A *light-client server* is a party that answers the queries of a wallet from its own view of the best chain, the best valid block chain of the *Consensus Guide*, “Work, and the best chain” (Definition “Best valid block chain”). The *network* is the set of the nodes that relay transactions and blocks and of the miners that produce blocks. The trust placed in the server is not fixed here; it is an assumption of “Architecture and authority” (§5.1).

A wallet handles funds in four value pools. The first is the *transparent pool*. The second is the *Sapling pool* (protocol specification, §“Chain Value Pool Balances”), whose constructions are cited, never constructed. The third and fourth are the *Ironwood pool* and the sealed *Orchard pool*, the two value pools of the Orchard protocol (*Ironwood Guide*, “The Orchard protocol and its two pools”, Definitions “Orchard protocol” and “Orchard pool, Ironwood pool”; ZIP 229, Terminology); each has its own note commitment tree, its own nullifier set and its own chain value pool balance. The rules that seal the Orchard pool are cited there (ZIP 258, “Consensus rules from NU6.3 activation”), and are stated for wallets in “Payments to pools” (§11.2).

Heights are counted as in the *Ironwood Guide*, “Scope and status” (Definition “Block chain, transaction, consensus rules”), the genesis block having height 0. The *tip* T is the height of the last block of the wallet’s view of the best chain. Every other height (the target height, the anchor

Table 1. Status of the ZIPs cited, as given in the preamble of each ZIP.

ZIP	Title	Status
32	Shielded Hierarchical Deterministic Wallets	Final
48	Transparent Multisig Wallets	Draft
173	Bech32 Format	Final
203	Transaction Expiry	Final
211	Disabling Addition of New Value to the Sprout Chain Value Pool	Final
212	Allow Recipient to Derive Ephemeral Secret from Note Plaintext	Final
218	25-second Block Target Spacing	Draft
225	Version 5 Transaction Format	Final
227	Issuance of Zcash Shielded Assets	Draft
229	Version 6 Transaction Format	Draft
231	Memo Bundles	Draft
244	Transaction Identifier Non-Malleability	Final
248	Extensible Transaction Format	Draft
256	Deployment of Consensus Bug Fixes Between NU6.1 and NU6.2	Final
258	Deployment of the NU6.3 Network Upgrade	Draft
259	Deployment of the NU7 Network Upgrade	Draft
302	Standardized Memo Field Format	Draft
307	Light Client Protocol for Payment Detection	Draft
312	FROST for Spend Authorization Multisignatures	Draft
314	Privacy upgrades to the Zcash light client protocol	Reserved
315	Best Practices for Wallet Implementations	Draft
316	Unified Addresses and Unified Viewing Keys	Revision 0 Active; Revision 1 Withdrawn; Revision 2 Draft
317	Proportional Transfer Fee Mechanism	Revision 0 Active; Revisions 1 and 2 Draft
318	Orchard to Ironwood Migration	Draft
320	Defining an Address Type to which funds can only be sent from Transparent Addresses	Active
321	Payment Request URIs	Active
326	NU6.3 Consequences for Wallets	Draft
374	Partially Created Zcash Transaction Format	Revision 0 Draft
401	Addressing Mempool Denial-of-Service	Active
2003	Disallow version 4 transactions	Draft
2005	Ironwood Quantum Recoverability	Proposed

height, the expiry height) is defined where it is first used.

Definition 1.1 (Classes of claim). The classes *specified* and *designed but unspecified* are those of the *Ironwood Guide*, Definition “Epistemic classes” (“Scope and status”), applied here to wallet rules as well as to claims about the deployed protocol. A wallet rule is *specified* when normative text of a ZIP or of the protocol specification fixes it, the status of the ZIP being that of Table 1. A wallet rule is *designed but unspecified* when wallets are designed to follow it and no normative text fixes it. This volume adds a third class: a question is an *open problem* when neither a ZIP nor a stated design settles it.

Every design-stage statement of the volume carries one class of Definition 1.1, and every deployed rule is cited at its use to the section of the protocol specification or of the ZIP that fixes it. A specified rule keeps the normative level of its source (MUST, SHOULD, MAY).

2 Hierarchical key derivation (ZIP 32)

This section derives every key of a wallet from one seed. It fixes the seed and its fingerprint, the hardened-only derivation of Orchard spending keys and the standard account path, and the passage from an account spending key to the key components that the *Ironwood Guide* constructs from a spending key, including the alternative branch of ZIP 2005. It then constructs the internal keys of an account and proves the separations between the two scopes, bounds the probability that an account key is invalid, and closes with the domains of key derivation.

2.1 Wallet seeds

Definition 2.1 (Wallet seed). A *wallet seed* is a byte string S with $32 \leq \text{length}(S) \leq 252$, where $\text{length}(S)$ is its length in bytes (MUST). The method that generates S MUST give it at least 256 bits of entropy; the requirement extends to any mnemonic or input randomness from which S is obtained (ZIP 32, “Specification: Wallet seeds”). Every key of every account of the wallet is derived from S : the account spending key in §2.3, and what recovery needs beyond S in §2.4.

Every key of every account is a deterministic function of S , so a seed space of lower entropy is searched once for all accounts, and across many wallets as a multi-target search that matches the first derived addresses or viewing keys against a set of observed ones (ZIP 32, “Rationale for the 256-bit entropy requirement”).

Construction 2.2 (Seed fingerprint). For a wallet seed S , the *seed fingerprint* is

$$\text{SeedFP}(S) := \text{BLAKE2b-256}(\text{Zcash_HD_Seed_FP}, [\text{length}(S)] \parallel S),$$

unkeyed BLAKE2b with a 32-byte output and the 16-byte personalisation shown, over the single length byte of S followed by S ; the length fits one byte because $\text{length}(S) \leq 252$. It MAY be used to identify the seed of a hierarchical deterministic wallet. Its string form is Bech32m (BIP 350) with human-readable part `zip32seedfp` and the 32 fingerprint bytes as data; no short tag is defined (ZIP 32, “Seed Fingerprints”). A partially created transaction carries the fingerprint together with a derivation path, so that a signing party locates its key (§12.3).

A partially created transaction relies on collision resistance of BLAKE2b-256 under this personalisation (*Crypto Guide*, §“Security notions: preimage, second-preimage, and collision resistance”, Definition “Collision resistance”); Lemma 2.5 additionally models it as a random oracle, so that the fingerprint reveals nothing of the keys. The fingerprint of a full viewing key is constructed with the key components, in §2.4.

2.2 Hardened-only derivation

ZIP 32 instantiates its hardened-only derivation for Orchard with the master-key domain `ZcashIP320rchain`, a 16-byte BLAKE2b personalisation that SHOULD be disjoint from every other BLAKE2b domain separator of Zcash, and the child-key domain `0x81`, a lead byte of PRFexpand (ZIP 32, “Specification: Hardened-only key derivation”, “Instantiation”, and “Specification: Orchard key derivation”). An *Orchard extended spending key* is a pair (sk, c) of 32-byte strings, where sk is a spending key in the sense of the *Ironwood Guide*, §“The spending key and the spend-side secrets”, and c is the *chain code* (ZIP 32, “Orchard extended keys”). Only hardened derivation is defined for Orchard: child derivation fails at every index $i < 2^{31}$. ZIP 32 defines no extended full viewing key for Orchard; a full viewing key has no child-derivation capability for which a chain code would serve. For $0 \leq i < 2^{31}$ write $i' := i + 2^{31}$.

Construction 2.3 (Orchard master key generation). For a wallet seed S (Definition 2.1), let

$$I := \text{BLAKE2b-512}(\text{ZcashIP320rchain}, S),$$

unkeyed BLAKE2b with a 64-byte output, and let $sk_m := I[0..32)$ and $c_m := I[32..64)$, the first and the last 32 bytes. The *master extended spending key* is $m_{\text{Orchard}} := (sk_m, c_m) =: \text{MKG}(S)$ (ZIP 32, “Hardened-only master key generation” and “Orchard master key generation”).

Construction 2.4 (Orchard child key derivation). For a parent extended spending key $(sk_{\text{par}}, c_{\text{par}})$ and an index $i \in \{0, \dots, 2^{32} - 1\}$: if $i < 2^{31}$, derivation fails; otherwise

$$I := \text{PRFexpand}_{c_{\text{par}}}([0x81] \parallel sk_{\text{par}} \parallel \text{l2LEOSP}_{32}(i)),$$

a 64-byte string, and $\text{CKD}((sk_{\text{par}}, c_{\text{par}}), i) := (I[0..32), I[32..64))$. Here $\text{l2LEOSP}_{32}(i)$ is the 4-byte little-endian encoding of i , and $\text{PRFexpand}_k(t) = \text{BLAKE2b-512}(\text{Zcash_ExpandSeed}, k \parallel t)$ is the *Ironwood Guide*, §“Pseudorandom functions and field reductions”, Construction “Expansion function” (ZIP 32, “Hardened-only child key derivation” and “Orchard child key derivation”). The parent chain code is the key of the expansion; the parent spending key is message data.

Lemma 2.5 (Pseudorandomness of derived spending keys). *Assume the following.*

- (i) *BLAKE2b-512 under the personalisation `ZcashIP320rchain` and BLAKE2b-256 under the personalisation `Zcash_HD_Seed_FP` are modelled as random oracles* (*Crypto Guide*, §“*The random oracle model*”, Definition “*Random oracle*”), *independent of each other and of every other personalised use of BLAKE2b* (*Crypto Guide*, §“*Domain separation and per-*

sonalisation”, Proposition “Domain separation yields independent oracles”), and of every other derivation from S that is modelled as a random oracle.

- (ii) The seed S has min-entropy at least 256 bits, the reading of the entropy requirement of Definition 2.1 that the lemma uses.
- (iii) The Ironwood Guide’s Assumption “Pseudorandomness of PRF expansion” (§“Pseudorandom functions and field reductions”) holds.

Let N be a fixed finite set of distinct nodes of the Orchard derivation tree rooted at m_{Orchard} , each reached by Construction 2.4 along hardened indices, and let T be the subtree spanned by the paths from the root to the nodes of N . Then the spending keys of the nodes of N , the chain codes of those nodes of N that are not a proper ancestor of another node of N , and $\text{SeedFP}(S)$ are jointly computationally indistinguishable from independent uniform 32-byte strings. For an adversary making q random-oracle queries, the distinguishing advantage is at most $q \cdot 2^{-256}$ plus one PRF advantage for each node of T whose chain code keys a derivation in T ; there are at most as many such nodes as T has edges.

Proof. A hybrid argument (Crypto Guide, §“The hybrid argument”, Lemma “Hybrid lemma”). Game 0 is the real experiment. Game 1 answers every oracle query whose input is determined by S (the master query of Construction 2.3, the input of SeedFP and the inputs of the other modelled derivations) by fresh uniform strings, independent of S . The two games differ only if the adversary itself queries such an input. In Game 1 its view is independent of S until that happens, so each of its q queries does so with probability at most 2^{-256} by hypothesis (ii), and the games differ by at most $q \cdot 2^{-256}$. In Game 1 the pair $(\text{sk}_m, \text{c}_m)$ and $\text{SeedFP}(S)$ are independent uniform strings.

Then visit the nodes of T that have a child in T , each parent before its children. At such a node v the chain code c_v is, by the earlier hops, a uniform string that is used only as a PRFexpand key and is not released, because v is a proper ancestor of a node of N . The hop replaces $\text{PRFexpand}_{\text{c}_v}$ by a uniformly random function. The children of v in T have distinct indices, hence distinct inputs $[\text{0x81}] \parallel \text{sk}_v \parallel \text{I2LEOSP}_{32}(i)$, so their 64-byte outputs become fresh uniform strings (Ironwood Guide, §“Pseudorandom functions and field reductions”, Lemma “Independence of domain-separated expansions”, first conclusion, applied to a uniform key; its statistical-distance clause is not used). The hop costs the PRF advantage of a reduction that samples every other node itself and forwards the queries of v to its oracle. In the last game every released value is a fresh uniform string; the chain codes of proper ancestors, which are not fresh relative to their children, are exactly those the lemma does not release. $\square$

Remark 2.6 (The spending key as a leaf capability). In Construction 2.4 the parent chain code keys the expansion and the parent spending key is message data, so deriving children requires the pair (sk, c) . A 32-byte spending key alone derives no child: by Lemma 2.5, without c the children are pseudorandom (ZIP 32, “Orchard child key derivation”). A wallet that keeps sk_{acct} and not c_{acct} keeps no power to derive keys below the account.

2.3 The standard account path

Definition 2.7 (Account spending key). For a wallet seed S , a coin type t and an account index $a \in \{0, \dots, 2^{31} - 1\}$,

$$(sk_{\text{acct}}, c_{\text{acct}}) := \text{CKD}(\text{CKD}(\text{CKD}(\text{MKG}(S), 32'), t'), a'),$$

the path $m_{\text{Orchard}}/purpose'/coin_type'/account'$. The purpose index is $32' = 0x80000020$, following BIP 43; the coin type is the SLIP 44 value, 133 on Mainnet and 1 on Testnet, every test network sharing the value 1; accounts are numbered sequentially from 0, as in BIP 44. All three levels are hardened. The *account spending key* is sk_{acct} ; no construction of this volume uses c_{acct} (Remark 2.6). A wallet implementing Orchard ZIP 32 derivation **MUST** support this path for every account in $\{0, \dots, 2^{31} - 1\}$, **MUST** support generating the default payment address, at diversifier index 0, of every account it supports, and **MAY** generate further diversified addresses of an account (ZIP 32, “Key path levels” and “Orchard key path”).

The path ends at the account. Addresses are not path children: an account has 2^{88} diversified addresses per scope, the images of the diversifier indices (*Ironwood Guide*, §“Diversified addresses”, Construction “Diversified address”). There is no change level. A shielded output does not show its address on chain, so returning change to the originating address is indistinguishable from using a change address; the wallet separates change from received funds by the internal scope below the account (§2.5; ZIP 32, “Key path levels”). Figure 1 shows the derivation from the seed to the addresses of both scopes.

Example 2.8 (The account path of a fixed seed). Let S be the 32 bytes $0x00, 0x01, \dots, 0x1f$, on Mainnet (coin type 133), account 0. The pairs (sk, c) along the path, in hexadecimal, are:

Node	Value
m	sk 7eee3c1017870990a3dd6891b82f80be8976c1e7dc20d60817a5e88e8b2cd4b8 c ab8b7a00509ef20e469b5292b61d474b7cfcfb1657924cda720250ae40526677
$m/32'$	sk 93af0a87c2e4704d8825e145557c6a5d4dcbc9016170e276a2080a2dc9eb0f87 c 6e1e808d410d6d1f84e62e67c8b60e80e5ee4de5021d575c0433a625d609c45c
$m/32'/133'$	sk 149fcee594a2a03de277ab919f22611d0ef6aa8e8e9a0bef32f01a0bcb0e8c4c c 07d13fdeccd21da16b467f8e785edc32f74cee541bede0394647f10fdb7114f
$m/32'/133'/0'$	sk b67d8d87cab9189500afb45dbca9f92c924c1de9d9ee1451be4a78313cb223b4 c 7ff0a6392e144d4c8f45bca21fb7827b508446ac069890aefbebbe388b7a84c0

The last spending key is sk_{acct} , and it is valid in the sense of §2.6; the last chain code is c_{acct} , which no construction uses. The master spending key agrees with the first Orchard test vector of ZIP 32, “Test Vectors”.

2.4 Orchard key components of an account

The *Ironwood Guide* constructs from a spending key sk the components ask , nk and r_{ivk} (§“The spending key and the spend-side secrets”), the spend validating key ak^P and its coordinate ak (same subsection), the full viewing key $fvk = (ak, nk, r_{\text{ivk}})$, the incoming viewing key scalar $ivk = \text{Commit}_{r_{\text{ivk}}}^{\text{ivk}}(ak, nk)$, and (dk, ovk) as the two halves of

$$\text{PRFexpand}_{\text{LE}_{256}(r_{\text{ivk}})}([0x82] \parallel \text{I2LEOSP}_{256}(ak) \parallel \text{I2LEOSP}_{256}(nk))$$

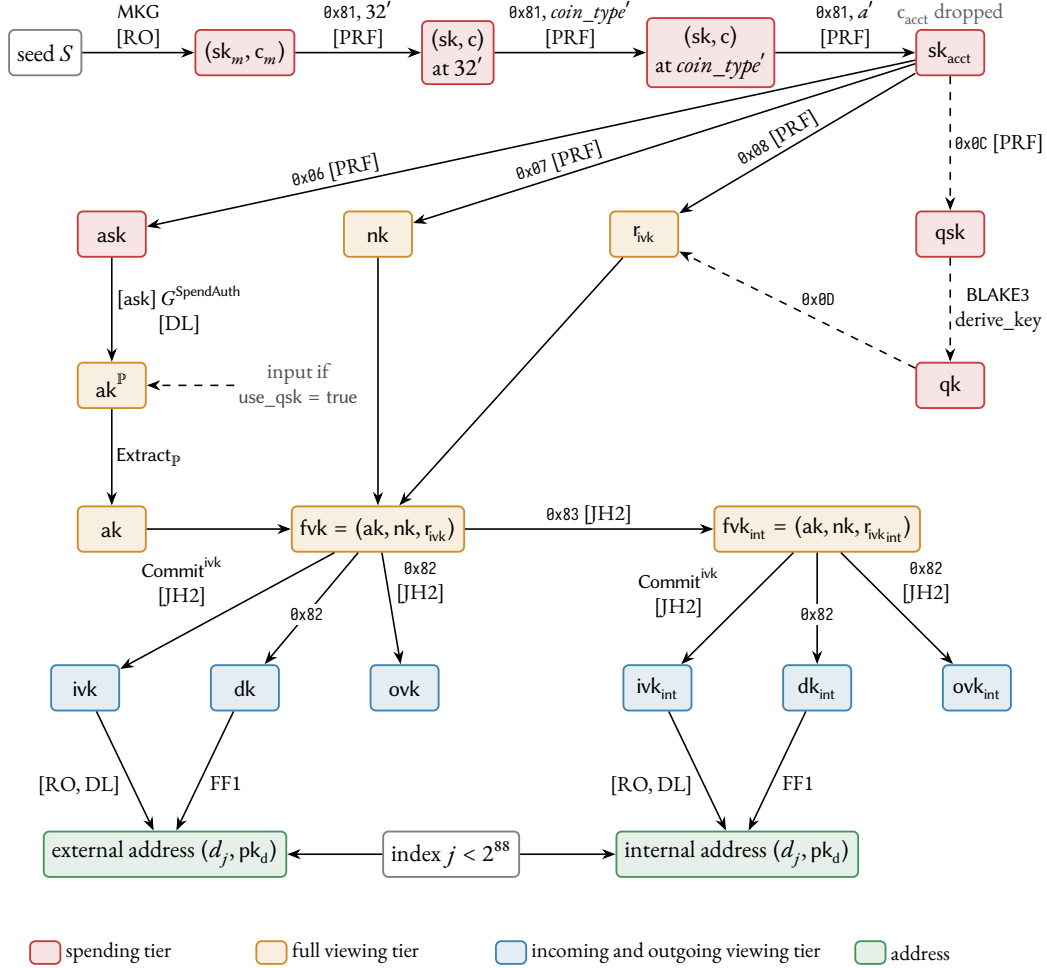


Figure 1. Derivation of the keys of one account, from the seed S through the master key and the hardened levels $32'$, $coin_type'$ and a' to the account spending key sk_{acct} , whose chain code is dropped; then the key components, the full viewing key fvk , the internal full viewing key fvk_{int} , the incoming and outgoing viewing keys of each scope, and the addresses at each index j . Solid edges are those of a key with $use_qsk = \text{false}$; dashed edges are the $use_qsk = \text{true}$ sources of ak^P , an input, and of r_{ivk} , through qsk and qk . Each derivation edge carries its expansion byte or operation; unlabelled edges are projections into a tuple or inputs. Each one-way edge also carries the assumption under which it is one-way: PRF, the *Ironwood Guide*’s Assumption “Pseudorandomness of PRF expansion” (§“Pseudorandom functions and field reductions”); RO, a random oracle (BLAKE2b under ZcashIP320rchain for MKG, GroupHash for the diversified base); DL, its Assumption “Discrete logarithms on Pallas”; JH2, the joint hiding assumption of both scopes stated in §2.5. Colours give the lowest key tier that computes a node. The separations the figure depicts are proved in §2.5.

(§“Viewing keys”), and the diversified addresses (§“Diversified addresses”). These constructions are the `use_qsk = false` branch (*Ironwood Guide*, §“The spending key and the spend-side secrets”, Remark “Scope of the key derivations”; protocol specification, §“Orchard Key Components”). They are cited, not restated; this subsection supplies $\text{sk} := \text{sk}_{\text{acct}}$ and the `use_qsk = true` branch.

Lemma 2.9 (Account keys as uniform spending keys). *Let R be a result of the Ironwood Guide about a key generated with `use_qsk = false`, stated as a bound on the probability of a win event W in a game G whose challenger receives the output sk of key generation (Ironwood Guide, §“The spending key and the spend-side secrets”, Definition “Spending key and key generation”), runs in time polynomial given sk , and decides W . Let G_{acct} be G with $\text{sk} := \text{sk}_{\text{acct}}$ at a fixed account path, the adversary $\mathcal{A}$ also receiving $\text{SeedFP}(S)$ and the spending keys at a fixed finite set of other account paths, and let V be the event that sk_{acct} passes the discard conditions of that Definition (made exact in §2.6). Let $\mathcal{B}$ be the adversary of G that samples those extra inputs as independent uniform 32-byte strings, answers the random-oracle queries of $\mathcal{A}$ by lazy sampling, and runs $\mathcal{A}$. Then*

$$|\Pr[W \text{ in } G_{\text{acct}} \mid V] - \Pr[\mathcal{B} \text{ wins } G]| \leq \frac{2\delta}{1 - \rho - \delta},$$

where δ is the advantage bound of Lemma 2.5 for the distinguisher of the proof and ρ is the probability that a single draw from a uniform sk is discarded, negligible by the Ironwood Guide’s Lemma “Rejection in key generation” (§“Viewing keys”); the bound assumes $\rho + \delta < 1$. Every key-side result of the Ironwood Guide therefore holds for sk_{acct} up to this loss.

Proof. The key sk_{acct} is only computationally close to uniform, so the statistical-distance clause of the Ironwood Guide’s Lemma “Independence of domain-separated expansions” (§“Pseudorandom functions and field reductions”) does not apply; the proof is a reduction. Both V and W are decided efficiently by the challenger. A distinguisher for Lemma 2.5 receives either the real values (sk_{acct} , the other spending keys and $\text{SeedFP}(S)$) or independent uniform strings, runs the challenger and $\mathcal{A}$ on them, and outputs 1 exactly on $W \wedge V$; a second distinguisher outputs 1 exactly on V . Write $a := \Pr[W \wedge V]$ and $v := \Pr[V]$ for the real values, and a_u, v_u for the uniform ones. Then $|a - a_u| \leq \delta$ and $|v - v_u| \leq \delta$, with $v_u = 1 - \rho$. With uniform inputs, the extra inputs are independent of the key and distributed as $\mathcal{B}$ samples them. Key generation outputs a single uniform draw conditioned on acceptance, so $\Pr[\mathcal{B} \text{ wins } G] = a_u/v_u$. Hence

$$\left| \frac{a}{v} - \frac{a_u}{v_u} \right| \leq \frac{|a - a_u|}{v} + \frac{a_u}{v_u} \cdot \frac{|v_u - v|}{v} \leq \frac{2\delta}{v} \leq \frac{2\delta}{1 - \rho - \delta},$$

using $a_u \leq v_u$ and $v \geq v_u - \delta$. □

Construction 2.10 (Key components under `use_qsk`). The flag `use_qsk` $\in \{\text{false}, \text{true}\}$ is chosen at key generation. For a spending key sk , in both branches

$$\text{nk} := \text{ToBase}(\text{PRFexpand}_{\text{sk}}([\text{0x07}])).$$

1. If `use_qsk = false`: $\text{ask}, \text{ak}^{\text{P}}, \text{ak}$ and r_{ivk} are those of the *Ironwood Guide*, and $\text{qsk} = \text{qk} = \perp$.

2. If `use_qlsk = true`: a spend validating key $ak^P \in \mathcal{E}(\mathbb{F}_{P_{\text{Pallas}}}) \setminus \{\mathcal{O}\}$ with even y -coordinate, that is, with bit 255 of its star encoding equal to 0, is an input, obtained by a method outside this construction, for example a directly generated ask with $ak^P = [\text{ask}] G^{\text{SpendAuth}}$, or FROST distributed key generation. Then

$$\begin{aligned} ak &:= \text{Extract}_P(ak^P), \\ qsk &:= \text{truncate}_{32}(\text{PRFexpand}_{sk}([0x0C])), \\ qk &:= \text{BLAKE3.derive_key}(\text{Zcash ZIP 2005 qk-derivation v1}, qsk, 32), \\ r_{ivk} &:= \text{ToScalar}(\text{PRFexpand}_{qk}([0x0D] \parallel \text{I2LEOSP}_{256}(ak) \parallel \text{I2LEOSP}_{256}(nk))), \end{aligned}$$

where truncate_{32} keeps the first 32 bytes and

$$\text{BLAKE3.derive_key}(\text{context}, \text{material}, \text{length})$$

is the function that ZIP 2005 specifies, cited to it and not constructed; no property of it is assumed.

The components ivk , dk , ovk and every address are computed from (ak, nk, r_{ivk}) by the *Ironwood Guide*'s constructions in both branches, so they depend on `use_qlsk` (ZIP 2005, "Changes to the Protocol Specification", §4.2.3 "Orchard Key Components", and "Usage with FROST"). No edition of the protocol specification carries this branch.

Wallet rules. A wallet MUST generate every key of one ZIP 32 account with the same value of `use_qlsk` (ZIP 326, "Wallet key-generation restrictions"). The rule of the same section on Orchard-pool funds and `use_qlsk = true` keys is stated once, with routing (§11.2). The value of `use_qlsk` is not part of the recovery information, so a wallet restoring from a seed scans with both values (ZIP 326, "Scanning when restoring from seed"; §6.2).

Class (Definition 1.1): how an account whose keys are to be recoverable from the seed with `use_qlsk = true` derives ak^P from sk_{acct} is fixed by no ZIP; it is designed but unspecified.

Remark 2.11 (Scope of the key results). Lemma 2.9 and every later key-side result of this volume (separations, hiding, validity bounds) is claimed for `use_qlsk = false` only. For `use_qlsk = true` the trapdoor r_{ivk} is an expansion keyed by qk , and a result would need BLAKE3 in its key-derivation mode modelled as a PRF, which is not assumed, and a hypothesis on the source of ak^P . Constructive statements, which state what is computed from what, hold in both branches.

Proposition 2.12 (Key recoverability). *For `use_qlsk = false`, every spending, full viewing, incoming viewing and outgoing viewing key and every address of an account is a deterministic function of the seed, the coin type, the account index, the scope and the diversifier index; the seed with the account index recovers all of them. For `use_qlsk = true` the same holds of nk , qsk and qk , and, given ak , of r_{ivk} , ivk , dk , ovk and every address; recovery needs ak in addition, and, to spend, the key material that signs under ak , held collectively by the signers under FROST. In both cases the wallet must know the account's value of `use_qlsk` or try both. The function is partial: it is undefined at an account index whose key is invalid (§2.6).*

Proof. Composition of Constructions 2.3 and 2.4 and Definition 2.7, which map the seed to sk_{acct} ; of Construction 2.10, which maps sk_{acct} and use_qsk , together with ak^P when $use_qsk = \text{true}$, to (ak, nk, r_{ivk}) ; and of the *Ironwood Guide*’s constructions from (ak, nk, r_{ivk}) (§“Viewing keys” and §“Diversified addresses”). The keys of the internal scope are a function of fvk (§2.5), and the composition only adds that function. ZIP 2005, “Changes to the Protocol Specification”, §4.2.3 “Orchard Key Components”, notes, and “Usage with FROST” state the same requirements for recovery. $\square$

Construction 2.13 (Full viewing key fingerprint and tag). For a full viewing key (ak, nk, r_{ivk}) of either branch,

$$FVFP(ak, nk, r_{ivk}) := \text{BLAKE2b-256}(\text{Zcash0rchainFVFP}, \\ \text{I2LEOSP}_{256}(ak) \parallel \text{I2LEOSP}_{256}(nk) \parallel \text{I2LEOSP}_{256}(r_{ivk})),$$

over the 96-byte string shown, which is the raw encoding of an Orchard full viewing key (protocol specification, §“Orchard Raw Full Viewing Keys”). It MAY be used to identify the key uniquely; its uses rely on collision resistance, cited as for Construction 2.2; Lemma 2.24 models it as a random oracle. The *tag* is $FVFP[0..4)$, its first 4 bytes; it serves key lookup and MUST NOT be assumed unique, and nothing is claimed of it (ZIP 32, “Orchard Full Viewing Key Fingerprints and Tags”). In ZIP 32’s encoding of an extended spending key, a key names its parent by this tag, the master key’s parent tag being four zero bytes (ZIP 32, “Orchard extended spending keys”).

2.5 Internal keys: change and auto-shielding

With no change level, the separation between externally visible receipt and wallet-internal operations runs below the account. The internal operations are change and *auto-shielding*, the automatic transfer of received transparent funds to the most recent shielded pool (ZIP 315, “Terminology”; its rules in §11.3). The derivation applies to keys at the account level (ZIP 32, “Orchard internal key derivation”).

Construction 2.14 (Internal full viewing key). For a full viewing key $fvk = (ak, nk, r_{ivk})$ let $K := \text{LE}_{256}(r_{ivk})$, written $\text{I2LEBSP}_{256}(r_{ivk})$ in ZIP 32, and

$$r_{ivk_{int}} := \text{ToScalar}(\text{PRFexpand}_K([\text{0x83}] \parallel \text{I2LEOSP}_{256}(ak) \parallel \text{I2LEOSP}_{256}(nk))) \in \mathbb{F}_{p_{Vesta}}.$$

The *internal full viewing key* is $fvk_{int} := (ak, nk, r_{ivk_{int}})$. The internal keys are $ivk_{int} := \text{Commit}_{fvk_{int}}^{ivk}(ak, nk)$ and (dk_{int}, ovk_{int}) , the two halves of the 0x82 expansion keyed by $\text{LE}_{256}(r_{ivk_{int}})$ over the same message. The input of the 0x83 expansion has the key and message body of the 0x82 expansion and another lead byte. The internal expanded spending key is $(ask, nk, r_{ivk_{int}})$; no 32-byte internal spending key exists (ZIP 32, “Orchard internal key derivation”; protocol specification, §“Orchard Key Components”).

Hence $ak_{int} = ak$ and $nk_{int} = nk$, as the protocol specification, §“Orchard Key Components”, notes: spend authorisation and nullifier derivation serve both scopes, and ivk , dk and ovk are de-

rived again from $r_{ivk_{int}}$. The construction does not force the keys of the two scopes to differ; their equality is an event of negligible probability (Proposition 2.17), not an excluded case. For Sapling, ZIP 32 keeps the spend authorising key and derives again the key that enters the proof and the nullifier, the outgoing viewing key and the diversifier key, so the two scopes share spend authority and differ in the key that enters the proof and the nullifier (ZIP 32, “Sapling internal key derivation”); this is cited, not constructed. Payments are sent from an account and a scope, not from an address: no outgoing viewing key per diversified address exists (ZIP 32, “Orchard diversifier and OVK derivation”; ZIP 316, “Usage of Outgoing Viewing Keys”).

Proposition 2.15 (Capability of a full viewing key). *From fvk alone one computes, for both scopes, ivk , dk , ovk and the address at every diversifier index $j \in \{0, \dots, 2^{88} - 1\}$. For a key with $use_qsk = \text{false}$, no efficient algorithm given fvk outputs ask , and no efficient algorithm given the keys of one account outputs the spending key of another account, except with negligible probability. Recovering the index and scope of a given address is stated with the index (§3.1).*

Proof. Computation: fvk_{int} is a function of fvk (Construction 2.14), and each address is a function of (dk, ivk) and j (*Ironwood Guide*, §“Diversified addresses”, Construction “Diversified address”). No ask : the *Ironwood Guide*, §“Key capabilities and scope”, Proposition “Capability separation”, part (a), under its Assumptions “Pseudorandomness of PRF expansion” and “Discrete logarithms on Pallas”, transferred to sk_{acct} by Lemma 2.9. No other account: the spending key of another account lies at a distinct path, and only hardened derivation exists; by Lemma 2.5 the two keys are jointly indistinguishable from independent uniform strings, so an algorithm given every key of this account outputs the other key with probability at most 2^{-256} plus the advantage bound of that lemma. $\square$

Assumption 2.16 (Joint hiding of the viewing keys of both scopes). For keys with $use_qsk = \text{false}$: for every $(ak, nk) \in \{1, \dots, p_{Pallas} - 1\} \times \mathbb{F}_{p_{Pallas}}$, given to the distinguisher, and r_{ivk} uniform on $\mathbb{F}_{p_{Vesta}}$, the six keys $(ivk, dk, ovk, ivk_{int}, dk_{int}, ovk_{int})$, computed by the *Ironwood Guide*’s constructions and Construction 2.14, are computationally indistinguishable from

$$(\text{Commit}_{r'}^{ivk}(ak, nk), u_1, u_2, \text{Commit}_{r'}^{ivk}(ak, nk), u_3, u_4),$$

where r and r' are uniform on $\mathbb{F}_{p_{Vesta}}$ and $u_1, \dots, u_4$ are uniform 32-byte strings, all independent. It extends the *Ironwood Guide*’s Assumption “Joint hiding of the viewing keys” (§“Viewing keys”), which covers the external triple only, to the 0x83 row, whose key $\text{LE}_{256}(r_{ivk})$ is also the Commit^{ivk} trapdoor of the external ivk and so lies outside the hypothesis of the *Ironwood Guide*’s Lemma “Independence of domain-separated expansions” (§“Pseudorandom functions and field reductions”) (Remark “Hiding of ivk and the double use of r_{ivk} ” (§“Viewing keys”)); the PRF assumption does not give it (protocol specification, §“Orchard Key Components”, note on r_{ivk} as trapdoor and key). It is applied after Lemma 2.9, averaging over the key-generation distribution of (ak, nk) .

Proposition 2.17 (Separation of scopes under trial decryption). *Let an account key have $use_qsk = \text{false}$, with external and internal incoming viewing keys outside $\{0, \perp\}$. An output*

honestly addressed to an internal-scope address of the account is accepted under the account's external ivk only with negligible probability, under the Ironwood Guide's Assumptions "Discrete logarithms on Pallas" (§"Sinsemilla: hash, commitment, and short forms") and "GroupHash as a random oracle" (§"GroupHash, domain separation, and nothing-up-my-sleeve generators"), Assumption 2.16 and the hypotheses of Lemma 2.9. Consequently an external incoming viewing key reveals no change or auto-shielding output. Since fvk_{int} is computable from fvk , a full viewing key reveals both scopes, and the boundary between the scopes lies at the incoming viewing tier.

Proof. Acceptance under a key ivk derives the transmission key again as $[\text{ivk}] g_d$ from the decrypted diversifier and checks the note commitment of the derived note against cmx (Ironwood Guide, §"Trial decryption and note acceptance", Construction "Acceptance of an Ironwood-pool output", check (iv); for an Orchard-pool output the check of the protocol specification, §"Decryption using an Incoming Viewing Key (Sapling and Orchard)"). The honest output commits to a note n with transmission key $[\text{ivk}_{\text{int}}] g_d$. If the note n' derived under ivk_{ext} differs from n , acceptance exhibits two distinct notes with one extracted commitment, which happens with negligible probability (Ironwood Guide, §"Note commitments", Proposition "Hiding and binding of note commitments", part (b)). If $n' = n$, then $[\text{ivk}_{\text{ext}}] g_d = [\text{ivk}_{\text{int}}] g_d$; since $g_d \neq \mathcal{O}$ in a group of prime order and both scalars lie in $\{1, \dots, p_{\text{Pallas}} - 1\}$ with $p_{\text{Pallas}} < p_{\text{Vesta}}$, this forces $\text{ivk}_{\text{ext}} = \text{ivk}_{\text{int}}$ (Ironwood Guide, §"Viewing keys"). By Lemma 2.9 and Assumption 2.16 the pair is replaced by $(\text{Commit}_{r'}^{\text{ivk}}(\text{ak}, \text{nk}), \text{Commit}_{r'}^{\text{ivk}}(\text{ak}, \text{nk}))$ with independent uniform r and r' , at the cost of the distinguishing advantage, since equality is efficiently decidable. By the Ironwood Guide's Lemma "Distribution of ivk " (§"Viewing keys"), the second value equals any fixed value with probability at most $2/p_{\text{Vesta}}$, independently of the first, so the two are equal with probability at most $2/p_{\text{Vesta}}$. $\square$

Construction 2.18 (Transparent outgoing viewing keys). A P2PKH (pay-to-public-key-hash) output pays to the hash of one secp256k1 public key (Crypto Guide, §"The transparent layer's schemes: ECDSA and BIP-340 over secp256k1"). P2PKH keys have no outgoing viewing key, yet auto-shielding outputs need one. For the BIP 32 account-level extended public key (c, pk) of a transparent account, at path $m/44'/\text{coin_type}'/\text{account}'$ (BIP 44), let

$$I_{\text{ovk}} := \text{PRFexpand}_c([0x00] \parallel \text{ser}_p(\text{pk})),$$

with the chain code c used as a 32-byte key and $\text{ser}_p(\text{pk})$ the 33-byte compressed encoding of BIP 32; then $\text{ovk}_{\text{external}} := I_{\text{ovk}}[0..32]$ and $\text{ovk}_{\text{internal}} := I_{\text{ovk}}[32..64]$ (ZIP 316, "Deriving Internal Keys"). The external transparent incoming viewing key is the extended public key of the non-hardened child 0 of (c, pk) and the internal one that of the non-hardened child 1 (ZIP 316, "Deriving a UIVK from a UFVK"; BIP 44), each obtained by BIP 32 public-to-public derivation, which is HMAC-SHA512 keyed by c (BIP 32, "Public parent key $\rightarrow$ public child key"; Crypto Guide, §"PRFs from hash functions: length extension and keyed BLAKE2").

⋮ *Assumption 2.19* (Hiding of the transparent account key). For an honestly generated account-
⋮ level extended public key (c, pk) , the extended public keys of its non-hardened children 0 and

1 and the two halves of I_{ovk} are jointly computationally indistinguishable from two independently generated extended public keys, each a uniform 32-byte chain code with the public key of a uniform secret key, and two independent uniform 32-byte strings. The chain code c keys both HMAC-SHA512 in BIP 32 non-hardened derivation and PRFexpand in the $0x00$ row, so this is not an instance of the *Ironwood Guide*’s Lemma “Independence of domain-separated expansions” (§“Pseudorandom functions and field reductions”); it is assumed.

Proposition 2.20 (Capability separation of internal keys). *The following hold, where “derives” means outputs the key with non-negligible probability.*

- (i) *A holder of a full viewing key derives the external and internal incoming viewing keys and the external and internal outgoing viewing keys.*
- (ii) *A holder of the external incoming viewing key derives neither the internal incoming viewing key nor any outgoing viewing key.*
- (iii) *A holder of the external outgoing viewing key derives neither the internal outgoing viewing key nor any incoming viewing key.*

For Orchard the statements hold for keys with $\text{use_qsk} = \text{false}$ under Lemma 2.9 and Assumption 2.16; for Sapling they are the properties that ZIP 316, “Deriving Internal Keys”, states for the construction of ZIP 32, “Sapling internal key derivation”, and are cited, not proved; for transparent P2PKH keys under Assumption 2.19 (ZIP 316, “Deriving Internal Keys”).

Proof. (i) Constructive: Construction 2.14 and the *Ironwood Guide*’s constructions; for transparent keys, Construction 2.18 from (c, pk) .

(ii) and (iii), Orchard: by Lemma 2.9 and Assumption 2.16 the six keys are replaced by the ideal distribution, at the cost of the distinguishing advantage, since success is decided by comparing the output with the key. In the ideal distribution every key not computed from the held one is independent of it: either a uniform 32-byte string, output with probability 2^{-256} , or a $\text{Commit}^{\text{ivk}}$ value under an independent trapdoor, output with probability at most $2/p_{\text{Vesta}}$ (*Ironwood Guide*, §“Viewing keys”, Lemma “Distribution of ivk ”). Transparent: the same argument in the ideal distribution of Assumption 2.19, where an independently generated extended public key has a uniform chain code, output with probability 2^{-256} . Sapling: cited to ZIP 316, “Deriving Internal Keys”, which asserts them from the one-wayness of PRFexpand and CRH^{ivk} ; not proved here. $\square$

2.6 Valid account keys

Definition 2.21 (Valid account key). For $\text{use_qsk} = \text{false}$, an account spending key sk_{acct} is *valid* if and only if $\text{ask} \neq 0$, $\text{ivk} \notin \{0, \perp\}$ and $\text{ivk}_{\text{int}} \notin \{0, \perp\}$. The first two conditions are discard conditions of the *Ironwood Guide*, §“The spending key and the spend-side secrets”, Definition “Spending key and key generation”; the third is the specification’s condition on the internal key of Construction 2.14, which that Definition names without constructing (protocol specification, §“Orchard Key Components”; ZIP 32, “Orchard internal key derivation”). ZIP 2005

amends the derivation for `use_ysk = true` (“Changes to the Protocol Specification”, §4.2.3), for which this volume states no validity condition. Only keys a wallet uses need be valid; intermediate path nodes are never expanded into components.

Remark 2.22 (Derivation failure). ZIP 32 notes that a child spending key yields an invalid external or internal full viewing key “with small probability” and prescribes no wallet behaviour (“Orchard child key derivation”). The specification’s instruction to discard the key and repeat with a new `sk` applies to a freshly random key, not to a fixed path, where nothing is drawn again: an account index whose key is invalid has no key. Class (Definition 1.1): the wallet’s handling of such an index is an open problem; Proposition 2.23 bounds its probability.

Proposition 2.23 (Probability of an invalid account key). *Model the Sinsemilla generators, the points $S(j)$ for $0 \leq j < 2^{10}$ and the initial point $Q := Q(z.cash:Orchard-CommitIvk-M)$, as independent uniform points of the Pallas group, independent of every other quantity (Ironwood Guide, §“GroupHash, domain separation, and nothing-up-my-sleeve generators”, Assumption “GroupHash as a random oracle”). Let*

$$\varepsilon := \frac{3[2^{512}/p_{\text{Vesta}}]}{2^{512}} + \frac{408}{p_{\text{Vesta}}} < \frac{412}{p_{\text{Vesta}}} = 2^{-245.31}.$$

For $\text{use_ysk} = \text{false}$ and a fixed account path, $\Pr[\text{sk}_{\text{acct}} \text{ is invalid}] \leq \varepsilon + \delta$, where δ is the sum of the advantages of efficient reductions, each deciding validity, against Lemma 2.5, the Ironwood Guide’s Assumption “Pseudorandomness of PRF expansion” (§“Pseudorandom functions and field reductions”) (rows 0x06, 0x07 and 0x08) and Assumption 2.16 (row 0x83), plus the statistical distance of ToScalar from uniform, below 2^{-257} (Ironwood Guide, §“Pseudorandom functions and field reductions”). For the n accounts a wallet uses, $\Pr[\text{some key is invalid}] \leq n\varepsilon + \delta_n$, where δ_n is the corresponding sum for the n paths jointly; the union runs over the keys used, not over path nodes. The bound averages over generators modelled as random and is not a frequency for the fixed deployed generators.

Proof. Replace `skacct` by a uniform string (Lemma 2.5); the 512-bit outputs of rows 0x06, 0x07 and 0x08 by independent uniform strings (the Ironwood Guide’s Lemma “Independence of domain-separated expansions” (§“Pseudorandom functions and field reductions”), first conclusion); `rivk` by a uniform element of $\mathbb{F}_{p_{\text{Vesta}}}$ (the reduction distance); and, on the event `ask` $\neq 0$, the pair `(ivk, ivkint)` by `(Commitrivk(ak, nk), Commitr'ivk(ak, nk))` with independent uniform r and r' (Assumption 2.16). Each replacement changes the probability of invalidity by at most one term of δ , since validity is efficiently decidable. Then, by the union bound (Math Guide, §“The union bound and a birthday calculation”, Theorem “Union bound / Boole’s inequality”), the key is invalid only on one of the following events.

- (a) `ask = 0`. The sign normalisation preserves 0, so this happens exactly when a uniform 512-bit integer is divisible by p_{Vesta} , with probability $[2^{512}/p_{\text{Vesta}}]/2^{512}$ (Math Guide, §“Uniform sampling and the bias of modular reduction”).
- (b) The Sinsemilla hash point $\hat{M}$ of the message $\text{LE}_{255}(\text{ak}) \parallel \text{LE}_{255}(\text{nk})$ is $\perp$. The message has 510 bits, 51 chunks of 10 bits, so the hash performs 102 incomplete additions, two per

round, $\text{Acc}_i := (\text{Acc}_{i-1} + S(m_i)) + \text{Acc}_{i-1}$ (protocol specification, §“Sinsemilla Hash Function”; *Ironwood Guide*, §“Sinsemilla: hash, commitment, and short forms”, Construction “Sinsemilla hash on Pallas”). An incomplete addition $A + B$ of points is exceptional exactly when $A = \mathcal{O}$, $B = \mathcal{O}$, $A = B$ or $A = -B$. At the first exceptional addition every accumulator so far equals its formal value, an integer combination of Q and the $S(j)$, so one of four linear relations among the generators holds. Each relation is nontrivial: it either states that the single generator $S(m_i)$ is $\mathcal{O}$, or gives Q a coefficient 2^t or 2^{t+1} , non-zero modulo the odd prime p_{Vesta} . The message depends only on (ask, nk) and $G^{\text{SpendAuth}}$, which are independent of Q and the $S(j)$, so each relation holds with probability $1/p_{\text{Vesta}}$, and the 102 additions cost at most $408/p_{\text{Vesta}}$. Both scopes commit to the same message, so this event is counted once.

- (c) $\text{ivk} = 0$ with $\hat{M} \neq \perp$. The addition of $[r] H_D$ is complete, and $\text{ivk} = 0$ exactly when $\hat{M} + [r] H_D = \mathcal{O}$ (*Ironwood Guide*, §“Fields, groups, and encodings”, Lemma “Extract vanishes only at the identity”), which, since $H_D \neq \mathcal{O}$, holds for a single value of r : probability $1/p_{\text{Vesta}} \leq \lceil 2^{512}/p_{\text{Vesta}} \rceil / 2^{512}$.
- (d) $\text{ivk}_{\text{int}} = 0$ with $\hat{M} \neq \perp$: likewise, under the independent r' , with the same bound.

The sum is ε . Negligibility alone is the *Ironwood Guide*’s Lemma “Rejection in key generation” (§“Viewing keys”). For n accounts, apply the replacements to the n paths jointly and the union bound over the n keys; path nodes that are not account keys are never expanded into components and do not enter the union. $\square$

2.7 The PRFexpand lead-byte registry

Domain separation of PRFexpand is per key: under one key, distinct lead bytes give jointly pseudorandom outputs (*Ironwood Guide*, §“Pseudorandom functions and field reductions”, Lemma “Independence of domain-separated expansions”), while expansions under different keys may reuse a byte. Byte conventions: the key that ZIP 32 writes $\text{I2LEBSP}_{256}(\text{r}_{\text{ivk}})$ is $\text{LE}_{256}(\text{r}_{\text{ivk}})$; the bytes of ak are $\text{I2LEOSP}_{256}(\text{ak})$ of the field element $\text{ak} = \text{Extract}_{\text{p}}(\text{ak}^{\text{p}})$, which for the sign-normalised ak^{p} equals the byte form LEBS2OSP_{256} of its star encoding, bit 255 being 0 (*Ironwood Guide*, §“The spending key and the spend-side secrets”); the key nk enters as $\text{I2LEOSP}_{256}(\text{nk})$ (protocol specification, §“Orchard Key Components”).

Table 2 lists the expansions and personalisations that this volume constructs. The expansions of the key components and of notes are the rows of the *Ironwood Guide*’s Construction “Domain bytes of PRF expansion” (§“Pseudorandom functions and field reductions”) and are not restated. The list of expansion bytes that the protocol specification maintains, as amended by ZIP 2005, is §“Pseudo Random Functions” (ZIP 2005, “Changes to the Protocol Specification”, §4.1.2); bytes this volume does not use are not listed.

Lemma 2.24 (Independence of the key-derivation domains). *Under the Ironwood Guide’s Assumption “Pseudorandomness of PRF expansion” (§“Pseudorandom functions and field reductions”), with BLAKE2b under distinct personalisations modelled as independent random oracles, the expansions of rows 0x81 and 0x0C, whose keys (a chain code of the Orchard tree, and sk_{acct}) are*

Table 2. Domains of key derivation. Upper part: each expansion $\text{PRFexpand}_{\text{key}}(b \parallel \text{message})$ that this volume constructs, with its lead byte b , key, message and output; the rows $0x0C$ and $0x0D$ are those of ZIP 2005. Lower part: the 16-byte BLAKE2b personalisations of key derivation, with the function, output and constructing environment of each.

Byte	Key	Message	Output	Constructed in
$0x81$	c_{par}	$sk_{\text{par}} \parallel \text{I2LEOSP}_{32}(i)$	(sk_i, c_i)	Constr. 2.4
$0x83$	$\text{LE}_{256}(r_{\text{ivk}})$	$\text{I2LEOSP}_{256}(ak) \parallel \text{I2LEOSP}_{256}(nk)$	$r_{\text{ivkint}}(\text{ToScalar})$	Constr. 2.14
$0x0C$	sk	ε (empty)	$qsk(\text{truncate}_{32})$	Constr. 2.10
$0x0D$	qk	$\text{I2LEOSP}_{256}(ak) \parallel \text{I2LEOSP}_{256}(nk)$	r_{ivk} if $\text{use_qsk} = \text{true}$ (ToScalar)	Constr. 2.10
$0xD0$	transparent chain code c	$ser_p(pk)$	$ovk_{\text{external}} \parallel ovk_{\text{internal}}$	Constr. 2.18
Personalisation		Function	Output	Constructed in
ZcashIP320rchaind		BLAKE2b-512 of S	(sk_m, c_m)	Constr. 2.3
Zcash_HD_Seed_FP		BLAKE2b-256 of $[\text{length}(S)] \parallel S$	SeedFP(S)	Constr. 2.2
Zcash0rchaindFVFP		BLAKE2b-256 of the 96-byte encoding of fvk	FVFP	Constr. 2.13

pseudorandom by Lemma 2.5 and enter no function other than PRFexpand , and the three personalised BLAKE2b functions of Table 2 are jointly indistinguishable from independent random functions. The lemma excludes the other rows, each outside the hypothesis of the Ironwood Guide’s Lemma “Independence of domain-separated expansions” (§“Pseudorandom functions and field reductions”): the row $0x83$, whose key is also the $\text{Commit}^{\text{ivk}}$ trapdoor, is covered by Assumption 2.16; the row $0xD0$, whose key also keys HMAC-SHA512, by Assumption 2.19; the row $0x0D$, keyed by the BLAKE3 output qk of which nothing is assumed (Remark 2.11), carries no claim.

Proof. For each key, the *Ironwood Guide*’s Lemma “Independence of domain-separated expansions” (§“Pseudorandom functions and field reductions”), applied after the hybrid of Lemma 2.5 has made the keys uniform; this is a reduction, as in Lemma 2.9, not the statistical-distance clause of that lemma. For the three personalisations, the *Crypto Guide*’s Proposition “Domain separation yields independent oracles” (§“Domain separation and personalisation”): they are pairwise distinct 16-byte strings, and the two BLAKE2b-256 uses and the BLAKE2b-512 use differ also in the output length, a parameter of BLAKE2b. The proof of Lemma 2.5 uses the same two citations directly. $\square$

3 Addresses and viewing keys (ZIP 316)

This section passes from the keys of an account to the addresses and viewing keys that a wallet hands out. It defines the diversifier index and the recovery of an index from an address, the raw encodings of receivers and keys, the unified container with its jumbled string encoding and its viewing-key form, the unified address at an index of a viewing key, the extensions of Revision 2 of ZIP 316, and the recipient addresses a sender handles.

3.1 Diversifier indices

The address layer meets two requirements.

- (R1) Each payee receives an address unlinkable to every other address of the same holder, while all of them are spendable by one key and recoverable from one seed (protocol specification, §“DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions”, security requirements; ZIP 32; ZIP 316, “Rationale for address derivation”). Each shielded protocol meets (R1) with diversified addresses.
- (R2) Transparent, Sapling and Orchard receivers coexist, each with its own format. A recipient publishes one string, from which the sender selects the most preferred receiver type it supports, and whose characters a sender can check (ZIP 316, “Motivation” and “Requirements”). ZIP 316 meets (R2) above the protocols, with unified containers (§3.3) and their jumbled string encoding (§3.4).

The *Ironwood Guide* constructs the Orchard key tiers, the diversifier map and the diversified base (§“Viewing keys” and §“Diversified addresses”). This subsection adds the diversifier index, shared by Sapling and Orchard and used by transparent derivation.

The Orchard address at index j of an incoming viewing key (dk, ivk) is that of the *Ironwood Guide*, §“Diversified addresses”, Construction “Diversified address”, cited and not re-derived: $d := \text{FF1-AES256}_{dk}(\text{LE}_{88}(j))$, $g_d := \text{DiversifyHash}^{\text{Orchard}}(d)$, which is $\text{GroupHash}(z.\text{cash}:\text{Orchard-gd}, \text{LEBS2OSP}_{88}(d))$ unless that is the identity and the fallback point $\text{GroupHash}(z.\text{cash}:\text{Orchard-gd}, \epsilon)$ otherwise, and $pk_d := [ivk] g_d$; the address is (d, pk_d) . For each scope $s \in \{\text{ext}, \text{int}\}$ of an account (Construction 2.14) the pair (dk_s, ovk_s) is the first and the second 32 bytes of

$$\text{PRFexpand}_{\text{LE}_{256}(r_{ivk_s})}([0x82] \parallel \text{I2LEOSP}_{256}(ak) \parallel \text{I2LEOSP}_{256}(nk))$$

(*Ironwood Guide*, §“Viewing keys”), and ivk_s is the incoming viewing key of the scope. The address at index 0 is the *default address* of the scope (protocol specification, §“Orchard Key Components”; ZIP 32, “Orchard diversifier and OVK derivation”).

Definition 3.1 (Diversifier index). For a 32-byte diversifier key dk and an index $j \in \{0, \dots, 2^{88} - 1\}$, the *diversifier at index j* is

$$d_j := \text{FF1-AES256}.\text{Encrypt}(dk, "", \text{LE}_{88}(j)) \in \{0, 1\}^{88},$$

the FF1 mode of NIST SP 800-38G over AES-256 with key dk , radix 2, length 88 and the empty tweak (*Crypto Guide*, §“Format-preserving encryption and FF1”, Construction “The FF1 mode of NIST SP 800-38G”; protocol specification, §“Pseudo Random Permutations”). The map is the same in both shielded protocols; only the key differs.

1. For Orchard, $dk = dk_s$ of the scope s , as above.
2. For Sapling, dk is the diversifier-key component of the account’s Sapling extended spending key: $dk_m = \text{truncate}_{32}(\text{PRFexpand}_{sk_m}([0x10]))$ at the master key and

$dk_i = \text{truncate}_{32}(\text{PRFexpand}_{I_L}([\emptyset \times 16] \parallel dk_{\text{par}}))$ for a hardened child, with I_L the left half of the child's derivation output (ZIP 32, “Sapling master key generation” and “Sapling child key derivation”); Sapling key derivation is cited and not constructed.

The diversifier d_j , and the index j , are *valid* for a protocol if and only if that protocol's DiversifyHash does not return $\perp$ at d_j .

3. Orchard: $\text{DiversifyHash}^{\text{Orchard}}$ is total, so every index is valid, a scope has exactly 2^{88} addresses, and the default diversifier is d_0 (ZIP 32, “Orchard diversifier and OVK derivation”).

4. Sapling:

$$\text{DiversifyHash}^{\text{Sapling}}(d) := \text{GroupHash}_{\text{URS}}^{J(r)*}(\text{Zcash_gd}, \text{LEBS2OSP}_{88}(d)),$$

a hash into the points of order r_j of the Jubjub curve that returns $\perp$ on some inputs (protocol specification, §“DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions”, §“Jubjub” and §“Group Hash into Jubjub”, cited and not constructed). Approximately half of all indices are valid (ZIP 32, “Sapling diversifier derivation”); the probability under the specification's random-oracle model is derived in §3.6. The default Sapling diversifier is d_j for the least valid j (same section).

Totality of $\text{DiversifyHash}^{\text{Orchard}}$ removes the search for a valid index. The Sapling failure rate matters when one index must serve several protocols at once (§3.6).

Index-choice rules, specified (protocol specification, §“Orchard Key Components” and §“Sapling Key Components”, notes). Orchard diversifier indices SHOULD NOT be chosen at random; ZIP 32 specifies their use in hierarchical deterministic wallets. Address generators MAY encode information in the diversifier index, recoverable by a holder of dk (Lemma 3.3). For Sapling, generators MAY encode information in the diversifier itself, and it is RECOMMENDED to encode it in the ZIP 32 diversifier index instead, so that only a holder of dk reads it.

Proposition 3.2 (Distinct diversifiers). *(a) For every diversifier key dk the map $j \mapsto d_j$ is a bijection from $\{0, \dots, 2^{88} - 1\}$ onto $\{0, 1\}^{88}$. In particular distinct indices give distinct diversifiers, and a wallet issues the indices $0, 1, 2, \dots$ over the whole index space without a repeated diversifier.*

(b) If instead k diversifiers are drawn independently and uniformly from $\{0, 1\}^{88}$, the probability P that two coincide satisfies

$$1 - e^{-k(k-1)/2^{89}} \leq P \leq \frac{k(k-1)}{2^{89}},$$

so a collision becomes likely once k is of order $2^{44} = \sqrt{2^{88}}$; at $k = 2^{44}$ the bounds are 0.393 and 0.500.

Proof. (a) The map LE_{88} is a bijection from $\{0, \dots, 2^{88} - 1\}$ onto $\{0, 1\}^{88}$, and FF1-AES256 under a fixed key and tweak is a bijection of $\{0, 1\}^{88}$ (*Crypto Guide*, §“Format-preserving encryption and FF1”, Proposition “FF1 is a keyed bijection on $\mathcal{X}$ ”); the composition is a bijection. (b) The *Math Guide*, §“The union bound and a birthday calculation”, Proposition “Birthday bound”, with $N = 2^{88}$. $\square$

The specification notes that an 88-bit diversifier is not unguessable at the 128-bit security level and that randomly chosen diversifiers collide near 2^{44} choices; the keyed permutation removes the collisions, provided indices do not repeat (protocol specification, §“DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions”, notes). One further reason for a keyed permutation is ZIP 32’s: it makes the diversifier sequence of each account pseudorandom and uncorrelated with that of any other account, so issued diversifiers do not reveal how many addresses have been generated (ZIP 32, “Sapling diversifier derivation” and “Orchard diversifier and OVK derivation”). The *Ironwood Guide* supplies Assumption “FF1-AES256 as a pseudorandom permutation” (§“Diversified addresses”) and the unlinkability proposition only.

In Orchard, dk_s is derived from the full viewing key of scope s , so a holder of the account’s Orchard full viewing key computes dk_s for both scopes, hence the index of any diversifier of the account (Lemma 3.3; ZIP 32, “Orchard diversifier and OVK derivation”). In Sapling the same capability belongs to the extended full viewing key, of which dk is a component. A holder of the incoming viewing key (dk, ivk) of one scope has this capability for that scope only.

Lemma 3.3 (Index recovery and ownership). *Let (d, pk_d) be an Orchard address and (dk, ivk) the incoming viewing key of some scope.*

- (i) *The value $j := \text{LE}_{88}^{-1}(\text{FF1-AES256.Decrypt}(\text{dk}, "", d))$ is defined for every d and every dk , and $d_j = d$. Decryption alone therefore proves nothing about ownership.*
- (ii) *The address (d, pk_d) is the address of (dk, ivk) at some index if and only if $\text{pk}_d = [\text{ivk}] \text{DiversifyHash}^{\text{Orchard}}(d)$; the index is then unique and equals j .*
- (iii) *A holder of an account’s full viewing key decides whether (d, pk_d) belongs to the account, and at which scope and index, by applying (ii) with $(\text{dk}_{\text{ext}}, \text{ivk}_{\text{ext}})$ and then with $(\text{dk}_{\text{int}}, \text{ivk}_{\text{int}})$. Both tests succeed only if $\text{ivk}_{\text{ext}} = \text{ivk}_{\text{int}}$, which has negligible probability for keys with $\text{use_qsk} = \text{false}$ under Assumption 2.16 and the hypotheses of Lemma 2.9.*

The Sapling case of (i) and (ii) is the same with $\text{DiversifyHash}^{\text{Sapling}}$, the test of (ii) failing when it returns $\perp$. For (iii), both tests succeed only if the two Sapling incoming viewing keys are equal; this volume does not bound that probability for Sapling.

Proof. (i) FF1 decryption is the two-sided inverse of encryption under every key and tweak (*Crypto Guide*, §“Format-preserving encryption and FF1”, Proposition “FF1 is a keyed bijection on $\mathcal{X}$ ”), and LE_{88} is a bijection. (ii) If (d, pk_d) is the address at index j' , then $d = d_{j'}$, so $j' = j$ by (i) and Proposition 3.2(a), and $\text{pk}_d = [\text{ivk}] \text{DiversifyHash}^{\text{Orchard}}(d)$ by the *Ironwood Guide*’s Construction “Diversified address” (§“Diversified addresses”). Conversely, if $\text{pk}_d = [\text{ivk}] \text{DiversifyHash}^{\text{Orchard}}(d)$, the address at index j is $(d_j, [\text{ivk}] \text{DiversifyHash}^{\text{Orchard}}(d_j)) = (d, \text{pk}_d)$. (iii) If both tests succeed,

then $[\text{ivk}_{\text{ext}}] g_d = [\text{ivk}_{\text{int}}] g_d$ with $g_d \neq \mathcal{O}$ in a group of prime order p_{Vesta} , and both scalars lie in $\{1, \dots, p_{\text{Pallas}} - 1\}$ with $p_{\text{Pallas}} < p_{\text{Vesta}}$, so $\text{ivk}_{\text{ext}} = \text{ivk}_{\text{int}}$. Its probability is bounded as in the proof of Proposition 2.17: after the replacement of Assumption 2.16, at most $2/p_{\text{Vesta}}$ by the *Ironwood Guide*'s Lemma "Distribution of ivk" (§"Viewing keys"), plus the distinguishing advantage, equality being efficiently decidable. $\square$

Unlinkability of the addresses of one key is cited with its assumptions. For Orchard it is the *Ironwood Guide*, §"Diversified addresses", Proposition "Unlinkability of diversified addresses": for two keys generated independently with $\text{use_qsk} = \text{false}$, an adversary that holds ak , nk and ovk of both keys and chooses indices j_0, j_1 and $j' \notin \{j_0, j_1\}$ distinguishes which key produced the third address with negligible advantage, conditioned on the three diversifiers being pairwise distinct. Its assumptions are the *Ironwood Guide*'s "Pseudorandomness of PRF expansion", "Joint hiding of the viewing keys", "FF1-AES256 as a pseudorandom permutation", "GroupHash as a random oracle", "DDH on Pallas" and "Discrete logarithms on Pallas". Account keys are admitted as such keys by Lemma 2.9, the win event being decidable by the challenger; the addresses of one key at distinct indices have distinct diversifiers by Proposition 3.2 (a). For Sapling the protocol specification argues the same property under DDH on the prime-order subgroup of Jubjub, with $\text{GroupHash}^{J(r)*}$ modelled as a random oracle, through IK-CPA key privacy of ElGamal (§"DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions", notes; *Crypto Guide*, §"Key privacy"). This volume does not prove the Sapling case.

Remark 3.4 (Chosen-diversifier oracle). Suppose that the holder of an incoming viewing key gives an adversary the address of that key at any diversifier the adversary chooses. The adversary then breaks unlinkability outright: given an address (d', pk_d') , it requests the address at d' and compares the returned transmission key with pk_d' . The two are equal if and only if the key is the same, the base $g_{d'}$ being a point other than the identity in a group of prime order. No other privacy property is affected. Implementations SHOULD avoid providing such a chosen-diversifier oracle (protocol specification, §"DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions", notes). An interface returning the address at a requested *index* gives no chosen diversifier (Lemma 3.5), but it returns the issued addresses themselves: it links every address whose index the requester queries, and under sequential issuance (Proposition 3.2) small indices reach every issued address.

Lemma 3.5 (Index queries give no chosen diversifier). *Let dk be either diversifier key of an Orchard key with $\text{use_qsk} = \text{false}$. Under Assumption 2.16, the hypotheses of Lemma 2.9, and the Ironwood Guide's Assumption "FF1-AES256 as a pseudorandom permutation" (§"Diversified addresses"), an adversary without dk that obtains the diversifiers at q indices of its choice obtains a given d , fixed independently of dk, with probability at most $q/2^{88}$ plus its distinguishing advantages against the two assumptions and, for an account key, the loss of Lemma 2.9. This volume does not prove the Sapling case.*

Proof. Replace dk by a uniform 32-byte string independent of the other keys; by Assumption 2.16, applied after Lemma 2.9, the probability changes by at most that distinguishing advantage and the loss of that Lemma, the win event being decidable. Then replace FF1-AES256 keyed by dk with a uniform permutation of $\{0, 1\}^{88}$; the probability changes by at most the distinguishing advantage.

Repeated indices add nothing, so let the adversary query $k \leq q$ distinct indices. Each image is uniform on the values not yet returned, so the i -th image is the first to equal d with probability

$$\prod_{j=1}^{i-1} \frac{2^{88} - j}{2^{88} - j + 1} \cdot \frac{1}{2^{88} - i + 1} = 2^{-88};$$

summing over $i \leq k$, d is returned with probability $k \cdot 2^{-88} \leq q \cdot 2^{-88}$. $\square$

3.2 Raw encodings

The items of unified containers carry the byte encodings below. Each is stated with the validity condition that a decoder applies.

Definition 3.6 (Raw receiver and key encodings). (a) An *Orchard raw payment address* has 43 bytes, $\text{LEBS2OSP}_{88}(d) \parallel \text{pk}_d^*$, where $\text{pk}_d^* = \text{LEBS2OSP}_{256}(\text{repr}_p(\text{pk}_d))$ is the 32-byte compressed encoding of the Pallas point pk_d (*Ironwood Guide*, §“Fields, groups, and encodings”). A decoder MUST treat the address as invalid if abst_p returns $\perp$ or the identity (protocol specification, §“Orchard Raw Payment Addresses”).

(b) A *Sapling raw payment address* has 43 bytes,

$$\text{LEBS2OSP}_{88}(d) \parallel \text{LEBS2OSP}_{256}(\text{repr}_j(\text{pk}_d)),$$

with the compressed Jubjub encoding. It MUST be considered invalid if abst_j returns $\perp$, if the encoding of pk_d is non-canonical, or if pk_d is not a point of order η_j , the identity included (protocol specification, §“Sapling Payment Addresses”).

(c) An *Orchard raw full viewing key* has 96 bytes,

$$\text{I2LEOSP}_{256}(\text{ak}) \parallel \text{I2LEOSP}_{256}(\text{nk}) \parallel \text{I2LEOSP}_{256}(\text{r}_{\text{ivk}}).$$

It MUST be considered invalid if ak , nk or r_{ivk} is not the canonical encoding of an element of its field ($\mathbb{F}_{p_{\text{Pallas}}}$, $\mathbb{F}_{p_{\text{Pallas}}}$, $\mathbb{F}_{p_{\text{Vesta}}}$), if ak is not the x -coordinate of a point of $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ other than the identity, or if the external or the internal incoming viewing key derived from it is 0 or $\perp$ (protocol specification, §“Orchard Raw Full Viewing Keys”).

(d) An *Orchard raw incoming viewing key* has 64 bytes, $\text{dk} \parallel \text{I2LEOSP}_{256}(\text{ivk})$. It MUST be considered invalid unless $\text{ivk} \in \{1, \dots, p_{\text{Pallas}} - 1\}$ (protocol specification, §“Orchard Raw Incoming Viewing Keys”).

(e) String forms. No Bech32 or Bech32m encoding exists for an individual Orchard payment address, incoming viewing key or full viewing key; the unified encodings of §3.4 are their only strings. A Sapling payment address keeps a standalone Bech32 string, with human-readable part `zs` on Mainnet and `ztestsapling` on Testnet (same sections).

(f) Transparent receivers. A *P2PKH* (pay-to-public-key-hash) address pays to the 20-byte hash160 of a 33-byte compressed ECDSA validating key; a *P2SH* (pay-to-script-hash) address pays to the 20-byte hash160 of a redeem script that the spender reveals and satisfies

(*Crypto Guide*, §“The transparent layer’s hashes”). Their raw encoding prefixes two version bytes to the hash and is written in Base58Check (protocol specification, §“Transparent Addresses”). Inside a unified container a transparent receiver is the bare 20-byte hash, without the two version bytes (ZIP 316, “Requirements for both Unified Addresses and Unified Viewing Keys”).

One Orchard raw address serves both pools of the Orchard protocol (§3.3).

3.3 Unified addresses

ZIP 316 defines Revision 0, the format of this subsection and of §3.4, and Revision 2, which adds metadata items, splits the address prefix into a shielded-only and a transparent-enabled form, and renames the viewing-key prefixes (§3.7); Revision 1 is withdrawn (ZIP 316, “Revisions”). The revision of a container is read from its human-readable part. The status of ZIP 316 is that of Table 1; everything Revision 2 adds is specified in the sense of Definition 1.1.

Definition 3.7 (Unified container). For $n \in \{0, \dots, 2^{64} - 1\}$, $\text{compactSize}(n)$ is the variable-length integer of Bitcoin that ZIP 316 cites:

$$\text{compactSize}(n) := \begin{cases} [n] & \text{if } n < 253, \\ [0xFD] \parallel \text{I2LEOSP}_{16}(n) & \text{if } 253 \leq n < 2^{16}, \\ [0xFE] \parallel \text{I2LEOSP}_{32}(n) & \text{if } 2^{16} \leq n < 2^{32}, \\ [0xFF] \parallel \text{I2LEOSP}_{64}(n) & \text{otherwise,} \end{cases}$$

the range of n fixing the form. An *item* is a pair (τ, v) of a *typecode* $\tau \in \mathbb{N}$ and a byte string v . The *raw encoding* of a container with items $(\tau_1, v_1), \dots, (\tau_k, v_k)$, where $\tau_1 < \dots < \tau_k$, is

$$\text{raw} := \parallel_{i=1}^k \text{compactSize}(\tau_i) \parallel \text{compactSize}(\text{length}(v_i)) \parallel v_i.$$

Both τ_i and $\text{length}(v_i)$ MUST be at most $0x2000000$; since $0x2000000 \in [2^{16}, 2^{32})$, each takes at most 5 bytes and an item header at most 10.

Typecodes $0x00$ and $0x01$ are *transparent*, $0x02$ and $0x03$ *shielded*, and $0xC0$ to $0xFC$ designate *metadata items*, items that are neither receivers nor viewing keys (§3.7). A *unified address* (UA) is a container whose non-metadata items are receivers, in the raw encodings of Definition 3.6 under the typecodes of Table 3. A *unified full viewing key* (UFVK) and a *unified incoming viewing key* (UIVK) are containers whose non-metadata items are full, respectively incoming, viewing keys (§3.5); UVK denotes either. No typecode exists for Sprout.

Validity. Every container holds at least one non-metadata item. A Revision 0 UA, and a Revision 0 UVK, holds at least one shielded item. A Revision 2 shielded-only UA holds no transparent item; a Revision 2 transparent-enabled UA may consist of a single P2PKH receiver, and a Revision 2 UVK need hold no shielded item. A Consumer MUST reject a container

- (i) that violates the containment rule of its revision;
- (ii) in which a typecode repeats, or which holds both $0x00$ and $0x01$;

Table 3. Typecodes and item lengths of Revision 0, in bytes. In Revision 0 Producers MUST NOT include typecode $0x01$ in a UFVK or UIVK, and Consumers MUST treat it as unrecognised there. The Priority List of preference is $0x03$, then $0x02$, then $0x01$ or $0x00$ (at most one of the two is present), the reverse of the encoding order. Receiver lengths are those of Definition 3.6; the viewing-key lengths are those of §3.5: $65 = 32 + 33$ (chain code and compressed key), 128 for (ak, nk, ovk, dk), 64 for (dk, ivk), and 96 for (ak, nk, r_{ivk}) (ZIP 316, “Encoding of Unified Addresses” and “Encoding of Unified Full/Incoming Viewing Keys”).

Typecode	Item	Receiver	UFVK item	UIVK item
$0x00$	transparent P2PKH	20	65	65
$0x01$	transparent P2SH	20	—	—
$0x02$	Sapling	43	128	64
$0x03$	Orchard	43	96	64

- (iii) which holds only metadata items;
- (iv) whose items are not in ascending typecode order;
- (v) which has bytes after the last item;
- (vi) any of whose items fails the validation of its encoding (Definition 3.6; §3.5).

A Consumer MUST ignore items with typecodes it does not recognise, with the exception stated in §3.7; a Consumer therefore parses a container that holds unrecognised items.

Same-account rule. Within one UA or UVK, all receivers, full viewing keys and incoming viewing keys derived hierarchically SHOULD belong to the same account at the ZIP 32 or BIP 44 account level, or, for ZIP 48 and FROST UVKs, to one set of signers able to generate independent addresses for that spending authority. FROST key material MUST be derived according to ZIP 312 and ZIP 2005, in particular ZIP 2005, “Usage with FROST”.

Sources: ZIP 316, “Encoding of Unified Addresses”, “Requirements for both Unified Addresses and Unified Viewing Keys” and “Receivers”.

The ascending order makes the raw encoding a function of the set of items, so a representation that discards the order reproduces the same encoding, provided it retains unrecognised items (ZIP 316, “Rationale for Item ordering”). This order is not the priority order below.

Receiver selection. The Sender of a payment to a UA MUST use the receiver of the most preferred type that it supports, by the Priority List Orchard ($0x03$), Sapling ($0x02$), then P2SH ($0x01$) or P2PKH ($0x00$); an unrecognised item is treated as absent, with the exception of §3.7. A wallet MAY let its users configure which receiver types it can send to, and MUST NOT let them change the order of the Priority List except by opting into an experiment, whose specification MAY add to the list and SHOULD keep the preference for more recent shielded protocols (ZIP 316, “Encoding of Unified Addresses”, “Transfers” and “Experimental Usage”). The Orchard receiver belongs to the Orchard protocol, not to a pool: one ivk trial-decrypts both Orchard-pool and Ironwood-pool note ciphertexts (ZIP 326, “Scanning for Ironwood-pool and Orchard-pool notes”), and the Ironwood pool changes no address structure or encoding (ZIP 229, “Abstract”). The pool in which a payment to an Orchard receiver is made is fixed by the routing rule of §11.2,

which carries the prohibition of ZIP 326, “Wallet key-generation restrictions”, on paying external receivers in the Orchard pool; the Priority List is unchanged.

3.4 String encoding: F4Jumble and Bech32m

The Bech32m checksum does not cascade: a user who checks only the beginning and the end of a long string can miss an adversarial substitution of an inner receiver. ZIP 316 therefore passes the whole payload through F4Jumble, an unkeyed four-round Feistel construction that approximates a random permutation of the entire byte string, so that every output character depends on every input byte. It states two goals (ZIP 316, “Jumbling”).

- (i) *Near second-preimage resistance.* Given q honestly generated UAs or UVKs, an adversary does not produce a valid one that it controls, knowing at least one constituent private key, whose string matches one of the targets on a chosen set of characters, for concreteness the first n and the last m .
- (ii) *Nonmalleability.* Without any private key, an adversary does not produce a valid UA or UVK usefully different from a target, for example with its shielded receiver removed, that matches the target on the checked characters.

Generic attacks bound what is achievable: about $w c^{n+m}/q$ for (i) and w/p for (ii), where w is the work to produce and verify one UA or UVK, c the size of the character set and p the probability that a random decoding is valid and useful (ZIP 316, “Discussion”). Jumbling is not authentication. F4Jumble is unkeyed, so anyone can jumble and checksum a wholly substituted address, and an address is still obtained or verified through a trusted channel; jumbling raises only the cost of partial-match and malleation attacks against a genuine string.

Construction 3.8 (F4Jumble). Let $\ell_H := 64$ and $\ell_{\text{MAX}} := (2^{16} + 1) \ell_H = 4194368$. For $i \in \{0, 1\}$ and byte strings u , with the first argument of BLAKE2b the personalisation,

$$H_i(u) := \text{BLAKE2b}(8\ell_L)(\text{UA_F4Jumble_H} \parallel [i, 0, 0], u),$$

$$G_i(u) := \text{the first } \ell_R \text{ bytes of } \bigparallel_{j=0}^{\lfloor \ell_R/\ell_H \rfloor - 1} \text{BLAKE2b-512}(\text{UA_F4Jumble_G} \parallel [i] \parallel \text{l2LEOSP}_{16}(j), u),$$

an ℓ_L -byte hash and an extendable-output function in the sense of the *Crypto Guide*, §“Hash functions and the random oracle model”, Definition “Extendable-output function”, for the lengths ℓ_L and ℓ_R below. For a byte string M of length ℓ_M with $38 \leq \ell_M \leq \ell_{\text{MAX}}$, let

$$\ell_L := \min(\ell_H, \lfloor \ell_M/2 \rfloor), \quad \ell_R := \ell_M - \ell_L,$$

split $M = a \parallel b$ with $\text{length}(a) = \ell_L$, and let

$$\begin{aligned} x &:= b \oplus G_0(a), & y &:= a \oplus H_0(x), \\ d &:= x \oplus G_1(y), & c &:= y \oplus H_1(d), \end{aligned}$$

and $\text{F4Jumble}(M) := c \parallel d$ (ZIP 316, “Solution”). Figure 2 shows the four rounds.

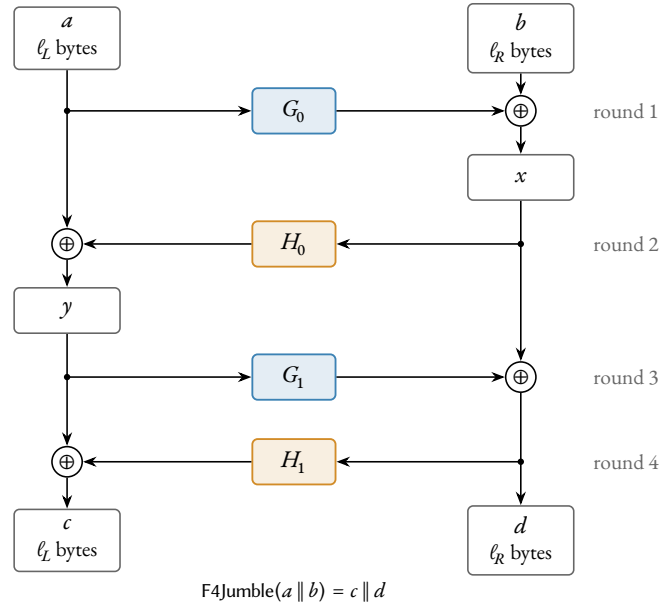


Figure 2. The four rounds of F4Jumble (Construction 3.8) on the unequal halves a (ℓ_L bytes) and b (ℓ_R bytes). Each round replaces one half by its XOR with a function of the other half, which it leaves unchanged: G_0 of a into b gives x , H_0 of x into a gives y , G_1 of y into x gives d , and H_1 of d into y gives c . Blue boxes are the extendable-output functions G_i , with ℓ_R output bytes; amber boxes are the hashes H_i , with ℓ_L output bytes. The names are those of the cells of Example 3.11. The structure is the content of Proposition 3.9.

Each personalisation has 16 bytes: $13 + 3$ for H_i and $13 + 1 + 2$ for G_i . They are pairwise distinct, differing at byte 12 between the H and the G family and at bytes 13 to 15 within a family, and distinct from the three personalisations of key derivation in Table 2 and from that of PRFexpand, Zcash_ExpandSeed. Being of one length and distinct, they are prefix-free tags, so BLAKE2b under these personalisations is modelled as independent random oracles (*Crypto Guide*, §“Domain separation and personalisation”, Proposition “Domain separation yields independent oracles”).

Proposition 3.9 (F4Jumble is a permutation). *For every ℓ_M with $38 \leq \ell_M \leq \ell_{\text{MAX}}$, F4Jumble is well defined and is a bijection of the ℓ_M -byte strings. Its inverse maps $c \parallel d$, with $\text{length}(c) = \ell_L$, to $a \parallel b$ computed by*

$$y := c \oplus H_1(d), \quad x := d \oplus G_1(y), \quad a := y \oplus H_0(x), \quad b := x \oplus G_0(a).$$

Proof. Well defined. From $\ell_M \geq 38$, $\ell_L \geq 19$; from $\ell_L \leq 64$, the output length $8\ell_L$ of H_i is admissible for BLAKE2b. If $\ell_M \leq 128$ then $\ell_R \leq 64$ and one block of G_i suffices. Otherwise $\ell_L = 64$ and $\ell_R = \ell_M - 64 \leq 2^{16} \cdot 64$, so $\lceil \ell_R / \ell_H \rceil \leq 2^{16}$ and every counter j fits l2LEOSP_{16} ; this is the reason for $\ell_{\text{MAX}} = (2^{16} + 1) \ell_H$.

Bijection. Each of the four steps replaces one half by its XOR with a function of the other half, which it leaves unchanged, and $r = p \oplus q$ implies $p = r \oplus q$. Each step is therefore inverted by recomputing the same function of the unchanged half, and the composition of the four inverses in reverse order is the stated map, a two-sided inverse. This is the argument of the *Crypto*

Guide, §“Format-preserving encryption and FF1”, Proposition “Feistel is a bijection regardless of the round functions”. Unequal halves change nothing, since G_i maps into ℓ_R bytes and H_i into ℓ_L bytes, the lengths of the halves they are XORed into. $\square$

ZIP 316, “Heuristic analysis”, motivates the number of rounds.

Construction 3.10 (Unified string encoding). Human-readable parts of Revision 0: u for a UA, $uview$ for a UFVK and $uivk$ for a UIVK on Mainnet; on Testnet $test$ is appended ($utest$, $uviewtest$, $uivktest$). Let raw be the raw encoding of Definition 3.7, hrp the human-readable part and padding the human-readable part in US-ASCII right-padded with zero bytes to 16 bytes.

1. Producer. It MUST ensure $\text{length}(raw) \leq \ell_{\text{MAX}} - 16 = 4194352$, and outputs

$$\text{Bech32m}(hrp, \text{F4Jumble}(raw \parallel \text{padding})),$$

the Bech32m encoding of BIP 350 with its length restriction ignored.

2. Consumer. It MUST decode as follows, rejecting the string at the first failure:

- (1) decode with Bech32m, rejecting an incorrect checksum;
- (2) reject a decoded byte string shorter than 38 or longer than ℓ_{MAX} bytes;
- (3) apply F4Jumble^{-1} (Proposition 3.9);
- (4) reject a result that does not end in the 16-byte padding of the string’s human-readable part, and remove that padding;
- (5) parse the rest as a raw encoding, rejecting the whole string if any rule of Definition 3.7 fails.

UFVKs and UIVKs use the same procedure under their own human-readable parts (ZIP 316, “Encoding of Unified Addresses”, “Encoding of Unified Full/Incoming Viewing Keys” and “Usage for Unified Addresses, UFKs, and UIVKs”; BIP 350).

The padding supplies the redundancy that defeats the generic nonmalleability attack, since a random decoding must end in the exact 16 bytes, and gives the human-readable part the same protection against malleation as the payload, which may help against cross-network substitution and against confusion of an address with a viewing key (ZIP 316, “Usage for Unified Addresses, UFKs, and UIVKs” and “Heuristic analysis”).

Length rules (ZIP 316, “Rationale for length restrictions” and “Efficiency”). The 38-byte minimum is Revision 2’s: one P2PKH item of 22 bytes and the padding. Revision 0 specified 48 bytes. No set of items valid in Revision 0 has an encoding of 22 to 31 bytes (the smallest Revision 0 input is $45 + 16 = 61$ bytes), so the difference is unobservable except in the error reported, and a Consumer supporting Revision 2 MAY apply either minimum to Revision 0 strings. A Consumer that cannot support strings up to ℓ_{MAX} MUST support those with ℓ_M of at least 256 bytes; a full implementation supports ℓ_{MAX} .

Display rule, specified (ZIP 316, “Requirements for both Unified Addresses and Unified Viewing Keys”). If the string of a UA or UVK is shown in an abridged form, at least its first 20 characters

MUST be shown. The rationale is the ZIP’s (“Rationale for showing at least the first 20 characters”). After the longest Mainnet human-readable part of Revision 0, `uvview`, and the separator 1, 14 characters remain, 70 bits of jumbled data at 5 bits per character, below the target of 2^{125} against classical attacks, and multi-target attacks are possible; 20 is an absolute minimum and a wallet may show more. Only a prefix counts, because displays that show different substrings let a user compare only the characters common to all of them, and because checksum characters complicate the analysis of partial matching.

Item sizes. A Sapling or Orchard receiver item has $1 + 1 + 43 = 45$ bytes (typecode, length, 11-byte diversifier and 32-byte transmission key), a P2PKH receiver item $1 + 1 + 20 = 22$ bytes, and the padding 16 bytes. Hence a UA with Sapling and Orchard receivers has $\ell_M = 106$, and one with P2PKH, Sapling and Orchard receivers $\ell_M = 128$ (ZIP 316, “Efficiency”).

Example 3.11 (A unified address, end to end). Let the seed be the 32 bytes $0x00, 0x01, \dots, 0x1f$, on Mainnet, with the Orchard account 9 at $m_{\text{Orchard}}/32'/133'/9'$ (Definition 2.7), the external scope and the index $j = 1$. Every value below is computed by script from the seed.

Receiver. The diversifier key dk is the first half of the $0x82$ expansion of the account’s full viewing key, and FF1 encryption of $LE_{ss}(1)$ gives d_1 (Definition 3.1):

dk	82cc9d79742fe5ae9a142b9336a98677b154fe20401eb18998dbed915b0453ce
$LEBS2OSP_{ss}(LE_{ss}(1))$	010000000000000000000000
d_1	3fadb8edb20a3301e8260a
pk_d^*	a311f4cbd54d7d6a76baac88c244b0b121c6dc22a8bcce15898e267829fc1e01

Decryption FF1-AES256. $\text{Decrypt}(dk, "", d_1)$ returns $LE_{ss}(1)$, so $j = 1$, and the ownership test $pk_d = [\text{ivk}] \text{DiversifyHash}^{\text{Orchard}}(d_1)$ of Lemma 3.3 holds for the external ivk . It fails for the internal ivk_{int} , although dk_{int} also decrypts d_1 , to the index 184814530803446824004762235: recovery alone proves nothing, and the test on pk_d decides. The neighbouring diversifiers are $d_0 = e340636542ece1c81285ed$ and $d_2 = 987fd74a2256c596a66f83$; that consecutive indices give unrelated strings illustrates the *Ironwood Guide*’s Assumption “FF1-AES256 as a pseudorandom permutation” (§“Diversified addresses”) and is not evidence for it.

Container. The UA has one item, so its raw encoding (Definition 3.7) is the header $\text{compactSize}(0x03) \parallel \text{compactSize}(43) = 032b$ followed by the 43-byte receiver $d_1 \parallel pk_d^*$, 45 bytes in all. The padding is 75 (u) followed by fifteen zero bytes, so M has $\ell_M = 61$, $\ell_L = 30$ and $\ell_R = 31$. The cells of Construction 3.8:

$a = M[0..30)$	032b3fadb8edb20a3301e8260aa311f4cbd54d7d6a76baac88c244b0b121
$b = M[30..61)$	c6dc22a8bcce15898e267829fc1e017500000000000000000000000000000000
$x = b \oplus G_0(a)$	e6f1529b03a0ad0a4209da8e4365ca14a5988bf1d03084a1c326f929c574ca
$y = a \oplus H_0(x)$	37b358eb784bb2b315fd35c595628cfa9aa8045ef271728dd0cbf84dc81a
$d = x \oplus G_1(y)$	ad26d7fb043e09cbf18e8f4d80d2b423ecb01719170463d644be4c76daab6b
$c = y \oplus H_1(d)$	98979f9a7d2dfce7b1616a8d2b9975f87878171b6fcf33b303f770b4aa64

The header, the diversifier and the padding are legible in a and b , and in neither c nor d .

String. The 61 bytes of $c \parallel d$ regroup into 98 five-bit symbols, the last padded with two zero bits. With the human-readable part u , the separator 1 and the 6-symbol checksum, the string has $1 + 1 + 98 + 6 = 106$ characters:

u1nzte1xna9h7w0vtpd2xjxht41pu8s9cmd18n8vcr7actf2ny45nd07cy8cyuhuvw3axcp545y0ktq9cezuzx84jyhex8dk4tdvwu4d1

Decoding reverses each stage of Construction 3.10; $F4Jumble^{-1}(c \parallel d) = M$. The string equals the unified-address test vector for account 9 and index 1 of the zcash-test-vectors suite, which ZIP 32, “Test Vectors”, names.

3.5 Unified viewing keys

Definition 3.12 (Unified full and incoming viewing keys). A UFVK and a UIVK are containers of Definition 3.7 under the string encoding of Construction 3.10 with their own human-readable parts; a Consumer **MUST** use the decoding procedure of Construction 3.10. Their items, at the receiver typecodes, are the following.

1. Orchard, 0x03: the 96-byte raw full viewing key, respectively the 64-byte raw incoming viewing key, of Definition 3.6.
2. Sapling, 0x02: in a UFVK the 128 bytes

$$\begin{aligned} &\text{EncodeExtFVKParts}(ak, nk, ovk, dk) \\ &:= \text{LEBS2OSP}_{256}(\text{repr}_J(ak)) \parallel \text{LEBS2OSP}_{256}(\text{repr}_J(nk)) \parallel ovk \parallel dk \end{aligned}$$

of ZIP 32, “Sapling helper functions”, which **SHOULD** be derived from the account-level extended full viewing key; in a UIVK the 64 bytes $dk \parallel \text{I2LEOSP}_{256}(\text{ivk})$, valid only if $\text{ivk} \in \{1, \dots, 2^{251} - 1\}$ (protocol specification, §“Sapling Incoming Viewing Keys”; ZIP 316 notes the exclusion of 0). The Sapling UIVK item carries dk beyond the specification’s Sapling incoming viewing key, which is ivk alone (protocol specification, §“Sapling Incoming Viewing Keys”); dk maps an index to a diversifier and back, so a UIVK yields addresses at indices (Construction 3.16) and recovers the index of a diversifier (Lemma 3.3).

3. Transparent P2PKH, 0x00: the 65 bytes $c \parallel \text{ser}_P(pk)$ of a 32-byte BIP 32 chain code and a 33-byte compressed public key, the last 65 bytes of the BIP 32 extended public key serialisation (BIP 32, “Serialization format”). The UFVK item is the key at $m/44'/\text{coin_type}'/a'$, the account level of BIP 44, and the UIVK item its external (non-change) child at $m/44'/\text{coin_type}'/a'/0$.

The current viewing-key types reuse the receiver typecodes; future types **MAY** use others. The Priority List does not govern viewing keys: a Consumer normally takes the union of the information of all items (ZIP 316, “Encoding of Unified Full/Incoming Viewing Keys”).

The transparent item omits the BIP 32 depth, parent fingerprint and child number; the format is ZIP 316’s. That the child number of the account-level key would reveal the account index is this volume’s inference, not a statement of the ZIP. The Sapling UFVK item carries no chain code, so a UFVK supports no non-hardened Sapling child derivation below the account.

Construction 3.13 (Attenuation of a UFVK to a UIVK). Item by item, with the typecode preserved:

1. a Sapling full viewing key to its incoming viewing key (protocol specification, §“Sapling Key Components”), with dk carried over;
2. an Orchard full viewing key (ak, nk, r_{ivk}) to the external-scope incoming viewing key (dk, ivk) (*Ironwood Guide*, §“Viewing keys”);
3. a transparent P2PKH account key to its child at non-hardened index 0 by BIP 32 public-to-public derivation (BIP 32, “Public parent key $\rightarrow$ public child key”).

Items with unrecognised typecodes, and items of experiments the user has not opted into, MUST be dropped (ZIP 316, “Deriving a UIVK from a UFVK”). The treatment of metadata items is stated with them, in §3.7.

Assumption 3.14 (Independence of the per-protocol key trees). For a wallet seed S (Definition 2.1) and an account index a , the Orchard account key of Definition 2.7, the Sapling extended spending key at $m_{\text{Sapling}}/32'/\text{coin_type}'/a'$ of ZIP 32 and the BIP 32 extended private key at $m/44'/\text{coin_type}'/a'$ are jointly computationally indistinguishable from three independently generated keys of the respective protocols: a uniform Orchard spending key with a uniform chain code, a uniform Sapling master-derived key and a uniform BIP 32 key.

The assumption has a heuristic basis and no proof. It follows if BLAKE2b-512 under ZcashIP32Sapling (the Sapling master key) and HMAC-SHA512 under the key Bitcoin seed (the BIP 32 master key) are modelled as random oracles independent of BLAKE2b-512 under ZcashIP32Orchard, and the Sapling and BIP 32 hardened child derivations as PRFs keyed by the parent chain code (HMAC as a PRF: *Crypto Guide*, §“PRFs from hash functions: length extension and keyed BLAKE2”; ZIP 32, “Sapling master key generation” and “Sapling child key derivation”; BIP 32, “Master key generation” and “Private parent key $\rightarrow$ private child key”). Lemma 2.5 proves the Orchard component and nothing more.

Proposition 3.15 (Capabilities of a unified viewing key). *Let the keys of one account be generated from a seed, with use_qsk = false for the Orchard component, and let Assumption 3.14 hold.*

- (a) *From a UFVK the following are computed: for Orchard, the incoming and outgoing viewing keys of both scopes and nk, hence receipt, spend and outgoing detection (Proposition 2.15); for Sapling, the same through ZIP 32’s internal key derivation; for transparent keys, every external and internal address and the transparent outgoing viewing keys (Construction 2.18). No secret that is infeasible to compute from its own component becomes feasible from the whole UFVK: in particular the Orchard ask (*Ironwood Guide*, §“Key capabilities and scope”, Proposition “Capability separation”, part (a)), the Sapling spend authorising key and a transparent private key.*
- (b) *A UIVK gives receipt detection for the external scope in each shielded protocol it carries, and no spend detection for its shielded items: nk is infeasible to compute from an Orchard (dk, ivk) (*Ironwood Guide*, §“Key capabilities and scope”, Proposition “Capability separation”, part (b)), and internal-scope receipt is neither detected (Proposition 2.17)*

nor derivable (Proposition 2.20). A transparent UIVK item yields the external addresses $m/44'/\text{coin_type}'/a'/0/j$ and not the internal ones. Spends of transparent funds are public on the transparent chain, so their visibility is a property of the chain, not of the key.

Metadata items confer no viewing authority (§3.7), so the statements hold for containers of either revision.

Proof. Componentwise. The computations of (a) and (b) are those of the cited constructions applied to each item. For the separations: by Assumption 3.14 the components are indistinguishable from independently generated keys, so a reduction given one component generates the others itself. An algorithm that outputs a secret of one protocol from the whole container thus yields an algorithm that outputs it from that protocol’s component alone, with the advantage of Assumption 3.14 added; the success of either is efficiently decidable, by recomputing the public key or viewing key from the output. The per-component statements are Propositions 2.15, 2.17 and 2.20 with the *Ironwood Guide*, §“Key capabilities and scope”, Proposition “Capability separation”, under the assumptions named there, for Orchard; ZIP 32 and the protocol specification, §“Sapling Key Components”, cited, for Sapling; and Assumption 2.19 with BIP 32, “Security”, for the transparent keys. The transparent spend clause is a property of transparent inputs, which name the outpoints they spend in the clear. $\square$

3.6 Deriving unified addresses

Construction 3.16 (Unified address at an index). Let U be a UIVK. Choose one index $j \in \{0, \dots, 2^{88} - 1\}$, which MUST be valid for every item of U :

1. for an Orchard item, always;
2. for a Sapling item, if and only if $\text{DiversifyHash}^{\text{Sapling}}(d_j) \neq \perp$, with d_j the diversifier of the item’s dk at j (Definition 3.1);
3. for a transparent P2PKH item, if and only if $0 \leq j \leq 2^{31} - 1$ and BIP 32 derivation of the child at index j succeeds; it fails with probability below 2^{-127} (BIP 32, “Public parent key $\rightarrow$ public child key”).

Each receiver is derived from its item at the same j : a Sapling or Orchard receiver as the address at index j of (dk, ivk) (protocol specification, §“Sapling Key Components” and §“Orchard Key Components”; *Ironwood Guide*, §“Diversified addresses”, Construction “Diversified address”), and a transparent P2PKH receiver as the hash160 of the non-hardened BIP 32 public child j of the external key, so that its path is $m/44'/\text{coin_type}'/a'/0/j$ (BIP 44). Items with unrecognised typecodes, and items of experiments the user has not opted into, MUST be dropped; metadata items are treated in §3.7 (ZIP 316, “Deriving a Unified Address from a UIVK”).

Proposition 3.17 (Indices valid for a unified key). Let $V(U)$ be the set of indices valid for every item of a UIVK U .

- (a) If U has only an Orchard item, $V(U) = \{0, \dots, 2^{88} - 1\}$.

(b) If U has a transparent item, $V(U) \subseteq \{0, \dots, 2^{31} - 1\}$, a fraction $2^{31}/2^{88} = 2^{-57}$ of the index space, so U has at most 2^{31} unified addresses.

(c) Let U have a Sapling item, and let BLAKE2s-256 in $\text{GroupHash}^{J(r)*}$ be modelled as a random oracle, the model of the protocol specification, §“Group Hash into Jubjub”, notes. The events that the indices are valid for the Sapling item are independent over distinct indices, each of probability

$$\pi := \frac{b_J (r_J - 1)}{2^{256}} = \frac{8 (r_J - 1)}{2^{256}} \approx 0.4528,$$

with $b_J = 8$ the cofactor and r_J the prime subgroup order of Jubjub (protocol specification, §“Jubjub”); an index is invalid with probability $1 - \pi \approx 0.5472$. ZIP 32’s “approximately half” and ZIP 316’s probability $1/2$ are this value rounded.

(d) With a Sapling and a transparent item, $V(U)$ is the set of indices below 2^{31} that are Sapling-valid and at which BIP 32 derivation succeeds; its expected size lies within $2^{31} \cdot 2^{-127} = 2^{-96}$ of $\pi \cdot 2^{31}$.

For the Sapling key of account 9 of the seed of Example 3.11, 4527 of the indices $0, \dots, 9999$ are valid, a fraction 0.4527, and index 0 is valid.

Proof. The set $V(U)$ is the intersection of the per-item valid sets of Construction 3.16, which gives (a) and (b), distinct indices giving distinct receivers.

(c) Distinct indices give distinct diversifiers (Proposition 3.2 (a)), hence distinct inputs to BLAKE2s-256 , whose outputs h are then independent and uniform on $\{0, 1\}^{256}$. The group hash returns a point if and only if $P := \text{abst}_J(h) \neq \perp$ and $[h_J]P \neq \mathcal{O}$ (protocol specification, §“Group Hash into Jubjub”). The map abst_J reads h as a 255-bit integer $v < q_J$ and a sign bit $\tilde{u}$, rejecting $v \geq q_J$ and v with no curve point (protocol specification, §“Jubjub”). Each point with $u \neq 0$ is the image of exactly one h , since u and $q_J - u$ have different parities, and each of the two points $(0, \pm 1)$ is the image of two. The curve has $b_J r_J$ points and its b_J -torsion subgroup, the kernel of $[h_J]$, has 8 points: $(0, 1)$, $(0, -1)$ and six with $u \neq 0$. The strings h for which the hash succeeds are thus in bijection with the points with $u \neq 0$ outside that subgroup, of which there are $b_J r_J - 2 - 6 = 8 (r_J - 1)$, so $\pi = 8 (r_J - 1) / 2^{256}$.

(d) Parts (b) and (c) and Construction 3.16 (3) combined by linearity of expectation over the 2^{31} indices below 2^{31} , each of which fails BIP 32 derivation with probability below 2^{-127} . $\square$

Remark 3.18 (Default addresses). ZIP 316 fixes neither an order in which a wallet searches for a valid index nor a default unified address. The only normative defaults are ZIP 32’s per-protocol default diversifiers: for Sapling d_j with j the least valid index, for Orchard d_0 (ZIP 32, “Sapling diversifier derivation” and “Orchard diversifier and OVK derivation”). Issuing the unified address at the least index valid for every item is designed but unspecified (Definition 1.1).

Index 0 and recovery across wallets, specified (ZIP 316, “Deriving a Unified Address from a UIVK”). Index 0 is invalid for a Sapling item with probability $1 - \pi$ (Proposition 3.17), so a wallet that requires a Sapling receiver often issues its first unified address at a later index, while some

wallets, earlier than ZIP 316 or without a Sapling component in their unified addresses, always generate a transparent P2PKH address at index 0. Therefore every Zcash wallet, whether or not it supports unified addresses, MUST assume that transparent funds may exist at diversifier index 0 of each ZIP 32 account, even where it would never generate a unified address at that index; reliable recovery of funds when key material moves between wallets depends on this. Its use in restoration is stated in §6.2.

3.7 Revision 2 containers

Definition 3.19 (Metadata items). The following is Revision 2 of ZIP 316, specified (Definition 1.1).

1. Human-readable parts. A UA is either *shielded-only*, with human-readable part *zu*, which MUST NOT hold a transparent item (typecode `0x00` or `0x01`), a Consumer rejecting any that does, or *transparent-enabled*, with *tu*. A UFVK uses *uvf* and a UIVK *uvi*; on Testnet test is appended. The human-readable part distinguishes a Revision 0 container (*u*, *uview*, *uivk*) from a Revision 2 one.
2. Metadata items, typecodes `0xC0` to `0xFC` (Definition 3.7), MAY affect the interpretation of the container, MUST NOT be selected as receivers and MUST NOT confer viewing authority.
3. MUST-understand items. Typecodes `0xE0` to `0xFC` are MUST-understand: a Consumer that does not recognise such an item MUST regard the whole container as unsupported and not process it further. This is the exception to the ignore rule of Definition 3.7.
4. Registry. Typecodes `0x04` to `0xBF` are unassigned; `0xC0` to `0xDF` are metadata that need not be understood, none assigned; `0xE0` is the Address Expiry Height and `0xE1` the Address Expiry Time, permitted in *zu* and *tu* addresses and not in Revision 0; `0xE2` to `0xFC` are unassigned MUST-understand metadata; `0xFD` to `0xFFF9` are unassigned; and `0xFFFA` to `0xFFFF` are experimental. Producers MUST NOT include items whose typecodes are unassigned or not permitted for the type of the container.
5. Address expiry items. An item of typecode `0xE0` holds the *Address Expiry Height*, an unsigned 32-bit little-endian integer; a UA carrying it MUST be considered expired when the height of the chain exceeds that value. An item of typecode `0xE1` holds the *Address Expiry Time*, an unsigned 64-bit little-endian Unix time; a UA carrying it MUST be considered expired when the current time is after that value. A Producer MAY include either, both or neither.
6. Retroactive rule. A Revision 0 container MUST NOT hold a MUST-understand metadata item, and a Consumer MUST reject one that does.
7. Derivation. To derive a UIVK from a UFVK, or a UA from a UFVK or UIVK, a Consumer MUST understand every MUST-understand item of the source. The derived container MUST be Revision 2 if the source holds any metadata item, and SHOULD be

Revision 2 if the source is. A UIVK derived from a UFVK MUST retain its 0xE0 and 0xE1 items unmodified. A UA derived from a UFVK or UIVK that carries 0xE0 (respectively 0xE1) MUST carry 0xE0 (respectively 0xE1) with a value at most that of the viewing key. Metadata items that need not be understood and are unrecognised are dropped under Constructions 3.13 and 3.16.

8. Containment. Revision 2 drops the rule that a UVK holds a shielded item, and admits a tu UA that consists of one P2PKH receiver (Definition 3.7). Revision 2 also admits a P2SH viewing-key item, which lies outside this volume; Table 3 is that of Revision 0.

Sources: ZIP 316, “Revisions”, “Metadata Items”, “Address Expiration Metadata”, “Typecode Registry”, “Requirements for both Unified Addresses and Unified Viewing Keys”, “Deriving a UIVK from a UFVK” and “Deriving a Unified Address from a UIVK”.

A Consumer that predates Revision 2 does not recognise the Revision 2 human-readable parts. Requiring MUST-understand items to appear only under those parts therefore enforces the MUST-understand rule even against such a Consumer (ZIP 316, “Rationale for making Revision 0 UA/UVKs with MUST-understand Typecodes invalid”).

Sender rules for expiring addresses, specified (ZIP 316, “Address Expiration Metadata” and its “Rationale”). The expiry height of a transaction is its field `nExpiryHeight` (*Ironwood Guide*, §“The transaction format”). A Sender supporting Revision 2 MUST set a nonzero `nExpiryHeight` in a transaction to a UA with an Address Expiry Height, and MUST NOT send the transaction if the `nExpiryHeight` it would otherwise construct exceeds that height; lowering `nExpiryHeight` to the address’s value instead would reveal the destination’s expiry on chain. With only an Address Expiry Time, it SHOULD choose `nExpiryHeight` so that the transaction expires no more than 24 hours after the current time. With both items both restrictions apply, and with several recipient UAs in one transaction all their constraints apply. A wallet MUST NOT send to an address it knows to have expired, and its error SHOULD suggest obtaining a current address. The choice of `nExpiryHeight` is stated in §9.3.

Linkability through metadata (ZIP 316, “Address Expiration Metadata”). Diversified UAs derived from one UIVK that hold only Sapling or Orchard receivers and no metadata are unlinkable (§3.1); with metadata items present this no longer holds, and it is RECOMMENDED that the expiry height and time of distinct derived addresses vary. Re-issuing the same receivers with different expiry metadata yields a visually distinct address that is directly linkable to the original, since the receivers are shared.

3.8 Recipient addresses

Definition 3.20 (Recipient address). A *recipient address* is one of the following:

1. a transparent address, P2PKH or P2SH, in its Base58Check encoding (Definition 3.6 (f));
2. a Sapling payment address in its Bech32 encoding (Definition 3.6 (e));

3. a unified address (Definition 3.7, Construction 3.10 and Definition 3.19);
4. a *TEX address* (transparent-source-only address): the Bech32m encoding, with human-readable part *tex* on Mainnet and *textest* on Testnet, of the 20-byte validating-key hash of a P2PKH address. It is obtained from a Base58Check P2PKH address whose decoding has 22 bytes beginning with the lead bytes $[0x1c, 0xb8]$ (Testnet $[0x1d, 0x25]$) by removing them, and it is parsed by requiring the human-readable part and exactly 20 decoded bytes (ZIP 320, “Specification”).

Sprout addresses are not recipient addresses: no transaction adds value to the Sprout pool, and no typecode exists for them (§14.2; ZIP 316, “Requirements for both Unified Addresses and Unified Viewing Keys”). A Sender paying a TEX address, as any output of a transaction, **MUST** spend only transparent (P2SH or P2PKH) outputs in that transaction, and **SHOULD** send only to transparent outputs (ZIP 320, “Specification”; §11.3).

Two predicates on recipient addresses, defined here, are used by payment construction and by ZIP 321 (ZIP 321, “memo”; §14.2).

- (i) An address is *memo-capable* if and only if it is a Sapling payment address or a unified address that contains a Sapling or Orchard receiver, since a memo travels only in a note plaintext (*Ironwood Guide*, §“The note plaintext”). A memo reaches the recipient of a unified address only when the Sender selects a shielded receiver.
- (ii) An address is *transparent-only* if and only if it is a transparent or TEX address, or a unified address whose non-metadata items are all transparent receivers, which is possible only as a Revision 2 tu address.

4 Compact blocks (ZIP 307)

This section states the compact block as an abstract message format: the records of a block and of a transaction, the compact outputs, spends and Actions and the fields they omit, the compact variant of trial decryption with the soundness of accepting a plaintext that no tag authenticates, and the placement of notes in the note commitment trees from per-block tree sizes. The service that delivers compact blocks is the subject of “The light-client service” (§5).

4.1 The compact block and the compact transaction

A *compact block* carries only the data of a block that a wallet needs to detect payments to its addresses, to detect spends of its notes and to maintain the witnesses of its notes (ZIP 307, “Compact Stream Format”); for the transparent pool it carries, on request, the data needed to detect spends of the wallet’s transparent coins.

Remark 4.1 (Normative status of the compact format). ZIP 307, “Compact Stream Format”, “Output Compression” and “Spend Compression”, specifies the compact format for the Sapling pool only: a compact block of height, block hash and transactions, and a compact transaction of index, transaction identifier, spends and outputs. It names neither the Orchard pool nor the Ironwood pool, nor per-block tree sizes, nor transparent data. ZIP 318 and ZIP 326 address light

wallets and specify no compact format. The remainder of the format below (the parent hash of a block, the tree sizes, the compact Actions of both Orchard-protocol pools, the transparent part and pool selection) is designed but unspecified (Definition 1.1). The volume states the format as an abstract message format, not as a serialisation, and states no field number, field width or encoded size. The status of each ZIP cited is that of Table 1.

Definition 4.2 (Compact block). A *compact block* of height h is a tuple

$$CB = (h, \text{hash}, \text{prevHash}, \text{hdr}, \text{vtx}, S)$$

where

1. $h \in \mathbb{N}$ is the block height;
2. hash and prevHash are the 32-byte hashes of the block and of its parent block (protocol specification, §“Block Header Encoding and Consensus”);
3. hdr is either the encoded block header or empty; the abstract format does not guarantee that it is present;
4. $\text{vtx} = (ctx_0, \dots, ctx_{m-1})$ is the sequence of compact transactions (Definition 4.3) of the block, in block order;
5. $S = (S^{\text{Sapling}}, S^{\text{Orchard}}, S^{\text{Ironwood}})$ gives, for each shielded pool pool, the number S^{pool} of leaves of that pool’s note commitment tree after every transaction of the block has been appended, written $S_{\text{end}}^{\text{pool}}(h)$.

Each S^{pool} is an integer in $[0, 2^{32}]$, where 2^{32} is the leaf capacity of a tree of depth $\text{MerkleDepth}^{\text{Sapling}} = \text{MerkleDepth}^{\text{Orchard}} = 32$ (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”; protocol specification, §“Constants”). The width of any serialisation of S^{pool} lies outside the abstract format.

Definition 4.3 (Compact transaction). A *compact transaction* is a tuple

$$ctx = (i, \text{txid}, SSp, SOut, A^{\text{Orchard}}, A^{\text{Ironwood}}, T)$$

where

1. $i \in \mathbb{N}$ is the index of the transaction in its block;
2. txid is the 32-byte transaction identifier in protocol byte order, the form in which it is computed, not the byte-reversed form used for display (*Ironwood Guide*, §“Transaction digests and signatures”, Definition “Transaction identifier”; protocol specification, §“Transaction Identifiers”);
3. SSp and $SOut$ are the sequences of compact Sapling spends and compact Sapling outputs, and A^{Orchard} and A^{Ironwood} the sequences of compact Actions of the Orchard pool and of the Ironwood pool (Definition 4.5), each in the order of the transaction’s descriptions, on which note positions depend (§4.4);

4. T is an optional *transparent part* $(O_{\text{in}}, O_{\text{out}})$: O_{in} is the sequence of outpoints (txid', n) spent by the transaction's transparent inputs, in input order, and O_{out} the sequence of pairs (value, script) of its transparent outputs, in output order, with value in zatoshi.

The coinbase transaction of a block is its first transaction and the one whose single transparent input has a null previous outpoint (protocol specification, §“Coinbase Transactions”); it spends no outpoint, its O_{in} is empty, and it is recognised by $i = 0$. No fee field is part of the abstract format.

Definition 4.4 (Pool selection). Let $\mathcal{P} \subseteq \{\text{transparent}, \text{Sapling}, \text{Orchard}, \text{Ironwood}\}$. The compact block *restricted to* $\mathcal{P}$ is obtained from a compact block by keeping, in each compact transaction, only the components of the pools in $\mathcal{P}$ (SSp and $SOut$ for Sapling, the sequence A^{pool} for an Orchard-protocol pool, and T only when transparent $\in \mathcal{P}$), and by omitting every compact transaction left with no component. The tuple S is kept whole, and a compact block is returned for every block of a requested range, possibly with empty vtx. The default is $\mathcal{P} = \{\text{Sapling}, \text{Orchard}, \text{Ironwood}\}$. The range query of “The query interface” (§5.2) takes $\mathcal{P}$ as an argument. Pool selection is designed but unspecified (Definition 1.1); ZIP 307 includes by default only the transactions that have Sapling spends or outputs (ZIP 307, “Block header validation”).

For every shielded pool in $\mathcal{P}$ the restriction keeps every commitment of that pool in its order, since an omitted compact transaction has no component of the pool. The per-pool counts and offsets of Definition 4.11 are therefore the same on a restricted block as on the unrestricted one.

4.2 Compact outputs, spends, and Actions

Definition 4.5 (Compact spend, output, and Action). Let C_{enc} be the note ciphertext of a Sapling output or of an Action: 580 bytes, the encryption of the 564-byte note plaintext followed by the 16-byte tag. Its split at the offsets 52 and 564 is that of the *Ironwood Guide*, §“Transaction digests and signatures”, Definition “Transaction identifier”.

- (a) A *compact Sapling spend* is the 32-byte nullifier nf of a Spend description.
- (b) A *compact Sapling output* is the triple $(\text{cmu}, \text{ephemeralKey}, C_{\text{enc}}[0:52])$, a payload of $32 + 32 + 52 = 116$ bytes.
- (c) A *compact Action*, of the Orchard pool and of the Ironwood pool alike, is the tuple $(\text{nf}, \text{cmx}, \text{ephemeralKey}, C_{\text{enc}}[0:52])$, a payload of $32 + 32 + 32 + 52 = 148$ bytes.

Parts (a) and (b) are specified (ZIP 307, “Output Compression” and “Spend Compression”); part (c) is designed but unspecified (Definition 1.1). The fields are those of the protocol specification, §“Spend Description Encoding and Consensus”, §“Output Description Encoding and Consensus” and §“Action Description Encoding and Consensus”. Only payload sizes are stated; the size of any serialisation lies outside the abstract format.

The nullifier nf of a compact Action has two roles. It detects a spend of the note that the Action consumes, and it is the element ρ of the note that the Action creates (*Ironwood Guide*, §“Nullifier

chaining”, Construction “Chaining rule”), an input of esk , ψ and rcm and hence of the commitment that acceptance recomputes (Definition 4.9). A compact Action without nf therefore cannot be accepted.

A compact block omits the memo $C_{\text{enc}}[52:564]$, the tag $C_{\text{enc}}[564:580]$, the outgoing ciphertext C_{out} , value commitments, randomised validating keys, anchors, proofs and signatures (ZIP 307, “Output Compression”, “Spend Compression” and “Transaction privacy”). Recovery of a memo or of an outgoing plaintext requires the full transaction, fetched by its identifier (“Leakage of transaction queries”, §8.4).

Proposition 4.6 (Sufficiency of the compact fields). *Let CB be a compact block of height h , and let a wallet hold incoming viewing keys and the nullifiers of its notes.*

- (i) *Receipt detection, that is compact decryption and acceptance of an output (§4.3), is a function of the key, of h , and of the record $(\text{cmu}, \text{ephemeralKey}, C_{\text{enc}}[0:52])$ of a Sapling output or $(\text{nf}, \text{cmx}, \text{ephemeralKey}, C_{\text{enc}}[0:52])$ of an Action.*
- (ii) *Spend detection is a function of the fields nf of SSp , A^{Orchard} and A^{Ironwood} .*
- (iii) *Note placement is a function of the order of the commitments in vtx and of S (§4.4).*

No field omitted by the compact block enters (i) to (iii).

Proof. (i) Trial decryption and acceptance read the ephemeral key, the ciphertext prefix that encrypts the lead byte, diversifier, value and note seed, the commitment, the height through the admissible lead bytes, and, for an Action, nf as ρ (*Ironwood Guide*, §“Trial decryption and note acceptance”, Constructions “Trial decryption” and “Acceptance of an Ironwood-pool output”; ZIP 307, “Scanning for relevant transactions” for Sapling); the memo and the tag enter only the full decryption. (ii) A spend of a note is detected by membership of its nullifier among the published ones (ZIP 307, “Detecting spends”). (iii) The chain appends each pool’s commitments in block, transaction and description order to the least unused leaf (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”; protocol specification, §“Note Commitment Trees”), so positions follow from that order and a tree size. $\square$

The omitted fields serve the verification of the chain, which the client delegates to the server; that delegation and its assumption are stated in “Architecture and authority” (§5.1).

4.3 Compact trial decryption

Trial decryption and acceptance of full outputs are those of the *Ironwood Guide*, §“Trial decryption and note acceptance” (Constructions “Trial decryption” and “Acceptance of an Ironwood-pool output”), cited and not re-derived. This subsection states the compact variant.

Definition 4.7 (Compact note plaintext). The *compact note plaintext* is the tuple $\text{np}_c = (\text{leadByte}, d, v, \text{rseed})$, encoded as

$$\text{leadByte} \parallel \text{LEBS2OSP}_{88}(d) \parallel \text{I2LEOSP}_{64}(v) \parallel \text{rseed},$$

of $1 + 11 + 8 + 32 = 52$ bytes: the note plaintext of the *Ironwood Guide*, §“The note plaintext”, Definition “Note plaintext”, without its memo (protocol specification, §“Note Plaintexts and Memo Fields” and §“Encodings of Note Plaintexts and Memo Fields”). The AEAD encryption is a stream-cipher encryption followed by a tag, so $C_{\text{enc}}[0:52]$ is the encryption of exactly these 52 bytes, which are the contents and the opening of the note commitment (ZIP 307, “Output Compression”).

Construction 4.8 (Compact decryption of an output). Input: an incoming viewing key ivk of the pool’s protocol and a compact output or compact Action with fields ephemeralKey and $c = C_{\text{enc}}[0:52]$.

1. Compute $\text{epk} := \text{abst}(\text{ephemeralKey})$ on the pool’s curve, abst_j on Jubjub for Sapling and abst_p on Pallas for either Orchard-protocol pool; return $\perp$ if $\text{epk} = \perp$.
2. Compute $K := \text{KDF}(\text{KA.Agree}(\text{ivk}, \text{epk}), \text{ephemeralKey})$ with the key agreement and key derivation of the pool’s protocol.
3. Compute $\text{np}_c := c \oplus \text{KS}_1(K, 0^{96}, 52)$, where $\text{KS}_1(K, 0^{96}, 52)$ is the first 52 bytes of the ChaCha20 keystream under key K and the all-zero nonce 0^{96} of the protocol specification, §“Symmetric Encryption”, starting at block counter 1, the counter from which ChaCha20-Poly1305 encrypts (block 0 yields the Poly1305 key; *Crypto Guide*, §“The ChaCha20-Poly1305 AEAD”, Construction “ChaCha20-Poly1305”; RFC 8439, §2.8); equivalently, bytes 64 to 115 of the keystream from counter 0.
4. Return np_c parsed as a compact note plaintext (Definition 4.7).

The tag is never checked. The result is a *candidate*, not an accepted note (ZIP 307, “Scanning for relevant transactions”).

Definition 4.9 (Acceptance of a compact candidate). Let $\text{np}_c = (\text{leadByte}, d, v, \text{rseed})$ be the candidate returned by Construction 4.8 under ivk for an output of pool pool in a compact block of height b . Let H_c be the Canopy activation height and $\text{ZIP212GracePeriod} = 32,256$ blocks (protocol specification, §“Constants”). The set of admissible lead bytes is

$$\text{allowedLeadBytes}^{\text{pool}}(b) := \begin{cases} \{0x01\} & \text{if } b < H_c, \\ \{0x01, 0x02\} & \text{if } H_c \leq b < H_c + \text{ZIP212GracePeriod}, \\ \{0x02\} & \text{otherwise, if pool} \neq \text{Ironwood}, \\ \{0x03\} & \text{otherwise} \end{cases}$$

(protocol specification, §“Note Plaintexts and Memo Fields”), so that an Orchard-pool note has lead byte $0x02$ and an Ironwood-pool note $0x03$. The wallet accepts np_c if and only if every check passes:

- (i) $\text{leadByte} \in \text{allowedLeadBytes}^{\text{pool}}(b)$; for Sapling, moreover, $\text{rcm} < r_j$ and $\text{g}_d := \text{DiversifyHash}(d) \neq \perp$, with $\text{rcm} := \text{rseed}$ read as an integer for lead byte $0x01$;

- (ii) with $pk_d := [ivk] g_d$, and with $\rho := nf$ of the same Action for either Orchard-protocol pool, the note commitment recomputed from $(g_d, pk_d, v, \rho, \psi, rcm)$, where ψ and rcm are derived from $rseed$ as the pool and the lead byte prescribe (ρ and ψ absent for Sapling), is not $\perp$ and extracts to the record's cmu (Sapling) or cmx (either Orchard-protocol pool);
- (iii) for every lead byte other than $0x01$, the field `ephemeralKey` equals the encoding of $[esk] g_d$, with esk re-derived from $rseed$, and from ρ for either Orchard-protocol pool.

The accepted note is $n = ((d, pk_d), v, \rho, \psi, rcm)$ without memo, with ρ and ψ absent for a Sapling note. The derivations of ψ , rcm and esk are cited and not restated: for Sapling and the Orchard pool to the protocol specification, §“Decryption using an Incoming Viewing Key (Sapling and Orchard)” (ZIP 212, “Changes to the process of sending Sapling or Orchard notes”); for the Ironwood pool to the *Ironwood Guide*, §“Trial decryption and note acceptance”, Construction “Acceptance of an Ironwood-pool output”, whose checks (i), (ii) with (iv), and (iii) are checks (i), (ii) and (iii) here (ZIP 2005, “Changes to the Protocol Specification”). The method of comparison is not part of the rule.

Check (iii) is a step of the specification’s decryption procedure and of ZIP 307, not an addition (ZIP 212, “Changes to the process of receiving Sapling or Orchard notes”). For an Ironwood-pool note it is made directly after the computation of g_d , in the order of ZIP 2005. Without it a sender could agree the key with one address of a recipient while naming the diversifier of another, and link the two addresses by observing acceptance (*Ironwood Guide*, §“Trial decryption and note acceptance”, Remark “Role of check (iii)”); ZIP 212, “Motivation”). A Sapling note with lead byte $0x01$, created under the rules before ZIP 212, skips check (iii), since its esk is not derivable from $rseed$.

For a note n the *commitment input* is the pair of committed values and trapdoor from which check (ii) computes its commitment: $((g_d, pk_d, v), rcm)$ for Sapling and $((g_d, pk_d, v, \rho, \psi), rcm)$ for either Orchard-protocol pool. An *address of ivk* is a pair $(d, [ivk] \text{DiversifyHash}(d))$ with $\text{DiversifyHash}(d) \neq \perp$.

Proposition 4.10 (Soundness of compact acceptance). *Let cm^* be the commitment, cmu or cmx , that a compact output or compact Action carries.*

- (a) *Every note n accepted by Definition 4.9 is an opening of cm^* . For every efficient sender, which chooses `ephemeralKey`, C_{enc} and the keys under which they are formed, the probability that the wallet accepts n while the sender outputs a note n' whose commitment input differs from that of n and whose commitment extracts to cm^* is negligible, under the binding of the pool’s note commitment scheme. The tag of the AEAD gives no integrity against the sender, who holds K , so its omission loses none.*
- (b) *For an output honestly formed for a note n' whose address is not an address of ivk, the probability that ivk accepts a candidate is negligible under the same assumption.*

Whether cm^ is the commitment of an output of the best chain is not a property of decryption; it is settled under the assumption of “Architecture and authority” (§5.1).*

Proof. (a) Check (ii) recomputes the commitment of n deterministically from its commitment input and compares it with cm^* , whatever the ciphertext and whichever key decrypted it, so n opens cm^* . Consider the algorithm that generates the wallet's key, runs the sender and the compact decryption and acceptance, and outputs the commitment inputs of n and n' . When the event of (a) occurs it outputs two distinct commitment inputs whose commitments extract to the same $cm^* \neq \perp$. For either Orchard-protocol pool this is excluded, except with negligible probability, by the argument of the *Ironwood Guide*, §“Trial decryption and note acceptance”, Proposition “Consistency of accepted notes”, part (c), under Proposition “Hiding and binding of note commitments” (§“Note commitments”). That proof uses neither the integrity of ChaCha20-Poly1305 nor key commitment, so it applies unchanged to a candidate decrypted from the 52-byte prefix; and binding holds for every rcm, hence also for the trapdoor derivation of the Orchard pool. For Sapling the named assumption is the computational binding of $\text{NoteCommit}^{\text{Sapling}}$, the security requirement of the protocol specification, §“Windowed Pedersen commitments”, together with the injectivity of $\text{Extract}_{\mathbb{J}^{(r)}}$ on $\mathbb{J}^{(r)}$ (protocol specification, §“Coordinate Extractor for Jubjub”).

(b) Let n be accepted. By check (ii) its address is $(d, [\text{ivk}] g_d)$ with $g_d = \text{DiversifyHash}(d) \neq \perp$, an address of ivk . Suppose the commitment inputs of n and n' were equal. Then the diversified base $\text{DiversifyHash}(d')$ of the address (d', pk_d') of n' equals g_d , and $\text{pk}_d' = [\text{ivk}] g_d = [\text{ivk}] \text{DiversifyHash}(d')$, so (d', pk_d') is an address of ivk , contrary to the hypothesis. The commitment inputs therefore differ, and the honest sender outputs n' , whose commitment extracts to cm^* ; the honest sender is an efficient sender of (a), which bounds the probability. $\square$

4.4 Note positions and tree sizes

For each shielded pool, every note commitment of a block, not only those of the wallet's notes, is appended to the wallet's copy of that pool's note commitment tree in block order, transaction order and description order: cmx of each Action of the pool and cmu of each Sapling output (ZIP 307, “Creating and updating note witnesses”; *Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”; protocol specification, §“Note Commitment Trees”). The leaf of each accepted note is marked for witness retention. The part of the tree a wallet must retain is fixed in “Commitment-tree synchronisation” (§7).

Definition 4.11 (Note position). Let pool be a shielded pool, b a block height with compact block CB , and $n_{\text{pool}}(b)$ the number of commitments of pool in CB . The *start size* of b is

$$S_{\text{start}}^{\text{pool}}(b) := \begin{cases} S_{\text{end}}^{\text{pool}}(b-1) & \text{if the wallet knows it,} \\ 0 & \text{otherwise, if } b \text{ lies below the activation of pool,} \\ S_{\text{end}}^{\text{pool}}(b) - n_{\text{pool}}(b) & \text{otherwise,} \end{cases}$$

the wallet knowing $S_{\text{end}}^{\text{pool}}(b-1)$ when it holds the compact block of height $b-1$ or a tree state of pool at the end of $b-1$; in the third case the value is taken from CB alone, and the block is rejected if it is negative. For the j -th output o of pool, counted from $j = 0$, of a compact transaction ctx of b ,

$$\text{position}(o) := S_{\text{start}}^{\text{pool}}(b) + \text{off}(\text{ctx}) + j,$$

where $\text{off}(ctx)$ is the number of commitments of pool in the compact transactions of b before ctx . Positions are per pool. A block scanned out of order thus places its notes from its own compact block. Note positions are designed but unspecified (Definition 1.1).

For a pool outside the pool set $\mathcal{P}$ of the query that returned b , the count $n_{\text{pool}}(b)$ is not served, and neither position nor check is defined.

The position of a note is the leaf index of its commitment (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”, item 1). Every spend needs it for the authentication path. A Sapling nullifier depends on it, through $\rho = \text{MixingPedersenHash}(\text{cm}, \text{pos})$ (protocol specification, §“Computing ρ values and Nullifiers”), while the nullifier of an Orchard-protocol note does not.

Lemma 4.12 (Consistency of tree sizes). *Let the wallet reject a block b unless, for each shielded pool of the pool set $\mathcal{P}$ of the query that returned b ,*

$$S_{\text{end}}^{\text{pool}}(b) \geq n_{\text{pool}}(b) \quad \text{and} \quad S_{\text{start}}^{\text{pool}}(b) + n_{\text{pool}}(b) = S_{\text{end}}^{\text{pool}}(b).$$

- (a) *If every reported size is the true size, no block is rejected, and every position of Definition 4.11 is the leaf index of the commitment in the pool’s tree.*
- (b) *A reported $S_{\text{end}}^{\text{pool}}(b)$ that is inconsistent with the block’s own commitment count, or with the known end size of $b - 1$, is rejected.*
- (c) *A block with $n_{\text{pool}}(b) > 0$ and reported size 0 is rejected. A size read as zero therefore yields a position only where 0 is consistent with the block, and the check needs no means of telling an absent size from zero.*
- (d) *A size that is false but consistent with both conditions is not detected; its exclusion is clause (a) of the server assumption of “Architecture and authority” (§5.1).*

Proof. (a) The chain appends the commitments of a pool in block, transaction and description order, each to the least unused index (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”). The true start size of b is thus the true end size of $b - 1$, which equals $S_{\text{end}}^{\text{pool}}(b) - n_{\text{pool}}(b) \geq 0$, so both conditions hold in every case of the start size; and the j -th commitment of ctx lands at index $S_{\text{start}}^{\text{pool}}(b) + \text{off}(ctx) + j$. (b) A size $S_{\text{end}}^{\text{pool}}(b) < n_{\text{pool}}(b)$ fails the first condition. With a known start size, a size that differs from $S_{\text{start}}^{\text{pool}}(b) + n_{\text{pool}}(b)$ fails the second. (c) A size 0 with $n_{\text{pool}}(b) > 0$ fails the first condition. (d) Let the wallet not know the end size of $b - 1$, and let b lie at or above the activation of pool, so that its start size is that of the third case of Definition 4.11. Adding one integer $\delta \neq 0$ to $S_{\text{end}}^{\text{pool}}(b)$ and to the end sizes of all later scanned blocks, such that every shifted size stays in $[0, 2^{32}]$ and at least the count of its block, preserves both conditions for b , whose start size is computed from its own end size, and for every later block, whose start and end sizes shift by δ together. $\square$

Comparison of the running position with $S_{\text{end}}^{\text{pool}}(b)$ identifies the last commitment of pool in b , at position $S_{\text{end}}^{\text{pool}}(b) - 1$. The tree after that commitment, or for $n_{\text{pool}}(b) = 0$ the tree at the end

of $b - 1$, is the tree state at the end of b . Its root is the anchor that a later spend against height b uses (*Ironwood Guide*, §“Anchors”; protocol specification, §“Transactions and Treestates”), and the wallet records it as a candidate anchor.

5 The light-client service

This section states the server from which a light wallet obtains chain data. It fixes the architecture and the one assumption on the server that the volume uses, states which checks a light client can make on the server’s responses and what the server can fabricate or omit undetected, and defines the service as an abstract interface of typed queries on the chain, the note commitment trees, transactions, submission and transparent addresses.

5.1 Architecture and authority

Definition 5.1 (Light-client architecture). A *light-client system* has three components:

1. a *full node*, which validates the chain and is the root of trust for chain state;
2. a *proxy*, which extracts from the node’s view of the best chain the data of compact blocks (Definition 4.2) and answers queries;
3. a *light client*, which holds keys and no copy of the chain, queries the proxy, and maintains its own view of chain state and of its notes.

This volume calls the node and the proxy together the *server*. The wallet of “Parties, pools, and classifications” (§1.2) is the light client. The best chain is that of the *Consensus Guide*, §“Work, and the best chain” (ZIP 307, “High-Level Design”).

Assumption 5.2 (Honest-but-curious server). The server satisfies two clauses.

- (a) *Integrity*. It answers every query from a correct view of the current best chain, each response being the value that the interface of “The query interface” (§5.2) assigns to the query on that chain, and it transmits queries and responses faithfully.
- (b) *Curiosity*. It records every query with its arguments, order and timing, every response, and the transport metadata of every connection, and computes arbitrarily on that record.

Every correctness result of this volume that relies on the server cites clause (a); the privacy results of §8 take clause (b) as the power of the adversary. No other server model is used. ZIP 307, “Security Model”, states the same model: the node and proxy together are honest but curious, and are trusted to provide a correct view of the current best chain and to transmit queries and responses faithfully. Its goal is payment-detection privacy: the proxy does not learn which transactions are addressed to a given wallet and, given network privacy provided separately, does not link different connections or queries to one wallet; the goal is defined in “Payment-detection privacy” (§8.1). The ZIP does not address private spending.

Proposition 5.3 (Light-client checks). *Let a wallet receive responses of the service (§5.2), with compact blocks that carry no header, which the abstract format does not guarantee (Definition 4.2). Then every check the wallet can make compares two responses, or a response with a value the wallet computes from responses. The relations among response fields that the format fixes are the following:*

- (i) *height continuity and prevHash linkage of consecutive compact blocks, and of the first block of a range with the block the wallet has stored, the checks of ZIP 307, “Client-server interaction”, phases B and D, and “Block header validation”;*
- (ii) *consistency of the per-block tree sizes with the commitments served (Lemma 4.12);*
- (iii) *recomputation of the note commitment of each accepted candidate (Definition 4.9, Proposition 4.10);*
- (iv) *agreement of a served tree node (a frontier, a subtree root, a root) with its recomputation from commitments the wallet has scanned;*
- (v) *agreement of a fetched transaction with its queried identifier and with the compact records of that identifier (Remark “Checks of a fetched transaction”);*
- (vi) *equality of a height, block hash or transaction identifier that several responses report.*

No check relates a response to a block header or to proof of work: the wallet verifies no tree root, nullifier, transaction identifier or block hash against a header.

Proof. The fields of a compact block and of its elements (Definitions 4.2, 4.3 and 4.5) and the responses of the service are values of the chain or tree data derived from it; without hdr no response carries a header, so no field, not even hash or prevHash, which the header determines, can be checked against one. A check is a predicate on the wallet’s inputs, which are these responses and its keys, so it compares responses with one another or with values computed from them. Items (i) to (vi) relate pairs of fields that the format makes functions of one another: the heights and hashes of adjacent blocks; a block’s commitment count and its sizes; a record’s commitment and its plaintext; a tree node and the leaves below it; a fetched transaction and its identifier and compact records; and a height, hash or identifier repeated across responses. Proof of work is a relation between a header and its solution and target (*Consensus Guide*, §“Equihash and the memoryless model”), which no response carries. $\square$

ZIP 307, “Block header validation”, proposes checks against a header when a compact block carries one; the abstract format does not guarantee a header, and no result of this volume uses them.

Corollary 5.4 (Fabrication of compact outputs).

- (a) *Under clause (a) of Assumption 5.2, every compact candidate accepted by Definition 4.9 is a note of the best chain: its commitment is a leaf of its pool’s note commitment tree at the position that Definition 4.11 assigns. The fields a compact record omits (proofs, signatures,*

value commitments, anchors, full ciphertexts) are checked by consensus for every transaction of the best chain, so delegating their verification to the server loses no check a light client could make.

- (b) *Without clause (a), no check authenticates the compact fields against the chain. A server that knows one of the wallet’s addresses can construct a note to it, compute its commitment and note ciphertext, choose the remaining compact fields (nf of an Action, which is p of the note), and serve the result as a compact output or Action, with tree sizes consistent with it; the result passes Definition 4.9. Unless a transaction of the chain appends the same commitment, no authentication path from it to an anchor of the chain can be found except with negligible probability, so the wallet displays a note whose every spend is rejected.*

Proof. (a) By Proposition 4.10(a) an accepted candidate opens the commitment of its compact record. By clause (a) that record is the corresponding field of a transaction of the best chain, whose commitments are appended to the pool’s tree in the order that fixes the position (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”; Lemma 4.12(a)). The omitted fields are validated by the consensus rules for every transaction of the chain (*Ironwood Guide*, §“Transaction digests and signatures” and §“Anchors”).

(b) Acceptance checks only the lead byte, the field ranges, the commitment recomputed from the plaintext and $\text{epk} = [\text{esk}] g_d$ (Definition 4.9), and an honestly constructed note to an address of the wallet satisfies all of them; the tree-size checks of Lemma 4.12 pass on sizes the server chooses consistently, by part (d) of that lemma. A spend proves a valid path from the commitment to an anchor of the chain (*Ironwood Guide*, §“Authentication paths”), and a valid path from a value that is not the leaf at its position yields a collision of the Merkle hash, found with at most negligible probability (*Ironwood Guide*, §“Anchors”, Proposition “Membership soundness”; protocol specification, §“Merkle Path Validity”, for Sapling). ZIP 307, “Output Compression”, grounds the commitment check on the same trust, in the entity that assembles transactions. $\square$

Remark 5.5 (Omission is undetected). Completeness of the served data is trusted, not verified. The size check of Lemma 4.12 detects the omission of a commitment only while the served sizes are correct. A server that omits compact outputs or Actions and serves tree sizes, frontiers, subtree roots and roots consistent with the omission, or omits or alters revealed nullifiers, of which a compact block carries no count, passes every check of Proposition 5.3. Omitted outputs hide receipts. Omitted nullifiers hide spends: the wallet shows spent notes as spendable, and a re-spend of such a note is rejected for a repeated nullifier (*Ironwood Guide*, §“Nullifier sets”) while it discloses to every party that receives it that it spends what an earlier transaction spent, the leak of repeated spends of ZIP 315, “Obtaining user consent for information leakage”. Only clause (a) of Assumption 5.2 excludes omission (ZIP 307, “Security Model”).

Remark 5.6 (Malformed responses). A response with a field outside its type (a nullifier not of 32 bytes, a non-canonical encoding of a field element, a malformed compact output or Action, a block outside the requested range) is an error of the response: the results of the query are not applied, and the query may be repeated. It is not a continuity error and triggers no rollback (“Reorganisation detection and recovery”, §7.6). This rule is designed but unspecified (Definition 1.1).

Remark 5.7 (Checks of a fetched transaction). A full transaction fetched by its identifier SHOULD be checked against the corresponding compact outputs, in addition to verification of its signatures (ZIP 307, “Block header validation”). The checks are: the identifier computed from the fetched transaction equals the queried one (*Ironwood Guide*, §“Transaction digests and signatures”, Definition “Transaction identifier”; protocol specification, §“Transaction Identifiers”); its outputs and Actions agree with the compact records; and its signatures verify over its signature digest (*Ironwood Guide*, §“Transaction digests and signatures”, Definition “Signature digest”). They bind the full transaction to the compact data and its authorisation to its effects. They bind neither to the chain: compact blocks carry too little to check an identifier against the block’s transaction tree, and that binding rests on clause (a) of Assumption 5.2.

5.2 The query interface

A wallet holds keys and no chain state (Definition 5.1), so the interface delivers what the protocols below consume: every compact output and Action of every shielded pool, for trial decryption (*Ironwood Guide*, §“Trial decryption and note acceptance”); every revealed nullifier, for spend detection (*Ironwood Guide*, §“Nullifier sets”); for each pool, tree sizes, frontiers and the roots of complete subtrees, for positions and authentication paths (*Ironwood Guide*, §“Authentication paths”); the full transactions that concern the wallet; submission of its transactions and their status; and, for transparent receivers, the transparent outputs paid to its addresses.

Definition 5.8 (Subtree root and completing height). Let $\text{pool} \in \{\text{Sapling}, \text{Orchard}, \text{Ironwood}\}$, whose note commitment tree has depth 32 and layers numbered from layer 0, the root, to layer 32, the leaves (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”; protocol specification, §“Note Commitment Trees”, for Sapling). For $0 \leq i < 2^{16}$, *subtree* i of the tree is the subtree whose leaves are the positions $[i \cdot 2^{16}, (i + 1) \cdot 2^{16})$; its *root* is the node M_i^{16} , sixteen layers above the leaves. When the tree holds n leaves, subtree i is *complete* if $n \geq (i + 1) \cdot 2^{16}$, after which its root no longer changes; the complete subtrees are those with $i < \lfloor n/2^{16} \rfloor$. The *completing height* of a complete subtree i is the height of the block whose transactions append position $(i + 1) \cdot 2^{16} - 1$.

Definition 5.9 (Transaction status). The *status* of a transaction identifier txid on the server’s best chain, with tip height T , is one of:

1. *unknown*: the server knows no transaction with identifier txid ;
2. *unmined*: the server knows the transaction, in its mempool or in a block off the best chain, and no block of the best chain contains it;
3. *mined*(b): the block of the best chain at height $b \leq T$ contains it.

The encoding of the status lies outside the abstract interface. The status is designed but unspecified (Definition 1.1).

Table 4. Queries, arguments, and responses of the light-client service (Definition 5.10). Each response is the value on the server’s best chain; $\perp$ answers an argument that names nothing on it.

Query	Arguments	Response
<i>Chain</i>		
Tip()	none	(b, hash) of the tip of the best chain; the tip input of every confirmation count (§9)
Block(b)	block identifier b	the compact block of b , with all shielded pools and transparent data
Blocks($s, e, \mathcal{P}$)	heights $s \leq e$; pool set $\mathcal{P}$	the compact blocks at heights $s, s + 1, \dots, e$ in ascending order, restricted to $\mathcal{P}$; every height of $[s, e]$ present, a block left with no transaction carrying an empty list
<i>Trees</i>		
TreeState(b)	block identifier b	height and hash of b , and for each shielded pool the size n and the frontier of its tree in the final treestate of b ; an empty frontier denotes the empty tree
SubtreeRoots(pool, i)	shielded pool pool; start index i	in index order, the pair (root, completing height) of every complete subtree of the tree of pool with index at least i
<i>Transactions</i>		
Tx(txid)	32-byte identifier	the full serialised transaction
Submit(bytes)	serialised transaction	accepted(txid) if the server’s node admits it to its mempool, else rejected
Status(txid)	32-byte identifier	the status of Definition 5.9
<i>Transparent</i>		
Utxos(A)	set A of transparent addresses	the unspent outputs of the best chain paying to A , each with outpoint, value, script and height

Definition 5.10 (Light-client query interface). The *light-client service* is the set of typed queries of Table 4, each answered from the server’s best chain under clause (a) of Assumption 5.2. A *block identifier* is a height or a 32-byte block hash. A *pool set* is a set $\mathcal{P}$ of the pool selection of “The compact block and the compact transaction” (§4.1): shielded pools, with transparent data selectable alongside. The service serves every pool set and transparent data. A query whose argument names no block of the best chain, or a transaction that the server does not know, is answered with $\perp$. Queries of the mempool and of server metadata are not part of the interface.

The frontier in a response of TreeState is that of the *Crypto Guide*, §“Append-only and incrementally updatable trees”, Definition “Frontier”: for a tree of n leaves, the left-sibling nodes along the path of the next leaf, one for each set bit of n , from which the root and every later append are computed. The empty frontier is the state before the pool’s first commitment. The final treestate of a block is that of the protocol specification, §“Transactions and Treestates” (*Ironwood Guide*, §“Anchors”, Definition “Treestate and anchor”). The serialisation of the frontier is fixed by no ZIP and is not restated. ZIP 307, “Proxy operation”, also identifies a transaction by its block and its index in the block; the interface uses the identifier alone.

Remark 5.11 (Classification of the queries). ZIP 307, “Proxy operation”, specifies the queries Tip, Block, Blocks and Tx. No ZIP specifies TreeState, SubtreeRoots, Submit, Status or the transparent query Utxos, and ZIP 307, “Client-server interaction”, leaves the acquisition of a starting tree state open: “Or, it has to set X to the block height at which Sapling activated, so as to be sent the entire commitment tree. [TODO: Decide which to specify.]” These queries, and pool selection in Blocks, are designed but unspecified (Definition 1.1).

Remark 5.12 (Minimal query set). A shielded synchronisation uses Tip, SubtreeRoots for each shielded pool, Blocks and TreeState; the full transactions that concern the wallet add Tx. A wallet that watches transparent receivers adds Utxos, whose argument names its addresses in the clear (“Key independence of queries”, §8.2). This set is the requirement of the synchronisation protocols of “Note discovery” (§6) and “Commitment-tree synchronisation” (§7), not a deployed fact.

6 Note discovery

This section states the protocol by which a wallet reconstructs its notes from the chain. It fixes the chain state and the account birthday that bound the scan, the keys an account is scanned with under the rules of ZIP 326, the note state of a wallet and the scan of one compact block that updates it, the continuity predicate and the conditions under which a scan in any order is correct, the theorems of sound and complete discovery and of recovery from the seed, and the work that scanning costs. The tree information a wallet retains for its witnesses is the subject of “Commitment-tree synchronisation” (§7).

Throughout, P ranges over the shielded pools Sapling, Orchard and Ironwood; H_S is the Sapling activation height, and $H_{\text{NU6.3}}$ the NU6.3 activation height of ZIP 258. For a compact block b , $n_P(b)$ and $S_{\text{end}}^P(b)$ are the commitment count and the end size of Definition 4.11.

6.1 The account birthday

Definition 6.1 (Chain state). For a height b , the *chain state* at b is

$$C(b) := (b, \text{hash}_b, (m_P(b), F_P(b))_{P \in \{\text{Sapling}, \text{Orchard}, \text{Ironwood}\}}),$$

where hash_b is the 32-byte hash of block b ; $m_P(b) \in [0, 2^{32}]$ is the number of leaves of the note commitment tree of pool P in the final treestate of block b ; and $F_P(b)$ is the frontier of that tree (*Crypto Guide*, §“Append-only and incrementally updatable trees”, Definition “Frontier”): for each set bit t of $m_P(b)$, the root of the completed left subtree of height t on the rightmost filled path. The empty tree has $m_P(b) = 0$ and the empty frontier, and the full tree, $m_P(b) = 2^{32}$, has its root as frontier. The three trees are distinct trees of depth 32 ($\text{MerkleDepth}^{\text{Sapling}} = \text{MerkleDepth}^{\text{Orchard}} = 32$; protocol specification, §“Constants” and §“Note Commitment Trees”): the Sapling tree of the specification, and the Orchard-pool and Ironwood-pool trees, each built with the Orchard Merkle hash (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”). The chain state $C(b)$ is the response of TreeState(b) (Table 4).

By the *Crypto Guide*’s Definition “Frontier” (§“Append-only and incrementally updatable trees”), the pair $(m_P(b), F_P(b))$ determines the root of the tree of P after block b and suffices to append every later leaf without any leaf appended before block $b + 1$. That it also yields every later authentication path is shown in “Frontiers as scan seeds” (§7.3).

Definition 6.2 (Account birthday). The *birthday* of an account is a height B , known to the wallet, that is a lower bound on the height of the first block in which the account could have received funds (ZIP 326, “Terminology”). With the birthday the wallet stores the chain state $C(B - 1)$, which summarises every note commitment of every pool before B . The scan of the account starts at block B , and no block below B is scanned for it. The birthday of a wallet is the minimum of the birthdays of its accounts.

- (i) *Added accounts*. Let an account with birthday B' be added while the greatest height of a block the wallet has scanned exceeds B' . Every block of $[B', \text{that height}]$ is then scanned again with the scanning keys of the new account (§6.2), since earlier scans did not try them; the results of the other accounts are unchanged.
- (ii) *No recorded birthday*. A wallet that restores a seed without a recorded birthday takes $B := H_S$: no shielded address of a light client can have received a relevant transaction before Sapling activation (ZIP 307, “Client-server interaction”, “Importing a pre-existing seed”). The bound concerns shielded receipts only; the transparent history of the account’s addresses may lie below it.

Remark 6.3 (Choosing a birthday). Let the tip be at height T when an account is created, and let d be at least the wallet’s assumed maximum reorganisation depth. The birthday of the new account is taken as $B \leq T - d + 1$, with stored chain state $C(B - 1)$ at height $T - d$ or below. A reorganisation that replaces at most d blocks re-mines no transaction below height $T - d + 1$; with B closer to the tip, such a reorganisation could re-mine a transaction paying the account at a height below B , which the wallet never scans, and the payment would be missed. Consensus imposes no maximum reorganisation depth (*Consensus Guide*, §“Reorg rails, maturity, and the finality floor”), so d is a wallet parameter, not a guarantee. The rollback window of 600 blocks that ZIP 218, “Block-count-based constants”, recommends concerns nodes and bounds no wallet’s choice of d . An earlier birthday is always safe for discovery and costs scanning work (§6.5).

6.2 Scanning keys

Definition 6.4 (Scanning keys of an account). Let an account a have Sapling keys and Orchard-protocol keys derived as in “Hierarchical key derivation (ZIP 32)” (§2), the latter with the flag `use_qlsk` of Construction 2.10. Scopes are $s \in \{\text{ext}, \text{int}\}$.

- (i) *Sapling*. For each scope s , the pair $(\text{ivk}_s^S, \text{nk}_s^S)$ of incoming viewing key and nullifier deriving key (ZIP 32, “Sapling internal key derivation”). Sapling outputs are scanned from B .

- (ii) *Orchard protocol*. For each scope s and each $u \in \{\text{false}, \text{true}\}$, the incoming viewing key $\text{ivk}_{s,u}$ of scope s computed with $\text{use_qsk} = u$, and the nullifier deriving key nk , the same in both branches of Construction 2.10 and in both scopes (Construction 2.14).

A receiver and its ivk are scoped to the Orchard protocol, not to a pool: the same ivk trial-decrypts Orchard-pool and Ironwood-pool note ciphertexts (ZIP 326, “Scanning for Ironwood-pool and Orchard-pool notes”); the pool of an Action is that of the bundle that carries it, and Definition 4.9 admits lead byte $0x02$ in the Orchard pool and $0x03$ in the Ironwood pool. The *use_qsk set* U_a is $\{u\}$ when the wallet knows the account’s value u and $\{\text{false}, \text{true}\}$ when it does not.

The following rules of ZIP 326, with their normative levels, fix which ivk is tried on which output.

- (R1) *Restoration*. A wallet restoring from a seed MUST take $U_a = \{\text{false}, \text{true}\}$ and scan with both values until rule (R2) narrows it, because use_qsk is not part of the recovery information (“Scanning when restoring from seed”).
- (R2) *Narrowing*. Once the wallet sees funds received in either Orchard-protocol pool under a value u , it SHOULD stop scanning with the other value, $U_a := \{u\}$. The evidence stands if the transaction that provided it is later reorganised away, since it was valid on some chain (“Scan ranges”).
- (R3) *Ironwood pool*. The Ironwood pool is scanned from $\max(B, H_{\text{NU6.3}})$ to the tip with $\text{ivk}_{s,u}$ for $u \in U_a$, hence with up to four keys ($\{\text{ext}, \text{int}\} \times \{\text{false}, \text{true}\}$) while use_qsk is unknown, subject to rule (R4). The value $u = \text{true}$ is kept for an account whose birthday precedes NU6.3 activation, since a recorded birthday may be conservatively early (“Scan ranges”).
- (R4) *First funds*. ZIP 326 describes this rule as an optimisation and does not require it (“First-funds scanning in the Ironwood pool”). While the wallet knows that the account has received no Ironwood-pool funds in any block below the one scanned, only the external keys $\text{ivk}_{\text{ext},u}$ are needed for an output, except in a transaction whose $\text{valueBalanceIronwood}$ is negative, where both scopes are used. The knowledge is kept only for the contiguous prefix of heights from $\max(B, H_{\text{NU6.3}})$ scanned without a receipt, and a reorganisation whose last unchanged block is at height h_f cuts the prefix back to $\max(h_f, H_{\text{NU6.3}})$. A compact transaction carries no value balance (Definition 4.3), so a scan of compact blocks applies the exception only where it obtains $\text{valueBalanceIronwood}$ otherwise; without it, both scopes are used.
- (R5) *Orchard pool* (“Scan ranges”). Orchard-pool note ciphertexts SHOULD NOT be scanned with $\text{use_qsk} = \text{true}$ keys. For an account with birthday after NU6.3 activation they SHOULD NOT be scanned at all. For an account with birthday before NU6.3 activation they SHOULD NOT be scanned with the external $\text{use_qsk} = \text{false}$ key after NU6.3 activation. If the account’s Orchard-pool balance is known to be zero at a block at or after NU6.3 activation, they SHOULD NOT be scanned in any block that descends from it.

(R6) *Order*. A wallet MAY scan blocks in any order (“First-funds scanning in the Ironwood pool”).

(R7) *Outgoing notes*. Outgoing notes are detected with the ovk of the same use_qsk value, which adds no key dimension (“Outgoing-note detection”).

Under (R5) the Orchard pool is scanned with use_qsk = false only: with $ivk_{\text{int},\text{false}}$ from B until the Orchard-pool balance is known to be zero after NU6.3 activation, and with $ivk_{\text{ext},\text{false}}$ from B to $H_{\text{NU6.3}} - 1$. The *scanning keys* $K_a(P, b, t)$ of account a for pool P , block b and transaction t are the triples (ivk, nk, s) that items (i) and (ii) and rules (R1) to (R5) select. Every ivk is labelled (a, s) , which attributes each accepted note to its account and scope.

Transparent receipts are detected at the transparent addresses the wallet watches, which MUST include index 0 of each account (“Deriving unified addresses”, §3.6; ZIP 316, “Deriving a Unified Address from a UIVK”). The Ironwood pool is scanned as a pool of its own: its outputs are accepted under its lead byte, its revealed nullifiers are matched only against nullifiers of Ironwood-pool notes, and its tree is its own (*Ironwood Guide*, §“Nullifier sets”, Remark “Pool locality”).

Remark 6.5 (Omitted keys and conformant senders). Rules (R2), (R4) and (R5) omit a key only where the restrictions of ZIP 326, “Wallet key-generation restrictions”, exclude an output of the account under it. Every key of an account has one use_qsk value (MUST). No wallet sends Orchard-pool funds to use_qsk = true keys, or to any external receiver after NU6.3 activation (MUST NOT). Internal receivers are exposed to no party outside the wallet (ZIP 326, “Scanning for Ironwood-pool and Orchard-pool notes”), so before the account’s first Ironwood-pool funds a note at its internal Ironwood receiver arises only from the account’s own transaction, funded by a negative valueBalanceIronwood (“First-funds scanning in the Ironwood pool”). No value enters the Orchard pool after NU6.3 activation, so a zero Orchard-pool balance stays zero (“Scan ranges”). An output created in violation of these restrictions by a non-conformant sender may be missed; this is the cost of the SHOULD NOT rules, and it is part of the statement of Theorem 6.14.

Proposition 6.6 (Spend detection requires the nullifier key). *The nullifier of an Orchard-protocol note is $\text{DeriveNullifier}_{\text{nk}}(\rho, \psi, \text{cm})$ (Ironwood Guide, §“The nullifier”, Definition “Nullifier”), and that of a Sapling note is $\text{PRF}_{\text{nk}^*}^{\text{nfSapling}}(\rho^*)$ with*

$$\rho^* = \text{repr}_J(\text{MixingPedersenHash}(\text{cm}, \text{pos})),$$

a function of the nullifier deriving key and of the note’s position (protocol specification, §“Computing ρ values and Nullifiers”).

(a) *Let an Orchard-protocol account key have use_qsk = false. No efficient algorithm that receives the incoming viewing key (dk, ivk) of one scope and ak^P , but not nk, and is then given notes of its choice with pairwise distinct ρ outputs the nullifier of one of them, except with negligible probability.*

(b) Assume that $\text{PRF}^{\text{nfSapling}}$, keyed by nk^* of a Sapling key generated as the specification prescribes, is pseudorandom to an efficient algorithm that holds the incoming viewing key and the spend validating key of that key: the specification's requirement that $\text{PRF}^{\text{nfSapling}}$ be a pseudorandom function (§“Pseudo Random Functions”), applied to this key distribution. Then the statement of (a) holds for the Sapling key.

A holder of (dk, ivk) therefore detects receipts, since trial decryption needs only ivk (Construction 4.8), and cannot recognise a revealed nullifier as that of its note. A wallet that holds only incoming viewing keys has an empty tracked-nullifier set, and the set of notes it considers unspent is a superset of the true set. Links through other public data are outside the statement (Ironwood Guide, §“Nullifier uniqueness and unlinkability”, Remark “Limits”). For $\text{use_qsk} = \text{true}$ no claim is made (Remark 2.11).

Proof. (a) Let $\mathcal{C}$ be an algorithm that outputs the nullifier of a submitted note with probability ε . It yields a distinguisher for the Ironwood Guide's Proposition “Nullifier unlinkability” (§“Nullifier uniqueness and unlinkability”): the distinguisher runs $\mathcal{C}$ on its inputs (dk, ivk) and ak^{p} , submits the notes $\mathcal{C}$ is given, and outputs 1 exactly when an answer equals the value $\mathcal{C}$ outputs for that note. In the real game it outputs 1 with probability at least ε . In the ideal game each answer is $\text{Extract}_{\text{p}}$ of a point uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, independent of the notes, except with probability $1/p_{\text{Vesta}}$; it equals a given value with probability at most $2/\#\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, since $\text{Extract}_{\text{p}}$ is at most two-to-one. Hence ε is bounded by the advantage bound of that proposition plus a negligible term, under the assumptions it names. For the internal scope the proof of that proposition applies with its hybrid H_2 changed: the pair $(\text{ivk}_{\text{int}}, \text{dk}_{\text{int}})$ is replaced by $(\text{Commit}_{\text{r}}^{\text{ivk}}(\text{ak}, \text{nk}), u_3)$ by Assumption 2.16, whose reduction, like that of H_2 , draws ak and nk itself and passes on the fourth and fifth components of its tuple. The cost is the advantage against that assumption in place of the term for the Ironwood Guide's Assumption “Joint hiding of the viewing keys” (§“Viewing keys”), and the later hybrids are unchanged, since the game after the hop is that of H_2 . That proposition concerns a key generated from a uniform spending key; for an account key the bound holds up to the loss of Lemma 2.9. (b) An algorithm that outputs $\text{PRF}_{\text{nk}^*}^{\text{nfSapling}}(\rho^*)$ for a submitted ρ^* not queried before distinguishes the function from a random function, which outputs that value with probability 2^{-256} , contradicting the assumption. $\square$

6.3 Receipt and spend detection

Definition 6.7 (Note state of a wallet). The *note state* of a wallet consists of:

- (i) per account a and pool P , the set of *detected notes*, each with its value, scope, position pos (Definition 4.11), creating transaction, its nullifier when derivable, its memo once fetched, and a *spent mark*: either unspent or the triple (height, index in block, txid) of the transaction that revealed its nullifier;
- (ii) per pool P , the *tracked set* W_P of the nullifiers of detected notes of P not marked spent, and the set R_P of *retained unlinked nullifiers*, each with its revealing transaction;
- (iii) for each wallet-relevant transaction, its mined height, or $\perp$ if it is unmined;

- (iv) the chain state (Definition 6.1) of the last block of each maximal interval of scanned heights; for a scan in increasing height, that of the last scanned block.

A transaction is *wallet-relevant* if and only if it creates a detected note or reveals the nullifier of one. A light wallet stores no copy of the chain; its note state is reconstructed from compact blocks (Definition 4.2) and fetched full transactions alone, supplied by a server that the wallet does not trust for privacy (Assumption 5.2, clause (b)). The information retained for witnesses is adjoined in “The information a wallet retains” (§7.4).

Construction 6.8 (Scanning a block). Let b be a compact block that continues the wallet’s chain, in the sense fixed in “Scan order” (§6.4), scanned for a set A of accounts. For each pool P , in block, transaction and description order, the start size of P in b (Lemma 4.12) and Definition 4.11 give each compact output or Action its position.

1. *Receipts*. For each compact Sapling output (for $P = \text{Sapling}$) and each compact Action of P (for an Orchard-protocol pool), in transaction t , and each account $a \in A$, run Construction 4.8 and Definition 4.9 under every $(\text{ivk}, \text{nk}, s) \in K_a(P, b, t)$ (Definition 6.4). Record an accepted note with its value, scope s , position and transaction. When nk is held, compute its nullifier nf (for Sapling from its position), and: if $\text{nf} \in R_P$, mark the note spent at the retained transaction and remove nf from R_P ; otherwise add nf to W_P .
2. *Spends*. For each nullifier revealed in P by b (each compact Sapling spend, and the nf of each compact Action of P): if it lies in W_P , mark the matching note spent at $(\text{height}(b), \text{index}, \text{txid})$ and remove the nullifier from W_P ; otherwise retain it in R_P with its revealing transaction.
3. *Chain state*. Compute $C(\text{height}(b))$ from the stored $C(\text{height}(b) - 1)$ and b : the hash of b , and for each pool the size $S_{\text{end}}^P(b)$ and the frontier obtained by appending the commitments of P in b , in order, to $F_P(\text{height}(b) - 1)$. It replaces $C(\text{height}(b) - 1)$ in the note state.

Step 1 precedes step 2, so a receipt and its spend in one block, or in one range scanned in increasing height, are both found in one pass. A nullifier is compared only with nullifiers of its own pool (*Ironwood Guide*, §“Nullifier sets”). The retention period of R_P is fixed by Lemma 6.12 (§6.4).

Enhancement. For each wallet-relevant transaction the wallet may fetch the full transaction by $\text{Tx}(\text{txid})$ (phase C of ZIP 307, “Client-server interaction”), which yields the memo, by decryption of the full C_{enc} , and the outgoing plaintexts recoverable under the account’s ovk , at the privacy cost stated in “Leakage of transaction queries” (§8.4).

Class (Definition 1.1). Trial decryption, nullifier matching in increasing height and enhancement are specified (ZIP 307, “Local processing”, “Detecting spends” and “Client-server interaction”); the scanning keys are those of ZIP 326. The retained set R_P and the chain state of step 3 are designed but unspecified.

6.4 Scan order

Remark 6.9 (The interaction of ZIP 307). ZIP 307, “Client-server interaction”, divides the interaction between client and server into four phases.

- (A) *First start.* The client obtains the tip height X and the commitment tree state at X , or sets $X := H_5$; the ZIP leaves the choice open. For an imported seed $X := H_5$ (“Importing a pre-existing seed”).
- (B) *First update.* The client obtains the tip Y and the compact blocks $X, \dots, Y$. It checks that the hash of block X equals its stored copy, a mismatch meaning that X was orphaned by a reorganisation, and discards block X without further processing. For each later block it validates the header if one is present, trial-decrypts for its notes, creates and updates witnesses of detected notes, and scans for their nullifiers.
- (C) *Transactions.* At user direction, the client fetches full transactions.
- (D) *Later updates.* The client obtains the tip Z and the blocks $Y, \dots, Z$, rechecks block Y as in (B), updates witnesses, marks the notes whose nullifiers appear as spent at that height, and scans for new notes as in (B).

In the queries of Table 4 the phases use Tip, TreeState or Block, Blocks and Tx.

Scanning a range $[b, b']$ requires the chain state $C(b - 1)$: its hash for the continuity check of block b , and each pool’s size and frontier for positions and authentication paths. The wallet holds it after scanning block $b - 1$, or obtains it by $\text{TreeState}(b - 1)$; a server-supplied frontier is accepted under clause (a) of Assumption 5.2 and is checked against no header (Proposition 5.3).

Definition 6.10 (Continuity predicate). Let the wallet hold the chain state $C(b) = (b, \text{hash}_b, (m_p(b), F_p(b))_p)$. A compact block b *continues the wallet’s chain at b* if and only if

- (i) $\text{height}(b) = b + 1$;
- (ii) $\text{prevHash}(b) = \text{hash}_b$; and
- (iii) for every pool P , the start size of P in b , $S_{\text{end}}^P(b) - n_P(b)$ (Lemma 4.12), equals $m_P(b)$.

Within a range each block is checked against the chain state computed from its predecessor, and the first block against the stored chain state. When a range is scanned directly below an already scanned block $b' + 1$, that block is checked again against the chain state $C(b')$ that the new range computes. This extends ZIP 307’s single hash comparison at the start of each phase to every block and to both sides of every range boundary.

Rule (verify first). The scan results of a block are applied to the note state only after the block is checked to continue the wallet’s chain, and the results of a range only after each of its blocks is. Failure of the predicate is a *continuity error*, handled by rollback (“Reorganisation detection and recovery”, §7.6). A malformed or insufficient field is an error of the response, not a continuity error (“Architecture and authority”, §5.1). The predicate beyond ZIP 307’s comparison is designed but unspecified (Definition 1.1).

Definition 6.11 (Scanned heights). For a wallet with birthday B , the *maximum scanned height* is the greatest height of a scanned block, and the *fully scanned height* f is the greatest $h \geq B - 1$ such that every block of $[B, h]$ has been scanned. Under out-of-order scanning f may lie below the maximum scanned height.

Lemma 6.12 (Out-of-order spend detection). *Fix a chain. Let a note of an account in pool P be created at height h_n and its nullifier revealed at height $h_s \geq h_n$, and let the wallet hold the note's nullifier deriving key. If each nullifier retained in R_p (Construction 6.8) that was revealed at height h is kept until the fully scanned height is at least h , the note is marked spent at the revealing transaction whichever of the blocks h_n and h_s is scanned first. The retention bound is the least that guarantees this: if a nullifier revealed at h is discarded while $f < h$, some block below h is unscanned, and a note of the account created there and spent at h is never marked spent.*

Proof. If block h_n is scanned no later than block h_s , the note's nullifier is in W_p when block h_s is scanned (steps 1 and 2 of Construction 6.8, in that order when $h_n = h_s$), and it is matched on arrival. Otherwise block h_s is scanned first; the nullifier matches no tracked note and is retained in R_p . Since $B \leq h_n \leq h_s$ (Definition 6.2), f reaches h_s only after block h_n is scanned, so the nullifier is still in R_p when step 1 finds the note, and the note is marked spent. Minimality: block h has been scanned, since it revealed the nullifier, so $f < h$ means that some $h' \in [B, h - 1]$ is unscanned; a note created at h' and spent at h has then lost the only record of its spend. $\square$

Lemma 6.13 (Order independence of scanning). *Fix a chain and a range $[B, T]$. Let a schedule scan every block of $[B, T]$ once, in any order and in ranges of any lengths, each block b with key sets $K_a(P, b, t)$, and let*

- (a) *the results of each range be applied only after continuity is verified (Definition 6.10), and*
- (b) *unlinked nullifiers be retained as Lemma 6.12 requires.*

Then the resulting notes, positions, nullifiers and spent marks equal those of a scan of $[B, T]$ in increasing height that applies to each block the same key sets, and the chain state held for height T is $C(T)$. The key sets of Definition 6.4 depend on the order only through rules (R2), (R4) and the last clause of (R5), whose omitted keys accept no output of the account under the restrictions of ZIP 326; under those restrictions every order of Definition 6.4 yields the note state of the scan in increasing height. The witness half of order independence is stated in "Witness readiness" (§7.5).

Proof. Receipt detection of an output is local to its block: it depends only on the compact output and the key set (Construction 4.8, Definition 4.9). Positions are local: by Lemma 4.12 the start size of each pool in b follows from the compact block b alone, and (a) makes it agree with the end size of the predecessor. Nullifiers are functions of the note, the key and, for Sapling, the position. Spent marks follow from Lemma 6.12 in either order. The chain state at T is computed by step 3 of Construction 6.8 from the chain state below the last range that ends at T , which (a) makes the true one, by induction over the ranges. For the last clause, a key omitted by (R2), (R4) or the last clause of (R5) in one order and tried in another accepts no output of the account (the Remark on

omitted keys above; for (R5), no value enters the Orchard pool after NU6.3 activation, so a zero Orchard-pool balance stays zero), so both orders accept the same outputs. $\square$

Theorem 6.14 (Soundness and completeness of note discovery). *Assume clause (a) of Assumption 5.2, and let the server’s best chain be unchanged while an account a with birthday B is scanned by Construction 6.8 with the scanning keys of Definition 6.4, under conditions (a) and (b) of Lemma 6.13. Let f be the fully scanned height. For every $h \leq f$, restricted to notes created at heights at most h and to spent marks set by nullifiers revealed at heights at most h :*

- (i) *the note state holds a note of a in pool P at position p if and only if the best chain has, at or below h , an output of P at position p that Definition 4.9 accepts under a key of $K_a(P, b, t)$ for its block b and transaction t ;*
- (ii) *when the wallet holds the note’s nullifier deriving key, the note is marked spent if and only if its nullifier is revealed in pool P of the best chain at or below h , and, except with negligible probability, a revealed nullifier equal to it marks a spend of that note and of no other.*

Completeness is over accepted outputs under the keys that Definition 6.4 applies, not over all outputs addressed to the account: an output that its sender encrypted malformed, one that acceptance rejects, and one that a non-conformant sender created under a key omitted by the rules of ZIP 326 are excluded. Ranges scanned out of order above f add notes and spent marks above h , which the statement does not cover. A holder of incoming viewing keys alone obtains (i) only (Proposition 6.6). The assumptions of (ii) are those named in the results cited in the proof.

Proof. By clause (a) the scanned compact blocks are those of the best chain, and every output of each is trial-decrypting under the applicable keys (Construction 6.8). Soundness of acceptance is Proposition 4.10, and membership of the accepted note in the best chain is Corollary 5.4(a). Positions are exact by Lemma 4.12(a) and Definition 6.10. The result does not depend on the order, by Lemma 6.13; this gives (i).

For (ii), spent status is nullifier matching within P , with spends found in either order by Lemma 6.12. For Orchard-protocol notes, one nullifier per note and distinct nullifiers for distinct notes are the *Ironwood Guide*’s Lemma “One nullifier per note” (§“Double-spend resistance”) and Proposition “Nullifier uniqueness” (§“Nullifier uniqueness and unlinkability”), under the assumptions they name. For Sapling notes they are the Spend statement, which fixes the nullifier of the consumed note (protocol specification, §“Spend Statement (Sapling)”), and the requirement that $\text{PRF}^{\text{nfSapling}}$ be collision-resistant across all keys (§“Pseudo Random Functions”), with ρ^* distinct for distinct positions (§“Computing ρ values and Nullifiers”). Without clause (a), Remark 5.5 defeats completeness and Corollary 5.4(b) defeats soundness. $\square$

Theorem 6.15 (Recovery from the seed). *Let an account have $\text{use_qsk} = \text{false}$ and a valid account key (“Valid account keys”, §2.6), so that its keys are determined by the seed S and the account index (Proposition 2.12). Under clause (a) of Assumption 5.2, scanning the best chain from a lower bound on the account’s birthday, or from H_s when none is recorded (Definition 6.2), with the scanning keys of Definition 6.4, in particular with $U_a = \{\text{false}, \text{true}\}$ by (R1) until (R2) narrows it, reconstructs the note state of the account up to the fully scanned height, as characterised by*

Theorem 6.14. Memos, and the outgoing plaintexts of outputs that the account encrypted under its ovk, are recovered once the wallet-relevant transactions are fetched. Transparent funds are recovered at the addresses the wallet watches: index 0 of each account by obligation (§3.6); how far beyond index 0 a wallet looks is fixed by no ZIP and is designed but unspecified (Definition 1.1). Data that never reached the chain, such as the memos of transactions never mined, is not recoverable. An account with use_qsk = true, whose keys S does not determine, is outside the theorem, although Definition 6.4 scans with both values when restoring.

Proof. Proposition 2.12 gives the keys, hence the scanning keys. Definition 6.2 makes the scan start at or below the first receipt. Theorem 6.14 gives the note state. Memos and outgoing plaintexts are functions of the fetched full transactions and the keys, and under clause (a) Tx returns the transactions of the best chain. $\square$

Remark 6.16 (A reference interaction). The following interaction of a wallet that synchronises by ranges out of order is designed but unspecified (Definition 1.1). Per synchronisation the wallet runs:

1. SubtreeRoots(P , 0) for each pool, storing the roots of the complete subtrees;
2. Tip();
3. selection of ranges: those near the tip, and those completing the subtrees of found notes (§7.5), before historic ranges;
4. for each range $[s, e]$: TreeState($s - 1$) unless $C(s - 1)$ is held, then Blocks(s, e, P), the continuity check and the scan;
5. a return to step 2 when a scan changes the selection: a newly found note, or a rollback after a continuity error.

The order is a wallet policy fixed by no ZIP, and it is observable by the server (“Leakage of range queries”, §8.3).

6.5 The cost of scanning

Trial decryption costs one key agreement per compact output or Action per candidate ivk, for every block from the birthday to the tip, independently of the wallet’s own activity. Compaction (Definition 4.5) reduces the bandwidth, not the number of trials. Restoring an account repeats this work from its birthday. Retrieval of full transactions and the maintenance of authentication paths scale with the wallet’s own notes, the latter linearly in its unspent notes.

Proposition 6.17 (Scanning work per target day). *At NU7’s target spacing of 25 seconds (ZIP 218, “Block target spacing”; Consensus Guide, §“Zcash time: 25-second blocks”), a target day of 86,400 seconds holds 3456 blocks. A block holds at most 330 Orchard-pool Actions (ZIP 218, “Shielded pool action limits”), so a target day holds at most 1,140,480 compact Orchard-pool Actions, with at most 168,791,040 bytes of their compact payload at 148 bytes each (Definition 4.5), and requires at most 1,140,480 trial decryptions of Orchard-pool Actions per candidate ivk; the*

number of candidate keys of an account is fixed by Definition 6.4. The same bounds hold for Orchard-pool Actions, Sapling spends and Sapling outputs together, since the shared budget of the same section limits their sum to 330 per block, and a compact Sapling output (116 bytes) or spend (32 bytes) is smaller than a compact Action. The figures count target blocks: the number of blocks mined in a day varies, and the target spacing is not a rate limit.

The bound covers no Ironwood-pool Action: ZIP 218 counts none against any limit, so the specified text bounds neither their number nor the trial decryptions they cost. A proposed amendment would charge them to the shared budget of 330 units, under which the same figures would bound the Orchard-protocol total (Consensus Guide, §“A block budget for shielded work”). The Ironwood term is an open problem (Definition 1.1).

Proof. The counts are $86,400/25 = 3456$, $330 \cdot 3456 = 1,140,480$ and $148 \cdot 1,140,480 = 168,791,040$. The per-block limits are $\text{OrchardBlockActionLimit} = 330$ and $\text{GlobalShieldedBudget} = 330$ of ZIP 218, “Shielded pool action limits”. Each compact Action, and each compact Sapling output, costs one trial decryption per candidate key (Construction 4.8); a compact Sapling spend costs none. $\square$

7 Commitment-tree synchronisation

This section states the tree information that a wallet retains in order to spend its notes. It shows that the witness maintenance of ZIP 307 forces a scan from each note to its anchor; decomposes each note commitment tree at level 16 into subtrees and a cap built from downloaded subtree roots; shows that a frontier seeds every later witness; defines the information a wallet retains and proves it sufficient; states exactly when a witness is computable; and states the detection of reorganisations and the recovery from them, with the bound on the rollback height, the safety theorem and the correctness theorem of synchronisation.

Throughout, P ranges over the shielded pools Sapling, Orchard and Ironwood, with the chain state $C(b)$ of Definition 6.1, the birthday B of Definition 6.2, and the commitment count $n_P(b)$ and end size $S_{\text{end}}^P(b)$ of a compact block b (Definition 4.11). Assumption 5.2 is the server assumption; only its clause (a) is used here.

7.1 Witness maintenance by linear scanning

Remark 7.1 (What a spend needs). A detected note is not yet spendable. A spend needs, besides the note, its position p and the authentication path of p to the anchor of a block at or after the note’s block (*Ironwood Guide*, §“Authentication paths”, Construction “Authentication path”; anchors as in §“Anchors”, Definition “Treestate and anchor”). By the *Ironwood Guide*’s Lemma “Authentication paths from public data” (§“Authentication paths”), that path is a function of p and of the first n leaves of the pool’s tree, n being the tree size at the anchor. Path construction and anchor selection are cited, not re-derived; the anchor height is fixed in “Anchors, confirmations, and expiry” (§9). The lemma is stated for the trees of the Orchard protocol; its proof uses only the recurrence of the tree and applies unchanged to the Sapling tree, which has the same depth 32 and the same layer convention (protocol specification, §“Note Commitment Trees” and §“Merkle Path Validity”). Everything below applies to each pool P separately.

An *incremental witness* of a note at position p , after $m > p$ leaves have been appended to the pool's tree, is the information from which the authentication path of p in the tree of size m is computed: the position p , the siblings of the path whose ranges lie below m , and the frontier of the leaves appended after p (ZIP 307, “Local processing”, “Creating and updating note witnesses”; *Crypto Guide*, §“Append-only and incrementally updatable trees”, Definition “Frontier”).

Construction 7.2 (Linear witness maintenance). ZIP 307, “Local processing”, “Creating and updating note witnesses”, states the protocol for the Sapling pool and its commitments cmu ; it applies to each pool with that pool's commitments, cmx for Actions. Compact blocks are processed in increasing height, with the order of transactions and outputs preserved. The wallet holds its copy of the pool's tree as a frontier and one incremental witness per tracked note. For each note commitment c of the block, in order:

- (i) append c to the tree;
- (ii) append c to every held witness;
- (iii) if the output of c is accepted by Definition 4.9, create a witness of it from the current tree.

A witness updated through block X yields the authentication path of its note to the anchor of block X . No ZIP specifies any other tree-maintenance protocol.

Lemma 7.3 (Linear maintenance consumes every commitment). *Under linear witness maintenance, let a note be created in block h_n and let $a \geq h_n$. The wallet holds the authentication path of the note to the anchor of block a only after it has processed, in order, every note commitment of the pool in every block from h_n through a . It creates the note's witness only after it holds the tree state at the end of block $h_n - 1$, obtained either by processing every earlier block or from a chain state (Definition 6.1). In particular, a wallet restored from a seed spends no note created at h_n before it has scanned forward from h_n to the anchor.*

Proof. By induction on the processed commitments. Steps (i) and (ii) are the only operations on the tree and the witnesses, and each consumes the next commitment in chain order; step (iii) creates a witness from the current tree, which after the commitment of the note is the tree state at the end of block $h_n - 1$ followed by the commitments of h_n up to that of the note. A witness therefore reflects block X only after every commitment through X has passed through step (ii), and it yields no path to the anchor of a before it reflects block a . $\square$

Remark 7.4 (The retained-information method). The method of “Subtree roots and the cap” (§7.2) to “Witness readiness” (§7.5) removes the constraint of Lemma 7.3. It replaces per-note witnesses by subtree roots, obtained through the query `SubtreeRoots` (Definition 5.10), and a frontier, so that the path of a note needs the commitments of its own subtree only. The method is designed but unspecified (Definition 1.1): no ZIP specifies it or the subtree-root query.

7.2 Subtree roots and the cap

Definition 7.5 (Subtrees and the cap). Fix a pool P and its note commitment tree of depth 32, with nodes M_i^b in layer b counted from the root (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”; protocol specification, §“Merkle Path Validity”). Let MerkleCRH_P be its Merkle hash, $\text{MerkleCRH}^{\text{Sapling}}$ for Sapling and $\text{MerkleCRH}^{\text{Orchard}}$ for either Orchard-protocol pool, and Uncommitted_P its unused-leaf value, $\text{Uncommitted}^{\text{Sapling}}$ or $\text{Uncommitted}^{\text{Orchard}} = 2$ (protocol specification, §“Constants”). For $0 \leq l \leq 32$ and $0 \leq i < 2^{32-l}$, the node at level l and index i is

$$\text{node}(l, i) := M_i^{32-l},$$

the root over the positions $[i \cdot 2^l, (i+1) \cdot 2^l]$; its *range* is that interval. The nodes satisfy the first line below, and the second defines the values $\text{Empty}_{P,l}$:

$$\text{node}(l+1, i) = \text{MerkleCRH}_P(31-l, \text{node}(l, 2i), \text{node}(l, 2i+1)),$$

$$\text{Empty}_{P,0} := \text{Uncommitted}_P, \quad \text{Empty}_{P,l+1} := \text{MerkleCRH}_P(31-l, \text{Empty}_{P,l}, \text{Empty}_{P,l})$$

for $0 \leq l < 32$. Each $\text{Empty}_{P,l}$ is the root over a range of unused leaves (*Ironwood Guide*, §“Authentication paths”, Lemma “Authentication paths from public data”, for Orchard-protocol trees; the same recurrence for Sapling). *Subtree i* is the node $\text{node}(16, i)$ with its range, and its completeness, root and completing height are those of Definition 5.8. There are $2^{32}/2^{16} = 2^{16} = 65536$ subtrees, with indices 0 to $2^{16} - 1$, each over 65536 positions. The subtree of a position p is $\sigma(p) := \lfloor p/2^{16} \rfloor$. The *cap* is the tree formed by the nodes $\text{node}(l, i)$ with $16 \leq l \leq 32$; its leaves are the 2^{16} level-16 nodes, and its root is the root $\text{node}(32, 0)$ of the tree. The same geometry holds for the Sapling, Orchard-pool and Ironwood-pool trees.

Construction 7.6 (The cap over level-16 values). For a pool P and a list $r_0, \dots, r_{2^{16}-1}$ of level-16 values, the *cap over (r_i)* is the append-only Merkle tree of depth 16 (*Crypto Guide*, §“Append-only and incrementally updatable trees”, Definition “Append-only Merkle tree”) whose cap-level-0 labels are the r_i , whose empty-leaf value is $\perp_0 := \text{Empty}_{P,16}$, and whose combination of two cap-level- u labels into a cap-level- $(u+1)$ label is

$$\text{NodeHash}_{u+1} := \text{MerkleCRH}_P(15-u, \cdot, \cdot) \quad (0 \leq u < 16),$$

the tree’s Merkle hash at level $u+16$, that is, in layer $31-(u+16)$. Complete subtree roots obtained from $\text{SubtreeRoots}(P, i)$ enter at cap positions $i, i+1, \dots$ in index order.

Lemma 7.7 (Cap root equals the tree root). *Let the tree of P hold n leaves, and for each i let $r_i := \text{node}(16, i)$ of that tree: the complete root when $(i+1) \cdot 2^{16} \leq n$; the root over the appended leaves of subtree i padded with Uncommitted_P when $i \cdot 2^{16} < n < (i+1) \cdot 2^{16}$; and $\text{Empty}_{P,16}$ when $i \cdot 2^{16} \geq n$. Then the cap-level- u node of index i of the cap over (r_i) equals $\text{node}(16+u, i)$ for $0 \leq u \leq 16$. In particular the root of the cap equals $\text{node}(32, 0)$, the anchor of the pool for that tree (*Ironwood Guide*, §“Anchors”, Definition “Treestate and anchor”).*

Proof. Induction on u . For $u = 0$ the claim is the definition of r_i . For the step, the cap combines

the cap-level- u nodes of indices $2i$ and $2i + 1$ with $\text{MerkleCRH}_p(15 - u, \cdot, \cdot)$; by the induction hypothesis they are $\text{node}(16 + u, 2i)$ and $\text{node}(16 + u, 2i + 1)$, and the recurrence of the tree gives $\text{node}(17 + u, i) = \text{MerkleCRH}_p(31 - (16 + u), \dots)$, in the same layer $15 - u$. The value $r_i = \text{Empty}_{p,16}$ for an unused subtree follows from the leaf rule, by which unused positions hold Uncommitted_p (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”, item “Leaves”), and from the induction in the proof of the *Ironwood Guide*’s Lemma “Authentication paths from public data” (§“Authentication paths”). The empty labels of the cap, $\perp_{u+1} := \text{NodeHash}_{u+1}(\perp_u, \perp_u)$, are then $\text{Empty}_{p,16+u}$ by the same induction, so the cap’s labels of empty ranges agree with the tree’s. $\square$

Lemma 7.8 (Path decomposition at level 16). *Let the tree hold n leaves, let $p < n$ and $\sigma := \sigma(p)$, and let $(s_0, \dots, s_{31})$ be the authentication path of p (*Ironwood Guide*, §“Authentication paths”, Construction “Authentication path”), $s_t = \text{node}(t, j_t \oplus 1)$ with $j_t := \lfloor p/2^t \rfloor$.*

- (i) *For $t < 16$, the range of s_t lies inside subtree σ , and s_t is a function of the leaves of subtree σ below n ; it is $\text{Empty}_{p,t}$ if its range starts at or beyond n .*
- (ii) *For $t \geq 16$, s_t is the cap node of cap level $t - 16$ and index $j_t \oplus 1$, a function of the level-16 nodes r_i with i in its range and $i \neq \sigma$. These are complete subtree roots; the root over the incomplete subtree $\lfloor (n - 1)/2^{16} \rfloor$ when n is not a multiple of 2^{16} , a function of the frontier of the tree after its n -th leaf; and $\text{Empty}_{p,16}$.*

Hence the leaves of subtree σ , the roots of the complete subtrees other than σ and the frontier of the tree of size n suffice for the path, and the wallet needs the commitments of no subtree other than σ .

Proof. The range of s_t is $[(j_t \oplus 1)2^t, ((j_t \oplus 1) + 1)2^t)$, and s_t is a function of the leaves in its range (*Ironwood Guide*, §“Authentication paths”, Lemma “Authentication paths from public data”, whose proof shows this for every node). For $t < 16$ the range lies inside $[\sigma 2^{16}, (\sigma + 1)2^{16})$, since j_t and $j_t \oplus 1$ have the same quotient by 2^{16-t} , which is σ . For $t \geq 16$ the range is a union of whole subtree ranges, and it excludes σ because $j_t \oplus 1 \neq j_t$ and $\lfloor \sigma/2^{t-16} \rfloor = j_t$; Lemma 7.7 computes s_t from the level-16 nodes of its range. The root of the incomplete subtree is the fold, with the $\text{Empty}_{p,}$ values as right siblings, of the carries below level 16 of the frontier of the tree of size n , one for each set bit of $n \bmod 2^{16}$ (*Crypto Guide*, §“Append-only and incrementally updatable trees”, Proposition “Incremental append in logarithmic time”, proof). $\square$

Construction 7.9 (Acquisition of subtree roots). At each synchronisation the wallet queries $\text{SubtreeRoots}(P, 0)$ for each pool $P \in \{\text{Sapling}, \text{Orchard}, \text{Ironwood}\}$ (Table 4) and records, for each complete subtree i , the pair (root, completing height) of Definition 5.8; the roots enter the cap at index i (Construction 7.6). The roots are trusted under clause (a) of Assumption 5.2: no root is checked against a header, and a root is verified only if the wallet also computes it from scanned commitments (Proposition 5.3, item (iv)). The acquisition is designed but unspecified (Definition 1.1).

7.3 Frontiers as scan seeds

Lemma 7.10 (A frontier suffices to continue the tree). *Let F be the frontier of the tree of pool P after block $b - 1$ (Crypto Guide, §“Append-only and incrementally updatable trees”, Definition “Frontier”), holding m leaves, with carries f_t for the set bits t of m , and let $c_m, c_{m+1}, \dots$ be the note commitments of P in blocks $b, b + 1, \dots$ in chain order. From F and these commitments alone the wallet computes*

- (i) *the root of the tree after every block $b \geq b - 1$, and*
- (ii) *for every position $p \geq m$ appended by a block at most b , the authentication path of p to that root.*

The commitments at positions below m are not needed. Consequently the chain state $C(B - 1)$ stored with the birthday B of an account (Definitions 6.1 and 6.2) seeds every later witness of the account.

Proof. (i) Repeated application of the *Crypto Guide*’s Proposition “Incremental append in logarithmic time” (§“Append-only and incrementally updatable trees”), each append producing the next frontier and root.

(ii) By the *Ironwood Guide*’s Lemma “Authentication paths from public data” (§“Authentication paths”), a right sibling of p ($b_t = 0$) has a range above $p \geq m$, hence over later leaves or unused positions, and is computed from the c_q with Empty_{P_t} values. A left sibling ($b_t = 1$) has the range $[(j_t - 1) 2^t, j_t 2^t)$. If the range starts at or above m , it is computed from later leaves. If it lies below m , then $j_t 2^t \leq m$ and $j_t = \lfloor p/2^t \rfloor \geq \lfloor m/2^t \rfloor$ give $j_t = \lfloor m/2^t \rfloor$, which is odd since $b_t = 1$; so bit t of m is set and the range is the block of the frontier at level t , whose root is the carry f_t . If the range contains m , its part below m is the union of the frontier’s blocks at the set bits of m below t , since the range is aligned at level t ; its root is the fold of those carries with the later leaves of the range. $\square$

Definition 7.11 (Seeded batch). A batch of consecutive blocks $b_0, \dots, b_k$, of heights increasing by one, is *seeded* with the chain state $C(h_s)$ of the block preceding b_0 (Definition 6.1), with frontiers $F_P(h_s)$ of sizes $m_P(h_s)$. Before any retained information is modified, the wallet accepts the batch only if b_0 continues the seed in the sense of Definition 6.10: $\text{height}(b_0) = h_s + 1$, $\text{prevHash}(b_0) = \text{hash}_{h_s}$, and $m_P(h_s) + n_P(b_0) = S_{\text{end}}^P(b_0)$ for each pool P (Definition 4.2, Lemma 4.12); each later block is checked against its predecessor in the same way. A violation is a continuity error (Definition 6.10), handled in “Reorganisation detection and recovery” (§7.6). Under clause (a) of Assumption 5.2, the root of each seed frontier $F_P(h_s)$ equals the anchor of block b_s in P ; nothing else checks it, since the abstract format does not guarantee a header (Proposition 5.3). The per-block sizes are designed but unspecified (Definition 4.2, Remark 4.1).

7.4 The information a wallet retains

Definition 7.12 (Retained tree information). The *retained tree information* of a wallet is adjoined to its note state (Definition 6.7). It comprises a finite set $\mathcal{H}$ of heights, the *retained block boundaries*, chosen by the wallet and the same for every pool, which contains $B - 1$, and for each pool P :

- (a) for each unspent note of the wallet in P , its position p and its leaf, and each sibling s_t with $t < 16$ of the path of p (Lemma 7.8(i)) from the moment its range lies wholly below the tree size; such a node is a function of appended leaves and does not change thereafter (*Ironwood Guide*, §“Authentication paths”, Lemma “Authentication paths from public data”, items (i) and (ii) and proof);
- (b) the complete subtree roots with their completing heights (Definition 5.8), and the frontier of the tree after the last scanned block;
- (c) for each $r \in \mathcal{H}$, the chain state $C(r)$, whose frontiers are the tree states after block r : the anchors of the blocks in $\mathcal{H}$ are the wallet’s candidate anchors, and the heights in $\mathcal{H}$ are its rollback points.

The *rollback window* is the span of $\mathcal{H} \setminus \{B - 1\}$. Because $\mathcal{H}$ is common to the pools, a rollback restores all trees to one block. Any other node may be discarded once no retained path needs it. An anchor older than the window is retained only by explicit choice: ZIP 318, “Anchor-height bucketing and cohorts”, requires the anchor of each migration transaction to be the tree state at a shared network-wide boundary height, so a wallet that follows it retains those boundaries; any other retention of old anchors is wallet policy. The definition fixes information, not a representation, and is designed but unspecified (Definition 1.1).

Proposition 7.13 (Sufficiency of the retained information). *Fix a pool P .*

- (i) *Let p be the position of a retained note, $\sigma := \sigma(p)$, and $A \in \mathcal{H}$ a retained block boundary with tree size $n > p$. If the blocks that hold the commitments of subtree σ below n and at or above the birthday seed are scanned, the authentication path of p to the anchor of A is computable from the retained information.*
- (ii) *For every height r at which the wallet holds a chain state $C(r)$ for all pools, with $n_r(P) := m_P(r)$ the size of the tree of P after r , discard every retained node over a range that meets $[n_r(P), 2^{32})$, every note position at or above $n_r(P)$, every subtree root with completing height above r and every $C(r')$ with $r' > r$, and take the frontier of $C(r)$ as the frontier after the last scanned block. The result is a function of the blocks at heights at most r , and it is information that a wallet which scanned only those blocks retains.*

Proof. (i) By Lemma 7.8, each sibling s_t is of one of the following sorts. A sibling whose range lies below n is the same node at every later tree state: for $t < 16$ it is in item (a), or, where its range lies below the birthday seed, it is a carry of the seed frontier (Lemma 7.10); for $t \geq 16$ it is a cap node

over complete subtree roots of item (b) (Lemma 7.7). The sibling ranges are pairwise disjoint, so at most one contains $n - 1$ without lying below n . That sibling is the root over its range, whose part below n is the union of the blocks of the frontier of $C(A)$ at the set bits of n below its level; it is the fold of those carries with $\text{Empty}_{P,}$ values, and the frontier of $C(A)$ is in item (c). A sibling whose range starts at or beyond n is $\text{Empty}_{P,}$.

(ii) After the discards, every retained node lies over a range below $n_r(P)$ and is a function of the leaves appended through block r (*Ironwood Guide*, §“Authentication paths”, Lemma “Authentication paths from public data”); every retained subtree root has completing height at most r , and is likewise such a function; the retained chain states are those at heights at most r ; and the frontier after r is that of $C(r)$, which the wallet holds for every pool at the same r . A wallet that scanned only the blocks at most r can hold each of these items, and item (a) is satisfied, since a sibling whose range is not below $n_r(P)$ is not yet due. $\square$

7.5 Witness readiness

Theorem 7.14 (Witness from retained information). *Let A be a retained block boundary with tree size n in pool P , let p be a position, $\sigma := \sigma(p)$, and let $(s_0, \dots, s_{31})$ be the authentication path of p to the anchor of A . If $p \geq n$, the Ironwood Guide’s Construction “Authentication path” (§“Authentication paths”) defines no path, and by its Proposition “Membership soundness” (§“Anchors”), item (iii), no valid path to that anchor with position p exists, except with negligible probability, for the Orchard-protocol pools; for Sapling, by the same argument under the collision resistance of PedersenHash that the protocol specification requires of $\text{MerkleCRH}^{\text{Sapling}}$ (§“MerkleCRH^{Sapling} Hash Function”), since $\text{Uncommitted}^{\text{Sapling}}$ is not in the range of $\text{NoteCommit}^{\text{Sapling}}$ (§“Windowed Pedersen commitments”). Otherwise the path is computable from the wallet’s information if every s_t is of one of the following kinds:*

- (a) $t < 16$, and s_t is computable from the commitments of subtree σ below n that the wallet has scanned or holds summarised in a carry, whose range lies below n , of a seed frontier (Lemma 7.10) or in the frontier of $C(A)$;
- (b) $t = 16$, and s_t is a complete subtree root that the wallet holds;
- (c) $t \geq 16$, and s_t is a cap node computed from held complete roots, carries at levels at least 16 of the frontier of $C(A)$ or, with range below n , of a seed frontier, $\text{Empty}_{P,16}$ and, where its range meets the incomplete subtree, the root over that subtree’s leaves below n , computed from the frontier of $C(A)$;
- (d) the range of s_t starts at or beyond n , and $s_t = \text{Empty}_{P,}$;
- (e) $t \geq 16$, and s_t is a carry of the frontier of $C(A)$ or a carry, whose range lies below n , of a seed frontier.

Proof. Kinds (a) to (d) are the sorts of sibling in the proof of Proposition 7.13(i), extended in kind (c) by frontier carries, and a sibling of kind (e) is a retained node. $\square$

Example 7.15 (Kinds of the siblings of one path). Let $p = 65537$, so $\sigma = 1$, and let the anchor's tree hold $n = 471098 = 7 \cdot 65536 + 12346$ leaves, so that subtrees 0 to 6 are complete and subtree 7 is incomplete. The siblings $s_0, \dots, s_{15}$ are of kind (a); s_{16} , the root of subtree 0, is of kind (b); s_{17} , over subtrees 2 and 3, and s_{18} , over subtrees 4 to 7, which meets the incomplete subtree, are of kind (c); and $s_{19}, \dots, s_{31}$, whose ranges start at $8 \cdot 65536$ or beyond, are of kind (d).

Definition 7.16 (Block extent of a subtree). For pool P and subtree s , let $c_p(s)$ be the completing height of subtree s (Definition 5.8), and let $c_p(-1)$ be the activation height of P . The *block extent* of subtree s is the interval of heights $[\max(c_p(s-1), B), c_p(s)]$, both ends included, the part below B being summarised by the chain state $C(B-1)$ (Lemma 7.10); for the incomplete last subtree the interval ends at the anchor height. After detecting a note at position p , the wallet scans the extent of subtree $\sigma(p)$ beyond the ranges it has already scanned. The incomplete subtree of every pool is covered by the range from $\max(B, h_c)$ to the tip, where h_c is the least, over the pools, of the completing height of the pool's last complete subtree, or of the pool's activation height if it has none.

The start of an extent is inclusive because the completing block of subtree $s-1$ may also hold the first commitments of subtree s ; the positions within that block are fixed by Lemma 4.12.

Proposition 7.17 (Readiness of a witness). *Let a note of pool P lie at position p , $\sigma := \sigma(p)$, and let the anchor block A , of height a , have tree size $n > p$ and $\kappa := \lfloor n/2^{16} \rfloor$ complete subtrees. The witness of p at A is computable if*

- (1) *the blocks of subtree σ are scanned from the start of its extent (Definition 7.16) up to a , or through its completing height if $\sigma < \kappa$;*
- (2) *the wallet holds the root of every complete subtree $i < \kappa$ with $i \neq \sigma$, including those after σ ; and*
- (3) *the frontier of $C(A)$ is retained, which scanning the extent of subtree κ through a yields when n is not a multiple of 2^{16} .*

Condition (2) can be weakened through frontier carries: every complete subtree $i \neq \sigma$ lies under a sibling of the path of p , and the root of subtree i is not needed when its range lies inside a carry, at a level at least 16 and at most that of the sibling, of the frontier of $C(A)$ or, provided the carry's range lies below n , of a seed frontier. Note eligibility, anchor choice and confirmation depth are separate conditions ("Spendable notes", §9.5).

Proof. Each condition supplies one kind of Theorem 7.14: condition (1) kind (a); condition (2) kind (b) and the complete part of kind (c); condition (3) the incomplete part of kind (c). Kind (d) needs nothing. For condition (3), the carries of the frontier at levels $t \geq 16$ are cap nodes over complete roots, and those below level 16 are functions of the leaves of subtree κ below n , appended in its extent. For the last claim, the ranges of $s_{16}, \dots, s_{31}$ partition the subtree indices other than σ ; if the range of subtree i lies inside such a carry, which is the same node at A , its range lying below n , the sibling over it is that carry, of kind (e), or a cap node computed from it, of kind (c), without the root of subtree i . $\square$

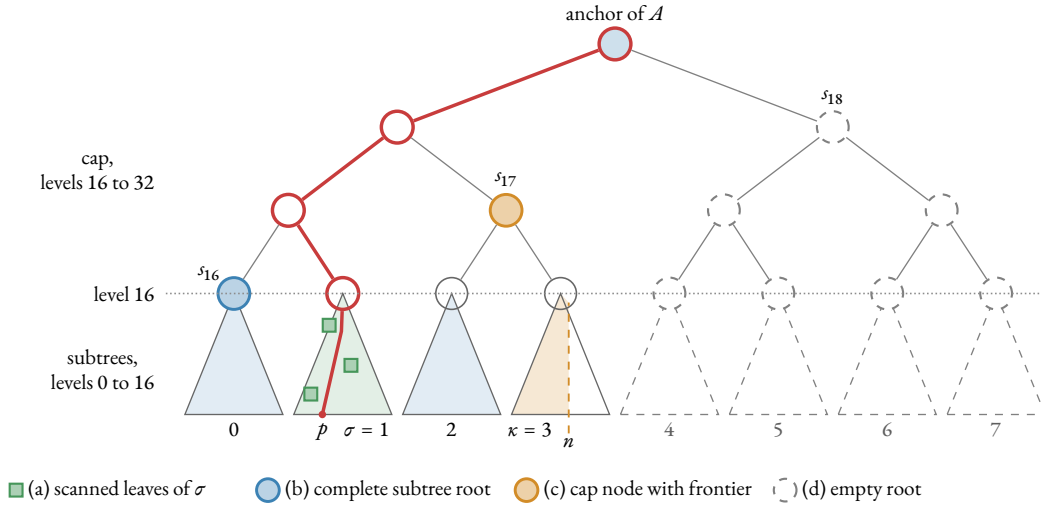


Figure 3. A note commitment tree cut at level 16, drawn with eight subtrees in place of 2^{16} and the cap above them compressed to three levels. Subtrees 0 and 2 are complete, subtree $\sigma = 1$ holds the note at position p , subtree $\kappa = 3$ is incomplete at the anchor’s tree size n , and subtrees 4 to 7 are unused. The route of p to the anchor is drawn in red. Its siblings are marked by the kind of Theorem 7.14 and the source that supplies them: the siblings $s_0, \dots, s_{15}$ inside subtree σ , from its scanned leaves (a); s_{16} , a downloaded complete subtree root (b); s_{17} , a cap node over a complete root and the root of subtree κ computed from the frontier after A (c); and s_{18} , the empty root beyond n (d). The split of the path at level 16 is Lemma 7.8.

Remark 7.18 (Scheduling). Scanning the tip range and the extents of the subtrees of found notes before historic ranges is a priority rule of wallet policy, designed but unspecified (Definition 1.1). The order of range requests is observable by the server (“Leakage of range queries”, §8.3).

Corollary 7.19 (Witnesses under any scan order). *Under the hypotheses of Lemma 6.13, a schedule of ranges that covers $[B, T]$ in any order yields, for every note and every retained anchor, the witness of a scan in increasing height from B , provided a note enters a witness only once the extent of its subtree (Definition 7.16) is scanned up to the anchor and the conditions of Proposition 7.17 hold.*

Proof. By the *Ironwood Guide*’s Lemma “Authentication paths from public data” (§“Authentication paths”), the path is a function of the position and of the leaves below n , which do not depend on the scan order. Positions are local to their blocks by Lemma 4.12, and the detection of the note is order-independent by Lemma 6.13. Once the stated extents are scanned, Theorem 7.14 makes the path computable. $\square$

Remark 7.20 (Cost of a restored wallet). A restored wallet seeds each tree from the chain state $C(B - 1)$ (Lemma 7.10), holds one root per complete subtree (Construction 7.6), and scans for witnesses only the extents of the subtrees of its notes and the tip range, instead of replaying the tree from the pool’s first block. By Proposition 7.17, the witness of a found note is ready once those ranges are scanned. The trial decryption of every block from B onward (Proposition 6.17) remains, and so does the scanning of the extents.

7.6 Reorganisation detection and recovery

Remark 7.21 (What a reorganisation invalidates). A reorganisation (*Ironwood Guide*, §“Anchors”, Definition “Reorganisation”) makes three kinds of wallet information wrong: the transactions of the wallet mined in replaced blocks become unmined; the anchors of replaced blocks vanish; and tree information over positions appended in replaced blocks no longer describes the chain.

Construction 7.22 (Detection of a reorganisation). Each synchronisation after the first requests blocks from its previously synchronised height X inclusive and checks that the hash of the received block X equals the stored one; a mismatch implies that block X was orphaned by a reorganisation (ZIP 307, “Client-server interaction”, phases B and D). This volume extends the check to every block and to both sides of every range boundary (Definition 6.10). Failure of the continuity predicate is a *continuity error* and triggers recovery (Construction 7.24). A malformed field, or a tree size that is absent, outside $[0, 2^{32}]$ or below the block’s own commitment count (Lemma 4.12(b)), is an error of the response and triggers none. The extension beyond ZIP 307’s comparison is designed but unspecified (Definition 1.1).

Definition 7.23 (Rollback to a height). *Rollback* to a height r at which the wallet holds a chain state $C(r)$ consists of the following changes.

- (i) The chain state of the wallet becomes $C(r)$, and each pool’s retained tree information becomes that of Proposition 7.13(ii) for r .
- (ii) Every transaction mined above r is marked unmined. The spend marks that such transactions set, on shielded notes and on transparent outputs, are cleared, and the nullifier of each note so unmarked returns to the tracked set $\mathcal{W}_p$ of its pool.
- (iii) Scan results above r are discarded: the unlinked nullifiers of R_p revealed above r , and the scanned ranges above r , so that the scanned heights of Definition 6.11 are at most r . A note whose creating transaction is now unmined loses its position and, for Sapling, its nullifier, which depends on the position; both are derived again if a later scan detects the note.
- (iv) The knowledge of rule (R4) of Definition 6.4 is cut back as that rule states; the evidence of rule (R2) stands.
- (v) Local-only data is kept: sent and received notes stay recorded, since the memos and transactions that the wallet created may be unrecoverable from the chain, while a note whose transaction is unmined is excluded from balances and from spendability (“Spendable notes”, §9.5).

Construction 7.24 (Recovery from a continuity error). Let $k \geq 2$ be an offset, a wallet parameter. On a continuity error at height e , that is, when a block of height e fails Definition 6.10 against the wallet’s chain state $C(e - 1)$, the wallet runs the following steps.

- (1) If $e - k < B - 1$, it obtains $C'(B - 1) := \text{TreeState}(B - 1)$, replaces its stored chain state at $B - 1$ by $C'(B - 1)$, rolls back to $B - 1$ (Definition 7.23), and goes to step (4).
- (2) Otherwise let r be the greatest height with $B - 1 \leq r \leq e - k$ that is a retained block boundary ($r \in \mathcal{H}$) or at which the wallet has recorded the hash of its scanned block. The height $B - 1$ qualifies, since $B - 1 \in \mathcal{H}$.
- (3) If $r \in \mathcal{H}$, the wallet rolls back to r . Otherwise it obtains $C'(r) := \text{TreeState}(r)$ (Definition 6.1), trusted under clause (a) of Assumption 5.2. If the hash of $C'(r)$ equals the recorded hash, the wallet takes the frontiers of $C'(r)$ as its tree states after r , discards every retained node over a position at or above their sizes, and rolls back to r . If the hashes differ, its block r is off the server's chain, and it repeats step (1) with $e := r + 1$.
- (4) It restarts synchronisation from the query $\text{Tip}()$.

A fetched chain state extends the depth to which the wallet recovers beyond its rollback window, but only at a height whose hash the wallet has recorded: without that comparison, a fetched chain state cannot show that the wallet's blocks at most r lie on the server's chain, and results from replaced blocks below r would survive. The construction is designed but unspecified (Definition 1.1).

Proposition 7.25 (Recovery reaches the fork). *Assume clause (a) of Assumption 5.2, let the server's best chain be unchanged during recovery, and exclude collisions of the block hash. Let the wallet's scanned heights below e form the interval $[B, e - 1]$, as in a scan in increasing height. The wallet's block at a height $h \in [B - 1, e - 1]$ is the block it scanned at h , or for $h = B - 1$ the block of its stored $C(B - 1)$. The fork point h_f is the greatest $h \in [B - 1, e - 1]$ at which the wallet's block is the server's block at h , and $h_f < B - 1$ if there is none. Agreement at a height implies agreement at every lower height of $[B - 1, e - 1]$: the wallet's blocks are linked by the checks of Definition 6.10, and each header contains the hash of its parent (protocol specification, §“Block Header Encoding and Consensus”).*

- (i) A continuity error at e implies $h_f \leq e - 2$.
- (ii) Repeated recovery by Construction 7.24 strictly decreases the rollback height r until $r \leq h_f$, or until step (1) replaces the stored chain state at $B - 1$; after either, no continuity error recurs.
- (iii) No offset below 2 guarantees progress: a rollback to $e - 1$ keeps the orphaned block, and the next scan fails again at e .

Proof. (i) Under clause (a) the server's block e has the requested height, so the failing clause of Definition 6.10 is (ii) or (iii). The server's block e has as parent its chain's block $e - 1$, and its sizes are functions of that chain's prefix (Lemma 4.12(a)). If the wallet's block $e - 1$ were the server's block $e - 1$, its hash and its sizes, which the wallet computed from the same prefix, would satisfy both clauses. So the chains disagree at $e - 1$, and $h_f \leq e - 2$.

(ii) Consider one recovery, with rollback height $r \leq e - k \leq e - 2$. If r is a retained boundary and $r > h_f$, the wallet's block r is off the server's chain, so the server's block $r + 1$, whose prevHash is the hash of the server's block r , fails continuity at $e' = r + 1$, and the next rollback height is at most $e' - 2 < r$. If r is a recorded height and the hashes differ, step (3) repeats with $e := r + 1$ and the next height is at most $r + 1 - k < r$. If the hashes agree, $r \leq h_f$ by the definition of the fork point. Heights are bounded below by $B - 1$, and a rollback height that would lie below it is replaced by step (1). Once $r \leq h_f$, the wallet's block r is on the server's chain, and every later block of that chain continues its predecessor. After step (1) the wallet's chain state at $B - 1$ is the server's and every result above it is discarded, so the same holds.

(iii) After a rollback to $e - 1$ the wallet's block $e - 1$ is unchanged, and by (i) it is off the server's chain; the server's block e fails against it again. $\square$

Theorem 7.26 (Reorganisation safety). *Under the hypotheses of Proposition 7.25, recovery by Construction 7.24 ends at a height r such that every block of the wallet at most r is a block of the server's chain: either a retained boundary or a recorded height $r \leq h_f$, the latter with the chain state $\text{TreeState}(r)$, or $r = B - 1$ with the chain state $\text{TreeState}(B - 1)$. Scanning the server's chain from $r + 1$ then yields the note state and the retained tree information of a scan of that chain from the birthday, with local-only data kept.*

Proof. Proposition 7.25 gives the end height r and its property. After the rollback, the retained information is a function of the blocks at most r (Proposition 7.13(ii), Definition 7.23), and those blocks are shared by both chains; where a fetched chain state is used, its frontiers are those of the server's chain under clause (a). Theorem 6.14 and Lemma 7.10 then apply to the server's chain from $r + 1$, seeded with $C(r)$. $\square$

Theorem 7.27 (Correctness of synchronisation). *Assume clause (a) of Assumption 5.2 and exclude collisions of the block hash. Let the server's best chain be unchanged from some time on, the final chain, and let T be the height of a tip of the final chain that the wallet obtains after that time. Let the wallet*

- (1) *check continuity before applying the results of a range (Definition 6.10), on both sides of every range boundary;*
- (2) *retain unlinked nullifiers as Lemma 6.12 requires;*
- (3) *take positions from consistent sizes (Lemma 4.12);*
- (4) *recover from every continuity error by Construction 7.24, scanning a height again only after a rollback below it;*
- (5) *enter a note into a witness only once the extent of its subtree is scanned up to the anchor (Definition 7.16);*
- (6) *scan the range that contains T after obtaining T , with continuity checked at its lower boundary (Definition 6.10), and discard scan results above T ; and*

(7) hold, as its complete subtree roots, those returned by a SubtreeRoots query made after obtaining T , restricted to completing height at most T .

If, after its last rollback, its scanned ranges cover $[B, T]$, in any order, the wallet holds the note state and the retained tree information of a scan of the final chain in increasing height from B .

Proof. The theorem composes earlier results and adds no mechanism. By (6) the range that contains T was requested after the tip query that returned T , so under clause (a) the wallet’s block T is that of the final chain. By (1), each block of $[B, T]$ was checked against the wallet’s chain state of its predecessor, and by (4) no height below it was scanned again afterwards without a rollback that also discarded the block; so the wallet’s blocks of $[B - 1, T]$ are linked by prevHash, and by the hash chain of the headers each of them is the final chain’s block at its height. By (4) a height is scanned again only after a rollback below it, which discards the earlier result at that height (Definition 7.23 (iii)), so every kept result of $[B, T]$ is that of the wallet’s current block; by (6) no result above T is kept. The scan of $[B, T]$ is therefore one of the final chain without a continuity error. By (7) and clause (a), the retained subtree roots are those of the final chain with completing height at most T . Lemma 6.13 gives the note state of the scan in increasing height, Corollary 7.19 the witnesses, and Theorem 6.14 characterises the notes found. $\square$

Remark 7.28 (The rollback window). Consensus bounds no reorganisation depth, and node rollback windows are local policy (*Consensus Guide*, §“Reorg rails, maturity, and the finality floor”). ZIP 307, “Local processing”, advises caching tree and witness state for 100 recent blocks, on the ground that no full node rolls back the chain by more than 100 blocks. That ground no longer holds: ZIP 218, “Block-count-based constants”, recommends a node rollback limit of 600 blocks at its 25-second target spacing and keeps coinbase maturity at 100 blocks, and neither figure bounds the window of a wallet. The depth that a wallet repairs from its retained information is the span of its retained block boundaries, a wallet parameter; a chain state fetched by TreeState at a height whose hash the wallet has recorded extends it under clause (a) of Assumption 5.2 (Construction 7.24).

8 Leakage to the light-client server

This section states what the server of “The light-client service” (§5) learns about a light wallet. It defines payment-detection privacy and unlinkability as indistinguishability games against the server of Assumption 5.2 and the view on which the server decides; proves that the view is a deterministic function of the chain, a tuple of schedule parameters and transport metadata, never of the wallet’s keys; derives the leakage of range queries, transaction queries, transparent-address queries and submission as instances of that theorem; and closes with a table of the server’s observables and the class of each mitigation.

8.1 Payment-detection privacy

The server is the node and the proxy together (Definition 5.1), under Assumption 5.2: clause (a) makes the response to each query a function of the server state and the query, and clause (b) is

the power of the adversary. No second adversary is defined. The *server state* Σ_t at time t is the server's best chain at t together with the state of its node's transaction pool; the latter enters only the responses of Submit and Status (Definition 5.9). An *evolution* is a family $\Sigma = (\Sigma_t)_{t \leq t^*}$ up to a horizon t^* . A *wallet configuration* is a pair (K, u) : K is the wallet's key material, the spending, full viewing, incoming viewing, outgoing viewing and nullifier deriving keys of every account; u is every other input of the wallet, namely the birthdays, the user's actions and their times, the scheduling policy's configuration, any state retained from earlier sessions, and the law from which the network endpoint of each connection is drawn.

Definition 8.1 (Server view). The *view* V of a wallet is the record of clause (b) of Assumption 5.2: the transcript

$$((c_j, q_j, a_j, t_j, r_j))_{j=1, \dots, J}$$

of the wallet's queries in the order issued, each with its connection c_j , its query name q_j of Definition 5.10, its argument tuple a_j , its time t_j and its response r_j ; and, for each connection c , the *transport metadata* $M_c = (E_c, t_c^{\text{open}}, t_c^{\text{close}})$, the network endpoint that the server sees and the open and close times. Times are recorded at a finite resolution, and a run up to a horizon issues finitely many queries, so every view ranges over a countable set.

Definition 8.2 (Payment-detection privacy game). Let π be a scheduling policy, a public randomised rule that emits the wallet's queries, made precise in “Key independence of queries” (§8.2). The *payment-detection privacy game* PD_π is the following indistinguishability game, with the advantage of a distinguishing game of the *Crypto Guide*, §“Security as a game”, Definition “Security game and advantage”; the adversary is not restricted to polynomial time.

1. The adversary fixes an evolution Σ , whose chain may contain outputs to either key set, and two configurations (K_0, u) and (K_1, u) with the same u .
2. The challenger draws a uniform bit $b \in \{0, 1\}$, runs the wallet (K_b, u) under π against a server with evolution Σ up to t^* , and hands the adversary the view V of that run (Definition 8.1).
3. The adversary outputs $b' \in \{0, 1\}$ and wins if and only if $b' = b$; its advantage is $\text{Adv} = |\Pr[b' = b] - 1/2|$.

Network privacy holds when the endpoint E_c of each connection is drawn afresh, independently of everything else, from a law that does not depend on which wallet opens the connection. The *unlinkability game* UL_π , under network privacy, differs in two points: the adversary fixes Σ and two configurations (K_0, u_0) and (K_1, u_1) ; and the challenger runs two consecutive synchronisation sessions, both of wallet 0 when $b = 0$, the second resuming from the state in which the first ends, and a session of wallet 0 followed by a session of wallet 1 when $b = 1$. The adversary receives the views of both sessions. Policy π provides *payment-detection privacy* (respectively *unlinkability*) if every adversary has advantage zero in PD_π (respectively UL_π), and provides it *within* ε if every advantage is at most ε .

Remark 8.3 (The goal of ZIP 307). ZIP 307, “Security Model”, states the goal informally and

with a lowercase “should”: the proxy should not learn which transactions received from the chain are addressed to a given light wallet, and, assuming network privacy (“via Tor or similar”), should not be able to link different connections or queries as deriving from the same wallet. It takes the node and proxy combination to be honest but curious, trusted to provide a correct view of the current best chain and to transmit queries and responses faithfully. Game PD_π formalises the first sentence and game UL_π the second; the trust clause is clause (a) of Assumption 5.2, an assumption distinct from the goal. The chain is common to both worlds of each game. What the chain itself reveals to a party without keys is the subject of the *Ironwood Guide*, §“Privacy”, Theorem “Privacy of an Action”, whose Remark “Non-claims” excludes wallet and network metadata, the subject of this section.

Remark 8.4 (Scope and status). ZIP 307, “Security Model”, does not address how to spend notes privately. Submission through the same server is nevertheless part of the view of Definition 8.1 and is analysed in “The submission channel” (§8.5). The status of ZIP 307 and of ZIP 314 is that of Table 1. ZIP 314, titled “Privacy upgrades to the Zcash light client protocol”, carries no specification text, so a specified successor to the privacy treatment of ZIP 307 is an open problem (Definition 1.1).

ZIP 307 makes two statements about its interaction of phases A to D (“Scan order”, §6.4). First, since synchronisation proceeds over a broadcast medium, it leaks no information about which transactions interest the client (“Transaction privacy”). Second, the start of each of phases B and D reveals the height to which the client last synchronised, an information leak under the security model, assuming network privacy (“Block privacy via bucketing”). The two statements hold together: the first concerns the content of the responses, the second the bounds of the queries, two components of the view that Theorem 8.8 separates. The second is a leak in UL_π , since the resume height links a session to the session that reached it; both are made precise in “Key independence of queries” (§8.2) and “Leakage of range queries” (§8.3).

8.2 Key independence of queries

A light wallet delegates the storage and filtering of the chain to the server and keeps its keys; its note state (Definition 6.7) is rebuilt on the device. What reaches the server is its query schedule, recorded as in Definition 8.1.

Lemma 8.5 (Query arguments carry no key material). *Every argument of every query of Definition 5.10 is one of the following: a block identifier, a height or a block hash (Block, TreeState); a pair of heights $s \leq e$ with a pool set $\mathcal{P}$ (Blocks); a shielded pool with a subtree index (SubtreeRoots); a transaction identifier (Tx, Status); a set A of transparent addresses (Utxos); the bytes of a transaction (Submit). Query Tip takes none. No argument is a spending, full viewing, incoming viewing, outgoing viewing or nullifier deriving key, and no argument other than that of Submit is a nullifier of a note that the wallet holds unspent. The argument of Submit carries the nullifiers of the notes it spends; it is a component of L (“transactions submitted”) whose law depends on K , so submission is not covered by PD_π (“The submission channel”, §8.5). Against adversaries restricted to polynomial time, what the public data of its Ironwood-pool bundle reveal of its keys is bounded by the Ironwood Guide, §“Privacy”, Theorem “Privacy of an Action”, under the preconditions and*

assumptions named there; that theorem claims nothing for an Orchard-pool bundle (its Remark “Non-claims”) and does not treat Sapling components. A transparent address is a receiver, disclosed by declaration (the Remark “Transparent boundary” below). Which arguments the wallet chooses may depend on its keys; that dependence is carried by the schedule parameters of Theorem 8.8.

Proof. By inspection of the argument types of Definition 5.10 (Table 4). $\square$

Lemma 8.6 (Local detection and witness assembly). *Trial decryption and acceptance (Construction 4.8, Definition 4.9), nullifier matching and the search of retained unlinked nullifiers (Construction 6.8), and the assembly of authentication paths from retained information (Definition 7.12, Proposition 7.13) take as inputs the wallet’s keys, responses already received and retained data, and issue no query.*

Proof. By inspection of the cited definitions. The underlying procedures are the *Ironwood Guide*’s, §“Trial decryption and note acceptance” (Construction “Trial decryption”) and §“Authentication paths” (Lemma “Authentication paths from public data”), both computed from data already held. $\square$

Definition 8.7 (Schedule parameters). A *scheduling policy* π is a public randomised rule that, from the responses received so far and a tuple L , emits the next query with its argument, its connection and its time. The *schedule parameters* L collect every other input on which the emitted schedule depends: the wallet birthday (Definition 6.2) and the resume heights; the bounds, the pool sets and the order of range queries; the subtree extents requested (Definition 7.16); the transaction identifiers fetched or queried; the transactions submitted; the sets of transparent addresses queried; and the policy’s random coins, a finite bit string that fixes times, orders and connection choices. Every component takes values in a countable set. Keys enter the schedule only through L (Lemmas 8.5 and 8.6). Given Σ , the law of L is induced by the configuration (K, u) . The endpoints $E = (E_c)_c$ are drawn from the law that u fixes, independently of L ; without network privacy that law may be a single point.

For random variables X and Y with values in a countable set, $\Delta(X, Y) := \frac{1}{2} \sum_s |\Pr[X = s] - \Pr[Y = s]|$ is their statistical distance.

Theorem 8.8 (The server’s view of a light wallet). *Assume Assumption 5.2, and let π be a scheduling policy. There is a deterministic function F_π with*

$$V = F_\pi(\Sigma, L, E).$$

Hence:

- (i) *two wallets with equal L and equal endpoints produce the same view, whatever their keys;*
- (ii) *in PD_π , with L_b the schedule parameters of wallet b and $V_b := F_\pi(\Sigma, L_b, E)$, every adversary that fixes Σ has*

$$\text{Adv} \leq \frac{1}{2} \Delta(V_0, V_1) \leq \frac{1}{2} \Delta(L_0, L_1),$$

and the first bound is attained by an adversary that is not restricted in time; in particular π provides payment-detection privacy if and only if, for every Σ , K_0 , K_1 and u , the view has the same law for K_0 and K_1 , which holds whenever the law of L given Σ is the same for both;

(iii) the bounds of (ii) hold for UL_π with the pair of the two sessions' parameter tuples in place of L , the endpoints being independent of b under network privacy.

Privacy holds exactly when the view has the same law for both key sets, and in particular whenever the law of L given Σ does not depend on the wallet's receipts, its detected notes and spends, or the transparent addresses it watches; each leak of "Leakage of range queries" (§8.3) to "The submission channel" (§8.5) is an instance.

Proof. Induction on the transcript. Query j , with its name, argument, connection and time, is emitted by π from L and responses $r_1, \dots, r_{j-1}$. Response r_j is, by clause (a) of Assumption 5.2, a function of Σ_{t_j} and query j . The metadata $\mathcal{M}_c$ is E_c with open and close times that the schedule fixes. So V is a function F_π of the three inputs, which gives (i).

For (ii), fix Σ . An adversary is, after its choice, a randomised decision rule; write $D(v) \in [0, 1]$ for the probability that it outputs 1 on view v . By the *Crypto Guide's* Definition "Security game and advantage" (§"Security as a game"), $\text{Adv} = \frac{1}{2} |\Pr[b' = 1 \mid b = 1] - \Pr[b' = 1 \mid b = 0]|$, and

$$\left| \sum_v (\Pr[V_1 = v] - \Pr[V_0 = v]) D(v) \right| \leq \sum_{v \in S} (\Pr[V_1 = v] - \Pr[V_0 = v]) = \Delta(V_0, V_1),$$

where S is the set of v with $\Pr[V_1 = v] > \Pr[V_0 = v]$ (or its complement, for the opposite sign); equality holds for D the indicator of S . For the second bound, E is independent of L_b with the same law in both worlds, so $\Delta((L_0, E), (L_1, E)) = \Delta(L_0, L_1)$; and for a deterministic f , $\Delta(f(X), f(Y)) \leq \Delta(X, Y)$, since $\Pr[f(X) = s]$ is the sum of $\Pr[X = x]$ over the preimage of s and the triangle inequality applies term by term. An adversary that chooses Σ at random has an advantage at most the maximum over its choices. The characterisation follows, since the advantage of the optimal adversary is $\frac{1}{2} \Delta(V_0, V_1)$. For (iii), apply the same argument to the pair of sessions; under network privacy E is independent of b and of the parameter tuples. $\square$

Corollary 8.9 (Compact responses reveal no receipt). *The response to $\text{Blocks}(s, e, \mathcal{P})$, $\text{Block}(b)$, $\text{TreeState}(b)$ and $\text{SubtreeRoots}(\text{pool}, i)$ is the same for every wallet that issues the same query at the same time. Let a policy's shielded synchronisation consist of Tip , $\text{SubtreeRoots}(\text{pool}, 0)$ for each shielded pool, $\text{TreeState}(B - 1)$, Blocks queries that cover $[B, T]$ in increasing height with bounds fixed by B , the resume heights, the returned tips and the policy's coins and one pool set $\mathcal{P}$ in every Blocks query, fixed independently of receipts, and the TreeState queries of Construction 7.24, with the retained boundaries $\mathcal{H}$ and the heights at which block hashes are recorded fixed by B , the resume heights, the responses and the policy's coins; and let its query times be fixed by its coins and the responses, not by the duration of local computation. Its L is then a function of B , the resume heights, the pool set and the coins, none a function of receipts, so its compact-block phase leaks no receipt. The commitment recomputation that replaces the discarded AEAD tag as the integrity check of compact decryption (Definition 4.9, Proposition 4.10) is local by Lemma 8.6. Requesting*

every complete subtree root from index 0 is independent of the positions of the wallet's notes; requesting a particular extent is not (Proposition 8.15). A policy that drops the Orchard pool from $\mathcal{P}$ once rule (R5) of Definition 6.4 stops its scanning reveals the height at which the Orchard-pool balance became known to be zero; keeping the pool in $\mathcal{P}$ and omitting only the trial decryption avoids this.

Proof. The first sentence is clause (a) of Assumption 5.2. The rollback heights of Construction 7.24 are functions of $\mathcal{H}$, the stored block hashes and the responses, hence, by the hypothesis on $\mathcal{H}$ and on the recorded heights, of Σ , B , the resume heights and the coins. The remainder is Theorem 8.8(i) and (ii) with the listed L , whose law given Σ does not depend on K . $\square$

The hypothesis on times is necessary: a policy that issues its next query when the processing of a range ends, when that processing takes longer for a range with detected notes, puts a function of receipts into the times of L .

Remark 8.10 (Transparent boundary). Query $\text{Utxos}(A)$ puts the addresses of A into L and into the view in the clear, by declaration, not by inference, and a query with $|A| \geq 2$ presents the members of A as one requester's. The addresses are derived from K , so in PD_π a policy that queries its watched transparent addresses has advantage $\frac{1}{2}$ against the adversary that reads A . The guarantee of Corollary 8.9 is a property of the shielded layer only.

Proposition 8.11 (Transparent-address queries). *Let a wallet watch n transparent addresses and query them one address per query, in rounds that visit every address once in a uniformly random order, at times whose successive gaps are independent draws of law $\text{Exp}(1/T)$ with $T = 86400/n$ seconds, of mean T , one expected query per address per day (Math Guide, §“Beyond finite spaces: countable additivity, limits, and densities”, Example “A jittered schedule”). Then no query argument contains two addresses, and the server cannot link two addresses by co-occurrence in one query. It can still link them through the connection and its endpoint when their queries share either, and through the times in L : per-address randomised timing does not unlink addresses queried over one connection. The schedule removes one linking channel, not linkability. The schedule is designed but unspecified (Definition 1.1); ZIP 318, “Transfer scheduling”, specifies an analogous schedule, a uniformly random shuffle and independent exponential delays, for its migration transactions.*

Proof. The arguments are singletons by construction. The remaining channels are the connection, endpoint and time components of the view, which Theorem 8.8 retains unchanged. The law of the gaps is cited, not derived. $\square$

8.3 Leakage of range queries

Proposition 8.12 (Birthday bound from range queries). *Let m be the least height of any block that the wallet requests, where $\text{Blocks}(s, e, \mathcal{P})$ requests the heights of $[s, e]$, $\text{Block}(h)$ the height h , and $\text{TreeState}(h)$ the height $h + 1$ whose chain state below it is fetched. A policy that requests no block below the wallet birthday B (Definition 6.2, the minimum of its accounts' birthdays) reveals*

$B \leq m$, and $B = m$ when it requests a range that starts at B , which need not be its first range under a prioritised order. Since B lower-bounds the first block in which any account could have received funds (ZIP 326, “Terminology”), the server learns the upper bound m on the height below which the wallet expects no receipt; once the wallet requests a range that starts at B , so that $m = B$, it learns that bound exactly. Adding an account with an earlier birthday lowers m later. A policy that starts at Sapling activation, as ZIP 307 prescribes for a seed that the client did not generate (“Importing a pre-existing seed”), and as this volume does for a seed restored without a recorded birthday (Definition 6.2), reveals only that constant. The birthday is a component of u , common to both worlds of PD_π ; its disclosure is a disclosure of u and, where birthdays differ, a linking component in UL_π .

Proof. The birthday B is a component of L and m a function of the view; the statement is an instance of Theorem 8.8. $\square$

Proposition 8.13 (Range bounds and scan state). *In the interaction of ZIP 307, “Client-server interaction”, the range of phase B starts at the height X that the previous synchronisation reached, and that of phase D at the height Y that phase B reached. Each start therefore reveals the previous synchronisation height, the tip at the time of the previous session, and so links consecutive sessions: in UL_π an adversary that chooses u_1 with a resume height other than the tip returned to the first session wins with probability 1. Under a prioritised policy the bounds are those of the policy’s ranges (verification ranges near the stored tip, extents of subtrees, historic ranges); they reveal the scan state, the maximum and fully scanned heights of Definition 6.11, to the extent that the ranges are functions of it, and need not equal the previous synchronisation height.*

Proof. The bounds are components of L , and the statement is an instance of Theorem 8.8(ii) and (iii). For the adversary: when $b = 0$ the second session starts at the tip returned to the first, when $b = 1$ at the resume height of u_1 , and the two differ with probability 1. $\square$

Remark 8.14 (Order of range queries). The order of range queries is a component of L . A policy that orders its ranges by a wallet-dependent priority (“Scan order”, §6.4, the reference interaction; “Witness readiness”, §7.5, scheduling) reveals that order, by Theorem 8.8. Every such priority is designed but unspecified (Definition 1.1).

Proposition 8.15 (Out-of-order subtree completion). *Let a policy, on detecting a note at position p of pool P , always request the block extent of subtree $s = \lfloor p/2^{16} \rfloor$ (Definition 7.16) as a query that the in-order schedule would not issue at that time. The extent starts at $\max(c_p(s-1), B)$ and ends at the completing height $c_p(s)$, or at the anchor height for the incomplete last subtree; the server knows every completing height from the responses to $\text{SubtreeRoots}(P, 0)$, so it learns s and that the wallet holds a detected note among the at most 2^{16} leaves of subtree s . Against such a policy, an adversary in PD_π that places an output to K_0 alone in a subtree outside both wallets’ in-order schedule at that time, and chooses K_1 with no receipt in Σ , wins with probability 1. The statement is an inference over the abstract policy of “Witness readiness” (§7.5), not a measurement of any deployment; no mitigation is specified, and one is an open problem (Definition 1.1).*

Proof. The requested extent is a function of p , a receipt, and a component of L ; the map from extent bounds to s is public. Wallet 0 issues the extent query and wallet 1 does not, so the adversary that outputs $b' = 0$ exactly when the view contains that query wins with probability 1, an instance of Theorem 8.8(ii). $\square$

Construction 8.16 (Bucketing). Fix a bucket size $N \geq 1$ and let $\lfloor X \rfloor_N := X - (X \bmod N)$. Phase B requests $\text{Blocks}(\lfloor X \rfloor_N, Y, \mathcal{P})$ in place of $\text{Blocks}(X, Y, \mathcal{P})$, and phase D requests $\text{Blocks}(\lfloor Y \rfloor_N, Z, \mathcal{P})$ in place of $\text{Blocks}(Y, Z, \mathcal{P})$. The blocks from $\lfloor X \rfloor_N$ to X are checked, height by height, to carry the block hash of the wallet’s chain view, and are then discarded without further processing. On a mismatch at height $Q > \lfloor X \rfloor_N$ the wallet rolls back (Definition 7.23) to $Q - 1$ if it holds $C(Q - 1)$. Otherwise it obtains $C'(Q - 1) := \text{TreeState}(Q - 1)$, compares hashes as in step (3) of Construction 7.24, takes its frontiers and rolls back to $Q - 1$. Both chains then agree at $Q - 1$, and it sets $X := Q - 1$. A mismatch at $Q = \lfloor X \rfloor_N$ goes to Construction 7.24 with $e := Q$. In phase D, Y replaces X throughout this check. Most reorganisations are thereby handled within the same range. The construction is specified, as the modification of phases B and D in ZIP 307, “Block privacy via bucketing”, which the ZIP describes as a proposed enhancement.

Proposition 8.17 (Leakage under bucketing). *Under Construction 8.16 the start of a range of phase B or D reveals $\lfloor X \rfloor_N$ and no more of the resume height X : two wallets whose resume heights share a bucket and whose parameter tuples agree elsewhere issue ranges with equal starts. The ends of the ranges, their order and their timing are unchanged, and so is their leakage. The gain is in the set of resume heights consistent with a start, from one height to one bucket of N heights; it is not an advantage bound of UL_π , in which an adversary that chooses u_1 with a resume height in another bucket still wins with probability 1.*

Proof. The start is $\lfloor X \rfloor_N$, a function of X that is constant on buckets; the other components of L are untouched (Theorem 8.8). The last clause is the argument of the proof of Proposition 8.13 with starts in distinct buckets. $\square$

8.4 Leakage of transaction queries

Proposition 8.18 (Uncovered transaction queries). *A query $\text{Tx}(\text{txid})$ or $\text{Status}(\text{txid})$ issued without cover puts that transaction into L and reveals it as one that the wallet is fetching or tracking: a received transaction whose memo or outgoing plaintext the wallet recovers (the enhancement of Construction 6.8), a sent one, or a transparent dependency. This is the set that payment-detection privacy protects. Against a policy that fetches exactly its wallet-relevant transactions, an adversary in PD_π that places an output to K_0 alone, at a height that the common schedule scans before the horizon, and chooses K_1 with no receipt in Σ , wins with probability 1. A transaction identifier queried in two sessions links them in UL_π .*

Proof. The fetched set is a function of receipts and a component of L . Wallet 0 fetches the transaction of the output and wallet 1 does not; the statement is an instance of Theorem 8.8(ii) and (iii). $\square$

Remark 8.19 (Rules of ZIP 307 for full transactions). ZIP 307, “Transaction privacy”, states the following interoperability rules. A light client needs full transactions for the memo fields of received notes and for outgoing ciphertexts, to recover spend information when a seed is imported. It SHOULD obscure the exact transactions of interest by also downloading numerous uninteresting transactions; it SHOULD download all transactions of any block from which it fetches a single full transaction; and it MUST convey to the user that fetching full transactions reduces privacy. The MUST is unconditional, not a fallback for omitting the SHOULDs. Under the second SHOULD the fetched set reveals the blocks that hold a transaction of interest, not the transactions. The rules are specified (Definition 1.1).

No query of Definition 5.10 returns the full transactions of a block, since Block returns the compact block; the second SHOULD therefore costs one Tx query per transaction of the block.

Remark 8.20 (Timing of transaction queries). A Tx query issued at a time fixed by the completion of the range in which its transaction was detected is correlated in time with that range; under network privacy with separate connections, the times in L still link the two (Theorem 8.8(iii)). A delay drawn independently of that completion time would remove the correlation. No ZIP specifies one for transaction queries, and no stated design fixes its law: an open problem (Definition 1.1).

8.5 The submission channel

Proposition 8.21 (Submission through the server). *Let the wallet submit a transaction by Submit on a connection c to its synchronisation server. The server then holds the complete transaction before it is broadcast, at a time in L , on a connection with endpoint E_c . Without separation at the network layer it links the transaction to that endpoint and to the other queries of the same connection or endpoint, among them the birthday bound, the range bounds and the transaction queries of the session. The disclosure lies outside the goal of ZIP 307 and inside the view of Theorem 8.8.*

Proof. The transaction, its time and its connection are components of the view; the statement is an instance of Theorem 8.8(i). $\square$

The server’s operator terminates the transport, so channel encryption such as TLS protects queries from third parties and not from the server. Every connection exposes to the server its endpoint E_c and its open and close times, the endpoint being the wallet’s network address or that of an exit of an anonymity network or of a proxy. Without network privacy E_c is a component common to all sessions of one wallet, which is why the unlinkability goal of ZIP 307, “Security Model”, assumes network privacy.

Proposition 8.22 (Separate circuits). *Let submission and synchronisation use connections whose endpoints are drawn independently, as separate anonymity-network circuits give; or let submission go to a server that does not share its view with the synchronisation server, in which case the Submit query is absent from that server’s view and the separation rests on that non-collusion assumption. Then the endpoint no longer links the Submit query to the synchronisation session. Correlation remains through the times in L ; through content, the anchor and expiry heights of the*

transaction against the range bounds of the session; and through application-level identifiers, such as a later Status query for the submitted transaction identifier on the synchronisation connection. For payments the separation is designed but unspecified (Definition 1.1). For migration transactions ZIP 318 specifies the network-privacy step (“Network-layer privacy”: the wallet MUST offer Tor and, where available, Nym, and MAY broadcast through a server other than the synchronisation server) and the separation in time of synchronisation from broadcast (“Decoupling synchronization from broadcast”).

Proof. The endpoint components of the two connections are independent, so their joint law is the same whether or not the connections belong to one wallet; every other component of L is unchanged, and the remaining channels are those components (Theorem 8.8(iii)). In the second case the synchronisation server’s view contains no Submit query, and under non-collusion the other server’s view is not joined to it. $\square$

8.6 Summary of server observables

Table 5 lists the observables of the server, what each reveals, its mitigation and the class of that mitigation (Definition 1.1). The interface of Definition 5.10 omits mempool queries, so no row concerns the mempool.

Every row of Table 5 is a component of L or of the transport metadata (Theorem 8.8); none is a function of the content of compact blocks (Corollary 8.9). The size of each leak depends on policies that the protocol leaves open: beyond the rules of ZIP 307 and the migration schedule of ZIP 318, no ZIP fixes the scheduling policy of a wallet.

9 Anchors, confirmations, and expiry

This section fixes the heights and the set of notes from which a wallet builds a transaction. It states the confirmation policy of ZIP 315; the target height and the anchor height, with the lemma that ties the anchor depth to the trusted confirmation depth; the expiry rule of ZIP 203 and the expiry height a wallet sets; the record a wallet keeps of the transactions it creates and the notes those transactions lock; and the spendability categories of ZIP 315, with the spendable set defined from the wallet’s state.

9.1 The confirmation policy

An output mined in a block of the best chain leaves the chain when a reorganisation replaces that block (*Ironwood Guide*, §“Anchors”, Definition “Reorganisation”), so a wallet spends an output only once it has a number of confirmations fixed by a policy. The rules of this subsection are those of ZIP 315 and are specified (Definition 1.1); the status of ZIP 315 is that of Table 1.

Definition 9.1 (Confirmation policy). A *confirmation policy* is a triple $(c_{\text{tr}}, c_{\text{untr}}, z)$ of integers $1 \leq c_{\text{tr}} \leq c_{\text{untr}}$, the *trusted* and the *untrusted confirmation depth*, and a flag $z \in \{\text{true}, \text{false}\}$ that permits zero-confirmation shielding. Confirmations are those of the *Consensus Guide*, §“Work, and the best chain”, Definition “Confirmations”: at a tip of height T , an output mined in the

Table 5. Observables of the light-client server, what each reveals about the wallet, the mitigation of each, and the class of that mitigation (Definition 1.1). Every row is a component of the schedule parameters or of the transport metadata of Theorem 8.8.

Observable	Reveals	Mitigation	Class
Network endpoint and connection times	network location and activity windows	network anonymity (Proposition 8.22)	designed but unspecified; specified for migration broadcasts (ZIP 318)
Least requested height	an upper bound on the birthday (Proposition 8.12)	a start at Sapling activation, at the cost of Proposition 6.17	specified for an imported seed (ZIP 307)
Range bounds and order	resume height and scan state (Proposition 8.13)	bucketing of the start (Construction 8.16)	specified (ZIP 307)
Out-of-order completion ranges	the subtree of a detected note (Proposition 8.15)	none	open problem; the leak is inferred
Transaction queries	the transactions of interest (Proposition 8.18)	decoys and whole-block fetches (SHOULD)	specified (ZIP 307)
Submission	the transaction before broadcast, linked to its session (Proposition 8.21)	separate circuits	designed but unspecified
Transparent-address queries	the addresses, by declaration (§8.2)	one address per query at exponential times in random order (Proposition 8.11); linkage by connection and timing remains	designed but unspecified
Pool sets of range queries	if narrowed by wallet state, the height at which the Orchard-pool balance became known to be zero (Corollary 8.9)	one pool set fixed independently of receipts; only trial decryption omitted	designed but unspecified

block of height $m \leq T$ of the best chain has $T - m + 1$ confirmations, its own block being the first; an output of a transaction that no block of the best chain contains has 0. ZIP 315 RECOMMENDS the depths $c_{\text{tr}} = 3$ and $c_{\text{untr}} = 10$ (“Trusted and untrusted TXOs”) and permits, without recommending, $z = \text{true}$ (“Transparent UTXOs”, MAY); a wallet SHOULD have a policy that it communicates to the user in a clear form (“Trusted and untrusted TXOs”).

Definition 9.2 (Trusted and untrusted outputs). A *TXO* is an output, transparent coin or shielded note, of a transaction of the best chain or of a mempool visible to the wallet (ZIP 315, “Terminology”). A TXO received by the wallet is *trusted* if it is an output of a transaction created by the wallet itself, which a wallet SHOULD treat as trusted, or of an external transaction that the user has marked trusted, a marking that a wallet MAY offer; every other received TXO is *untrusted* (ZIP 315, “Trusted and untrusted TXOs”; ZIP 315, “Prompt accessibility of funds”). A trusted TXO requires c_{tr} confirmations and an untrusted TXO c_{untr} . The output of a shielding transaction is not trusted by virtue of its internal receiver (ZIP 315, “Transparent UTXOs”); its depth rule is stated below.

The rationale of the default depths is ZIP 315’s and is cited, not derived (ZIP 315, “Rationale for the given numbers of confirmations”): the trusted depth is 3 because reorganisations are anecdotally shorter than three blocks, and the two depths differ because the value of a trusted TXO is recoverable after a rollback, the wallet re-creating its transaction, whereas an untrusted TXO may need the sender to pay again. The modelled double-spend risk of a given count is that of the *Consensus Guide*, §“Waiting for confirmations” and §“Wallet confirmation policy”: three confirmations give an attacker with 10% of the hash rate a success probability of 1.712%, and ten about 0.0008%. A confirmation count bounds the modelled double-spend probability only for a stated attacker share q of the hash rate and a tolerated probability ε (*Consensus Guide*, §“Wallet confirmation policy”); it is not a guarantee.

Confirmation counts are in blocks, not in time. At the 25-second target spacing (ZIP 218, “Block target spacing”; *Consensus Guide*, §“Zcash time: 25-second blocks”), 10 blocks are $10 \times 25 = 250$ s and 3 blocks are $3 \times 25 = 75$ s. ZIP 218 does not argue for reducing confirmation counts (ZIP 218, “Motivation”) and keeps the anchor depth at 3 blocks (ZIP 218, “Anchor selection depth”). Neither an equal count nor an equal elapsed time establishes an equal risk, whose model is that of the *Consensus Guide*.

Two kinds of transaction enter the rules below; both are constructed in “TEX transfers and shielding transactions” (§11.3). A *shielding transaction* spends transparent UTXOs of the wallet and pays their value, less the fee, to an internal shielded receiver of the spending account, with no other recipient. An *ephemeral UTXO* is the transparent output that the first transaction of a transfer to a TEX address creates and its second transaction spends (ZIP 320, “Design considerations for Senders”; ZIP 315, “Trusted and untrusted TXOs”).

The following rules are specified. *Floor*: wallets SHOULD NOT permit the spending of TXOs with fewer than 3 confirmations, except (i) transparent UTXOs spent in shielding transactions and (ii) ephemeral UTXOs of a ZIP 320 transfer, whose second transaction spends an output that may be unmined (ZIP 315, “Trusted and untrusted TXOs”). *Zero-confirmation shielding*: wallets MAY spend received transparent UTXOs in wallet-internal shielding transactions with zero confirmations (ZIP 315, “Transparent UTXOs”); the flag z of Definition 9.1 records whether the

wallet does. *Shielding outputs*: a shielded output of a wallet-internal shielding transaction meets its depth requirement if and only if every transparent UTXO that the transaction spent has at least $c_{\text{untr}} - c_{\text{tr}}$ confirmations and the shielding transaction has at least c_{tr} (ZIP 315, “Shielded TXOs”). With h_s the greatest mining height of the spent UTXOs and T the tip, the first condition is $T - h_s + 1 \geq c_{\text{untr}} - c_{\text{tr}}$.

Remark 9.3 (Two readings of the shielding rule). ZIP 315 states the rule for shielding outputs twice, and the two statements differ. The paragraph “Shielded TXOs” requires $c_{\text{untr}} - c_{\text{tr}}$ confirmations of the source UTXOs; the paragraph “Transparent UTXOs” requires c_{untr} confirmations of the transaction that produced them. Both require c_{tr} confirmations of the shielding transaction. Let the sources be mined at greatest height h_s and the shielding transaction at $h_s + d$ with $d \geq 0$. At the policy (3, 10, true), the first reading is met at every tip

$$T \geq h_s + \max(6, 2 + d),$$

and the second at every tip $T \geq h_s + \max(9, 2 + d)$; the two agree for $d \geq 7$. Indeed, a block at height m has $T - m + 1$ confirmations at tip T (Definition 9.1), and

$$\begin{aligned} T - h_s + 1 \geq 7 &\iff T \geq h_s + 6, \\ T - h_s + 1 \geq 10 &\iff T \geq h_s + 9, \\ T - (h_s + d) + 1 \geq 3 &\iff T \geq h_s + 2 + d. \end{aligned}$$

The rule stated above is the first reading, the one inside ZIP 315’s definition of a confirmed-spendable TXO. Which reading the ZIP intends is an open problem (Definition 1.1).

9.2 Target and anchor heights

Definition 9.4 (Target height). Let T be the height of the tip of the wallet’s view of the chain, the height returned by `Tip()` (Definition 5.10). The *target height* of a transaction built at that view is $H := T + 1$, the height of the first block that could include it. Every height-dependent rule of construction is fixed by H : the consensus branch, the fee rule and the expiry height are those of a block of height H , and confirmation counts are taken at the tip $T = H - 1$.

Definition 9.5 (Anchor height). Let $(c_{\text{tr}}, c_{\text{untr}}, z)$ be a confirmation policy (Definition 9.1) and $H \geq c_{\text{tr}}$ a target height. The *anchor height* of a transaction with target height H is

$$A := H - c_{\text{tr}},$$

a function of H and the policy alone. One anchor height serves every anchor that the transaction carries, including that of an Orchard-protocol bundle whose Actions spend no note. For a pool $P \in \{\text{Sapling, Orchard, Ironwood}\}$, the anchor of the transaction’s bundle of P is the root of P ’s note commitment tree in the final treestate of the block at height A (*Ironwood Guide*, §“Anchors”, Definition “Trestate and anchor”), and the authentication path of every note that the transaction spends from P is computed to that root (*Ironwood Guide*, §“Authentication paths”). Since $c_{\text{tr}} \geq 1$, the block at height A precedes the block at height H , as the consensus

anchor rules require (*Ironwood Guide*, §“Anchors”; protocol specification, §“Action Transfers and their Descriptions” and §“Spend Transfers, Output Transfers, and their Descriptions”). A transaction therefore carries, for each pool it touches, its own anchor and authentication paths, the Orchard pool and the Ironwood pool of the Orchard protocol separately.

Lemma 9.6 (Anchor height and confirmations). *Let T be the tip, $H = T + 1$ and $A = H - c_{\text{tr}}$. For a note mined at height $m \leq T$ of the best chain, the following are equivalent:*

- (i) *the note’s commitment is a leaf of the tree rooted by the anchor of its pool at A ;*
- (ii) *$m \leq A$;*
- (iii) *the note has at least c_{tr} confirmations at T .*

The anchor depth thus equals the trusted confirmation depth: a note with exactly c_{tr} confirmations lies in the block at height A . At $c_{\text{tr}} = 3$ the anchor height is $A = T - 2$, and a note with at least $c_{\text{untr}} = 10$ confirmations is mined at a height at most $T - 9 < A$.

Proof. The final treestate of the block at height A holds exactly the commitments appended in the blocks at heights $0, \dots, A$ (*Ironwood Guide*, §“Anchors”, Definition “Treestate and anchor” and Remark “Old anchors”), so (i) and (ii) are equivalent. By Definition 9.1 the note has $T - m + 1$ confirmations, and $T - m + 1 \geq c_{\text{tr}}$ if and only if $m \leq T + 1 - c_{\text{tr}} = A$, so (ii) and (iii) are equivalent. The instances follow with $c_{\text{tr}} = 3$, $A = T + 1 - 3$, and $T - m + 1 \geq 10$ if and only if $m \leq T - 9$. $\square$

The rule for the anchor is specified: a wallet SHOULD choose an anchor a number of blocks back from the head of the chain equal to the trusted confirmation depth, the ZIP’s instance being the final treestate of the block at height $H_{315} - 3$ when the current block is at height H_{315} (ZIP 315, “Anchor selection”). ZIP 218 recommends that the depth stay 3 blocks at the 25-second spacing, $3 \times 25 = 75$ s (ZIP 218, “Anchor selection depth”). The rule is a recommendation to the builder, not a consensus rule: the anchor rules admit the final treestate of any earlier block.

Remark 9.7 (The anchor convention). ZIP 315 counts the anchor depth back from the head of the chain, and ZIP 218, “Anchor selection depth”, reads it as the tip, so that a 4-block reorganisation is needed to remove the anchor; the *Ironwood Guide*, §“Anchors”, Remark “Recommended anchor depth”, follows that reading, which puts the anchor at $T - 3$ with $c_{\text{tr}} + 1$ confirmations. This volume’s convention counts from the block being built, $H = T + 1$, one block shallower, which gives Lemma 9.6: the anchor block has exactly c_{tr} confirmations. Both satisfy the consensus anchor rules.

The anchor depth is c_{tr} whatever the trust of the notes spent. The eligibility of each note is decided by its own depth (Definition 9.2), so an untrusted note spent at depth c_{untr} is proved against the anchor at depth c_{tr} , whose tree contains it by Lemma 9.6. ZIP 315 gives three reasons for the rule, specified and cited (ZIP 315, “Rationale for anchor selection”): a rollback past the anchor block invalidates the transaction, whose revealed nullifiers then link it to any later transaction spending the same notes; too deep an anchor keeps recently received notes from being spent; and a fixed depth, rather than one depending on whether the notes spent are trusted, does not

reveal their trust. The anchor is published data (*Ironwood Guide*, §“Anchors”, Remark “Anchor choice as public data”).

Every Orchard-protocol bundle with at least one Action carries an anchor, which must refer to the final treestate of an earlier block in its pool, whether or not any Action spends a note of nonzero value (protocol specification, §“Action Transfers and their Descriptions”; consensus rule). Giving such a bundle the anchor at A makes it indistinguishable by its anchor from a bundle that spends (*Ironwood Guide*, §“Anchors”, Remark “Anchor choice as public data”).

9.3 Expiry (ZIP 203)

Definition 9.8 (Transaction expiry). Every transaction of version at least 3 carries a field $\text{nExpiryHeight} \in \{0, \dots, 2^{32} - 1\}$, a block height and never a timestamp; for a non-coinbase transaction, $\text{nExpiryHeight} \leq 499,999,999$ (consensus rule). A non-coinbase transaction with $N := \text{nExpiryHeight} \neq 0$ is *expired* at height b if and only if $b > N$, and it MUST NOT be mined at a height greater than N (consensus rule), so the block at height N is the last that may include it; the value $N = 0$ disables expiry (protocol specification, §“Transaction Encoding and Consensus”; ZIP 203, “Specification”). Coinbase transactions, which a wallet does not create, are outside the definition.

Lemma 9.9 (Includability at the next block). *Let t be a non-coinbase transaction with expiry height N , and T the tip. The expiry rule permits t in the block at the target height $H = T + 1$ if and only if $N = 0$ or $T < N$; when $N \neq 0$, it permits t in no block of height greater than N . Inclusion requires the other consensus rules as well; the lemma concerns expiry only.*

Proof. By Definition 9.8, the rule permits t at height H if and only if $N = 0$ or $H \leq N$, and $H = T + 1 \leq N$ if and only if $T < N$. The second claim is the rule itself. $\square$

Construction 9.10 (Expiry height of a wallet transaction). A wallet sets $\text{nExpiryHeight} := H + \delta$, where H is the target height (Definition 9.4) and $\delta \geq 1$ is an *expiry delta* chosen by the wallet. Reading ZIP 203’s “current height” as the target height is designed but unspecified (Definition 1.1). Every later rule that depends on the delta, the locking of notes (§9.4) and the polling and re-sending of transactions (§13), uses the δ actually chosen.

ZIP 315 specifies (SHOULD) that a wallet create transactions using the default expiration height of ZIP 203, which it gives as 40 blocks from the current height (ZIP 315, “Expiration height”). ZIP 218 changes the default of node implementations that create transactions to $\text{NU7PoWTargetSpacingRatio} \times 40 = 120$ blocks (SHOULD), to keep an expiry of about 50 minutes, and keeps an explicitly configured delta (ZIP 218, “Default expiry delta”); ZIP 315 is not updated for the 25-second spacing. At that spacing, ZIP 315’s SHOULD therefore has two readings: the literal 40 blocks, about $40 \times 25 = 1000$ s, and the ZIP 203 default as ZIP 218 changes it, 120 blocks, about $120 \times 25 = 3000$ s. Which one a wallet uses is an open problem (Definition 1.1). Either time is an estimate, since block intervals vary, and not a guaranteed period of validity.

The choice of delta has a privacy consequence, specified. ZIP 315 lists the expiry height, with the anchor position, among the data that reveal to which height the creator was synchronised and some of its policy (ZIP 315, “Kinds of information leakage”), and it specifies a common default (ZIP 315, “Expiration height”). An expiry delta other than the common one therefore distinguishes its creator, as a fee other than the conventional fee does (§10).

The expiry height is fixed by the transaction identifier: `nExpiryHeight` is effecting data, an input of the header digest (ZIP 244, “T.1: header_digest”), so by Lemma 13.1(ii) a party matches a transaction, and rejects it once expired, by its identifier alone.

When a transaction expires, nodes remove it from their mempools together with every transaction that depends on it (ZIP 203, “Specification”). This is node policy layered on the consensus rule, which constrains only the contents of blocks.

9.4 The transaction record and locked notes

Definition 9.11 (Transaction record). The *transaction record* of a wallet holds, for each transaction t that the wallet creates:

- (i) the identifier $\text{txid}(t)$ and the encoding of t , once the wallet holds it;
- (ii) its fee, its target height H_t and its expiry height N_t ;
- (iii) the set of its inputs: the notes it spends in each shielded pool and the transparent out-points it spends;
- (iv) its mined height, or $\perp$ while it is unmined;
- (v) the earliest height h_t^{obs} at which it was observed: initially H_t for a transaction the wallet builds, and the height of the block in which it was first seen mined for a record entered from compact blocks;
- (vi) the latest tip at which the server reported it unmined.

A record entered without the encoding of the transaction has N_t unknown; such is the record of a transaction learnt from compact blocks, whose compact transaction carries no expiry height (Definition 4.3), and later unmined by a rollback (Definition 7.23). The transitions of the record are those of “Status tracking and terminal states” (§13.2). The record is information that the wallet retains, not a storage layout.

Definition 9.12 (Locked note). Fix an expiry delta $\delta \geq 1$, a parameter of the definition, whose default at the 25-second spacing is the open problem of §9.3. At target height H , a note is *locked* if and only if some recorded transaction t that spends it satisfies one of:

- (a) t is mined, necessarily at a height below H ;
- (b) $N_t = 0$;
- (c) $N_t \geq H$;

(d) N_t is unknown and $h_t^{\text{obs}} + \delta \geq H$.

A locked note is excluded from selection.

Proposition 9.13 (Locks follow includability). *Let t be a recorded transaction with known expiry height N_t that spends a note. At target height $H = T + 1$, the conditions of Definition 9.12 hold for t if and only if t is mined or the expiry rule permits t at H (Lemma 9.9). Hence, for a note that no mined recorded transaction spends and whose recorded spending transactions all have known expiry heights, being unlocked is the clause of ZIP 315 that the TXO is not committed to be spent in another unexpired transaction created by the wallet (ZIP 315, “Categories of TXOs according to spendability”); a mined spend, which condition (a) locks, makes the note spent, and ZIP 315’s clause that the TXO is unspent excludes it; and an unmined t with $N_t > 0$ releases its inputs, as far as t is concerned, at every target height $H > N_t$.*

Proof. For known N_t , condition (d) of Definition 9.12 is void. By Lemma 9.9, the expiry rule permits t at H if and only if $N_t = 0$ or $H \leq N_t$, which are conditions (b) and (c); with condition (a) this gives the first claim. By Definition 9.8, t is unexpired at H if and only if $N_t = 0$ or $H \leq N_t$, so for an unmined t the conditions hold exactly when t is an unexpired transaction of the wallet that spends the note, and a note is unlocked exactly when no such transaction exists, which is ZIP 315’s clause (ZIP 203, “Specification”). For $H > N_t > 0$ and t unmined, each of (a) to (d) fails. $\square$

Remark 9.14 (Unknown expiry). Condition (d) of Definition 9.12 assumes that t was created with the delta δ at a target height at most h_t^{obs} , so that $N_t \leq h_t^{\text{obs}} + \delta$. If $N_t = 0$, or its creator used a larger delta with $N_t > h_t^{\text{obs}} + \delta$, the note is released from $H = h_t^{\text{obs}} + \delta + 1$ while t can still be mined (for $H \leq N_t$, or at every H if $N_t = 0$), and a re-spend conflicts with the late mining of t . If its creator used a smaller delta, the note stays locked longer than necessary. Either misjudgement lasts until the wallet sees t mined, or obtains its encoding and hence N_t by the query Tx (Definition 5.10).

Definition 9.15 (Watched nullifiers). The *watched nullifiers* are the set $W := \bigcup_p W_p$ of the tracked sets of Definition 6.7: the nullifiers of the wallet’s detected notes not marked spent. A lock (Definition 9.12) removes a note from selection and never removes its nullifier from W . A nullifier leaves W only when Construction 6.8 marks its note spent, that is, when a transaction revealing it is mined, whatever that transaction’s expiry height, $N = 0$ included; or when a rollback unmines the creating transaction of a Sapling note, whose nullifier is then discarded until a later scan derives it again (Definition 7.23(iii)). A rollback that clears the spent mark returns the nullifier to W .

The mining of any transaction that spends a note of the wallet is therefore detected by scanning through W , independently of status queries (*Ironwood Guide*, §“Nullifier sets”). A rule that removed from W the nullifiers spent by transactions with $N = 0$, combined with status polling that ends at the first report of an unmined transaction, would leave such a transaction, when it has no output to the wallet, mined and never observed as mined: no block would reveal a watched nullifier, and no note of the wallet would be created by it.

9.5 Spendable notes

Definition 9.16 (Spendability categories). Relative to a chain and a wallet state (ZIP 315, “Categories of TXOs according to spendability”):

- (i) a TXO is *known-spendable* if and only if (1) it is unspent at the wallet’s view of the tip and that view is reasonably up to date, (2) it is not committed to be spent in another unexpired transaction created by the wallet (Proposition 9.13), and (3) the wallet can effect the spend, by holding its spending key or by mediating interaction with a hardware device or another offline signing protocol;
- (ii) a shielded TXO is *confirmed-spendable* if and only if it is known-spendable, the wallet can construct its witness (Theorem 7.14), and either it is not an output of a wallet-internal shielding transaction and has c_{tr} confirmations if trusted, c_{untr} if untrusted (Definition 9.2), or it is such an output and meets the rule for shielding outputs of §9.1 (ZIP 315, “Shielded TXOs”);
- (iii) a known-spendable TXO that is not confirmed-spendable is *unconfirmed-spendable*;
- (iv) a TXO is *watch-only* if and only if the wallet holds its full viewing key, or for a transparent TXO its address, but not its spending key;
- (v) a transparent UTXO is *confirmed-shieldable* if and only if it is known-spendable and the wallet can construct a correct redeem script for it (ZIP 315, “Transparent UTXOs”).

The following rule is specified: a wallet MUST NOT attempt to spend, in a user-initiated transaction, a TXO that is not confirmed-spendable. ZIP 315 notes that the confirmed-spendable balance is 0 until the wallet has synchronised at least the nullifier set to the tip (ZIP 315, “Reporting of balances”).

Definition 9.17 (Spendable note at a target height). Let T be the tip, $H = T + 1$, $A = H - c_{\text{tr}}$ (Definition 9.5), and f the fully scanned height (Definition 6.11). A shielded note n of pool P , mined at height m , is *spendable at H* if and only if:

- (a) the note n is in the note state (Definition 6.7), it is not marked spent, and $f \geq T$;
- (b) it is not locked at H (Definition 9.12);
- (c) the wallet holds the spending key of the account of n , or mediates a signer for it;
- (d) it meets its depth: $T - m + 1 \geq c_{\text{tr}}$ or $T - m + 1 \geq c_{\text{untr}}$ according to its trust (Definition 9.2), or, if n is an output of a wallet-internal shielding transaction, the rule for shielding outputs of §9.1;
- (e) the authentication path of the position of n to the anchor of P at A is computable from retained information (Theorem 7.14, Proposition 7.17).

The *spendable set* at H is the set of notes spendable at H , each with its value, pool, position and authentication path at A .

Condition (d) implies $m \leq \mathcal{A}$ in each case, since $c_{\text{untr}} \geq c_{\text{tr}}$ and the shielding transaction has at least c_{tr} confirmations, so n lies in the tree of the anchor by Lemma 9.6. The spendable set at H is the set of confirmed-spendable shielded notes of Definition 9.16, with ZIP 315’s “reasonably up-to-date” instantiated as $f \geq T$; the instantiation is designed but unspecified (Definition 1.1). Clause (a) is clause (1) of known-spendability: with $f \geq T$, every block up to the tip has been scanned, and a note is marked spent exactly when its nullifier is revealed on the best chain at or below T (Theorem 6.14, item (ii)). Clause (b) is clause (2) by Proposition 9.13, with condition (d) of Definition 9.12 in place of the expiry height where that is unknown; clause (c) is clause (3); clauses (e) and (d) are the witness and depth clauses of confirmed-spendability. The spendable set is the input state of construction (“Transaction construction”, §11): it is reconstructed by synchronisation (§6, Theorem 6.14) and taken as given by fee computation and note selection.

Remark 9.18 (Spending before the nullifier set is complete). A policy that admits a note once the block extent of its own subtree is scanned up to $\mathcal{A}$ (Definition 7.16), before every range up to T is scanned, departs from clause (a) of Definition 9.17 and from ZIP 315, whose clause on an up-to-date view carries an open TODO in the ZIP. The departure fails as follows. A note whose spend was mined in a range not yet scanned passes the policy; a transaction that spends it again is rejected by the consensus rule that a nullifier must not repeat (*Ironwood Guide*, §“Nullifier sets”), and its repeated nullifier links it to the earlier spend for every party that sees it. The departure is designed but unspecified (Definition 1.1).

10 Fees (ZIP 317)

This section states the fee a wallet pays. It fixes the fee as a consensus quantity, defines the conventional fee of ZIP 317 over logical actions and bounds it before signing; it states the padded Action counts on which the fee is computed; and it states the relay, mempool and block-template rules that apply to a transaction paying less.

10.1 The conventional fee

For a transaction other than a coinbase transaction, the *transaction fee* is the value remaining in its transparent transaction value pool: the sum of its transparent input values and of the balancing values of its shielded pools, minus the sum of its transparent output values (protocol specification, §“Transactions and Treestates”; *Ironwood Guide*, Definition “Transparent transaction value pool” and Definition “Balancing value”, in “The balancing value”). Consensus requires it to be non-negative. No transaction field encodes it, and every term of the sum is public, so the fee is public.

A fee rule for a shielded protocol meets the requirements of ZIP 317, “Requirements”: the fee is computable from the public data of the transaction alone, since a fee that depended on private construction data, such as which notes were spent or what change was returned, would disclose that data; it scales linearly with the numbers of inputs and outputs, which makes transaction-rate denial of service costly (ZIP 317, “Motivation” and “Denial of Service”); it neither penalises padding for privacy nor favours any pool; and it lets a small number of inputs of value below the marginal fee be spent. A fee other than the one every wallet pays distinguishes the wallet that chose it: non-standard fees may reveal specific users, wallets or wallet versions (ZIP 317, “Security and

Privacy considerations”).

The conventional fee (Definition 10.2) is not a consensus rule. ZIP 317, “Fee calculation”, states that it is not a consensus requirement that fees follow the formula, and that wallets SHOULD create transactions that pay it, to reduce information leakage, unless overridden by the user. It binds through relay and block-template policy only (§10.3). ZIP 259, “NU7 consensus changes”, changes neither the formula nor its parameters.

In what follows, $\lceil x \rceil$ is the least integer $\geq x$ and $\lfloor x \rfloor$ the greatest integer $\leq x$, for real x .

Definition 10.1 (Logical actions). Let a transaction of version 5 or version 6 have the following public fields of its final encoding (protocol specification, §“Transaction Encoding and Consensus”; ZIP 229, “Transaction Format”), each a non-negative integer:

- $tx_in_total_size$ and $tx_out_total_size$, the byte lengths of the encoded fields tx_in and tx_out , the concatenations of the encoded transparent inputs and outputs;
- $nJoinSplit$, the number of Sprout JoinSplit descriptions;
- $nSpendsSapling$ and $nOutputsSapling$, the numbers of Sapling spend and output descriptions;
- $nActionsOrchard$, the number of Orchard-pool Actions;
- $nActionsIronwood$, the number of Ironwood-pool Actions, 0 in a version 5 transaction.

With the parameters $p2pkh_standard_input_size := 150$ and $p2pkh_standard_output_size := 34$, in bytes, the number of *logical actions* of the transaction is

$$\begin{aligned} logical_actions := & \max(\lceil tx_in_total_size / 150 \rceil, \lceil tx_out_total_size / 34 \rceil) \\ & + 2\ nJoinSplit + \max(nSpendsSapling, nOutputsSapling) \\ & + nActionsOrchard + nActionsIronwood, \end{aligned}$$

the sum of the transparent, Sprout, Sapling, Orchard and Ironwood contributions, in that order (ZIP 317, “Fee calculation”, the contributions of Revision 0 and Revision 1).

The Sprout contribution $2\ nJoinSplit$ is 0 in every valid transaction: transactions of version 5 and version 6 have no JoinSplit field, and transactions of version 4 are invalid under NU7 (ZIP 2003; ZIP 259, “NU7 consensus changes”). The term is kept so that the definition is that of ZIP 317. The two divisors are the maximal encoded sizes of a P2PKH input and output (ZIP 317, “Rationale for the chosen parameters”):

$$\begin{aligned} 150 &= 36 \text{ (outpoint: 32-byte txid, 4-byte index)} + 110 \text{ (script)} + 4 \text{ (sequence)}, \\ 110 &= 1 \text{ (length)} + 1 \text{ (signature length)} + 72 \text{ (signature)} + 1 \text{ (sighash type)} \\ &\quad + 1 \text{ (key length)} + 33 \text{ (compressed key)} + 1 \text{ (margin)}, \\ 34 &= 8 \text{ (value)} + 1 \text{ (script length)} + 25 \text{ (P2PKH script)}. \end{aligned}$$

Charging transparent parts by size rather than by count charges a large script in proportion to its

size; a 2-of-3 P2SH multisig input, about 70% larger than a P2PKH input, is counted so (same section).

Definition 10.2 (Conventional fee). Let $\text{marginal_fee} := 5000$ zatoshi per logical action and $\text{grace_actions} := 2$. The *conventional fee* of a transaction with logical_actions as in Definition 10.1 is

$$\text{conventional_fee} := \text{marginal_fee} \cdot \max(\text{grace_actions}, \text{logical_actions}) \text{ zatoshi}$$

(ZIP 317, “Fee calculation”).

The parameter values and their rationale are ZIP 317’s (“Rationale for the chosen parameters”).

Remark 10.3 (Revisions of the fee rule). ZIP 317, “Fee calculation”, defines logical_actions as the sum of the contributions of the applicable revision, and a contribution defined by a revision is included in every later one (ZIP 317, “Revisions”). Revision 0 defines the transparent, Sprout, Sapling and Orchard contributions. Revision 1 adds the Ironwood contribution $n\text{ActionsIronwood}$: Ironwood-pool Actions have the Action structure of Orchard-pool Actions and are counted identically. Revision 2 adds a contribution for memo bundles (ZIP 231), defined on the transaction format of ZIP 248, which NU7 does not introduce (ZIP 259, “Rationale for no new transaction format”). Definition 10.1 is therefore Revision 0 with the Revision 1 term. The statuses of these ZIPs are those of Table 1.

Corollary 10.4 (The minimum fee). *Every transaction with $\text{logical_actions} \leq 2$ has conventional fee $\text{marginal_fee} \cdot \text{grace_actions} = 5000 \cdot 2 = 10,000$ zatoshi, and no transaction has a smaller conventional fee.*

Proof. By Definition 10.2, since $\max(2, \text{logical_actions}) = 2$ when $\text{logical_actions} \leq 2$, and $\max(2, \text{logical_actions}) \geq 2$ always. $\square$

A wallet computes the fee before it signs, and the encoded size of a transparent input depends on its signature. The following proposition bounds the fee from the unsigned transaction.

Proposition 10.5 (Fee estimate before signing). *Let a wallet fix, before signing, every field of a transaction except the scripts of its transparent inputs $1, \dots, k$, and assign to each input i a bound $b_i \in \mathbb{N}$ on its encoded size after signing, as follows:*

- (i) $b_i = 150$ for a P2PKH input spending a public key with a compressed 33-byte encoding, such as each of the wallet’s own transparent keys;
- (ii) otherwise any b_i known to bound the encoded size of the signed input.

Let fee_{est} be the conventional fee computed with tx_in_total_size replaced by $\sum_{i=1}^k b_i$. Then fee_{est} is at least the conventional fee of the signed transaction.

Proof. Signatures enter only the scripts of the transparent inputs, so the field `tx_out` and every shielded count of Definition 10.1 are the same in the unsigned and the signed transaction. A DER-encoded signature has at most 72 bytes, so the script of a P2PKH input with a compressed key has at most $1 + 1 + 72 + 1 + 1 + 33 = 109$ bytes and the input at most $36 + 109 + 4 = 149 \leq 150$ bytes; under clause (ii) the bound holds by hypothesis. The signed `tx_in_total_size` is therefore at most $\sum_i b_i$. The maps $x \mapsto \lceil x/150 \rceil$, $(a, b) \mapsto \max(a, b)$, the addition of the unchanged remaining contributions, and $n \mapsto 5000 \cdot \max(2, n)$ are each non-decreasing in every argument, so their composition is, and fee_{est} is at least the conventional fee of the signed transaction. $\square$

Hypothesis (i) does not extend to a P2PKH input with an uncompressed 65-byte key encoding, which has up to 181 bytes and needs $b_i \geq 181$ under (ii). For an input with no known bound on its signed size, the fee is unknown before signing.

10.2 The fee on padded counts

The fields of Definition 10.1 are those of the final encoded transaction: every bundle padded and every change output included (ZIP 317, “Fee calculation”). A wallet therefore prices a transaction by predicting its padded shape (Definition 10.6) and its change before it selects inputs; a fee computed on unpadded counts underpays whenever padding adds a logical action. This rule is ZIP 317’s and is not a consensus rule.

Definition 10.6 (Padded Action count). Let a bundle of the Orchard protocol, in the Orchard pool or the Ironwood pool, have $s \in \mathbb{N}$ real spends and $o \in \mathbb{N}$ real outputs, and let `enableCrossAddress` $\in \{0, 1\}$ be its flag value (*Ironwood Guide*, §“Bundle flags”, Definition “Flag values and expanded receiver”). Its *requested Action count* is

$$r(s, o) := \begin{cases} \max(s, o) & \text{if enableCrossAddress} = 1, \\ s + o & \text{if enableCrossAddress} = 0, \end{cases}$$

and its *padded Action count* is $n(s, o) := 0$ if $s = o = 0$ and $n(s, o) := \max(r(s, o), 2)$ otherwise. The counts `nActionsOrchard` and `nActionsIronwood` of Definition 10.1 are the padded Action counts of the two bundles. A non-empty Sapling bundle with s real spends and o real outputs has `nSpendsSapling` $= s$ and `nOutputsSapling` $= \max(o, 2)$.

The count $s + o$ is that of the pairing a wallet performs when cross-address transfers are disabled: the note an Action creates has the expanded receiver of the note it consumes, so each real spend occupies an Action with a fabricated zero-valued output at its own receiver, and each real output an Action with a fabricated zero-valued spend at its receiver (ZIP 326, “Fabricated same-address outputs and randomized note ciphertexts”; ZIP 374, “Fabricated same-address outputs (NU6.3)”), respectively. Every Orchard-pool bundle has `enableCrossAddress` $= 0$ (*Ironwood Guide*, “Chain state and pool rules”, rule (i); ZIP 258, “Consensus rules from NU6.3 activation”); an Ironwood-pool bundle may set it to 1.

The floor 2 and the Sapling padding are a padding policy, designed but unspecified in the sense of Definition 1.1. Their only general ZIP anchor is non-normative: ZIP 317, “Requirements”,

notes that standard builders pad to two inputs and two outputs for each shielded pool, and “Rationale for the chosen parameters” calls padding to two actions the standard approach; ZIP 318 fixes the padding of its migration and note-preparation transactions only (below). The Sapling rule departs from ZIP 317’s remark by adding no spend beyond those needed. Dummy Sapling spends, which this policy does not use, are constructed as in protocol specification, §“Dummy Notes (Sapling)”; the dummy Actions of the Orchard protocol are those of §11.7. Logical actions count an Action once: a charge per input plus per output would penalise the padding that the Action structure forces, since each Action consumes one note and creates one, and the maximum in each contribution favours no pool (ZIP 317, “Rationale for logical actions”).

ZIP 318 specifies two exceptions. Under “Canonical migration transaction structure”, a canonical migration transaction has one Orchard-pool spend in an Orchard-pool bundle of exactly two Actions, one spend Action and one output Action that carries either change or a zero-valued dummy output (ZIP 318, “Rationale for the canonical transaction shape”), and one Ironwood-pool output in an unpadded Ironwood-pool bundle of one Action. Its *logical_actions* is 3 and its conventional fee 15,000 zatoshi. The “canonical fee (provisionally the ZIP 317 minimum fee)” of ZIP 318 is read here as this conventional fee: read as 10,000 zatoshi, it leaves one unpaid action (Definition 10.8). A note-preparation transaction SHOULD have an Orchard-pool bundle of exactly 16 Actions, padded as needed (ZIP 318, “Note preparation transactions”), so its count is 16, not $n(s, o)$.

Proposition 10.7 (Fee of a single-pool payment). *Consider a transaction with no transparent inputs or outputs, one input note and one payment output, with or without one change output, whose bundles are padded as in Definition 10.6.*

- (a) *If the input note and every output lie in the Ironwood pool, in a bundle with $\text{enableCrossAddress} = 1$, or all lie in the Sapling pool, the conventional fee is 10,000 zatoshi.*
- (b) *If the input is a single Orchard-pool note, the payment output lies in the Ironwood pool, and the change output, if any, lies in the Orchard pool or the Ironwood pool, then the conventional fee is 20,000 zatoshi.*

Proof. (a) In the Ironwood pool $s = 1$ and $o \in \{1, 2\}$, so $r = \max(1, o) \leq 2$ and $n = 2$. In the Sapling pool the contribution is $\max(1, \max(o, 2)) = 2$. In either case *logical_actions* = 2, and Corollary 10.4 gives 10,000 zatoshi.

(b) No value enters the Orchard pool, since $\text{valueBalanceOrchard} \geq 0$ for every transaction (*Ironwood Guide*, “Chain state and pool rules”, rule (ii); ZIP 258, “Consensus rules from NU6.3 activation”), and an Orchard-pool output lies at the receiver of the note consumed in its Action; so a payment to another party cannot lie in the Orchard pool, and the full routing rule is that of §11.2. The payment lies in the Ironwood pool by hypothesis. The Orchard-pool bundle has $s = 1$ and $o \in \{0, 1\}$, so $r = s + o \leq 2$ and $n = 2$. The Ironwood-pool bundle has $s = 0$ and $o \in \{1, 2\}$, so $n = 2$. Hence *logical_actions* = 4 and the conventional fee is $5000 \cdot 4 = 20,000$ zatoshi. $\square$

10.3 Relay, mempool, and block template

The conventional fee binds through three mechanisms, none of them a consensus rule. Nodes that normally relay transactions are expected to relay those paying at least the conventional fee, unless other reasons of robustness or denial-of-service mitigation apply (ZIP 317, “Transaction relaying”). Mempool size limiting adds a penalty *low_fee_penalty* to the eviction weight of a transaction paying less than the conventional fee (ZIP 401; ZIP 317, “Mempool size limiting”). Block-template construction limits the unpaid actions of a block (Definition 10.8).

Definition 10.8 (Unpaid actions). For a transaction tx with fee $tx.fee$ as in §10.1 and $tx.logical_actions$ as in Definition 10.1, the number of *unpaid actions* is $unpaid_actions(tx) := 0$ if tx is a coinbase transaction, and otherwise

$$unpaid_actions(tx) := \max(0, \max(grade_actions, tx.logical_actions) - \lfloor tx.fee / marginal_fee \rfloor).$$

The unpaid actions of a block are the sum over its transactions. The recommended block-template algorithm adds a transaction that pays less than the conventional fee only while that sum stays at most $block_unpaid_action_limit = 50$ (ZIP 317, “Recommended algorithm for block template construction”).

Lemma 10.9 (Zero unpaid actions). *A transaction other than a coinbase transaction has $unpaid_actions = 0$ if and only if its fee is at least its conventional fee.*

Proof. Let $m := \max(grade_actions, logical_actions)$, an integer. By Definition 10.8, $unpaid_actions = 0$ if and only if $\lfloor fee / marginal_fee \rfloor \geq m$. For an integer m and real x , $\lfloor x \rfloor \geq m$ if and only if $x \geq m$; and since $marginal_fee > 0$, $fee / marginal_fee \geq m$ if and only if $fee \geq marginal_fee \cdot m$, which is the conventional fee of Definition 10.2. $\square$

A transaction with more than $block_unpaid_action_limit = 50$ unpaid actions is never included by the recommended algorithm, and nodes MAY drop it, or any transaction with more unpaid actions than a configurable limit (ZIP 317, “Transaction relaying”). A relay policy may set that limit to 0; by Lemma 10.9 it then drops exactly the transactions that pay less than the conventional fee. The conventional fee is therefore the least fee that passes every unpaid-action limit, whatever its value. For example, the migration transaction of §10.2 with $logical_actions = 3$ has one unpaid action at a fee of 10,000 satoshi.

A logical action is not the unit of the block limits of ZIP 218, “Shielded pool action limits”: these count the sum $nSpendsSapling + nOutputsSapling$ of a block’s Sapling descriptions, not the maximum that Definition 10.1 charges, so paying the conventional fee does not ensure that a transaction fits a given block (*Consensus Guide*, “A block budget for shielded work”). The limits add no transaction field. ZIP 259, “Rationale for no new transaction format”, refers to a block-level fee contribution that needs no transaction field; no published ZIP specifies that calculation, so it is designed but unspecified (Definition 1.1), and the conventional fee does not depend on it.

11 Transaction construction

This section states how a wallet turns payments into transactions. It defines the transaction plan and its balance equation; the pool in which each payment and each change output lies; transfers to TEX addresses and shielding transactions; input selection as a fixed point of the fee; change; the outgoing viewing key of every output; the assembly of Orchard-protocol bundles with dummy Actions in uniform order; and the authorisation of a built transaction, with the theorem that the result pays what the plan states. The input state is the spendable set of Definition 9.17; the fee is the conventional fee of §10.

11.1 The transaction plan

Definition 11.1 (Payment and payment request). A *payment* is a transfer of funds implemented by a single output of a transaction: a triple (a, v, m) of a recipient address a (Definition 3.20), a value $v \in \mathbb{N}$ in zatoshi and an optional memo m , a memo being admitted only when a is memo-capable. A *payment request* is a finite map R from indices to payments; it asks for one transaction that pays every payment in its range (ZIP 321, “Terminology”). ZIP 321 encodes payment requests as URIs (§14.1); the label and message parameters of that encoding are display data and enter no transaction.

Construction has two phases. *Planning* maps a payment request and the wallet state, its spendable notes (Definition 9.17) and its tip, to a plan exact to the zatoshi, and uses no spending key. *Execution* builds, proves and authorises the transactions of the plan and records them (§13); submission is a separate act. Every choice of inputs, fee and change is fixed in the plan before a spending key is used. No ZIP defines plans: the two-phase structure is designed but unspecified (Definition 1.1).

Definition 11.2 (Transaction plan). A *transaction plan* is a non-empty sequence $P = (S_1, \dots, S_\ell)$ of *steps*. Each step describes one transaction by:

- (i) its target height H (Definition 9.4), anchor height A (Definition 9.5) and expiry height N (Definition 9.8);
- (ii) its *spending account*, the account of the wallet to which every input of the step belongs;
- (iii) its inputs, partitioned by pool (transparent, Sapling, Orchard, Ironwood), each either a note of the spending account spendable at H (Definition 9.17), a transparent UTXO of that account that meets its depth (§9.1), or a reference (j, i) to output i of an earlier step S_j ;
- (iv) its payment outputs, each a payment of the request together with the pool assigned to it (§11.2);
- (v) its change outputs, each a value and a pool;
- (vi) its fee $f \in \mathbb{N}$, computed under the conventional fee rule (Definition 10.2).

A transparent output of a step may be marked *ephemeral*: it pays a fresh transparent address of the wallet and exists only to be spent by a later step (§11.3). Each step satisfies the *balance equation*

$$\sum_{\text{inputs}} v = \sum_{\text{payments}} v + \sum_{\text{change}} v + f,$$

the sums taken over all pools, ephemeral outputs counted among the payments of their step. In the built transaction f is the value remaining in the transparent transaction value pool: the sum of the transparent inputs, minus the transparent outputs, plus every shielded balancing value (*Ironwood Guide*, §“The balancing value”, Definitions “Transparent transaction value pool” and “Balancing value”).

Clause (ii) restricts this section to steps funded by one account; ZIP 316 also admits transfers whose sending account is undetermined, a case that enters only the choice of outgoing viewing key (§11.6).

Definition 11.3 (Well-formed plan). A plan $(S_1, \dots, S_\ell)$ is *well formed* if and only if

- (i) every reference (j, i) among the inputs of step S_k has $j < k$;
- (ii) every output of an earlier step that a step spends is ephemeral; no step spends a change output of an earlier step;
- (iii) no output, of the wallet or of a step, is spent twice within the plan;
- (iv) every ephemeral output of a step is spent by exactly one later step.

Proposition 11.4 (Chaining through transparent outputs). *Let step S_k of a plan spend output i of an earlier step S_j , and let t_j be the transaction built for S_j .*

- (a) *If the output is transparent, every field of the transaction of S_k is computable once t_j is built, before t_j is mined.*
- (b) *If the output is a shielded note of non-zero value, the proof that covers its spend (its Sapling spend proof, or the proof of its Orchard-protocol bundle) is not computable before t_j is mined.*
- (c) *If the output is an Orchard-protocol note and the transaction of S_k has version 6, its signature digest, and hence every spend-authorisation and binding signature of it, is computable once t_j is built; only proving waits for the mining of t_j .*
- (d) *If the output is a Sapling note, the signature digest of the transaction of S_k is not computable before t_j is mined.*

A plan whose steps are all completed before any of them is submitted therefore passes value between steps only through transparent outputs, which Definition 11.3 (ii) and the ephemeral outputs of Definition 11.2 realise.

Proof. (a) A transparent input names the outpoint $(\text{txid}(t_j), i)$ and is signed over a digest that commits to the value and script of the spent coin (ZIP 244, “S.2: transparent_sig_digest”). The

identifier is a function of the effecting data of t_j (*Ironwood Guide*, §“Transaction digests and signatures”, Definition “Transaction identifier”), and value and script are fixed by S_j ; none depends on the mining of t_j .

(b) A proof for a spend of a note of non-zero value needs the note’s authentication path to the anchor (*Ironwood Guide*, §“Authentication paths”; condition A3 of the Action statement, which exempts only a consumed note of value 0). The path is a function of the note’s position in its tree, which is fixed only when t_j is mined (Definition 4.11).

(c) The version-6 signature digest commits to no anchor and to no proof (ZIP 229, “Anchor commitment (version 6)”; *Ironwood Guide*, §“Transaction digests and signatures”, Definition “Authorising-data digest; effecting and authorising data” and Definition “Signature digest”). The effecting data of an Orchard-protocol spend are computable before mining: its nullifier is $\text{DeriveNullifier}_{\text{rk}}(\rho, \psi, \text{cm})$, a function of note data and not of the position (*Ironwood Guide*, §“The nullifier”), and ρ of the note is fixed by the Action of t_j that creates it; the key rk and the commitment cv^{net} depend on keys, values and fresh randomness. ZIP 374, “Anchors and pre-authorization”, states this workflow, and ZIP 318, “Note preparation transactions”, uses it.

(d) A Sapling nullifier is computed from

$$\rho = \text{MixingPedersenHash}(\text{cm}, \text{pos}),$$

where pos is the note’s position (protocol specification, §“Computing ρ values and Nullifiers”), and it is effecting data, committed by the signature digest (ZIP 244, “Signature Digest”).

For the final claim: by (b) and (d), a step that spends a shielded note of non-zero value of an earlier step cannot be completed before that step’s transaction is mined, so a plan completed as a unit passes value only through transparent outputs, which (a) admits; a zero-valued Orchard-protocol note carries no value. $\square$

The partially created form of (c) is that of §12.5. Fee and change are fixed jointly: the change outputs enter the padded counts of Definition 10.6 and hence the conventional fee, and the fee determines the change value; §11.5 resolves the dependence.

Execution is not deterministic. The note seeds rseed , hence esk , ψ and rcm , the value-commitment trapdoors rcv , the randomisers α , the proofs and the permutations of §11.7 are drawn fresh (*Ironwood Guide*, §“Construction of a transaction”), so one plan executed twice yields transactions with different effecting data and different transaction identifiers. A plan fixes values and shape, not the transaction identifier.

11.2 Payments to pools

Construction 11.5 (Output pool of a payment). The output of a payment (a, v, m) is determined by a :

- (i) a transparent address yields a transparent output to it;
- (ii) a TEX address yields a transparent P2PKH output of the second transaction of a ZIP 320 transfer (§11.3);

- (iii) a Sapling address yields a Sapling output;
- (iv) a unified address yields an output to its receiver of the most preferred type in the Priority List of §3.3 that the sender supports, which ZIP 316 makes a MUST (ZIP 316, “Encoding of Unified Addresses”); an Orchard receiver is paid by an Ironwood-pool output;
- (v) an address with no receiver that the sender supports is refused, and so is a payment with a memo whose output is transparent.

Case (iv) is sound because both pools of the Orchard protocol share keys, addresses and receivers (*Ironwood Guide*, §“The Orchard protocol and its two pools”; ZIP 326, “Scanning for Ironwood-pool and Orchard-pool notes” and “Receiving funds into the Ironwood pool”).

Orchard-pool rules. Consensus admits no value into the Orchard pool: for every transaction, $\text{valueBalanceOrchard} \geq 0$; and every Orchard-pool Action has cross-address transfers disabled, so the note it creates lies at the expanded receiver of the note it consumes (ZIP 258, “Consensus rules from NU6.3 activation”; *Ironwood Guide*, §“Chain state and pool rules”, rules (i) and (ii), and §“Bundle flags”). A wallet MUST NOT send funds to any external receiver, its own included, in the Orchard pool (ZIP 326, “Wallet key-generation restrictions”), and MUST NOT use its knowledge of the internal addresses of all its accounts to send funds from one account to an internal address of another (ZIP 326, “Scanning for Ironwood-pool and Orchard-pool notes”). A wallet that generates $\text{use_qsk} = \text{true}$ keys MUST NOT send Orchard-pool funds to them at any point (ZIP 326, “Wallet key-generation restrictions”). This volume reads “external receiver” as every receiver that is not an internal-scope receiver (Construction 2.14) of an account of the wallet.

Change. A change output lies at an internal-scope address of the spending account (§11.5) in the Ironwood pool, in the Sapling pool, or, only as Proposition 11.6 allows, in the Orchard pool. The choice among these is wallet policy, designed but unspecified (Definition 1.1); a change output in a pool other than that of the inputs moves value between pools, and the moved amount is published in the balancing values.

Proposition 11.6 (Orchard-pool change). *In a transaction built by a wallet that follows Construction 11.5, every Orchard-pool output of non-zero value lies at the expanded receiver of the note consumed by its own Action, a real note or a fabricated zero-valued note, and that receiver is an internal-scope receiver of the spending account. Hence no payment output lies in the Orchard pool; non-zero change remains in the Orchard pool only at internal-scope receivers of the spending account, with total at most the value of the Orchard-pool notes spent; any other change lies in the Ironwood or the Sapling pool.*

Proof. With $\text{enableCrossAddress} = 0$, condition A10 of the Action statement (*Ironwood Guide*, §“Bundle flags” and §“Chain state and pool rules”, rule (i)) places each Orchard-pool output at the expanded receiver of the note consumed in its Action. By the first rule of ZIP 326 under the reading of Construction 11.5, an output of non-zero value lies at an internal-scope receiver of an account of the wallet, and by the second that account is the spending account. A payment output is routed by cases (i) to (iv) of Construction 11.5, none of which yields an Orchard-pool output. Let V_{in} and V_{out} be the totals of the consumed and created Orchard-pool notes; the balancing

value of the Orchard pool is $V_{\text{in}} - V_{\text{out}} \geq 0$ (rule (ii)), so the Orchard-pool change, a part of V_{out} , is at most V_{in} . Every other change output lies, by Construction 11.5, in the Ironwood or the Sapling pool. $\square$

The following consequences are cited later rather than restated. Orchard-pool bundles are priced by Definition 10.6 with the requested count $s + o$; Orchard-pool fabricated outputs and spends are assembled in §11.7; and the canonical migration and note-preparation transactions of ZIP 318 are the exceptions to the padding floor stated in §10.2.

One Orchard-protocol spending key authorises spends in both Orchard-protocol pools. Ironwood-pool and Orchard-pool spends use the same full viewing key and the same spend authorising key ask, each pool’s bundle with its own note commitment tree, anchor and nullifier set (*Ironwood Guide*, §“Key capabilities and scope”, §“The Orchard protocol and its two pools” and §“Chain state and pool rules”); the two bundles of one transaction are built, proved and signed side by side.

Migration is specified. ZIP 2005 states that wallets SHOULD move all funds they control, transparent, Sprout and Sapling funds included, into recoverable Ironwood-pool notes as soon as practically possible, and that other wallets are REQUIRED to be able to receive them (ZIP 2005, “Proactive movement of funds to recoverable notes”). ZIP 318 specifies a scheduled migration from the Orchard pool to the Ironwood pool; its scheduling rules and privacy analysis are cited, not restated (ZIP 318, “Specification” and “Privacy Implications”). Its block-count constants are stated for a 75-second block spacing; their treatment under the 25-second spacing of ZIP 218, which NU7 deploys (ZIP 259, “NU7 consensus changes”), is an open problem (Definition 1.1). The privacy of the choice of source pool is stated in §11.4.

11.3 TEX transfers and shielding transactions

Construction 11.7 (Transfer to a TEX address). A sender paying one or more TEX addresses from shielded funds MUST create two transactions (ZIP 320, “Design considerations for Senders”), realised as two steps of one plan.

- (1) The first transaction tr_0 spends shielded notes only and pays, in place of the n TEX payments of values $v_1, \dots, v_n$, one ephemeral transparent output of value

$$v_{\text{eph}} := \sum_{i=1}^n v_i + \text{fee}(tr_1)$$

to a fresh ephemeral transparent address of the wallet.

- (2) The second transaction tr_1 spends exactly that output and pays the n TEX payments as P2PKH outputs (*Crypto Guide*, §“The transparent layer’s hashes”), with no memo and no change, so that its input equals its outputs plus its fee.

Here $\text{fee}(tr_1)$ is the conventional fee of one P2PKH input priced at 150 bytes (Proposition 10.5 (i), the ephemeral key being a compressed key of the wallet) and n P2PKH outputs of 34 bytes each:

$$\text{fee}(tr_1) = 5000 \cdot \max(2, \max(\lceil 150/150 \rceil, \lceil 34n/34 \rceil)) = 5000 \cdot \max(2, n) \text{ zatoshi},$$

which is 10,000 zatoshi for $n = 1$ and $n = 2$ and 15,000 zatoshi for $n = 3$. A single transaction that pays a TEX address and spends shielded inputs is refused: a sender to a TEX address **MUST** spend only transparent (P2SH or P2PKH) UTXOs in that transaction, and **SHOULD** pay only transparent outputs in it (ZIP 320, “Specification”). ZIP 320 does not require that those UTXOs come from a single transparent address.

The plan of Construction 11.7 is well formed (Definition 11.3): its one reference points from tr_1 to the ephemeral output of tr_0 , which is spent exactly once. By Proposition 11.4 (a) both transactions are completed before either is mined, and ZIP 315 exempts the ephemeral UTXO from the confirmation floor (§9.1).

Ephemeral addresses are specified. A wallet based on ZIP 32 **SHOULD** choose ephemeral addresses so that their private keys are recoverable from the seed, **SHOULD NOT** choose them so that they can be linked between transactions without the seed or the relevant transparent viewing keys, and so **SHOULD** avoid collisions with the addresses of earlier outputs, change outputs included; a wallet **MUST** recognise, and be able to spend, funds that a recipient returns to an ephemeral address (ZIP 320, “Design considerations for Senders”). An ephemeral address reserved for a construction that fails is not reused, since reuse would link the two transactions; this rule is designed but unspecified (Definition 1.1).

Construction 11.8 (Shielding transaction). A *shielding transaction* is the transaction of a one-step plan with an empty payment request. It spends transparent UTXOs of the spending account and pays their total, less the conventional fee, to change outputs at an internal-scope shielded receiver of that account (Construction 2.14), with no other recipient and no transparent output. Its outputs lie in the Ironwood pool, since no value enters the Orchard pool (Construction 11.5) and ZIP 2005 asks for recoverable Ironwood-pool notes (ZIP 2005, “Proactive movement of funds to recoverable notes”); its balancing value in that pool is negative. Its UTXOs may be spent with zero confirmations (§9.1). The value threshold at which a wallet shields is wallet policy, designed but unspecified (Definition 1.1).

Shielding rules are specified. A wallet that receives transparent funds **SHOULD** auto-shield them by default; it **SHOULD NOT** auto-shield from several transparent addresses in one transaction, and **SHOULD NOT** use *opportunistic shielding*, the shielding of received transparent funds within a user-initiated transaction, which would show its recipients the link between the transparent addresses and the payment (ZIP 315, “Long-term storage of funds”). Users **MUST** be able to disable auto-shielding (ZIP 315, “Auto-shielding”). Spent transparent UTXOs **SHOULD** be sent only to an internal shielded receiver of the wallet, except the ephemeral outputs of a ZIP 320 transfer (ZIP 315, “Linkability of transactions or addresses”).

Transparent inputs are public, so a transaction that spends UTXOs of different transparent receivers links those receivers. ZIP 315 states that UTXOs received on different transparent receivers **SHOULD NOT** be shielded in one transaction, and that shielding several UTXOs of one address together leaks nothing further (ZIP 315, “Linkability of transactions or addresses”). The same holds for ephemeral addresses, so a transaction spends the ephemeral output of at most one ZIP 320 transfer.

11.4 Input selection

Definition 11.9 (Covering selection). Fix a step with payments of total out , target height H , anchor height A and a number k of change outputs (§11.5 fixes k). Let U be the finite set of inputs of the spending account that Definition 11.2 (iii) admits at (H, A) , each $u \in U$ with value $v(u)$, and let $\mathcal{S} \subseteq 2^U$ be the family of *admissible* selections, those that meet the pool rules stated at the end of this subsection. For $I \in \mathcal{S}$ let $shape(I)$ be the public counts of Definition 10.1 of the step funded by I , with its payments and k change outputs, after padding (Definition 10.6), and let $fee(I) := \text{conventional_fee}(shape(I))$ (Definition 10.2). A selection $I \in \mathcal{S}$ is *covering* if and only if

$$\sum_{u \in I} v(u) \geq out + fee(I).$$

Adding an input never lowers a count of Definition 10.1, so fee is non-decreasing under inclusion, and a covering selection is a fixed point of the fee: its total must cover a fee that depends on it. A plan requires a covering selection for each step; any procedure that returns one conforms. The following construction is one such procedure.

Construction 11.10 (Fee escalation). Set $I_0 := \emptyset$ and $t_0 := 0$. At round $k \geq 0$:

- (1) compute the balance of the step on I_k (§11.5 gives its exact form);
- (2) if I_k is covering, return I_k ;
- (3) otherwise let $r_k := out + fee(I_k) - t_k > 0$ be the shortfall, and choose $I_{k+1} \in \mathcal{S}$ with total $t_{k+1} := \sum_{u \in I_{k+1}} v(u) \geq t_k + r_k$; if no such I_{k+1} exists, fail with insufficient funds.

The choice of I_{k+1} among the admissible sets that step (3) allows is the wallet's.

Proposition 11.11 (Termination of fee escalation). *Fee escalation terminates after finitely many rounds. On success the returned selection is covering, and the fee of the step is the conventional fee of its final padded shape. On failure at round k , no admissible selection has total at least $out + fee(I_k)$. No bound on the number of rounds is claimed beyond this finiteness.*

Proof. A round that does not stop has $t_{k+1} \geq t_k + r_k > t_k$, so the totals increase strictly. Every t_k lies in the finite set $\{\sum_{u \in I} v(u) : I \in \mathcal{S}\} \cup \{0\}$, which admits no infinite strictly increasing sequence. On success the returned I_k is covering by step (2), and its fee is $fee(I_k)$, the conventional fee of $shape(I_k)$ by Definition 11.9. On failure, step (3) found no admissible set of total at least $t_k + r_k = out + fee(I_k)$. $\square$

A failure does not show that no covering selection exists: an admissible set J with $fee(J) < fee(I_k)$ and total in $[out + fee(J), out + fee(I_k))$ covers. Fee escalation is therefore sound, returning only covering selections, and complete only for choice rules that exclude this case.

Definition 11.12 (Uneconomic input). An input u is *uneconomic* if and only if $v(u) < \text{marginal_fee}$, where $\text{marginal_fee} = 5000$ zatoshi (Definition 10.2). The strict

inequality is ZIP 317's wording, "below the marginal fee" (ZIP 317, "Requirements" and "Rationale for the chosen parameters").

Lemma 11.13 (Cost of an additional input). *Let L be the number of logical actions of a step (Definition 10.1) and L' that after adding one input.*

- (i) *A P2PKH input priced at 150 bytes gives $L' - L \in \{0, 1\}$.*
- (ii) *A shielded spend added to a non-empty bundle with s real spends and o real outputs, $s + o \geq 1$, gives $L' - L \in \{0, 1\}$, and leaves the bundle's padded count unchanged if and only if $s + 1 \leq \max(o, 2)$ in an Orchard-protocol bundle with cross-address transfers enabled, $s + o = 1$ in an Orchard-pool bundle, and $s + 1 \leq \max(o, 2)$ in a Sapling bundle.*
- (iii) *A shielded spend that opens an empty bundle gives $L' - L = 2$, the padded count going from 0 to 2 for every kind of bundle.*
- (iv) *The fee rises by $\text{marginal_fee} \cdot (\max(2, L') - \max(2, L)) \leq \text{marginal_fee} \cdot (L' - L)$, and not at all while $L' \leq \text{grace_actions} = 2$.*

Hence an uneconomic input of positive value adds positive net value to the step if and only if $\max(2, L') = \max(2, L)$: when padding absorbs its spend, or while the step stays within the grace window.

Proof. (i) For every $x \in \mathbb{N}$, $\lceil (x + 150) / 150 \rceil = \lceil x / 150 \rceil + 1$; the transparent contribution is the maximum of this term and the unchanged output term, so it rises by 0 or 1.

(ii) and (iii). By Definition 10.6 the contribution of a non-empty bundle is $\max(\max(s, o), 2)$ with cross-address transfers enabled, $\max(s + o, 2)$ in an Orchard-pool bundle, and $\max(s, \max(o, 2))$ for Sapling, whose outputs are padded to $\max(o, 2)$; an empty bundle contributes 0. Each non-empty form is $\max(x, c)$ with $c \geq 2$ and a count x that rises by exactly one, so it rises by 0 or 1, and by 0 if and only if $x + 1 \leq c$: that is $s + 1 \leq \max(o, 2)$ in the first and third forms, and $s + o + 1 \leq 2$, which with $s + o \geq 1$ is $s + o = 1$, in the second. A single spend in an empty bundle gives $\max(1, 2) = 2$ with cross-address transfers enabled, $\max(1 + o, 2) = 2$ in the Orchard pool and $\max(1, \max(o, 2)) = 2$ for Sapling. No other contribution changes.

(iv) From Definition 10.2, since $n \mapsto \max(2, n)$ is non-decreasing and rises by at most the rise of n . For the final claim: if $\max(2, L') = \max(2, L)$ the fee is unchanged and the input adds $v(u) > 0$; otherwise the fee rises by at least $\text{marginal_fee} > v(u)$.

The instances $(s, o, \text{rise of the padded count})$ are, with cross-address transfers enabled and for Sapling, $(0, 1, 0)$, $(1, 1, 0)$, $(1, 2, 0)$ and $(2, 2, 1)$; in an Orchard-pool bundle $(0, 1, 0)$, $(1, 1, 1)$, $(1, 2, 1)$ and $(2, 2, 1)$; and $(0, 0, 2)$ for the empty bundle of every kind. $\square$

ZIP 315 states that automatic shielding and automatic or opportunistic migration SHOULD NOT be applied to inputs whose cost of shielding or migrating exceeds their economic value (ZIP 315, "Auto-shielding"); under the conventional fee, Lemma 11.13 decides which inputs these are.

The choice of source pool is constrained by privacy. Value that crosses pools is published in the balancing values (*Ironwood Guide*, §"The balancing value", Remark "Disclosure by the balancing value"), so selecting inputs in the pool of the payment outputs keeps the amount private,

and ZIP 315 states the aim of minimising pool crossing (ZIP 315, “Linkability of transactions or addresses”). Wallets MUST NOT automatically combine funds across pools to satisfy a transfer, since that could reveal the total the user holds in a pool (ZIP 315, “Information leakage for transfers between pools”). One policy serving these rules is designed but unspecified (Definition 1.1): order the shielded pools with the pool of the outputs first, then the other Orchard-protocol pool, then Sapling, or, for Sapling outputs, Sapling, then Ironwood, then Orchard; remove the pools that the user’s policy forbids; and spend from the first pool whose notes alone cover the payment.

Transparent sources and recipients are specified. A wallet MUST NOT send funds to a transparent address unless all of the source funds come from shielded pools, and this SHOULD be a single shielded pool (ZIP 315, “Linkability of transactions or addresses”); the ZIP 320 transfer of Construction 11.7 meets this rule, its first transaction spending only shielded notes. A wallet SHOULD NOT spend funds from a transparent address in a transaction with an external recipient unless the user gives explicit consent for that transaction, and MAY restrict transfers further (ZIP 315, “Allowed transfers”). The admissible family $\mathcal{S}$ of Definition 11.9 consists of the selections that meet the rules of this and the preceding paragraph, those of §11.3, and the wallet’s policy.

11.5 Change

Construction 11.14 (Balance and change). *Change* is the excess of a step’s inputs over its payments and fee, returned to the wallet in $k \geq 0$ outputs whose pool follows Construction 11.5. Let in be the total of the inputs, over all pools and ephemeral inputs included, out the total of the payment outputs, and f_k the conventional fee of the padded shape with k change outputs, which is non-decreasing in k .

- (1) If $in < out + f_0$, fail with shortfall $out + f_0 - in$, the requirement fed back to selection (Construction 11.10).
- (2) If $in = out + f_0$, the step has no shielded input or output and no change memo is requested, set $k := 0$ and fee f_0 .
- (3) Otherwise let k be the least number of change outputs, 1, or a larger number chosen by the wallet to split change across notes, a choice designed but unspecified (Definition 1.1), subject to $in \geq out + f_k$. The change is $c := in - out - f_k \geq 0$, divided among the k outputs, each carrying the change memo if one is requested. If $out + f_0 \leq in < out + f_1$, no $k \geq 1$ qualifies: the step fails with requirement $out + f_1 - in$, or, if $in > out + f_0$, may instead set $k := 0$ and fold the positive remainder $in - out - f_0$ into the fee; the plan records which.

Balance invariant. Before building, the transparent, Sapling, Orchard and Ironwood balances of the step, each the value of the step’s inputs in that pool minus the value of its outputs in that pool, sum to the planned fee. A sum above the fee means that change was omitted; a sum below it, insufficient funds.

Change goes to an internal-scope address of the spending account (ZIP 32, “Orchard internal key derivation”; Construction 2.14; ZIP 316, “Deriving Internal Keys”), its diversifier index

a wallet policy; non-zero change remains in the Orchard pool only at an internal-scope receiver of the spending account, the receiver of the note consumed in its Action (Proposition 11.6). A zero-valued fabricated output at the receiver of a spent note is not change.

The balance invariant is the balance equation of Definition 11.2 with every term moved to one side, summed pool by pool.

Proposition 11.15 (Zero-valued change). *Let a step have a shielded input or output, and let Construction 11.14 complete it with $k \geq 1$ change outputs, of total 0 when $\text{in} = \text{out} + f_k$.*

- (i) *The public counts of the step are the same function of its inputs, payments and k whether the change is 0 or positive.*
- (ii) *If change were instead omitted at exact balance, the counts at exact balance would differ from those with change exactly when one more output of the change pool raises that pool's padded count or opens a bundle (Definition 10.6). An observer who knows the payment outputs, such as a party that receives every payment, and sees the counts would then infer that the inputs sum exactly to the payments plus the fee.*
- (iii) *In the Ironwood pool, bundles with equal public leakage that differ only in whether the change is 0 or positive are computationally indistinguishable.*

The zero-valued output also carries a requested change memo. Where padding already absorbs the change output, as in the two-Action Orchard-pool bundle of ZIP 318, the omission of (ii) reveals nothing (ZIP 318, “Canonical migration transaction structure”).

Proof. (i) and (ii) follow from Construction 11.14 and Definition 10.6: the counts depend on k , which is at least 1 whatever the value of c , and omission replaces k by 0 at exact balance only. (iii) is *Ironwood Guide*, §“Privacy”, Theorem “Privacy of an Action”, under its preconditions (P1) to (P4) and its assumptions: Actions whose private inputs have equal public leakage are indistinguishable, and a created note’s value is a private input, not public leakage. No claim is made for Sapling outputs beyond (i). $\square$

A zero-valued note needs no witness. Condition A3 of the Action statement, $v^{\text{old}} \cdot (\text{root} - \text{rt}) = 0$, checks the authentication path only of a consumed note of non-zero value (*Ironwood Guide*, §“The Action statement”), so a zero-valued change note can later be spent without tracking its path.

Only a step with no shielded input or output and no change memo omits change at exact balance. The second transaction tr_1 of Construction 11.7 is the instance, and a change memo is not carried there. A change note of value $v < \text{marginal_fee}$ is an uneconomic input when it is later spent (Definition 11.12), and adds positive net value only where Lemma 11.13 gives no rise of the fee. The wallet’s policy for such change is designed but unspecified (Definition 1.1), constrained by the following proposition.

Proposition 11.16 (Fee uniformity). *The fee of a transaction is public, the value remaining in its transparent transaction value pool, and its conventional fee is a function of public fields (Definitions 10.1 and 10.2), so every observer computes both. A transaction whose fee differs from the conventional fee of its shape is distinguished from every transaction of that shape that pays the conventional fee. Folding a remainder into the fee (Construction 11.14, case (3)) produces exactly such a fee whenever the remainder is positive. The folding threshold is the wallet’s policy, designed but unspecified (Definition 1.1).*

Proof. The fee is the sum of the transparent input values and the balancing values, minus the transparent output values, each a public field (*Ironwood Guide*, §“The balancing value”); the transparent sizes and the shielded counts of Definition 10.1 are public fields. The predicate “the fee differs from the conventional fee of the shape” is therefore a function of the public data; it holds for a transaction with a folded remainder $in - out - f_0 > 0$, whose fee exceeds f_0 , and fails for every transaction of the same shape that pays the conventional fee. ZIP 317 states that non-standard fees may reveal specific users or wallets (ZIP 317, “Security and Privacy considerations”). $\square$

11.6 Outgoing viewing keys

Definition 11.17 (Outgoing viewing key of an output). For a transfer, the sender determines:

- (a) the *sending account*: the account that the transfer names, if it names one; otherwise, if every fund used comes from addresses of one account, that account, a rule that ZIP 316 makes a SHOULD; otherwise the sending account is undetermined;
- (b) the *preferred sending protocol*, the most preferred receiver type (the Priority List of §3.3) among the funds used: Orchard if any Orchard-protocol note, of either pool, is spent, else Sapling, else transparent.

The *outgoing viewing key of an output* is, with the sending account determined, that account’s external or internal ovk of the preferred protocol at the account level, external for a payment and internal for a wallet-internal output such as change or a shielding output (SHOULD); with the sending account undetermined, the external or internal ovk of the full viewing key of one address of the preferred protocol from which funds are sent, for example the first (SHOULD) (ZIP 316, “Usage of Outgoing Viewing Keys”). The Orchard external ovk is that of the *Ironwood Guide*, Construction “Diversifier key and outgoing viewing key” in §“Viewing keys”, and the internal one that of Construction 2.14; the transparent pair is that of Construction 2.18, the two halves of $I_{\text{ovk}} = \text{PRFexpand}_c([\text{0xD0}] \parallel \text{ser}_P(\text{pk}))$; the Sapling keys are those of ZIP 32, “Sapling internal key derivation”. The choice governs the outgoing ciphertext C_{out} of every payment and change output. Its construction, which also admits $\text{ovk} = \perp$ with a uniformly random ock, is cited (*Ironwood Guide*, §“The outgoing ciphertext”, Construction “Outgoing ciphertext”; protocol specification, §“Sending Notes (Sapling)” and §“Sending Notes (Orchard)”).

For change the rule gives the internal ovk. The specification also permits no ovk: with $\text{ovk} = \perp$ the sender draws ock and the outgoing plaintext uniformly, no viewing key recovers the outgoing plaintext of the change, and the wallet finds its change by trial decryption under the internal

ivk (*Ironwood Guide*, §“Trial decryption and note acceptance”; protocol specification, §“Sending Notes (Orchard)”). Encrypting change with no ovk departs from the SHOULD of ZIP 316.

The holder of an account’s external ovk therefore recovers the recipient, value and memo of the account’s Ironwood-pool payment outputs (*Ironwood Guide*, §“The outgoing ciphertext”, Construction “Recovery with ovk”), and those of its change only with the internal ovk.

11.7 Padding, dummies, and shuffling

Construction 11.18 (Assembly of an Orchard-protocol bundle). Fix an Orchard-protocol bundle with s real spends and o real outputs, and let n be its padded Action count (Definition 10.6), or $n = 1$ for the Ironwood-pool bundle of a canonical migration transaction, or $n = 16$ for the Orchard-pool bundle of a note-preparation transaction (§10.2). The wallet sets $\text{enableSpend} = \text{enableOutput} = 1$, the flags that ZIP 315 notes depend under normal circumstances only on whether a transaction is a coinbase transaction (ZIP 315, “Kinds of information leakage”).

- (A) *Cross-address transfers enabled*, an Ironwood-pool bundle with $\text{enableCrossAddress} = 1$, the normal case of ZIP 229 (ZIP 229, “enableCrossAddress polarity”), which the wallet uses for every Ironwood-pool bundle. Pad the list of real spends with $n - s$ dummy consumed notes and the list of real outputs with $n - o$ dummy created notes (*Ironwood Guide*, §“Actions, bundles, and dummy notes”, Definition “Dummy notes”), each at an independent fresh random address; draw independent uniform permutations σ and τ of $\{1, \dots, n\}$; place spend j in Action $\sigma(j)$ and output j in Action $\tau(j)$.
- (B) *Orchard pool*, cross-address transfers disabled (Construction 11.5). Form
 - (i) for each real spend, one Action whose output is a fabricated zero-valued note at the spent note’s address, with its C_{enc} filled with random bytes (MUST), since a real encryption would let a holder of that receiver’s ivk link the spend’s nullifier to the address (ZIP 326, “Fabricated same-address outputs and randomized note ciphertexts”);
 - (ii) for each real output, one Action whose spend is a fabricated zero-valued note at that output’s address, a real spend signed with the account’s ask for that receiver, carrying no dummy spending key (ZIP 374, “Fabricated same-address outputs (NU6.3)”);
 - (iii) $n - s - o$ fully dummy Actions, whose dummy spend and dummy output share one common fresh random address, the only Actions that carry a dummy spending key;

then draw a uniform permutation π of $\{1, \dots, n\}$ and place the j -th Action so formed at position $\pi(j)$.

The second Action of the canonical Orchard-pool bundle of ZIP 318 is of kind (B)(iii) when that bundle carries no change, and of kind (B)(ii) when it does (ZIP 318, “Canonical migration

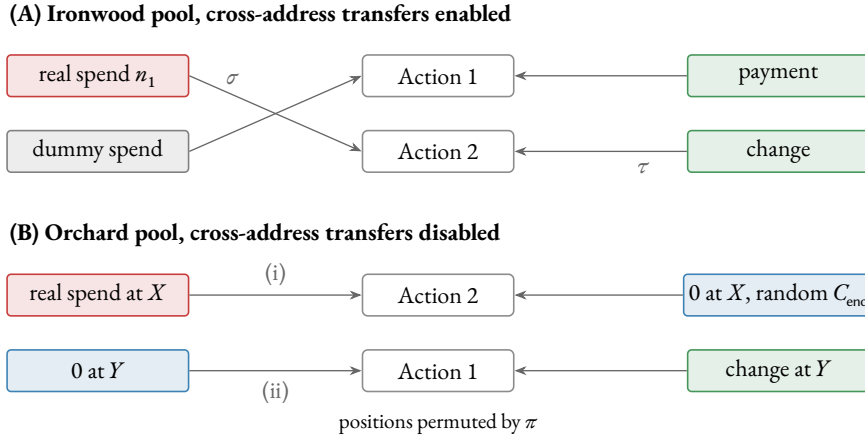


Figure 4. Assembly of Orchard-protocol bundles with $n = 2$ Actions (Construction 11.18). (A) An Ironwood-pool bundle with one real spend and two real outputs: the padded spend list and the output list are permuted independently by σ and τ , so any spend may pair with any output. (B) An Orchard-pool bundle with one real spend at receiver X and one change output at an internal receiver Y of the spending account: each real side is paired with a fabricated zero-valued side at its own receiver, the one beside the real spend with a random C_{enc} , and only the order of the Actions is permuted, by π . Red marks real spends, green real outputs, blue fabricated sides and grey dummies.

transaction structure”). A dummy output at an address independent of its Action’s consumed note exists only in bundles with cross-address transfers enabled; in kind (B)(iii) the dummy output shares the fresh random address of its dummy spend. A dummy spend coexists with any anchor, since condition A3 of the Action statement checks no path for a zero-valued consumed note. Per Action, rcv , α and the created note’s rseed are drawn fresh, and ρ of the created note is the Action’s nf^{old} (*Ironwood Guide*, §“Construction of a transaction”). A Sapling bundle samples rcv and α per spend and rcv per output, pads its outputs as in Definition 10.6, and its binding signing key is the sum of the spends’ rcv minus the sum of the outputs’ (protocol specification, §“Balance and Binding Signature (Sapling)”). The builder keeps the permutations, which are not part of the transaction, so that later roles (§12) attach per-output data to the right Action.

Proposition 11.19 (Uniform order and pairing of Actions). *In a bundle with cross-address transfers enabled built by Construction 11.18, the position of each spend and the pairing of spends with outputs are uniform and independent of the order of the plan: the map from spend index to Action position is a uniform permutation, and the spend-to-output pairing is a uniform permutation independent of it. In an Orchard-pool bundle the order of the Actions is a uniform permutation independent of the order of the plan. Positions therefore carry no information about the plan beyond the Action count; for an Ironwood-pool bundle, the public data of the Actions reveal no more than their public leakage.*

Proof. With σ and τ independent uniform permutations of $\{1, \dots, n\}$ applied to the padded spend and output lists, Action i pairs spend $\sigma^{-1}(i)$ with output $\tau^{-1}(i)$, so spend j is paired with output

$\pi(j)$ for $\pi := \tau^{-1} \circ \sigma$. The map $(\sigma, \tau) \mapsto (\sigma, \pi)$ is a bijection of pairs of permutations, since $\tau = \sigma \circ \pi^{-1}$; it maps the uniform distribution to the uniform distribution, so π is uniform and independent of σ . The plan's order enters only as the indexing of the lists, which both permutations randomise. The Orchard-pool case is the single uniform permutation π . The last clause is *Ironwood Guide*, §“Privacy”, Theorem “Privacy of an Action”, under its preconditions (P1) to (P4) and its assumptions, applied to the bundle as a whole. No indistinguishability is claimed between the kinds (B)(i), (B)(ii) and (B)(iii) of Orchard-pool Actions, whose bundles that theorem does not cover. $\square$

11.8 Authorisation of a transaction

The transaction carries the consensus branch identifier in force at its target height H (*Ironwood Guide*, §“The transaction format”). Transaction versions 5 and 6 are valid and version 4 is not (ZIP 2003, “Specification”; ZIP 259, “Abstract”); a transaction with an Ironwood-pool component is of version 6. The lock time is specified: the field `nLockTime` is public and may distinguish a transaction if used, and SHOULD be zero (ZIP 315, “Kinds of information leakage”); the wallet sets it to 0, and ZIP 318 lists `lock_time = 0` among the SHOULDs of its canonical migration transaction structure (ZIP 318, “Canonical migration transaction structure”). The reason is the one proved for the fee in Proposition 11.16: a public field set otherwise than by every other wallet distinguishes the transaction. The expiry height is that of §9.3.

Construction 11.20 (Authorising a built transaction). Let a transaction be built from one step by Construction 11.18, every field of its effecting data fixed.

- (1) Compute its signature digest `SigHash` over the effecting data (ZIP 244, “Signature Digest”, as amended by ZIP 229, “Anchor commitment (version 6)”, which omits the anchors from the version-6 digest; *Ironwood Guide*, §“Transaction digests and signatures”). Every spend-authorisation signature and every binding signature signs `SigHash`.
- (2) For each Orchard-protocol Action of either pool, with the Action's randomiser α , set $\text{rsk} := \text{ask} + \alpha$ and $\text{spendAuthSig} := \text{Sign}_{\text{rsk}}(\text{SigHash})$ (*Ironwood Guide*, §“Randomised validating keys”). Every Action carries one: a fully dummy Action's spend is signed with its own random key, and a fabricated zero-valued spend beside a real Orchard-pool output with the account's ask, not with a dummy key. Each Sapling spend likewise signs `SigHash` under its randomised key (protocol specification, §“Spend Authorization Signature (Sapling and Orchard)”).
- (3) For each bundle, set $\text{bsk} := \sum \text{rcv} \bmod p_{\text{Vesta}}$ over its Actions, check it against `bvk`, and sign `SigHash` under `bsk` (*Ironwood Guide*, §“The binding signature”); the Sapling `bsk` is that of Construction 11.18.
- (4) Sign each transparent input under the key of its address, by the scheme of the *Crypto Guide*, §“The transparent layer's schemes: ECDSA and BIP-340 over secp256k1”, over the signature digest of that input (ZIP 244, “S.2: transparent_sig_digest”).

Keys. One Orchard-protocol ask serves both scopes and both pools (Construction 2.14; ZIP 32, “Orchard internal key derivation”). One Sapling ask serves both scopes, the internal scope differing in *nsk*, which enters the proof and the nullifier, not the signature (ZIP 32, “Deriving a Sapling internal spending key”), so a Sapling spend is added under the scope of its note. Each transparent input needs the key of its own address.

Lemma 11.21 (Scope of the signature digest). *The signature digests of version-5 and version-6 transactions commit to the effecting data and, for transparent inputs, to the values and scripts of the coins spent (ZIP 244, “S.2: transparent_sig_digest”), excluding proofs and signatures, and the version-6 digest also excludes the anchors. Hence steps (2) to (4) of Construction 11.20 need no proof; proofs and signatures can be computed in either order; in version 6, signing can precede the choice of anchor; and the transaction identifier is unchanged by proofs, by signatures and, in version 6, by anchors.*

Proof. By the *Ironwood Guide*, §“Transaction digests and signatures”, Definition “Signature digest”, SigHash has the header and shielded children of *txid* and the transparent signature digest as its children, all computed over effecting data and, for the transparent signature digest, the values and scripts of the coins spent; proofs, signatures and transparent scripts are authorising data, under the authorising-data digest (Definition “Authorising-data digest; effecting and authorising data”). ZIP 229, “Anchor commitment (version 6)”, moves the anchors to the authorising data in version 6; in version 5 they are effecting data (ZIP 244, “Signature Digest”). A step of Construction 11.20 computes a signature over SigHash from keys and randomisers, and the binding key from the *rcv* values, none of which is a proof, so the order of proving and signing is free. The transaction identifier is a function of the effecting data alone (Definition “Transaction identifier”). The multi-party form of the lemma is that of §12.9. $\square$

Theorem 11.22 (Payment integrity). *Let a well-formed plan (Definition 11.3) be executed by Construction 11.18 and Construction 11.20, with Orchard-protocol keys generated with `use_qlsk = false`. In each resulting transaction:*

- (i) *each payment is paid by exactly one output, in the pool that Construction 11.5 assigns, to the routed receiver, of the payment’s value and with its memo, none for a transparent output;*
- (ii) *every other output is a change output of Construction 11.14, an ephemeral output of the step, or a zero-valued dummy or fabricated output;*
- (iii) *the fee equals the planned fee: the conventional fee of the final shape, or, where Construction 11.14 folds a remainder into it, that fee plus the remainder.*

*Moreover, for a version-6 transaction whose signing keys meet hypotheses (H) and (S) of Ironwood Guide, §“Transaction digests and signatures”, Proposition “Bundle binding”, and under the assumptions of that proposition and of its Theorem “Spend authority” (§“Authorisation of spends”), no other party produces, except with negligible probability, an accepted transaction that carries any of the transaction’s Orchard-protocol randomised validating keys *rk* with different effecting data. The theorem claims content, not inclusion: inclusion further requires that no spent nullifier*

| *be revealed first, that the anchor block remain on the best chain and that inclusion occur by the expiry height, none of which the wallet controls (§13).*

Proof. (i) and (ii). By Definition 11.2 each payment is one output of its step, placed by Construction 11.5, and the only other outputs of a step are its change and ephemeral outputs. Construction 11.18 adds only notes of value 0: dummy created notes, fabricated outputs and the outputs of fully dummy Actions. The padding therefore creates no output of non-zero value, and each real output keeps the receiver, value and memo that the plan gives it.

(iii) In the built transaction the fee is the value remaining in the transparent transaction value pool, the transparent inputs minus the transparent outputs plus the balancing value of each shielded pool, and each balancing value is the value consumed minus the value created in its pool. Since dummy and fabricated notes have value 0, the remaining value is the total of the step's inputs minus the total of its payment, ephemeral and change outputs, which is the planned fee by the balance equation of Definition 11.2 and the balance invariant of Construction 11.14; the planned fee is f_k , or f_0 plus the folded remainder.

Binding. Every signature of the transaction signs SigHash (Construction 11.20), which covers the effecting data (Lemma 11.21). By *Ironwood Guide*, §“Transaction digests and signatures”, Proposition “Bundle binding”, an accepted transaction that carries some rk_i of the transaction carries the same effecting data, except with negligible probability; Theorem “Spend authority” excludes a valid signature under a key of the wallet on a digest that the wallet did not sign. For transparent and Sapling signatures the commitment to the effecting data is that of ZIP 244; no lower volume proves the unforgeability of those schemes, and none is claimed here. $\square$

The wallet records each of its shielded outputs from the data it built the output with: the address, the value, rseed and the memo, and, for an Orchard-protocol output, ρ , the nullifier of the spend of its Action. For such an output,

$$\text{esk} = \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}([\text{0x04}] \parallel \text{I2LEOSP}_{256}(\rho))),$$

and the note plaintext follow without any viewing key (*Ironwood Guide*, §“Encryption to the recipient”, and §“The outgoing ciphertext”, Construction “Recovery with ovk”). A fabricated zero-valued Orchard-pool output at a spent note's receiver carries a random C_{enc} , which trial decryption under any ivk rejects except with negligible probability (*Ironwood Guide*, §“Encryption to the recipient”, Assumption “Confidentiality and integrity of ChaCha20-Poly1305”; ZIP 326, “Fabricated same-address outputs and randomized note ciphertexts”). Non-zero Orchard-pool change lies only at internal-scope receivers (Proposition 11.6), so scanning finds it under the internal ivk (Definition 6.4), and no external ivk is needed for it (ZIP 326, “Wallet key-generation restrictions”).

12 Partially created transactions (PCZT)

This section states the protocol by which the construction, proving and authorisation of one transaction are divided among several parties (ZIP 374). It defines the partially created transaction, its versions and the kinds of its fields; the roles that act on it and the modification state they maintain;

its fields, its canonical empty bundles and its encoding; the version-2 rules for deferred anchors and the Ironwood bundle; construction and IO finalisation; signing, with the theorem that a Signer’s signatures authorise exactly what it checked; proving; combining and redaction, with the order independence of proving and signing and the data a PCZT discloses; and spend finalisation and extraction.

12.1 The PCZT and its requirements

Definition 12.1 (Partially created transaction). A *partially created transaction* (PCZT) of one transaction is a value of the typed record of §12.3 for one PCZT version, exchanged among the parties to the creation of that transaction. It carries the data each role needs for its step (adding inputs and outputs, proving, signing, extracting) and accumulates proofs and signatures until the transaction is complete; since it carries all data a signer needs, a signer can be offline. A PCZT consists of a *Global* record and one *bundle* per protocol: transparent, Sapling, Orchard and, from version 2, Ironwood. Every bundle is present in every PCZT, empty bundles included, because metadata of a protocol may have to be held before it is known whether that protocol has inputs or outputs (ZIP 374, “Abstract”, “Specification” and “Structure”).

The PCZT protocol is specified (Definition 1.1) in ZIP 374, Revision 0, whose status is that of Table 1; no consensus rule depends on it. It covers transactions of versions 5 and 6 only: it supports neither Sprout nor version 4 or earlier, since version 4 predates the ZIP 244 digest and several of the role separations of this section, the independence of proving from signing in particular, fail for it (ZIP 374, “Specification”). The protocol serves signing devices that hold spending keys but can neither compute proofs nor hold a whole transaction; proof delegation, the prover needing note data and the full viewing key but no spending key; transactions with several signers (threshold signing, the transparent multisignature wallets of ZIP 48) or several contributors of inputs, whose parallel contributions are merged; and, for version 6, pre-authorisation, in which the signatures precede the choice of anchors, the witnesses and the proofs (ZIP 374, “Motivation”).

ZIP 374, “Requirements”, states six requirements, recorded here as the ZIP’s and not proved as such:

- (R1) every intermediate state of the creation of a version-5 or version-6 transaction is representable, and adding inputs and outputs, finalising the input and output set, attaching lookup data, proving, signing, merging parallel contributions and extracting are each performable by distinct entities, in an order constrained only by data dependencies;
- (R2) no spending key material is required, the keys of dummy spends excepted, which authorise nothing of value and are removable before any Signer sees the PCZT;
- (R3) a Signer determines from the PCZT alone what it authorises (amounts, recipients, and the correspondence between the fields it signs and the values shown to its user), and data a Signer does not need is removable;
- (R4) merging two PCZTs of the same transaction is deterministic and fails on any conflict;

- (R5) for version 6, signatures can be produced while anchors, witnesses and proofs are absent, and supplying them later leaves the signatures valid;
- (R6) the encoding is compact and strictly versioned, so that no data is read under another version's rules.

Requirement (R3) is made a theorem in §12.7, (R5) a proposition in §12.5 and (R4) a construction in §12.9.

Definition 12.2 (PCZT versions). Version 1 represents transactions of version 5 (ZIP 225); version 2 represents transactions of versions 5 and 6 (ZIP 229). The PCZT version does not fix the transaction version: a version-2 PCZT carries a transaction of version 5 or 6, subject to the per-field rules, and a version-1 PCZT only one of version 5. Relative to version 1, version 2 adds the Ironwood bundle; a note version in {V2, V3} per Orchard-protocol bundle; anchors that may be absent; and an encoding that writes a bundle equal to its slot's canonical empty bundle as absent (ZIP 374, "Versioning"). The fields are made exact in §12.3 and the encoding in §12.4.

No field can be added within a version: the encoding is positional, and a parser cannot skip a field it does not know. A PCZT has no non-proprietary key-value maps, so every field of a version is known in advance and a Signer's view of a PCZT is complete; the proprietary maps are the only extension point within a version, and MUST be used for internal processes only. The format evolves by new versions, which the four-byte version of the header distinguishes. Implementations SHOULD parse every specified version and SHOULD emit the oldest version able to represent a PCZT's state when interoperating with entities that may lack the newest; conversion between versions is lossless exactly when the target version represents the state (ZIP 374, "Extensibility" and "Proprietary Use fields"). The states that version 1 cannot represent are listed in §12.4.

Definition 12.3 (Field kinds of a PCZT). Every field of a PCZT is of one of three kinds (ZIP 374, "Structure"):

- (i) *effecting data*, required fields that are part of the final transaction and are always present, committed to by the signature digest (*Ironwood Guide*, §"Transaction digests and signatures");
- (ii) *authorising data*, initially absent and filled as roles act: proofs, signatures, binding signing keys and, in a transaction of version 6, the anchors;
- (iii) *context data*, which enables roles to act and never enters the final transaction: note openings, keys, witnesses, derivation records and address strings.

In a transaction of version 5 the anchors are effecting data (ZIP 374, "Anchors and pre-authorization").

The signature digest commits to every non-transparent effecting field as soon as any input is signed, so no shielded analogue of the transparent hash types that leave inputs or outputs open exists. The transparent and the shielded modifiable flags of §12.2 are therefore kept apart, so that

a purely transparent PCZT behaves as a partially signed Bitcoin transaction of BIP 174 and BIP 370 does (ZIP 374, “Separate Shielded Modifiable Flag from Transparent Modifiable Flags”).

12.2 Roles and the modification state

Definition 12.4 (Roles). A *role* is a partial map from PCZTs to PCZTs, undefined where the role fails; a role that draws randomness is a family of such maps indexed by its random coins, and the Combiner maps a finite set of PCZTs to one. The roles, with the parties that may execute them (ZIP 374, “Specification” and “Roles”), are:

- (1) *Creator* (a single entity): chooses the transaction version and consensus branch identifier, and sets the Global fields and the empty bundles;
- (2) *Constructor*: adds transparent inputs and outputs, Sapling spends and outputs, and Orchard-protocol Actions;
- (3) *IO Finalizer* (anyone): declares the input and output set final, computes the binding signing keys and signs dummy spends;
- (4) *Updater* (anyone): adds data later roles need, such as derivation records, full viewing keys, witnesses and, for version 6, anchors;
- (5) *Redactor* (anyone): removes data later roles do not need;
- (6) *Prover* (a holder of the note data): adds proofs;
- (7) *Signer* (a holder of a spending key): adds signatures;
- (8) *Combiner* (anyone): merges PCZTs;
- (9) *Spend Finalizer* (anyone): assembles transparent unlocking scripts;
- (10) *Transaction Extractor* (anyone): checks completeness and produces the transaction.

The lifecycle runs the Creator, the Constructors, the IO Finalizer and the Updaters, then the Provers and the Signers, the Combiners and the Redactors, and last the Spend Finalizer and the Transaction Extractor. A *Verifier* is not a role: it is an inspection of a PCZT’s internal consistency, for example the relations of §12.7, performed on behalf of an entity, such as a hardware signer, that trusts it and cannot perform the checks itself (ZIP 374, “Verifier”).

Definition 12.5 (Modification state). The Global field $\text{tx_modifiable} \in \{0, 1\}^8$, bits numbered from the least significant, has the following bits (ZIP 374, “Global” and “Combiner”):

- (i) bit 0, transparent inputs modifiable, and bit 1, transparent outputs modifiable: each set by the Creator, checked, and possibly cleared, by a Constructor before it adds an input or output of its kind, cleared by the IO Finalizer when shielded spends or outputs exist, and merged by AND;

- (ii) bit 2, a signature with hash type SIGHASH_SINGLE is present: cleared by the Creator, set by a Signer that adds such a signature, read by a Constructor, which then preserves the pairing of each such input with its output, and merged by OR;
- (iii) bits 3 to 6: zero, which ZIP 374 requires in version 1 and the Combiner requires of every PCZT it merges;
- (iv) bit 7, shielded modifiable: set by the Creator, checked, and possibly cleared, by a Constructor before it adds a shielded spend or output, cleared by the IO Finalizer when shielded spends or outputs exist and by every Signer, and merged by AND.

The Creator’s value is therefore 1000 0011 in binary.

Signing freezes the effects. After adding a shielded signature a Signer clears bits 0, 1 and 7: every shielded signature, spend-authorisation and binding alike, is on the one signature digest, which commits to all effecting data (*Ironwood Guide*, §“Transaction digests and signatures”, Proposition “Bundle binding”), and no shielded hash type leaves an input or output open. After adding a transparent signature it clears bit 0 unless the hash type has ANYONECANPAY, clears bit 1 unless its base type is NONE, sets bit 2 if its base type is SINGLE, with or without ANYONECANPAY, and clears bit 7 (ZIP 374, “Global” and “Signer”).

A wallet executing a plan (Definition 11.2) acts as Creator, Constructor, IO Finalizer and Updater. It builds the bundles by Construction 11.18 and then, as Updater, attaches to each non-dummy Orchard-pool, Ironwood-pool and Sapling spend the derivation record of §12.3, with the seed fingerprint of Construction 2.2 and the path $(32', \text{coin_type}', \text{account}')$ of Definition 2.7; to each transparent input the BIP 44 path $(44', \text{coin_type}', \text{account}', \text{scope}, \text{index})$, the last two components non-hardened; and to each output that pays a payment of the plan (Definition 11.1) that payment’s address string as `user_address`, which a Signer checks (§12.7) (ZIP 374, “Updater” and “Zip32Derivation”; ZIP 32; BIP 44).

12.3 Fields

Definition 12.6 (PCZT fields). *Types.* The types `u8`, `u32` and `u64` are the unsigned integers of that width, `i128` the signed integers of 128 bits and `bool` = {false, true}; $\mathbb{B}^n$ is the set of byte strings of length n , T^n the sequences of n elements of T , and $T_1 \times T_2$ the pairs; $\text{Opt}(T)$ is T together with an absent value $\perp$; $\text{List}(T)$ is the finite sequences over T , $\text{Map}(K, V)$ the finite maps from K to V , and `Str` the UTF-8 strings; $\text{Enum}\{A_0, A_1, \dots\}$ is a set of tags, A_i with index i . A *derivation record* is a value of $D := \mathbb{B}^{32} \times \text{List}(\text{u32})$, a seed fingerprint (Construction 2.2) and a derivation path, an index with bit 31 set being hardened. Let $P := \text{Map}(\text{Str}, \text{List}(\text{u8}))$, the type of a proprietary map. Each record below lists its fields in order, the order of the encoding (§12.4); a field marked (v1) or (v2) is present only in that version (ZIP 374, “Structure” and the sections of each record).

Global. `tx_version` $\in$ `u32`, which is 5 in version 1 and 5 or 6 in version 2;
`version_group_id` $\in$ `u32`, consistent with it (ZIP 225, ZIP 229); `consensus_branch_id` $\in$ `u32`,
 which for `tx_version` = 6 names an upgrade at which version 6 is supported, NU6.3 or later,

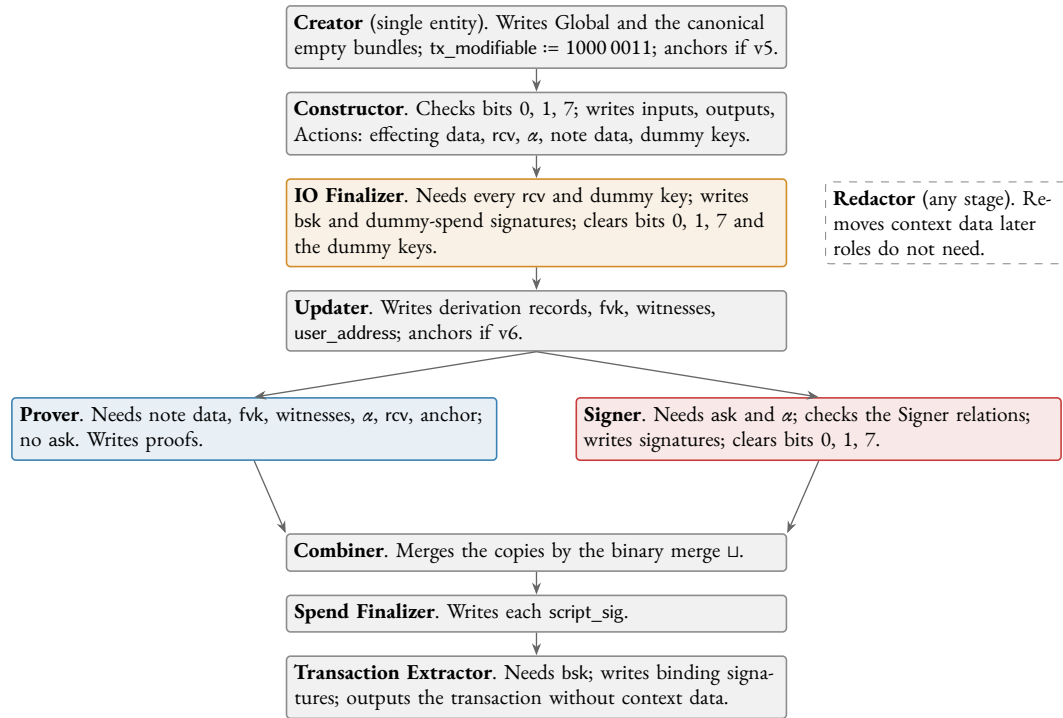


Figure 5. The roles of a PCZT (Definition 12.4), with the fields each writes, the secrets it needs and the bits of tx_modifiable it clears (Definition 12.5). Red marks the one role that needs the spend authorising key ask, amber the role that needs the value-commitment trapdoors and the dummy keys, blue the role that needs the note data and fvk but no ask, and grey the roles that need no secret. The Prover and the Signer run in either order or in parallel on copies joined by the Combiner; what each recipient learns is stated in §12.9.

that of NU7 being 0x77190AD9 (ZIP 259, “NU7 network constants”);
 fallback_lock_time $\in \text{Opt}(u32)$, absence meaning 0; expiry_height $\in u32$ (ZIP 203);
 coin_type $\in u32$, the SLIP 44 coin type, which is not part of the transaction and against which
 roles check network-specific data; tx_modifiable $\in u8$ (Definition 12.5); proprietary $\in P$.

Transparent bundle. inputs $\in \text{List}(\text{TI})$ and outputs $\in \text{List}(\text{TO})$. An input TI:
 prevout_txid $\in \mathbb{B}^{32}$; prevout_index $\in u32$; sequence $\in \text{Opt}(u32)$, absence meaning $2^{32} - 1$;
 required_time_lock_time $\in \text{Opt}(u32)$, at least 500,000,000;
 required_height_lock_time $\in \text{Opt}(u32)$, in $\{1, \dots, 499,999,999\}$; script_sig $\in \text{Opt}(\text{List}(u8))$;
 value $\in u64$; script_pubkey $\in \text{List}(u8)$; redeem_script $\in \text{Opt}(\text{List}(u8))$;
 partial_signatures $\in \text{Map}(\mathbb{B}^{33}, \text{List}(u8))$; sighash_type $\in u8$; bip32_derivation $\in \text{Map}(\mathbb{B}^{33}, D)$;
 four preimage maps, for RIPEMD160, SHA256, HASH160 and HASH256, of types
 $\text{Map}(\mathbb{B}^{20}, \text{List}(u8))$, $\text{Map}(\mathbb{B}^{32}, \text{List}(u8))$, $\text{Map}(\mathbb{B}^{20}, \text{List}(u8))$ and $\text{Map}(\mathbb{B}^{32}, \text{List}(u8))$ in that
 order; proprietary $\in P$. An output TO: value $\in u64$; script_pubkey $\in \text{List}(u8)$;
 redeem_script $\in \text{Opt}(\text{List}(u8))$; bip32_derivation $\in \text{Map}(\mathbb{B}^{33}, D)$; user_address $\in \text{Opt}(\text{Str})$;
 proprietary $\in P$.

Sapling bundle. spends $\in \text{List}(\text{SS})$; outputs $\in \text{List}(\text{SO})$; value_sum $\in i128$, the net value of
 the spends minus the outputs; anchor $\in \mathbb{B}^{32}$ (v1) or $\text{Opt}(\mathbb{B}^{32})$ (v2); bsk $\in \text{Opt}(\mathbb{B}^{32})$. A spend
 SS: effecting cv, nf, rk $\in \mathbb{B}^{32}$; zkproof $\in \text{Opt}(\mathbb{B}^{192})$; spend_auth_sig $\in \text{Opt}(\mathbb{B}^{64})$;
 recipient $\in \text{Opt}(\mathbb{B}^{43})$; value $\in \text{Opt}(u64)$; rcm $\in \text{Opt}(\mathbb{B}^{32})$ and rseed $\in \text{Opt}(\mathbb{B}^{32})$, at most one
 set, by whether the note predates ZIP 212; rcv $\in \text{Opt}(\mathbb{B}^{32})$; the proof generation key
 (ak, nsk) $\in \text{Opt}(\mathbb{B}^{32} \times \mathbb{B}^{32})$; witness $\in \text{Opt}(u32 \times (\mathbb{B}^{32})^{32})$; $\alpha \in \text{Opt}(\mathbb{B}^{32})$; a derivation record in
 $\text{Opt}(D)$; dummy_ask $\in \text{Opt}(\mathbb{B}^{32})$; proprietary $\in P$. An output SO: effecting cv, cmu, epk $\in \mathbb{B}^{32}$
 and the ciphertexts $C_{\text{enc}}, C_{\text{out}} \in \text{List}(u8)$, whose lengths depend on the transaction version;
 zkproof $\in \text{Opt}(\mathbb{B}^{192})$; recipient $\in \text{Opt}(\mathbb{B}^{43})$; value $\in \text{Opt}(u64)$; rseed, rcv, ock $\in \text{Opt}(\mathbb{B}^{32})$; a
 derivation record in $\text{Opt}(D)$; user_address $\in \text{Opt}(\text{Str})$; proprietary $\in P$. The Sapling
 constructions are those of the protocol specification and are cited, not restated.

Orchard bundle. actions $\in \text{List}(\text{OA})$; flags $\in u8$, with bit 0 enableSpends, bit 1
 enableOutputs, bit 2 enableCrossAddress (v2; zero in version 1 and in the Orchard bundle of
 every transaction mined under NU6.3 or later, the rule of Construction 11.5), and bits 3 to 7
 zero; value_sum $\in u64 \times \text{bool}$; anchor $\in \mathbb{B}^{32}$ (v1) or $\text{Opt}(\mathbb{B}^{32})$ (v2);
 note_version $\in \text{Enum}\{V2, V3\}$ (v2), where V2 is the note plaintext of ZIP 212, lead byte 0x02,
 and V3 that of ZIP 2005, lead byte 0x03, which MUST be V2 in the Orchard bundle and is
 implicitly V2 in version 1; zkproof $\in \text{Opt}(\text{List}(u8))$, one proof for the bundle; bsk $\in \text{Opt}(\mathbb{B}^{32})$.
 An Action OA: cv^{net} $\in \mathbb{B}^{32}$; a spend OS; an output OO; rcv $\in \text{Opt}(\mathbb{B}^{32})$, one per Action, since
 an Action has one net value commitment (*Ironwood Guide*, §“Value commitments”). A spend
 OS: effecting nf, rk $\in \mathbb{B}^{32}$; spend_auth_sig $\in \text{Opt}(\mathbb{B}^{64})$; recipient $\in \text{Opt}(\mathbb{B}^{43})$, a raw Orchard
 address; value $\in \text{Opt}(u64)$; ρ , rseed $\in \text{Opt}(\mathbb{B}^{32})$; fvk $\in \text{Opt}(\mathbb{B}^{96})$, the encoding of (ak, nk, r_{fvk});
 witness $\in \text{Opt}(u32 \times (\mathbb{B}^{32})^{32})$, a position and a 32-level authentication path; $\alpha \in \text{Opt}(\mathbb{B}^{32})$; a
 derivation record in $\text{Opt}(D)$; dummy_sk $\in \text{Opt}(\mathbb{B}^{32})$; proprietary $\in P$. In a transaction of
 version 6 the witness may be absent until proving (§12.5). An output OO: effecting
 cmx, epk $\in \mathbb{B}^{32}$ and $C_{\text{enc}}, C_{\text{out}} \in \text{List}(u8)$; recipient $\in \text{Opt}(\mathbb{B}^{43})$; value $\in \text{Opt}(u64)$;

$\text{rseed}, \text{ock} \in \text{Opt}(\mathbb{B}^{32})$; a derivation record in $\text{Opt}(D)$; $\text{user_address} \in \text{Opt}(\text{Str})$;
 $\text{proprietary} \in P$.

Ironwood bundle (v2). The fields of the Orchard bundle, with $\text{note_version} = V3$ and its anchor a root of the Ironwood note commitment tree (§12.5).

Which role sets, requires or clears each field is as ZIP 374 states per field; this section restates it where it defines a role.

Every transparent input's value and script_pubkey are required, not optional, because the signature digest commits to the value and script of every coin a transaction spends (ZIP 244, as amended by ZIP 229; *Ironwood Guide*, §“Transaction digests and signatures”), and the binding signatures are on that digest (ZIP 374, “TransparentInput”).

Remark 12.7 (The sign of an Orchard-protocol value sum). The pair $\text{value_sum} = (m, b)$ denotes the net value of the bundle's spends minus its outputs, but ZIP 374 does not state the meaning of b : a gap in the specification. The reference implementation of ZIP 374 reads $b = \text{true}$ as the net value $-m$ and $b = \text{false}$ as $+m$, and the canonical empty bundle has $(0, \text{false})$. This volume adopts that reading, and writes v^{balance} for the integer that a bundle's value_sum denotes; for Sapling it is the i128 value itself.

A Signer **MUST** parse the user_address of each output that carries one and confirm that it contains the output's recipient, directly or as a receiver of a unified address (ZIP 374, “TransparentOutput”, “SaplingOutput” and “OrchardOutput”; Definition 3.7 and Construction 3.10). The anchors, witnesses and commitments that these fields carry are those of the *Ironwood Guide*, §“The note commitment tree”, with its subsections “Anchors” and “Authentication paths”; they are cited, not rebuilt.

Definition 12.8 (Canonical empty bundles). The *canonical empty bundle* of each slot is (ZIP 374, “v2 Encoding”):

- (i) transparent: empty input and output lists;
- (ii) Sapling: empty spend and output lists, $\text{value_sum} = 0$, $\text{anchor} = \perp$, $\text{bsk} = \perp$;
- (iii) Orchard: no Actions, $\text{flags} = 0000\,0011$ in binary (spends and outputs enabled), $\text{value_sum} = (0, \text{false})$, $\text{anchor} = \perp$, $\text{note_version} = V2$, $\text{zkproof} = \text{bsk} = \perp$;
- (iv) Ironwood: no Actions, $\text{flags} = 0000\,0111$ in binary (spends, outputs and cross-address transfers enabled), $\text{value_sum} = (0, \text{false})$, $\text{anchor} = \perp$, $\text{note_version} = V3$, $\text{zkproof} = \text{bsk} = \perp$.

A bundle that differs from it in any field, its flags, note version or anchor included, is not canonical empty.

12.4 Encoding

Construction 12.9 (PCZT encoding). The encoding of a PCZT of version n is

$$50\ 43\ 5a\ 54 \parallel \text{l2LEOSP}_{32}(n) \parallel \text{body}_n,$$

the first four bytes being the ASCII string PCZT. A string that does not begin with this 8-byte header is not a PCZT encoding, and a parser that meets an unknown version **MUST NOT** parse the remainder (ZIP 374, “Encoding”). The body is built from the following encodings of the types of Definition 12.6 (ZIP 374, “Data type encodings”):

- (i) u8: one raw byte;
- (ii) u32 and u64: $\text{varint}(x)$, the digits of x in base 128, least significant first, one per byte, with bit 7 set on every byte but the last. Encoders **MUST** emit the shortest form. A k -bit integer takes at most $\lceil k/7 \rceil$ bytes, 5 for u32 and 10 for u64, and in an encoding of that length the last byte contributes only the high $k \bmod 7$ bits, so the fifth byte of a u32 is at most $0x0f$; decoders **MUST** reject encodings that exceed either bound and **MAY** accept non-minimal encodings within them;
- (iii) i128: $\text{varint}(\text{zigzag}(x))$, with $\text{zigzag}(x) = 2x$ for $x \geq 0$ and $2|x| - 1$ for $x < 0$;
- (iv) bool: the byte $0x00$ for false or $0x01$ for true, other bytes rejected;
- (v) $\mathbb{B}^n$, T^n and pairs: the concatenation of the elements, with no length prefix, so that a witness path is 1024 raw bytes;
- (vi) $\text{Opt}(T)$: the byte $0x00$ for $\perp$, and $0x01$ followed by the value otherwise, other tag bytes rejected;
- (vii) $\text{List}(T)$ and Str : the varint of the number of elements, of bytes for a string, followed by the elements; invalid UTF-8 is rejected;
- (viii) $\text{Map}(K, V)$: the varint of the number of entries, followed by each key and its value. Encoders **MUST** emit the entries in strictly ascending lexicographic byte order of the unencoded keys, without duplicates; decoders **MAY** accept other orders and duplicates;
- (ix) Enum: the varint of the tag’s index;
- (x) a record: the concatenation of its fields in the order of Definition 12.6, without tags or padding, a field absent from version n being omitted entirely.

Encoders **MUST NOT** emit data after the last field; decoders **MAY** ignore or reject it. *Layout* (ZIP 374, “v1 Encoding” and “v2 Encoding”): body_1 is the Global record followed by the transparent, Sapling and Orchard bundles, each Action encoded as cv^{net} , spend, output, rcv. The body body_2 is the Global record followed by the four bundles, transparent, Sapling, Orchard and Ironwood, each as a value of Opt ; a bundle equal to its slot’s canonical empty bundle (Definition 12.8) **MUST** be encoded as absent and any other as present, and a parser **MUST**

restore the canonical empty bundle from absence. Encoding in version 1 **MUST** fail when `tx_version` $\neq$ 5, when the Ironwood bundle is not canonical empty, when the Orchard note version is not V2, or when an anchor required before signing a transaction of version 5 is absent: these are the states version 1 cannot represent.

Lemma 12.10 (Canonicity of the encoding). *For every version n and every PCZT P representable in version n , exactly one byte string is an output for P permitted to an encoder that obeys the **MUST** rules of Construction 12.9, and distinct PCZTs have distinct encodings. Canonicity is a property of encoder output: since decoders may accept non-minimal variants, unordered or repeated map entries and trailing data, several strings may decode to one PCZT.*

Proof. By induction on the type structure. *Uniqueness.* The permitted encoding of each primitive is a function of its value: the shortest base-128 representation is unique, its last byte non-zero unless the value is 0; the map zigzag is a bijection from the integers onto the naturals; the tags of options, booleans and enumerations are fixed; the counts of lists and strings are determined by the value. A finite map admits exactly one strictly ascending order of its entries without duplicates, since lexicographic order on byte strings is total. A record concatenates uniquely encoded fields in a fixed order. In version 2 the presence of each bundle is determined by equality with Definition 12.8, and no trailing data is permitted. *Injectivity.* Each encoding is self-delimiting given its type, being of fixed length or a count followed by self-delimiting elements, so a decoder that reads the fields in order recovers every value; an absent version-2 bundle is restored as the canonical empty bundle, the only bundle encoded as absent. The header fixes n , so encodings of distinct versions differ as well. $\square$

The canonical-empty rule is exact equality so that two copies of one PCZT encode identically whatever software produced them, and re-encoding cannot move a bundle between its present and absent forms (ZIP 374, “Omitting empty bundles in PCZT v2”).

Example 12.11 (A minimal version-2 PCZT). The Creator’s PCZT (§12.6) for a transaction of version 6, with version group identifier `0xD884B698` (ZIP 229), consensus branch identifier `0x77190AD9` (NU7, ZIP 259), expiry height 10,000,000, coin type 133 (Mainnet), no fallback lock time, no proprietary entries and every bundle canonical empty, encodes in version 2 as the 32 bytes of Table 6. With coin type 1 the coin type takes the one byte `01`, the total is 31 bytes, and every later field moves one byte earlier: the position of a field depends on the values before it, which is why no field can be added within a version.

12.5 PCZT v2: deferred anchors and the Ironwood bundle

In version 2 the Ironwood bundle carries the Ironwood-pool component of a transaction of version 6, with the structure, field meaning and role interactions of the Orchard bundle, the Ironwood pool being a second value pool of the Orchard protocol (ZIP 229; *Ironwood Guide*, §“The Orchard protocol and its two pools”). It differs in three respects: its `enableCrossAddress` bit is meaningful and normally 1; its anchor is a root of the Ironwood note commitment tree, distinct from the Orchard tree; and its note version **MUST** be V3. Every role **MUST** reject a PCZT with

Table 6. The minimal version-2 PCZT of Example 12.11: offset, bytes and field, in the encoding of Construction 12.9.

Offset	Bytes	Field
0	50 43 5a 54	header, PCZT
4	02 00 00 00	version 2
8	06	tx_version = 6
9	98 ed 92 c4 0d	version_group_id
14	d9 95 e4 b8 07	consensus_branch_id
19	00	fallback_lock_time = $\perp$
20	80 ad e2 04	expiry_height
24	85 01	coin_type = 133
26	83	tx_modifiable, bits 0, 1, 7
27	00	empty proprietary map
28 to 31	00 00 00 00	four absent bundles

tx_version = 5 whose Ironwood bundle is not canonical empty (Definition 12.8) (ZIP 374, “IronwoodBundle (PCZT v2)”).

Definition 12.12 (Anchors and witnesses in version 2). The following rules hold (ZIP 374, “Anchors and pre-authorization”).

- (i) For tx_version = 5, every shielded bundle’s anchor is set when the bundle’s first spend is added and never changes, since the version-5 digest commits to it.
- (ii) For tx_version = 6, which requires version 2, an anchor MAY be absent until proving; an Updater MAY set an absent anchor, or replace a set one, at any time before the Prover runs for that bundle; setting or replacing an anchor MUST clear every proof already in that bundle, the Sapling spends’ proofs or the Orchard-protocol bundle’s proof, since a proof takes the anchor as a primary input; and the witness of a Sapling or Orchard-protocol spend MAY be absent until proving, and is set by an Updater once known.
- (iii) Whenever a bundle has both an anchor and witnesses, the Updater that sets either MUST check that every witness of a spend of non-zero value roots to the anchor, and the Prover MUST check the same before proving.

Deferral is governed by the transaction version, not by the PCZT version: a transaction of version 5 in a version-2 PCZT fixes its anchors as in version 1. Absence is an absent option, not an all-zero string, because zero is an element of $\mathbb{F}_{p_{\text{Pallas}}}$ and so a syntactically valid anchor (ZIP 374, “Optional anchors in PCZT v2”).

Proposition 12.13 (Deferred anchors). *Let P be a PCZT of a transaction of version 6.*

- (i) *If every spend of P is signed, replacing the anchor rt of a bundle and its proof π by any anchor rt' and a proof π' valid for it leaves every signature valid and the transaction identifier unchanged: re-anchoring needs only re-proving.*

- (ii) *A spend of an Orchard-protocol note whose creating transaction is not yet mined can be fully signed before that note is in the tree, and its signatures remain valid once an anchor, a witness and a proof are supplied: every field that the signature digest commits to is computable from the note data, and the anchor is not among them.*

Neither holds for a transaction of version 5, and (ii) fails for a Sapling spend in version 6.

Proof. (i) The version-6 digest and transaction identifier omit the anchors, which are authorising data (ZIP 229, “Anchor commitment (version 6)”), and by *Ironwood Guide*, §“Transaction digests and signatures”, Proposition “Bundle binding”, replacing (rt, π) by (rt', π') changes neither the transaction identifier nor any signed message, so every signature remains valid.

(ii) The effecting fields of an Orchard-protocol spend are nf , rk and its Action’s cv^{net} . The nullifier is $DeriveNullifier_{nk}(\rho, \psi, cm)$ (*Ironwood Guide*, §“The nullifier”), a function of the key and of note data, with ρ fixed by the creating Action and ψ and cm derived from $rseed$, ρ , the address and the value; it does not depend on the note’s position. The key rk and the commitment cv^{net} depend on keys, values and fresh randomness only.

For version 5 the digest commits to the anchors (ZIP 244, “Signature Digest”), so replacing an anchor invalidates every signature, which refutes (i); and an anchor fixed before the note is in the tree is the root of a tree state that does not contain the note, for which no authentication path of the note exists except with negligible probability (*Ironwood Guide*, §“Anchors”, Proposition “Membership soundness”), so a spend of non-zero value signed then admits no valid proof, so no signature made before the note is in the tree survives completion, which refutes (ii). A Sapling nullifier depends on the note’s position in the tree (protocol specification, §“Computing ρ values and Nullifiers”), which is unknown until the note is mined, and is effecting data; for Sapling only (i) holds. $\square$

ZIP 374 calls the two workflows re-anchoring and the spending of unmined notes; Proposition 11.4 (c) is the latter for the steps of a plan.

12.6 Construction and IO finalisation

The transaction of a PCZT is of version 5, with version group identifier $0x26A7270A$ (protocol specification, §“Transaction Consensus Rules”), or of version 6, with $0x0884B698$ (ZIP 229), whose consensus branch identifier names NU6.3 or a later upgrade; version 4 is not representable, and is invalid from NU7 on (ZIP 2003, deployed by ZIP 259). An Ironwood component requires version 6, the mixed case being rejected by the rule of §12.5. The consensus branch identifier fixes the rules in force, and a role that does not recognise it cannot check the PCZT against them (ZIP 374, “Global”).

Creator. The Creator chooses $tx_version$, $version_group_id$ and $consensus_branch_id$, sets the other Global fields with $tx_modifiable = 1000\ 0011$ in binary, and sets every bundle to its canonical empty bundle (Definition 12.8); for $tx_version = 5$ it sets the Sapling and Orchard anchors, and for $tx_version = 6$ it MAY leave them to an Updater (Definition 12.12) (ZIP 374, “Creator”).

Constructor. Before adding anything the Constructor MUST check the bit of $tx_modifiable$ for the kind added (Definition 12.5), and it updates the bundle’s $value_sum$. Per Orchard-protocol

Action, in the notation of the *Ironwood Guide*, it sets $cv^{\text{net}} := \text{ValueCommit}_{\text{rcv}}^{\text{Orchard}}(v^{\text{old}} - v^{\text{new}})$ for a fresh rcv (§“Value commitments”); the nullifier nf^{old} of the consumed note (§“The nullifier”); a uniform α and $rk := ak^P + [\alpha] G^{\text{SpendAuth}}$ (§“Randomised validating keys”); and the created note with $\rho^{\text{new}} := nf^{\text{old}}$, its cmx (§“Note commitments”), epk and C_{enc} (§“Encryption to the recipient”) and C_{out} (§“The outgoing ciphertext”); together with the context fields of the spend and the output. Dummy and fabricated Actions and their order are those of Construction 11.18, only fully dummy Actions carrying dummy_sk. Spends and outputs MUST agree with the bundle flags. When the bundle’s anchor is set, each spend added carries a witness rooting to it, or, for a note of value zero, whose path the Action statement does not check, any placeholder; for tx_version = 6 a spend MAY be added without a witness, even of a note not yet in the tree; for tx_version = 5 every non-dummy spend is of a note in the tree, with a witness to the anchor. A transparent input with a lock-time requirement is added only if its lock type is compatible with those of the other inputs and, when inputs are already signed, it leaves the lock time unchanged (§12.10) (ZIP 374, “Constructor” and “OrchardBundle”).

Construction 12.14 (IO finalisation). Once the input and output set is final, the IO Finalizer performs the following steps, failing where stated (ZIP 374, “IO Finalizer”).

- (1) If shielded spends or outputs exist, it clears bits 0, 1 and 7 of tx_modifiable.
- (2) For each Orchard-protocol bundle with Actions 1, ..., n , it requires every rcv _{i} , sets bsk := $\sum_i rcv_i \in \mathbb{F}_{p_{\text{Vesta}}}$, computes

$$bvk := \sum_i cv_i^{\text{net}} - [v^{\text{balance}}] V^{\text{Orchard}},$$

with v^{balance} the integer that the bundle’s value_sum denotes (§12.3), and fails unless $[bsk] R^{\text{Orchard}} = bvk$ (*Ironwood Guide*, §“The binding signature”). The Orchard and the Ironwood bundles are finalised separately, each by this computation on its own Actions.

- (3) For the Sapling bundle, it sets bsk to the sum of the spends’ rcv minus the sum of the outputs’ rcv, checked in the same way against the Sapling binding validating key (protocol specification, §“Balance and Binding Signature (Sapling)”).
- (4) It computes the signature digest over the effecting data (*Ironwood Guide*, §“Transaction digests and signatures”), signs each dummy spend under the spend authorising key derived from its dummy_sk or given by its dummy_ask, randomised by the spend’s α , and clears that key.
- (5) For each bundle with enableCrossAddress = 0, it MUST verify that each Action’s output recipient equals its spend recipient, and fail otherwise.

The condition of step (5) is condition A10 of the Action statement (*Ironwood Guide*, §“The Action statement”), so no valid proof exists for an Action that violates it; the step reports the violation at the role responsible for it.

Lemma 12.15 (Value-sum consistency). *Let an Orchard-protocol bundle have Actions with $\text{cv}_i^{\text{net}} = \text{ValueCommit}_{\text{rcv}_i}^{\text{Orchard}}(v_i^{\text{net}})$, and let v^{balance} be the integer that its `value_sum` denotes. Step (2) of Construction 12.14 passes if and only if $\sum_i v_i^{\text{net}} \equiv v^{\text{balance}} \pmod{p_{\text{Vesta}}}$, and `bsk` is then a binding signing key whose validating key is `bvk`. The check reads no value v_i^{net} , which may therefore have been redacted.*

Proof. With $\text{bsk} = \sum_i \text{rcv}_i$,

$$\text{bvk} - [\text{bsk}] R^{\text{Orchard}} = \left[\sum_i v_i^{\text{net}} - v^{\text{balance}} \right] V^{\text{Orchard}},$$

which is $\mathcal{O}$ if and only if the coefficient vanishes modulo p_{Vesta} , since V^{Orchard} is a non-identity point of the group of prime order p_{Vesta} . This is *Ironwood Guide*, §“The binding signature”, Proposition “Balance and key knowledge”, part (i). No independence of V^{Orchard} and R^{Orchard} is used. The check computes `bvk` from the commitments and `value_sum` alone. $\square$

The binding signing key cannot wait for the Transaction Extractor, since every `rcv` would then travel to it and open every value commitment to every intermediate party; nor can Constructors maintain it incrementally, since serial Constructors would have to update it and parallel ones, whose results a Combiner merges, must not. One role run once the set is final removes both problems, and the same role signs dummy spends so that no Signer parses spending-key material (ZIP 374, “Adding an IO Finalizer role”).

12.7 Signing

Digests. The Signer derives the effecting data from the PCZT and computes the shielded signature digest `SigHash` itself (ZIP 244 for version 5, as amended by ZIP 229 for version 6; *Ironwood Guide*, §“Transaction digests and signatures”), neither proofs nor signatures entering it; for each transparent input it signs, it computes that input’s digest under the input’s `sighash_type` (ZIP 244, “S.2: transparent_sig_digest”) and signs it by the scheme of the *Crypto Guide*, §“The transparent layer’s schemes: ECDSA and BIP-340 over secp256k1”. A Signer that cannot sign under an input’s hash type MUST NOT sign that input (ZIP 374, “TransparentInput”).

Orchard-protocol spends. For an Action it signs, with α present, the Signer sets $\text{rsk} := \text{ask} + \alpha$, fails unless $\text{rk} = [\text{rsk}] G^{\text{SpendAuth}}$, and sets `spend_auth_sig` to the RedPallas signature on `SigHash` under `rsk` (*Ironwood Guide*, §“Randomised validating keys”); Ironwood-pool Actions are signed in the same way in their own bundle. A signature produced elsewhere, for example by a hardware device, is attached if and only if it is valid under the Action’s `rk` on `SigHash`. The Signer then updates `tx_modifiable` (Definition 12.5) (ZIP 374, “Signer” and “OrchardSpend”).

Definition 12.16 (Signer relations). For an Orchard-protocol Action whose explicit data are present, with v^{old} and v^{new} the values of its `spend` and its `output`, the *Signer relations* are, in the notation of the *Ironwood Guide*:

- (a) $\text{cv}^{\text{net}} = \text{ValueCommit}_{\text{rcv}}^{\text{Orchard}}(v^{\text{old}} - v^{\text{new}})$ (§“Value commitments”);

- (b) the consumed note rebuilt from (recipient, v^{old} , ρ , rseed) has nullifier nf under the nk of fvk (§“The nullifier”), and fvk owns the recipient: $\text{pk}_d = [\text{ivk}] \text{g}_d$ for the ivk that fvk determines (§“Diversified addresses”);
- (c) $\text{rk} = \text{ak}^{\text{P}} + [\alpha] G^{\text{SpendAuth}}$ for the ak of fvk (§“Randomised validating keys”);
- (d) cmx is the commitment of the created note rebuilt from (recipient, v^{new} , rseed) with $\rho^{\text{new}} = \text{nf}$ (§“Note commitments”);
- (e) for an output that is not fabricated, esk derived from rseed and ρ^{new} gives epk and a C_{enc} that decrypts to a plaintext with the explicit recipient, value and rseed (§“Encryption to the recipient”), and, when ock is present, C_{out} decrypts under it to $(\text{pk}_d, \text{esk})$ (§“The outgoing ciphertext”).

A fabricated zero-valued output at a spent note’s receiver carries a random C_{enc} (Construction 11.18) and is exempt from (e): a Signer MUST NOT reject it on that ground alone, and SHOULD classify it as a tolerable dummy, checking (d) if it wishes (ZIP 374, “Signer”, “Verifier” and “Fabricated same-address outputs (NU6.3)”).

Obligations before signing (ZIP 374, “Signer”). A Signer MUST reject a PCZT that carries any dummy_sk or dummy_ask: the IO Finalizer signs dummy spends and clears these keys, so their presence asks the Signer to parse spending-key material. It MUST, for each output with a user_address, parse it and confirm that it contains the output’s recipient (§12.3). It SHOULD verify that the explicit values and note data it shows its user are consistent with the effecting data it signs, through the relations of Definition 12.16; a field absent because it was redacted is not an inconsistency, and only a present field that contradicts the effecting data is. The relations need the value and note commitments, BLAKE2b and the functions built on it, nullifier derivation, key agreement, the key derivation and the AEAD of note encryption, the signature digest and RedPallas signing (RedJubjub for Sapling), and no proof system.

Theorem 12.17 (Signer safety). *Let a Signer hold the ask of an Orchard-protocol key generated with use_ksk = false, and suppose that for every PCZT P it signs it computes $m_P := \text{SigHash}(T_P)$ from the effecting data T_P of P , performs the obligations above, verifies relations (a) to (e) of Definition 12.16, with all their fields present, for every Action whose spend it signs, displays the values, recipients and address strings so checked, and signs under keys $\text{ask} + \alpha$ no message other than such digests m_P . Under the assumptions of Ironwood Guide, §“Authorisation of spends”, Theorem “Spend authority”, and Assumption “Collision resistance of BLAKE2b” (§“Transaction digests and signatures”), except with negligible probability:*

- (i) every accepted Action that consumes a note of the key carries a signature on some m_P ;
- (ii) every accepted transaction that carries an Action with a randomised validating key rk under which the Signer signed has the effecting data T_Q of some PCZT Q that the Signer signed with an Action of key rk; in it the Actions whose spends the Signer signed consume and create the notes, with the values and recipients, that the Signer displayed for Q , and

every other effecting field, balancing values and transparent data included, equals that of Q .

The theorem concerns content, not inclusion.

Proof. (i) The Signer is the signing oracle of the *Ironwood Guide*, §“Authorisation of spends”, Definition “Spend-authority experiment”, queried with (α, m_p) for α taken from the PCZT and so possibly chosen by an adversary, which that experiment admits; the full viewing key that the PCZT discloses is given to the adversary there. Theorem “Spend authority” bounds by a negligible function the probability of an accepted Action that consumes a note of the key without a queried digest.

(ii) Let T' be accepted and carry an Action with key rk and a signature σ' valid under rk on $\text{SigHash}(T')$. The reduction of the proof of Theorem “Spend authority”, which simulates the key holder from ak^P and answers queries with adversarial randomisers, turns a triple $(rk, \text{SigHash}(T'), \sigma')$ that is not a triple of an answer of the Signer into a forgery in *Ironwood Guide*, §“RedPallas unforgeability”, Proposition “Unforgeability under re-randomisation”; hence, except with negligible probability, $\text{SigHash}(T') = m_Q$ for a PCZT Q that the Signer signed under rk . The collision case of the proof of Proposition “Bundle binding” uses only that each node of the digest tree hashes a string from which its children and fields are recovered uniquely, which holds equally for the version-5 tree (ZIP 244, “TxId Digest” and “Signature Digest”), where the anchors are among the hashed fields and are effecting data (Definition 12.3); it therefore applies to both transaction versions and gives $T' = T_Q$ as effecting data, except with negligible probability under Assumption “Collision resistance of BLAKE2b”. The digest covers the value and script of each transparent input (§12.3), so a false value in Q yields a digest that validation does not compute and a signature that is not accepted. For each Action whose spend the Signer signed, relations (a) and (d) make the displayed data an opening of cv^{net} and of cmx , and by Lemma “Hiding and binding of net value commitments” and Proposition “Hiding and binding of note commitments” of the *Ironwood Guide* no other opening is feasible; relation (b) identifies the consumed note, whose commitment fixes its value; relation (e) makes the created note decryptable by its recipient (Proposition “Decryption correctness”); and the check of `user_address` ties each recipient to the displayed string. $\square$

Pre-authorisation (ZIP 374, “Signer” and “Anchors and pre-authorization”). For a transaction of version 6 a Signer MAY sign while the shielded anchors, witnesses or proofs are absent, since signatures commit to none of them (Proposition 12.13). A Signer asked to do so SHOULD be aware, and where applicable make its user aware, that a spend without a witness may be of a note that never exists on chain: the signature authorises the spend whether or not the note is created. For a transaction of version 5 the anchors are in the digest and MUST be present before signing.

Lemma 12.18 (Binding key after IO finalisation). *If IO finalisation (Construction 12.14) succeeded for an Orchard-protocol bundle, and its commitments cv_i^{net} and its value_sum are unchanged afterwards, then a binding signature under bsk on the signature digest is valid under the bvk that validation derives from the transaction’s cv_i^{net} and balancing value v^{balance} .*

Proof. Validation computes bvk from the same commitments and balancing value (*Ironwood Guide*, §“The binding signature”, Definition “Binding validating key”), and IO finalisation checked that it equals $[\text{bsk}] R^{\text{Orchard}}$ (Lemma 12.15). Correctness of RedPallas in the binding-signature instance (*Ironwood Guide*, §“The binding signature”, Construction “Binding signature”) gives validity. $\square$

The hypothesis of Lemma 12.18 holds for every PCZT reached by the roles after finalisation: cv^{net} is required effecting data, which no later role changes and the merge of §12.9 keeps equal, and a finalised bundle merges only with a bundle of equal Action count and value_sum .

12.8 Proving

Per Orchard-protocol Action the Prover requires the spend’s recipient, value, ρ , rseed, fvk, witness and α , the output’s recipient, value and rseed, and the Action’s rcv; the bundle’s anchor MUST be set, and before proving the Prover MUST verify that the path of the witness of every spend of non-zero value roots to it, and fail otherwise. For Sapling it requires per spend the recipient, the value, rcm or rseed, rcv, the proof generation key and the witness, and per output the recipient, the value, rseed and rcv. The Prover needs no spending key; a delegated Prover learns the note data of the bundle but gains no spending authority (§12.9) (ZIP 374, “Prover”).

Construction 12.19 (Proof creation from a PCZT). For an Orchard-protocol bundle with no Actions the Prover does nothing. Otherwise, for each Action i , it rebuilds the consumed note from its explicit data and the created note from its explicit data with $\rho_i^{\text{new}} = \text{nf}_i$; forms the primary input

$$(rt, \text{cv}_i^{\text{net}}, \text{nf}_i, \text{rk}_i, \text{cmx}_i, \text{enableSpend}, \text{enableOutput}, \text{disableCrossAddress})$$

with rt the bundle’s anchor, and the auxiliary input from the witness, the two notes, α_i , the components of fvk_i and rcv_i (*Ironwood Guide*, §“The Action statement”); and fails if the pair does not satisfy the Action statement, in particular if fvk_i does not own the consumed note or cmx_i differs from the commitment of the rebuilt created note. It then creates one Halo 2 proof for all Actions of the bundle (*Ironwood Guide*, §“The Action circuit and the Halo 2 proof”; *Halo 2 Guide*, §“The Halo 2 proof system”) and sets zkproof . The Ironwood bundle is proved in the same way with its own anchor. The primary input consists of effecting data and the anchor only.

An Action whose rk is the identity point $\mathcal{O}$ is invalid (protocol specification, §“Action Descriptions”; ZIP 256, “Orchard identity rk ”), and the Transaction Extractor MUST refuse it (ZIP 374, “Transaction Extractor”). A Prover that refuses it as well spares a proof of a transaction that cannot be valid; ZIP 374 does not state that check for the Prover, and it is designed but unspecified (Definition 1.1).

12.9 Combining, redaction, and privacy

Construction 12.20 (Binary merge). The *merge* $P \sqcup Q$ of two PCZTs is defined, or fails, as follows (ZIP 374, “Combiner”).

- (1) *Global*. The fields `tx_version`, `version_group_id`, `consensus_branch_id`, `fallback_lock_time`, `expiry_height` and `coin_type` MUST be equal. The field `tx_modifiable` merges bitwise, bits 0, 1 and 7 by AND and bit 2 by OR, bits 3 to 6 being required zero on both sides.
- (2) *Lists* of inputs, outputs, spends and Actions. If either side’s bundle is IO-finalised, its `bsk` being present, both sides have equally many entries and equal `value_sum`. Otherwise the entries at the positions common to both lists merge pairwise, and the extra entries of the longer list are appended provided the shorter side’s `tx_modifiable` permits additions of that kind, the merge failing otherwise. The merged shielded bundle’s `value_sum` is that of the side each of whose lists of the bundle is at least as long as the other side’s, the merge failing if neither side is; with lists of equal lengths the two MUST be equal. ZIP 374 leaves this unstated, so it is designed but unspecified (Definition 1.1).
- (3) *Required fields* of merged entries, and the bundle flags, `note_version` and, in version 1, `anchor`, MUST be equal.
- (4) *Optional fields* merge as $x \sqcup \perp = \perp \sqcup x = x$, and $x \sqcup y = x$ if $x = y$, failing otherwise; in version 2 this rule covers the anchors.
- (5) *Maps* merge by union of their keys, the values of a common key being required equal.

The Combiner of k PCZTs merges them into one that contains every field of each, or fails. It MUST NOT combine PCZTs that do not represent the same transaction, that is, PCZTs not derived by roles from a common ancestor, which rules (1) to (3) check structurally. Once the input and output set is final, the effecting data determine the transaction identifier, which then identifies the PCZT; for version 6, re-anchored copies keep that identifier.

BIP 174 lets a combiner keep one of two conflicting values and discard the other. The PCZT merge requires equality instead, which is always possible because every field is typed and known in advance, so every conflict fails at the Combiner and none diverges later (ZIP 374, “Combiner rejecting non-identical duplicates”).

Lemma 12.21 (Commutativity of the merge). *For PCZTs P and Q , the merge $P \sqcup Q$ is defined if and only if $Q \sqcup P$ is, and then the two are equal.*

Proof. By cases on the rules of Construction 12.20. The equality tests of rules (1) and (3) are symmetric; AND and OR are commutative; the rule for optional fields and the union of maps with equal values on common keys are symmetric; the condition on IO-finalised bundles is symmetric; in rule (2) the longer list and the side whose permission is consulted are determined by the lengths, not by the order of the arguments, and so is the merged `value_sum`, which is required equal at equal lengths; and the pairwise merges of common entries commute by induction on the

structure of the entries. □

ZIP 374 requires further that combining SHOULD be functionally order-independent, $\text{Combine}(f_A(p), f_B(p)) = f_A(f_B(p)) = f_B(f_A(p))$ for participants that apply f_A and f_B to p . That is a requirement on roles, not a property proved here for all of them; Proposition 12.22 proves it for the Prover and the Signer.

Proposition 12.22 (Order independence of proving and signing). *Let p be an IO-finalised PCZT on which the Prover (Construction 12.19) and a Signer (§12.7) are both defined, the anchors being set for proving and, for a transaction of version 5, fixed before signing (Definition 12.12). Fix the random coins of both roles. Then $\text{Prove}(\text{Sign}(p))$ and $\text{Sign}(\text{Prove}(p))$ are defined and both equal $\text{Prove}(p) \sqcup \text{Sign}(p)$: the two roles run in either order, or in parallel on copies whose outputs are merged by Construction 12.20. For a transaction of version 6 signing may moreover precede the setting of the anchors (Proposition 12.13). If a copy is redacted before either role, and retains the fields that role reads, the equality holds up to the redacted fields, which the merge restores from the other side.*

Proof. Signing. The signed digest is computed from effecting data only, excluding proofs and signatures (Lemma 11.21), and the relations of Definition 12.16 read effecting and context data; Prove writes neither, so Sign reads the same values on p and on $\text{Prove}(p)$ and, with its coins fixed, writes the same signatures and bits. *Proving.* The primary input of the Action statement consists of effecting data and the anchor, and the auxiliary input of context data (Construction 12.19; *Ironwood Guide*, §“The Action statement”); Sign writes none of them, and the Prover reads no spending key, so Prove writes the same proofs on p and on $\text{Sign}(p)$. *Merge.* The role Prove sets only proof fields, absent in p , and Sign only signatures, absent in p , and bits of tx_modifiable, which it only clears, or sets in the case of bit 2; the bundle of p is finalised on both sides with equal counts and value sums. Neither role writes a required field, so rule (3) holds, and a transparent signature is a new key of partial_signatures, merged by rule (5). Rules (1) to (5) of Construction 12.20 therefore give $\text{Prove}(p) \sqcup \text{Sign}(p) = \text{Prove}(\text{Sign}(p)) = \text{Sign}(\text{Prove}(p))$. ZIP 374, “Prover”, states the same independence. □

Redactor (ZIP 374, “Redactor” and the field descriptions). Anyone may clear fields that later roles do not need: per Orchard-protocol Action the spend’s spend_auth_sig, recipient, value, ρ , rseed, fvk, witness, α , derivation record, dummy key and proprietary entries, the output’s recipient, value, rseed, ock, derivation record, user_address and proprietary entries, and rcv; the corresponding Sapling fields; and the transparent and Global lookup data. A PCZT with inputs of several independent Signers is sent to each with only what that Signer needs, for example without the other Signers’ values of α .

Proposition 12.23 (Disclosure by a PCZT). *A PCZT carries strictly more than its transaction: for each shielded spend or output it may carry the counterparty address, the value, the note randomness, rcv, which opens cv^{net} , the full viewing key of the spending account, derivation records, ock, which opens C_{out} , and address strings. Every recipient of a PCZT at a stage learns all data present at that stage; a delegated Prover learns the note data of the spends and outputs but gains no spending authority; the extracted transaction contains none of these data. Participants SHOULD*

send a PCZT only to entities needed for the remaining roles and *SHOULD* redact what those do not need, and wallets *SHOULD* protect stored PCZTs as they protect their note records (ZIP 374, “Privacy Implications”).

Proof. The listed fields are context data (Definition 12.3), which extraction omits (§12.10); a recipient reads every field present. The Prover’s data (Construction 12.19) contain fvk but no ask, and an accepted spend of a note of the key without a signature under $\text{ask} + \alpha$ on a digest that the key holder signed is excluded, except with negligible probability, by *Ironwood Guide*, §“Authorisation of spends”, Theorem “Spend authority”, under its assumptions, whose adversary holds the full viewing key. $\square$

12.10 Spend finalisation and extraction

Spend Finalizer (ZIP 374, “Spend Finalizer” and “TransparentInput”; BIP 11; BIP 16). For each transparent input with enough data, the Spend Finalizer builds `script_sig`: for P2PKH, the push of the one signature, with its hash-type byte, under the key that hashes to the address, followed by the push of that key; for P2SH multisignature, the opcode `OP_0`, the pushes of m signatures in the key order of the redeem script and the push of the redeem script, failing with fewer than m signatures. It *MUST* fail on a signature whose hash type is not the input’s `sighash_type`. It *SHOULD* then clear the input’s redeem script, partial signatures, derivation records and preimages. It keeps value and `script_pubkey`, required fields that the digest needs, and sequence and the lock-time requirements, which the digest and Definition 12.24 read at extraction; ZIP 374’s “all other optional data” would include these last fields, a gap in the specification.

Definition 12.24 (Lock-time determination). If no transparent input has a required time or height lock, the lock time is `fallback_lock_time`, or 0 if it is absent. Otherwise the lock type is one supported by every input that states a requirement, an input that states both types supporting both; an input that states neither supports both. If both types remain possible, the height type is chosen; if none remains, the requirements are unsatisfiable. The lock time is the maximum required value of the chosen type (ZIP 374, “Determining Lock Time”).

Construction 12.25 (Transaction extraction). The Transaction Extractor checks, failing on any violation (ZIP 374, “Transaction Extractor”), that

- (i) `tx_version`, `version_group_id` and `consensus_branch_id` name a supported format, and the lock-time requirements are satisfiable (Definition 12.24);
- (ii) for `tx_version` = 5 the Ironwood bundle is canonical empty;
- (iii) every `script_sig` is set;
- (iv) every Sapling spend has its proof and signature, and every Sapling output its proof;
- (v) every Orchard-protocol bundle with $n \geq 1$ Actions has a zkproof of the canonical length of $2720 + 2272 n$ bytes (*Ironwood Guide*, §“The transaction format”), and every Action a `spend_auth_sig`;

- (vi) every non-empty shielded bundle has its anchor and bsk;
- (vii) no rk is $\mathcal{O}$.

It then sets each bundle’s balancing value to v^{balance} , failing if it lies outside the signed 64-bit range of the transaction field; sets the lock time by Definition 12.24; computes SigHash; for each bundle derives bvk from its value commitments and balancing value, fails unless it matches bsk, and signs SigHash under bsk, by RedPallas for the Orchard and Ironwood bundles and by RedJubjub for Sapling (Lemma 12.18); and outputs the transaction, without any context data (ZIP 229). It SHOULD then verify every proof and signature of that transaction, each `spend_auth_sig` under its rk on SigHash included, so that an invalid transaction is found before submission.

A party that holds an Orchard-protocol output’s recipient, value and rseed, with ρ^{new} the nf of its Action’s spend, recovers the full note plaintext, memo included, without ock or any viewing key, by the derivation of esk stated with Theorem 11.22 (§11.8; *Ironwood Guide*, §“Encryption to the recipient”, Proposition “Decryption correctness”; ZIP 212). The random C_{enc} of a fabricated output decrypts to nothing, as the construction intends.

13 The transaction lifecycle

This section states what a wallet does with a transaction between its creation and a terminal state: the encoding it submits and the identifier by which it tracks the transaction; the rule that a transaction is recorded before it is released, with the completeness of the record that follows; submission and rebroadcast; the status observations that update the record; and the terminal states, re-sending and the consensus branch. A transaction is *created* when it is built, proved and authorised from one step of a plan (Definition 11.2; Constructions 11.18 and 11.20). The record is that of Definition 9.11.

13.1 Submission and the transaction identifier

The wallet serialises a created transaction in the encoding of its version: version 5 (ZIP 225, “Transaction Format”) or version 6 (ZIP 229, “Transaction Format”), the only versions in force; a transaction with Ironwood-pool Actions has version 6 (*Ironwood Guide*, §“The transaction format”, Construction “Version-6 transaction”; protocol specification, §“Transaction Encoding and Consensus”). The argument of Submit (Definition 5.10) is this byte string. The encoding is cited, not restated.

The wallet tracks a transaction t by its *transaction identifier* $\text{txid}(t)$: the root of the BLAKE2b-256 digest tree of ZIP 244 over the effecting data of t ,

$$\text{txid}(t) = \text{h}(\text{ZcashTxHash_} \parallel \beta, d_{\text{hdr}} \parallel d_{\text{tr}} \parallel d_{\text{Sap}} \parallel d_{\text{Orc}} \parallel d_{\text{Irw}}),$$

where $\text{h}(P, x)$ is BLAKE2b-256 of x under the 16-byte personalisation P , β is the 4-byte little-endian consensus branch identifier, and the child d_{Irw} is absent in version 5 (*Ironwood Guide*, §“Transaction digests and signatures”, Definition “Transaction identifier”, for version 6, and

Remark “Version 5”; ZIP 244, “txid_digest”; ZIP 229, “Transaction Identifiers, Auth Digests, and Signature Digests”; protocol specification, §“Transaction Identifiers”). The header digest d_{hdr} hashes, under ZTxIdHeadersHash , the 4-byte little-endian encodings of header, nVersionGroupId , $\text{nConsensusBranchId}$, lock_time and nExpiryHeight (ZIP 244, “T.1: header_digest”), so the expiry height N of Definition 9.8 is effecting data. The wallet computes $\text{txid}(t)$ from t and needs no response of the server to know it.

Lemma 13.1 (Non-malleability of the identifier). *Let t be a transaction of version 5 or 6.*

- (i) *Replacing the authorising data of t , namely its proofs, signatures and transparent scripts and, in version 6, its anchors (anchorSapling, anchorOrchard and anchorIronwood, ZIP 229, “Anchor commitment (version 6)”); Ironwood Guide, §“Transaction digests and signatures”, Definition “Authorising-data digest; effecting and authorising data”, leaves $\text{txid}(t)$ unchanged.*
- (ii) *Under the Ironwood Guide’s Assumption “Collision resistance of BLAKE2b” (§“Transaction digests and signatures”), no efficient algorithm outputs, except with negligible probability, two transactions with different effecting data and equal identifiers.*
- (iii) *Under the same assumption, a transaction built for one consensus branch and the same payment built for another have different identifiers, except with negligible probability.*

Proof. (i) Authorising data lie under the authorising-data digest and under no node of the tree whose root is txid (Ironwood Guide, §“Transaction digests and signatures”, Definition “Authorising-data digest; effecting and authorising data”). (ii) Each node of the tree hashes an injective encoding (fixed-size fields and length-prefixed scripts, concatenated in order) of effecting data or of the digests of its children (ZIP 244, “txid_digest”). If the effecting data differ, the inputs of some node differ. A node whose inputs differ either has equal digests, which is a collision, or passes different digests to its parent, whose inputs then differ; at the root, equal identifiers from different inputs are a collision. (iii) The branch identifier enters both the root personalisation $\text{ZcashTxHash}_\parallel \beta$ and the header digest through $\text{nConsensusBranchId}$, so the two root pairs differ, and equal identifiers are a collision (Ironwood Guide, §“Transaction digests and signatures”, Remark “Consensus-branch binding”). $\square$

By (i) and (ii), the identifier tracks a pending transaction from its creation on, before and after any re-signing or re-proving. The identifier of versions up to 4, the double SHA-256 of the encoding (protocol specification, §“Transaction Identifiers”), is malleable; it belongs to no version in force and is not used.

Construction 13.2 (Recording and submission). The following rules are designed but unspecified (Definition 1.1).

- (L1) *Recording before release.* The wallet enters each transaction t it creates into its transaction record (Definition 9.11), which locks the notes t spends (Definition 9.12), in storage that survives restarts and rollbacks (Definition 7.23, item (v)), before it submits t or releases its bytes to any other party. It never removes an entry of the record.

- (L2) *Multi-step plans.* For a plan of $\ell > 1$ steps (Definitions 11.2 and 11.3), the wallet creates every step before it records any, records all ℓ transactions in one update of the record, and submits them after that update, in step order. A construction that fails at some step leaves no recorded, hence by (L1) no transmittable, prefix. Every step can be created before any is mined, since a well-formed plan chains steps only through transparent outputs (Proposition 11.4); the transfer to a TEX address of Construction 11.7 is the instance (ZIP 320, “Design considerations for Senders”).
- (L3) *Submission.* The wallet submits t by `Submit(bytes)`, whose response is `accepted(txid)` or `rejected` (Definition 5.10). Neither response changes the record. It submits at a tip T that leaves a margin of blocks, a wallet parameter, before the last includable height N (Definition 9.8), rather than at $T = N - 1$.
- (L4) *Rebroadcast.* While t is pending and unmined (Definition 13.8), the wallet resubmits it at intervals whenever, at its tip T , $N = 0$ or $T < N$ (Lemma 9.9) and no transaction included in its view of the chain spends an input of t . For $N = 0$ this continues until t is mined or an input of t is spent by a mined transaction. The rule is motivated by ZIP 203, “Specification”, under which nodes drop a transaction from their mempools at its expiry.

The semantics of the two responses of `Submit` are those of the server’s node and are fixed by no ZIP: `accepted` means that the node admitted t to its mempool, `rejected` that it refused it. Under clause (a) of Assumption 5.2, acceptance implies neither propagation to other nodes nor inclusion in a block, since relay policy is local to each node and may refuse a transaction that consensus still admits in the next block (Lemma 9.9); and rejection does not imply that t is invalid. Rule (L3) keeps a margin for this reason. Each submission is an observation of the server (Proposition 8.21).

Proposition 13.3 (Completeness of the transaction record). *Let the wallet follow rule (L1), and let no party other than the wallet hold the bytes of a created transaction t before the wallet submits them. Then, if a block of any chain includes a transaction with the effecting data of t , the record contains t , and the included transaction has identifier $\text{txid}(t)$, except with negligible probability. The converse fails: a recorded transaction that the network never received is stale, not lost, and, for $N \neq 0$, reaches the terminal state expired unmined of Definition 13.8; for $N = 0$ it stays pending by item (c) until the end of its consensus branch is known, and then reaches expired unmined with the bound N^* (“Consensus-branch bound” below).*

Proof. A block includes a transaction only if the party that assembled the block held its bytes. The first party other than the wallet to hold bytes of t received them by the wallet’s submission, by hypothesis; by (L1) the wallet recorded t before that event, and the entry survives restarts and rollbacks and is never removed. A valid transaction with the effecting data of t carries, for each spend of t other than a dummy, a signature under a key of the wallet on the signature digest those data fix; except with negligible probability no party obtains such a signature except from bytes the wallet released (*Ironwood Guide*, §“RedPallas unforgeability”, Proposition “Unforgeability under re-randomisation”, for Orchard-protocol spends; for transparent and Sapling spends the unforge-

ability of those schemes is assumed, as in §11.8, where no lower volume proves it). Bytes derived from those of t by another party with the effecting data of t differ from t at most in authorising data, so the included transaction has identifier $\text{txid}(t)$ by part (i) of the lemma above. The converse fails because recording does not imply delivery. $\square$

Where a partially created transaction is extracted by another party (“Spend finalisation and extraction”, §12.10), that party holds the bytes of t before any submission, and the hypothesis is arranged differently: the wallet records $\text{txid}(t)$, N and the inputs of t , locking them, before it releases the partially created transaction to any party that may extract it, and at a point where the effecting data of t are final. After IO finalisation (Construction 12.14) of a partially created transaction with shielded spends or outputs, or, for one that is transparent-only, once signatures without SIGHASH_ANYONECANPAY and without SIGHASH_NONE have cleared bits 0 and 1 of tx_modifiable (ZIP 374, “Global”), the inputs and outputs are final, the lock time is determined by Definition 12.24, and the proofs and signatures still to be added are authorising data; by part (i) of the lemma the extracted transaction then has the recorded identifier, and the proof above applies with release in place of submission. The entry acquires the encoding of t when the wallet obtains it.

Remark 13.4 (Retained data). From creation until it is mined, t exists only in the wallet’s record, in remote mempool and, for an extracted partially created transaction, at the extracting party. Once the wallet holds the encoding of t (Definition 9.11, item (i)) the record keeps it, and rule (L4) needs it; before that the wallet does not rebroadcast t . It may hold data that the chain never will: a transaction that ends expired and unmined lies on no chain that agrees with the server’s up to N (Lemma 13.7), and its entry may be the only trace of it.

13.2 Status tracking and terminal states

Remark 13.5 (Scope of scanning). Scanning detects a transaction only through a nullifier of the watched set W (Definition 9.15) or an output accepted under the wallet’s scanning keys (Construction 6.8). A recorded transaction that spends no note of W and has no such output, every fully transparent transaction among them, is invisible to scanning. Its fate is learnt only by queries $\text{Status}(\text{txid})$ (Definition 5.9), each of which puts its identifier into the view of the server (Proposition 8.18).

For a recorded transaction t write N for its expiry height, h_t for its mined height ($\perp$ while unmined), h_t^{obs} for its earliest observation height, and u_t for the latest tip at which the server reported it unmined ($\perp$ initially): items (ii), (iv), (v) and (vi) of Definition 9.11. A *status observation* (s, T) is a response s of $\text{Status}(\text{txid}(t))$ received while the wallet’s view of the chain has tip T .

Construction 13.6 (Status transitions). The record entry of t changes only as follows.

- (i) On an observation (s, T) with $s \in \{\text{unknown}, \text{unmined}\}$: $u_t := T$.
- (ii) On an observation $(\text{mined}(b'), T)$: $h_t := b'$, $h_t^{\text{obs}} := \min(h_t^{\text{obs}}, b')$ and $u_t := \perp$.
- (iii) Scanning a block of height b' in which t is wallet-relevant (Construction 6.8) has the effect of (ii).

- (iv) A rollback to r (Definition 7.23) sets $h_t := \perp$ if $h_t > r$, and $u_t := \min(u_t, r)$ if $u_t \neq \perp$, since a report at a tip above r concerned blocks that the rollback discards.

Query termination. The wallet issues $\text{Status}(\text{txid}(t))$ for a pending transaction t with $h_t = \perp$: if $N \neq 0$, until $u_t \geq N$, that is, until t is reported unmined at a tip of at least N ; if $N = 0$, for as long as t is pending and unmined. No rule stops at a first report of unmined or at a fixed depth after first observation. A rollback that lowers u_t below N , or sets $h_t := \perp$, resumes the queries. The transitions and the termination rule are designed but unspecified (Definition 1.1); the termination rule rests on the consensus rule of Definition 9.8 (ZIP 203, “Specification”). For an entry with N_t unknown (Definition 9.11), the wallet obtains the encoding by $\text{Tx}(\text{txid})$ (Definition 5.10); until then it queries as for $N = 0$, does not rebroadcast, and its inputs are locked by condition (d) of Definition 9.12.

Lemma 13.7 (Expired unmined is final on its chain). *Assume clause (a) of Assumption 5.2. Let t be a transaction created by the wallet, with expiry height $N \neq 0$, and let (s, T) be an observation with $s \in \{\text{unknown}, \text{unmined}\}$ and $T \geq N$, answered from a best chain C of the server (Consensus Guide, §“Work, and the best chain”) whose blocks at heights at most N are those of the wallet’s view. Then no chain whose blocks at heights at most N are those of C includes t .*

Proof. The wallet’s view has tip $T \geq N$, so C has blocks at every height up to N . By clause (a), s is the status of $\text{txid}(t)$ on C (Definition 5.9): no block of C contains t , so no block of height at most N of a chain that agrees with C up to N does. The transaction t is not a coinbase transaction, and by Lemma 9.9 the expiry rule permits it in no block of height greater than N (protocol specification, §“Transaction Consensus Rules”; ZIP 203, “Specification”). $\square$

A reorganisation with fork point below N reopens the question, since the new chain need not agree with C up to N . The wallet meets it as a continuity error (Definition 6.10), and the rollback of Construction 7.24 to some $r < N$ sets $u_t \leq r < N$ by transition (iv), which resumes the queries.

Let B be the birthday of the wallet (Definition 6.2) and $\mathcal{H}$ its retained block boundaries (Definition 7.12). Write $r_{\min}$ for the least element of $\mathcal{H} \setminus \{B - 1\}$, the lowest rollback point of the rollback window, and $r_{\min} := B - 1$ when that set is empty.

Definition 13.8 (Terminal states). A recorded transaction t is *pending* from its recording until it reaches one of the following states.

- (a) *Mined*: $h_t \neq \perp$ and $h_t \leq r_{\min}$, so that no rollback to a point of the window unmines t . The state is terminal relative to the rollback window only, since consensus bounds no reorganisation depth (Remark 7.28); a deeper rollback, to $B - 1$ or to a fetched chain state in steps (1) and (3) of Construction 7.24, returns t to pending by transition (iv).
- (b) *Expired unmined*: $N \neq 0$, $h_t = \perp$ and $u_t \geq N$, which gives the conclusion of Lemma 13.7 when the server’s chain agrees with the wallet’s view up to N ; a disagreement surfaces as a continuity error (Definition 6.10), whose rollback to $r < N$ returns t to pending by transition (iv). Its inputs are released (Proposition 9.13, every later target height exceeding N), its entry is kept, and the user is notified.

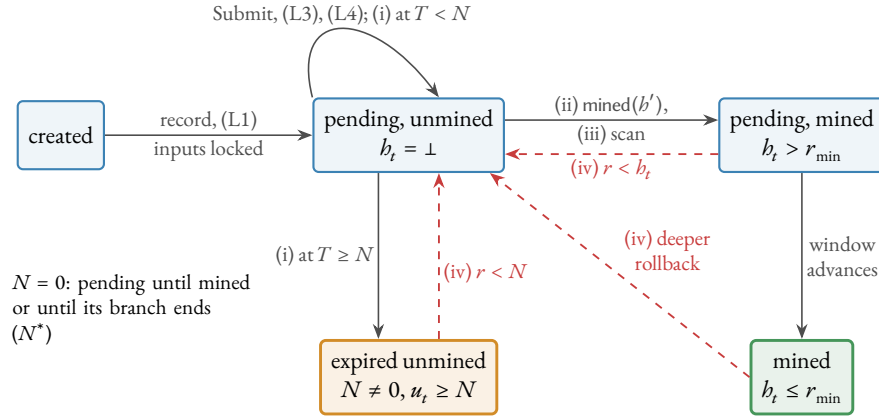


Figure 6. The states of a recorded transaction t (Definition 13.8). Solid edges are transitions of the record: recording by rule (L1), which locks the inputs of t ; submission, rebroadcast and reports unmined or unknown at a tip below N , which leave t pending; a report mined(b') or the scan of a block containing t , transitions (ii) and (iii); burial below the lowest rollback point $r_{\min}$ of the window; and, for $N \neq 0$, a report of an unmined t at a tip $T \geq N$, after which Lemma 13.7 excludes t from every chain that agrees with the server's up to N . Dashed red edges are rollbacks, transition (iv): below b_t from either mined state, from the terminal one only by a rollback deeper than the window, and below N from expired unmined. The green and amber states are terminal; a transaction with $N = 0$ stays pending until mined, unless its consensus branch ends.

- (c) For $N = 0$, a value chosen explicitly: t remains pending until it is mined, rebroadcast by rule (L4), or, once the end of its consensus branch is known, until it reaches state (b) with the bound N^* of that branch (“Consensus-branch bound” below). Its mining is detected by scanning through $\mathcal{W}$, from which a spend, $N = 0$ included, removes a nullifier only once mined (Definition 9.15), or by continued status queries.

The states and their transitions are designed but unspecified (Definition 1.1). The notification is specified: the wallet should notify the user of expired transactions that must be re-sent, and user interfaces and services must never rely on zero-confirmation transactions (ZIP 203, “Wallet behavior and UI”).

Remark 13.9 (Re-sending). Re-sending an expired payment usually spends the same TXOs, and ZIP 315 lists which spends of given TXOs are repeated across transactions among the kinds of information leakage, the case arising when a previous transaction expired (ZIP 315, “Kinds of information leakage”): the repeated spends link the expired transaction to its replacement. Selecting other notes avoids that link, but the nullifiers that the expired transaction revealed still link it to any later transaction that spends those notes (ZIP 315, “Rationale for anchor selection”). A wallet that re-sends accepts one of the two links. ZIP 315, “Note selection”, records as a TODO, not as a rule, that such revealed nullifiers be spent in a note-management transaction; how to avoid both links is an open problem (Definition 1.1).

Consensus-branch bound. The consensus branch bounds a pending transaction as well as its expiry height does. An authorisation binds the consensus branch identifier through β and `nConsensusBranchId`, which must equal the identifier of the branch under which t is validated, so a transaction authorised for one branch is invalid under another whatever its expiry height

(*Ironwood Guide*, §“Transaction digests and signatures”, Remark “Consensus-branch binding”, under its Assumption “Collision resistance of BLAKE2b”; ZIP 244, “txid_digest” and “T.1: header_digest”; ZIP 259, “Backward compatibility”; protocol specification, §“Transaction Consensus Rules”). A pending transaction built for a branch that ends at a height X , with $N = 0$ or $X \leq N$, can be mined only below X . Once X is known, write $N^* := X - 1$ if $N = 0$ or $N \geq X$, and $N^* := N$ otherwise. Rule (L4), the query termination of Construction “Status transitions”, state (b) of Definition 13.8 and conditions (b) and (c) of Definition 9.12 apply with N^* in place of N , and Lemma 13.7 holds with N^* , by the branch binding in place of the expiry rule. Thus t becomes expired unmined from a report at a tip of at least N^* , and its inputs are released for the rebuild at every target height $H \geq X$. This rule is designed but unspecified (Definition 1.1). To pay across the activation of the next branch, the wallet rebuilds and re-authorises the payment for the branch in force at its target height (Definition 9.4): a new transaction, with a new identifier by part (iii) of the lemma of §13.1 and a new entry of the record.

14 Payment requests (ZIP 321)

This section states the encoding by which a payee states payments to a payer. It gives the URI grammar of ZIP 321, the grouping of parameters into payments and the conditions under which a parser accepts a URI; then the meaning of the address, amount, memo, label and message parameters, the capacity of one request, and the passage from an accepted request to a transaction plan.

14.1 The URI grammar

A payment request precedes planning (§11.1): the payee states to the payer, in a form that a wallet reads mechanically, the recipient addresses, amounts and memos of the payments it asks for, and ZIP 321 fixes this encoding as a URI scheme. Its terms *payment* and *payment request* (ZIP 321, “Terminology”) are those of Definition 11.1. Everything that follows once the recipients are fixed is constructed in the earlier sections of this volume and in the *Ironwood Guide*, §“Note encryption”, §“Trial decryption and note acceptance”, §“Authentication paths”, §“Anchors” and §“Transaction digests and signatures”; this section re-derives none of it. The status of ZIP 321 is that of Table 1.

Definition 14.1 (Payment request URI). Let L be the set of strings derived from *zcashurn* in the following grammar in ABNF (RFC 5234), reproduced from ZIP 321, “URI Syntax”, with its longest rule continued on an indented line:

```
zcashurn      = "zcash:" ( zcashaddress [ "?" zcashparams ]
                        / "?" zcashparams )
zcashaddress  = 1*( ALPHA / DIGIT )
zcashparams   = zcashparam [ "&" zcashparams ]
zcashparam    = [ addrparam / amountparam / assetparam / memoparam
                        / messageparam / labelparam / otherreqparam / otherparam ]
NONZERO       = %x31-39
DIGIT         = %x30-39
paramindex    = "." NONZERO 0*3DIGIT
addrparam     = "address" [ paramindex ] "=" zcashaddress
```



```

amountparam  = "amount"    [ paramindex ] "=" 1*DIGIT [ "." 1*8DIGIT ]
assetparam    = "req-asset" [ paramindex ] "=" *base64url
labelparam    = "label"     [ paramindex ] "=" *qchar
memoparam     = "memo"      [ paramindex ] "=" *base64url
messageparam  = "message"   [ paramindex ] "=" *qchar
paramname     = ALPHA *( ALPHA / DIGIT / "+" / "-" )
otherreqparam = "req-" paramname [ paramindex ] [ "=" *qchar ]
otherparam    = paramname [ paramindex ] [ "=" *qchar ]
qchar         = unreserved / pct-encoded / allowed-delims / ":" / "@"
allowed-delims = "!" / "$" / "'" / "(" / ")" / "*" / "+" / "," / ";"

```

Here ALPHA, unreserved and pct-encoded are those of RFC 3986, Appendix A, and base64url is the alphabet of RFC 4648, Section 5, in which the values 62 and 63 are written - and _, with padding omitted. The grammar is read as follows: a parameter whose name is address, amount, label, memo, message or req-asset is derived only by its own production, and otherreqparam and otherparam derive only parameters with other names. A *payment request URI* is a string of L that meets the acceptance conditions (a) to (f) stated below in this subsection.

Without the reading, otherparam would derive amount=1%30, with name amount, no index and a qchar value; ZIP 321, “Invalid Examples”, rejects a URI with this parameter.

Surface forms. The grammar derives no // after the scheme, so a payment request URI has no authority component and is not hierarchical in the sense of RFC 3986, Section 1.2.3; ZIP 321, “Invalid Examples”, rejects zcash:// followed by an address. A URI zcash: A ? P , with A a zcashaddress, MUST be treated as zcash:?address= A & P (ZIP 321, “URI Semantics”). A request for a single payment SHOULD place its address in the hier-part; several payments are written with addrparam at distinct indices (same section). By this equivalence and condition (b) below, a URI zcash: A ?address= B &... is not accepted: after the rewriting it carries address twice at the empty index.

Indices and grouping. Identify the empty index with 0 and let $J := \{0, 1, \dots, 9999\}$. The *rendering* ι maps 0 to the empty string and $i \geq 1$ to $\cdot \parallel \text{dec}(i)$, where $\text{dec}(i)$ is the decimal representation of i without leading zeros. The strings that match [paramindex] are the empty string and the strings $\cdot d_1 \dots d_k$ with $1 \leq k \leq 4$, $d_1 \in \{1, \dots, 9\}$ and $d_2, \dots, d_k$ decimal digits; these are exactly the decimal representations without leading zeros of the integers 1 to 9999. Hence ι is a bijection from J onto the index suffixes, and its inverse is parsing. The strings $\cdot 0$, $\cdot 01$ and $\cdot 10000$ match no production, and ZIP 321, “Invalid Examples”, rejects address. $\cdot 0$ = and amount. $\cdot 0$ =. Parameters whose suffixes denote the same index $i \in J$, the empty index included, belong to one payment P_i ; the order of the parameters is insignificant, and indices need not be consecutive (ZIP 321, “URI Semantics”). A URI therefore decodes to a finite map $i \mapsto P_i$ on a subset of J , and carries at most $|J| = 10,000$ payments: 9999 at indices 1 to 9999 and one at the empty index.

Lexical rules. Percent encoding (RFC 3986, Section 2.1) occurs only inside qchar, that is, only in the values of label, message, otherreqparam and otherparam. Addresses, amounts, indices and parameter names are literal, so amount=1%30, %61mount=1 and an address beginning with %74 are not in L (ZIP 321, “URI Syntax” and “Invalid Examples”). The values of memo and req-asset are base64url without padding, and implementations MUST NOT accept the characters +, / and =, which occur only in the standard base64 alphabet (ZIP 321, “URI Syntax”; RFC 4648, Section 5).

Acceptance. A parser accepts a string u as a payment request URI only if all of the following hold (ZIP 321, “URI Syntax”, “URI Semantics” and “Forward compatibility”):

- (a) $u \in L$ under the reading of Definition 14.1; a purported ZIP 321 URI that cannot be so parsed **MUST NOT** be accepted;
- (b) after the hier-part address is rewritten as address at the empty index, no pair of a parameter name and an index occurs twice (**MUST NOT**);
- (c) every index that carries a parameter other than address carries an address (**MUST**);
- (d) u names no parameter with prefix req- that the parser does not recognise, since such a parameter makes the entire URI invalid (**MUST**); other unrecognised parameters are ignored (**SHOULD**);
- (e) the address, amount and memo rules of §14.2 hold;
- (f) u decodes to at least one payment, a payment request containing one or more payments (ZIP 321, “Terminology”).

The prefix req- is part of the name of a parameter defined by a later revision of ZIP 321, not a modifier of an arbitrary parameter; the parameters address, amount, label, memo and message never carry it, since every conformant parser is required to understand them (ZIP 321, “Forward compatibility”). An accepted URI asks for one transaction: implementations **SHOULD** construct a single transaction that pays every address of the request (ZIP 321, “URI Semantics”).

Custom assets. The parameter req-asset is specified in ZIP 321, “Custom Assets”, with an encoding defined by ZIP 227, which the protocol described in this volume does not include. A parser of this protocol does not recognise req-asset, so by condition (d) a URI that carries it is invalid to that parser, and the further rule that amount and req-asset exclude each other at one index (ZIP 321, “URI Semantics”) never applies.

Remark 14.2 (Case of literals). Quoted strings in ABNF are case-insensitive (RFC 5234, Section 2.3), so L contains strings beginning ZCASH: and strings with the parameters AMOUNT=1 and REQ-ASSET=; the value ranges NONZERO and DIGIT, given as %x ranges, and the base64url alphabet are case-sensitive. ZIP 321 does not address case: it states neither whether such URIs are to be accepted nor whether amount and AMOUNT at one index are the same parameter for condition (b). The volume states the grammar as written and resolves neither question; no parser’s behaviour is normative. Class: open problem (Definition 1.1).

14.2 Parameter semantics

Addresses. Implementations **SHOULD** check that each zcashaddress is a valid encoding of an address other than a Sprout address under protocol specification, §“Encodings of Addresses and Keys”: a transparent address (§“Transparent Addresses”, Base58Check), a Sapling payment address (§“Sapling Payment Addresses”, Bech32, ZIP 173) or a unified address (§“Unified Payment Addresses and Viewing Keys” and ZIP 316, Bech32m, BIP 350). Address formats later added to that section **SHOULD** be supported; where the network of the request is known, each address

SHOULD be checked to belong to it; and every requirement of ZIP 316 applies to a unified address (ZIP 321, “URI Semantics”). Sprout addresses MUST NOT be supported (same section). ZIP 321 gives as its reason the restriction of ZIP 211 on adding value to the Sprout pool; in the protocol described, version-4 transactions are invalid, and Sprout transfers exist only in transactions of versions 2 to 4, so Sprout funds are unspendable (ZIP 2003, “Abstract”; ZIP 259, “Abstract”; *Ironwood Guide*, §“The transaction format”). A TEX address (ZIP 320) is in no section of the specification, and ZIP 321 names none; its reading as the recipient address of item 4 of Definition 3.20 is designed but unspecified (Definition 1.1). The addresses of an accepted request are thus recipient addresses in the sense of Definition 3.20.

Construction 14.3 (Amount codec). Let $\text{COIN} = 10^8$ zatoshi per ZEC (*Ironwood Guide*, Definition “Unit of value” in §“Notes”) and $\text{MAX_MONEY} = 2.1 \cdot 10^{15}$ zatoshi (*Ironwood Guide*, §“The transaction format”), so that 21,000,000 ZEC is MAX_MONEY zatoshi. For a string x of decimal digits, let $\text{int}(x)$ be the natural number it denotes, leading zeros ignored, and let $\emptyset^k$ be the string of k characters $\emptyset$.

- (1) *Parse*. Let s match $1^*\text{DIGIT} [\text{"."} 1^*8\text{DIGIT}]$, and write $s = c$ or $s = c \parallel \cdot \parallel f$ with c its whole part and f its fraction. Set $z := \text{int}(f \parallel \emptyset^{8-|f|})$, or $z := 0$ without a fraction, and

$$v := \text{int}(c) \cdot \text{COIN} + z.$$

Accept s if and only if $v \leq \text{MAX_MONEY}$; an accepted s denotes v zatoshi, and $\text{parse}(s) := v$.

- (2) *Render*. For $v \in \{0, \dots, \text{MAX_MONEY}\}$ write $v = c \cdot \text{COIN} + z$ with $0 \leq z < \text{COIN}$. Let $\text{render}(v)$ be $\text{dec}(c)$ if $z = 0$, and otherwise $\text{dec}(c) \parallel \cdot \parallel f$, where f is the 8-digit zero-padded decimal representation of z with its trailing characters $\emptyset$ removed.

Leading zeros of the whole part and trailing zeros of the fraction are ignored, the bound MAX_MONEY is ZIP 321’s “MUST NOT be greater than 21000000 ZEC”, and a string with a comma or other grouping separator, with an empty whole part ($.5$), an empty fraction ($5\emptyset.$) or more than 8 fraction digits matches no production (ZIP 321, “ZEC Transfer Amount”). ZIP 321 fixes only the value that an amount denotes; the rendering is canonical, not required.

Lemma 14.4 (Round trip of amounts). *For every integer v with $0 \leq v \leq \text{MAX_MONEY}$, the string $\text{render}(v)$ matches $1^*\text{DIGIT} [\text{"."} 1^*8\text{DIGIT}]$ and $\text{parse}(\text{render}(v)) = v$. The map parse is not injective. For every accepted s , $\text{render}(\text{parse}(s))$ is the unique string of value $\text{parse}(s)$ whose whole part has no leading zero unless it is the single character $\emptyset$, whose fraction has no trailing zero, and which has no fraction when $\text{parse}(s)$ is a multiple of COIN .*

Proof. Write $v = c \cdot \text{COIN} + z$ with $0 \leq z < \text{COIN}$. The string $\text{dec}(c)$ is non-empty. If $z = 0$, then $\text{render}(v) = \text{dec}(c)$ matches the production, and parse returns $c \cdot \text{COIN} = v$. If $z \neq 0$, the 8-digit form of z has a non-zero digit, so removing its trailing zeros leaves a fraction f with $1 \leq |f| \leq 8$, and the production matches. Appending $8 - |f|$ zeros to f restores the 8-digit form of z , so parse returns $c \cdot \text{COIN} + z = v$, and accepts since $v \leq \text{MAX_MONEY}$. The strings $5\emptyset$, $05\emptyset$ and $5\emptyset.00$ all

denote 5,000,000,000 zatoshi, so parse is not injective. Finally, an accepted s determines $(\text{int}(c), z)$, which are the quotient and remainder of $\text{parse}(s)$ by COIN since $z < \text{COIN}$. A string of the stated form is fixed by these two numbers: its whole part is $\text{dec}(\text{int}(c))$, and its fraction is absent if $z = 0$ and otherwise the 8-digit form of z without trailing zeros. That string is $\text{render}(\text{parse}(s))$. $\square$

Examples. On the examples of ZIP 321, “ZEC Transfer Amount” and “Examples”, Construction 14.3 gives the following. The parameter $\text{amount}=1$ denotes 100,000,000 zatoshi, $\text{amount}=123.456$ denotes 12,345,600,000 zatoshi, and $\text{amount}.1=0.789$ denotes 78,900,000 zatoshi for the payment at index 1. The values 50, 050 and 50.00 denote 5,000,000,000 zatoshi, and 0.5 and 00.500 denote 50,000,000 zatoshi. The value 21000000 denotes $\text{MAX_MONEY} = 2,100,000,000,000,000$ zatoshi and is accepted, while 21000000.00000001 exceeds MAX_MONEY and is rejected. The values 50,000.00, 50,00, 50., .5 and 0.123456789 are invalid. The parameters $\text{address}.0=$ and $\text{amount}.0=$ are invalid, their index having a leading zero, and so is any parameter with the five-digit suffix .10000.

Absent amount. ZIP 321 gives a payment a value only “if an amount parameter is provided” (ZIP 321, “ZEC Transfer Amount”). A payment without one has no value under ZIP 321, while Definition 11.1 requires a value, so the payer fixes the value before a plan is formed. Class: designed but unspecified (Definition 1.1).

Construction 14.5 (Memo codec). A *memo* is a 512-byte string M , the memo field of a note plaintext (*Ironwood Guide*, §“The note plaintext”; protocol specification, §“Encodings of Note Plaintexts and Memo Fields”). Let b64u be the encoding of RFC 4648, Section 5, with padding omitted, and let $\text{strip}(M)$ be M with its trailing $0x00$ bytes removed.

- (1) *Encode.* $\text{Enc}(M) := \text{b64u}(\text{strip}(M))$, a string of at most $\lceil 512 \cdot 8/6 \rceil = 683$ characters.
- (2) *Decode.* Reject a value that contains a character outside the `base64ur1` alphabet, in particular $+$, $/$ or $=$, or whose length is congruent to 1 modulo 4, a length to which no octet string encodes. Otherwise let b be its decoding under RFC 4648, Section 5, and require $|b| \leq 512$ (MUST); then $\text{Dec}(\text{value}) := b \parallel 0x00^{512-|b|}$ (ZIP 321, “Query Keys”).

A payment without a `memo` parameter has no memo (Definition 11.1); its output, if shielded, carries the memo field that ZIP 302 designates as supplying no memo, $0xF6 \parallel 0x00^{511}$, whose encoding is 9g (ZIP 302, “Specification”). A parameter `memo=` with the empty value differs from an absent parameter: it decodes to $0x00^{512}$, which a reader under ZIP 302 reads as the empty text. If the address at the index of a `memo` parameter does not permit memos, that is, is not memo-capable in the sense of predicate (i) of Definition 3.20, as for a transparent or TEX address, parsers MUST consider the entire URI invalid (ZIP 321, “Query Keys”), since a memo travels only in a note plaintext.

Lemma 14.6 (Round trip of memos). *For every memo M , $\text{Dec}(\text{Enc}(M)) = M$.*

Proof. The string $\text{Enc}(M)$ uses only the `base64ur1` alphabet, and an unpadded encoding of n octets has length $\lceil 8n/6 \rceil$, never congruent to 1 modulo 4, so decoding does not reject it. The encoding `b64u` is injective with inverse the decoding of RFC 4648, so $b = \text{strip}(M)$, and $|b| \leq 512$. The bytes that `strip` removes are zeros, so appending $512 - |b|$ zeros to b returns M . $\square$

ZIP 321 does not say whether a value whose final character carries non-zero unused bits (RFC 4648, Section 3.5) is accepted; the encoding Enc produces none. Stripping trailing zeros gives the shortest encoding and is not required: $\text{b64u}(M)$ decodes to the same memo. As an instance, ZIP 321’s example value `VGhpcyBpcyBhIHNPbXBsZSBtZW1vLg`, of 30 characters, decodes to the 22 bytes of the text “This is a simple memo.”, padded with zeros to 512 bytes.

Labels and messages. The `label` at an index names its address, for example by the name of the recipient; a client that renders the payment for inspection SHOULD display the label together with the address, and a displayed label MUST be identifiable as distinct from the address. The `message` at an index is descriptive text that a client may display for that payment (ZIP 321, “Query Keys”). Both values are `qchar` strings, percent-decoded to octet strings; an octet outside `qchar` reaches them only by percent encoding. Neither enters the transaction (Definition 11.1).

Capacity. A request carries at most 10,000 payments (§14.1), and the one transaction it asks for is bounded further by consensus; the bound that binds is the least of the following.

- (i) *Size.* A block has at most 2,000,000 bytes (protocol specification, §“Block Header Encoding and Consensus”), and a Sapling output contributes at least 948 bytes to a transaction, so a transaction has at most $\lfloor 2,000,000/948 \rfloor = 2109$ Sapling outputs (protocol specification, §“Balance and Binding Signature (Sapling)”). This is the bound that ZIP 321, “URI Semantics”, states.
- (ii) *Shielded budget.* ZIP 218, part of NU7, requires of every block, with S the number of Sapling spends plus outputs, A the number of Orchard-pool Actions and n_{js} the number of JoinSplit descriptions, each summed over the block’s transactions,

$$S \leq 300, \quad A \leq 330, \quad A + S + 2 n_{js} \leq 330$$

(ZIP 218, “Shielded pool action limits”). Versions 5 and 6 carry no JoinSplit description (*Ironwood Guide*, §“The transaction format”), so $n_{js} = 0$. Each of these sums over a block is at least the term of any one of its transactions. A transaction that pays n payments by Sapling outputs has $o \geq n$ Sapling outputs; with s Sapling spends and a Orchard-pool Actions it is mined only if $s + o \leq 300$ and $a + s + o \leq 330$, hence only if

$$n \leq o \leq \min(300, 330 - a) - s.$$

The all-Sapling bound is therefore $\min(2109, 300) = 300$ outputs, fewer with spends or Orchard-pool Actions, and the bound of (i) never binds; one transaction carries at most 330 units of the shared budget.

- (iii) *Orchard pool.* No payment output lies in the Orchard pool (Construction 11.5, Proposition 11.6), so the Orchard-pool Actions of the transaction only consume the shared budget of (ii).
- (iv) *Ironwood pool and transparent outputs.* ZIP 218 as written counts neither Ironwood-pool Actions nor transparent components, so payments by Ironwood-pool or transparent outputs are bounded by the size limit alone. Whether Ironwood-pool Actions enter the shared budget is an open proposal (*Consensus Guide*, §“A block budget for shielded work”). Class of (iv): open problem (Definition 1.1).

From request to plan. Each payment $P_i = (a, v, m)$ of an accepted request, its value fixed by the payer if absent, is a payment in the sense of Definition 11.1 and becomes one output of the plan: its pool is that of Construction 11.5, a TEX address is paid by Construction 11.7, and its memo, if any, is the memo field of that output’s note plaintext (Construction 14.5). A TEX payment funded from shielded notes needs the two transactions of Construction 11.7, so the MUST of ZIP 320, “Design considerations for Senders”, and not the SHOULD of ZIP 321 that asks for a single transaction, fixes the shape of such a plan. From there construction is that of §11.1 and Constructions 11.18 and 11.20: an Orchard-protocol output is sealed as in the *Ironwood Guide*, §“Encryption to the recipient”, a Sapling output as in the protocol specification, §“Sending Notes (Sapling)”, and the *Ironwood Guide*, §“Transaction digests and signatures”, gives the digest that every signature signs.

VII

FlyClient Guide

Merkle mountain ranges, the chain-history commitment, and the FlyClient sampling protocol

Abstract

This volume of *The Zcash Arboretum* states the protocol by which a light client decides which of several claimed chains carries the most work while receiving a number of block headers logarithmic in the chain length. It constructs Merkle mountain ranges under the two bagging orders in use, with inclusion paths, prefix roots, position binding and aggregate binding; fixes the security model of the FlyClient paper of Bünz, Kiffer, Luu and Zamani; constructs the FlyClient protocol from the header chain commitment and the per-sample check through the optimal sampling distribution to the non-interactive protocol and its security theorem, with every hypothesis named; and states the chain-history commitment of ZIPs 221, 244 and 258, to which every Zcash block header after the Heartwood activation block commits. Each component of a Zcash FlyClient is classified as specified, designed but unspecified, or an open problem. The volume describes the protocol as specified for NU7, whose deployment ZIP, ZIP 259, has status Draft. It assumes the *Math Guide*, the *Crypto Guide* and the *Consensus Guide*, and cites the *Ironwood Guide* for network upgrades, ZIP statuses and transaction-level objects. It is non-normative: the Zcash Protocol Specification and the Zcash Improvement Proposals are authoritative.

1 Introduction

This section fixes the subject of the volume, its status, its sources and its notation before any construction: the scope; the named network upgrades and ZIP statuses on which later rules depend; the three classes in which the deployment boundary places each component of a Zcash FlyClient; the primary paper and the policy for its printed defects; the normative sources; and one table of conventions closing with a symbol index.

1.1 Scope and status

This volume of *The Zcash Arboretum* states three objects and the relation between them. The first is the FlyClient protocol of Bünz, Kiffer, Luu and Zamani (ePrint 2019/226). The second is

the chain-history commitment of ZIP 221 (Final), to which every block header after the Heartwood activation block (Definition 1.1) commits: directly, as the header field `hashLightClientRoot`, in blocks validated under the Heartwood or Canopy rules, and, from NU5, as the first input of the hash `hashBlockCommitments` of ZIP 244 (Final). The header fields are named in “The header consensus rules” (§6.7), and their values are defined in “The epoch tree and its root commitment” (§6.4) and “The block commitment hash” (§6.6). The third is the boundary between the protocol and the commitment: each component of a Zcash FlyClient is classified as specified, designed but unspecified, or an open problem (Definition 1.3). The volume is non-normative: the Zcash Protocol Specification, cited as the protocol specification with the title of a section, and the Zcash Improvement Proposals are authoritative.

The volume assumes the following, each cited by *italic* volume name and section title. From the *Math Guide*: probability, densities and asymptotics (§“Probability, asymptotics, and computation”, with §“Beyond finite spaces: countable additivity, limits, and densities” and §“Asymptotic notation”). From the *Crypto Guide*: hash functions and their security notions (§“Hash functions and the random oracle model”, with §“Security notions: preimage, second-preimage, and collision resistance” and §“The random oracle model”); the provable-security model (§“The provable-security model”, with §“Adversaries and the security parameter”, §“Security as a game” and §“Reductions”); interactive arguments (§“Interactive protocols, proofs, and arguments”); the Fiat–Shamir transform (§“The Fiat–Shamir transform: from interactive to non-interactive”); and the binding and inclusion soundness of Merkle trees (§“Merkle trees and commitments to sets”, with its subsections §“Binary Merkle trees”, §“Authentication paths and membership proofs”, §“Soundness: forging a path implies a collision”, §“Layer and domain separation”, §“Append-only and incrementally updatable trees” and §“Vector commitments”). From the *Consensus Guide*: work and the most-work rule (§“The block tree and the most-work rule” and §“Work, and the best chain”), memoryless clocks (§“Memoryless clocks and the attribution sequence”), the retarget rule under NU7 (§“Difficulty in motion”) and Equihash (§“Equihash and the memoryless model”). From the *Ironwood Guide*, only for the objects it constructs: block chain, height, network upgrades, ZIPs and the status Draft (§“Scope and status”); the Orchard pool and the Ironwood pool (§“The Orchard protocol and its two pools”); the encoding LEBS2OSP (§“Fields, groups, and encodings”); pool roots, final treestates and anchors (§“The note commitment tree” and §“Anchors”); branch IDs and the valid transaction formats, version 5 and version 6 under NU7 (§“The transaction format”; ZIP 2003 (Draft); ZIP 259 (Draft)); and transaction identifiers, the personalised hash h and the branch-ID encoding β (§“Transaction digests and signatures”). The light-client model is cited directly to ZIP 307 (Draft), “Security Model” and “Block header validation”.

Definition 1.1 (Named network upgrades). A *network upgrade*, its *activation height* and the fixed order in which network upgrades activate are those of the *Ironwood Guide*, §“Scope and status”, Definition “Network upgrade” (ZIP 200 (Final), “Terminology” and “Activation mechanism”). This volume names the following network upgrades, in activation order, each with its deployment ZIP and the status of that ZIP:

Upgrade	ZIP	Status	Upgrade	ZIP	Status
Sapling	205	Final	NU6	253	Final
Blossom	206	Final	NU6.1	255	Final
Heartwood	250	Final	NU6.2	257	Final
Canopy	251	Final	NU6.3	258	Draft
NU5	252	Final	NU7	259	Draft

An upgrade is named only where its name fixes which header rule or tree-node rule applies. The *Heartwood activation block* is the block whose height is the activation height of Heartwood. No activation height, date or branch ID value is given here; the branch IDs of the upgrades from Heartwood on, whose upgrade epochs bear an epoch tree (“Upgrade epochs and epoch trees”, §6.2), are tabulated in Table 3.

Definition 1.2 (ZIP statuses). A ZIP and its status *Draft*, the status of every ZIP on submission, are those of the *Ironwood Guide*, §“Scope and status”, Definition “ZIP”. Two further statuses occur in this volume (ZIP 0 (Active), “ZIP Status Field”):

- (i) *Final*, the status of a Consensus or Standards ZIP that is both implemented and activated on the Zcash network;
- (ii) *Active*, the status, typically of a Process or Informational ZIP, reached once rough consensus on it has been reached; ZIP 0 itself has this status.

The status is a property of the document and does not decide whether its rules are in force. The rules of ZIP 258 (Draft) and ZIP 229 (Draft) apply from NU6.3 activation: ZIP 258, “NU6.3 deployment”, lists ZIP 229 among the primary sources of the NU6.3 consensus changes, and ZIP 258, “Changes to ZIP 221”, amends the history-tree node of ZIP 221. Every ZIP is cited with its status at its first citation in the volume.

The volume describes the protocol as specified for NU7, whose deployment ZIP, ZIP 259, has status Draft (ZIP 259, preamble). NU6.3 is active.

Definition 1.3 (Epistemic classes). A *component* of a Zcash FlyClient is a data structure, rule, check or proof format of the protocol, or a hypothesis of its security theorem. A component is

- (i) *specified* when normative text of the protocol specification or of a ZIP fixes it, whatever the status of that ZIP (Definition 1.2);
- (ii) *designed but unspecified* when a published design with an analysis exists but no ZIP or section of the protocol specification fixes it for Zcash;
- (iii) an *open problem* when no design exists.

A property proved in this volume of a specified object is classified by its object: the object is specified, and the property is proved here.

The class *specified* is that of the *Ironwood Guide*, §“Scope and status”, Definition “Epistemic classes”; that volume applies *designed but unspecified* to a property that a deployed construction is

designed to satisfy but that no specification, ZIP or proof establishes, whereas this volume applies it to a component that exists only as a published design; the two uses share the condition that no normative text fixes the object, and the sense of the *Ironwood Guide* is not used here.

1.2 Sources and conventions

The primary source is B. Bünz, L. Kiffer, L. Luu and M. Zamani, “FlyClient: Super-Light Clients for Cryptocurrencies”, IACR ePrint 2019/226, in its sixth revision, the last revision on the ePrint page and the revision of record; the paper also appeared at the IEEE Symposium on Security and Privacy. Every statement taken from the paper is cited by the section, definition, theorem, lemma, algorithm or printed page of that revision. A section of the paper is written “Section 5.3”, without the section sign, which this volume reserves for sections of the protocol specification and of the *Arboretum*.

Where a statement of this volume rests on a printed statement of the paper that is defective, the volume states the corrected form at that point and names the departure. Only defects that touch a stated result are mentioned, and there is no separate register of them. Claims of the paper that this volume does not prove are stated as the paper’s, with their gaps: for example Assumption 5.11 (“Transition rewriting”) and Remark 4.11 (“Lemma 1 of the paper”).

The normative sources are the following. The protocol specification, in the sections §“The Block Chain”, §“Integers, Bit Sequences, and Endianness”, §“Constants”, §“BLAKE2 Hash Functions”, §“Transaction Encoding and Consensus”, §“Block Header Encoding and Consensus”, §“Difficulty filter”, §“Difficulty adjustment”, §“nBits conversion” and §“Definition of Work”. ZIP 0 (Active), which defines the ZIP process and its statuses; ZIP 200 (Final); ZIP 204 (Draft); ZIPs 205, 206, 250, 251, 252, 253, 255 and 257 (Final), the deployment ZIPs of the upgrades named in Definition 1.1 before NU6.3; ZIP 218 (Draft); ZIP 221 (Final); ZIP 229 (Draft); ZIP 244 (Final); ZIP 258 (Draft); ZIP 259 (Draft); ZIP 307 (Draft); and ZIP 2003 (Draft). A ZIP is cited by its number and the title of a section, the title following the number in the same sentence; the protocol specification is cited as protocol specification, §“Title”.

The conventions of the volume and the symbols of later sections are fixed in Table 1. The paper overloads several symbols: n denotes both the honest participation and the chain length; k the persistence depth, the samples per interval and $\log_c \delta$; λ both the security parameter and a statistical exponent; δ both the weight fraction checked in full and the backbone parameter for which $\mu = 1 - \delta$ bounds the adversary’s mining power; L both the fork threshold and the number of blocks of the checked suffix; and c both the weight bound and a ratio of mining power. The table assigns one meaning to each symbol, and the index that closes it records where each is defined.

2 The verification problem

This section fixes the object a light client verifies, the Zcash block header, and the baseline procedure that verifies a chain of headers in full. It states the trust model of the light-client protocol specified for Zcash and the problem the volume solves, with each term of its solution defined in one clause and constructed in a later section.

Conventions	
Logarithms	The logarithm $\log x$ is natural and $\log_2 x$ is to base 2 (<i>Math Guide</i> , §“Symbols and their pronunciation”). For real $b > 0$, $b \neq 1$, $\log_b x := \log x / \log b$, as in $\log_{1/c}$: a real-base logarithm, distinct from the discrete logarithm $\log_g b$, whose base is a group element.
Floor, ceiling	The floor $\lfloor x \rfloor$ and ceiling $\lceil x \rceil$ of a real x (<i>Math Guide</i> , §“Symbols and their pronunciation”; §“The division algorithm”).
Asymptotics	The symbols O and Θ and negligible functions are as in the <i>Math Guide</i> , §“Asymptotic notation” and §“Polynomial, exponential, and negligible functions”; and $\text{polylog}(N) := (\log N)^{O(1)}$. With several parameters, n or N grows while λ , σ , c and L are fixed; the budget R is either fixed or, as in the paper, $R = cn$, and each bound states which.
Three parameters	λ the security parameter (<i>Math Guide</i> , §“The security parameter”); σ the statistical exponent of a failure target $2^{-\sigma}$; κ the output length in bits of the paper’s hash H . The paper writes λ for all three.
Node fields	Fields of a node v of the difficulty Merkle mountain range are sans-serif and accessed with a dot, as $v.w$, $v.t$.
Personalised hash	The hash h and the encoding β are those of the <i>Ironwood Guide</i> , §“Transaction digests and signatures”; the hash is always written with its personalisation and input in full.
Two epochs	The <i>retarget epoch</i> (“Retargeting and the difficulty-raising attack”) and the <i>upgrade epoch</i> (“Upgrade epochs and epoch trees”) are distinct objects; “epoch” is not used alone where it is ambiguous.
Unused symbols	The <i>Crypto Guide</i> ’s c (completeness error) and L (language) do not occur.
Symbol index (pointers to the defining subsection; nothing is defined here)	
Header-only verification	N the length of a claimed chain; η a block height in the sense of the <i>Ironwood Guide</i> , §“Scope and status”; $\omega(C)$ the total work of a chain C ; hdr a block header; every chain-length quantity before “Merkle mountain ranges” is written N or “the chain length”, never n .
Merkle mountain ranges (section preamble)	the hash $H: \{0, 1\}^* \rightarrow \{0, 1\}^*$ and its output length κ .
The recursive definition	n the leaf count of a Merkle mountain range, hence of the range a head commits to, never a block height; j a leaf index and x_j the value of leaf j ; M_n the range over n values and rt_n its root; $\text{alt}(v)$ the altitude of a node.
Peaks and bagging	$P_1, \dots, P_d$ the peaks, $d = \text{popcount}(n)$; Bag_R and Bag_L the two bagging orders.
Aggregating ranges	ser the serialisation of a node tuple; W the cumulative weight before a leaf.
The variable-difficulty backbone model	D a difficulty level (larger is harder); T a target (smaller is harder); ω , restated, never written D ; q the queries per player per round; r a round; v_r and t_r the honest and adversarial players of round r ; (γ, s) the participation parameters; μ the adversary’s power bound; ℓ the persistence depth.
Retargeting and the difficulty-raising attack	E the retarget-epoch length; Δ the rounds a retarget epoch takes; τ the dampening factor; f the expected blocks per round.
The (c, L) -adversary	c and L the parameters of the adversary; ρ the power ratio of the paper’s evaluation.
The difficulty Merkle mountain range	y and z the level exponent and the retarget-epoch count of the feasibility bounds; the dotted node fields.
The sampling distribution	δ the weight fraction checked in full; k the catch parameter, $\delta = c^k$, its only meaning; a a fork point; ξ a point of $[0, 1]$; φ a density on $[0, 1]$; K the constant of the indifference density; g the FlyClient density and G its distribution function; p the single-draw catch probability; Q the sample count; u the samples per dyadic interval; U a uniform variable on $[0, 1]$.
The non-interactive protocol	R the head-recomputation budget.
Header chain commitments and per-sample checks	B_η the block at height η ; heights are written η , never i .

Table 1. Conventions and symbol index. Each symbol has the one meaning listed, fixed in the subsection named; a letter bound inside a single definition, construction or proof, such as a node or an index, keeps its meaning only there.

Upgrade epochs and epoch trees	$\mathcal{A}$ an activation height; $\mathcal{A}_H$ that of Heartwood, distinct from the hash H ; $\alpha(\eta)$ the greatest activation height below η ; $\mathcal{T}_\eta$ the epoch tree committed by B_η , distinct from a target T , with $n = \eta - \alpha(\eta)$ leaves.
The epoch tree and its root commitment	$b(\mathcal{T})$ the branch ID of an epoch tree $\mathcal{T}$.
The header consensus rules	$\mathcal{A}_{\text{NU5}}$ the activation height of NU5.
The committed range	$\mathcal{A}_{\text{NU7}}$ the activation height of NU7.

2.1 Header-only verification

Definition 2.1 (Light client). A *light client* is a client that is not a full participant in the network of Zcash peers. It can send and receive payments, but does not store or validate a copy of the block chain (ZIP 221, “Terminology”).

The verifier of the FlyClient paper is a light client given the additional inputs fixed in Definition 4.16 (“Security and succinctness of chain proofs”, §4.4).

Byte strings and integers are related by the maps of the *Ironwood Guide*, §“Fields, groups, and encodings”, Definition “Integer and byte encodings”: the map I2LEOSP_k sends an integer in $\{0, \dots, 2^k - 1\}$ to its $\lceil k/8 \rceil$ -byte little-endian encoding, and the map LEOS2IP_k sends a $k/8$ -byte string to the integer it encodes in little-endian order.

Definition 2.2 (CompactSize). The *CompactSize encoding* of an integer $v \in \{0, \dots, 2^{64} - 1\}$ is the byte string

$$\text{CompactSize}(v) := \begin{cases} \text{I2LEOSP}_8(v) & \text{if } 0 \leq v \leq 252, \\ 0xFD \parallel \text{I2LEOSP}_{16}(v) & \text{if } 253 \leq v \leq 0xFFFF, \\ 0xFE \parallel \text{I2LEOSP}_{32}(v) & \text{if } 0x10000 \leq v \leq 0xFFFFFFFF, \\ 0xFF \parallel \text{I2LEOSP}_{64}(v) & \text{if } 0x100000000 \leq v \leq 0xFFFFFFFFFFFFFFFF, \end{cases}$$

of length 1, 3, 5 or 9 bytes respectively. Only this encoding, the shortest, is valid for v : a byte string that decodes to v under one of the other three prefixes **MUST** be rejected (ZIP 204, “CompactSize”).

ZIP 204 has status Draft; the minimal-encoding rule binds block headers independently, through the protocol specification, §“Block Header Encoding and Consensus”, which requires the header field solutionSize to be encoded with the minimum number of bytes. That rule prevents a miner from testing several encodings of one Equihash solution against the difficulty filter.

The upgrade epochs delimited by the network upgrades of Definition 1.1 are fixed in “Upgrade epochs and epoch trees” (§6.2).

Definition 2.3 (Block header). A Zcash *block* consists of a block header and a sequence of transactions. A *block header* *hdr* is the concatenation, in the following order, of the fields

Bytes	Field	Content
4	nVersion	signed 32-bit integer; the consensus rules require $nVersion \geq 4$
32	hashPrevBlock	the block hash of the previous block
32	hashMerkleRoot	the root of the transaction Merkle tree over the identifiers of the block's transactions, stated in "The authorising-data root" (§6.5)
32	commitment field	named as below
4	nTime	unsigned 32-bit integer, the block timestamp
4	nBits	unsigned 32-bit integer, an encoding of the target threshold
32	nNonce	32 arbitrary bytes
3	solutionSize	CompactSize(1344)
1344	solution	an Equihash solution

with integers encoded little-endian. The commitment field is named `hashReserved` before Sapling, `hashFinalSaplingRoot` in Sapling and Blossom, `hashLightClientRoot` in Heartwood and Canopy, and `hashBlockCommitments` from NU5; its value in each upgrade epoch is fixed in "The header consensus rules" (§6.7). The *block hash* of *hdr* is the 32-byte string

$$\text{BlockHash}(\text{hdr}) := \text{SHA256d}(\text{hdr}), \quad \text{SHA256d}(x) := \text{SHA256}(\text{SHA256}(x)),$$

taken over the whole header serialisation, `solutionSize` and `solution` included, in internal byte order, the order in which the hash function outputs its bytes. Let $\text{ToTarget}: \{0, \dots, 2^{32} - 1\} \rightarrow \mathbb{N}$ be the conversion of protocol specification, §"nBits conversion". The header *hdr* *satisfies proof of work* when *solution* is a valid Equihash solution for *hdr* and

$$\text{LEOS2IP}_{256}(\text{BlockHash}(\text{hdr})) \leq \text{ToTarget}(n\text{Bits})$$

(protocol specification, §"Difficulty filter"). The consensus rules further require, for a block at height η , that $n\text{Bits} = \text{ThresholdBits}(\eta)$, the value the difficulty-adjustment rule of protocol specification, §"Difficulty adjustment" assigns to that height.

The definition follows protocol specification, §"Block Header Encoding and Consensus" for the field table and its consensus rules, and ZIP 221, "Block header semantics and consensus rules" and ZIP 244 for the renaming of the commitment field. The hash `SHA256d` is registered in the *Crypto Guide*, §"The transparent layer's hashes". The Equihash condition is cited, not reconstructed: *Consensus Guide*, §"Equihash and the memoryless model". The *work* of a block whose header carries *nBits* is

$$\text{work}(\text{hdr}) := \left\lfloor \frac{2^{256}}{\text{ToTarget}(n\text{Bits}) + 1} \right\rfloor,$$

and the *total work* $\omega(C)$ of a chain *C* is the sum of the work of its blocks; a node considers the valid chain of greatest total work to be best, breaking ties between leaf blocks in favour of the block received first (*Consensus Guide*, §"The block tree and the most-work rule", and its subsection

§“Work, and the best chain”; protocol specification, §“The Block Chain” and §“Definition of Work”).

The fixed-width fields of Definition 2.3 occupy $4 + 32 + 32 + 32 + 4 + 4 + 32 = 140$ bytes. Since $253 \leq 1344 \leq 0xFFFF$, the field `solutionSize` occupies 3 bytes, and with the 1,344-byte solution a Zcash block header is 1,487 bytes, of which the solution is $1344/1487$, about 90.4 per cent.

Definition 2.4 (Simplified payment verification). A *claimed chain* of length $N \geq 1$ is a sequence of block headers $C = (\text{hdr}_0, \dots, \text{hdr}_{N-1})$ in which hdr_0 is the header of the genesis block; header hdr_η is the header at height η . Given one or more claimed chains, a *simplified-payment-verification* (SPV) client downloads every header of every claimed chain and checks, for each chain C :

- (S1) *linkage*: for $1 \leq \eta \leq N - 1$, the field `hashPrevBlock` of hdr_η equals $\text{BlockHash}(\text{hdr}_{\eta-1})$;
- (S2) *proof of work*: for $0 \leq \eta \leq N - 1$, header hdr_η satisfies proof of work in the sense of Definition 2.3, and its field `nBits` equals $\text{ThresholdBits}(\eta)$, computed from $\text{hdr}_0, \dots, \text{hdr}_{\eta-1}$.

It rejects every chain that fails a check and accepts, among the rest, the chain C of greatest total work $\omega(C)$, under the best-chain rule of the *Consensus Guide*, §“Work, and the best chain”.

The definition is that of ePrint 2019/226, Section 1, p. 2, instantiated with the header checks of protocol specification, §“Block Header Encoding and Consensus”. For a claimed chain of N blocks the client receives N headers, 1,487 N bytes, so its communication is $\Theta(N)$. A block is at most 2,000,000 bytes (same section), so the saving over downloading full blocks is at most a constant factor, not an asymptotic one.

An SPV client checks neither the validity of transactions nor any consensus rule beyond those of the header, and therefore relies on the following assumption.

Assumption 2.5 (SPV assumption). “The chain with the most PoW solutions follows the rules of the network and will eventually be accepted by the majority of miners.” (ePrint 2019/226, Section 1, p. 2, Assumption 1.)

The paper formalises the property an SPV client checks as Definition 4.14, and strengthens Assumption 2.5 quantitatively as Assumption 4.8 (“The (c, L) -adversary”, §4.3).

2.2 The light-client model of ZIP 307

ZIP 307 (Draft) specifies a light-client protocol in which a light client obtains, from a Zcash node and a proxy, *compact blocks*: per-block messages that may carry the block header. The trust model is stated in ZIP 307, “Security Model”: the node and proxy combination is assumed to be “honest but curious” and “is trusted to provide a correct view of the current best chain state and to faithfully transmit queries and responses”.

ZIP 307, “Block header validation” describes, as a proposed enhancement that the ZIP records as only partially implemented (only check (Z2) is made), checks of the SPV kind on each compact block that carries a header. With the fields named as in Definition 2.3, the checks are:

- (Z1) the field `nVersion` is at least the minimum block version;
- (Z2) the field `hashPrevBlock` equals the block hash of the previous compact block;
- (Z3) the Equihash solution is valid;
- (Z4) the target `ToTarget(nBits)` is non-zero and at most the proof-of-work limit `PoWLimit` (protocol specification, §“Constants”);
- (Z5) when the last 27 compact blocks all carry headers, the field `nBits` is correct under the difficulty-adjustment rule;
- (Z6) the block hash, read as a little-endian integer, is at most `ToTarget(nBits)`.

In that proposal, a compact block that fails a check **MUST** be discarded. The same section of ZIP 307 further describes a check that the commitment field equals the root of the Sapling note commitment tree recomputed from the compact block; from Heartwood that check no longer applies, since the field changes meaning (Definition 2.3; ZIP 221, “Block header semantics and consensus rules”). Checks (Z1) to (Z6), where made on every header, cost communication linear in the chain length N , as in Definition 2.4.

Of the trust that ZIP 307 places in the node and proxy, a FlyClient verifier would replace only the trust in which header chain carries the most work, under Assumption 4.8 (“The (c, L) -adversary”). It would replace the linear header checks by the logarithmic sampling of “The FlyClient protocol” (§5); it does not replace faithful transmission of queries and of compact-block contents. Such a verifier is designed but unspecified in the sense of Definition 1.3; its status is fixed in “A Zcash verifier” (§7.4).

2.3 Statement of the problem

The problem is the following. Under the SPV assumption (Assumption 2.5), strengthened quantitatively as Assumption 4.8 in “The (c, L) -adversary” (§4.3), a light client is to accept the head of the chain of greatest total work after communication sublinear in the chain length N . The inputs and the chain to be selected are those of Definition 2.4; the assumption is strengthened to Assumption 4.8, the selection holds except with a failure probability fixed in “The FlyClient protocol” (§5), and the communication falls from $\Theta(N)$ to sublinear.

A client that does not check every proof of work “can be tricked into accepting an invalid chain by a malicious prover who can precompute a sufficiently-long chain using its limited computational power” (ePrint 2019/226, Section 1, p. 3); the adversary is fixed in “The FlyClient security model” (§4).

The solution uses four objects. A *Merkle mountain range* is a binary hash tree over a sequence that admits appending a leaf without changing its peaks or any node beneath them (“Merkle mountain ranges”, §3). A *header chain commitment* is a root, carried in every header after the genesis header, of such a range over all blocks strictly before that header (“Header chain commitments and per-sample checks”, §5.1). A *work-weighted sample* is a block drawn at a random point of cumulative work and checked against the commitment of the head (“The sampling distribution”,

§5.4). A *suffix* of the chain is downloaded and checked in full, its extent fixed in the same subsection. *FlyClient* is the protocol that verifies a proof-of-work chain by drawing logarithmically many such samples and checking each against the commitment of the head (ePrint 2019/226, Section 1). The commitment of the paper covers the chain from the genesis block, whereas the commitment of ZIP 221 covers one upgrade epoch: the blocks from the activation height of the preceding network upgrade up to the committing block, exclusive (ZIP 221, “Specification”; “Upgrade epochs and epoch trees”, §6.2).

3 Merkle mountain ranges

This section constructs the commitment on which the FlyClient protocol and the chain-history commitment of ZIP 221 rest: a binary hash tree over a sequence of values whose shape is fixed by the length of the sequence, extended by one value at logarithmic cost, and opened at one position j by a path from which the root over the prefix of length $j - 1$ is recomputed, and, under the right fold, also the root over the prefix of length j . The construction is parametrised by the order in which its perfect subtrees are joined: the right fold of ePrint 2019/226 and the left fold of ZIP 221, “Background”. The fixed-depth trees of the *Crypto Guide*, §“Binary Merkle trees” and §“Append-only and incrementally updatable trees”, are the special case of a sequence whose length is a power of two.

Throughout, $H: \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$ is a hash function drawn from a family that is collision resistant in the sense of the *Crypto Guide*, Definition “Collision resistance” (§“Security notions: preimage, second-preimage, and collision resistance”); ePrint 2019/226, Definition 8, states the same assumption. A statement holds *barring a collision* when a deterministic procedure, given any input on which the statement fails, outputs two distinct strings with equal images under H , using the number of evaluations of H stated with it. Such a procedure is a tight reduction in the sense of the *Crypto Guide*, Definition “Reduction loss and tightness” (§“Reductions”), so a probabilistic polynomial-time adversary makes the statement fail with negligible probability.

3.1 The recursive definition

Definition 3.1 (Merkle mountain range). Let $n \geq 1$ and $x_1, \dots, x_n \in \{0, 1\}^\kappa$. The *Merkle mountain range* M_n over $(x_1, \dots, x_n)$ is the rooted ordered binary tree, every internal node of which has exactly two children, a left and a right one, with a value in $\{0, 1\}^\kappa$ at every node, defined by recursion on n :

1. if $n = 1$, M_1 is the single leaf with value x_1 ;
2. if $n > 1$, let $i = \lfloor \log_2(n - 1) \rfloor$, so that $2^i < n \leq 2^{i+1}$ and 2^i is the largest power of two strictly below n ; the root has as left child the range M_{2^i} over $(x_1, \dots, x_{2^i})$ and as right child the range M_{n-2^i} over $(x_{2^i+1}, \dots, x_n)$, where $1 \leq n - 2^i \leq 2^i$.

Every internal node with children of values v_ℓ and v_r has value $H(v_\ell \parallel v_r)$, the image of the 2κ -bit concatenation of its children’s values. The leaves are ordered from left to right and leaf j holds x_j . The *root* rt_n is the value of the root node; *range* abbreviates Merkle mountain range.

The *depth* of a node is the number of edges from the root to it, and the depth of a tree the maximal depth of a leaf. A *perfect subtree of altitude a* is a subtree with 2^a leaves, all at depth a below its root; its root node v has *altitude* $\text{alt}(v) = a$. The *right spine* is the sequence of nodes that starts at the root and continues from each internal node to its right child, ending at a leaf.

ZIP 221, “Terminology”, calls a Merkle mountain range a binary hash tree that admits appends of new leaves without changing the value of existing nodes; Theorem 3.11 and Proposition 3.12 below establish it for Definition 3.1 for the peaks and every node beneath them. An append rehashes only the right-spine nodes above the first perfect one, the counterparts of the bagging nodes of ZIP 221, “Background”. Definition 3.1 is ePrint 2019/226, Definition 11, with the base case $n = 1$, implicit there, made explicit. Every node of M_n is the root of a range over a run of consecutive values, by the recursion. Its depth is fixed in Lemma 3.2.

Lemma 3.2 (Left children are perfect). *Let $n \geq 1$.*

1. *A range over 2^a values is a perfect subtree of altitude a .*
2. *Every left child in M_n is perfect: the left child of a node over $m \geq 2$ leaves is the perfect range over $2^{\lfloor \log_2(m-1) \rfloor}$ leaves.*
3. *Descend the right spine of M_n from the root, stopping at the first node whose leaf count is a power of two. The left children of the nodes passed, followed by the node where the descent stops, are perfect subtrees of pairwise distinct power-of-two sizes, decreasing from the root downwards, covering consecutive runs of leaves from left to right; their sizes are the terms of the binary expansion of n .*
4. *The range M_n has depth exactly $\lfloor \log_2 n \rfloor$, which is 0 for $n = 1$.*

Proof. (1) By induction on a . For $a = 0$ the range is a leaf. For $a \geq 1$, $\lfloor \log_2(2^a - 1) \rfloor = a - 1$, so the root has two children over 2^{a-1} values each, perfect of altitude $a - 1$ by hypothesis; all leaves lie at depth a .

(2) Every node over $m \geq 2$ leaves is a range M_m , whose left child is a range over $2^{\lfloor \log_2(m-1) \rfloor}$ values, perfect by (1).

(3) Let a right-spine node have m leaves. If m is a power of two, the node is perfect by (1) and the descent stops. Otherwise $2^{\lfloor \log_2 m \rfloor} < m$, hence $m - 1 \geq 2^{\lfloor \log_2 m \rfloor}$ and $\lfloor \log_2(m - 1) \rfloor = \lfloor \log_2 m \rfloor$: the left child takes the highest term $2^{\lfloor \log_2 m \rfloor}$ of the binary expansion of m and the right child the remaining terms. By induction along the spine, the left children passed take the terms of the expansion of n from the highest downwards, and the descent stops when one term is left. The runs are consecutive because each left child precedes its sibling’s leaves.

(4) By induction on n . For $n = 1$ the depth is 0. For $n > 1$ let $i = \lfloor \log_2(n - 1) \rfloor$. The left child is perfect over 2^i leaves, of depth i ; the right child has $n - 2^i \leq 2^i$ leaves and depth $\lfloor \log_2(n - 2^i) \rfloor \leq i$ by hypothesis. The depth of M_n is therefore $i + 1$, which equals $\lfloor \log_2 n \rfloor$ since $2^i < n \leq 2^{i+1}$. $\square$

Example 3.3 (Six and seven leaves). For $n = 6$, $i = \lfloor \log_2 5 \rfloor = 2$ and $6 = 4 + 2$. The left child is the perfect range P_1 over $x_1, \dots, x_4$; the right child has two leaves, is perfect, and has value $P_2 =$

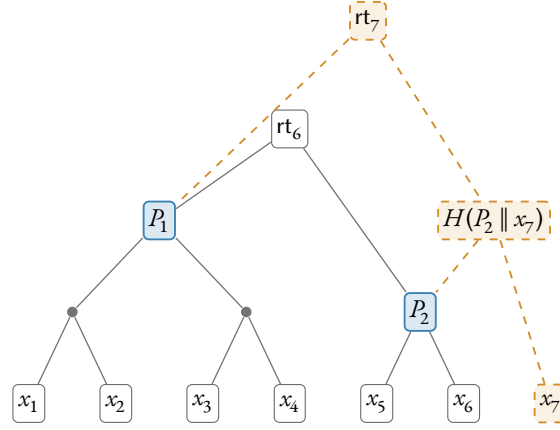


Figure 1. The range M_6 over $x_1, \dots, x_6$ in single-rooted form (solid): the root rt_6 has left child P_1 , the perfect range over $x_1, \dots, x_4$, and right child $P_2 = H(x_5 \parallel x_6)$, the perfect range over x_5, x_6 . The nodes P_1 and P_2 are the peaks of Definition 3.4; P_2 is the first perfect node on the right spine of M_6 , whose last node is the leaf x_6 . Dashed: appending x_7 creates only the nodes $H(P_2 \parallel x_7)$ and $rt_7 = H(P_1 \parallel H(P_2 \parallel x_7))$; the nodes P_1, P_2 and every node beneath them are shared by M_6 and M_7 .

$H(x_5 \parallel x_6)$; hence $rt_6 = H(P_1 \parallel P_2)$. For $n = 7$, $i = \lfloor \log_2 6 \rfloor = 2$ and $7 = 4 + 3$; the three-leaf right child splits as $3 = 2 + 1$, so $7 = 4 + (2 + 1)$ and

$$rt_7 = H(P_1 \parallel H(H(x_5 \parallel x_6) \parallel x_7)).$$

3.2 Peaks and bagging

For $n \geq 1$, $\text{popcount}(n)$ is the number of ones in the binary expansion of n .

Definition 3.4 (Peaks). Let $n \geq 1$ and write $n = 2^{a_1} + \dots + 2^{a_d}$ with $a_1 > \dots > a_d \geq 0$, so that $d = \text{popcount}(n)$, and let $s_i = \sum_{l < i} 2^{a_l}$ for $1 \leq i \leq d$. The *peaks* of the sequence $(x_1, \dots, x_n)$ are $P_1, \dots, P_d$, where P_i is the perfect range over $(x_{s_i+1}, \dots, x_{s_i+2^{a_i}})$, of altitude $\text{alt}(P_i) = a_i$. There is one peak per set bit of n , of altitude that bit's index; the altitudes strictly decrease from the left, and the peaks cover consecutive runs of leaves. The symbol P_i denotes both the subtree and its root value.

The peaks are the mountains of ZIP 221, “Background”, whose leftmost peak is the highest. The same section numbers nodes in the order of their creation; that numbering enters no committed value and no check, and the root is a function of the leaf sequence and the bagging order of Definition 3.5 alone. No node numbering is used in this volume.

Definition 3.5 (Bagging orders). Let $P_1, \dots, P_d$ be the peaks of $(x_1, \dots, x_n)$. The *right fold* and the *left fold* of the peaks are

$$\begin{aligned} \text{Bag}_R(P_1, \dots, P_d) &= H(P_1 \parallel H(P_2 \parallel \dots H(P_{d-1} \parallel P_d) \dots)), \\ \text{Bag}_L(P_1, \dots, P_d) &= H(\dots H(H(P_1 \parallel P_2) \parallel P_3) \dots \parallel P_d); \end{aligned}$$

a single peak is its own bag under either fold. The left fold is the order of ZIP 221, “Background”, which repeatedly joins the two leftmost peaks. For $b \in \{R, L\}$ the tree M_n^b consists of the d perfect peak subtrees joined by the $d - 1$ internal *bagging nodes* of Bag_b , and rt_n^b is its root value. Both are determined by n , b and the leaf sequence.

Proposition 3.6 (Peaks of the recursive form). *For every $n \geq 1$ the peaks of $(x_1, \dots, x_n)$ are exactly the left children of the right-spine nodes of M_n , from the root down to the first perfect right-spine node, followed by that node. Hence $M_n = M_n^R$ and $\text{rt}_n = \text{Bag}_R(P_1, \dots, P_d)$.*

Proof. By Lemma 3.2(3) the nodes named are perfect subtrees whose sizes are the terms of the binary expansion of n , largest first, covering consecutive runs of leaves from the left: they are $P_1, \dots, P_d$ in order. Along the spine, the t -th node passed has value $H(P_t \parallel v_{t+1})$ with v_{t+1} the value of the next spine node, and the last spine node considered is P_d ; unwinding gives the right fold. $\square$

Lemma 3.7 (Path lengths under the two baggings). *Let $n \geq 1$ have peaks $P_1, \dots, P_d$ of altitudes $a_1 > \dots > a_d$, let $b \in \{R, L\}$, and let leaf j lie in P_i . The depth $\text{depth}_b(n, j)$ of leaf j in M_n^b is*

$$\text{depth}_R(n, j) = \begin{cases} a_i + i & i < d, \\ a_d + d - 1 & i = d, \end{cases} \quad \text{depth}_L(n, j) = \begin{cases} a_1 + d - 1 & i = 1, \\ a_i + d - i + 1 & i \geq 2. \end{cases}$$

Consequently the depth of M_n^R is $\lceil \log_2 n \rceil$; the depth of M_n^L is $\lceil \log_2 n \rceil + \text{popcount}(n) - 1 \leq 2\lceil \log_2 n \rceil$, attained by the leaves of P_1 ; and the two depths differ exactly when $d \geq 3$. For $n = 7$ they are 3 and 4; for $n = 127$ they are 7 and 12. Leaf depths are functions of (b, n, j) ; for $d \geq 2$ they are not all equal.

Proof. The leaf lies at depth a_i below the root of P_i ; it remains to find the depth of that root in the bag. Under Bag_R the t -th bagging node lies at depth $t - 1$ and has left child P_t for $t \leq d - 1$, and the last bagging node has right child P_d ; so P_i lies at depth i for $i < d$ and P_d at depth $d - 1$. Under Bag_L the root has right child P_d and left child $\text{Bag}_L(P_1, \dots, P_{d-1})$, and inductively P_i lies at depth $d - i + 1$ for $i \geq 2$ while P_1 shares depth $d - 1$ with P_2 ; for $d = 1$ the single peak lies at depth 0.

Maxima. Under R , $a_i \leq a_1 - (i - 1)$ gives $a_i + i \leq a_1 + 1$ and $a_d + d - 1 \leq a_1$; the maximum is $a_1 + 1$ for $d \geq 2$ and a_1 for $d = 1$, which is $\lceil \log_2 n \rceil$ in both cases, as n is a power of two exactly when $d = 1$. Under L , for $i \geq 2$, $a_i + d - i + 1 \leq a_1 + d - 2i + 2 \leq a_1 + d - 2$, so the maximum is $a_1 + d - 1$, attained by P_1 ; with $a_1 = \lceil \log_2 n \rceil$ and $d = \text{popcount}(n) \leq \lceil \log_2 n \rceil + 1$ this is the stated value and bound. The maxima $a_1 + 1$ and $a_1 + d - 1$ coincide for $d = 2$, both equal a_1 for $d = 1$, and differ for $d \geq 3$. For $d \geq 2$ a leaf of P_1 and a leaf of P_d differ in depth under R ($a_1 + 1$ against at most a_1), and a leaf of P_1 and a leaf of P_2 under L ($a_1 \neq a_2$). $\square$

A *cycle* of H is a value v and a nonempty sequence of evaluations of H that, starting from inputs that include v , recomputes v .

Theorem 3.8 (Fold order). *Let $n \geq 1$ have d peaks. The shapes M_n^R and M_n^L coincide if and only if $d \leq 2$. For $d \geq 3$:*

1. the roots rt_n^R and rt_n^L are distinct hash expressions over the same peaks, and from values of the peaks for which $\text{rt}_n^R = \text{rt}_n^L$ a collision or a cycle of H is computed;
2. for every leaf j , the sequence of sides of the siblings on leaf j 's path above its peak differs between M_n^R and M_n^L ;
3. the peaks and the perfect subtrees beneath them coincide, and so do the asymptotic costs of append, of storage of the peaks and of a path, each logarithmic in n ; the leaf depths differ as in Lemma 3.7, so the bound $\lceil \log_2 n \rceil$ on the path length of ePrint 2019/226 does not hold for M_n^L .

Proof. Definition 3.1 splits $n = 2^{\lceil \log_2 (n-1) \rceil} + (\text{rest})$ with the first peak on the left and recurses on the rest, which is the right fold (Proposition 3.6). For $d = 1$ both folds are P_1 and for $d = 2$ both are $H(P_1 \parallel P_2)$. For $d \geq 3$ the root's left child is P_1 under R and a bagging node under L , so the shapes differ.

(1) The root inputs are $P_1 \parallel \text{Bag}_R(P_2, \dots, P_d)$ and $\text{Bag}_L(P_1, \dots, P_{d-1}) \parallel P_d$. If the root values agree and these 2κ -bit strings differ, they are a collision. Otherwise $P_1 = \text{Bag}_L(P_1, \dots, P_{d-1})$ and $P_d = \text{Bag}_R(P_2, \dots, P_d)$, each a value recomputed from itself by $d - 2 \geq 1$ evaluations of H : a cycle.

(2) Above P_1 the path has under R the single sibling $\text{Bag}_R(P_2, \dots, P_d)$ and under L the $d - 1 \geq 2$ siblings $P_2, \dots, P_d$. Above P_d it has under R the $d - 1 \geq 2$ siblings $P_{d-1}, \dots, P_1$ and under L the single sibling $\text{Bag}_L(P_1, \dots, P_{d-1})$. Above P_i with $2 \leq i < d$ the first sibling is on the right under R , namely $\text{Bag}_R(P_{i+1}, \dots, P_d)$, and on the left under L , namely $\text{Bag}_L(P_1, \dots, P_{i-1})$.

(3) Both trees are built on the peaks of Definition 3.4, which depend on the leaf sequence alone; there are $d \leq \lceil \log_2 n \rceil + 1$ of them. Appending x_{n+1} joins the last peaks of equal altitude with x_{n+1} , at most $\lceil \log_2 n \rceil + 1$ evaluations of H , and re-bagging the at most $\lceil \log_2 (n + 1) \rceil + 1$ new peaks costs one evaluation fewer than their number, under either fold. A path has at most $2\lceil \log_2 n \rceil$ siblings under either fold by Lemma 3.7, whose depths are the stated ones. $\square$

Collision resistance alone does not exclude a cycle, so part (1) does not assert that the two roots differ numerically. No later statement of this volume uses such an inequality; the later sections use the shapes and path lengths of part (2) and Lemma 3.7.

Example 3.9 (Eleven leaves). The eleven-leaf tree of ZIP 221, “Background”, has $11 = 8 + 2 + 1$ and peaks P_1, P_2, P_3 of altitudes 3, 1 and 0, joined by two bagging nodes. The left fold that ZIP 221 commits and the right fold are

$$\text{rt}_{11}^L = H(H(P_1 \parallel P_2) \parallel P_3), \quad \text{rt}_{11}^R = H(P_1 \parallel H(P_2 \parallel P_3)).$$

A leaf of P_1, P_2, P_3 has path length 4, 3, 2 under Bag_R and 5, 3, 1 under Bag_L . Figure 2 shows both trees.

3.3 Appending a leaf

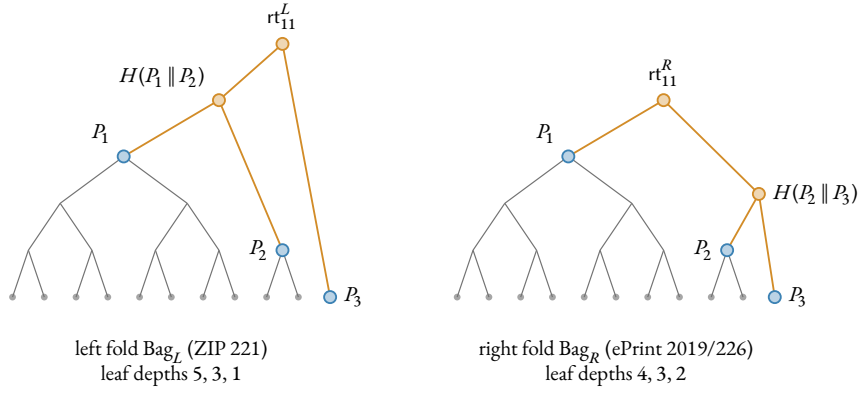


Figure 2. The eleven-leaf tree of Example 3.9, without node numbering. Both panels carry the same peaks P_1, P_2, P_3 (blue) of altitudes 3, 1, 0 over the same leaves. Left: the left fold $rt_{11}^L = H(H(P_1 \parallel P_2) \parallel P_3)$ that ZIP 221 commits. Right: the right fold $rt_{11}^R = H(P_1 \parallel H(P_2 \parallel P_3))$ of the recursion of Definition 3.1. Bagging nodes are amber. The leaves of P_1 lie one level deeper under the left fold; the depths below each panel are those of a leaf of P_1, P_2, P_3 .

Construction 3.10 (Append). Every node v of a range carries its leaf count m . Procedure $\text{Append}(v, x)$ takes the root v of a range over m leaves and a value $x \in \{0, 1\}^k$ and returns the root of a range over $m + 1$ leaves:

1. if m is a power of two (v is perfect; $m = 1$ included), return a new node with left child v , right child the leaf x , value $H(v \parallel x)$ and leaf count $m + 1$;
2. otherwise, with children v_ℓ and v_r , replace v_r by $\text{Append}(v_r, x)$, set the value of v to $H(v_\ell \parallel v_r)$ with the new v_r , set its leaf count to $m + 1$, and return v .

The construction is ePrint 2019/226, Algorithm 5. The order $H(v \parallel x)$ in step 1 is that of the algorithm and of Definition 3.1; the base case of the paper's proof of its Theorem 6 writes the reverse order.

Theorem 3.11 (Correctness of append). *For every leaf count $n \geq 1$, $\text{Append}(M_n, x_{n+1})$ is M_{n+1} over $(x_1, \dots, x_{n+1})$: the leaves of M_n followed by x_{n+1} as the rightmost leaf.*

Proof. By induction on n , following ePrint 2019/226, Theorem 6. If $n = 2^i$, Definition 3.1 for $n + 1$ has $\lfloor \log_2 n \rfloor = i$ and right part 1: left child M_n , right child the leaf x_{n+1} , which is step 1. Otherwise $n = 2^i + m$ with $1 \leq m < 2^i$, and $n + 1 = 2^i + (m + 1)$ with the same i , since $m + 1 \leq 2^i$. The left child M_{2^i} is unchanged, and the right child is Append applied to the range M_m over $(x_{2^i+1}, \dots, x_n)$, which is M_{m+1} over $(x_{2^i+1}, \dots, x_{n+1})$ by hypothesis; step 2 rehashes the root. $\square$

Proposition 3.12 (Cost of append). *Procedure $\text{Append}(M_n, x_{n+1})$ creates or rebashes only right-spine nodes of M_n and one new node, with at most $\lfloor \log_2 n \rfloor + 1$ evaluations of H . Every left child of M_n , and the perfect right-spine node at which the recursion stops, is a subtree of M_{n+1} with unchanged values.*

Proof. The recursion descends the right spine until the first node whose leaf count is a power of two, at depth at most $\lceil \log_2 n \rceil$ (Lemma 3.2(4)), evaluates H once to create the new node, and once more at each spine node above it. Left children and the node where the descent stops are never modified. $\square$

Figure 1 shows the case $n = 6$: two evaluations of H create $H(P_2 \parallel x_7)$ and rt_7 .

3.4 Inclusion paths and prefix roots

Definition 3.13 (Inclusion path). Let $b \in \{R, L\}$, $n \geq 1$, $1 \leq j \leq n$, and $m = \text{depth}_b(n, j)$ (Lemma 3.7). Let $u_0, u_1, \dots, u_m$ be the nodes from leaf j to the root of M_n^b . The *inclusion path* of leaf j is $\pi = (\pi_1, \dots, \pi_m)$, where π_t is the value of the sibling of u_{t-1} , listed bottom-up. The *side* $\sigma_t \in \{\text{left}, \text{right}\}$ records whether that sibling is the left or the right child of u_t . The sides and m are functions of (b, n, j) through the shape of M_n^b .

The verifier $\text{Verify}_b(\text{rt}, n, j, x, \pi)$, on $\text{rt}, x \in \{0, 1\}^*$ and a sequence π of κ -bit values:

1. rejects unless $|\pi| = \text{depth}_b(n, j)$;
2. sets $y_0 = x$ and, for $t = 1, \dots, m$, $y_t = H(\pi_t \parallel y_{t-1})$ if $\sigma_t = \text{left}$ and $y_t = H(y_{t-1} \parallel \pi_t)$ otherwise;
3. accepts if and only if $y_m = \text{rt}$.

By construction Verify_b accepts (x_j, π) with π the inclusion path of leaf j against rt_n^b . Under Bag_R every path has at most $\lceil \log_2 n \rceil$ siblings (Lemma 3.7); ePrint 2019/226, Definition 10, counts at most $\log_2 n + 1$ hashes, the leaf pair included. For $d \geq 2$, path lengths differ between leaves (Lemma 3.7). The paper’s Algorithm 4 requires $|\pi| = \lceil \log_2(n - 1) \rceil$ and so rejects valid paths: 1,225 of the 8,384 leaf paths of the ranges M_n , $2 \leq n \leq 129$, among them both leaves of M_2 and leaf 5 of M_6 . The verifier above fixes the length from (b, n, j) instead. For n a power of two, M_n^b is a perfect tree, and the inclusion path and its verifier coincide with the *Crypto Guide*’s Definition “Authentication path” and Definition “Path verification” (§“Authentication paths and membership proofs”) at index $j - 1$, with the bits b_t fixed by j instead of carried in the path.

Theorem 3.14 (Prefix roots from one path). For $1 \leq j \leq n$, from x_j and the inclusion path of leaf j in M_n a verifier computes rt_j and, for $j \geq 2$, rt_{j-1} , the roots of the ranges over $(x_1, \dots, x_j)$ and $(x_1, \dots, x_{j-1})$.

Proof. For rt_j , by induction on n , following ePrint 2019/226, Theorem 7, with siblings bottom-up. If $j = n$, $\text{rt}_j = \text{rt}_n$ is the recomputed root y_m . Otherwise $n > 1$; let 2^i be the largest power of two strictly below n , so that the root’s children are M_{2^i} and a range over $n - 2^i$ values.

1. If $j = 2^i$, rt_j is the value of the root’s left child, which is y_{m-1} .
2. If $j < 2^i$, the path without its top sibling is the inclusion path of leaf j in the left child M_{2^i} , and the hypothesis applies.

3. If $2^i < j < n$, then $j - 1 \geq 2^i$ and $j < 2^{i+1}$, so 2^i is also the largest power of two strictly below j . The top sibling is P , the value of M_{2^i} , and the path without it is the inclusion path of leaf $j - 2^i$ in the right child; by hypothesis it yields the root r' over $(x_{2^i+1}, \dots, x_j)$, and $\text{rt}_j = H(P \parallel r')$ by Definition 3.1.

For rt_{j-1} , the bottom-up fold of Lemma 3.15, whose proof does not use this theorem. The paper's proof lists the path top-down with the path nodes included, while its Algorithm 3 outputs the siblings bottom-up; the bottom-up format of Definition 3.13 is the one fixed here. $\square$

Lemma 3.15 (Left siblings cover the prefix). *Let $b \in \{R, L\}$, $1 \leq j \leq n$, and consider the inclusion path of leaf j in M_n^b . Its left siblings are subtrees whose leaf runs partition $\{1, \dots, j - 1\}$, higher siblings covering earlier leaves; its right siblings cover $\{j + 1, \dots, n\}$ and enter none of the folds below. Let $j \geq 2$ and let leaf j lie in peak P_i .*

1. Under Bag_R the left siblings, from the top down, are the peaks of $(x_1, \dots, x_{j-1})$. The bottom-up fold (set r to the lowest left sibling, then $r \leftarrow H(\pi \parallel r)$ for each further left sibling π upwards) yields rt_{j-1} ; started instead from $r = x_j$ and applied to every left sibling, it yields rt_j .
2. Under Bag_L , for $i \geq 2$ the topmost left sibling is $\text{Bag}_L(P_1, \dots, P_{i-1})$ and the others lie in P_i , with sizes the terms of the binary expansion of $j - 1 - s_i$; for $i = 1$ all left siblings lie in P_1 . The top-down fold (set r to the topmost left sibling, then $r \leftarrow H(r \parallel \pi)$ for each further left sibling π downwards) yields rt_{j-1}^L .

Proof. A leaf $l < j$ leaves the path of leaf j at their lowest common ancestor, in whose left child l lies; that left child is a left sibling on the path. Distinct ancestors give disjoint runs, higher ancestors earlier runs. The same argument for $l > j$ gives the right siblings. Inside a perfect subtree the left siblings of the leaf at offset o from its first leaf are perfect subtrees whose sizes are the terms of the binary expansion of o , largest highest.

Leaf j lies at offset $o = j - 1 - s_i$ in P_i , and $j - 1 = s_i + o$, so the peaks of $(x_1, \dots, x_{j-1})$ are $P_1, \dots, P_{i-1}$ followed by the left siblings inside P_i , top-down. Under R the siblings $P_{i-1}, \dots, P_1$ lie above P_i as left children of bagging nodes (proof of Lemma 3.7); the bottom-up fold is then the right fold of the peaks of $(x_1, \dots, x_{j-1})$, which is rt_{j-1} by Proposition 3.6. Started from x_j , it is the right fold of those peaks followed by x_j . The largest power of two strictly below j is the highest term of $j - 1$, the size of the first of these peaks, and the recursion of Definition 3.1 on j splits off the peaks of $j - 1$ in turn and ends at the single leaf x_j ; this fold is therefore rt_j . Under L with $i \geq 2$ the root of P_i is the right child of the bagging node $H(\text{Bag}_L(P_1, \dots, P_{i-1}) \parallel P_i)$, and every higher node on the path is a left child; the top-down fold is the left fold of the peaks of $(x_1, \dots, x_{j-1})$. For $i = 1$ the root of P_1 is a left child and every higher node on the path too. $\square$

Construction 3.16 (Prefix root). Procedure $\text{PrefixRoot}_b(n, j, \pi)$ takes $b \in \{R, L\}$, $2 \leq j \leq n$ and a sequence π of κ -bit values with $|\pi| = \text{depth}_b(n, j)$, with sides σ_t as in Definition 3.13. It selects the π_t with $\sigma_t = \text{left}$ and outputs their bottom-up fold for $b = R$ and their top-down fold for $b = L$ (Lemma 3.15).

Correctness of the construction is Lemma 3.15. The construction corrects the printed pseudocode of ePrint 2019/226, Algorithm 6, which uses an undefined index, ends in a vestigial boolean comparison, and has a call signature that does not match its call in Algorithm 2, and it fixes the siblings bottom-up.

Theorem 3.17 (Position binding). *Let $b \in \{R, L\}$, $n \geq 1$ and $1 \leq j \leq n$. From any $\text{rt} \in \{0, 1\}^k$, chosen by the adversary, and two pairs $(x, \pi) \neq (x', \pi')$ both accepted by $\text{Verify}_b(\text{rt}, n, j, \cdot, \cdot)$, a collision of H is computed with at most $2 \text{depth}_b(n, j)$ evaluations of H . In particular $x \neq x'$ yields a collision, and the siblings of an accepted path and every value it recomputes are functions of (rt, b, n, j) , barring a collision.*

Proof. Both chains $y_0, \dots, y_m$ and $y'_0, \dots, y'_m$ of Definition 3.13 have the length $m = \text{depth}_b(n, j)$ and the sides fixed by (b, n, j) , and $y_m = y'_m = \text{rt}$. Since $(x, \pi) \neq (x', \pi')$, some step has differing inputs; let t be the highest step with $(y_{t-1}, \pi_t) \neq (y'_{t-1}, \pi'_t)$. Then $y_t = y'_t$ for $t = m$ both equal rt , and for $t < m$ the inputs of step $t + 1$ agree. The two 2κ -bit inputs of step t place their operands on the same sides and differ, and H maps both to y_t : a collision. For $n = 1$ the path is empty and Verify_b accepts only $x = \text{rt}$, so no two distinct pairs are accepted. The extractor is deterministic and the reduction tight in the sense of the *Crypto Guide*, §“Reductions”; it is the first-level-of-agreement argument of the *Crypto Guide*, Proposition “Binding of the root” (§“Binary Merkle trees”) and §“Soundness: forging a path implies a collision”. $\square$

The theorem extends the *Crypto Guide*’s Theorem “Merkle trees are succinct position-binding vector commitments” (§“Vector commitments”) from perfect trees to the shapes fixed by n and the bagging order. It is stated for a root chosen by the adversary, not for a fixed honest root as in the *Crypto Guide*’s Remark “Scope: a fixed honest root”: in FlyClient the root of the head comes from the prover.

Corollary 3.18 (Prefix binding and root sensitivity). *Let $b \in \{R, L\}$ and $1 \leq j \leq n$.*

1. *If $\text{rt} = \text{rt}_n^b$ over $(x_1, \dots, x_n)$ and Verify_b accepts (x, π) at position j , then $x = x_j$ and, for $j \geq 2$, $\text{PrefixRoot}_b(n, j, \pi) = \text{rt}_{j-1}^b$ over $(x_1, \dots, x_{j-1})$, barring a collision: one accepted path binds $x_1, \dots, x_j$ as a prefix of the committed sequence.*
2. *If $(x'_1, \dots, x'_m)$ differs from $(x_1, \dots, x_m)$ at some $i \leq m$, then $\text{rt}_m^b(x') \neq \text{rt}_m^b(x)$, barring a collision: changing x_i changes every root rt_m^b with $m \geq i$.*

When x_i is the hash of block i , binding the blocks themselves requires in addition collision resistance of that hash.

Proof. (1) The honest pair (x_j, π^*) is accepted. By Theorem 3.17 the pairs (x, π) and (x_j, π^*) are equal barring a collision, so $x = x_j$ and the fold of Construction 3.16 on π is the fold on π^* , which is rt_{j-1}^b by Lemma 3.15. (2) The two trees have the same shape M_m^b . The honest paths of leaf i in both are accepted against the common root, with $x_i \neq x'_i$; Theorem 3.17 yields a collision. $\square$

Part (1) is ePrint 2019/226, Corollary 3, and part (2) its Corollary 4, stated for both bagging orders and with the collision named.

3.5 Aggregating ranges

A verifier that samples leaves by cumulative weight needs each path to bind the sampled leaf's weight interval, and a verifier that constrains unsampled leaves needs aggregates over their runs; both follow from metadata propagated up the tree (ePrint 2019/226, Section 2 and Section 6).

Definition 3.19 (Aggregating range). An *aggregating range* extends Definition 3.1, after ZIP 221, “Specification”, which propagates metadata “by either summing the metadata of both children, or inheriting the metadata of a specific child”. It is given by:

1. a set of *node tuples* $v = (v.h, v.f_1, \dots, v.f_r)$ with $v.h \in \{0, 1\}^k$;
2. a *serialisation* ser from tuples to bit strings that is injective and prefix-free, no value of ser being a proper prefix of another, so that $\text{ser}(\ell) \parallel \text{ser}(r)$ determines the pair (ℓ, r) ;
3. a *leaf map* from a leaf's data to its tuple;
4. for each field f_i one rule: inherit from the left child, inherit from the right child, or sum, in $\mathbb{Z}$ or in $\mathbb{Z}/2^{256}\mathbb{Z}$;
5. the *parent map* $\text{par}(\ell, r)$, whose hash field is $H(\text{ser}(\ell) \parallel \text{ser}(r))$, over the children's full serialisations, and whose other fields follow the rules.

A labelling of the shape M_n^b with leaf tuples at the leaves is *consistent* when every internal node is par of its children; its *root* is the root's tuple. Inclusion paths and Verify_b are those of Definition 3.13 with tuples in place of values and par in place of $H(\cdot \parallel \cdot)$; Verify_b accepts if and only if the recomputed root tuple equals the given one.

A serialisation of fixed width is prefix-free. Injectivity alone does not suffice: with $\text{ser}(x) = 0$, $\text{ser}(y) = 01$ and $\text{ser}(z) = 10$, the pairs (x, z) and (y, x) have the same concatenation 010 .

Lemma 3.20 (Aggregate binding). *Let ser be injective and prefix-free, $b \in \{R, L\}$ and $n \geq 1$.*

1. *From two consistent labellings of M_n^b with equal root tuples that differ at some node, a collision of H is computed; barring a collision, equal roots imply agreement on every field of every node.*
2. *From any root tuple, chosen by the adversary, and two distinct accepted paths (leaf tuple and sibling tuples) at one position j , a collision of H is computed: an accepted path binds the full tuple of every node it recomputes and of every sibling it carries.*
3. *For $2 \leq j \leq n$ and the inclusion path of leaf j in a consistent labelling, the folds of Lemma 3.15 run with par in place of $H(\cdot \parallel \cdot)$ yield the root tuple of the consistent labelling of M_{j-1}^b over the first $j - 1$ leaf tuples.*

Proof. Unique decoding: if $\text{ser}(\ell) \parallel \text{ser}(r) = \text{ser}(\ell') \parallel \text{ser}(r')$, one of $\text{ser}(\ell)$, $\text{ser}(\ell')$ is a prefix of the other, hence they are equal, and $\ell = \ell'$ by injectivity; then $\text{ser}(r) = \text{ser}(r')$ and $r = r'$.

(1) Take a node at which the labellings differ and whose parent they agree on; one exists because they agree at the root. The parent tuples are equal, so their hash fields are, while the child pairs differ; by unique decoding the two hash inputs differ: a collision.

(2) As in the proof of Theorem 3.17, with tuples: at the highest step whose inputs differ, the outputs are equal tuples, hence have equal hash fields, and the inputs are distinct pairs, hence by unique decoding distinct strings: a collision.

(3) In a consistent labelling the tuple of every subtree, a bagging node included, is determined by its leaf tuples through par . By Lemma 3.15 the left siblings are, under R , the peak tuples of the consistent labelling over the first $j - 1$ leaves, and so are they under L with $i \leq 2$. Under L with $i \geq 3$, the topmost is the tuple of the bagging node $\text{Bag}_L(P_1, \dots, P_{i-1})$ of that labelling, and the others are its peak tuples inside P_i . The folds of that lemma, run with par , therefore yield the root tuple of M_{j-1}^b . The engine of (1) and (2) is the *Crypto Guide*, Proposition “Binding of the root” (§“Binary Merkle trees”). $\square$

Lemma 3.21 (Cumulative-weight interval of a leaf). *Let ser be injective and prefix-free, let w be a summed field, w_j the w -field of the tuple of leaf j , and W the sum of the w -fields of the left siblings on an accepted path of leaf j .*

1. *Binding, for any root tuple chosen by the adversary: barring a collision, W and w_j are functions of the root, b , n and j . One accepted path therefore binds the interval $[W, W + w_j)$ of leaf j .*
2. *Meaning, for the path of a consistent labelling: $W = \sum_{i < j} w_i$, under either bagging order. For a sum taken modulo 2^{256} , this holds as an identity of integers when the sum of w over all n leaves is below 2^{256} .*

Part (2) is not claimed for a root chosen by the adversary, whose siblings off the path are never recomputed from their children.

Proof. (1) Lemma 3.20(2). (2) In a consistent labelling the summed field of a node is the sum over its leaves, by induction on height; with the total below 2^{256} no reduction occurs. The left siblings partition $\{1, \dots, j - 1\}$ by Lemma 3.15, a property of any binary tree. $\square$

Two instances follow. The difficulty Merkle mountain range of ePrint 2019/226, whose nodes aggregate difficulty and time, is Definition 5.6 in §5.3. The node vector of ZIP 221, whose nodes aggregate work, heights and pool data, is Definition 6.3 in §6.3.

4 The FlyClient security model

This section fixes the model in which the FlyClient theorem is stated: the variable-difficulty backbone with its unit of work, the adversary and persistence; the retarget rule, the attack it excludes and the limit of that exclusion for a sampling verifier; the (c, L) assumption; and the definitions of security and succinctness for chain proofs.

4.1 The variable-difficulty backbone model

The model of this section is the one ePrint 2019/226 adopts in Section 3.1 and Section 3.2, after Garay, Kiayias and Leonardos (the variable-difficulty backbone, CRYPTO 2017), and it is stated as the paper adopts it. Whether a Zcash chain satisfies its hypotheses is classified in §7, not here.

Definition 4.1 (Variable-difficulty backbone model). Fix the security parameter λ (*Math Guide*, §“The security parameter”) and a hash length κ . The execution proceeds in rounds $r = 1, 2, \dots$. The proof-of-work function is a random oracle $H: \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$ (*Crypto Guide*, §“The random oracle model”), its outputs read as integers in $[0, 2^\kappa)$. In round r there are ν_r honest and t_r adversarial *active players*, and each active player makes q sequential queries to H per round. A block B declares a *target* $T \in [1, 2^\kappa]$ in its header, and B carries a *valid proof of work* at T if and only if $H(B) < T$. A message broadcast in round r is delivered to every player in round $r + 1$, in an order the adversary chooses. The honest participation sequence $(\nu_r)_r$ is (γ, s) -*respecting*: for every set S of at most s consecutive rounds,

$$\max_{r \in S} \nu_r \leq \gamma \min_{r \in S} \nu_r.$$

Each block also records the round in which it was mined, which Definition 4.5 reads.

The paper’s paraphrase in Section 3.1 omits “consecutive”; read literally, for $s \geq 3$ it bounds participation over the whole execution.

The paper writes λ for the oracle output length and n_r for the honest player count; here the length is κ , λ being the security parameter, and the count is ν_r , n being a leaf count (§1.2).

Definition 4.2 (Difficulty level and total work). A single query to H meets target T with probability $T/2^\kappa$. The *difficulty level* of T is

$$D(T) = 2^\kappa / T,$$

the expected number of queries per block valid at T . A harder target is a smaller T and a larger $D(T)$: the level is a work-like quantity inversely proportional to the target and is never the target itself. The *total work* of a finite sequence of blocks $B_1, \dots, B_m$ with declared targets $T_1, \dots, T_m$ is

$$\omega = \sum_{i=1}^m D(T_i),$$

the paper’s *cumulative difficulty*. A chain is *heavier* than another when its total work is larger.

The paper also calls a level a “target” in its Definition 7; in this volume D denotes only a level and T only a target. The Zcash analogue, the work of a block computed from its compact target, is defined in the *Consensus Guide*, §“Work, and the best chain”, from protocol specification, §“Definition of Work”. It is a different normalisation of the same ordering and is not used by the model.

Definition 4.3 (Adversary). Fix μ with $0 < \mu < 1$. The *adversary* is a PPT algorithm (*Math Guide*, §“Algorithms, running time, and PPT”; *Crypto Guide*, §“Adversaries and the security parameter”) that controls the t_r adversarial players of each round r , with $t_r < \mu v_r$, so that it makes at most $t_r q$ queries to H in round r . It is *rushing*: in each round it receives every honest message of the round before choosing its own, and it orders the delivery of all messages. Its goal is to make a light client accept a chain whose total work is at least that of the honest chain.

The prior knowledge of the verifier is fixed once, in Definition 4.16.

Assumption 4.4 (Persistence and liveness). The backbone parameters $(\mu, \gamma, s, \tau, E, f)$, with τ , E and f the constants of Definition 4.5, are such that there is a *persistence depth* ℓ with, except with probability negligible in λ :

1. (persistence) for all honest players P_1, P_2 and rounds $r_1 \leq r_2$, the chain of P_1 at round r_1 with its last ℓ blocks removed is a prefix of the chain of P_2 at round r_2 ;
2. (liveness) a transaction given to every honest player for a number of consecutive rounds fixed by the parameters lies in that stable prefix of every honest chain.

ePrint 2019/226, Section 3.1 (p. 7), adopts this as a parameter choice, citing Theorems 26 and 27 of its reference [22], Garay, Kiayias and Leonardos, “The Bitcoin backbone protocol: analysis and applications” (EUROCRYPT 2015), a fixed-difficulty analysis; it names no theorem of its reference [23] (CRYPTO 2017), the source of the variable-difficulty model. No cited theorem establishes the assumption in the model of Definition 4.1.

The tips of honest chains need not coincide: persistence gives agreement only on all but the last ℓ blocks. The paper’s sentence that persistence and liveness give “a single chain adopted by all honest full nodes” overstates this; the assumption states the corrected form. The persistence depth ℓ is distinct from the catch parameter k of §5.

4.2 Retargeting and the difficulty-raising attack

Definition 4.5 (Target recalculation with dampening). Fix constants $E \geq 1$ (the *retarget-epoch length*, in blocks), $\tau > 1$ (the *dampening factor*), an initial participation estimate $v_0 > 0$ and an initial target T_0 . Targets are constant within each *retarget epoch* of E consecutive blocks. At the end of an epoch whose E blocks were mined at target T in Δ rounds, read from the rounds the blocks record, the *participation estimate* is

$$\hat{v}(T, \Delta) = \frac{2^* E}{q T \Delta},$$

and the target of the next epoch is

$$\text{Retarget}(T, \Delta) = \begin{cases} T/\tau & \text{if } (v_0/\hat{v}(T, \Delta)) T_0 < T/\tau, \\ \tau T & \text{if } (v_0/\hat{v}(T, \Delta)) T_0 > \tau T, \\ (v_0/\hat{v}(T, \Delta)) T_0 & \text{otherwise,} \end{cases}$$

that is, the value $(\nu_0/\hat{\nu}) T_0$ clamped to $[T/\tau, \tau T]$. Let $f = q\nu_0 T_0/2^\kappa$, the expected number of blocks per round of ν_0 players at T_0 . A chain *respects* the rule when every epoch's target is the image under Retarget of the previous epoch's target and round span.

The unclamped value is the *raw retarget*

$$\frac{\nu_0}{\hat{\nu}(T, \Delta)} T_0 = \frac{\nu_0 T_0 q T \Delta}{2^\kappa E} = \frac{T \Delta f}{E},$$

which restores f blocks per round at the estimated participation. The current target T is an input of the rule; the paper's list of constants omits it. The definition is ePrint 2019/226, Definition 1 (Section 3.1, p. 7), after Garay, Kiayias and Leonardos, Definition 2. The function is written Retarget rather than the paper's $D(n_0, T_0)$, because D is a difficulty level, and ν_0 and T_0 are the initial constants of the cited definition, not the previous-epoch quantities of the gloss in the paper's Definition 1. The retarget epoch is distinct from the upgrade epoch of §6.

Under variable difficulty the total work of a chain, not its length, decides between chains. An adversary may therefore mine fewer blocks at higher levels and use the larger variance of a sum of few heavy terms to exceed a more powerful honest chain: the *difficulty-raising attack* of Bahack (arXiv:1312.7013), recalled in ePrint 2019/226, Section 3.1.

Proposition 4.6 (Success of the difficulty-raising attack). *Model block discovery by exponential clocks (Consensus Guide, §“Memoryless clocks and the attribution sequence”): honest queries arrive at rate Λ per unit time and adversarial queries at rate $\Lambda/3$, one third of the honest power; each query succeeds independently with probability $1/D$ at level D , so that the successes of a party querying at rate Λ' at level D form an exponential clock of rate Λ'/D . Fix a time $t_0 > 0$ and let $W = \Lambda t_0$.*

- (i) *The expected total work of the blocks the honest players find in $[0, t_0]$ is W , whatever their targets, so the expected total work of the honest chain at t_0 is at most W .*
- (ii) *Suppose the adversary may mine one block at an arbitrary level, in violation of Definition 4.5, and sets its level to $D = W$. Then it finds that block, whose work alone equals W , before t_0 with probability*

$$1 - e^{-1/3} \approx 0.2835,$$

about 28.3 per cent, a constant independent of λ .

Proof. (i) An honest query at target T yields a block of work $D(T)$ with probability $T/2^\kappa = 1/D(T)$, hence has expected work 1 whatever the target. By linearity of expectation the expected work found in $[0, t_0]$ equals the expected number of honest queries in $[0, t_0]$, which is $\Lambda t_0 = W$ (*Math Guide*, §“Exponential clocks, Poisson counts, and the Chernoff method”, Theorem “Poisson arrival counts”). The honest chain carries at most the work found.

(ii) At level $D = W$ the adversary's successes form an exponential clock of rate $(\Lambda/3)/W = 1/(3t_0)$. Its first arrival exceeds t_0 with probability $e^{-t_0/(3t_0)} = e^{-1/3}$ (*Math Guide*, same section, Lemma “Tail, mean, and memorylessness”), so it occurs before t_0 with probability $1 - e^{-1/3}$. $\square$

In the discrete query model the same statement holds as an inequality: if $W/3$ is an integer, the adversary's $W/3$ queries at level W all fail with probability $(1 - 1/W)^{W/3} \leq e^{-1/3}$, since $1 - x \leq e^{-x}$, so it succeeds with probability at least $1 - e^{-1/3}$. Since $\log(1 - x) \geq -x - x^2$ for $0 \leq x \leq 1/2$, $(1 - 1/W)^{W/3} \geq e^{-1/3 - 1/(3W)} \geq e^{-1/3}(1 - 1/(3W))$, so for every such $W \geq 3$ the two probabilities differ by at most $e^{-1/3}/(3W) < 1/W$. The paper states “roughly 28%” without derivation (ePrint 2019/226, Section 3.1, p. 7); the comparison, as in the paper, is with the expected honest work.

Remark 4.7 (The clamp as a defence). Definition 4.5 excludes the attack of Proposition 4.6: a target changes only at epoch ends and by at most the factor τ , so no block may declare an arbitrary level. The paper, citing Garay, Kiayias and Leonardos, states that this rule suffices against difficulty-raising attacks for full nodes, which see every transition and reject a chain that violates it (ePrint 2019/226, Section 3.1, p. 7); that sufficiency is the cited authors’ result and is not proved here. A verifier that samples few blocks sees few transitions. The clamp constrains the transitions it does not see only through data that the chain commits to, which §5.3 constructs; no conclusion about that construction is drawn here. The Zcash retarget rule is a different function, a per-block rule over an averaging window, defined in protocol specification, §“Difficulty adjustment”, and described in the *Consensus Guide*, §“Difficulty in motion”; it is not an instance of Definition 4.5.

4.3 The (c, L) -adversary

Fix an honest player and let C be its chain at round r , the *honest chain*. A *fork* of a chain C' against C has *fork point* a , the last block common to C and C' ; its blocks are those of C' after a . A block of the fork is *validly mined* if it carries a valid proof of work at its declared target (Definition 4.1), and *invalid* otherwise; its *validly mined work* is the total work of its validly mined blocks.

Assumption 4.8 ((c, L) -adversary). The printed form (ePrint 2019/226, Assumption 2, Section 3.2, p. 8) reads: “There exists no adversary in the variable-difficulty model that respects the target recalculation function from Definition 1 and can, with non-negligible probability, produce a fork that contains more than L blocks such that a c fraction of the difficulty weight in these blocks is honest.”

The operative form, for $0 < c < 1$ and $L \geq 1$: for every PPT adversary respecting Definition 4.5 in the model of Definition 4.1, except with probability negligible in λ , every fork it produces that has more than L blocks after its fork point a , and total work after a at least the total work of C after a at the same round, has validly mined work less than c times its total work after a . Such a fork is therefore at least a $(1 - c)$ fraction invalid by weight.

The operative form departs from the printed one in three stated respects. The word “honest” in the printed form means validly mined. The length condition is “more than L blocks”, the printed assumption’s own wording, where the paper’s preceding sentence (Section 3.2, p. 8) says “of length L or longer”; the choice is consistent with checking the last L blocks in full (Remark 4.10). The operative form quantifies only over forks whose work after the fork point reaches that of the honest chain, following the same sentence (“a c fraction of the honest forks weight”): the unrestricted reading fails for every $\mu > 0$, since the adversary’s own fully valid private fork of more than L blocks has valid fraction 1, and only forks that reach the honest weight can win under

Definition 4.16.

Remark 4.9 (Two readings of c). In Assumption 4.8 the parameter c bounds the validly mined fraction of a competitive long fork’s weight. The paper’s evaluation (ePrint 2019/226, Section 7.1, p. 20, and Section 7.2) reads it instead as a power ratio ρ , adversarial to honest mining power, so that the adversary’s share of total power is $\rho/(1 + \rho)$: one third at $\rho = 1/2$, and $9/19 \approx 0.4737$, about 47.4 per cent, at $\rho = 0.9$, where the paper prints 47.3 per cent by truncation. The identification of c with ρ is not part of the assumption; it needs a bridge from mining power to fork weight, which Remark 4.13 classifies.

Remark 4.10 (Short forks and the checked suffix). Assumption 4.8 constrains only forks of more than L blocks, by design: over a short window the variance of mining lets even an adversary with small power produce a heavier short fork with non-negligible probability (ePrint 2019/226, Section 3.2, p. 8). A verifier relying on the assumption must therefore check in full a suffix of the claimed chain containing at least its last L blocks. At constant difficulty the paper takes $L = \delta n$; under variable difficulty it states the checked suffix as a weight fraction δ (Section 6, p. 17), leaving its relation to the block count L implicit. The size of the suffix under variable difficulty is fixed in §5.4 (Remark 5.24); no conclusion about it is drawn here.

Remark 4.11 (Lemma 1 of the paper). The paper’s only derivation of Assumption 4.8 from mining power is its Lemma 1 with Corollary 1 (ePrint 2019/226, pp. 8–9), stated here to register their defect. Lemma 1 as printed: in the constant-difficulty backbone, if X is the number of blocks any adversary mines while the honest chain adopts L blocks, and the adversary’s block rate is at most μ times the honest adoption rate, then for $c > \mu$,

$$\Pr[X \geq cL] \leq e^{L(c-\mu)} (c/\mu)^{-cL}.$$

Corollary 1: for $L = \Theta(\lambda)$ and every μ there is $c < 1$ for which the assumption holds at constant difficulty.

The printed bound is the Chernoff bound of a Poisson variable of mean μL (*Math Guide*, §“Exponential clocks, Poisson counts, and the Chernoff method”, Corollary “Chernoff bound for a Poisson variable”). The proof asserts without proof that such a variable models X , and it switches to the undefined symbols $1 - \delta$ and n , a footnote setting $\mu = 1 - \delta$. Under independent exponential block clocks the count X is negative binomial (same section, Theorem “Negative-binomial arrival count”), and the printed expression is not an upper bound for it in general: the *Math Guide*’s Example “The Poisson expression is not a bound for the arrival count”, in the same section, exhibits $\mu = 1/2$, $c = 3/4$, $L = 200$ and $L = 400$ with the exact tail above the printed expression. Lemma 1 and Corollary 1 are therefore unproved as printed; Proposition 4.12 replaces them in the clock model.

The clock model identifies, at constant difficulty, honest adoption and adversarial block discovery after a fork point a with independent exponential clocks started at a , of rates 1 and μ , the rates normalised to the honest adoption rate (*Consensus Guide*, §“Memoryless clocks and the attribution sequence”). For $M \geq 1$ let $X_{a,M}$ be the number of adversarial arrivals strictly before the M -th honest arrival after a . The count $X_{a,M}$ is nondecreasing in M .

Proposition 4.12 (Constant-difficulty bound). *In the clock model, count as validly mined in a fork from a only blocks the adversary finds after a . Let $0 < \mu < c < 1$ and*

$$\beta = \left(\frac{1+c}{1+\mu} \right)^{1+c} \left(\frac{\mu}{c} \right)^c, \quad \theta = \left(\frac{c(1+\mu)}{\mu(1+c)} \right)^c.$$

Then:

- (i) *for every fork point a and $M \geq 1$, $\Pr[X_{a,M} \geq cM] \leq \beta^M$, and $\beta < 1$;*
- (ii) *for any set A of candidate fork points, the probability that for some $a \in A$ a fork from a exists at some time with more than L blocks, total work after a at least that of the honest chain, and validly mined fraction at least c , is at most*

$$|A| \frac{(1+\theta) \beta^{L+1}}{1-\beta}.$$

For $|A|$ polynomial in λ and $L = \Theta(\lambda)$ this is negligible in λ , so Assumption 4.8 holds in the clock model, which recovers Corollary 1 there. The proposition says nothing about the variable-difficulty backbone.

Proof. (i) By memorylessness the clocks restart at a , and the bound is the *Math Guide*’s Theorem “Chernoff bound for the arrival count”, with the law of Theorem “Negative-binomial arrival count” (§“Exponential clocks, Poisson counts, and the Chernoff method”), which also shows $\beta < 1$. That proof attains β at $e^{t^*} = c(1+\mu)/(\mu(1+c)) > 1$ as $e^{-t^*c} \Phi_1(t^*) = \beta$, where $\Phi_M(t) = (1+\mu-\mu e^t)^{-M}$ is the moment generating function of $X_{a,M}$; note that $\theta = e^{t^*c}$.

(ii) Fix $a \in A$. At constant difficulty the work of a block is constant, so work is block count. Consider a time at which the honest chain has m blocks after a , and a fork from a at that time with L'' blocks, total work at least that of the honest chain and validly mined fraction at least c . Then $L'' \geq \max(m, L+1)$. The validly mined blocks were found by the adversary after a and before the $(m+1)$ -th honest arrival, so there are at most $X_{a,m+1}$ of them, and the fraction condition gives

$$X_{a,m+1} \geq cL'' \geq c \max(m, L+1).$$

If $m \leq L$, then $X_{a,L+1} \geq X_{a,m+1} \geq c(L+1)$, an event of probability at most β^{L+1} by (i). If $m = M-1$ with $M \geq L+2$, the event is $X_{a,M} \geq cM - c$; the Chernoff method (*Math Guide*, same section, Theorem “The Chernoff method”) at t^* bounds its probability by

$$e^{-t^*(cM-c)} \Phi_M(t^*) = e^{t^*c} (e^{-t^*c} \Phi_1(t^*))^M = \theta \beta^M.$$

The union bound (*Math Guide*, §“The union bound and a birthday calculation”) over these events gives, for the fork point a ,

$$\beta^{L+1} + \theta \sum_{M \geq L+2} \beta^M \leq \beta^{L+1} + \frac{\theta \beta^{L+2}}{1-\beta} \leq \frac{(1+\theta) \beta^{L+1}}{1-\beta},$$

and the union over A contributes the factor $|A|$. For negligibility, β and θ are constants with $\beta < 1$, so for $L = \Theta(\lambda)$ the bound is a polynomial in λ times a function exponentially small in λ , which is negligible (*Math Guide*, §“Polynomial, exponential, and negligible functions”). $\square$

The source omits step (ii). The identification of the fork experiment with the clock model is the paper’s (proof of Corollary 1) and is not derived from Definition 4.1; the counting hypothesis on validly mined blocks is returned to in §5.2.

Remark 4.13 (Status of the (c, L) assumption). The paper conjectures that (c, L) can be derived from the backbone parameters $(\mu, \gamma, s, \tau, E, f)$ and leaves the derivation out of scope (“not trivial and out of the scope of this paper”, ePrint 2019/226, Section 3.2, p. 8). No such derivation exists: in the classes of Definition 1.3 it is an open problem. Proposition 4.12 covers only the constant-difficulty clock model and does not connect the variable-difficulty backbone to Assumption 4.8. Every result of this volume that uses the assumption is conditional on it.

4.4 Security and succinctness of chain proofs

Definition 4.14 (SPV predicate). Let $C = (B_0, \dots, B_\eta)$ be a chain with genesis block B_0 and head B_η . The *SPV predicate* of C is 1 if and only if every block $B_{\eta'}$ carries a valid proof of work at the target that Definition 4.5 assigns to its position, and for $1 \leq \eta' \leq \eta$ the hash of $B_{\eta'-1}$ is contained in the header of $B_{\eta'}$. The statement a prover proves at round r is that it knows a chain ending in B_η , with total work ω (Definition 4.2), whose SPV predicate is 1.

The definition is ePrint 2019/226, Definition 2 (Section 3.3, p. 9). The paper writes B_n and D for the head and the cumulative difficulty; here η is a height and ω the total work, n and D being reserved.

Definition 4.15 (Equivalence up to the last blocks). Two chains are *equivalent up to the last ℓ blocks*, with ℓ the persistence depth of Assumption 4.4, if removing the last ℓ blocks of each leaves one a prefix of the other. The verifier treats valid proofs of equivalent chains as proofs of one chain and takes the one of greatest total work as its representative; no predicate is evaluated on the last ℓ blocks.

The paper writes k for this depth (Section 3.3, p. 9); here ℓ, k being the catch parameter.

Definition 4.16 (Verifier setting). The verifier is a light client (Definition 2.1) whose only prior knowledge of the chain is the genesis block B_0 , and which has oracle access to H to check proofs of work. It receives a set $\mathcal{P}$ of proofs from several provers, at least one of which is honest, and accepts the SPV predicate of the proof whose head carries the greatest total work, up to Definition 4.15.

The honest-prover hypothesis places eclipse attacks, in which every prover is adversarial, outside the model (ePrint 2019/226, Section 3.3, p. 9).

Definition 4.17 (Security of a chain proof protocol). A *chain proof protocol* is a pair (P, V) of algorithms with oracle access to H : the prover P maps a chain to a proof, and the verifier V maps a set of proofs to an output (B, ω) , a head and a total work, or to rejection. The *security game*, for an environment, a PPT adversary $\mathcal{A}$ as in Definition 4.3 and a round r , is as follows.

1. The backbone execution of Definition 4.1 runs to round r , with $\mathcal{A}$ controlling the adversarial players.

2. At the start of round r the verifier receives a set $\mathcal{P}$ containing at least one proof that P computes on the chain of an honest full node; $\mathcal{A}$ chooses the other proofs.
3. The adversary wins if either of the following clauses fails.
 - (a) At the end of round r the verifier outputs the SPV predicate of greatest total work for some head B : a pair (B, ω) such that the SPV predicate holds for a chain ending in B of total work ω , and no proof in $\mathcal{P}$ of a chain whose SPV predicate holds carries more total work, up to Definition 4.15.
 - (b) Every honest full node at round r holds a chain of which the chain committed by B , with its last ℓ blocks removed, is a prefix.

The protocol is *secure* if for every environment, every PPT $\mathcal{A}$ and every round r the probability that $\mathcal{A}$ wins is negligible in λ .

The definition states both clauses of ePrint 2019/226, Definition 3 (Section 3.3, p. 9). The game idiom is that of the *Crypto Guide*, §“Security as a game”; the multi-prover game is defined here.

Definition 4.18 (Succinctness). A chain proof protocol (P, V) is *succinct* if for every PPT prover and every round r , every proof produced at round r has size $O(\text{polylog}(N))$ bits, where N is the number of blocks of the honest chain, $\text{polylog}(N) = (\log N)^{O(1)}$ as fixed in §1.2, and the asymptotics are in N with λ , c and L fixed.

The definition is ePrint 2019/226, Definition 4 (p. 9), after Kiayias, Miller and Zindros, “Non-interactive proofs of proof-of-work”, Definition 4.

Definition 4.19 (Non-interactive proof of proof of work). A *non-interactive proof of proof of work* is a chain proof protocol for the SPV predicate in which each prover sends a single message and no message passes from verifier to prover, and which is secure (Definition 4.17) and succinct (Definition 4.18).

The term names this class of protocols. Kiayias, Miller and Zindros use the same name for their superblock construction, a different member of the class.

5 The FlyClient protocol

This section constructs the protocol of ePrint 2019/226 in the model of §4, each object before its first use: the header chain commitment and the check applied to one sampled block, two lemmas on forks, the difficulty Merkle mountain range with its node checks, the sampling distribution, the interactive and the non-interactive protocol, the security theorem with its hypotheses and status, and verification after acceptance. Throughout, $H: \{0, 1\}^* \rightarrow \{0, 1\}^k$ is the hash function of §3, which is also the proof-of-work oracle of Definition 4.1; $\log$ is the natural logarithm and $\text{Adv}_H^{\text{col}}(\mathcal{B})$ the collision advantage of an algorithm $\mathcal{B}$ (*Crypto Guide*, Definition “Collision resistance”, §“Security notions: preimage, second-preimage, and collision resistance”), the security parameter left implicit.

5.1 Header chain commitments and per-sample checks

Blocks are written B_0 (the genesis block), $B_1, B_2, \dots$, with B_η the block at height η and $\text{hdr}(B)$ the header of B .

Construction 5.1 (Header chain commitment). Fix an aggregating range (Definition 3.19) under the bagging Bag_R (Definition 3.5) whose leaf map sends a block B to a node tuple $\text{leaf}(B)$ with hash field $\text{leaf}(B).h = H(\text{hdr}(B))$. Leaf j of every range is $\text{leaf}(B_{j-1})$, so leaf j is the block at height $j - 1$. For every height $\eta \geq 1$ let M_η be the range over the η leaves $\text{leaf}(B_0), \dots, \text{leaf}(B_{\eta-1})$ and rt_η its root tuple. The producer of B_η obtains M_η from $M_{\eta-1}$ by Construction 3.10 applied to $\text{leaf}(B_{\eta-1})$, with the parent map par in place of $H(\cdot \parallel \cdot)$ (M_1 is the single leaf $\text{leaf}(B_0)$), and records the hash field $\text{rt}_\eta.h$ in the header of B_η . The genesis header records no root. A full node accepts B_η only if its header records $\text{rt}_\eta.h$.

The construction has the following invariant: the header of B_η commits, through one root, to exactly the blocks strictly before it, and a head B_η commits to $n = \eta$ leaves, n denoting this leaf count below. The root's fields other than its hash field are not recorded; a verifier recomputes the root tuple from its two children, which the hash field binds (Lemma 3.20). The range of ePrint 2019/226, Section 2, is the instance whose tuples carry the hash field alone; the paper's difficulty Merkle mountain range is the instance of Definition 5.6. The one further header field is the paper's only consensus change (Section 4.2). The paper indexes the committing block and its range inconsistently; the invariant fixes one indexing, and the paper's is not carried. In the classes of Definition 1.3, this commitment from the genesis block is the paper's design and no Zcash rule specifies it; the specified commitment is the epoch-scoped instance of "The chain-history commitment" (§6), from which nothing is concluded here.

One sample at index $j \in \{1, \dots, n\}$ against a head B_η must establish three facts: that the sampled block is leaf j under the head's root, that the leaf's tuple is the one its header determines, and that the root in the sampled header commits to exactly the first $j - 1$ of the same leaves.

Construction 5.2 (Per-sample check). The inputs are a root tuple rt with leaf count n and an index $1 \leq j \leq n$. The prover sends the header hdr_j of the block at height $j - 1$, the tuple v_j of leaf j , and its inclusion path π in M_n (Definition 3.13) with every sibling as a full tuple. The verifier performs the following checks.

- (a) *Leaf*: $v_j.h = H(\text{hdr}_j)$, and every other field of v_j that the leaf map determines from the header alone equals its value computed from hdr_j .
- (b) *Inclusion*: $\text{Verify}_R(\text{rt}, n, j, v_j, \pi)$ of Definition 3.13, in the tuple form of Definition 3.19, accepts; the side of each sibling and the path length $\text{depth}_R(n, j)$ are fixed by (j, n) .
- (c) *Prefix*: for $j \geq 2$, the hash field of $\text{PrefixRoot}_R(n, j, \pi)$ (Construction 3.16), run with par in place of $H(\cdot \parallel \cdot)$, equals the root recorded in hdr_j ; for $j = 1$, hdr_1 equals the genesis header, which the verifier knows.
- (d) *Proof of work*: hdr_j carries a valid proof of work at the target T it declares, $H(\text{hdr}_j) < T$ (Definition 4.1).

Any failure rejects.

ePrint 2019/226, Algorithm 2, lists checks (b) and (c); checks (a) and (d) come from its Definition 5 and Section 6.1. Check (b) uses the length rule of Definition 3.13, not that of the paper's Algorithm 4.

Proposition 5.3 (Soundness of the per-sample check). *Let rt be any root tuple, possibly chosen by the adversary, with leaf count n , and let $1 \leq j \leq n$. There is an explicit reduction $\mathcal{B}$ such that, for every adversary $\mathcal{A}$ that outputs samples at position j against (rt, n) , the probability that every sample passes Construction 5.2 while one of the following fails is at most $\text{Adv}_H^{\text{col}}(\mathcal{B})$:*

- (i) *any two passing samples at position j carry the same leaf tuple and the same header, so the header is bound to position j of the sequence that rt commits;*
- (ii) *every header-determined field of that leaf tuple is its value computed from that header;*
- (iii) *for $j \geq 2$, the root recorded in that header is the hash field of a prefix-root tuple that is the same for every passing sample at j , hence a function of (rt, n, j) ; if rt is the root of a consistent labelling over $v_1, \dots, v_n$, it is the root of the consistent labelling over $v_1, \dots, v_{j-1}$.*

Proof. The reduction $\mathcal{B}$ runs $\mathcal{A}$ and inspects two passing samples (hdr, v, π) and (hdr', v', π') at position j . (i) If $(v, \pi) \neq (v', \pi')$, Lemma 3.20(2), the tuple form of Theorem 3.17 under the prefix-free serialisation of Definition 3.19, yields a collision at the highest step whose inputs differ. If $v = v'$ but $\text{hdr} \neq \text{hdr}'$, check (a) gives $H(\text{hdr}) = v.h = H(\text{hdr}')$, a collision. (ii) is check (a). (iii) Barring the collisions of (i), $\pi = \pi'$, so the fold of check (c) over the left siblings of π is the same tuple for every passing sample, and check (c) equates its hash field with the recorded root. For a consistent labelling, Lemma 3.15 with Lemma 3.20(3) identifies the fold with the root over $v_1, \dots, v_{j-1}$, as Theorem 3.14 does for plain values. $\square$

5.2 Forks against a committed chain

A claimed chain C *forks* from the honest chain at its fork point, the last common block (§4.3). Its head records the root of a range over the leaf tuples $v'_1, \dots, v'_n$ of its blocks, and C *places* a block B at leaf position j when $v'_j.h = H(\text{hdr}(B))$.

Lemma 5.4 (Honest blocks after the fork point). *Let the head of C record the hash field of the root of a consistent labelling over $v'_1, \dots, v'_n$. Let B be a block of the honest chain at height η above the fork point, whose header records the hash field of the honest root over the honest leaf tuples $v_1, \dots, v_\eta$, and let C place B at leaf position $j \geq 2$. Suppose that*

1. *the sequences $(v'_1.h, \dots, v'_{j-1}.h)$ and $(v_1.h, \dots, v_\eta.h)$ differ, possibly in length; and*
2. *no block header equals $\text{ser}(\ell) \parallel \text{ser}(r)$ for node tuples ℓ, r (leaf-internal separation).*

Then from a sample of B that passes Construction 5.2 against the head of C an explicit reduction computes a collision of H , or finds a leaf position $i < j$ of C whose tuple v'_i has hash field $H(\text{ser}(\ell) \parallel \text{ser}(r))$ for node tuples ℓ, r . A sample at i passes check (a) only on a header hashing to that value, which by hypothesis 2 is a collision. Consequently, barring a collision, either a sample

on B rejects or leaf i does, and that block or leaf belongs, in the analysis of §5.4, to the invalid set of the fork.

Proof. By Proposition 5.3(iii) a passing sample makes the root recorded in $\text{hdr}(B)$ equal to the hash field of the root of the consistent labelling of M_{j-1} over $v'_1, \dots, v'_{j-1}$. That value is also the hash field of the honest root of M_j . Descend both labellings from their roots, keeping a pair of nodes with equal hash fields whose leaf subsequences differ in their hash fields. At a pair of internal nodes, equal hash fields $H(\text{ser}(\ell) \parallel \text{ser}(r)) = H(\text{ser}(\ell') \parallel \text{ser}(r'))$ with $(\ell, r) \neq (\ell', r')$ are a collision by unique decoding (proof of Lemma 3.20); otherwise the children agree as tuples, the leaf subsequences of the left children or of the right children differ, and the descent continues there. Two leaves with equal hash fields do not have different hash-field sequences, so the descent ends at a leaf opposite an internal node. If the leaf is honest, its hash field is $H(\text{hdr})$ for the header hdr of an honest block, and $H(\text{hdr}) = H(\text{ser}(\ell) \parallel \text{ser}(r))$ with distinct inputs by hypothesis 2 is a collision. If the leaf is a leaf v'_i of C , $i \leq j - 1$, opposite an honest internal node with children ℓ, r , then $v'_i.h = H(\text{ser}(\ell) \parallel \text{ser}(r))$, which is the second outcome. The descent takes at most $\max(\lceil \log_2(j-1) \rceil, \lceil \log_2 \eta \rceil)$ steps. $\square$

The lemma is ePrint 2019/226, Section 5.1 with Corollary 4 (here Corollary 3.18), extended to a block placed at a height other than its own, where the two roots have different leaf counts and hypothesis 2 is needed. It concerns honest blocks placed after the fork point and states nothing about blocks the adversary mines; those are the subject of the next lemma.

Lemma 5.5 (No reuse of work across the fork point). *Model H as a random oracle with κ -bit outputs (Crypto Guide, §“The random oracle model”), and let at most q_H queries to H be made in the experiment, by $\mathcal{A}$, by the verification of its output, and by the computation of the prefix root at every position of the output fork after its fork point. Call a query $H(x)$ reused if x is the header of a block B of the output fork after its fork point, x records the hash field of the prefix root of the fork at B ’s position, and the query precedes the first query whose answer is that recorded value. Then*

$$\Pr[\mathcal{A} \text{ outputs a fork with a block mined by a reused query}] \leq q_H^2 2^{-\kappa} + \text{Adv}_H^{\text{col}}(\mathcal{B})$$

for an explicit reduction $\mathcal{B}$. Consequently, except with that probability, only adversarial work done after the fork’s prefix is fixed counts against the honest chain.

Proof. By Construction 5.1 the root recorded in a header after the fork point is the hash field of a tuple over the fork’s prefix; by Proposition 5.3(iii) that tuple is a function of the fork’s root, position and leaf count, barring a collision, which accounts for the term $\text{Adv}_H^{\text{col}}(\mathcal{B})$. Its hash field is the answer of a query, made at the latest during that computation. A reused query fixes a recorded value r before any query has answer r . The first query with answer r has an input not queried before, since a repeated input returns an earlier answer; its answer is uniform on $\{0, 1\}^\kappa$ and independent of all earlier queries, so it equals one of the at most q_H values recorded in earlier query inputs with probability at most $q_H 2^{-\kappa}$. The union bound over the at most q_H queries (Math Guide, §“The union bound and a birthday calculation”) gives $q_H^2 2^{-\kappa}$. $\square$

The paper asserts the statement in one sentence (ePrint 2019/226, Section 3.2); the proof above is this volume's. The bound is in the number of oracle queries, not in the mining rate.

5.3 The difficulty Merkle mountain range

A sampling verifier sees few difficulty transitions, so the transitions it does not see must be constrained by committed aggregates.

Definition 5.6 (Difficulty Merkle mountain range). The *difficulty Merkle mountain range* (*difficulty MMR*) is the instance of Definition 3.19 whose node v is the tuple

$$v = (v.h, v.w, v.t, v.D_{\text{start}}, v.D_{\text{next}}, v.n),$$

with a serialisation that concatenates injective fixed-width encodings of the six fields, hence prefix-free. For a block B whose header declares target T , the leaf map sets

- $v.h = H(\text{hdr}(B))$;
- $v.w = v.D_{\text{start}} = D(T)$, the difficulty level of T (Definition 4.2);
- $v.t$ the number of rounds between the round recorded by the predecessor of B and the round recorded by B (0 for the genesis block);
- $v.D_{\text{next}}$ the level of the target that Definition 4.5 assigns to the block after B ;
- $v.n = 1$.

The parent of ℓ and r has hash field $H(\text{ser}(\ell) \parallel \text{ser}(r))$, over both children's complete tuples; its fields w , t and n are the sums of the children's; it inherits D_{start} from ℓ and D_{next} from r .

The fields h , w , D_{start} and n of a leaf are determined by its header alone and are checked by item (a) of Construction 5.2; the fields t and D_{next} depend on neighbouring blocks and are constrained only by Construction 5.8. The definition makes three precisions against ePrint 2019/226, Definition 7. The paper writes $H(lc, rc)$ without fixing its input; hashing the children's hash fields alone would leave their other fields unbound. The paper's "timestamp" t is an elapsed time, and its "targets" w , D_{start} and D_{next} are difficulty levels. The paper's checks name fields t_{start} , t_{end} and D_{end} that its Definition 7 never declares; the declared aggregate t suffices for every timing check below, so the field set is the declared one. The paper's data field is omitted, since no check uses it. No encoding of the fields is fixed by the paper.

Lemma 5.7 (Weight binding for the difficulty MMR). *Let a sample at leaf j pass Construction 5.2 against a root rt with leaf count n , the shape being fixed by n under Bag_R . Except with probability $\text{Adv}_H^{\text{col}}(\mathcal{B})$ for an explicit reduction $\mathcal{B}$:*

1. *the leaf weight $v_j.w$ is the difficulty level $D(T)$ of the target T at which the header's proof of work is valid;*

2. the interval $[W, W + v_j.w]$, with W the sum of the fields w of the left siblings on the path, is a function of (rt, n, j) .

Proof. (1) Check (a) sets $v_j.w = D(T)$ for the declared target T , check (d) validates the proof of work at that T , and Proposition 5.3(i) and (ii) bind the header and its fields to position j . (2) Lemma 3.21(1). Whether W equals the true prefix weight is part (2) of that lemma, which holds for a consistent labelling only and is not claimed here. $\square$

Construction 5.8 (Node checks). Assume, with ePrint 2019/226, Section 6.1, that the retarget-epoch length E of Definition 4.5 and the leaf count n are powers of two, so that the retarget epochs are the perfect subtrees of altitude $\log_2 E$. For every parent p on a sample's path, with children ℓ and r (one of them a carried sibling), the verifier checks:

1. $p = \text{par}(\ell, r)$ under Definition 5.6;
2. $\ell.D_{\text{next}} = r.D_{\text{start}}$;
3. for each of ℓ and r , internal consistency:
 - (a) a node v of altitude below $\log_2 E$ lies inside one retarget epoch and has $v.w = v.n \cdot v.D_{\text{start}}$, and $v.D_{\text{next}} = v.D_{\text{start}}$ unless v contains the last block of its epoch;
 - (b) a node v of altitude $\log_2 E$ is one retarget epoch, has $v.w = E \cdot v.D_{\text{start}}$, and $v.D_{\text{next}} = D(\text{Retarget}(2^x / v.D_{\text{start}}, v.t))$, the level that Definition 4.5 assigns after an epoch of $v.t$ rounds at level $v.D_{\text{start}}$;
 - (c) a node v above altitude $\log_2 E$ is *feasible*: its $(v.t, v.w, v.D_{\text{start}}, v.D_{\text{next}}, v.n)$ is attained by some sequence of clamped retargets (Propositions 5.9 and 5.10).

At the sampled leaf the checks (a) and (d) of Construction 5.2 apply; their effect on w and D_{start} is Lemma 5.7. *Weight-position check*: when a leaf is requested at a point x of cumulative weight, $0 \leq x < rt.w$, the returned leaf j satisfies $W \leq x < W + v_j.w$, with W the sum of the fields w of the left siblings on its path; this interval is bound to (rt, n, j) by Lemma 3.21(1). Any failure rejects.

The construction is ePrint 2019/226, Section 6.1 (“Difficulty MMR Verification”); the paper states the weight-position check only informally (Section 2).

Proposition 5.9 (Feasibility envelope). *Let consecutive retarget epochs $0, \dots, z-1$ have levels $D_0 = D_{\text{start}}, D_1, \dots, D_{z-1}$, and let $D_z = D_{\text{next}} = \tau^y D_{\text{start}}$, where every retarget obeys the clamp of Definition 4.5, so that $D_{e+1}/D_e \in [1/\tau, \tau]$. Then:*

1. $|y| \leq z$ and, for every e ,

$$D_{\text{start}} \max(\tau^{-e}, \tau^{y-z+e}) \leq D_e \leq D_{\text{start}} \min(\tau^e, \tau^{y+z-e});$$

2. for integer y with $z-y$ even both envelopes are attained by clamped sequences, so the total weight $E \sum_{e < z} D_e$ is maximal when the difficulty is raised by τ per epoch and then lowered by τ per epoch to D_{next} , and minimal when it is lowered first and then raised;

3. an epoch at target T that lowers the difficulty maximally (new target τT) lasts $\Delta \geq \tau E / f$ rounds, and one that raises it maximally (new target T / τ) lasts $\Delta \leq E / (f\tau)$ rounds.

Proof. (1) Since $|\log_\tau D_{e+1} - \log_\tau D_e| \leq 1$, induction on e from epoch 0 gives $|\log_\tau (D_e / D_{\text{start}})| \leq e$, and from epoch z gives $|\log_\tau (D_e / D_{\text{start}}) - y| \leq z - e$; at $e = 0$ the second bound is $|y| \leq z$. (2) For $z - y$ even the pointwise upper envelope has exponent steps $+1$ up to $e = (y + z)/2$ and -1 after, and the lower envelope steps -1 up to $e = (z - y)/2$ and $+1$ after; both are clamped, and the weight is increasing in each D_e . (3) The raw retarget is $T\Delta f / E$ (§4.2); the clamp is reached from above when $T\Delta f / E \geq \tau T$ and from below when $T\Delta f / E \leq T / \tau$. A script checks the sums against brute force over exponent sequences, and the two thresholds. $\square$

Proposition 5.10 (Feasibility bounds). *In the setting of Proposition 5.9, let y be an integer with $y \geq 0$ and $z - y$ even. If a node spanning the z epochs is attained by a clamped sequence, then*

1. $y \leq z$ (Proposition 5.9(1));
2. $w \leq E D_{\text{start}} \frac{(\tau + 1) \tau^{(y+z)/2} - \tau^{1+y} - 1}{\tau - 1}$;
3. $w \geq E D_{\text{start}} \frac{\tau^y + \tau - (\tau + 1) \tau^{-(z-y)/2}}{\tau - 1}$;
4. if $y = z$, then $t \leq zE / (f\tau)$.

Proof. Bounds (2) and (3) are the geometric sums of the envelopes of Proposition 5.9(2); at $y = z$ both reduce to $(\tau^z - 1) / (\tau - 1)$. For (4), $y = z$ forces a maximal raise in each of the z epochs, each of at most $E / (f\tau)$ rounds by Proposition 5.9(3). A script confirms that the closed forms equal the brute-force extremes in all 123 cases with $\tau \in \{2, 4, 3/2\}$, $1 \leq z \leq 11$, $0 \leq y \leq z$ and $z - y$ even; at $\tau = 4$, $y = 0$, $z = 2$ they give 5 and $5/4$ per unit of E . $\square$

The paper states these checks with k and n in place of y and z , which here are the catch parameter and the leaf count (ePrint 2019/226, Section 6.1, pp. 18–19). Three departures are made. The printed lower bound has exponent $(y + z)/2$ in place of $-(z - y)/2$ and is then negative, -5 against the corrected $5/4$ at $\tau = 4$, $y = 0$, $z = 2$. The printed bounds count one level per epoch, whereas the leaf rule $w = D_{\text{start}}$ sums per block, so both scale by E . The printed upper summation overcounts by τ^z at $y = z$, where its closed form is right. The paper further requires that t cover the epochs in which the difficulty is lowered, without a formula, and calls the cases of non-integer and negative y “similar” without stating them. The conditions above are necessary, not sufficient: the general feasibility predicate that item 3(c) of Construction 5.8 requires is not given by the paper.

Assumption 5.11 (Transition rewriting). In the model of Definition 4.1, for every adversary $\mathcal{A}$ that with non-negligible probability p outputs a chain C accepted by the node checks on every sample, some of whose difficulty transitions violate Definition 4.5, there is an adversary $\mathcal{A}'$ that makes the same number of oracle queries, respects the retarget rule, and with probability at

least $p - \text{negl}(\kappa)$ outputs a chain C' whose validly mined blocks occupy the same positions as those of C .

The assumption is the statement of ePrint 2019/226, Lemma 4, taken as a hypothesis because its proof is a sketch. It handles subtrees whose aggregates are inconsistent with their children, which Lemma 3.21(2) does not cover. The paper’s proof strategy is as follows. Let x' be the highest node on a claimed path at which the node checks fail. Every path through x' , or carrying it as a sibling, rejects, so barring a collision every leaf of the subtree M' under the parent of x' is unsampleable without rejection. The adversary $\mathcal{A}'$ replaces the transitions inside M' by clamped ones consistent with the aggregates $(w, t, D_{\text{start}}, D_{\text{next}}, n)$ of the root of M' , which is possible because that node passes the feasibility check and the leaves of M' , being invalid, need no proof of work; this is repeated until no invalid transition remains. The paper writes “consistency with x_0 ” where the parent of x' is meant.

Remark 5.12 (Gaps in the rewriting argument). (1) The feasibility step presumes a complete feasibility predicate, which the paper gives only in the simplified setting of Proposition 5.10. (2) Rewriting the leaves of M' changes every later root and hence every later header, so $\mathcal{A}'$ must fix the rewrite before it mines the later valid blocks, and must mine them anew; the paper asserts without a simulation argument that this succeeds with probability at least $p - \text{negl}(\kappa)$. (3) “The same valid blocks” holds for positions, not for header contents. Assumption 5.11 is therefore the paper’s claim, not a result of this volume.

5.4 The sampling distribution

Uniform sampling fails on forks near the tip: almost every uniform sample lands before a late fork point, where the two chains agree. If the fork point were known, each sample after it would be invalid with probability at least $1 - c$ under Assumption 4.8.

Construction 5.13 (Dyadic-suffix sampling). At constant difficulty, over a claimed chain of n blocks and with u samples per interval, the verifier draws, for every integer i with $0 \leq i < \log_2 n$, u samples uniformly and independently from the last $n/2^i$ blocks, and checks every one of the last $\min(L, u)$ blocks.

The samples do not depend on the prover’s answers, so the protocol is one-shot and public-coin (*Crypto Guide*, §“Interactive protocols, proofs, and arguments”). The construction is ePrint 2019/226, Section 5.3, with u in place of the paper’s k .

Lemma 5.14 (Dyadic miss probability). *Under Assumption 4.8, at constant difficulty, for every fork point a and every fork from a of more than L blocks whose total work after a is at least that of the honest chain, the probability that the $u \log_2 n$ samples of Construction 5.13 include no invalid block of the fork is at most $((1 + c)/2)^u$, except with probability negligible in λ .*

Proof. Let the fork point be a and choose i with $(2^i - 1)n/2^i \leq a < (2^{i+1} - 1)n/2^{i+1}$. The interval of the last $n' = n/2^i$ blocks then contains a in its first half, so the fork length $l = n - a$ satisfies $l > n'/2$, and by Assumption 4.8, at constant difficulty, the interval holds at least $(1 - c)l > (1 - c)n'/2$ invalid blocks. Each of its u independent uniform samples misses them with

probability at most $1 - (1 - c)/2 = (1 + c)/2$, and the samples of the other intervals are discarded. A script checks the bound for $c \in \{0.25, 0.5, 0.9\}$, $u \in \{1, 5, 20\}$ and intervals of 8, 64 and 1,024 blocks. $\square$

The lemma is ePrint 2019/226, Lemma 2. A fork of at most $\min(L, u)$ blocks lies in the checked suffix. When $u < L$, a fork of l blocks with $u < l \leq L$ is covered by neither clause, since Assumption 4.8 does not constrain it; the construction is sound for $u \geq L$, or with the last L blocks checked in full.

Definition 5.15 (Continuum model and catch probability). The claimed chain is the interval $[0, 1]$ in relative cumulative work: a block whose cumulative-work interval is $[W, W + w)$ within total committed work ω occupies $[W/\omega, (W + w)/\omega)$; genesis is at 0 and the head at 1. A *fork point* is a point $a \in [0, 1]$. By Assumption 4.8 in its operative form, the adversary designates an *invalid set* $S \subseteq [a, 1]$ of Lebesgue measure at least $(1 - c)(1 - a)$; every such S is *admissible*. A *sampling density* is a probability density φ on $[0, 1]$ (*Math Guide*, §“Beyond finite spaces: countable additivity, limits, and densities”). When a suffix $[1 - \delta, 1]$ is checked in full, a set S that meets it in positive measure is caught with probability 1, and any other S is caught by one draw with probability $\int_S \varphi$. The *single-draw catch probability* at a is the infimum $p_\varphi(a)$ of these probabilities over admissible S . For Q independent draws (*Math Guide*, §“Conditional probability and independence”) against a set S fixed before the draws, all miss with probability $(1 - \int_S \varphi)^Q \leq (1 - p_\varphi(a))^Q$. The *design problem* is to choose φ to maximise $\min_a p_\varphi(a)$.

Lemma 5.16 (Best response). Let φ be non-decreasing on an interval $[a, b]$ with $b \leq 1$, and let $0 \leq m \leq b - a$. Among measurable $S \subseteq [a, b]$ of measure m , the integral $\int_S \varphi$ is minimal for $S = [a, a + m]$. Hence against a density non-decreasing on $[a, 1 - \delta]$, with $[1 - \delta, 1]$ checked in full, an adversary forking at $a \leq 1 - \delta/c$ places its valid weight last, and its invalid set is $[a, 1 - (1 - a)c]$.

Proof. With $I = [a, a + m]$, $\int_S \varphi - \int_I \varphi = \int_{S \setminus I} \varphi - \int_{I \setminus S} \varphi$; the two sets have equal measure, $S \setminus I$ lies right of $a + m$ and $I \setminus S$ left of it, and φ is non-decreasing, so the difference is non-negative. A set meeting the suffix in positive measure is caught with certainty, so the adversary’s best set lies in $[a, 1 - \delta]$ and has the least admissible measure $(1 - c)(1 - a)$; it ends at $a + (1 - c)(1 - a) = 1 - (1 - a)c \leq 1 - \delta$. $\square$

Construction 5.17 (Indifference density). Equal catch probability at every fork point, that is $\Phi(1 - c(1 - a)) - \Phi(a)$ constant in a for an antiderivative Φ of φ , gives on differentiation

$$\varphi(a) = c\varphi(1 - c(1 - a)).$$

The function $\varphi(\xi) = K/(1 - \xi)$ solves it. With $K = (1 - c)/c$ every invalid segment $[a, 1 - c(1 - a)]$ has mass $(c - 1) \log(c)/c$, which is $\log 2 \approx 0.693147$ at $c = 1/2$. The equation fixes φ only up to a factor periodic in $\log(1 - \xi)$ with period $\log(1/c)$. The function is not integrable on $[0, 1]$, its mass on $[0, 1 - \epsilon]$ being $K \log(1/\epsilon)$, so it is not a density there.

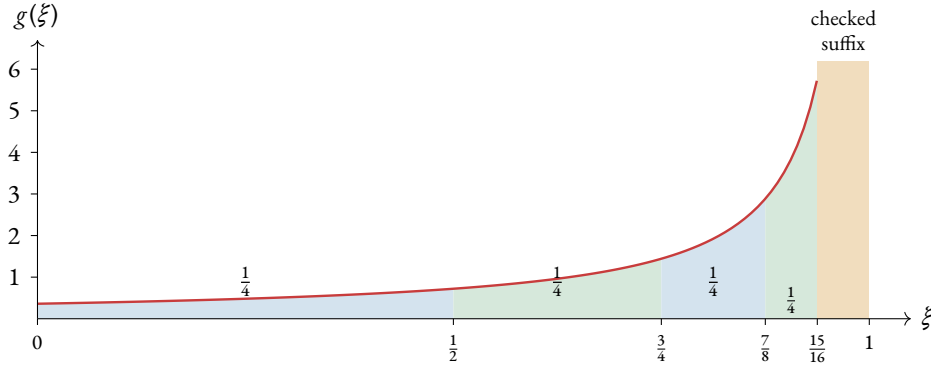


Figure 3. The FlyClient sampling density g of Construction 5.18 for $c = 1/2$ and $k = 4$, so $\delta = 1/16$ and $g(\xi) = 1/((1 - \xi) \log 16)$ on $[0, 15/16]$. The amber strip is the suffix $[15/16, 1]$, checked in full, on which g vanishes. The four shaded segments $[1 - c^i, 1 - c^{i+1}]$, $i = 0, \dots, 3$, are the invalid sets of an adversary forking at $1 - c^i$ with its valid weight last; each has mass $1/4$ under g , and together they tile the support, which is the proof of Theorem 5.21(1).

The segment mass is $K(\log(1 - a) - \log(c(1 - a))) = -K \log c$; a script checks it for $c \in \{0.3, 0.5, 0.9\}$ and $a \in \{0, 0.2, 0.7\}$, and the divergence. The construction is ePrint 2019/226, Section 5.4, p. 16.

Construction 5.18 (FlyClient sampling density). Fix $0 < c < 1$ and $\delta \in (0, c]$; the suffix $[1 - \delta, 1]$ is checked in full. The *FlyClient sampling density* is

$$g(\xi) = \begin{cases} \frac{1}{(\xi - 1) \log \delta} & 0 \leq \xi \leq 1 - \delta, \\ 0 & \text{otherwise,} \end{cases} \quad G(\xi) = \frac{\log(1 - \xi)}{\log \delta} \quad (0 \leq \xi \leq 1 - \delta).$$

Both factors of the denominator are negative, so $g > 0$ on its support, and g is increasing there; G is its distribution function, with $G(1 - \delta) = 1$. For $a \leq (c - \delta)/c$ the invalid segment $[a, 1 - (1 - a)c]$ lies in $[0, 1 - \delta]$ and has catch probability

$$G(1 - (1 - a)c) - G(a) = \frac{\log c}{\log \delta},$$

which is $1/k$ at $\delta = c^k$. For $a > (c - \delta)/c$ every admissible invalid set meets the checked suffix in positive measure, and the fork is caught with probability 1.

Normalisation, G and the catch probability are direct integrations. For the last clause, $[a, 1 - \delta]$ has measure $1 - \delta - a < (1 - c)(1 - a)$ exactly when $a > 1 - \delta/c = (c - \delta)/c$. A script checks normalisation, G and the catch probability $1/k$ for $c \in \{0.25, 0.3, 0.5, 0.75, 0.9\}$ and $2 \leq k \leq 9$. The density is ePrint 2019/226, Section 5.4, p. 16; Figure 3 shows it.

Definition 5.19 (Sampling map). Let the head's root tuple rt carry committed work rt.w , with leaf count n . The *sampling map* sends a uniform $U \in [0, 1]$ to the point $\xi = 1 - \delta^U$, the inverse of G at U , so that ξ has density g ; it sends ξ to the point $x = \xi \cdot \text{rt.w}$ of cumulative work, and

x to the leaf j whose interval $[W, W + v_j.w)$ contains x . That interval is bound to (rt, n, j) by Lemma 3.21(1) and is checked by the weight-position check of Construction 5.8. The *checked suffix* is the set of blocks whose interval meets $[(1 - \delta) rt.w, rt.w)$, together with the last L blocks and the head.

Assumption 5.20 (Discretisation). For every chain C output by an adversary respecting Definition 4.5, and for the fork point a of C against the honest chain, if C has more than L blocks after a and total work after a at least that of the honest chain, then one draw of Definition 5.19 returns an invalid block, or a block on which the per-sample check fails, with probability at least $p_g(a)$ of Definition 5.15 minus a function negligible in κ ; the Q draws are independent.

The paper asserts that the continuum analysis “still produces a good distribution for the discrete case” (ePrint 2019/226, Section 5.4) and gives no proof; the hypothesis is named here.

Theorem 5.21 (Optimal sampling distribution). *Let $0 < c < 1$, let $k \geq 1$ be real and $\delta = c^k$. Suppose that the suffix $[1 - \delta, 1]$ is checked in full and that the adversary’s valid weight after any fork point a is at most a c fraction. Then:*

1. *if k is an integer, for every sampling density φ , $\min_a p_\varphi(a) \leq 1/k$;*
2. *the density g of Construction 5.18 attains $\min_a p_g(a) = 1/k$, against every placement of the invalid weight.*

Proof. (1) Tiling. At the fork points $a_i = 1 - c^i$, $i = 0, \dots, k - 1$, the placement with valid weight last is available to the adversary against every φ , and leaves the invalid segment $[1 - c^i, 1 - c^{i+1}]$, of measure $c^i(1 - c) = (1 - c)(1 - a_i)$, inside $[0, 1 - \delta]$. These k segments tile $[0, 1 - c^k] = [0, 1 - \delta]$, so $1 \geq \sum_i \int_{[1 - c^i, 1 - c^{i+1}]} \varphi \geq k \min_i p_\varphi(a_i)$. No monotonicity of φ is used. (2) The density g is increasing on $[0, 1 - \delta]$, so for $a \leq (c - \delta)/c$ Lemma 5.16 makes $[a, 1 - (1 - a)c]$ the adversary’s best set, whose catch probability is $1/k$ by Construction 5.18; later fork points meet the suffix and are caught with probability 1. A script checks the k tiling masses. $\square$

The theorem is ePrint 2019/226, Theorem 2, pp. 16–17. The paper prints $c \in (0, 1]$, where $\log \delta = 0$ at $c = 1$ and g is undefined; the domain here is $0 < c < 1$. The paper’s proof sums over $k + 1$ segments, $i = 0, \dots, k$, the last of which lies in the checked suffix; the range is $i = 0, \dots, k - 1$. The paper’s Lemma 3, a monotonicity lemma, is not needed.

Proposition 5.22 (Sample count). *Let $k > 1$. Then Q independent draws from g all miss an optimally placed invalid set with probability at most $(1 - 1/k)^Q$, and*

$$(1 - 1/k)^Q \leq 2^{-\sigma} \iff Q \geq \frac{\sigma}{\log_2(1/(1 - 1/k))}.$$

At constant difficulty with the last $L = c^k n$ blocks checked, $k = \log_{1/c}(n/L)$ and the least such Q is $\Theta(\sigma k)$, which is $O(\sigma \log_{1/c} n)$; for constant L it satisfies

$$\lim_{n \rightarrow \infty} \frac{Q}{\sigma \log_c(1/2) \log n} = 1.$$

| Here σ is the statistical exponent, not the security parameter.

Proof. The miss bound is Definition 5.15 with Theorem 5.21(2); the equivalence follows on taking logarithms. Since $\log_2(1/(1 - 1/k)) = 1/(k \log 2) + O(1/k^2)$, the least Q satisfies $Q \sim \sigma k \log 2 = \sigma \log_c(1/2) \log(n/L)$, and $\log(n/L)/\log n \rightarrow 1$ for constant L . A script cross-checks the least Q against exact rational arithmetic for $2 \leq k \leq 9$ at $\sigma = 50$. $\square$

The proposition is ePrint 2019/226, Section 5.4 (“Optimizing the Proof Size”), p. 17.

Remark 5.23 (Domain of the catch parameter). At $k = 1$, $\delta = c$: one draw suffices in the continuum model, and $\log_2(1/(1 - 1/k))$ is undefined, so Proposition 5.22 is stated for $k > 1$. For $k < 1$, $\delta > c$ and the checked suffix catches every adversary; the paper assumes $k \geq 1$ without loss of generality. For non-integer $k > 1$, Construction 5.18 still gives catch probability $\log c / \log \delta = 1/k$ and hence the sample bound. The tiling of part (1) of Theorem 5.21 needs integral k .

Remark 5.24 (Length of the checked suffix). Under variable difficulty the suffix checked in full is the weight fraction δ , and a sound instantiation makes it contain at least the last L blocks, as Definition 5.19 does (Remark 4.10). The length L is bounded below by the L of the (c, L) assumption in force and trades against Q : a larger L means a larger δ , a smaller k and fewer samples. The asymptotic size bound holds for constant L ; within it, L may be chosen numerically. Which (c, L) holds for Zcash is classified in “The deployment boundary” (§7).

Example 5.25 (Sample counts). At $c = 1/2$ and $\sigma = 50$, with $k = \log_2(n/L)$, the least Q with $(1 - 1/k)^Q \leq 2^{-50}$ is as follows.

n	L	k	bound	Q
2^{20}	100	13.36	445.3	446
2^{22}	100	15.36	514.7	515
2^{22}	1,000	12.03	399.5	400
2^{24}	100	17.36	584.0	585
2^{26}	100	19.36	653.4	654

Every k here is non-integer, so Remark 5.23 applies: the counts are sufficient, and their optimality is not claimed.

5.5 The interactive protocol

Construction 5.26 (Interactive FlyClient). The parties are a verifier $\mathcal{V}$ in the setting of Definition 4.16, which knows the genesis block B_0 , and provers $\mathcal{P}_1, \dots, \mathcal{P}_m$, at least one of them honest. The parameters are c, L , a real $k > 1$, $\delta = c^k$ and Q . The ranges are difficulty MMRs (Definition 5.6) committed by Construction 5.1.

1. Each prover sends its head B_γ , whose header records $\text{rt}_\gamma.\text{h}$ over $B_0, \dots, B_{\gamma-1}$, together with the two children of the root tuple (for $n = 1$, the single leaf tuple). The verifier recomputes the root tuple rt with par , rejects unless rt.h equals the recorded value, and reads $n = \text{rt.n}$ and the claimed total work $\omega = \text{rt.w} + D(T_{\text{head}})$, where T_{head} is the target the head declares.

2. If all heads are identical, $\mathcal{V}$ accepts that head.
3. Otherwise $\mathcal{V}$ orders the claims by claimed total work, heaviest first, and for each prover draws Q independent uniforms, maps each to a point of cumulative work by Definition 5.19, and requests the leaf at each point.
4. For each requested point the prover returns the header, the leaf tuple and its inclusion path with full node tuples. The sample must pass Construction 5.2, whose checks (a) and (d) require a proof of work valid at a target that the difficulty-MMR fields declare, and Construction 5.8 on every node of its path, including the weight-position check that the drawn point lies in the returned leaf's interval.
5. The prover sends the checked suffix of Definition 5.19, and $\mathcal{V}$ checks it in full: every block passes the per-sample and node checks, and consecutive blocks, the head included, satisfy the linkage and target conditions of Definition 4.14, the target of each block being the one that the field D_{next} of its predecessor's leaf declares.
6. Any failure rejects that prover. Among the surviving claims $\mathcal{V}$ accepts the heaviest, outputting its head and claimed total work; claims equivalent under Definition 4.15 count as one claim.

The construction is ePrint 2019/226, Algorithms 1 and 2 with Definition 5 and Sections 5.3, 5.4 and 6.1. Step 5 belongs to the protocol by Sections 5.3 and 5.4 of the paper, although its Algorithm 1 omits it. For claims of different weight the paper delegates the comparison to the generic verifier of Kiayias, Miller and Zindros (“Non-interactive proofs of proof-of-work”), which checks the longest claim first; under variable difficulty the order is by claimed total work, as in step 3.

5.6 The non-interactive protocol

Construction 5.27 (Non-interactive FlyClient). Construction 5.26 is public-coin and one-round: the only message of $\mathcal{V}$ is its sampling coins, which do not depend on the provers' answers. The non-interactive protocol applies the Fiat–Shamir transform (*Crypto Guide*, §“The Fiat–Shamir transform: from interactive to non-interactive”), the challenge hashing the instance. Let $\mathcal{O}$ be a random oracle (*Crypto Guide*, §“The random oracle model”); the uniforms are $U_i = \mathcal{O}(H(\text{hdr}(B_\eta)), i)$, $i = 1, \dots, Q$, read as points of $[0, 1)$, seeded by the proof-of-work output of the head. Through rt_n the head commits to the whole claimed chain.

1. *Prover*. It computes $U_1, \dots, U_Q$ from its head and the sampled leaves by Definition 5.19, and sends one message: the head with the root's children, for each i the sampled header, leaf tuple and inclusion path with full node tuples, and the checked suffix.
2. *Verifier*. For each proof received it recomputes $U_1, \dots, U_Q$ from the head and runs steps 1 and 4 to 6 of Construction 5.26 on the points they determine.

The paper suggests a standard hash (ePrint 2019/226, Section 6.2) and fixes no domain separation, no map from hash outputs to points and no proof encoding. In the classes of Definition 1.3 the construction is designed but unspecified for Zcash.

Assumption 5.28 (Head-recomputation budget). During one execution the adversary produces at most R heads that carry a valid proof of work.

Assumption 4.8 bounds the validly mined weight of a fork, not the number of head recomputations, and does not imply Assumption 5.28. The paper’s budget $R = cn$ (“our security assumption gives a concrete bound on the number of PoW puzzles the adversary can solve, which is cn ”, ePrint 2019/226, Section 6.2) is asserted, not derived.

Proposition 5.29 (Grinding bound). *Assume Assumptions 5.28, 5.20, 5.11 and 4.8, let $\mathcal{O}$ be a random oracle, let H be the random oracle of Definition 4.1, with κ -bit outputs, and let $k > 1$. Let at most $q_{\mathcal{O}}$ queries to $\mathcal{O}$ and q_H queries to H be made. The probability that an adversary makes Construction 5.27 accept a claimed chain whose fork is not caught is at most*

$$R (1 - 1/k)^Q + q_{\mathcal{O}} q_H 2^{-\kappa} + \text{Adv}_H^{\text{col}}(\mathcal{B}) + \text{negl}(\kappa)$$

for an explicit reduction $\mathcal{B}$. Hence

$$Q > \frac{\sigma + \log_2 R}{\log_2(1/(1 - 1/k))}$$

suffices for failure probability below $2^{-\sigma}$ plus negligible terms. This Q is $\Theta((\sigma + \log R) k)$ and, at constant difficulty with constant L , $\Theta((\sigma + \log R) \log_{1/c} n)$; it exceeds the interactive count of Proposition 5.22 by the $\log_2 R$ bits of grinding.

Proof. Each head with a valid proof of work fixes, through rt_n , the claimed chain and its invalid set before its challenge $\mathcal{O}(H(\text{hdr}(B_\eta)), \cdot)$ is known, barring a collision of H (Proposition 5.3). The challenge of a header cannot be computed before $H(\text{hdr})$ is queried, and that same query decides validity. Since $\mathcal{O}$ is independent of H , the challenges of the at most R valid heads are uniform and independent of which heads are valid, except when $\mathcal{O}$ was queried at a value before it arose as an H -output, which has probability at most $q_{\mathcal{O}} q_H 2^{-\kappa}$. So by Assumption 5.20 and Theorem 5.21 the Q samples of each valid head all miss with probability at most $(1 - 1/k)^Q$ plus a negligible term. A head without a valid proof of work fails step 5 of Construction 5.26. By Assumption 5.28 at most R heads qualify, and the union bound (*Math Guide*, §“The union bound and a birthday calculation”) gives the bound. The threshold follows on taking logarithms, and its order as in the proof of Proposition 5.22. $\square$

The bound is proved here because the *Crypto Guide* proves Fiat–Shamir security only for Sigma protocols (§“The Fiat–Shamir transform: from interactive to non-interactive”); the argument above uses only that the challenge is a random-oracle image of the proof-of-work output of a header committing to the whole instance. Seeded by the header itself, the challenge could be evaluated before the proof of work is attempted, and an adversary could mine only heads whose samples miss.

Example 5.30 (Grinding cost). At $c = 1/2$, $\sigma = 50$, $L = 100$ and $n = 2^{22}$, $k = \log_2(n/L) \approx 15.36$. Proposition 5.22 gives $Q \geq 514.7$, so 515 samples. With the paper's $R = cn = 2^{21}$ the budget adds 21 bits, and Proposition 5.29 gives $Q > 730.8$, so 731 samples. At the other (n, L) of Example 5.25, namely $(2^{20}, 100)$, $(2^{22}, 1,000)$, $(2^{24}, 100)$ and $(2^{26}, 100)$, the non-interactive counts are 615, 568, 853 and 981.

Proposition 5.31 (Transferability). *A proof of Construction 5.27 is publicly verifiable: its verification uses only the proof, the genesis block and the public oracles, so any party can relay it unchanged, and it convinces every verifier that the original convinces. For a given head the sampled points are deterministic; under a canonical proof encoding there is therefore one proof per head, which one party can compute for every verifier.*

Proof. The verifier's decision is a deterministic function of the proof and public data, and the points are a function of the head. $\square$

The paper's statement that “there only exists a single valid non-interactive proof” (ePrint 2019/226, Section 8.1) holds only under a canonical encoding, which no ZIP specifies. ZIP 221, “Security and Privacy Considerations” records that, because FlyClient proofs are non-interactive and publicly verifiable, “they could be shared among many light clients after the initial server interaction”.

5.7 The security theorem

Theorem 5.32 (FlyClient security and succinctness). *Assume:*

1. $0 < c < 1$, a real $k > 1$ and $\delta = c^k$;
2. the variable-difficulty backbone model (Definition 4.1), in which every adversary is a (c, L) -adversary (Assumption 4.8);
3. H collision resistant and, as in Definition 4.1, a random oracle, and the sampling oracle $\mathcal{O}$ a random oracle;
4. Assumptions 5.28, 5.20, 5.11 and 4.4;
5. a verifier that knows only the genesis block, and at least one honest prover (Definition 4.16);
6. no block header equals $\text{ser}(\ell) \parallel \text{ser}(r)$ for node tuples ℓ, r (leaf-internal separation, hypothesis 2 of Lemma 5.4).

Then an adversary wins the game of Definition 4.17 against Construction 5.27 with probability at most $R(1 - 1/k)^Q$ plus negligible terms, which is at most $2^{-\sigma}$ plus negligible terms when Q satisfies Proposition 5.29. Proofs have

$$O(L_\delta + (\sigma + \log R) k \log n)$$

headers and hashes, where L_δ is the number of blocks of the checked suffix. The construction is succinct (Definition 4.18) whenever L_δ , k and $\log R$ are $O(\text{polylog}(n))$, in particular at constant

difficulty with constant L and $\delta = L/n$, where $L_\delta = L$ and $k = \log_{1/c}(n/L)$; there, with the paper's $R = cn$, the size is $O(L + (\sigma + \log n) \log_{1/c}(n) \log n)$.

Proof structure. The proof composes the obligations O1 to O6 of Remark 5.33, following the five steps of ePrint 2019/226, Section 6.3. (0) The honest prover's proof passes every check: its labelling is consistent, its transitions are clamped, and Lemma 3.21(2) makes each weight interval its true prefix weight. (1) Assumption 5.11 replaces an adversary with invalid transitions by one that respects the retarget rule, to which Assumption 4.8 applies (O3). (2) O1 pins every sampled answer to one committed chain; O2 makes every block after the fork point adversarial, or places a leaf of the fork in its invalid set, and makes its validly mined weight post-fork; O3 binds each sampled leaf's weight interval to the root, while the agreement of that interval with the chain's true prefix weight rests on the node checks and Assumption 5.11. (3) O4: a fork heavier than the honest chain and longer than L blocks is caught by one draw with probability at least $1/k$, and a fork within the checked suffix with certainty. (4) O5: the Fiat–Shamir step with the grinding bound. (5) O6: the size count. Agreement of honest full nodes on the prefix up to the last ℓ blocks, the second clause of Definition 4.17, is Assumption 4.4. $\square$

The theorem is ePrint 2019/226, Theorem 1 (p. 10, restated p. 20); ZIP 221, “Security and Privacy Considerations” restates its honest-connection and genesis hypotheses. The paper states the size as $O(L + \lambda \log_{1/c}(n) \log n)$. That equals the constant-difficulty bound above with $R = cn$ only when $\log n = O(\lambda)$, for instance for n polynomial in λ ; under the convention that n grows with σ fixed, the theorem states the $\log R$ form and no stronger bound.

Remark 5.33 (Proof obligations). Each obligation is listed with the results that discharge it.

- O1 Binding: Theorem 3.17, Corollary 3.18, Lemma 3.20 and Proposition 5.3.
- O2 Fork structure: Lemmas 5.4 and 5.5.
- O3 Weight binding and transitions: Lemma 5.7, Construction 5.8 and, for subtrees whose aggregates are inconsistent, Assumption 5.11.
- O4 Catch probability and sample count: Theorem 5.21, Definition 5.19, Assumption 5.20 and Proposition 5.22.
- O5 Grinding: Proposition 5.29, which fixes the non-interactive Q .
- O6 Size: Corollary 5.34.

Corollary 5.34 (Proof size). *For the interactive protocol with Q from Proposition 5.22, constant L and constant difficulty, a proof has size $O(\sigma \log n \log_{1/c} n + L)$: $Q = \Theta(\sigma \log_{1/c} n)$ samples, each a header and a path of at most $\lceil \log_2 n \rceil$ nodes under Bag_R (Lemma 3.7), and the L suffix blocks. For the non-interactive protocol, with Q from Proposition 5.29, σ is replaced by $\sigma + \log_2 R$.*

Proof. Multiply Q by the cost of one sample. The suffix blocks form a contiguous run of leaves ending at leaf n , so the union of their paths has $O(L + \log n)$ nodes and is sent once. $\square$

The corollary is ePrint 2019/226, Corollary 2 (Section 5.4), which states Θ ; the matching lower bound is not proved here.

Remark 5.35 (Status of the security theorem). Theorem 5.32 is the paper’s conditional claim, not an independently completed proof. Deriving the (c, L) assumption from the backbone parameters is open (Remark 4.13); the paper’s Lemma 1 is invalid as printed (Remark 4.11); its Lemma 4 is a sketch with recorded gaps and enters as Assumption 5.11; the step from the continuum to blocks is Assumption 5.20; and the budget $R = cn$ is asserted. Proved in this volume are binding (O1), fork structure (O2) under leaf–internal separation, the binding of sampled weight intervals to the root, optimality, the sample count and the grinding bound.

5.8 Verification after acceptance

Proposition 5.36 (Inclusion after acceptance). *Let a head B_η with root tuple rt_n be accepted. Membership of the block at height $j - 1$ in the accepted chain is established by one inclusion path of leaf j against rt_n , and membership of a transaction in that block by one further Merkle path against the transaction root its header carries; no intermediate header is needed. Both are binding barring a collision.*

Proof. For the block, Proposition 5.3(i); for the transaction path, *Crypto Guide*, §“Soundness: forging a path implies a collision”. $\square$

The proposition is ePrint 2019/226, Sections 1 and 4.2.

Proposition 5.37 (Verification against a fresh head). *A client may run the protocol against the freshest heads and store one recent block B^* of its previously accepted chain lying more than 2ℓ blocks below that chain’s head. Let the height of B^* be less than that of the newly accepted head minus ℓ . Under the hypotheses of Theorem 5.32, except with its soundness error, the accepted head’s committed range contains B^* at its height, which the client checks by one inclusion path (Proposition 5.36); a head is rejected if a sample exposes an invalid block.*

Proof. By Theorem 5.32 both accepted chains are equivalent up to their last ℓ blocks to chains of honest full nodes. A head B_η commits to heights 0 to $\eta - 1$, so clause (b) of Definition 4.17 at the earlier round places B^* in every honest chain of that round with its last ℓ blocks removed, and Assumption 4.4 places it in every later honest chain. The height of B^* is below that of the new head minus ℓ , so B^* lies in the part of the new committed chain that clause (b) makes a prefix of an honest chain. $\square$

The paper states without qualification that the range of a fresh head “cannot contain invalid blocks and it must contain all stable blocks” (ePrint 2019/226, Section 4.2, “Unstable Blocks”); that sentence is not carried.

Proposition 5.38 (Subchain proofs, the paper’s claim). *Assume the hypotheses of Theorem 5.32. Suppose a verifier accepted a valid proof for a chain with head B_η when the honest chain had that head, and later, when the honest chain has head $B_{\eta'}$, $\eta' > \eta$, receives a subproof for the blocks*

from B_η to $B_{\eta'}$, sampled as an instance of the protocol with B_η in place of genesis, together with one inclusion path showing that B_η lies in the range committed by $B_{\eta'}$. Then, except with the soundness error of Theorem 5.32 for the instance from B_η , it accepts no chain that a verifier given a full proof for height η' would reject. The instance from B_η , which samples only leaves after B_η of a range committed from B_0 , is the paper's design (ePrint 2019/226, Section 8.2) and is not constructed here.

Proof idea. This is ePrint 2019/226, Theorem 3 (Section 8.2), with the paper's proof idea. A fork after B_η faces a fresh instance of the protocol with B_η as genesis. A fork before B_η fails the inclusion path, since the verifier's B_η is not in the adversary's range, barring a collision (Proposition 5.36). $\square$

The proposition concerns one commitment range from the genesis block.

6 The chain-history commitment

This section states the commitment that Zcash block headers carry, each rule cited to the ZIP or the protocol specification section that fixes it, and each object before its first use: the upgrade epochs over which it ranges, the node vector of its Merkle mountain range, the hashing of the tree and its root commitment, the authorising-data root and the block commitment hash of ZIP 244 that wraps the root from NU5, the header consensus rules, and the range of blocks to which each header commits.

6.1 The deployed chain commitment

ZIP 221 (Final), “FlyClient - Consensus-Layer Changes”, changes the semantics of the block header and the consensus rules from Heartwood. The 32-byte header field that held the root of the Sapling note commitment tree instead commits to the root of a Merkle mountain range (Definition 3.1; ZIP 221, “Terminology”: a binary hash tree that admits appends of new leaves “without changing the value of existing nodes”) whose nodes carry aggregated metadata. Its leaves are the blocks from the last activation height below the committing block up to the parent of that block: the *epoch tree* of Definition 6.2. ZIP 221 thus deploys the invariant of Construction 5.1 restricted to one *upgrade epoch*, the heights from one activation height up to the next (Definition 6.1), and not from the genesis block. The tree and its node fields 1 to 14 are ZIP 221's; fields 15 to 17 are ZIP 258's (Draft, in force from NU6.3 by Definition 1.2); and from NU5 the root is committed as the first input of the *block commitment hash*, a personalised hash of the root together with the block's authorising-data root (ZIP 244; “The block commitment hash”, §6.6). Seven node fields serve reasoning about difficulty without matching the node of Definition 5.6 (ePrint 2019/226, Definition 7) field for field (Remark 6.4). Each term of this paragraph carries here only its defining clause; its definition is in the subsection named.

The purpose of the commitment, after ZIP 221, “Motivation”, is to let a light client verify, with a number of headers logarithmic rather than linear in the chain length and correctly with high probability, the validity of a chain received from a full node, the inclusion of a block in it, and metadata of any block or range of blocks; the commitment also admits subtree proofs, which

check only blocks later than the most recently verified one. Transaction inclusion within a block is left to ZIP 307 (Draft). No claim of this paragraph is proved in this section; the protocol that would realise it is that of “The FlyClient protocol” (§5), and its relation to the deployed commitment is classified in “The deployment boundary” (§7).

6.2 Upgrade epochs and epoch trees

Definition 6.1 (Upgrade epoch). Let $0 = A_0 < A_1 < A_2 < \dots$ be the distinct activation heights of the network upgrades of a chain (ZIP 200, “Activation mechanism”), height 0 counted as an activation height (ZIP 221, “Specification”). The named upgrades and their order are those of Definition 1.1; the list of activation heights includes those of upgrades that definition does not name. The *upgrade epoch* beginning at A_i is the interval of heights $[A_i, A_{i+1})$, unbounded for the last activation height. For $i \geq 1$ its blocks are validated under one consensus rule set and one *consensus branch ID*, the 32-bit identifier of the last upgrade that activates at A_i (ZIP 200, “Specification”; *Ironwood Guide*, §“The transaction format”). The upgrade epoch is distinct, by name and by object, from the retarget epoch of Definition 4.5; the word “epoch” is not used alone. The Heartwood activation height is written A_H , and B_{A_H} is the Heartwood activation block.

Definition 6.2 (Epoch tree). For a block height $\eta > 0$, the *preceding activation height* $\alpha(\eta)$ is the largest activation height $A_i < \eta$, the “preceding network upgrade height” of ZIP 221, “Specification”; $\alpha(0)$ is undefined. For $\eta > A_H$ the *epoch tree* $\mathcal{T}_\eta$ is the Merkle mountain range $\mathcal{M}_n^L$ under the left fold Bag_L (Definition 3.5), bagging nodes included as internal nodes, with the

$$n = \eta - \alpha(\eta) \geq 1$$

leaves $B_{\alpha(\eta)}, \dots, B_{\eta-1}$ in height order: leaf j is $B_{\alpha(\eta)+j-1}$. Since $\alpha(\eta)$ is the last activation height below η , all its leaves lie in the one upgrade epoch beginning at $\alpha(\eta)$. Its node contents are fixed in “The node vector” (§6.3) and its hashing in “The epoch tree and its root commitment” (§6.4). No epoch tree is defined for $\eta \leq A_H$.

Which header commits to $\mathcal{T}_\eta$ is not part of Definition 6.2: it is Definition 6.22 in “The header consensus rules” (§6.7), and its consequences for the committed ranges are stated in “The committed range” (§6.8).

6.3 The node vector

Definition 6.3 (Node vector). The epoch tree $\mathcal{T}_\eta$ is an instance of Definition 3.19 under Bag_L . Every node v of it, leaf, internal node, bagging node or root, carries one *node vector* of fields, serialised canonically by the map ser of Definition 6.5; that serialisation is used whenever a parent commitment is formed. The fields are numbered as in ZIP 221, “Tree Node specification”. For a block $B_{\eta'}$ with header hdr' , the *leaf map* sets both members of each earliest/latest pair from $B_{\eta'}$ itself:

- (1) field `hashSubtreeCommitment` is $\text{BlockHash}(\text{hdr}')$ (Definition 2.3), in internal byte order and unpersonalised;
- (2), (3) fields `nEarliestTimestamp` and `nLatestTimestamp` are the field `nTime` of hdr' ;
- (4), (5) fields `nEarliestTargetBits` and `nLatestTargetBits` are the field `nBits` of hdr' (the semantics of `nTime` and `nBits`: *Consensus Guide*, §“Difficulty in motion”);
- (6), (7) fields `hashEarliestSaplingRoot` and `hashLatestSaplingRoot` are $\text{LEBS2OSP}_{256}(\text{rt})$ for rt the root of the Sapling note commitment tree of the final Sapling treestate of $B_{\eta'}$ (protocol specification, §“Block Header Encoding and Consensus”), an opaque 32-byte value in this volume;
- (8) field `nSubTreeTotalWork` is the work

$$\lfloor 2^{256} / (\text{ToTarget}(\text{nBits}) + 1) \rfloor$$
 of $B_{\eta'}$ (*Consensus Guide*, §“Work, and the best chain”, Definition “Work of a block”);
- (9), (10) fields `nEarliestHeight` and `nLatestHeight` are the height η' ;
- (11) field `nSaplingTxCount` is the number of transactions of $B_{\eta'}$ in which `vSpendsSapling` or `vOutputsSapling` is non-empty;
- (12), (13) fields `hashEarliestOrchardRoot` and `hashLatestOrchardRoot` are the Orchard-pool anchor of $B_{\eta'}$, the root of the Orchard pool’s note commitment tree in the final treestate of $B_{\eta'}$ (*Ironwood Guide*, §“Anchors”, Definition “Treestate and anchor”);
- (14) field `nOrchardTxCount` is the number of transactions of $B_{\eta'}$ in which `vActionsOrchard` is non-empty (*Ironwood Guide*, §“The transaction format”);
- (15)–(17) fields `hashEarliestIronwoodRoot`, `hashLatestIronwoodRoot` and `nIronwoodTxCount` are the Ironwood-pool anchor of $B_{\eta'}$ and the number of its transactions in which `vActionsIronwood` is non-empty, computed as fields 12 to 14.

The *parent rule*: an internal node with left child ℓ and right child r takes each earliest field (2, 4, 6, 9, 12, 15) from ℓ and each latest field (3, 5, 7, 10, 13, 16) from r ; it sums the additive fields, field 8 modulo 2^{256} and the per-pool counts 11, 14 and 17 in $\mathbb{Z}$; and it sets field 1 by Construction 6.10, the personalised hash of $\text{ser}(\ell) \parallel \text{ser}(r)$. Fields 12 to 14 are ZIP 221’s own, added at NU5 (ZIP 252). Fields 15 to 17 are ZIP 258’s, “Changes to ZIP 221” (Draft), whose rules apply from NU6.3 activation (Definition 1.2).

For field 8, ZIP 221, “Tree Node specification”, states that computations modulo 2^{256} are fine because cumulative chain work is assumed to be at most 2^{256} , an aggregate effort of 2^{256} or more being infeasible; under that assumption, with the total below 2^{256} , the field of a node is the integer sum of the work of its leaves (Lemma 3.21(2)). Table 2 summarises the definition.

Table 2. The fields of the node vector of Definition 6.3: serialised type, value at the leaf for the block $B_{\eta'}$ with header hdr' , rule at an internal node (“hash” is Construction 6.10), and source: ZIP 221, “Tree Node specification”, for fields 1 to 14, and ZIP 258, “Changes to ZIP 221”, for fields 15 to 17.

	Field	Type	Leaf value for $B_{\eta'}$	Internal node	ZIP
1	hashSubtreeCommitment	char[32]	BlockHash(hdr')	hash	221
2	nEarliestTimestamp	uint32	nTime	inherit left	221
3	nLatestTimestamp	uint32	nTime	inherit right	221
4	nEarliestTargetBits	uint32	nBits	inherit left	221
5	nLatestTargetBits	uint32	nBits	inherit right	221
6	hashEarliestSaplingRoot	char[32]	final Sapling root	inherit left	221
7	hashLatestSaplingRoot	char[32]	final Sapling root	inherit right	221
8	nSubTreeTotalWork	uint256	work of the block	sum mod 2^{256}	221
9	nEarliestHeight	CompactSize	height η'	inherit left	221
10	nLatestHeight	CompactSize	height η'	inherit right	221
11	nSaplingTxCount	CompactSize	transactions with Sapling spends or outputs	sum	221
12	hashEarliestOrchardRoot	char[32]	Orchard-pool anchor	inherit left	221
13	hashLatestOrchardRoot	char[32]	Orchard-pool anchor	inherit right	221
14	nOrchardTxCount	CompactSize	transactions with Orchard-pool Actions	sum	221
15	hashEarliestIronwoodRoot	char[32]	Ironwood-pool anchor	inherit left	258
16	hashLatestIronwoodRoot	char[32]	Ironwood-pool anchor	inherit right	258
17	nIronwoodTxCount	CompactSize	transactions with Ironwood-pool Actions	sum	258

Remark 6.4 (The FlyClient fields). ZIP 221, “FlyClient Requirements and Recommendations”, designates seven fields, 2 to 5 and 8 to 10, which “enable FlyClient in the variable-difficulty model” by letting a light client reason about the application of the difficulty adjustment over a range of blocks. They serve the purpose of Definition 5.6 without matching it field for field. The leaf count of a node, the field n of the difficulty MMR, is $n_{\text{LatestHeight}} - n_{\text{EarliestHeight}} + 1$ (Lemma 6.6). The field w of the difficulty MMR sums difficulty levels (Definition 4.2), whereas $n_{\text{SubTreeTotalWork}}$ sums work, a different normalisation. The field $n_{\text{EarliestTargetBits}}$ is a compact target, ordered inversely to D_{start} . No field carries the elapsed time t ; the timestamps of the first and last block stand in its place. No field carries D_{next} : the next target of a Zcash block is a function of an averaging window of preceding blocks (*Consensus Guide*, §“Difficulty in motion”), which the aggregates of a node do not determine.

Fields 6, 7 and 11 to 14 are ZIP 221’s “Non-FlyClient Commitments”, chosen “primarily to improve light client security guarantees”, with no counterpart in ePrint 2019/226. Field 1 “forms the structure of the commitment tree” (ZIP 221, “Tree nodes”), and fields 15 to 17 are the ZIP 258 counterparts for the Ironwood pool. The earliest roots let a client check the treestate transitions over a range of blocks without recomputing the roots from genesis. ZIP 221, “Non-FlyClient Commitments”, states that `hashLatestSaplingRoot` serves the purpose of the former header field `hashFinalSaplingRoot`, one block delayed (Proposition 6.23). The per-pool counts follow the rule of the same section that a new shielded pool adds a count of its transactions, applied at NU5 (fields 12 to 14) and at NU6.3 (fields 15 to 17, ZIP 258).

Definition 6.5 (Field vector by upgrade epoch). The fields are encoded as follows: a `char[32]` field as its 32 bytes; a `uint32` or `uint256` field little-endian, as `l2LEOSP32` or `l2LEOSP256` (protocol specification, §“Integers, Bit Sequences, and Endianness”); a height or count as its `CompactSize` encoding in minimal form (Definition 2.2); and each pool-root field as `LEBS2OSP256` of the root, as the specification encodes the Orchard anchor (protocol specification, §“Transaction Encoding and Consensus”). For a node v , $\text{ser}(v)$ is the concatenation of the encodings of its fields in field order. The *field vector* of an upgrade epoch is:

1. fields 1 to 11 in the Heartwood and Canopy upgrade epochs;
2. fields 1 to 14 in the NU5, NU6, NU6.1 and NU6.2 upgrade epochs;
3. fields 1 to 17 in the NU6.3 and NU7 upgrade epochs.

The vectors are named by their fields. ZIP 221, “Tree Node specification”, omits fields 12 to 14 “before NU5 activation”, and ZIP 258, “Changes to ZIP 221”, omits fields 15 to 17 “before NU6.3 activation”; neither says whether the height of a leaf’s block or that of the committing block governs, and the two differ only for the tree committed by an activation block. *Adopted reading*, resolving that ambiguity: every node of $\mathcal{T}_\eta$ uses the field vector of the upgrade epoch beginning at $\alpha(\eta)$, the upgrade epoch of its leaves.

The reading is adopted for the following reason. Under the other reading the tree $\mathcal{T}_A$ committed by B_A , at an activation height A that adds fields, would serialise every node of the previous upgrade epoch with the added fields; every internal node would change value, and $\mathcal{T}_A$ would not be an append of $\mathcal{T}_{A-1}$, contrary to the property by which ZIP 221, “Terminology”, defines a Merkle mountain range. The ZIP’s pseudocode, which takes a leaf’s Orchard fields from the leaf’s own block and serialises them only when present, is consistent with the reading. Two consequences rest on it: $\mathcal{T}_A$ uses the previous upgrade epoch’s vector, and $\mathcal{T}_{A+1}$ is the first tree with the new one; under the other reading $\mathcal{T}_A$ would carry the new vector. For NU7, no ZIP among the sources of NU7 consensus changes listed in ZIP 259 (Draft), “NU7 consensus changes”, amends ZIP 221 or ZIP 244, so the nodes of the trees of the NU7 upgrade epoch carry fields 1 to 17 under either reading.

The fixed-width fields occupy 144, 208 and 272 bytes in the three vectors, and the three, four and five `CompactSize` fields 1 to 9 bytes each, so a serialised node has 147 to 171, 212 to 244 and 277 to 317 bytes, the ranges that ZIP 221 and ZIP 258 state. A toy chain computed by script illustrates the reading, with an activation block that commits a tree of the previous vector and the next block the first to commit a tree of the new one; it establishes nothing.

Lemma 6.6 (Leaf count and shape). *Every node v of $\mathcal{T}_\eta$, bagging nodes included, spans a run of consecutive leaves $B_{\eta_1}, \dots, B_{\eta_2}$ with $v.\text{nEarliestHeight} = \eta_1$ and $v.\text{nLatestHeight} = \eta_2$, so its leaf count is $\text{nLatestHeight} - \text{nEarliestHeight} + 1$. The node v roots a perfect subtree of $\mathcal{T}_\eta$ if and only if that count is a power of two. In particular a leaf is the only kind of node with count 1.*

Proof. By induction on the parent rule of Definition 6.3. A leaf has both heights equal to its block’s. An internal node inherits `nEarliestHeight` from its left child and `nLatestHeight` from its

right child, and in a range under Bag_L the run of the left child immediately precedes that of the right child, the heights of the leaves of $\mathcal{T}_\eta$ being consecutive by Definition 6.2. A perfect subtree of altitude a spans 2^a leaves (Definition 3.1), and every node inside a peak roots a perfect subtree. A bagging node of the left fold spans the peaks $P_1, \dots, P_i$ for some $i \geq 2$ (Definition 3.5), whose sizes are distinct powers of two, set bits of n (Definition 3.4); a sum of two or more distinct powers of two has at least two set bits and is not a power of two. An internal node spans at least two leaves. A script checks the statement for $1 \leq n \leq 1,024$. $\square$

Lemma 6.7 (Leaves and header linkage). *For $\alpha(\eta) \leq \eta' \leq \eta - 1$, field 1 of the leaf of $\mathcal{T}_\eta$ for $B_{\eta'}$ equals the field `hashPrevBlock` of the header of $B_{\eta'+1}$, which for $\eta' = \eta - 1$ is the committing block B_η . Consequently a received header is matched to its leaf by computing its block hash and comparing it with field 1, and the leaves add no hashing beyond the block hashes.*

Proof. By Definition 6.3, field 1 of the leaf is `BlockHash` of the header of $B_{\eta'}$ in internal byte order; by Definition 2.3, the field `hashPrevBlock` of $B_{\eta'+1}$ is that same hash. ZIP 221, “Tree Node specification”, states the identity. $\square$

Remark 6.8 (Non-monotone timestamps). Consensus bounds the field `nTime` of a block below, strictly above the median of the timestamps of the preceding eleven blocks, and above, by that median plus 90 minutes (protocol specification, §“Block Header Encoding and Consensus”; *Consensus Guide*, §“Difficulty in motion”). It orders two timestamps only at a distance: the median-time-past never decreases and rises by at least one second every eleven blocks, so a block at least $11 \cdot 5,400 = 59,400$ heights after another has the strictly later timestamp. By the same count, $\text{nLatestTimestamp} \geq \text{nEarliestTimestamp}$ holds for every node with $\text{nLatestHeight} - \text{nEarliestHeight} \geq 11 \cdot 5,399 = 59,389$, and for a leaf, whose two timestamps are equal; for smaller spans this argument gives no order. ZIP 221, “Tree Node specification”, says that the reverse order “may occur within subtrees smaller than $\text{PoWMedianBlockSize}$ blocks”; this is an example, not a bound. A block whose timestamp lies up to 90 minutes ahead of the median exceeds the timestamps of the later blocks whose timestamps track real time, until real time passes it, and 90 minutes spans more than eleven target spacings (protocol specification, §“Constants”; from NU7, ZIP 218).

6.4 The epoch tree and its root commitment

The tree is defined from the prose of ZIP 221, “Background”, “Specification”, “Tree Node specification” and “Block header semantics and consensus rules”, with ZIP 258, “Changes to ZIP 221”. That prose is ambiguous at two points, both at an activation block: the field vector, settled by the adopted reading of Definition 6.5, and the branch ID of the personalisation, settled by Definition 6.9. The pseudocode of ZIP 221 is cited only as corroboration of the two readings; it carries editorial defects, among them in the serialisation of the Orchard fields, none of which bears on either reading.

Definition 6.9 (Branch ID of a tree). ZIP 221, “Tree Node specification”, states that the personalisation encodes “the consensus branch ID for the epoch of the block containing the commitment”, and continues: “Which is to say, each node in the tree commits to the consensus

branch that produced it.” The two sentences agree at every $\eta > A_H$ that is not an activation height, since B_η and the leaves of $\mathcal{T}_\eta$ then lie in one upgrade epoch. At an activation height $A > A_H$ the first names the upgrade epoch beginning at A and the second the upgrade epoch of the leaves $B_{\alpha(A)}, \dots, B_{A-1}$. At A_H no epoch tree is defined (Definition 6.2). *Adopted reading*, resolving an ambiguity of specified text: the branch ID $b(\mathcal{T})$ of an epoch tree $\mathcal{T}$ is the consensus branch ID of the upgrade epoch whose blocks are the leaves of $\mathcal{T}$, and every internal node and the root commitment of $\mathcal{T}$ are hashed under it.

The reading is adopted for three reasons. (1) “Which is to say” introduces the second sentence as a restatement of the first, and it is the sentence that assigns a branch ID to each node. (2) Under the first reading the tree committed by B_A would not be an append of the tree committed by B_{A-1} , every internal node changing value, contrary to the property by which ZIP 221, “Terminology”, defines a Merkle mountain range (the reason for which Definition 6.5 adopts the leaves’ vector); and the whole previous upgrade epoch would be rehashed in one block, the cost that ZIP 221, “Header Format Change”, calls impractical and avoids by a special case at the Heartwood activation block and at no later one. (3) The pseudocode of ZIP 221 hashes each parent under the branch ID of its left child and the root commitment under that of the root node, each leaf carrying the branch ID of its own block. Every later statement about the tree committed by an activation block rests on this reading and cites it; under the first reading each holds with the branch ID of the new upgrade epoch in place of $b(\mathcal{T}_A)$.

Construction 6.10 (Hashing of the epoch tree). Let $h(P, x)$ be BLAKE2b-256 with the 16-byte personalisation P on input x (*Ironwood Guide*, §“Transaction digests and signatures”; *Crypto Guide*, §“Domain separation and personalisation”), and let β be the 4-byte little-endian encoding of $b(\mathcal{T}_\eta)$. The personalisation is the 12 ASCII bytes `ZcashHistory` followed by β , 16 bytes in all. The tree $\mathcal{T}_\eta$ is the range M_n^L whose nodes are node vectors (Definition 6.3) under the field vector of Definition 6.5. Its bagging nodes, which join the two leftmost remaining peaks until one remains,

$$(\dots ((P_1 \parallel P_2) \parallel P_3) \dots) \parallel P_d,$$

are ordinary internal nodes, and a single peak is the root node. For an internal node with children ℓ and r , field 1 is

$$h(\text{ZcashHistory} \parallel \beta, \text{ser}(\ell) \parallel \text{ser}(r)),$$

over the children’s full serialisations; its other fields follow the parent rule of Definition 6.3. The branch ID is not a node field: it enters only through the personalisation, and whoever recomputes a node hash supplies $b(\mathcal{T}_\eta)$ by Definition 6.9, the upgrade epoch being fixed by the node’s heights. For the Heartwood branch ID `0xF5B9230B` the last four bytes of the personalisation are `0B 23 B9 F5`.

Definition 6.11 (Chain-history root). For $\eta > A_H$ the *chain-history root* of $\mathcal{T}_\eta$ is

$$\text{hashChainHistoryRoot}(\mathcal{T}_\eta) := h(\text{ZcashHistory} \parallel \beta, \text{ser}(\text{root node of } \mathcal{T}_\eta)),$$

with β encoding $b(\mathcal{T}_\eta)$ (ZIP 221, “Tree Node specification”). It is not field 1 of the root node: the committed value covers the root’s aggregates, namely the total work modulo 2^{256} , the first

Table 3. The branch IDs entering the personalisation $\text{ZcashHistory} \parallel \beta$ of Construction 6.10: one row per upgrade epoch that bears an epoch tree, each tree hashed under its own personalisation, with the ZIP that fixes the branch ID (ZIP 258, “NU6.3 deployment”; ZIP 259, “NU7 network constants”), the little-endian bytes β and the field vector of Definition 6.5.

Upgrade epoch	Branch ID	Fixed by	Bytes 13 to 16 of the personalisation	Fields
Heartwood	0xF5B9230B	ZIP 250	0B 23 B9 F5	1 to 11
Canopy	0xE9FF75A6	ZIP 251	A6 75 FF E9	1 to 11
NU5	0xC2D6D0B4	ZIP 252	B4 D0 D6 C2	1 to 14
NU6	0xC8E71055	ZIP 253	55 10 E7 C8	1 to 14
NU6.1	0x4DEC4DF0	ZIP 255	F0 4D EC 4D	1 to 14
NU6.2	0x5437F330	ZIP 257	30 F3 37 54	1 to 14
NU6.3	0x37A5165B	ZIP 258 (Draft)	5B 16 A5 37	1 to 17
NU7	0x77190AD9	ZIP 259 (Draft)	D9 0A 19 77	1 to 17

and last timestamps, target bits and heights, the boundary pool roots and the per-pool counts. When $\alpha(\eta) = \eta - 1$ the tree has one leaf, which is its root node, and the value is defined there too. ZIP 258, “Changes to ZIP 221”, leaves the definition unchanged apart from the added fields.

Definition 6.12 (History input of a block). For $\eta \geq A_H$ the *history input* of B_η is

$$\text{hashLightClientRoot}(\eta) := \begin{cases} \text{the string of 32 zero bytes} & \text{if } \eta = A_H, \\ \text{hashChainHistoryRoot}(\mathcal{T}_\eta) & \text{if } \eta > A_H. \end{cases}$$

The value at A_H is the sentinel that ZIP 221, “Block header semantics and consensus rules”, requires in the Heartwood activation block and states “MUST NOT be interpreted as a root hash”. The history input is a total function of the height from A_H on, and it is the value that ZIP 244, “Block Header Changes”, names “hashLightClientRoot (as described in ZIP 221)”.

Proposition 6.13 (Binding of the chain-history root). *Let $\mathcal{T}$ and $\mathcal{T}'$ be epoch trees hashed under one branch ID b . If $\text{hashChainHistoryRoot}(\mathcal{T}) = \text{hashChainHistoryRoot}(\mathcal{T}')$, then either $\mathcal{T}$ and $\mathcal{T}'$ agree on every field of every node, or a collision of $\text{h}(\text{ZcashHistory} \parallel \beta, \cdot)$ is computed from them efficiently. If in addition SHA-256d is collision resistant, the leaves’ block hashes bind the headers of the leaf blocks.*

Proof. The proposition is the ZIP 221 instance of Lemma 3.20(1), whose hypotheses are checked as follows. *Serialisation.* The branch ID b fixes one upgrade epoch and hence one field vector for both trees (Table 3; Definitions 6.9 and 6.5). Its field order is fixed, and each field is of fixed width or a minimal CompactSize, whose first byte fixes its length (Definition 2.2). A serialised node therefore parses field by field: ser is injective, and no serialised node is a proper prefix of another, since the two parse to the same fields up to the end of the shorter and the number of fields is fixed; minimality makes ser a function of the node. *Root and shape.* If the root commitments are equal, then the two root serialisations are equal, or they are distinct inputs with equal images, a collision. Equal root nodes have equal nEarliestHeight and nLatestHeight , hence equal leaf count

n by Lemma 6.6, hence the same shape $\mathcal{M}_n^L$. The shape fixes which positions are leaves, so no leaf of one tree is compared with an internal node of the other, although the tree carries no layer tags (ZIP 221, “Tree nodes”, identifies leaves by their block range of 1). *Extraction.* Lemma 3.20(1) then yields a collision at the first level of disagreement. *Headers.* Field 1 of each leaf is the SHA-256d block hash of its header (Definition 2.3; *Crypto Guide*, §“The transparent layer’s hashes”); equal leaves with distinct headers are a collision of SHA-256d. $\square$

Proposition 6.14 (Binding of range transaction counts). *Let a node v of $\mathcal{T}_\eta$ be authenticated by an inclusion path (Definition 3.13, in the tuple form of Definition 3.19, siblings as full node vectors) accepted against the chain-history root of the epoch tree of a chain the client trusts. Barring a collision as in Proposition 6.13, $v.n\text{SaplingTxCount}$, and where present $v.n\text{OrchardTxCount}$ and $v.n\text{IronwoodTxCount}$, equal the numbers of transactions in the blocks of the height range of v in which $v\text{SpendSapling}$ or $v\text{OutputsSapling}$, $v\text{ActionsOrchard}$ and $v\text{ActionsIronwood}$, respectively, are non-empty. Hence a client that obtains the transactions of those blocks, each authenticated against its block’s transaction root, detects the withholding of any such transaction by a count mismatch, and a range whose count is zero contains none and may be skipped; the count does not identify the missing transaction.*

Proof. By Lemma 3.20(2), an accepted path binds the full vector of every node it recomputes and of every sibling it carries, so v is the node of the trusted tree at its position, barring a collision; the serialisation is prefix-free by the proof of Proposition 6.13. In that tree the count fields are consistent, and by the parent rule of Definition 6.3 and Lemma 6.6 the value of each count field of v is its sum over the leaves of v ’s height range. The reading as detection of withholding is ZIP 221, “Non-FlyClient Commitments”. $\square$

Proposition 6.15 (Cross-branch separation). *For branch IDs $b \neq b'$ with encodings $\beta \neq \beta'$ the personalisations $\text{ZcashHistory} \parallel \beta$ and $\text{ZcashHistory} \parallel \beta'$ differ. Model $h(P, \cdot)$, for each personalisation P , as an independent random oracle with 256-bit output. An adversary making q oracle queries, the queries of the verification of its output included, outputs x, x' with*

$$h(\text{ZcashHistory} \parallel \beta, x) = h(\text{ZcashHistory} \parallel \beta', x')$$

with probability at most $q^2 / 2^{256}$. Hence no internal-node hash or root commitment computed under b equals one computed under b' , except with that probability. Leaves are unpersonalised block hashes, identical in every branch, and are not separated.

Proof. The modelling hypothesis is that of the *Crypto Guide*, §“Domain separation and personalisation”, Proposition “Domain separation yields independent oracles”: distinct tags give independent random oracles. Both values must be answers to queries, one to each oracle; each of the at most q^2 pairs of such queries agrees with probability 2^{-256} , and the union bound (*Math Guide*, §“The union bound and a birthday calculation”) gives the bound. ZIP 221, “Tree nodes”, states the design intent: “valid nodes from one consensus branch cannot be used to make false statements about parallel consensus branches”. $\square$

6.5 The authorising-data root

The header field `hashMerkleRoot` (Definition 2.3) is the root of Bitcoin’s SHA-256d transaction Merkle tree over the transaction identifiers of the block (protocol specification, §“Block Header Encoding and Consensus”). The identifier of a version-5 or version-6 transaction excludes its authorising data (ZIP 244; *Ironwood Guide*, §“Transaction digests and signatures”, Definition “Authorising-data digest; effecting and authorising data”), so for such transactions `hashMerkleRoot` does not commit to the authorising data; the identifier of a version-4 transaction covers its whole encoding. The version-4 format is valid until NU7, from which ZIP 2003 (Draft), deployed by ZIP 259 (Draft), makes it invalid.

Construction 6.16 (Authorising-data root). Let a block have the transactions $t_1, \dots, t_m$, where $m \geq 1$ because of the coinbase transaction. Leaf i is the *authorising-data commitment* of t_i : for version 5, the digest `auth_digest` of ZIP 244, “Authorizing Data Commitment”; for version 6, the digest of the *Ironwood Guide*’s Definition “Authorising-data digest; effecting and authorising data” (ZIP 229 (Draft), “Block Commitments”: `hashAuthDataRoot` takes the version-6 digest “with no further structural change”); and for a transaction of a version before 5, the 32 bytes `0xFF`, a placeholder used only in this tree and occurring only in blocks before NU7. The leaves are padded with leaves of 32 zero bytes to $2^{\lceil \log_2 m \rceil}$, and `hashAuthDataRoot` is the root of the binary Merkle tree of height $\lceil \log_2 m \rceil$ over them, each non-leaf node being $h(\text{ZcashAuthDatHash}, \text{left} \parallel \text{right})$, under a 16-byte personalisation without branch ID. The insertion order MUST equal that of the transaction identifiers in the tree under `hashMerkleRoot`, so that one path identifies the same transaction in both trees (ZIP 244, “Block Header Changes”).

Proposition 6.17 (Joint binding of the two transaction roots). *Assume, with ZIP 244, “Authorizing Data Commitment”, that the pair of a transaction’s identifier and its authorising-data commitment is binding: distinct transactions with equal pairs form a collision of that pair map. Let two lists of m transactions have equal `hashMerkleRoot` and equal `hashAuthDataRoot`. Then the lists are equal, or a collision of SHA-256d, of $h(\text{ZcashAuthDatHash}, \cdot)$ or of the pair map is computed from them. The binding is claimed for lists of the given length only: the identifier tree is Bitcoin’s transaction Merkle tree and inherits its duplicate-last-node caveat, and the header commits to no transaction count.*

Proof. For fixed m , Bitcoin’s duplication rule and the zero padding of Construction 6.16 are fixed maps of the list, so the path argument of the *Crypto Guide*, §“Binary Merkle trees”, Proposition “Binding of the root”, applies to each tree: the two lists of identifiers agree, else a collision of SHA-256d is extracted, and the two lists of authorising-data commitments agree, else a collision of $h(\text{ZcashAuthDatHash}, \cdot)$. The common insertion order aligns the two leaf sequences, so each position carries equal pairs, and distinct transactions there form a collision of the pair map. ZIP 244, “Authorizing Data Commitment”, states that the pair “constitutes a commitment to all the data of a serialized transaction”; for a transaction of a version before 5 the identifier covers all data and the second member is the constant placeholder. The caveat is the *Crypto Guide*, §“Layer and domain separation”, Remark “Unbalanced trees, padding, and a deployed failure”. $\square$

6.6 The block commitment hash

In this subsection 0^{32} denotes the string of 32 zero bytes.

Definition 6.18 (Block commitment hash). For a block B_η at a height in or after the NU5 upgrade epoch, the *block commitment hash* is

$$\text{hashBlockCommitments}(B_\eta) := h(\text{ZcashBlockCommit}, \text{hashLightClientRoot}(\eta) \parallel \text{hashAuthDataRoot}(B_\eta) \parallel 0^{32}),$$

with $\text{hashLightClientRoot}(\eta)$ the history input of Definition 6.12, $\text{hashAuthDataRoot}(B_\eta)$ the root of Construction 6.16 for B_η , and 0^{32} the *terminator* (ZIP 244, “Block Header Changes”). The input has $32 + 32 + 32 = 96$ bytes, and the 16-byte personalisation carries no branch ID. The history input is a function of the blocks before B_η ; the authorising-data root is a function of B_η itself.

Proposition 6.19 (Reopening the block commitment). *From NU5 the chain-history root appears in no header. Let c be the value $\text{hashBlockCommitments}(B_\eta)$ of a block in or after the NU5 upgrade epoch. A verifier given a claimed history input r , or a claimed root node vector from which it computes $r = h(\text{ZcashHistory} \parallel \beta, \text{ser}(\text{root}))$ with β encoding the branch ID of Definition 6.9, and a 32-byte value a , accepts r if and only if*

$$c = h(\text{ZcashBlockCommit}, r \parallel a \parallel 0^{32});$$

only the 32-byte value of hashAuthDataRoot is needed, not its tree. Two distinct accepted pairs $(r, a) \neq (r', a')$ exhibit a collision of $h(\text{ZcashBlockCommit}, \cdot)$. Hence an accepted r is the block’s history input except with that collision probability, and an inclusion path in the epoch tree is checked against a header through it.

Proof. The pair $(\text{hashLightClientRoot}(\eta), \text{hashAuthDataRoot}(B_\eta))$ is accepted by Definition 6.18. The input $r \parallel a \parallel 0^{32}$ has fixed length, so distinct pairs give distinct inputs, and two accepted pairs with equal image c are a collision. $\square$

Remark 6.20 (Extension of the block commitment hash). ZIP 244, “Block Header Changes”, treats the value as “a linked list of hashes terminated by arbitrary data”: a later upgrade may replace the terminator by the hash of a new structure that ends in its own terminator. The same section states two interoperability rules: fully validating nodes MUST always use the entire structure defined by the latest activated protocol version they support, and a third-party server that supports a later protocol version SHOULD provide a value that can be used in place of the all-zero terminator, so that a light client can validate the parts it needs. No upgrade through NU7 replaces the terminator: ZIP 258, “Changes to ZIP 221”, leaves $\text{hashBlockCommitments}$ “unaffected” beyond the added node fields; ZIP 229, “Block Commitments”, changes no structure; and no source of NU7 consensus changes listed in ZIP 259, “NU7 consensus changes”, amends ZIP 244.

6.7 The header consensus rules

Definition 6.21 (The commitment field by upgrade epoch). The 32-byte header field that follows `hashMerkleRoot` (Definition 2.3) is named `hashReserved` before Sapling, `hashFinalSaplingRoot` in the Sapling and Blossom upgrade epochs, `hashLightClientRoot` in the Heartwood and Canopy upgrade epochs, and `hashBlockCommitments` from NU5 (protocol specification, §“Block Header Encoding and Consensus”; ZIP 221, “Block header semantics and consensus rules”; ZIP 244, “Block Header Changes”). The header byte format and version are unchanged by each renaming.

Definition 6.22 (Header commitment rule). For a block B_η the commitment field satisfies the following rules.

- (i) For $\eta < A_H$ it follows the earlier rules of the protocol specification, §“Block Header Encoding and Consensus”, not restated here: in the Sapling and Blossom upgrade epochs it is LEBS2OSP_{256} of the root of the Sapling note commitment tree of the block’s final Sapling treestate.
- (ii) For $A_H \leq \eta$ in the Heartwood or Canopy upgrade epoch it MUST equal the history input `hashLightClientRoot( $\eta$ )` of Definition 6.12, which supplies the sentinel at A_H .
- (iii) For η in or after the NU5 upgrade epoch, the NU5 activation block included and without sentinel, it MUST equal `hashBlockCommitments( $B_\eta$ )` of Definition 6.18.

(ZIP 221, “Block header semantics and consensus rules”; ZIP 244, “Block Header Changes”; protocol specification, §“Block Header Encoding and Consensus”.) The value is undefined for the genesis block, which no case covers.

In the NU5 activation block $B_{A_{\text{NU5}}}$, A_{NU5} the NU5 activation height, the history input is `hashChainHistoryRoot( $\mathcal{T}_{A_{\text{NU5}}}$ )`, the root of the completed tree of the Canopy upgrade epoch. It is hashed under the Canopy branch ID `0xE9FF75A6` (ZIP 251) by the adopted reading of Definition 6.9, the first sentence of ZIP 221 giving the NU5 branch ID instead, and it is built from nodes of fields 1 to 11 by the adopted reading of Definition 6.5. By Definition 6.22 every valid block after the Heartwood activation block commits to the root of its epoch tree: directly in the Heartwood and Canopy upgrade epochs, and from NU5 as the first input of the block commitment hash.

6.8 The committed range

Proposition 6.23 (One-block delay). *For $\eta > A_H$ the tree to which B_η commits (Definition 6.22) has the leaves $B_{a(\eta)}, \dots, B_{\eta-1}$. No header commits to a tree that contains its own block, and $B_{\eta-1}$ first enters a committed tree at B_η . Consequently the final Sapling root of a block at a height of at least A_H is committed one block later, as the field `hashLatestSaplingRoot` of the root node committed by the next block. ZIP 221, “Non-FlyClient Commitments”, accepts the delay because “using the most recent treestate as a transaction anchor is already unsafe in reorgs”.*

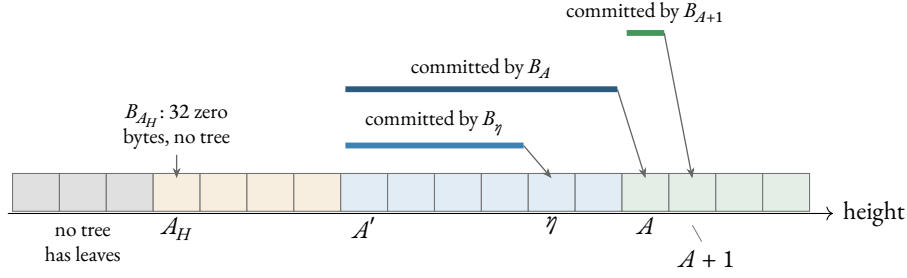


Figure 4. Committed ranges on a height axis, one cell per block, with the Heartwood activation height A_H and two later activation heights $A' < A$, consecutive, and $A' < \eta < A$. Grey: heights below A_H , a leaf of no epoch tree (Corollary 6.25). The Heartwood activation block carries the sentinel of Definition 6.12 and commits no tree. Each bar is the leaf range of the tree to which the header at the arrow's head commits: B_{η} commits $B_{A'}, \dots, B_{\eta-1}$; the activation block B_A commits the whole upgrade epoch $B_{A'}, \dots, B_{A-1}$ (Theorem 6.24(1)); and B_{A+1} commits the single leaf B_A (Theorem 6.24(2)). No bar reaches below A_H .

Proof. By Definition 6.2, η lies outside $[\alpha(\eta), \eta - 1]$. Since $\eta - 1 \geq A_H$ and $\alpha(\eta)$ is the largest activation height below η , $\alpha(\eta) \leq \eta - 1$, so $B_{\eta-1}$ is the last leaf of $\mathcal{T}_{\eta}$; it is no leaf of $\mathcal{T}_{\eta'}$ for $\eta' \leq \eta - 1$, whose leaves lie below η' . Then Definition 6.22. The latest-root field of the root node is that of its last leaf, by the parent rule of Definition 6.3. $\square$

Theorem 6.24 (Reset at each activation height). *Let $A > A_H$ be an activation height and $A' = \alpha(A)$ the preceding one. Then:*

1. B_A commits to the complete tree $\mathcal{T}_A$ over $B_{A'}, \dots, B_{A-1}$, the whole upgrade epoch beginning at A' , hashed under that upgrade epoch's branch ID $b(\mathcal{T}_A)$ by the adopted reading of Definition 6.9 and built with that upgrade epoch's field vector by the adopted reading of Definition 6.5; the commitment field equals $\text{hashChainHistoryRoot}(\mathcal{T}_A)$ before NU5, and $\text{hashBlockCommitments}(B_A)$ with it as first input from NU5;
2. B_{A+1} commits to the single-leaf tree over B_A ;
3. for $A' < \eta < A$, the leaves of $\mathcal{T}_{\eta}$ are a proper prefix of those of $\mathcal{T}_A$, so every ended upgrade epoch from Heartwood on is committed completely by exactly one header, that of the next activation block;
4. the reset occurs at every upgrade, including those that leave the field vector unchanged (Canopy, NU6, NU6.1, NU6.2 and NU7).

Proof. By the definition of α , $\alpha(A) = A'$ and $\alpha(A+1) = A$. For $A' < \eta < A$, $\alpha(\eta) = A'$, so $\mathcal{T}_{\eta}$ has the leaves $B_{A'}, \dots, B_{\eta-1}$, a proper prefix of those of $\mathcal{T}_A$; only $\mathcal{T}_A$ has the full leaf set, and for $\eta > A$ no tree has the leaf B_A , since $\alpha(\eta) \geq A$. The commitment fields follow from Definition 6.22. Part (4) holds because α depends on the activation heights alone. A script illustrates (1) to (3) on a toy chain, a one-block upgrade epoch included; it establishes nothing. $\square$

Figure 4 shows the ranges of the theorem.

Corollary 6.25 (No commitment below the Heartwood activation height). *The Heartwood activation block B_{A_H} commits to no tree, its history input being the sentinel of Definition 6.12. For $\eta > A_H$, $\alpha(\eta) \geq A_H$, so no committed tree has a leaf below A_H . Below A_H the commitment field holds a Sapling root or is reserved (Definition 6.21).*

Proof. The height A_H is an activation height below every $\eta > A_H$, and $\alpha(\eta)$ is the largest such; then Definition 6.22, item (ii). $\square$

Corollary 6.26 (The NU7 upgrade epoch). *Let A_{NU7} be the NU7 activation height. The block $B_{A_{\text{NU7}}}$ commits, as the first input of its block commitment hash, to the completed tree of the NU6.3 upgrade epoch, hashed under the NU6.3 branch ID $0x37A5165B$ by the adopted reading of Definition 6.9. The block $B_{A_{\text{NU7}}+1}$ commits to a new tree over $B_{A_{\text{NU7}}}$, hashed under the NU7 branch ID $0x77198AD9$ (ZIP 259, Draft) with the field vector of Definition 6.5 for the NU7 upgrade epoch.*

Proof. The instance $A = A_{\text{NU7}}$ of Theorem 6.24, NU6.3 being the upgrade that precedes NU7 (Definition 1.1), with the constants of ZIP 259, “NU7 network constants”, and ZIP 258, “NU6.3 deployment”. $\square$

7 The deployment boundary

This section classifies every component of a Zcash FlyClient in the classes of Definition 1.3. It opens with one table that places each hypothesis of Theorem 5.32 and each obligation of Remark 5.33 in its class, with the cited fact that places it there. The subsections that follow elaborate the rows: the specified commitment and the absence of a specified consumer, the differences between the paper’s commitment and the deployed one, the conditions that the ZIPs impose on any sample check of a Zcash verifier, and the open problems.

7.1 Classification of the security hypotheses

Remark 7.1 (Classification of the hypotheses). The classes of Definition 1.3 apply to components, not to properties. A property proved in this volume of a specified object is recorded as “the object is specified, and the property is proved here”, never as specified. Table 4 classifies every hypothesis of Theorem 5.32 and every obligation O1 to O6 of Remark 5.33, together with two components of the deployed commitment, the chain commitment (row (1)) and its composition across upgrade epochs (row (4)); O1 and O2 share row (3). Each row gives its class and the cited fact that places it there or, for an assumption on the client’s environment, the source that records it. Hypothesis 1 of the theorem fixes the parameters c , k and δ , whose values an instantiation chooses; rows (7) and (9) classify what those values rest on. The rows read as follows.

- (1) *Chain commitment*: specified. It is the epoch-scoped, metadata-bearing, branch-separated epoch tree $\mathcal{T}_\eta$ (Definition 6.2, Construction 6.10), to whose chain-history root every header after the Heartwood activation block commits under Definition 6.22, enforced by consensus (ZIPs 221 and 244; ZIP 258, Draft, whose rules are in force from NU6.3 by Definition 1.2).

- (2) *Collision resistance of the hash functions*: the functions are specified, namely personalised BLAKE2b-256 h for tree nodes, the chain-history root, the authorising-data root and the block commitment hash (ZIPs 221 and 244; protocol specification, §“BLAKE2 Hash Functions”), and SHA-256d for block hashes, hence for leaves (protocol specification, §“Block Header Encoding and Consensus”). Their collision resistance is a standard assumption (*Crypto Guide*, §“Security notions: preimage, second-preimage, and collision resistance”).
- (3) *O1, binding within one upgrade epoch*: the commitment is specified (Definition 6.22), and the property is proved here of the specified object (Proposition 6.13); no normative text fixes the property. *O2, fork structure for ZIP 221 trees*: open, since it is conditional on a sufficient sample check for those trees, which no specification or result supplies (Proposition 7.4); Lemmas 5.4 and 5.5 are stated for the paper’s construction only. Hypothesis 6 of Theorem 5.32, leaf–internal separation, enters only through Lemma 5.4 and shares the class of O2: it is stated for the paper’s construction, and for ZIP 221 trees no specification or result supplies its counterpart.
- (4) *Composition across upgrade epochs*: open. Each upgrade epoch has its own tree (Theorem 6.24), and ZIP 221 states no rule that chains them.
- (5) *The verifier’s anchor*, the genesis block in the paper: open (Corollary 6.25).
- (6) *O3, weight binding*: the node vector is specified (Definition 6.3), and the binding of the field-8 interval of a leaf to the root and its position within one upgrade epoch is proved here of the specified object (Lemma 3.21(1) with Proposition 6.13). The link from a sampled header to its leaf’s weight is conditional on the same sufficient sample check, and so open. Sampling by the chain’s total work is open, field 8 of a root summing one upgrade epoch (Definition 6.3; Proposition 7.8).
- (7) *Assumption 4.8, the transition checks of O3 (Construction 5.8) and Assumption 5.11 under the Zcash retarget rule*, in both of its regimes: open (protocol specification, §“Difficulty adjustment”; ZIP 218, Draft).
- (8) *Assumption 5.20*: open for Zcash; no analysis relates the work of individual blocks to the invalid segments of a Zcash fork.
- (9) *O4, catch probability and sample count*: designed but unspecified, and proved in the paper’s model under its hypotheses (Theorem 5.21; Proposition 5.22).
- (10) *The random-oracle hypothesis on the sampling hash*: open. No ZIP or specification section fixes a sampling hash for Zcash, so the hypothesis has no object; the model is that of the *Crypto Guide*, §“The random oracle model”.
- (11) *O5, grinding*: the transform of Construction 5.27 is designed but unspecified. Its Zcash transcript (header bytes, hash, domain separation, and map from output to weight points across upgrade epochs) and a derivation of the budget R of Assumption 5.28 are open.
- (12) *O6, size*: the bound of Corollary 5.34 is designed but unspecified; a Zcash proof format is open.

Table 4. The components of a Zcash FlyClient, covering every hypothesis of Theorem 5.32 and every obligation of Remark 5.33, with its class under Definition 1.3 and the result or normative text that places it there. Rows are numbered as in Remark 7.1, which states each in full.

	Hypothesis or obligation	Class	Placed by
(1)	chain commitment $\mathcal{T}_\eta$	specified	Definition 6.22; ZIPs 221, 244, 258
(2)	collision resistance of h and SHA-256d	functions specified; resistance assumed	ZIPs 221, 244; <i>Crypto Guide</i>
(3)	O1: binding within one upgrade epoch	commitment specified; property proved here	Proposition 6.13
	O2: fork structure for ZIP 221 trees; hypothesis 6 (leaf–internal separation)	open problem	Proposition 7.4; sufficiency not claimed
(4)	composition across upgrade epochs	open problem	Theorem 6.24; ZIP 221 silent
(5)	anchor of the verifier	open problem	Corollary 6.25
(6)	O3: weight binding	node vector specified; interval binding proved here; header-to-weight link and total work open	Lemma 3.21(1); Proposition 7.8
(7)	(c, L) , transition checks, transition rewriting	open problem	retarget rule of the specification; ZIP 218
(8)	discretisation	open problem	no analysis for Zcash
(9)	O4: catch probability, sample count	designed but unspecified	Theorem 5.21; Proposition 5.22
(10)	random oracle for sampling	open problem	no sampling hash specified
(11)	O5: grinding	transform designed but unspecified; transcript and R open	Construction 5.27; Assumption 5.28
(12)	O6: size	designed but unspecified; format open	Corollary 5.34
(13)	at least one honest prover	environment; no class	ZIP 221, “Security and Privacy Considerations”

- (13) *At least one honest prover*: an assumption on the client’s connectivity, not a component, so no class applies. ZIP 221, “Security and Privacy Considerations”, summarises the analysis of the paper, not the setting of ZIP 221 itself, as follows: “The security analysis assumes adversary mining power is bounded by a known fraction of combined mining power of honest nodes, and cannot drop or tamper with messages between client and full nodes. It also assumes the client is connected to at least one full node and knows the genesis block.” The bound on mining power is row (7); the absence of dropping or tampering and the connection to at least one full node are this row; knowledge of the genesis block is row (5).

Open problems (§7.5) derives rows (4), (5), (7), (11) and (12), and the total work of row (6), from cited rules; O2 of row (3), with the header-to-weight link of row (6), and rows (8) and (10) are open for the reasons stated above.

The verifier as a whole is designed but unspecified. The protocol of “The FlyClient protocol” (§5) exists with the paper’s theorem in the paper’s model; no ZIP specifies a Zcash instantiation;

and the paper’s model does not cover Zcash’s retarget rule. The paper proves security only for its model of retargeting once per retarget epoch (Definition 4.5), places rules that retarget from a moving average outside it, and credits them with heuristic guarantees only (ePrint 2019/226, Section 3.1, pages 7–8); it names Ethereum there and does not name Zcash. ZIP 221, “Security and Privacy Considerations”, names Zcash among the chains “with rapidly adjusting difficulty” that the paper leaves open.

7.2 The specified commitment and its consumers

Row (1) of Remark 7.1 is the following. ZIP 221 (Final) specifies the tree, with FlyClient as its stated motivation (ZIP 221, “Motivation”). ZIP 221, “FlyClient Requirements and Recommendations”, designates seven fields for reasoning about the difficulty adjustment over a range of blocks: $n_{\text{EarliestTimestamp}}$, $n_{\text{LatestTimestamp}}$, $n_{\text{EarliestTargetBits}}$, $n_{\text{LatestTargetBits}}$, $n_{\text{EarliestHeight}}$, $n_{\text{LatestHeight}}$ and $n_{\text{SubTreeTotalWork}}$ (Remark 6.4). The pool roots and transaction counts are its “Non-FlyClient Commitments”. No ZIP or specification section specifies a consumer of the tree, a proof object or a verifier. The specified class depends on no implementation: Definition 1.2 places the fields of ZIP 258 (Draft) in force from NU6.3.

Remark 7.2 (Recomputation against proof verification). The consensus rule of Definition 6.22 is an equality between the commitment field of a header and a value that the validator recomputes from the chain it holds: $\text{hashChainHistoryRoot}(\mathcal{T}_\eta)$ in the Heartwood and Canopy upgrade epochs, and the block commitment hash from NU5 (protocol specification, §“Block Header Encoding and Consensus”; ZIP 221, “Block header semantics and consensus rules”; ZIP 244, “Block Header Changes”). No ZIP or specification section defines an inclusion proof for the tree, a proof object or a verification algorithm. The operation that the paper’s verifier performs, Construction 5.2, is therefore unspecified for Zcash.

No specified light-client protocol consumes the commitment. ZIP 307 (Draft) does not reference ZIP 221, while ZIP 221, “Motivation”, defers the verification of transaction inclusion to ZIP 307: it “follows the usual reference protocol”.

7.3 The paper’s commitment and the deployed commitment

Remark 7.3 (Deltas from the paper’s commitment). Against Construction 5.1 and Definition 5.6, the deployed commitment differs in five respects.

- (1) *Scope*. It is epoch-scoped, not rooted at the genesis block (Theorem 6.24). The one-block delay of Proposition 6.23 is shared with the paper, whose header also commits to exactly the blocks strictly before it.
- (2) *Metadata*. The seven FlyClient fields serve the purpose of Definition 5.6 (ePrint 2019/226, Definition 7) without matching it field for field (Remark 6.4); the pool roots and counts have no counterpart.
- (3) *Hashing*. Every parent is hashed by the personalised, branch-separated BLAKE2b-256 $h(\text{ZcashHistory} \parallel \beta, \text{ser}(\ell) \parallel \text{ser}(r))$ over full serialisations, the root commitment $\text{hashChainHistoryRoot}$ covering the root’s aggregates, and every leaf carries the SHA-256d

block hash; the paper hashes with one generic collision-resistant H (Propositions 6.13 and 6.15).

- (4) *Placement*. The root enters through the repurposed commitment field (Definition 6.21), and from NU5 only as the first input of the block commitment hash (Proposition 6.19), whereas the paper adds a header field that can replace the previous block hash (ePrint 2019/226, Section 4.2, page 12).
- (5) *Bagging*. The deployed tree bags its peaks by the left fold Bag_L , the paper by the right fold Bag_R (ePrint 2019/226, Definition 11). Path tops, prefix folds and the maximal path length therefore differ whenever the leaf count has three or more set bits (Theorem 3.8; Lemma 3.7).

ZIP 221, “Tree Node specification”, states: “Other than the metadata commitments, the MMR tree’s construction is standard.” Items (1), (3), (4) and (5) record the respects in which the deployed construction also differs from the paper’s.

7.4 A Zcash verifier

The verifier is designed but unspecified (Remark 7.1). The conditions below are obligations that ZIPs 221 and 244 impose on any sample check for ZIP 221 trees; they are not a construction of this volume, and no Zcash verifier is specified here. A *sample check for ZIP 221 trees* is a deterministic algorithm that takes the header of a head, an index j and a prover’s message, and accepts or rejects; its purpose is to establish, for the head’s epoch tree, the conclusions of Proposition 5.3 and Lemma 5.7. A condition is *necessary* when some message that meets every other condition and violates it also violates such a conclusion, so that a check that does not reject that message fails the conclusion.

Proposition 7.4 (Necessary conditions of a Zcash sample check). *Let the head B_η , $\eta > A_H$, commit to $\mathcal{T}_\eta$ with $n = \eta - \alpha(\eta)$ leaves, let β encode $b(\mathcal{T}_\eta)$, and let a sample be leaf j of $\mathcal{T}_\eta$, $1 \leq j \leq n$, the block at height $\alpha(\eta) + j - 1$, with received header hdr_j . Each of the following conditions, the height clause of (ii) and the parsing clause of (iv) excepted, is necessary for a sample check for ZIP 221 trees. Sufficiency is not claimed. The height clause of (ii) is required for the leaf’s stated heights to match its position (Lemma 6.6); parsing under the field vector is the hypothesis under which Proposition 6.13 and Lemma 3.20(2) apply; the necessity of neither is claimed.*

- (i) *Field 1 of the leaf equals $\text{BlockHash}(\text{hdr}_j)$, the SHA-256d hash of the full received header, Equihash solution included (Lemma 6.7).*
- (ii) *Fields 2 and 3 of the leaf equal the field $n\text{Time}$ of hdr_j , fields 4 and 5 its field $n\text{Bits}$; field 8 equals the work $\lfloor 2^{256} / (\text{ToTarget}(n\text{Bits}) + 1) \rfloor$ of that $n\text{Bits}$; and fields 9 and 10 equal $\alpha(\eta) + j - 1$, the height that the position fixes, the header carrying no height (Definition 2.3).*
- (iii) *The header hdr_j satisfies proof of work in the sense of Definition 2.3: a valid Equihash solution, and the difficulty filter at the target $\text{ToTarget}(n\text{Bits})$ it declares.*

- (iv) The sampled leaf of (i) and (ii) and the path siblings are received as full node serialisations, parsed under the field vector of Definition 6.5 for the upgrade epoch beginning at $\alpha(\eta)$. Starting from that leaf, every parent is recomputed: field 1 as $h(\text{ZcashHistory} \parallel \beta, \text{ser}(\ell) \parallel \text{ser}(r))$ (Construction 6.10), the other fields by the parent rule of Definition 6.3. The recomputed root node yields $\text{hashChainHistoryRoot}(\mathcal{T}_\eta)$, which is compared with the head's commitment field under Definition 6.22: directly in the Heartwood and Canopy upgrade epochs, and from NU5 through Proposition 6.19 with the head's hashAuthDataRoot .
- (v) The sides and the length of the path are those that j and n fix under Bag_L , the length being $\text{depth}_L(n, j)$ (Lemma 3.7).
- (vi) For $j \geq 2$: the root node ρ output by $\text{PrefixRoot}_L(n, j, \pi)$ (Construction 3.16), run on the path π with the parent rule in place of $H(\cdot \parallel \cdot)$, is the claimed root node of $\mathcal{T}_{\alpha(\eta)+j-1}$, and the value

$$r = h(\text{ZcashHistory} \parallel \beta, \text{ser}(\rho))$$

is accepted against the commitment field of hdr_j under Definition 6.22: r equals that field in the Heartwood and Canopy upgrade epochs, and from NU5 r is accepted by Proposition 6.19 with the hashAuthDataRoot of the sampled block.

Proof. For each condition, a message that meets the other five and violates it also violates the conclusion named.

- (i) An honest header with valid proof of work, paired with a leaf whose field 1 is the hash of a different, unmined block of the claimed tree. The proof of work checked is not that of the committed block, contrary to Proposition 5.3(i) and Lemma 5.7(1).
- (ii) An honest header paired with a leaf of inflated field 8, in a tree that the adversary builds over such leaves under a head that it mines, so that every path verifies. The header-determined fields are not those of the header, contrary to Proposition 5.3(ii), and the interval that Lemma 3.21(1) binds belongs to a forged weight.
- (iii) A tree built over fabricated headers without valid solutions. Every other condition is met by construction, and unmined headers pass, contrary to Lemma 5.7(1).
- (iv) If a node is taken in any other form, the fields that do not enter the recomputation are unbound. With the siblings supplied as field-1 digests and the path checked on field 1 alone, the siblings' field 8, which fixes the interval of Lemma 3.21, and the leaf's pool fields (6, 7 and, where the vector carries them, 11 to 17), which its header does not determine, are chosen freely; two passing samples at j then carry distinct leaf tuples, contrary to Proposition 5.3(i). If the recomputation starts from a node other than the leaf matched in (i) and (ii), the matched header is bound to nothing. Proposition 6.13 binds full serialisations only.
- (v) The single-block tuple of a leaf placed at a position where the shape $\mathcal{M}_n^L$ has an internal node: a tree with fewer than n leaves then passes, and a tuple committed at another depth is

accepted as leaf j . Theorem 3.17, which assumes the shape fixed by n and the bagging order, does not apply.

- (vi) Honest blocks, mined on the honest chain after the fork point, appended as leaves of the fork's tree. They pass (i) to (v), while their own commitments bind the honest prefix, not the fork's first $j - 1$ leaves, contrary to Proposition 5.3(iii). This is the case that Lemma 5.4 excludes by the prefix check.

No condition is shown sufficient. □

The conditions are the ZIP 221 counterparts of the checks of Construction 5.2: (i) and (ii) of check (a), (iii) of check (d), (iv) and (v) of check (b), and (vi) of check (c). Three further statements bound their reach. For $j = 1$ the sample is the block $B_{\alpha(\eta)}$. For $\alpha(\eta) > A_H$ it is an activation block, which commits to the completed tree of the previous upgrade epoch (Theorem 6.24); for $\alpha(\eta) = A_H$ it carries the sentinel of Definition 6.12 (Corollary 6.25). In both cases its prefix check belongs to epoch composition, which is open. The consensus condition $n\text{Bits} = \text{ThresholdBits}(\alpha(\eta) + j - 1)$ of Definition 2.3 depends on preceding headers and is not decided by one sample; it is the object of the transition checks of row (7), which are open. And condition (iv) makes Proposition 6.13 apply, but the fork structure of row (3) additionally needs a sufficient check, which is open.

Example 7.5 (Per-sample size). The size is evaluated at the scale of one upgrade epoch, since n is the leaf count of one epoch tree and never the length of the chain. The message of Proposition 7.4 for one sample consists of the 1,487-byte header (§2.1), the sampled leaf's own serialisation of at most 317 bytes (Definition 6.5), at most $\lfloor \log_2 n \rfloor + \text{popcount}(n) - 1$ siblings of at most 317 bytes each (Lemma 3.7), and the 32 bytes of the sampled block's `hashAuthDataRoot`. For $n = 2^{20}$ a path has at most 20 siblings, and a sample at most

$$1,487 + 317 + 20 \cdot 317 + 32 = 8,176 \text{ bytes,}$$

of which the header is 18.2 per cent. For $n = 2^{20} - 1$, with 20 peaks, a path has at most 38 siblings, and a sample at most

$$1,487 + 317 + 38 \cdot 317 + 32 = 13,882 \text{ bytes,}$$

of which the header is 10.7 per cent. The leaf's fields fixed by its header and its position, 1 to 5 and 8 to 10, could be omitted by a proof format, which is open. The figures are computed by script.

7.5 Open problems

The following open items, each a row of Remark 7.1 except the serving interface, are derived below from cited rules: epoch composition, the anchor below the Heartwood activation height, total work, the proof format, the serving interface, the Fiat–Shamir transcript, and the security analysis under Zcash's retarget rule with its NU7 regime. Composition with compact-block scanning follows. Availability of prover data is not a protocol question.

Remark 7.6 (Epoch composition). Epoch trees are hashed under distinct branch IDs, and each complete root is committed only by the next activation header (Theorem 6.24). No specified rule

chains them, and ZIP 221, “Specification”, is silent. Composition is needed for prefix proofs and for difficulty checks: the averaging window and the median-time span of the first blocks of an upgrade epoch reach into the previous upgrade epoch (protocol specification, §“Difficulty adjustment”), so a transition check at an upgrade boundary needs two epoch trees. Proposition 5.38 needs an inclusion path of the accepted head against the later head’s root, and so applies within one epoch tree. Across epoch trees it does not apply: no tree of a later upgrade epoch contains a leaf of an earlier one (Definition 6.2), and only the activation header commits to the completed earlier tree. No design exists: the item is open.

Remark 7.7 (Anchor). The anchor below the Heartwood activation height is open, by Corollary 6.25.

Proposition 7.8 (No committed node aggregates total work). *Let $\eta > A_H$.*

- (a) *If SHA-256d is collision resistant, the header of B_η commits through hashPrevBlock to the headers of $B_0, \dots, B_{\eta-1}$, hence to their fields nBits and to the total work ω of that chain (§2.1; the model’s counterpart is Definition 4.2).*
- (b) *For every node v of $\mathcal{T}_\eta$, field 8 is the sum modulo 2^{256} of the work of the blocks at heights $v.\text{nEarliestHeight}$ to $v.\text{nLatestHeight}$, all in $[\alpha(\eta), \eta - 1]$; field 8 of no node of $\mathcal{T}_\eta$ depends on $B_0, \dots, B_{\alpha(\eta)-1}$.*
- (c) *Hence no inclusion path against the root to which B_η commits opens ω . Sampling by the chain’s cumulative work (Definition 5.19) and the acceptance rule need field 8 of the root of every completed epoch tree $\mathcal{T}_A$, $A_H < A \leq \alpha(\eta)$ an activation height, each committed only by B_A (Theorem 6.24), together with the work of the blocks below A_H , which no tree commits (Corollary 6.25).*

No ZIP addresses the item; it is open.

Proof. (a) The block hash covers the whole header, nBits and hashPrevBlock included (Definition 2.3); induction down the chain, a divergence yielding a collision of SHA-256d. (b) Field 8 of a leaf is its block’s work and that of an internal node the sum of its children’s (Definition 6.3); the leaves of $\mathcal{T}_\eta$ are $B_{\alpha(\eta)}, \dots, B_{\eta-1}$ (Definition 6.2); the height range of v is that of Lemma 6.6. (c) A path opens only fields of nodes of $\mathcal{T}_\eta$, and by (b) the only work-valued field, field 8, sums work over leaves of $\mathcal{T}_\eta$; the pool-root fields depend on earlier blocks, but through treestates, from which work is not computed. The work of each block is at least 1, since $\text{ToTarget}(\text{nBits}) + 1 \leq 2^{256}$, so ω exceeds the work of the leaves of $\mathcal{T}_\eta$ whenever $\alpha(\eta) > 0$. The remaining work is committed only as stated, by Theorem 6.24 and Corollary 6.25. ZIP 221, “Tree Node specification”, assumes that cumulative chain work stays below 2^{256} , which makes each modular sum exact (Lemma 3.21(2)). $\square$

Part (b) does not contradict the statement of (a) that every header commits to the total work: that commitment is through the linkage of every header, which a sampling verifier does not receive.

Remark 7.9 (Proof format). The proof format is open; no ZIP or specification section defines one. From NU5 any format carries, for every header whose commitment it opens, the 32-byte `hashAuthDataRoot` needed to reopen the block commitment hash (Proposition 6.19).

Remark 7.10 (Serving interface). The serving interface is open: no ZIP or specification section defines a message for requesting or serving tree nodes, inclusion paths or assembled proofs. Availability of prover data is not a protocol question. Every node, and hence every inclusion path, of T_η is a function of the chain up to its last leaf $B_{\eta-1}$, not of the blocks of the upgrade epoch alone, since the pool-root fields (6, 7, 12, 13, 15 and 16) are roots of final treestates, which depend on every note commitment since the activation of the pool (Definition 6.3; Construction 6.10; ZIP 221, “Tree Node specification”; ZIP 258, “Changes to ZIP 221”). Any party that holds that chain computes them; what is unspecified is the message.

Remark 7.11 (Transcript). The transcript is open. Construction 5.27 derives the sampling randomness from a hash of the head (ePrint 2019/226, Section 6.2). For Zcash no ZIP or specification section fixes the header bytes, the hash, the domain separation or the map from output to weight points across upgrade epochs, so Proposition 5.29 has no Zcash instance.

Remark 7.12 (Security analysis). The security analysis is open. Zcash retargets after every block from an averaging window with median-time spans, damping and asymmetric clamps, which is not Definition 4.5. The blocks of the upgrade epochs from Heartwood through NU6.3 are governed by a 17-block window at a 75-second target spacing (protocol specification, §“Difficulty adjustment”); NU7 as specified uses 102 blocks at 25 seconds (ZIP 218, Draft, “Averaging window”; *Consensus Guide*, §“Difficulty in motion”). Neither regime is covered by Theorem 5.32, and the paper contains no derivation of (c, L) from either (Remark 4.13). The published analysis of a Zcash instantiation (Perazzo and Capecchi, arXiv 2604.26736, Section IV-A, Definition IV.1) assumes its adversarial work budget rather than deriving it from the retarget rule, so it does not close the gap; it is recorded here because the open classification must account for it. ZIP 221, “Security and Privacy Considerations”, records that the paper leaves such chains with only heuristic guarantees, that “The use of these fields has not been analysed in the academic security literature”, and that “A more in-depth security analysis of FlyClient should be performed before designing a FlyClient-based light client ecosystem for Zcash”. No ZIP addresses the gap.

Remark 7.13 (NU7). ZIP 218 (Draft) changes the target spacing from 75 to 25 seconds, the averaging window from 17 to 102 blocks and the expected stale rate (ZIP 218, “Block target spacing”, “Averaging window”, “Stale block rate”). These bear on the (c, L) that an instantiation could assume, not on Theorem 5.21, which is stated in c and δ . Transition checks must use the height-dependent spacing and window, span two regimes across one more upgrade boundary (Corollary 6.26), and meet first adjustments that ZIP 218, “Effect on difficulty adjustment”, expects to be limited by `PoWMaxAdjustDown` (*Consensus Guide*, §“NU7: 25-second blocks and bounded shielded work”).

Remark 7.14 (Compact-block scanning). Composition with compact-block scanning is open. A FlyClient replaces trust in the header chain, not in the channel through which a wallet detects its transactions (ZIP 307, Draft).

VIII

ZSA Guide

Zcash Shielded Assets: asset identity, issuance, transfer, and burn

Abstract

This volume of *The Zcash Arboretum* describes the OrchardZSA protocol of ZIP 226 and ZIP 227, both with status Draft, as a delta on the Orchard protocol that the *Ironwood Guide* constructs. The protocol extends the Orchard protocol by the issuance, transfer and burn of Custom Assets. The volume constructs, in dependency order, the identity of an Asset; the note, its commitment and its value commitment, extended by the Asset Base; the Action statement and the Action Group; burn and the binding validating key; issuance and the global issuance state; and the status of the transaction carrier. It proves no counterfeiting across Assets, supply integrity, issuance authority and privacy of the Asset type under named assumptions. NU7, specified by ZIP 259 (Draft), introduces no transaction format and includes neither ZIP, so no network upgrade includes the protocol; its transaction format survives only in withdrawn text, and every construction is therefore classified as specified, designed but unspecified, or an open problem. The volume assumes the *Math Guide*; the *Crypto Guide* for signatures (§“Digital signatures”), Pedersen commitments (§“Commitment schemes”, §“The Pedersen commitment”) and random oracles (§“Hash functions and the random oracle model”); the *Halo 2 Guide* for the proof system; and the *Ironwood Guide* for the Orchard protocol. It is non-normative: the Zcash Protocol Specification, cited as the protocol specification with the title of a section, and the Zcash Improvement Proposals are authoritative.

1 Introduction

This section fixes the subject of the volume, its status and its vocabulary before any construction: the sources and their status, and the notation (§1.1); the three epistemic classes, the status of a result proved here and the carrier of a construction (§1.2); the Assets, the OrchardZSA protocol and the premise on the pool that carries its notes (§1.3); and the requirements that a protocol for shielded assets must meet, each paired with the result of this volume that discharges it (§1.4).

1.1 Scope and status

This volume of *The Zcash Arboretum* describes the OrchardZSA protocol, defined in §1.3, as a delta on the Orchard protocol that the *Ironwood Guide* constructs. Every construction of the volume modifies an object of that volume, or adds an object beside it, and the objects of the Orchard protocol are cited by the *Ironwood Guide*’s titles, never re-derived: §“Notes and note commitments”, with Definition “Note” and Definition “Note commitment”; §“The note seed: Ironwood derivations (lead byte 0x03)”; §“Actions, bundles, and dummy notes”, with Definition “Dummy notes”; §“Value commitments and the binding signature”, with §“Value commitments” (Definition “Net value commitment”), §“The balancing value” and §“The binding signature”; §“The Action statement and its proof”, with §“The Action statement” (Definition “Orchard Action statement”); §“Nullifiers”; §“Sinsemilla: hash, commitment, and short forms”; §“GroupHash, domain separation, and nothing-up-my-sleeve generators”; §“The Orchard protocol and its two pools”; §“The transaction format”; §“Value conservation”, with Theorem “Balance”; §“Fields, groups, and encodings”; and §“Requirements on a shielded payment”. The volume further assumes the *Math Guide*; the *Crypto Guide* for digital signatures (§“Digital signatures”), Pedersen commitments (§“Commitment schemes”, §“The Pedersen commitment”) and random oracles (§“Hash functions and the random oracle model”); and the *Halo 2 Guide* for the proof system. No other volume is cited. The volume is non-normative: the Zcash Protocol Specification, cited as the protocol specification with the title of a section, and the Zcash Improvement Proposals are authoritative.

Sources. Two ZIPs specify the protocol. ZIP 227, “Issuance of Zcash Shielded Assets”, specifies the issuance of Custom Assets; ZIP 226, “Transfer and Burn of Zcash Shielded Assets”, specifies their transfer and burn. Both have status Draft and category Consensus (ZIP 226 and ZIP 227, preambles). ZIP 226 is written relative to the protocol with ZIP 2005 applied (ZIP 226, Abstract), and calls ZIP 2005 “Orchard Quantum Recoverability”; the current title of ZIP 2005 is “Ironwood Quantum Recoverability”, and its status is Proposed (ZIP 2005, preamble). Every construction of the OrchardZSA protocol is therefore fixed, where it is fixed at all, by Draft text, and none belongs to the consensus rules of any network upgrade. The class of each construction (§1.2) is stated once, where the construction is made.

Definition 1.1 (ZIP statuses). The status field of a ZIP takes one of the values that ZIP 0, “ZIP Status Field”, fixes; a ZIP with several revisions carries one status per revision. This volume uses five of them.

- *Draft*: the status of every ZIP on submission. ZIP 226, ZIP 227, ZIP 229, ZIP 258 and ZIP 259 have it.
- *Proposed*: typically the stage after Draft, set after consideration, feedback and rough consensus of the community. ZIP 2005 has it.
- *Active*: typically used for Process or Informational ZIPs, set once rough consensus on a Proposed ZIP is reached. Revision 0 of ZIP 317 has it.
- *Final*: the status of a Consensus or Standards ZIP that is both implemented and activated on the Zcash network.

- *Withdrawn*: the owners of the ZIP have removed it from consideration by the community. ZIP 230 and ZIP 246 have it.

A ZIP text is *live* when its status, or the status of the revision concerned, is Draft, Proposed, Active or Final. Every published ZIP that this volume cites is either live or Withdrawn. The *Ironwood Guide*’s Definition “ZIP” names only the statuses Draft and Reserved; Reserved is not used here.

Status with respect to network upgrades. NU6.3 is the active network upgrade, deployed by ZIP 258, and NU7 is specified by ZIP 259 (Draft). The sources of the consensus changes of NU7 are the protocol specification, ZIPs 200, 218 and 2003, and ZIP 237, which ZIP 259 names but which is unpublished; none of these is ZIP 226 or ZIP 227; NU7 introduces no new transaction format, and version-5 and version-6 transactions remain valid (ZIP 259, Abstract and Specification, “NU7 consensus changes”). ZIP 258 does not mention Zcash Shielded Assets (ZIP 258, Specification). ZIP 226, “Deployment”, still names NU7 as the network upgrade that deploys the protocol, which ZIP 259 contradicts, and ZIP 227, “Deployment”, reads “TBD”. The assignment of the OrchardZSA protocol to a network upgrade is therefore an open problem in the sense of Definition 1.2, stated in §1.2.

Notation. The notation is that of the *Ironwood Guide*, whose table in §“Fields, groups, and encodings” maps it to the names of the protocol specification. Table 1 extends that map to the names that ZIP 226 and ZIP 227 use and that the table of the *Ironwood Guide* does not list. The names of the ZIPs appear in that table only. The table lists no symbol that this volume defines: every such symbol, among them the Asset Base, the Asset Identifier and Asset Digest, the OrchardZSA note and value commitments, the split flag, the pause flag, the issuance keys, the global issuance state and the burn set, is defined at its first use. The same convention applies to the generator L^{Orchard} of Construction 4.4, which ZIP 226 writes calligraphically.

Table 1. ZIP notation and series notation. The set $\mathcal{E}^*$ is that of the *Ironwood Guide*’s Definition “Orchard Action statement”; the star encoding is its Definition “Star encoding” and the group hash its Construction “GroupHash on Pallas”; the three generators are those of its §“Value commitments” and §“Nullifiers”. The value $\ell_p = 256$ is fixed in the protocol specification, §“Pallas and Vesta”.

ZIP notation	This volume	Meaning
$\mathbb{P}$	$\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, also $\mathcal{E}$	the Pallas group
$\mathbb{P}^*$	$\mathcal{E}^* := \mathcal{E} \setminus \{\mathcal{O}\}$	the Pallas group without its identity
$q_{\mathbb{P}}$	p_{Pallas}	the Pallas base-field modulus
$\sharp_{\mathbb{P}}$	p_{Vesta}	the order of the Pallas group
$\ell_{\mathbb{P}}$	256	the bit length of the star encoding
l2LEBSP_k	LE_k	k -bit little-endian encoding
$\text{repr}_{\mathbb{P}}(P)$	P^*	the star encoding of $P \in \mathcal{E}$
$\text{GroupHash}^{\mathbb{P}}$	GroupHash	the group hash into $\mathcal{E}$
$\mathcal{V}^{\text{Orchard}}, \mathcal{R}^{\text{Orchard}}, \mathcal{K}^{\text{Orchard}}$	$\mathcal{V}^{\text{Orchard}}, \mathcal{R}^{\text{Orchard}}, \mathcal{K}^{\text{Orchard}}$	fixed generators of the Orchard protocol

Prose uses British spelling, as in finalisation and authorisation. Identifiers and titles taken from the ZIPs keep their own spelling, as in the flag finalize and the title “Issuance Authorization Signature Scheme” of ZIP 227.

1.2 Epistemic classes

Definition 1.2 (Three epistemic classes). Each construction of this volume is in exactly one of three classes.

- (i) It is *specified* when normative text of the protocol specification or of a live ZIP (Definition 1.1), Draft included, fixes it.
- (ii) It is *designed but unspecified* when no live normative text and no published proof fixes it, but a complete design of it exists, in withdrawn ZIP text, in text removed from a ZIP, in a document outside the ZIPs, or as an argument of a ZIP’s rationale without proof, and that design conflicts with no live text.
- (iii) It is an *open problem* when the protocol needs it and either no text, live, withdrawn or removed, supplies a design of it; or every existing design conflicts with live text, in that a value or position that the design assigns is assigned otherwise by live text; or live texts contradict one another about it.

The *Ironwood Guide*’s Definition “Epistemic classes” uses the first two names for claims about the deployed protocol; here they are redefined for constructions, and the third is added.

Each construction is classified where it is made. Among the designed but unspecified constructions are the OrchardZSA circuit (§4.4), the derivation of the commitment trapdoor of an OrchardZSA note (§3.3) and the fee terms (§8.3). Among the open problems are the pool that carries OrchardZSA notes (Remark 1.7); the live carrier, whose withdrawn version 6 is now the format of ZIP 229, which need not support Zcash Shielded Assets (ZIP 229, Non-requirements); the bit of the pause flag enableZSA, whose position in withdrawn ZIP 230, bit 2 of the flags byte, is that of enableCrossAddress in ZIP 229 (§8.1); and the value of the lead byte, whose removed value 0x03 is the Ironwood lead byte (§3.3).

Definition 1.3 (Status of a result proved in this volume). A theorem, proposition, lemma or corollary of this volume is proved here for the constructions as the ZIPs state them, and carries no class: the classes of Definition 1.2 apply to constructions and to the rationale arguments of the ZIPs. The hypotheses of a result are named in its statement. A hypothesis that concerns a designed but unspecified object, or an open problem (Definition 1.2(iii)), makes the result *conditional* on that object:

- on the OrchardZSA circuit, through Assumption 4.9 and Assumption 4.10 of “The OrchardZSA Action proof” (§4.4);
- on the signed message, through Assumption 6.13 of “Issuance authorisation” (§6.5);
- on an open problem, through a hypothesis that names it: Lemma 7.6, Proposition 6.11(a) and Theorem 9.11.

A result about notes created by issuance, such as the provenance of Asset Bases or supply integrity, holds for every deterministic derivation of the commitment trapdoor *rcm* from the

published fields of such a note. It depends on the designed but unspecified derivation of *rcm* only in that some such derivation must be fixed for a validator to compute the commitment. Where a ZIP argues the same property in its rationale only, that argument remains designed but unspecified; a proof in this volume does not change its class.

Every construction states its class where it is made, and every result the premises on which it is conditional. “Classification of the components” (§10) tabulates both.

Definition 1.4 (Carrier of a construction). The *carrier* of a construction is the set of transaction fields that encode it, together with the digests that commit to it: the transaction identifier, the authorising-data digest and the signature digest, the three digests of the *Ironwood Guide*, §“Transaction digests and signatures”. A construction can be specified while its carrier is not.

1.3 The OrchardZSA protocol

Definition 1.5 (Asset, Custom Asset, Native Asset). An *Asset* is a type of note that can be transferred on the Zcash block chain. Each Asset is identified by an Asset Identifier (ZIP 227, Terminology). ZEC is the default, and currently the only defined, Asset on Mainnet, and TAZ the default, and currently the only defined, Asset on Testnet; each is the *Native Asset* of its network. A *Custom Asset* is an Asset other than ZEC and TAZ (ZIP 227, Terminology; ZIP 226, “Burn Mechanism”). The Asset Identifier of a Custom Asset is constructed in “Asset descriptions and Asset Identifiers” (§2.2); no ZIP constructs one for the Native Asset, whose Asset Base is fixed directly (Definition 2.15). In this volume *issuance* means the issuance of a Custom Asset, in the sense of ZIP 227. It is distinct from the creation of ZEC by the block subsidy in a coinbase transaction (*Ironwood Guide*, Definition “Coinbase transaction”), which the OrchardZSA protocol does not modify.

Definition 1.6 (OrchardZSA protocol). The *OrchardZSA protocol* is the extension of the Orchard protocol that ZIP 226 and ZIP 227 define jointly: ZIP 227 specifies the issuance of Custom Assets, and ZIP 226 their transfer and burn. ZIP 227 is to be implemented only together with ZIP 226, because the notes that issuance creates can be transferred only by the OrchardZSA transfer protocol (ZIP 227, Abstract, where “must” is lower-case and is not a key word of BCP 14). Relative to the Orchard protocol it makes the following changes, each constructed in the section named with it.

- (1) Each note gains one field, a point of the Pallas group that names its Asset (“Notes and note commitments”, §3).
- (2) The note commitment becomes a case split that leaves the commitments of ZEC notes unchanged (same section).
- (3) The net value commitment takes that point as its value base, in place of the fixed base V^{Orchard} (same section).
- (4) The binding validating key further subtracts the burnt amounts (“Burn and the binding signature”, §5).

- (5) An Action of a Custom Asset that consumes no note pads its consumed side with a copy of a note already committed, of the same Asset and counted at value 0, instead of a dummy note (“OrchardZSA Actions and Action Groups”, §4).
- (6) The Action statement changes accordingly (same section).
- (7) ZIP 227 adds issuance and a global issuance state (“Issuance of Custom Assets”, §6; “The global issuance state”, §7).

The sources of these changes are ZIP 227, Abstract, and ZIP 226, Abstract and Specification, with “Split Notes”, “Value Balance Verification” and “Backward Compatibility”.

Remark 1.7 (The Orchard-pool premise). ZIP 226 places OrchardZSA notes in the Orchard pool, which the Orchard and the OrchardZSA protocols then share, and adds their note commitments to the same note commitment tree (ZIP 226, “Privacy Implications”, “Backward Compatibility” and “Rationale for Note Commitment”). It also assumes ZIP 2005 applied (ZIP 226, Abstract). ZIP 2005 applies its recoverable note plaintext format, of lead byte $0x03$, only to notes of the Ironwood pool (ZIP 258, “ZIP 2005 activation”), and note plaintexts of the Orchard pool take the lead byte $0x02$ only (protocol specification, §“Note Plaintexts and Memo Fields”). Since NU6.3 the Orchard pool is sealed (ZIP 258, “Consensus rules from NU6.3 activation”; *Ironwood Guide*, §“The Orchard protocol and its two pools”):

- (a) a coinbase transaction has no Orchard-pool Actions;
- (b) every Orchard-pool Action creates its note at the address of the note it consumes, cross-address transfers being disabled;
- (c) the balancing value of the Orchard pool is non-negative.

Read literally, these rules have consequences that no ZIP states. Rule (b) excludes, for a Custom Asset, every created note at an address other than that of the note which the same Action consumes or copies; it thus excludes a transfer of a Custom Asset to such an address, and every padding copy at an address other than that of its Action’s created note. Rule (c) is stated for ZEC only, and leaves unaddressed the entry of issued value into a pool that admits no inflow. ZIP 226 and ZIP 227 have not been revised for the pool rules of NU6.3 (ZIP 226 has only given up lead byte $0x03$), ZIP 258 does not mention them, and no ZIP places OrchardZSA notes in the Ironwood pool: ZIP 2005 treats the extension as undeployed (“if that were deployed”, Terminology) and requires it to take a new lead byte (“Usage with other proposals requiring note plaintext format changes”).

Class: which pool carries OrchardZSA notes is an open problem (Definition 1.2). Live texts contradict one another about it: ZIP 258 states that no new value may enter the Orchard pool, while ZIP 227 adds the commitments of issued notes to the note commitment tree (ZIP 227, “Addition to the Note Commitment Tree”) that ZIP 226 places in that pool, and no live text places the notes elsewhere. The later sections state each construction as ZIP 226 and ZIP 227 state it, and recall this remark where a consequence falls: at Split Inputs, at reference notes and at the order of the note commitment tree.

1.4 Requirements on shielded assets

Definition 1.8 (Requirements Z1 to Z5). A protocol for shielded assets meets the following requirements. Each refines one of the *Ironwood Guide*’s Definition “Requirements R1 to R5”, or is new (ZIP 227, Requirements; ZIP 226, “Privacy Implications”, “Rationale for Value Commitment” and “Backward Compatibility”).

- Z1. *Per-Asset conservation*. Requirement R4 holds Asset by Asset: in every valid bundle, for each Asset, the value consumed equals the value created plus the value that leaves the pool, burnt value counted as leaving.
- Z2. *No counterfeiting across Assets*. No transaction creates value of one Asset from value of another Asset, or from nothing; value of a Custom Asset enters only by issuance.
- Z3. *Supply integrity*. The amount of a Custom Asset in circulation equals the amount issued minus the amount burnt, can be tracked publicly, and stops growing once its issuer finalises the Asset.
- Z4. *Issuance authority*. Only the holder of an issuer’s key issues or finalises that issuer’s Assets.
- Z5. *Hiding of the Asset type*. A shielded transfer reveals which Assets it moves no more than burn and issuance publish. This refines the hiding parts of R1 and R4.

Two design requirements are recorded with Z2. First, the identification of an Asset should be unique among all shielded pools, and different issuer keys should not produce the same Asset Identifier (ZIP 227, Requirements, where “should” is lower-case). Second, the identity of an Asset maps to a point of the Pallas group that serves as a private value base inside the Action proof (ZIP 226, Overview, “Rationale for Value Commitment” and “Value Commitment Correctness”).

Table 2 names, for each requirement, the result of this volume that discharges it. The checks that serve each requirement are mapped in Table 6 of “Verification of an OrchardZSA transaction” (§7.4), where all of them are defined.

Table 2. Requirements and the results that discharge them. Each result states its assumptions and the premises on which it is conditional (Definition 1.3).

Requirement	Result
Z1, Z2	Theorem 9.5, “No counterfeiting across Assets”
Z3	Theorem 9.7, “Supply integrity”; Proposition 7.8, “Finalised supply never grows”
Z4	Theorem 9.8, “Issuance authority”
Z5	Theorem 9.11, “Privacy of the Asset type”

2 Asset identity

This section constructs the identity of an Asset in dependency order: the issuance key pair, the issuance authorisation signature scheme and the issuer identifier (§2.1); the asset description and the Asset Identifier (§2.2); the Asset Digest, the Asset Base and its byte encoding (§2.3); and the Asset Base of the Native Asset (§2.4). Every construction of the section is taken from ZIP 227 and ZIP 226, both Draft. The results of the section separate Assets computationally: distinct pairs of issuer and description yield distinct Asset Identifiers and distinct Asset Bases, and no Asset Base of a Custom Asset coincides with a fixed generator of the Orchard protocol, except with negligible probability.

Throughout, $\mathbb{B}^{\mathbb{Y}[k]}$ denotes the set of byte strings of length k and $\mathbb{B}^{\mathbb{Y}[N]}$ the set of byte strings of arbitrary length; $\parallel$ denotes concatenation. For a 16-byte personalisation P and a byte string x , $\text{BLAKE2b-256}(P, x)$ and $\text{BLAKE2b-512}(P, x)$ denote unkeyed BLAKE2b with personalisation P , input x and an output of 32 and 64 bytes respectively (protocol specification, §“BLAKE2 Hash Functions”). The former is the function h of the *Ironwood Guide*, §“Transaction digests and signatures”.

2.1 Issuance keys and the issuer identifier

Definition 2.1 (Issuance key pair). The *issuance authorising key* is a byte string $\text{isk} \in \mathbb{B}^{\mathbb{Y}[32]}$. The *issuance validating key* is $\text{ik} := \text{DerivePublic}(\text{isk})$, with DerivePublic the algorithm of Definition 2.2; it lies in $\mathbb{B}^{\mathbb{Y}[32]} \cup \{\perp\}$, and a key pair is usable only when $\text{ik} \neq \perp$. The key isk authorises the issuance of Asset Identifiers by one issuer and is used only by that issuer; the key ik validates those issuances. Neither key is used for spend authorisation, which remains with the Orchard spend authorising key (ZIP 227, “Issuance Keys” and “Rationale”). Class: specified.

Definition 2.2 (Issuance authorisation signature scheme). The *issuance authorisation signature scheme* IssueAuthSig is a digital signature scheme in the sense of the *Crypto Guide*’s Definition “Digital signature scheme” (protocol specification, §“Signature”), instantiated as BIP-340 Schnorr signatures over secp256k1 with the standard secp256k1 parameters (*Crypto Guide*, §“‘The transparent layer’s schemes: ECDSA and BIP-340 over secp256k1 ”, Construction “BIP-340 Schnorr over secp256k1 , schematically”; cited, not re-derived). Let n be the order of the secp256k1 group. The types are

$$\begin{aligned} \text{Message} &= \mathbb{B}^{\mathbb{Y}[N]}, & \text{Signature} &= \mathbb{B}^{\mathbb{Y}[64]} \cup \{\perp\}, \\ \text{Public} &= \mathbb{B}^{\mathbb{Y}[32]} \cup \{\perp\}, & \text{Private} &= \mathbb{B}^{\mathbb{Y}[32]}, \end{aligned}$$

a public key being the 32-byte big-endian x -coordinate of BIP 340. The algorithms are the following.

1. The algorithm $\text{DerivePublic}(\text{isk})$ returns the BIP-340 public key of isk , or $\perp$ exactly when BIP-340 key derivation fails, that is, when isk read as a big-endian integer is 0 or at least n .

2. The algorithm $\text{Sign}(\text{isk}, M)$ runs BIP-340 signing on M under isk with auxiliary data $a := [0x00]^{32}$ and returns its output $\sigma \in \mathbb{B}^{Y[64]}$, or $\perp$ if it fails. With a fixed, signing is a deterministic function of (isk, M) .
3. The algorithm $\text{Validate}(\text{ik}, M, \sigma)$ returns 0 if $\sigma = \perp$; otherwise it returns 1 if BIP-340 verification of σ on M under ik succeeds, and 0 if not.

ZIP 227 writes the verification key in Validate as key , although the parameter is named ik ; the reading ik is the only one that type-checks. Batch verification MAY be used; precomputation MAY be used if and only if it produces equivalent results (ZIP 227, “Issuance Authorization Signature Scheme”, “Orchard ZSA Issuance Authorization Signature Scheme”, “Derivation of issuance validating key” and “Issuance Authorization Signing and Validation”). Class: specified.

Assumption 2.3 (Unforgeability of the issuance signature). The scheme IssueAuthSig , with key generation that samples isk uniformly from $\mathbb{B}^{Y[32]}$ conditioned on $\text{DerivePublic}(\text{isk}) \neq \perp$, and with $H_{\text{BIP0340}/\text{challenge}}$ and $H_{\text{BIP0340}/\text{nonce}}$ modelled as independent random oracles, is existentially unforgeable under chosen-message attack (*Crypto Guide*, Definition “Existential unforgeability under chosen-message attack”).

The assumption is justified, not proved. The *Crypto Guide*’s Theorem “EUF-CMA security of Schnorr signatures in the ROM” (§“Security in the random oracle model and the forking lemma”) proves the property for Schnorr signatures under the discrete-logarithm assumption, and that volume’s remark following Construction “BIP-340 Schnorr over secp256k1, schematically” carries the proof to BIP-340 with the tagged challenge hash $H_{\text{BIP0340}/\text{challenge}}$ modelled as a random oracle. With a fixed, the nonce is $H_{\text{BIP0340}/\text{nonce}}(t \parallel \text{ik} \parallel M)$ for the secret $t := d \oplus H_{\text{BIP0340}/\text{aux}}(a)$, where d is the BIP-340 secret scalar of isk , fixed per key. With $H_{\text{BIP0340}/\text{nonce}}$ also modelled as a random oracle, independent of the challenge oracle, the nonce is a fresh uniform value per message to any party ignorant of t , and the proof carries over as for deterministic nonces (*Crypto Guide*, Construction “EdDSA, schematically”).

Definition 2.4 (Issuer identifier and signature encodings). For $\text{ik} \neq \perp$, the *validating-key encoding* is $\text{ik_encoding} := [0x00] \parallel \text{ik}$, of 33 bytes, and the *issuer identifier* is $\text{issuer} := \text{ik_encoding}$. For $\sigma \neq \perp$, the *signature encoding* is $\text{issueAuthSig} := [0x00] \parallel \sigma$, of 65 bytes. In both encodings the initial byte names the signature scheme and MUST be $0x00$, which denotes BIP 340 (ZIP 227, “Derivation of issuance validating key”, “Issuance Authorization Signing and Validation”). The map $\text{ik} \mapsto \text{issuer}$ is injective. Note owners and validators use the identifier to associate an Asset with its issuer; ZIP 227, “Issuer Identifier”, records that the equality $\text{issuer} = \text{ik_encoding}$ may cease to hold once key rotation is specified. Class: specified; its carrier is not.

The encoding ik_encoding occurs only in the issuer field of the Issuance Bundle of “Issuance Actions and Issuance Bundles” (§6.2). No transaction format carrying that field is specified.

2.2 Asset descriptions and Asset Identifiers

Definition 2.5 (Asset description). An *asset description* asset_desc is a byte string chosen by the issuer, carrying any information about the issuance. ZIP 227 requires it to be non-empty and states that it SHOULD be a well-formed UTF-8 code-unit sequence according to Unicode 15.0.0 or later; no maximum length is imposed (ZIP 227, “ZIP 227 Asset Identifiers”). Only its hash assetDescHash (Definition 2.6) is visible to consensus, so neither non-emptiness nor the UTF-8 condition can be enforced by consensus: both are issuer-side conventions. Within the scope of one issuer, distinct Assets are obtained from distinct descriptions. Class: specified.

Definition 2.6 (Asset Identifier and its encoding). For an issuer identifier $\text{issuer} \in \mathbb{B}^{\mathbb{Y}[33]}$ and an asset description asset_desc , let

$$\begin{aligned}\text{assetDescHash} &:= \text{BLAKE2b-256}(\text{ZSA-AssetDescCRH}, \text{asset_desc}) \in \mathbb{B}^{\mathbb{Y}[32]}, \\ \text{AssetId} &:= (\text{issuer}, \text{assetDescHash}), \\ \text{EncodeAssetId}(\text{AssetId}) &:= [\text{0x00}] \parallel \text{issuer} \parallel \text{assetDescHash} \in \mathbb{B}^{\mathbb{Y}[66]}.\end{aligned}$$

The pair AssetId is the *Asset Identifier* and $\text{EncodeAssetId}(\text{AssetId})$ its canonical encoding (ZIP 227, “ZIP 227 Asset Identifiers”). The leading byte of the encoding is a version byte, reserving room for other issuance protocols; it is distinct in role from the scheme byte that begins issuer, so every current encoding begins with two 0x00 bytes of different meaning. The encoding is injective, since its three components have the fixed lengths 1, 33 and 32 bytes. Class: specified.

Proposition 2.7 (Uniqueness of Asset Identifiers). *Consider two distinct pairs of issuer identifier and asset description, $(\text{issuer}, \text{asset_desc})$ and $(\text{issuer}', \text{asset_desc}')$, with Asset Identifiers AssetId and $\text{AssetId}'$.*

1. *If $\text{issuer} \neq \text{issuer}'$, then $\text{AssetId} \neq \text{AssetId}'$.*
2. *If $\text{issuer} = \text{issuer}'$, then $\text{asset_desc} \neq \text{asset_desc}'$, and $\text{AssetId} = \text{AssetId}'$ holds exactly when the two descriptions collide under $\text{BLAKE2b-256}(\text{ZSA-AssetDescCRH}, \cdot)$.*

Hence, under the Ironwood Guide’s Assumption “Collision resistance of BLAKE2b”, no efficient algorithm outputs two distinct pairs of issuer and description with equal Asset Identifiers, or with equal encodings EncodeAssetId , except with negligible probability. Status: proved here for the specified construction; conditional on no unspecified object.

Proof. The issuer identifier is a component of AssetId , so distinct issuers give distinct pairs, which is claim 1. For one issuer the descriptions differ, and equal identifiers mean equal values assetDescHash on distinct descriptions, a BLAKE2b-256 collision under one personalisation; this is claim 2. An algorithm that outputs such a collision outputs two distinct inputs $(P, x) \neq (P, x')$ of h with equal values, which the *Ironwood Guide’s* Assumption “Collision resistance of BLAKE2b” (§“Transaction digests and signatures”) excludes except with negligible probability. Equal encodings imply equal identifiers by the injectivity recorded in Definition 2.6. $\square$

The requirement of ZIP 226, “Asset Identifiers”, that every new Asset have a new and unique

Asset Identifier is met by the choice of a new description. ZIP 227, “Rationale”, argues the separation of issuers informally; that argument is a rationale of the ZIP, whose class is designed but unspecified, and Proposition 2.7 supplies its proof. Since the proposition separates Assets only through the issuer and any issuer may reuse any description, ZIP 227, “ZIP 227 Asset Identifiers”, requires that wallets MUST NOT display `asset_desc` alone as the name of an Asset (see also ZIP 227, “Displaying Asset Identifier information to users”).

2.3 Asset Digests and Asset Bases

OrchardZSA adds three inputs of GroupHash to those of the Orchard protocol, which the *Ironwood Guide* lists in its table of §“GroupHash, domain separation, and nothing-up-my-sleeve generators”. Table 3 lists them as strings, each with the subsection that uses it (ZIP 227, “OrchardZSA Asset Bases”; ZIP 226, “Split Notes” and “Note Structure and Commitment”). The objects they define are constructed at their uses.

Table 3. Group-hash inputs of OrchardZSA, in addition to those of the Orchard protocol. The subsections are “Asset Digests and Asset Bases”, “Nullifiers of OrchardZSA notes” and “Note commitments”. The commitment to a Custom Asset note reuses the blinding input (`z.cash:Orchard-NoteCommit-r`, ϵ) and the Sinsemilla table entries of the Orchard protocol, so they add no row. The table lists input strings only.

Domain separator D	Message M	Use	Subsection
<code>z.cash:OrchardZSA</code>	64-byte Asset Digest	Asset Base of a Custom Asset	§2.3
<code>z.cash:Orchard</code>	L	additional term of a split nullifier	§4.2
<code>z.cash:SinsemillaQ</code>	<code>z.cash:ZSA-NoteCommit-M</code>	hash base, Custom Asset note commitment	§3.2

Lemma 2.8 (Distinctness of the OrchardZSA group-hash inputs). *The input pairs of Table 3 are pairwise distinct and distinct from every input pair of the Ironwood Guide’s table of §“GroupHash, domain separation, and nothing-up-my-sleeve generators”. Hence, under the Ironwood Guide’s Assumption “GroupHash as a random oracle”, the values of GroupHash at all these pairs are independent uniform points of $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, and, under that assumption and the Ironwood Guide’s Assumption “Discrete logarithms on Pallas”, that volume’s Lemma “Relations among fixed generators” applies to any polynomial collection of them together with the Orchard generators. Status: proved here; conditional on no unspecified object.*

Proof. The separator `z.cash:OrchardZSA` occurs in no row of the Orchard table. Under `z.cash:Orchard` the Orchard table has only the messages G and K , both distinct from L . Under `z.cash:SinsemillaQ` it has only the messages `z.cash:Orchard-NoteCommit-M`, `z.cash:Orchard-CommitIvk-M` and `z.cash:Orchard-MerkleCRH`, all distinct from `z.cash:ZSA-NoteCommit-M`. The three new pairs have pairwise distinct separators or messages. The map from (D, M) to the input hashed by GroupHash is injective, as the *Ironwood Guide*’s Remark “Domain separation” records, so distinct pairs are distinct oracle inputs; the random-oracle assumption then gives independent uniform values, and the cited lemma applies verbatim, its hypothesis being only that the inputs are distinct. $\square$

The *Ironwood Guide*'s Assumption “Collision resistance of BLAKE2b” concerns the 32-byte output only. The Asset Digest is a 64-byte output, hence a separate assumption.

Assumption 2.9 (Collision resistance of BLAKE2b-512). No efficient algorithm outputs, except with negligible probability, two inputs $x \neq x'$ with $\text{BLAKE2b-512}(P, x) = \text{BLAKE2b-512}(P, x')$ under one 16-byte personalisation P , in the sense of the *Crypto Guide*'s Definition “Collision resistance” (§“Security notions: preimage, second-preimage, and collision resistance”).

Construction 2.10 (Asset Digest and Asset Base). For the Asset Identifier AssetId of a Custom Asset, the *Asset Digest* is

$$\text{AssetDigest}_{\text{AssetId}} := \text{BLAKE2b-512}(\text{ZSA-Asset-Digest}, \text{EncodeAssetId}(\text{AssetId})) \in \mathbb{B}^{\mathbb{Y}[64]}.$$

For $D \in \mathbb{B}^{\mathbb{Y}[64]}$ let

$$\text{ZSAValueBase}(D) := \text{GroupHash}(\text{z.cash:OrchardZSA}, D) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}),$$

with GroupHash the *Ironwood Guide*'s Construction “GroupHash on Pallas” (protocol specification, §“Group Hash into Pallas and Vesta”); no cofactor is cleared. The separator gives a string DST within the 255-byte bound of that construction. The *Asset Base* of the Asset is

$$\text{AssetBase}_{\text{AssetId}} := \text{ZSAValueBase}(\text{AssetDigest}_{\text{AssetId}})$$

(ZIP 227, “Asset Digests”, “Asset Bases” and “OrchardZSA Asset Bases”). The subscript is dropped when the identifier is clear. The whole chain, from issuer and description to Asset Base, is

$$\begin{aligned} \text{asset_desc} &\xrightarrow{\text{BLAKE2b-256}(\text{ZSA-AssetDescCRH}, \cdot)} \text{assetDescHash}, \\ (\text{issuer}, \text{assetDescHash}) = \text{AssetId} &\xrightarrow{\text{EncodeAssetId}} \text{EncodeAssetId}(\text{AssetId}), \\ \text{EncodeAssetId}(\text{AssetId}) &\xrightarrow{\text{BLAKE2b-512}(\text{ZSA-Asset-Digest}, \cdot)} \text{AssetDigest}, \\ \text{AssetDigest} &\xrightarrow{\text{ZSAValueBase}} \text{AssetBase}. \end{aligned}$$

Class: specified.

The Asset Identifier is global across the shielded protocols that support Zcash Shielded Assets; the Asset Base is per protocol, the map ZSAValueBase being instantiated per protocol, here for OrchardZSA (ZIP 227, “Asset Identifier”, “Asset Bases”).

Let $\mathcal{E}^* := \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}$. ZIP 226, “Note Structure and Commitment”, types the Asset Base of a note in $\mathcal{E}^*$, as “a valid group element that is not the identity”; ZIP 227 types the Asset Base of an Issue Note (“Issue Note”) and the domain of the issuance state (“Global Issuance State”) likewise. The codomain of GroupHash , by contrast, is the whole group.

Lemma 2.11 (Non-identity of derived Asset Bases). *Under the Ironwood Guide’s Assumption “GroupHash as a random oracle”, for every $D \in \mathbb{B}^{\mathbb{Y}[64]}$,*

$$\Pr[\text{ZSAValueBase}(D) = \mathcal{O}] = 1/p_{\text{Vesta}},$$

and an algorithm that makes q queries to the oracle finds an Asset Identifier whose Asset Base is $\mathcal{O}$ with probability at most q/p_{Vesta} , the inputs of its output being counted among the queries. Status: proved here; conditional on no unspecified object.

Proof. In the model each value is uniform on the Pallas group, of order p_{Vesta} (Ironwood Guide, §“Fields, groups, and encodings”), so it is $\mathcal{O}$ with probability $1/p_{\text{Vesta}}$. A union bound over the q queries gives the second claim. $\square$

Remark 2.12 (Identity output of the Asset Base derivation). The map ZSAValueBase has no case for the identity, and ZIP 227 states no rule for an issuance whose derived Asset Base is $\mathcal{O}$. The typing in $\mathcal{E}^*$ of the Asset Base of an Issue Note and of the domain of the issuance state makes such an issuance unconstructible, hence invalid. Class: specified; the rejection follows from the normative typing in $\mathcal{E}^*$, and ZIP 227 states no separate rule. By Lemma 2.11 the event has negligible probability.

Proposition 2.13 (Separation of Asset Bases). *Let the Ironwood Guide’s Assumption “GroupHash as a random oracle” hold, and consider an algorithm that makes q queries to GroupHash, the inputs of its output being counted among the queries.*

1. *The map $\text{AssetId} \mapsto \text{AssetBase}_{\text{AssetId}}$ is not injective; “unique” in ZIP 226, “Note Structure and Commitment”, means only that it is deterministic.*
2. *If the algorithm outputs $\text{AssetId} \neq \text{AssetId}'$ with equal Asset Bases, then either it has found a collision of*

$$\text{BLAKE2b-512}(\text{ZSA-Asset-Digest}, \cdot)$$

on distinct encodings, or it is in an event of probability at most $q^2/(2p_{\text{Vesta}})$.

3. *It outputs an Asset Identifier whose Asset Base equals one of t fixed generators, among them V^{Orchard} , R^{Orchard} and K^{Orchard} , each the value of GroupHash at an input of the Orchard table or of Table 3 other than the first row, with probability at most qt/p_{Vesta} .*

Hence, under Assumption 2.9, the Ironwood Guide’s Assumptions “Collision resistance of BLAKE2b” and “GroupHash as a random oracle”, and by Proposition 2.7, distinct pairs (issuer, asset_desc) yield distinct Asset Bases, and no Asset Base of a Custom Asset is an Orchard generator, except with negligible probability. Status: proved here for the specified chain; conditional on no unspecified object.

Proof. Claim 1. The map is defined on all pairs in $\mathbb{B}^{\mathbb{Y}[33]} \times \mathbb{B}^{\mathbb{Y}[32]}$, of which there are more than 2^{256} , and takes values in a group of prime order $p_{\text{Vesta}} < 2^{255}$; it is therefore not injective. It is a composition of deterministic maps.

Claim 2. By the injectivity of `EncodeAssetId` (Definition 2.6), distinct identifiers have distinct encodings; equal digests of distinct encodings are a BLAKE2b-512 collision under ZSA-Asset-Digest. Otherwise the digests are distinct, hence distinct oracle inputs under `z.cash:OrchardZSA`, whose values are independent uniform points in the model. Among q queried inputs two share a value with probability at most $\binom{q}{2}/p_{\text{Vesta}} \leq q^2/(2p_{\text{Vesta}})$.

Claim 3. By Lemma 2.8 every input under `z.cash:OrchardZSA` differs from the inputs of the t generators, so each queried Asset Base is uniform and independent of them, and equals a given one with probability $1/p_{\text{Vesta}}$. A union bound over the q queries and the t generators gives qt/p_{Vesta} .

The final sentence composes claims 2 and 3 with Proposition 2.7: distinct pairs of issuer and description yield distinct identifiers except with negligible probability, distinct identifiers yield distinct Asset Bases except with negligible probability by claim 2 and Assumption 2.9, and the bounds of claims 2 and 3 are negligible for polynomial q and t . $\square$

Definition 2.14 (Byte encoding of an Asset Base). The *byte encoding* of an Asset Base is the 32-byte string

$$\text{asset_base} := \text{LEBS2OSP}_{256}(\text{AssetBase}^*) \in \mathbb{B}^{\mathbb{Y}[32]},$$

with LEBS2OSP_{256} and the star encoding those of the *Ironwood Guide*, §“Fields, groups, and encodings”, Definitions “Integer and byte encodings” and “Star encoding” (ZIP 226, “Note Structure and Commitment”). It is typed as a byte string throughout, although ZIP 226 also types it as a bit string in its clause on the note plaintext. The encoding is canonical on the Pallas group, by the canonicity of the star encoding established after the same Definition; ZIP 226 notes that canonicity may fail for future shielded protocols. Class: specified; its carrier is not.

The placements of `asset_base` in a note plaintext and in the entries of a burn set are encodings of ZIP 230, which is Withdrawn; they are not specified.

Table 4. Sizes along the identity chain, in bytes. ZIP 227 publishes no test vectors, and no worked instance is given.

Object	Bytes	Defined in	Source
<code>ik</code>	32	Definition 2.2	ZIP 227, “Derivation of issuance validating key”
<code>issuer = ik_encoding</code>	33	Definition 2.4	ZIP 227, “Issuer Identifier”
<code>issueAuthSig</code>	65	Definition 2.4	ZIP 227, “Issuance Authorization Signing and Validation”
<code>assetDescHash</code>	32	Definition 2.6	ZIP 227, “ZIP 227 Asset Identifiers”
<code>EncodeAssetId(AssetId)</code>	66	Definition 2.6	ZIP 227, “ZIP 227 Asset Identifiers”
<code>AssetDigest</code>	64	Construction 2.10	ZIP 227, “Asset Digests”
<code>asset_base</code>	32	Definition 2.14	ZIP 226, “Note Structure and Commitment”

Every formula of the chain is specified, in Draft ZIP text: the issuance keys and `IssueAuthSig`, the identifier issuer, the hash `assetDescHash`, the identifier `AssetId` and its encoding, the Asset Digest, the Asset Base, its encoding `asset_base` and the Native Asset Base of §2.4. The identity case is specified, by typing (Remark 2.12). The encodings that would carry the chain on chain belong to the carrier.

2.4 The Native Asset Base

Definition 2.15 (Native Asset Base). The Native Asset, ZEC on Mainnet and TAZ on Testnet (ZIP 227, “Terminology”), has no Asset Identifier under ZIP 227 and does not pass through the chain of Construction 2.10. Its Asset Base is fixed as

$$\text{AssetBase} := V^{\text{Orchard}} = \text{GroupHash}(\text{z.cash:orchard-cv}, v),$$

the value base of the *Ironwood Guide*’s Definition “Net value commitment” (§“Value commitments”), where V^{Orchard} and its companion $R^{\text{Orchard}} = \text{GroupHash}(\text{z.cash:orchard-cv}, r)$ are constructed; they are cited, not re-derived. ZIP 226, “Asset Identifiers” and “Value Commitment”, writes the definition for ZEC; its burn rules (“Burn Mechanism”, “Additional Consensus Rules for the assetBurn set”) identify the Native Asset, ZEC or TAZ, with the base V^{Orchard} . Class: specified.

The value commitment and the note commitment of a Native Asset note are computed by the Orchard formulas. Two things differ: the derivation of rcm takes the OrchardZSA lead byte (Remark 3.6), and the note plaintext carries asset_base (Definition 3.7; ZIP 226, “Note Structure and Commitment”). The value commitment of ZIP 226, “Value Commitment”, is $[v^{\text{net}}] \text{AssetBase} + [\text{rcv}] R^{\text{Orchard}}$, which at the base V^{Orchard} is the Orchard net value commitment $[v^{\text{net}}] V^{\text{Orchard}} + [\text{rcv}] R^{\text{Orchard}}$ of the cited Definition. The note commitment of ZIP 226, “Note Structure and Commitment”, is by its case split the Orchard note commitment when the Asset Base is V^{Orchard} ; the case split is stated in “Note commitments” (§3.2). The base V^{Orchard} is a group-hash output under a separator distinct from z.cash:orchardZSA , so by Proposition 2.13 no Asset Base of a Custom Asset equals it, except with negligible probability.

3 Notes and note commitments

This section extends the Orchard note by one field, the Asset Base (§3.1); extends the note commitment by a case split whose native branch is the Orchard commitment (§3.2); states the derivations of the note’s randomness from its seed and what no live text fixes of them (§3.3); states the field set of the note plaintext (§3.4); and replaces the fixed value base of the net value commitment by the Asset Base (§3.5). It proves hiding and binding of both commitments within and across Assets. Every construction of the section is taken from ZIP 226, which is Draft; the Orchard objects it extends are those of the *Ironwood Guide* and are cited, not restated.

3.1 The OrchardZSA note

Definition 3.1 (OrchardZSA note). An *OrchardZSA note* is a tuple

$$n = (d, \text{pk}_d, v, \text{AssetBase}, \rho, \psi, \text{rcm})$$

of type

$$\text{Note}^{\text{OrchardZSA}} := \{0, 1\}^{88} \times (\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}) \times \{0, \dots, 2^{64} - 1\} \\ \times \mathcal{E}^* \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{F}_{p_{\text{Vesta}}},$$

with $\mathcal{E}^* = \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}$ as in §2.3. It is the note of the *Ironwood Guide*’s Definition “Note” (§“Notes”) with exactly one added field, the *Asset Base* $\text{AssetBase} \in \mathcal{E}^*$ of the note’s Asset: for a Custom Asset the point of Construction 2.10, for the Native Asset the point V^{Orchard} of Definition 2.15. The components $(d, \text{pk}_d, v, \rho, \psi, \text{rcm})$ and their types, with rcm typed in $\mathbb{F}_{p_{\text{Vesta}}}$, are those of the cited definition. ZIP 226 types the added field in $\mathbb{P}^*$, a group element that is neither the identity nor $\perp$ (Table 1). The value v counts units of the note’s Asset; the *Ironwood Guide*’s Definition “Unit of value” (the zatoshi) applies only when $\text{AssetBase} = V^{\text{Orchard}}$ (ZIP 226, “Note Structure and Commitment”; protocol specification, §“Shielded Pools and Notes”). Class: specified.

3.2 Note commitments

Definition 3.2 (OrchardZSA note commitment). Let $n = (d, \text{pk}_d, v, \text{AssetBase}, \rho, \psi, \text{rcm})$ be an OrchardZSA note, $\text{g}_d := \text{DiversifyHash}(d)$, and

$$M(n) := \text{g}_d^* \parallel \text{pk}_d^* \parallel \text{LE}_{64}(v) \parallel \text{LE}_{255}(\rho) \parallel \text{LE}_{255}(\psi)$$

the 1086-bit Orchard message of the *Ironwood Guide*’s Definition “Note commitment”. Let

$$M^{\text{ZSA}}(n) := M(n) \parallel \text{AssetBase}^*,$$

where the 256-bit star encoding AssetBase^* has the bytes `asset_base` of Definition 2.14. The string $M^{\text{ZSA}}(n)$ has the fixed length 1342: 135 chunks of 10 bits with padding 8, against 109 chunks for $M(n)$, within the chunk bound $c = 253$ of the *Ironwood Guide*’s Construction “Sinsemilla hash on Pallas”. With the Orchard blinding base and the hash point

$$H_D := \text{GroupHash}(\text{z.cash:Orchard-NoteCommit-r}, \varepsilon), \\ \hat{M}^{\text{ZSA}} := \text{SinsemillaHashToPoint}(\text{z.cash:ZSA-NoteCommit-M}, M^{\text{ZSA}}(n)),$$

the *OrchardZSA note commitment* $\text{cm} := \text{NoteCommit}_{\text{rcm}}^{\text{OrchardZSA}}(n)$ of n is defined in two branches. In the *native branch*, $\text{AssetBase} = V^{\text{Orchard}}$, it is

$$\text{cm} := \text{NoteCommit}_{\text{rcm}}(M(n)),$$

with NoteCommit the Orchard commitment of the cited definition. In the *custom branch*, $\text{AssetBase} \neq V^{\text{Orchard}}$, it is

$$\text{cm} := \begin{cases} \hat{M}^{\text{ZSA}} + [\text{rcm}] H_D & \text{if } \hat{M}^{\text{ZSA}} \neq \perp, \\ \perp & \text{otherwise.} \end{cases}$$

In both branches $\text{cm} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \cup \{\perp\}$, and the *extracted note commitment* is $\text{cmx} := \text{Extract}_{\text{p}}^{\perp}(\text{cm}) \in \mathbb{F}_{p_{\text{Pallas}}} \cup \{\perp\}$, as for Orchard. The custom branch is not the routed *SinsemillaCommit* of the *Ironwood Guide*’s Construction “Sinsemilla commitment and short commitment” for any domain: its hash domain is new, while its blinding base is Orchard’s. The hash base $Q(\text{z.cash:ZSA-NoteCommit-M})$ is a row of Table 3. The new domain hashes the single length 1342, as the *Ironwood Guide*’s Remark “Fixed length” requires of every domain (ZIP 226, “Note Structure and Commitment”; protocol specification, §“Sinsemilla commitments”). Class: specified.

In the native branch the Asset Base is not an input of the commitment. Hence the OrchardZSA commitment of a note with $\text{AssetBase} = V^{\text{Orchard}}$ equals, bit for bit, the Orchard commitment of its Orchard components; an Orchard note and a native OrchardZSA note with the same Orchard components have the same cmx ; and every leaf of the Orchard-pool note commitment tree created before OrchardZSA remains a valid opening target. Both branches output in $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \cup \{\perp\}$, so custom-branch commitments enter the same tree. Each statement is immediate from Definition 3.2 (ZIP 226, “Note Structure and Commitment” and “Rationale for Note Commitment”).

Proposition 3.3 (Hiding and binding of OrchardZSA note commitments).

- (a) *Hiding with one output distribution.* Let rcm be uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and independent of the content. For every content $(d, \text{pk}_d, v, \text{AssetBase}, \rho, \psi)$ whose hash point in its branch is not $\perp$, the commitment cm is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$. Hence the distributions of cm and of cmx depend neither on the content nor on the branch, and commitments to ZEC and to Custom Assets are identically distributed.
- (b) *Binding.* Under the *Ironwood Guide*’s Assumptions “Discrete logarithms on Pallas” and “GroupHash as a random oracle”, no efficient algorithm outputs, except with negligible probability, OrchardZSA notes $n \neq n'$ with $\text{cmx}(n) = \text{cmx}(n') \neq \perp$; in particular none with $\text{AssetBase} = V^{\text{Orchard}} \neq \text{AssetBase}'$, and none whose contents are equal except for the Asset Base.
- (c) *Under the same assumptions, no efficient algorithm outputs, except with negligible probability, a custom-branch content with $\hat{M}^{\text{ZSA}} = \perp$.*

Status (Definition 1.3): proved here for the specified construction; conditional on no unspecified object. Part (a) is stated for uniform rcm and does not cover a derived trapdoor, whose derivation is classified in Remark 3.6.

Proof. Write

$$Q_{\text{O}} := Q(\text{z.cash:Orchard-NoteCommit-M}), \quad Q_{\text{Z}} := Q(\text{z.cash:ZSA-NoteCommit-M}).$$

The generators $Q_{\text{O}}, Q_{\text{Z}}, S(0), \dots, S(1023)$ and H_D are values of GroupHash at pairwise distinct inputs, by Lemma 2.8 and the *Ironwood Guide*’s table of §“GroupHash, domain separation, and nothing-up-my-sleeve generators”.

(a) In both branches $\text{cm} = \hat{M} + [\text{rcm}] H_D$, with the same blinding base $H_D \neq \mathcal{O}$ and a point $\hat{M}$ fixed by the content. The proof of the *Ironwood Guide*'s Lemma "Hiding and binding of SinsemillaCommit" (i) uses only these two facts, the first established for H_D before that volume's Proposition "Hiding and binding of note commitments"; so cm is uniform, whatever the content and the branch. The value cmx is a fixed function of cm .

(b) Step 1. The map from a content to its branch and message is injective except on a diversifier collision: the branch is fixed by whether $\text{AssetBase} = V^{\text{Orchard}}$; in the native branch the Asset Base is determined, and the message $M(n)$ determines the Orchard components; in the custom branch the message $M^{\text{ZSA}}(n)$ determines them and, by the injectivity of the star encoding, the Asset Base. A diversifier collision is excluded as in Step 1 of the proof of the *Ironwood Guide*'s Proposition "Hiding and binding of note commitments" (b). Outside that event, $n \neq n'$ gives distinct branches, or one branch with distinct messages, or one branch and one message with $\text{rcm} \neq \text{rcm}'$.

Step 2. Let $\text{cmx}(n) = \text{cmx}(n') \neq \perp$. Both commitments are points, and by the *Ironwood Guide*'s Lemma "Coordinate extraction identifies opposite points", $\text{cm}' = \pm \text{cm}$; both hash points are not $\perp$. Three cases remain.

Both native. The Orchard components differ, since the Asset Bases agree, and the *Ironwood Guide*'s Proposition "Hiding and binding of note commitments" (b) applies.

Both custom. The three steps of the *Ironwood Guide*'s Proposition "Collision resistance of the Sinsemilla instances" hold for the message length $l = 1342$ and $n = 135$ chunks. Since $2^{135} \leq 2^e \leq (p_{\text{Vesta}} - 1)/2$, every difference $\chi_j(m) - \chi_j(m')$ of the unfolding in its proof has absolute value below p_{Vesta} , so χ is injective, and the coefficient 2^{136} on Q_Z in the opposite case is non-zero modulo p_{Vesta} . Equal points on distinct messages, or opposite points, therefore give a non-trivial relation among Q_Z , the $S(j)$ and H_D . Equal messages with $\text{rcm} \neq \text{rcm}'$ and equal points give $[\text{rcm} - \text{rcm}'] H_D = \mathcal{O}$, impossible for $H_D \neq \mathcal{O}$ in a group of prime order p_{Vesta} ; opposite points on equal messages are the opposite case above.

Across branches. Let n be native and n' custom, with pieces m and m' of $M(n)$ and $M^{\text{ZSA}}(n')$. Unfolding both hash points gives

$$[2^{109}] Q_O \mp [2^{135}] Q_Z + \sum_j [\chi_j(m) \mp \chi_j(m')] S(j) + [\text{rcm} \mp \text{rcm}'] H_D = \mathcal{O}.$$

The coefficient 2^{109} on Q_O is non-zero modulo p_{Vesta} .

In each case the relation is non-trivial among values of GroupHash at distinct inputs, which the *Ironwood Guide*'s Lemma "Relations among fixed generators" excludes except with negligible probability. The first particular case of (b) is the case across branches; the second is either that case or the case of two custom notes with distinct messages.

(c) Step (2) of the proof of the cited proposition applies to the domain of Q_Z : an operand $\mathcal{O}$ arises only if Q_Z or some $S(j)$ is $\mathcal{O}$, itself a non-trivial relation, and an exceptional case at step i yields a relation with coefficient $\alpha 2^{i-1}$ on Q_Z , where $\alpha \in \{-1, 1, 2\}$ and $0 < |\alpha 2^{i-1}| \leq 2^{135} < p_{\text{Vesta}}$. The same lemma excludes both. $\square$

ZIP 226 argues the property in rationale only, attributing it to the Sinsemilla hash function ("by definition of the Sinsemilla hash function", "Rationale for Note Commitment"; see also "Privacy Implications"). The unblinded hash is deterministic and hides nothing; the hiding comes

from the term $[rcm] H_D$. The plaintext equivalence of Orchard and native OrchardZSA notes, which is withdrawn ZIP 230 text, is not used.

Remark 3.4 (Contextual distinguishability of commitments). Commitments are hidden only as group elements. ZIP 226, “Privacy Implications”, states that Orchard note commitments are distinguishable from OrchardZSA note commitments by context, since the two protocols are carried by different transaction versions (in its text, version 5 for Orchard and version 6 for OrchardZSA), whether or not both are active. No current format assigns OrchardZSA a version: version 6 is the format of ZIP 229, “Transaction Format”, which carries Orchard-pool and Ironwood-pool Actions and no ZSA. ZIP 248, “Potential Future Bundle Types”, lists OrchardZSA provisionally as variant 2 of bundle type 3, the Orchard bundle type, whose public variant identifier would likewise separate the two. Within one OrchardZSA bundle, commitments to ZEC and to Custom Assets are identically distributed under a uniform trapdoor (Proposition 3.3(a)); the derived trapdoor, undefined in live text, is not covered (Remark 3.6). The carrier is treated in “The transaction carrier” (§8).

3.3 The note seed and the lead byte

ZIP 226, “Abstract”, is defined relative to the protocol with ZIP 2005 applied, and reads H^{rcm} , H^ψ and the recoverable plaintexts as ZIP 2005 defines them; it calls ZIP 2005 “Orchard Quantum Recoverability”. ZIP 2005 is now “Ironwood Quantum Recoverability” (Proposed). It is active with NU6.3 and merged into the protocol specification (ZIP 258, “ZIP 2005 activation”; ZIP 2005, “Specification Updates”), and it applies its recoverable plaintext format, lead byte $0x03$, only to Ironwood-pool notes (ZIP 229, “Transaction Format”), while ZIP 226 places OrchardZSA notes in the Orchard pool (Remark 1.7). The two premises no longer coincide. Two constructions depend on them: the lead byte, here, and the seed of split nullifiers, at “Nullifiers of OrchardZSA notes” (§4.2). The derivations of ZIP 2005 are constructed in the *Ironwood Guide*, §“The note seed: Ironwood derivations (lead byte $0x03$)”, and are cited, not restated.

Definition 3.5 (Seed derivations of an OrchardZSA note). Let an OrchardZSA note be created with recipient (d, pk_d) , value v , element $\rho \in \mathbb{F}_{p_{\text{Pallas}}}$, note seed $rseed$ uniform on the 32-byte strings, and lead byte $leadByte$. Let $\underline{\rho} := \text{I2LEOSP}_{256}(\rho)$ and $g_d := \text{DiversifyHash}(d)$. Then

$$\begin{aligned}\psi &:= \text{ToBase}(\text{PRFexpand}_{rseed}([0x09] \parallel \underline{\rho})), \\ rcm &:= \text{Derive_rcm}_{rseed}(leadByte, g_d^*, pk_d^*, v, \underline{\rho}, \psi),\end{aligned}$$

where ψ does not depend on the lead byte, and Derive_rcm is defined only for two lead bytes:

$$\text{Derive_rcm}_{rseed}(leadByte, \dots) := \begin{cases} \text{ToScalar}(\text{PRFexpand}_{rseed}([0x05] \parallel \underline{\rho})) & \text{if } leadByte = 0x02, \\ \text{the pre_rcm derivation} & \text{if } leadByte = 0x03, \end{cases}$$

the latter being that of the *Ironwood Guide*’s Construction “Derivations under lead byte $0x03$ ”. Neither input list contains AssetBase . ZIP 226, “Note Structure and Commitment”, requires $leadByte$ to be the placeholder $\{\{ZSALEADBYTE\}\}$ whenever §“Sending Notes (Orchard)” or §“Dummy Notes (Orchard)” of the protocol specification is invoked, directly or indirectly, for

an OrchardZSA note. That rule speaks of “the computation of ρ and ψ ”; under the amended specification the lead byte enters only `Derive_rcm`. The specification admits lead byte $0x02$ in the Orchard pool and $0x03$ in the Ironwood pool only (protocol specification, §“Note Plaintexts and Memo Fields”), and ZIP 2005, “Usage with other proposals requiring note plaintext format changes”, requires such proposals to use a new lead byte value or values. Classes: the derivation of ψ is specified; the derivation of `rcm` and the value of the lead byte are classified in Remark 3.6.

Remark 3.6 (Unassigned lead byte). Each part carries one class of Definition 1.2.

- (i) The derivation of ψ is specified.
- (ii) The derivation of `rcm` is designed but unspecified. A complete design existed: ZIP 226 fixed $0x03$ with H^{rcm} before $0x03$ passed to ZIP 2005, and the design needs only a lead byte.
- (iii) The value of the lead byte is an open problem. The removed design’s $0x03$ is the Ironwood lead byte in live text, a design that conflicts with live text; the placeholders `{{ZSALEADBYTE}}` (ZIP 226), `{{LEADBYTE}}` (withdrawn ZIP 230) and `{{MBLEADBYTE}}` (ZIP 231, “Note plaintext lead byte assignment”) are unassigned.

Until the byte is assigned, the `rcm` of an OrchardZSA note is undefined in live text, and the *Ironwood Guide*’s Proposition “Hiding under the derived trapdoor” does not transfer. The OrchardZSA Action statement, like the Orchard one, does not check the derivation (*Ironwood Guide*, Remark “What the statement does not check” (ii)), so for the outputs of a transfer the gap binds only sender and recipient. Its consequence for validators is stated at “Issue Notes” (§6.1).

3.4 The note plaintext

Definition 3.7 (OrchardZSA note plaintext). The *OrchardZSA note plaintext* of a created OrchardZSA note carries the fields (`leadByte`, d , v , `rseed`, `memo`) of the *Ironwood Guide*’s Definition “Note plaintext” and, in addition, the 32-byte string `asset_base` of Definition 2.14 (ZIP 226, “Note Structure and Commitment”). The recipient needs `asset_base` to recompute `cmx` by Definition 3.2 and to spend the note. Given an encoding of fixed length and an assigned lead byte, the *Ironwood Guide*’s Constructions “Encryption to the recipient” and “Trial decryption” apply unchanged. Its acceptance checks (§“Trial decryption and note acceptance”) apply with the lead-byte check and the derivation of `rcm` replaced by those of that lead byte (Remark 3.6), `asset_base` required to decode to a point of $\mathcal{E}^*$, and the recomputed commitment that of Definition 3.2. Classes: the field set is specified; the encoding is an open problem.

ZIP 226 delegates the encoding to ZIP 230, which is Withdrawn. Its layout

$$\text{leadByte} \parallel d \parallel v \parallel \text{rseed} \parallel \text{asset_base} \parallel K^{\text{memo}}$$

presupposes the memo key K^{memo} of ZIP 231, which is now defined only for version-7 transactions (ZIP 231, “Specification”), and a lead byte that live text assigns otherwise (Remark 3.6). No current ZIP defines an OrchardZSA plaintext encoding. The statement of withdrawn ZIP 230 that Orchard and native OrchardZSA notes need not be distinguished in plaintexts carries no weight; the commitment half of that statement follows from Definition 3.2.

3.5 The per-Asset value commitment

Definition 3.8 (Per-Asset value commitment). Let an Action (*Ironwood Guide*, Definition “Action”) consume an OrchardZSA note of value v^{old} and create an OrchardZSA note of value v^{new} , both with Asset Base $\text{AssetBase} \in \mathcal{E}^*$. Its *net value* is the integer

$$v^{\text{net}} := v^{\text{old}} - v^{\text{new}} \in \{-2^{64} + 1, \dots, 2^{64} - 1\}.$$

For rcv drawn by the *Ironwood Guide*’s Construction “Value-commitment trapdoor”, its *per-Asset value commitment* is

$$\text{cv}^{\text{net}} := \text{ValueCommit}_{\text{rcv}}^{\text{OrchardZSA}}(\text{AssetBase}, v^{\text{net}}) := [v^{\text{net}} \bmod p_{\text{Vesta}}] \text{AssetBase} + [\text{rcv}] R^{\text{Orchard}},$$

with R^{Orchard} and the reading of v^{net} modulo p_{Vesta} as in the *Ironwood Guide*’s Definition “Net value commitment”. For $\text{AssetBase} = V^{\text{Orchard}}$ it equals $\text{ValueCommit}_{\text{rcv}}^{\text{Orchard}}(v^{\text{net}})$. The only change against Orchard is that the fixed-base multiplication by V^{Orchard} becomes a variable-base multiplication by the Action’s Asset Base; the randomness term is unchanged. The condition that controls the base is stated in “The OrchardZSA Action statement” (§4.3) (ZIP 226, “Value Commitment”; protocol specification, §“Homomorphic Pedersen commitments (Sapling and Orchard)”). Class: specified.

Lemma 3.9 (Hiding and binding of per-Asset value commitments).

- (i) *Perfect hiding.* For every $B \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ and every integer v , with rcv uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and independent of (B, v) , the point $[v \bmod p_{\text{Vesta}}] B + [\text{rcv}] R^{\text{Orchard}}$ is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$.
- (ii) *Binding across bases.* Call a base admissible if it is given as the value of GroupHash at the input $(z.\text{cash}:\text{Orchard-cv}, v)$ or at $(z.\text{cash}:\text{OrchardZSA}, D)$ for a byte string D . Under the *Ironwood Guide*’s Assumptions “Discrete logarithms on Pallas” and “GroupHash as a random oracle”, no efficient algorithm outputs, except with negligible probability, two families $((B_i, v_i))_i$ and $((B'_j, v'_j))_j$ of polynomial size, of admissible bases with integers, and trapdoors rcv, rcv' , with

$$\sum_i [v_i] B_i + [\text{rcv}] R^{\text{Orchard}} = \sum_j [v'_j] B'_j + [\text{rcv}'] R^{\text{Orchard}}$$

while the base-value vectors differ: for some group-hash input x , the sum of the v_i at bases with input x and the sum of the v'_j at bases with input x differ modulo p_{Vesta} . For one pair each, (B, v) and (B', v') with inputs x and x' , equal commitments force $x = x'$, hence $B = B'$, and $v \equiv v'$, or else $v \equiv v' \equiv 0 \pmod{p_{\text{Vesta}}}$.

- (iii) *Binding fails for arbitrary bases.* For $B \neq \mathcal{O}$, $v \neq 0$ and $v' \neq 0, v \pmod{p_{\text{Vesta}}}$, the point $B' := [v v'^{-1}] B$ satisfies $B' \notin \{\mathcal{O}, B\}$ and $[v] B = [v'] B'$, so the two openings (B, v) and (B', v') share every trapdoor and every commitment. Hence the Action statement must control the base.

Status (Definition 1.3): proved here; conditional on no unspecified object.

Proof. (i) The proof of the *Ironwood Guide*’s Lemma “Hiding and binding of net value commitments” (i) uses only $R^{\text{Orchard}} \neq \mathcal{O}$ and the prime order p_{Vesta} of the group, not the value base: $[\text{rcv}] R^{\text{Orchard}}$ is uniform, and so is its translate by the fixed point $[v \bmod p_{\text{Vesta}}] B$.

(ii) Let P_x denote the value of GroupHash at input x , and c_x the difference, modulo p_{Vesta} , of the two sums of values at input x . Grouping both sides by input, a double opening gives

$$\sum_x [c_x] P_x + [\text{rcv} - \text{rcv}'] R^{\text{Orchard}} = \mathcal{O}$$

with some $c_x \neq 0$, the relation of the proof of the *Crypto Guide*’s Theorem “Properties of the vector commitment” (§“Pedersen vector commitments”). The inputs x are pairwise distinct by construction, and distinct from the input $(z.\text{cash}:\text{Orchard-cv}, r)$ of R^{Orchard} : the input of V^{Orchard} differs from it in the message, and every input under $z.\text{cash}:\text{OrchardZSA}$ differs from both by Lemma 2.8. The relation is therefore non-trivial among values of GroupHash at distinct inputs, and the *Ironwood Guide*’s Lemma “Relations among fixed generators” excludes it except with negligible probability. For one pair each, $x \neq x'$ gives $c_x = v$ and $c_{x'} = -v'$, non-trivial unless $v \equiv v' \equiv 0$; and $x = x'$ gives $c_x = v - v'$, non-trivial unless $v \equiv v'$.

(iii) Direct computation: $[v'] B' = [v' v v'^{-1}] B = [v] B$; the scalar $v v'^{-1}$ is neither 0 nor 1 modulo p_{Vesta} , and B generates the group of prime order p_{Vesta} , so $B' \neq \mathcal{O}$ and $B' \neq B$. $\square$

ZIP 226, “Rationale for Value Commitment”, states the change of value base without a binding argument; Lemma 3.9 supplies it for bases of the admissible form, and part (iii) shows that the form cannot be dropped.

4 OrchardZSA Actions and Action Groups

This section shows that the membership exemption of Orchard dummy notes is unsound once the value base of an Action is a witness, and defines the Split Inputs that replace dummy consumed notes for Custom Assets (§4.1); constructs the split nullifier and proves its independence from the nullifier of the copied note (§4.2); states the OrchardZSA Action statement as a delta on the Orchard one, with named conditions (§4.3); states the assumptions on its proof and proves that a Split Input requires the spend authority of the copied note (§4.4); and defines the Action Group (§4.5). The objects of the section are taken from ZIP 226 and ZIP 228, both Draft, with the rules of ZIP 227 that bound the number of Action Groups. The Orchard objects they modify are those of the *Ironwood Guide* and are cited, not restated.

4.1 Dummy notes and Split Inputs

By the *Ironwood Guide*’s Definition “Dummy notes” (§“Actions, bundles, and dummy notes”), an Action whose consumed side is unused consumes a dummy note of value 0: under a fresh key or, when `enableCrossAddress = 0` and the created note is real, at the expanded receiver of the created note and under the key from which that receiver derives. Condition A3 of that volume’s Definition “Orchard Action statement”, namely $v^{\text{old}} \cdot (\text{root} - \text{rt}) = 0$ or some evaluation of `SinsemillaHash` in the fold returns $\perp$, exempts every consumed note of value 0 from membership (protocol specification, §“Dummy Notes (Orchard)” and §“Action Statement (Orchard)”). In the Orchard protocol the exemption is sound for value: condition A4 multiplies the value by the

fixed generator V^{Orchard} (*Ironwood Guide*, Definition “Net value commitment”), so a consumed note of value 0 contributes $[0] V^{\text{Orchard}} = \mathcal{O}$, whatever path it fails to prove. The per-Asset value commitment of Definition 3.8 multiplies by a witnessed Asset Base instead. The next proposition shows that the exemption, kept unchanged, then admits a balance violation.

Proposition 4.1 (Unsoundness of the dummy exemption under a witnessed base). *Let $\mathcal{R}^-$ be the relation obtained from the Ironwood Guide’s Definition “Orchard Action statement” by adding an auxiliary input $A \in \mathcal{E}^*$ and making three replacements: in A1 and A2, the commitment NoteCommit is replaced by NoteCommit^{OrchardZSA} of Definition 3.2 with Asset Base A for both notes; in A4, the net value commitment becomes*

$$cv^{\text{net}} = [v^{\text{old}} - v^{\text{new}}] A + [\text{rcv}] R^{\text{Orchard}},$$

as in Definition 3.8; condition A3 and conditions A5 to A10 are unchanged, so a consumed note of value 0 is exempt from membership whatever A is. Then for every $v \in \{1, \dots, 2^{64} - 1\}$ there is a bundle of two Actions whose witnesses satisfy $\mathcal{R}^-$, with balancing value 0 and no burn, whose binding signature verifies, and which creates a ZEC note, of Asset Base V^{Orchard} , of value v while every consumed note has value 0:

Action 1 witnesses $A := [-1] V^{\text{Orchard}}$, $v^{\text{old}} = 0$ and $v^{\text{new}} = v$;

Action 2 witnesses $A := V^{\text{Orchard}}$, a dummy consumed note and $v^{\text{new}} = v$.

More generally, a witnessed base $A = \sum_i [a_i] B_i$ with known coefficients a_i over Asset Bases B_i contributes $\sum_i [a_i v] B_i$ for a committed value v , so a value committed on A cancels values committed on the B_i . This is the attack of ZIP 226, “Rationale for Split Notes”: without the enforcement “the prover could input a multiple (or linear combination) of an existing Asset Base, and thereby attack the network by overflowing the ZEC value balance and hence counterfeiting ZEC funds”. Status (Definition 1.3): proved here; ZIP 226 states the attack in words only.

Proof. The point $[-1] V^{\text{Orchard}}$ is not $\mathcal{O}$, since V^{Orchard} has prime order p_{Vesta} , so Action 1 is well typed. Both consumed notes have value 0, so A3 holds for both Actions without a path, and A8 holds for either value of enableSpends; A9 holds with enableOutputs = 1. The remaining conditions hold when the commitments, the nullifiers and rk are computed from the witness by their definitions, under a key of the prover; each consumed note is placed at the expanded receiver of the note its Action creates, so that A10 holds for either value of disableCrossAddress, as in the *Ironwood Guide*’s Remark “Meaning of A10”. The net value commitments are

$$\begin{aligned} cv_1^{\text{net}} &= [0 - v] ([-1] V^{\text{Orchard}}) + [\text{rcv}_1] R^{\text{Orchard}} = [v] V^{\text{Orchard}} + [\text{rcv}_1] R^{\text{Orchard}}, \\ cv_2^{\text{net}} &= [0 - v] V^{\text{Orchard}} + [\text{rcv}_2] R^{\text{Orchard}}. \end{aligned}$$

With balancing value 0 and no burn, the *Ironwood Guide*’s Definition “Binding validating key” gives

$$bvk = cv_1^{\text{net}} + cv_2^{\text{net}} = [\text{rcv}_1 + \text{rcv}_2] R^{\text{Orchard}},$$

whose discrete logarithm $\text{bsk} = \text{rcv}_1 + \text{rcv}_2$ to the base R^{Orchard} the prover knows, so that volume’s Construction “Binding signature” produces a valid signature. Action 2 creates a ZEC note of

value v , and no consumed note carries value. For the general case, $[w] \left(\sum_i [a_i] B_i \right) = \sum_i [a_i w] B_i$ for every integer w , by bilinearity of scalar multiplication, and the same computation applies. $\square$

Definition 4.2 (Split Input and Split Action). Let $B \in \mathcal{E}^*$ with $B \neq V^{\text{Orchard}}$. A *Split Input* of Asset Base B is an OrchardZSA note $n = (d, \text{pk}_d, v, B, \rho, \psi, \text{rcm})$ (Definition 3.1), whose extracted commitment

$$\text{Extract}_p(\text{NoteCommit}_{\text{rcm}}^{\text{OrchardZSA}}(n))$$

(Definition 3.2) is a leaf of the note commitment tree under the anchor of the consuming Action, consumed by an Action that sets the witness bit `split_flag` := 1 of the statement defined in “The OrchardZSA Action statement” (§4.3). A Split Input differs from a spend of n in two respects only: its value is replaced by 0 in the net value commitment, while the commitment still opens with the true value v ; and the Action publishes the split nullifier of “Nullifiers of OrchardZSA notes” (§4.2) in place of the nullifier of n . A *Split Action* is an Action whose consumed note is a Split Input. ZIP 226 calls n “a previously issued input note (that is, a note that has previously been included in the Merkle tree)”; any leaf of Asset Base B qualifies, whether its note was created by an Action or by issuance (ZIP 226, “Terminology” and “Split Notes”). Class: specified.

Padding rule. In a bundle (*Ironwood Guide*, Definition “Bundle”) of OrchardZSA Actions, for each Custom Asset whose Asset Base B has fewer consumed notes than created notes, the sender gives each input-less Action of B a Split Input of B (ZIP 226, “Split Notes”). Actions of ZEC keep the dummy consumed notes of the *Ironwood Guide*’s Definition “Dummy notes”; Actions of Custom Assets take no dummy consumed note (ZIP 226, “Backward Compatibility”). Both regimes occur within one bundle of OrchardZSA Actions. That bundle is not the Orchard-pool bundle of a version-6 transaction: version 6 is the format of ZIP 229, which carries no ZSA fields (ZIP 229, “Non-requirements”). ZIP 226 enforces in the statement that every consumed note of a Custom Asset, split or not and of any value, is a leaf: “for Custom Assets we enforce that *every* input note to an OrchardZSA Action must be proven to exist in the set of note commitments” (“Rationale for Split Notes”). The condition is constructed in “The OrchardZSA Action statement” (§4.3), and Corollary 9.4, in “Security”, states what it yields for the Asset Base of the created note.

Remark 4.3 (Choice of the copied note). ZIP 226, “Split Notes”, lets the sender copy one of three notes of Asset Base B :

- (1) another note of B consumed by a real spend in the same bundle, but not by this Split Input;
- (2) a different unspent note of B , which is not thereby spent;
- (3) an already spent note of B , the zeroed value preventing a second spend.

It states that “we do not care about whether the previously issued note copied to create a Split Input is owned by the sender, or whether it was nullified before”. Proposition 4.12, in “The OrchardZSA Action proof” (§4.4), states which of these choices are available to a sender who does not hold the spend authority of the copied note. Under Remark 1.7, if OrchardZSA notes are in the Orchard pool, which ZIP 258, “Consensus rules from NU6.3 activation”, restricts to Actions

with `enableCrossAddress = 0`, then a Split Action whose created note is at an expanded receiver other than that of the copied note violates the cross-address rule. ZIP 226 does not address this case.

4.2 Nullifiers of OrchardZSA notes

An OrchardZSA note consumed by a real spend, with `split_flag = 0`, has the Orchard nullifier of the *Ironwood Guide*’s Definition “Nullifier”:

$$\text{nf} = \text{DeriveNullifier}_{\text{nk}}(\rho, \psi, \text{cm}) = \text{Extract}_{\text{p}}\left(\left[(\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho) + \psi) \bmod p_{\text{Pallas}}\right] K^{\text{Orchard}} + \text{cm}\right),$$

with $\text{cm} = \text{NoteCommit}_{\text{rcm}}^{\text{OrchardZSA}}(n)$ of Definition 3.2 (ZIP 226, “Note Structure and Commitment”: “generated in the same manner as in the Orchard protocol”; protocol specification, §“Computing ρ values and Nullifiers”). It depends on the Asset Base only through cm ; for a note with $\text{AssetBase} = V^{\text{Orchard}}$ it is the Orchard nullifier bit for bit. The extension of the *Ironwood Guide*’s Lemma “One nullifier per note” to OrchardZSA notes is Lemma 9.6, in “Security”.

A Split Action publishes one nullifier, which enters the nullifier set of its pool, and a nullifier must not repeat, either within a transaction or across the block chain (*Ironwood Guide*, §“Nullifier sets”; protocol specification, §“Nullifier Sets”). The copied note may be unspent, choice (2) of Remark 4.3, or spent in the same bundle, choice (1). Publishing its nullifier would, in the first case, make it unspendable and, in the second, repeat a nullifier within the transaction, which invalidates the transaction. The nullifier of a Split Input must therefore differ from that of the copied note. ZIP 226 states the purpose of the change as privacy: it is made “to prevent the identification of notes” (“Circuit Statement”).

Construction 4.4 (Split nullifier). Let

$$L^{\text{Orchard}} := \text{GroupHash}(\text{z.cash:Orchard}, L),$$

the input of the second row of Table 3. For a Split Input with nullifier deriving key nk , element ρ^{old} and commitment cm^{old} , the sender draws rseed_nf uniformly from $\mathbb{B}^{\text{Y}[32]}$ and sets

$$\psi^{\text{nf}} := \text{ToBase}\left(\text{PRFexpand}_{\text{rseed_nf}}([\text{0x0A}] \parallel \text{I2LEOSP}_{256}(\rho^{\text{old}}))\right) \in \mathbb{F}_{p_{\text{Pallas}}}.$$

The *split nullifier* is

$$\text{nf}^{\text{old}} := \text{Extract}_{\text{p}}\left(\left[(\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho^{\text{old}}) + \psi^{\text{nf}}) \bmod p_{\text{Pallas}}\right] K^{\text{Orchard}} + \text{cm}^{\text{old}} + L^{\text{Orchard}}\right).$$

Against the *Ironwood Guide*’s Definition “Nullifier” there are two changes: ψ^{old} is replaced by the fresh ψ^{nf} , and the fixed point L^{Orchard} is added before extraction. The derivation adds one row to that volume’s Construction “Domain bytes of PRF expansion”:

b	Key κ	Remainder t'	f	Output	Section
0x0A	rseed_nf	$\text{I2LEOSP}_{256}(\rho^{\text{old}})$	ToBase	ψ^{nf}	§4.2

The key rseed_nf is secret, uniform and used once. The byte 0x0A is reserved for this use by ZIP 2005’s amendment of §“Sending Notes (Orchard)” (ZIP 2005, “Changes to the Protocol Specification”; ZIP 226, “Split Notes”); the specification’s own list of expansion uses (§“Pseudo Random Functions”) does not record it, and ZIP 2005’s amendment of that list adds 0x0A and 0x0B .

It differs from the bytes 0x04 of esk, 0x05 and 0x0B of rcm and 0x09 of ψ of the Orchard sending derivations (protocol specification, §“Sending Notes (Orchard)”); Definition 3.5). Class: specified, as a sender rule; Remark 4.8 states what the statement enforces.

Proposition 4.5 (Independence of split nullifiers). *Let n be an OrchardZSA note with commitment $\text{cm} \neq \perp$, elements ρ, ψ , and nullifier nf under nk . For $\psi' \in \mathbb{F}_{p_{\text{Pallas}}}$ let $\text{nf}^{\text{split}}(\psi')$ be the split nullifier of a copy of n with $\psi^{\text{nf}} = \psi'$.*

(i) *Under the Ironwood Guide’s Assumptions “GroupHash as a random oracle” and “Discrete logarithms on Pallas”, no efficient algorithm outputs nk , a note n with the opening of its commitment, and $\psi' \in \mathbb{F}_{p_{\text{Pallas}}}$ with $\text{nf}^{\text{split}}(\psi') = \text{nf}$, except with negligible probability. This holds for every choice of ψ' , honest or not.*

(ii) *Let rseed_nf be uniform and used only for this derivation, and let $K^{\text{Orchard}} \neq \mathcal{O}$. To an adversary that knows n, nk and the full viewing key, the split nullifier is indistinguishable from $\text{Extract}_{\text{P}}(U)$ for U uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, with advantage at most*

$$\epsilon_{\text{expand}} + \epsilon_{\text{ToBase}} + \delta,$$

where ϵ_{expand} bounds the Ironwood Guide’s Assumption “Pseudorandomness of PRF expansion” for one query, $\epsilon_{\text{ToBase}} \leq p_{\text{Pallas}}/2^{512} < 2^{-257}$ is the statistical distance of ToBase on a uniform 64-byte input from the uniform distribution on $\mathbb{F}_{p_{\text{Pallas}}}$, and $\delta := (p_{\text{Vesta}} - p_{\text{Pallas}})/p_{\text{Vesta}}$ as in that volume’s Proposition “Nullifier unlinkability”.

Status (Definition 1.3): proved here for the specified construction. Part (ii) concerns honest senders, since the statement does not enforce the derivation (Remark 4.8).

Proof. (i) Write

$$s := (\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho) + \psi) \bmod p_{\text{Pallas}}, \quad s' := (\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho) + \psi') \bmod p_{\text{Pallas}},$$

and $K := K^{\text{Orchard}}, L := L^{\text{Orchard}}$. By the Ironwood Guide’s Lemma “Coordinate extraction identifies opposite points”, $\text{nf}^{\text{split}}(\psi') = \text{nf}$ means $[s'] K + \text{cm} + L = \pm([s] K + \text{cm})$. With the sign +,

$$L + [s' - s] K = \mathcal{O};$$

with the sign −,

$$L + [s + s'] K + [2] \text{cm} = \mathcal{O}.$$

Given its opening, $\text{cm} \neq \perp$ is a known integer combination of the Sinsemilla generators of its domain and of its blinding base, by the unfolding of the Ironwood Guide’s Construction “Sinsemilla hash on Pallas” used in the proof of its Proposition “Collision resistance of the Sinsemilla instances”; in either branch of Definition 3.2 these generators are values of GroupHash at inputs other than $(z.\text{cash}:\text{Orchard}, L)$ and $(z.\text{cash}:\text{Orchard}, K)$. In both cases L has coefficient 1, and its input differs from every other input involved (Lemma 2.8), so the algorithm has output a non-trivial relation among values of GroupHash at distinct inputs, which the Ironwood Guide’s Lemma “Relations among fixed generators” excludes except with negligible probability. This is the role of the offset L , which ZIP 226 does not state: without it, $\psi' = \psi$ reproduces nf .

(ii) The hybrids follow the proof of the *Ironwood Guide*'s Proposition "Nullifier unlinkability". First, $\text{PRFexpand}_{\text{rseed_nf}}$, evaluated at the single input $[\text{0x0A}] \parallel \text{I2LEOSP}_{256}(\rho)$, is replaced by a uniform 64-byte string, at cost ϵ_{expand} : the key is used nowhere else, and the rest of the game is an efficient function of the answer and of (n, nk) . Second, ToBase of that uniform string is replaced by a uniform $\psi^{\text{nf}} \in \mathbb{F}_{p_{\text{Pallas}}}$, at cost ϵ_{ToBase} (*Math Guide*, §"Uniform sampling and the bias of modular reduction"; *Ironwood Guide*, §"Pseudorandom functions and field reductions"). Translation by $\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho)$ is a bijection of $\mathbb{F}_{p_{\text{Pallas}}}$, so s' is then uniform on $\{0, \dots, p_{\text{Pallas}} - 1\}$ and independent of nk , ρ and all else. Third, s' , read in $\mathbb{F}_{p_{\text{Vesta}}}$, is replaced by a uniform scalar $u \in \mathbb{F}_{p_{\text{Vesta}}}$, at cost δ , the statistical distance computed in the cited proposition. Since $K \neq \mathcal{O}$ generates the group of prime order p_{Vesta} , the point $[u]K + \text{cm} + L$ is then uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ whatever cm and L are, and the split nullifier is $\text{Extract}_{\text{p}}$ of a uniform point. $\square$

In the model of the *Ironwood Guide*'s Assumption "GroupHash as a random oracle", the event $K^{\text{Orchard}} = \mathcal{O}$ excluded in part (ii) has probability $1/p_{\text{Vesta}}$.

4.3 The OrchardZSA Action statement

The statement is defined as a delta on the relation $\mathcal{R}_{\text{Action}}$ of the *Ironwood Guide*'s Definition "Orchard Action statement", itself a relation in the sense of the *Crypto Guide*'s Definition "Language, relation, witness" (§"Languages, relations, and witnesses"). Notation, types and conditions not named below are those of that definition; conditions A6 to A9 are cited by name and not restated.

Definition 4.6 (OrchardZSA Action statement). *Base.* The seven components of the primary input of the *Ironwood Guide*'s Definition "Orchard Action statement" other than $\text{disableCrossAddress}$, and its conditions A1 to A9. The input $\text{disableCrossAddress}$ and condition A10 are omitted (Remark 4.7).

Primary input.

$$\text{pub} = (rt, \text{cv}^{\text{net}}, \text{nf}^{\text{old}}, \text{rk}, \text{cmx}, \text{enableSpend}, \text{enableOutputs}, \text{enableZSA}),$$

with the first seven components typed as in the cited definition and $\text{enableZSA} \in \{0, 1\}$.

Auxiliary input. The auxiliary input aux of the cited definition, together with

$$\text{AssetBase} \in \mathcal{E}^*, \quad \text{is_native_asset} \in \{0, 1\}, \quad \text{split_flag} \in \{0, 1\}, \quad \psi^{\text{nf}} \in \mathbb{F}_{p_{\text{Pallas}}}.$$

Notation. Let $v' := 0$ if $\text{split_flag} = 1$ and $v' := v^{\text{old}}$ otherwise. With M^{old} and M^{new} the 1086-bit messages of the cited definition, built from the witnessed diversified bases, write $\text{NoteCommit}_{\text{rcm}}^{\text{OrchardZSA}}(\text{g}_d^*, \text{pk}_d^*, v, \rho, \psi, \text{AssetBase})$ for the commitment of Definition 3.2 with $M(n)$ replaced by the message built from these arguments: $\text{NoteCommit}_{\text{rcm}}(M)$ in the native branch, and $\text{SinemillaHashToPoint}(\text{z.cash:ZSA-NoteCommit-M}, M \parallel \text{AssetBase}^*) + [\text{rcm}]H_D$, or $\perp$, in the custom branch. Let L^{Orchard} be as in Construction 4.4.

Relation. The relation $\mathcal{R}_{\text{ActionZSA}}$ is the set of pairs (pub, aux) of the types above that satisfy the following conditions.

A1'. Old note commitment integrity:

$$\begin{aligned} & \text{NoteCommit}_{\text{rcm}^{\text{old}}}^{\text{OrchardZSA}}((\text{g}_d^{\text{old}})^*, (\text{pk}_d^{\text{old}})^*, v^{\text{old}}, \rho^{\text{old}}, \psi^{\text{old}}, \text{AssetBase}) \\ & \in \{\text{cm}^{\text{old}}, \perp\}. \end{aligned}$$

A2'. New note commitment integrity:

$$\text{Extract}_{\mathbb{P}}^{\perp}(\text{NoteCommit}_{\text{rcm}^{\text{new}}}^{\text{OrchardZSA}}((g_d^{\text{new}})^*, (pk_d^{\text{new}})^*, v^{\text{new}}, \rho^{\text{new}}, \psi^{\text{new}}, \text{AssetBase})) \\ \in \{\text{cmx}, \perp\},$$

with $\rho^{\text{new}} := \text{nf}^{\text{old}}$ and the same AssetBase as in A1'.

C1. Native indicator: $\text{is_native_asset} = 1$ if and only if $\text{AssetBase} = V^{\text{Orchard}}$.

C2. Pause: if $\text{enableZSA} = 0$, then $\text{is_native_asset} = 1$.

A3'. Merkle path validity: ($v^{\text{old}} = 0$ and $\text{is_native_asset} = 1$), or $\text{root} = \text{rt}$, or some evaluation of SinsemillaHash in the fold returns $\perp$; the value root , the fold and the encoding bits c are those of A3.

A4'. Value commitment integrity:

$$\text{cv}^{\text{net}} = \text{ValueCommit}_{\text{rcv}}^{\text{OrchardZSA}}(\text{AssetBase}, v' - v^{\text{new}}) \\ = [v' - v^{\text{new}}] \text{AssetBase} + [\text{rcv}] R^{\text{Orchard}},$$

a variable-base multiplication by the witnessed AssetBase (Definition 3.8).

C3. Split only for Custom Assets: if $\text{split_flag} = 1$, then $\text{is_native_asset} = 0$.

A5'. Nullifier integrity: if $\text{split_flag} = 0$, then $\psi^{\text{nf}} = \psi^{\text{old}}$ and

$$\text{nf}^{\text{old}} = \text{DeriveNullifier}_{\text{nk}}(\rho^{\text{old}}, \psi^{\text{old}}, \text{cm}^{\text{old}});$$

if $\text{split_flag} = 1$, then

$$\text{nf}^{\text{old}} = \text{Extract}_{\mathbb{P}}([(\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho^{\text{old}}) + \psi^{\text{nf}}) \bmod p_{\text{Pallas}}] K^{\text{Orchard}} \\ + \text{cm}^{\text{old}} + L^{\text{Orchard}}),$$

the split nullifier of Construction 4.4.

A6, A7. Spend authority and diversified address integrity: unchanged and unconditional; no factor of split_flag relaxes them.

A8, A9. Enable spend flag and enable output flag: unchanged; A8 reads v^{old} , not v' , and A9 reads v^{new} .

(ZIP 226, “Circuit Statement”, paragraphs “Asset Base Equality”, “The enableZSA Flag”, “Value Commitment Correctness” and “Asset Identifier Consistency for Split Actions”; ZIP 226, “Split Notes” and “Note Structure and Commitment”; protocol specification, §“Action Statement (Orchard)”). Class: specified at statement level; its status is stated in Remark 4.7.

The following consequences are read off Definition 4.6, Definition 3.2 and Definition 2.15.

One witness. Conditions A1', A2' and A4' use the single witnessed AssetBase, so the consumed

and the created note of an Action have one Asset Base, and its net value commitment is on that base.

Native branch. By C1 and the case split of Definition 3.2, an Action with $\text{AssetBase} = V^{\text{Orchard}}$ and $\text{split_flag} = 0$ satisfies $A1'$, $A2'$, $A4'$ and $A5'$ exactly when $\psi^{\text{nf}} = \psi^{\text{old}}$ and it satisfies the Orchard $A1$, $A2$, $A4$ and $A5$.

Pause. With $\text{enableZSA} = 0$, C2 and C1 force $\text{AssetBase} = V^{\text{Orchard}}$, and C3 then forces $\text{split_flag} = 0$, so the relation admits only Orchard Actions.

Membership. Since $0 \leq v^{\text{old}} < 2^{64} < p_{\text{Pallas}}$, the first disjunct of $A3'$ holds only for a consumed ZEC note of value 0. Every consumed note of a Custom Asset, split or not and of any value, satisfies $\text{root} = \text{rt}$ or the $\perp$ clause. The $\perp$ clause gives no escape: for witnesses extracted under Assumption 4.9, the argument of the *Ironwood Guide*'s Remark "1-weakened conditions", by that volume's Proposition "Collision resistance of the Sinsemilla instances" (iii) under its Assumptions "Discrete logarithms on Pallas" and "GroupHash as a random oracle", excludes it except with negligible probability; the same holds for the $\perp$ cases of $A1'$ and $A2'$ in both branches of $\text{NoteCommit}^{\text{OrchardZSA}}$, by Proposition 3.3(c) for the custom branch.

Split Actions. By $A4'$ the copied value never enters cv^{net} . By A8, which reads v^{old} , a Split Action that copies a note of non-zero value requires $\text{enableSpends} = 1$, while one that copies a note of value 0 does not.

These consequences are direct from the cited definitions and from the *Ironwood Guide*'s Remark "1-weakened conditions", with Assumption 4.9 in place of that volume's knowledge-soundness assumption (ZIP 226, "Circuit Statement" and "Overview"). Table 5 pairs each condition with the Orchard condition it replaces and its ZIP 226 paragraph; Figure 1 draws the new witnesses and the conditions that read them.

Table 5. Conditions of the OrchardZSA Action statement of Definition 4.6: each condition, the condition of the *Ironwood Guide*'s Definition "Orchard Action statement" that it replaces, and the paragraph of ZIP 226, "Circuit Statement", that states it.

Condition	Replaces	ZIP 226, "Circuit Statement", paragraph
$A1'$	A1	"Asset Base Equality"
$A2'$	A2	"Asset Base Equality"
C1	none	"Asset Base Equality" (the witness is <code>_native_asset</code>)
C2	none	"The enableZSA Flag"
$A3'$	A3	"Asset Identifier Consistency for Split Actions" (Merkle Path Validity)
$A4'$	A4	"Value Commitment Correctness"; "Asset Identifier Consistency for Split Actions" (v')
C3	none	"Asset Identifier Consistency for Split Actions"
$A5'$	A5	"Asset Identifier Consistency for Split Actions" (Nullifier Integrity), with "Split Notes" for the formula and "Note Structure and Commitment" for $\text{split_flag} = 0$
A6 to A9	themselves	none (unchanged)
A10	omitted	none (Remark 4.7)

Remark 4.7 (Status of the OrchardZSA Action statement). The delta is specified at statement level (ZIP 226, Draft, "Circuit Statement"). ZIP 226 states it against the specification's statement with seven primary inputs and conditions A1 to A9. The current Orchard statement has the eighth primary input `disableCrossAddress` and condition A10 (*Ironwood Guide*, Definition "Orchard Action statement" and Remark "Status"; protocol specification, §"Action Descriptions";

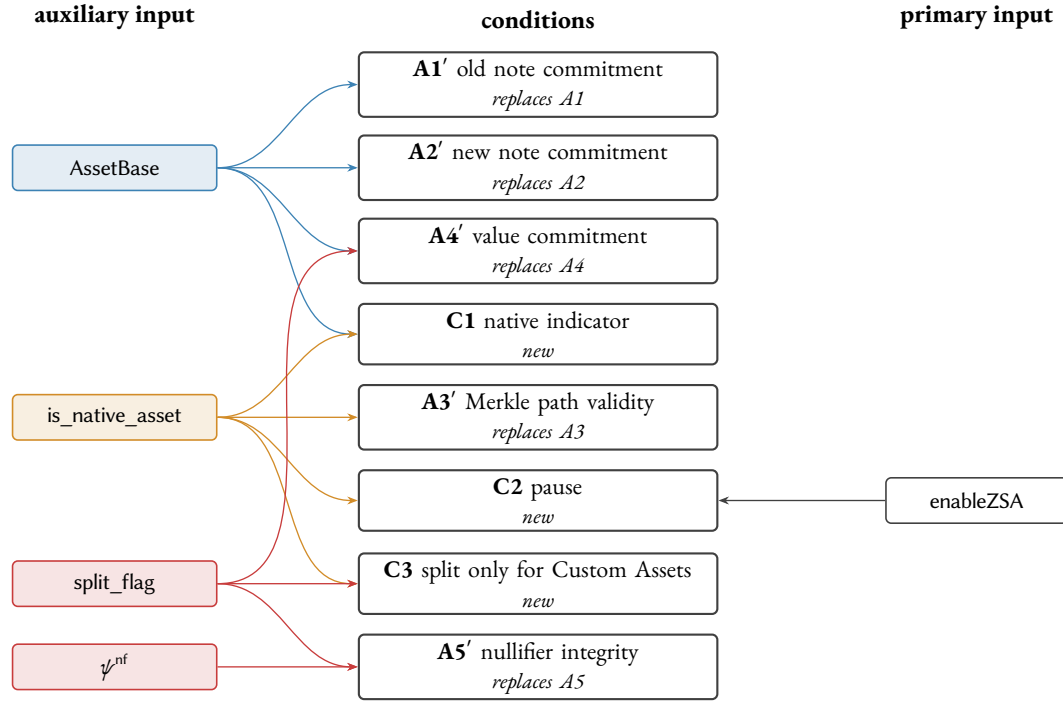


Figure 1. The witnessed Asset Base in the OrchardZSA Action statement of Definition 4.6. The four new witnesses are on the left, the new primary input `enableZSA` on the right, and the conditions that read them in the centre, each tagged with the Orchard condition it replaces or as new. One witness `AssetBase` enters `A1'`, `A2'` and `A4'`, which forces equal Asset Bases of the consumed and the created note and a value commitment on that base; through `C1` and `A3'`, only a consumed ZEC note of value 0 escapes membership.

ZIP 258, “Consensus rules from NU6.3 activation”), which ZIP 226 never mentions. The composition of the delta with `A10` is therefore an open problem (Definition 1.2). The statement uses `enableZSA`, which ZIP 226 never lists as a primary input. The encoded instance of the current Orchard statement has ten elements, the nine that encode its first seven primary inputs and `disableCrossAddress` (*Ironwood Guide*, §“The Action statement”); no live text places `enableZSA` in it, so its position is an open problem. ZIP 226 defers “All modifications in the Circuit” to an unversioned document outside the ZIPs, “Modifications to the Orchard circuit for the OrchardZSA Protocol”; no ZIP fixes a complete OrchardZSA circuit, which is designed but unspecified. The reading of `A8` on v^{old} is specified by the delta, which leaves `A8` unchanged; only its confirmation in that circuit document is designed but unspecified, and Table 7 records it.

Remark 4.8 (What the OrchardZSA statement does not check). Items (i) to (iii) of the *Ironwood Guide*’s Remark “What the statement does not check” carry over. In addition:

- (iv) The witness ψ^{nf} is constrained only through `A5'` to the published nullifier. For `split_flag = 1` only the equality $\psi^{\text{nf}} = \psi^{\text{old}}$ is switched off, and no condition derives ψ^{nf} from a seed. The derivation of Construction 4.4 is therefore a sender rule whose formula is specified and which nothing enforces: `rseed_nf` is transmitted nowhere, and no text proposes an enforcement. Proposition 4.5(i) holds for every ψ^{nf} ; part (ii) holds for senders that follow the rule.

- (v) The derivations of rcm and ψ from rseed are unchecked, as for Orchard.
 - (vi) The witnessed AssetBase is checked only against the consumed leaf, through $A1'$ and $A3'$, and never against a record of issued Assets. Lemma 9.3, in “Security”, concerns the leaves.
 - (vii) The statement does not check that the prover owns the copied note of a Split Input beyond $A6$ and $A7$; this is the subject of Proposition 4.12.
- (ZIP 226, “Split Notes” and “Circuit Statement”).

4.4 The OrchardZSA Action proof

The *Ironwood Guide*’s Assumptions “Knowledge soundness of the Action proof” and “Zero knowledge of the Action proof” name the relation $\mathcal{R}_{\text{Action}}$ and its verifying key. The corresponding assumptions for $\mathcal{R}_{\text{ActionZSA}}$ are stated here in the same form; the *OrchardZSA verifying key* denotes the verifying key of a circuit for $\mathcal{R}_{\text{ActionZSA}}$, which no ZIP names.

Assumption 4.9 (Knowledge soundness of the OrchardZSA Action proof). Let the hash of the Fiat–Shamir transform be modelled as a random oracle. For every efficient adversary that outputs primary inputs $\text{pub}_1, \dots, \text{pub}_n$ and an aggregate proof π , there is an efficient extractor, with access to the adversary and to its random-oracle queries, that outputs auxiliary inputs $\text{aux}_1, \dots, \text{aux}_n$ such that the probability of the event

$$\begin{aligned} &\pi \text{ is accepted for } (\text{pub}_1, \dots, \text{pub}_n) \text{ under the OrchardZSA verifying key} \\ &\text{and } (\text{pub}_i, \text{aux}_i) \notin \mathcal{R}_{\text{ActionZSA}} \text{ for some } i \end{aligned}$$

is negligible. This is the knowledge-soundness clause of the *Crypto Guide*’s Definition “SNARK” (§“SNARKs: succinct non-interactive arguments of knowledge”) for the relation of Definition 4.6, for a Halo 2 proof (*Halo 2 Guide*, §“The Halo 2 proof system” and §“A rigorous treatment of knowledge soundness”), the primary input bound to the proof as in the *Halo 2 Guide*, §“Instance columns and binding the public statement”. It includes that every satisfying assignment of the circuit yields a witness of $\mathcal{R}_{\text{ActionZSA}}$; in particular, the multiplication of $A4'$ is by the witnessed AssetBase and nothing else. No ZIP names the circuit or the verifying key: the assumption concerns a designed but unspecified object.

Assumption 4.10 (Zero knowledge of the OrchardZSA Action proof). Let the hash of the Fiat–Shamir transform be modelled as a programmable random oracle. There is an efficient simulator that, given only $\text{pub}_1, \dots, \text{pub}_n$ and programming the random oracle, outputs an aggregate proof statistically indistinguishable from an honest proof under the OrchardZSA verifying key, for any $\text{aux}_1, \dots, \text{aux}_n$ with $(\text{pub}_i, \text{aux}_i) \in \mathcal{R}_{\text{ActionZSA}}$ for every i , against every distinguisher that makes polynomially many random-oracle queries. This is zero knowledge of statistical grade in the non-interactive, random-oracle form of the *Crypto Guide*’s Definition “Zero knowledge” (§“The simulation paradigm and zero knowledge”), for a Halo 2 proof (*Halo 2 Guide*, §“Zero knowledge: hiding the witness in Halo 2”). It concerns the same designed but unspecified circuit and key.

Remark 4.11 (Circuit, verifying key and proof length). The statement is specified at delta level (ZIP 226). Its arithmetisation exists only in the document outside the ZIPs named in Remark 4.7, so the circuit and its verifying key are designed but unspecified, and Assumptions 4.9 and 4.10 are the only hypotheses through which later results depend on them. Condition A4' is sound only if the circuit multiplies by the witnessed AssetBase and nothing else. The Orchard variable-base scalar multiplication was corrected at NU6.2 because it left the base under-constrained (ZIP 257, “NU6.2 deployment”). ZIP 226, which turns the value base into a witness (“Value Commitment Correctness”), does not mention the correction, which any OrchardZSA circuit must carry. ZIP 257 made the Orchard proof length $2720 + 2272n$ bytes, for n Actions, a consensus rule (“Canonical Orchard Action proof length”). No ZIP fixes an OrchardZSA proof length, which the circuit determines and which no live text contradicts: it is designed but unspecified, as the circuit and the verifying key are.

Proposition 4.12 (Ownership of Split Inputs). *Consider the Ironwood Guide’s Definition “Spend-authority experiment” with $\mathcal{R}_{\text{ActionZSA}}$ in place of $\mathcal{R}_{\text{Action}}$: a key generated with `use_qsk = false`, the adversary holding its full viewing key but not ask. Under Assumption 4.9 and the Ironwood Guide’s Assumptions “Discrete logarithms on Pallas”, “GroupHash as a random oracle”, “The RedPallas hash as a random oracle” and “Pseudorandomness of PRF expansion”, no efficient adversary outputs, except with negligible probability, a block chain with an accepted Split Action whose witness copies a leaf that is a note of the key, and whose spend-authorisation signature is valid under its rk on a signature digest for which it made no signing query with that rk. Consequently the sentence of ZIP 226, “Split Notes”, “we do not care about whether the previously issued note copied to create a Split Input is owned by the sender, or whether it was nullified before” holds for spentness and not for ownership: a sender can use choices (2) and (3) of Remark 4.3 only with the spend authority of the copied note, except for a note whose key is public, such as a reference note (Proposition 6.11). Status (Definition 1.3): proved here from specified conditions; conditional on Assumption 4.9, which concerns the designed but unspecified circuit.*

Proof. The proof is a reduction to the proof of the Ironwood Guide’s Theorem “Spend authority”. By Assumption 4.9 the extractor yields, for the accepted Split Action, a witness of $\mathcal{R}_{\text{ActionZSA}}$ with `split_flag = 1`. By C3, `is_native_asset = 0`, so A3' gives $root = rt$ outside its $\perp$ case, which is excluded as in the consequences of Definition 4.6; with the anchor rule, the Ironwood Guide’s Proposition “Membership soundness” makes $\text{Extract}_p(\text{cm}^{\text{old}})$ a leaf. Condition A1' with Proposition 3.3(b) makes the witnessed consumed note the note of that leaf, so its witnessed expanded receiver $(g_d^{\text{old}}, pk_d^{\text{old}})$ is the receiver of the leaf’s note, a note of the key; the witness therefore consumes a note of the key in the sense of the experiment. Conditions A6 and A7, unchanged and unconditional, then bind rk to a randomisation of $\pm ak^P$ of that key, by the Ironwood Guide’s Proposition “Binding of ivk to (ak, nk)” and Lemma “Coordinate extraction identifies opposite points”, and a valid signature under rk without a matching signing query contradicts that volume’s Proposition “Unforgeability under re-randomisation”, exactly as in steps (1) to (5) of the proof of Theorem “Spend authority”. Neither `split_flag` nor v' enters A6 or A7, the only conditions that proof uses; A1' and membership serve here only to identify the witnessed receiver with that of the copied leaf’s note, so the steps apply unchanged.

For spentness: by $A4'$ the copied value is not committed, and by Proposition 4.5(i) the copied note's nullifier is not published, except with negligible probability. Choice (1) of Remark 4.3 copies a note that the same bundle spends, so its sender holds that note's spend authority in any case. $\square$

4.5 Action Groups

Definition 4.13 (Action Group). An *Action Group* is a non-empty finite sequence of OrchardZSA Actions together with one anchor rt , the flags `enableSpends`, `enableOutputs` and `enableZSA` in $\{0, 1\}$, and one aggregate proof for the relation of Definition 4.6 at the primary inputs

$$(rt, cv_i^{\text{net}}, nf_i, rk_i, cmx_i, \text{enableSpends}, \text{enableOutputs}, \text{enableZSA})$$

of its Actions i . Each Action carries the public fields $(cv^{\text{net}}, nf, rk, cmx, epk, C_{\text{enc}}, C_{\text{out}})$ of an Orchard Action description (*Ironwood Guide*, Definition “Action description”) and its own spend-authorisation signature under rk . The term is that of ZIP 228, “Terminology”: Actions that share the tuple

$$(rt, \text{enableSpends}, \text{enableOutputs}, \text{enableZSA}, \text{nAGExpiryHeight}).$$

ZIP 227 requires $\text{nAGExpiryHeight} = 0$ and at most one Action Group per transaction (“Specification: Consensus Rule Changes”), so the fifth component is constant and is omitted. The nullifier $nf_{0,0}$ of the first Action of the Action Group is the value that issuance uses (ZIP 227, “Computation of ρ ”). The Action Group has no burn set; the remaining components of a transaction's bundle are defined in “Burn sets and OrchardZSA bundles” (§5.1). Class: specified.

The structure is specified by ZIPs 226 to 228, all Draft. Its encoding exists only in ZIP 230, which is Withdrawn; the layout that ZIP 230 calls version 6 is not the version 6 of ZIP 229, which has no Action Groups (ZIP 229, “Transaction Format”). The byte lengths of the note ciphertext and of an Action depend on a plaintext encoding that no current ZIP defines (Definition 3.7). The flag `enableZSA` has no flag bit in any current format. ZIP 230 assigns it bit 2 of the Orchard flags byte, which ZIP 229, “Consensus Rules”, reserves as 0 in version 6 and which is `enableCrossAddress` in the Ironwood flags byte (*Ironwood Guide*, §“Bundle flags”; ZIP 258, “Consensus rules from NU6.3 activation”). The design conflicts with live text, so the flag bit is an open problem (Definition 1.2).

5 Burn and the binding signature

This section defines the burn set and the OrchardZSA bundle that it completes (§5.1); constructs the binding validating key, which subtracts the burnt amounts Asset by Asset (§5.2); and proves that a verifying binding signature forces per-Asset balance whenever every witnessed Asset Base and every Asset Base of a burn pair is a group-hash output at a suitably distinct input (§5.3). The premise on the bases concerns a single bundle; it is discharged for valid chains in “Security” (§9). The constructions are taken from ZIP 226 and ZIP 227, both Draft. The Orchard binding signa-

ture that they extend is that of the *Ironwood Guide*, §“The binding signature”, and is cited, not restated.

5.1 Burn sets and OrchardZSA bundles

Definition 5.1 (Burn set). *Burn* destroys a stated amount of a Custom Asset (Definition 1.5): it sends nothing to any address, lowers the amount of the Asset in circulation, and publishes the Asset Base and the amount, so that the destruction is publicly verifiable. A *burn set* assetBurn is a finite set of pairs $(\text{AssetBase}, v)$ with $\text{AssetBase} \in \mathcal{E}^*$ and $v \in \mathbb{Z}$. The author of a transaction adds one pair for each Custom Asset that the transaction burns, v being the amount burnt. The constant

$$\text{MAX_BURN_VALUE} := 2^{63} - 1 = 9\,223\,372\,036\,854\,775\,807$$

bounds the amount of one Custom Asset burnt in one transaction, so that every burnt amount is representable in a signed 64-bit balance field. A burn set is *valid* when it satisfies the following rules.

- (B1) For every pair, $\text{AssetBase} \neq V^{\text{Orchard}}$: the Native Asset Base (Definition 2.15), of ZEC and of TAZ alike, is never burnt.
- (B2) For every pair, $0 < v \leq \text{MAX_BURN_VALUE}$.
- (B3) Every Asset Base occurs in at most one pair. The *burnt amount* v_B of a point B is therefore well defined: the v of the pair with $\text{AssetBase} = B$, or 0 if there is none.

Rules (B1) to (B3) constrain the pairs only. Rule (B1) excludes the one point V^{Orchard} and no other, and no rule requires AssetBase to be the Asset Base of an issued Asset. The state rule that a burn cannot exceed the amount of the Asset in circulation is ZIP 227’s; it is rule (T4) of Construction 7.3, in “State transition of a transaction” (§7.2) (ZIP 226, “Burn Mechanism”, “Additional Consensus Rules for the assetBurn set” and “Rationale for MAX_BURN_VALUE ”; ZIP 227, “Specification: Consensus Rule Changes”). Class: specified; the carrier of the burn set is stated in “Transaction formats” (§8.1).

Definition 5.2 (OrchardZSA bundle). The *OrchardZSA bundle* of a transaction consists of at most one Action Group (Definition 4.13), one burn set assetBurn satisfying (B1) to (B3), one balancing value v^{balance} and one binding signature. It extends the *Ironwood Guide*’s Definition “Bundle” by the burn set; its anchor, flags and proof are those of the Action Group. ZIP 226 places one burn set in the transaction, and ZIP 227 limits the transaction to at most one Action Group, so the two readings coincide (ZIP 226, “Burn Mechanism”; ZIP 227, “Specification: Consensus Rule Changes”). Class: the structure is specified; its encoding, one burn set per Action Group in withdrawn ZIP 230, belongs to the carrier and is classified in Remark 8.1.

Remark 5.3 (No transparent pool for Custom Assets). ZIPs 226 and 227 leave the transparent protocol unchanged and single-asset. Transparent inputs and outputs, and the *Ironwood Guide*’s transparent transaction value pool (Definition “Transparent transaction value pool”), carry the Native Asset only; Custom Assets have no transparent value pool and cannot be unshielded by

a transfer. ZIP 226 conditions this on future consensus changes, which no ZIP specifies (ZIP 226, “Burn Mechanism”, note; “Rationale for Value Balance Verification”: “All Custom Assets are contained within the shielded pool, and cannot be unshielded via a regular transfer”). Structurally, the binding validating key of the next subsection (Definition 5.4) has, besides the Actions’ net value commitments, only the balancing value at V^{Orchard} and one burn term per pair; a non-zero net amount of a Custom Asset in a bundle can therefore be absorbed only by a burn pair of that Asset. The computational statement is Proposition 5.6. Class: specified.

5.2 The binding validating key

Definition 5.4 (Binding validating key of an OrchardZSA bundle). Let an OrchardZSA bundle have n Actions with net value commitments $cv_1^{\text{net}}, \dots, cv_n^{\text{net}}$ (Definition 3.8), trapdoors $rcv_1, \dots, rcv_n$, balancing value v^{balance} and burn set assetBurn . The *binding signing key* is unchanged:

$$\text{bsk} := rcv_1 + \dots + rcv_n \in \mathbb{F}_{p_{\text{Vesta}}}$$

(*Ironwood Guide*, Definition “Binding signing key”). The verifier computes, from public fields only, the *binding validating key*

$$\text{bvk} := \left(\sum_{i=1}^n cv_i^{\text{net}} \right) - [v^{\text{balance}}] V^{\text{Orchard}} - \sum_{(\text{AssetBase}, v) \in \text{assetBurn}} [v] \text{AssetBase},$$

where the subtracted points are the zero-trapdoor commitments

$$\text{ValueCommit}_0^{\text{OrchardZSA}}(V^{\text{Orchard}}, v^{\text{balance}}) \quad \text{and} \quad \text{ValueCommit}_0^{\text{OrchardZSA}}(\text{AssetBase}, v),$$

each signed integer entering as its residue modulo p_{Vesta} . Against the *Ironwood Guide*’s Definition “Binding validating key” two things are new: the bases of the Actions’ commitments vary, being the witnessed Asset Bases, and one burn term is subtracted per pair.

The balancing value keeps the meaning of the *Ironwood Guide*’s Definitions “Transparent transaction value pool” and “Balancing value”: the net flow of ZEC between the pool and the transparent transaction value pool, a signed integer in $\{-2^{63}, \dots, 2^{63} - 1\}$. It enters bvk at V^{Orchard} only. Its meaning rests on the key of ZIP 226; its encoding, the field `valueBalanceOrchard` of withdrawn ZIP 230, belongs to the carrier.

The binding signature is that of the *Ironwood Guide*’s Construction “Binding signature”, the binding-signature instance of RedPallas on the base R^{Orchard} (*Ironwood Guide*, §“The Red-Pallas signature scheme”): the author signs `SigHash` with bsk , and the verifier MUST validate the signature on `SigHash` under bvk . Here `SigHash` is the signature digest of the transaction (*Ironwood Guide*, Definition “Signature digest”) as modified for OrchardZSA; the modifications and their class are stated in “Transaction digests and the signature message” (§8.2, Remark 8.3) (ZIP 226, “Value Balance Verification” and “Value Commitment”; protocol specification, §“Balance and Binding Signature (Orchard)”). Class: specified; the signed message is not.

For an honestly constructed bundle each cv_i^{net} equals

$$[v'_i - v_i^{\text{new}}] \text{AssetBase}_i + [rcv_i] R^{\text{Orchard}},$$

by condition A4' of Definition 4.6. By the additive homomorphism (*Crypto Guide*, §“The Pedersen commitment”, Theorem “Additive homomorphism”), the trapdoor terms sum to $[\text{bsk}] R^{\text{Orchard}}$, and the value terms group by Asset Base. When the net values of each Custom Asset sum to its burnt amount and those of ZEC to v^{balance} , every value term cancels, so $\text{bvk} = [\text{bsk}] R^{\text{Orchard}}$, the pair (bsk, bvk) is a key pair of the instance, and the honest signature is accepted, as in the paragraph after the *Ironwood Guide*’s Construction “Binding signature”. In the notation of ZIP 226, with S_{CA} the indices of the Actions of Custom Assets, a correctly constructed bundle satisfies

$$\sum_{j \in S_{\text{CA}}} \text{cv}_j^{\text{net}} - \sum_{(\text{AssetBase}, v) \in \text{AssetBurn}} [v] \text{AssetBase} = \left[\sum_{j \in S_{\text{CA}}} \text{rcv}_j \right] R^{\text{Orchard}}$$

(ZIP 226, “Rationale for Value Balance Verification”). For a Split Action $v' = 0$, so the value of the copied note never enters bvk : the commitment $[0 - v^{\text{new}}] \text{AssetBase} + [\text{rcv}] R^{\text{Orchard}}$ cancels against the value of the notes of the same Asset consumed by real spends in other Actions (ZIP 226, “Split Notes”).

The key bvk is computed from the Actions’ commitments, the balancing value and the burn set, and of these only the burn set names an Asset, by publishing each burnt Asset Base with its amount. The hiding of the Actions’ Asset Bases is Theorem 9.11, in “Security”.

5.3 Per-Asset balance

Lemma 5.5 (Modular per-Asset balance is integer balance). *Let $n \leq 62\,500$. For $1 \leq i \leq n$ let v'_i and v_i^{new} be integers in $\{0, \dots, 2^{64} - 1\}$ and AssetBase_i a point. Let assetBurn satisfy (B2) and (B3), with burnt amounts v_B , and let v^{balance} be an integer in $\{-2^{63}, \dots, 2^{63} - 1\}$. For a point B put*

$$\begin{aligned} c_B &:= \sum_{i: \text{AssetBase}_i = B} (v'_i - v_i^{\text{new}}) - v_B & (B \neq V^{\text{Orchard}}), \\ c_{V^{\text{Orchard}}} &:= \sum_{i: \text{AssetBase}_i = V^{\text{Orchard}}} (v'_i - v_i^{\text{new}}) - v^{\text{balance}}, \end{aligned}$$

rule (B1) giving $v_{V^{\text{Orchard}}} = 0$ for a valid burn set. If $c_B \equiv 0 \pmod{p_{\text{Vesta}}}$, then $c_B = 0$ as an integer.

*The hypotheses hold for every OrchardZSA bundle of a valid block. Every Action carries its own 32-byte nullifier (*Ironwood Guide*, Definition “Action description”; protocol specification, §“Action Description Encoding and Consensus”), and a block, hence a transaction, holds at most 2 000 000 bytes (protocol specification, §“Block Header Encoding and Consensus”), so*

$$n \leq 2\,000\,000/32 = 62\,500 < 2^{16}$$

whatever the carrier. The values are 64-bit by Definition 3.1, with $v' \in \{0, v^{\text{old}}\}$ by the notation of Definition 4.6; burnt amounts are bounded by (B2), and the balancing value by Definition 5.4. No carrier-specific bound on the number of Actions is needed. Status (Definition 1.3): proved here; conditional on no unspecified object.

Proof. The argument is that of the *Ironwood Guide*'s Lemma "Modular balance is integer balance". Each difference $v'_i - v_i^{\text{new}}$ has absolute value at most $2^{64} - 1$, and $n \leq 2^{16} - 1$. For $B \neq V^{\text{Orchard}}$, rule (B2) gives $0 \leq v_B \leq 2^{63} - 1$, so

$$|c_B| \leq (2^{16} - 1)(2^{64} - 1) + (2^{63} - 1) = 1\,208\,916\,596\,242\,592\,319\,864\,832;$$

for V^{Orchard} , with $|v^{\text{balance}}| \leq 2^{63}$,

$$|c_{V^{\text{Orchard}}}| \leq (2^{16} - 1)(2^{64} - 1) + 2^{63} = 1\,208\,916\,596\,242\,592\,319\,864\,833,$$

the upper end of the range in the protocol specification, §"Balance and Binding Signature (Orchard)". Both bounds are below $2^{80} < 2^{253} \leq (p_{\text{Vesta}} - 1)/2$ (*Ironwood Guide*, §"Fields, groups, and encodings"; protocol specification, §"Pallas and Vesta"). The hypothesis makes c_B a multiple of p_{Vesta} , the order of the Pallas group, and the only such multiple of absolute value below p_{Vesta} is 0. $\square$

The protocol specification's argument in §"Balance and Binding Signature (Orchard)" bounds the number of Actions by $2^{16} - 1$ through a separate consensus rule, which it notes is redundant given the 2 MB transaction size limit; the lemma uses the block size itself.

Fix an OrchardZSA bundle as in Definition 5.4, and let $\mathfrak{B}$ be the set of points consisting of V^{Orchard} , the Asset Bases of the Actions and the Asset Bases of the burn pairs. An *opening* of cv_i^{net} is a triple (B_i, u_i, r_i) of a point $B_i \in \mathfrak{B}$, an integer u_i and a scalar $r_i \in \mathbb{F}_{p_{\text{Vesta}}}$ with

$$\text{cv}_i^{\text{net}} = [u_i \bmod p_{\text{Vesta}}] B_i + [r_i] R^{\text{Orchard}}.$$

For openings, c_B is defined for $B \in \mathfrak{B}$ as in Lemma 5.5, with B_i in place of AssetBase_i and u_i in place of $v'_i - v_i^{\text{new}}$. The set $\mathfrak{B}$ meets the *premise on the bases* when every $B \in \mathfrak{B}$ other than V^{Orchard} is given with an input x_B of GroupHash such that $B = \text{GroupHash}(x_B)$ and x_B differs from the inputs $(z.\text{cash}:\text{Orchard-cv}, v)$ of V^{Orchard} and $(z.\text{cash}:\text{Orchard-cv}, r)$ of R^{Orchard} . As in the *Ironwood Guide*, a *binding signature* under a point X is a signature valid under X in the binding-signature instance of RedPallas .

Proposition 5.6 (Per-Asset balance and key knowledge). *For the bundle fixed above:*

- (i) *Identity.* If $\text{cv}_i^{\text{net}} = [v'_i - v_i^{\text{new}}] \text{AssetBase}_i + [\text{rcv}_i] R^{\text{Orchard}}$ for every i , and $\text{bsk} := \sum_i \text{rcv}_i$, then

$$\text{bvk} - [\text{bsk}] R^{\text{Orchard}} = \sum_{B \in \mathfrak{B}} [c_B] B;$$

hence $c_B \equiv 0 \pmod{p_{\text{Vesta}}}$ for every $B \in \mathfrak{B}$ implies $\text{bvk} = [\text{bsk}] R^{\text{Orchard}}$.

- (ii) *Knowledge.* Given openings of every cv_i^{net} and $b \in \mathbb{F}_{p_{\text{Vesta}}}$ with $\text{bvk} = [b] R^{\text{Orchard}}$,

$$\sum_{B \in \mathfrak{B}} [c_B] B + \left[\sum_{i=1}^n r_i - b \right] R^{\text{Orchard}} = \mathcal{O}.$$

If $\mathfrak{B}$ meets the *premise on the bases* and some c_B is non-zero modulo p_{Vesta} , this is a non-trivial relation among values of GroupHash at distinct inputs.

Consequently, under the Ironwood Guide's Assumptions "Discrete logarithms on Pallas", "GroupHash as a random oracle" and "The RedPallas hash as a random oracle", an efficient party that outputs a bundle whose set $\mathfrak{B}$ meets the premise on the bases, openings of every net value commitment and a binding signature under the resulting bvk on a message of its choice satisfies $c_B \equiv 0 \pmod{p_{\text{Vesta}}}$ for every $B \in \mathfrak{B}$, and knows $b = \sum_i r_i$, except with negligible probability. The statement holds for every message and does not depend on the signed message.

Under Assumption 4.9 in addition, let an efficient adversary output OrchardZSA bundles of valid blocks, hence with at most 62 500 Actions each, whose Action proofs and binding signatures verify and whose witnessed and burn-pair Asset Bases meet the premise on the bases. Then, except with negligible probability, as integers:

- (a) for every Asset Base $B \neq V^{\text{Orchard}}$, $\sum_{i: \text{AssetBase}_i=B} (v'_i - v_i^{\text{new}}) = v_B$, which is 0 when B is not burnt;
- (b) $\sum_{i: \text{AssetBase}_i=V^{\text{Orchard}}} (v'_i - v_i^{\text{new}}) = v^{\text{balance}}$;
- (c) no burn pair has an Asset Base that no Action witnesses;
- (d) the signer knows the trapdoor sum $\sum_i \text{rcv}_i$.

Here AssetBase_i , v'_i , v_i^{new} and rcv_i are read from the extracted witnesses. Status (Definition 1.3): proved here for the specified bvk ; conditional on Assumption 4.9, which concerns the designed but unspecified circuit, and not on Assumption 6.13. The premise on the bases is discharged for every accepted bundle of a valid chain only in "Security", in Theorem 9.5: for witnessed bases by Corollary 9.4, for burn-pair bases by rule (T4) of Construction 7.3 and Lemma 7.7. ZIP 226 argues per-Asset balance in rationale only ("Rationale for Value Balance Verification"), and that argument stays designed but unspecified.

Proof. (i) Expand each cv_i^{net} and apply the additive homomorphism (Crypto Guide, §"The Pederсен commitment", Theorem "Additive homomorphism"):

$$\sum_{i=1}^n \text{cv}_i^{\text{net}} = \sum_{B \in \mathfrak{B}} \left[\sum_{i: \text{AssetBase}_i=B} (v'_i - v_i^{\text{new}}) \right] B + [\text{bsk}] R^{\text{Orchard}}.$$

By (B3) each Asset Base of a burn pair occurs in exactly one subtracted term $[v_B] B$, and by (B1) none of them is V^{Orchard} , whose only subtracted term is $[v^{\text{balance}}] V^{\text{Orchard}}$. Subtracting these terms as in Definition 5.4 and grouping by base gives the identity. If every c_B vanishes modulo p_{Vesta} , the right-hand side is $\mathcal{O}$.

(ii) With the openings in place of the honest values, the same grouping gives $\text{bvk} = \sum_B [c_B] B + [\sum_i r_i] R^{\text{Orchard}}$; equating with $[b] R^{\text{Orchard}}$ gives the relation. Its coefficients are scalars modulo p_{Vesta} . Under the premise, the base V^{Orchard} is the value at its input, and each other $B \in \mathfrak{B}$ at x_B . The points of $\mathfrak{B}$ are pairwise distinct, so the chosen inputs are pairwise distinct, one per point; two distinct inputs with one value would themselves be a relation of the same kind. The inputs differ from that of R^{Orchard} , by the premise and since the inputs of V^{Orchard} and R^{Orchard} differ in the message. A non-zero c_B makes the relation non-trivial.

Consequence. Let an efficient party $\mathcal{A}$ output, with probability ϵ , a bundle meeting the premise, openings, a message and a binding signature on it under the resulting bvk , with some c_B non-zero modulo p_{Vesta} ; the event is efficiently decidable from the output. The value b is extracted exactly as in the proof of the *Ironwood Guide*'s Proposition "Balance and key knowledge": if $\text{bvk} = \mathcal{O}$, then $b := 0$; otherwise special soundness with forking in the random-oracle model for the RedPallas hash yields b with $\text{bvk} = [b] R^{\text{Orchard}}$ (*Crypto Guide*, §"Binding signatures as a proof of knowledge of a discrete logarithm", Theorem "The binding signature is a proof of knowledge of $\log_p B$ "). Key prefixing places bvk in the critical query, so the openings and the bases of the first run satisfy the hypothesis of part (ii) for it. Part (ii) then outputs a non-trivial relation among values of GroupHash at distinct inputs, which the *Ironwood Guide*'s Lemma "Relations among fixed generators" excludes except with negligible probability; so ϵ is negligible. In the model of that volume's Assumption "GroupHash as a random oracle" the inputs x_B may equally be read from the queries of $\mathcal{A}$: a point that equals the value at an input $\mathcal{A}$ never queried occurs with probability at most $1/p_{\text{Vesta}}$ for each point. For Asset Bases of Custom Assets, Lemma 2.8 places every input under z.cash:OrchardZSA apart from those of V^{Orchard} and R^{Orchard} . When every c_B vanishes, the relation reduces to $[\sum_i r_i - b] R^{\text{Orchard}} = \mathcal{O}$, and R^{Orchard} has prime order p_{Vesta} , so $b = \sum_i r_i$.

Bundles. The adversary outputs polynomially many bundles, and the union bound reduces the claim to one of them, as in the proof of the *Ironwood Guide*'s Theorem "Balance". The algorithm that runs the adversary with the extractor of Assumption 4.9 outputs, for that bundle, the triples $(\text{AssetBase}_i, v'_i - v_i^{\text{new}}, \text{rcv}_i)$ read from the witnesses; by A4' of Definition 4.6 each is an opening of cv_i^{net} , except with the negligible probability that a witness fails the relation. The bundle's binding signature verifies on SigHash under bvk , so the algorithm is a party of the consequence, and every c_B vanishes modulo p_{Vesta} and $b = \sum_i \text{rcv}_i$, which is (d). The hypotheses of Lemma 5.5 hold for the witnesses, the bundle being in a valid block, and the lemma lifts each congruence to an equality of integers, which is (a) and (b). A burn pair of base B that no Action witnesses has $c_B = -v_B$, with $0 < v_B < p_{\text{Vesta}}$ by (B2), so $c_B \neq 0$; this is (c). $\square$

The premise on the bases cannot be dropped, for witnessed bases or for the bases of burn pairs. For witnessed bases this is Proposition 4.1. For burn pairs, let B be the Asset Base of an issued Asset and $0 < v \leq \text{MAX_BURN_VALUE}$. Split Actions of base B contribute $v' = 0$, so Actions of base B whose net values sum to $-2v$ create $2v$ units of B from nothing. The pair $([-2] B, v)$ satisfies (B2) and (B3), and (B1) unless $[-2] B = V^{\text{Orchard}}$, a relation that Lemma "Relations among fixed generators" of the *Ironwood Guide* excludes except with negligible probability. With balancing value 0 and $t := \sum_i \text{rcv}_i$, Definitions 3.8 and 5.4 give

$$\text{bvk} = [-2v] B + [t] R^{\text{Orchard}} - [v] ([-2] B) = [t] R^{\text{Orchard}},$$

whose discrete logarithm the author knows, so the binding signature verifies. Likewise, with balancing value 0, ZEC Actions whose net values sum to $-v$ and the pair $([-1] V^{\text{Orchard}}, v)$ give

$$\text{bvk} = [-v] V^{\text{Orchard}} + [t] R^{\text{Orchard}} - [v] ([-1] V^{\text{Orchard}}) = [t] R^{\text{Orchard}} :$$

an amount of v zatoshi of ZEC appears from nothing, and (B1) does not exclude the pair, since $[-1] V^{\text{Orchard}} \neq V^{\text{Orchard}}$ in a group of odd order. Rule (B1) excludes only V^{Orchard} ; the premise

on burn-pair bases is discharged by the issuance state, through rule (T4) of Construction 7.3 and Lemma 7.7, not by the burn rules (ZIP 226, “Rationale for Split Notes”, on inputs that are a multiple or linear combination of an existing Asset Base; ZIP 226, “Additional Consensus Rules for the assetBurn set”).

Remark 5.7 (Homomorphism across Asset Bases). ZIP 226, “Rationale for Value Balance Verification”, states that the relation between bv_k and bs_k holds “if and only if”, per Custom Asset, the net values sum to the burnt amount (0 if absent) and, for ZEC, to the balancing value, and that the commitments “add up homomorphically only with respect to the same value base point”. Neither is a literal statement about the Pallas group. The group is cyclic of prime order p_{Vesta} (protocol specification, §“Pallas and Vesta”), so any two non-identity bases satisfy $B' = [\lambda] B$ for some λ ; commitments to distinct bases add homomorphically as group elements; and $\sum_B [c_B] B = \mathcal{O}$ has non-zero solutions (c_B). The “if” direction is part (i) of Proposition 5.6. The “only if” direction holds computationally, for group-hash bases and an efficient signer, as part (ii) and its consequence prove, and then as integers by Lemma 5.5. The argument of the ZIP is rationale and keeps its class (Definition 1.3).

6 Issuance of Custom Assets

This section constructs the objects of ZIP 227 that create value of a Custom Asset: the Issue Note and the rule by which validators compute its commitment (§6.1); the Issuance Action and the Issuance Bundle that carry Issue Notes (§6.2); the element ρ of an Issue Note, derived from the first nullifier of the Action Group (§6.3); the reference note (§6.4); and the rules and the digest hypothesis that bind an Issuance Bundle to its issuer and to one transaction (§6.5). Issuance is public. The one privacy property that it keeps, the unlinkability of a later spend of an Issue Note, is proved in §6.1. The rules that update the global issuance state are those of “The global issuance state” (§7). Every construction of the section is taken from ZIP 227, which is Draft, together with the objects of ZIP 226 that it uses.

6.1 Issue Notes

Definition 6.1 (Issue Note). An *Issue Note* represents the issuance of a value v of one Custom Asset to one recipient. It is a tuple

$$(d, \text{pk}_d, v, \text{AssetBase}, \rho, \text{rseed})$$

of type

$$\begin{aligned} \text{Note}^{\text{Issue}} := & \{0, 1\}^{88} \times (\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}) \times \{0, \dots, 2^{64} - 1\} \\ & \times \mathcal{E}^* \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{B}^{\mathbb{Y}[32]}, \end{aligned}$$

with the following components.

1. The pair (d, pk_d) is the recipient’s address, of the type of the *Ironwood Guide*’s Construction “Diversified address”: the diversifier d and the diversified transmission key pk_d , a

point other than the identity (ZIP 227 types it in $KA^{\text{Orchard}}.\text{Public}$, protocol specification, §“Orchard Key Components”).

2. The value v counts units of the Asset. The lengths 88 and 64 are the constants ℓ_d and ℓ_{value} of the protocol specification, §“Constants”.
3. The point $\text{AssetBase} \in \mathcal{E}^*$ is the Asset Base of the Custom Asset issued (Construction 2.10).
4. The element ρ is fixed by Construction 6.7, in “Computation of ρ for Issue Notes” (§6.3).
5. The note seed rseed is a 32-byte string that the issuer MUST sample uniformly at random.

An Issue Note carries no ψ and no rcm ; both are to be derived from rseed (Remark 6.2). ZIP 227 writes the fourth and fifth factors of the type as P^* and F_{q_p} (Table 1; ZIP 227, “Issue Note”). Class: specified.

Commitments of Issue Notes. No note commitment of an Issue Note is transmitted. Every validator MUST compute the commitment cm of each Issue Note by $\text{NoteCommit}^{\text{OrchardZSA}}$ (Definition 3.2), exactly as for an OrchardZSA note with the same fields, and, when the transaction is included in the block chain, append its extracted form cmx to the note commitment tree after the commitments of the Action Group’s Actions. The order is Definition 7.4, in “State transition of a transaction” (§7.2), and the tree is the one that Remark 1.7 leaves open (ZIP 227, “Issue Note”, “Specification: Consensus Rule Changes” and “Addition to the Note Commitment Tree”). Class: specified; its evaluability is qualified by Remark 6.2.

Remark 6.2 (Commitment inputs of an Issue Note). The OrchardZSA note of an Issue Note $(d, \text{pk}_d, v, \text{AssetBase}, \rho, \text{rseed})$ is $(d, \text{pk}_d, v, \text{AssetBase}, \rho, \psi, \text{rcm})$, with ψ and rcm derived from rseed as in Definition 3.5. With $\underline{\rho} := \text{I2LEOSP}_{256}(\rho)$ and $\text{g}_d := \text{DiversifyHash}(d)$,

$$\begin{aligned}\psi &:= \text{ToBase}(\text{PRFexpand}_{\text{rseed}}([\text{0x09}] \parallel \underline{\rho})), \\ \text{rcm} &:= \text{Derive_rcm}_{\text{rseed}}(\text{leadByte}, \text{g}_d^*, \text{pk}_d^*, v, \underline{\rho}, \psi).\end{aligned}$$

The derivation of ψ does not depend on the lead byte and is specified. The derivation of rcm is designed but unspecified, and the value of its lead byte is an open problem (Remark 3.6). An Issue Note has no plaintext that could carry a lead byte. Three consequences follow.

- (a) Until the value is assigned, the commitment that every validator MUST compute is not computable from live text. The gap is at consensus level. For the created note of a transfer the same gap binds only sender and recipient, since consensus sees only cmx .
- (b) The recipient needs the same rcm to open the commitment when it spends the note, in condition A1’ of Definition 4.6.
- (c) The *Ironwood Guide*’s Proposition “Hiding under the derived trapdoor” does not transfer

to Issue Notes. Hiding plays no role for them: every field, rseed included, is public (Remark 6.6).

(ZIP 227, “Issue Note” and “Specification: Consensus Rule Changes”; ZIP 226, “Note Structure and Commitment”; ZIP 2005, “Changes to the Protocol Specification”, amending the protocol specification’s §“Sending Notes (Orchard)”.)

Remark 6.3 (Commitment $\perp$ for an Issue Note). The commitment $\text{NoteCommit}^{\text{OrchardZSA}}$ may return $\perp$. For a note created by an Action, the sender repeats the derivation with a new rseed when the extracted commitment is $\perp$ (protocol specification, §“Sending Notes (Orchard)”). ZIP 227 states no rule for an Issue Note whose commitment is $\perp$, and no text proposes one: the consequence for the transaction is an open problem. An Issue Note is in the custom branch of Definition 3.2, its Asset Base being a derived one (Proposition 2.13), where the commitment is $\perp$ only if the hash point is; for uniform rseed the event therefore has negligible probability, by Proposition 3.3(c) (ZIP 227, “Issue Note” and “Specification: Consensus Rule Changes”).

Proposition 6.4 (Unlinkability of spends of Issue Notes). *Let a key be generated with $\text{use_qsk} = \text{false}$ from a uniform spending key, with nullifier deriving key nk . Consider the game of the Ironwood Guide’s Proposition “Nullifier unlinkability”, modified as follows. The adversary, which receives (dk, ivk) and ak^P but not nk , submits adaptively Issue Notes n_i of its choice, in particular to addresses of the key, each published in full as issuance publishes it, together with elements $\psi_i \in \mathbb{F}_{\text{Pallas}}$ and $\text{rcm}_i \in \mathbb{F}_{\text{Pesta}}$ of its choice, such that the elements ρ_i are pairwise distinct and the OrchardZSA commitments cm_i of the notes $(d_i, \text{pk}_{d_i}, v_i, \text{AssetBase}_i, \rho_i, \psi_i, \text{rcm}_i)$ are not $\perp$. In the real game it receives the nullifier $\text{DeriveNullifier}_{\text{nk}}(\rho_i, \psi_i, \text{cm}_i)$ that a later spend of the note publishes by condition $A5'$ with $\text{split_flag} = 0$; in the ideal game it receives $\text{Extract}_P([u_i] K^{\text{Orchard}} + \text{cm}_i)$ for independent uniform $u_i \in \mathbb{F}_{\text{Pesta}}$. Then its distinguishing advantage is at most the bound of the cited proposition, which is negligible under the assumptions that proposition names, the Ironwood Guide’s Assumption “Poseidon is a PRF” among them.*

The ideal answers are independent of the published notes when $K^{\text{Orchard}} \neq \mathcal{O}$, so a spend’s nullifier does not link it to its issuance for any party without nk , the issuer included. The statement holds for every ψ_i and every rcm_i , in particular for the derived ψ_i and for any value that a future lead byte gives rcm_i ; it is therefore independent of the open lead byte. It concerns the nullifier only; the other published components of the spending Action are treated in “Hiding of the Asset type” (§9.5). It fails for the reference note (Definition 6.10), whose recipient key is the all-zero spending key: its nk is public, and its nullifier is computable by anyone. Status (Definition 1.3): proved here for the specified construction; conditional on the assumptions of the cited proposition. ZIP 227, “Issue Note”, states the property as rationale.

Proof. The proof is that of the Ironwood Guide’s Proposition “Nullifier unlinkability”. Its game already lets the adversary choose every field of every submitted note, so publishing an Issue Note in full, rseed included, gives the adversary nothing that it does not hold. Its hybrids use a submitted note only through the triple $(\rho_i, \psi_i, \text{cm}_i)$, with cm_i a point other than $\perp$ and the ρ_i pairwise distinct: the hops H_1 to H_3 change the key material only, the hop H_4 uses the distinctness of the ρ_i for the lazily sampled random function, and the hop H_5 and the final statement use cm_i only as a point to which $[s_i] K^{\text{Orchard}}$ or $[u_i] K^{\text{Orchard}}$ is added. The nullifier of an OrchardZSA note is DeriveNullifier

of the *Ironwood Guide*'s Definition "Nullifier" with the OrchardZSA commitment in place of the Orchard one (ZIP 226, "Note Structure and Commitment"; §4.2). Every hybrid and every bound therefore carries over verbatim. The hypothesis that the ρ_i are pairwise distinct is supplied, for the Issue Notes of a valid chain, by Proposition 6.9, except with negligible probability.

For the reference note, $\text{nk} := \text{ToBase}(\text{PRFexpand}_{\text{sk}}([\text{0x07}]))$ with sk the all-zero string (*Ironwood Guide*, §"The spending key and the spend-side secrets") is a public constant, so the real answer is computable from public data and differs from the ideal one. $\square$

6.2 Issuance Actions and Issuance Bundles

ZIP 227 specifies issuance through three components: the Issuance Bundle of a transaction, in which a publicly named issuer creates Issue Notes; the flag `finalize` of each Issuance Action, which, once set, ends issuance of its Asset permanently; and the global issuance state kept by every node, constructed in "The global issuance state" (§7).

Definition 6.5 (Issuance Action and Issuance Bundle). Let

$$\text{IssueNoteFields} := \{0, 1\}^{88} \times (\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}) \times \{0, \dots, 2^{64} - 1\} \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{B}^{\mathbb{Y}[32]},$$

the fields $(d, \text{pk}_d, v, \rho, \text{rseed})$ of an Issue Note without its Asset Base.

1. An *Issuance Action* is a triple

$$(\text{assetDescHash}, \text{vNotes}, \text{flagsIssuance}),$$

where the hash $\text{assetDescHash} \in \mathbb{B}^{\mathbb{Y}[32]}$ is the description hash of Definition 2.6; vNotes is a finite sequence of elements of IssueNoteFields , the unencrypted Issue Notes of the Asset; and flagsIssuance is a byte that carries the Boolean `finalize`, which the issuer sets to signal that no further issuance of the Asset will occur.

2. An *Issuance Bundle* is a triple

$$(\text{issuer}, \text{vIssueActions}, \text{issueAuthSig}),$$

where $\text{issuer} = \text{ik_encoding} \in \mathbb{B}^{\mathbb{Y}[33]}$ is the issuer identifier of Definition 2.4; vIssueActions is a finite sequence of Issuance Actions; and $\text{issueAuthSig} = [\text{0x00}] \parallel \sigma$ encodes a signature σ by isk on the transaction's SigHash (Definition 6.12).

The pair $(\text{issuer}, \text{assetDescHash})$ is an Asset Identifier and determines AssetBase by Construction 2.10. With it, each element $(d, \text{pk}_d, v, \rho, \text{rseed})$ of vNotes determines the Issue Note $(d, \text{pk}_d, v, \text{AssetBase}, \rho, \text{rseed})$ of Definition 6.1 completely. The commitment is omitted, because every validator recomputes it (ZIP 227, "Issuance Action", "Issuance Bundle", "Issue Note" and "Issuance Protocol"). Class: specified.

Encoding status. The byte encoding of the Issuance Bundle and of the Issuance Action exists only in withdrawn ZIP 230 and is designed but unspecified; it is not the version-6 format of ZIP 229 (Remark 8.1; ZIP 227, "Issuance Action" and "Issuance Bundle").

Issuance Actions without notes. The rules of ZIP 227 place no requirement on the number of Issue Notes of an Issuance Action. Withdrawn ZIP 230 admits an Issuance Action with none, so that an issuer can finalise an issued Asset without issuing more of it. The validity of such an action for an Asset without an entry in the issuance state is the open case of Remark 7.5, in “State transition of a transaction” (§7.2) (ZIP 227, “Specification: Consensus Rule Changes”; ZIP 230, “Issuance Action Description”).

Remark 6.6 (Public issuance). ZIP 227 enables only public issuance, so that the supply of each Asset can be tracked (ZIP 227, “Motivation”; “Requirements”: the issuance mechanism should enable public tracking of the supply). The Issue Notes are unencrypted. The issuer, the Asset (through `assetDescHash`, from which anyone recomputes `AssetBase`), every value, every recipient address, every ρ and `rseed`, and every flag `finalize` are public, so issuance publicly links each recipient address to an Asset and an amount. Since `rseed` is public, so is ψ , and so, once the lead byte is assigned, are the commitment and the leaf of each Issue Note. With the public burn set (Definition 5.1), the amount of each Asset in circulation, issued less burnt, is a public function of the chain, which ZIP 227 records as the balance of the global issuance state (“Global Issuance State”). The later use of an Issue Note remains private, to the extent of Proposition 6.4. In this volume issuance is called public and Issue Notes unencrypted (ZIP 227, “Issuance Action”).

6.3 Computation of ρ for Issue Notes

An Issue Note is created outside any Action, so the *Ironwood Guide*’s Construction “Chaining rule” (§“Nullifier chaining”) gives it no ρ .

Construction 6.7 (Derivation of issued rho values). Define

$$\text{DerivelIssuedRho}: \mathbb{F}_{p_{\text{Pallas}}} \times \{0, \dots, 2^{32} - 1\} \times \{0, \dots, 2^{32} - 1\} \rightarrow \mathbb{F}_{p_{\text{Pallas}}}$$

by

$$\begin{aligned} &\text{DerivelIssuedRho}(\text{nf}, i_A, i_N) \\ &:= \text{ToBase}(\text{PRFexpand}_{\text{I2LEOSP}_{256}(\text{nf})}([\text{0x84}] \parallel \text{I2LEOSP}_{32}(i_A) \parallel \text{I2LEOSP}_{32}(i_N))), \end{aligned}$$

with `PRFexpand` the *Ironwood Guide*’s Construction “Expansion function” and `ToBase` its Definition “Field reductions”. The expansion input after the key has $1 + 4 + 4 = 9$ bytes. The element ρ of the Issue Note at zero-based position i_N of the Issuance Action at zero-based position i_A of the Issuance Bundle is

$$\rho := \text{DerivelIssuedRho}(\text{nf}_{0,0}, i_A, i_N),$$

where $\text{nf}_{0,0}$ is the nullifier published by the first Action of the transaction’s Action Group (Definition 4.13), whatever the kind of its consumed note. The definition presupposes an Action Group, which rule (I1) of Definition 6.12 enforces. The derivation adds one row to the *Ironwood Guide*’s Construction “Domain bytes of PRF expansion”:

b	Key κ	Remainder t'	f	Output	Section
<code>0x84</code>	$\text{I2LEOSP}_{256}(\text{nf}_{0,0})$	$\text{I2LEOSP}_{32}(i_A) \parallel \text{I2LEOSP}_{32}(i_N)$	<code>ToBase</code>	ρ	§6.3

The key is a public nullifier. No other ZIP and no section of the protocol specification uses the byte 0x84, and no registry of a lower volume lists it. Because the key is public, neither the *Ironwood Guide*’s Assumption “Pseudorandomness of PRF expansion” nor its Lemma “Independence of domain-separated expansions” applies (ZIP 227, “Computation of ρ ”; protocol specification, §“Pseudo Random Functions” and §“Orchard Key Components”). Class: specified.

Assumption 6.8 (BLAKE2b-512 as a random oracle). For each fixed 16-byte personalisation P , the function $x \mapsto \text{BLAKE2b-512}(P, x)$ is a random oracle with 512-bit output in the sense of the *Crypto Guide*’s Definition “Random oracle” (§“The random oracle model”), independent of the oracles of the other personalisations. It is used for the personalisation Zcash_ExpandSeed of PRFexpand , where the key of Construction 6.7 is public. For a secret uniform key and an adversary that makes polynomially many oracle queries, it implies the *Ironwood Guide*’s Assumption “Pseudorandomness of PRF expansion”. Its uses in this volume are Proposition 6.9 and Lemma 9.6, whose proof programs one query of the oracle.

Proposition 6.9 (Uniqueness of issued rho values).

- (a) *The expansion inputs, key and remainder, of the Issue Notes of a valid chain are pairwise distinct, and each differs from the input of every other row of PRFexpand .*
- (b) *Under Assumption 6.8, consider an execution in which at most q queries in total are made to the oracle of personalisation Zcash_ExpandSeed . The probability that two Issue Notes of the chain have equal ρ , or that the ρ of an Issue Note equals one of M elements ρ of notes created by Actions and fixed before the query that yields it, is at most*

$$\left(\frac{q(q-1)}{2} + qM \right) \left(\frac{1}{p_{\text{Pallas}}} + 2^{-512} \right) \leq \frac{(q+M)^2}{p_{\text{Pallas}}}.$$

With no query beyond the N issued values, $q = N$.

Literal uniqueness is not claimed, nor distinctness from the ρ of notes created by later Actions, which the derivation does not control. Distinct OrchardZSA notes have distinct commitments by Proposition 3.3(b). The proof of the Ironwood Guide’s Proposition “Nullifier uniqueness” then applies with the hash base and message of each note’s branch of Definition 3.2, which unfold as in the proof of Proposition 4.5(i). So distinct notes have distinct nullifiers whatever their ρ , and a collision can affect only notes equal in every field. The proposition extends to Issue Notes part (a) of the Ironwood Guide’s Corollary “Faerie Gold resistance”, which covers only notes created by Actions. Status (Definition 1.3): proved here for the specified derivation; conditional on Assumption 6.8.

Proof. (a) Within a transaction the key is one and the index pairs (i_A, i_N) differ. Across transactions of a valid chain the keys $\text{l2LEOSP}_{256}(\text{nf}_{0,0})$ differ: $\text{nf}_{0,0}$ is a published nullifier, and the nullifier-set rule forbids a nullifier to repeat within a pool (*Ironwood Guide*, §“Nullifier sets”; the pool is that of Remark 1.7), and l2LEOSP_{256} is injective on $\mathbb{F}_{p_{\text{Pallas}}}$. The oracle input is the 32-byte key followed by the remainder. Every key of PRFexpand is a 32-byte string (*Ironwood Guide*, Construction “Expansion function”), so the byte after the key is the lead byte, and 0x84 differs from

the lead byte of every other row.

(b) By (a) each issued ρ is ToBase of the oracle value at an input of its own. In the lazy form of the *Crypto Guide*'s Definition "Random oracle", the first query at an input returns a fresh uniform 512-bit string, independent of all earlier values. The map ToBase takes each value of $\mathbb{F}_{p_{\text{Pallas}}}$ on at most $\lceil 2^{512}/p_{\text{Pallas}} \rceil$ strings, so the fresh output equals a value fixed before the query with probability at most $\lceil 2^{512}/p_{\text{Pallas}} \rceil / 2^{512} < 1/p_{\text{Pallas}} + 2^{-512}$. Every issued ρ is the reduced output of one of the q queries. A union bound over the at most $q(q-1)/2$ pairs of queries and the qM pairs of a query and a fixed value (*Math Guide*, §"The union bound and a birthday calculation", Theorem "Union bound / Boole's inequality" and Proposition "Birthday bound") gives the first bound. The second follows from $2(q(q-1)/2 + qM) \leq (q+M)^2$ and $1 + p_{\text{Pallas}} 2^{-512} < 2$. $\square$

6.4 Reference notes

Definition 6.10 (Reference note). The first Issue Note of the Issuance Action in which a Custom Asset is issued for the first time MUST be a *reference note*: an Issue Note with $v = 0$, the Asset Base of that Asset, and recipient (d, pk_d) the default diversified address, of diversifier index 0, of the all-zero Orchard spending key $\text{sk} = [\text{0x00}]^{32}$. The address is derived by the *Ironwood Guide*'s constructions of §"The spending key and the spend-side secrets", §"Viewing keys" and Construction "Diversified address". The all-zero key passes every acceptance condition of key generation: $\text{ask} \neq 0$; $\text{ivk} \notin \{0, \perp\}$; and the incoming viewing key of its internal key (ZIP 32, "Orchard internal key derivation") is not in $\{0, \perp\}$, the condition of the protocol specification, §"Orchard Key Components". At index 0 the diversified base g_d is not the identity. The raw encoding of the address, $\text{LEBS2OSP}_{88}(d) \parallel \text{LEBS2OSP}_{256}(\text{pk}_d^*)$ (protocol specification, §"Orchard Raw Payment Addresses"), is the 43-byte string, given in ZIP 227, "Reference Notes", as

(204, 54, 96, 25, 89, 33, 59, 107, 12, 219, 150, 167, 92, 23, 195, 166,
104, 169, 127, 13, 106, 140, 92, 225, 100, 165, 24, 234, 155, 169, 165,
14, 167, 81, 145, 253, 134, 27, 15, 241, 14, 98, 176).

Recomputing the address and every acceptance check from the cited constructions reproduces this array. Validators enforce the rule and store the reference note as note_{ref} in the global issuance state, by rule (I5) of Construction 7.3, in "State transition of a transaction" (§7.2) (ZIP 227, "Reference Notes" and "Specification: Consensus Rule Changes"). Class: specified; the reference note is mandated and stored in consensus state, and no section of ZIP 227 states its purpose.

Proposition 6.11 (Reference notes as universal Split Inputs). *Let n_{ref} be the reference note of a Custom Asset with Asset Base B , its extracted commitment a leaf of the note commitment tree.*

- (a) *Any party can construct a Split Action (Definition 4.2) whose Split Input copies n_{ref} with a witness of Definition 4.6 and a spend-authorisation signature that verifies under its rk , for a created note of Asset Base B and value 0 to any receiver of its choice. Proposition 4.12 therefore imposes nothing on n_{ref} , whose recipient key is public by construction.*

- (b) Let an efficient adversary output OrchardZSA bundles of valid blocks whose Action proofs and binding signatures verify and whose witnessed and burn-pair Asset Bases meet the premise on the bases of §5.3. Except with negligible probability, in every such bundle in which no extracted witness of Asset Base B has `split_flag` = 0, every created note of Asset Base B has value 0, and the burn set has no pair of Asset Base B .

Hence n_{ref} serves exactly the created notes of B that a party holding no note of B can produce. Under Remark 1.7, a copy to a receiver other than that of the copied note would violate the cross-address rule if the notes were in the Orchard pool. Status (Definition 1.3): proved here for the specified constructions; part (a) is conditional on the open lead byte (Remark 6.2); part (b) is conditional on Assumption 4.9 through Proposition 5.6, and its premise on the bases is discharged for valid chains in “Security” (§9). The reading of the purpose is the volume’s; ZIP 227 states none.

Proof. (a) The construction is explicit. The all-zero spending key is accepted (Definition 6.10), so its ask, nk , r_{ivk} and ak^{p} are public constants. The note n_{ref} is published unencrypted with ρ and rseed (Remark 6.6), so ψ is public and, once the lead byte is assigned (Remark 6.2), so are rcm and the commitment. The position of its leaf in the tree is public, by Definition 7.4. The party takes a valid anchor rt whose tree contains the leaf and computes its authentication path, which satisfies $\text{A3}'$; it opens $\text{A1}'$ with the fields of n_{ref} and $v^{\text{old}} = 0$; it sets `split_flag` = 1, `is_native_asset` = 0 and `enableZSA` = 1, which satisfy C1 to C3 since $B \neq V^{\text{Orchard}}$; it draws ψ^{nf} and computes the split nullifier of $\text{A5}'$ with the public nk ; it creates a note of base B , value 0 and any receiver, whose commitment satisfies $\text{A2}'$, and takes any rcv for $\text{A4}'$, which reads $v' = 0$ and $v^{\text{new}} = 0$. Conditions A6 and A7 hold with the public $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$, from which the reference address is derived; A8 holds since $v^{\text{old}} = 0$, and A9 since $v^{\text{new}} = 0$. The party signs under $\text{rsk} = \text{ask} + \alpha$ for its randomiser α , as any owner does.

(b) By Proposition 5.6(a), except with negligible probability, the net values of the Actions of base B , read from the extracted witnesses, sum as integers to the burnt amount v_B , which is 0 if B is not burnt. Every such Action has `split_flag` = 1, so its value v' is 0 by the notation of Definition 4.6, and the sum is $-\sum_i v_i^{\text{new}} \leq 0$. A burn pair has $v_B > 0$ by rule (B2) of Definition 5.1, so $v_B \geq 0$. Both sides therefore vanish: every v_i^{new} of base B is 0, and $v_B = 0$ excludes a pair of base B . $\square$

6.5 Issuance authorisation

Definition 6.12 (Issuance authorisation rules). For a transaction that contains an Issuance Bundle (`issuer`, `vIssueActions`, `issueAuthSig`):

- (I1) It MUST contain an OrchardZSA Action Group; by Definition 4.13 it contains exactly one.
- (I2) The encoding `ik_encoding` of the issuance validating key `ik` MUST be taken from the field `issuer` and MUST start with the byte `0x00`, which indicates a BIP-340 public key; `ik` is the remaining 32 bytes (Definition 2.4).
- (I3) The first byte of `issueAuthSig` MUST be `0x00`, and

$$\text{IssueAuthSig.Validate}(\text{ik}, \text{SigHash}, \sigma) = 1$$

MUST hold for the 64-byte σ that follows it (Definition 2.2). ZIP 227 writes the rule with `issueAuthSig` in place of σ ; the scheme byte is stripped because `Validate` is typed over 32-byte keys and 64-byte signatures.

For every Issuance Action the Issue Notes are then constructed from the elements of its `vNotes`, with `AssetBase` derived from its `assetDescHash` and from issuer (Remark 6.14). The rules on the issuance state, (T1) to (T4) and (I4) to (I7), are Construction 7.3. The message `SigHash` is the transaction's signature digest (*Ironwood Guide*, Definition "Signature digest"), which ZIP 227 defines by the protocol specification's "SIGHASH Transaction Hashing" "with the modifications described in ZIP 226", under `SIGHASH_ALL`. The cited section of ZIP 226 does not exist; ZIP 226, "Sighash modifications relative to ZIP 244", defers to ZIP 246, which is Withdrawn; and the version-6 digest of ZIP 229 has no issuance child (ZIP 227, "Specification: Consensus Rule Changes" and "Issuance Authorization Signing and Validation"). Class: rules (I1) to (I3) specified; the signed message designed but unspecified (Remark 8.3).

Assumption 6.13 (Coverage of the signature digest). The signature digest `SigHash` of an OrchardZSA transaction is computed without any signature of the transaction: neither `issueAuthSig`, nor a spend-authorisation signature, nor the binding signature is an input. No efficient algorithm outputs, except with negligible probability, two transactions with equal `SigHash` that differ in

- (i) the effecting data of the Issuance Bundle: issuer, the number and order of the Issuance Actions, and, per Issuance Action, `assetDescHash`, `finalize` and every field ($d, pk_d, v, \rho, rseed$) of every Issue Note;
- (ii) the burn set; or
- (iii) the published data of the Actions of the Action Group, in particular `nf0,0`.

This hypothesis (H-SIG) concerns a designed but unspecified object. "Transaction digests and the signature message" (§8.2) shows that the digest of withdrawn ZIP 246 meets it under the *Ironwood Guide*'s Assumption "Collision resistance of BLAKE2b", and states it as the obligation on any future digest (Remark 8.2). The version-6 digest of ZIP 229 does not meet (i).

Remark 6.14 (Recomputation of issued Asset Bases). Of the identity chain of Construction 2.10, consensus sees exactly two links on the issuance side: issuer in the Issuance Bundle and `assetDescHash` in each Issuance Action. The Asset Identifier, the Asset Digest and the Asset Base of issued notes are never transmitted. Every validator derives

$$\text{AssetBase} := \text{ZSAValueBase}(\text{BLAKE2b-512}(\text{ZSA-Asset-Digest}, \text{EncodeAssetId}(\text{issuer}, \text{assetDescHash})))$$

by that construction. The same bytes issuer name the Asset and yield, by (I2), the key under which (I3) verifies the signature; under Assumption 6.13(i), both issuer and `assetDescHash` are in the signed message. The consequence for authority over issuance is Theorem 9.8, in "Authority over issuance" (§9.4) (ZIP 227, "Specification: Consensus Rule Changes" and "Issuance Bundle").

Remark 6.15 (Signing order of an issuance bundle). Every ρ of an Issue Note is a function of $\text{nf}_{0,0}$ (Construction 6.7), fixed once the first Action of the Action Group is fixed. Under Assumption 6.13 the signed digest covers every ρ and every other effecting field of the transaction. The issuer can therefore sign only after the nullifier of the first Action and all effecting data are fixed. Its message is the SIGHASH_ALL digest that every spend-authorisation signature and the binding signature sign (*Ironwood Guide*, Definition “Signature digest”; ZIP 227, “Computation of ρ ”, “Issuance Protocol” and “Specification: Consensus Rule Changes”).

Proposition 6.16 (Replay protection of issuance). *Let ik be the issuance validating key of an honest issuer, which signs only digests SigHash of transactions it builds. Except with negligible probability, in a valid chain produced by an efficient adversary that obtains the honest issuer’s signatures:*

- (a) *every Issuance Bundle whose issuer is ik_encoding of ik belongs to a transaction whose SigHash the issuer signed;*
- (b) *no transaction of the chain that contains an Issuance Bundle has the SigHash of another transaction of the chain.*

Hence each digest that the issuer signs authorises the issuance of at most one transaction of the chain. Rule (I1) is what makes (b) hold: without an Action Group, the digest of a transaction that carries only an Issuance Bundle depends on no value that the chain forbids to repeat, and a duplicate of the bundle in a transaction with the same remaining digest inputs has the same SigHash, so its signature verifies again (ZIP 227, “Rationale”). Status (Definition 1.3): proved here for the specified rules (I1) to (I3); conditional on Assumption 6.13, a hypothesis on the designed but unspecified signed message, and on Assumption 2.3. ZIP 227, “Rationale”, states the replay argument informally.

Proof. (a) An accepted bundle satisfies (I2) and (I3), so its signature verifies on the transaction’s SigHash under the key whose encoding is issuer, which is ik by the injectivity of $\text{ik} \mapsto \text{issuer}$ (Definition 2.4). A verifying signature on a digest that the issuer never signed is an existential forgery against Assumption 2.3: the reduction answers the adversary’s requests for the issuer’s signatures with its signing oracle and outputs the digest and the signature of the chain.

(b) By (I1) a transaction with an Issuance Bundle has an Action Group, and so a nullifier $\text{nf}_{0,0}$. Another transaction of the chain with equal SigHash has, by Assumption 6.13(iii), the same published Action data, except with negligible probability, since the chain is the output of an efficient algorithm; in particular it publishes $\text{nf}_{0,0}$ as well. The nullifier-set rule forbids a nullifier to repeat across the transactions of a valid chain (*Ironwood Guide*, §“Nullifier sets”), so no such transaction exists. By (a) and (b) the map from the transactions of the chain that carry a bundle under ik to the digests that the issuer signed is injective, which is the final claim. $\square$

7 The global issuance state

This section constructs the record that every node keeps of the issued Custom Assets: the global issuance state of ZIP 227 and its chaining through blocks and transactions (§7.1); the transition

of that state under one transaction, in which burn precedes issuance, together with the order in which the note commitments of a transaction enter the note commitment tree (§7.2); the properties of finalisation and of the recorded balance (§7.3); and the complete verification of an OrchardZSA transaction, with each check paired with the requirement that it serves (§7.4). The rules are those of ZIP 227, which is Draft; one case that they leave undetermined is an open problem (Remark 7.5).

7.1 The issuance state and its chaining

Definition 7.1 (Global issuance state). Let

$$\text{MAX_ISSUE} := 2^{64} - 1 = 18\,446\,744\,073\,709\,551\,615,$$

the largest value of one Issue Note (Definition 6.1). The *global issuance state* is a finite partial map

$$\begin{aligned} \text{issued_assets}: \mathcal{E}^* &\rightarrow \{0, \dots, \text{MAX_ISSUE}\} \times \{0, 1\} \times (\text{Note}^{\text{Issue}} \cup \{\perp\}), \\ \text{AssetBase} &\mapsto (\text{balance}, \text{final}, \text{note}_{\text{ref}}), \end{aligned}$$

from Asset Bases to triples, with $\mathcal{E}^*$ as in §2.3. The components are written

$$\begin{aligned} \text{issued_assets}(\text{AssetBase}).\text{balance}, \quad & \text{issued_assets}(\text{AssetBase}).\text{final}, \\ & \text{issued_assets}(\text{AssetBase}).\text{note}_{\text{ref}}. \end{aligned}$$

An Asset Base outside the domain of the map, written

$$\text{issued_assets}(\text{AssetBase}) = \perp,$$

reads as $(0, 0, \perp)$, and a write to a component of such an Asset Base creates its entry from that default. The components mean the following.

1. The *balance* is the amount of the Asset in circulation: the amount issued less the amount burnt.
2. The *finalisation bit* *final* records whether *finalize* = 1 occurred in an earlier issuance of the Asset; it never changes from 1 to 0.
3. The *reference note* *note_{ref}* is the Asset’s reference note (Definition 6.10), or $\perp$.

ZIP 227 types the third component in $\text{Note}^{\text{Issue}}$ while its default reads $\perp$; it is typed here in $\text{Note}^{\text{Issue}} \cup \{\perp\}$. The bound *MAX_ISSUE* applies to the balance, the amount in circulation, and not to cumulative issuance: after burns an unfinalised Asset can be issued again, so the sum of all values issued of one Asset may exceed *MAX_ISSUE*. The phrases “maximum total supply” and “limit the total issuance” of ZIP 227 are looser than its rule (I6) of Construction 7.3, which the volume follows.

Chaining. Every node MUST update the map while processing any transaction that contains a burn set or an Issuance Bundle. A transaction has an *input state* $\text{issued_assets}_{\text{IN}}$ and an *output state* $\text{issued_assets}_{\text{OUT}}$:

1. the input state of the first transaction of the OrchardZSA activation block is the empty map;
2. the input state of the first transaction of any later block is the final state of the preceding block;
3. the input state of every later transaction of a block is the output state of the preceding transaction;
4. the *final state* of a block is the output state of its last transaction.

The chaining is that of treestates in the *Ironwood Guide*'s Definition "Treestate and anchor" (§"Anchors") (ZIP 227, "Global Issuance State" and "Management of the Global Issuance State"). Class: specified. The OrchardZSA activation block is not assigned, since ZIP 227, "Deployment", reads "TBD"; this is an open problem, listed in "Open problems" (§10.2).

ZIP 227, "Rationale for Global Issuance State", gives three reasons, which are rationale of the ZIP. The balance of an issued Custom Asset must never become negative, along the lines of ZIP 209, and no single transaction field carries both the issued and the burnt amounts, so nodes keep a running record of the amount in circulation. The bound MAX_ISSUE is a practical limit that lets an issuer issue the complete supply of an Asset in one transaction, indeed in one Issue Note. The finalisation bit lets nodes reject further issuance of a finalised Asset. The properties themselves are proved in "Finalisation" (§7.3).

Remark 7.2 (Issuance state under reorganisation). By the chaining of Definition 7.1, the final state of a block is a function of the branch from the OrchardZSA activation block to that block. A reorganisation (*Ironwood Guide*, Definition "Reorganisation") therefore replaces the state by the one obtained by chaining along the new branch from the last common block, as it replaces the treestates. ZIP 227 states forward chaining only and no rule for reorganisations: the statement here is a consequence of the definition, not a rule of the ZIP. Every result of this section, and Theorem 9.7, holds per branch.

7.2 State transition of a transaction

Construction 7.3 (Transition of the issuance state). *Input.* A transaction T that may contain an OrchardZSA bundle (Definition 5.2), with burn set assetBurn , empty when T has none, and an Issuance Bundle (issuer, vIssueActions , issueAuthSig) (Definition 6.5); and its input state $\text{issued_assets}_{\text{IN}}$.

Output. The output state $\text{issued_assets}_{\text{OUT}}$, defined only when every requirement below holds; otherwise T is invalid and contributes no state. Every write below is to $\text{issued_assets}_{\text{OUT}}$, and every read is of its current value. The labels follow the order of ZIP 227, which numbers none.

For every transaction:

- (T1) The transaction carries at most one Action Group, and that Action Group has no expiry height of its own.

(T2) $\text{issued_assets}_{\text{OUT}} := \text{issued_assets}_{\text{IN}}$.

(T3) The burn set assetBurn satisfies (B1) to (B3) of Definition 5.1.

(T4) For every $(\text{AssetBase}, v) \in \text{assetBurn}$, it MUST hold that

$$\text{issued_assets}_{\text{OUT}}(\text{AssetBase}).\text{balance} \geq v;$$

the node then sets, for every pair,

$$\text{issued_assets}_{\text{OUT}}(\text{AssetBase}).\text{balance} := \text{issued_assets}_{\text{OUT}}(\text{AssetBase}).\text{balance} - v,$$

before any issuance is processed. By (B3) the pairs have distinct Asset Bases, so checking every pair and then subtracting equals processing the pairs one by one.

If T contains an Issuance Bundle: rules (I1) to (I3) of Definition 6.12 hold, and for each Issuance Action $(\text{assetDescHash}, v\text{Notes}, \text{flagsIssuance})$ of $v\text{IssueActions}$, in order, with zero-based position i_A in the bundle, let AssetBase be derived from issuer and assetDescHash by Construction 2.10 (Remark 6.14). Then:

(I4) Every Issue Note description of the action MUST be a valid encoding, and each yields the Issue Note of Definition 6.5 with that AssetBase . It MUST hold that $\text{issued_assets}_{\text{OUT}}(\text{AssetBase}).\text{final} \neq 1$, evaluated against the state as updated by the earlier actions of the same bundle.

(I5) If $\text{issued_assets}_{\text{OUT}}(\text{AssetBase}).\text{note}_{\text{ref}} = \perp$, the first Issue Note of the action MUST be a reference note (Definition 6.10): its value MUST be 0, and its recipient (d, pk_d) MUST be the default diversified address of the all-zero Orchard spending key. The node then sets $\text{issued_assets}_{\text{OUT}}(\text{AssetBase}).\text{note}_{\text{ref}}$ to that note.

(I6) For each Issue Note of the action, at zero-based position i_N and of value v : its element ρ MUST equal $\text{DerivelIssuedRho}(\text{nf}_{0,0}, i_A, i_N)$ (Construction 6.7); it MUST hold that

$$\text{issued_assets}_{\text{OUT}}(\text{AssetBase}).\text{balance} + v \leq \text{MAX_ISSUE},$$

and the node then adds v to that balance; and the node MUST compute the note commitment of the Issue Note by Definition 3.2, whose inputs are qualified in Remark 6.2.

(I7) If $\text{finalize} = 1$ in flagsIssuance , the node sets

$$\text{issued_assets}_{\text{OUT}}(\text{AssetBase}).\text{final} := 1.$$

(ZIP 227, “Specification: Consensus Rule Changes”; ZIP 226, “Additional Consensus Rules for the assetBurn set”.) Class: specified. The encodings that (T1) and (I4) name belong to the carrier: ZIP 227 states (T1) “for every transaction” as $\text{nActionGroupsOrchard} \in \{0, 1\}$ and $\text{nAGExpiryHeight} = 0$, fields that exist only in withdrawn ZIP 230, and (I4) by reference to the field encodings of ZIP 230; that carrier is open (Remark 8.1). ZIP 227 writes the balance of (I6) as $\text{issued_assets}_{\text{OUT}}.\text{balance}$, without the argument AssetBase , which is restored here.

Four consequences are read off Construction 7.3. Burns are checked against the balance before issuance, so a transaction cannot burn units that it issues. A burn of an Asset Base without an entry fails (T4), since the default balance is 0 and (B2) gives $v > 0$. A finalised Asset remains burnable, since no rule of the burn half reads final. Since (I4) precedes (I7) within an action, one action may issue notes and finalise its Asset.

Definition 7.4 (Insertion order of note commitments). When a transaction T that passes Construction 7.3 is added to the block chain, the leaves that it appends to the note commitment tree of its treestate are, in this order:

1. for each Action of the Action Group of T , in order, the extracted commitment cmx of the note that the Action creates, whatever its value and whether or not the Action is a Split Action;
2. then, for each Issuance Action of the Issuance Bundle of T , in order, and each of its Issue Notes, in order, the extracted commitment of the note commitment computed by (I6).

The transactions of a block are taken in block order. The definition extends the *Ironwood Guide*'s Construction "State update" (§"Verification of a transaction"), which appends the commitments of Actions only. The position of every leaf, and hence every authentication path, is a function of the chain under this order, so nodes and wallets agree on positions only under it. Which pool's tree the order extends is the open problem of Remark 1.7 (ZIP 227, "Addition to the Note Commitment Tree"; protocol specification, §"Note Commitment Trees"). Class: specified.

Remark 7.5 (Empty Issuance Actions). The rules of ZIP 227 place no requirement on the number of Issue Notes of an Issuance Action, and withdrawn ZIP 230, "Issuance Action Description (IssueAction)", admits an action without notes so that an issuer can finalise an Asset without issuing more of it. Consider such an action on the Asset Base AssetBase.

- (a) If $\text{note}_{\text{ref}} \neq \perp$ when the action is processed, an action with $\text{finalize} = 0$ changes nothing, and one with $\text{finalize} = 1$ sets final by (I7).
- (b) If $\text{note}_{\text{ref}} = \perp$, rule (I5) refers to a first Issue Note that does not exist, and the rules do not determine whether the action, finalising or not, is valid. If a finalising one were accepted, it would create the entry $(0, 1, \perp)$: an Asset finalised with no reference note and no issuance.

Class: case (b) is an open problem, listed in "Open problems" (§10.2); a rule that fixes the validity of such an action closes it.

An issuance state is *reachable* if it is the input or the output state of a transaction in a sequence of transactions, chained from the empty map as in Definition 7.1, each of which has an output state under Construction 7.3. A *valid chain* in the results of this section is such a sequence; an invalid transaction contributes no state. Every chain accepted by Construction 7.10 is a valid chain in this sense, since that procedure includes the transition.

Lemma 7.6 (First-issuance detection). *Assume that no valid chain contains an Issuance Action without Issue Notes on an Asset Base whose note_{ref} is $\perp$ when the action is processed, which is case (b) of Remark 7.5. Then in every reachable issuance state S , and for every AssetBase , the Asset Base is in the domain of S if and only if $S(\text{AssetBase}).\text{note}_{\text{ref}} \neq \perp$. Hence conditioning (I5) on the absence of an entry is equivalent to conditioning it on $\text{note}_{\text{ref}} = \perp$. Status (Definition 1.3): proved here for the specified rules; conditional on the hypothesis, which concerns the open case of Remark 7.5.*

Proof. Induction over the transactions of a valid chain, the invariant being checked after every write of Construction 7.3. It holds for the empty map. Rule (T2) copies a state. Rule (T4) writes only Asset Bases with $\text{balance} \geq v > 0$, by (B2), hence Asset Bases already in the domain, and leaves note_{ref} unchanged. Rule (I4) only reads. For an action on an Asset Base outside the domain, $\text{note}_{\text{ref}} = \perp$ and the action has at least one Issue Note by the hypothesis, so (I5) sets note_{ref} to that note before (I6) and (I7) write: the entry is created together with its reference note. For an action on an Asset Base in the domain, $\text{note}_{\text{ref}} \neq \perp$ by the invariant, and (I5) does not apply. No rule writes $\perp$ to note_{ref} or removes an entry. Both the burn step and the hypothesis are needed: without the hypothesis, a finalising action of case (b), if accepted, creates the entry $(0, 1, \perp)$ by (I7). $\square$

Lemma 7.7 (Keys of the issuance state). *In every reachable issuance state, every Asset Base in the domain equals $\text{ZSAValueBase}(\text{AssetDigest})$ for the Asset Identifier (issuer, assetDescHash) of some accepted Issuance Action (Construction 2.10), and is therefore the value of GroupHash at an input under $z.\text{cash}:\text{OrchardZSA}$ (Table 3). For every transaction of a valid chain, the Asset Base of every pair of its burn set is in the domain of its input state, and is therefore of the same form. Status (Definition 1.3): proved here for the specified rules; unconditional, case (b) of Remark 7.5 included. The lemma discharges, in Theorem 9.5, the premise of Proposition 5.6 on the bases of burn pairs, which (B1) alone does not discharge.*

Proof. Induction over the transactions of a valid chain. The initial map is empty. Rules (I4) to (I7) write only the entry at the Asset Base that Construction 7.3 derives from issuer and assetDescHash by Construction 2.10, whether or not the action has Issue Notes, and so does (I7) in case (b) of Remark 7.5 if such an action is accepted. Rule (T4) writes only an Asset Base in the domain: an Asset Base outside it reads $\text{balance} = 0$, and (B2) requires $v > 0$, so the check of (T4) fails. The same argument gives the second claim, since (T4) reads the input state, copied by (T2), before any issuance of the transaction. $\square$

7.3 Finalisation

Every Issuance Action carries the Boolean `finalize` in `flagsIssuance` (Definition 6.5). An Asset is *finalised* in a reachable issuance state S when $S(\text{AssetBase}).\text{final} = 1$ for its Asset Base. Setting `finalize` finalises the Asset by (I7), and every later Issuance Action for its Asset Base, with or without Issue Notes, fails (I4). Only (I7) writes `final`, and it writes 1, so `final` never returns from 1 to 0, as Definition 7.1 states. Finalisation is the only mechanism of the issuance life cycle that ZIP 227 specifies (ZIP 227, “Requirements”, “Issuance Action” and “Specification: Consensus Rule Changes”). Class: specified.

Proposition 7.8 (Finalised supply never grows). *In a reachable issuance state S of a valid chain, if $S(\text{AssetBase}).\text{final} = 1$, then in every later reachable state S' of the same chain $S'(\text{AssetBase}).\text{final} = 1$ and $S'(\text{AssetBase}).\text{balance} \leq S(\text{AssetBase}).\text{balance}$, and the balance is non-increasing from S on. The Asset can still be burnt. This is a property of the map; its relation to the notes in the tree is Theorem 9.7. Status (Definition 1.3): proved here for the specified rules; unconditional.*

Proof. Induction over the later transactions. Only (I7) writes final, and it writes 1. Rule (I4) rejects every later Issuance Action for AssetBase, with or without Issue Notes, since it reads final = 1, so (I6) never runs for that Asset Base again. The only other rule that writes balance is (T4), which subtracts. No rule of the burn half reads final, so a burn of the Asset is accepted whenever (T3) and (T4) hold. $\square$

Proposition 7.9 (The recorded balance). *In every reachable issuance state S of a valid chain and for every Asset Base B ,*

$$S(B).\text{balance} = \sum v_{\text{iss}} - \sum v_{\text{burn}} \in \{0, \dots, \text{MAX_ISSUE}\},$$

the first sum over the values of the Issue Notes of Asset Base B of the accepted Issuance Actions up to S , the second over the values of the pairs (B, v_{burn}) of the burn sets of the accepted transactions up to S . The balance increases only under (I6) and decreases only under (T4), so burn is the only mechanism that lowers it, and a burn of an Asset Base that has never been issued is rejected. The bound is on circulation, not on cumulative issuance (Definition 7.1). This is a property of the map; its equality with the value of the notes created and not consumed is Theorem 9.7. Status (Definition 1.3): proved here for the specified rules; unconditional.

Proof. Induction over the transactions, the claim being checked after every write. For the empty map both sides are 0. Rule (T2) copies a state. Rule (T4) subtracts v after checking balance $\geq v$, which keeps the identity and balance ≥ 0 ; an Asset Base without an entry reads 0, and (B2) gives $v > 0$, so a burn of a never-issued Asset Base fails. Rule (I6) adds v after checking that the sum is at most MAX_ISSUE, which keeps the identity and the upper bound; the reference note of (I5) has value 0 and enters both sides as 0. No other rule writes balance, and a rejected transaction contributes no state. $\square$

Figure 2 draws the transitions of one entry of the map.

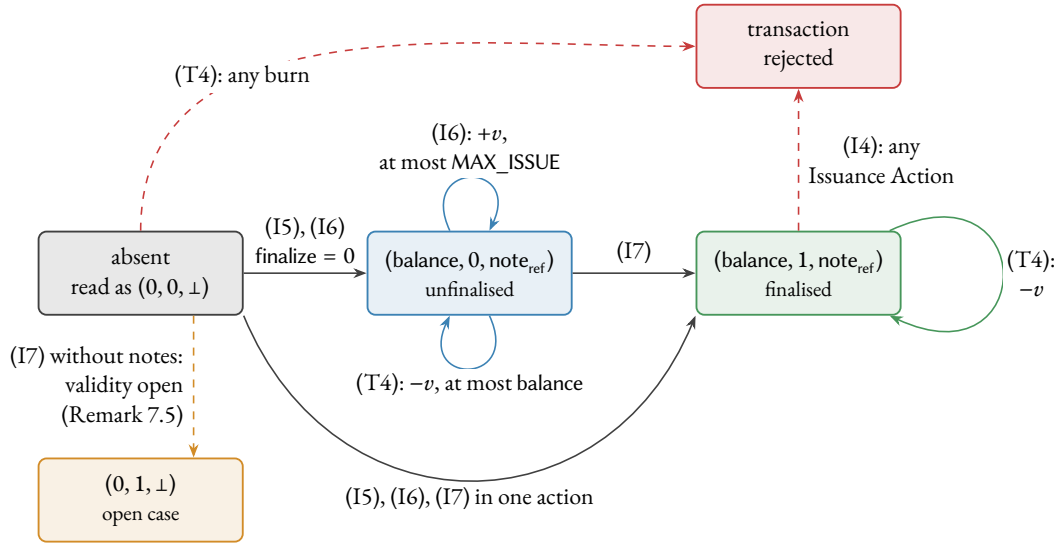


Figure 2. Transitions of one entry of the global issuance state of Definition 7.1. Solid edges are accepted transitions, labelled with the rules of Construction 7.3 that take them. The first issuance, by (I5) and (I6), creates the entry together with its reference note, as in Lemma 7.6; one action may also issue and finalise at once. The unfinalised entry grows under (I6), up to MAX_ISSUE , and shrinks under (T4), down to 0; the finalised entry only shrinks under (T4), again down to 0, and every Issuance Action on it is rejected by (I4), as in Proposition 7.8. A burn of an absent entry is rejected by (T4). The dashed edge to $(0, 1, \perp)$ is case (b) of Remark 7.5, an open problem.

7.4 Verification of an OrchardZSA transaction

The following procedure has the form of the *Ironwood Guide*’s Construction “Verification” (§“Verification of a transaction”). Each check is tagged with the requirements of Definition 1.8 that it serves and with the conditions of Definition 4.6 that it completes, and with its class in the sense of Definition 1.2: “specified” for the rule, followed by “carrier designed but unspecified” when its encoding exists only in withdrawn text, or “carrier open” when no design without conflict exists (Remark 8.1).

Construction 7.10 (Verification procedure for OrchardZSA). Given the note commitment tree and the nullifier set of the pool that carries OrchardZSA notes (Remark 1.7), and the issuance state $\text{issued_assets}_{\text{IN}}$, a transaction T is accepted, as far as its OrchardZSA bundle and its Issuance Bundle are concerned, if and only if the following checks pass.

Per Action.

- (1) *Proof.* The aggregate proof of the Action Group verifies for the relation $\mathcal{R}_{\text{ActionZSA}}$ of Definition 4.6, under the OrchardZSA verifying key of Assumption 4.9, at the primary input

$$(\mathcal{rt}, \text{cv}^{\text{net}}, \text{nf}, \text{rk}, \text{cmx}, \text{enableSpend}, \text{enableOutput}, \text{enableZSA})$$

of each Action. [Z1, Z2, Z3 and Z5, through A1’ to A5’ and C1 to C3; statement specified, circuit and verifying key designed but unspecified.]

- (2) *Spend authorisation*. Each Action’s spend-authorisation signature is valid under its rk on $SigHash$ (*Ironwood Guide*, §“Randomised validating keys”; unchanged by ZIP 226). [Requirement R5 of the *Ironwood Guide*’s Definition “Requirements R1 to R5”, with A6 and A7; specified, signed message designed but unspecified.]

Per Action Group and OrchardZSA bundle.

- (3) *Flags*. The values $enableSpends$, $enableOutputs$ and $enableZSA$ are read from the Action Group into every primary input of step (1) (Definition 4.13). [Z2, through C2; flags specified, the bit of $enableZSA$ open.]
- (4) *Anchor*. The anchor rt of the Action Group is the root of the carrying pool’s note commitment tree in the final treestate of an earlier block of the chain (*Ironwood Guide*, §“Anchors”). [Z2, through A3’; specified, the pool open.]
- (5) *Nullifiers*. The nullifiers of T , real and split, are pairwise distinct and absent from the carrying pool’s nullifier set (*Ironwood Guide*, §“Nullifier sets”). [Z3, through A5’; specified.]
- (6) *Burn set*. The burn set satisfies (B1) to (B3) of Definition 5.1; this is (T3). [Z1 and Z3; specified, carrier designed but unspecified.]
- (7) *Balance*. The binding signature is valid on $SigHash$ under the key bvk of Definition 5.4, computed from the Actions’ net value commitments, the balancing value and the burn set; the balancing value enters the transparent transaction value pool as in the *Ironwood Guide*’s Definition “Balancing value”. [Z1 and Z2, through A4’; specified, carrier designed but unspecified; the fee terms are classified in Remark 8.4.]

Per transaction.

- (8) *Issuance authorisation*. If T contains an Issuance Bundle, rules (I1) to (I3) of Definition 6.12 hold. [Z4; specified, signed message designed but unspecified.]
- (9) *Issuance state*. The output state $issued_assets_{OUT}$ of Construction 7.3 is defined: rules (T1), (T2) and (T4), and (I4) to (I7) for every Issuance Action. [(T1): carrier open; (T4): Z2 and Z3; (I4): Z2, Z3 and Z4; (I5) to (I7): Z3; specified.]

State update, on accepting the block that contains T .

- (10) The leaves of Definition 7.4 are appended to the carrying pool’s note commitment tree. [Z2 and Z3; specified, the pool open.]
- (11) Every nullifier of T , real and split, enters the carrying pool’s nullifier set. [Z3; specified.]
- (12) The issuance state becomes $issued_assets_{OUT}$, chained as in Definition 7.1; the ZEC chain value pool balance of the carrying pool decreases by the balancing value, as in the *Ironwood Guide*’s Construction “State update”. [Z3; specified.]

The remaining checks of the *Ironwood Guide*’s Construction “Verification”, on the transaction’s other components and on its expiry, apply unchanged; the parsing of the OrchardZSA components depends on the carrier, which is open (Remark 8.1) (ZIP 226, “Circuit Statement”, “Additional Consensus Rules for the assetBurn set” and “Value Balance Verification”; ZIP 227, “Specification: Consensus Rule Changes” and “Addition to the Note Commitment Tree”; protocol specification, §“Transactions and Treestates”, §“Nullifier Sets” and §“Note Commitment Trees”). Class: every rule specified; the classes of the carrier, the circuit, the signed message and the pool are as tagged.

The division of the checks follows the *Ironwood Guide*’s Remark “Division of the checks”. Under Assumption 4.9 the proof of step (1) yields, for each Action, a witness of Definition 4.6. Whether rt is a root that the chain recorded, whether a nullifier is new, and whether a burn or an issuance is admitted by the issuance state are facts about the chain’s history, which no proof over one Action’s data attests; the verifier checks them against its state, in steps (4), (5) and (9). Issuance is checked entirely outside the proof, from public data (Remark 6.6). Table 6 maps each requirement to the conditions of the statement, the issuance and state rules, the checks outside the proof and the result that discharges it, the last as in Table 2.

Table 6. Checks against the requirements they serve. For each requirement of Definition 1.8: the conditions of the OrchardZSA Action statement (Definition 4.6), the rules of Construction 7.3 and Definition 6.12, the numbered checks of Construction 7.10 outside the proof, and the result of “Security” that discharges it.

Requirement	Conditions	Issuance and state rules	Checks	Result
Z1, per-Asset conservation	A1', A2', A4', C1	none	(6), (7)	Theorem 9.5, “No counterfeiting across Assets”
Z2, no counterfeiting across Assets	A1', A2', A3', A4', C1, C2, C3	(I4), the derived base; (T4) with Lemma 7.7; Definition 7.4	(3), (4), (7), (10)	Theorem 9.5, “No counterfeiting across Assets”
Z3, supply integrity	A4', value 0 of Split Inputs; A5', split nullifier	(T4), (I4) to (I7)	(5), (6), (9) to (12)	Theorem 9.7, “Supply integrity”; Proposition 7.8, “Finalised supply never grows”
Z4, issuance authority	none	(I1) to (I3); (I4), the base derived from issuer	(8); (5), through $nf_{0,0}$	Theorem 9.8, “Issuance authority”
Z5, hiding of the Asset type	the statement as a whole under Assumption 4.10; A4', A5'	none, issuance being public (Remark 6.6)	none beyond the published burn set	Theorem 9.11, “Privacy of the Asset type”

8 The transaction carrier

This section states the status of every text that would carry OrchardZSA in a transaction: the transaction formats (§8.1), the digests and the message that the three kinds of signature sign (§8.2), and the fee terms (§8.3). No carrier is constructed here. The withdrawn design is described abstractly and classified as designed but unspecified; the absence of a live carrier is classified as an open problem (Definition 1.2).

8.1 Transaction formats

The carrier of a construction is the set of transaction fields that encode it together with the digests that commit to it (Definition 1.4).

Deferral of the encodings. ZIP 226, “OrchardZSA Transaction Structure”, states that the transaction format that carries OrchardZSA “is described in ZIP 230”, and defers to it the transaction structure and the note plaintexts. ZIP 227, “Issue Note”, “Issuance Action” and “Issuance Bundle”, defers to ZIP 230 the encodings of Issue Notes, of Issuance Actions and of the Issuance Bundle. ZIP 230, now titled “Withdrawn Version 6 Transaction Format”, has status Withdrawn (Definition 1.1): it “has been obsoleted by ZIP 229, and will not be deployed”, and transaction version 6 “is now defined by ZIP 229” (ZIP 230, “Abstract”). The consensus rules of ZIP 227 nonetheless name fields that exist only in ZIP 230, the Action-Group count and the per-group expiry field of rule (T1) of Construction 7.3 (“State transition of a transaction”, §7.2), and require every Issue Note description to be “a valid field encoding as defined in ZIP 230” (ZIP 227, “Specification: Consensus Rule Changes”). A Draft consensus rule thus delegates to withdrawn text. ZIP 226, “Privacy Implications” and “Implications for Wallets”, still identifies OrchardZSA with version-6 transactions.

Version 6. Version 6 is the transaction format of ZIP 229, “Version 6 Transaction Format” (Draft), which the active NU6.3 deploys (ZIP 258, Draft). It carries the Ironwood component beside the Orchard component and no bundle of OrchardZSA or of issuance (*Ironwood Guide*, §“The transaction format”, Construction “Version-6 transaction”). By ZIP 229, “Non-requirements”, the format “need not support ZSAs”. The withdrawn format and the format of ZIP 229 share the version number 6 and are distinct and incompatible.

Version 7. ZIP 248, “Extensible Transaction Format” (Draft), defines transaction version 7, whose version group identifier is not yet assigned and whose “Deployment” section is empty. Its bundle registry allocates no bundle to ZSA. OrchardZSA, proposed as a provisional variant of the Orchard bundle type (ZIP 226), and ZSA Issuance, with no bundle type (ZIP 227), appear only under “Potential Future Bundle Types”, kinds that “have no allocated (bundleType, bundleVariant) pair”. ZIP 248 specifies no encoding and no digest for either (ZIP 248, “Terminology”, “Abstract”, “Potential Future Bundle Types” and “Deployment”).

Content and withdrawn design. The abstract content that the rules of this volume need is fixed earlier and does not depend on a carrier: the OrchardZSA bundle of Definition 5.2, with at most one Action Group, one burn set, the balancing value and the binding signature, and the Issuance Bundle of Definition 6.5. The withdrawn design of ZIP 230 encodes this content as follows; no part of it is a live layout.

- (a) The burn set is encoded inside the Action Group as a list of 40-byte entries, each the 32-byte encoding `asset_base` of Definition 2.14 followed by the amount as an unsigned 64-bit little-endian integer. ZIP 230 thus places one burn set in each Action Group, which under the limit of ZIP 227 of one Action Group per transaction is one burn set per transaction. Its field description of the amount, “non-zero, and less than `MAX_BURN_VALUE`”, is superseded by rule (B2) of Definition 5.1, $0 < v \leq \text{MAX_BURN_VALUE}$, which is ZIP 226’s (ZIP 230, “OrchardZSA Action Group Description” and “OrchardZSA Asset Burn Description”; ZIP 226, “Additional Consensus Rules for the `assetBurn` set”).

- (b) The flags byte of the Action Group assigns bit 2 to enableZSA, an assignment without live counterpart (“Action Groups”, §4.5).
- (c) Each Action carries the public fields of an Orchard Action description; the Action Group carries, in Action order, one spend-authorisation signature per Action, and the aggregate proof. The byte lengths of an Action depend on a plaintext encoding that no current ZIP defines (“Action Groups”, §4.5).

The pool into which a carrier would place OrchardZSA notes is the subject of Remark 1.7.

Remark 8.1 (Status of the carrier). Under Definition 1.2:

- (i) the transfer, burn and issuance constructions are specified, in ZIPs 226 and 227, both Draft;
- (ii) the field sets and digests of withdrawn ZIPs 230 and 246, read as a composition of those constructions into one transaction and abstracted from the version number, the version group and the positions of the flag bits, are designed but unspecified: a complete design exists in withdrawn text, and no live text fixes it. The exception is the authorising-data digest, which ZIP 246 itself marks as not updated for sighashInfo (ZIP 246, “v0 Digests”), and whose issuance child states its empty case for a transaction with no Orchard Actions (ZIP 246, “A.4: issuance_auth_digest”); no result of this volume uses that digest;
- (iii) which live transaction format carries OrchardZSA bundles and Issuance Bundles is an open problem. The only design assigns the version number 6 and bit 2 of the Orchard flags, both of which live text assigns otherwise: version 6 is the format of ZIP 229, in which that bit is reserved. ZIP 229 need not support ZSAs, and ZIP 248 allocates no bundle to them.

The texts that leave (iii) open and the obligation that closes it are enumerated in “Open problems” (§10.2) (ZIP 229, “Non-requirements”; ZIP 230 and ZIP 246, “Abstract”; ZIP 248, “Potential Future Bundle Types”).

8.2 Transaction digests and the signature message

The signed message. Every signature that OrchardZSA uses signs one message, SigHash: the issuance authorisation signature, by rule (I3) of Definition 6.12, the spend-authorisation signature of each Action, and the binding signature of the bundle (Definition 5.4; ZIP 226, “Value Balance Verification”; *Ironwood Guide*, Definition “Signature digest”). ZIP 227 defines SigHash as the digest of the protocol specification, §“SIGHASH Transaction Hashing”, under SIGHASH_ALL, “with the modifications described in ZIP 226”, and cites for them a ZIP 226 section “TxId Digest” that does not exist. ZIP 226, “Sighash modifications relative to ZIP 244”, and ZIP 227, “Modifications relative to ZIP 244”, defer those modifications to ZIP 246, “Digests for the Withdrawn Version 6 Transaction Format”, whose status is Withdrawn and whose digests “are now specified by ZIP 229” (ZIP 227, “Specification: Consensus Rule Changes”; ZIP 246, “Abstract”).

Withdrawn design of the transaction identifier. ZIP 246 extends the digest tree of ZIP 244 (*Ironwood Guide*, Definition “Transaction identifier”). Each node is the function $h = \text{BLAKE2b-256}$ under a personalisation of its own, applied to the concatenation of fixed-length

digests and field encodings; a node that ZIP 246 does not redefine keeps its definition in ZIP 244. The root has six children:

$$\text{txid} := h(\text{ZcashTxHash_} \parallel \beta, d_{\text{hdr}} \parallel d_{\text{tr}} \parallel d_{\text{sap}} \parallel d_{\text{orc}} \parallel d_{\text{iss}} \parallel d_{\text{memo}}),$$

with β the encoding of the consensus branch identifier as in the *Ironwood Guide*. The header digest d_{hdr} , redefined with fields of ZIPs outside this volume, the transparent digest d_{tr} and the Sapling digest d_{sap} carry no ZSA content. The other three children do.

- (a) The *Orchard branch* d_{orc} hashes one digest over the Action Groups and the balancing value. For each Action Group, the digest over the Action Groups commits to every Action’s nullifier nf , note commitment cmx , ephemeral key epk and ciphertexts C_{enc} and C_{out} , value commitment cv^{net} and randomised validating key rk , in Action order, and to the flags byte, the anchor, the per-group expiry field and a *burn digest*. The burn digest hashes every burn pair ($\text{asset_base}, v$) in encoding order, and is the hash of the empty string when the burn set is empty. Without an Action Group the Orchard branch is the hash of the empty string; since the burn set is encoded inside the Action Group, a transaction then has no burn set. With at most one Action Group per transaction (ZIP 227), the digest over the Action Groups is one hash over the elements of that group.
- (b) The *issuance branch* d_{iss} hashes the length-prefixed issuer and an *actions digest*, which commits, per Issuance Action and in order, to assetDescHash , to a *notes digest* over every Issue Note’s recipient, value, ρ and rseed , and to flagsIssuance . ZIP 246 states empty cases for the issuance digest, “no issuance components”, and for the notes digest, and none for the actions digest.
- (c) The *memo branch* d_{memo} hashes a nonce and a digest over memo chunks, and presupposes the memo bundles of ZIP 231.

The personalisation strings of the nodes below the root are not listed, since no proof or rule of this volume uses them (ZIP 246, “TxId Digest”, nodes T.4, T.4a, T.4a.vi, T.5, T.5a, T.5a.i and T.6; ZIP 244, “TxId Digest”).

Withdrawn design of the signature digest and the authorising-data digest. The signature digest has the same six children as the transaction identifier. Its Orchard, issuance and memo children are those of txid , and its transparent child is specific to the input, as in ZIP 244 (*Ironwood Guide*, Definition “Signature digest”). ZIP 246 produces a signature digest for each transparent input, each Sapling input, each Action and each Issuance Action; all of these except those of transparent inputs equal the SIGHASH_ALL digest SigHash , since no other child depends on the signer. The authorising-data digest (*Ironwood Guide*, Definition “Authorising-data digest; effecting and authorising data”) has transparent, Sapling, Orchard and issuance children. The Orchard child hashes, per Action Group, the proof and the spend-authorisation signatures, and then the binding signature; the issuance child hashes the encoding of issueAuthSig . Proofs and signatures enter this digest only. ZIP 246 marks this digest as not yet updated for the sighashInfo fields of its format (ZIP 246, “v0 Digests”). ZIP 246 states that the pair of the transaction identifier and the authorising-data digest commits to all data of the transaction (ZIP 246, “Signature Digest” and

“Authorizing Data Commitment”, nodes A.3, A.3a and A.4; ZIP 244, “Signature Digest” and “Authorizing Data Commitment”).

Remark 8.2 (Commitments required of a digest). Under the withdrawn design, the digest SigHash commits

- (i) to the effecting data of the Issuance Bundle: issuer and, per Issuance Action and in order, assetDescHash, finalize and every field of every Issue Note, namely the recipient (d, pk_d) , v , ρ and rseed; the Asset Base is derived from issuer and assetDescHash (Remark 6.14);
- (ii) to the burn set;
- (iii) to the published data of the Actions of the Action Group, hence to $nf_{0,0}$;

and to no signature. The argument runs node by node. The signature digest’s Orchard child is the Orchard branch (a), and its issuance child is the issuance branch (b); their inputs include the data listed in (i) to (iii), the flag finalize as a bit of flagsIssuance. Signatures and proofs enter the authorising-data digest only. In the withdrawn design every node input is a concatenation of fixed-length digests and of fixed-length or length-prefixed field encodings, so that it parses uniquely, and the number of records of each list is determined by the length of the input. Let two transactions with equal SigHash differ in a listed datum. If their consensus branches differ, the root personalisations differ and equal outputs are already a collision in the sense of the *Ironwood Guide*’s Assumption “Collision resistance of BLAKE2b”; otherwise the differing datum yields two distinct inputs with equal outputs at some node, a collision of h . The withdrawn design thus meets Assumption 6.13 under the *Ironwood Guide*’s Assumption “Collision resistance of BLAKE2b”. The three commitments (i) to (iii), together with the exclusion of every signature, are the obligation on any future signature digest that carries OrchardZSA: Proposition 6.16 and Theorem 9.8 assume exactly them and nothing more (ZIP 246, “TxId Digest” and “Signature Digest”).

Current text. The version-6 digests of ZIP 229 have header, transparent, Sapling, Orchard and Ironwood children and no issuance, burn or memo-bundle data (no counterpart of the branch d_{memo} of ZIP 246); their Orchard child carries Orchard-pool Actions, which have no Asset Bases (*Ironwood Guide*, §“Transaction digests and signatures”; ZIP 229, “Transaction Identifiers, Auth Digests, and Signature Digests” and “Summary of the resulting digest structure”). ZIP 248 lets each bundle type define its own versions of the signature digest, carried with each signature as sighashInfo, and defines none for a ZSA bundle (ZIP 248, “Sighash Versioning” and “Potential Future Bundle Types”). No current text specifies a digest that commits to OrchardZSA or issuance data.

Remark 8.3 (Status of the signature message). The rules by which the three kinds of signature sign SigHash are specified, in ZIPs 226 and 227, both Draft. The message SigHash itself is designed but unspecified: its only definition is withdrawn ZIP 246, a complete design. Its live definition, as part of a live carrier, is an open problem (Remark 8.1). Every result of this volume that uses the message is conditional on Assumption 6.13 (Definition 1.3) (ZIP 226, “Sighash modifications relative to ZIP 244”; ZIP 227, “Modifications relative to ZIP 244”; ZIP 246).

8.3 Fees

The conventional fee. The transaction fee is the value remaining in the transparent transaction value pool, and ZIP 317, “Fee calculation”, specifies the conventional fee

$$\begin{aligned} & \text{marginal_fee} \cdot \max(\text{grace_actions}, \text{logical_actions}) \text{ zatoshi,} \\ & \text{marginal_fee} = 5000, \quad \text{grace_actions} = 2, \end{aligned}$$

where *logical_actions* is a sum of contributions per component, fixed by the applicable revision of ZIP 317. The fee and the formula are those of the *Ironwood Guide*, §“The balancing value”, and are cited, not re-derived. Paying the conventional fee is not a consensus requirement: “It is not a consensus requirement that fees follow this formula; however, wallets SHOULD create transactions that pay this fee” (ZIP 317, “Fee calculation”).

Fees in ZEC. Fees are paid in ZEC only. The balancing value is an amount of ZEC and enters the binding validating key at the base V^{Orchard} only (Definition 5.4), and a Custom Asset has no transparent value pool (Remark 5.3), so no Custom Asset reaches the pool whose remainder is the fee. ZIP 227 records the alternative, a fee denominated in the Custom Asset, as not adopted, since for shielded transactions any design that lifts value from the transaction leaks information about it (ZIP 226, “Value Balance Verification”; ZIP 227, “Rationale for paying fees in ZEC”).

Missing ZSA terms. ZIP 227, “Changes to ZIP 317”, states that the conventional fee in ZEC is altered for the Issuance Actions of a transaction and for Custom Assets newly created in the global issuance state, and refers to ZIP 317, “Fee calculation”. No revision of ZIP 317 has such a term; Revision 0 is Active and Revisions 1 and 2 are Draft. The Change History of ZIP 317 records “Removed the ZSA issuance and asset-creation fee contributions (ZIP 227)”. ZIP 226, “Transaction Fees”, links a ZIP 227 section “OrchardZSA Fee Calculation” that does not exist. Both references dangle. The removed design had one contribution per Issue Note and one per newly created Custom Asset; their values are not quoted (ZIP 317, “Revisions”, “Fee calculation” and “Change History”).

Remark 8.4 (Status of the fee terms). The ZSA fee terms have one class, designed but unspecified (Definition 1.2): a complete design existed as text of ZIP 317 and was removed, and it conflicts with no live text. Live ZIP 227, “Changes to ZIP 317”, states that the conventional fee is altered, and live ZIP 317, “Fee calculation”, has no such term. The conventional fee is not a consensus rule, so the protocol of this volume does not need the fee terms in the sense of Definition 1.2(iii), and they are not an open problem. That fees are paid in ZEC is specified (ZIP 317, “Change History”; ZIP 227, “Changes to ZIP 317”).

9 Security

This section composes the propositions stated at their objects into the five requirements of Definition 1.8, as Table 2 names them: Theorem 9.5 discharges Z1 and Z2; Theorem 9.7, with Proposition 7.8, discharges Z3; Theorem 9.8 discharges Z4; and Theorem 9.11 discharges Z5. The model and the assumptions are fixed in §9.1, and the four theorems are proved in §9.2 to §9.5.

The conventions are those of the *Ironwood Guide*, §“Security”, cited, not restated. Efficient adversaries and negligible functions are those of the *Crypto Guide*, §“Adversaries and the security pa-

parameter”, and computational indistinguishability is that of its Definition “Perfect, statistical, and computational indistinguishability” (§“Distribution ensembles and indistinguishability”). Every reduction simulates the honest parties and the random oracles, which are independent of one another. Every result holds except with negligible probability, and every result that uses a property of key generation is claimed for keys generated as the *Ironwood Guide*, §“Security”, claims them, with `use_qsk = false`.

9.1 Accepted transactions and assumptions

Definition 9.1 (Accepted OrchardZSA transactions and witnesses). The definition extends the *Ironwood Guide*’s Definition “Pools, accepted Actions and witnesses” to OrchardZSA.

- (a) *Pool*. The pool is the pool that carries OrchardZSA notes, which Remark 1.7 leaves open. It has one note commitment tree, one nullifier set and one issuance state `issued_assets` (Definition 7.1); every statement below uses these.
- (b) *Valid chain*. An adversary outputs a *valid chain*: a sequence of blocks each of whose transactions passes Construction 7.10 against the tree, the nullifier set and the issuance state that the transactions before it produce, the tree being extended in the order of Definition 7.4 and the issuance state by Construction 7.3. An Action, an Action Group (Definition 4.13), an OrchardZSA bundle (Definition 5.2) or an Issuance Bundle (Definition 6.5) of a transaction of a valid chain is *accepted*.
- (c) *Witnesses*. The *witness* of an accepted OrchardZSA Action is the auxiliary input that the extractor of Assumption 4.9 outputs for the proof of its Action Group, the assumption being applied to each Action Group as the *Ironwood Guide* applies its knowledge-soundness assumption to each bundle (the paragraph after its Definition “Pools, accepted Actions and witnesses”). It contains `AssetBase`, `is_native_asset`, `split_flag` and ψ^{nf} besides the Orchard witness, and satisfies Definition 4.6. The Action *consumes* the note

$$(g_d^{\text{old}}, pk_d^{\text{old}}, v^{\text{old}}, \text{AssetBase}, \rho^{\text{old}}, \psi^{\text{old}}, \text{rcm}^{\text{old}})$$

of its witness, with value $v' := 0$ if `split_flag = 1` and $v' := v^{\text{old}}$ otherwise (the notation of $A4'$), and *creates* the note

$$(g_d^{\text{new}}, pk_d^{\text{new}}, v^{\text{new}}, \text{AssetBase}, \rho^{\text{new}}, \psi^{\text{new}}, \text{rcm}^{\text{new}}),$$

with ρ^{new} the published nullifier. Notes are given by expanded receivers as in item (d) of the cited definition. An accepted Action with `split_flag = 0` is a *real spend* of its consumed note; one with `split_flag = 1` is a Split Action (Definition 4.2). The shared pool also admits Orchard-protocol Actions (ZIP 226, “Privacy Implications”); the witness of such an Action is that of the *Ironwood Guide*’s Assumption “Knowledge soundness of the Action proof”, and every such Action is a real spend.

- (d) *Issue Notes*. An Issue Note of an accepted Issuance Bundle is given by its published fields and the Asset Base that rule (I4) of Construction 7.3 derives. Its note commitment is the

one that the validator computes by rule (I6), which presupposes a fixed deterministic derivation of rcm (Remark 6.2, Definition 1.3).

- (e) *Committed notes and openings.* A *committed note* is a note whose extracted commitment is a leaf of the tree. An *opening* of a leaf is a note whose extracted commitment equals it.
- (f) *Adversary.* The adversary controls every party except the honest ones that it attacks. It may generate issuance keys, issue Assets under them with any descriptions, choose every field of its own transactions, copy any leaf as a Split Input, and obtain from each honest issuer the signed transactions that the issuer composes, in an order that it schedules.

(ZIP 226, “Specification”; ZIP 227, “Specification: Consensus Rule Changes”).

By the union bound over the polynomially many Action Groups of a valid chain, every witness satisfies Definition 4.6, except with negligible probability. Conditions $A1'$, $A2'$, $A3'$ and $A7$ then hold in their unweakened forms, without the $\perp$ case, except with negligible probability, by the *Ironwood Guide*’s Remark “ $\perp$ -weakened conditions” (§“The Action statement”) and, for the custom branch of $A1'$ and $A2'$, by Proposition 3.3(c), under the *Ironwood Guide*’s Assumptions “Discrete logarithms on Pallas” and “GroupHash as a random oracle”. The results below include this event in their negligible terms.

Remark 9.2 (Standing assumptions). The results of this section rest on the following assumptions, each stated in the text named.

1. The *Ironwood Guide*’s Assumptions “GroupHash as a random oracle” (§“GroupHash, domain separation, and nothing-up-my-sleeve generators”) and “Discrete logarithms on Pallas” (§“Sinsemilla: hash, commitment, and short forms”).
2. The *Ironwood Guide*’s Assumption “Collision resistance of BLAKE2b” (§“Transaction digests and signatures”), and Assumption 2.9 (“Asset Digests and Asset Bases”).
3. Assumption 6.8 (“Computation of ρ for Issue Notes”).
4. Assumption 2.3 (“Issuance keys and the issuer identifier”).
5. Assumptions 4.9 and 4.10 (“The OrchardZSA Action proof”), and the *Ironwood Guide*’s Assumption “Knowledge soundness of the Action proof” for Orchard-protocol Actions.
6. The *Ironwood Guide*’s Assumption “The RedPallas hash as a random oracle” (§“The Red-Pallas signature scheme”).
7. The assumptions of the *Ironwood Guide*’s Proposition “Confidentiality and key privacy of note encryption” (§“Encryption to the recipient”), and its Assumptions “Poseidon is a PRF” (§“The nullifier”), “Pseudorandomness of PRF expansion” (§“Pseudorandom functions and field reductions”) and “Joint hiding of the viewing keys” (§“Viewing keys”).
8. The hypothesis H-SIG, Assumption 6.13 (“Issuance authorisation”), in the form that Remark 8.2 states as the obligation on a digest.

Following Definition 1.3, three of them concern designed but unspecified objects: Assumptions 4.9 and 4.10, the circuit and its verifying key, and Assumption 6.13, the signed message. Each result below names which of these it is conditional on, and the results about Issue Notes add the fixed derivation of rcm . The *Ironwood Guide*'s results on its own statement, for example its Theorems "Balance" and "Privacy of an Action", are not cited as proved for OrchardZSA; only their proof steps are reused, each cited at its use.

9.2 Asset Bases and balance of accepted bundles

Lemma 9.3 (Provenance of Asset Bases). *Under Assumption 4.9, the Ironwood Guide's Assumptions "Knowledge soundness of the Action proof", "GroupHash as a random oracle" and "Discrete logarithms on Pallas", in every valid chain, except with negligible probability:*

- (i) *every leaf of the tree has at most one opening that an efficient algorithm finds;*
- (ii) *every leaf whose opening has $\text{AssetBase} \neq V^{\text{Orchard}}$ opens to a note whose Asset Base is*

$$\text{ZSAValueBase}(\text{AssetDigest}_{(\text{issuer}, \text{assetDescHash})})$$

(Construction 2.10) for the issuer of an accepted Issuance Bundle and the assetDescHash of one of its Issuance Actions, whose Issue Notes precede the leaf, or include it, in the order of Definition 7.4;

- (iii) *every leaf appended by an Orchard-protocol Action, before or after OrchardZSA, opens only with $\text{AssetBase} = V^{\text{Orchard}}$, and the empty leaf opens to nothing.*

Status (Definition 1.3): proved here for the specified statement and rules; conditional on Assumption 4.9, which concerns the designed but unspecified circuit, and, for the leaves of Issue Notes, on a fixed derivation of rcm , any deterministic one.

Proof. Every step is carried out by the efficient algorithm that runs the adversary with the extractors and inspects the chain; the chain has polynomially many leaves, and the union bound collects the negligible events.

Uniqueness of openings. By Proposition 3.3(b), no efficient algorithm outputs two distinct OrchardZSA notes with equal extracted commitments other than $\perp$, within one branch of Definition 3.2 or across the two; the sign ambiguity of extraction is handled in its proof by the *Ironwood Guide*'s Lemma "Coordinate extraction identifies opposite points". An Orchard note is the native-branch OrchardZSA note with the same components, by the case split of that definition. Hence (i), and an opening found in the native branch excludes one in the custom branch.

Induction. The claim is proved for the leaves in the order of Definition 7.4, the hypothesis being (ii) and (iii) for all earlier leaves. A position beyond the appended leaves holds $\text{Uncommitted}^{\text{Orchard}} = 2$, which by the *Ironwood Guide*'s Lemma "Uncommitted leaves" is the extracted commitment of no note, since in both branches cmx is $\perp$ or the extraction of a point. Every appended leaf is of one of three kinds.

Orchard-protocol Action. The witness of the Action that appended the leaf creates a note whose

commitment $\text{NoteCommit}^{\text{Orchard}}$, by the unweakened condition A2, extracts to the leaf. That commitment is the native branch of $\text{NoteCommit}^{\text{OrchardZSA}}$ (Definition 3.2), so the leaf has an opening with $\text{AssetBase} = V^{\text{Orchard}}$, and by (i) no other. This holds for leaves appended before OrchardZSA and after it.

OrchardZSA Action. Let the witness have $\text{AssetBase} = B$. If $B = V^{\text{Orchard}}$ there is nothing to show. Otherwise $\text{is_native_asset} = 0$ by C1, so the first disjunct of A3' fails, and the unweakened A3' gives a valid path from $\text{Extract}_p(\text{cm}^{\text{old}})$ to the anchor rt of the Action Group. By check (4) of Construction 7.10, the anchor is the root of the tree in the final treestate of an earlier block. By the *Ironwood Guide*'s Proposition "Membership soundness", and since $\text{cm}^{\text{old}} \neq \perp$ by the unweakened A1', the value $\text{Extract}_p(\text{cm}^{\text{old}})$ is a leaf appended by an earlier transaction. By A1', the consumed note, of Asset Base B , opens that leaf, and by (i) it is its opening. By the hypothesis, B is the Asset Base derived from the issuer and an assetDescHash of an earlier accepted Issuance Action whose Issue Notes precede that leaf. By A2', the created note carries the same B and opens the new leaf, and by (i) it is its opening. The case covers Split Actions: A3' demands the path for every consumed note with $\text{is_native_asset} = 0$, whatever its value and split_flag , and C3 forbids $\text{split_flag} = 1$ with $\text{is_native_asset} = 1$.

Issue Note. Rule (I4) derives the Asset Base from the bundle's issuer and the action's assetDescHash by Construction 2.10; rule (I6) makes the validator compute the commitment from that base and the published fields (Remark 6.14), with the fixed derivation of rcm ; its commitment is not $\perp$ except with negligible probability, by Proposition 3.3(c), its Asset Base being a custom one (Proposition 2.13). The Issue Note therefore opens the leaf, and by (i) it is its opening; its Asset Base is the derived one, which differs from V^{Orchard} except with negligible probability (Proposition 2.13), and its own Issuance Action includes the leaf. $\square$

ZIP 226 states the conclusion in rationale: "This ensures that the value base points of all output notes of a transfer are actual outputs of a GroupHash, as they originate in the Issuance protocol which is publicly verified" ("Rationale for Split Notes"), and the output notes "receive an Asset Base that exists on the global state" ("Rationale for Asset Identifiers"). In Lemma 9.3, existence on the global state is realised by membership in the tree; the proof never consults issued_assets (ZIP 227, "Addition to the Note Commitment Tree"). Figure 3 draws the induction.

Corollary 9.4 (Asset Bases of accepted Actions). *Under the assumptions of Lemma 9.3, in every valid chain, except with negligible probability, every Asset Base witnessed by an accepted OrchardZSA Action is V^{Orchard} or $\text{ZSAValueBase}(\text{AssetDigest})$ of the Asset Identifier of an Issuance Action of an accepted Issuance Bundle of an earlier transaction. No transactor chooses an Asset Base: every Custom Asset Base in an accepted Action is one that a validator derived. Status (Definition 1.3): proved here; conditional on Assumption 4.9 and, through Lemma 9.3, on a fixed derivation of rcm for Issue Notes.*

Proof. For a witness with $\text{AssetBase} = V^{\text{Orchard}}$ there is nothing to show. The exemption of A3' applies only to such consumed notes, since C1 makes $\text{is_native_asset} = 1$ exactly when $\text{AssetBase} = V^{\text{Orchard}}$. Otherwise, as in the proof of Lemma 9.3, A3' and the *Ironwood Guide*'s Proposition "Membership soundness" make the consumed note an opening of a leaf of the treestate at the anchor, appended by an earlier transaction; A1' fixes its Asset Base as the witnessed one; and

bundles: the Asset Bases of burn pairs are constrained only by rule (T4) of Construction 7.3 against the issuance state. Status (Definition 1.3): proved here for the specified constructions; conditional on Assumption 4.9, which concerns the designed but unspecified circuit, and on a fixed derivation of rcm for Issue Notes, and not on Assumption 6.13. ZIP 226 argues the property in rationale only.

Proof. Witnessed Asset Bases. By Corollary 9.4, each is V^{Orchard} or

$$\text{GroupHash}(z.\text{cash}:\text{OrchardZSA}, D)$$

for the Asset Digest D of an accepted Issuance Action.

Asset Bases of burn pairs. By rule (T4) every burn pair of an accepted transaction has an Asset Base in the domain of its input state, and by Lemma 7.7 every such Asset Base is $\text{GroupHash}(z.\text{cash}:\text{OrchardZSA}, D)$ for the Asset Digest D of an accepted Issuance Action. Rule (B1) alone does not give this.

Premise and balance. Every input $(z.\text{cash}:\text{OrchardZSA}, D)$ differs from the inputs of V^{Orchard} and R^{Orchard} (Lemma 2.8). Each such input is computable from the chain, by searching its Issuance Actions for one whose derived Asset Base is the point. So the algorithm that runs the adversary with the extractors, and outputs the accepted bundles together with these inputs whenever it finds them, outputs bundles whose set $\mathfrak{B}$ meets the premise on the bases of §5.3; by the two preceding steps it fails to find an input only with negligible probability. Parts (a) and (b) of Proposition 5.6 then give (i) and (ii), the congruences modulo p_{Vesta} being lifted to integers by Lemma 5.5, whose hypotheses hold for every bundle of a valid block. A Split Action contributes $v' = 0$ by A4', so the value of a copied note never enters these sums. No other term of bvk carries a Custom Asset (Remark 5.3).

Part (iii). Claims 2 and 3 of Proposition 2.13: distinct Asset Identifiers have distinct Asset Bases, and no Asset Base of a Custom Asset equals V^{Orchard} , except with negligible probability, under Assumption 2.9 and the *Ironwood Guide's* Assumption “GroupHash as a random oracle”. $\square$

Each premise of the proof is necessary. For witnessed Asset Bases, Proposition 4.1 creates ZEC from a witnessed base $[-1] V^{\text{Orchard}}$ when the exemption of A3' is not confined to ZEC. For burn pairs, the example after Proposition 5.6 creates $2v$ units of an issued Asset B with the pair $([-2] B, v)$, which rules (B1) to (B3) admit and rule (T4) rejects. ZIP 226, “Rationale for Split Notes”, “Value Balance Verification” and “Additional Consensus Rules for the assetBurn set”, gives the argument informally; the proof here does not change the class of that rationale, and ZIP 227, “Specification: Consensus Rule Changes”, supplies rule (T4).

9.3 Circulating supply

Lemma 9.6 (One consumption per OrchardZSA note). *Under Assumption 4.9, Assumption 6.8, and the Ironwood Guide's Assumptions “Knowledge soundness of the Action proof”, “GroupHash as a random oracle” and “Discrete logarithms on Pallas”, in every valid chain, except with negligible probability:*

- (a) the openings of the leaves of the tree have pairwise distinct elements ρ , and are therefore pairwise distinct notes;
- (b) every committed note is consumed by at most one accepted real spend;
- (c) a Split Action contributes $v' = 0$ and publishes a nullifier that is not the nullifier of its copied note.

Status (Definition 1.3): proved here; conditional on Assumption 4.9, which concerns the designed but unspecified circuit, and, for the leaves of Issue Notes, on a fixed derivation of rcm, any deterministic one.

Proof. By Lemma 9.3(i), the opening of a leaf appended by an Action is the note that the Action's witness creates, and the opening of a leaf appended by issuance is the Issue Note.

(a) Three cases cover every pair of leaves.

Two Action outputs. The note created by an accepted Action, of either protocol, has ρ equal to that Action's published nullifier (A2' with $\rho^{\text{new}} := \text{nr}^{\text{old}}$, or A2), and the published nullifiers, real and split, are pairwise distinct by the nullifier-set rule, check (5) of Construction 7.10.

Two Issue Notes. Their elements ρ are pairwise distinct by Proposition 6.9, under Assumption 6.8.

An Action output and an Issue Note. It suffices that no accepted Action publishes a nullifier equal to the ρ of an Issue Note of the chain, in whichever order the two occur; Proposition 6.9 covers only the Action outputs fixed before the issued value. Let an efficient algorithm $\mathcal{A}'$, running the adversary with the extractors, find such a pair with probability ϵ . The published nullifier is $\text{Extract}_{\text{p}}(Q)$, where by A5' (or A5)

$$Q = [s] K^{\text{Orchard}} + \text{cm}^{\text{old}} \quad \text{or} \quad Q = [s] K^{\text{Orchard}} + \text{cm}^{\text{old}} + L^{\text{Orchard}},$$

the second for a split nullifier, with s computed from the witness. By the unweakened A1' (or A1), $\text{cm}^{\text{old}} \neq \perp$ is, by the unfolding of the *Ironwood Guide's* Construction “Sinsemilla hash on Pallas” used in the proof of Proposition 4.5(i), an integer combination of values of GroupHash with coefficients computable from the witness; so is Q . A reduction to the *Ironwood Guide's* Assumption “Discrete logarithms on Pallas” receives a challenge Y to a base G , programs every value of GroupHash as $[a] G$ for a fresh uniform scalar a , as in the proof of that volume's Lemma “Relations among fixed generators”, chooses uniformly one of the at most q queries to the oracle of personalisation Zcash_ExpandSeed , and answers it with a uniform 64-byte string u conditioned on $\text{ToBase}(u) = \text{Extract}_{\text{p}}(Y + [b] G)$, for a uniform scalar b . If $\mathcal{A}'$ succeeds at that query, then $\text{Extract}_{\text{p}}(Q) = \text{Extract}_{\text{p}}(Y + [b] G)$, so $Q = \pm(Y + [b] G)$ by the *Ironwood Guide's* Lemma “Coordinate extraction identifies opposite points”; with $Q = [c] G$ for the computed c , the reduction outputs $\pm c - b$, the discrete logarithm of Y . The programmed answer differs from a uniform one in that its reduction is the x -coordinate of a point, up to the statistical distance of ToBase from uniform, below 2^{-257} (Proposition 4.5); success requires the issued ρ to be such a coordinate in any case, and, given that it is one, the two distributions of ρ are within negligible statistical distance. The reduction therefore succeeds with probability at least ϵ/q less a negligible term, and ϵ is negligible.

Notes with distinct ρ are distinct.

(b) The steps of the proof of the *Ironwood Guide*'s Lemma "One nullifier per note" apply, with A1' in place of A1 and A5' with `split_flag = 0`, which sets $\psi^{\text{nf}} = \psi^{\text{old}}$ and publishes $\text{DeriveNullifier}_{\text{nk}}(\rho^{\text{old}}, \psi^{\text{old}}, \text{cm}^{\text{old}})$, in place of A5; A7 is unchanged. Two real spends, of either protocol, whose consumed notes have the same extracted commitment consume the same note, by Proposition 3.3(b), with the same nk, by the *Ironwood Guide*'s Proposition "Binding of ivk to (ak, nk)", and publish the same nullifier. The nullifier-set rule rejects a repeated nullifier within a transaction and across the chain.

(c) The value $v' = 0$ is the notation of A4' for `split_flag = 1`. The nullifier is the split nullifier of A5', and by Proposition 4.5(i), which holds for every ψ^{nf} , including one that the adversary chooses, it is not the nullifier of the copied note, except with negligible probability. A split nullifier that equals the nullifier of some other note only prevents the later spend of that note; it creates no value, and the note is not thereby consumed in the sense of Definition 9.1(c). (ZIP 226, "Split Notes"; ZIP 227, "Computation of ρ "; protocol specification, §"Nullifier Sets".) $\square$

Theorem 9.7 (Supply integrity). *Under the assumptions of Theorem 9.5 and Lemma 9.6, for every valid chain, every point $B \neq V^{\text{Orchard}}$ and every state of the chain after a transaction, except with negligible probability,*

$$\sum_n v(n) = \text{issued_assets}(B).\text{balance},$$

the sum being over the committed notes n with Asset Base B that no accepted real spend has consumed, and $v(n)$ being the value of n . By Proposition 7.9, the right-hand side is the amount of B issued less the amount burnt. Consequently the amount of B in circulation is never negative and is at most $\text{MAX_ISSUE} = 2^{64} - 1$, and it is constant or falling once B is finalised (Proposition 7.8). The invariant holds per branch, since the issuance state is a function of the branch (Remark 7.2). Status (Definition 1.3): proved here for the specified constructions; conditional on Assumption 4.9, which concerns the designed but unspecified circuit, and on a fixed derivation of rcm for Issue Notes, any deterministic one. ZIP 227 argues the property in rationale only.

Proof. Induction over the transactions of the chain. Write U_B for the left-hand side.

Base. Before the first transaction of the OrchardZSA activation block the issuance state is the empty map (Definition 7.1), whose balances read 0, and every leaf was appended by an Orchard-protocol Action, so by Lemma 9.3(iii) no committed note has a Custom Asset Base: $U_B = 0$.

Step. Let T be a transaction of the chain, $B \neq V^{\text{Orchard}}$, and let the claim hold before T . The leaves that T appends open uniquely (Lemma 9.3(i)) to notes distinct from the openings of all other leaves (Lemma 9.6(a)), and each is unconsumed when appended.

Orchard-protocol Actions of T . They create native notes only (Lemma 9.3(iii)). A note that such an Action consumes is a native note; by Lemma 9.3(i) it is not the opening of a leaf of Asset Base B . They leave U_B unchanged.

The Action Group of T and its burn set. Let R_B be its real spends witnessing B , and A_B all its Actions witnessing B . A note of Asset Base B consumed by an Action of T is consumed by an Action witnessing B , by A1'. The consumed notes of R_B are committed, by A3' with C1 and the *Ironwood Guide*'s Proposition "Membership soundness", as in the proof of Lemma 9.3; and they

were unconsumed before T and are pairwise distinct, by Lemma 9.6(b). Split Actions are not real spends and remove no note from U_B ; each contributes $v' = 0$ (Definition 9.1(c), Lemma 9.6(c)). Every Action of A_B creates one note of Asset Base B whose leaf T appends (Definition 7.4). Hence U_B changes by

$$\sum_{i \in A_B} v_i^{\text{new}} - \sum_{i \in R_B} v_i^{\text{old}} = - \sum_{i \in A_B} (v'_i - v_i^{\text{new}}) = -v_B,$$

the first equality because $v'_i = v_i^{\text{old}}$ on R_B and $v'_i = 0$ on $A_B \setminus R_B$, and the second by Theorem 9.5(i), whose premise on burn-pair bases rests on (T4) and Lemma 7.7. Rule (T4) lowers the balance of B by the same v_B . No transparent term carries B (Remark 5.3).

The Issuance Bundle of T . Each Issue Note of Asset Base B , that is of an Issuance Action whose derived base is B by (I4), enters the tree after the Action outputs (Definition 7.4) with its public value and unconsumed, and no Action of T consumes it: a consumed note of T opens a leaf of an earlier block, and by Lemma 9.6(a) that opening is not the Issue Note. Rule (I6) raises the balance of B by exactly that value; a reference note adds 0 to both sides. Issuance Actions of other Asset Bases write other entries.

Both sides therefore change equally, and the claim holds after T . The bounds follow from $U_B \geq 0$ and Proposition 7.9, and the behaviour after finalisation from Proposition 7.8. $\square$

The theorem, not the issuance state, excludes counterfeit value. ZIP 227, “Rationale for Global Issuance State”, keeps the state along the lines of ZIP 209 (“Specification”), and ZIP 227, “Global Issuance State” and “Management of the Global Issuance State”, maintain it. The state detects counterfeit value directly only when a burn exceeds the recorded balance, by rule (T4). A bundle that created unbacked value of B and did not burn it would pass every check on the map; Theorem 9.5 excludes such a bundle, and Theorem 9.7 inherits the exclusion.

9.4 Authority over issuance

Theorem 9.8 (Issuance authority). *Let an honest issuer generate isk as in Definition 2.1 and Assumption 2.3 and publish issuer = ik_encoding (Definition 2.4), and let the adversary of Definition 9.1 obtain the transactions that the issuer signs. An Asset of the honest issuer is an Asset whose Asset Identifier has that issuer. Under Assumption 2.3, Assumption 6.13, Assumption 2.9 and the Ironwood Guide’s Assumptions “Collision resistance of BLAKE2b” and “GroupHash as a random oracle”, in every valid chain that the adversary outputs, except with negligible probability:*

- (i) *every accepted Issuance Action whose Asset Base is that of an Asset of the honest issuer, and which raises its balance by rule (I6), sets its finalisation bit by rule (I7) or creates its entry with the reference note by rule (I5), belongs to a transaction whose Issuance Bundle has the effecting data, and whose transaction has the burn set and the nullifier $\text{nf}_{0,0}$, of a transaction that the honest issuer signed;*
- (ii) *the Issuance Bundle of each signed transaction is accepted at most once.*

Hence only the holder of *isk* issues or finalises the issuer's Assets (requirement Z4). Status (Definition 1.3): proved here for the specified rules; conditional on Assumption 6.13, which concerns the designed but unspecified signed message.

Proof. Reduction. Let an accepted Issuance Action change the entry at an Asset Base B of an Asset of the honest issuer, with Asset Identifier (issuer_b, b) . Rule (I4) derives B from the bundle's issuer and the action's `assetDescHash`. If $(\text{issuer}, \text{assetDescHash}) \neq (\text{issuer}_b, b)$, two distinct Asset Identifiers have the Asset Base B , which claim 2 of Proposition 2.13 excludes, under Assumption 2.9 and the *Ironwood Guide*'s Assumption "GroupHash as a random oracle", except with negligible probability. So the bundle's issuer is the honest `isk_encoding`. Rules (I2) and (I3) then require a signature valid under the honest `isk` on the transaction's `SigHash`.

If the honest issuer made no signature on that digest, the reduction, which answers the honest issuer's signing through the signing oracle of the EUF-CMA experiment, outputs the digest and the signature of the chain, a forgery against Assumption 2.3. If it made one, on a transaction T_b , then the two transactions have equal `SigHash`, and Assumption 6.13, which for the withdrawn digest rests on the *Ironwood Guide*'s Assumption "Collision resistance of BLAKE2b" (Remark 8.2), makes their Issuance Bundles' effecting data equal (issuer and, per Issuance Action, `assetDescHash`, `finalize` and every field of every Issue Note), and their burn sets and nullifiers $\text{nf}_{0,0}$ equal. Every change of the entry, the finalisation bit included, is therefore one that the issuer authorised; this is (i).

Part (ii) is Proposition 6.16, by rule (I1) and the nullifier set on $\text{nf}_{0,0}$. Burns of the issuer's Assets by note holders are outside the claim: rule (T4) only lowers the balance (ZIP 227, "Specification: Consensus Rule Changes" and "Issuance Authorization Signature Scheme"). $\square$

Remark 9.9 (Issuance key compromise). No rotation of issuance keys exists, and issuer is the encoding `isk_encoding` of the key; ZIP 227 notes that this equality "might not hold in future when key rotation is specified for issuance key pairs" ("Issuer Identifier"). The Asset Identifier of an Asset, and hence its Asset Base, stays bound to the key under which it was issued. On compromise of *isk*, ZIP 227 recommends, without a requirement keyword, setting `finalize` = 1 for every Asset issued with that key, changing to a new issuance authorising key, hence a new issuer, and issuing new Assets, each with a new Asset Identifier ("Issuance Key Compromise"). By rule (I4), finalisation bounds only the future issuance of the finalised Asset. It does not undo issuance by the attacker before the finalising transaction is accepted. An attacker that holds *isk* can issue up to `MAX_ISSUE` less the recorded balance of each unfinalised Asset, can finalise first, and can issue Assets under new descriptions, which no finalisation prevents; by consensus these are indistinguishable from the issuer's own, and ZIP 227 treats their presentation only as a matter of display ("Security and Privacy Considerations", "Displaying Asset Identifier information to users"). Theorem 9.8 assumes the secrecy of *isk* and says nothing after its compromise. Key rotation has no design in any text. The specified protocol needs it for no result of this volume, so its absence is a stated limitation, not an open problem in the sense of Definition 1.2 (ZIP 227, "Issuance Keys").

9.5 Hiding of the Asset type

The *Ironwood Guide*'s Theorem "Privacy of an Action" is confined to Ironwood-pool bundles by precondition (P2) of its Definition "Public leakage and privacy preconditions". It is not cited as proved here; the definition below has the form of that definition.

Definition 9.10 (Public leakage of an OrchardZSA bundle). The *public leakage* of an OrchardZSA bundle is its number of Actions, its anchor, its flags enableSpends, enableOutputs and enableZSA, its balancing value, and its burn set, which publishes each burnt Asset Base and amount. The rest of the transaction is public as well, including any Issuance Bundle with every field of its Issue Notes (Remark 6.6) and the carrier that holds the bundle. The *public fields* are, per Action, cv^{net} , nf, rk, cmx, epk, C_{enc} , C_{out} and the spend-authorisation signature, and, per bundle, the proof and the binding signature. The *private inputs* are, per Action, its witness (Definition 4.6, with AssetBase, is_native_asset, split_flag and ψ^{nf}), the plaintext of its created note (Definition 3.7), the outgoing viewing key under which its C_{out} is formed, or $\perp$, and whether its consumed side is a real note, a Split Input or a dummy. The *privacy preconditions* are:

- (Q1) every key involved is generated as in (P1) of the cited definition, except the all-zero spending key of a reference note copied as a Split Input, which is public (Definition 6.10);
- (Q2) honest construction: each created note has a fresh uniform rseed, and its rcm is derived from rseed by the designed Derive_rcm under an assigned lead byte (Remark 3.6); rcv is uniform; α is drawn by GenRandom; each Split Input has a fresh uniform rseed_nf (Construction 4.4); dummy consumed notes occur only for ZEC and are formed as in the protocol specification, §"Dummy Notes (Orchard)"; each Split Input copies a leaf with the Action's Asset Base whose recipient key is a key of (Q1) held by the sender, or the reference note; consumed notes are padded per Asset by the padding rule after Definition 4.2; each real consumed note is a note of a key of (Q1) that the sender accepted by the *Ironwood Guide*'s §"Trial decryption and note acceptance"; and the plaintext encoding has a length independent of the Asset;
- (Q3) authentication paths are computed from public data (the *Ironwood Guide*'s Lemma "Authentication paths from public data");
- (Q4) the adversary holds no viewing key of a key of (Q1).

Within these preconditions the adversary may choose the consumed notes, the recipients, the Assets and the values, and may know the contents of the consumed notes (ZIP 226, "Burn Mechanism", "Split Notes" and "Backward Compatibility"; protocol specification, §"Dummy Notes (Orchard)").

Theorem 9.11 (Privacy of the Asset type). Assume (Q1) to (Q4) of Definition 9.10, and let two sequences of private inputs of one OrchardZSA bundle have equal public leakage, both satisfy Definition 4.6 with the common anchor and flags, and each balance per Asset against the common burn set and balancing value. Assume Assumption 4.10 and the *Ironwood Guide*'s Assumptions "Pseudorandomness of PRF expansion", "Joint hiding of the viewing keys", "Poseidon

is a PRF”, “DDH on Pallas”, “GroupHash as a random oracle”, “The note-encryption hashes as random oracles”, “Confidentiality and integrity of ChaCha20-Poly1305”, “The RedPallas hash as a random oracle” and “Discrete logarithms on Pallas”. Then the public fields of the two bundles are computationally indistinguishable.

Precisely, every efficient adversary has negligible advantage in the experiment of the Ironwood Guide’s Theorem “Privacy of an Action”, with the following changes: the leaves that the adversary outputs are those of the tree of Definition 9.1(a); it outputs in addition the common burn set and the flag enableZSA; and a specification gives, per Action, its Asset Base, a consumed side that is a dummy (for V^{Orchard} only), a note of a key of (Q1) whose extracted commitment is a leaf, or a Split Input as in (Q2), and a created side as in that experiment. Each specification balances per Asset, the consumed notes of one key have pairwise distinct ρ , and the construction of (Q2) yields witnesses of Definition 4.6.

In particular a bundle reveals, beyond its leakage, neither which Assets its Actions move, nor how many distinct Assets, nor their values, nor which consumed sides are real, split or dummies; Remark 9.12 states what the leakage itself reveals (requirement Z5). Status (Definition 1.3): proved here; conditional on Assumption 4.10, which concerns the designed but unspecified circuit, on the unassigned lead byte for the uniformity of rcm, and on the open plaintext encoding. ZIP 226 argues the property in rationale only (“Overview”, “Privacy Implications”).

Proof. For each of the two specifications, the hybrids H_0 to H_{10} below change the probability that the adversary outputs 1 by a negligible amount (Crypto Guide, §“The hybrid argument”, Lemma “Hybrid lemma”; each hybrid is reached in polynomially many steps, one key or one Action at a time), and the two hybrids H_{10} are identically distributed. Every reduction runs the experiment with the adversary and samples every value that its hop does not concern. The keys involved are those of (Q1), the keys of the dummy consumed notes and the keys of dummy created notes.

Hybrid H_0 is the real bundle.

Hybrid H_1 (*proof*). The aggregate proof is the output of the simulator of Assumption 4.10 on the primary inputs, which are public, enableZSA included; the witnesses satisfy Definition 4.6 by hypothesis. The cost is the statistical distance of that assumption. From H_1 on, a witness enters the game only through the public fields computed from it.

Hybrid H_2 (*keys*). Each key of (Q1) and each dummy key is replaced as in game G_2 of the proof of the Ironwood Guide’s Theorem “Privacy of an Action” (its Lemmas “Rejection in key generation” and “Independence of domain-separated expansions”, and Assumption “Joint hiding of the viewing keys”), so that ask, nk, ivk, dk and ovk of each key are independent and dk and ovk uniform. The public all-zero key is left unchanged.

Hybrid H_3 (*randomisers*). Each randomiser α_i is replaced by a uniform scalar, as in game G_4 of that proof (Ironwood Guide, Lemma “Distance of GenRandom from uniform”, and Assumption “The RedPallas hash as a random oracle”). Then $\text{rsk}_i = \text{ask} + \alpha_i$ is uniform and independent of ask, the public ask of the all-zero key included, and each rk_i is a uniform point independent of ak .

Hybrid H_4 (*signatures*). Every spend-authorisation signature and the binding signature are replaced by simulated signatures under rk_i and bvk , programming the RedPallas hash, by the Crypto Guide’s Lemma “Signature simulation” (§“Security in the random oracle model and the forking lemma”). Its simulator applies to RedPallas, a Schnorr signature whose challenge is the RedPallas

hash of the nonce commitment, the validating key and the message (*Ironwood Guide*, Construction “RedPallas”); the costs are the abort probability of the lemma and the statistical distance of the reduced hash output from a uniform scalar. No signing key enters the game from H_4 on: neither rsk_i nor bsk .

Hybrid H_5 (*value commitments*). After H_4 , each trapdoor rcv_i enters the game only through cv_i^{net} . Each cv_i^{net} is replaced by an independent uniform point, by perfect hiding for every Asset Base (Lemma 3.9(i)); the key bvk is computed from these points, the public burn set and the balancing value by Definition 5.4. The hop is perfect.

Hybrid H_6 (*split nullifiers*). Each split nullifier is replaced by the x -coordinate of a uniform point, by Proposition 4.5(ii), at cost $\epsilon_{\text{expand}} + \epsilon_{\text{ToBase}} + \delta$ each. That proposition holds against an adversary that knows the copied note and nk , so the reduction samples every key itself; it holds in particular for the public key of a reference note, and for a copied note that the same bundle spends.

Hybrid H_7 (*nullifiers of consumed notes*). For each key that consumes a real or dummy note, nk enters the game after H_6 only through the nullifiers of those notes, $\text{Extract}_P([\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho) + \psi] K^{\text{Orchard}} + \text{cm}^{\text{old}})$, where cm^{old} depends on the Asset Base. As in hybrids H_4 and H_5 of the proof of the *Ironwood Guide*’s Proposition “Nullifier unlinkability” and in game G_5 , the PRF is replaced by a random function (Assumption “Poseidon is a PRF”) and each multiplier of K^{Orchard} by a uniform scalar, so each nullifier is the x -coordinate of a uniform point whatever cm^{old} is. The real consumed notes of one key have pairwise distinct ρ by the definition of the experiment, which in a valid chain holds by Lemma 9.6(a), and a dummy consumed note has a key of its own.

Hybrid H_8 (*outgoing ciphertexts*). Each C_{out} is replaced by the encryption of a fixed string under an independent uniform key, as in game G_3 (*Ironwood Guide*, Proposition “Confidentiality of the outgoing ciphertext”).

Hybrid H_9 (*ephemeral keys and note ciphertexts*). For each created note, epk is replaced by a uniform non-identity point and C_{enc} by the encryption of a fixed plaintext of the common length, keeping every note commitment, by steps (1) to (4) of the proof of the *Ironwood Guide*’s Proposition “Confidentiality and key privacy of note encryption”, as in game G_6 . The common length is the hypothesis of (Q2) on the plaintext encoding.

After H_9 , the seed rseed of each created note enters the game only through ψ and rcm , that is through cm . With rcm a PRF-expansion output of rseed under an assigned lead byte, as for either derivation of Definition 3.5, the argument of the proof of the *Ironwood Guide*’s Proposition “Hiding under the derived trapdoor” places rcm within negligible distance of a uniform scalar independent of the rest of the game.

Hybrid H_{10} (*note commitments*). Each cmx is replaced by the x -coordinate of a uniform point, by hiding with one output distribution for both branches (Proposition 3.3(a)), as in game G_7 .

In H_{10} every public field is a uniform value independent of the specification, a simulated proof or signature on public values, or a function of the leakage of Definition 9.10: the value commitments, nullifiers and extracted commitments are uniform, each epk is a uniform non-identity point, the ciphertexts encrypt fixed strings under fresh keys, each rk_i is uniform, and bvk is a function of these and of the common burn set and balancing value. The two hybrids H_{10} are therefore identically distributed, and the advantage is at most the sum of the costs for both specifications, which is negligible. $\square$

Remark 9.12 (Limits of Asset-type privacy).

- (a) *Leakage*. Each burn publishes its Asset Base and amount, “the amount and identifier of the Asset being burnt” (ZIP 226, “Rationale for Value Balance Verification”). The flag $\text{enableZSA} = 0$ shows that a bundle moves ZEC only (C2). The balancing value is an amount of ZEC. Issuance is public (Remark 6.6), although a later spend of an Issue Note is unlinkable to it (Proposition 6.4), except for a reference note.
- (b) *Action count*. Every Action has one Asset Base ($A1'$, $A2'$), consumes at most one real note and creates one note. A bundle whose Actions move k Assets, Asset j with i_j real consumed notes and o_j created notes, therefore has at least $\sum_{j=1}^k \max(i_j, o_j) \geq k$ Actions, and the public number of Actions bounds k . Theorem 9.11 hides the Assets only among bundles with equal Action counts.
- (c) *Order*. ZIP 226 fixes no Action order. Theorem 9.11 holds for every order against an adversary bound by (Q4); it claims nothing against a recipient, which by (e) is outside its scope.
- (d) *Anonymity set*. ZIP 226, “Privacy Implications”, separates Orchard and OrchardZSA note commitments by transaction version, version 5 for Orchard and version 6 for OrchardZSA (Remark 3.4); no current format assigns these versions so (Remark 8.1). The anonymity set of an Action is the set of OrchardZSA Actions of whatever carrier is specified, and an Orchard-protocol bundle reveals that its notes are ZEC. The pool that carries OrchardZSA notes is the open problem of Remark 1.7.
- (e) *Non-claims*. The theorem claims nothing against a holder of a viewing key of a key involved; nothing for the copied note of a Split Input beyond the unlinkability of its split nullifier; and nothing about timing, network metadata or information outside the transaction.

(ZIP 226, “Privacy Implications”, “Rationale for Value Balance Verification” and “Asset Base Equality”).

10 Classification of the components

This section collects the classes that the preceding sections assign. It tabulates the class of every construction of the volume under Definition 1.2 and the unspecified objects on which every result is conditional under Definition 1.3 (§10.1), and it enumerates the open problems, each with the texts that leave it open and the obligation that closes it (§10.2). No class is derived here; each is the one stated where the construction or result is made.

10.1 Classes of the constructions and results

Remark 10.1 (Reading of the table). Each row of the first part of Table 7 is a construction, or a named part of one, with the subsection that states it; the rows are grouped by their class under Definition 1.2. “Specified” means specified by Draft text: no ZIP of OrchardZSA is Final and no network upgrade includes one (item (1) of “Open problems”, §10.2). The second part lists

every theorem, proposition, lemma and corollary of the volume. A result carries no class (Definition 1.3); its row states only the unspecified objects that its hypotheses concern, and the rows are grouped by those objects. The standing assumptions of “Accepted transactions and assumptions” (§9.1) on specified primitives, namely the group hash, discrete logarithms on Pallas, BLAKE2b, RedPallas, IssueAuthSig, note encryption, Poseidon as a PRF, PRF expansion and the joint hiding of the viewing keys, are not repeated per row.

The reading of condition A8 has a group of its own. ZIP 226 leaves conditions A8 and A9 unchanged by its delta, so A8 reads v^{old} and A9 reads v^{new} , and a Split Action that copies a note of non-zero value needs `enableSpend` = 1; this is specified by the delta. The confirmation of that reading in the circuit document outside the ZIPs is designed but unspecified (Remark 4.7).

The groups of the second part read as follows. A fixed derivation of `rcm` for Issue Notes is needed only so that the validator’s commitment is computable; any deterministic derivation from the published fields suffices (Definition 1.3), and the corollary and the theorem that use Lemma 9.3 carry that qualification only through it. Theorem 9.11 needs the lead byte for `rcm` to be uniform as a pseudorandom output, and the plaintext encoding to have a length independent of the Asset; both are open, items (2) and (8). No result is conditional on the carrier otherwise than through Assumption 6.13.

Table 7. Classification of the OrchardZSA components, first part: every construction with its class under Definition 1.2 and the subsection that states it. “Specified” means specified by Draft ZIP text.

Construction	Subsection
<i>Specified (Draft text)</i>	
Issuance key pair, IssueAuthSig, issuer and signature encodings	§2.1
Asset description, Asset Identifier and its encoding	§2.2
Asset Digest, Asset Base and its byte encoding	§2.3
Rejection of an issuance whose derived Asset Base is the identity, by typing (Remark 2.12)	§2.3
Native Asset Base	§2.4
OrchardZSA note	§3.1
OrchardZSA note commitment	§3.2
Derivation of ψ	§3.3
Field set of the OrchardZSA note plaintext	§3.4
Per-Asset value commitment	§3.5
Split Inputs, Split Actions and the padding rule	§4.1
Split-nullifier formula, a sender rule that no condition of the statement enforces (Remark 4.8)	§4.2
Statement delta: conditions A1' to A5' and C1 to C3 against A1 to A9, with A8 and A9 unchanged and reading v^{old} and v^{new}	§4.3
Structure of the Action Group	§4.5
Burn set, rules (B1) to (B3), structure of the OrchardZSA bundle	§5.1
Binding validating key; its signed message excepted	§5.2
Issue Note; the rule that validators compute its commitment	§6.1
Issuance Action and Issuance Bundle	§6.2
Derivation of issued ρ values	§6.3
Reference note	§6.4
Rules (I1) to (I3); the signed message excepted	§6.5
Global issuance state and its chaining	§7.1
Transition rules (T1) to (T4) and (I4) to (I7); insertion order of note commitments	§7.2
Finalisation	§7.3
Checks of the verification procedure, each carrier tagged with its own class	§7.4
<i>Specified by the delta; realisation designed but unspecified</i>	
Reading of A8 on v^{old} : specified; its confirmation in the circuit document outside the ZIPs designed but unspecified	§4.3
<i>Designed but unspecified</i>	
OrchardZSA circuit, its verifying key and the proof length it determines	§4.4
Derivation of rcm, up to its lead byte	§3.3
Withdrawn transaction layouts, read as a composition of the constructions	§8.1
Withdrawn transaction-identifier and signature digests, and the signature message SigHash they define (the authorising-data digest excepted, Remark 8.1(ii))	§8.2, §6.5
ZSA fee terms	§8.3
Rationale arguments of the ZIPs for the properties proved in “Security”	§9
<i>Open problem (item of “Open problems”)</i>	
(1) Network upgrade; activation block of the issuance state	§7.1
(2) Value of the lead byte	§3.3
(3) Live carrier: encodings, digests, signature message, and the bit and instance position of enableZSA	§8.1, §8.2
(4) Carrying pool (Remark 1.7)	§1.3
(5) Composition of the delta with A10 and disableCrossAddress	§4.3
(6) Empty Issuance Action for a new Asset	§7.2
(7) Issue Note whose commitment is $\perp$	§6.1
(8) Encoding of the note plaintext	§3.4

Table 7. Classification of the OrchardZSA components, second part: every result of the volume, grouped by the unspecified objects that its hypotheses concern (Definition 1.3). Standing assumptions on specified primitives are omitted.

Result	Number
<i>Conditional on no unspecified object</i>	
Proposition “Uniqueness of Asset Identifiers”	2.7
Lemma “Distinctness of the OrchardZSA group-hash inputs”	2.8
Lemma “Non-identity of derived Asset Bases”	2.11
Proposition “Separation of Asset Bases”	2.13
Proposition “Hiding and binding of OrchardZSA note commitments”, stated for uniform rcm	3.3
Lemma “Hiding and binding of per-Asset value commitments”	3.9
Proposition “Unsoundness of the dummy exemption under a witnessed base”	4.1
Proposition “Independence of split nullifiers”; part (ii) for senders that follow the sender rule	4.5
Lemma “Modular per-Asset balance is integer balance”	5.5
Proposition “Unlinkability of spends of Issue Notes”	6.4
Proposition “Uniqueness of issued rho values”	6.9
Lemma “Keys of the issuance state”	7.7
Proposition “Finalised supply never grows”	7.8
Proposition “The recorded balance”	7.9
<i>Conditional on an open case</i>	
Lemma “First-issuance detection”: on item (6)	7.6
Proposition “Reference notes as universal Split Inputs”, part (a): on item (2), the lead byte	6.11
<i>Conditional on the circuit, through Assumption 4.9</i>	
Proposition “Ownership of Split Inputs”	4.12
Proposition “Per-Asset balance and key knowledge”	5.6
Proposition “Reference notes as universal Split Inputs”, part (b), on zero-valued outputs, through Proposition 5.6	6.11
<i>Conditional on the circuit, through Assumption 4.9, and, for leaves of Issue Notes, on a fixed derivation of rcm</i>	
Lemma “Provenance of Asset Bases”	9.3
Lemma “One consumption per OrchardZSA note”	9.6
Theorem “Supply integrity”	9.7
Corollary “Asset Bases of accepted Actions”, through Lemma 9.3	9.4
Theorem “No counterfeiting across Assets”, through Lemma 9.3	9.5
<i>Conditional on the signed message, through Assumption 6.13</i>	
Proposition “Replay protection of issuance”	6.16
Theorem “Issuance authority”	9.8
<i>Conditional on the circuit, through Assumption 4.10, on the lead byte and on the plaintext encoding</i>	
Theorem “Privacy of the Asset type”	9.11

10.2 Open problems

Each item below is an open problem in the sense of Definition 1.2: no text supplies a design, every existing design conflicts with live text, or live texts contradict one another. Each item names the texts that leave it open and the obligation that closes it.

- (1) *Network upgrade.* ZIP 226, “Deployment”, names NU7; ZIP 227, “Deployment”, reads “TBD”. ZIP 259 (Draft), which specifies NU7, lists its consensus changes without either ZIP (“NU7 consensus changes”) and introduces no transaction format (“Rationale for no new transaction format”); ZIP 258, which specifies NU6.3, mentions neither. Consequently the first block of OrchardZSA, whose input issuance state is the empty map (Definition 7.1), is not assigned. Closed by a deployment ZIP that includes ZIPs 226 and 227.
- (2) *Value of the lead byte of OrchardZSA notes and Issue Notes.* ZIP 226, “Note Structure and Commitment”, fixes only the placeholder `{{ZSALEADBYTE}}`; ZIP 2005, “Usage with other proposals requiring note plaintext format changes”, requires a new lead byte for plaintext changes; and the removed design’s `0x03` is the Ironwood lead byte in live text (protocol specification, §“Note Plaintexts and Memo Fields”; Remark 3.6). Consequently the commitment of an Issue Note is not computable from live text (Remark 6.2). Closed by an assignment of the value and an extension of `Derive_rcm` to it.
- (3) *Live carrier.* ZIP 226, “OrchardZSA Transaction Structure”, and ZIP 227 defer the encodings to ZIP 230, and ZIP 226, “Sighash modifications relative to ZIP 244”, and ZIP 227, “Modifications relative to ZIP 244”, defer the signature message to ZIP 246; both are Withdrawn. Version 6 is the format of ZIP 229, which carries no ZSA bundle, and ZIP 248 lists OrchardZSA and ZSA Issuance only as potential bundle types without an allocated type (Remarks 8.1 and 8.3). The flag `enableZSA` has no bit and no instance position in any current format: the withdrawn design places it at bit 2 of `flagsOrchard`, which the consensus rules of ZIP 229 reserve and require to be 0, the format table of that ZIP naming the bit `enableCrossAddress`. Closed by a live format that encodes the OrchardZSA bundle, the Issuance Bundle and `enableZSA`, with a signature digest that meets Remark 8.2.
- (4) *Carrying pool.* ZIP 226, “Backward Compatibility”, places OrchardZSA notes in the Orchard pool, which ZIP 258, “Consensus rules from NU6.3 activation”, closes to new value and to cross-address transfers from NU6.3 (Remark 1.7). The rules of that pool then forbid transfers of Custom Assets between distinct addresses, leave issuance into it unaddressed, and leave undetermined the tree that Definition 7.4 extends. Closed by a reconciliation of ZIP 226 with ZIP 258 that names the pool and its rules.
- (5) *Composition of the delta with condition A10.* ZIP 226, “Circuit Statement”, states its delta against A1 to A9; the current statement adds condition A10 and the primary input `disableCrossAddress` (*Ironwood Guide*, Definition “Orchard Action statement”; Remark 4.7). Closed by a statement text that includes both, which depends on the resolution of item (4).
- (6) *Empty Issuance Action for a new Asset.* For an Issuance Action without Issue Notes on an Asset Base without a reference note, rule (I5) refers to a first Issue Note that does not exist,

so its validity, finalising or not, is undetermined (ZIP 227, “Specification: Consensus Rule Changes”; Remark 7.5). Lemma 7.6 is conditional on this case. Closed by a rule that rejects such an action or fixes its effect on the issuance state.

- (7) *Issue Note whose commitment is $\perp$* . The protocol specification, §“Sending Notes (Orchard)”, makes a sent note whose commitment is $\perp$ invalid and resamples `rseed`, which covers sent notes only; ZIP 227, “Issue Note”, states no rule (Remark “Commitment $\perp$ for an Issue Note” in §6.1). The event has negligible probability. Closed by a rule: resampling by the issuer or rejection by validators.
- (8) *Encoding of the OrchardZSA note plaintext*. The field set is specified (Definition 3.7); ZIP 226, “Note Structure and Commitment”, delegates the encoding to ZIP 230, which is Withdrawn. Theorem 9.11 is conditional on it. Closed by a live encoding whose length does not depend on the Asset.

Remark 10.2 (Items not open). Four items are not open problems. The bound on the number of Actions that Lemma 5.5 needs, at most $62\,500 < 2^{16}$ Actions per transaction, follows from the block-size rule (protocol specification, §“Block Header Encoding and Consensus”) whatever the carrier. The ZSA fee terms (Remark 8.4), the derivation of `rcm` (Remark 3.6) and the OrchardZSA proof length (Remark 4.11) are designed but unspecified, as the first part of Table 7 records: each has a design that no live text contradicts. ZIP 227 and ZIP 317 disagree about the fee terms, which the protocol does not need (Remark 8.4).

IX

FROST Guide

Threshold RedPallas: FROST and its Zcash integration surfaces

Abstract

This volume of *The Zcash Arboretum* constructs threshold spend authorisation for Orchard-protocol notes: FROST, the two-round threshold Schnorr protocol of RFC 9591, with key generation by a trusted dealer and by a distributed protocol, and its re-randomised variant specified in ZIP 312 over the ciphersuite FROST(Pallas, BLAKE2b-512), whose aggregate is an ordinary RedPallas spend-authorisation signature. Any threshold number of participants signs, and no signing run reconstructs the signing key. The volume proves correctness of each protocol and states its security with the assumptions and the status of each result, marking open problems as such. It assumes the *Math Guide* and the *Crypto Guide*, and cites the *Ironwood Guide* for RedPallas, randomised validating keys, the signature digest and the construction of a transaction. It is non-normative: the protocol specification, the ZIPs and RFC 9591 are authoritative.

1 Introduction

This section fixes the object of the volume and its scope, the standing of the normative and scholarly sources with the classes into which its claims fall, and the notation shared with the lower volumes.

1.1 Scope

This volume of *The Zcash Arboretum* constructs threshold spend authorisation for Orchard-protocol notes, for Actions of either pool of the Orchard protocol (*Ironwood Guide*, §“The Orchard protocol and its two pools”, Definitions “Orchard protocol” and “Orchard pool, Ironwood pool”). Any threshold number of the participants, the threshold and the number of participants being those of Definition 2.1, jointly produce, for an Action spending a note of a threshold key, a signature that consensus accepts as an ordinary RedPallas spend-authorisation signature, with the same format and the same validation as a signature by a single holder of the spend authorising key ask. No signing run reconstructs the signing key; a trusted dealer computes it at key generation; no step of distributed key generation computes it, and no source cited in

this volume proves that protocol secure (Remark 3.9). RedPallas and spend authorisation are cited, not re-derived: *Ironwood Guide*, §“The RedPallas signature scheme” and §“Randomised validating keys”.

The construction has two layers. The first is FROST, the two-round threshold Schnorr protocol of Komlo and Goldberg as RFC 9591 specifies it, run on a Shamir sharing of ask. The second is its re-randomised variant (ePrint 2024/436), specified for Zcash in ZIP 312, which shifts the shares, the verification shares and the group key of each signing run by one randomiser so that the aggregate verifies under the Action’s randomised validating key; it is defined in §5.2 and constructed in §5.5 (Construction 5.10) with the randomiser of §5.3, and its correctness is Theorem 5.11.

The Orchard pool is spend-only: no transaction increases its value, and every Orchard-pool Action creates its note at the address of the note it consumes (ZIP 258, Draft, “Consensus rules from NU6.3 activation”; *Ironwood Guide*, §“The Orchard protocol and its two pools”), so no value enters the pool and, by consensus, only a party able to authorise spends for an address can create an Orchard-pool note to it. ZIP 326 (Draft) forbids conforming wallets to send Orchard-pool funds to a key derived with `use_qsk = true` (ZIP 2005), which every key from distributed key generation must use, so under that rule such a key holds no Orchard-pool notes, a rule that consensus does not enforce, while a dealer-shared key whose ask is derived from a spending key may spend notes of either pool; both facts are established, with their sources, in §6.4.

The volume constructs FROST as RFC 9591 specifies it; key generation by a trusted dealer and by a distributed protocol; the re-randomised variant for RedPallas that ZIP 312 specifies, with its ciphersuite FROST(Pallas, BLAKE2b-512); the proof that its output is an ordinary spend-authorisation signature; the key-generation constraints of ZIP 2005; and the security of each part, with its assumptions and status. ZIP 312 also defines the ciphersuite FROST(Jubjub, BLAKE2b-512) for Sapling spend authorisation by RedJubjub; it is named here and not developed. Transparent inputs, transports and particular tools are outside the scope. The volume assumes the *Math Guide* and the *Crypto Guide*, and cites the *Ironwood Guide* for RedPallas, randomised validating keys, the signature digest and the construction of a transaction. It is non-normative: the protocol specification, the ZIPs and RFC 9591 are authoritative.

The volume describes the protocol as specified for NU7 (*Ironwood Guide*, §“Scope and status”, Definition “Network upgrade”), whose deployment ZIP 259 has status Draft; NU7 changes neither spend authorisation nor the transaction formats on which the volume relies, version-5 and version-6 transactions remaining valid (ZIP 259, Abstract). NU6.3 is active, and its rules, including those of ZIP 2005 that ZIP 258 deploys, are part of the protocol so described.

1.2 Sources and their standing

The *Ironwood Guide*, §“Scope and status”, Definition “ZIP”, names the ZIP statuses Draft and Reserved. A ZIP has status *Proposed* when, after Draft, it has been advanced upon consideration, feedback and rough consensus of the community (ZIP 0, “ZIP Status Field”). A ZIP of status Draft, Proposed, Active, Implemented or Final carries normative text; a Reserved, Withdrawn, Rejected or Obsolete ZIP carries none.

Definition 1.1 (Classes of claims).

- (i) A claim about the deployed protocol is placed in one of the two classes of the *Ironwood Guide*, §“Scope and status”, Definition “Epistemic classes”, with their meaning unchanged. It is *specified* when normative text of the protocol specification or of a ZIP fixes it. It is *designed but unspecified* when the deployed construction is designed to satisfy it but neither the protocol specification, nor a ZIP, nor a published proof establishes it for the deployed instance; knowledge soundness of the Action proof is that volume’s example.
- (ii) *Normative text* comprises the protocol specification, RFC 9591, an Informational RFC of the IRTF stream, and every ZIP of status Draft, Proposed, Active, Implemented or Final.
- (iii) A claim about a construction outside the deployed protocol, namely FROST, its key generation and its re-randomised variant, to which no consensus rule refers, is *specified* when normative text fixes it, *established* when a published proof establishes it, stated with that proof’s assumptions and the venue status of its source (§1.2), *proved here* when this volume proves it, under the hypotheses it states, and an *open problem* when neither normative text, nor a published proof, nor this volume establishes it.
- (iv) A construction itself is *specified* when normative text fixes it; *designed but unspecified* when a published design, or this volume, fixes it and no normative text does, any part first stated here, such as Construction 6.7, being marked as the volume’s own; and an *open problem* when nothing fixes it.

Every design-stage claim carries its class where it is stated; Table 3 collects them.

The sources of the volume, each with its standing, are the following.

- (a) D. Connolly, C. Komlo, I. Goldberg and C. A. Wood, “The Flexible Round-Optimized Schnorr Threshold (FROST) Protocol for Two-Round Schnorr Signatures”, RFC 9591, June 2024, a product of the IRTF Crypto Forum Research Group in the category Informational, not an Internet Standards Track specification, specifies two-round FROST signing over any prime-order group and hash, with ciphersuites for none of the Zcash groups, leaves key generation out of scope, gives a trusted dealer for completeness in its Appendix C, and is normative text in the sense of Definition 1.1.
- (b) ZIP 312, “FROST for Spend Authorization Multisignatures”, status Draft, category Wallet, specifies the re-randomised variant for Zcash as changes to RFC 9591 (randomiser generation, Round Two, and re-randomised signing, aggregation and share verification) together with the ciphersuites FROST(Jubjub, BLAKE2b-512) and FROST(Pallas, BLAKE2b-512), names ask as the key to be shared, requires key generation to be consistent with FROST without specifying one, and specifies no consensus rule.
- (c) ZIP 2005, “Ironwood Quantum Recoverability”, status Proposed, category Consensus, whose key rules take effect with NU6.3, which ZIP 258 (Draft) deploys, and stand in the

protocol as specified for NU7, is used for its constraints on FROST key generation and the keys they produce: “Usage with FROST”, its replacement text for § 4.2.3 “Orchard Key Components” (“Changes to the Protocol Specification”), and “Deployment”.

- (d) C. P. L. Gouvêa and C. Komlo, “Re-Randomized FROST”, IACR ePrint 2024/436, an unrefereed preprint with no peer-reviewed venue, defines re-randomisable threshold signatures and their unforgeability game and claims unforgeability of Rerandomized FROST in the random-oracle model under the algebraic one-more discrete logarithm assumption, with a proof that §7.4 finds incomplete (Remark 7.17), so that the claim is an open problem (Table 3); a result that rests on it alone is stated as the preprint’s, not as established.
- (e) C. Komlo and I. Goldberg, “FROST: Flexible Round-Optimized Schnorr Threshold Signatures”, IACR ePrint 2020/852, published at SAC 2020 (LNCS 12804, pp. 34–65) and cited by RFC 9591 as [FROST20], gives the original scheme, its distributed key generation, in which every participant acts as a Feldman dealer and proves knowledge of its constant coefficient by a Schnorr proof (its Figure 1), and the original security argument.
- (f) M. Bellare, S. Tessaro and C. Zhu, “Stronger Security for Non-Interactive Threshold Signatures: BLS and FROST”, IACR ePrint 2022/833, cited in its current revision, has a preliminary version that is part of M. Bellare, E. Crites, C. Komlo, M. Maller, S. Tessaro and C. Zhu, “Better than Advertised Security for Non-Interactive Threshold Signatures”, CRYPTO 2022, which RFC 9591 cites as [StrongerSec22]; each work is cited by its own title and authors.
- (g) E. Crites, C. Komlo and M. Maller, “How to Prove Schnorr Assuming Schnorr: Security of Multi- and Threshold Signatures”, IACR ePrint 2021/1375, which defines FROST2 and analyses it with its own key generation, is cited only for FROST2 in §7.2 and for the nearest result recorded in Remark 7.13.
- (h) The revision of ZIP 312 proposed in zcash/zips pull request 895, linked from ZIP 2005, “Deployment”, is unmerged and not normative text, and is cited only as a published design.

1.3 Notation

Notation is additive throughout; the multiplicative forms of the papers are converted at citation. The generic sections work in a group $\mathbb{G}$ of prime order q with generator B and identity $\mathcal{O}$, whose discrete logarithm problem and random oracles are those of the *Crypto Guide*, §“The discrete logarithm problem”, Definition “Discrete logarithm problem, DLP”, and §“The random oracle model”. The Pallas instance sets

$$\mathbb{G} = \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}), \quad q = p_{\text{Vesta}}, \quad B = G^{\text{SpendAuth}}$$

(*Ironwood Guide*, §“Fields, groups, and encodings”, and §“The spending key and the spend-side secrets”, Construction “Spend validating key and sign normalisation”). Scalar multiplication is written $[k]P$, and scalars are elements of $\mathbb{F}_q$. The star encoding P^* , the encodings LE_k and l2LEOSP_k , concatenation $\parallel$ and monospace byte-string literals are those of the *Ironwood Guide*, §“Fields,

Table 1. Notation of the sources. Each row names an object in this volume, in RFC 9591, in ZIP 312 and in the multiplicative notation of the papers (ePrints 2020/852, 2022/833 and 2024/436). The names r_p , repr_p , abst_p and $\mathcal{G}^{\text{Orchard}}$ are those of the protocol specification, which ZIP 312 uses, and which §6 also writes where it follows the protocol specification.

Object	This volume	RFC 9591	ZIP 312	Papers
Group order	q	$p, G.\text{Order}()$	$G.\text{Order}()$; r_p for Pallas	q (2020/852); p (2022/833, 2024/436)
Generator	$B; G^{\text{SpendAuth}}$ for Pallas	B	$\mathcal{G}^{\text{Orchard}}$	g
Scalar base multiplication	$[k]B$	$\text{ScalarBaseMult}(k)$	$G.\text{ScalarBaseMult}(k)$	g^k
Scalar multiplication	$[k]A$	$\text{ScalarMult}(A, k)$	as RFC 9591	A^k
Element encoding	$\text{ser}_G, \text{deser}_G$; on Pallas LEBS2OSP ₂₅₆ (P^*) and abst_p , failing on $\perp$ and on $\mathcal{O}$ (Construction 6.1)	$\text{SerializeElement}, \text{DeserializeElement}$	$\text{repr}_p, \text{abst}_p$	—
Scalar encoding	$\text{ser}_{F_q}, \text{deser}_{F_q}$; on Pallas I2LEOSP ₂₅₆	$\text{SerializeScalar}, \text{DeserializeScalar}$	32-byte little-endian	—
Threshold, number of participants	t, n	$\text{MIN_PARTICIPANTS}, \text{MAX_PARTICIPANTS}$	as RFC 9591	t, n
Randomiser	effective randomiser $\hat{\alpha}$, the scalar that shifts the key material; in an accepted Action the Action’s randomiser α (Corollary 6.4)	—	randomizer	$\hat{\alpha} = H_r(\alpha, \text{aux})$ (2024/436)
Hash roles	$H_1, \dots, H_5, H_R$	$H1, \dots, H5$	$H1, \dots, H5, HR$	—

groups, and encodings”. The element and scalar serialisations $\text{ser}_G, \text{ser}_{F_q}$ and their inverses $\text{deser}_G, \text{deser}_{F_q}$ are components of a ciphersuite (Definition 4.1).

The spend-side symbols follow the *Ironwood Guide*, §“The spending key and the spend-side secrets”: ask is the spend authorising key, $\text{ak}^P = [\text{ask}] G^{\text{SpendAuth}}$ the spend validating key as a point, $\text{ak} = \text{Extract}_P(\text{ak}^P)$ its extracted coordinate, and nk, rsk and rk are as there. ZIP 2005 uses the same notation.

A signing set is written I and its Lagrange coefficients $\lambda_{I,i}$ (Definition 2.3), the symbols of the *Crypto Guide*, §“Secret sharing”, in Construction “Shamir t -of- n secret sharing” and in Remark “Leakage of Feldman sharing”, which names this volume as the consumer of $\lambda_{I,i}$ -weighted recombination. The generator G of that Guide’s Construction “Feldman verifiable secret sharing” is B here. The group key and verification shares are PK and PK_i , the secret shares sk_i (Definition 2.5). Query counts of cited bounds are written Q , or q_s and q_b with subscripts, never a bare q , which is the group order. The British spelling *randomiser* replaces ZIP 312’s *randomizer*; titles of ZIPs and papers keep their own spelling. Table 1 maps the names of the sources to those of the volume; later text uses the latter, and §6 also the protocol specification’s names where it follows that text.

2 Threshold Schnorr signatures

This section fixes the setting that every later protocol assumes: the participants, their identifiers, the signing sets and their Lagrange coefficients; the threshold key material and the group information that coefficient commitments determine; the syntax and correctness of a two-round threshold Schnorr signature scheme with a Coordinator; and the channel types and the static corruption model. Notation is that of §1.3.

2.1 Participants, identifiers and signing sets

Definition 2.1 (Threshold parameters and signing sets). Let G be a group of prime order q with generator B and scalar field $\mathbb{F}_q$ (§1.3). *Threshold parameters* are integers t, n with

$$1 \leq t \leq n < q.$$

There are n participants $P_1, \dots, P_n$. The *identifier* of P_i is $i \in \{1, \dots, n\}$, read as an element of $\mathbb{F}_q$; identifiers are distinct and nonzero in $\mathbb{F}_q$ because $n < q$. A *signing set* is a set $I \subseteq \{1, \dots, n\}$ with $t \leq |I| \leq n$. In the notation of RFC 9591 (Table 1), t is MIN_PARTICIPANTS, n is MAX_PARTICIPANTS and $|I|$ is NUM_PARTICIPANTS.

The constraints are those of RFC 9591, Section 5 (“Two-Round FROST Signing Protocol”): the identifiers are distinct nonzero scalars in $[1, \text{MAX_PARTICIPANTS}]$; the bound MAX_PARTICIPANTS is less than the group order; and

$$\text{MIN_PARTICIPANTS} \leq \text{NUM_PARTICIPANTS} \leq \text{MAX_PARTICIPANTS}.$$

The dealer of RFC 9591, Appendix C (“Trusted Dealer Key Generation”), shares at the identifiers $1, \dots, n$. Every statement below that uses only interpolation holds verbatim for any distinct nonzero identifiers; the identifiers $1, \dots, n$ are the ones RFC 9591 requires.

Remark 2.2 (Shamir sharing). Shamir sharing is cited, not re-derived: *Crypto Guide*, §“Secret sharing”, Construction “Shamir t -of- n secret sharing”. Its parameters $1 \leq t \leq n < q$ and identifiers $1, \dots, n$ are those of Definition 2.1; its share symbol s_i is this volume’s sk_i , and the generator G of the *Crypto Guide*’s Construction “Feldman verifiable secret sharing” is B . Any t shares determine the sharing polynomial and its constant term (Lemma 2.4 gives this for every signing set). Fewer than t shares are jointly uniform whatever the secret (*Crypto Guide*, Theorem “Perfect secrecy below the threshold”). That theorem concerns the shares alone. For key material shared by a dealer, once the coefficient commitments of §2.2 are published, the point $[s]B$ is public and secrecy below the threshold is computational: the view of $t - 1$ participants is computable from $[s]B$ and $t - 1$ uniform scalars. Since $[s]B$ is the group key, which is published in any case, nothing further escapes (*Crypto Guide*, Remark “Leakage of Feldman sharing”). Distributed key generation is treated in §7.3.

Definition 2.3 (Lagrange coefficients of a signing set). Let I be a signing set and $i \in I$. The *Lagrange coefficient* of i in I is

$$\lambda_{I,i} := \prod_{j \in I, j \neq i} \frac{j}{j-i} \in \mathbb{F}_q,$$

whose denominators are nonzero because identifiers are distinct. For the Lagrange basis polynomial $\ell_i(X) = \prod_{j \in I, j \neq i} (X - j)/(i - j)$ of the nodes I (*Math Guide*, Theorem “Lagrange interpolation”), $\lambda_{I,i} = \ell_i(0)$, since $(0 - j)/(i - j) = j/(j - i)$; it is the reconstruction weight of the *Crypto Guide*’s Construction “Shamir t -of- n secret sharing”. The specified computation takes the list of identifiers of I and the identifier i , and fails with the error “invalid parameters” if i is absent from the list or some identifier occurs in it more than once (RFC 9591, Section 4.2, “Polynomials”).

Lemma 2.4 (Recombination over a signing set). Let I be a signing set. For every $f \in \mathbb{F}_q[X]$ with $\deg f \leq t - 1$,

$$\sum_{i \in I} \lambda_{I,i} f(i) = f(0).$$

In particular $\sum_{i \in I} \lambda_{I,i} = 1$. The statement and its proof hold verbatim for any finite set I of distinct nonzero elements of $\mathbb{F}_q$ with $|I| \geq \deg f + 1$, the coefficients being defined by the same product.

Proof. Put $d := |I| - 1$, so $d \geq t - 1 \geq \deg f$. By *Math Guide*, Theorem “Lagrange interpolation”, with nodes I and targets $f(i)$, the polynomial $g := \sum_{i \in I} f(i) \ell_i$ has degree at most d and satisfies $g(i) = f(i)$ for every $i \in I$, that is at $d + 1$ distinct points. The polynomial f also has degree at most d , so $g = f$ by *Math Guide*, Corollary “Few points determine a low-degree polynomial”. Evaluating at 0 and using $\ell_i(0) = \lambda_{I,i}$ (Definition 2.3) gives the identity. The constant polynomial $f = 1$ has degree $0 \leq t - 1$, because $t \geq 1$, and gives the sum one. $\square$

The proof is given in place: the *Math Guide*’s Proposition “Lagrange polynomials sum to one” concerns its roots-of-unity domain and does not apply to the nodes I . ZIP 312, “Rationale”, uses the sum $\sum_{i \in I} \lambda_{I,i} = 1$ and refers to an external note for its proof; ePrint 2024/436, Section 5, justifies it in one line as the interpolation of the constant polynomial 1.

The coefficients $\lambda_{I,i}$ depend on the identifiers of I alone, not on the shares or on the polynomial (Definition 2.3). Hence the signing set is chosen per signing run, after key generation, by the Coordinator of Definition 2.8, which determines the participants, at least MIN_PARTICIPANTS in number (RFC 9591, Section 5).

2.2 Key material and group information

Definition 2.5 (Threshold key material). Let (t, n) be threshold parameters. *Threshold key material* consists of:

1. a *sharing polynomial* $f(X) = \sum_{k=0}^{t-1} a_k X^k \in \mathbb{F}_q[X]$ with $f(0) = a_0 = s$, the *signing key*;

2. for each identifier j , the *secret share* (j, sk_j) with $sk_j = f(j)$, held by P_j alone;
3. the *coefficient commitments* $A_k := [a_k]B$ for $k = 0, \dots, t-1$;
4. the *group information* $(PK, (PK_j)_{j=1}^n)$, with *group key* $PK := [s]B$ and *verification shares* $PK_j := [sk_j]B$, held by every party, the Coordinator included.

Shares $(sk_j)_{j=1}^n \in \mathbb{F}_q^n$ and group information $(PK, (PK_j)_{j=1}^n) \in \mathbb{G}^{n+1}$ are *valid key material* if some $f \in \mathbb{F}_q[X]$ with $\deg f \leq t-1$ has $f(j) = sk_j$ for every $j \in \{1, \dots, n\}$, $PK = [f(0)]B$ and $PK_j = [sk_j]B$ for every j . Such f is unique, since $n \geq t$ (*Math Guide*, Corollary “Few points determine a low-degree polynomial”). No party is given s or f ; the definition names them. Valid key material is the output of both key generations (§3) and the input of signing.

The components are those of RFC 9591, Section 5: each participant is configured with its identifier, the share $sk_i = f(i)$ and $PK_i = [sk_i]B$; the group information consists of PK and PK_i for each i in $[1, \text{MAX_PARTICIPANTS}]$.

Lemma 2.6 (Group information from coefficient commitments). *Let $f = \sum_{k=0}^{t-1} a_k X^k$ and $A_k = [a_k]B$.*

(i) *The group key is $PK = [f(0)]B = A_0$, and for every identifier j*

$$PK_j = [f(j)]B = \sum_{k=0}^{t-1} [j^k]A_k.$$

Every party holding $(A_0, \dots, A_{t-1})$ therefore computes the group information from the commitments alone.

(ii) *Let $(A_0, \dots, A_{t-1}) \in \mathbb{G}^t$, however chosen, and let $sk_1, \dots, sk_n \in \mathbb{F}_q$ satisfy $[sk_j]B = \sum_{k=0}^{t-1} [j^k]A_k$ for every $j \in \{1, \dots, n\}$. Then $(sk_j)_{j=1}^n$, with the group information computed by (i), is valid key material (Definition 2.5) for the unique polynomial $f^* = \sum_k a_k^* X^k$ with $[a_k^*]B = A_k$.*

Proof. Part (i) is the identity displayed in *Crypto Guide*, Construction “Feldman verifiable secret sharing”, where it follows from the homomorphism $a \mapsto [a]B$; that construction’s G and Φ_k are this volume’s B and A_k . For (ii), by *Crypto Guide*, Proposition “What the Feldman check certifies”, the vector $(A_0, \dots, A_{t-1})$ determines a unique polynomial f^* of degree at most $t-1$, and each passing check gives $sk_j = f^*(j)$. Then $A_0 = [f^*(0)]B$ and $\sum_k [j^k]A_k = [sk_j]B$, which are the conditions of Definition 2.5. $\square$

RFC 9591, Appendix C.2 (“Verifiable Secret Sharing”), derives the group information from the commitment vector in this way. Both key generations of §3 cite the lemma.

2.3 Two-round threshold signature schemes

Remark 2.7 (Prerequisites). The syntax of a signature scheme, existential unforgeability under chosen-message attack and strong unforgeability are those of *Crypto Guide*, §“Syntax and security goal” (Definitions “Digital signature scheme”, “Existential unforgeability under chosen-message attack” and “Strong unforgeability”), which admits correctness with overwhelming probability. The single-party Schnorr signature is *Crypto Guide*, §“From identification to signature via the Fiat–Shamir transform”, Construction “Schnorr signature”, perfectly correct by its Proposition “Perfect correctness”. In additive notation with generator B , signing key s and public key $PK = [s]B$, a signature on a message m is

$$r \xleftarrow{\$} \mathbb{F}_q, \quad R = [r]B, \quad c = H(R \parallel PK \parallel m), \quad z = r + cs, \quad \sigma = (R, z),$$

and verification accepts (R, z) if and only if

$$[z]B = R + [c] PK.$$

The *Crypto Guide*’s response s is written z here, s being the signing key. The challenge input order (nonce commitment, public key, message) is that of RFC 9591, Section 4.6 (“Signature Challenge Computation”), and of RedPallas; the hash and the encodings of FROST are fixed by the ciphersuite (Definition 4.1). Discrete logarithms and random oracles are those of the *Crypto Guide*, §“The discrete logarithm problem” and §“The random oracle model”, as §1.3 records.

Definition 2.8 (Two-round threshold signature scheme with a Coordinator). Let (t, n) be threshold parameters (Definition 2.1). A *two-round threshold signature scheme with a Coordinator* is run by the participants $P_1, \dots, P_n$ and one *Coordinator*: a role, which a participant may also play, that uses no share of the signing key and no nonce, and that chooses the signing set, relays messages, aggregates and publishes the signature. It consists of:

1. *Key generation*, a dealer algorithm or a protocol, which outputs valid key material (Definition 2.5): sk_j to P_j , and the group information to every party.
2. *Round One*, run by each P_i independently of any message and of any signing set, which outputs a public *commitment* com_i , sent to the Coordinator, and a secret *state* st_i , which P_i keeps and uses in at most one Round Two.
3. *Round Two*. The Coordinator chooses a signing set I and a message m , and sends the request $(m, (com_j)_{j \in I})$ to each P_i with $i \in I$. Participant P_i either returns a *signature share* z_i , computed from sk_i, st_i , the request and the group information, or aborts; in both cases it deletes st_i .
4. *Aggregation*, by which the Coordinator maps m , the commitment list $(com_j)_{j \in I}$ and the shares $(z_j)_{j \in I}$ to a signature σ or to $\perp$.
5. *Verification*, the single-party Schnorr verification of a signature on m under the group key PK .

The scheme is *correct* if, for all valid key material output by key generation, every signing set I and every message m , a run in which every party is honest outputs either $\perp$ or a signature that verification accepts under PK , and outputs $\perp$ with probability negligible in the security parameter.

The parties and the rounds are those of RFC 9591, Section 5: the Coordinator determines the participants, coordinates the rounds, aggregates the signature shares and publishes the signature; Round One produces commitments; Round Two produces a signature share over the message and the commitments that the Coordinator supplies. Theorem 4.11 bounds the probability of $\perp$ for FROST. Unforgeability notions are defined in §7, before any use.

Remark 2.9 (Scope of the syntax). The two-round form is the one RFC 9591 specifies (Section 5). Single-round signing with commitments preprocessed in batches (ePrint 2020/852) is out of the scope of RFC 9591 (Section 1, “Introduction”) and of this volume.

The parties compute (R, z) jointly, and no step computes s or r . Round Two binds each share to the message and the whole commitment list (Construction 4.8); unforgeability under concurrent runs is Theorem 7.9, under its assumptions (Remark 7.10).

2.4 Channels and corruption

Definition 2.10 (Channels). Every message of every protocol in this volume travels on a channel of one of the following four types, and the protocol names the type.

- (a) An *authenticated channel with reliable delivery*: the receiver learns the sender and the unmodified message; delivery is assured, as completion of the protocol requires; the content is visible to an observer of the network. It carries the signing messages: the Round One commitments, the Round Two request and the returned shares.
- (b) A *confidential and authenticated channel*: a channel of type (a) whose content is hidden from every party but its endpoints. It carries the Round Two request of the re-randomised variant with its confidential inputs, named in Construction 5.10.
- (c) A *mutually authenticated confidential channel*: a channel of type (b) in which each endpoint is authenticated to the other. It carries dealt shares and the evaluations of round two of distributed key generation.
- (d) *Consistent broadcast*: every honest recipient outputs the same value from the sender, whether or not the sender is honest, or every honest recipient aborts. If the sender is honest, an honest recipient that does not abort outputs the value sent. It carries the co-efficient commitments and the proofs of knowledge of key generation.

Realisations are not described. For what authentication provides against an adversary that controls the network, see *Crypto Guide*, §“Active attacks and the role of authentication”.

Each type is tied to its source. Type (a) is that of RFC 9591, Section 5, which assumes reliable delivery for completion and an authenticated channel, confidentiality not being required; Section 7

(“Security Considerations”) adds that signing may run over a public channel provided it is authenticated and reliable. Authentication is what identifies a misbehaving participant: without it, a party masquerading as another causes only an invalid signature (RFC 9591, Section 5). Type (b) is that of ZIP 312, “Round Two - Signature Share Generation”, which requires a confidential and authenticated channel and records the difference from RFC 9591. Type (c) is that of RFC 9591, Appendix C, which requires a mutually authenticated secure channel with confidentiality and integrity for dealt shares, and of ePrint 2020/852, Figure 1, in which round two of key generation sends the evaluations securely. Type (d) is required by RFC 9591, Appendix C, under which participants must abort if they do not hold the same view of the commitment vector, and by ePrint 2020/852, Figure 1, which broadcasts commitments and proofs.

Definition 2.11 (Static corruption). Before key generation the adversary fixes a set $C \subseteq \{1, \dots, n\}$ with $|C| \leq t - 1$. It receives the entire state of each P_j with $j \in C$ throughout, and chooses their messages. It may in addition control the Coordinator, which holds no share of the signing key and no nonce; in the re-randomised variant it also holds the randomiser and the transaction data (Definition 5.14). It observes and schedules the network, subject to the guarantees of Definition 2.10. The set C never changes, and every other participant follows the protocol.

A malicious Coordinator can deny service, by withholding requests, the aggregate or its publication, and can falsely report a participant as misbehaving; no honest party can prevent either, since FROST provides no robustness (RFC 9591, Section 7; ePrint 2020/852, Section 5, “Signature Aggregator Role”). What a malicious Coordinator cannot do is stated in §7, either as a theorem (Theorem 7.9, Theorem 7.20) or as an open problem (Remark 7.22); Definition 2.11 asserts none of it. The re-randomised variant further trusts the Coordinator and every share holder with the privacy of the transaction (Definition 5.14). Adaptive corruption, metadata privacy and network privacy are out of scope. The model is that of RFC 9591, Section 7, under which the Coordinator and at most $\text{MIN_PARTICIPANTS} - 1$ participants may be corrupted and robustness and metadata protection are not goals, and of ZIP 312, “Threat Model”.

3 Key generation

This section states the status of key generation in RFC 9591 and ZIP 312 and the two protocols that output the threshold key material of Definition 2.5: key generation by a trusted dealer and distributed key generation. Each is proved to output valid key material and is classified by Definition 1.1.

3.1 Trusted-dealer key generation

RFC 9591 places key generation out of its scope and, for completeness, specifies key generation by a trusted dealer in Appendix C (“Trusted Dealer Key Generation”); Section 1 (“Introduction”) says so. Its Section 5 (“Two-Round FROST Signing Protocol”) names the two mechanisms by which the key material of Definition 2.5 may be configured, a trusted dealer and a distributed key generation protocol, and specifies neither beyond that appendix. ZIP 312 specifies no key

generation (“Non-requirements”). Its “Key Generation” requires key generation to be consistent with FROST, referring to the dealer appendix of RFC 9591 for guidance, and names the spend authorising key ask as the key to be shared. It records that ask is usually derived from the spending key sk but need not be; that not deriving it permits distributed key generation, since the key so generated is unpredictable; and that not deriving it prevents recovery of the secret from a seed phrase, which the ZIP notes may be desirable for FROST. The security analysis of Bellare, Tessaro and Zhu (ePrint 2022/833, in its current revision, whose preliminary version is part of the analysis RFC 9591 cites) models key generation as an algorithm run by a trusted party. Two protocols output key material: the dealer of RFC 9591, Appendix C, in this subsection, and the distributed key generation of ePrint 2020/852, Figure 1, in §3.2.

Assumption 3.1 (Honest dealer). The dealer runs Construction 3.2 as written:

1. it samples s and the coefficients $a_1, \dots, a_{t-1}$ uniformly and independently from its own randomness;
2. it reveals s , the coefficients and the shares only as the construction prescribes, the share of P_j to P_j alone;
3. it deletes s , the coefficients and the shares once the shares are distributed.

The adversary of Definition 2.11 does not control the dealer and never obtains its state.

The three clauses are the trust requirements of RFC 9591, Appendix C: the dealer is trusted to generate good randomness, to delete secret values after distributing the shares, and to keep secret values confidential; the appendix further requires that the dealer delete the secret key and the secret key shares upon completion.

Construction 3.2 (Dealer sharing of a threshold key). Let (t, n) be threshold parameters with identifiers $1, \dots, n$ (Definition 2.1), over the group G of prime order q with generator B and scalar field $\mathbb{F}_q$. The dealer proceeds as follows.

1. It samples the signing key $s \xleftarrow{\$} \mathbb{F}_q$ uniformly and $a_1, \dots, a_{t-1} \xleftarrow{\$} \mathbb{F}_q$ independently and uniformly.
2. It sets $a_0 := s$ and

$$f(X) := \sum_{k=0}^{t-1} a_k X^k \in \mathbb{F}_q[X],$$

a sharing of s by *Crypto Guide*, Construction “Shamir t -of- n secret sharing”.

3. For $j = 1, \dots, n$ it sends the secret share (j, sk_j) , with $sk_j := f(j)$, to P_j over a mutually authenticated confidential channel (Definition 2.10, type (c)).
4. It sends the coefficient commitments

$$A_k := [a_k]B, \quad k = 0, \dots, t-1,$$

to every party, the participants and the Coordinator, by consistent broadcast (Definition 2.10, type (d)).

5. It deletes s , the coefficients and the shares.

Participant P_j aborts if its view of $(A_0, \dots, A_{t-1})$ differs from that of another party, and aborts if

$$[sk_j]B \neq \sum_{k=0}^{t-1} [j^k]A_k.$$

Every party computes the group information

$$PK := A_0, \quad PK_i := \sum_{k=0}^{t-1} [i^k]A_k \quad (i = 1, \dots, n)$$

by Lemma 2.6; by this computation every party learns every verification share. *Output*: the share (j, sk_j) to P_j , and $(PK, PK_1, \dots, PK_n)$ to every party. The dealer knows s . Class (Definition 1.1): specified.

The construction is that of RFC 9591, Appendix C, with its Appendices C.1 (“Shamir Secret Sharing”) and C.2 (“Verifiable Secret Sharing”). The secret is generated uniformly at random and must be derived from at least N_s bytes of entropy, N_s being the length of an encoded scalar; Appendix D (“Random Scalar Generation”) gives the generation of uniform scalars. The shares are the evaluations of f at $1, \dots, n$, the secret prepended to the coefficients, and an identifier is never 0 (Appendix C.1). The commitment is the vector of the $[a_k]B$; a participant must abort if it does not hold the same view of it as the other participants, which a secure broadcast channel ensures (Appendix C), and must abort if its share fails the check, the failure being investigated out of band (Appendix C.2). The channel for shares is mutually authenticated and provides confidentiality and integrity (Appendix C). Consistent broadcast realises the first abort rule: under Definition 2.10, type (d), every honest recipient outputs the same vector or every honest recipient aborts.

Proposition 3.3 (Validity of dealt key material). *Let the commitments of Construction 3.2 travel by consistent broadcast (Definition 2.10, type (d)).*

- (i) *Under Assumption 3.1, if the broadcast does not abort, no honest participant aborts, and the shares (j, sk_j) with the group information $(PK, PK_1, \dots, PK_n)$ are valid key material for s (Definition 2.5), with $PK = [s]B$.*
- (ii) *Without Assumption 3.1, the commitments determine a unique polynomial $f^* = \sum_k a_k^* X^k$ of degree at most $t - 1$ with $[a_k^*]B = A_k$ for every k ; a participant whose check passes holds $sk_j = f^*(j)$; and every honest party computes the same group information, that of f^* . The shares that pass and the group information are thus consistent with the one polynomial f^* , whose constant term $\log_B A_0$ the dealer chooses freely.*
- (iii) *Under Assumption 3.1, for every set C of fewer than t participants there is a randomised algorithm that, given $PK = [s]B$ alone, outputs a tuple distributed identically to the joint*

view of C : its shares and the commitments. The commitments reveal $[s]B$ and nothing further to C .

Proof. (i) By steps (1) to (3), $sk_j = f(j)$ for the polynomial f of degree at most $t - 1$ with $f(0) = s$. Each honest share passes its check by the identity $\sum_k [j^k]A_k = [f(j)]B$ displayed in *Crypto Guide*, Construction “Feldman verifiable secret sharing”, and consistent broadcast from an honest dealer that does not abort gives every honest party the dealer’s vector, so no honest participant aborts. Lemma 2.6, part (i), gives $PK = A_0 = [f(0)]B = [s]B$ and $PK_j = [f(j)]B = [sk_j]B$, which are the conditions of Definition 2.5.

(ii) Consistent broadcast gives every honest party the same vector $(A_0, \dots, A_{t-1})$, or every honest party aborts. By *Crypto Guide*, Proposition “What the Feldman check certifies”, that vector determines the unique f^* with $a_k^* = \log_B A_k$, a check passes if and only if $sk_j = f^*(j)$, and nothing constrains $f^*(0)$. Lemma 2.6, part (i), applied to f^* , gives the common group information $PK = [f^*(0)]B$ and $PK_j = [f^*(j)]B$.

(iii) Suppose first $|C| = t - 1$. By *Crypto Guide*, Theorem “Perfect secrecy below the threshold”, the shares $(sk_i)_{i \in C}$ are jointly uniform on $\mathbb{F}_q^{t-1}$ whatever s is. Given the shares, the points $[f(0)]B = PK$ and $[f(i)]B = [sk_i]B$, $i \in C$, are t values of the map $x \mapsto [f(x)]B$ at distinct points; since each A_k is a fixed $\mathbb{F}_q$ -linear combination of these values (interpolation in the exponent), the commitments are a function of PK and the shares. The algorithm samples $t - 1$ uniform scalars as the shares and computes the commitments so; this is the argument of *Crypto Guide*, Remark “Leakage of Feldman sharing”. If $|C| < t - 1$, then, since $t \leq n$, some set $C' \supseteq C$ of $t - 1$ participants exists; the view of C is a projection of that of C' , and the algorithm for C' followed by the projection serves for C . $\square$

Remark 3.4 (Key material of an Orchard-protocol key). For an Orchard-protocol key the shared secret is ask (ZIP 312, “Key Generation”). The output of Construction 3.2 then passes through Construction 6.7 in §6.3 and is completed by Construction 6.8. ZIP 312 also admits a dealer that shares an ask derived from a spending key; that ask is not sampled as in Assumption 3.1, clause (1), and §7.3 treats both cases.

3.2 Distributed key generation

Assumption 3.5 (The proof-of-knowledge hash). The challenge hash

$$H_{\text{dkg}}: \{1, \dots, n\} \times \{0, 1\}^* \times \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{F}_q$$

of Construction 3.6, applied to an injective encoding of its input (identifier, context string, point, point), is modelled as a random oracle (*Crypto Guide*, §“The random oracle model”) independent of every other hash of the volume, in particular of the ciphersuite hashes of Assumption 4.2.

An instantiation achieves the independence by domain separation: a random oracle restricted to prefix-free tags yields independent random oracles (*Crypto Guide*, §“Domain separation and personalisation”, Proposition “Domain separation yields independent oracles”). No ZIP fixes H_{dkg}

or the context string for the Pallas ciphersuite of ZIP 312: that ciphersuite defines the hashes H_1 to H_5 and H_R only (“FROST(Pallas, BLAKE2b-512)”), and key generation is outside the ZIP’s scope (“Key Generation”). The volume therefore takes H_{dkg} as the named oracle of Assumption 3.5.

Construction 3.6 (Pedersen key generation with proofs of knowledge). Let (t, n) be threshold parameters (Definition 2.1) and H_{dkg} the hash of Assumption 3.5. Before Round One the participants agree, by means outside the construction, on a context string $\Phi \in \{0, 1\}^*$ unique to the ceremony.

Round One, participant P_i :

1. It samples $a_{i,0}, \dots, a_{i,t-1} \xleftarrow{\$} \mathbb{F}_q$ independently and uniformly and sets $f_i(X) := \sum_{k=0}^{t-1} a_{i,k} X^k$.

2. It proves knowledge of $a_{i,0}$: it samples $v_i \xleftarrow{\$} \mathbb{F}_q$ and sets

$$R_i := [v_i]B, \quad c_i := H_{\text{dkg}}(i, \Phi, [a_{i,0}]B, R_i), \\ \mu_i := v_i + a_{i,0} c_i, \quad \sigma_i := (R_i, \mu_i).$$

3. It computes $A_{i,k} := [a_{i,k}]B$ for $k = 0, \dots, t-1$.
4. It sends $(A_{i,0}, \dots, A_{i,t-1}, \sigma_i)$ to every other party, the Coordinator included, by consistent broadcast (Definition 2.10, type (d)).
5. For every $\ell \neq i$ it computes $c_\ell := H_{\text{dkg}}(\ell, \Phi, A_{\ell,0}, R_\ell)$ and aborts unless

$$R_\ell = [\mu_\ell]B - [c_\ell]A_{\ell,0}.$$

If every check passes, it deletes the proofs σ_ℓ .

Round Two, participant P_i :

1. It sends $f_i(\ell)$ to each P_ℓ , $\ell \neq i$, over a mutually authenticated confidential channel (Definition 2.10, type (c)), keeps $f_i(i)$, and deletes f_i and the values sent.
2. For every $\ell \neq i$ it aborts unless

$$[f_\ell(i)]B = \sum_{k=0}^{t-1} [i^k]A_{\ell,k}.$$

3. It sets $sk_i := \sum_{\ell=1}^n f_\ell(i)$ and deletes the $f_\ell(i)$.

Group information. Every party sets $A_k := \sum_{\ell=1}^n A_{\ell,k}$ for $k = 0, \dots, t-1$ and computes

$$PK := A_0, \quad PK_j := \sum_{k=0}^{t-1} [j^k]A_k \quad (j = 1, \dots, n)$$

by Lemma 2.6.

Output: (i, sk_i) to P_i , and $(PK, PK_1, \dots, PK_n)$ to every party. A failed check leads to abort only; its cause is investigated out of band.

The construction is that of ePrint 2020/852, Figure 1, in additive notation: the paper’s $g, \phi_{i,k}, k, s_i, Y_i$ and Y are $B, A_{i,k}, v_i, sk_i, PK_i$ and PK . The paper’s Section 5.1 describes it as Pedersen’s protocol, n parallel runs of *Crypto Guide*, Construction “Feldman verifiable secret sharing”, one with each participant as dealer, each share being the sum of the n shares received. Round One, step (2), is the Fiat–Shamir transform of *Crypto Guide*, Construction “Schnorr identification” (§“The Schnorr identification protocol”; §“The Fiat–Shamir transform: from interactive to non-interactive”), with statement $A_{i,0}$, witness $a_{i,0}$, and the prover’s identifier and Φ bound into the challenge; the verification equation of step (5) is that of the identification protocol rearranged. ePrint 2020/852, Figure 1, binds the context string into the challenge to prevent replay of proofs across ceremonies, a design claim that no cited source proves (Remark 3.9). The last part of Figure 1 computes every verification share from the broadcast commitments, which is how parties other than P_j learn PK_j .

Consistent broadcast of Round One is a hypothesis of the construction: ePrint 2020/852, Section 5.1, assumes that participants hold a consistent view of the commitments and does not provide it. A participant that sent different commitment vectors to different parties would lead them to compute different group information: different verification shares, and different group keys when the constant terms $A_{\ell,0}$ differ.

Lemma 3.7 (Sum of dealt sharings). *Let $f_1, \dots, f_n \in \mathbb{F}_q[X]$ have degree at most $t - 1$, with coefficients $a_{\ell,k}$ and coefficient commitments $A_{\ell,k} = [a_{\ell,k}]B$. Then $f := \sum_{\ell=1}^n f_\ell$ has degree at most $t - 1$, constant term $f(0) = \sum_{\ell} a_{\ell,0}$, and coefficient commitments*

$$\left[\sum_{\ell=1}^n a_{\ell,k} \right] B = \sum_{\ell=1}^n A_{\ell,k}, \quad k = 0, \dots, t - 1;$$

and $f(j) = \sum_{\ell} f_\ell(j)$ for every $j \in \mathbb{F}_q$.

Proof. Polynomial addition adds coefficients and does not raise the degree, so the coefficient of X^k in f is $\sum_{\ell} a_{\ell,k}$, zero for $k \geq t$. Evaluation at a point is additive in the polynomial. The map $a \mapsto [a]B$ is a group homomorphism $\mathbb{F}_q \rightarrow \mathbb{G}$, which gives the commitments. $\square$

Proposition 3.8 (Validity of distributed key material). *Consider an execution of Construction 3.6 without abort in which the Round One messages travel by consistent broadcast. For each ℓ let f_ℓ^* be the unique polynomial of degree at most $t - 1$ whose coefficient commitments are $A_{\ell,0}, \dots, A_{\ell,t-1}$, let $f^* := \sum_{\ell} f_\ell^*$, and let $sk_j := f^*(j)$ for every identifier j . Then:*

- (i) *every honest participant P_i outputs $sk_i = f^*(i)$;*
- (ii) *every honest party computes the same group information, $PK = [f^*(0)]B$ and $PK_j = [f^*(j)]B$;*
- (iii) *the shares sk_j with this group information are valid key material (Definition 2.5) for the signing key*

$$s = f^*(0) = \sum_{\ell=1}^n \log_B A_{\ell,0}.$$

| If every participant is honest, $f_\ell^* = f_\ell$ for every ℓ .

Proof. The polynomials f_ℓ^* exist and are unique by *Crypto Guide*, Proposition “What the Feldman check certifies”. An honest P_i broadcasts the commitments of its own f_i , so $f_i = f_i^*$ and its kept value is $f_i^*(i) = f_i^*(i)$. Each received $f_\ell(i)$, $\ell \neq i$, passed the check of Round Two, step (2), so equals $f_\ell^*(i)$ by the same proposition. Hence $sk_i = \sum_\ell f_\ell^*(i) = f^*(i)$ by Lemma 3.7, which is (i). Consistent broadcast gives every honest party the same $A_{\ell,k}$, hence the same $A_k = \sum_\ell A_{\ell,k}$, which by Lemma 3.7 are the coefficient commitments of f^* , of degree at most $t - 1$. Lemma 2.6, part (i), applied to f^* gives $PK = [f^*(0)]B$ and $PK_j = [f^*(j)]B = [sk_j]B$, which is (ii); with the shares $sk_j = f^*(j)$ these are the conditions of Definition 2.5, which is (iii), and $f^*(0) = \sum_\ell f_\ell^*(0) = \sum_\ell \log_B A_{\ell,0}$. If every participant is honest, each broadcasts the commitments of its own polynomial, so $f_\ell^* = f_\ell$. $\square$

Remark 3.9 (Rogue keys and robustness). The statements of this remark are design claims of ePrint 2020/852; none is a theorem. The proofs of knowledge of Round One, step (2), are the change from Pedersen’s protocol: the paper adds them against rogue-key attacks, in which a participant chooses its commitment as a function of the others’ commitments to bias the group key, in the setting $t \geq n/2$ (Section 5.1). No source cited in this volume proves Construction 3.6 secure, or proves FROST as RFC 9591 specifies it unforgeable with it; §7.3 states what is known (Remark 7.13). The protocol is not robust: any failed proof or share check aborts the whole execution and its cause is investigated out of band (Figure 1; Section 3), so one participant can prevent key generation.

Construction 3.6 is designed but unspecified for Zcash (Definition 1.1). The protocol is published (ePrint 2020/852, Figure 1), but no RFC or ZIP specifies a distributed key generation: RFC 9591 places key generation out of scope (Section 1), and ZIP 312 specifies none (“Key Generation”). ZIP 2005 (Proposed), “Usage with FROST”, names the FROST distributed key generation protocol without a specification behind the reference; the revision of ZIP 312 proposed in pull request 895 names a different key-generation protocol, so its adoption would not specify this one. Its Zcash instantiation, the hash H_{dkg} and the context string Φ , is fixed by no ZIP (Assumption 3.5).

4 Signing and aggregation

This section states FROST signing as RFC 9591 specifies it, in execution order: the ciphersuite interface with its hash roles and the encoding of a commitment list; Round One, which draws hedged nonces and commits to them; Round Two, which binds each signature share to the message and to the whole commitment list; aggregation with share verification; the correctness of an honest run; and the single use of nonces, with the exact condition under which reuse of a nonce pair exposes a share. Threshold parameters, signing sets, Lagrange coefficients and key material are those of §2; the key material is the output of either key generation of §3.

4.1 Ciphersuites and encodings

Definition 4.1 (Ciphersuite). A *ciphersuite* fixes the following data.

- (a) A group $\mathbb{G}$ of prime order q , written additively, with identity $\mathcal{O}$ and generator B , and its scalar field $\mathbb{F}_q$. Write $\mathbb{G}^* := \mathbb{G} \setminus \{\mathcal{O}\}$.
- (b) An *element serialisation* $\text{ser}_{\mathbb{G}}$, mapping either every element of $\mathbb{G}$ or every element of $\mathbb{G}^*$ injectively to a byte string of fixed length N_e , and an *element deserialisation* $\text{deser}_{\mathbb{G}}$, which maps a byte string to an element of $\mathbb{G}^*$ or fails. It fails on every string that is not the canonical encoding $\text{ser}_{\mathbb{G}}(P)$ of some $P \in \mathbb{G}$, and on the encoding of $\mathcal{O}$ where one exists; on every other string it inverts $\text{ser}_{\mathbb{G}}$. The serialisation is *total* if its domain is $\mathbb{G}$.
- (c) A *scalar serialisation* $\text{ser}_{\mathbb{F}_q}$, mapping $\mathbb{F}_q$ injectively to byte strings of fixed length N_s , and a *scalar deserialisation* $\text{deser}_{\mathbb{F}_q}$, which fails on every string whose integer value is not below q and inverts $\text{ser}_{\mathbb{F}_q}$ otherwise.
- (d) Hash functions by role:

$$H_1, H_2, H_3: \{0, 1\}^{8*} \rightarrow \mathbb{F}_q, \quad H_4, H_5: \{0, 1\}^{8*} \rightarrow \{0, 1\}^{8N_h},$$

where $\{0, 1\}^{8*}$ denotes the byte strings and N_h is the digest length of the suite. The roles are: H_1 the binding factor, H_2 the challenge, H_3 the nonce, H_4 the message pre-hash and H_5 the commitment-list pre-hash.

- (e) The *signature encoding* $\text{ser}_{\mathbb{G}}(R) \parallel \text{ser}_{\mathbb{F}_q}(z)$ of a pair $(R, z) \in \mathbb{G} \times \mathbb{F}_q$, and the *single-party verification*: a pair (R, z) is accepted on a message m under a public key PK if and only if

$$[z]B = R + [c] PK, \quad c := H_2(\text{ser}_{\mathbb{G}}(R) \parallel \text{ser}_{\mathbb{G}}(PK) \parallel m).$$

This is the verification of the *Crypto Guide*'s Construction "Schnorr signature" (§"From identification to signature via the Fiat–Shamir transform") in additive form, with hash H_2 .

A ciphersuite for *re-randomised signing* adds

- (f) the *randomiser hash* $H_R: \{0, 1\}^{8*} \rightarrow \mathbb{F}_q$ (ZIP 312, "Specification" and "Randomizer Generation").

The ciphersuites of RFC 9591 do not define H_R .

The data are those of RFC 9591, Sections 3.1 ("Prime-Order Group") and 3.2 ("Cryptographic Hash Function"), and Section 6 ("Ciphersuites"), whose Appendix A ("Schnorr Signature Encoding") fixes the encoding and whose Appendix B ("Schnorr Signature Generation and Verification for Prime-Order Groups") the verification equation. The requirements that RFC 9591, Section 6.6 ("Ciphersuite Requirements"), places on every ciphersuite are four: the group is of prime order and every deserialisation outputs an element of its set or fails; every hash function is

domain-separated with a per-suite context string, FROST(Ed25519, SHA-512) excepted for H_2 for compatibility with RFC 8032; H_1 , H_2 and H_3 have output distributions close to uniform; and the signature encoding is specified. The suite FROST(Pallas, BLAKE2b-512) separates its hashes by BLAKE2b personalisations instead, its H_2 being that of RedPallas (Construction 6.1).

The generic element serialisation of RFC 9591, Section 3.1, fails on $\mathcal{O}$; a suite may instead serialise every element, as FROST(Pallas, BLAKE2b-512) does with the star encoding (*Ironwood Guide*, Definition “Star encoding”; ZIP 312, “FROST(Pallas, BLAKE2b-512)”). In both cases the element deserialisation rejects $\mathcal{O}$. Whether ser_G is total is the one property of a suite on which Theorem 4.11 distinguishes cases.

Assumption 4.2 (The ciphersuite hashes). The hashes H_1 , H_2 , H_3 and H_R are modelled as random oracles into $\mathbb{F}_q$ (*Crypto Guide*, §“The random oracle model”, Definition “Random oracle”), independent of each other and of the proof-of-knowledge hash H_{dkg} (Assumption 3.5): each answer at a new input is an independent sample from a distribution on $\mathbb{F}_q$ within statistical distance δ of uniform (*Math Guide*, §“Statistical distance”), where δ is a parameter of the suite. The hashes H_4 and H_5 are modelled as independent random oracles into $\{0, 1\}^{8N_h}$; hence an algorithm making Q queries finds a collision of either with probability at most

$$\frac{Q(Q-1)}{2} \cdot 2^{-8N_h}$$

(*Math Guide*, §“The union bound and a birthday calculation”, Proposition “Birthday bound”). For a suite that builds every role from one hash function under distinct personalisations, the assumption is that this function, keyed by its personalisation, is a random oracle; distinct personalisations then give independent oracles (*Crypto Guide*, §“Domain separation and personalisation”, Proposition “Domain separation yields independent oracles”), and δ is the bias of the reduction of its output into $\mathbb{F}_q$ (*Crypto Guide*, §“Hashing to a field element”).

The assumption is the one hash assumption of every ciphersuite in this volume. RFC 9591, Section 3.2, models the ciphersuite hash as a random oracle in the security analyses it cites. The suite FROST(Pallas, BLAKE2b-512) meets the assumption with $\delta \leq p_{\text{vesta}}/2^{512} < 2^{-257}$ and no further assumption, by Proposition 6.2.

Definition 4.3 (Commitment list and its encoding). Let a ciphersuite be fixed and let I be a signing set (Definition 2.1). A *commitment list* over I is a sequence

$$L = \left((i, D_i, E_i) \right)_{i \in I}, \quad D_i, E_i \in \mathbb{G},$$

with one triple for each member of I : D_i is the *hiding commitment* and E_i the *binding commitment* of member i . The identifiers are distinct and the triples are sorted in ascending order of the least non-negative representative of the identifier. The signing set is read off L as the set of its identifiers. The *encoding* of L is the byte string

$$\text{enc}(L) := \parallel_{i \in I, \text{ ascending}} \left(\text{ser}_{\mathbb{F}_q}(i) \parallel \text{ser}_G(D_i) \parallel \text{ser}_G(E_i) \right)$$

of length $|I|(N_s + 2N_e)$.

A commitment equal to $\mathcal{O}$ occurs only in a suite with total element serialisation, and check (3) of Construction 4.8 rejects it.

The ordering is a requirement of RFC 9591, which states that the list must be sorted in ascending order by identifier, and compares scalars by their least non-negative representative (Sections 3.1 and 4.3, “List Operations”); the encoding is that of Section 4.3. Every component has fixed length, each serialisation is injective, and the order of the triples is fixed; hence the length of $\text{enc}(L)$ determines $|L|$, and the string parses uniquely into the triples. The map enc is therefore injective on commitment lists over one suite, which the binding factors of Definition 4.7 require.

4.2 Round One: nonces and commitments

Construction 4.4 (Round One). Participant P_i , holding the share sk_i (Definition 2.5), proceeds as follows.

1. It draws two independent strings T_d, T_e , each uniform on the 32-byte strings, and sets the *hiding nonce* and the *binding nonce*

$$d_i := H_3(T_d \parallel \text{ser}_{\mathbb{F}_q}(sk_i)), \quad e_i := H_3(T_e \parallel \text{ser}_{\mathbb{F}_q}(sk_i)).$$

2. It sets $D_i := [d_i]B$ and $E_i := [e_i]B$.
3. It stores (d_i, e_i, D_i, E_i) , keeps (d_i, e_i) secret, and sends $(\text{ser}_G(D_i), \text{ser}_G(E_i))$ to the Coordinator over an authenticated channel (Definition 2.10, type (a)). If a serialisation fails, which happens only in a suite whose element serialisation is not total and only if D_i or E_i is $\mathcal{O}$, the participant discards the pair and this execution of Round One fails.

The strings T_d and T_e must be sampled uniformly from a secure source of randomness, and a nonce pair must not be used in more than one signing run.

The construction is RFC 9591, Section 5.1 (“Round One - Commitment”), with the hedged nonce generation of Section 4.1 (“Nonce Generation”), which always samples 32 bytes of fresh randomness and hashes them together with the serialised share. Section 5.1 requires that the nonces not be used in more than one signing and be generated from a source of secure randomness; Section 7.3 (“Nonce Reuse Attacks”) requires the randomness to be sampled uniformly. Nonces lie in $\mathbb{F}_q$, not in $\mathbb{F}_q^*$: the zero nonce is not excluded, and its consequence is case (1) of Theorem 4.11. Round One does not depend on the message or on the signing set, so a participant may run it ahead of any signing run and hold several unused pairs. The paper that introduced FROST (ePrint 2020/852, Figure 2) samples the pair uniformly from $\mathbb{F}_q^* \times \mathbb{F}_q^*$; the specified rule is the hedged derivation above, and this volume states it instead.

Lemma 4.5 (Hedged nonces are close to uniform). *Under Assumption 4.2, let an algorithm, the observer, make at most q_b queries to H_3 and, at one point, choose a byte string w and receive $d := H_3(T \parallel w)$, for T uniform on the 32-byte strings and independent of all else. The pair formed by the observer’s view and d is within statistical distance*

$$\delta + q_b \cdot 2^{-256}$$

of the pair obtained when d is replaced by a uniform element of $\mathbb{F}_q$ independent of all else. For the two nonces of one execution of Round One, received for one string w with independent strings T_d and T_e , the corresponding distance is at most

$$2(\delta + q_b \cdot 2^{-256}) + 2^{-256}.$$

Proof. The argument is that of the *Ironwood Guide*'s Lemma "Distance of GenRandom from uniform" (§"Randomised validating keys"), with 32 fresh bytes in place of 80 and H_3 in place of the RedPallas hash. Let E be the event that the observer queries H_3 at $T \parallel w$. In a second experiment the answer $H_3(T \parallel w)$ is replaced, in the computation of d only, by an independent sample of the answer distribution. In the lazy realisation of the random oracle the answers at other inputs are independent of the answer at $T \parallel w$, so the two experiments are identical until E occurs, and their outputs are within statistical distance $\Pr[E]$ (*Crypto Guide*, §"Security as a game", Remark "Game-hopping"). In the second experiment the observer's view is independent of T , so each query has prefix T with probability at most 2^{-256} , and $\Pr[E] \leq q_b \cdot 2^{-256}$ by the union bound. There the sample is independent of everything else, and replacing it by a uniform scalar changes the pair by at most δ (*Math Guide*, §"Statistical distance", Theorem "Properties of statistical distance", part (4)). The triangle inequality, part (2) of the same theorem, gives the first bound.

For the pair, the event $T_d = T_e$, of probability 2^{-256} , is excluded at that cost; outside it the inputs $T_d \parallel w$ and $T_e \parallel w$ are distinct. The event that the observer queries either input has probability at most $2q_b \cdot 2^{-256}$, and two independent samples are replaced by uniform scalars at cost 2δ . $\square$

The nonces are therefore close to uniform, not uniform, whatever the share; RFC 9591, Section 7.3, states that they are indistinguishable from values sampled uniformly at random, and the lemma quantifies that statement. For FROST(Pallas, BLAKE2b-512), with the δ of Proposition 6.2, the first bound at $q_b = 2^{64}$ is below $2^{-191.99}$, the term $q_b \cdot 2^{-256}$ dominating; in a suite with a larger δ , such as a 48-byte hash to field, δ dominates.

RFC 9591, Section 4.1, hashes the share with the fresh bytes to hedge against a generator whose output is predictable to an observer; Lemma 4.5 assumes T uniform, and the volume proves no statement for a predictable T . Nonces computed from the share and the message alone permit complete key recovery (RFC 9591, Sections 1 and 7.3), which is why fresh bytes are used.

Over $N = 2^{64}$ executions of Round One by one participant, the 32-byte strings of one role, hiding or binding, repeat with probability at most $\binom{N}{2} \cdot 2^{-256} < 2^{-129}$ (*Math Guide*, Proposition "Birthday bound"), so, by the union bound, a repeat in either role has probability at most $2^{-128}(1 - 2^{-64}) < 2^{-128}$, the figure of RFC 9591, Section 4.1. A repeated nonce *pair*, the hypothesis of Lemma 4.13, requires in the same two executions equal hiding nonces and equal binding nonces.

Lemma 4.6 (Repetition of a nonce pair). *Under Assumption 4.2, two of N executions of Round One by one participant yield equal nonce pairs with probability at most*

$$\binom{N}{2} (2^{-255} + 1/q + \delta)^2.$$

Proof. For two executions $j \neq k$, the hiding strings are equal with probability 2^{-256} , and so are the binding strings, independently. Two answers of H_3 at distinct inputs are equal with probability at most $1/q + \delta$, since every value has probability at most $1/q + \delta$ under a distribution within δ of uniform (*Math Guide*, Theorem “Properties of statistical distance”, part (3)); and when both roles have distinct strings, the two equalities constrain answers at two distinct pairs of inputs, whose probability of holding together is at most $(1/q + \delta)^2$, except when the two pairs of inputs coincide, an event of probability at most $2 \cdot 2^{-512}$. Summing the cases gives at most $2^{-512} + 2 \cdot 2^{-256}(1/q + \delta) + (1/q + \delta)^2 + 2^{-511}(1/q + \delta)$, which is below the square displayed; the union bound over the $\binom{N}{2}$ pairs of executions gives the claim. $\square$

4.3 Round Two: binding factors and signature shares

Definition 4.7 (Binding factors and group commitment). Let PK be the group key, m a message, that is a byte string, and L a commitment list over a signing set I (Definition 4.3). For $i \in I$ the *binding factor* of i is

$$\rho_i := H_1(\text{ser}_G(PK) \parallel H_4(m) \parallel H_5(\text{enc}(L)) \parallel \text{ser}_{\mathbb{F}_q}(i));$$

the *commitment share* of i is $R_i := D_i + [\rho_i]E_i$; the *group commitment* is $R := \sum_{i \in I} R_i$; and the *challenge* is

$$c := H_2(\text{ser}_G(R) \parallel \text{ser}_G(PK) \parallel m),$$

with ρ_i defined whenever $\text{ser}_G(PK)$ is, and c whenever $\text{ser}_G(R)$ and $\text{ser}_G(PK)$ are. All four are functions of the public data (PK, m, L) , computed identically by every member of I and by the Coordinator.

The binding factor is that of RFC 9591, Section 4.4 (“Binding Factors Computation”), the group commitment that of Section 4.5 (“Group Commitment Computation”), and the challenge that of Section 4.6 (“Signature Challenge Computation”). The challenge input has the order nonce commitment, public key, message of the single-party challenge of Definition 4.1, part (e), so a verifier recomputes c from (R, PK, m) alone. The paper’s binding value $\rho_i = H_1(i, m, B)$, with B its commitment list, and its challenge $c = H_2(R, Y, m)$ (ePrint 2020/852, Figure 3, in multiplicative notation with $R = \prod_i D_i E_i^{\rho_i}$) differ as follows: the specified binding-factor input adds the group key and pre-hashes the message and the encoded list; the challenge input is the paper’s, in the same order. For FROST(Pallas, BLAKE2b-512), H_2 is the RedPallas hash and ser_G the star encoding, so c is the challenge that RedPallas validation recomputes (Construction 6.1; ZIP 312, “Rationale”).

Construction 4.8 (Round Two). *Coordinator.* It selects a signing set I with $t \leq |I| \leq n$; takes, for each $i \in I$, one commitment pair (D_i, E_i) received from P_i in Round One and not placed in any earlier request; forms the commitment list L over I (Definition 4.3); and sends (m, L) , serialised componentwise, to each P_i with $i \in I$ over an authenticated channel (Definition 2.10, type (a)).

Member P_i , on receiving (m, L) :

1. it checks that m is a message it is willing to sign; for spend authorisation this is the check of Definition 5.9;
2. it checks that its identifier i appears in L with a commitment pair (D_i, E_i) that it stores and has not used;
3. it deserialises every commitment of L with deser_G , which fails on a non-canonical encoding and on the encoding of $\mathcal{O}$;
4. it checks that the identifiers of L are distinct, without which $\lambda_{I,i}$ is undefined, and in ascending order, as RFC 9591 requires of a commitment list (Section 4.3).

It aborts if any check fails. Otherwise it computes ρ_i , R and c (Definition 4.7), aborting if ρ_i or c is undefined, and $\lambda_{I,i}$ (Definition 2.3), with I the set of identifiers of L ; sets the *signature share*

$$z_i := d_i + e_i \rho_i + \lambda_{I,i} sk_i c \in \mathbb{F}_q;$$

deletes (d_i, e_i) and (D_i, E_i) ; and only then sends $\text{ser}_{\mathbb{F}_q}(z_i)$ to the Coordinator over an authenticated channel (Definition 2.10, type (a)).

The construction is RFC 9591, Section 5.2 (“Round Two - Signature Share Generation”). The Coordinator’s choice of at least t and at most n participants is required by Section 5. Each participant must validate the commitment list by deserialising each element, must abort if deserialisation fails, and must ensure that its identifier and commitments from Round One appear in the list (Section 5.2); applications that restrict the messages a participant signs perform further validation, which Section 7.7 (“Input Message Validation”) recommends so that participants do not act as signing oracles for arbitrary messages. The computation of $\lambda_{I,i}$ fails on an absent or repeated identifier (Section 4.2, “Polynomials”). The share equation is that of Section 5.2. After computing its share, each participant must delete the nonce and the corresponding commitment, and must not use the nonce as input more than once. The deletion is the step on which Proposition 4.14 rests.

4.4 Aggregation and share verification

Construction 4.9 (Aggregation). The Coordinator holds (m, L) and the group information $(PK, (PK_j)_{j=1}^n)$ stored from key generation (Definition 2.5). The shares arrive over authenticated channels without confidentiality (Definition 2.10, type (a)). The Coordinator proceeds as follows.

1. It deserialises each share with $\text{deser}_{\mathbb{F}_q}$ and aborts if any deserialisation fails.
2. It computes ρ_i and R_i for $i \in I$, R and c (Definition 4.7), aborting if ρ_i or c is undefined.
3. It sets $z := \sum_{i \in I} z_i$ and outputs the signature (R, z) , encoded $\text{ser}_G(R) \parallel \text{ser}_{\mathbb{F}_q}(z)$.

It should verify (R, z) on m under PK by the single-party verification (Definition 4.1, part (e)) before publishing it, and should abort if the signature is invalid. It may check each share by the

share equation

$$[z_i]B = R_i + [c \lambda_{I,i}] PK_i,$$

in particular when the aggregate is invalid. The keys PK and PK_i in these checks are taken from the stored group information, never from the messages of the run.

The construction is RFC 9591, Section 5.3 (“Signature Share Aggregation”). The Coordinator must validate each share by deserialisation and must abort if validation fails; it should verify the signature under the group key before publishing or releasing it; if that verification fails, it may verify each share individually to identify and act on misbehaving participants; and the group key and the keys PK_i used there must come from the stored group information. Section 5 requires an authenticated channel for identifying misbehaving participants and does not require confidentiality. The paper checks every share unconditionally before aggregating (ePrint 2020/852, Figure 3); the specified duties are the recommendation and the permission above, and this volume states those.

Proposition 4.10 (Identifiable abort). *Let the key material be valid (Definition 2.5), let the Coordinator be honest, and let c be defined.*

- (i) *If the share of every member satisfies its share equation, then $[z]B = R + [c] PK$, so the aggregate (R, z) is accepted by the single-party verification on m under PK .*
- (ii) *The share of every member that follows Construction 4.8 satisfies its share equation.*
- (iii) *Hence, if the aggregate is not accepted, the share equation of some member fails, and that member deviated from the protocol; the authenticated channel identifies it as the sender of the share.*

Proof. (ii) For a member following the construction,

$$[z_i]B = [d_i]B + [\rho_i][e_i]B + [c \lambda_{I,i}][sk_i]B = R_i + [c \lambda_{I,i}] PK_i,$$

since $D_i = [d_i]B$, $E_i = [e_i]B$ and, by validity, $PK_i = [sk_i]B$.

(i) Let f be the polynomial of degree at most $t - 1$ that validity provides, with $f(j) = sk_j$ and $PK = [f(0)]B$. Summing the share equations over I gives

$$[z]B = \sum_{i \in I} R_i + \left[c \sum_{i \in I} \lambda_{I,i} sk_i \right] B = R + [c f(0)]B = R + [c] PK,$$

by Lemma 2.4 applied to f . The challenge c is the one the single-party verification recomputes from (R, PK, m) (Definition 4.7), so the verification accepts.

(iii) is the contrapositive of (i), with (ii). $\square$

The mechanism for acting on an identified member is outside RFC 9591 (Section 5.4, “Identifiable Abort”). FROST provides no robustness (Sections 5.4 and 7): one member that withholds or corrupts its share prevents the signature, and a failed run is restarted from Round One with fresh nonces, since the consumed pairs are deleted.

4.5 Correctness of FROST signing

Theorem 4.11 (Correctness of FROST). *Let the key material be valid (Definition 2.5), as key generation by a dealer or by the distributed protocol outputs it (Propositions 3.3 and 3.8); let I be a signing set and m a message that every member of I accepts; and let every party follow Constructions 4.4, 4.8 and 4.9. Then, outside the three cases below, the run outputs (R, z) with*

$$R = [r]B, \quad z = r + c s, \quad r := \sum_{i \in I} (d_i + e_i \rho_i),$$

which the single-party verification accepts on m under PK . In the three cases the run aborts.

- (1) Zero nonce. *Some d_i or e_i , $i \in I$, is 0, so D_i or E_i is $\mathcal{O}$. In a suite whose element serialisation is not total, Round One fails on it; in every suite the run aborts no later than check (3) of Round Two, since deser_G rejects $\mathcal{O}$. Under Assumption 4.2 the probability of this case is at most $2|I|(\delta + 1/q)$ per run.*
- (2) Identity group commitment. *Only in a suite whose element serialisation is not total, as the generic serialisation of RFC 9591 is not: $R = \mathcal{O}$, so c is undefined. Under Assumption 4.2 the probability of this case, outside case (1), is at most $\delta + 1/q$ per run.*
- (3) Identity group key. *Only in a suite whose element serialisation is not total: $s = 0$, so $PK = \mathcal{O}$ and ρ_i and c are undefined. For s uniform in $\mathbb{F}_q$, as under Assumption 3.1, its probability is $1/q$ per key generation.*

A suite with a total element serialisation, such as FROST(Pallas, BLAKE2b-512), has only case (1).

Proof. Write $r_i := d_i + e_i \rho_i$, so that $z_i = r_i + c \lambda_{i,i} sk_i$. The nonce $r = \sum_{i \in I} r_i$ is additively shared by construction: each summand is formed locally, with no dealer and no interaction; and $R = \sum_{i \in I} R_i = \sum_{i \in I} [r_i]B = [r]B$. The Lagrange coefficients turn the Shamir shares $sk_i = f(i)$ of $s = f(0)$ into additive shares $\lambda_{i,i} sk_i$ with sum s , by Lemma 2.4, since f has degree at most $t - 1$ by validity. Hence

$$z = \sum_{i \in I} z_i = r + c s, \quad [z]B = [r]B + [c][s]B = R + [c] PK,$$

where $c = H_2(\text{ser}_G(R) \parallel \text{ser}_G(PK) \parallel m)$ is the challenge the verifier recomputes (Definition 4.7). Outside the three cases, every check of Round Two passes: m is accepted, each member's pair is stored and unused, every commitment lies in $\mathbb{G}^*$ and deserialises, the identifiers of L are distinct and sorted, and ρ_i and c are defined. Every share deserialises, being an honest serialisation, so aggregation outputs (R, z) .

(1) Each of the $2|I|$ nonces is an answer of H_3 , equal to 0 with probability at most $1/q + \delta$, because under a distribution within δ of uniform every value has probability at most $1/q + \delta$ (*Math Guide*, Theorem “Properties of statistical distance”, part (3)). Since B generates a group of prime order, $[d]B = \mathcal{O}$ if and only if $d = 0$. The union bound gives $2|I|(1/q + \delta)$.

(2) Outside case (1), fix $i \in I$; then $E_i \neq \mathcal{O}$. Condition on all answers of H_3 , H_4 and H_5 , and on the answers of H_1 at every input other than that of ρ_i . Then $R = \mathcal{O}$ if and only if $[\rho_i]E_i$ equals the

element $-D_i - \sum_{j \in I \setminus \{i\}} R_j$, which is fixed; since E_i generates $\mathbb{G}$, this holds for exactly one value of ρ_i . The binding factor ρ_i is the answer of H_1 at an input with suffix $\text{ser}_{\mathbb{F}_q}(i)$, distinct from the inputs of the other members, whose suffixes differ; so it takes that value with probability at most $1/q + \delta$.

(3) Since B generates a group of prime order, $PK = [s]B = \mathcal{O}$ if and only if $s = 0$, and $\text{ser}_{\mathbb{G}}(PK)$, an input of every ρ_i and of c , is then undefined. A uniform $s \in \mathbb{F}_q$ is 0 with probability $1/q$. $\square$

For FROST(Pallas, BLAKE2b-512), with the δ of Proposition 6.2, the bound of case (1) is below $2^{-251.9}$ for $|I| = 2$ and below $2^{-251.3}$ for $|I| = 3$. In a suite with the parameters $q = p_{\text{Vesta}}$ and that δ but a serialisation that is not total, the bound of case (2) would be below $2^{-253.9}$.

The output of a threshold run is a single-party Schnorr signature under the group key. It carries no threshold marker: a verifier applies to it, with (PK, m) alone and no knowledge of t , n or I , the verification it applies to a signature of one signer. RFC 9591 states this in Section 5, under which signatures can be verified as if produced by a single signer holding s , and in Section 6, under which the verification instructions of each suite are equivalent to those for a single participant; Appendix B gives that verification.

Example 4.12 (A two-of-three signing run). Every number below is output by the volume's toy script; none is computed by hand. The group is the curve $y^2 = x^3 + 11$ over $\mathbb{F}_{1009}$, whose group of points has prime order $q = 967$, with generator $B = (1, 298)$. A point (x, y) is serialised as the two-byte little-endian integer $x + 2^{15}(y \bmod 2)$, and $\mathcal{O}$ as zero, the analogue of the star encoding; a scalar is serialised as a two-byte little-endian integer. The hashes are BLAKE2b-512 under the personalisations that FROST(Pallas, BLAKE2b-512) assigns to H_1 to H_5 (Construction 6.1), the scalar-valued ones read as little-endian integers and reduced modulo 967. The 32 fresh bytes of each nonce are fixed strings, so that the run is reproducible; the example therefore does not follow the sampling rule of Construction 4.4.

Key material. A dealer with $t = 2$, $n = 3$ uses $f(X) = 321 + 778X$, so $s = 321$ and

$$sk_1 = 132, \quad sk_2 = 910, \quad sk_3 = 721.$$

The coefficient commitments are $A_0 = (526, 125)$ and $A_1 = (433, 783)$, and by Lemma 2.6 the group information is $PK = A_0$ and $PK_j = A_0 + [j]A_1$:

$$PK_1 = (963, 797), \quad PK_2 = (587, 458), \quad PK_3 = (30, 509),$$

each equal to $[sk_j]B$.

Round One. Members 1 and 3 draw

$$\begin{array}{llll} d_1 = 155, & e_1 = 457, & D_1 = (942, 518), & E_1 = (309, 697), \\ d_3 = 464, & e_3 = 724, & D_3 = (958, 797), & E_3 = (582, 220). \end{array}$$

Round Two. The signing set is $I = \{1, 3\}$, with

$$\lambda_{I,1} = 3 \cdot (3 - 1)^{-1} = 3/2 = 485, \quad \lambda_{I,3} = 1 \cdot (1 - 3)^{-1} = -1/2 = 483$$

modulo 967. Then $\lambda_{I,1} + \lambda_{I,3} = 1$ and $\lambda_{I,1}sk_1 + \lambda_{I,3}sk_3 = 321 = s$, as Lemma 2.4 requires with $f = 1$ and with the sharing polynomial. For the message m equal to the ASCII string toy message,

the binding factors are $\rho_1 = 869$ and $\rho_3 = 285$, the group commitment is $R = (972, 62)$ and the challenge is $c = 628$. The shares are $z_1 = 419$ and $z_3 = 717$, and each satisfies its share equation $[z_i]B = D_i + [\rho_i]E_i + [c \lambda_{I,i}] PK_i$.

Aggregation. The aggregate is $z = 169$, and $[z]B = R + [c] PK$ holds. The toy group gives no security; the example checks the algebra only.

4.6 Single use of nonces

Lemma 4.13 (Reuse of a nonce pair exposes the share). *Let member P_i answer k Round Two requests, $j = 1, \dots, k$, with one nonce pair (d, e) and share sk_i . Its responses are*

$$z_j = d + e \rho_j + \mu_j sk_i, \quad \mu_j := \lambda_{I_j, i} c_j,$$

where ρ_j , c_j and I_j are the binding factor of i , the challenge and the signing set of request j , all computable from public data. Each response is affine in the unknown $(d, e, sk_i) \in \mathbb{F}_q^3$ with coefficient vector $v_j := (1, \rho_j, \mu_j)$. Say that sk_i is determined by $(z_j, v_j)_{j \leq k}$ if every $x \in \mathbb{F}_q^3$ with $v_j \cdot x = z_j$ for all j has third coordinate sk_i . The following are equivalent:

- (a) sk_i is determined by $(z_j, v_j)_{j \leq k}$;
- (b) $(0, 0, 1)$ lies in the span of $v_1, \dots, v_k$ (Math Guide, §“Linear combinations, span, and linear independence”, Definition “Linear combination and span”);
- (c) the points (ρ_j, μ_j) do not all lie on one line $\mu = \alpha + \beta \rho$ with $\alpha, \beta \in \mathbb{F}_q$.

In particular, three responses with linearly independent coefficient vectors determine sk_i ; two responses with distinct binding factors never do, and every value of sk_i is then consistent with them; and two responses with equal binding factors and $\mu_1 \neq \mu_2$ determine

$$sk_i = (z_1 - z_2) / (\mu_1 - \mu_2).$$

Proof. (b) implies (a): if $(0, 0, 1) = \sum_j a_j v_j$, then every consistent x has third coordinate $\sum_j a_j (v_j \cdot x) = \sum_j a_j z_j$, one value, which the true unknown attains.

(a) implies (c): suppose every point lies on $\mu = \alpha + \beta \rho$, and put $u := (\alpha, \beta, -1)$. Then $v_j \cdot u = \alpha + \beta \rho_j - \mu_j = 0$ for every j , so $(d, e, sk_i) + \gamma u$ is consistent for every $\gamma \in \mathbb{F}_q$, and its third coordinate $sk_i - \gamma$ takes every value; sk_i is not determined.

(c) implies (b): if two points have equal ρ and distinct μ , then $v_1 - v_2$ (for those two) is a nonzero multiple of $(0, 0, 1)$. Otherwise distinct points have distinct ρ . Two points with distinct ρ lie on a line $\mu = \alpha + \beta \rho$, so (c) forces at least three distinct points, not all on one such line; no vertical line contains them either, since their ρ are distinct. Hence some three of them are not collinear, their vectors $(1, \rho, \mu)$ are linearly independent, and they span $\mathbb{F}_q^3$, which contains $(0, 0, 1)$.

The particular cases follow: independent vectors span $\mathbb{F}_q^3$; two points with distinct ρ lie on a line $\mu = \alpha + \beta \rho$, and the proof of (a) implies (c) shows every value of sk_i consistent; and with $\rho_1 = \rho_2$, $z_1 - z_2 = (\mu_1 - \mu_2) sk_i$. $\square$

The last case is the *Crypto Guide*’s Proposition “Nonce reuse reveals the secret key” (§“Security in the random oracle model and the forking lemma”) applied to the effective nonce $d + e\rho$. Whoever sees the shares and the public data can compute sk_i in the determined cases, since the shares travel without confidentiality (Definition 2.10, type (a)). RFC 9591 states the risk (Sections 1 and 7.3) and forbids the reuse (Section 5.2).

In the toy group of Example 4.12, member 1 answers three requests on the message `toy message` over $I = \{1, 3\}$ with its pair $(d, e) = (155, 457)$, the requests differing only in the commitments of member 3. The triples (ρ_j, μ_j, z_j) are $(682, 413, 819)$, $(403, 331, 773)$ and $(947, 46, 955)$. The determinant of the three coefficient vectors is $347 \neq 0$, and solving the system gives $sk_1 = 132$. From the first two responses alone, the value 133 with $(d, e) = (802, 325)$ is also consistent, so sk_1 is not determined.

Proposition 4.14 (Replayed commitments). *A Coordinator that places in a Round Two request a commitment pair of member P_i that P_i has already used to answer a request obtains no response from P_i , unless P_i still stores a second copy of that pair from another execution of Round One, an event whose probability over N executions is at most $\binom{N}{2}(2^{-255} + 1/q + \delta)^2$ (Lemma 4.6). Outside that event a replay denies service only, and yields no second response under one nonce pair, the hypothesis of Lemma 4.13.*

Proof. A member responds only for a pair that it stores and has not used, by check (2) of Construction 4.8, and it deletes a pair when it answers with it. Outside that event no second copy of the pair is stored, so a used pair is no longer stored, check (2) fails and the member aborts. $\square$

The Coordinator may in addition track, per group key, the commitments already used (RFC 9591, Section 7.3).

5 Re-randomised FROST for spend authorisation

This section states the re-randomised variant of FROST that ZIP 312 specifies, in the order of the mechanism: why spend authorisation requires a shifted key; the re-randomisable threshold scheme; the derivations of the randomiser, with the bound showing that the specified derivation, fed the signature digest as its message, succeeds only with negligible probability; the signers’ duty to check the signed message; the re-randomised rounds with their correctness; and the requirements and trust model of ZIP 312 that this flow forces. The ciphersuite is generic, with the randomiser hash H_R of Definition 4.1, part (f); the Pallas instance is Construction 6.1. ZIP 312 is Draft.

5.1 Spend authorisation under randomised validating keys

The single-signer objects are cited, not re-derived. The spend-authorisation instance of RedPallas (*Ironwood Guide*, §“The RedPallas signature scheme”, Construction “RedPallas” and Definition “Spend-authorisation and binding-signature instances”) works in the Pallas group, of prime order $q = p_{\text{Vesta}}$, with base

$$B = G^{\text{SpendAuth}} = \text{GroupHash}(z.\text{cash}:\text{Orchard}, G).$$

A key pair is $(x, X = [x]B)$. A signature on a message M is $\sigma = \underline{R} \parallel \text{I2LEOSP}_{256}(S)$, with challenge $c = H^{\otimes}(\underline{R} \parallel \underline{X} \parallel M)$, where $H^{\otimes}$ reduces modulo p_{Vesta} the BLAKE2b-512 output under the personalisation `Zcash_RedPallasH`. Validation accepts if and only if the first 32 bytes decode to a point R , a non-canonical encoding decoding to $\perp$ and being rejected; $S < p_{\text{Vesta}}$; and $[S]B = R + [c]X$, the cofactor being one on Pallas. The nonce and challenge hashes take $\underline{X}$: the scheme is key-prefixed (Remark “Key prefixing” there). The message of every spend-authorisation signature is the signature digest `SigHash` (*Ironwood Guide*, §“Transaction digests and signatures”, Definition “Signature digest”). The deployed rules are those of the protocol specification, §“RedDSA, RedJubjub, and RedPallas” and §“Spend Authorization Signature (Sapling and Orchard)”.

Re-randomisation by a randomiser $\alpha \in \mathbb{F}_q$ maps a key pair (x, X) to $(x + \alpha, X + [\alpha]B)$ (*Crypto Guide*, §“Key re-randomisation and unlinkability”, Definition “Key re-randomisation”); the identity randomiser is 0. The randomiser generator `GenRandom` draws T uniformly from the 80-byte strings and returns $H^{\otimes}(T)$ (*Ironwood Guide*, Definition “Randomiser generator and key re-randomisation”).

For each Action the spender draws $\alpha := \text{GenRandom}()$ and sets

$$\text{rsk} := \text{ask} + \alpha, \quad \text{rk} := \text{ak}^{\text{P}} + [\alpha]G^{\text{SpendAuth}} = [\text{rsk}]G^{\text{SpendAuth}},$$

with `ask` sign-normalised; the Action publishes `rk` and the signature under `rsk` on `SigHash` (*Ironwood Guide*, §“Randomised validating keys”, Construction “Randomised validating key”). Two consensus rules apply: $\text{rk} \neq \mathcal{O}$, and the signature is valid under `rk` on `SigHash` (protocol specification, §“Action Descriptions” and §“Spend Authorization Signature (Sapling and Orchard)”).

The verifier’s key is therefore `rk`, which equals ak^{P} only for $\alpha = 0$, an event of probability at most $1/p_{\text{Vesta}} + 2^{-512}$ under `GenRandom` (*Ironwood Guide*, Assumption “The RedPallas hash as a random oracle”). Only these scheme-level statements are inherited. The *Ironwood Guide* constructs `ask` only for keys with `use_qsk = false` (Remark “Scope of the key derivations”), so its key-derived theorems are not cited for threshold keys (§7.5).

For every Action of the Orchard protocol, in either pool, the single signer proceeds in the order: draw α ; compute `rk`; compute `SigHash` of the transaction, which commits to `rk`; sign it under `rsk`. The Action proof takes α as auxiliary (secret) input and `rk` as primary input and, being authorising data, may be computed after signing (*Ironwood Guide*, §“Construction of a transaction”, Construction “Ironwood-pool bundle”, steps “Randomised keys”, “Proof”, “Digest” and “Spend authorisation”; ZIP 229, “Motivation”). Sapling spends are outside the volume.

Definition 5.1 (Strong unforgeability under re-randomised keys). A *re-randomisable signature scheme* is a signature scheme (`GenPrivate`, `DerivePublic`, `Sign`, `Validate`) with private-key set `Private` and public-key set `Public`, together with a randomiser set $\mathcal{R}$, a randomised generator `GenRandom` with values in $\mathcal{R}$, maps

$$\text{RandomizePrivate}: \mathcal{R} \times \text{Private} \rightarrow \text{Private}, \quad \text{RandomizePublic}: \mathcal{R} \times \text{Public} \rightarrow \text{Public},$$

and an identity randomiser $\alpha_{\text{id}} \in \mathcal{R}$, such that:

1. each $\text{RandomizePrivate}(\alpha, \cdot)$ is injective and efficiently invertible;
2. $\text{RandomizePrivate}(\alpha_{\text{id}}, \cdot)$ is the identity;

3. for every sk , the value $\text{RandomizePrivate}(\alpha, sk)$ with $\alpha \leftarrow \text{GenRandom}()$ is distributed as $\text{GenPrivate}()$;
4. for every α and sk ,

$$\begin{aligned} \text{RandomizePublic}(\alpha, \text{DerivePublic}(sk)) \\ = \text{DerivePublic}(\text{RandomizePrivate}(\alpha, sk)). \end{aligned}$$

Let $sk \leftarrow \text{GenPrivate}()$ and $vk := \text{DerivePublic}(sk)$. The signing oracle keeps a set $\mathcal{Q}$, initially empty; on a query (m, α) it returns $\sigma := \text{Sign}_{\text{RandomizePrivate}(\alpha, sk)}(m)$ and adds (m, σ) to $\mathcal{Q}$. The scheme is *SURK-CMA secure* if no efficient adversary, given vk and the oracle, outputs, except with negligible probability, a triple (m', σ', α') with

$$\text{Validate}_{\text{RandomizePublic}(\alpha', vk)}(m', \sigma') = 1 \quad \text{and} \quad (m', \sigma') \notin \mathcal{Q}.$$

Freshness is on the message–signature pair; the forgery may be under any re-randomisation of vk , the identity included.

The definition is that of the protocol specification, §“Signature with Re-Randomizable Keys”, adapted there from Fleischhacker et al. (PKC 2016, Section 3) with the randomiser and key arguments swapped; the specification requires every spend-authorisation instance to meet it (§“Spend Authorization Signature (Sapling and Orchard)”). The single-signer spend-authorisation instance meets it: the *Ironwood Guide*’s Proposition “Unforgeability under re-randomisation” (“RedPallas unforgeability”), part (ii), under discrete logarithms on Pallas and the RedPallas hash as a random oracle, requires freshness only of the triple of key, message and signature, and states the implication. The threshold form of the notion is an open problem (Definition 1.1; Table 3).

Proposition 5.2 (A fixed-key aggregate under a randomised key). *Let key material with signing key s and group key $PK = [s]B$ be valid (Definition 2.5), over a ciphersuite meeting Assumption 4.2 with distance δ .*

- (i) *For every signing set I , $\sum_{i \in I} \lambda_{I,i} sk_i = s$. Hence every honest run of FROST (Constructions 4.4, 4.8 and 4.9) outputs a signature valid under the one key PK , whatever I , apart from the failed runs of Theorem 4.11.*
- (ii) *Fix $\alpha \in \mathbb{F}_q$ with $\alpha \neq 0$ and $PK + [\alpha]B \neq \mathcal{O}$ before the experiment. In any experiment whose parties make at most Q_2 queries to H_2 in total, the probability that one output pair $(m, (R, z))$, designated in advance (such as the aggregate of a run), is accepted by the single-party verification (Definition 4.1, part (e)) both under PK and under $PK + [\alpha]B$ is at most*

$$(Q_2 + 2) (1/q + \delta).$$

Proof. (i) Lemma 2.4 with f the sharing polynomial gives the sum; Theorem 4.11 gives the rest.

- (ii) Write $PK' := PK + [\alpha]B$. Acceptance of (R, z) on m under both keys means $[z]B = R +$

$[c] PK = R + [c'] PK'$, where

$$c = H_2(\text{ser}_G(R) \parallel \text{ser}_G(PK) \parallel m), \quad c' = H_2(\text{ser}_G(R) \parallel \text{ser}_G(PK') \parallel m);$$

hence $[c] PK = [c'] PK'$, that is $cs = c'(s + \alpha)$. The two inputs differ, since $PK' \neq PK$ and ser_G is injective. Add the two queries at the inputs of the designated pair if they are missing, so that at most $Q_2 + 2$ queries are made. Call a query with key component PK and one with key component PK' and the same R and m partners; each query has at most one partner. For each query define a *target*, where $s + \alpha \neq 0$ because $PK' \neq \mathcal{O}$. If $s \neq 0$ and the partner was answered earlier, the target is the value the query's own answer must take for the equation to hold: $cs/(s + \alpha)$ for a query with key PK' , and $c'(s + \alpha)/s$ for a query with key PK . If $s = 0$, the equation reads $c'\alpha = 0$, and every query with key PK' has target 0. A pair valid under both keys makes some query hit its target: the later-answered query of the pair if $s \neq 0$, the query with key PK' if $s = 0$. Each target is fixed before the fresh answer is drawn, and that answer takes any fixed value with probability at most $1/q + \delta$ (*Math Guide*, §“Statistical distance”, Theorem “Properties of statistical distance”, part (3)). The union bound over the at most $Q_2 + 2$ queries gives the claim. $\square$

The argument is that of the *Ironwood Guide*'s Remark “Key prefixing” for the unshifted response. The bound is for α fixed in advance; with α chosen adaptively, one query may pair with many. For FROST(Pallas, BLAKE2b-512) the per-query term $1/q + \delta$ is below $2^{-253.9}$.

Spend authorisation re-randomises the validating key of every Action with a randomiser from GenRandom, which makes validating keys unlinkable (*Ironwood Guide*, Proposition “Unlinkability of randomised validating keys”); a signer bound to one key cannot follow it (ZIP 312, “Motivation”). By part (i), a plain FROST aggregate is valid under $PK = ak^P$, which consensus accepts only in an Action with $\text{rk} = ak^P$, that is with randomiser zero, publishing the spend validating key in every such Action. By part (ii), it is valid under rk for a nonzero randomiser only with the stated probability. The re-randomised variant shifts every participant's share, every verification share and the group key by one common scalar.

5.2 Re-randomisable threshold signatures

Definition 5.3 (Re-randomisable threshold signature scheme). A *re-randomisable threshold signature scheme* is a scheme of Definition 2.8 over a group G of prime order q with generator B , together with a randomiser set $\mathcal{X}$ and:

1. a randomised algorithm GenRand with values in $\mathcal{X}$;
2. an *effective-randomiser map* $H_r: \mathcal{X} \times \{0, 1\}^* \rightarrow \mathbb{F}_q$, sending a randomiser α and an *auxiliary string* aux to the *effective randomiser* $\hat{\alpha} := H_r(\alpha, aux)$;
3. the *shifts*

$$\begin{aligned} \text{RandShare}(sk_i, \alpha, aux) &:= sk_i + \hat{\alpha}, & \text{RandPKShare}(PK_i, \alpha, aux) &:= PK_i + [\hat{\alpha}]B, \\ \text{RandPK}(PK, \alpha, aux) &:= PK + [\hat{\alpha}]B; \end{aligned}$$

such that the following *correctness* holds. For valid key material with signing key s , every (α, aux) with $PK + [\hat{\alpha}]B$ in the domain of ser_G (every (α, aux) when ser_G is total), every signing set I and every message m : the shifted shares, verification shares and group key are valid key material for $s + \hat{\alpha}$; and a run of the signing rounds on the shifted material over I on m in which every party is honest outputs either $\perp$ or a signature that verification accepts under $PK + [\hat{\alpha}]B$, and outputs $\perp$ with negligible probability.

The definition is that of ePrint 2024/436, Definition 5, which names the algorithms, with general RandShare, RandPKShare and RandPK and a distribution on $\mathcal{X}$, without a correctness relation; the correctness clause is this volume's. The auxiliary string models a contribution from the transcript of another protocol, so that a new session changes the shifted key even under a repeated α (Remark 1 there).

FROST is an instance, with the identity exception of Theorem 5.11 in a suite whose element serialisation is not total, with H_r a hash into scalars, whose role the randomiser hash H_R of ZIP 312 plays, and with Rounds One and Two and aggregation run unchanged on the shifted material (ePrint 2024/436, Figure 5, read in additive notation and with the challenge input order of RFC 9591, nonce commitment, key, message, in place of the figure's key, nonce commitment, message; ZIP 312, "Re-randomizable FROST"); its rounds are constructed once, in Construction 5.10. The shifted material is valid key material: for the sharing polynomial f , the polynomial $f + \hat{\alpha}$ has degree at most $t - 1$, constant term $s + \hat{\alpha}$ and values $sk_i + \hat{\alpha}$, and $[s + \hat{\alpha}]B = PK + [\hat{\alpha}]B$, $[sk_i + \hat{\alpha}]B = PK_i + [\hat{\alpha}]B$. Recombination on it is Lemma 2.4 applied to $f + \hat{\alpha}$, whose case $f = 1$ gives $\sum_{i \in I} \lambda_{I,i} = 1$. Correctness of the instance is Theorem 5.11.

5.3 Randomiser generation

Construction 5.4 (Randomiser generation of ZIP 312). After Round One, let m be the message and L the commitment list of Definition 4.3, sorted in ascending order of the nonzero identifiers. The Coordinator draws u uniformly from the N_s -byte strings, N_s the length of a serialised scalar (32 for FROST(Pallas, BLAKE2b-512)), and sets

$$\hat{\alpha} := H_R(u \parallel \text{enc}(L, m)),$$

where enc is any encoding from which m and, for each member of L , its identifier, hiding commitment and binding commitment are recovered uniquely. It sends $\hat{\alpha}$ itself to the signers. Class (Definition 1.1): specified (ZIP 312, Draft).

The construction is that of ZIP 312, "Randomizer Generation" and "Round Two - Signature Share Generation". The ZIP's default encoding is a byte serialisation of the message and the commitment list; it permits any other encoding under which all these values are unambiguously encoded.

In spend authorisation the message is $m = \text{SigHash}(T)$ for the transaction T , and SigHash commits to the randomised validating key of every Action: ZIP 244, "T.4c: orchard_actions_noncompact_digest", field T.4c.ii, kept for version 6, and with the same structure for the Ironwood pool, by ZIP 229, "Transaction Identifiers, Auth Digests, and Signature

Digests” (*Ironwood Guide*, Definition “Signature digest”). The aggregate is accepted only under rk , which for the shifted key of Definition 5.3 is $PK + [\hat{a}]B$. A valid threshold spend by Construction 5.4 with this message therefore needs a fixed point: a randomiser $\hat{a}$ equal to H_R of an input containing the digest of a transaction that contains $PK + [\hat{a}]B$.

Proposition 5.5 (No randomiser derived from the signature digest). *Let PK be the group key of an Orchard-protocol threshold key, with $q = p_{\text{vesta}}$ and $B = G^{\text{SpendAuth}}$. Let $H_R = \text{ToScalar} \circ H'_R$, where H'_R is a random oracle onto a set of size b and ToScalar maps a uniform element of that set to within δ of uniform on $\mathbb{F}_q$, the one-function form of Assumption 4.2; for $\text{FROST}(\text{Pallas}, \text{BLAKE2b-512})$, $b = 2^{512}$. Let an algorithm $\mathcal{A}$, given PK and making at most q_b queries to H'_R , output a transaction T with at most N_A Actions, an index j , and a string $x = u \parallel \text{enc}(L, m)$ with u of N_s bytes, as in Construction 5.4. It wins if $m = \text{SigHash}(T)$ and the randomised validating key of Action j of T equals $PK + [H_R(x)]B$. With $Q := q_b + 1$,*

$$\Pr[\mathcal{A} \text{ wins}] \leq \frac{Q}{b} + \sqrt{Q \cdot \text{Adv}^{\text{cr}}(\mathcal{B}) + Q^2 N_A (1/q + \delta)}$$

for an algorithm $\mathcal{B}$ of about twice the running time of $\mathcal{A}$, where $\text{Adv}^{\text{cr}}(\mathcal{B})$ is its probability of breaking the Ironwood Guide’s Assumption “Collision resistance of BLAKE2b”.

Proof. Only H'_R is modelled as a random oracle; the digest tree of SigHash is computed with BLAKE2b under other personalisations, and only its collision resistance is used. Let ϵ be the winning probability. Define an algorithm $\mathcal{F}$ for the *Crypto Guide*’s Theorem “General forking lemma” (§“Security in the random oracle model and the forking lemma”), with challenge set the range of H'_R , of size b , and Q challenges. On input PK and challenges $c_1, \dots, c_Q$, it runs $\mathcal{A}$, answering its k -th distinct query to H'_R with c_k ; on the output (T, j, x) it queries x itself if $\mathcal{A}$ did not; it returns $(I, (T, j, x))$, with I the index of the query at x if $\mathcal{A}$ wins and $I = 0$ otherwise. Its acceptance probability is ϵ , so the forking algorithm succeeds with probability

$$\text{frk} \geq \epsilon \left(\frac{\epsilon}{Q} - \frac{1}{b} \right).$$

On success the two runs coincide until the I -th query is asked, so both query the same string x at index I , and both win with that x , with outputs T_1 and T_2 and challenges $c_I \neq c'_I$. The string x determines m , since u has fixed length and the encoding is unambiguous; hence $\text{SigHash}(T_1) = \text{SigHash}(T_2) = m$.

If the tuples of randomised validating keys of T_1 and T_2 differ, descent through the two digest trees from their equal roots yields two distinct pairs of personalisation and input with equal BLAKE2b output, each node hashing a uniquely decodable input (ZIP 244, “TxId Digest” and “Signature Digest”; ZIP 229, “Transaction Identifiers, Auth Digests, and Signature Digests”): at the first node whose inputs differ, the two pairs collide. Running the forking algorithm and this descent is $\mathcal{B}$.

Otherwise T_2 contains the point $PK + [\text{ToScalar}(c_I)]B$, which the first run placed at index j of T_1 . The second run is independent of c_I . For a fixed fork index i , each of the at most N_A keys of T_2 equals $PK + [\text{ToScalar}(c_i)]B$ only if $\text{ToScalar}(c_i)$ takes one value fixed by the second run, which

has probability at most $1/q + \delta$ (*Math Guide*, Theorem “Properties of statistical distance”, part (3)); summing over the keys and over the Q indices,

$$frk \leq \text{Adv}^{\text{cr}}(\mathcal{B}) + QN_A(1/q + \delta).$$

With the lower bound, $\epsilon^2 - (Q/h)\epsilon - K \leq 0$ for $K := Q \cdot \text{Adv}^{\text{cr}}(\mathcal{B}) + Q^2N_A(1/q + \delta)$, so $\epsilon \leq (Q/h + \sqrt{(Q/h)^2 + 4K})/2 \leq Q/h + \sqrt{K}$. $\square$

The derivation of Construction 5.4 with the digest as message therefore yields a valid threshold spend only with negligible probability: infeasible in the random-oracle model, not impossible. For FROST(Pallas, BLAKE2b-512), $1/q + \delta < 2^{-253.9}$ and $Q/h = Q \cdot 2^{-512}$. Construction 5.4 is the only specified derivation, so no specified randomiser derivation is usable for spend authorisation; the order that a usable one must respect is Proposition 6.12.

Construction 5.6 (Seed-and-commitments randomiser). After Round One, let L be the commitment list over the signing set I . The Coordinator draws a seed v uniformly from the 32-byte strings and sends it to every member of I over a confidential and authenticated channel (Definition 2.10, type (b)). The Coordinator and every member of I compute

$$\hat{\alpha} := H_R(v \parallel \text{enc}(L)),$$

with $\text{enc}(L)$ the encoding of Definition 4.3; the message is not an input. Two rules are part of the construction and are this volume’s own: H_R in both computations, the Coordinator’s and every member’s, so that all parties obtain one scalar; and the confidential and authenticated channel for the seed, which the proof of Proposition 7.24, step (iv), shows necessary. Class (Definition 1.1): designed but unspecified.

The interface is that of ePrint 2024/436, Figure 5 and Remark 1, with $(\alpha, aux) = (v, \text{enc}(L))$ and every signer applying H_R ; the encoding of the commitment list is that of RFC 9591, Section 4.3. A seed flow is published only in the revision of ZIP 312 proposed in pull request 895, and the merged ZIP 312 has none. That hashing the signer-chosen commitments hedges a faulty generator at the Coordinator is asserted informally in ePrint 2024/436, Section 7, and proved nowhere; the Coordinator sees L before it chooses the seed, and it remains trusted for unlinkability (Definition 5.14).

Lemma 5.7 (Distance of the randomiser from uniform). *Under Assumption 4.2, let an algorithm, the observer, make at most q_b queries to H_R and, at one point, choose a byte string w and receive $\hat{\alpha} := H_R(u \parallel w)$, for u uniform on the N_u -byte strings and independent of all else. The pair formed by the observer’s view and $\hat{\alpha}$ is within statistical distance*

$$\delta + q_b \cdot 2^{-8N_u}$$

of the pair in which $\hat{\alpha}$ is replaced by a uniform element of $\mathbb{F}_q$ independent of all else. For Construction 5.4, $N_u = N_s$ and $w = \text{enc}(L, m)$; for Construction 5.6, $N_u = 32$ and $w = \text{enc}(L)$; with $N_u = 32$ the bound is $\delta + q_b \cdot 2^{-256}$.

Proof. The proof is that of Lemma 4.5 with H_R in place of H_3 and N_u bytes in place of 32. The observer queries H_R at $u \parallel w$ with probability at most $q_b \cdot 2^{-8N_u}$, by the union bound over its queries, each having prefix u with probability 2^{-8N_u} in the experiment in which the answer used for $\hat{a}$ is resampled. Outside that event the two experiments coincide, and the resampled answer is within δ of uniform; the triangle inequality (*Math Guide*, §“Statistical distance”, Theorem “Properties of statistical distance”) gives the bound. $\square$

At $q_b = 2^{64}$ and $N_u = 32$ the bound is below $2^{-191.99}$. The randomiser is thus close to uniform, not exactly uniform as ZIP 312, “Rationale”, states.

Remark 5.8 (Effective randomiser). The randomiser of ZIP 312 is the effective randomiser $\hat{a}$ of ePrint 2024/436, not its α . The correspondence carries correctness (Theorem 5.11), not the paper’s unforgeability game, whose signing oracle applies H_r itself to an adversarial (α, aux) (Definition 6 and Figure 4, with RandShare and RandPK of Figure 5). Only Construction 5.6 presents the paper’s interface. Wherever the Coordinator sends $\hat{a}$ itself, as in the wire form of Construction 5.4 or when it forwards a randomiser chosen elsewhere, it chooses the effective randomiser outright and the signers use it unchecked; the paper’s game does not model this. This remark is the volume’s one statement of that fact; later statements cite it.

5.4 Authorisation of the signed message

Definition 5.9 (Message check). In spend authorisation the signed message is $m = \text{SigHash}(T)$ for a transaction T (*Ironwood Guide*, Definition “Signature digest”), which alone does not show a signer what it authorises. With m the Coordinator sends *transaction data* D_T , further data of T , over the confidential and authenticated channel of Definition 2.10, type (b). A signer *performs the message check* if, before returning a share, it verifies that m is the signature digest determined by D_T , or computes m from D_T itself. A signature share counts as an *approval* of the pair (rk, m) , with rk the shifted group key of its request, only if its signer performed the message check.

ZIP 312, “Round Two - Signature Share Generation”, makes the check a MUST and names as possible data the transaction itself, openings of value commitments and decryptions of note ciphertexts; RFC 9591, Section 7.7 (“Input Message Validation”), recommends such input validation. The honest participants of the security games of §7 perform the check by definition, and their returned shares are the approvals the games count. A signer that omits it authorises whatever the transaction builder chose. Class (Definition 1.1): the obligation is specified; the mechanism by which a signer obtains and checks D_T is outside ZIP 312 and designed but unspecified.

5.5 Re-randomised signing, aggregation and share verification

Construction 5.10 (Re-randomised signing of ZIP 312). The key material is valid (Definition 2.5); the ciphersuite has the randomiser hash H_R (Definition 4.1, part (f)).

- (1) *Round One.* Each participant runs Construction 4.4 unchanged and sends its commitments over an authenticated channel (Definition 2.10, type (a)).

- (2) *Randomiser*. The Coordinator chooses the signing set I and, for each $i \in I$, one unused commitment pair, forms the commitment list L , and obtains $\hat{\alpha}$ by Construction 5.4 or Construction 5.6, fresh for this run. It sends to each $P_i, i \in I$, the tuple $(m, L, \hat{\alpha}, D_T)$, or (m, L, v, D_T) with the seed v of Construction 5.6, where D_T is the data of Definition 5.9, over a confidential and authenticated channel (Definition 2.10, type (b)); RFC 9591 requires authentication only.
- (3) *Round Two*. Member P_i performs the checks of Construction 4.8, its check (1) being the message check of Definition 5.9; in the seed variant it computes $\hat{\alpha} := H_R(v \parallel \text{enc}(L))$. It computes $PK' := PK + [\hat{\alpha}]B$ from its stored group key and runs Construction 4.8 with $sk_i + \hat{\alpha}$ in place of sk_i and PK' in place of PK . The shifted group key PK' , not PK , is the key serialised into H_1 for the binding factors and into H_2 for the challenge (Definition 4.7). Its share is

$$z_i := d_i + e_i \rho_i + \lambda_{T,i} (sk_i + \hat{\alpha}) c,$$

returned as in RFC 9591 over an authenticated, not confidential, channel (Definition 2.10, type (a)).

- (4) *Aggregation*. The Coordinator runs Construction 4.9 with PK' in place of PK , and verifies the aggregate under PK' .
- (5) *Share verification*. The share equation of Construction 4.9 is checked with $PK_i + [\hat{\alpha}]B$ in place of PK_i and with the shifted group key:

$$[z_i]B = R_i + [c \lambda_{T,i}] (PK_i + [\hat{\alpha}]B).$$

Every signing run uses a fresh randomiser, generated in its own Round Two: a randomiser used for two runs of one key gives both the same shifted key and links them. The output (R, z) is a signature under $\text{rk} := PK'$. Class (Definition 1.1): specified (ZIP 312, Draft), the seed variant designed but unspecified.

The construction is that of ZIP 312, “Round One - Commitment”, “Round Two - Signature Share Generation” and “Signature Share Verification and Aggregation”, over RFC 9591, Sections 5.1 to 5.4: the randomised signing, aggregation and share-verification functions are the regular ones with the share, the verification share and the group key shifted by $\hat{\alpha}$ and by $[\hat{\alpha}]B$. Round One keeps the unshifted share in the nonce hash, the randomiser not yet existing. The keys PK and PK_i are those stored from key generation, never taken from the messages of the run (Construction 4.9). What the returned shares reveal to a party that sees them is Proposition 7.25.

Theorem 5.11 (Correctness of re-randomised FROST). *Let the key material be valid, with signing key s and group key PK ; let $\hat{\alpha} \in \mathbb{F}_q$, I a signing set and m a message that every member of I accepts. Outside the exceptions below, a run of Construction 5.10 in which every party is honest*

outputs (R, z) with

$$R = [r]B, \quad r := \sum_{i \in I} (d_i + e_i \rho_i), \quad z = r + c(s + \hat{a}),$$

which the single-party verification accepts on m under $PK + [\hat{a}]B$, and every share satisfies its shifted share equation. The exceptions are those of Theorem 4.11 for the shifted material, together with the identity case of the shifted group key: in a suite whose element serialisation is not total, the run fails when $PK + [\hat{a}]B = \mathcal{O}$.

Proof. By Definition 5.3 and the paragraph following it, the shifted shares $sk_j + \hat{a}$, verification shares $PK_j + [\hat{a}]B$ and group key $PK + [\hat{a}]B$ are valid key material for $s + \hat{a}$, with sharing polynomial $f + \hat{a}$. Steps (3) to (5) of Construction 5.10 are Constructions 4.8 and 4.9 run on that key material. Round One enters those constructions only through the nonces, each an answer of H_3 whichever share enters its input. Theorem 4.11 applied to the shifted material gives $R = [r]B$, $z = r + c(s + \hat{a})$ and acceptance under $PK + [\hat{a}]B$, outside its cases; and Proposition 4.10, part (ii), applied to it gives the shifted share equations. Both use only validity of the key material. When $PK + [\hat{a}]B = \mathcal{O}$ in a suite whose element serialisation is not total, $\text{ser}_G(PK')$ is undefined, and so are the binding factors of Definition 4.7; the run fails. $\square$

ZIP 312, “Rationale”, makes the same argument, and ePrint 2024/436, Equation (2), displays it; the ZIP cites, rather than proves, the coefficient sum one on which it rests, which Lemma 2.4 proves.

Example 5.12 (The two-of-three run, re-randomised). Every number below is output by the volume’s toy script. The toy curve, the key material, the encodings, the message toy message and the signing set $I = \{1, 3\}$ are those of Example 4.12. Members 1 and 3 draw fresh Round One nonces: reusing that example’s nonces would reuse a nonce pair (Lemma 4.13). The randomiser is that of Construction 5.6, with a fixed seed and H_R BLAKE2b-512 under the personalisation of FROST(Pallas, BLAKE2b-512) reduced modulo 967: $\hat{a} = 565$. The shifted group key and shares are

$$PK + [565]B = (268, 139), \quad sk_1 + \hat{a} = 697, \quad sk_3 + \hat{a} = 319,$$

and $\lambda_{1,1} + \lambda_{1,3} = 1$ modulo 967. With the shifted key in H_1 and H_2 , the binding factors are $\rho_1 = 480$ and $\rho_3 = 303$, the group commitment is $R = (302, 757)$ and the challenge is $c = 151$. The shares are $z_1 = 151$ and $z_3 = 652$, and each satisfies its shifted share equation against $PK_i + [\hat{a}]B$. The aggregate is $z = 803 = r + c(s + \hat{a})$, and $[z]B = R + [c](PK + [\hat{a}]B)$ holds.

5.6 Trust model of ZIP 312

Remark 5.13 (Requirements of ZIP 312). ZIP 312, “Requirements”, states three requirements, in lower case, although its “Terminology” invokes the key words of RFC 2119:

- (a) all signatures generated by following the ZIP must verify as Sapling or Orchard spend-authorisation signatures under the appropriate validating key, which Proposition 6.3 proves for the Pallas suite, the Sapling suite being outside the volume;

- (b) they should meet the security criteria of the protocol specification for signatures with re-randomisable keys, Definition 5.1, whose threshold form is an open problem (Table 3);
- (c) the threat model must be taken into account, which Definition 5.14 makes exact.

Definition 5.14 (Threat model of ZIP 312). The threat model of ZIP 312 consists of two clauses.

- (i) The Coordinator is trusted with the privacy of the transaction, unlinkability included. A rogue Coordinator can break both, but “should not be able to create signed transactions without the approval of” t participants, “as specified in FROST”; the ZIP’s MIN_PARTICIPANTS is t . An approval is a share returned after the message check (Definition 5.9).
- (ii) Every key-share holder is likewise trusted with the privacy of the transaction and with unlinkability.

The clauses are those of ZIP 312, “Threat Model”. The should of clause (i) is lower case, within a threat model that the Requirements say must be taken into account; it is not a must not. ePrint 2024/436, Section 5.1, states the same split: the party choosing the randomiser is trusted with privacy, not with security. The privacy trust is forced by the flow: the Coordinator generates $\hat{a}$, every signer receives it, or the seed with L , and whoever holds it computes $PK = rk - [\hat{a}]B$ and links the run to the key (the proof of Proposition 7.24, step (iv)). The unforgeability clause has no proof, as §7.5 records (Remark 7.22). In RFC 9591, Section 7, the Coordinator holds no private information; trust in it for privacy is what re-randomisation adds.

ZIP 312, “Non-requirements”, excludes four things: removal of the Coordinator role (RFC 9591, Section 7.5, “Removing the Coordinator Role”); prevention of key-share holders from linking a signing run to its transaction on the chain; a key-generation procedure, for which it refers to RFC 9591; and network privacy.

Remark 5.15 (Unlinkability as ZIP 312 states it). ZIP 312, “Terminology”, defines unlinkability as statistical independence of signatures from the signers’ long-term keys, ensuring, “for perfectly uniform generation of Randomizers and no leakage of metadata”, that it is “impossible to determine whether two transactions were generated by the same party”. The statement is informal: it names no observer and no view. Definition 7.23 makes it exact against a specified observer, and Lemma 5.7 replaces perfect uniformity.

6 The aggregate as a spend-authorisation signature

This section instantiates the re-randomised protocol of §5 over the ciphersuite FROST(Pallas, BLAKE2b-512) and composes it with the Orchard protocol. After the ciphersuite comes the section’s main result: the aggregate is an ordinary RedPallas spend-authorisation signature under the shifted group key, and consensus accepts it for an Action exactly when the effective randomiser is the Action’s randomiser. There follow the sign normalisation that makes a threshold group key a spend validating key; the key tree of ZIP 2005 for an Orchard-protocol threshold key, with the

lemma that its common secrets are simulable; the order of randomiser, randomised validating key, signature digest and proof that the main result forces; and the threshold construction of an Action in that order. Throughout, $q = p_{\text{Vesta}}$, $B = G^{\text{SpendAuth}}$, the effective randomiser is $\hat{\alpha}$ (Definition 5.3) and the randomiser of an Action is α .

6.1 The ciphersuite FROST(Pallas, BLAKE2b-512)

For a point P of the Pallas group write $\underline{P} := \text{LEBS2OSP}_{256}(P^*)$, the 32-byte form of the star encoding (*Ironwood Guide*, §“Fields, groups, and encodings”, Definition “Star encoding”), and abst_{P} for its partial inverse.

Construction 6.1 (FROST(Pallas, BLAKE2b-512)). The ciphersuite FROST(Pallas, BLAKE2b-512) is the following instance of Definition 4.1, for re-randomised signing.

- (a) *Group*. The Pallas group $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, of prime order $q = p_{\text{Vesta}}$ (ZIP 312’s n_{P}) and cofactor 1, with identity $\mathcal{O}$ and generator

$$B = G^{\text{SpendAuth}} = \text{GroupHash}(\text{z.cash:0rchar}, G)$$

(*Ironwood Guide*, §“GroupHash, domain separation, and nothing-up-my-sleeve generators”; Definition “Spend-authorisation and binding-signature instances”), ZIP 312’s $\mathcal{G}^{\text{Orchard}}$. A random scalar is uniform on $\{0, \dots, p_{\text{Vesta}} - 1\}$.

- (b) *Elements*. $N_e = 32$ and $\text{ser}_G(P) := \underline{P}$, which is total: $\mathcal{O}^* = \text{LE}_{256}(0)$. The deserialisation maps a 32-byte string b to $\text{abst}_{\text{P}}(\text{LEBS2OSP}_{256}^{-1}(b))$, and fails if that value is $\perp$ and, in addition, if it is $\mathcal{O}$. The function abst_{P} itself decodes $\text{LE}_{256}(0)$ to $\mathcal{O}$ (step 2 of Definition “Star encoding”); the rejection of the identity is the suite’s, not abst_{P} ’s.
- (c) *Scalars*. $N_s = 32$, $\text{ser}_{\mathbb{F}_q}(k) := \text{l2LEOSP}_{256}(k)$, and $\text{deser}_{\mathbb{F}_q}$ fails on a 32-byte string whose little-endian integer is not below p_{Vesta} .
- (d) *Hashes*. BLAKE2b-512 with a 16-byte personalisation, $N_h = 64$, the roles H_1 to H_5 as in Table 2.
- (e) *Signature encoding and verification* as in Definition 4.1, part (e), with these data.
- (f) *Randomiser hash*. The role H_R of Table 2.

Class (Definition 1.1): specified (ZIP 312, Draft).

The data are those of ZIP 312, “FROST(Pallas, BLAKE2b-512)”, which takes the group, the order and the encoding repr_{P} from the protocol specification, §“Pallas and Vesta”, and the base point from §“Spend Authorization Signature (Sapling and Orchard)”. The generic element serialisation of RFC 9591, Section 3.1, fails on the identity; this one does not. Three consequences for Theorem 4.11 follow. Its case (2), $R = \mathcal{O}$, does not arise, the serialisation being total. Its case (1), a zero nonce, remains: a commitment D_i or E_i equal to $\mathcal{O}$ is serialised in Round One and rejected by deser_G in check (3) of Round Two. The only identity condition left on the output is the consensus rule $\text{rk} \neq \mathcal{O}$ (Corollary 6.4).

Table 2. Hash functions of FROST(Pallas, BLAKE2b-512). Each role maps x to $\text{BLAKE2b-512}(\text{pers}, x)$, followed where stated by $\text{ToScalar}(y) = \text{LEOS2IP}_{512}(y) \bmod p_{\text{Vesta}}$ (*Ironwood Guide*, §“Pseudorandom functions and field reductions”, Definition “Field reductions”). The challenge hash H_2 is, as a function, the RedPallas hash $H^{\otimes}(z) = \text{ToScalar}(\text{BLAKE2b-512}(\text{zcash_RedPallasH}, z))$ (*Ironwood Guide*, §“The RedPallas signature scheme”, Definition “RedPallas hash”), so the FROST challenge is the challenge that RedPallas validation recomputes; the other five personalisations carry the prefix FROST_. Source: ZIP 312, “FROST(Pallas, BLAKE2b-512)”.

Role	Personalisation	Output
H_1 , binding factor	FROST_RedPallasR	ToScalar
H_2 , challenge	Zcash_RedPallasH	ToScalar
H_3 , nonce	FROST_RedPallasN	ToScalar
H_4 , message pre-hash	FROST_RedPallasM	64-byte digest
H_5 , commitment-list pre-hash	FROST_RedPallasC	64-byte digest
H_R , randomiser	FROST_RedPallasA	ToScalar

The reduction ToScalar is ZIP 312’s “interpreting the 64 bytes as a little-endian integer, and reducing the resulting integer modulo $G.\text{Order}()$ ”. ZIP 312 records the equality of H_2 and $H^{\otimes}$ (protocol specification, §“RedDSA, RedJubjub, and RedPallas”).

Proposition 6.2 (Random-oracle model of the Pallas ciphersuite). *Assume Assumption 4.2 in its form for a suite built from one hash function under distinct personalisations: the map $(\text{pers}, x) \mapsto \text{BLAKE2b-512}(\text{pers}, x)$ is modelled as a random oracle. Its restriction to $\text{pers} = \text{Zcash_RedPallasH}$ is the Ironwood Guide’s Assumption “The RedPallas hash as a random oracle”. No further assumption is introduced. Then:*

- (i) *the hashes H_1, H_2, H_3, H_4, H_5 and H_R are independent random oracles;*
- (ii) *each value of H_1, H_2, H_3 and H_R at a new input is within statistical distance*

$$\delta_{512} := p_{\text{Vesta}}/2^{512} < 2^{-257}$$

of a uniform scalar, so the suite meets Assumption 4.2 with $\delta = \delta_{512}$;

- (iii) *the hashes H_4 and H_5 are random oracles onto the 64-byte strings, so Q queries find a collision of either with probability at most $Q^2/2^{513}$;*
- (iv) *the independence of H_{dkg} that Assumptions 3.5 and 4.2 require holds for every instantiation of H_{dkg} by BLAKE2b-512 under a 16-byte personalisation distinct from the six of Table 2. No such instantiation is specified (ZIP 312, “Key Generation”).*

Proof. (i) The six personalisations are pairwise distinct and all 16 bytes long, hence prefix-free; *Crypto Guide*, §“Domain separation and personalisation”, Proposition “Domain separation yields independent oracles”, gives independence. (ii) A value at a new input is ToScalar of a uniform 64-byte string, whose distance from uniform is at most $p_{\text{Vesta}}/2^{512}$ by *Math Guide*, §“Uniform sampling and the bias of modular reduction”, Proposition “Bias of modular reduction”, with

$L = 512$, the bound that the Ironwood assumption records; and $p_{\text{Vesta}}/2^{512} = 2^{-258}(1 + e)$ with $0 < e < 2^{-128}$. (iii) Birthday bound, *Math Guide*, §“The union bound and a birthday calculation”, Proposition “Birthday bound”: $(\frac{Q}{2})2^{-512} \leq Q^2/2^{513}$. (iv) Part (i) applied to seven distinct personalisations. $\square$

At $Q = 2^{64}$ the collision bound of part (iii) is 2^{-385} .

6.2 Validity of the aggregate

Proposition 6.3 (Aggregates are spend-authorisation signatures). *Let valid key material (Definition 2.5) with group key Y be given over $\text{FROST}(\text{Pallas}, \text{BLAKE2b-512})$, let $\hat{a} \in \mathbb{F}_q$ be an effective randomiser and m a message that every member of the signing set accepts (Definition 5.9). Every run of Construction 5.10 in which every party is honest either fails, in the zero-nonce case of Theorem 4.11, or outputs (R, z) whose encoding*

$$\sigma := \text{ser}_G(R) \parallel \text{ser}_{\mathbb{F}_q}(z)$$

(RFC 9591, Appendix A) is accepted by the validation of the spend-authorisation instance of RedPallas (Ironwood Guide, Construction “RedPallas”, part 3, with base $G^{\text{SpendAuth}}$) under $X := Y + [\hat{a}]G^{\text{SpendAuth}}$ on $M := m$. No completed honest run yields an aggregate that validation rejects.

Proof. Theorem 5.11 gives

$$[z]B = R + [c]X \quad \text{with} \quad c = H_2(\text{ser}_G(R) \parallel \text{ser}_G(X) \parallel m)$$

(RFC 9591, Section 4.6), its only exception in this suite being the zero nonce, since ser_G is total. Validation of σ under X on m proceeds as follows.

1. The first 32 bytes are $\bar{R} = \underline{R}$, and $\text{abst}_p(R^*) = R$ because the star encoding is canonical and total, $R = \mathcal{O}$ included; so the decoded point is $R \neq \perp$.
2. The last 32 bytes give $S = z$, an integer below p_{Vesta} since $z \in \mathbb{F}_q$.
3. Validation computes $H^*(\bar{R} \parallel \underline{X} \parallel m)$, where $\underline{X} = \text{ser}_G(X)$: the inputs of the FROST challenge in the same order. Since $H_2 = H^*$ (Table 2), it recomputes c .
4. The equation $[S]B = R + [c]X$, whose cofactor is 1 on Pallas, is the FROST equation of RFC 9591, Appendix B, which holds.

How R was produced does not enter validation. These are the conditions that ZIP 312, “Rationale”, lists. $\square$

ZIP 312, “FROST(Pallas, BLAKE2b-512)”, states that the suite is meant to produce signatures “indistinguishable from RedPallas Orchard Spend Authorization Signatures”. This volume reads that phrase as the statement of format and validation above; the distributional statement, for randomisers of Construction 5.6, is Proposition 7.24.

Corollary 6.4 (The effective randomiser is the Action’s randomiser). *Let $Y = \text{ak}^P = [s]G^{\text{SpendAuth}}$ be the spend validating key of the key of the note that an Action consumes, and let the Action carry $\text{rk} = \text{ak}^P + [\alpha]G^{\text{SpendAuth}}$ (Ironwood Guide, §“Randomised validating keys”, Construction “Randomised validating key”). Then*

$$Y + [\hat{\alpha}]G^{\text{SpendAuth}} = \text{rk} \iff \hat{\alpha} = \alpha \text{ in } \mathbb{F}_{p_{\text{Vesta}}}.$$

Consequently:

- (i) if $\hat{\alpha} = \alpha$, the aggregate of Proposition 6.3 on the signature digest is valid under rk;
- (ii) if $\hat{\alpha} \neq \alpha$, with α and $\hat{\alpha}$ fixed before the queries to H_2 whose inputs contain either key, the aggregate is valid under rk with probability at most $(Q_2 + 2)(1/p_{\text{Vesta}} + \delta_{512})$ in an experiment with at most Q_2 queries to H_2 (Proposition 5.2, applied to the key $Y + [\hat{\alpha}]G^{\text{SpendAuth}}$ and the shift $\alpha - \hat{\alpha} \neq 0$);
- (iii) the consensus rule $\text{rk} \neq \mathcal{O}$ (protocol specification, §“Action Descriptions”; Ironwood Guide, Construction “Verification”, part (1)) fails exactly when $\alpha = -s$. For $\alpha = \hat{\alpha}$ the output of Construction 5.6, fixed after the key, this has probability at most

$$1/p_{\text{Vesta}} + \delta_{512} + q_b \cdot 2^{-256},$$

with q_b the number of queries to H_R made by the party that fixed the key (Lemma 5.7).

Hence, up to these probabilities, consensus accepts the aggregate as the Action’s spend-authorisation signature if and only if the effective randomiser equals the Action’s α .

Proof. The map $k \mapsto [k]G^{\text{SpendAuth}}$ is injective on $\mathbb{F}_{p_{\text{Vesta}}}$, since $G^{\text{SpendAuth}}$ generates a group of prime order p_{Vesta} ; and $Y + [\hat{\alpha}]G^{\text{SpendAuth}} - \text{rk} = [\hat{\alpha} - \alpha]G^{\text{SpendAuth}}$. Part (i) is Proposition 6.3 with $X = \text{rk}$. Part (ii) is the cited proposition, whose hypotheses hold: the shifted material is valid key material (Definition 5.3 and the paragraph following it), and $\text{rk} \neq \mathcal{O}$ in an accepted Action. For (iii), $\text{rk} = [s + \alpha]G^{\text{SpendAuth}}$ is $\mathcal{O}$ exactly when $\alpha = -s$; a value within statistical distance ϵ of uniform takes a fixed value with probability at most $1/p_{\text{Vesta}} + \epsilon$ (Math Guide, Theorem “Properties of statistical distance”, part (3)), and Lemma 5.7 gives $\epsilon = \delta_{512} + q_b \cdot 2^{-256}$. $\square$

For FROST(Pallas, BLAKE2b-512) the term $1/p_{\text{Vesta}} + \delta_{512}$ of parts (ii) and (iii) is below $2^{-253.9}$.

Remark 6.5 (Consensus is unchanged). No consensus rule refers to FROST or to a threshold key. The rules that bear on spend authorisation, $\text{rk} \neq \mathcal{O}$ and validity of the spend-authorisation signature under rk on the signature digest (Ironwood Guide, §“Verification of a transaction”, Construction “Verification”), read only rk, the 64-byte signature and the digest; validation does not depend on how R was produced (ZIP 312, “Rationale”). The first requirement of ZIP 312, “Requirements”, that every signature produced by following the ZIP be verified successfully as an Orchard spend-authorisation signature under the appropriate validating key, is met by Proposition 6.3. The remark makes no claim about the security of the signers, which is the subject of §7.

6.3 Sign normalisation of a shared key

The spend validating key ak^P of an Orchard-protocol key has even y -coordinate: the last bit, the $\tilde{y}$ bit, of $\text{repr}_P(ak^P)$ is 0. For keys with $\text{use_qsk} = \text{false}$ the protocol specification, §“Orchard Key Components”, produces this by negating ask when the bit is 1 (*Ironwood Guide*, §“The spending key and the spend-side secrets”, Construction “Spend validating key and sign normalisation”). For keys with $\text{use_qsk} = \text{true}$, ZIP 2005’s replacement text for that section (“Changes to the Protocol Specification”, § 4.2.3 “Orchard Key Components”) requires $ak^P \in \mathbb{P}^*$ to be obtained “by any suitably secure method that ensures the last bit of $\text{repr}_P(ak^P)$ is 0”; the published specification does not yet carry that text. The requirement is one of key derivation, not a consensus rule: the Action statement admits either sign (protocol specification, §“Action Statement (Orchard)”; *Ironwood Guide*, Remark “Sign of ak^P ”). A full viewing key records only $ak = \text{Extract}_P(ak^P)$ (*Ironwood Guide*, §“Viewing keys”, Definition “Full viewing key”), from which a transaction builder recovers the point by the even- y lift. For a group key PK of odd y the lift yields $-PK$; the builder forms $rk = -PK + [\alpha]G^{\text{SpendAuth}}$, which with $\alpha = \hat{\alpha}$ differs from $PK + [\hat{\alpha}]G^{\text{SpendAuth}}$ by $[2]PK \neq \mathcal{O}$, and by Proposition 5.2, part (ii), applied to the key $PK + [\hat{\alpha}]G^{\text{SpendAuth}}$ and the shift $-2s \neq 0$, the aggregate is accepted under rk with probability at most $(Q_2 + 2)(1/p_{\text{Vesta}} + \delta_{512})$. The randomised key rk carries no parity condition. Class (Definition 1.1): specified (ZIP 2005, Proposed, taking effect with NU6.3, which ZIP 258, Draft, deploys; ZIP 258, “ZIP 2005 activation”).

Lemma 6.6 (Negation of key material). *Let shares sk_j , coefficient commitments $A_0, \dots, A_{t-1}$, verification shares PK_j and group key PK be valid key material for the signing key s with sharing polynomial f (Definition 2.5). Then $-sk_j$, $-A_k$, $-PK_j$ and $-PK$ are valid key material for $-s$, with sharing polynomial $-f$. If $PK \neq \mathcal{O}$, the last bit of $\text{repr}_P(-PK)$ differs from that of $\text{repr}_P(PK)$, and $\text{Extract}_P(-PK) = \text{Extract}_P(PK)$.*

Proof. The polynomial $-f$ has degree at most $t - 1$, coefficients $-a_k$ and constant term $-s$; and $-sk_j = (-f)(j)$, $[-a_k]B = -A_k$, $[-sk_j]B = -PK_j$ and $[-s]B = -PK$, which are the conditions of Definition 2.5. For $PK = (x, y) \neq \mathcal{O}$, $-PK = (x, p_{\text{Pallas}} - y)$. Here $y \neq 0$, since a point with $y = 0$ has order 2 and the group has odd order; and p_{Pallas} is odd, so y and $p_{\text{Pallas}} - y$ have opposite parities. The last bit of the star encoding is $y \bmod 2$ (*Ironwood Guide*, Definition “Star encoding”). The extracted coordinate is x for both points (*Ironwood Guide*, §“Fields, groups, and encodings”, Lemma “Coordinate extraction identifies opposite points”). $\square$

On the toy curve of Example 4.12, the negated material has $-s = 646$ and shares 835, 57, 246; the group key $PK = (526, 125)$ becomes $-PK = (526, 884)$, with the same x -coordinate and the parity of y changed from 1 to 0.

Construction 6.7 (Sign normalisation of key material). *Input:* valid key material output by Construction 3.2 or by Construction 3.6. Every party computes PK from public data (Lemma 2.6).

1. If $PK = \mathcal{O}$, key generation is repeated: ZIP 2005 requires $ak^P \in \mathbb{P}^*$, and neither key generation excludes $s = 0$.

2. Otherwise, if the last bit of $\text{repr}_p(PK)$ is 1, every participant P_j replaces its share sk_j by $-sk_j$, and every party replaces each A_k , each PK_j and PK by its negative.

The test reads public data only, so all parties decide identically without interaction. *Output*: the resulting key material and $ak^P := PK$. Class (Definition 1.1): the requirement is specified (ZIP 2005, Proposed); the method is this volume’s.

Correctness. Validity of the output and even y of ak^P follow from Lemma 6.6; $PK \neq \mathcal{O}$ on output by step (1). $\square$

6.4 The key tree of an Orchard-protocol FROST key

The *Ironwood Guide* constructs only keys with $\text{use_qsk} = \text{false}$ (Remark “Scope of the key derivations”). Write PRFexpand for the *Ironwood Guide*’s Construction “Expansion function” and ToBase , ToScalar for its Definition “Field reductions” (“Pseudorandom functions and field reductions”); truncate_{32} returns the first 32 bytes of its input.

Construction 6.8 (Orchard-protocol threshold key). This is the branch $\text{use_qsk} = \text{true}$ of ZIP 2005’s replacement for § 4.2.3 “Orchard Key Components”.

1. Run a key generation (§3) and Construction 6.7; let $ak^P := PK$ and $ak := \text{Extract}_p(ak^P)$, non-zero since $ak^P \neq \mathcal{O}$ (*Ironwood Guide*, Lemma “Extract vanishes only at the identity”).
2. The participants privately agree on a spending key sk , a 32-byte string drawn uniformly and independently of the key material.
3. Every participant computes

$$\begin{aligned}
 nk &:= H^{nk}(sk) = \text{ToBase}(\text{PRFexpand}_{sk}([0x07])), \\
 qsk &:= H^{qsk}(sk) = \text{truncate}_{32}(\text{PRFexpand}_{sk}([0x0C])), \\
 qk &:= H^{qk}(qsk) \\
 &= \text{BLAKE3.derive_key}(\text{Zcash ZIP 2005 qk-derivation v1}, qsk, 32), \\
 r_{ivk} &:= H_{qk}^{rivk_ext}(ak, nk) \\
 &= \text{ToScalar}(\text{PRFexpand}_{qk}([0x0D] \parallel \text{I2LEOSP}_{256}(ak) \parallel \text{I2LEOSP}_{256}(nk))),
 \end{aligned}$$

where H^{qk} is the key-derivation mode of the BLAKE3 hash function with the displayed context string and a 32-byte output, ZIP 2005’s derivation, of which no property is used below.

4. The remainder of § 4.2.3 is unchanged: $ivk := \text{Commit}_{r_{ivk}}^{ivk}(ak, nk)$, the internal keys and the other keys derived from (ak, nk, r_{ivk}) , and the rejection of a key whose ivk or internal ivk lies in $\{0, \perp\}$. On rejection step (2) is repeated with a new sk , and ak^P is kept.
5. The protocol MUST ensure that all participants obtain the same ak , nk and r_{ivk} , which ZIP 2005 calls r_{ivk_ext} .

The spend authorising key $ask = s$ stays threshold-shared. Figure 1 shows the tree.

Steps (3) and (5), the agreement of step (2) and the rejection of step (4) quote ZIP 2005, “Usage with FROST” and “Changes to the Protocol Specification”, § 4.2.3; the uniform, independent draw of sk in step (2) and keeping ak^P on rejection in step (4) are this volume’s reading of “privately agree” and of the repetition with a new sk . Step (1) meets the even- γ clause of that section by Construction 6.7. Attempts of step (4) are independent, sk being fresh on each; the volume proves no bound on their rejection probability.

ZIP 2005 names two sources of ak^P for $use_qsk = \text{true}$: direct generation of an Orchard-protocol $ask \in \mathbb{F}_{p_{Vesta}}^*$ with $ak^P = [ask]G^{\text{SpendAuth}}$, which a dealer then shares by Construction 3.2; and FROST distributed key generation in which the spend-authorisation material is t -of- n shared, here Construction 3.6. Its “Deployment” states that “FROST distributed key generation requires the $use_qsk = \text{true}$ case”. The key-generation section that ZIP 2005 cites for the second method is ZIP 312, “Key Generation”, which leaves key generation out of scope.

The Orchard pool is spend-only by consensus: no transaction increases its value, and every Orchard-pool Action creates its note at the address of the note it consumes (ZIP 258, Draft, “Consensus rules from NU6.3 activation”; *Ironwood Guide*, § “The Orchard protocol and its two pools”). By consensus, therefore, only a party able to authorise spends for an address can create an Orchard-pool note to it, the consumed side of the Action being possibly a dummy note at that address (ZIP 326, “Rationale for key-generation restrictions”). ZIP 326 (Draft), “Wallet key-generation restrictions”, forbids conforming wallets to send Orchard-pool funds to a key with $use_qsk = \text{true}$; under that rule such a key, in particular every key from distributed key generation, holds no Orchard-pool notes, and consensus does not enforce the rule. A dealer-shared key with $use_qsk = \text{false}$ may spend notes of either pool. This qualifies the statement of §1.1 that both pools are served.

Remark 6.9 (Scope of the ZIP 2005 constraints). ZIP 2005, “Usage with FROST”, states its constraints for keys whose ak is derived jointly by distributed key generation, and this volume reads its MUST as binding those keys. A dealer that shares an ask derived from a spending key, which ZIP 312, “Key Generation”, permits, produces an ordinary key with $use_qsk = \text{false}$, known to the dealer and outside that constraint. A dealer that shares a directly generated ask may use Construction 6.8.

Remark 6.10 (Custody of the common secrets). Under Construction 6.8 the spend authorising key ask is threshold-shared and no signing run assembles it, while sk , nk , r_{ivk} and qsk are known to every participant. Each participant MUST treat qsk as a secret held at least as securely and reliably as ask (ZIP 2005, “Usage with FROST”).

Lemma 6.11 (Viewing material and the signing key). (a) For Construction 6.8, the tuple $V := (sk, nk, qsk, qk, r_{ivk}, ivk, \dots)$, with every key derived from ak , nk and r_{ivk} , is a randomised function of ak alone, sk being drawn independently of the key material. Hence, for every experiment in which an adversary receives V together with other inputs, an adversary that receives only the other inputs, ak^P among them, draws sk uniformly, computes V with the same rejection, and runs the first, has identical output distribution.

(b) For a dealer-shared key with $use_qsk = \text{false}$ and $ask = \pm \text{ToScalar}(\text{PRFexpand}_{sk}([0x06]))$ derived from sk , the spending key determines ask , so no participant may hold sk . The par-

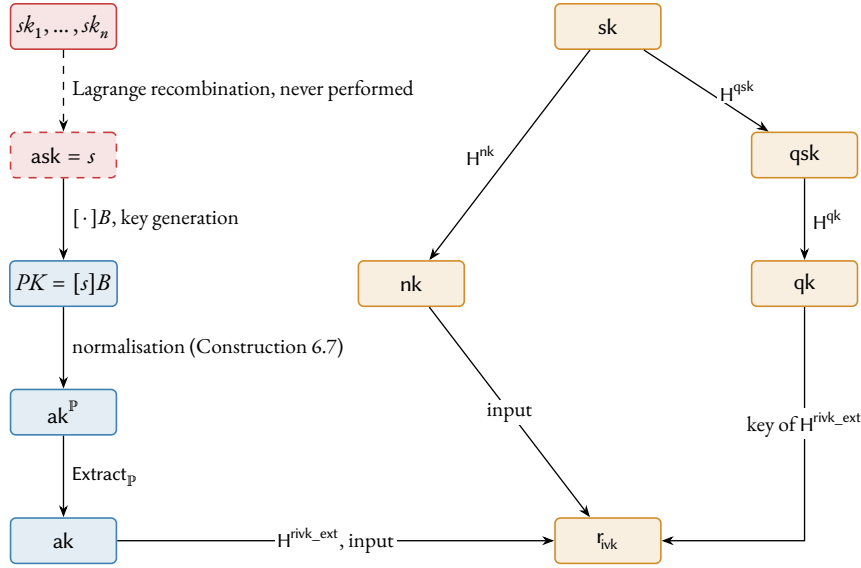


Figure 1. The Orchard-protocol threshold key of Construction 6.8. Red: threshold-shared values; the shares $sk_1, \dots, sk_n$ determine ask by Lagrange recombination, which no signing run performs (dashed). Amber: secrets common to all participants, derived from the privately agreed spending key sk : nk and qsk from sk , qk from qsk , and r_{ivk} from (ak, nk) keyed by qk . Blue: values known to every party, the group key PK , normalised to ak^P and extracted to ak . Every edge is labelled by its operation.

participants' $nk = \text{ToBase}(\text{PRFexpand}_{sk}([0x07]))$ and $r_{ivk} = \text{ToScalar}(\text{PRFexpand}_{sk}([0x08]))$, hence the full viewing key, are simulated from ak^P and independent uniform values with the same rejection, at a cost, per key attempt, of the advantage against the Ironwood Guide's Assumption "Pseudorandomness of PRF expansion" plus the sum of the three reduction distances, below $3 \cdot 2^{-257}$.

Proof. (a) Every component of V is computed from sk and ak , and the rejection test reads only sk and ak ; the construction draws sk independently of the key material, as the simulating adversary does. (b) By the Ironwood Guide's Lemma "Independence of domain-separated expansions" ("Pseudorandom functions and field reductions"), under Assumption "Pseudorandomness of PRF expansion" and with the three distinct leading bytes $0x06$, $0x07$, $0x08$, the triple (ask, nk, r_{ivk}) is replaced by independent uniform values at the stated cost, each reduction distance being below 2^{-257} . After the replacement nk and r_{ivk} are independent of ask , hence of ak^P , and the simulator draws them and applies the rejection test, which reads ak , nk and r_{ivk} . No property of H^{qk} is used. $\square$

Section 7 uses the lemma to give the corrupt participants the common secrets. Class (Definition 1.1): ZIP 2005's constraints on FROST key generation are specified (ZIP 2005, Proposed); how the participants privately agree on sk is designed but unspecified: ZIP 2005, "Usage with FROST", requires only that they agree privately, no specification gives a protocol for it, and the revision of ZIP 312 proposed in pull request 895 publishes one.

6.5 Order of randomiser, validating key and signature digest

Proposition 6.12 (Order of a threshold spend authorisation). *Consider a signing run over an Orchard-protocol threshold key, re-randomised as in Definition 5.3 by an effective randomiser $\hat{\alpha}$, whose aggregate consensus accepts as the spend-authorisation signature of an Action with randomiser α and randomised validating key rk , in a transaction with signature digest m . Except with the probabilities of Corollary 6.4 and of Proposition 5.5:*

- (i) *the randomiser $\hat{\alpha} = \alpha$ is fixed before rk , which is the function $\text{ak}^P + [\hat{\alpha}]G^{\text{SpendAuth}}$ of it;*
- (ii) *the key rk is fixed before m , which commits to it (Ironwood Guide, §“Transaction digests and signatures”, Definition “Signature digest”);*
- (iii) *the digest m is fixed before any Round Two share, being the message of the binding factors and of the challenge;*
- (iv) *Round One precedes $\hat{\alpha}$ under Construction 5.6, which hashes the commitment list, and need not precede it for a randomiser fixed before signing, as in Remark 6.13, where (i) to (iii) still hold;*
- (v) *the Action proof takes α as auxiliary input and rk as instance, so it follows (i), and it is unconstrained with respect to m and to Round Two, proofs being authorising data outside the digest.*

Proof. (i) is Corollary 6.4. (ii) and (iii) are the data dependencies of the cited definition and of Construction 4.8, as run in step (3) of Construction 5.10: ρ_i takes $H_4(m)$ and c takes m . (iv) is the input of H_R in Construction 5.6. (v) follows from the *Ironwood Guide*’s Definition “Authorising-data digest; effecting and authorising data” and “Construction of a transaction”, where signing can precede proving (ZIP 244, “Authorizing Data Commitment”, field A.3a). A randomiser that is a function of m , as under Construction 5.4 with the digest as message, would by (i) and (ii) be a fixed point, which Proposition 5.5 bounds; so that derivation admits no order except with that negligible probability. $\square$

Class (Definition 1.1): the order of Proposition 6.12 under Construction 5.6 is designed but unspecified; no specification composes ZIP 312 with a transaction format.

Remark 6.13 (Randomisers chosen by a transaction constructor). ZIP 312 derives the randomiser in Round Two (“Randomizer Generation” and “Round Two - Signature Share Generation”), and a derived randomiser equals an α fixed earlier, as Corollary 6.4 requires, only with probability at most $1/p_{\text{Vesta}} + \delta_{512} + q_b \cdot 2^{-256}$ (Lemma 5.7). A flow in which the transaction constructor fixes α and rk before signing, and the Coordinator sends that α as the effective randomiser, respects the order of Proposition 6.12, but no specification or published design states it: its design is an open problem (Definition 1.1), and under it the Coordinator chooses the effective randomiser outright (Remark 5.8).

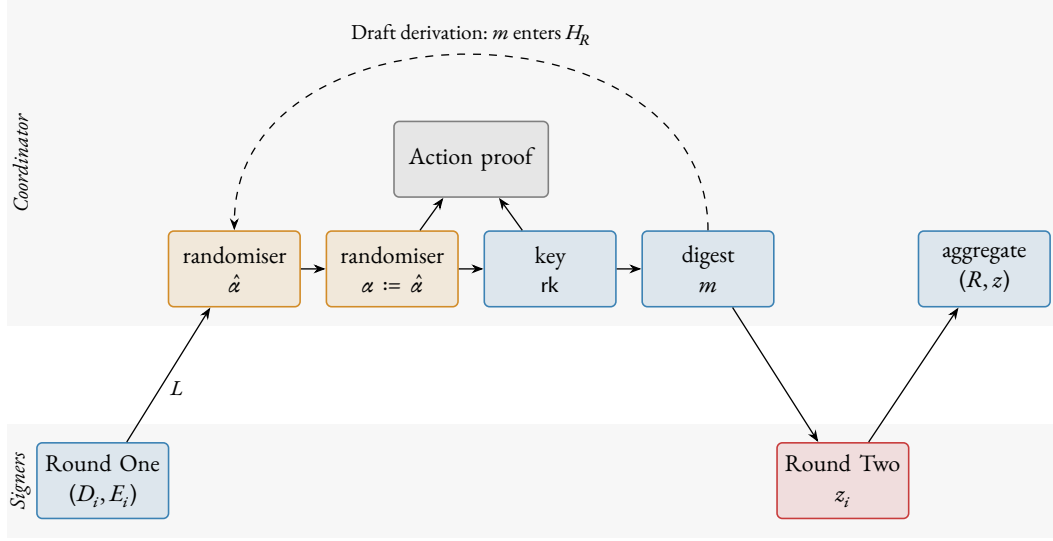


Figure 2. Order of one threshold spend authorisation (Proposition 6.12), in two lanes, Coordinator and signers, time running left to right: Round One commitments (D_i, E_i) ; the seed-and-commitments randomiser $\hat{\alpha}$ (Construction 5.6), which hashes the commitment list L ; the Action’s randomiser α ; the randomised validating key rk ; the signature digest m , which commits to rk ; the Round Two shares z_i ; the aggregate (R, z) . The Action proof is a branch from α and rk , connected neither to m nor to Round Two. The dashed edge closes the cycle of the Draft derivation (Construction 5.4 with m as message): randomiser, rk , digest and back, which Proposition 5.5 bounds.

6.6 Threshold signing of an Action

The single-signer construction is cited, not rebuilt. Each Action draws α by GenRandom , sets $\text{rsk} := \text{ask} + \alpha$ and $\text{rk} := \text{ak}^{\text{P}} + [\alpha]G^{\text{SpendAuth}}$, proves the Action statement with α as auxiliary input, computes the signature digest and signs it under rsk (*Ironwood Guide*, §“Randomised validating keys”, Construction “Randomised validating key”; “Construction of a transaction”, Construction “Ironwood-pool bundle”, steps (5), (7), (10) and (11); protocol specification, §“Sending Notes (Orchard)” and §“Spend Authorization Signature (Sapling and Orchard)”).

Construction 6.14 (Threshold spend authorisation of an Action). The transaction builder acts as Coordinator: it creates the Action proof and so already controls the privacy of the transaction (ZIP 312, “Threat Model”). For each Action that consumes a note of an Orchard-protocol threshold key (Construction 6.8), in the order of Proposition 6.12, it proceeds as follows.

1. It runs Round One (Construction 5.10, step (1)) with a signing set I of at least t participants.
2. It derives the effective randomiser $\hat{\alpha}$ by Construction 5.6, the only designed derivation that respects that order: Proposition 5.5 excludes the Draft derivation, and a constructor-chosen α has no published design (Remark 6.13).
3. It sets $\alpha := \hat{\alpha}$ and $\text{rk} := \text{ak}^{\text{P}} + [\alpha]G^{\text{SpendAuth}}$.

4. Once every Action of the transaction is fixed, it computes the one signature digest m of the transaction.
5. It runs Round Two of Construction 5.10 with message m , each signer performing the message check of Definition 5.9, and aggregates.
6. At any point after step (3), it proves the Action statement with α as auxiliary input (*Ironwood Guide*, §“The Action statement”).

These steps replace the draw of α and the single-holder signature in the cited construction; every other Action, dummy spends included, is built as there. No party holds rsk . The builder alone makes the binding signature (*Ironwood Guide*, §“The binding signature”). Every threshold-spent Action has its own α and rk and its own signing run, with fresh nonces and randomiser, all on the one digest m ; since m commits to the rk of every Action, the Round One and the randomiser of every threshold-spent Action precede m , and hence every Round Two (Proposition 6.12, (i) to (iv), applied to each Action). Class (Definition 1.1): designed but unspecified; the composition of ZIP 312 with the transaction is this volume’s own.

Acceptance. After normalisation $ak^P = PK$, and $\alpha = \hat{\alpha}$; so Proposition 6.3 and Corollary 6.4, part (i), give a signature valid under rk on m , except in the zero-nonce case and when $rk = \mathcal{O}$, whose probabilities Theorem 4.11 and Corollary 6.4, part (iii), bound. $\square$

The circuit and the consensus rules are unchanged (Remark 6.5). Both pools of the Orchard protocol use the same RedPallas spend authorisation, so one flow serves an Action of either pool (*Ironwood Guide*, §“The Orchard protocol and its two pools”; ZIP 229, “Reuse of the Orchard protocol with minimal changes”); which pool’s notes a key may hold is fixed in §6.4.

7 Security

This section states the assumptions and games, then each security statement at the strength its support allows: the cited unforgeability theorem for FROST, with the non-interactive syntax in which it is stated and its correspondence to RFC 9591; its transfer to Orchard-protocol dealer keys, and the open composition with distributed key generation; the claimed unforgeability of the re-randomised scheme and the gaps in its proof; threshold spend authority, proved from a target notion that is itself open; unlinkability against specified outside observers, and the linkage that returned signature shares permit; and the class of the principal constructions and claims of the volume. Throughout, G is a group of prime order q with generator B and identity $\mathcal{O}$; a count of oracle queries is written Q , never q .

7.1 One-more discrete logarithms

Definition 7.1 (One-more discrete logarithm problem). The *one-more discrete logarithm game* for G is a search game in the sense of the *Crypto Guide*, §“Security as a game”, Definition

“Security game and advantage”. The challenger keeps a challenge counter k and a query counter ℓ , both initially 0, and offers two oracles.

$\text{Chal}()$: set $k := k + 1$, draw $x_k \xleftarrow{\$} \mathbb{F}_q$, and return $X_k := [x_k]B$.

$\text{DLog}(X)$, for any $X \in \mathbb{G}$: if X was queried before, return the stored answer; otherwise set $\ell := \ell + 1$, store and return the discrete logarithm of X to base B .

The adversary outputs $(\gamma_1, \dots, \gamma_k) \in \mathbb{F}_q^k$ and *wins* if $\gamma_j = x_j$ for every $j \leq k$ and $\ell < k$. Its advantage $\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{A})$ is its probability of winning. The case of $\ell + 1$ challenges and at most ℓ queries is the ℓ -OMDL problem; the case of one challenge and no query is the discrete logarithm problem of the *Crypto Guide*, §“The discrete logarithm problem”.

The *Crypto Guide* names the problem in prose only. The game is that of ePrint 2022/833, Figure 9, in additive notation; its challenge queries are those that Theorem 7.9 counts.

Assumption 7.2 (One-more discrete logarithms on Pallas). Let $\mathbb{G}$ be the Pallas group, of order $q = p_{\text{Vesta}}$, with generator $B = G^{\text{SpendAuth}}$ (*Ironwood Guide*, §“Fields, groups, and encodings”; “GroupHash, domain separation, and nothing-up-my-sleeve generators”). Every efficient adversary has negligible advantage $\text{Adv}_{\mathbb{G}}^{\text{omdl}}$ (*Math Guide*, §“Polynomial, exponential, and negligible functions”) in the game of Definition 7.1.

By its case of one challenge, the assumption implies the *Ironwood Guide*’s Assumption “Discrete logarithms on Pallas”.

Definition 7.3 (Algebraic one-more discrete logarithm problem). Let $\mathbb{G}$, q and B be as in Definition 7.1 and let $\ell \geq 0$ be an integer. In the ℓ -AOMDL game the challenger draws $x_0, \dots, x_\ell \xleftarrow{\$} \mathbb{F}_q$ and gives the adversary $X_i := [x_i]B$ for $0 \leq i \leq \ell$. Its oracle takes a *declaration* $(\alpha, \beta_0, \dots, \beta_\ell) \in \mathbb{F}_q^{\ell+2}$ and returns

$$\alpha + \sum_{i=0}^{\ell} \beta_i x_i,$$

the discrete logarithm of $[\alpha]B + \sum_i [\beta_i]X_i$; it answers at most ℓ declarations. The adversary wins by outputting $(x_0, \dots, x_\ell)$, and $\text{Adv}_{\mathbb{G}}^{\ell\text{-aomdl}}(\mathcal{A})$ is its probability of winning.

The game is the intended one of ePrint 2024/436, Assumption 1 and Figure 1. The restriction lies on the oracle’s queries, which are declared combinations of B and the challenges, and not on the adversary, so no statement about algebraic adversaries is needed; the oracle of Definition 7.1 answers any element. The printed Figure 1 is ill-formed in three clauses: its oracle takes $\beta_1, \dots, \beta_\ell$ but returns an expression in β_0 ; it never checks the queried element against the declared combination; and its query counter is never incremented, so its final test, that the count is below $\ell + 1$, bounds nothing. This volume reads the game as stated above.

Lemma 7.4 (OMDL hardness implies AOMDL hardness). *For every $\mathbb{G}$ of prime order q , every $\ell \geq 0$ and every adversary $\mathcal{A}$ in the ℓ -AOMDL game of Definition 7.3, there is an adversary $\mathcal{A}'$*

in the game of Definition 7.1, making $\ell + 1$ challenge queries and at most ℓ oracle queries, with

$$\text{Adv}_G^{\text{omdl}}(\mathcal{A}') = \text{Adv}_G^{\ell\text{-aomdl}}(\mathcal{A}),$$

running in the time of $\mathcal{A}$ plus $\ell + 2$ scalar multiplications per declaration.

Proof. Algorithm $\mathcal{A}'$ queries Chal $\ell + 1$ times, obtaining $X_1, \dots, X_{\ell+1}$, and runs $\mathcal{A}$ on them as $X_0, \dots, X_\ell$. On a declaration $(\alpha, \beta_0, \dots, \beta_\ell)$ it queries DLog on $[\alpha]B + \sum_i [\beta_i]X_i$ and returns the answer, which is $\alpha + \sum_i \beta_i x_i$, the value the AOMDL oracle returns. It outputs the output of $\mathcal{A}$. The view of $\mathcal{A}$ is that of the AOMDL game, the query counts agree, and $\mathcal{A}'$ wins exactly when $\mathcal{A}$ wins. $\square$

The assumption of ePrint 2024/436, Theorem 1, is therefore no stronger than the OMDL assumption of the analysis RFC 9591 cites (ePrint 2022/833). Hence the analysis of ePrint 2024/436 needs no assumption beyond that of ePrint 2022/833.

7.2 Security of FROST signing

Definition 7.5 (Threshold signatures with leader requests). A *threshold signature scheme with leader requests* TS has n servers, the participants of Definition 2.1 with identifiers $1, \dots, n$, a threshold t , and the following algorithms, each with access to a random oracle h .

1. *Key generation* $\text{Kg}[h]$, run by a trusted party, outputs a verification key vk , public auxiliary information vk_s (the paper's *aux*), which holds the verification shares, and secret keys $sk_1, \dots, sk_n$.
2. *Token generation*: server i issues a first-round *token* pp and keeps a secret state for it.
3. A *leader request* lr , formed by the *leader*, a party holding no secret of the scheme, carries a message $lr.\text{msg}$, a signer set $lr.\text{SS} \subseteq \{1, \dots, n\}$, and a map $lr.\text{PP}$ from $lr.\text{SS}$ to tokens. The scheme is an *echo scheme*: requests carry the tokens.
4. *Partial signing* $\text{PS}[h](lr, i)$: server i answers lr with a partial signature computed from the state of the token $lr.\text{PP}(i)$, or with the failure symbol $\perp$ if $lr.\text{PP}(i)$ is not an unused token of its own; each token is thus answered at most once.
5. *Aggregation* $\text{Agg}[h]$ maps partial signatures to a signature sig .
6. *Verification* $\text{Vf}[h](vk, M, sig)$.
7. *Strong verification* $\text{SVf}[h](vk, lr, sig)$, which accepts, for each pair (vk, lr) , at most one signature.

The syntax is that of ePrint 2022/833, Section 3.1, restricted to what this volume uses. The leader is the Coordinator of Definition 2.8.

Definition 7.6 (Threshold unforgeability levels). Let TS be an echo scheme (Definition 7.5). The game has the following procedures.

Init(C): require $C \subseteq \{1, \dots, n\}$ and $|C| < t$; draw b ; run $(vk, vks, sk_1, \dots, sk_n) \leftarrow \text{Kg}[b]$; set $HS := \{1, \dots, n\} \setminus C$; return vk, vks and sk_j for $j \in C$.

Token oracle on $i \in HS$: server i issues a token, which is added to the set PP_i and returned.

Partial-signing oracle on $(i, lr), i \in HS$: record lr ; run $\text{PS}[b](lr, i)$; if the answer is not $\perp$, add i to $S_2(lr)$; return the answer.

Random oracle: return $b(x)$.

Fin(M, sig): for every recorded lr set

$$S_3(lr) := \{i \in HS \cap lr.SS : lr.PP(i) \in PP_i\}, \quad S_4(lr) := HS \cap lr.SS.$$

The adversary loses unless $\forall f[b](vk, M, sig)$ holds.

The adversary acts as the leader. The *trivial-forgery predicates* are

$$\begin{aligned} \text{tf}_2(lr) &: |S_2(lr)| \geq t - |C|, \\ \text{tf}_3(lr) &: \text{tf}_2(lr) \text{ and } S_2(lr) = S_3(lr), \\ \text{tf}_4(lr) &: \text{tf}_2(lr) \text{ and } S_2(lr) = S_4(lr), \end{aligned}$$

and, for $i \in \{2, 3, 4\}$, the *trivial strong-forgery predicate* $\text{tsf}_i(lr, vk, sig)$ is $\text{tf}_i(lr)$ and $\text{SVf}[b](vk, lr, sig)$. The game TS-UF- i is won by a valid (M, sig) such that no recorded lr with $lr.\text{msg} = M$ satisfies $\text{tf}_i(lr)$; TS-SUF- i replaces $\text{tf}_i(lr)$ by $\text{tsf}_i(lr, vk, sig)$. The advantage $\text{Adv}_{\text{TS}}^{\text{ts-suf-}i}(\mathcal{A})$ is the probability of winning.

The games are those of ePrint 2022/833, Figures 2 and 3, the predicates transcribed exactly. Under TS-SUF-3 a forgery on M is trivial only if one request lr for M was answered by at least $t - |C|$ honest servers, those servers are exactly the honest servers of $lr.SS$ listed with a token they issued, and sig is the unique signature that strong verification associates with (vk, lr) . Only these levels are defined: the volume uses TS-SUF-3 and the failure of TS-UF-4.

Construction 7.7 (FROST1 in the non-interactive syntax). Let $t \leq n < q$, and let $h_j(x) := b(j, x)$ for one random oracle b . The scheme FROST1[G] is the following echo scheme.

1. Kg: draw $a_0, \dots, a_{t-1} \xleftarrow{\$} \mathbb{F}_q$; set $sk_i := \sum_j i^j a_j$ and $vk_i := [sk_i]B$ for each i , $vk := [a_0]B$ and $vks := (vk_1, \dots, vk_n)$.
2. Token of server i : draw $u, v \xleftarrow{\$} \mathbb{F}_q$; the token is $pp := ([u]B, [v]B)$, and (u, v) is kept for it.
3. For a request lr and each $i \in lr.SS$ let $(U_i, V_i) := lr.PP(i)$ and $b_i := h_1(vk, lr, i)$; let

$$R := \sum_{i \in lr.SS} (U_i + [b_i]V_i), \quad c := h_2(vk, lr.\text{msg}, R).$$

4. $\text{PS}(lr, i)$: if $lr.\text{PP}(i)$ is an unused token of i with secret (u, v) , erase (u, v) and return (R, z_i) with $z_i := u + b_i v + c \lambda_{lr.\text{SS}, i} sk_i$; otherwise return $\perp$.
5. Agg : if all partial signatures carry one R , return $(R, \sum_i z_i)$.
6. $\text{Vf}(vk, M, (R, z))$: accept if and only if $[z]B = R + [h_2(vk, M, R)] vk$.
7. $\text{SVf}(vk, lr, (R^*, z^*))$: accept if and only if R^* equals the R computed from lr and $[z^*]B = R^* + [c] vk$.

The construction is ePrint 2022/833, Figure 8, in additive notation; the paper’s token secrets r, s and binding factors d_i are written u, v and b_i here, those letters being taken. FROST2 (Crites, Komlo and Maller, ePrint 2021/1375) replaces the factors b_i by one $b := b_1(vk, lr)$; ePrint 2022/833, Section 5, proves FROST2 TS-SUF-2-secure and not TS-UF-3-secure. RFC 9591 specifies FROST1: its binding factor takes the identifier (Section 4.4), and its Section 7.2 (“Optimizations”) rules the shared factor NOT RECOMMENDED, because it removes the guarantee that the set of participants that started Round One is the set that produced the signature.

Proposition 7.8 (RFC 9591 signing realises FROST1). *Let a ciphersuite (Definition 4.1) whose element serialisation is total, as that of FROST(Pallas, BLAKE2b-512) is, meet Assumption 4.2 with distance δ and digest length N_h bytes. Map RFC 9591 signing (Constructions 4.4, 4.8 and 4.9, Definition 4.7) over key material of Construction 3.2 at identifiers $1, \dots, n$ to the syntax of Definition 7.5 as follows: the Coordinator is the leader; the Round One pair (D_i, E_i) is the token; the request (m, L) is lr with $lr.\text{msg} = m$, $lr.\text{SS}$ the identifiers of L and $lr.\text{PP}(i)$ the pair of i in L ; $vk := PK$ and $vks := (PK_1, \dots, PK_n)$, from which the coefficient commitments are computed, since t verification shares determine the committed polynomial in the exponent (Lemmas 2.6 and 2.4). For every adversary $\mathcal{A}$ in the TS-SUF-3 game of Definition 7.6 for RFC 9591 signing under this map whose requests list only identifiers in $\{1, \dots, n\}$, making q_s token queries and q_b hash queries, there is an adversary $\mathcal{A}'$ against FROST1[G] (Construction 7.7) with the same query counts and about the same running time, such that, with $Q := q_s + q_b + 1$,*

$$\begin{aligned} \text{Adv}_{\text{RFC}}^{\text{ts-suf-3}}(\mathcal{A}) &\leq \text{Adv}_{\text{FROST1[G]}}^{\text{ts-suf-3}}(\mathcal{A}') \\ &\quad + 2q_s(\delta + (q_b + 2q_s) \cdot 2^{-256}) + 3Q^2 \cdot 2^{-8N_h} + (q_b + (n+1)q_s + 1)\delta. \end{aligned}$$

A request naming another identifier lies outside the FROST1 game and is not covered, and so is a suite whose serialisation is not total: its abort on $R = \mathcal{O}$ follows hash evaluations that consume no token. The correspondence rests on four facts.

(a) *The binding-factor input*

$$\text{ser}_G(PK) \parallel H_4(m) \parallel H_5(\text{enc}(L)) \parallel \text{ser}_{F_q}(i)$$

determines (vk, lr, i) , except on a collision of H_4 or H_5 or on a digest used before its preimage is queried.

(b) *The challenge input $\text{ser}_G(R) \parallel \text{ser}_G(PK) \parallel m$ is an injective reordering of (vk, M, R) , the two element encodings having fixed length.*

- (c) The separated hashes H_1 and H_2 realise h_1 and h_2 as independent oracles.
- (d) Hedged nonces replace the uniform token secrets of FROST1 at the cost of Lemma 4.5 per nonce, a hop the reduction needs because it does not know the honest shares that enter H_3 .

Proof. Game hops, with additive losses (Crypto Guide, §“Security as a game”, Remark “Game-hopping”).

Game G_0 is the RFC game.

Game G_1 : every honest nonce is a uniform scalar. The $2q_s$ nonces are replaced one at a time, in the order in which they are drawn. At each hop Lemma 4.5 applies with the rest of the game as the observer: its queries to H_3 are the q_b queries of $\mathcal{A}$ and at most $2q_s$ evaluations by honest servers, each at an input with a fresh 32-byte prefix. Hence $|\Pr[G_0] - \Pr[G_1]| \leq 2q_s(\delta + (q_b + 2q_s) \cdot 2^{-256})$. Game G_2 aborts on a collision of H_4 or of H_5 , or when a hash input contains an output value of H_4 or H_5 before the query that produces it. Each of the two oracles receives at most Q queries, from $\mathcal{A}$ and one per answered request, at most q_s of them. By Assumption 4.2 a collision of either has probability at most $Q^2 \cdot 2^{-8N_h}$ in total; each new answer equals one of at most Q digest strings already present in inputs with probability at most $Q \cdot 2^{-8N_h}$, which over Q queries and both oracles gives $2Q^2 \cdot 2^{-8N_h}$.

Game G_3 answers H_1 and H_2 at distinct inputs by uniform scalars. Under Assumption 4.2 each answer is an independent sample within distance δ of uniform, and the distinct inputs number at most q_b queries of $\mathcal{A}$, at most n binding factors and one challenge per answered request, and one challenge of the final verification; replacing the answers one at a time costs $(q_b + (n + 1)q_s + 1)\delta$ (Math Guide, Theorem “Properties of statistical distance”).

In G_3 , algorithm $\mathcal{A}'$ plays the FROST1 game. It relays token queries. It simulates H_3 , H_4 and H_5 lazily and records their queries. It answers an H_1 query $\text{ser}_G(PK') \parallel u \parallel v \parallel \text{ser}_{\mathbb{F}_q}(i)$ by $h_1(vk, lr, i)$ when $PK' = PK$ and u, v are recorded outputs of H_4 on m and of H_5 on $\text{enc}(L)$, with lr formed from (m, L) , and by a fresh value otherwise; by (a) and the abort rule of G_2 the request is unique. It answers an H_2 query $\text{ser}_G(R) \parallel \text{ser}_G(PK') \parallel m$ by $h_2(vk, m, R)$ when $PK' = PK$, and by a fresh value otherwise; by (b) the input determines (m, R) , and by (c) the two simulations are independent. The checks of Construction 4.8, namely the message check, the presence of the member’s own unused pair in L , deserialisation of every commitment and the order of the identifiers, are decided from public data. Algorithm $\mathcal{A}'$ therefore queries the partial-signing oracle exactly when the RFC participant would answer, does not query the oracle when the participant would abort, leaving the token unused in both games, since an aborting participant keeps its pair; it never submits a token already answered, and returns z_i , discarding R . The sets S_2, S_3 and S_4 of the two games coincide; a valid RFC signature $\text{ser}_G(R) \parallel \text{ser}_{\mathbb{F}_q}(z)$ is a valid FROST1 signature (R, z) ; and strong verification agrees. So $\mathcal{A}'$ wins when $\mathcal{A}$ wins in G_3 . $\square$

Theorem 7.9 (Unforgeability of FROST). *Let $\mathbb{G}$ have prime order q , and let $\text{FROST1}[\mathbb{G}]$ be Construction 7.7 with n servers and threshold t . For every adversary $\mathcal{A}$ in the TS-SUF-3 game of Definition 7.6 making at most q_s token queries, the first-round queries, and at most q_b random-oracle queries, there is an adversary $\mathcal{B}$ in the game of Definition 7.1, making at most $2q_s + t$*

challenge queries, such that, with $Q := q_s + q_b + 1$,

$$\text{Adv}_{\text{FROST1}[\mathbf{G}]}^{\text{ts-suf-3}}(\mathcal{A}) \leq 4nQ\sqrt{\text{Adv}_{\mathbf{G}}^{\text{omdl}}(\mathcal{B}) + 6Q/q},$$

only the two terms $\text{Adv}_{\mathbf{G}}^{\text{omdl}}(\mathcal{B})$ and $6Q/q$ lying under the root. Algorithm $\mathcal{B}$ runs in about twice the time of $\mathcal{A}$, plus at most $6nQ + 4q_s + 2n^2$ exponentiations and group operations. For some parameters, for instance $n = 20$ and $t = 3$, the scheme $\text{FROST1}[\mathbf{G}]$ is not TS-UF-4-secure. The hypotheses are those of the game: static corruption of fewer than t servers, trusted key generation, and the random-oracle model.

Proof. Cited: ePrint 2022/833, Theorem 5.4, whose bound, query counts and running time are those stated, the root spanning only $\text{Adv}_{\mathbf{G}}^{\text{omdl}}(\mathcal{B}) + 6Q/q$; the TS-UF-4 separation is its Section 5.4, by an explicit attack at $n = 20$ and $t = 3$. $\square$

RFC 9591, Section 7 (“Security Considerations”), claims security against existential unforgeability under chosen-message attacks as defined in that analysis, assuming that the shares are generated and distributed securely, by a trusted dealer or by a distributed key generation protocol, and that at most the Coordinator and $\text{MIN_PARTICIPANTS} - 1$ participants are corrupted. For dealer keys in a suite with total element serialisation and requests listing only identifiers in $\{1, \dots, n\}$, the claim rests on Theorem 7.9 at level TS-SUF-3, through Proposition 7.8; with distributed key generation it is open (Remark 7.13). It does not rest on the theorem of the original paper, which concerns an interactive variant.

Remark 7.10 (Role of the binding factors). The share z_i is bound through ρ_i to PK , the message, the commitment list L and the identifier (Definition 4.7). Changing the message, the signing set or any commitment in L changes some ρ_j , hence R and c , except on a collision of H_1 , H_4 or H_5 ; a share moved into another run fails its share equation, and the aggregate fails verification. Without such binding, concurrent sessions of two-round Schnorr signing admit forgeries: in time subexponential in the bit length of q , by an attack that combines the challenges of concurrent sessions (Drijvers, Edalatnejad, Ford, Kiltz, Loss, Neven and Stepanovs, “On the Security of Two-Round Multi-Signatures”, IEEE Symposium on Security and Privacy 2019), which applies to t -of- n threshold signing with up to $t - 1$ corrupt participants (ePrint 2020/852, Section 2.5); and in polynomial time once more than $\log_2 q$ sessions are open concurrently (Benhamouda, Lepoint, Loss, Orrù and Raykova, “On the (in)security of ROS”, EUROCRYPT 2021). The remark explains Theorem 7.9 and adds no claim; the security of FROST rests on that theorem.

7.3 Unforgeability and key generation

Lemma 7.11 (Sign normalisation and unforgeability). *Consider a game over dealer key material (Construction 3.2) that decides its winning condition from public data, the adversary’s output and the transcript of its oracles, as the games of Definition 7.6 do, over a group in which exactly half of the $q - 1$ non-identity points pass the parity test of Construction 6.7; for Pallas this test is the last bit of $\text{repr}_{\mathbf{p}}$. An adversary with advantage ϵ in the game whose key material is normalised*

by Construction 6.7 is, run unchanged, an adversary with advantage at least

$$\frac{q-1}{2q} \epsilon$$

in the game with unnormalised dealer key material.

Proof. The reduction plays the unnormalised game, runs the adversary unchanged on it, and stops unless $PK \neq \mathcal{O}$ and PK passes the parity test; since s is uniform, this event has probability $(q-1)/(2q)$, and it is decided from public data before the adversary receives anything else. On the event normalisation changes nothing. The normalised key material is distributed as unnormalised dealer key material conditioned on the event: step (1) of Construction 6.7 redraws a zero secret, and negation (Lemma 6.6) maps the sharings whose group key fails the test bijectively onto those whose group key passes it, the coefficients staying uniform. The conditioned view is therefore that of the normalised game, and the win carries over. $\square$

For $q = p_{\text{Vesta}}$ the factor is $1/2 - 1/(2q)$ with $1/(2q) = 2^{-255}$ to the precision stated: normalisation costs a factor 2 in advantage, up to that term.

Corollary 7.12 (Unforgeability for Orchard-protocol dealer keys). *Assume Assumption 7.2, and Assumption 4.2 as Proposition 6.2 instantiates it for $\text{FROST}(\text{Pallas}, \text{BLAKE2b-512})$ (Construction 6.1). Then Theorem 7.9, through Proposition 7.8, holds for RFC 9591 signing against adversaries whose requests list only identifiers in $\{1, \dots, n\}$, with the effective randomiser zero on dealer key material (Construction 3.2) normalised by Construction 6.7 and completed in either of two ways, the corrupt participants holding the common secrets:*

- (a) *by Construction 6.8 (`use_qsk = true`), with sk drawn independently of the key material and given to every participant; the TS-SUF-3 advantage of such an adversary is at most $2q/(q-1)$ times the sum of the bound of Theorem 7.9 and the loss of Proposition 7.8, with $q = p_{\text{Vesta}}$; the simulation of Lemma 6.11, part (a), being exact;*
- (b) *for a dealer that shares an ask derived from a spending key that no participant holds (`use_qsk = false`; Ironwood Guide, Construction “Spend-side secrets”), with the further terms of Lemma 6.11, part (b), of the Ironwood Guide’s Assumption “Pseudorandomness of PRF expansion” and Lemma “Rejection in key generation”, and of the bias below 2^{-257} of ToScalar (Definition “Field reductions” there), by a hybrid that replaces ask with a uniform scalar.*

The corollary covers signing with the randomiser zero only; the re-randomised protocol is the subject of §7.4.

Proof. Hybrids, then the cited statements. For (a), Lemma 6.11, part (a), replaces the adversary that receives sk and the values derived from it by one that draws sk itself and computes them from $ak = \text{Extract}_p(PK)$, rejection included, with no loss. For (b), Lemma 6.11, part (b), simulates nk and r_{ivk} from independent uniform values; the key-derivation hybrid then replaces ask, before normalisation, by a uniform scalar, at the cost of Assumption “Pseudorandomness of PRF ex-

pansion”, of Lemma “Rejection in key generation” and of the ToScalar bias. These hybrids apply because the win is efficiently decidable from the key: the deciding algorithm answers the signing queries with the shares. What remains is the game of Definition 7.6 over normalised dealer keys. Lemma 7.11 passes to unnormalised dealer keys, which are the output of Kg of FROST1; Proposition 7.8 passes such an adversary to FROST1[Pallas]; and Theorem 7.9 with Assumption 7.2 bounds that advantage. $\square$

Remark 7.13 (Distributed key generation and unforgeability). Theorem 7.9 assumes trusted key generation, and no checked source proves FROST1 unforgeable with a distributed key generation, in particular with Construction 3.6. RFC 9591, Section 7, lists secure generation by a dealer or by a distributed protocol as an assumption, not as a result. The nearest results are two. Crites, Komlo and Maller (ePrint 2021/1375, Theorem 7) prove FROST2 with their key generation unforgeable under the one-more discrete logarithm assumption and a further assumption: that an algorithm outputting a group element with a valid Schnorr proof of knowledge for it knows the element’s discrete logarithm, in that an extractor computes the logarithm from the algorithm’s code and random coins. The original paper (ePrint 2020/852) proves, under discrete logarithms, an interactive variant with its own key generation, not the specified scheme. Classification (Definition 1.1): unforgeability of FROST1 with distributed key generation is an open problem.

7.4 Unforgeability of re-randomised FROST

Definition 7.14 (Re-randomisable threshold unforgeability). The game TRUF, for the instance of Definition 5.3 given by FROST with effective-randomiser map H_r , runs as follows.

1. The adversary chooses (n, t) and a static set cor with $|cor| \leq t - 1$.
2. The challenger runs centralised key generation, a Shamir sharing of a uniform x with $PK = [x]B$, and gives PK , every PK_i and x_j for $j \in cor$.
3. Oracle $\text{Sign}(k, ssid)$, for honest k , opens session $ssid$ and returns Round One commitments $\rho_k = (R_k, S_k)$.
4. Oracle $\text{Sign}'(k, ssid, m, S, \alpha, aux, (\rho_i)_{i \in S})$, once per open session: add m to the set $\mathcal{Q}_m$; set $\hat{\alpha} := H_r(\alpha, aux)$, $\tilde{x}_k := x_k + \hat{\alpha}$ and $\overline{PK} := PK + [\hat{\alpha}]B$; return the Round Two share z_k computed with $\tilde{x}_k$ and $\overline{PK}$.
5. The adversary outputs $(m^*, \sigma^*, \alpha^*, aux^*)$ and wins if $m^* \notin \mathcal{Q}_m$ and σ^* is valid on m^* under PK or under $PK + [H_r(\alpha^*, aux^*)]B$.

The randomisers are adversarial, and the oracle applies H_r itself. The advantage $\text{Adv}^{\text{truf}}(\mathcal{A})$ is the probability of winning.

The game is that of ePrint 2024/436, Definition 6 and Figure 4, in additive notation and with the evident reading of its interface; the symbol ρ_k is the paper’s name for a commitment pair, not a binding factor.

Remark 7.15 (Scope of the game). Four clauses bound what TRUF covers.

- (a) TRUF is existential: its oracle records messages only, and a new signature on a signed message does not win. It is therefore weaker than Definition 5.1, which records message–signature pairs, and Theorem 1 of ePrint 2024/436 would not, as printed, give the strong notion that ZIP 312, “Requirements”, asks for (Remark 5.13, (b)).
- (b) The choice $\alpha = 0$ does not give the identity shift, since $H_r(0, aux)$ need not be 0, contrary to the preprint’s statement that an adversary obtains unrandomised signatures by submitting the randomiser zero. The game has no unrandomised signing oracle and does not contain ordinary unforgeability.
- (c) Freshness is on the message alone. All Actions of a transaction sign one digest, so a forgery under a second Action’s randomised key on a digest already signed for another Action is not a win.
- (d) A Coordinator that chooses the effective randomiser outright (Remark 5.8) is outside the game, whose oracle applies H_r .

Each clause is a reason why the target notion of §7.5, Definition 7.18, is stated separately.

Remark 7.16 (The claimed unforgeability of re-randomised FROST). Theorem 1 of ePrint 2024/436 claims that Rerandomized-FROST is unforgeable in the sense of Definition 7.14, in the random-oracle model under the assumption of Definition 7.3, with its Equation (3) as printed:

$$\text{Adv} \leq \sqrt{Q \cdot \text{Adv}_D^{\text{aomdl}}(\lambda) + 2Q^2/q - \text{negl}(\lambda)},$$

all three terms under one root, with $Q := q_b + q_s$ counting random-oracle and signing queries and q the group order (p in the paper). No number is used from it. Its assumptions are static corruption and centralised key generation (Section 2.3 there); the paper states that its definition can be adapted to distributed key generation, and gives no proof. The abstract’s “same security assumptions underlying plain FROST” is correct only in the sense of Lemma 7.4.

Remark 7.17 (Gaps in the printed reduction). The reduction of ePrint 2024/436, Section 6, has five gaps, one per clause.

- (a) The simulated verification shares do not interpolate to PK as printed. For $|C| = t - 1$ and $L_j(i)$ the Lagrange coefficient of the node j of $\{0\} \cup C$ at i , the required relation is

$$PK_i = [L_0(i)] PK + \sum_{j \in C} [L_j(i)][x_j]B.$$

- (b) The Round Two logarithm query omits the Lagrange coefficient: the share $z_k = r_k + s_k a_k + c \lambda_k \tilde{x}_k$ needs the logarithm of $R_k + [a_k]S_k + [c \lambda_k] \overline{PK}_k$.
- (c) The random-oracle simulation is not a consistent lazy sampler: the table of H_r stores no outputs, index and message are mixed in the binding-factor table, one branch returns c in place of a_k , and an unrandomised case $\alpha = \perp$ is used that neither $\mathbb{F}_q$ nor the game defines.

- (d) The forking step reprograms the signature hash at a key PK' left undefined, does not show that the reprogrammed query is the challenge query of both forgeries when their randomised keys differ, and derives no bound, so Equation (3) is not derived.
- (e) Nonce extraction is unsound: the printed numerator has sign errors and uses the randomisers in place of their H_r images; and when $c_0 = c_1$ and $a_0 = a_1$ but $b_0 \neq b_1$, the two responses give one linear equation in two unknowns.

With the query counter of Definition 7.3 restored, these remain material gaps: a repair may exist, but no complete reduction is given, and the claim of Remark 7.16 is recorded, not used as a theorem. Classification (Definition 1.1): unforgeability of re-randomised FROST in the game of Definition 7.14 is an open problem.

7.5 Threshold spend authority

Definition 7.18 (Threshold unforgeability under re-randomisation). Fix a key generation K for threshold parameters (t, n) , followed by Construction 6.7, and the ciphersuite $\text{FROST}(\text{Pallas}, \text{BLAKE2b-512})$, whose hashes are available as Assumption 4.2 models them.

1. *Corruption.* The adversary names a set C of fewer than t participants before key generation and takes their part in K : it receives every message K delivers to a member of C and, when K is interactive, sends the messages of the members of C in each round after seeing the honest participants' messages of that round. The honest participants, and a dealer, run K as specified and abort as it prescribes. For Construction 3.2 the adversary receives the shares of C and the coefficient commitments.
2. *Keys.* After a run of K without abort it receives PK and every PK_j , and it controls the Coordinator.
3. *Signing.* It obtains Round One commitments from honest participants (Construction 4.4) and Round Two shares on requests $(m, L, \hat{a}, D_T)$ with transaction data D_T (Definition 5.9), in which it supplies the effective randomiser $\hat{a}$ directly, as the wire form of ZIP 312 lets a Coordinator do; each honest participant runs Construction 5.10 with that $\hat{a}$ and D_T .
4. *Output.* The adversary outputs $(\theta^*, \alpha^*, m^*, \sigma^*)$ with $\theta^* \in \{1, -1\}$, and wins if σ^* is valid on m^* under

$$\text{rk}^* := [\theta^*] PK + [\alpha^*] B$$

and fewer than $t - |C|$ honest participants returned shares to requests whose message is m^* and whose shifted group key $PK + [\hat{a}]B$ equals rk^* .

The scheme *satisfies the notion* for K if every efficient adversary wins with negligible probability.

The notion is the target for ZIP 312, “Round Two - Signature Share Generation”; no source states it. Naming (θ^*, α^*) relates rk^* to PK , so a key whose logarithm the adversary knows is no win; $\theta^* = -1$ admits the opposite point that an Action may witness (*Ironwood Guide*, Remark “Sign of

ak^P). Freshness is on the pair (rk, m) , the counterpart of the *Ironwood Guide*'s Definition "Spend-authority experiment", whose queries (α, m) fix rk .

Definition 7.19 (Threshold spend-authority experiment). The adversary names a static set C of fewer than t participants. An Orchard-protocol threshold key is generated by K , the adversary taking the part of C as in Definition 7.18, normalised by Construction 6.7, and completed in one of two ways:

- (a) by Construction 6.8 ($use_qsk = \text{true}$), with sk drawn uniformly and independently of the key material and given to every participant, the ideal form of the agreement on sk ; or
- (b) when K is a dealer that shares an ask derived from a spending key, by the *Ironwood Guide*'s Construction "Spend-side secrets" from that spending key, which no participant holds ($use_qsk = \text{false}$).

The adversary controls the Coordinator and the participants of C , receives their state, the common secrets included, and every public field, may create notes to any address of the key, and runs signing runs of Construction 5.10 with the honest participants, supplying the message, the transaction data they check (Definition 5.9) and the effective randomiser. A *note of the key* is a note whose transmission key is $[ivk]g_d$ for the key's ivk . The adversary *wins* by outputting a block chain containing an accepted Action whose witness consumes a note of the key and whose signature is valid under its rk on a digest m such that fewer than $t - |C|$ honest participants returned shares to requests with message m and shifted key rk .

The experiment is the formal content of the clause of ZIP 312, "Threat Model", that a rogue Coordinator "should not be able to create signed transactions without the approval of" t participants (Definition 5.14). Freshness is on (rk, m) because every Action of a transaction signs one digest.

Theorem 7.20 (Threshold spend authority from the target notion). *Suppose that re-randomised FROST over FROST(Pallas, BLAKE2b-512) with keys from K satisfies Definition 7.18. Then, under the Ironwood Guide's Assumptions "Knowledge soundness of the Action proof", "Discrete logarithms on Pallas" and "GroupHash as a random oracle", and, for keys completed as in Definition 7.19, (b), its Assumption "Pseudorandomness of PRF expansion", no efficient adversary wins Definition 7.19 except with negligible probability. The statement is conditional: its premise is an open problem (Remark 7.22).*

Proof. The proof cites statements only. Let $\mathcal{A}$ be an efficient adversary in Definition 7.19.

(1) *Simulation.* A reduction $\mathcal{B}$ plays Definition 7.18 with the same C , relaying the part of $\mathcal{A}$ in K . It completes the key. For $use_qsk = \text{true}$ it draws sk and computes nk , qsk , qk , r_{ivk} and ivk from sk and $ak = \text{Extract}_p(PK)$ (Lemma 6.11, part (a)); for $use_qsk = \text{false}$ it simulates nk and r_{ivk} (Lemma 6.11, part (b)), at a negligible cost. It gives $\mathcal{A}$ the state of C and every public field, and simulates the chain and every other honest party. It relays each signing run to its oracles with the message, transaction data, commitment list and effective randomiser of $\mathcal{A}$; the honest oracles perform the check of Definition 5.9, and a failed check aborts identically in both games. The view

of $\mathcal{A}$ is thus that of the experiment, and the number of honest shares returned for a pair (rk, m) is the same in both games.

(2) *Extraction.* On the block chain output by $\mathcal{A}$, $\mathcal{B}$ runs the extractor of Assumption “Knowledge soundness of the Action proof” and finds an Action that meets the winning condition, stopping if there is none. Its witness gives ak_w^p , α_w and ivk_w with $rk = ak_w^p + [\alpha_w]G^{\text{SpendAuth}}$ and the consumed note’s transmission key $[ivk_w]g_d$, where $ivk_w = \text{Commit}_{ivk_w}^{ivk}(\text{Extract}_p(ak_w^p), nk_w)$ (*Ironwood Guide*, Definition “Orchard Action statement”).

(3) *The witnessed key.* The note is a note of the key, so $[ivk_w]g_d = [ivk]g_d$. The point $g_d \neq \mathcal{O}$ has prime order p_{Vesta} , and ivk_w and ivk are integers below $p_{\text{Pallas}} < p_{\text{Vesta}}$, so $ivk_w = ivk$. The *Ironwood Guide*’s Proposition “Binding of ivk to (ak, nk) ” gives $\text{Extract}_p(ak_w^p) = \text{Extract}_p(PK)$, except with negligible probability, and its Lemma “Coordinate extraction identifies opposite points” gives $ak_w^p = [\theta] PK$ for some $\theta \in \{1, -1\}$.

(4) *Output.* The Action is accepted, so its signature σ is valid under rk on m , and $rk = [\theta] PK + [\alpha_w]B$. Algorithm $\mathcal{B}$ outputs $(\theta, \alpha_w, m, \sigma)$, which wins Definition 7.18 by the winning condition of the experiment and step (1).

The corrupt participants’ nk and the other common secrets enter only through the simulation of Lemma 6.11, and no step uses their secrecy; there the proof departs from the single-holder case. The losses are the failure probability of the extractor, the binding term and, for $\text{use_qsk} = \text{false}$, the simulation term, all negligible. $\square$

Corollary 7.21 (Bundle binding for threshold keys). *The Ironwood Guide’s Proposition “Bundle binding” is stated for single holders under its hypothesis (S), that the holder signs under rk_i only $\text{SigHash}(T)$. Its threshold counterpart is the hypothesis*

(S_t) for each threshold-signed Action i of a transaction T , with randomised validating key rk_i , fewer than $t - |C|$ honest participants return shares to requests whose shifted group key is rk_i and whose message is not $\text{SigHash}(T)$.

If the premise of Theorem 7.20 and (S_t) hold, then under the Ironwood Guide’s Assumption “Collision resistance of BLAKE2b” and, for keys completed as in Definition 7.19, (b), its Assumption “Pseudorandomness of PRF expansion”, an adversary in Definition 7.19 that receives T outputs, only with negligible probability, an accepted transaction that contains an Action with key rk_i and whose effecting data differ from those of T .

Proof. Let T' be the output and $m' := \text{SigHash}(T')$. If $m' = \text{SigHash}(T)$, descending the digest tree of the *Ironwood Guide*’s Definition “Signature digest”, whose nodes hash unambiguous encodings under their personalisations, from the equal roots to a node whose inputs differ gives a collision, which has negligible probability by Assumption “Collision resistance of BLAKE2b” there. Otherwise $m' \neq \text{SigHash}(T)$, and by (S_t) fewer than $t - |C|$ honest participants returned shares for the pair (rk_i, m') ; the signature σ' of T' for that Action, valid under rk_i on m' , is fresh on the pair. The reduction of Theorem 7.20, which relayed the effective randomiser α_i with $rk_i = PK + [\alpha_i]B$ and simulated the common secrets by Lemma 6.11, outputs $(1, \alpha_i, m', \sigma')$, a win in Definition 7.18. $\square$

Hypothesis (S_t) does not follow from Definition 5.9: a Coordinator may send a second request with the same effective randomiser together with its own matching transaction data, and the check passes. It is an approval policy that the corollary assumes. The corollary is conditional, as the theorem is.

Remark 7.22 (Status of threshold spend authority). The premise of Theorem 7.20 is an open problem for every randomiser flow. Theorem 7.9 has no randomisation. The game of Definition 7.14 applies the randomiser hash inside its oracle, which only the flow of Construction 5.6 matches; wherever the Coordinator sends the effective randomiser, as in the wire form of ZIP 312 or when it forwards a randomiser fixed elsewhere, the flow is outside that game (Remark 5.8). The preprint’s freshness is on messages only, and its proof is incomplete (Remark 7.17). Composition with distributed key generation is open (Remark 7.13). The *Ironwood Guide*’s Theorem “Spend authority” does not apply: it fixes a key with `use_qsk = false` and the signing oracle of one holder. Unconditional threshold spend authority is an open problem (Definition 1.1); the clause of ZIP 312 is itself a “should”.

7.6 Unlinkability

Neither ePrint 2024/436 nor ZIP 312 proves threshold unlinkability. ZIP 312, “Rationale”, argues by matching distributions: its randomiser generation “generates randomizer uniformly at random as required by RedDSA.GenRandom”, and signing is compatible with the randomisation, signing and validation of the protocol specification. Its “Requirements” defer the formal notion to the criteria of the protocol specification for signatures with re-randomisable keys. The randomiser is close to uniform, not uniform (Lemma 5.7). This subsection states a notion against specified observers and proves it.

Definition 7.23 (Outsider unlinkability). Fix threshold parameters (t, n) , the identifiers $1, \dots, n$ and a key generation K with these parameters. The game generates two threshold keys independently by K and gives both group keys to an *observer*, which is neither the Coordinator nor a key-share holder and makes at most q_b random-oracle queries. The observer chooses a number N of run positions, for each position k a signing set I_k of at least t identifiers, and two assignments a^0, a^1 of the run positions to the two keys. For $b \in \{0, 1\}$, run position k signs with signing set I_k under key $a^b(k)$ by Construction 5.10, the observer choosing the message m_k , a signature digest, after receiving the run’s randomised validating key rk_k . Signing sets and their sizes, and messages as functions of the view, thus do not depend on the assignment. The *view* of a run is taken at one of three levels:

the *chain level*: rk_k , the aggregate and m_k ;

the *commitment level*: the chain level with the commitment list of Round One, which RFC 9591, Section 5, sends without confidentiality;

the *share level*: the commitment level with the returned signature shares, which RFC 9591 also sends without confidentiality.

The scheme is *outsider-unlinkable at a level with distance ϵ* if the observer's views under a^0 and a^1 , its oracle answers included, are within statistical distance ϵ (*Math Guide*, §“Statistical distance”).

The Coordinator and the share holders are excluded because ZIP 312 trusts them with unlinkability (Definition 5.14).

Proposition 7.24 (Unlinkability against outside observers). *Let signing be over FROST(Pallas, BLAKE2b-512), with distance $\delta < 2^{-257}$ (Proposition 6.2). Assume:*

- (1) *an honest Coordinator and honest signers following Construction 5.10;*
- (2) *a randomiser fresh per run and hidden from the observer, drawn by Construction 5.6, hence within the distance of Lemma 5.7;*
- (3) *the seed and the transaction data on the confidential and authenticated channel of Definition 2.10, type (b), and Round One and the returned shares on the authenticated channels of RFC 9591.*

Let $Q_b := q_b + N + 2 \sum_k |I_k|$, which bounds the number of distinct inputs at which the observer and the honest parties together query H_3 and H_R . Then the scheme is outsider-unlinkable (Definition 7.23) at the chain and commitment levels with distance at most

$$2 \sum_{k=1}^N (1 + 2|I_k|)(\delta + Q_b \cdot 2^{-256}).$$

The hypotheses that the observer is neither the Coordinator nor a signer of the runs, and that the randomiser channel is confidential, are necessary.

Proof. Sufficiency is proved by hybrids (i) to (iii), from each assignment towards one common game. The argument is statistical and programs no oracle, so it has no abort term.

(i) Each randomiser $\hat{a}_k$ is replaced by a uniform scalar, one run at a time, at $\delta + Q_b \cdot 2^{-256}$ per run, by Lemma 5.7 applied with the rest of the game as the observer; its queries to H_R are among the Q_b , each other honest evaluation having its own fresh 32-byte prefix. Then every $\text{rk}_k = PK + [\hat{a}_k]B$ is uniform and independent of the group keys and of the other runs (*Ironwood Guide*, Proposition “Unlinkability of randomised validating keys”, part (i)).

(ii) Each honest nonce is replaced by a uniform scalar, at $\delta + Q_b \cdot 2^{-256}$ per nonce, by Lemma 4.5 applied in the same way, whatever the share. The commitments are then uniform and independent of the keys, and so is the event of a zero nonce and the abort it causes.

(iii) The rest of the view is a function of these values, the observer's choices and the oracle. For run k : ρ_i is H_1 at an input holding $\text{ser}_G(\text{rk}_k)$, $H_4(m_k)$, H_5 of the list encoding and i ; $R = \sum_{i \in I_k} (D_i + [\rho_i]E_i)$; $c = H_2(\text{ser}_G(R) \parallel \text{ser}_G(\text{rk}_k) \parallel m_k)$; and z is the unique scalar with $[z]B = R + [c]\text{rk}_k$ (Theorem 5.11). The binding-factor input holds rk_k and not PK because Construction 5.10 substitutes the shifted key; with PK there, the check that R is the sum of the $D_i + [\rho_i]E_i$ under a candidate PK would test whether a run belongs to that key. After (i) and (ii) the view has one distribution under a^0 and under a^1 . The triangle inequality (*Math Guide*, §“Statistical distance”, Theorem

“Properties of statistical distance”) over both chains of hybrids, each of N randomiser hops and $2 \sum_k |I_k|$ nonce hops, gives the bound.

(iv) *Necessity*. The Coordinator and every signer of a run hold its seed with L , hence its randomiser, and so does any party that reads a randomiser channel without confidentiality. Each computes $ak^P = PK = rk - [\hat{a}]G^{\text{SpendAuth}}$ and so links the run to every run of the key, distinguishing two assignments that differ on it, except when the two independent group keys coincide. A randomiser reused across two runs of one key gives both the same shifted key (ePrint 2024/436, Section 7). Hence the exclusions and the confidential randomiser channel are necessary; this part is the volume’s one statement of why that channel is confidential. $\square$

The proposition is scoped against the *Ironwood Guide*’s Definition “Public leakage and privacy preconditions” (“Privacy”). A key with `use_qsk = true` violates (P1), which requires `use_qsk = false`; every randomiser of ZIP 312 violates (P2), which requires α drawn by `GenRandom`; and (P4) excludes an adversary holding nk , which every participant of Construction 6.8 holds. The *Ironwood Guide*’s Theorem “Privacy of an Action” and the key-dependent clauses of its Theorem “End-to-end security” therefore cover no threshold-signed Action, and for the other fields of such an Action this volume claims nothing.

Proposition 7.25 (Linkage through returned signature shares). *Let $t \geq 2$. At the share level of Definition 7.23 every run of a key is linked. From a share z_i of run k , the commitment list, the public rk_k and m_k , the observer computes ρ_i , R , c and $\lambda_{I_k,i}$, and then the tag*

$$\left[\frac{1}{\lambda_{I_k,i} c} \right] ([z_i]B - D_i - [\rho_i]E_i) - rk_k = PK_i - PK,$$

which is the same for every run of the key; the division is defined except when $c = 0$, which has probability at most $\delta + 1/q$. From the at least t tags of one run, interpolation in the exponent gives $PK_j - PK$ for every identifier j , so runs with disjoint signing sets are linked too. For two keys generated independently by Construction 3.2, the tags at one identifier agree with probability $1/q$. The chain and commitment levels are therefore the strongest that the channels of RFC 9591 and ZIP 312 support: unlinkability against an observer of the network needs confidential channels for the returned shares, which neither requires. For $t = 1$ the tag is $\mathcal{O}$ and carries no key.

Proof. An honest share is $z_i = d_i + e_i \rho_i + \lambda_{I,i} c (sk_i + \hat{a})$ (Construction 5.10), so

$$[z_i]B - D_i - [\rho_i]E_i = [\lambda_{I,i} c] (PK_i + [\hat{a}]B);$$

dividing by $\lambda_{I,i} c$ and subtracting $rk = PK + [\hat{a}]B$ gives $PK_i - PK$. Here $\lambda_{I,i} \neq 0$ for distinct non-zero identifiers, and c is an answer of H_2 , which takes the value 0 with probability at most $1/q + \delta$ (*Math Guide*, Theorem “Properties of statistical distance”, part (3)). With f the sharing polynomial, the tags are the values $[g(j)]B$ of $g := f - f(0)$, of degree at most $t - 1$; t of them determine $[g(j)]B$ for every j by Lagrange interpolation in the exponent (*Math Guide*, Theorem “Lagrange interpolation”), the map $a \mapsto [a]B$ being linear. For a dealer key with $t \geq 2$, $g(j) = \sum_{k \geq 1} a_k j^k$ is uniform for $j \neq 0$, since a_1 is uniform and independent of the other coefficients; so the tags of two independent keys at j agree with probability $1/q$. For $t = 1$, f is constant and $g = 0$. $\square$

On the toy curve of Example 4.12, two re-randomised runs of the key with randomisers 565 and 908 have randomised keys $(268, 139)$ and $(436, 331)$, and both give the tags $PK_1 - PK = (433, 783)$ and $PK_3 - PK = (933, 275)$.

7.7 Status of the constructions

Table 3 classifies the principal constructions and claims of the volume by Definition 1.1, with its source and the section that states it. Specified objects are those of an RFC, a ZIP or the protocol specification, with the ZIP's status; designed objects are published or constructed here without a specification; open problems have no proof in any checked source; established claims rest on a published proof. The last group lists the principal statements proved in this volume.

Table 3. Classification of the principal constructions and claims of the volume (Definition 1.1), with its source and the section or statement that gives it.

Object and source	Home
<i>Specified</i>	
FROST signing, aggregation and share verification; RFC 9591	§4
Trusted-dealer key generation; RFC 9591, Appendix C	§3.1
Even- γ requirement on the spend validating key of an Orchard-protocol key; ZIP 2005 (Proposed)	§6.3
Re-randomised signing, aggregation and share verification; FROST(Pallas, BLAKE2b-512); the threat model; ZIP 312 (Draft), which omits key generation	§§5.5, 6.1, 5.6
Randomiser derivation of ZIP 312 (Draft); with the signature digest as message it yields a valid threshold spend only with negligible probability (Proposition 5.5), so no randomiser derivation usable for spend authorisation is specified	§5.3
Derivation with <code>use_qsk = true</code> and the constraints on FROST keys; ZIP 2005 (Proposed), taking effect with NU6.3 (ZIP 258)	§6.4
Message-check obligation; ZIP 312	§5.4
Consensus rule under which the aggregate is accepted; protocol specification, “Action Descriptions”	§6.2
<i>Designed but unspecified</i>	
Distributed key generation for Zcash; ePrint 2020/852, specified by no RFC or ZIP	§3.2
Agreement on the common spending key	§6.4
Sign normalisation of shared key material (Construction 6.7), this volume’s own	§6.3
Seed-and-commitments randomiser and the order under it	§§5.3, 6.5
Message-check mechanism	§5.4
Threshold spend authorisation of an Action under the seed-and-commitments randomiser	§6.6
<i>Open problem</i>	
Unforgeability of FROST with distributed key generation	§7.3
Unforgeability of re-randomised FROST in the game of ePrint 2024/436, claimed with a gapped proof; and in the target notion, Definition 7.18	§§7.4, 7.5
Threshold strong unforgeability under re-randomised keys, Definition 5.1	§5.6
Unconditional threshold spend authority	§7.5
Composition of a constructor-chosen α with ZIP 312	§6.5
<i>Established</i>	
TS-SUF-3 of FROST1 under OMDL in the random-oracle model; ePrint 2022/833, Theorem 5.4	Theorem 7.9
<i>Proved in this volume</i>	
Correctness	Theorems 4.11, 5.11
Validity of the aggregate	Proposition 6.3, Corollary 6.4
Correspondence to FROST1	Proposition 7.8
OMDL hardness implies AOMDL hardness	Lemma 7.4
No randomiser derived from the signature digest	Proposition 5.5
Distance of nonces and randomisers from uniform	Lemmas 4.5, 5.7
Replay and identifiable abort	Propositions 4.14, 4.10
Validity of dealt and jointly generated key material	Propositions 3.3, 3.8
Correctness of sign normalisation of a dealt or jointly generated sharing	Lemma 6.6, Construction 6.7
Transfer of unforgeability to Orchard-protocol dealer keys	Lemmas 7.11, 6.11, Corollary 7.12
Order of a threshold spend authorisation	Proposition 6.12
Nonce-reuse criterion	Lemma 4.13
Threshold spend authority and bundle binding from the target notion	Theorem 7.20, Corollary 7.21
Outsider unlinkability at the chain and commitment levels	Proposition 7.24
Linkage at the share level	Proposition 7.25